# Errores pendientes de OpenMarkov

Lista unificada, comprobada contra el código y ordenada por temática

14 de septiembre de 2026; revisado el 22 de septiembre de 2026; correcciones marcadas el 23 de septiembre de 2026

**Qué es este documento.** La lista de los errores de OpenMarkov que siguen sin corregir, reunida a partir de las revisiones de código hechas en verano de 2026 y de los defectos nuevos que aparecieron al comprobarlas. Cada error se ha vuelto a comprobar contra el código de hoy, y se describe con lo necesario para ponerse a arreglarlo sin haber leído nada más: qué ve el usuario, por dónde se llega, en qué clase y en qué líneas está, y por dónde podría ir el arreglo.

**Qué no es.** No es un plan de trabajo, y no incluye las propuestas de rediseño. Lo que se comprobó y no es un error por corregir (porque ya está corregido, porque el código hace lo correcto, porque nada llega a él o porque una decisión del equipo lo deja fuera) está en el apéndice A, para que no haga falta volver a mirarlo.

## Índice


**Cómo leer este documento** **9**

**Resumen** **12**

**1 Operaciones con potenciales** **23**
1 Multiplicar potenciales constantes pierde el criterio de la utilidad . . . . . 23
2 Tablas de más de $2^{31}$ casillas: excepciones distintas o resultado falso . . . . . 25
3 Copiar un potencial a una red sin todas sus variables mete nulos . . . . . 27

**2 Modelos canónicos y modelo de ajuste** **28**
4 El modelo de ajuste se proyecta con otro orden y otros decimales cada vez . . . . 29
5 Parámetros que no suman uno se abren sin aviso y el muestreo discrepa . . . . . 31
6 El modelo de ajuste se ofrece a políticas sin padres y en redes de sucesos . . . . . 33

**3 Relaciones definidas por fórmulas, regresiones, árboles y sucesos** **35**
7 Una utilidad por fórmula calcula mal si sus padres no se llaman U1, U2… . . . . . 35
8 «Absorber padres» muere con ClassCastException en 12 redes del repositorio . . . 37
9 «Absorb parents» de una suma falla si un padre es una distribución (formato 1.0) 39
10 Descontar una utilidad exponencial cambia el coeficiente equivocado . . . . . 42
11 Escalar o descontar una fórmula o un producto de utilidades da otro número . . . 44
12 Una regresión no se calcula si un padre vale menos de una milésima . . . . . 47
13 Una covariable con el nombre sin llaves se acepta pero no se calcula nunca . . . . 49
14 Una regresión sin `Constant` o `Gamma` muere con un índice −1 sin mensaje . . . . . 52

15 Una utilidad de regresión no cuenta como utilidad al calcular . . . . . 54
16 Una tabla de fórmulas con `Complement` en vez de `#` no se puede proyectar . . . . . 57
17 Quitar una etiqueta referenciada del árbol deja la red sin poder evaluarse . . . . . 59
18 Añadir a una hoja del árbol la variable del nodo deja una red que no abre . . . . . 61
19 Tras cancelar el potencial de un suceso, la simulación usa el primer padre . . . . . 64
20 Un nodo con tabla de transiciones copiado ya no sabe a qué estado pasa . . . . . 67
21 Los nodos de sucesos pegados siguen leyendo los padres del nodo original . . . . . 69
22 Aceptar un árbol con sucesos deja un deshacer que borra la relación . . . . . 72
23 Se ofrece «Tree with Excluded Events» con solo padres suceso, y falla . . . . . 74
193 «Absorb parents» falla si un padre es una regresión . . . . . 76
234 Un nodo pegado junto con su padre no se calcula si su fórmula lo nombra . . . . . 78
257 Un riesgo de Weibull pegado con su variable de tiempo sigue usando la del nodo original . . . . . 79
260 Pegar un nodo o deshacer un cambio de estados borra el comentario de cinco clases de relación . . . . . 81
278 Un riesgo de Weibull da «probabilidades» negativas, mayores que uno o que no son números, sin avisar . . . . . 83

**4 Ediciones de la red: deshacer, cancelar y pegar 84**
24 Un dominio estándar de otro tamaño borra tablas que «Cancel» no devuelve . . . 84
25 Restringir un enlace cambia la tabla del hijo y deshacer no la devuelve . . . . . 86
26 Enlazar a una utilidad producto la convierte en suma y cambia el resultado . . . . 88
27 Rehacer un enlace y su restricción deja el enlace sin la restricción . . . . . 90
28 Pasar a suceso una decisión numérica sin política la deja a medias . . . . . 92
29 Mover un estado con un hijo sin potencial deja la tabla permutada a medias . . . 94
30 Deshacer un cambio de estados duplica tablas de los hijos y cambia resultados . . 96
31 Pegar en una red que ya incumple una restricción se rechaza siempre . . . . . 98
33 Deshacer o cancelar «mostrar el comentario al abrir» no la devuelve . . . . . 100
34 Deshacer o cancelar «Always append» de un suceso no devuelve su valor . . . . . 101
35 Una red expandida o aprendida no se marca como modificada ni pide guardar . . 103
36 En una red expandida o aprendida, deshacer no recuerda nada al principio . . . . 105
239 Quitar un padre a un nodo con fórmula la cambia por una uniforme que aún lo nombra . . . . . 107
243 Un dominio estándar de otro tamaño da un error inesperado en una decisión o en su padre . . . . . 108
258 Pegar un trozo con `X` y `X'` en una red que ya tiene `X` confunde las dos en las fórmulas 110
271 Un nodo temporal no se puede pegar en su propia red . . . . . 111
275 Quitarle un padre a un nodo le borra el comentario de su relación, salvo si es una fórmula o un modelo canónico . . . . . 112
280 Añadir o quitar un estado a una decisión con política impuesta deja la política con el tamaño viejo . . . . . 114
281 Convertir un nodo de azar en decisión conserva su tabla de probabilidad como política impuesta . . . . . 115
282 Cambiar el tipo de variable de una decisión sin política le impone una política uniforme . . . . . 117
283 El portapapeles guarda los nodos originales, no una copia: cortar y pegar deja las relaciones sin sus padres . . . . . 118
284 Pegar pierde las restricciones y las condiciones de revelación de los enlaces, y la marca «always observed» . . . . . 119
285 Quitar un padre a un nodo de utilidad deja una relación uniforme que impide evaluar la red . . . . . 121

286 Pasar una variable de estados finitos a discretizada borra las restricciones y las revelaciones de sus enlaces . . . 122
287 Deshacer un cambio en los criterios de decisión después de rehacerlo no lo deshace 123
288 Cambiar los estados o el tipo de variable de un padre de una utilidad deja su relación uniforme e impide evaluar la red . . . 124
289 Quitar el padre de un nodo de azar numérico con valores exactos, o pegarlo sin él, deja una relación uniforme que impide evaluar . . . 126

**5 Restricciones de red 127**
37 Abrir un fichero no comprueba las restricciones de la red . . . 127
38 Un estado puede quedarse con el nombre vacío o repetido . . . 129
39 Ocho restricciones opcionales no llegan a ninguna red . . . 131

**6 Evidencia, inferencia y variables numéricas 133**
40 Una variable numérica sin precisión se discretiza a un único valor cero . . . 133
41 Un hallazgo sobre un nodo numérico de azar hace fallar la propagación . . . 135
42 Entrar en modo inferencia falla si hay nodos de utilidad intermedios . . . 137
43 Un nodo numérico de azar sin potencial tumba la discretización . . . 140
44 Un hallazgo numérico con más de dos decimales se pierde al discretizar . . . 141
194 Un hallazgo sobre un nodo con delta se ignora antes de resolver y se rechaza después 144
45 Observar un nodo numérico de azar no informa sobre sus padres . . . 145
46 Un nodo de utilidad sin rango impide entrar en modo inferencia . . . 148
47 «Add finding» marca siempre el primer estado y pisa el hallazgo anterior . . . 150
48 La evidencia imposible se trata distinto según el gesto . . . 151
49 Construir una propagación cambia el tipo de análisis de la red del usuario . . . 154
227 En una DAN con restricciones, la propagación dice que los hallazgos son imposibles sin haber ninguno . . . 156
228 La propagación de MID-HPV falla al podar la red . . . 157
229 Tres MID no propagan porque la edad inicial es uniforme . . . 158
230 MID-CHAP-Ryan-Griffin no propaga: una expresión usa la edad sin valor . . . 159
231 La propagación de MID-Cochlear no termina en un minuto . . . 160
232 La red desplegada con «Expand network» no entra en modo inferencia . . . 161
233 Con transiciones a medio ciclo no se evalúa una red con una utilidad dada por un árbol . . . 162
267 Un nodo numérico con varios padres se discretiza mal: pierde valores y rechaza hallazgos posibles . . . 163
268 Observar un nodo numérico con un parámetro imposible acaba en un error de Commons Math que nadie explica . . . 165
269 Observar un nodo numérico con distribución muere si se ha visto otro número o tiene un padre numérico . . . 167
270 Un hallazgo donde la densidad es infinita deja la probabilidad de los padres en NaN 169
272 Una beta con un parámetro 0 solo se rechaza como tal en su extremo; dentro, el mensaje dice otra cosa . . . 170

**7 Decisiones: políticas, criterios, agentes y árbol de decisión 171**
50 El árbol de decisión falla en casi todas las redes en formato 1.0 . . . 172
51 «Show optimal policy» y «Show expected utility» no llegan a abrirse . . . 174
52 «Show expected utility» falla con utilidad constante y rellena ceros . . . 177
53 La utilidad esperada de una decisión ignora la evidencia previa . . . 179
54 Un hallazgo que descarta la primera opción de una decisión anula el cálculo . . . 181
55 Las ventanas de utilidad esperada y política óptima muestran la clave . . . 184
56 Deshacer «quitar la política impuesta» pierde la tabla de la política . . . 185

57 Renombrar un criterio de decisión no se deshace ni se cancela . . . 187
58 Confirmar un criterio sin cambiar su nombre se rechaza como repetido . . . 188
59 Evaluar una red de decisiones sin criterios falla sin explicar qué falta . . . 190
60 El agente de cada decisión se pierde al guardar y no se lee al abrir . . . 192
61 Quitar, renombrar o mover un agente descuadra las decisiones . . . 193
62 Las propiedades de una decisión no se abren si su agente ya no figura . . . 196
63 Evaluar DAN-arthronet falla con NullPointerException al instanciar . . . 197
64 Instanciar una variable de una red de decisiones duplica un enlace . . . 199
65 La estrategia óptima de una red sin decisiones lanza una excepción . . . 201
66 El botón del árbol de decisión se enciende en redes que no admiten árbol . . . 203
200 Una rama de una red de decisiones suma como utilidad el factor de un producto borrado . . . 204
210 Alt+clic sobre una decisión sin política impuesta da un error inesperado . . . 205
222 En una red de análisis de decisiones, saber qué se eligió en una decisión no cambia nada . . . 207
223 La estrategia óptima de una DAN no se abre si ya se sabe qué se eligió en su única decisión . . . 209
224 En una DAN, un hallazgo previo sobre una variable que restringe a otras impide evaluarla . . . 210
290 En una DAN, un hallazgo cuya probabilidad depende de una decisión cambia la utilidad o la estrategia . . . 211

**8 Modelos temporales y evolución temporal 212**
67 Con evidencia, las curvas de la evolución temporal no suman uno . . . 213
68 La evolución temporal de una variable que empieza en el ciclo 1 falla . . . 215
69 La evolución temporal de una decisión temporal muere con una excepción . . . 217
70 «Same as previous» revienta si el ciclo anterior no tiene la variable . . . 220
71 Al expandir un modelo temporal, las regresiones siguen nombrando el ciclo 0 . . . 222
72 Pasar una variable a atemporal puede dejar dos con el mismo nombre . . . 224
73 La duración del ciclo escrita en las propiedades de la red no se aplica . . . 226
211 Al sumar los estados, el nombre de la curva sale cortado: «Therapy [0» . . . 228
259 Una tabla de fórmulas copiada al ciclo siguiente se calcula sobre el ciclo anterior . 229
261 Un «Same as previous» en el primer ciclo de una variable pasa sin aviso y acaba en un error confuso . . . 231

**9 Coste-efectividad y análisis de sensibilidad 233**
74 El jar publicado mezcla dos JFreeChart y muchas gráficas no se abren . . . 233
75 La araña de coste-efectividad reescribe la red abierta con las medias . . . 236
76 Una columna con incertidumbre solo en algunas casillas se sortea mal . . . 239
77 El análisis probabilístico de sensibilidad no se puede repetir . . . 241
78 El diálogo de incertidumbre rechaza toda Dirichlet sobre dos o más estados . . . 243
79 Editor y muestreador reparten distinto el complemento junto a una Dirichlet . . . 245
80 El mapa de dos parámetros elige mal la mejor opción si hay tres o más . . . 247
81 En el mapa de dos parámetros el empate y la tercera opción salen en gris . . . 248
82 El mapa de dos parámetros pinta toda la zona de datos de un solo color . . . 250
83 El mapa no se abre si gana en todo él la misma opción y no es la primera . . . 251
84 La tabla de intervenciones fronterizas nombra otra opción si dos empatan . . . 253
85 El coste-efectividad falla si un hallazgo hace imposible una sola opción . . . 255
86 Las ventanas de coste-efectividad fallan si una opción es imposible . . . 257
87 El escenario del ámbito de decisión puede ser imposible y no se comprueba . . . 260
88 Con valores todos negativos, el tornado dibuja barras hasta cero . . . 262
89 El tornado por decisión no dibuja el rango real de la diferencia . . . 264

90 Con utilidades negativas, «Plot» y la araña aplastan y recortan las curvas . . . . . 266
91 El tornado con dieciséis parámetros o más deja barras fuera del recuadro . . . . . 268
92 El análisis de sensibilidad no se abre sobre una red sin decisiones . . . . . 269
93 Los parámetros inciertos salen en otro orden en cada apertura del análisis . . . . . 271
94 Ocultar una opción en una pestaña deja marcada su casilla en la otra . . . . . 273
255 El mapa de dos parámetros estira sus ejes hasta el cero aunque el intervalo no llegue 274
256 Un 0 % o un intervalo sin anchura impide abrir «Plot», el tornado y la araña, y el mapa . . . . . 276
262 Aceptar el análisis de sensibilidad devuelve el escenario a su primer estado y analiza otro . . . . . 277
277 Después de un análisis de sensibilidad, el escenario de otra decisión sale fijo con lo elegido antes . . . . . 279

**10 Simulación de sucesos discretos 280**
95 Una utilidad acumulativa vale cero hasta el primer suceso que la alcanza . . . . . 280
96 La simulación se cae si un nodo espera a un padre de un suceso no ocurrido . . . . 283
97 Un ciclo entre nodos de azar de una red de sucesos impide simular . . . . . 285
98 Una tabla o valor exacto con un suceso padre hace caer la simulación . . . . . 287
99 Añadir o quitar un enlace deja una relación uniforme y la simulación falla . . . . . 290
100 Una tabla de vida «con tasas» se simula como si fueran probabilidades . . . . . 293
101 La relación «External» se ofrece, elegirla falla y no trae valores de fuera . . . . . 294
102 Resultados de la simulación: la media descontada va al criterio equivocado . . . . 296
103 El CSV de la simulación de sucesos tiene los títulos corridos una columna . . . . . 298
104 Simular con traza una red de sucesos sin guardar falla antes de empezar . . . . . 299
105 Un fallo de «Tree with Excluded Events» abre una ventana desde el núcleo . . . . 301
106 Un nodo de azar discretizado impide simular con un error inesperado . . . . . 303
107 La hoja de la propagación estocástica cambia el orden de los hallazgos . . . . . 305
108 La propagación estocástica no rechaza nodos numéricos: falla sin explicarlo . . . . 307
109 Las casillas de estadísticos del panel de Monte Carlo no hacen nada . . . . . 309
110 «Sort probnet» falla en una red de sucesos con un nodo enlazado a sí mismo . . . 311
214 «Cost effectiveness analysis» del menú Tools no sabe evaluar una red de sucesos . 312
252 El CSV de una simulación pierde columnas cuando una media se escribe con exponente . . . . . 313

**11 Abrir y guardar redes 314**
111 Un .pgmx mal escrito se abre sin aviso y con todas las tablas uniformes . . . . . 315
112 Guardar en formato 1.0 deja la red sin poder evaluarse al reabrirla . . . . . 316
113 Guardar en formato 0.2 convierte los nodos de producto en uniformes . . . . . 320
114 «Save» falla en tres redes con un papel de potencial mal escrito . . . . . 323
115 Una distribución desconocida en una red de sucesos falla lejos de la causa . . . . . 326
116 Una tabla de sucesos con incertidumbre en el fichero impide abrir la red . . . . . 328
117 Abrir un .xml que no es ProbModelXML falla con una excepción de índice . . . . 331
202 El aviso de agente desconocido no dice de qué ciclo es la variable . . . . . 333
235 Guardar una red de sucesos en formato 0.2 pierde sus relaciones y luego no se abre 334
236 Un árbol con sucesos mal escrito en el fichero se abre sin aviso, con el árbol por omisión . . . . . 335
237 Los ficheros guardados con la versión de la rama de sucesos no se abren . . . . . 336
273 El mensaje que se niega a guardar una relación nombra su nodo con el instante dos veces . . . . . 337
274 El escritor de 0.2 guarda una tabla de fórmulas como una tabla, y al abrirla las fórmulas ya no están . . . . . 337

276 Un indicador sin «TTE» o sin «ProbabilityOccurrence» en el fichero revienta con un puntero nulo . . . . . 338

**12 Bases de casos** **339**

118 Un «?» en una base de Weka descuadra las filas e inventa casos sin avisar . . . . . 339
119 Un fichero de Weka en UTF-8 con tildes o «ñ» pierde estados y lee ceros . . . . . 342
120 Una columna numérica, de texto o de fecha en Weka da NullPointerException . . 343
121 Weka con @Relation, @Attribute o @Data en mayúsculas mezcladas no se abre . . 344
122 Una base con una columna de números guardada en Weka no se puede reabrir . . 346
123 Un fichero de Weka en una carpeta con tilde, «ñ» o «+» no se puede reabrir . . . 347
124 Una línea de CSV con campos de menos, de más o en blanco inventa casos . . . . 348
218 En el generador de casos, cancelar «Load Net from file» deja «Open» apagado . . 350
246 Una base con una casilla en blanco, guardada en formato Weka, no se vuelve a abrir 351
247 Un comentario de Weka detrás de lo que dice una línea hace que se rechace el fichero 352
248 Un fichero escrito por Weka con « %», comillas dobles o un tabulador en un valor se lee con barras de más . . . . . 354
249 Un «`'?'`» que la columna no declara se rechaza o se acepta según lo que haya en las líneas de antes . . . . . 355
250 Un fichero de Weka en UTF-8 con marca de orden de bytes ya no se abre, y el mensaje no dice por qué . . . . . 356

**13 Aprendizaje y evaluación de redes** **357**

125 La exactitud del Bayes ingenuo selectivo usa el caso evaluado al entrenar . . . . . 357
126 El Bayes ingenuo selectivo elige características sin el alfa del diálogo . . . . . 359
127 El superpadre pesa sus enlaces con cuentas de casillas equivocadas . . . . . 361
128 El superpadre penaliza la clase verdadera: ningún enlace sube la exactitud . . . . 362
129 El superpadre puntúa los enlaces candidatos con la probabilidad al revés . . . . . 365
130 El superpadre cuenta dos veces la hija de cada enlace al medir la exactitud . . . . 367
131 La validación cruzada de dos clasificadores no prueba un tercio de casos . . . . . 370
132 El superpadre por defecto no aprende nada: propone un enlace a sí mismo . . . . 371
133 El superpadre acepta el primer enlace aunque empeore al Bayes ingenuo . . . . . 373
134 Superpadre interactivo: llenar la tabla avanza el algoritmo y la vacía . . . . . 375
135 El Bayes ingenuo selectivo hacia delante no da dos veces el mismo modelo . . . . . 379
136 El árbol aumentado (TAN) orienta sus enlaces distinto en cada aprendizaje . . . . 380
137 El EM deja sin aprender los nodos con potencial uniforme . . . . . 382
138 El esqueleto del algoritmo PC cambia al renombrar una variable . . . . . 383
139 El PC interactivo ofrece las dos orientaciones de un enlace, contra Meek . . . . . 386
140 El PC interactivo adelanta la fase al llenar la tabla y descoloca botones . . . . . 388
141 «Finish» del PC interactivo empieza de cero y descarta lo aplicado a mano . . . . 392
142 PC interactivo: con un borrado bloqueado, «Finish» da NullPointerException . . 395
143 El bosque aumentado (FAN) falla sin características ligadas a la clase . . . . . 398
144 Discretizar por frecuencias iguales con decimales falla o cuenta mal . . . . . 399
145 «Split Dataset» lanza siempre una excepción, elija la opción que elija . . . . . 401
146 Validación cruzada: casos por pliegue truncados y pérdida por caso inflada . . . . 403
147 Quedarse sin memoria al aprender sale como error no previsto . . . . . 405
148 Una variable de la red modelo que falta en los datos da error no previsto . . . . . 406
149 Red modelo sin alguna columna de la base: error no previsto al cargarla . . . . . 408
215 En «Data preprocessing» no se ve la lista «Outliers», y «Cross validation» sale cortada . . . . . 410
216 El rótulo «Classification variable» se queda gris con la base de casos cargada . . . 411
217 «Reset values» de la ventana de aprender deja «Learn» encendido, y pulsarlo dice que no hay datos . . . . . 412

219 Al cargar otra base en «Data preprocessing», las casillas de las variables salen apagadas . . . 413
240 El árbol aumentado (TAN) falla con NoSuchElementException si no hay ninguna característica . . . 413
241 El TAN interactivo sin «Only allowed edits» no enseña los mejores enlaces si hay más de 50 . . . 415
251 El PC interactivo sigue pasando de fase al pedir la mejor propuesta, según las casillas marcadas . . . 417

**14 Interfaz: editor de red y paneles de las relaciones 418**
150 Volver a elegir la clase de relación que ya tiene un nodo borra sus números . . . 418
151 En una regresión, lo escrito como Cholesky se pierde al guardar la red . . . 421
152 Añadir una fila a la tabla de vida y aceptar borra el primer tramo . . . 423
153 El editor de una distribución univariante enseña «1» en cada parámetro . . . 424
154 En la tabla de utilidad, una restricción de enlace bloquea otras casillas . . . 426
155 La ventana de restricción de un enlace edita la tabla del padre . . . 428
156 La tabla de utilidad enseña 922 337 203,685478 en vez de valores mayores . . . 431
157 El panel de Weibull deja borrar «Gamma» y la relación revienta al calcular . . . 432
158 En el panel de una regresión no se ve la columna de las covariables . . . 434
159 «Add variables to potential» en una hoja del árbol no añade ningún padre . . . 436
160 El árbol ofrece repetir una variable discretizada y elegirla da un error . . . 437
161 «Split interval» con el campo vacío o sin un número da un error inesperado . . . 439
162 La pestaña «Parents» ofrece padres que las restricciones rechazan . . . 441
163 El límite superior del último intervalo de revelación no se puede cambiar . . . 443
164 Un número no válido en el indicador da un error y vacía la pestaña . . . 444
165 El doble clic en la ventanilla de expresiones mete dos veces el texto . . . 446
166 Tras cambiar el tipo de un nodo, una copia suya no encuentra sus padres . . . 447
167 Una columna con incertidumbre fuera del primer estado se deja editar . . . 449
168 Tras alejar el zoom, los nombres de estado siguen recortados al acercarlo . . . 451
169 Con precisión 0.0, 2.0 o 0.5, los límites de los tramos acaban en punto . . . 452
170 Tras A, B y C, el nodo de azar nuevo se llama «E», la letra de los sucesos . . . 454
207 El selector de precisión enseña «1» si la precisión es 0.0001 o no está en su lista . 455
209 «Distribution» y «Parametrization» se quedan a la vista sin lista al lado . . . 456
225 En el panel de una distribución no se puede escribir ningún parámetro . . . 457
226 El diálogo de incertidumbre no acepta una columna con distribuciones de Dirichlet 458
238 Marcar «Use first interval» no cambia la fórmula que se ve en el panel . . . 459
242 En modo inferencia, el valor de una utilidad sin rango se sale de la caja del nodo . 460
245 En el árbol de un nodo padre de sí mismo, «Add subtree» ofrece su variable y «Change root variable» no . . . 461
263 En la pestaña Domain de un nodo de utilidad, la precisión dice siempre «1» . . . 463
264 Una precisión «NaN» o «Infinity» en el fichero impide abrir las propiedades del nodo 465
265 La casilla del tiempo del indicador acepta «NaN», «∞» y «1E400», y el suceso deja de ocurrir . . . 466
266 Un indicador leído con un número fuera de rango hace fallar el diálogo del nodo . 468
279 En un modelo temporal, «OK» en las propiedades de un nodo con gemelo en otro ciclo se rechaza como nombre repetido . . . 470

**15 Interfaz: menús, diálogos y herramientas 472**
171 Abrir una red desde internet apaga «Guardar» para todas las redes . . . 472
172 «Edits history» deja un núcleo al cien por cien y puede colgar el programa . . . 474
173 Aceptar los dominios estándar sin marcar ninguno lanza una excepción . . . 475
174 Aceptar los criterios estándar sin marcar nada borra los criterios de la red . . . 477

175 «OpenMarkov configuration» edita unas preferencias que el programa no usa . . . 480
176 Las escalas de criterio se redondean a tres decimales y pueden quedar en 0 . . . . 482
201 El modo inferencia solo se ofrece a diagramas de influencia y redes bayesianas . . 484
203 «Select all» está apagada hasta que se elige una herramienta de edición . . . . . 485
204 La ventana «About» se abre con 24 píxeles de ancho . . . . . 486
206 «Delete» sigue encendido sin ninguna fila señalada, y pulsarlo da un error . . . . . 487
220 En el tema oscuro hay cinco cosas que no se ven o no se leen . . . . . 488
221 Tras pasar por otro tema, «Sync with OS» puede dejar de seguir al sistema . . . . 490
253 Un descuento escrito en tanto por ciento se guarda redondeado y se enseña con dieciocho decimales . . . . . 491
254 Con una duración del ciclo no positiva en el fichero, «OK» no cierra las propiedades de la red . . . . . 492

**16 Textos y títulos que se ven mal 494**
177 La ayuda de atajos da dos teclas equivocadas, y una cierra el programa . . . . . 494
178 Pegar un nodo de sucesos donde no se admite da «»> eventNode is null «<» . . . 495
179 El selector del tipo de relación enseña nombres internos como «ProbTable» . . . . 497
180 El diálogo de dominios estándar se abre sin título . . . . . 498
181 Seis de las ventanillas del editor de árboles se abren sin título . . . . . 498
182 Varios títulos de error parten las siglas: «tree a d d», «X m l invalid» . . . . . 500
183 El aviso de zoom fuera de rango dice «Requested zoomManager» . . . . . 502
184 El rótulo del panel del indicador dice «Probability of ocurrence» . . . . . 503
185 La ayuda de «Add → Utility node» dice «Create an utility node» . . . . . 503
186 El mensaje de evidencia incompatible dice «old evidente» y es ilegible . . . . . 504
187 El error de muestras sin peso dice «Samples weigth is 0» . . . . . 506
188 La ayuda de «Unique superparent» dice «commont» . . . . . 508
205 Sin ninguna red abierta, el menú Edit dice «Switch to edition mode» . . . . . 508
208 Varios selectores dicen «Cost criterion named …» y «state no» en vez del nombre . 509
212 El diálogo del análisis de sensibilidad se abre sin título . . . . . 510
213 El título de las opciones de la evolución temporal pega el nombre: «of:MID-…» . . 511

**17 Rendimiento 511**
189 Entrar en modo inferencia con una red grande congela la ventana . . . . . 511
190 Cada resultado de una operación sobre tablas reserva una tabla que tira . . . . . 513
191 Cada tabla de probabilidad nueva se rellena dos veces con el mismo valor . . . . . 514
192 La maximización reserva la tabla de la unión de variables sin usarla . . . . . 515

**18 Exportación a Amua 516**
195 Amua recibe coste y efectividad cruzados si la red pone antes la efectividad . . . . 517
196 El escenario base de Amua es siempre la primera opción de la decisión . . . . . 517
197 Probabilidades que no suman uno se exportan a Amua como uniformes . . . . . 518
198 Amua recibe las probabilidades y los valores redondeados a cuatro decimales . . . 519
199 Una decisión por debajo del primer nivel se rechaza con «Decision node not found» 520
244 Un criterio sin unidad deja el modelo de Amua con una dimensión de menos . . . 521

**A Lo que se comprobó y no entra en la lista 522**
Ya corregidos . . . . . 522
No son errores . . . . . 522
Defectos sin consecuencia visible hoy . . . . . 527
Defectos que hoy no alcanza nadie . . . . . 530
Fuera de alcance por decisión de equipo . . . . . 542
Propuestas de diseño y limpieza . . . . . 546

**B Procedencia de cada ficha** **552**

## Cómo leer este documento

**Qué código se ha comprobado.** El de la rama `development` tal como queda al unir los últimos commits locales (hasta `6260a4c`, del 5 de septiembre) con los últimos de `origin/development` (hasta `fb19d55`). El 22 de septiembre se revisó la lista contra `40a9de9`, que añade diez commits: se volvió a comprobar cada error cuyo código o cuyo camino tocan esos commits. Uno quedó corregido y ha salido de la lista (nota 184); tres están ahora corregidos en parte (errores 29, 43 y 99). Los errores conservan su número, así que falta el 32. En las fichas revisadas, las líneas de la fila *Ficheros* son las de `40a9de9`; las que se citan en el texto pueden ser todavía las del código anterior. Con el tiempo se irán desplazando, así que conviene buscar por el nombre del método.

**Revisión del 7 de octubre.** Se revisó la lista contra `2b4fc61`, que añade dos commits de Jorge Rico (`eafc412` y `2b4fc61`). Cambian tres cosas que tocan a esta lista: los gestos del editor de red pasan de `EditorInputHandler` a una clase nueva, `EditorCommandHandlerGenerator`, con teclas y botones del ratón reasignables; quince clases de la simulación de sucesos pasan del módulo `inference.des` al módulo `gui` (paquete `org.openmarkov.gui.desnets`); y se borran `Filter` y `PluginManager`. Ningún error queda corregido ni cambia de estado. En las fichas de los errores sin corregir se han puesto al día el módulo, el paquete y las rutas; y las líneas de la fila *Ficheros* y de los títulos de los extractos de código, para los ficheros que tocan esos commits, son ahora las de `2b4fc61`. Las líneas que se citan en el texto no se han tocado. Los errores 48 y 170 pasan por la clase nueva, y las notas 172 y 175 se han reescrito porque esos commits hacen parte de lo que proponían.

Ese día se revisaron además las filas *Corregir antes*: se han quitado las que nombraban un error ya corregido (en los errores 24, 132 y 133), y las que quedan están dibujadas en la fila *Grafo* de cada ficha y, todas juntas, en el grafo del resumen. Y se volvió a medir el error 134, que se apoyaba en los errores 132 y 133, ya corregidos: pasa a estar presente en parte y su ficha se ha reescrito con las medidas nuevas.

**Correcciones marcadas el 23 de septiembre.** Los catorce commits que van de `bbde667` a `522d926` corrigen errores de esta lista. Esos errores no salen de ella: se quedan en su sitio con la marca **Corregido** o **Corregido en parte** en la tabla del resumen y en su título, y la fila *Estado* de su ficha dice qué commit los corrige y, si la corrección es parcial, qué sigue presente. Corregidos del todo: errores 1, 2, 3, 4, 6, 7, 8, 10, 11, 12, 13 y 14. Corregidos en parte: errores 9 y 157. El resto de la ficha (qué pasa, el código, cómo arreglarlo) describe el código anterior a esos commits. El error 1 se da por corregido sin tocar lo que quedaba de él, por lo que explica la nota 185; al revisarlo aparecieron el error 193 y la nota 186.

**Correcciones marcadas desde el 29 de septiembre.** Se marcan del mismo modo, cada una con el commit que la hace: errores 15, 193, 26, 29, 30, 35, 36, 41, 43, 46, 47, 49, 50, 125, 126, 127, 128, 129, 130, 131, 132, 133, 135, 137, 138, 142, 143, 144, 145, 146, 147, 148, 149, 52, 53, 85, 54, 56, 57, 59, 62, 61, 60, 63, 65, 66, 113, 117, 151, 153, 225, 169, 167, 226, 76, 202, 42, 223, 222, 68, 83, 74, 69, 67, 48, 86, 77, 136, 97, 227, 228, 232, 230, 229, 5, 16, 40, 111, 112, 115, 116, 19, 20, 21, 82, 81, 80, 102, 103, 176, 75, 150, 89, 88, 90, 18, 159, 140, 200, 73, 84, 195, 196, 198, 118, 119, 121, 122, 123, 152, 107, 93, 22, 34, 70, 72, 166, 170, 207, 164, 45, 44, 79, 87, 95, 235, 236, 239, 267, 270, 269, 268, 265, 266, 262, 257, 258, 160, 23, 161, 260, 105, 199, 244, 17, 245, 55, 280, 281, 282, 283, 279, 285, 286, 288, 284, 274, 287, 231, 201, 259, 243, 224, 289, 78, 275, 253 y 261. Al corregir el 21 apareció el error 234, corregido también. Al corregir el 41 apareció el error 194; al corregir el 63, el error 200, al corregir el 280, el error 288, al corregir el 285, el error 289, y al comprobar el arreglo del 224, el error 290. El error 201 lo vio Manuel Arias en la ventana al comprobar el 66. Al comprobar en la ventana el arreglo del 153 apareció el error 225, y al medir las reglas de

familia para el 76, el error 226. El error 24 queda corregido en parte por el commit `461cd46`, de Jorge Rico, y el 71, por el `971f541`, el mismo que corrige el 230. Los errores 51 y 209 los corrige el commit `295a7a3`, de Jorge Rico. El error 64 lleva la marca **Cerrado sin arreglo**: el defecto sigue en el código, pero no se ha encontrado nada que un usuario pueda ver por su causa, y la fila *Estado* de su ficha dice qué se midió. El error 44 la llevó hasta que lo corrigió el arreglo del 45.

**Errores añadidos el 30 de septiembre.** Los errores 195 a 199, del exportador a Amua (sección 18), se anotaron el 28 de septiembre en una copia aparte de esta lista; se han pasado a ella tras comprobar que siguen presentes.

**Errores añadidos el 9 de octubre.** Los errores 202 a 224 y las notas 187 a 191 se encontraron al portar OpenMarkov a C++ y Qt, es decir, al escribir una segunda versión del programa traduciendo esta clase por clase y comparando lo que hacen las dos: unos al traducir una clase o al llevar allí los arreglos del 30 de septiembre al 8 de octubre, y otros al poner las ventanas de las dos versiones una al lado de la otra. Cada uno se ha vuelto a comprobar contra `6312fd4`, el último commit a día de hoy, y la fila *Comprobado* de su ficha dice cómo. Lo que no se pudo confirmar, lo que ya estaba en la lista y lo que una decisión de equipo deja fuera no se ha añadido.

Varias de esas fichas dicen «visto en una captura automática». El 8 de octubre se arrancó el programa con un guion que abre redes, pulsa las entradas de los menús y guarda de cada ventana que se abre una imagen, su título y el texto de sus componentes. Las capturas son del commit `b2e4c1f`; de las clases que nombran esas fichas, entre ese commit y `6312fd4` solo ha cambiado `Main` `PanelMenuAssistant`, que se ha vuelto a leer. Nadie ha repetido esos pasos a mano en la ventana.

Con lo visto en esa comparación se han ampliado además tres fichas que ya estaban: los errores 42, 51 y 153.

**Errores añadidos el 15 de octubre.** Los errores 235 a 239 y las notas 192 a 195 salen de revisar la rama `DES-development` de los antiguos repositorios por módulos, en la que Carmen Yago desarrolló la simulación de sucesos entre 2018 y 2026 y que nunca se fusionó. Jorge Rico trajo su trabajo a mano el 20-05-2026 y después se reorganizó la interfaz, así que se comparó cada función de la rama, no cada fichero, con el código de `9adb77b`. No se ha perdido ninguna función: lo que no llegó es código viejo o de prueba, o cosas que la rama dejó a medias. Lo que sí falta o llegó mal está en esos errores, comprobados uno a uno con la fila *Comprobado* de su ficha; el error 99 y el 105 ya lo recogían en parte. Lo que solo afectaba a los textos en castellano no se ha añadido.

**Más errores añadidos el 15 de octubre.** Los errores 240 a 271 y las notas 196 a 199 se encontraron también al portar OpenMarkov a C++ y Qt, como los del 9 de octubre: al llevar a esa versión los arreglos del 9 al 15 de octubre y los cambios de Jorge Rico en el editor de red. Cada uno se ha vuelto a comprobar contra `9adb77b`, y la fila *Comprobado* de su ficha dice cómo. Varios son lo que quedó de un error que ya está marcado como corregido, y lo citan en la fila *Relacionados*. Lo que ya estaba en la lista, lo que ya se ha corregido, lo que una decisión de equipo deja fuera y lo que era un fallo de una prueba y no del programa no se ha añadido. Con lo visto se han ampliado además dos fichas que ya estaban: los errores 114 y 210.

**Errores añadidos la tarde del 15 de octubre.** Los errores 272 a 278 se encontraron al llevar a la versión en C++ y Qt los arreglos que esta lista recibió la tarde del 15 de octubre, de `208605c` a `aa4bcfb`. Cada uno se ha comprobado con el Java de ese momento —el jar del 14 de octubre con las clases de esos commits delante—, y la fila *Comprobado* de su ficha dice cómo. Tres son lo que dejaron atrás arreglos de ese día: el error 272 el del 268, los 273 y 274 el del 235, y el 275 el del 239.

**Errores añadidos la noche del 15 de octubre.** Los errores 279 a 287 salen de revisar el código de la edición de redes —las ediciones de `core/action` y de `gui/action`, el diálogo de propiedades del nodo y el portapapeles— buscando defectos que no estuvieran ya en la lista. Cada uno se ha

comprobado con el código de `3a93386`, ejecutando la edición sobre redes pequeñas construidas en código y, cuando la ficha lo dice, abriendo de verdad el diálogo; la fila *Comprobado* de cada ficha dice cómo. El error 279 afecta a todos los modelos temporales: las propiedades de un nodo con gemelo en otro ciclo no se pueden aceptar. Lo que ya estaba en la lista no se ha repetido; las fichas nuevas lo citan en *Relacionados*.

**Errores asignados el 29 de septiembre.** Jorge Rico se encarga de los errores que llevan la marca **Jorge Rico** en la tabla del resumen y en su título, y la fila *Asignado a* en su ficha: errores 24, 25, 27, 28, 33, 37, 38, 114, 124, 154, 155, 156, 157, 158, 162, 163, 165, 168, 171, 173, 174, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 188, 189, 190, 191 y 192. De los errores 24 y 157 le queda la parte que sigue presente.

**Cómo se ha comprobado.** Para cada error se leyó el código de hoy, se buscaron todos los sitios que llaman a la parte afectada y se estableció por dónde llega un usuario real (qué menú, qué diálogo, qué fichero) o un llamador real del programa. Muchos se ejecutaron además con pequeños programas de prueba sobre el jar de la aplicación, sin abrir la ventana. La fila *Comprobado* de cada ficha dice si el error se comprobó leyendo el código, ejecutándolo o de las dos maneras; cuando una consecuencia solo se ha deducido leyendo, la descripción lo dice.

**Cómo está hecha cada ficha.** Cada error tiene un número, corrido en todo el documento, y un título que dice la consecuencia. Debajo va una ficha con:

- **Estado**: presente, o presente en parte (y entonces qué parte ya está corregida).
- **Asignado a**, solo si lo hay: quién se encarga de corregirlo.
- **Módulo**, **paquete** y **ficheros** con sus líneas, con las rutas relativas a la raíz del repositorio.
- **Esfuerzo** estimado: pequeño (horas), medio (días) o grande (semanas).
- **Decisión de equipo**: si el arreglo no se puede hacer sin que el equipo decida antes algo (qué debe hacer el programa, qué ficheros se aceptan…), qué hay que decidir. Estos errores conviene no empezarlos hasta tener la decisión.
- **Corregir antes**, solo si lo hay: el error que hay que corregir antes que este, o a la vez como mínimo, porque sin él este arreglo no basta o rompe otra cosa; y por qué. Solo se nombran errores que siguen sin corregir: cuando el otro se corrige, la fila se quita.
- **Grafo**, debajo de la fila anterior: todos los errores sin corregir que van antes que este, también los que van antes de aquellos, y en qué orden. Una flecha de A a B dice que A se corrige antes que B; un error al que no llega ninguna flecha no espera a otro. El error de la ficha es el del borde grueso.
- **Tapado por**, solo si lo hay: el error que hoy impide ver este, o comprobar su arreglo desde la aplicación, y dónde. Este sí se puede corregir sin esperar al otro.
- **Relacionados**: otros errores que tocan el mismo código o conviene arreglar a la vez, y notas del apéndice sobre lo mismo.

Después vienen tres apartados: *Qué pasa* (la consecuencia, la causa en el código y lo que se midió), *El código* (extractos del código actual, cuando ayudan) y *Cómo arreglarlo* (por dónde ir, qué otros sitios deben cumplir la misma regla y qué prueba lo demostraría).

**Decisiones de equipo ya tomadas que se han tenido en cuenta.**

- **La aplicación es solo en inglés.** No se añaden traducciones. Lo que solo afectaba a los textos en castellano, o a que un texto inglés esté escrito en el código en vez de en un fichero

de textos, no entra en la lista; sí entra lo que se ve mal en la aplicación en inglés.

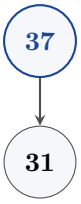
- **El formato Elvira está congelado**: no se arregla, no se migra y no se retira.
- **Dos cambios de la interfaz son intencionados**: reabrir al arrancar las redes de la última sesión, y crear los nodos con doble clic en el lienzo en vez de con botones.

## Resumen

La lista tiene **289 errores**; 145 de ellos son defectos nuevos: 69 encontrados al comprobar o corregir los demás, 62 al portar OpenMarkov a C++/Qt, 5 al revisar la rama `DES-development` y 9 al revisar el código de la edición de redes. **41** necesitan antes una decisión del equipo. Por esfuerzo estimado: 249 pequeños, 40 medios y 0 grandes. De los 289, 183 están ya corregidos, 10 lo están en parte y 1 está cerrado sin arreglo; 36 están asignados a Jorge Rico. El apéndice recoge además 199 notas con lo que se comprobó y no entra en la lista.

| Tema | Errores |
|---|---|
| 1. Operaciones con potenciales | 3 |
| 2. Modelos canónicos y modelo de ajuste | 3 |
| 3. Relaciones definidas por fórmulas, regresiones, árboles y sucesos | 22 |
| 4. Ediciones de la red: deshacer, cancelar y pegar | 27 |
| 5. Restricciones de red | 3 |
| 6. Evidencia, inferencia y variables numéricas | 23 |
| 7. Decisiones: políticas, criterios, agentes y árbol de decisión | 23 |
| 8. Modelos temporales y evolución temporal | 10 |
| 9. Coste-efectividad y análisis de sensibilidad | 25 |
| 10. Simulación de sucesos discretos | 18 |
| 11. Abrir y guardar redes | 14 |
| 12. Bases de casos | 13 |
| 13. Aprendizaje y evaluación de redes | 32 |
| 14. Interfaz: editor de red y paneles de las relaciones | 33 |
| 15. Interfaz: menús, diálogos y herramientas | 14 |
| 16. Textos y títulos que se ven mal | 16 |
| 17. Rendimiento | 4 |
| 18. Exportación a Amua | 6 |

**Errores que hay que corregir antes que otros.** El grafo recoge las filas *Corregir antes* de las fichas. Una flecha de A a B dice que A hay que corregirlo antes que B, o a la vez como mínimo. Solo aparecen errores sin corregir; en azul, los asignados a Jorge Rico. Los demás errores sin corregir no esperan a ninguno.




La tabla siguiente es la lista completa, en el orden del documento. La última columna da el esfuerzo (S, pequeño; M, medio; L, grande) y, en rojo, si hace falta una decisión de equipo.

| Nº | Error | Módulo | Esf./Dec. |
|---|---|---|---|
| **1. Operaciones con potenciales** | | | |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 1 | **Corregido** · Multiplicar potenciales constantes pierde el criterio de la utilidad | `core` | S | No |
| 2 | **Corregido** · Tablas de más de $2^{31}$ casillas: excepciones distintas o resultado falso | `core` | S | No |
| 3 | **Corregido** · Copiar un potencial a una red sin todas sus variables mete nulos | `core` | S | No |
| **2. Modelos canónicos y modelo de ajuste** | | | | |
| 4 | **Corregido** · El modelo de ajuste se proyecta con otro orden y otros decimales cada vez | `core` | S | No |
| 5 | **Corregido** · Parámetros que no suman uno se abren sin aviso y el muestreo discrepa | `core` | S | No |
| 6 | **Corregido** · El modelo de ajuste se ofrece a políticas sin padres y en redes de sucesos | `core` | S | No |
| **3. Relaciones definidas por fórmulas, regresiones, árboles y sucesos** | | | | |
| 7 | **Corregido** · Una utilidad por fórmula calcula mal si sus padres no se llaman U1, U2… | `core` | S | No |
| 8 | **Corregido** · «Absorber padres» muere con ClassCastException en 12 redes del repositorio | `core` | S | No |
| 9 | **Corregido en parte** · «Absorb parents» de una suma falla si un padre es una distribución (formato 1.0) | `core` | M | Sí |
| 10 | **Corregido** · Descontar una utilidad exponencial cambia el coeficiente equivocado | `core` | S | No |
| 11 | **Corregido** · Escalar o descontar una fórmula o un producto de utilidades da otro número | `core` | M | Sí |
| 12 | **Corregido** · Una regresión no se calcula si un padre vale menos de una milésima | `core` | S | No |
| 13 | **Corregido** · Una covariable con el nombre sin llaves se acepta pero no se calcula nunca | `io` | M | Sí |
| 14 | **Corregido** · Una regresión sin `Constant` o `Gamma` muere con un índice $-1$ sin mensaje | `core` | S | No |
| 15 | **Corregido** · Una utilidad de regresión no cuenta como utilidad al calcular | `core` | S | No |
| 16 | **Corregido** · Una tabla de fórmulas con `Complement` en vez de `#` no se puede proyectar | `core` | S | No |
| 17 | **Corregido** · Quitar una etiqueta referenciada del árbol deja la red sin poder evaluarse | `gui` | S | Sí |
| 18 | **Corregido** · Añadir a una hoja del árbol la variable del nodo deja una red que no abre | `core` | M | Sí |
| 19 | **Corregido** · Tras cancelar el potencial de un suceso, la simulación usa el primer padre | `core` | S | No |
| 20 | **Corregido** · Un nodo con tabla de transiciones copiado ya no sabe a qué estado pasa | `core` | S | No |
| 21 | **Corregido** · Los nodos de sucesos pegados siguen leyendo los padres del nodo original | `gui` | M | No |
| 22 | **Corregido** · Aceptar un árbol con sucesos deja un deshacer que borra la relación | `core` | S | No |
| 23 | **Corregido** · Se ofrece «Tree with Excluded Events» con solo padres suceso, y falla | `core` | S | No |
| 193 | **Corregido** · «Absorb parents» falla si un padre es una regresión | `core` | S | No |
| 234 | **Corregido** · Un nodo pegado junto con su padre no se calcula si su fórmula lo nombra | `core` | S | No |
| 257 | **Corregido** · Un riesgo de Weibull pegado con su variable de tiempo sigue usando la del nodo original | `core` | S | No |
| 260 | **Corregido** · Pegar un nodo o deshacer un cambio de estados borra el comentario de cinco clases de relación | `core` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 278 | Un riesgo de Weibull da «probabilidades» negativas, mayores que uno o que no son números, sin avisar | `core` | S | Sí |
| **4. Ediciones de la red: deshacer, cancelar y pegar** | | | | |
| 24 | **Corregido en parte** · **Jorge Rico** · Un dominio estándar de otro tamaño borra tablas que «Cancel» no devuelve | `core` | M | No |
| 25 | **Jorge Rico** · Restringir un enlace cambia la tabla del hijo y deshacer no la devuelve | `gui` | S | No |
| 26 | **Corregido** · Enlazar a una utilidad producto la convierte en suma y cambia el resultado | `core` | S | No |
| 27 | **Jorge Rico** · Rehacer un enlace y su restricción deja el enlace sin la restricción | `core` | M | No |
| 28 | **Jorge Rico** · Pasar a suceso una decisión numérica sin política la deja a medias | `core` | S | No |
| 29 | **Corregido** · Mover un estado con un hijo sin potencial deja la tabla permutada a medias | `core` | S | No |
| 30 | **Corregido** · Deshacer un cambio de estados duplica tablas de los hijos y cambia resultados | `core` | S | No |
| 31 | Pegar en una red que ya incumple una restricción se rechaza siempre | `core` | M | Sí |
| 33 | **Jorge Rico** · Deshacer o cancelar «mostrar el comentario al abrir» no la devuelve | `core` | S | No |
| 34 | **Corregido** · Deshacer o cancelar «Always append» de un suceso no devuelve su valor | `core` | S | No |
| 35 | **Corregido** · Una red expandida o aprendida no se marca como modificada ni pide guardar | `core` | S | No |
| 36 | **Corregido** · En una red expandida o aprendida, deshacer no recuerda nada al principio | `core` | S | No |
| 239 | **Corregido** · Quitar un padre a un nodo con fórmula la cambia por una uniforme que aún lo nombra | `core` | S | No |
| 243 | **Corregido** · Un dominio estándar de otro tamaño da un error inesperado en una decisión o en su padre | `core` | S | No |
| 258 | **Corregido** · Pegar un trozo con `X` y `X'` en una red que ya tiene `X` confunde las dos en las fórmulas | `gui` | S | No |
| 271 | Un nodo temporal no se puede pegar en su propia red | `gui`, `core` | S | No |
| 275 | **Corregido** · Quitarle un padre a un nodo le borra el comentario de su relación, salvo si es una fórmula o un modelo canónico | `core` | S | No |
| 280 | **Corregido** · Añadir o quitar un estado a una decisión con política impuesta deja la política con el tamaño viejo | `core` | S | No |
| 281 | **Corregido** · Convertir un nodo de azar en decisión conserva su tabla de probabilidad como política impuesta | `gui`, `core` | S | No |
| 282 | **Corregido** · Cambiar el tipo de variable de una decisión sin política le impone una política uniforme | `core` | S | No |
| 283 | **Corregido** · El portapapeles guarda los nodos originales, no una copia: cortar y pegar deja las relaciones sin sus padres | `gui`, `core` | S | No |
| 284 | **Corregido** · Pegar pierde las restricciones y las condiciones de revelación de los enlaces, y la marca «always observed» | `gui` | S | No |
| 285 | **Corregido** · Quitar un padre a un nodo de utilidad deja una relación uniforme que impide evaluar la red | `core` | S | No |
| 286 | **Corregido** · Pasar una variable de estados finitos a discretizada borra las restricciones y las revelaciones de sus enlaces | `core` | S | No |
| 287 | **Corregido** · Deshacer un cambio en los criterios de decisión después de rehacerlo no lo deshace | `core` | S | No |
| 288 | **Corregido** · Cambiar los estados o el tipo de variable de un padre de una utilidad deja su relación uniforme e impide evaluar la red | `core` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 289 | **Corregido** · Quitar el padre de un nodo de azar numérico con valores exactos, o pegarlo sin él, deja una relación uniforme que impide evaluar | `core`, `gui` | S | No |
| **5. Restricciones de red** | | | | |
| 37 | **Jorge Rico** · Abrir un fichero no comprueba las restricciones de la red | `io` | S | Sí |
| 38 | **Jorge Rico** · Un estado puede quedarse con el nombre vacío o repetido | `core` | S | Sí |
| 39 | Ocho restricciones opcionales no llegan a ninguna red | `core` | M | Sí |
| **6. Evidencia, inferencia y variables numéricas** | | | | |
| 40 | **Corregido** · Una variable numérica sin precisión se discretiza a un único valor cero | `core` | S | No |
| 41 | **Corregido** · Un hallazgo sobre un nodo numérico de azar hace fallar la propagación | `core` | M | No |
| 42 | **Corregido** · Entrar en modo inferencia falla si hay nodos de utilidad intermedios | `inference` | M | No |
| 43 | **Corregido** · Un nodo numérico de azar sin potencial tumba la discretización | `core` | S | No |
| 44 | **Corregido** · Un hallazgo numérico con más de dos decimales se pierde al discretizar | `core` | S | No |
| 194 | Un hallazgo sobre un nodo con delta se ignora antes de resolver y se rechaza después | `core` | S | Sí |
| 45 | **Corregido** · Observar un nodo numérico de azar no informa sobre sus padres | `core` | M | Sí |
| 46 | **Corregido** · Un nodo de utilidad sin rango impide entrar en modo inferencia | `gui` | S | No |
| 47 | **Corregido** · «Add finding» marca siempre el primer estado y pisa el hallazgo anterior | `gui` | S | No |
| 48 | **Corregido** · La evidencia imposible se trata distinto según el gesto | `gui` | M | No |
| 49 | **Corregido** · Construir una propagación cambia el tipo de análisis de la red del usuario | `core` | S | No |
| 227 | **Corregido** · En una DAN con restricciones, la propagación dice que los hallazgos son imposibles sin haber ninguno | `inference` | S | No |
| 228 | **Corregido** · La propagación de MID-HPV falla al podar la red | `inference` | S | No |
| 229 | **Corregido** · Tres MID no propagan porque la edad inicial es uniforme | `inference` | M | No |
| 230 | **Corregido** · MID-CHAP-Ryan-Griffin no propaga: una expresión usa la edad sin valor | `inference` | S | No |
| 231 | **Corregido** · La propagación de MID-Cochlear no termina en un minuto | `inference` | M | No |
| 232 | **Corregido** · La red desplegada con «Expand network» no entra en modo inferencia | `gui` | S | No |
| 233 | Con transiciones a medio ciclo no se evalúa una red con una utilidad dada por un árbol | `core` | M | Sí |
| 267 | **Corregido** · Un nodo numérico con varios padres se discretiza mal: pierde valores y rechaza hallazgos posibles | `core` | S | No |
| 268 | **Corregido** · Observar un nodo numérico con un parámetro imposible acaba en un error de Commons Math que nadie explica | `core` | S | No |
| 269 | **Corregido** · Observar un nodo numérico con distribución muere si se ha visto otro número o tiene un padre numérico | `core` | S | No |
| 270 | **Corregido** · Un hallazgo donde la densidad es infinita deja la probabilidad de los padres en NaN | `core` | S | No |
| 272 | Una beta con un parámetro 0 solo se rechaza como tal en su extremo; dentro, el mensaje dice otra cosa | `core` | S | No |
| **7. Decisiones: políticas, criterios, agentes y árbol de decisión** | | | | |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 50 | **Corregido** · El árbol de decisión falla en casi todas las redes en formato 1.0 | `inference` | M | No |
| 51 | **Corregido** · «Show optimal policy» y «Show expected utility» no llegan a abrirse | `gui` | S | Sí |
| 52 | **Corregido** · «Show expected utility» falla con utilidad constante y rellena ceros | `core` | S | No |
| 53 | **Corregido** · La utilidad esperada de una decisión ignora la evidencia previa | `inference` | S | No |
| 54 | **Corregido** · Un hallazgo que descarta la primera opción de una decisión anula el cálculo | `inference` | M | No |
| 55 | **Corregido** · Las ventanas de utilidad esperada y política óptima muestran la clave | `gui` | S | No |
| 56 | **Corregido** · Deshacer «quitar la política impuesta» pierde la tabla de la política | `core` | S | No |
| 57 | **Corregido** · Renombrar un criterio de decisión no se deshace ni se cancela | `core` | S | No |
| 58 | Confirmar un criterio sin cambiar su nombre se rechaza como repetido | `core` | S | Sí |
| 59 | **Corregido** · Evaluar una red de decisiones sin criterios falla sin explicar qué falta | `core` | S | No |
| 60 | **Corregido** · El agente de cada decisión se pierde al guardar y no se lee al abrir | `io` | S | No |
| 61 | **Corregido** · Quitar, renombrar o mover un agente descuadra las decisiones | `gui` | M | No |
| 62 | **Corregido** · Las propiedades de una decisión no se abren si su agente ya no figura | `gui` | S | No |
| 63 | **Corregido** · Evaluar DAN-arthronet falla con NullPointerException al instanciar | `inference` | S | No |
| 64 | **Cerrado sin arreglo** · Instanciar una variable de una red de decisiones duplica un enlace | `inference` | S | No |
| 65 | **Corregido** · La estrategia óptima de una red sin decisiones lanza una excepción | `gui` | S | No |
| 66 | **Corregido** · El botón del árbol de decisión se enciende en redes que no admiten árbol | `gui` | S | No |
| 200 | **Corregido** · Una rama de una red de decisiones suma como utilidad el factor de un producto borrado | `inference` | S | No |
| 210 | Alt+clic sobre una decisión sin política impuesta da un error inesperado | `gui` | S | No |
| 222 | **Corregido** · En una red de análisis de decisiones, saber qué se eligió en una decisión no cambia nada | `inference` | S | No |
| 223 | **Corregido** · La estrategia óptima de una DAN no se abre si ya se sabe qué se eligió en su única decisión | `gui` | S | No |
| 224 | **Corregido** · En una DAN, un hallazgo previo sobre una variable que restringe a otras impide evaluarla | `inference` | S | No |
| 290 | En una DAN, un hallazgo cuya probabilidad depende de una decisión cambia la utilidad o la estrategia | `inference` | M | Sí |
| **8. Modelos temporales y evolución temporal** | | | | |
| 67 | **Corregido** · Con evidencia, las curvas de la evolución temporal no suman uno | `inference` | M | No |
| 68 | **Corregido** · La evolución temporal de una variable que empieza en el ciclo 1 falla | `inference` | S | No |
| 69 | **Corregido** · La evolución temporal de una decisión temporal muere con una excepción | `inference` | M | No |
| 70 | **Corregido** · «Same as previous» revienta si el ciclo anterior no tiene la variable | `core` | S | No |
| 71 | **Corregido en parte** · Al expandir un modelo temporal, las regresiones siguen nombrando el ciclo 0 | `core` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 72 | **Corregido** · Pasar una variable a atemporal puede dejar dos con el mismo nombre | `core` | S | No |
| 73 | **Corregido** · La duración del ciclo escrita en las propiedades de la red no se aplica | `gui` | S | No |
| 211 | Al sumar los estados, el nombre de la curva sale cortado: «Therapy [0» | `gui` | S | No |
| 259 | **Corregido** · Una tabla de fórmulas copiada al ciclo siguiente se calcula sobre el ciclo anterior | `core` | S | No |
| 261 | **Corregido** · Un «Same as previous» en el primer ciclo de una variable pasa sin aviso y acaba en un error confuso | `core` | S | No |
| **9. Coste-efectividad y análisis de sensibilidad** | | | | |
| 74 | **Corregido** · El jar publicado mezcla dos JFreeChart y muchas gráficas no se abren | `full` | S | No |
| 75 | **Corregido** · La araña de coste-efectividad reescribe la red abierta con las medias | `core` | M | No |
| 76 | **Corregido** · Una columna con incertidumbre solo en algunas casillas se sortea mal | `core` | S | No |
| 77 | **Corregido** · El análisis probabilístico de sensibilidad no se puede repetir | `core` | M | No |
| 78 | **Corregido** · El diálogo de incertidumbre rechaza toda Dirichlet sobre dos o más estados | `gui` | S | No |
| 79 | **Corregido** · Editor y muestreador reparten distinto el complemento junto a una Dirichlet | `core` | S | Sí |
| 80 | **Corregido** · El mapa de dos parámetros elige mal la mejor opción si hay tres o más | `sensitivity Analysis` | S | No |
| 81 | **Corregido** · En el mapa de dos parámetros el empate y la tercera opción salen en gris | `sensitivity Analysis` | S | No |
| 82 | **Corregido** · El mapa de dos parámetros pinta toda la zona de datos de un solo color | `sensitivity Analysis` | S | No |
| 83 | **Corregido** · El mapa no se abre si gana en todo él la misma opción y no es la primera | `sensitivity Analysis` | S | No |
| 84 | **Corregido** · La tabla de intervenciones fronterizas nombra otra opción si dos empatan | `cost Effectiveness` | S | No |
| 85 | **Corregido** · El coste-efectividad falla si un hallazgo hace imposible una sola opción | `inference` | S | No |
| 86 | **Corregido** · Las ventanas de coste-efectividad fallan si una opción es imposible | `cost Effectiveness` | S | No |
| 87 | **Corregido** · El escenario del ámbito de decisión puede ser imposible y no se comprueba | `gui` | S | Sí |
| 88 | **Corregido** · Con valores todos negativos, el tornado dibuja barras hasta cero | `sensitivity Analysis` | S | No |
| 89 | **Corregido** · El tornado por decisión no dibuja el rango real de la diferencia | `sensitivity Analysis` | S | No |
| 90 | **Corregido** · Con utilidades negativas, «Plot» y la araña aplastan y recortan las curvas | `sensitivity Analysis` | S | No |
| 91 | El tornado con dieciséis parámetros o más deja barras fuera del recuadro | `sensitivity Analysis` | S | No |
| 92 | El análisis de sensibilidad no se abre sobre una red sin decisiones | `sensitivity Analysis` | S | No |
| 93 | **Corregido** · Los parámetros inciertos salen en otro orden en cada apertura del análisis | `core` | S | No |
| 94 | Ocultar una opción en una pestaña deja marcada su casilla en la otra | `sensitivity Analysis` | S | No |
| 255 | El mapa de dos parámetros estira sus ejes hasta el cero aunque el intervalo no llegue | `sensitivity Analysis` | S | No |
| 256 | Un 0 % o un intervalo sin anchura impide abrir «Plot», el tornado y la araña, y el mapa | `sensitivity Analysis` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 262 | **Corregido** · Aceptar el análisis de sensibilidad devuelve el escenario a su primer estado y analiza otro | `sensitivity Analysis` | S | No |
| 277 | Después de un análisis de sensibilidad, el escenario de otra decisión sale fijo con lo elegido antes | `sensitivity Analysis` | S | No |
| **10. Simulación de sucesos discretos** | | | | |
| 95 | **Corregido** · Una utilidad acumulativa vale cero hasta el primer suceso que la alcanza | `gui`, `inference. des` | M | Sí |
| 96 | La simulación se cae si un nodo espera a un padre de un suceso no ocurrido | `gui` | S | No |
| 97 | **Corregido** · Un ciclo entre nodos de azar de una red de sucesos impide simular | `gui`, `inference. des` | M | No |
| 98 | Una tabla o valor exacto con un suceso padre hace caer la simulación | `core` | M | Sí |
| 99 | Añadir o quitar un enlace deja una relación uniforme y la simulación falla | `core` | M | Sí |
| 100 | Una tabla de vida «con tasas» se simula como si fueran probabilidades | `core` | S | Sí |
| 101 | La relación «External» se ofrece, elegirla falla y no trae valores de fuera | `core` | M | Sí |
| 102 | **Corregido** · Resultados de la simulación: la media descontada va al criterio equivocado | `gui` | S | No |
| 103 | **Corregido** · El CSV de la simulación de sucesos tiene los títulos corridos una columna | `gui` | S | No |
| 104 | Simular con traza una red de sucesos sin guardar falla antes de empezar | `gui` | S | No |
| 105 | **Corregido** · Un fallo de «Tree with Excluded Events» abre una ventana desde el núcleo | `core` | S | No |
| 106 | Un nodo de azar discretizado impide simular con un error inesperado | `gui` | S | Sí |
| 107 | **Corregido** · La hoja de la propagación estocástica cambia el orden de los hallazgos | `stochastic Propagation Output` | S | No |
| 108 | La propagación estocástica no rechaza nodos numéricos: falla sin explicarlo | `stochastic Propagation Output` | S | No |
| 109 | Las casillas de estadísticos del panel de Monte Carlo no hacen nada | `gui` | S | Sí |
| 110 | «Sort probnet» falla en una red de sucesos con un nodo enlazado a sí mismo | `full` | S | No |
| 214 | «Cost effectiveness analysis» del menú Tools no sabe evaluar una red de sucesos | `cost Effectiveness` | S | No |
| 252 | El CSV de una simulación pierde columnas cuando una media se escribe con exponente | `gui` | S | No |
| **11. Abrir y guardar redes** | | | | |
| 111 | **Corregido** · Un .pgmx mal escrito se abre sin aviso y con todas las tablas uniformes | `io` | S | No |
| 112 | **Corregido** · Guardar en formato 1.0 deja la red sin poder evaluarse al reabrirla | `io` | M | No |
| 113 | **Corregido** · Guardar en formato 0.2 convierte los nodos de producto en uniformes | `io` | S | No |
| 114 | **Jorge Rico** · «Save» falla en tres redes con un papel de potencial mal escrito | `io` | S | Sí |
| 115 | **Corregido** · Una distribución desconocida en una red de sucesos falla lejos de la causa | `core` | S | No |
| 116 | **Corregido** · Una tabla de sucesos con incertidumbre en el fichero impide abrir la red | `io` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 117 | **Corregido** · Abrir un .xml que no es ProbModelXML falla con una excepción de índice | `io` | S | No |
| 202 | **Corregido** · El aviso de agente desconocido no dice de qué ciclo es la variable | `io` | S | No |
| 235 | **Corregido** · Guardar una red de sucesos en formato 0.2 pierde sus relaciones y luego no se abre | `io`, `gui` | S | No |
| 236 | **Corregido** · Un árbol con sucesos mal escrito en el fichero se abre sin aviso, con el árbol por omisión | `io` | S | No |
| 237 | Los ficheros guardados con la versión de la rama de sucesos no se abren | `io`, `core` | S | Sí |
| 273 | El mensaje que se niega a guardar una relación nombra su nodo con el instante dos veces | `io` | S | No |
| 274 | **Corregido** · El escritor de 0.2 guarda una tabla de fórmulas como una tabla, y al abrirla las fórmulas ya no están | `io` | S | No |
| 276 | Un indicador sin «TTE» o sin «ProbabilityOccurrence» en el fichero revienta con un puntero nulo | `io`, `gui` | S | No |
| **12. Bases de casos** | | | | |
| 118 | **Corregido** · Un «?» en una base de Weka descuadra las filas e inventa casos sin avisar | `io` | M | No |
| 119 | **Corregido** · Un fichero de Weka en UTF-8 con tildes o «ñ» pierde estados y lee ceros | `io` | M | No |
| 120 | Una columna numérica, de texto o de fecha en Weka da NullPointerException | `io` | M | Sí |
| 121 | **Corregido** · Weka con @Relation, @Attribute o @Data en mayúsculas mezcladas no se abre | `io` | S | No |
| 122 | **Corregido** · Una base con una columna de números guardada en Weka no se puede reabrir | `io` | S | No |
| 123 | **Corregido** · Un fichero de Weka en una carpeta con tilde, «ñ» o «+» no se puede reabrir | `io` | S | No |
| 124 | **Jorge Rico** · Una línea de CSV con campos de menos, de más o en blanco inventa casos | `io` | S | No |
| 218 | En el generador de casos, cancelar «Load Net from file» deja «Open» apagado | `dbGenerator` | S | No |
| 246 | Una base con una casilla en blanco, guardada en formato Weka, no se vuelve a abrir | `io` | S | No |
| 247 | Un comentario de Weka detrás de lo que dice una línea hace que se rechace el fichero | `io` | S | No |
| 248 | Un fichero escrito por Weka con « %», comillas dobles o un tabulador en un valor se lee con barras de más | `io` | S | No |
| 249 | Un «`'?'`» que la columna no declara se rechaza o se acepta según lo que haya en las líneas de antes | `io` | S | No |
| 250 | Un fichero de Weka en UTF-8 con marca de orden de bytes ya no se abre, y el mensaje no dice por qué | `io` | S | Sí |
| **13. Aprendizaje y evaluación de redes** | | | | |
| 125 | **Corregido** · La exactitud del Bayes ingenuo selectivo usa el caso evaluado al entrenar | `learning.metric` | S | No |
| 126 | **Corregido** · El Bayes ingenuo selectivo elige características sin el alfa del diálogo | `learning.algorithm` | S | No |
| 127 | **Corregido** · El superpadre pesa sus enlaces con cuentas de casillas equivocadas | `learning.metric` | S | No |
| 128 | **Corregido** · El superpadre penaliza la clase verdadera: ningún enlace sube la exactitud | `learning.metric` | S | No |
| 129 | **Corregido** · El superpadre puntúa los enlaces candidatos con la probabilidad al revés | `learning.metric` | S | No |
| 130 | **Corregido** · El superpadre cuenta dos veces la hija de cada enlace al medir la exactitud | `learning.metric` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 131 | **Corregido** · La validación cruzada de dos clasificadores no prueba un tercio de casos | `learning.metric` | S | No |
| 132 | **Corregido** · El superpadre por defecto no aprende nada: propone un enlace a sí mismo | `learning.algorithm` | S | No |
| 133 | **Corregido** · El superpadre acepta el primer enlace aunque empeore al Bayes ingenuo | `learning.algorithm` | S | No |
| 134 | Superpadre interactivo: llenar la tabla avanza el algoritmo y la vacía | `learning.algorithm` | M | Sí |
| 135 | **Corregido** · El Bayes ingenuo selectivo hacia delante no da dos veces el mismo modelo | `learning.algorithm` | S | No |
| 136 | **Corregido** · El árbol aumentado (TAN) orienta sus enlaces distinto en cada aprendizaje | `learning.algorithm` | S | No |
| 137 | **Corregido** · El EM deja sin aprender los nodos con potencial uniforme | `learning.algorithm` | S | No |
| 138 | **Corregido** · El esqueleto del algoritmo PC cambia al renombrar una variable | `learning.algorithm` | S | No |
| 139 | El PC interactivo ofrece las dos orientaciones de un enlace, contra Meek | `learning.algorithm` | S | Sí |
| 140 | **Corregido** · El PC interactivo adelanta la fase al llenar la tabla y descoloca botones | `learning.algorithm` | M | Sí |
| 141 | «Finish» del PC interactivo empieza de cero y descarta lo aplicado a mano | `learning.algorithm` | M | Sí |
| 142 | **Corregido** · PC interactivo: con un borrado bloqueado, «Finish» da NullPointerException | `learning.algorithm` | S | No |
| 143 | **Corregido** · El bosque aumentado (FAN) falla sin características ligadas a la clase | `learning.algorithm` | S | No |
| 144 | **Corregido** · Discretizar por frecuencias iguales con decimales falla o cuenta mal | `learning.core` | S | No |
| 145 | **Corregido** · «Split Dataset» lanza siempre una excepción, elija la opción que elija | `bnEvaluation` | S | No |
| 146 | **Corregido** · Validación cruzada: casos por pliegue truncados y pérdida por caso inflada | `bnEvaluation` | S | No |
| 147 | **Corregido** · Quedarse sin memoria al aprender sale como error no previsto | `learning.gui` | S | No |
| 148 | **Corregido** · Una variable de la red modelo que falta en los datos da error no previsto | `learning.gui` | S | No |
| 149 | **Corregido** · Red modelo sin alguna columna de la base: error no previsto al cargarla | `learning.gui` | S | No |
| 215 | En «Data preprocessing» no se ve la lista «Outliers», y «Cross validation» sale cortada | `bnEvaluation` | S | No |
| 216 | El rótulo «Classification variable» se queda gris con la base de casos cargada | `bnEvaluation` | S | No |
| 217 | «Reset values» de la ventana de aprender deja «Learn» encendido, y pulsarlo dice que no hay datos | `learning.gui` | S | No |
| 219 | Al cargar otra base en «Data preprocessing», las casillas de las variables salen apagadas | `bnEvaluation` | S | No |
| 240 | El árbol aumentado (TAN) falla con NoSuchElementException si no hay ninguna característica | `learning.algorithm` | S | No |
| 241 | El TAN interactivo sin «Only allowed edits» no enseña los mejores enlaces si hay más de 50 | `learning.algorithm` | S | No |
| 251 | El PC interactivo sigue pasando de fase al pedir la mejor propuesta, según las casillas marcadas | `learning.algorithm` | S | Sí |
| **14. Interfaz: editor de red y paneles de las relaciones** | | | | |
| 150 | **Corregido** · Volver a elegir la clase de relación que ya tiene un nodo borra sus números | `gui` | S | No |
| 151 | **Corregido** · En una regresión, lo escrito como Cholesky se pierde al guardar la red | `gui` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 152 | **Corregido** · Añadir una fila a la tabla de vida y aceptar borra el primer tramo | `gui` | S | No |
| 153 | **Corregido** · El editor de una distribución univariante enseña «1» en cada parámetro | `core` | M | No |
| 154 | **Jorge Rico** · En la tabla de utilidad, una restricción de enlace bloquea otras casillas | `core` | S | No |
| 155 | **Jorge Rico** · La ventana de restricción de un enlace edita la tabla del padre | `gui` | S | No |
| 156 | **Jorge Rico** · La tabla de utilidad enseña 922 337 203,685478 en vez de valores mayores | `core` | S | No |
| 157 | **Corregido en parte** · **Jorge Rico** · El panel de Weibull deja borrar «Gamma» y la relación revienta al calcular | `gui` | S | No |
| 158 | **Jorge Rico** · En el panel de una regresión no se ve la columna de las covariables | `gui` | S | No |
| 159 | **Corregido** · «Add variables to potential» en una hoja del árbol no añade ningún padre | `gui` | S | No |
| 160 | **Corregido** · El árbol ofrece repetir una variable discretizada y elegirla da un error | `gui` | S | No |
| 161 | **Corregido** · «Split interval» con el campo vacío o sin un número da un error inesperado | `gui` | S | No |
| 162 | **Jorge Rico** · La pestaña «Parents» ofrece padres que las restricciones rechazan | `gui` | S | No |
| 163 | **Jorge Rico** · El límite superior del último intervalo de revelación no se puede cambiar | `gui` | S | No |
| 164 | **Corregido** · Un número no válido en el indicador da un error y vacía la pestaña | `gui` | S | No |
| 165 | **Jorge Rico** · El doble clic en la ventanilla de expresiones mete dos veces el texto | `gui` | S | No |
| 166 | **Corregido** · Tras cambiar el tipo de un nodo, una copia suya no encuentra sus padres | `core` | S | No |
| 167 | **Corregido** · Una columna con incertidumbre fuera del primer estado se deja editar | `core` | S | No |
| 168 | **Jorge Rico** · Tras alejar el zoom, los nombres de estado siguen recortados al acercarlo | `gui` | S | No |
| 169 | **Corregido** · Con precisión 0.0, 2.0 o 0.5, los límites de los tramos acaban en punto | `gui` | S | No |
| 170 | **Corregido** · Tras A, B y C, el nodo de azar nuevo se llama «E», la letra de los sucesos | `core` | S | No |
| 207 | **Corregido** · El selector de precisión enseña «1» si la precisión es 0.0001 o no está en su lista | `gui` | S | No |
| 209 | **Corregido** · «Distribution» y «Parametrization» se quedan a la vista sin lista al lado | `gui` | S | No |
| 225 | **Corregido** · En el panel de una distribución no se puede escribir ningún parámetro | `gui` | S | No |
| 226 | **Corregido** · El diálogo de incertidumbre no acepta una columna con distribuciones de Dirichlet | `gui` | S | No |
| 238 | Marcar «Use first interval» no cambia la fórmula que se ve en el panel | `gui` | S | No |
| 242 | En modo inferencia, el valor de una utilidad sin rango se sale de la caja del nodo | `gui` | S | No |
| 245 | **Corregido** · En el árbol de un nodo padre de sí mismo, «Add subtree» ofrece su variable y «Change root variable» no | `gui` | S | Sí |
| 263 | En la pestaña Domain de un nodo de utilidad, la precisión dice siempre «1» | `gui` | S | No |
| 264 | Una precisión «NaN» o «Infinity» en el fichero impide abrir las propiedades del nodo | `gui`, `io` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 265 | **Corregido** · La casilla del tiempo del indicador acepta «NaN», «∞» y «1E400», y el suceso deja de ocurrir | `gui`, `core` | S | No |
| 266 | **Corregido** · Un indicador leído con un número fuera de rango hace fallar el diálogo del nodo | `io`, `core`, `gui` | S | Sí |
| 279 | **Corregido** · En un modelo temporal, «OK» en las propiedades de un nodo con gemelo en otro ciclo se rechaza como nombre repetido | `gui`, `core` | S | No |
| **15. Interfaz: menús, diálogos y herramientas** | | | | |
| 171 | **Jorge Rico** · Abrir una red desde internet apaga «Guardar» para todas las redes | `gui` | M | No |
| 172 | «Edits history» deja un núcleo al cien por cien y puede colgar el programa | `full` | S | Sí |
| 173 | **Jorge Rico** · Aceptar los dominios estándar sin marcar ninguno lanza una excepción | `gui` | S | No |
| 174 | **Jorge Rico** · Aceptar los criterios estándar sin marcar nada borra los criterios de la red | `gui` | S | No |
| 175 | «OpenMarkov configuration» edita unas preferencias que el programa no usa | `gui` | M | Sí |
| 176 | **Corregido** · Las escalas de criterio se redondean a tres decimales y pueden quedar en 0 | `gui` | S | No |
| 201 | **Corregido** · El modo inferencia solo se ofrece a diagramas de influencia y redes bayesianas | `gui` | S | No |
| 203 | «Select all» está apagada hasta que se elige una herramienta de edición | `gui` | S | No |
| 204 | La ventana «About» se abre con 24 píxeles de ancho | `gui` | S | No |
| 206 | «Delete» sigue encendido sin ninguna fila señalada, y pulsarlo da un error | `gui` | S | No |
| 220 | En el tema oscuro hay cinco cosas que no se ven o no se leen | `gui` | S | No |
| 221 | Tras pasar por otro tema, «Sync with OS» puede dejar de seguir al sistema | `gui` | S | No |
| 253 | **Corregido** · Un descuento escrito en tanto por ciento se guarda redondeado y se enseña con dieciocho decimales | `gui` | S | No |
| 254 | Con una duración del ciclo no positiva en el fichero, «OK» no cierra las propiedades de la red | `gui` | S | No |
| **16. Textos y títulos que se ven mal** | | | | |
| 177 | **Jorge Rico** · La ayuda de atajos da dos teclas equivocadas, y una cierra el programa | `gui` | S | No |
| 178 | **Jorge Rico** · Pegar un nodo de sucesos donde no se admite da «»> eventNode is null «<» | `core` | S | No |
| 179 | **Jorge Rico** · El selector del tipo de relación enseña nombres internos como «ProbTable» | `gui` | S | Sí |
| 180 | **Jorge Rico** · El diálogo de dominios estándar se abre sin título | `gui` | S | No |
| 181 | **Jorge Rico** · Seis de las ventanillas del editor de árboles se abren sin título | `gui` | S | No |
| 182 | **Jorge Rico** · Varios títulos de error parten las siglas: «tree a d d», «X m l invalid» | `core` | S | No |
| 183 | **Jorge Rico** · El aviso de zoom fuera de rango dice «Requested zoomManager» | `gui` | S | No |
| 184 | **Jorge Rico** · El rótulo del panel del indicador dice «Probability of ocurrence» | `gui` | S | No |
| 185 | **Jorge Rico** · La ayuda de «Add → Utility node» dice «Create an utility node» | `gui` | S | No |
| 186 | **Jorge Rico** · El mensaje de evidencia incompatible dice «old evidente» y es ilegible | `core` | S | No |
| 187 | El error de muestras sin peso dice «Samples weigth is 0» | `core` | S | No |

| Nº | Error | Módulo | Esf. | Dec. |
|---|---|---|---|---|
| 188 | **Jorge Rico** · La ayuda de «Unique superparent» dice «commont» | `learning.gui` | S | No |
| 205 | Sin ninguna red abierta, el menú Edit dice «Switch to edition mode» | `gui` | S | No |
| 208 | Varios selectores dicen «Cost criterion named …» y «state no» en vez del nombre | `gui`, `cost Effectiveness` | S | No |
| 212 | El diálogo del análisis de sensibilidad se abre sin título | `sensitivity Analysis` | S | No |
| 213 | El título de las opciones de la evolución temporal pega el nombre: «of:MID-…» | `gui` | S | No |
| **17. Rendimiento** | | | | |
| 189 | **Jorge Rico** · Entrar en modo inferencia con una red grande congela la ventana | `gui` | M | Sí |
| 190 | **Jorge Rico** · Cada resultado de una operación sobre tablas reserva una tabla que tira | `core` | S | No |
| 191 | **Jorge Rico** · Cada tabla de probabilidad nueva se rellena dos veces con el mismo valor | `core` | S | No |
| 192 | **Jorge Rico** · La maximización reserva la tabla de la unión de variables sin usarla | `core` | S | No |
| **18. Exportación a Amua** | | | | |
| 195 | **Corregido** · Amua recibe coste y efectividad cruzados si la red pone antes la efectividad | `io` | S | No |
| 196 | **Corregido** · El escenario base de Amua es siempre la primera opción de la decisión | `io` | S | No |
| 197 | Probabilidades que no suman uno se exportan a Amua como uniformes | `io` | S | Sí |
| 198 | **Corregido** · Amua recibe las probabilidades y los valores redondeados a cuatro decimales | `io` | S | No |
| 199 | **Corregido** · Una decisión por debajo del primer nivel se rechaza con «Decision node not found» | `io` | S | No |
| 244 | **Corregido** · Un criterio sin unidad deja el modelo de Amua con una dimensión de menos | `io` | S | No |

# 1 Operaciones con potenciales

Las operaciones aritméticas sobre potenciales (multiplicar, marginalizar, copiar) que usan todos los algoritmos. Un potencial es la función que cuantifica un nodo: una tabla de probabilidad, una utilidad, una fórmula.

### Error 1 · [Corregido] Multiplicar potenciales que son todos un solo número pierde el criterio, y una utilidad sale convertida en algo que no es aditivo

| | |
|---|---|
| **Estado** | Corregido en el commit `bbde667` (23-09-2026): multiplicar solo potenciales constantes conserva el criterio y el árbol de estrategia, así que una utilidad constante sigue siendo aditiva. Que el producto deje fuera las variables de un solo estado no se corrige: no cambia ningún número, y conservarlas rompe dos cosas que hoy funcionan (nota 185). |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialArithmetic.java:69-99`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ope` |

| | |
|---|---|
| | ration/TablePotentialArithmetic.java:674-678<br>core/src/main/java/org/openmarkov/core/model/network/potential/operation/AuxiliaryOperations.java:48-56<br>core/src/main/java/org/openmarkov/core/model/network/potential/operation/MaxOutVariable.java:59<br>core/src/main/java/org/openmarkov/core/model/network/potential/operation/MaxOutVariable.java:159 |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 93, nota 185 |

**Qué pasa.** Quien multiplica con `DiscretePotentialOperations.multiply` una lista de potenciales que son todos constantes (una sola casilla) recibe un resultado **sin criterio de decisión**. El criterio es lo único que distingue un potencial de utilidad (aditivo) de uno de probabilidad: `Potential.isAdditive()` devuelve `criterion != null`. Así, una utilidad con criterio multiplicada por la probabilidad unidad sale convertida en un potencial que ya no es aditivo.

El mismo atajo pierde también la variable de un solo estado del resultado y los árboles de estrategia de un `StrategicTablePotential` constante, porque tampoco los pasa al constructor.

Dónde está tapado hoy: `MaxOutVariable` (eliminación de una decisión) multiplica la probabilidad por la utilidad en la línea 59 y, al final, vuelve a poner a mano el criterio de la utilidad de entrada (línea 159). Cuando una utilidad queda reducida a un solo número (por ejemplo, sobre una decisión con un único camino posible), el producto sale sin criterio y es esa línea la que lo arregla. No se ha encontrado un resultado visible distinto.

Hay dos llamadores de producción que no lo corrigen y pueden recibir utilidades constantes: `ProductPotential.tableProject` (nodo de utilidad producto de sus padres: si todos los padres se proyectan a un solo número, el resultado sale sin criterio) y `UtilityFunctionComputer` (cálculo de rangos de utilidad para las barras de la ventana). No se ha comprobado con una red que alguno de ellos cambie lo que ve el usuario.

Por qué pasa: `DiscretePotentialOperations.multiply` delega en `TablePotentialArithmetic.multiply(List, boolean)`. Ese método calcula al principio el criterio del resultado (`findFirstNonNullCriterion`, línea 81). Después aparta los potenciales «constantes» —para `AuxiliaryOperations`, cualquiera cuya tabla tenga una sola casilla, aunque tenga variables de un solo estado— y, si no queda ninguno, devuelve `buildConstantPotential(constantFactor, role)` (línea 98), que no recibe el criterio. El criterio ya calculado se descarta.

Medido:

```
utilidad constante (cost) x probabilidad unidad:            criterio=(ninguno)
↪  aditivo=false valores=[10.0]
utilidad de una casilla sobre E (cost) x probabilidad unidad: criterio=(ninguno)
↪  aditivo=false valores=[10.0]
utilidad sobre D (cost) x probabilidad unidad:                criterio=cost aditivo=true
↪  valores=[10.0, 20.0]
utilidad constante (cost) x probabilidad sobre D:             criterio=cost aditivo=true
↪  valores=[5.0, 5.0]
```

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialArithmetic.java:79-99, 674-678

```
        // Find out if some potential has criterion. In that case, set that criterion in
        // the resulting potential
        Criterion criterion = findFirstNonNullCriterion(tablePotentials);
```

```
        ...
        // Gets constant factor: The product of constant potentials
        double constantFactor = AuxiliaryOperations.getConstantFactor(potentials);

        // get role
        PotentialRole role = getRole(potentials);

        potentials = AuxiliaryOperations.getNonConstantPotentials(potentials);
        if (potentials.isEmpty()) {
            return buildConstantPotential(constantFactor, role);
        }
...
    private static TablePotential buildConstantPotential(double constantFactor,
    ↪  PotentialRole role) {
        TablePotential constantTablePotential = new TablePotential(null, role);
        constantTablePotential.getValues()[0] = constantFactor;
        return constantTablePotential;
    }
```

**Cómo arreglarlo.** Pasar el criterio al resultado también en el atajo (`buildConstantPotential` con criterio, igual que `buildResultPotential`), y conservar los árboles de estrategia si el único potencial con intervenciones era constante. Es la misma regla que ya siguen la suma y la maximización sobre una colección: el criterio es el primero no nulo de la lista entera, constantes incluidas. El atajo de la línea 77 (lista vacía, devuelve la probabilidad unidad) no tiene criterio que conservar.

Otros sitios que deben cumplir la misma regla: los tres atajos de «todo son constantes» de marginalizar y maximizar (sus valores ya son correctos; la tabla de elecciones vacía que devuelve la maximización está descrita en nota 93, donde hoy no hay nada que corregir) y `SumOutVariable`. Con el arreglo, la línea 159 de `MaxOutVariable` deja de ser necesaria, pero puede quedarse.

Prueba de regresión: multiplicar una utilidad constante con criterio por la probabilidad unidad y exigir el criterio en el resultado.

### Error 2 · [Corregido] Una operación cuya tabla resultado pasa de $2^{31}$ casillas falla con una excepción distinta según el número, o devuelve un resultado equivocado sin avisar

| | |
|---|---|
| **Estado** | Corregido en el commit `76b0a41` (23-09-2026): `multiply`, `sum`, `evaluate FunctionPotential`, `merge` y `calculateOffsets` calculan el tamaño con `computeTableSize` antes de reservar memoria, y `multiplyAndMarginalize` también el número de combinaciones que elimina; toda tabla de más de $2^{31} - 1$ casillas lanza la misma `InvalidArgumentException` en lugar de otra excepción o un resultado falso. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialArithmetic.java:125-127,272-278,600-602`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialMerge.java:68-71`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialElimination.java:115-121,148-166`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/AbstractIndexedPotential.java:142-169` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |

**Decisión de equipo** No

**Qué pasa.** Una tabla tiene una casilla por cada combinación de estados de sus variables, así que su tamaño es el producto de los números de estados. Cuando ese producto no cabe en un `int` (más de $2^{31} - 1$ casillas), el llamador debería recibir siempre la misma `InvalidArgumentException` con el mensaje «the product of their numbers of states exceeds the maximum table size, $2^{31}$ - 1». Hoy recibe una excepción distinta según el número, y en un caso ninguna: un resultado falso.

Camino de llegada: cualquier inferencia sobre una red con una camarilla muy grande. Con las redes bayesianas de `integrationTests/src/main/resources/clone_of_probmodelxml_networks/bn` (20 ficheros) y con `integrationTests/src/main/resources/networks/bn/BN-cpcs360b.pgmx` e `integrationTests/src/main/resources/networks/bn/BN-cpcs422b.pgmx` (360 y 422 nodos), la eliminación de variables (lo que usa la ventana) termina bien en todas. La propagación de Hugin, que es la que usa la ventana de exportar la propagación estocástica a Excel, falla en `BN-nasonet.pgmx` y `1-0/BN-nasonet-1-0.pgmx` de esa misma carpeta, hoy con el mensaje correcto porque el desbordamiento cae en un número pequeño.

Por qué pasa: el cálculo protegido, `AbstractIndexedPotential.computeTableSize`, multiplica con `Math.multiplyExact` y lanza la excepción buena; el constructor de `TablePotential` lo usa. Pero varias operaciones no usan ese cálculo: construyen el tamaño con `dimensions[n-1] * offsets[n-1]` en un `int`, reservan `new double[tableSize]` con ese número y solo al final construyen el potencial, que es cuando la comprobación salta. Si el producto da la vuelta a un número negativo, se lanza `NegativeArraySizeException` antes de llegar a la comprobación; si da la vuelta a un número positivo grande, se intenta reservar esa memoria y puede salir `OutOfMemoryError`; solo si da la vuelta a un número pequeño llega al constructor y se ve el mensaje correcto. Los sitios son `TablePotentialArithmetic.multiply`, `TablePotentialArithmetic.sum`, `TablePotentialArithmetic.evaluateFunctionPotential` y `TablePotentialMerge.merge`. `calculateOffsets` también multiplica en `int` sin comprobar.

El caso peor está en `TablePotentialElimination.multiplyAndMarginalize` (multiplicar varios potenciales y sumar las variables que sobran). El tamaño del resultado sí usa `computeTableSize`, pero el número de combinaciones de las variables a eliminar se acumula en `int eliminationSize` sin control. Si ese producto da la vuelta a cero o a un negativo, el bucle interior no se ejecuta y cada casilla del resultado recibe solo la primera combinación. Como el potencial resultado tiene un tamaño legal, **no hay excepción**: el resultado es simplemente falso.

No se ha encontrado un llamador de producción que elimine de una vez tantas variables como para disparar el resultado falso de `multiplyAndMarginalize`: la eliminación de variables elimina de una en una y Hugin elimina sobre camarillas ya construidas. El defecto de la operación sí se ha comprobado.

Medido con pequeños programas de prueba (una ejecución de cada uno; son deterministas), con potenciales de todo unos y variables binarias:

- `multiply` de un potencial de 16 variables por otro de 16 distintas (2^32): `InvalidArgumentException` (el `int` da cero y llega al constructor).
- `multiply` y `sum` de 16 por 15 variables ($2^{31}$): `NegativeArraySizeException: -2147483648`.
- `multiply` de 15 binarias por 15 binarias más una de cinco estados ($5 \cdot 2^{30}$), con 2 GB de memoria: `OutOfMemoryError: Java heap space`.
- `multiplyAndMarginalize` de 16 por 16 variables conservando una (elimina 31 binarias): devuelve [1,0, 1,0]; lo correcto es [$2^{31}$, $2^{31}$]. Con 16 por 17 (elimina 32) devuelve también [1,0, 1,0] en vez de [2^32, 2^32].

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialArithmetic.java:125-127

```
int[] offsets = TablePotential.calculateOffsets(resultDimension);
int tableSize = numVariables > 0 ? resultDimension[numVariables - 1] * offsets[numVariables
↪  - 1] : 1;
double[] resultValues = new double[tableSize];
```

core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialElimination.java:115-121

```
int resultSize = TablePotential.computeTableSize(variablesToKeep);
double[] resultValues = new double[resultSize];
int eliminationSize = 1;
for (Variable variable : variablesToEliminate) {
    eliminationSize *= variable.getNumStates();
}
```

**Cómo arreglarlo.** Sustituir los cuatro cálculos de `tableSize` por `TablePotential.computeTableSize(resultVariables)` (o `mergedVariables`) antes de reservar nada, y calcular `eliminationSize` con el mismo método sobre `variablesToEliminate`; `calculateOffsets` debería usar también `Math.multiplyExact`. Así toda operación que se pase del límite lanza la misma `InvalidArgumentException` antes de reservar memoria.

Conviene buscar otros productos de números de estados en `int` en los módulos `core` e `inference` (el de `HybridElimination.cliqueSize` ya se corrigió en el commit `075ee9f`).

Pruebas: los cuatro casos medidos convertidos en pruebas unitarias, esperando `InvalidArgumentException` en todos, incluida `multiplyAndMarginalize` con 31 y 32 variables eliminadas.

### Error 3 · [Corregido] Copiar un potencial a una red que no tiene todas sus variables mete nulos en su lista de variables sin avisar

| | |
|---|---|
| **Estado** | Corregido en el commit `38e9782` (23-09-2026): `Potential.deepCopy` lanza una `InvalidArgumentException` que nombra la variable, la red y el potencial cuando la red de destino no tiene alguna de sus variables, en vez de meter un nulo; lo mismo la variable de tiempo de `WeibullHazardPotential` y las variables de los árboles (`TreeADDPotential`, `TreeADDBranch`). |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:801-821`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:446-455`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:401-408` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 50, error 166, nota 180 |

**Qué pasa.** Quien copia un potencial con `Potential.deepCopy(ProbNet copyNet)` a una red a la que le falta alguna de sus variables recibe, sin ningún aviso, un potencial mal formado: con un `null` en su lista de variables. En la mayoría de subclases el fallo aparece más tarde y lejos, cuando alguien pide el número de estados de esa variable. En `TableWithEvents` y su hija `UnivariateDistrPotential` aparece en la línea siguiente, con `NullPointerException`.

El camino de producción comprobado que llega aquí es `UtilityOperations.transformToUnicriterion`, que copia utilidades sobre la red de una rama del árbol de decisión que ya no tiene una variable. Lo que ve el usuario en ese camino, el árbol de decisión que no se construye, está descrito en error 50. Arreglar esta copia no hace que el árbol salga, pero convierte un puntero nulo en un mensaje que nombra el potencial y la variable que faltan.

Por qué pasa: `Potential.deepCopy` crea el potencial nuevo por reflexión y le pone como variables las de `copyNet` con el mismo nombre. Si alguna no existe en `copyNet`, `getVariable` devuelve `null` y ese `null` entra en la lista de variables del potencial copiado sin ninguna comprobación. `TableWithEvents.deepCopy` recorre enseguida la lista recién puesta, en `setEventAsStates`, y pregunta a cada variable su tipo.

El defecto es de contrato: una copia que no puede resolver una variable debería decirlo, no devolver un potencial mal formado.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:811-816

```
List<Variable> newReferences = new ArrayList<>();
for (Variable variable : this.variables) {
    newReferences.add(copyNet.getVariable(variable.getName()));
}

potential.setVariables(newReferences);
```

core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:446-447

```
@Override public Potential deepCopy(ProbNet copyNet) {
    TableWithEvents potential = (TableWithEvents) super.deepCopy(copyNet);
    potential.setEventAsStates(potential.variables.subList(1, potential.variables.size()));
```

**Cómo arreglarlo.** Que `Potential.deepCopy` falle con una excepción con mensaje (qué potencial, qué variable, qué red) cuando una variable no se resuelve, en vez de insertar `null`. La misma idea forma parte de la propuesta de diseño nota 180; allí no hay nada que corregir.

Hay que revisar las subclases que redefinen `deepCopy` y construyen listas auxiliares a partir de las variables (`TableWithEvents`, `UnivariateDistrPotential`, `ICIPotential` y las demás que llaman a `super.deepCopy`), para que no hagan nada con la lista antes de que la base haya validado. Los llamadores de producción (`UtilityOperations.transformToUnicriterion`, `applyCEUtilityScaling` y las copias de red) deben decidir si propagan la excepción o si es un error de programación.

Prueba: copiar con `deepCopy` un `TablePotential` y un `UnivariateDistrPotential` a una red a la que le falte uno de sus padres y comprobar que se lanza la excepción declarada con el nombre de la variable, y que con todas las variables presentes la copia sigue siendo igual que antes.

## 2 Modelos canónicos y modelo de ajuste

Los modelos canónicos (OR, MAX y MIN ruidosos) describen la influencia de muchas causas sobre un efecto con pocos parámetros; el modelo de ajuste (tuning) es una variante de tres estados.

### Error 4 · [Corregido] La proyección de un modelo de ajuste (tuning) cambia de una ejecución a otra el orden de las variables de su tabla y los últimos decimales de las posteriores

| | |
|---|---|
| **Estado** | Corregido en el commit `ee3200d` (23-09-2026): las variables a eliminar se recorren en un `LinkedHashSet` y la tabla proyectada de todo modelo canónico sale en el orden declarado del potencial, así que la proyección del modelo de ajuste da el mismo orden y los mismos valores en cada lectura y ejecución; también `getCPT` usa `LinkedHashSet`. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.canonical |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/canonical/ICIPotential.java:201-231`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:245-256`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/canonical/MinMaxPotential.java:124-136` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 136 |

**Qué pasa.** En una red con un nodo cuyo potencial es un modelo de ajuste («tuning»), la tabla proyectada de ese nodo sale con los padres en un orden distinto en cada ejecución, y las posteriores de la red no coinciden bit a bit de una ejecución a otra.

Dentro del programa las operaciones con tablas se guían por la lista de variables de cada tabla, así que el cambio de orden por sí solo no produce un número equivocado. Sí afecta a quien lea los valores por posición suponiendo el orden declarado (exportaciones, comparaciones con valores de referencia, pruebas): lee otra distribución. Y las últimas cifras cambian: una prueba fijada a valores exactos falla unas veces sí y otras no, y un resultado publicado no es exactamente reproducible.

Camino de llegada: toda propagación, evaluación o consulta de la tabla que pase por la proyección del modelo. El modelo de ajuste se elige desde el diálogo de propiedades del nodo para variables de tres estados, y se lee de ficheros `.pgmx` con `<Model>Tuning</Model>`.

Qué es proyectar el modelo. Un modelo canónico ICI (de influencia causal independiente: OR/MAX ruidoso, AND/MIN ruidoso o modelo de ajuste) no guarda su tabla de probabilidad completa. La calcula multiplicando unas tablas pequeñas —una por padre, una para la fuga (la parte de la probabilidad que no viene de ningún padre) y la de la función que combina— que hablan de unas variables intermedias que el modelo inventa: las «variables auxiliares» o variables z, y la variable de la fuga. Proyectar el modelo, es decir, obtener su tabla con la evidencia aplicada, consiste en multiplicar esas tablas y sumar fuera las variables auxiliares.

Por qué pasa: `ICIPotential.tableProject` reúne las variables que hay que sumar fuera en un `HashSet<Variable>`, las pasa a una lista y las elimina empezando por la última. `Variable` no redefine `equals` ni `hashCode`, así que el orden de recorrido de un `HashSet` de variables depende de las direcciones de memoria de los objetos. Ese orden cambia de una ejecución del programa a otra, e incluso entre dos lecturas del mismo fichero dentro de la misma ejecución, porque cada lectura crea objetos nuevos. Con el orden de eliminación cambian el orden en que se multiplican las tablas intermedias, el tamaño de esas tablas, el orden de las operaciones de coma flotante y el orden de las variables de la tabla que sale (salvo la primera, que `withConditionedVariableFirst` fija en la variable condicionada).

Solo el modelo de ajuste usa este método: las familias MAX y MIN tienen su propia proyección

(`MinMaxPotential.tableProject`, líneas 124-136), que recorre una lista en orden fijo, y por eso el MAX ruidoso medido abajo sale siempre igual.

Medido sobre la red `io/src/test/resources/test-ici-reading.pgmx`, que tiene un modelo de ajuste sobre `I` con padres `E`, `F` y `G`, y un MAX ruidoso sobre `H` con padres `A`, `B` y `C`.

Seis ejecuciones seguidas del mismo programa:

- La tabla proyectada de `I` salió con las variables en los órdenes `[I, G, F, E]`, `[I, F, E, G]`, `[I, E, G, F]`, `[I, F, G, E]`, `[I, E, F, G]` y `[I, G, E, F]`. Los números son los mismos, pero en otras posiciones.
- La propagación por eliminación de variables (`VEPropagation`) de la red entera dio P(I = tercer estado) = `0.0602075` en unas ejecuciones y `0.06020749999999999` en otras. La huella de todas las posteriores tomó dos valores distintos entre ejecuciones.

Cuatro lecturas seguidas del fichero en una misma ejecución, proyectando sin evidencia `H` e `I` y comparando las tablas después de reordenarlas al orden de variables del potencial:

```
H MaxPotential    order: H A B C   canonical-hash=1120680360   (igual en las ocho
↪  proyecciones)
I TuningPotential order: I G F E   first6=[0.9984000000000001, 0.0016, ...]
↪  canonical-hash=-349860942
I TuningPotential order: I F E G   first6=[0.9984, 0.0016000000000000003, ...]
↪  canonical-hash=-630171156
I TuningPotential order: I F G E   ...
↪  canonical-hash=827132973
I TuningPotential order: I G E F   ...
↪  canonical-hash=1234376430
```

Aun reordenadas al mismo orden, las tablas de `I` no son idénticas bit a bit (`0.9984` frente a `0.9984000000000001`).

`Potential.getCPT` (línea 248) también reúne variables en un `HashSet`, pero sin consecuencia: el conjunto se llena con las variables de la proyección del propio potencial y luego se le quitan las variables del potencial, así que en la práctica siempre queda vacío y no hay nada que ordenar.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/canonical/ICIPotential.java:201-231

```
public @NotNull TablePotential tableProject(EvidenceCase evidenceCase, InferenceOptions
↪  inferenceOptions, List<TablePotential> projectedPotentials) throws
↪  NonProjectablePotentialException {
    List<TablePotential> potentials = internalTableProject(evidenceCase, inferenceOptions);
    HashSet<Variable> variablesToEliminate = new HashSet<>();
    // Fill it with variables appearing in all potentials except this
    for (TablePotential tablePotential : potentials) {
        variablesToEliminate.addAll(tablePotential.getVariables());
    }
    variables.forEach(variablesToEliminate::remove);

    List<Variable> allVariables = new ArrayList<>(variables);
    allVariables.addAll(variablesToEliminate);
    while (allVariables.size() > variables.size()) {
        Variable variableToEliminate = allVariables.getLast();
        allVariables.removeLast();
        ...
        potentials.addFirst(DiscretePotentialOperations.multiplyAndMarginalize(
↪  relatedPotentials, allVariables));
    }
```

```
    return withConditionedVariableFirst(DiscretePotentialOperations.multiplyAndMarginalize
    ↪  (potentials, variables));
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:248

```
HashSet<Variable> variablesToEliminate = new HashSet<>();
```

**Cómo arreglarlo.** Recorrer las variables a eliminar en un orden fijo. Hay dos maneras:

- Cambiar el `HashSet` de `ICIPotential.tableProject` por un `LinkedHashSet` (orden de inserción). Basta, porque `internalTableProject` devuelve los potenciales en una lista de orden fijo y cada tabla tiene su lista de variables en orden fijo, así que el orden de inserción ya es determinista.
- Construir la lista de variables a eliminar directamente en el orden que la clase ya guarda: las variables auxiliares en el orden del `LinkedHashMap zVariables` (uno por padre, en el orden de los padres; se construye en los constructores, líneas 90-93 y 111-114) y después la variable de la fuga.

Además, la tabla que sale debería llevar las variables en el orden declarado del potencial (hijo y padres en el orden de `variables`), no solo el hijo delante: `multiplyAndMarginalize(potentials, variables)` las deja en el orden de la unión de los factores. Un `reorder(variables)` al final, en lugar de `withConditionedVariableFirst`, o pasar `variablesToKeep` explícitamente, lo resuelve, y así el orden tampoco depende de cómo multiplique `multiplyAndMarginalize`.

Cambiar también el `HashSet` de `Potential.getCPT` (línea 248) por `LinkedHashSet`, por coherencia, sin esperar ningún efecto.

El mismo patrón («conjunto de dispersión sobre variables sin `hashCode` propio») ya se corrigió en los árboles (`TreeADDPotential.blendPotentials` usa un `SequencedMap` desde el commit `c9a13a5`), en la maximización, en la heurística de eliminación de Cano y Moral y en el orden parcial de las decisiones. Conviene buscar si queda otro `HashSet` o `HashMap` de variables que se recorra para decidir un orden, por ejemplo con `grep -rn "HashSet<Variable>\|HashMap<Variable" core/src/main`.

Pruebas de regresión:

- En `core/src/test`, leer `test-ici-reading.pgmx` varias veces (cada lectura crea objetos nuevos), proyectar el nodo `I` y comprobar que la lista de variables es siempre `I, E, F, G` y que los valores son idénticos entre lecturas (`assertArrayEquals` sin tolerancia) e iguales bit a bit a una referencia.
- Para la reproducibilidad entre ejecuciones, lanzar la propagación de `test-ici-reading.pgmx` y comparar bit a bit con valores fijados, repitiendo la prueba en varias ejecuciones de la máquina virtual o creando las variables en distinto orden para mover sus códigos de dispersión.

### Error 5 · [Corregido] Un fichero puede traer parámetros de un modelo canónico (o una tabla) cuyas columnas no suman uno; se abre sin aviso y el muestreo y el cálculo exacto dan respuestas distintas

| | |
|---|---|
| **Estado** | Corregido en dos commits. `9dc49bb` (23-09-2026): al abrir un fichero en la ventana se avisa de las columnas (de tablas y de parámetros de modelos canónicos, fuga incluida) que se apartan de uno más de $10^{-3}$, y las que se apartan menos se normalizan en la copia de la red de cada algoritmo de |

| | |
|---|---|
| | inferencia sin tocar la del usuario. `79c82e2` (10-10-2026): el generador de bases de casos sortea también sobre una copia normalizada, con prueba (la misma semilla da los mismos casos con la columna redondeada y con la exacta). Que nada avise fuera de la ventana (lectores, llamadores del programa) es la pregunta del error 37. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.canonical |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/canonical/ICIPotential.java:294-304`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/canonical/ICIPotential.java:391-401`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/canonical/ICIPotential.java:568-598`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:68-75`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXPotentialParsers.java:141-163` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir si las columnas que no suman uno se rechazan, para qué potenciales y con qué tolerancia. Quedó decidido con el commit `9dc49bb`: no se rechazan; se avisa al abrir de las que se apartan más de $10^{-3}$ y las demás se normalizan antes de calcular |
| **Relacionados** | error 37 |

**Qué pasa.** Un fichero `.pgmx` editado a mano o generado por otra herramienta puede traer parámetros de un modelo canónico, o una tabla, cuyas columnas no suman uno. Se abre sin ningún aviso. Después, los algoritmos de muestreo (muestreo lógico, ponderación por verosimilitud, generador de bases de casos) y la eliminación de variables contestan cosas distintas sobre la misma red, y ninguno avisa. No se ha comprobado si el editor de parámetros de la ventana deja escribir esos números.

Por qué pasa: los parámetros de un modelo canónico son, por cada padre, una tabla con una columna por estado del padre, y cada columna debería ser una distribución sobre los estados del hijo (sumar uno); la fuga es otra distribución. `ICIPotential.setNoisyParameters` y `setLeakyParameters` solo comprueban la longitud del array, y el lector de `.pgmx` (`PGMXPotentialParsers.getICIPotential`) les pasa los números del fichero sin filtrar. El potencial de tabla (`TablePotential`) tampoco comprueba sumas.

El commit `afa64f9` hizo que los dos bucles de muestreo de `sampleConditionedVariable` no se salgan de la columna. Con eso ya no aparecen estados inexistentes, pero una columna que no suma uno sigue entrando, y el sorteo da al último estado de la columna toda la masa que falta.

Medido sobre un OR/MAX ruidoso `Y` (tres estados) con padre `X` (dos estados): `setNoisyParameters(X, [0.05, 0.03, 0.02, 0, 0, 1])` y la fuga `[0.5, 0.2, 0.2]` se aceptan sin queja. Con `X` en su primer estado, 10000 sorteos dieron `[253, 328, 9419]`: el 94 % al último estado. La tabla que usa el cálculo exacto (`getCPT()`) para esa misma columna es `[0.025, 0.031, 0.034]`, que no suma uno. `new TablePotential([Y, X], condicionada, [0.05, 0.03, 0.02, 0.2, 0.2, 0.2])` también se acepta.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/canonical/ICIPotential.java:294-304

```
public void setNoisyParameters(Variable parent, double[] parameters) {
    int row = parametersRowOf(parent);
    if (parameters.length != variables.getFirst().getNumStates() * parent.getNumStates()) {
```

```
            throw new UnrecoverableException(new InvalidArgumentException(...));
        }
        expandedPotential = null;
        noisyParameters[row] = parameters;
    }
```

**Cómo arreglarlo.** Hay que decidir antes si se rechazan esas columnas (ver `decision-de-equipo`).

Si se decide comprobar, los sitios son:

- `ICIPotential.setNoisyParameters`, que cubre `setNoisyPotentials`, los constructores de copia, `addVariable`/`removeVariable` de las tres familias y el lector;
- `ICIPotential.setLeakyParameters`;
- `TuningPotential.setNoisyParameters`, que construye la tabla desde cuatro números;
- si se decide para las tablas también, el lector de tablas del `.pgmx`, mejor que el constructor de `TablePotential`, que usan las operaciones intermedias, cuyas tablas no suman uno por diseño.

La alternativa sin rechazar es avisar al abrir. Encaja con error 37, que trata de que abrir un fichero no comprueba nada de la red al terminar de leerla.

Prueba: abrir un fichero con una columna de parámetros que suma 0,1 da el rechazo o el aviso elegido.

### Error 6 · [Corregido] El diálogo de imponer una política ofrece el modelo de ajuste para una decisión de tres estados sin padres, y en redes de simulación de eventos discretos, donde el OR y el AND sí se ocultan

| | |
|---|---|
| **Estado** | Corregido en el commit `f2c5239` (23-09-2026): el modelo de ajuste se ofrece solo donde se ofrecen OR/MAX y AND/MIN, y además con tres estados en todas las variables; ya no aparece para una decisión sin padres con política impuesta ni en redes de simulación de eventos discretos. Las comprobaciones comunes están en `ICIPotential.validate`. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.canonical |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/canonical/TuningPotential.java:83-88`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/canonical/MaxPotential.java:60-72`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/canonical/MinPotential.java:59-71`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:93-112`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:662-671`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/ImposePolicyDialog.java:19` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 96 |

**Qué pasa.** El desplegable de tipos de potencial del diálogo del nodo, y el del diálogo de imponer una política (`ImposePolicyDialog`, que usa el mismo `PotentialEditPanel`), ofrece el modelo de ajuste («tuning») en dos casos donde no corresponde: a una decisión de tres estados sin padres a

la que se impone una política, y a los nodos de una red de simulación de eventos discretos. En esos mismos casos el OR/MAX y el AND/MIN sí se ocultan.

Si el usuario lo elige para la decisión sin padres, `instanciateSafely(TuningPotential.class, [D], POLICY)` devuelve un `TuningPotential` con una sola variable, papel `conditional Probability` (no `POLICY`), sin parámetros por padre, y cuya tabla es solo la fuga por omisión `[0, 1, 0]`: una política determinista que elige siempre el estado del medio, presentada como un modelo de ajuste. No se ejecutó el diálogo (es un `JDialog`); no se ha comprobado lo que el panel del modelo canónico enseña con cero padres.

Por qué pasa: el desplegable se llena llamando por reflexión al `validate(nodo, variables, papel)` de cada clase anotada. El del modelo de ajuste es `if (!ICIPotential.validate(...) && role == PotentialRole.CONDITIONAL_PROBABILITY) return false;` y después `variables.stream().allMatch(v -> v.getNumStates() == 3)`.

La «y» hace que el requisito de tener algún padre (`ICIPotential.validate` pide más de una variable) solo se aplique cuando el papel es probabilidad condicionada. Con cualquier otro papel —en particular `POLICY`, el de una política impuesta a una decisión— una variable sola de tres estados pasa. Y a diferencia de `MaxPotential.validate` y `MinPotential.validate`, no rechaza las redes de simulación de eventos discretos (`DESNetworkType`), no limita el papel a probabilidad condicionada o política, ni exige variables de estados finitos o discretizadas.

Medido con `PotentialUtils.getFilteredPotentialClasses`:

- Decisión `D` de tres estados sin padres, con política impuesta (tabla de papel `POLICY`) en un diagrama de influencia: la lista contiene `TuningPotential` y no contiene ni `MaxPotential` ni `MinPotential`.
- Nodo de azar de tres estados sin padres: no lo contiene (correcto, ahí el papel es condicionada).
- Red de simulación de eventos discretos, nodo `B` de tres estados con padre `A` de tres: contiene `TuningPotential` y no `MaxPotential` ni `MinPotential`.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/canonical/TuningPotential.java:83-88

```
public static boolean validate(Node node, List<Variable> variables, PotentialRole role) {
    if (!ICIPotential.validate(node, variables, role) && role ==
    ↪  PotentialRole.CONDITIONAL_PROBABILITY) {
        return false;
    }
    return variables.stream().allMatch(variable -> variable.getNumStates() == 3);
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/canonical/MaxPotential.java:60-72

```
public static boolean validate(Node node, List<Variable> variables, PotentialRole role) {
    if (node.getProbNet().getNetworkType() instanceof DESNetworkType) {
        return false;
    }
    if (role != PotentialRole.CONDITIONAL_PROBABILITY && role != PotentialRole.POLICY) {
        return false;
    }
    if (!ICIPotential.validate(node, variables, role)) {
        return false;
    }
    return variables.stream().allMatch(variable -> variable.getVariableType() ==
    ↪  VariableType.FINITE_STATES
```

```
            || variable.getVariableType() == VariableType.DISCRETIZED);
}
```

**Cómo arreglarlo.** Escribir `TuningPotential.validate` con las mismas cuatro comprobaciones que `MaxPotential` y `MinPotential` (red que no sea de simulación de eventos discretos, papel condicionada o política, al menos un padre, variables de estados finitos o discretizadas) más la suya de tres estados, usando la constante `NUM_STATES` en vez del 3 escrito a mano. Mejor aún, subir las comprobaciones comunes a un método de `ICIPotential` que llamen las tres clases, para que no vuelvan a separarse. Si el equipo quiere el modelo de ajuste en redes de simulación de eventos discretos, habría que decirlo y documentarlo; hoy los otros dos modelos canónicos se ocultan ahí.

Defecto vecino, que no forma parte de este error: `PotentialUtils.instanciateSafely` busca un constructor `(List, PotentialRole)`, y como los tres modelos canónicos solo tienen `(List)`, cae en ese y el potencial nace con papel `conditionalProbability` aunque se esté imponiendo una política; pasa también al elegir OR/MAX para una política.

Prueba de regresión: `getFilteredPotentialClasses` no contiene `TuningPotential` para una decisión de tres estados sin padres ni para un nodo de una red de simulación de eventos discretos, y sí para un nodo de azar de tres estados con padres de tres estados.

## 3 Relaciones definidas por fórmulas, regresiones, árboles y sucesos

Los tipos de relación que no son una tabla: fórmulas, regresiones y riesgos de Weibull, árboles (TreeADD) y las tablas que dependen de sucesos.

### Error 7 · [Corregido] Un nodo de utilidad definido por una fórmula calcula con los valores cambiados si sus padres no se llaman U1, U2… en el orden de los enlaces, y revienta si se llaman de otra manera

| | |
|---|---|
| **Estado** | Corregido en el commit `fd093b0` (23-09-2026): cada tabla se asigna a la variable de su padre, no a `"U" + posición`, así que la fórmula da el valor correcto con cualquier nombre y cualquier orden de enlaces; una fórmula que nombra una variable que no es padre sale como `UnrecoverableException` con la variable que falta, en vez de `NoSuchElementException`. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialArithmetic.java:583-663`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:41-60`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:144-168`<br>`core/src/main/java/org/openmarkov/core/action/core/AbsorbParentsEdit.java:31-46`<br>`core/src/main/java/org/openmarkov/core/inference/tasks/TaskUtilities.java:119-122` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 8, error 11, error 12 |

**Qué pasa.** Un potencial de función (`FunctionPotential`, tipo «Function» en el diálogo del nodo y en los ficheros `.pgmx`) define el valor de un nodo numérico —normalmente un nodo de utilidad—

con una fórmula sobre los valores de sus padres, escrita con los nombres de los padres entre llaves, por ejemplo `abs({Coste})*{Beneficio}`. Para calcular con él, OpenMarkov «absorbe» los padres: sustituye el nodo por una tabla con el valor de la fórmula para cada configuración.

Lo que ve el usuario: si los padres no se llaman `U1`, `U2`…, la absorción muere con `NoSuchElement Exception` sin mensaje. Si se llaman así pero no en el orden de los enlaces, sale un número equivocado sin ningún aviso. Las redes de prueba del repositorio usan justo los nombres `U1`, `U2`… en orden, por eso no se nota.

Camino de llegada: el menú contextual «Absorber padres» (se habilita con `AbsorbParents Validator` cuando el nodo y sus padres son numéricos y los abuelos de estados finitos) y cualquier evaluación con eliminación de variables de una red con nodos numéricos intermedios (`Variable Elimination`, `TemporalEvaluation` y `MIDTemporalEvolution` llaman al preproceso).

Por qué pasa: la absorción la hace `TablePotentialArithmetic.evaluateFunctionPotential`, llamado desde `BasicOperations.absorbParentPotentials`. El método recibe la lista de tablas de los padres (`potentials`) y una lista de variables (`utilityVariables`) cuyos nombres calcula y no usa: la línea que los usaría está comentada. Para dar valor a cada operando de la fórmula busca, entre las variables del potencial de función, la que se llame exactamente `"U" + (iPotential + 1)` y le asigna el número de la tabla del padre que ocupa la posición `iPotential`. Si no la encuentra, `Optional.get()` lanza `NoSuchElementException`.

Además, el llamador (`absorbParentPotentials`) le pasa como `utilityVariables` las variables de la tabla del primer padre, que no son los operandos, y solo captura las dos excepciones comprobadas que el método declara (y las envuelve como «inalcanzables»), así que la `NoSuchElementException` sale tal cual.

Medido sobre copias de `integrationTests/src/main/resources/networks/dan/DAN-two-utility-and-chance-function-sv.pgmx` (nodo `U0` con fórmula sobre dos utilidades constantes; se cambió el valor del segundo padre a 5):

- Control, padres `U1 = -2` y `U2 = 5`, fórmula `abs({U1})*{U2}`: la absorción da `10`, correcto.
- Padres renombrados a `Coste` y `Beneficio` con fórmula `abs({Coste})*{Beneficio}`: `No SuchElementException: No value present` en `TablePotentialArithmetic.java:642`, tanto por `AbsorbParentsEdit` (la opción «Absorber padres» del menú contextual del nodo) como por `TaskUtilities.absorbAllIntermediateNumericNodes` (el preproceso que hacen las tareas de eliminación de variables antes de evaluar una red que no es solo de azar).
- Mismos nombres `U1` y `U2`, pero con el primer enlace saliendo del padre llamado `U2` (valor -2) y el segundo del llamado `U1` (valor 5): la fórmula `abs({U1})*{U2}` debe valer `abs(5)*(-2) = -10` y la absorción devuelve `10`. A la variable llamada `U1` se le da el valor del primer padre, se llame como se llame.

De paso, la fórmula se evalúa y su resultado de texto se convierte a número con `Double.parse Double` una vez por casilla de la tabla (línea 653), en vez de una sola vez por expresión compilada; es un coste, no un error.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialArithmetic.java:620-643

```
List<String> utilityVariablesNames = utilityVariables.stream()
                                                      .map(Variable::getName)
                                                      .toList();
...
for (int iPotential = 0; iPotential < numPotentials; iPotential++) {
    int potentialsPositionIPotential = potentialsPositions[iPotential];
    String varNameInExpressionToEvaluate = "U" + (iPotential + 1);
```

```
    // String varNameInExpressionToEvaluate = utilityVariablesNames.get(iPotential);
    assignment.put(
            utilityPotential.getVariables()
                            .stream()
                            .filter(variable ->
                            ↪  variable.getName().equals(varNameInExpressionToEvaluate))
                            .findFirst()
                            .get()
            , "" + tables[iPotential][potentialsPositionIPotential]);
```

core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:51-59

```
} else {   // FunctionPotential
    try {
        // To check (probable bug): are these two exceptions really unreachable?
        newTable = DiscretePotentialOperations.evaluateFunctionPotential(
                (FunctionPotential) nodePotential, parentsPotentials,
                ↪  parentsPotentials.get(0).getVariables());
    } catch (NonProjectablePotentialException.CannotEvaluate |
             NonProjectablePotentialException.CannotResolveVariable e) {
        throw new UnreachableException(e);
    }
}
```

**Cómo arreglarlo.** Emparejar cada tabla con su operando por la variable, no por la posición. La tabla `i` es la proyección del potencial del padre `i` de `network.getParents(node)`, así que el operando es la variable de ese nodo padre. Ojo: la tabla proyectada de una utilidad constante no lleva esa variable dentro, así que no sirve preguntársela a la tabla.

Para ello, `absorbParentPotentials` debe recibir y pasar la lista de variables de los padres en el mismo orden que `parentsPotentials` (hoy pasa las variables de la primera tabla), y `evaluateFunctionPotential` asignar `assignment.put(variableDelPadre_i, valor)`. Si una variable que la fórmula necesita no aparece, lanzar la excepción declarada (`NonProjectablePotentialException.CannotResolveVariable`) con el nombre que falta, y que `absorbParentPotentials` deje de declararla inalcanzable y la propague o la convierta en un error con mensaje.

Los sitios que llegan: `BasicOperations.absorbParents` (preproceso de inferencia) y `AbsorbParentsEdit.generateEdits` (menú); los dos pasan por `absorbParentPotentials`. Opcionalmente, compilar la expresión una vez fuera del bucle.

Pruebas de regresión: con la red de dos utilidades y fórmula `abs({Coste})*{Beneficio}`, absorber padres da `10`; con los padres `U2` (-2) y `U1` (5) enlazados en ese orden y la fórmula `abs({U1})*{U2}`, da `-10`; con una fórmula que nombra una variable que no es padre, la excepción dice qué variable falta.

### Error 8 · [Corregido] «Absorber padres» sobre un nodo numérico que no es suma, producto ni fórmula muere con ClassCastException (12 redes del repositorio)

| | |
|---|---|
| **Estado** | Corregido en el commit `e682a70` (23-09-2026): «Absorb parents» solo se ofrece, y el preproceso solo absorbe, cuando el potencial del nodo es una suma, un producto o una fórmula; con cualquier otro, `absorbParentPotentials` lanza `InvalidArgumentException` con mensaje en vez de `ClassCastException`. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.inference |

| | |
|---|---|
| **Ficheros** | `core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:41-60`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:144-219`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:279-299`<br>`core/src/main/java/org/openmarkov/core/action/core/AbsorbParentsEdit.java:32-46`<br>`gui/src/main/java/org/openmarkov/gui/validator/AbsorbParentsValidator.java:24-26`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/NodeContextualMenu.java:142` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 7, error 9, error 42, error 112, error 113, error 193, nota 28 |

**Qué pasa.** Absorber los padres de un nodo numérico sustituye su potencial por una tabla calculada a partir de los de sus padres. Si el potencial del nodo no es una suma, un producto ni una fórmula, la operación muere con `ClassCastException`, y en la aplicación sale como error inesperado.

Llegan dos caminos:

- El menú contextual del nodo, «Absorb parents». `NodeContextualMenu` lo enciende con `AbsorbParentsValidator.validate`, que solo mira que el nodo sea numérico, que sus padres sean numéricos y sus abuelos de estados finitos (`BasicOperations.haveParentsAndAreAllAbsorbable`), no la clase del potencial. La acción construye `AbsorbParentsEdit`, que llama a `absorbParentPotentials`.
- El preproceso de los algoritmos exactos, `TaskUtilities.absorbAllIntermediateNumericNodes` → `BasicOperations.absorbParents`, que usa el mismo método. Ahí se llega menos, porque la discretización previa deja de ser numéricas la mayoría de las variables de azar (no medido).

Por qué pasa: `BasicOperations.absorbParentPotentials` (módulo `core`) decide cómo combinar los potenciales de los padres por la clase del potencial del nodo: `SumPotential` → suma, `ProductPotential` → producto, y cualquier otra cosa → `(FunctionPotential) nodePotential`, sin comprobar.

Medido ejecutando `AbsorbParentsEdit` sobre cada nodo con el menú encendido en las redes de `0_miscellaneous/networks` (sin las copias de las carpetas `1-0`): de 30 nodos así, 10 son suma y 3 producto; los otros 17 fallan.

Con `ClassCastException`, 12 redes: en `0_miscellaneous/networks/mid/`, `MID-CHAP-Ryan-Griffin.pgmx`, `MID-Chancellor.pgmx`, `MID-Chancellor-corrected.pgmx`, `MID-Chancellor-new.pgmx`, `MID-Cochlear.pgmx`, `MID-HPV.pgmx`, `MID-HPV-without-supervalue.pgmx` y `MID-hip-Briggs.pgmx`; y en `0_miscellaneous/networks/mid/DMHEE/`, `MID-dmhee-2.5.pgmx`, `MID-dmhee-3.5.pgmx`, `MID-dmhee-4.7.pgmx` y `MID-dmhee-4.8.pgmx`. Las clases que lo provocan son `CycleLengthShift` (la relación «un ciclo más» de variables como «Age [1]» o «Time in treatment [1]»), `LinearCombinationPotential` y `TreeADDPotential` («AR QoL [0]» en `MID-HPV.pgmx`).

En otras tres redes (`0_miscellaneous/networks/mid/MID-CHD-Walker.pgmx`, `MID-Colorectal.pgmx` y `MID-mammography.pgmx`) el fallo es anterior y distinto: `AbsorbParentsEdit.generateEdits` pide la tabla del potencial de un padre y recibe `PotentialCannotBeConvertedToATable` envuelto en `UnreachableException`, es decir, una excepción que el código declara imposible. También sale

como error inesperado. Esa causa no es la de esta ficha y alcanza también a nodos de suma y de producto: está en error 9. En estas tres redes el nodo es un `CycleLengthShift`, así que el arreglo de aquí ya apaga su menú.

**El código.** core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:46-60

```
TablePotential newTable;
if (nodePotential instanceof SumPotential) {
    newTable = DiscretePotentialOperations.sum(parentsPotentials);
} else if (nodePotential instanceof ProductPotential) {
    newTable = DiscretePotentialOperations.multiply(parentsPotentials);
} else {   // FunctionPotential
    try {
        // To check (probable bug): are these two exceptions really unreachable?
        newTable = DiscretePotentialOperations.evaluateFunctionPotential(
                (FunctionPotential) nodePotential, parentsPotentials,
                ↪  parentsPotentials.get(0).getVariables());
    } catch (NonProjectablePotentialException.CannotEvaluate |
             NonProjectablePotentialException.CannotResolveVariable e) {
        throw new UnreachableException(e);
    }
}
```

**Cómo arreglarlo.** Añadir la comprobación `instanceof FunctionPotential` y, para cualquier otra clase, no absorber (dejar el nodo como está) o rechazar con una excepción del dominio con mensaje.

La misma condición debe estar en los tres sitios que deciden si se puede absorber: `AbsorbParents Validator.validate` / `BasicOperations.haveParentsAndAreAllAbsorbable` (para que el menú no se ofrezca), `BasicOperations.absorbAllIntermediateNumericNodes` (preproceso automático) y `absorbParentPotentials`. La condición de que los potenciales de los padres se puedan convertir en tabla es error 9 y debe ir en los mismos sitios.

Prueba de regresión: sobre `0_miscellaneous/networks/mid/MID-Chancellor-new.pgmx`, el menú «Absorb parents» debe estar apagado para «Time in treatment [1]» (o la edición no debe lanzar), y el recorrido de todas las redes de `0_miscellaneous/networks` no debe dar ninguna excepción al absorber los nodos con el menú encendido.

### Error 9 · [Corregido en parte] «Absorb parents» sobre un nodo de suma o de producto sale como error inesperado cuando un padre es una distribución, como ocurre en las redes guardadas en formato 1.0

| | |
|---|---|
| **Estado** | Presente en parte. El commit `551e715` (23-09-2026) hizo que una distribución «Exact» con números, como guarda el formato 1.0 las relaciones exactas, se convierta en tabla y se escale como el `ExactDistrPotential` que es, así que los trece nodos de suma y producto medidos ya se absorben y las redes 1.0 se evalúan; sigue que el menú no mira si los potenciales de los padres se pueden convertir en tabla, y `AbsorbParentsEdit.generateEdits` y `BasicOperations.absorbParents` siguen convirtiendo el fallo en `UnreachableException` para cualquier otra distribución de un padre (por ejemplo, una normal). Esto último se ha deducido leyendo el código. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core; org.openmarkov.core.inference |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/AbsorbParentsEdit.java:36-46`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.j` |

| | |
|---|---|
| | `ava:144-195`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:279-300`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:357-364`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UniformPotential.java:91-99`<br>`core/src/main/java/org/openmarkov/core/inference/tasks/TaskUtilities.java:119-121`<br>`gui/src/main/java/org/openmarkov/gui/validator/AbsorbParentsValidator.java:24-26`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/NodeContextualMenu.java:142`<br>`gui/src/main/java/org/openmarkov/gui/configuration/UserPreferences.java:114` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Si «Absorb parents» debe funcionar con padres cuya relación es una distribución exacta (como las guarda el formato 1.0) o solo rechazarse con un mensaje |
| **Relacionados** | error 8, error 46, error 50, error 112, error 153, error 193, nota 28 |

**Qué pasa.** «Absorb parents», en el menú contextual de un nodo numérico, sustituye su relación por una tabla calculada con las de sus padres. Si el potencial de algún padre no se puede convertir en tabla, la operación termina en el diálogo de error inesperado. No es un caso raro: en el formato ProbModelXML 1.0 las utilidades y muchos valores numéricos se guardan como distribuciones (`UnivariateDistrPotential`, con la distribución «exacta»), y ese potencial no se convierte nunca en tabla.

Camino del usuario:

- abrir `0_miscellaneous/networks/id/1-0/ID-arthronet-1-0.pgmx`, clic derecho sobre «EVAC Total» (una suma de utilidades) y «Absorb parents»;
- o abrir `0_miscellaneous/networks/id/ID-arthronet.pgmx` (formato 0.2, donde la misma operación funciona), guardarla con «Save network as…» eligiendo el formato «OpenMarkov.1.0» («Save» la guardaría en 0.2, el formato del fichero abierto), reabrirla y repetir: ya falla (error 112).

Por qué pasa: `AbsorbParentsEdit.generateEdits` (módulo `core`) pide a cada padre `getPotential().tableProject(null, null)` y convierte `NonProjectablePotentialException` en `UnreachableException`, es decir, en un fallo que el código declara imposible. `UnivariateDistrPotential.tableProject` lanza siempre `PotentialCannotBeConvertedToATable`, y `UniformPotential.tableProject` también cuando su variable es numérica. El menú se enciende con `AbsorbParentsValidator.validate` → `BasicOperations.haveParentsAndAreAllAbsorbable`, que solo mira los tipos de variable del nodo, de sus padres y de sus abuelos, no sus potenciales.

La misma conversión, con el mismo `UnreachableException`, está en `BasicOperations.absorbParents`, que usa el preproceso de los algoritmos exactos (`TaskUtilities.absorbAllIntermediateNumericNodes`): evaluar con eliminación de variables `0_miscellaneous/networks/id/1-0/ID-arthronet-1-0.pgmx`, `ID-mediastinet-1-0.pgmx` o `ID-used-car-buyer-1-0.pgmx` muere ahí. Arreglar esta ficha no basta para evaluar esas redes: las 15 de `0_miscellaneous/networks/id/1-0/` fallan al evaluarlas, la mayoría más adelante, al convertir en tabla esas mismas distribuciones (por ejemplo `ID-decide-test-1-0.pgmx`, que no tiene nodos de suma), mientras que sus versiones en formato 0.2 se evalúan; esa causa es error 112.

Medido ejecutando `AbsorbParentsEdit` sobre cada nodo con el menú encendido en todas las redes de `0_miscellaneous/networks`, incluidas las carpetas `1-0` (148 se abren):

- 13 nodos de suma o de producto fallan con `UnreachableException` causada por `Potential CannotBeConvertedToATable`, en 9 redes: en `0_miscellaneous/networks/id/1-0/`, «EVAC Total» y «Coste total» de `ID-arthronet-1-0.pgmx`, «Net_QALE» y «Total_Economic_Cost» de `ID-mediastinet-1-0.pgmx` y «Total» de `ID-used-car-buyer-1-0.pgmx`; en `0_miscellaneous/networks/dan/1-0/`, `DAN-arthronet-1-0.pgmx` (dos), `DAN-economic-mediastinet-1-0.pgmx`, `DAN-mediastinet-1-0.pgmx` (dos), `DAN-qale-mediastinet-1-0.pgmx` y `DAN-used-car-buyer-1-0.pgmx`; y «Treatment cost [1]» de `0_miscellaneous/networks/mid/1-0/MID-mammography-1-0.pgmx`. En los doce primeros los padres son `UnivariateDistrPotential`; en el último, un árbol cuyas hojas son distribuciones;
- los mismos 13 nodos en las versiones en formato 0.2 de esas redes (por ejemplo `0_miscellaneous/networks/id/ID-arthronet.pgmx`) se absorben sin error, y son las únicas sumas y productos con el menú encendido fuera de las carpetas `1-0`;
- los otros 6 casos con `UnreachableException` son «Age [1]» de `0_miscellaneous/networks/mid/MID-CHD-Walker.pgmx`, `MID-Colorectal.pgmx`, `MID-mammography.pgmx` y sus copias en `1-0`: un `CycleLengthShift` cuyo padre «Age [0]» tiene un `UniformPotential` sobre la variable numérica. Ahí el nodo tampoco se podría absorber aunque el padre se convirtiera en tabla (error 8);
- guardar `0_miscellaneous/networks/id/ID-arthronet.pgmx` con `PGMXWriter_1_0` y reabrirla convierte el `ExactDistrPotential` de «EVAC Implante» en `UnivariateDistrPotential`, y «Absorb parents» sobre «EVAC Total» pasa de funcionar a lanzar `UnreachableException: Potential P(EVAC Implante | Realizar Implante) cannot be converted to a table`.

**El código.** core/src/main/java/org/openmarkov/core/action/core/AbsorbParentsEdit.java:36-46

```
List<Node> parents = probNet.getParents(node);
ArrayList<TablePotential> parentsPotential = new ArrayList<>();
for (Node node : parents) {
    try {
        parentsPotential.add(node.getPotential().tableProject(null, null));
    } catch (NonProjectablePotentialException e) {
        throw new UnreachableException(e);
    }
}

Potential potential = BasicOperations.absorbParentPotentials(nodeVariable,node.
↪  getPotential(),parentsPotential,null);
```

core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:357-359

```
public @NotNull TablePotential tableProject(EvidenceCase evidenceCase, InferenceOptions
↪  inferenceOptions) throws
↪  NonProjectablePotentialException.PotentialCannotBeConvertedToATable {
    throw new NonProjectablePotentialException.PotentialCannotBeConvertedToATable(this);
}
```

**Cómo arreglarlo.** Hay que decidir qué debe pasar con un padre cuyo potencial no es una tabla:

- rechazarlo: añadir a la condición del menú que el potencial de cada padre se pueda convertir en tabla, y en `generateEdits` convertir el fallo en una excepción del dominio con un mensaje, en vez de `UnreachableException`;
- aceptarlo cuando tenga sentido: una distribución exacta sobre un número, la que escribe

el formato 1.0, dice lo mismo que un `ExactDistrPotential`, y se podría tabular antes de absorber. Es una de las opciones de arreglo de error 112; si se toma allí, los trece nodos de suma y producto medidos dejan de fallar por esta causa.

La condición tiene que ser la misma en todos los sitios que deciden si se absorbe: `AbsorbParents Validator.validate` / `BasicOperations.haveParentsAndAreAllAbsorbable` (menú), `Basic Operations.absorbAllIntermediateNumericNodes` (preproceso) y los dos lugares que convierten los padres, `AbsorbParentsEdit.generateEdits` y `BasicOperations.absorbParents`. Error 8 añade a esa misma condición la clase del potencial del nodo.

Prueba de regresión: sobre `0_miscellaneous/networks/id/1-0/ID-arthronet-1-0.pgmx`, el menú «Absorb parents» de «EVAC Total» debe estar apagado o la edición debe terminar sin `Unreachable Exception`; y el recorrido de todas las redes de `0_miscellaneous/networks`, incluidas las carpetas `1-0`, no debe dar `UnreachableException` al absorber.

### Error 10 · [Corregido] Escalar o descontar una utilidad exponencial cambia el coeficiente de otra covariable cuando el término constante no va el primero

| | |
|---|---|
| **Estado** | Corregido en el commit `27c5226` (23-09-2026): escalar o descontar un potencial exponencial suma `ln(escala)` al coeficiente de `Constant`, esté donde esté, también en los coeficientes muestreados; sin `Constant` lanza `InvalidArgument Exception` con mensaje en vez de indexar con −1. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/ExponentialPotential.java:133-140`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:308-318`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:349`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:363-391`<br>`core/src/main/java/org/openmarkov/core/model/network/UtilityOperations.java:28-86` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 11, error 14, error 15, error 71, nota 110 |

**Qué pasa.** En un modelo con una utilidad exponencial cuyo término constante no va el primero de la lista de covariables, el descuento o la escala de las utilidades dan un resultado equivocado sin aviso: en vez de multiplicar la utilidad por el factor, cambian el coeficiente de otra covariable.

Un potencial exponencial (`ExponentialPotential`, de la familia de regresiones `GLMPotential`) vale `exp(β_const + Σ β_j · covariable_j)`. El término constante es la covariable marcada con el texto `Constant`, que puede estar en cualquier posición de la lista.

Nada obliga a que vaya la primera: el lector de `.pgmx` conserva el orden del fichero, y el panel de edición del potencial (`GLMPanel`, que hereda de `KeyTablePanel` los botones de subir y bajar filas y que inserta las covariables nuevas detrás de la fila seleccionada) permite cambiarlo (esto último comprobado leyendo). Y hay redes del repositorio con `Constant` en último lugar: los potenciales exponenciales de los nodos de utilidad `Inpatient costs` y `Outpatient costs` de `0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx` (y de su copia `0_miscellaneous/networks/mid/1-`

`0/MID-CHAP-Ryan-Griffin-1-0.pgmx`, y de `integrationTests/src/main/resources/networks/mid/chap.pgmx` e `integrationTests/src/main/resources/networks/mid/chap-sv.pgmx`) tienen las covariables `{Age [0]}*12, CD4% 8–15 [0], CD4% > 15 [0], Therapy, Constant`.

Camino de llegada: `scalePotential` se ejecuta en tres sitios de la preparación de toda evaluación de un modelo con utilidades:

- el descuento de los nodos de utilidad temporales (`TemporalNetOperations.applyDiscountToUtilityNodes`, factor `1/(1+d)^t`, que se aplica con cualquier tasa de descuento distinta de cero);
- el paso a un único criterio (`UtilityOperations.transformToUnicriterion`);
- la escala de coste-efectividad (`UtilityOperations.applyCEUtilityScaling`).

Un árbol (`TreeADDPotential.scalePotential`) reenvía el escalado a cada rama, así que alcanza también a los exponenciales colgados de un árbol, como los de CHAP.

Por qué pasa: la clase busca la posición de `Constant` con `GLMPotential.getConstantIndex` y la usa al proyectar (`tableProject`, línea 84). Multiplicar el potencial por un factor `s` equivale a sumar `ln(s)` al coeficiente del término constante, que es lo que dice el comentario de `scalePotential`. Pero el código suma `ln(s)` a `coefficients[0]`, que solo es el del término constante si `Constant` va la primera.

Medido:

- Potencial `exp(0,5 + 1·X)` con `X = 3`: con `Constant` primero, escalar por 2 da 66,23 (el doble de 33,12, correcto); con las mismas covariables en orden `X, Constant`, da 264,92, porque el coeficiente de `X` pasa de 1 a 1,693.
- Potencial real de `Inpatient costs [0]` de `MID-CHAP-Ryan-Griffin.pgmx`, escalado por el descuento de 5 ciclos al 3,5 % (0,842, logaritmo −0,172): los coeficientes pasan de `[-0.0016163, -0.9416483, -1.226022, -0.8080182, 4.016486]` a `[-0.1736234, -0.9416483, -1.226022, -0.8080182, 4.016486]`. El que cambia es el de la edad en meses, no el término constante: con una edad de 40 años el coste hospitalario de ese ciclo queda multiplicado por `exp(-0,172·480)`, prácticamente cero, en vez de por 0,842.

Con la red tal como está no se puede medir el efecto en el resultado final del análisis de coste-efectividad de CHAP: `VECEAnalysis` se detiene antes por otro defecto, error 71. Reescribiendo a mano las fórmulas que ese defecto deja mal, sí se puede. Con los descuentos del fichero (0 %) el resultado no cambia, porque el factor es 1. Con un descuento del 3,5 % anual en los dos criterios, el coste esperado de cotrimoxazol sale 18,52; poniendo antes `Constant` la primera en cada exponencial, que equivale al arreglo, sale 162,4.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/ExponentialPotential.java:133-140

```
@Override public void scalePotential(double scale) {
    /*
     * Add ln(scale) to the first coefficient (constant covariate) is the same as
     * multiply all the exponential potential by the scale
     */
    coefficients[0] += Math.log(scale);

}
```

core/src/main/java/org/openmarkov/core/model/network/potential/ExponentialPotential.java:84 y 107

```
int constantIndex = getConstantIndex(covariates);
...
double regression = coefficients[constantIndex];
```

**Cómo arreglarlo.** Sumar `Math.log(scale)` a `coefficients[getConstantIndex(covariates)]`. Si `getConstantIndex` devuelve $-1$ (el potencial no tiene `Constant`), fallar con un mensaje que lo diga en vez de indexar con $-1$; es el mismo problema de fondo que error 14.

Conviene revisar dos detalles del mismo método al tocarlo:

- `scalePotential(0)` (lo usa `TemporalNetOperations`, línea 349, para anular las utilidades del ciclo que se poda según el momento de transición) suma `ln(0) = −∞`. Con `Constant` primero da `exp(−∞) = 0`, correcto. Con `Constant` en otra posición, el $-\infty$ cae en el coeficiente de otra covariable; medido con el potencial `X, Constant`, la utilidad que debía anularse vale 0 si `X = 3`, **NaN** si `X = 0` e **Infinity** si `X = -3`. El arreglo de arriba lo corrige también; aun así, conviene tratar el factor 0 aparte (devolver cero) para no dejar un coeficiente infinito en la copia.
- Si el potencial ya tiene coeficientes muestreados (`sampledCoefficients`, que es lo que usa `tableProject` durante un análisis probabilístico de sensibilidad), escalar solo `coefficients` no cambia lo que se proyecta. En ese análisis se muestrea antes de descontar, así que el descuento se perdería también con `Constant` en su sitio; hoy ningún análisis llega a ese punto (nota 110), pero conviene que el arreglo escale también `sampledCoefficients`.

Los demás potenciales de regresión no tienen este defecto: `LinearCombinationPotential.scalePotential` multiplica todos los coeficientes y `WeibullHazardPotential.scalePotential` no admite escalado.

Prueba de regresión: en `core/src/test`, un `ExponentialPotential` con covariables `{X}, Constant` y otro con `Constant, {X}` (mismos coeficientes emparejados), escalar ambos por 2 y comprobar que la proyección con `X = 3` es el doble de la original en los dos casos.

### Error 11 · [Corregido] Con una escala de criterio o un descuento, un nodo de utilidad que combina otras utilidades con una fórmula o un producto recibe el factor varias veces, y la fórmula además sin paréntesis: el resultado sale equivocado sin aviso

| | |
|---|---|
| **Estado** | Corregido en el commit `4e7c473` (23-09-2026): cuando un nodo de utilidad combina otras con un producto o una fórmula, la escala o el descuento se aplica una sola vez, en ese nodo, y no en sus padres; una fórmula se escala entera, entre paréntesis. Una suma sigue dejando el factor a sus padres. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network; org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/FunctionPotential.java:167-170`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/FunctionPotentialOld.java:167-173`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ProductPotential.java:117-119`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/SumPotential.java:146-148`<br>`core/src/main/java/org/openmarkov/core/model/network/UtilityOperations.java:28-60`<br>`core/src/main/java/org/openmarkov/core/model/network/UtilityOperat` |

| | |
|---|---|
| | `ions.java:67-88`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:346-353`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:363-392`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VariableElimination.java:66-116` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** A qué nodos se aplican la escala y el descuento cuando un nodo de utilidad combina otras utilidades (solo al nodo final que las combina, o solo a los padres y nunca al nodo que los combina), porque con productos y fórmulas que no son sumas las dos opciones dan resultados distintos |
| **Relacionados** | error 7, error 10, error 12, error 26, error 176 |

**Qué pasa.** Un nodo de utilidad puede definirse combinando otros nodos de utilidad: como suma, como producto o con una fórmula (relación «Function», que el diálogo del nodo ofrece para cualquier nodo de utilidad). Antes de calcular, OpenMarkov multiplica cada utilidad por un factor en tres casos:

- la escala de un solo criterio (`UtilityOperations.transformToUnicriterion`);
- la escala de coste-efectividad (`UtilityOperations.applyCEUtilityScaling`);
- el descuento de los nodos temporales, `1/(1+d)^t` (`TemporalNetOperations.applyDiscountToUtilityNodes`), con cualquier tasa distinta de cero.

Las dos escalas se fijan en el diálogo de opciones de inferencia, que en una red con más de un criterio aparece al entrar en modo inferencia, al pedir el análisis de coste-efectividad, el de sensibilidad o el árbol de decisión, y desde el menú de opciones de inferencia.

Los tres métodos recorren **todos** los nodos de utilidad y escalan el potencial de cada uno, sin mirar si combina otros. Después, `BasicOperations.absorbAllIntermediateNumericNodes` combina los padres ya escalados. Para una suma sale bien, porque `SumPotential.scalePotential` no hace nada: la suma de padres escalados queda escalada una vez. Para los otros dos, no:

- **Fórmula.** `FunctionPotential.scalePotential` se escala a sí misma además de a sus padres: una fórmula lineal queda escalada dos veces y un producto de dos padres, tres. Y lo hace anteponiendo `factor*` al texto de la fórmula sin paréntesis, así que en `{U1}+100` el factor solo multiplica al primer término.
- **Producto.** `ProductPotential.scalePotential` no hace nada, pero sus dos padres sí se escalan: el producto queda escalado dos veces.

Lo mismo pasa con la anulación de las utilidades del ciclo que se poda según el momento de la transición (`TemporalNetOperations`, línea 349), que escala por 0: una fórmula `{U1}+100` quedaría en `0.0*{U1}+100` y no se anularía (por lectura).

El nodo producto está en una red del repositorio: `0_miscellaneous/networks/mid/MID-HPV.pgmx`, donde «QoL [0]» es el producto de «AR QoL [0]» y «HSR QoL [0]», dos utilidades temporales de efectividad; basta con poner un descuento o una escala a la efectividad. Los nodos de fórmula sobre utilidades solo aparecen en redes de prueba (`integrationTests/src/main/resources/networks/dan/`), así que se llega construyendo el modelo en el editor. La fórmula tiene que nombrar a sus padres `U1`, `U2`… (error 7).

Medido:

- En `0_miscellaneous/networks/id/ID-CEA-minimal.pgmx`, cambiando el coste por un nodo

`U1` (0 sin terapia, 14 000 con terapia) y añadiendo el nodo «Total cost» del mismo criterio con distintas fórmulas sobre `U1`; coste de la opción «yes» con ámbito de decisión, según la escala de coste-efectividad del coste (con la escala de un solo criterio sale igual):

| fórmula | escala 1 | escala 2: debería / sale | escala 0,5: debería / sale |
|---|---|---|---|
| {U1} | 14000 | 28000 / 56000 | 7000 / 3500 |
| {U1}*2 | 28000 | 56000 / 112000 | 14000 / 7000 |
| {U1}+100 | 14100 | 28200 / 56100 | 7050 / 3600 |
| abs({U1})+abs({U1}) | 28000 | 56000 / 84000 | 14000 / 10500 |

- `integrationTests/src/main/resources/networks/dan/DAN-two-utility-and-chance-function-sv.pgmx` (`U0 = abs({U1})*{U2}`), evaluación con la escala de un solo criterio: $-6$ con escala 1; $-48$ con escala 2, en vez de $-12$; $-6 \cdot 10^{-9}$ con escala 0,001, en vez de $-0{,}006$.
- `0_miscellaneous/networks/mid/MID-HPV.pgmx`, análisis de coste-efectividad global: con escala 2 en la efectividad, las efectividades pasan de 60,16 a 240,63 ($\times 4$); en `0_miscellaneous/networks/mid/MID-HPV-without-supervalue.pgmx`, que no tiene el nodo producto, pasan de 71,82 a 143,64 ($\times 2$). Con un descuento del 1,5 % anual en la efectividad, en `MID-HPV.pgmx` queda el 42 % (de 60,16 a 25,39) y en la versión sin producto, el 60 % (de 71,82 a 43,40). Las dos redes no dan la misma efectividad ni sin escala; aquí solo se compara cuánto cambia cada una.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/FunctionPotential.java:167-170

```
@Override public void scalePotential(double scale) {
    String scaleString = String.valueOf(scale);
    covariates[0] = new VariableExpression(this.variables, scaleString + "*" +
↪   this.covariates[0].asStringExpression());
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/ProductPotential.java:117-119 (igual en `SumPotential.java`, líneas 146-148)

```
@Override public void scalePotential(double scale) {

}
```

core/src/main/java/org/openmarkov/core/model/network/UtilityOperations.java:67-80

```
public static void applyCEUtilityScaling(ProbNet probNet) {
    for (Node utilityNode : probNet.getNodes(NodeType.UTILITY)) {
        if (utilityNode.getVariable().getDecisionCriterion() != null) {
            // Save the actual criterion scale
            double scale = utilityNode.getVariable().getDecisionCriterion().getCeScale();
            if (scale != 0) {
                // Transform the potential with the scale
                Potential potential = utilityNode.getPotentials().get(0).deepCopy(probNet);
                try {
                    potential.scalePotential(scale);
```

**Cómo arreglarlo.** Hay dos cosas que arreglar.

La primera, local: `FunctionPotential.scalePotential` (y su gemela `FunctionPotentialOld.scalePotential`) debe escribir `factor*(fórmula)`, con el número sin notación científica, que es el arreglo de error 12 para esas mismas líneas.

La segunda exige decidir dónde se aplica el factor cuando un nodo combina utilidades, y es la que decide los números:

- **Solo en el nodo final.** Escalar y descontar solo los nodos de utilidad que no tienen hijos de utilidad, cada uno con el factor de su criterio, y no sus padres. Es correcto para sumas, productos y cualquier fórmula, pero cambia qué criterio se aplica cuando los padres tienen criterios distintos (por ejemplo, una fórmula `30000*{U1}-{U2}` que junta efectividad y coste), y el orden de `VariableElimination.generalPreprocessing`, que descuenta antes de absorber.
- **Solo en los padres.** Seguir escalando los padres y que ningún nodo que combina utilidades se escale a sí mismo, como ya hace la suma. Respeta el criterio de cada padre, pero un producto o una fórmula no lineal sigue recibiendo el factor más de una vez.

Sea cual sea, la regla tiene que cumplirse a la vez en `UtilityOperations.transformToUnicriterion`, `UtilityOperations.applyCEUtilityScaling`, `TemporalNetOperations.applyDiscountToUtilityNodes` y la anulación por factor 0 de `TemporalNetOperations` (línea 349), y en `scalePotential` de `SumPotential`, `ProductPotential` y `FunctionPotential`.

Prueba de regresión: con `0_miscellaneous/networks/mid/MID-HPV.pgmx`, una escala 2 en la efectividad debe duplicar exactamente las efectividades del análisis de coste-efectividad; y con un diagrama de influencia de dos criterios donde «Total cost» sea `{U1}+100` sobre un `U1` de 14 000, la escala 2 del coste debe dar 28 200.

### Error 12 · [Corregido] Una regresión deja de poder calcularse cuando el valor de un padre está por debajo de una milésima

| | |
|---|---|
| **Estado** | Corregido en el commit `f44ac1a` (23-09-2026): los números llegan al evaluador de expresiones sin notación científica, así que una regresión o una fórmula se calcula aunque un padre, un hallazgo o un factor de escala valga menos de 0,001. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:227-241`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/LinearCombinationPotential.java:104-111`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExponentialPotential.java:98-105`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:185-192`<br>`core/src/main/java/org/openmarkov/core/expression/ReferencedExpression.java:106-113` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 7, error 11, error 13, error 15, error 16, error 176, nota 110 |

**Qué pasa.** Un nodo con un potencial de regresión deja de poder calcularse en cuanto algún padre vale, por hallazgo o por nombre de estado, menos de 0,001 en valor absoluto sin ser cero. La proyección del potencial falla con una excepción `CannotEvaluate` que habla de la fórmula y no del número. Un valor pequeño —una probabilidad o una tasa pequeña, de lo más corriente en un modelo de coste-efectividad— basta. Un valor negativo pequeño (`-1.0E-4`) falla igual; el cero (`0.0`) no.

Camino de llegada: cualquier inferencia (propagación, evaluación, análisis de coste-efectividad, evolución temporal) sobre una red cuyos nodos lleven uno de estos potenciales, por ejemplo las redes de `0_miscellaneous/networks/mid/` con `Hazard (Exponential)`, `Hazard (Weibull)` o

`Linear combination`. No se ha encontrado en las redes de ejemplo un padre con ese valor; la consecuencia se ha comprobado construyendo el potencial en un pequeño programa de prueba, no abriendo una red del repositorio.

Por qué pasa: los potenciales de regresión —la familia `GLMPotential` (modelo lineal generalizado): combinación lineal, exponencial, riesgo de Weibull y riesgo exponencial— calculan cada covariable (cada término de la regresión, que puede ser una fórmula como `{Edad}*12`) con el evaluador de expresiones jeval, una biblioteca externa que trabaja sobre texto. `ReferencedExpression.processedExpression` sustituye cada referencia `{Variable}` por el texto que le den para esa variable, y `evaluateWith` le pasa la cadena resultante a jeval.

Quien prepara ese texto lo hace con `String.valueOf(double)`: `GLMPotential.tableProject` para los padres con hallazgo (línea 241), y `LinearCombinationPotential`, `ExponentialPotential` y `WeibullHazardPotential` para los padres sin hallazgo, cuyo valor sale del nombre del estado (líneas 111, 105 y 192). `String.valueOf` de un `double` cambia a notación científica cuando el valor absoluto es menor que 0,001 (o mayor o igual que $10^7$): `String.valueOf(0.0001)` es `"1.0E-4"`. Y jeval no acepta un exponente con signo: medido, `1e3*2` da 2000 y `1.0E7*2` da 2,0E7, pero `1.0E-4*2`, `1e-4*2` y también `1E+3*2` dan `EvaluationException: Expression is invalid`. El corte está exactamente donde `Double.toString` cambia de forma y no tiene relación con el modelo.

Medido con un `LinearCombinationPotential` de hijo numérico `Y` y padre numérico `X`, covariable `{X}*2`, y el hallazgo `X = v`:

```
X=50.0    -> [100.0]
X=0.001   -> [0.002]
X=1.0E-4  -> NonProjectablePotentialException$CannotEvaluate
X=1.0E-5  -> NonProjectablePotentialException$CannotEvaluate
X=1.0E7   -> [2.0E7]
```

Lo mismo ocurre sin hallazgo, con un padre de estados finitos cuyos estados se llaman `0.0001` y `1`, e incluso cuando la covariable es la variable sola (`{D}`), porque el nombre del estado se convierte a `double` y vuelve a texto con `String.valueOf`.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:227-241

```
double numericValue;
Finding finding = evidenceCase.getFinding(variable);
if (variable.getVariableType() == VariableType.NUMERIC
        || variable.getVariableType() == VariableType.DISCRETIZED) {
    numericValue = finding.getNumericalValue();
} else {
    ...
}
variableValues.put(variable, String.valueOf(numericValue));
```

core/src/main/java/org/openmarkov/core/expression/ReferencedExpression.java:106-113

```
public String evaluateWith(Map<T, String> variablesValues) throws ... {
    String processedExpression = this.processedExpression(variablesValues);
    String result;
    try {
        result = new net.sourceforge.jeval.Evaluator().evaluate(processedExpression);
    } catch (EvaluationException e) {
        throw new NonProjectablePotentialException.CannotEvaluate(processedExpression, e);
    }
```

**Cómo arreglarlo.** Escribir el número en una forma que jeval acepte siempre, en un único sitio. La opción más contenida es una función auxiliar (por ejemplo en `ReferencedExpression`

o junto a `VariableExpression`) que convierta un `double` a texto sin notación científica —`BigDecimal.valueOf(v).toPlainString()` conserva todas las cifras de `Double.toString` y nunca escribe exponente— y usarla en todos los sitios que hoy hacen `String.valueOf(double)` o `"" + double` antes de evaluar:

- `GLMPotential.tableProject` (línea 241);
- `LinearCombinationPotential.tableProject` (línea 111), `ExponentialPotential.tableProject` (línea 105) y `WeibullHazardPotential.tableProject` (línea 192);
- `FunctionPotential.sampleConditionedVariable` (`core/src/main/java/org/openmarkov/core/model/network/potential/FunctionPotential.java:103`, simulación de eventos discretos) y su gemela `FunctionPotentialOld` (línea 276);
- `TablePotentialArithmetic.evaluateFunctionPotential` (`core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialArithmetic.java:643`, utilidades combinadas con una fórmula);
- `FunctionPotential.scalePotential` (`FunctionPotential.java:167-169`, y `FunctionPotentialOld.java:171-172`), que antepone `String.valueOf(scale) + "*"` a la fórmula (sin paréntesis, y sobre padres que ya se han escalado: eso es otro defecto, error 11): un factor de descuento o de escala menor que 0,001 (por ejemplo 1/1,035^210 en un horizonte largo) deja escrita en el potencial una fórmula que jeval no acepta. Medido con `integrationTests/src/main/resources/networks/dan/DAN-two-utility-and-chance-function-sv.pgmx` (nodo `U0` con la fórmula `abs({U1})*{U2}`) y la escala de un solo criterio a 0,0005: la evaluación termina con `UnreachableException` causada por `Cannot evaluate '5.0E-4*abs(-0.001)*-0.0015'`. El diálogo de opciones de inferencia redondea las escalas a tres decimales, así que un factor así solo llega desde un fichero o con el descuento de un nodo de fórmula temporal en un ciclo lejano (con un 6 % anual y ciclos de un año, a partir del ciclo 119).

La alternativa de cambiar el lugar de la conversión —que `ReferencedExpression` reciba números y los escriba él— evita que un llamador futuro repita el error, pero cambia la firma `Map<T, String>` que usan también la tabla aumentada (`AugmentedProbTablePotential`) y la interfaz.

Prueba de regresión: en `core/src/test`, proyectar un `LinearCombinationPotential` con covariable `{X}*2` y hallazgos `X = 0.0001`, `X = -0.0001` y `X = 1.0E-12`, y comprobar que devuelve `2·X`; y otra con un padre de estados finitos llamados `0.0001` y `1` sin hallazgo.

### Error 13 · [Corregido] Una covariable escrita como fórmula con el nombre de la variable sin llaves se acepta al leer y al editar, pero no se puede calcular nunca

| | |
|---|---|
| **Estado** | Corregido en el commit `54a1aec` (23-09-2026): se eligió aceptar los nombres sin llaves; al construir la expresión se les ponen, así que la covariable se calcula, se guarda con llaves y lo que el diálogo acepta se puede calcular. Las redes CHAP pasan ahora de este punto y se paran en el error 71. |
| **Módulo** | io, core, gui |
| **Paquete** | org.openmarkov.io.probmodel.reader; org.openmarkov.core.expression; org.openmarkov.gui.dialog.common |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1480-1495`<br>`core/src/main/java/org/openmarkov/core/expression/ReferencedExpression.java:22-43`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:1060-1068`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/ArithmeticExpre` |

| | |
|---|---|
| | `ssionDialog.java:185-204` `gui/src/main/java/org/openmarkov/gui/dialog/common/VariableExpressionTextField.java:80-99` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Si una fórmula puede nombrar variables sin llaves (y entonces el evaluador debe reconocerlas) o si las llaves son obligatorias (y entonces el lector y el diálogo deben rechazar o avisar) |
| **Relacionados** | error 12, error 71, nota 28 |

**Qué pasa.** Una covariable de regresión escrita como fórmula que nombra una variable sin llaves, por ejemplo `pow(Age [1],8)`, se abre desde un fichero sin aviso y el diálogo de edición la da por buena, pero cualquier inferencia que proyecte ese potencial falla con `NonProjectablePotentialException.CannotEvaluate`. Guardar y volver a abrir no la repara.

Camino de llegada: abrir `0_miscellaneous/networks/mid/MID-mammography.pgmx` (o cualquier fichero con una fórmula así) y lanzar cualquier inferencia que proyecte esos potenciales; o, en cualquier red, escribir a mano en el diálogo de expresiones una covariable con el nombre de un padre sin llaves.

En `0_miscellaneous/networks/mid/`, las covariables con fórmula de `MID-CHAP-Ryan-Griffin.pgmx` y de `1-0/MID-mammography-1-0.pgmx` llevan llaves y se calculan. Van sin llaves las de `MID-mammography.pgmx` (cuatro covariables en los potenciales de `Death (OC) [1]` e `Invasive cancer [0]`) y, entre las redes de las pruebas, las de `integrationTests/src/main/resources/networks/mid/chap.pgmx` e `integrationTests/src/main/resources/networks/mid/chap-sv.pgmx` (covariable `Age [0]*12` de los costes hospitalarios y ambulatorios).

Por qué pasa: en los potenciales de regresión (familia `GLMPotential`: combinación lineal, exponencial, riesgos de Weibull y exponencial) cada covariable es una expresión de la clase `VariableExpression`. Su clase madre, `ReferencedExpression`, trocea el texto con la expresión regular `\{([^\{]+?)\}` y solo trata como referencia a una variable lo que va **entre llaves** y coincide con el nombre de una variable del potencial; el resto se guarda como texto sin interpretar y se le pasa tal cual al evaluador jeval. Por tanto `pow({Age [1]},8)` se calcula, y `pow(Age [1],8)` llega a jeval con el nombre de la variable dentro y falla.

Hay tres sitios que no siguen esa misma regla:

- **El lector de `.pgmx`** (`PGMXReader_0_2.getCovariates`, que también usa `PGMXReader_1_0` a través de `PGMXPotentialParsers`) pone las llaves solo cuando el texto de la covariable es **exactamente** el nombre de una variable (`Age [0]` → `{Age [0]}`). Si el nombre aparece dentro de una fórmula, la covariable se construye tal cual, sin referencias, y el fichero se abre sin ningún aviso.
- **El escritor** (`PGMXWriter_0_2.getCovariatesElement`) escribe `asStringExpression()`, que devuelve el texto sin llaves de lo que no se reconoció; así que guardar y volver a abrir deja la covariable igual de rota. No hay manera de repararla desde el propio programa salvo reescribirla a mano con llaves.
- **El diálogo de edición de expresiones** (`ArithmeticExpressionDialog`, que se abre al pulsar «añadir» o al hacer doble clic en una covariable en el panel `GLMPanel` del potencial) valida el texto con su propio `processExpression`, que sustituye por variables de jeval **tanto `{Nombre}` como `Nombre`** a secas. Si el usuario escribe el nombre a mano, sin pulsar en la lista de variables (que sí inserta las llaves), el diálogo pinta la expresión como válida y la acepta con «Aceptar»; la inferencia falla después. `VariableExpressionTextField` repite la misma validación. Esto se ha comprobado leyendo, no ejecutando (el diálogo no se puede construir sin pantalla).

Medido con un pequeño programa que abre la red, evalúa cada covariable con fórmula dando a todas las variables el valor 50, la guarda y la vuelve a abrir:

```
MID-mammography.pgmx (formato 0.2)
Death (OC) [1]      | ExponentialHazardPotential | pow(Age [1],8)                  | refs=0
↪  -> CannotEvaluate
Invasive cancer [0] | ExponentialHazardPotential | log(0.849828*(-0.0003717*Age [0]+ ...))
↪  | refs=0 -> CannotEvaluate
(dos covariables más iguales)                                                       ->
↪  CannotEvaluate
--- tras guardar y volver a abrir: las cuatro siguen con refs=0 y CannotEvaluate
1-0/MID-mammography-1-0.pgmx (formato 1.0)
Death (OC) [1]      | ExponentialHazardPotential | pow({Age [1]},8)                | refs=1
↪  = 3.90625E13
Invasive cancer [0] | ExponentialHazardPotential | log(0.849828*(-0.0003717*{Age [0]}+
↪  ...)) | refs=3 = -6.3118...
```

**El código.** io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1480-1495

```java
protected static VariableExpression[] getCovariates(Element xmlCovariates, List<Variable>
↪  variables) {
    VariableExpression[] covariates = new
↪  VariableExpression[xmlCovariates.getChildren().size()];
    int i = 0;
    for (Element xmlCovariate : xmlCovariates.getChildren()) {
        String xmlCovariateText = xmlCovariate.getText();
        var variableCovariate = variables.stream()
                                         .filter(var ->
                                         ↪  var.getName().equals(xmlCovariateText))
                                         .findFirst();
        if (variableCovariate.isPresent()) {
            covariates[i++] = variableCovariate.get().asVariableExpression();
        } else {
            covariates[i++] = new VariableExpression(variables, xmlCovariateText);
        }
    }
    return covariates;
}
```

gui/src/main/java/org/openmarkov/gui/dialog/common/ArithmeticExpressionDialog.java:195-204

```java
private String processExpression(String expression) {
    String processedExpression = expression;
    for (int i = 0; i < variables.size(); i++) {
        processedExpression = processedExpression.replace("{" + variables.get(i).getName()
↪  + "}", "#{v" + i + "}");
    }
    for (int i = 0; i < variables.size(); i++) {
        processedExpression = processedExpression.replace(variables.get(i).getName(), "#{v"
↪  + i + "}");
    }
    return processedExpression;
}
```

**Cómo arreglarlo.** Hace falta decidir cuál es la regla (ver `decision-de-equipo`), y aplicarla igual en los cuatro sitios: lector, escritor, diálogo y evaluador.

- **Opción A — las llaves son obligatorias.** El lector detecta una covariable con fórmula que

no contiene ninguna referencia pero sí el nombre de alguna variable del potencial, y lo dice (aviso al abrir o excepción de lectura con el nombre del nodo y el texto). `ArithmeticExpressionDialog.processExpression` y `VariableExpressionTextField.processExpression` quitan el segundo bucle (el que acepta nombres sin llaves), para que el diálogo no dé por buena una expresión que la inferencia no puede calcular. `MID-mammography.pgmx` se corrige a mano poniendo las llaves.

- **Opción B — los nombres sin llaves se aceptan al leer.** El lector reescribe el texto poniendo llaves a los nombres de variables que aparezcan. Ojo: una sustitución ingenua por `String.replace` (como la que ya hace `GLMPotential.processCovariate`, líneas 278-290, hoy sin llamadores) confunde un nombre con otro que lo contiene (`Age [0]` dentro de `Age at state entry [0]`), y los nombres llevan espacios y corchetes; habría que sustituir primero los nombres más largos y respetar los límites de palabra. El escritor ya escribiría entonces las llaves, y el fichero se repararía solo al guardarlo.

En ambos casos, la prueba de regresión: leer un `.pgmx` mínimo con una covariable `pow(X [0],2)` y comprobar el comportamiento elegido (aviso o rechazo, o que se evalúa a $X^2$ y que al guardar sale `pow({X [0]},2)`); y una prueba de la validación del diálogo con el mismo texto.

### Error 14 · [Corregido] Una regresión sin su covariable `Constant` (o un riesgo de Weibull sin `Gamma`) muere al calcularse con un índice −1 que no dice qué falta

| | |
|---|---|
| **Estado** | Corregido en el commit `522d926` (23-09-2026): al proyectarse, una regresión sin `Constant` o un riesgo de Weibull sin `Gamma` lanza `MissingMandatoryCovariate` con el nombre de la covariable que falta, y los accesores lo dicen también; el panel ya no deja quitar `Gamma` y la red de prueba lleva un riesgo exponencial. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:168-174`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:308-318`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:122-128`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:137-138`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:176`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:339-349`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/LinearCombinationPotential.java:97-113`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExponentialPotential.java:84-107` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 10, error 15, error 16, error 71, error 157 |

**Qué pasa.** Un potencial de regresión al que le falta su covariable `Constant`, o un riesgo de Weibull al que le falta `Gamma`, muere al calcularse con `ArrayIndexOutOfBoundsException: Index -1 out of bounds for length N`, que no dice qué covariable falta ni en qué nodo.

Caminos de llegada:

- **Fichero:** el lector de `.pgmx` no comprueba que estén los marcadores obligatorios; cualquier inferencia que proyecte un potencial así falla.
- **Interfaz:** en el panel de covariables (`GLMPanel`), el botón de quitar se apaga cuando la fila elegida es obligatoria, pero la lista de obligatorias se pide siempre a `GLMPotential.getMandatoryCovariates()`, que solo contiene `Constant`. Para un riesgo de Weibull se puede quitar la fila `Gamma` y la proyección falla después; es lo que describe error 157 (comprobado leyendo). `Constant` sí está protegida en la interfaz.

Hay una red del propio repositorio así: `integrationTests/src/main/resources/networks_for_every_potential/Dynamic-LIMID-every-potential.pgmx`, la que usan `PotentialsObeyTheEqualityContractTest` y `PotentialsCanBeClonedTest` para recorrer todos los tipos de potencial, tiene el nodo `ExponentialHazard [0]` escrito con `type="Hazard (Weibull)"` y `Constant` como única covariable. Esas pruebas no proyectan el potencial, así que no lo detectan.

Por qué pasa: los potenciales de regresión (familia `GLMPotential`) marcan dos covariables especiales con un texto fijo: `Constant` (el término independiente, obligatorio en todas) y `Gamma` (el logaritmo del parámetro de forma, obligatorio en el riesgo de Weibull, `WeibullHazardPotential`). Su posición se busca con `GLMPotential.getConstantIndex` y `WeibullHazardPotential.getGammaIndex`, que devuelven **−1** si el marcador no está. Quien las llama indexa directamente con lo devuelto:

- `WeibullHazardPotential.tableProject`: `Math.exp(coefficients[gammaIndex])` (línea 176) y `coefficients[constantIndex]`;
- `LinearCombinationPotential.tableProject` (línea 113) y `ExponentialPotential.tableProject` (línea 107): `coefficients[constantIndex]`;
- los accesores `GLMPotential.getConstant/setConstant` y `WeibullHazardPotential.getGamma/setGamma`.

Medido (el primer caso es el nodo de la red del repositorio citada arriba):

```
ExponentialHazard [0]  WeibullHazardPotential [Constant]         ->
↪  ArrayIndexOutOfBoundsException: Index -1 (WeibullHazardPotential.java:176)
WeibullHazard [0]      WeibullHazardPotential [Gamma, Constant]  -> [1.0, 0.0]
LinearCombination sin Constant                                    ->
↪  ArrayIndexOutOfBoundsException: Index -1 (LinearCombinationPotential.java:113)
Exponential sin Constant                                          ->
↪  ArrayIndexOutOfBoundsException: Index -1 (ExponentialPotential.java:107)
getConstant() sin Constant                                        ->
↪  ArrayIndexOutOfBoundsException: Index -1
```

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:308-318

```java
protected static int getConstantIndex(VariableExpression[] covariates) {
    int constantIndex = -1;
    int i = 0;
    while (i < covariates.length && constantIndex == -1) {
        if (covariates[i].asStringExpression().equals(VariableExpression.Common.CONSTANT.
        ↪  asStringExpression())) {
            constantIndex = i;
        }
        ++i;
    }
    return constantIndex;
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:137-138 y 176

```
int gammaIndex = getGammaIndex(covariates);
int constantIndex = getConstantIndex(covariates);
...
double shape = Math.exp(coefficients[gammaIndex]);
```

**Cómo arreglarlo.** Comprobar los marcadores en un único punto por el que pasan todas las proyecciones: `GLMPotential.tableProject(EvidenceCase, InferenceOptions, List)` (líneas 203-245), que ya comprueba que haya tantos coeficientes como covariables y lanza `NonProjectable PotentialException.CoefficientsDoNotMatchCovariates`. Añadir ahí una comprobación de que cada covariable obligatoria de la clase concreta está presente, lanzando una excepción del mismo tipo que diga el nodo y la covariable que falta.

Para que la lista de obligatorias dependa de la clase, hace falta un método de instancia (por ejemplo `getMandatoryCovariatesOfThisPotential()`), que `WeibullHazardPotential` redefina con `MANDATORY_COVARIATES` (líneas 40-43); `ExponentialHazardPotential` antepone `Gamma` por su cuenta al proyectar y no la necesita en su lista.

La misma regla debe cumplirse en:

- `GLMPanel.valueChanged` (`gui/src/main/java/org/openmarkov/gui/dialog/common/GLMPanel.java:136`) y en el doble clic de edición (línea 164), que deben usar ese método de instancia en vez del estático de `GLMPotential` (es el arreglo de error 157);
- los accesores `getConstant`/`setConstant`/`getGamma`/`setGamma`, que deben lanzar la misma excepción clara;
- opcionalmente, el lector de `.pgmx` (`PGMXPotentialParsers`), que podría avisar al abrir.

Además, corregir la red `Dynamic-LIMID-every-potential.pgmx`: o se cambia el tipo del nodo `ExponentialHazard [0]` a `Hazard (Exponential)`, o se le añade `Gamma`.

Prueba de regresión: en `core/src/test`, proyectar un `WeibullHazardPotential` sin `Gamma`, un `LinearCombinationPotential` y un `ExponentialPotential` sin `Constant`, y comprobar que lanzan la excepción nueva con el nombre de la covariable ausente; y una prueba de integración que proyecte todos los potenciales de `Dynamic-LIMID-every-potential.pgmx`.

### Error 15 · [Corregido] Un nodo de utilidad definido como regresión no cuenta como utilidad: el coste-efectividad falla con ClassCastException o pierde ese coste sin avisar, y la evaluación puede dar utilidad 0

| | |
|---|---|
| **Estado** | Corregido en el commit `c6f8655` (29-09-2026): `ProbNetPotentialQueries.tableProjectPotentials` da a toda tabla proyectada desde un nodo de utilidad el criterio del nodo cuando no lo trae, así que la combinación lineal, la exponencial y el árbol de regresiones cuentan como utilidad en la evaluación y en el coste-efectividad; también la utilidad constante definida como «Delta», que se perdía por la misma causa. No se ha hecho que `createCEPPotential` falle con un mensaje ante una utilidad sin criterio: con el arreglo, solo le llegaría la de un nodo de utilidad que no tenga criterio. |
| **Módulo** | core, inference |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.inference.algorithm.variableElimination |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/LinearCombinationPotential.java:84-131`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExponentialPotential.java:80-118`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Exa` |

| | |
|---|---|
| | `ctDistrPotential.java:69-75`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExactDistrPotential.java:93-100`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDPotential.java:669-690`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:432-434`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetPotentialQueries.java:101-121`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/VariableEliminationCore.java:192-217`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/DecisionVariableElimination.java:43-49` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Tapado por** | error 71: en las redes CHAP del repositorio, que fallan antes. |
| **Relacionados** | error 10, error 12, error 14, error 71, error 74, nota 28 |

**Qué pasa.** Un nodo de utilidad cuya relación es una regresión —«Linear combination» o «Exponential», que el diálogo del nodo ofrece para cualquier nodo de utilidad— no se trata como utilidad al calcular. Lo que ve el usuario depende de qué variables use el nodo:

- **Análisis de coste-efectividad con ámbito global** (menú Tools → «Cost-effectiveness analysis»), si el nodo depende de alguna decisión: aviso de error inesperado con `ClassCast Exception: GTablePotential cannot be cast to TablePotential`.
- **Ámbito de decisión**, si el nodo solo depende de la decisión elegida (el coste de una prueba o de una prótesis, por ejemplo): el análisis termina y la ventana enseña los costes, o las efectividades, **sin ese nodo**, sin ningún aviso. Si depende de otra decisión, falla como el global.
- **Si el nodo depende de una variable de azar**: la evaluación de un solo criterio da utilidad 0 y el coste-efectividad da efectividad (o coste) 0 en todas las opciones, sin aviso.

Lo mismo pasa con un árbol (`TreeADDPotential`) cuyas hojas son todas regresiones. Un árbol que mezcla una hoja de regresión con una «Exact» sí funciona, porque toma el criterio de la hoja «Exact»: por eso las redes CHAP del repositorio, cuyos costes son exponenciales colgadas de un árbol con una rama «Exact» para «dead», no lo muestran (fallan antes por error 71). Ninguna otra red de `0_miscellaneous/networks` en formato 0.2 tiene una regresión como utilidad, así que se llega construyendo el modelo en el editor. Con el jar ejecutable publicado, la ventana del ámbito de decisión no llega a abrirse (error 74); los costes equivocados se ven desde el IDE.

Por qué pasa. Durante la inferencia, un potencial cuenta como utilidad si lleva criterio de decisión (`Potential.isAdditive()` es `criterion != null`); en edición el criterio lo tiene la variable del nodo, no el potencial. `ExactDistrPotential.tableProject` copia a la tabla proyectada el criterio de la variable del nodo, pero `LinearCombinationPotential.tableProject` y `ExponentialPotential.tableProject` crean la tabla con `new TablePotential(variables, role)` y no se lo ponen, y `TreeADDPotential` toma el de la primera rama que lo tenga. Sin criterio:

- `VariableEliminationCore.createCEPPotential`, que junta costes y efectividades en la partición de coste-efectividad antes de eliminar la primera decisión, solo recoge los potenciales con criterio, así que esa tabla se queda fuera;
- al eliminar una decisión de la que depende, la tabla llega a `DecisionVariableElimination` junto a la partición, y la suma (`TablePotentialArithmetic.sum`, línea 218, llamada desde la línea 48) hace la conversión que falla;

- si depende de una variable de azar, el valor se pierde al eliminar esa variable y la utilidad sale 0 (medido; no se ha seguido paso a paso).

Medido, sustituyendo en redes del repositorio la utilidad «Exact» por la regresión que da los mismos números, creada como la crea el diálogo del nodo (`PotentialUtils.instanciateSafely` con el papel del potencial anterior):

- `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx`, «Prosthesis cost» como combinación lineal `394 + 185·{Prosthesis type}` (proyecta a 394 y 579, como el original): ámbito global → `ClassCastException`; ámbito de decisión «Prosthesis type» → costes 163,6 y 43,6, cuando con el original son 557,6 y 622,6 (la prótesis NP1 pasa a parecer la barata). La evaluación de un solo criterio da lo mismo que el original, 636,23.
- `0_miscellaneous/networks/id/ID-CEA-minimal.pgmx`, «Cost» como `14000·{Therapy}`: global → `ClassCastException`; ámbito de decisión → costes 0 y 0 en vez de 0 y 14 000. La exponencial da lo mismo.
- `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx`, «C: Cheap test» como `18·{Dec: Test}`: global y ámbitos de «Dec: New test» y «Therapy» → `ClassCastException`; ámbito de «Dec: Test» → el coste de 18 de la prueba desaparece de las dos opciones.
- La misma red, con «QALE» (depende de «Disease» y «Therapy») definida como «Exact» y como combinación lineal con los mismos números `5 − 2·{Disease} + {Therapy}`: con «Exact», utilidad 151 600 y efectividades 4,72, 5,72 y 6,72; con la combinación lineal, utilidad 0 y efectividades 0, 0 y 0.
- Un árbol con dos hojas de combinación lineal en «Cost» de `ID-CEA-minimal.pgmx` falla igual; con una hoja lineal y otra «Exact», da los costes correctos.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/ExactDistrPotential.java:69-75

```
public @NotNull TablePotential tableProject(EvidenceCase evidenceCase, InferenceOptions
↪  inferenceOptions) throws NonProjectablePotentialException {
    // get the projected TablePotential, which will be returned inside a list
    TablePotential projectedPotential = tablePotential.tableProject(evidenceCase,
↪  inferenceOptions);
    projectedPotential.setCriterion(getChildVariable().getDecisionCriterion());
    projectedPotential.setPotentialRole(PotentialRole.UNSPECIFIED);
    return projectedPotential;
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/LinearCombinationPotential.java:93-95 (lo mismo en `ExponentialPotential.java`, línea 88)

```
List<Variable> projectedPotentialVariables = new ArrayList<>(evidencelessVariables);
projectedPotentialVariables.addFirst(variables.getFirst());
TablePotential projectedPotential = new TablePotential(projectedPotentialVariables, role);
```

inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/VariableEliminationCore.java:197-204

```
for (Potential potential : markovDecisionNetwork.getAdditivePotentials()) {
    markovDecisionNetwork.removePotential(potential);
    if (potential.getCriterion().getCECriterion() == Criterion.CECriterion.Cost) {
        costPotentials.add((TablePotential) potential);
    } else {
        effectivenessPotentials.add((TablePotential) potential);
    }
```

```
}
```

**Cómo arreglarlo.** Que toda tabla proyectada desde el potencial de un nodo de utilidad lleve el criterio de la variable del nodo. Hay dos formas:

- clase por clase, como `ExactDistrPotential`: en `GLMPotential.tableProject` (por donde pasan combinación lineal, exponencial, riesgos de Weibull y exponencial y función), en `TablePotential.tableProject` (que tampoco lo copia: con una utilidad en forma de tabla y una política impuesta determinista, la utilidad esperada sale 0; medido sobre `integrationTests/src/main/resources/networks/id/ID-no-knowledge.pgmx` cambiando a mano el potencial de utilidad) y en `TreeADDPotential`, que hoy toma el criterio de sus ramas;
- en un solo sitio, `ProbNetPotentialQueries.tableProjectPotentials`, que proyecta todos los potenciales de la red antes de eliminar: si la variable del nodo tiene criterio y la tabla no, ponérselo. Así ninguna clase nueva puede olvidarlo.

Además, `VariableEliminationCore.createCEPPotential` no debería dejar fuera en silencio una tabla de un nodo de utilidad sin criterio: mejor fallar con un mensaje que lo diga.

Prueba de regresión: con `0_miscellaneous/networks/id/ID-CEA-minimal.pgmx` y «Cost» como combinación lineal `14000·{Therapy}`, `VECEAnalysis` debe dar costes 0 y 14 000 en ámbito global y en el de «Therapy»; y con `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx` y «QALE» como `5 − 2·{Disease} + {Therapy}`, `VEEvaluation` debe dar 151 600, lo mismo que la tabla «Exact» con esos números.

### Error 16 · [Corregido] Una tabla de fórmulas que marca «lo que sobra» con la palabra `Complement` en vez de `#` no se puede proyectar, y la red de prueba de todos los potenciales la usa

| | |
|---|---|
| **Estado** | Corregido en el commit `d71ecac` (10-10-2026): la red de pruebas escribe la marca como `#`, que es como la lee y la escribe el programa, y una prueba nueva proyecta ese nodo. El lector no se ha tocado: `Complement` sigue sin ser una marca. De los 1379 ficheros `.pgmx` que había en el equipo donde se comprobó, solo esa red (y sus copias) usaba la palabra. El mensaje para una fórmula mal escrita sigue sin nombrar el nodo ni la casilla. |
| **Módulo** | core, io, integrationTests |
| **Paquete** | org.openmarkov.core.expression; org.openmarkov.core.model.network.potential.operation; org.openmarkov.io.probmodel.reader |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/expression/VariableExpression.java:22`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/AugmentedProbTableInference.java:18-54`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:189-205`<br>`integrationTests/src/main/resources/networks_for_every_potential/Dynamic-LIMID-every-potential.pgmx:304-311` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir si la lectura acepta `Complement` como sinónimo de `#` o si basta con corregir la red de prueba. Manuel Arias lo decidió el 10-10-2026: solo `#` |
| **Relacionados** | error 12, error 14 |

**Qué pasa.** Una tabla de probabilidad aumentada («tabla de fórmulas») cuyas casillas de «lo que sobra» están escritas con la palabra `Complement` en vez de `#` se abre sin aviso, pero cualquier

inferencia que proyecte ese nodo falla con `NonProjectablePotentialException$CannotEvaluate`. El mensaje habla de una expresión inválida y no dice que la palabra esperada es `#`.

Camino de llegada: abrir un `.pgmx` cuya tabla de fórmulas use `Complement` y lanzar cualquier inferencia que proyecte ese nodo. Dentro del repositorio, el único fichero así es `integrationTests/src/main/resources/networks_for_every_potential/Dynamic-LIMID-every-potential.pgmx`, cuyo nodo `AugmentedProbTable [0]` trae `<Functions>"1" "Complement" "1" "Complement" "1" "Complement" "1" "Complement"</Functions>`. Las demás apariciones de `Complement` en los `.pgmx` son la distribución de incertidumbre `distribution="Complement"`, que es otra cosa y no está afectada. Las pruebas que usan esa red (`PotentialsObeyTheEqualityContractTest`, `PotentialsCanBeClonedTest`) no proyectan, así que no lo detectan.

`AugmentedProbTable` conserva comentada una constante `COMPLEMENT_FUNCTION` que usaba la palabra `Complement` (`core/src/main/java/org/openmarkov/core/model/network/potential/AugmentedProbTable.java:58-66`, en un bloque comentado), así que es posible que existan ficheros escritos con esa grafía fuera del repositorio.

Cómo funciona el marcador: una tabla de probabilidad aumentada (`AugmentedProbTable` `Potential`) guarda en cada casilla una expresión en vez de un número. Una casilla puede decir «lo que sobre»: al proyectar, `AugmentedProbTableInference` calcula primero las fórmulas de cada columna y reparte después lo que falta para sumar 1 entre las casillas marcadas como complemento. Una casilla se reconoce como complemento comparando su texto con `Variable Expression.Common.COMPLEMENT`, que es la cadena `#`.

Con `#` el mecanismo funciona también desde un fichero escrito por OpenMarkov. La interfaz usa ese mismo marcador: al editar una casilla, `AugmentedPotentialValueEdit` (`gui/src/main/java/org/openmarkov/gui/action/AugmentedPotentialValueEdit.java:158` y 199) pone `COMPLEMENT` en las demás casillas de la columna; el escritor `PGMXWriter_1_0` escribe el texto tal cual (`<Functions>"1" "#" ...`) y el lector `PGMXReader_1_0.getAugmentedProbTable` lo relee.

Lo que falla es la palabra `Complement`: el lector no la traduce, así que la casilla no se reconoce como complemento, se intenta calcular como fórmula y jeval la rechaza.

Medido con la red citada:

```
fichero con "Complement"  -> NonProjectablePotentialException$CannotEvaluate: Cannot
↪  evaluate 'Complement' due to Expression is invalid.
mismo fichero con "#"      -> [1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0]
escrito por el programa    -> <Functions>"1" "#" "1" "#" "1" "#" "1" "#"</Functions>, y al
↪  releer [1.0, 0.0, ...]
```

**El código.** core/src/main/java/org/openmarkov/core/expression/VariableExpression.java:22

```
public static final VariableExpression COMPLEMENT = new
↪  VariableExpression(Collections.emptyList(), "#");
```

io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:191-203

```
String functionsValue = xmlPotential.getChild(XMLTags.FUNCTIONS.toString()).getValue();
List<String> uncertainParametersList = Pattern.compile("\"(.*?)\"")
                                              .matcher(functionsValue)
                                              .results()
                                              .map(m -> m.group(1))
                                              .toList();
VariableExpression[] functionValues = new
↪  VariableExpression[uncertainParametersList.size()];
int i = 0;
for (String uncertainParameter : uncertainParametersList) {
    functionValues[i++] = new VariableExpression(
```

```
            Stream.concat(vDistributionTable.stream(),
↪  variables.stream()).distinct().toList(),
            uncertainParameter);
}
```

**Cómo arreglarlo.** Dos opciones, a decidir (ver `decision-de-equipo`):

- **Aceptar las dos grafías al leer.** En `PGMXReader_1_0.getAugmentedProbTable`, si el texto de una casilla es `Complement` (conviene decidir si sin distinguir mayúsculas), construir `VariableExpression.Common.COMPLEMENT`. El escritor seguiría escribiendo `#`, de modo que el fichero se normaliza al guardarlo. Es el único sitio de lectura: `getAugmentedProbTable` `Potential` y la lectura de `UnivariateDistrPotential` (líneas 155-160) pasan los dos por `getAugmentedProbTable`.
- **Solo `#`.** Corregir la red de pruebas (`Complement` → `#`) y, para que el usuario entienda el fallo, que el lector o `AugmentedProbTableInference` digan que una casilla no es una fórmula válida indicando el nodo y la casilla.

En ambos casos conviene añadir a las pruebas de integración una que proyecte todos los potenciales de `Dynamic-LIMID-every-potential.pgmx` (la misma que propone error 14), y en `io/src/test` una prueba que lea una tabla de fórmulas con el marcador elegido y compruebe el reparto.

### Error 17 · [Corregido] Quitar la etiqueta de una rama que otra rama referencia deja una referencia rota, y al guardar y reabrir la red no se puede evaluar

| | |
|---|---|
| **Estado** | Corregido en el commit `491a0d6` (15-10-2026), con la decisión que tomó Manuel Arias ese día: la opción (b), convertir cada referencia en una copia, y además un mensaje al abrir un fichero que ya tenga una referencia sin etiqueta. Tras cada operación de su menú, el editor del árbol da a toda rama cuya etiqueta ya no está en el árbol su propia copia del potencial que usaba (`TreeADDPotential.getReferencesWithoutLabel` y `TreeADDBranch.replace` `ReferenceWithACopy`). Esto cubre «Remove label» y también las otras dos operaciones que borran la rama etiquetada: «Remove subtree», medido, y «Associate states», solo leído, porque abre una ventana de selección. Al abrir un `.pgmx`, `PGMXReader_0_2.getPotentials` rechaza una referencia sin etiqueta, también en los árboles de las relaciones con sucesos, con «The relation of C cannot be read: a branch of its tree uses the label shared, which no branch of that tree has.». Con prueba en `gui` que usa el menú del editor de verdad: tras quitar la etiqueta, la red guardada y reabierta da las mismas probabilidades, y tras quitar el subárbol se reabre y se evalúa. `TreeADDBranch.copy()` no se ha tocado: ya no queda ninguna referencia sin su etiqueta que copiar. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.treeadd |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:1260-1264`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDBranch.java:196-212`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDBranch.java:368-372`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDPotential.java:594-606`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:1230-1240`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2` |

| | |
|---|---|
| | `.java:1214-1224` `core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDPotential.java:358` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Qué debe pasar al quitar una etiqueta en uso (prohibirlo, avisar, o convertir cada referencia en una copia del potencial) |

**Qué pasa.** En un potencial en forma de árbol (`TreeADDPotential`, módulo `core`), cada rama (`TreeADDBranch`) puede llevar una **etiqueta** («label»). Otra rama puede, en vez de tener su propio potencial, llevar una **referencia** a esa etiqueta: así las dos ramas comparten la misma tabla sin escribirla dos veces. En el editor de árboles (`TreeADDEditorPanel`, módulo `gui`; diálogo de propiedades del nodo, pestaña del potencial con relación de tipo árbol) se llega con el botón derecho sobre una rama: «Set label», «Set reference», «Remove label», «Remove reference».

Si el usuario quita con «Remove label» una etiqueta que otra rama referencia, el editor no dice nada, la referencia queda rota, y la red guardada y reabierta muere con `NullPointerException` al evaluarla. La red guardada queda inservible para cualquier inferencia sin que nada lo haya avisado.

Por qué pasa: «Remove label» (`removeLabel`, líneas 1260-1264) solo pone la etiqueta a `null`. No recorre el árbol para ver si alguna rama la estaba referenciando. La rama que la referenciaba se queda con su campo `reference` diciendo el nombre antiguo y con su puntero interno `referencedBranch` apuntando a la rama que ya no está etiquetada.

Medido sobre `0_miscellaneous/networks/id/ID-Monty-Hall-spanish.pgmx` (nodo «Puerta escogida 2»): se etiquetó la rama «no» como `theOne`, se hizo que la rama «sí» la referenciara, se abrió el editor y se ejecutó `removeLabel` sobre «no». Resultado:

1. En el editor, el árbol ya no tiene ninguna etiqueta, pero la rama «sí» sigue siendo una referencia (`isReference() == true`), el árbol la dibuja con el texto `theOne` y su menú sigue ofreciendo «Remove reference». Mientras dura la sesión, la rama sigue usando el potencial de «no» a través del puntero interno, así que la inferencia en memoria no falla.
2. Al aceptar el diálogo (`SetPotentialEdit`, que no copia el árbol) y guardar la red en ProbModelXML (`.pgmx`), el escritor (`PGMXWriter_0_2`, líneas 1230-1240) escribe `<Reference>theOne</Reference>` en la rama «sí» y ninguna `<Label>` en todo el árbol.
3. Al reabrir el fichero, el lector (`PGMXReader_0_2`, líneas 1214-1224) crea la rama con la referencia `theOne`; `TreeADDPotential.updateReferences` no encuentra ninguna rama con esa etiqueta y la rama se queda **sin potencial** (`getPotential() == null`).
4. Evaluar esa red (`VEEvaluation`) termina en `NullPointerException` en `TreeADDPotential.tableProject` (línea 358).

Hay además una incoherencia en la copia: `TreeADDBranch.copy()` (líneas 196-212) llama a `setReferencedBranch`, que toma la referencia de la etiqueta actual de la rama referenciada; como esa etiqueta ya es `null`, la copia pierde la referencia pero sigue apuntando a la rama del árbol **original**. Si en algún camino se copia el árbol antes de guardar, el resultado es otro distinto (se guardaría una copia del potencial en lugar de la referencia).

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:1260-1264

```
private void removeLabel(ActionEvent ae, TreeADDBranch branch, TreePath path) {
    branch.setLabel(null);
    TreeADDModel model = (TreeADDModel) jTree.getModel();
    model.notifyTreeStructureChanged(path);
```

```
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDBranch.java:368-372

```
public void setReferencedBranch(TreeADDBranch treeADDBranch) {
    this.reference = treeADDBranch.getLabel();
    this.referencedBranch = treeADDBranch;
    this.potential = null;
}
```

**Cómo arreglarlo.** Hace falta decidir el comportamiento (ver `decision-de-equipo`):

- (a) no ofrecer «Remove label», o rechazarlo con un mensaje, mientras alguna rama referencie la etiqueta;
- (b) quitarla y convertir cada rama que la referenciaba en una rama con su propia copia del potencial (lo mismo que hace hoy «Remove reference», líneas 1289-1295);
- (c) quitarla y quitar también las referencias, avisando.

Sitios donde la regla tiene que cumplirse: `TreeADDEditorPanel.removeLabel`; `TreeADDBranch.copy()` y `setReferencedBranch`, que no deberían depender de que la etiqueta siga puesta; y, como red de seguridad, el lector de `.pgmx`, que hoy acepta en silencio una referencia sin etiqueta y deja una rama sin potencial en lugar de dar un error de lectura claro.

Prueba de regresión: árbol con una rama etiquetada y otra que la referencia; quitar la etiqueta desde el panel; guardar y reabrir; comprobar que la red se evalúa (o que la operación se rechazó) y que ninguna rama queda sin potencial.

### Error 18 · [Corregido] El editor de árboles ofrece añadir a una hoja la propia variable del nodo; aceptarlo la escribe dos veces y la red guardada ya no se puede abrir

| | |
|---|---|
| **Estado** | Corregido en el commit `5555f2a` (14-10-2026), con el alcance que decidió Manuel Arias ese día: solo lo que se ofrece. `TreeADDBranch.getAddableVariables` ve un bucle sobre sí mismo cuando la variable del nodo está más veces entre las del árbol que en la hoja, como en una red de sucesos con un nodo padre de sí mismo (`Checkup` en `io/src/test/resources/DES-illness.pgmx`, cuya relación lleva la variable dos veces). El menú de la hoja y el diálogo «Add variables to potential» toman la lista de ese método. Medido en las 148 redes legibles del repositorio: sin el arreglo, a 1558 de las 1810 hojas de árbol se les ofrecía la variable del propio nodo; con él, a ninguna, y las demás variables ofrecidas son las mismas en todas las hojas. Con prueba en `core` de un árbol corriente y de un nodo padre de sí mismo. Una variable padre puede seguir apareciendo bajo una rama que agrupa varios de sus estados (`TreeADDEditorPanel.addSubtree` restringe entonces sus estados); eso no cambia. Lo que hace `addVariable` con una variable que ya está queda como estaba, sin camino de usuario (nota 164). |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDBranch.java:238-250`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:700-717`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:628-642` |

| | |
|---|---|
| | `gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:288-323`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:207-229` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué contrato tiene `addVariable` con una variable que ya está (hoy depende de la clase), y cómo se distingue de él el caso legítimo de los bucles sobre sí mismo de las redes de simulación de eventos discretos |
| **Relacionados** | error 159, nota 164, nota 182 |

**Qué pasa.** En el editor de árboles, el menú de una hoja (`TreeADDEditorPanel.setContextualMenuPotential`) y el diálogo «Add variables to potential» ofrecen siempre la propia variable del nodo, en cualquier tipo de red. Si el usuario la elige, la hoja queda como P(X | …, X), con la variable escrita dos veces. La comprobación de restricciones no dice nada y la evaluación en memoria sigue dando un número, pero la red guardada ya no se puede volver a abrir. Como se describe en error 159, la variable condicionada es además la única elección de ese diálogo que hace algo.

Los bucles sobre sí mismo solo son legítimos en las redes de simulación de eventos discretos, donde un nodo de azar o de suceso puede ser padre de sí mismo.

Por qué pasa: se combinan dos piezas.

**1. Qué se ofrece.** `TreeADDBranch.getAddableVariables` (módulo `core`, líneas 238-250) calcula las variables que se pueden añadir a la hoja de una rama. Parte de las variables del árbol, quita las que ya tiene el potencial de la hoja y, si «hay bucle sobre sí mismo», vuelve a añadir la variable condicionada. Pero la condición `selfLoop = addableVariables.contains(potential.getConditionedVariable())` pregunta si la lista de variables del árbol contiene la variable condicionada, y eso es verdad en **todo** árbol de probabilidad condicionada (la condicionada es siempre la primera).

**2. Qué hace al aceptarlo.** `Potential.addVariable` (líneas 700-717) añade la variable si no está; si ya está y es la primera, la **añade otra vez** (el propio código lo marca `FIXME` y cita los bucles de las redes de simulación de eventos discretos). `TablePotential.addVariable` (líneas 628-642) ni siquiera mira si ya está: siempre la añade.

Medido sobre `0_miscellaneous/networks/id/ID-Monty-Hall-spanish.pgmx` (nodo «Puerta escogida 2», hoja «no», una `TablePotential`):

- `addVariable` de la variable condicionada sobre un `UniformPotential` y sobre una `TablePotential` de una sola variable devuelve en ambos casos `[Puerta escogida 2, Puerta escogida 2]`.
- Poniendo en la hoja la tabla duplicada `[Puerta escogida 2, Puerta escogida 1, Puerta escogida 2]` (27 casillas), la comprobación de restricciones de la red (`checkProbNet`) no dice nada y la evaluación en memoria (`VEEvaluation`) sigue dando un número.
- Guardada la red con `PGMXWriter_1_0`, el fichero lleva la variable repetida en `<Variables>`; al **reabrirlo**, `PGMXReader` falla con `InvalidArgumentException: Argument table (27) is invalid because its length must be 9, the product of the numbers of states of the variables.` El usuario pierde la posibilidad de abrir la red que acaba de guardar.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/-TreeADDBranch.java:238-250

```
public List<Variable> getAddableVariables() {
    List<Variable> addableVariables = new ArrayList<>(parentVariables);
    boolean selfLoop = addableVariables.contains(potential.getConditionedVariable());
    addableVariables.removeAll(potential.getVariables());
    if (selfLoop) {
        addableVariables.add(potential.getConditionedVariable());
    }
    ...
```

core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:703-716

```
if (!variables.contains(variable)) {
    ...
} else {
    //DESnets - 18/03/2023 - due to self-loop, the conditioned variable can be repeated -
    ↪  FIXME
    if (variable.equals(variables.get(0))){
        List<Variable> newVariables = new ArrayList<Variable>(variables);
        newVariables.add(variable);
        newPotential = new UniformPotential(newVariables, role);
    } else{
        newPotential = this;
    }
}
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*, solo en la primera parte. La detección del bucle sobre sí mismo tiene que mirar si de verdad lo hay (por ejemplo, que el nodo sea padre de sí mismo en la red, o que la variable aparezca dos veces entre las del árbol), no si la condicionada está en la lista. Con eso, fuera de las redes de simulación de eventos discretos la variable condicionada deja de ofrecerse.

El contrato de `addVariable` con una variable que ya está debe ser uno solo para todas las clases: hoy `Potential` la duplica solo si es la primera, `TablePotential` siempre, y `TreeADDPotential` y `SumPotential` mutan el receptor. Hay que decidirlo (ver `decision-de-equipo`); los contratos sin escribir de las operaciones sobre potenciales están recogidos también en nota 164, donde no hay nada que corregir. Si se decide que duplicar solo es válido con bucle sobre sí mismo, la comprobación debe estar en `addVariable` (o en quien la llama) y no depender de cada llamador.

Sitios que llaman a `addVariable` y deben respetar la misma regla: `TreeADDEditorPanel.addVariablesToPotential` (línea 785), `AddLinkEdit.doEdit` (líneas 207-229, que ya trata aparte el bucle sobre sí mismo de las redes de simulación de eventos discretos), `InvertLinkEdit` (línea 124), `TreeADDPotential.addVariable` (línea 331), `DiscretizedCauchyPotential.addVariable` y `MIDTemporalEvolution` (línea 584).

Además, el lector de `.pgmx` debería dar un mensaje claro ante una variable repetida en un potencial (hoy da el error de tamaño de tabla), y la comprobación de red podría detectarla antes de guardar.

Prueba: en un diagrama de influencia, el diálogo «Add variables to potential» de una hoja no ofrece la variable del nodo; en una red de simulación de eventos discretos con bucle sobre sí mismo sí la ofrece; y guardar y reabrir la red tras añadir variables funciona.

### Error 19 · [Corregido] Tras cancelar la ventana de potencial de un suceso con dos sucesos padres, la simulación usa siempre los parámetros del primero

| | |
|---|---|
| **Estado** | Corregido en el commit `d53200e` (13-10-2026), junto con el error 20: el constructor de copia de `TableWithEvents` pone la tabla interna (y la de funciones, si la hay) sobre las variables recién calculadas para la copia, de modo que la tabla y `convert` usan la misma variable `Events`. Con prueba (`CopiedEventRelationsSampleLikeTheOriginalTest`): con dos sucesos padres y tasas 0,1 y 0,3, `copy()` y `deepCopy` de un `DistributionTablePotential` muestrean lo mismo que el original para cada suceso, también con un padre numérico; antes del arreglo fallaban. No se ha comprobado en la ventana. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:112-124`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:150-162`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:445-454`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java:104-112`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:238-268`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:148-153,570-579,709-715`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:116-118` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 20, error 21 |

**Qué pasa.** En una red de simulación de eventos discretos, un nodo de suceso cuyo tiempo hasta ocurrir depende de qué suceso lo desencadenó lleva un `DistributionTablePotential`: una distribución (exponencial, exacta…) cuyos parámetros se leen de una tabla interna, `TableWithEvents`.

Si ese nodo tiene dos o más sucesos padres y el potencial se copia, la copia lee siempre los parámetros del primer suceso, cualquiera que sea el que haya ocurrido. La simulación no falla ni avisa: da tiempos hasta el suceso con los parámetros de otra columna y, por tanto, costes y efectividades distintos de los que el modelo dice. Con un solo suceso padre no se nota (la única columna es la buena).

Caminos de llegada:

1. Ventana de edición del potencial de un nodo (doble clic en el nodo, pestaña de potencial, o «Editar potencial»). Al abrirse, `PotentialEditPanel` guarda `this.node.getPotential().deepCopy(...)` como potencial original; al pulsar Cancelar, `uncommitChanges` → `removePotentialOnClose` vuelve a poner en el nodo **esa copia**, no el objeto que tenía. `Potential.deepCopy` construye la copia con el constructor de copia de la clase (por reflexión) y `DistributionTablePotential` no redefine `deepCopy`, así que la copia defectuosa se queda en el nodo. Si se acepta con cambios, la edición `SetPotentialEdit` guarda esa misma copia como estado anterior y la restituye al deshacer. La ventana no se ha podido construir sin pantalla, así que este paso está comprobado leyendo `PotentialEditPanel`; la consecuencia sobre el potencial se midió ejecutando `deepCopy`.

2. Copiar y pegar un nodo de suceso con dos o más sucesos padres (`PasteEdit` usa `copy()`).

Por qué pasa: la tabla interna no tiene una columna por cada suceso padre como variable separada. `setEventAsStates` fabrica una variable auxiliar llamada `Events` cuyos estados son los nombres de los sucesos padres, y `convert` traduce «ha ocurrido el suceso X» a un hallazgo sobre esa variable `Events`.

El constructor de copia de `TableWithEvents` hace dos cosas que no encajan. Primero copia la tabla interna (`new TablePotential(potential.getTablePotential())`), que conserva las variables del original y, entre ellas, **la variable `Events` del original**. Después llama a `setEventAsStates`, que construye una variable `Events` **nueva**, y es esa la que usa `convert`. Como `Variable` compara por identidad de objeto, la tabla busca en la evidencia una variable que no está. Y `TablePotential.getValue(EvidenceCase)` no protesta cuando falta una de sus variables: la salta, lo que equivale a leer su estado 0. `DistributionTablePotential(DistributionTablePotential)` copia su tabla interna con ese mismo `copy()`, así que hereda el defecto.

Medido:

- Un `DistributionTablePotential` exponencial (parametrización `Lambda`) con dos sucesos padres A y B y tasas 0,1 y 0,3. Ocurrido B, el original devuelve 2,3104906018664844 (la mediana con tasa 0,3, con número aleatorio 0,5) y su `copy()` devuelve 6,931471805599453, que es la de tasa 0,1. En la copia, la tabla interna contiene la variable `Events` del original y no la suya.
- Camino real, en una red de sucesos de prueba en la que el suceso terminal `Death` depende del suceso `Onset` y del nodo `Ill`: se añade a `Death` un segundo suceso padre (`Checkup`), con parámetro «exacto» 3,0 si lo desencadena `Onset` y 1,0 si lo desencadena `Checkup` (con `Ill`=yes). Antes: tras `Checkup`, el tiempo hasta la muerte es 1,0. Tras sustituir el potencial por su `deepCopy(red)` —lo que hace la ventana al cancelar—, el mismo caso devuelve 3,0.

La copia de `TransitionTablePotential` por su constructor de copia tiene el mismo desajuste (su muestreo lanza `NullPointerException`); hoy no se ve porque `TableWithEvents.deepCopy` rehace las variables internas y porque `copy()` devuelve otra clase, lo que se describe en error 20.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:112-124

```
    public TableWithEvents(TableWithEvents potential) {
        super(potential);
        this.setTablePotential(new TablePotential(potential.getTablePotential()));
        this.useTableWithFunctions = potential.useTableWithFunctions;
        if (potential.tableWithFunctions != null) {
            this.tableWithFunctions = (TableWithFunctions)
            ↪  potential.tableWithFunctions.copy();
        }
        this.hasImpossibleConfigurations = potential.hasImpossibleConfigurations;
        this.impossibleConfigurations = potential.impossibleConfigurations == null
                ? new ArrayList<>() : new ArrayList<>(potential.impossibleConfigurations);
        setEventAsStates(variables.subList(1, variables.size()));
        rebuildTableVariables();
    }
```

core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:238-248

```
    public double getValue(List<Variable> variables, int[] statesIndices) {
        int position = 0;
        for (int i = 0; i < variables.size(); i++) {
            Variable variable = variables.get(i);
```

```
            int indexVariable = this.variables.indexOf(variable);
            if (indexVariable != -1) {
                position += offsets[indexVariable] * statesIndices[i];
            }
        }
        return values[position];
    }
```

gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:147-152 y 697-701

```
        Potential lastPotential = this.node.getPotential();
        if (lastPotential != null) {
            this.originalPotential =
            ↪  this.node.getPotential().deepCopy(this.node.getProbNet());
        ...
    protected void removePotentialOnClose(@Nullable Potential originalPotential) {
        if (originalPotential != null) {
            LinkRestrictionPotentialOperations.setPotentialWithRestrictions(this.node,
↪  originalPotential);
        }
    }
```

**Cómo arreglarlo.** En el constructor de copia de `TableWithEvents`, después de `setEventAsStates` y `rebuildTableVariables`, poner las variables de la tabla interna (y de `tableWithFunctions` si existe) sobre `tableVariables`, igual que ya hace `TableWithEvents.deepCopy` (líneas 445-454). Así la copia y su variable `Events` vuelven a hablar de lo mismo, y `deepCopy` puede dejar de repetirlo.

Sitios donde la misma regla («las variables de la tabla interna son las derivadas del potencial que la contiene») tiene que cumplirse:

- `TableWithEvents(TableWithEvents)` (copia) y `TableWithEvents.deepCopy`.
- `DistributionTablePotential(DistributionTablePotential)`: comparte `distributionVariable` y copia la tabla interna con `copy()`; y no redefine `deepCopy`, de modo que una copia profunda hacia **otra** red deja la tabla interna con las variables de la red de origen (hoy el único `deepCopy` alcanzable para este potencial es el de la ventana, dentro de la misma red).
- `TableWithEvents.setVariables` (líneas 257-260) pone en la tabla interna las variables exteriores, que no incluyen `Events`: cualquier llamador que lo use deja la tabla desajustada de otra forma.
- `TableWithEvents.replaceVariable` no está redefinido (lo usa `PasteEdit`); lo que eso provoca al pegar se describe en error 21.

Hay otra opción, que exigiría una decisión del equipo: que `TablePotential.getValue(EvidenceCase)` falle cuando falta un hallazgo de una variable de la tabla, en vez de leer el estado 0. Haría ruidoso este defecto y otros parecidos, pero cambia un método que usa todo el proyecto.

Pruebas de regresión: para `DistributionTablePotential` y `TransitionTablePotential` con dos sucesos padres, `copy()` y `deepCopy(misma red)` muestrean lo mismo que el original para cada suceso; lo mismo con un padre numérico (rama `tableWithFunctions`).

### Error 20 · [Corregido] Copiar un nodo de azar con tabla de transiciones deja un nodo que ya no sabe a qué estado pasa

| | |
|---|---|
| **Estado** | Corregido en el commit `d53200e` (13-10-2026), junto con el error 19: `TransitionTablePotential` redefine `copy()` y devuelve una tabla de transiciones. Con prueba (`CopiedEventRelationsSampleLikeTheOriginalTest`): la copia es de la misma clase y, con dos sucesos padres, pasa al mismo estado que el original para cada suceso. Pegar nodos de sucesos sigue sin funcionar del todo por el error 21. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/TransitionTablePotential.java:27-80`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:430-433`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:486-492`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:116-118` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 19, error 21, nota 184 |

**Qué pasa.** En una red de simulación de eventos discretos (en OpenMarkov, redes de tipo `DESNet`), un nodo de azar que cambia de estado cuando ocurre un suceso lleva un potencial `TransitionTablePotential`: una tabla que dice, para cada suceso que puede ocurrir, con qué probabilidad pasa el nodo a cada estado. Esa clase hereda de `TableWithEvents` y lo único que añade es saber muestrearse (`sampleConditionedVariable`).

Lo que ve el usuario: al copiar y pegar un nodo así, el nodo pegado se queda en su primer estado para siempre durante la simulación, sin aviso.

Camino de llegada: el único llamador de producción de `copy()` sobre estos potenciales es pegar nodos. `PasteEdit` (menú Edición, Copiar y Pegar en el editor de redes) copia cada potencial con `originalPotential.copy()`. En la simulación, el `NaN` («no es un número») que devuelve la copia acaba convertido a entero (`(int) NaN` vale 0 en Java) al leer el estado del nodo (`ChanceEvaluation.computeChanceValue`, `DESLogTextWriter.logChanceChange`, `Configuration.addFinding`). Esta última parte, lo que hace la simulación con el `NaN`, está comprobada leyendo el código, no ejecutando la simulación con la red pegada.

El otro camino de copia, la copia profunda `Potential.deepCopy` (que usa la ventana de edición de potenciales para poder cancelar), no pasa por `copy()`: invoca por reflexión el constructor de copia de la clase exacta, y `TableWithEvents.deepCopy` rehace después las variables internas, así que para esta clase sí funciona.

Por qué pasa: `TransitionTablePotential` no redefine `copy()`, así que se ejecuta el de `TableWithEvents`, que construye un objeto de la clase madre: la copia de una tabla de transiciones es una `TableWithEvents` a secas. Y el muestreo de `TableWithEvents` no está implementado: devuelve siempre `Double.NaN`. No hay excepción; quien usa la copia recibe `NaN`. Si alguien la moldea a su tipo original, `(TransitionTablePotential) p.copy()` lanza `ClassCastException`.

Medido:

- Una `TransitionTablePotential` sobre `[Ill, E1]` con valores `{0, 1}` (al ocurrir E1 el nodo pasa a «yes») contesta 1.0 al muestrearla con E1 ocurrido; su `copy()` es de clase `Table`

`WithEvents` y contesta `NaN`; el moldeado a `TransitionTablePotential` lanza `ClassCastException`.
- En una red de sucesos de prueba, con un suceso `Onset` que hace pasar `Ill` de «no» a «yes», copiar `Onset` e `Ill` y pegarlos deja a `Ill'` con un potencial `TableWithEvents`; muestrearlo con `Onset'` ocurrido devuelve `NaN`, mientras el original devuelve 1.0.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:430-433 y 486-492

```
//TODO
@Override public Potential copy() {
    return new TableWithEvents(this);
}
...
/**
 * Not implemented for this potential: always returns {@link Double#NaN}.
 */
@Override
public double sampleConditionedVariable(double[] randomNumbers, EvidenceCase parents)
↪  throws IncompatibleEvidenceException.EvidenceIsIncompatibleWithOther {
    return Double.NaN;
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/TransitionTablePotential.java:43-46 (tiene constructor de copia, pero no `copy()`)

```
//TODO
public TransitionTablePotential(TransitionTablePotential potential) {
    super(potential);
}
```

**Cómo arreglarlo.** Redefinir `copy()` en `TransitionTablePotential` para que devuelva `new TransitionTablePotential(this)`.

**No basta solo con eso**: el constructor de copia que hereda (`TableWithEvents(TableWithEvents)`) deja la tabla interna apuntando a la variable `Events` del original, que es el defecto de error 19. Con el `copy()` corregido, el muestreo de la copia pasa de devolver `NaN` a lanzar `NullPointerException` (medido llamando al constructor de copia directamente: `EvidenceCase.getFinding(...)` devuelve `null` dentro de `TablePotential.sampleConditionedVariable`). Hay que arreglar error 19 a la vez.

Conviene revisar en el mismo paso todas las subclases de `TableWithEvents` y de `Potential` que implementan `DESSimulablePotential`: cada una debe redefinir `copy()` (hoy `UnivariateDistrPotential` y `DistributionTablePotential` lo hacen; `TransitionTablePotential` no).

Aparte, y no es el mismo defecto: al pegar, las relaciones de sucesos siguen consultando por dentro las variables del nodo original, lo que se describe en error 21 (medido: la tabla interna de `Ill'` sigue siendo `[Ill, Events{Onset}]`). Arreglar `copy()` no hace por sí solo que pegar nodos de sucesos funcione.

Prueba de regresión: para una `TransitionTablePotential` con al menos dos sucesos padres, `copy()` devuelve la misma clase y, para cada suceso, la copia muestrea lo mismo que el original con los mismos números aleatorios.

### Error 21 · [Corregido] Pegar nodos de una red de sucesos deja sus relaciones consultando los padres del original: la simulación usa siempre los números del primer estado de cada padre, o se cae

| | |
|---|---|
| **Estado** | Corregido en el commit `07647a5` (13-10-2026): `replaceVariable` llega ahora a lo que cada relación de sucesos guarda por dentro. `TableWithEvents` (y con ella `TransitionTablePotential`) rehace la variable `Events` y pone sobre las variables nuevas su tabla y su tabla de funciones, también en `setVariables`; `DistributionTablePotential` lo pasa a su tabla de parámetros y a su lista de padres numéricos; `ExactDistrPotential`, a su tabla; y los dos árboles con sucesos, cuyo método estaba vacío, a sus listas de padres y a sus árboles. `TreeADDPotential.replaceVariable` cambia además la variable raíz y los padres que guardan sus ramas, y ya no falla con una rama que remite a otra. Con pruebas: `EventRelationsFollowAReplacedVariableTest` (para cada relación, la copia con todas las variables cambiadas responde como el original y no guarda en ningún campo una variable del original) y `PastedEventNodesAnswerLikeTheOriginalsTest` (una red de sucesos de siete nodos pegada con `Paste Edit` en otra red y en la misma: cada nodo pegado responde como su original). No se ha lanzado la simulación completa de la red pegada ni se ha comprobado en la ventana. Al corregirlo apareció el error 234. |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.action; org.openmarkov.core.model.network.potential; org.openmarkov.core.model.network.potential.treeadd |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:113-141`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:331-334`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:726-736`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java:104-112,260-301`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:112-124,172-187,445-454`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExactDistrPotential.java:57-60,158-160,166-169`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithEventsPotential.java:229-231`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:180-182`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:238-248`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDPotential.java:840-849` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 19, error 20, error 99, error 105 |

**Qué pasa.** En una red DES (simulación de eventos discretos), copiar nodos y pegarlos (menú Edición, Copiar y Pegar del editor de redes) deja los nodos pegados con relaciones que, por dentro, siguen consultando las variables de los nodos originales. El portapapeles es común a todas las redes abiertas, así que se puede pegar en otra pestaña, por ejemplo en una red de sucesos nueva.

La simulación de la red pegada no avisa de nada. Cada tiempo hasta un suceso y cada valor de una utilidad se lee como si todos sus padres de estados (la decisión, los nodos de azar) estuvieran en su primer estado. Con las relaciones de árbol con sucesos, muestrear el nodo pegado lanza una excepción (medido sobre la relación, sin lanzar la simulación).

Medido con una simulación completa, en una red de sucesos de prueba:

- una decisión `Therapy` (none, drug) y un nodo de azar `Sex` (male, female, al 50 %);
- un suceso `Onset` con tiempo exacto de 2 o 5 años para los hombres (none, drug) y de 3 o 6 para las mujeres;
- un suceso terminal `Death` 4 años después de `Onset`;
- dos utilidades acumulativas: `QALY`, 1 por año, y `Monitoring`, 10 por año con none y 40 con drug;
- 200 individuos y horizonte de 20 años.

```
red original:                          none: efectividad 6.52, coste 65.2   drug:
↪  efectividad 9.52, coste 380.8
todos sus nodos y enlaces pegados
en otra red con los mismos criterios: none: efectividad 6.0,  coste 60.0   drug:
↪  efectividad 6.0,  coste 60.0
```

En la red pegada, el tratamiento y el sexo no cambian nada: `Onset` ocurre siempre a los 2 años y `Monitoring` cuesta siempre 10.

Por qué pasa: `PasteEdit` (paso final, líneas 113-141) copia cada potencial con `copy()` y después cambia sus variables por las de los nodos pegados, una a una, con `potential.replaceVariable(i, variable)`. La versión de la clase base (`Potential.replaceVariable`, líneas 331-334) solo cambia la lista exterior `variables`. Los potenciales compuestos guardan otra lista por dentro, y ninguno de los de sucesos la actualiza:

- `DistributionTablePotential`: su `tableWithEvents`, copiado en el constructor de copia (líneas 104-112), sigue sobre `[distributionVariable, Therapy, Sex]` del original. Al muestrear (`sampleConditionedVariable`, líneas 260-301) busca cada parámetro con `tableWithEvents.getTablePotential().getValue(...)` usando los hallazgos de los padres pegados, y `TablePotential.getValue` se salta cualquier variable que no sea suya (líneas 238-248), lo que equivale a leer su estado 0.
- `ExactDistrPotential` (el valor exacto de las utilidades de una red de sucesos; el lector convierte en él las `UnivariateDistr` de las utilidades): su `tablePotential` sigue sobre los padres del original, y `sampleConditionedVariable` (líneas 158-160) lee `tablePotential.getValue(parents)`. Su `setVariables` (líneas 166-169) sí actualiza la tabla interna, pero `PasteEdit` no lo usa.
- `TableWithEvents` y `TransitionTablePotential`: la tabla interna sigue con las variables del original. (La copia de una tabla de transiciones falla antes por otro motivo, descrito en error 20.)
- `TreeWithEventsPotential` y `TreeWithExcludedEventsPotential` («Tree with Events» y «Tree with Excluded Events»): su `replaceVariable` está vacío (`//TODO`), así que ni siquiera cambian las variables exteriores.

`TreeADDPotential.replaceVariable` (líneas 840-849) sí propaga el cambio a sus ramas: es el modelo que les falta a los demás.

Medido también muestreando cada relación pegada con los mismos hallazgos que el original, en una segunda red de prueba. Es como la anterior, más un nodo de azar `Ill` que pasa a «yes» cuando ocurre `Onset`, una utilidad `CareCost` que vale 100 o 2000 según `Ill`, y un `Death` que ocurre a los 8 o a los 3 años según `Ill`. Se pegó en la misma red (nombres con apóstrofo) y en otra red (mismos nombres), con resultados iguales en los dos casos:

- `Onset` (tabla de distribución sobre `Therapy` y `Sex`), con drug y mujer: el original da 6,0 y la copia 2,0.

- `CareCost` (valor exacto sobre `Ill`), con `Ill`=yes: el original da 2000 y la copia 100. `Monitoring`, con drug: 40 y 10.
- `Death` (tabla de distribución sobre el suceso `Onset` y sobre `Ill`), con `Onset` ocurrido e `Ill`=yes: el original da 3,0 y la copia 8,0. Aquí se suma el desajuste de la variable interna `Events` descrito en error 19.
- Un `Ill` con «Tree with Events» sobre `[Ill, Onset, Sex]`, pegado en otra red: las tres variables de la relación pegada son las del original, y muestrearla lanza `NullPointerException` (`getTree` devuelve `null`). Con «Tree with Excluded Events», sin pantalla sale `Headless Exception` desde la ventana de aviso descrita en error 105.

Dónde se ve: pegar un modelo entero en la misma red no se puede simular, porque la simulación solo admite una decisión (`DESInference.initialize`, líneas 416-420, por lectura). Por eso el caso que llega a la simulación es pegar en otra red, o pegar en la misma red una parte que no incluya la decisión. Si un padre no se copia, `PasteEdit` lo quita con `removeVariable`, cuya versión de la clase base (`Potential.removeVariable`, líneas 726-736) devuelve una relación uniforme: medido, pegar `Onset` o `Ill` sin sus padres deja un `UniformPotential`, con la consecuencia que describe error 99.

**El código.** gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:117-131

```
                    for (Potential originalPotential : originalNode.getPotentials()) {
                        Potential potential = originalPotential.copy();
                        List<Variable> externalVars = new ArrayList<>();
                        for (int i = 0; i < potential.getNumVariables(); ++i) {
                            String variableName = potential.getVariable(i).getName();
                            if (PasteEdit.this.newVariables.containsKey(variableName)) {
                                Variable variable =
↪   this.probNet.getVariable(PasteEdit.this.newVariables.get(variableName));
                                potential.replaceVariable(i, variable);
                            } else {
                                externalVars.add(potential.getVariable(i));
                            }
                        }
                        for (Variable extVar : externalVars) {
                            potential = potential.removeVariable(extVar);
                        }
```

core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:331-334

```
    public void replaceVariable(int position, Variable variable) {
        variables.remove(position);
        variables.add(position, variable);
    }
```

core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithEventsPotential.java:229-231

```
    @Override public void replaceVariable(int position, Variable variable) {
        //TODO
    }
```

**Cómo arreglarlo.** Cada potencial compuesto debe propagar `replaceVariable` a lo que guarda por dentro, como ya hace `TreeADDPotential`:

- `DistributionTablePotential`: las variables de `tableWithEvents` (la variable de parámetros más los padres que no son numéricos), su tabla y su `tableWithFunctions`, y la lista `numeric Variables`.

- `TableWithEvents`: la tabla interna y `tableWithFunctions`. Si la variable cambiada es un suceso, hay que rehacer también la variable `events`, cuyos estados son los **nombres** de los sucesos padres: un suceso que pasa a llamarse `Onset'` tiene que cambiar el estado, o `convert` (líneas 172-187) no lo encontrará. `TableWithEvents.deepCopy` (líneas 445-454) ya rehace todo esto a partir de las variables exteriores y se puede reutilizar.
- `ExactDistrPotential`: su `tablePotential`, igual que hace ya `setVariables` (líneas 166-169).
- `TreeWithEventsPotential` (las claves de su mapa de árboles y cada árbol) y `TreeWithExcludedEventsPotential` (su `noEventTree`): escribir el método, que hoy está vacío.

Otra vía, que no quita la anterior: que `PasteEdit` construya la lista completa de variables nuevas y la ponga con `setVariables`, o copie con una correspondencia de nombres como hace `Potential.deepCopy`. `DistributionTablePotential` y los dos árboles tampoco propagan hoy `setVariables`, así que el trabajo en los potenciales hace falta igualmente.

`PasteEdit` es hoy el único llamador de producción de `replaceVariable` sobre estos potenciales. Con el arreglo, pegar sigue sin funcionar del todo para las tablas de transiciones y para los sucesos con dos o más sucesos padres mientras no se corrijan error 20 y error 19.

Pruebas de regresión:

- Para cada relación de sucesos (`DistributionTablePotential`, `TransitionTablePotential`, `ExactDistrPotential`, los dos árboles), `copy()` más `replaceVariable` de todas sus variables no deja ninguna referencia a las variables del original, tampoco en las listas internas.
- Pegar todos los nodos de la red de prueba descrita en otra red da los mismos resultados de simulación que el original.

### Error 22 · [Corregido] Abrir el editor de un «Tree with Events» o «Tree with Excluded Events» y aceptar deja un deshacer que borra la relación del nodo

| | |
|---|---|
| **Estado** | Corregido en el commit `0f217e1` (14-10-2026). Medido antes del arreglo con el panel real y las dos clases: no solo aceptar y deshacer dejaba el nodo sin relación (ya con la lista vacía, no con `[null]`); **cancelar sin tocar nada** también, porque `uncommitChanges` restaura la copia guardada al abrir, que era `null`. La copia de una red se rechazaba con «potential is invalid because it is null». Ahora `deepCopy` de las dos clases construye la relación sobre las variables de la red destino y copia con `deepCopy` sus árboles internos (el de cada suceso en `TreeWithEventsPotential`; `noEventTree` en `TreeWithExcludedEventsPotential`). Se quitan `sample`, `getInducedFindings` y `toString` sin escribir, así que valen los de la base (la propia relación, ningún hallazgo, «P(X \| E, Y)»). Quedan sin escribir `reorder`, `replaceNumericVariable` y `scalePotential`, fuera de este error. Pruebas: con el editor, aceptar y deshacer, y cancelar, conservan la relación; la copia de la red trae la relación con árboles nuevos sobre sus variables. Con el código anterior fallaban las seis. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.treeadd |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithEventsPotential.java:217-226,256-259,275-278`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:168-177,207-210,226-229`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:148-153,537-561,563-568`<br>`core/src/main/java/org/openmarkov/core/action/core/SetPotentialEdit.java:69-91`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetCopier` |

| | |
|---|---|
| | `.java:111` `core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:801-820` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 23, error 105 |

**Qué pasa.** En una red de simulación de eventos discretos, si el usuario abre las propiedades de un nodo cuya relación es «Tree with Events» o «Tree with Excluded Events» (o «Edit probability»), pulsa OK sin tocar nada y después Ctrl+Z, el nodo se queda sin relación: su lista de relaciones pasa a ser `[null]`. A partir de ahí `ProbNet.deepCopy` falla con `NullPointerException` en `ProbNetCopier`, línea 111. Por lectura, el diálogo siguiente sobre ese nodo no podrá construir su selector de relación (`getPotentials().getFirst().getClass()`), y el escritor de ficheros encontrará un `null`; esto no se ha comprobado con la aplicación abierta.

Por qué pasa: `TreeWithEventsPotential` y `TreeWithExcludedEventsPotential` (relaciones de la simulación de eventos discretos que combinan árboles con sucesos) tienen varios métodos sin escribir, con `//TODO`: `deepCopy(ProbNet)` devuelve `null` (líneas 256-259 y 207-210), `sample()` y `getInducedFindings` devuelven `null`, y `toString()` devuelve `""`. `deepCopy` anula el de la clase base (`Potential.deepCopy`, líneas 801-820), que sí copiaría la relación por su constructor de copia; bastaba con no anularlo. Medido: `deepCopy` de las dos devuelve `null` y `toString` la cadena vacía.

`deepCopy` tiene al menos dos llamadores de producción:

1. **El editor de relaciones.** `PotentialEditPanel` (módulo `gui`, líneas 145-151) guarda al abrirse `originalPotential = node.getPotential().deepCopy(...)`, que para estas dos clases es `null`. Al aceptar, `commitChanges` (líneas 537-549) compara la relación actual con `null` (`potentialHasChanged`, líneas 552-557; medido `ReflectionEquality.areEquals(null, relación) = false`), concluye que ha cambiado aunque el usuario no haya tocado nada, y ejecuta `SetPotentialEdit(nodo, null, relación)`. Deshacer esa edición (`SetPotentialEdit.undo`, líneas 88-91) pone como relación del nodo `null`. Medido: tras ejecutar y deshacer esa edición, la lista de relaciones del nodo es `[null]`, y `ProbNet.deepCopy` falla como se ha dicho.
2. **La copia profunda de una red.** `ProbNetCopier.deepCopy` (línea 111) copia cada relación con `deepCopy`; medido que la copia de una red deja el nodo con `[null]`. Hoy su único llamador de producción es la expansión temporal (`TemporalNetOperations`, línea 457), que no se usa con redes de simulación de eventos discretos, así que este camino no se alcanza hoy.

`sample()`, `getInducedFindings` y `toString` no tienen consecuencia medida; se listan porque son del mismo tipo y el arreglo es el mismo.

Otro hallazgo sobre estos árboles, que muestrearlos vacía la configuración de padres que reciben, está en nota 74; allí no hay nada que corregir.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:207-210

```
@Override public Potential deepCopy(ProbNet copyNet) {
    //TODO
    return null;
}
```

gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:145-151

```
Potential lastPotential = this.node.getPotential();
if (lastPotential != null) {
    this.originalPotential = this.node.getPotential().deepCopy(this.node.getProbNet());
} else {
    this.originalPotential = null;
}
```

**Cómo arreglarlo.** Quitar los `deepCopy` que devuelven `null` (la base ya copia por el constructor de copia) o implementarlos copiando también los árboles internos con las variables de la red destino, como hace `TreeADDPotential.deepCopy`; `TreeWithEventsPotential` debe copiar su mapa de árboles por suceso y `TreeWithExcludedEventsPotential` su `noEventTree`. Implementar `toString` con algo que identifique la relación; `sample()` y `getInducedFindings` deberían lanzar `UnsupportedOperationException` como la base en vez de devolver `null`.

Aparte, `SetPotentialEdit` con relación anterior `null` no debería poder dejar un `null` en la lista del nodo; eso protege a cualquier otra relación con el mismo fallo.

Prueba: las pruebas de copia (`copy`/`deepCopy`) del proyecto deberían cubrir estas dos clases: copiar una red de simulación de eventos discretos con cada una y comprobar que la relación copiada no es nula y usa las variables de la copia.

### Error 23 · [Corregido] El editor ofrece «Tree with Excluded Events» a un nodo cuyos padres son todos sucesos, y elegirlo lanza un error no previsto

| | |
|---|---|
| **Estado** | Corregido en el commit `4bec161` (15-10-2026): `TreeWithExcludedEvents Potential.validate` exige además algún padre que no sea suceso, así que el selector ya no la ofrece a un nodo cuyos padres son todos sucesos. Con prueba en `core`. Quitar después el último padre que no es suceso no deja un árbol imposible, porque la relación pasa a uniforme (error 99). Un `.pgmx` escrito a mano con un árbol así sigue sin abrirse, con el mensaje genérico de fichero dañado; OpenMarkov no escribe ficheros así. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.treeadd |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:67-81,102-111`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDPotential.java:189-191`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:90-109,118-135`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:488-522,675-683` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 22, error 101, error 105, error 162, nota 96 |

**Qué pasa.** En una red de simulación de eventos discretos, el selector de clase de relación ofrece «Tree with Excluded Events» a un nodo cuyos padres son todos sucesos, y elegirlo termina en un error no previsto. La relación del nodo no llega a cambiar, pero el selector queda mostrando la clase elegida.

Camino del usuario (la parte de la interfaz está comprobada leyendo): propiedades del nodo → pestaña de relación → selector de clase → «Tree with Excluded Events». `PotentialEditPanel.potentialTypeChanged` (líneas 476-507) llama a `instanciatePotential` (líneas 663-671)

→ `PotentialUtils.instanciateSafely` (líneas 93-112), que recibe la excepción envuelta en `InvocationTargetException`, prueba un constructor de un argumento que no existe y termina lanzando `UnreachableException`. El manejador global la enseña como error no previsto.

Por qué pasa: `TreeWithExcludedEventsPotential` es un árbol de relaciones que ramifica solo sobre los padres que **no** son sucesos. Su `validate` (líneas 102-111), que decide si el selector la ofrece, exige que la red sea de simulación de eventos discretos y que **algún** padre sea un suceso, pero no que haya algún padre que no lo sea. Su constructor (líneas 67-81) arma el árbol con la variable hija más los padres que no son sucesos y llama a `new TreeADDPotential(variables, role)`, que ramifica sobre `variables.get(1)` (`TreeADDPotential`, líneas 189-191). Si todos los padres son sucesos, la lista tiene un solo elemento y el constructor lanza `IndexOutOfBoundsException`.

La hermana `TreeWithEventsPotential` no tiene este problema: arma un árbol por suceso con la hija y ese suceso, así que siempre tiene dos variables.

Medido (red de tipo `DESNetworkType`, nodo de azar Z con dos padres suceso E y E2 y nada más):

```
validate = true
new TreeWithExcludedEventsPotential([Z, E, E2]) -> IndexOutOfBoundsException: Index 1 out
↪  of bounds for length 1
tipos ofrecidos para Z: [TransitionTablePotential, ExternalPotential, DeltaPotential,
↪  WeibullHazardPotential,
                         TreeWithExcludedEventsPotential, TreeWithEventsPotential,
                         ↪  TreeADDPotential, ICIPotential]
```

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:102-111

```java
public static boolean validate(Node node, List<Variable> variables, PotentialRole role) {
    if (!(node.getProbNet().getNetworkType() instanceof DESNetworkType)) return false;
    ...
    boolean validate = variables.subList(1, variables.size())
                                .stream()
                                .anyMatch(variable -> variable.getVariableType() ==
                                ↪  VariableType.EVENT);
    return validate;
}
```

**Cómo arreglarlo.** Añadir a `validate` la condición de que algún padre no sea suceso (`anyMatch( ... != VariableType.EVENT)`), para que el selector no la ofrezca donde no se puede construir.

Como la relación también puede venir de un fichero (`PGMXReader_1_0.getTreeWithExcluded EventsPotential`) y los padres pueden cambiar después (quitar el último padre que no es suceso), el constructor debería rechazar ese caso con una excepción con mensaje en vez de un índice fuera de rango; y quitar un enlace que deje la relación inválida debería tratarse igual que en las demás relaciones con requisitos sobre sus padres.

De paso, `instanciateSafely` convierte cualquier excepción del constructor en `Unreachable Exception`; un constructor que rechaza sus argumentos no es un caso inalcanzable, así que conviene dejar pasar la causa.

Prueba: para un nodo con solo padres suceso, `validate` devuelve `false` y el nodo no la ofrece; para uno con un suceso y un padre de azar, se construye sin error.

### Error 193 · [Corregido] «Absorb parents» sobre un nodo de suma, de producto o de fórmula sale como error inesperado cuando un padre está definido por una regresión, porque la tabla de la regresión trae la variable del propio padre

| | |
|---|---|
| **Estado** | Corregido en el commit `799a16e` (29-09-2026): `BasicOperations.tableOfAParentToAbsorb` deja fuera de la tabla de cada padre la variable del propio padre, y lo usan el menú (`AbsorbParentsEdit`) y el preproceso de la inferencia (`BasicOperations.absorbParents`). El menú absorbe ya un padre definido por una regresión, y la evaluación deja de contar su valor dos veces, consecuencia que esta ficha no recogía: con un coste constante de −4 en «Cost: Second Test», padre de «Total» en `ID-used-car-buyer.pgmx`, la utilidad esperada era 24,96 en vez de 28,96. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core, org.openmarkov.core.inference |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/AbsorbParentsEdit.java:33-68`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:43-76,201-238`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/LinearCombinationPotential.java:94-95`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExponentialPotential.java:88-89` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 1, error 8, error 9, nota 185 |

**Qué pasa.** Sobre un nodo numérico cuya relación es una suma, un producto o una fórmula, «Absorb parents» sale como error inesperado cuando alguno de sus padres está definido por una regresión (combinación lineal o exponencial). El menú está encendido, porque `BasicOperations.haveParentsAndAreAllAbsorbable` solo mira que el nodo y sus padres sean numéricos y que los abuelos tengan estados finitos.

Camino del usuario (la parte de la interfaz está comprobada leyendo): en el editor, un nodo numérico con relación de suma, de producto o de fórmula que tiene como padre un nodo numérico con relación de regresión; menú contextual del nodo → «Absorb parents». `GUIUtils.executeUIAction` envuelve el fallo en `UnrecoverableException`, y el manejador global lo enseña como error inesperado. En el repositorio no hay ninguna red así: las únicas regresiones de nodos numéricos, las de `integrationTests/src/main/resources/networks_for_every_potential/Dynamic-LIMID-every-potential.pgmx`, no tienen hijos.

Por qué pasa: la tabla de una regresión lleva delante la variable del propio nodo (`LinearCombinationPotential.tableProject`, línea 95; `ExponentialPotential.tableProject`, línea 89), una variable numérica de un solo estado. La absorción combina las tablas de los padres (`BasicOperations.absorbParentPotentials`) y convierte cada variable del resultado en un padre del nodo, así que el padre absorbido vuelve a aparecer como padre. `AbsorbParentsEdit.generateEdits` pide a la vez borrar ese padre y añadir el enlace desde él, y la comprobación de restricciones rechaza la edición porque ese enlace ya existe (`ConstraintViolatedException.LinkAlreadyExists`). El preproceso de la inferencia (`BasicOperations.absorbParents`) no falla, pero vuelve a enlazar el padre y, como ahora tiene un hijo, no lo borra.

Medido con `Dynamic-LIMID-every-potential.pgmx`, enlazando `LinearCombination [0]` y `Exponential [0]`, dos regresiones sin padres, con `Sum [0]` o con `Product [0]`:

| | menú «Absorb parents» | preproceso de la inferencia |
|---|---|---|
| suma, un padre de regresión | falla | el padre sigue en la red |
| suma, dos padres de regresión | falla | los padres siguen en la red |
| producto, un padre | falla | el padre sigue en la red |
| producto, dos padres | funciona | funciona |

El menú falla con `CannotDoEditException`, causada por `LinkAlreadyExists` (con dos padres, por `MultipleConstraintsViolateds`).

El producto de dos padres funciona solo porque la multiplicación deja fuera las variables de un solo estado de las tablas de una casilla (nota 185). Con regresiones que tengan padres propios, sus tablas tienen más de una casilla, el producto conserva la variable y falla también; esto último se ha deducido leyendo.

**El código.** core/src/main/java/org/openmarkov/core/action/core/AbsorbParentsEdit.java:49-65

```
        for (Node parent : parents) {
            PNEdit newEdit;
            if (parent.getChildren().size() > 1) {
                newEdit = new RemoveLinkEdit(probNet, parent.getVariable(), nodeVariable,
↪   true, false);
            } else {  // parent.getChildren().size() == 1
                newEdit = new CRemoveNodeEdit(probNet, parent);
            }
            edits.add(newEdit);
        }

        for (Variable variable : potential.getVariables()) {
            if (variable != nodeVariable) {
                AddLinkEdit addLinkEdit = new AddLinkEdit(probNet, variable, nodeVariable,
↪   true);
                ...
                edits.add(addLinkEdit);
            }
        }
```

**Cómo arreglarlo.** Al absorber, quitar de la tabla de cada padre la variable del propio padre antes de combinarlas: tiene un solo estado, así que ningún valor cambia (`TablePotential.removeVariable` la proyecta en su único estado). La regla tiene que cumplirse en los dos sitios que arman esas tablas, el menú (`AbsorbParentsEdit.generateEdits`, líneas 38-45) y el preproceso de la inferencia (`BasicOperations.absorbParents`, líneas 205-212); lo natural es que los dos llamen al mismo método.

La otra salida, que la tabla de una regresión no lleve la variable del nodo, cambia lo que reciben todos los que proyectan una regresión, la inferencia incluida.

Prueba de regresión: en `Dynamic-LIMID-every-potential.pgmx`, con `LinearCombination [0]` enlazado a `Sum [0]`, «Absorb parents» debe terminar sin error, borrar `LinearCombination [0]`, dejar `Sum [0]` sin padres y darle el valor de la regresión; lo mismo con dos padres, con un producto y con el preproceso de la inferencia.

### Error 234 · [Corregido] Un nodo pegado junto con un padre que su fórmula nombra no se puede calcular en la misma red, y en otra red sigue escrito sobre los nodos de la red de origen

| | |
|---|---|
| **Estado** | Corregido en el commit `0883462` (13-10-2026). `VariableExpression.replacing` devuelve la fórmula escrita sobre la variable nueva, tanto si guardaba la variable como si solo llevaba su nombre entre llaves, y `replaceVariable` la aplica en `GLMPotential` (y con él las fórmulas y las regresiones), `DistributionTablePotential` (sobre el texto de su `TableWithFunctions`, también con el nombre sin llaves), `AugmentedProbTablePotential`, `UnivariateDistrPotential` y `PiecewiseExponentialPotential`; los tres últimos pasan además a la variable nueva su tabla interna y sus listas de padres. Con pruebas: `FormulasFollowAReplacedVariableTest` (cada relación, pegada con nombre nuevo y con el mismo nombre) y `PastedFormulasNameThePastedParentsTest` (una utilidad con la fórmula `{Age}*3` pegada con `PasteEdit` junto con `Age` en la misma red y en otra). De la tabla de fórmulas y de la distribución con fórmulas se comprueba lo que guardan por dentro, no un cálculo. No se ha comprobado en la ventana. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.expression; org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/expression/VariableExpression.java`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithFunctions.java`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/AugmentedProbTablePotential.java`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/PiecewiseExponentialPotential.java`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:71-78, 117-131` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 21, error 13 |

**Qué pasa.** Visto al corregir el error 21; describe el código anterior al arreglo. Una fórmula guarda las variables sobre las que está escrita. Al pegar un nodo junto con un padre que su fórmula nombra, `PasteEdit` cambia las variables de la relación por las de los nodos pegados, pero la fórmula sigue apuntando al padre original. No es cosa solo de las redes de sucesos: pasa en cualquier tipo de red.

- Pegando en la misma red, el padre pegado recibe un nombre nuevo con apóstrofo (`Age'`) y la fórmula sigue diciendo `Age`: calcular el nodo pegado falla.
- Pegando en otra red se calcula bien, pero solo porque el nombre coincide: la fórmula sigue escrita sobre el nodo de la red de origen.
- Una distribución o una tabla de fórmulas pegadas conservan además su tabla interna sobre los padres originales, como las relaciones de sucesos del error 21.

Medido sobre cada relación, haciendo lo que hace `PasteEdit` (`copy()` y `replaceVariable` de todas

las variables), con `Age`=40:

- `FunctionPotential` con `{Age}*3`: el original da 120; la copia con `Age'` lanza `CannotResolveVariable` («no value resolved for variable Age»). Lo mismo una `LinearCombinationPotential` y la función de tiempo inicial de una `PiecewiseExponentialPotential`.
- `DistributionTablePotential` «Exact» con las fórmulas `{Age}*2` y `Age*3`: el original da 120 y la copia lanza `UnrecoverableException` («Cannot evaluate … Expression is invalid»).
- `UnivariateDistrPotential` «Exact» con los valores 100 y 2000 según `Sex`: la copia se proyecta como una tabla de dos valores sobre el `Sex` original, en vez de dar 2000 con el `Sex` pegado en «female».
- `AugmentedProbTablePotential` y `UnivariateDistrPotential` con fórmulas: la tabla interna y las fórmulas de la copia siguen sobre las variables originales (visto en sus campos, no calculando).

Con `PasteEdit`: una utilidad `Cost` con la fórmula `{Age}*3`, pegada junto con `Age` en su misma red, queda con la fórmula `{Age}*3` en vez de `{Age'}*3`.

No se ha mirado qué pasa al cambiar el nombre de un nodo desde su diálogo, que es otro camino.

**Cómo arreglarlo.** Hecho como dice la fila *Estado*: cada relación que guarda fórmulas las reescribe en `replaceVariable`, con un método común en `VariableExpression`.

### Error 257 · [Corregido] Al pegar un nodo cuya relación es un riesgo de Weibull junto con su variable de tiempo, la relación pegada sigue contando el tiempo con la variable del original: no se puede calcular, o da una tabla sobre una variable que no es de su familia

| | |
|---|---|
| **Estado** | Corregido en el commit `3228141` (15-10-2026): `WeibullHazardPotential.replaceVariable` cambia también `timeVariable` cuando es la variable sustituida, como ya hacían `shift`, `replaceNumericVariable` y `deepCopy`. Repasadas las demás relaciones que guardan una variable aparte: los árboles y las tablas con sucesos ya la cambian, y las distribuciones, los modelos canónicos y el máximo y el mínimo guardan variables internas propias |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:88-92,136-180,325-330,337-342,368-379`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:347-357`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:114-139`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:988-995` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición de pegar y la proyección, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 21, error 234, error 71, error 258 |

**Qué pasa.** Un riesgo de Weibull (`WeibullHazardPotential`, y con él `ExponentialHazardPotential`, que hereda de ella) da la probabilidad de que algo pase en un ciclo según el tiempo transcurrido. Ese tiempo lo dice uno de los padres del nodo, que la relación guarda además aparte, en el campo `timeVariable`; es la lista «Time variable» de su panel. Al pegar el nodo junto con ese padre, la relación pegada sigue apuntando a la variable de tiempo del nodo original, aunque su lista de variables ya nombre la pegada. Lo que pasa depende de cómo sea la variable de tiempo:

- si es numérica, la relación pegada pide un hallazgo de la variable del original, que no tiene, y no se puede calcular (`MissingEvidenceInVariable`);
- si tiene estados, la tabla que sale lleva una variable de más, la del original, que no es padre del nodo pegado.

Medido el 15-10-2026 sin ventana, con `PasteEdit` sobre una red pequeña: X, de dos estados, con un riesgo de Weibull sobre X y su padre T, que es la variable de tiempo; se pegan X y T juntos, en la misma red y en otra, y se proyecta la relación pegada (con `T' = 2` cuando `T` es numérica):

```
numeric T, same net | pasted relation over [X', T'] | time variable is the pasted one:
↪  false, the original: true
numeric T, same net | original projected | table over [X]
numeric T, same net | pasted projected   | THROWS
↪  ...NonProjectablePotentialException$MissingEvidenceInVariable
numeric T, other net | pasted projected   | THROWS
↪  ...NonProjectablePotentialException$MissingEvidenceInVariable
finite T, same net | original projected | table over [X, T]
finite T, same net | pasted projected   | table over [X', T', T]
finite T, other net | pasted projected   | table over [X, T, T]
```

Pegando en otra red, la T de la última línea es la de la red de origen. En la red de prueba las variables no tienen ciclo; en un modelo de Markov, que es donde el panel ofrece variables de tiempo (solo las temporales), el camino es copiar el nodo y su variable de tiempo y pegarlos en otro modelo.

Leído y no medido: al guardar, el escritor del PGMX (formato propio de OpenMarkov) escribe la variable de tiempo por su nombre base y su ciclo (`PGMXWriter_0_2`, líneas 988-995), y pegando en otra red ese nombre es también el de la variable pegada; así que la red guardada y vuelta a abrir ya está bien, pero la que el usuario tiene abierta, y con la que calcula, no.

Por qué pasa: `PasteEdit` (líneas 114-139) copia la relación con `copy()` y cambia una a una sus variables por las de los nodos pegados con `replaceVariable`. Desde `0883462` (error 234), `GLMPotential.replaceVariable` (líneas 347-357) reescribe también las fórmulas de las covariables, pero nadie cambia `timeVariable`, que el constructor de copia (líneas 88-92) toma del original. Las otras operaciones que mueven las variables de esta relación sí lo cambian: `shift` (líneas 325-330), `replaceNumericVariable` (líneas 337-342) y `deepCopy` (líneas 368-379). Al proyectar, `tableProject` quita de las variables sin hallazgo la de tiempo que guarda (línea 145), y como esa ya no está en la lista, la pegada se queda; luego pide el hallazgo de la guardada (líneas 172-177) o, si tiene estados, la añade a la tabla (líneas 154-157).

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:347-357

```java
@Override public void replaceVariable(int position, Variable variable) {
    Variable oldVariable = variables.get(position);
    super.replaceVariable(position, variable);
    if (covariates != null) {
        VariableExpression[] replaced = new VariableExpression[covariates.length];
        for (int i = 0; i < covariates.length; i++) {
            replaced[i] = covariates[i].replacing(oldVariable, variable);
        }
        covariates = replaced;
    }
}
```

**Cómo arreglarlo.** Redefinir `replaceVariable` en `WeibullHazardPotential` para que, si la variable sustituida es la de tiempo, `timeVariable` pase a ser la nueva, como ya hace `replaceNumericVariable`. Es la misma regla que `PasteEdit` aplica a mano a la variable hija de un `ExactDistr`

`Potential` (líneas 132-138); mejor dentro de la relación, que es quien sabe qué guarda aparte. Conviene repasar a la vez las demás relaciones que guardan una variable fuera de su lista.

Prueba: pegar con `PasteEdit` un nodo con un riesgo de Weibull junto con su variable de tiempo, en la misma red y en otra, y comprobar que `getTimeVariable()` de la relación pegada es la variable pegada y que, con un hallazgo en ella, la proyección es una tabla sobre el nodo pegado.

### Error 260 · [Corregido] Al pegar un nodo, o al deshacer un cambio del número de estados de uno de sus padres, el comentario de su relación se pierde si es un árbol con sucesos, una tabla de vida, una distribución por tabla o una tabla de fórmulas, porque su copia no se lo lleva

| | |
|---|---|
| **Estado** | Corregido en el commit `52033cb` (15-10-2026): los constructores de copia de las cinco clases llaman a `copyAttributesFrom`, y el `deepCopy` de los dos árboles con sucesos también, en vez de poner solo el comentario. Revisadas todas las clases de relación: solo estas cinco tenían el defecto; `CycleLengthShift` conserva el comentario y no se ha tocado. Con prueba en `core`: la copia y la copia profunda de las cinco conservan comentario, criterio y propiedades, y el hijo conserva el comentario tras dar a un padre un estado más y deshacerlo. Pegar un nodo usa esa misma copia. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.core.model.network.potential.treeadd |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:123-139`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithEventsPotential.java:89-97,253-261`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:86-92,199-204`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java:104-112`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/PiecewiseExponentialPotential.java:172-181`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/AugmentedProbTablePotential.java:64-68`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:118`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeReplaceStatesEdit.java:51-53` |
| **Comprobado** | Leyendo el código y ejecutándolo (las copias, el pegado y el cambio de estados, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 22, error 24, error 243 |

**Qué pasa.** Toda relación lleva, además de lo suyo, tres cosas que guarda `Potential`: el comentario, el criterio y unas propiedades sueltas. El comentario es el que se escribe al pie del diálogo de la relación y el que se guarda con ella en el fichero. Al menos dos ediciones copian la relación de un nodo con `copy()`, y en cinco clases esa copia no se lleva ninguna de las tres cosas, así que el comentario se pierde sin aviso:

- **pegar un nodo**: `PasteEdit` (línea 118) le pone al nodo pegado un `copy()` de la relación del original;
- **cambiar el número de estados de un nodo, o de un padre, y deshacerlo**: desde `461cd46`, `NodeReplaceStatesEdit` guarda un `copy()` de la relación del nodo y de la de cada

hijo (líneas 51-53), y al deshacer se la devuelve.

Las cinco clases son los dos árboles con sucesos de las redes de simulación de sucesos discretos (DES), «Tree with Events» y «Tree with Excluded Events»; la distribución por tabla (`DistributionTablePotential`, la que da en esas redes el tiempo hasta un suceso); la tabla de vida (`PiecewiseExponentialPotential`); y la tabla de fórmulas (`AugmentedProbTablePotential`). El fichero que se guarde después ya no lleva el comentario, porque el escritor del PGMX (el formato propio de OpenMarkov) escribe el de la relación que el nodo tiene.

Medido el 15-10-2026 sin ventana, con las tres cosas puestas en la relación de un suceso «Dies» que depende de los sucesos E1 y E2 y de «Sex», en una red DES: la copia, la copia profunda, el nodo pegado con sus padres en la misma red con `PasteEdit`, y el hijo después de darle a «Sex» un estado más y deshacerlo:

```
events   original       TreeWithEventsPotential comment «a comment» criterion cost
↪  properties {key=value}
events   copy           TreeWithEventsPotential comment «» criterion null properties {}
events   deepCopy       TreeWithEventsPotential comment «a comment» criterion null
↪  properties {}
events   pasted         TreeWithEventsPotential comment «» criterion null properties {}
events   undone         TreeWithEventsPotential comment «» criterion null properties {}
excluded pasted         TreeWithExcludedEventsPotential comment «» criterion null
↪  properties {}
excluded undone         TreeWithExcludedEventsPotential comment «» criterion null
↪  properties {}
```

y la copia de las otras tres clases, frente a una tabla:

```
copy of AugmentedProbTablePotential comment «» criterion null properties {}
copy of DistributionTablePotential comment «» criterion null properties {}
copy of PiecewiseExponentialPotential comment «» criterion null properties {}
copy of TablePotential comment «a comment» criterion cost properties {key=value}
```

El criterio solo lo usa la inferencia, y las propiedades solo las llena el lector de Elvira, que no tiene sucesos, así que esas dos pérdidas no se notan; el comentario, sí. El editor de la relación usa `deepCopy`, y en los árboles con sucesos el que escribió `0f217e1` (error 22) le vuelve a poner el comentario, así que abrir y cerrar el editor no lo pierde.

Por qué pasa: el constructor de copia de `Potential` (líneas 123-139) copia las tres cosas con `copyAttributesFrom`, y cada clase lo llama con `super(potential)` desde su propio constructor de copia, como `TablePotential` o `TreeADDPotential`. Los de estas cinco empiezan en cambio con `this(variables, role, …)`, que hace una relación nueva, y luego copian lo suyo (los árboles, la tabla de sucesos, la tabla de tramos, la tabla interna). El `deepCopy` de los dos árboles con sucesos construye también una relación nueva y le pone el comentario a mano, pero no el criterio ni las propiedades.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithEventsPotential.java:89-97

```java
public TreeWithEventsPotential(TreeWithEventsPotential treeWithEvents) {
    this(treeWithEvents.getVariables(), treeWithEvents.role);
    HashMap<Variable, TreeADDPotential> treesToCopy = treeWithEvents.getTrees();
    trees.clear();
    for (Variable event : treeWithEvents.getEvents()) {
        trees.put(event, (TreeADDPotential) treesToCopy.get(event).copy());
    }

}
```

**Cómo arreglarlo.** Que los cinco constructores de copia llamen a `copyAttributesFrom(original)` después de construirse (no pueden empezar con `super(original)` porque delegan en otro constructor de la clase), y que el `deepCopy` de los dos árboles con sucesos haga lo mismo en vez de poner solo el comentario. Es la regla que ya cumplen las demás relaciones: una copia se lleva el comentario, el criterio y las propiedades.

Prueba: para cada una de las cinco clases, poner las tres cosas, y comprobar que `copy()` y `deepCopy()` las conservan; y, con `PasteEdit` y con `NodeReplaceStatesEdit` deshecho, que el nodo pegado y el hijo conservan el comentario de su relación.

### Error 278 · La tabla de un riesgo de Weibull sale con probabilidades que no lo son, sin ningún aviso, si su tasa es negativa (lo que pasa sin el logaritmo, que es como nace la relación) o si el tiempo está entre 0 y 1

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:49,136-222` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | Sí: qué hacer con una tasa negativa (rechazarla, o exigir el logaritmo) y con un tiempo entre 0 y 1 (rechazarlo, o tratarlo como el primer ciclo) |
| **Relacionados** | error 257 |

**Qué pasa.** `WeibullHazardPotential.tableProject` (líneas 136-222) calcula la probabilidad de que el suceso pase en el ciclo que acaba en el instante t como $p = 1 - \exp(\lambda \cdot ((t-1)^k - t^k))$, donde k, la forma, es la exponencial del coeficiente de `Gamma`, y $\lambda$, la tasa, es la suma de las covariables por sus coeficientes, o la exponencial de esa suma si la relación está «en logaritmo» (el campo `log`). Nada comprueba que p quede entre 0 y 1, y hay dos maneras sencillas de que no quede:

- **Una tasa negativa.** Sin logaritmo, $\lambda$ es la suma tal cual, y nada impide que sea negativa. Como $(t-1)^k - t^k$ es negativo, el exponente sale positivo, p queda por debajo de cero y su complemento por encima de uno. Y sin logaritmo es como nace la relación: el campo `log` vale `false` en la clase (línea 49); el lector del formato propio, en cambio, lo pone a `true` cuando el fichero no dice nada.
- **Un tiempo entre 0 y 1.** $t - 1$ es negativo, y elevarlo a una forma que no es un número entero no da un número: p sale NaN, con logaritmo o sin él. Solo está previsto $t \leq 0$, que da $p = 0$.

Medido el 15-10-2026 con la clase de `3228141` delante del jar del 14 de octubre, con un riesgo sobre una edad numérica, $\ln k = 0{,}3740968$ y término constante $-5{,}490935$ (los números son $[1 - p, p]$):

```
sin logaritmo, edad 65     [8.991159255698918E22, -8.991159255698918E22]
sin logaritmo, edad 2      [14031.13472108565, -14030.13472108565]
sin logaritmo, edad 0,5    [NaN, NaN]
sin logaritmo, edad 0      [1.0, 0.0]
con logaritmo, edad 65     [0.9610820240119792, 0.038917975988020825]
con logaritmo, edad 0,5    [NaN, NaN]
```

La tabla entra en la propagación como cualquier otra. La propia prueba del arreglo del error 257 lo hace sin darse cuenta: toma los coeficientes del riesgo de revisión de una prótesis, que son los de

un modelo cuyo término lineal es el logaritmo de la tasa, sin poner `log`, y las tablas que compara valen $\pm 8{,}4 \cdot 10^{32}$. La prueba sigue sirviendo, porque compara la relación pegada con la original, pero lo que compara no son probabilidades.

**Cómo arreglarlo.** Hace falta decidir qué es una tasa negativa: si la relación sin logaritmo debe rechazarla al calcular (con un mensaje que nombre el nodo y la configuración de los padres), o si el logaritmo debe ser lo único admitido, como ya supone el lector cuando el fichero no dice nada. Y qué es un tiempo entre 0 y 1: rechazarlo, o tratarlo como el primer ciclo ($p = 1 - \exp(-\lambda \cdot t\hat{}k)$). En los dos casos, comprobar al final que p está entre 0 y 1 y negarse con un mensaje si no.

Prueba: un riesgo sin logaritmo con una tasa negativa y otro con un tiempo de 0,5: ninguno da una tabla con números fuera de [0, 1] ni NaN.

# 4 Ediciones de la red: deshacer, cancelar y pegar

Cada cambio que el usuario hace sobre una red es una edición que se apunta en el historial y se puede deshacer. Aquí van los cambios que no se deshacen bien, los que dejan la red a medias y los que se rechazan sin motivo.

### Error 24 · [Corregido en parte] [Jorge Rico] Elegir un dominio estándar con otro número de estados borra las tablas del nodo y de sus hijos, y ni «Cancel» ni «deshacer» las devuelven

| | |
|---|---|
| **Estado** | Presente en parte. El commit `461cd46` (12-10-2026), de Jorge Rico, hace que la edición guarde las tablas del nodo y de sus hijos, y las restricciones y condiciones de revelación de sus enlaces, y que las devuelva al deshacer o cancelar. Siguen sin volver los intervalos de una variable discretizada; el apartado *Lo que queda* da un ejemplo. |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/NodeReplaceStatesEdit.java`<br>`core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:69-110,292-320`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:167-210`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:1078-1117`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:406-409` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 25, error 30, error 173, nota 5, nota 15, nota 150 |

**Lo que queda.** Ejemplo: una variable discretizada «Temperature» con dos estados, «normal» de 0 a 37,5 y «fever» de 37,5 a 42. El usuario abre el diálogo del nodo, prueba el dominio estándar `low - medium - high` y pulsa «Cancel». Vuelven los nombres «normal» y «fever», y vuelven las tablas del nodo y de sus hijos; pero los intervalos ya no son los suyos: «normal» va ahora de $-\infty$ a 0 y «fever» de 0 a $\infty$. Nada avisa, y a partir de ahí una temperatura de 36 cae en «fever». Lo mismo pasa al aceptar y deshacer.

Tampoco hace falta cambiar el número de estados: elegir un dominio estándar de dos estados (`no`

- `yes`) deja los mismos intervalos de $-\infty$ a 0 y de 0 a $\infty$, y deshacer no devuelve los de antes.

Por qué pasa: `VariableStateOperations.replaceStates` pone siempre los intervalos por defecto en una variable discretizada, y `NodeReplaceStatesEdit` guarda las tablas y los enlaces, pero no los intervalos; además su `undo()` vuelve a llamar a `replaceStates`, que los pone otra vez por defecto. Falta guardar el intervalo partido de la variable (`getPartitionedInterval`) antes de la edición y reponerlo al deshacer.

Medido el 13-10-2026 ejecutando la edición, no en la ventana, sobre `BN-one-disease.pgmx` con «Disease» pasada a discretizada con los límites 0, 37,5 y 42: tras la edición y deshacer, los límites son $-\infty$, 0 e $\infty$, tanto con el dominio de tres estados como con el de dos. En la misma medida, «Disease» y «Symptom» sí recuperan sus tablas tras «Cancel» y tras deshacer, rehacer y deshacer.

**Qué pasa.** El resto de la ficha describe el código anterior a `461cd46`. Abrir el diálogo de un nodo, probar en la pestaña de dominio un dominio estándar («Standard domains») con otro número de estados y cancelar borra sin aviso la tabla del nodo y la de sus hijos, que el usuario cree intactas porque ha cancelado. Lo mismo pasa al aceptar y deshacer con el menú de edición.

Por qué pasa: el botón «Standard domains» sustituye los estados del nodo (y de sus gemelos en otros ciclos) mediante `NodeReplaceStatesEdit`. Su `doEdit()` llama a `Variable.replaceStates` → `VariableStateOperations.replaceStates`, que pone los estados nuevos y, si el número de estados cambia: sustituye el potencial del nodo por un `UniformPotential`, sustituye el primer potencial de cada hijo por otro `UniformPotential`, y `resetLink` quita las restricciones de los enlaces del nodo y las condiciones de revelación de sus enlaces a hijos. Además, si la variable es discretizada, pone siempre intervalos por defecto, aunque el número de estados no cambie.

La edición solo guarda `lastStates`. Su `undo()` vuelve a llamar a `replaceStates(node, lastStates)`, que devuelve los nombres pero, como el número de estados vuelve a cambiar, deja otra vez potenciales uniformes; nada de lo borrado vuelve. En cambio, `NodeStateEdit` (añadir, quitar o mover un estado) sí fotografía los potenciales, los intervalos y las restricciones, y los restaura en su `undo()`, aunque su fotografía de potenciales recoge también las tablas de los hijos (error 30).

«Cancel» llega a este `undo()` porque el diálogo del nodo deshace su sub-historia al cancelar.

Medido sobre `0_miscellaneous/networks/bn/BN-one-disease.pgmx`, reproduciendo la secuencia del diálogo: antes, «Disease» tiene `[0.97, 0.03]` y «Symptom» `[0.98, 0.02, 0.05, 0.95]`; tras elegir `low - medium - high` los dos pasan a `UniformPotential`; tras «Cancel» (`cancelLastSubEditHistory`) «Disease» vuelve a `absent, present` pero los dos siguen con `UniformPotential`. Lo mismo tras aceptar y deshacer con el menú de edición.

**El código.** core/src/main/java/org/openmarkov/core/action/core/NodeReplaceStatesEdit.java

```java
public NodeReplaceStatesEdit(Node node, State[] newStates) {
    super(node.getProbNet());
    this.node = node;
    this.lastStates = node.getVariable().getStates();
    this.newStates = newStates;
}

@Override protected void doEdit() {
    node.getVariable().replaceStates(node,newStates);
}

@Override public void undo() {
    super.undo();
    node.getVariable().replaceStates(node,lastStates);
}
```

core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:76-101

```
node.getVariable().setStates(newStates);
if (newStates.length != lastStates.length) {
    if (!lastPotential.isEmpty()) {
        UniformPotential newPotential = new UniformPotential(
                lastPotential.getFirst().getVariables(),
                lastPotential.getFirst().getPotentialRole());
        ...
        node.setPotentials(newPotentials);
    }
    nodes = probNet.getChildren(node);
    for (Node child : nodes) {
        ...
            child.setPotentials(container);
    }
    resetLink(node);
}
```

**Cómo arreglarlo.** Que `NodeReplaceStatesEdit` fotografíe en su constructor (o antes de `doEdit`) lo mismo que `NodeStateEdit`: la lista de potenciales del nodo y de cada hijo, el intervalo partido si la variable es discretizada, los potenciales de restricción de sus enlaces y las condiciones de revelación de los enlaces a sus hijos. Y que `undo()` ponga los estados antiguos directamente con `setStates` y restaure todo eso, sin volver a llamar a `replaceStates`. Lo más limpio es compartir la fotografía y la restauración con `NodeStateEdit` en vez de escribirla dos veces, una vez corregida su fotografía de potenciales (error 30).

El mismo código toca el `resetLink` de `NodeStateEdit` y la restauración de las restricciones de enlace. Los dos mapas sin uso de `VariableStateOperations.resetLink` están descritos en nota 5; allí no hay nada que corregir.

Prueba de regresión: sobre `0_miscellaneous/networks/bn/BN-one-disease.pgmx`, ejecutar `NodeReplaceStatesEdit` con un dominio de tres estados dentro de una sub-historia, cancelarla y exigir que los potenciales de «Disease» y «Symptom» sean los originales; repetir con un enlace con restricción y con una variable discretizada.

### Error 25 · [Jorge Rico] Marcar una casilla como incompatible en la restricción de un enlace cambia la tabla del hijo fuera del historial, y ni «Cancel», ni deshacer, ni volver a pulsar la casilla la devuelven

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.component; org.openmarkov.gui.action; org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/component/LinkRestrictionValuesTable.java:54-80`<br>`gui/src/main/java/org/openmarkov/gui/action/LinkRestrictionPotentialValueEdit.java:73-115`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/LinkRestrictionPotentialOperations.java:163-200,218-253`<br>`gui/src/main/java/org/openmarkov/gui/dialog/link/LinkRestrictionPanel.java:140-151`<br>`gui/src/main/java/org/openmarkov/gui/dialog/link/LinkRestrictionEditDialog.java:43,99-108`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssis` |

| | |
|---|---|
| | `tant.java:244-256` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 24, error 27, error 154, error 155, nota 1, nota 23 |

**Qué pasa.** En el menú contextual de un enlace, «Add restrictions» (o «Edit restrictions», si ya tiene) abre la ventana «Link Restriction: Link between … and …», con una casilla por cada pareja de estados del padre y del hijo. Pulsar una casilla la marca como incompatible (0) o compatible (1). Al marcarla incompatible, la ventana pone además esa casilla a cero en la tabla de probabilidad del hijo y reparte la probabilidad de la columna entre los estados que siguen permitidos. Ese cambio de la tabla no forma parte de ninguna edición:

- «Cancel» quita la restricción, pero la tabla del hijo se queda con el cero y la columna repartida;
- lo mismo al aceptar con «OK» y deshacer después con el menú de edición;
- volver a pulsar la casilla antes de aceptar la deja compatible, y al aceptar el enlace queda sin restricción, pero la tabla del hijo sigue cambiada.

Nada lo avisa: la red da otras probabilidades y se guarda así. Cada pulsación cambia además la tabla del padre, o lanza una excepción antes de llegar a la del hijo, según la casilla; eso es error 155.

Por qué pasa: la ventana (`LinkRestrictionEditDialog`) abre una sub-historia de ediciones al construirse (línea 43) y la cancela con «Cancel» (líneas 106-108). Cada pulsación llega a `LinkRestrictionValuesTable.setValueAt` (módulo `gui`), que ejecuta un `LinkRestrictionPotentialValueEdit` y, si el valor nuevo es 0, llama a `LinkRestrictionPotentialOperations.updatePotentialByAddLinkRestriction` y pone el resultado en el hijo con `node2.setPotentials` (líneas 66-76). Ese método modifica en su sitio la tabla que ya tiene el hijo: pone el cero y, si el hijo es de azar, llama a `redistributeProbabilities` (líneas 244-249). `LinkRestrictionPotentialValueEdit` solo guarda y restaura la tabla de la restricción del enlace (`lastTable`, `newTable`; `undo`, líneas 107-115), así que ni cancelar la sub-historia ni deshacer tocan la tabla del hijo. Con un hijo de utilidad se pone el cero sin repartir, con el mismo problema (por lectura).

Medido con un pequeño programa de prueba que repite los pasos de la ventana con su panel (`LinkRestrictionPanel`) y su tabla reales, sin construir la ventana, sobre `0_miscellaneous/networks/bn/BN-asia.pgmx` y el enlace VisitToAsia → Tuberculosis (la tabla de Tuberculosis vale `[0.99, 0.01, 0.95, 0.05]`: «no» y «yes» cuando VisitToAsia es «no», y lo mismo cuando es «yes»), marcando la casilla VisitToAsia = «no», Tuberculosis = «yes»:

- tras pulsar la casilla, la tabla de Tuberculosis es `[1.0, 0.0, 0.95, 0.05]`;
- tras «Cancel», la restricción ha desaparecido y la tabla sigue en `[1.0, 0.0, 0.95, 0.05]`; la probabilidad de Tuberculosis = «yes» sin hallazgos pasa de 0,0104 a 0,0005, y la de Dyspnea = «yes», de 0,4360 a 0,4324; no queda nada que deshacer;
- tras «OK» y deshacer, el mismo resultado; guardada la red y reabierta, la tabla sigue en `[1.0, 0.0, 0.95, 0.05]`;
- volviendo a pulsar la casilla y aceptando, el enlace queda sin restricción y la tabla sigue en `[1.0, 0.0, 0.95, 0.05]`.

**El código.** gui/src/main/java/org/openmarkov/gui/component/LinkRestrictionValuesTable.java:66-76

```
if (newNumericValue == 0) {
    if (!node2.getPotentials().isEmpty() && node2.getPotentials().getFirst() instanceof
    ↪  TablePotential) {
        Potential potential = LinkRestrictionPotentialOperations
                .updatePotentialByAddLinkRestriction(node2,
                                                     link.getRestrictionsPotential(),
↪  variable1Index,
                                                     variable2Index);
        ArrayList<Potential> potentials = new ArrayList<>();
        potentials.add(potential);
        node2.setPotentials(potentials);
    }
}
```

gui/src/main/java/org/openmarkov/gui/action/LinkRestrictionPotentialValueEdit.java:107-115

```
@Override public void undo() {
    super.undo();
    if (!link.hasRestrictions()) {
        LinkOperations.initializesRestrictionsPotential(link);
        this.tablePotential = link.getRestrictionsPotential();
    }
    tablePotential.setValues(lastTable);
    checkRestrictionPotential(lastTable);
}
```

**Cómo arreglarlo.** Meter el cambio de la tabla del hijo en la edición: que `LinkRestriction PotentialValueEdit` (o una edición compuesta con ella) guarde una copia de la tabla del hijo antes de cambiarla, aplique `updatePotentialByAddLinkRestriction` sobre la tabla del nodo en su `do Edit`, restaure la copia en `undo` y vuelva a poner la tabla cambiada en `redo`; y que `LinkRestriction ValuesTable.setValueAt` deje de tocar el hijo (en ese mismo método se edita también la tabla del padre: error 155). La copia es necesaria porque `updatePotentialByAddLinkRestriction` modifica la tabla en su sitio.

Con eso, «Cancel» y deshacer devuelven la tabla, y volver a pulsar la casilla dentro de la misma ventana también, si al pulsarla otra vez se deshace la edición que la marcó en lugar de añadir otra. Una casilla que se desmarca en otra visita a la ventana no tiene tabla anterior que devolver; eso no es este fallo.

`updatePotentialByAddLinkRestriction` solo se llama desde aquí. Al tocar esta edición conviene arreglar a la vez error 27, que es la misma clase: al rehacer escribe en un objeto enlace que puede no ser ya el de la red.

Prueba de regresión: sobre `0_miscellaneous/networks/bn/BN-asia.pgmx`, abrir una sub-historia, marcar como incompatible (VisitToAsia = «no», Tuberculosis = «yes») con la edición y cancelar la sub-historia: la tabla de Tuberculosis debe volver a `[0.99, 0.01, 0.95, 0.05]`; lo mismo cerrando la sub-historia y deshaciendo; y rehacer debe dejarla en `[1.0, 0.0, 0.95, 0.05]`.

### Error 26 · [Corregido] Trazar un enlace a un nodo de utilidad que multiplica a sus padres lo convierte en uno que suma, y el modelo cambia de resultado

| | |
|---|---|
| **Estado** | Corregido en el commit `8c39518` (29-09-2026): `PotentialOperations.super ValuePotential` conserva la suma, el producto o la fórmula de un nodo de utilidad con padres solo numéricos, con su nueva lista de padres, y solo pone una suma si tenía otra cosa; lo usan `AddLinkEdit`, `RemoveLinkEdit` y `Variable` |

| | |
|---|---|
| | `TypeEdit`. Quitar un padre que la fórmula usa la deja nombrando una variable que ya no es padre, y la evaluación lo dice con un mensaje en vez de sustituirla por una suma. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.base.linkEdits |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:189-233`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:92-105`<br>`core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:157-170`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:604-617`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/MultiAddLinkEdit.java:30-51` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 11 |

**Qué pasa.** Un nodo de utilidad cuyos padres son todos numéricos (un «supervalor») no guarda números propios: su potencial dice cómo combinar los de los padres, `SumPotential` (suma) o `ProductPotential` (producto). Si el usuario le traza un enlace a un nodo producto, el nodo pasa a sumar, y el modelo da otro resultado sin ningún aviso. Deshacer lo devuelve a producto, pero rehacer lo vuelve a sumar, y quitar después ese mismo enlace no lo devuelve a producto.

Camino de llegada: en modo edición, dibujar un enlace desde un nodo numérico hacia un nodo de utilidad producto (la ventana usa `MultiAddLinkEdit`, que construye un `AddLinkEdit` por par). En el repositorio hay nodos producto con padres solo numéricos en `0_miscellaneous/networks/id/ID-mediastinet.pgmx` (`Net_QALE`, `Weighted_Economic_Cost`), `0_miscellaneous/networks/dan/DAN-qale-mediastinet.pgmx` y `0_miscellaneous/networks/dan/DAN-mediastinet.pgmx` (`Net QALE`), `0_miscellaneous/networks/id/ID-arthronet.pgmx` y `0_miscellaneous/networks/dan/DAN-arthronet.pgmx` («Coste ajustado»), `0_miscellaneous/networks/mid/MID-HPV.pgmx` (`QoL [0]`) y varias redes de prueba de redes de análisis de decisiones.

Por qué pasa: al trazar el enlace, `AddLinkEdit.doEdit()` (módulo `core`), después de añadir el enlace, comprueba `nodeTo.getNodeType() == UTILITY && nodeTo.onlyNumericalParents()` y, para cada potencial antiguo, crea `new SumPotential(variables, rol)`, sea cual sea la clase que tenía. Un producto pasa a ser una suma. Lo mismo ocurre con cualquier otra clase que tenga el nodo en ese momento (en las redes del repositorio hay nodos así con `TreeADDPotential`, `Function Potential` y otros).

`RemoveLinkEdit.doEdit()` y `VariableTypeEdit.setPotentialsNodeAndChildren` repiten el mismo supuesto (crean `SumPotential` para un hijo de utilidad con padres numéricos), así que quitar un padre a un nodo producto, o cambiar el tipo de la variable de un padre, también lo convierte en suma (leído, no ejecutado para estas dos vías).

Medido sobre `0_miscellaneous/networks/id/ID-mediastinet.pgmx`: `Net_QALE` es `Product Potential` de tres padres; con `AddLinkEdit(TBNA_Morbidity → Net_QALE)` pasa a `SumPotential` de cuatro; deshacer lo devuelve a producto, pero rehacer lo vuelve a sumar; y quitando después ese mismo enlace con `RemoveLinkEdit` queda `SumPotential` de los tres padres originales. La utilidad esperada global (evaluación por eliminación de variables) pasa de 3,984 a 6,059 sin ningún aviso.

**El código.** core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:189-204

```
@Override protected void doEdit() {
    probNet.addLink(nodeFrom, nodeTo, isDirected);
    this.link = probNet.getLink(nodeFrom, nodeTo, isDirected);
    if (updatePotentials) {
        this.oldPotentials = nodeTo.getPotentials();
        // TODO Check if this UTILITY label is outdated
        if (nodeTo.getNodeType() == NodeType.UTILITY && nodeTo.onlyNumericalParents()) {
            // Add a default Sum potential to utility supervalue nodes
            for (Potential oldPotential : oldPotentials) {
                // Update potential
                List<Variable> variables = oldPotential.getVariables();
                if (!variables.contains(nodeFrom.getVariable())) {
                    variables.add(nodeFrom.getVariable());
                }
                Potential newPotential = new SumPotential(variables,
↪   oldPotential.getPotentialRole());
```

**Cómo arreglarlo.** Conservar la clase del potencial: si es `ProductPotential`, crear un `ProductPotential` con la variable añadida (o usar `addVariable` del propio potencial si da el resultado adecuado); usar `SumPotential` solo cuando el nodo no tenía ya un combinador (por ejemplo, venía de una tabla o de un potencial uniforme). Hay que decidir qué hacer con clases que no son suma ni producto (`FunctionPotential`, `TreeADDPotential`): mantenerlas y añadir la variable, o avisar.

La misma regla debe cumplirse en `AddLinkEdit.doEdit`, `RemoveLinkEdit.doEdit` (quitar la variable manteniendo la clase) y `VariableTypeEdit.setPotentialsNodeAndChildren`.

Prueba: en `0_miscellaneous/networks/id/ID-mediastinet.pgmx`, añadir y luego quitar un padre numérico a `Net_QALE` debe dejarlo como `ProductPotential` de sus tres padres y con la misma utilidad esperada que al principio.

### Error 27 · [Jorge Rico] Rehacer un enlace trazado y la restricción o la revelación que se le puso después deja el enlace sin ellas: se escriben en un enlace que ya no está en la red

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core; gui |
| **Paquete** | org.openmarkov.core.action.base.linkEdits; org.openmarkov.core.action.core; org.openmarkov.gui.action |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:254-264`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:171-187`<br>`gui/src/main/java/org/openmarkov/gui/action/LinkRestrictionPotentialValueEdit.java:73-115`<br>`core/src/main/java/org/openmarkov/core/action/core/RevelationStateEdit.java:28-52`<br>`gui/src/main/java/org/openmarkov/gui/action/RevelationIntervalEdit.java:67-69`<br>`core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:271-307`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:244-256`<br>`gui/src/main/java/org/openmarkov/gui/graphic/VisualLink.java:142` |
| **Comprobado** | Leyendo el código y ejecutándolo |

| | |
|---|---|
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 25, error 155, nota 23, nota 150 |

**Qué pasa.** Trazar un enlace A → B, ponerle una restricción (en su menú contextual, «Edit link restriction», marcar una casilla como incompatible y «OK»), deshacer las dos cosas con el menú de edición y rehacerlas deja el enlace A → B sin restricción. Rehacer no avisa ni falla: escribe la restricción en el objeto enlace que existía antes de deshacer, que ya no está en la red. Lo mismo ocurre con las condiciones de revelación de un enlace (qué estados del padre revelan al hijo), que se editan en otra ventanilla y con otras ediciones.

Por qué pasa: `AddLinkEdit.redo` (módulo `core`) vuelve a llamar a `probNet.addLink`, que crea un objeto `Link` nuevo, y actualiza su propio campo `link`. Pero las ediciones que se hicieron después sobre ese enlace guardan una referencia al objeto de su momento: `LinkRestrictionPotential ValueEdit` (módulo `gui`) guarda en el constructor el `Link` y su tabla de restricciones, y `Revelation StateEdit` y `RevelationIntervalEdit` guardan también el `Link`. Al rehacerse, escriben en el objeto viejo.

`RemoveLinkEdit.undo` también crea un objeto enlace nuevo, pero le pasa las restricciones y revelaciones del viejo, que comparten la misma tabla, así que ahí el daño es menor. Medido: poner una restricción, quitar el enlace, deshacer y deshacer después la restricción deja en el enlace de la red una tabla de restricciones con todo a 1, en lugar de ninguna tabla. Con esa tabla el enlace cuenta como restringido: por lectura, se dibuja rayado (`VisualLink`, línea 142) y se guarda con restricciones.

Medido con un pequeño programa de prueba que repite las ediciones de la ventana (el `MultiAdd LinkEdit` que crea el trazado, la sub-historia del diálogo de restricción con un `LinkRestriction PotentialValueEdit`, y deshacer y rehacer con `PNESupport`) sobre una red bayesiana con dos nodos A y B de dos estados:

- tras poner la restricción, el enlace de la red tiene la tabla `[1, 1, 0, 1]`;
- tras deshacer dos veces no hay enlace; tras rehacer dos veces hay un enlace distinto del original y sin restricciones, y es el objeto viejo el que tiene `[1, 1, 0, 1]`;
- en una red de análisis de decisiones, con `RevelationStateEdit` para que el estado «a1» de A revele B: tras deshacer y rehacer las dos ediciones, el enlace de la red no revela nada y el viejo revela «a1».

**El código.** core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:254-264

```
@Override public void redo() {
    setTypicalRedo(false);
    super.redo();

    nodeTo = probNet.getNode(variableTo.getName());
    probNet.addLink(nodeFrom, nodeTo, isDirected);
    this.link = probNet.getLink(nodeFrom, nodeTo, isDirected);
    if (updatePotentials) {
        nodeTo.setPotentials(newPotentials);
    }
}
```

gui/src/main/java/org/openmarkov/gui/action/LinkRestrictionPotentialValueEdit.java:73-105

```
public LinkRestrictionPotentialValueEdit(Link<Node> link, Integer newValue, int row, int
↪  col) {
    super(link.getFrom().getProbNet());
    this.link = link;
    ...
    this.tablePotential = link.getRestrictionsPotential();
    ...
}
...
@Override public void redo() {
    this.setTypicalRedo(false);
    super.redo();
    if (!link.hasRestrictions()) {
        LinkOperations.initializesRestrictionsPotential(link);
        this.tablePotential = link.getRestrictionsPotential();
    }
    tablePotential.setValues(newTable);
    checkRestrictionPotential(newTable);
}
```

**Cómo arreglarlo.** Dos caminos:

- que las ediciones que actúan sobre un enlace guarden sus extremos (y si es dirigido) en vez del objeto `Link`, y lo busquen en la red cada vez que hacen, deshacen o rehacen (`probNet.getLink(nodo1, nodo2, dirigido)`). Sitios: `LinkRestrictionPotentialValueEdit` (también su campo `tablePotential`), `RevelationStateEdit`, `RevelationIntervalEdit` y las ediciones internas `RawSetRevelation` y `RawSetRestriction` de `VariableTypeEdit`;
- que `AddLinkEdit.redo` y `RemoveLinkEdit.undo` vuelvan a meter en el grafo el mismo objeto `Link` que quitaron, en vez de crear otro; esto arregla todas las ediciones a la vez, pero exige que el grafo acepte reinsertar un enlace ya construido.

Conviene revisar con el mismo criterio las demás ediciones que quitan y vuelven a crear enlaces.

Prueba de regresión: la secuencia medida (trazar, restringir, deshacer dos veces, rehacer dos veces) debe dejar en el enlace de la red la tabla `[1, 1, 0, 1]`; lo mismo con una revelación; y restringir, quitar el enlace, deshacer y deshacer la restricción debe dejar el enlace sin tabla de restricciones.

### Error 28 · [Jorge Rico] Cambiar a suceso un nodo de decisión numérico sin política revienta y deja el nodo convertido a medias, sin poder deshacerlo

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:80-135`<br>`core/src/main/java/org/openmarkov/core/action/base/MultiStepEdit.java:36-48`<br>`gui/src/main/java/org/openmarkov/gui/action/ChangeNodeTypeEdit.java:47-62`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:332-342`<br>`core/src/main/java/org/openmarkov/core/model/network/VariableType.java:58-65` |

| | |
|---|---|
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 31, nota 184, nota 142 |

**Qué pasa.** Cambiar a suceso un nodo numérico que no tiene potenciales termina en el diálogo de «error no previsto», y el nodo queda convertido a medias: tipo de nodo `event`, variable de tipo `Event` y un solo estado. El cambio no entra en el historial, así que no se puede deshacer, y cancelar el diálogo de propiedades no lo repara porque el cambio no está en su historial provisional.

Camino del usuario, en una red de simulación de eventos discretos (la única en la que el botón «Event» está habilitado): crear un nodo de decisión; en su dominio elegir «Numeric» (el tipo de variable ofrece Numeric para decisiones, y eso le pone un potencial uniforme); quitarle la política con «quitar política» (`RemovePolicyEdit` vacía sus potenciales); abrir sus propiedades y pulsar el tipo «Event». También llega un nodo numérico de azar o de decisión sin potencial leído de un fichero.

Por qué pasa: `ChangeNodeTypeEdit` (el botón de tipo de nodo en la pestaña de definición del diálogo de propiedades del nodo) es una edición de varios pasos. Primero cambia el tipo de nodo (`SetNodeTypeEdit`) y, si el tipo de variable ya no vale para el tipo nuevo, ejecuta `VariableTypeEdit(node, tipo, false)`, es decir, sin recalcular el potencial del propio nodo. Un nodo de suceso solo admite variables de tipo EVENT (`VariableType.of`), así que pasar un nodo numérico a suceso entra en esa rama. `VariableTypeEdit`, cuando el tipo original es NUMERIC, construye un potencial uniforme con el papel del primer potencial del nodo: `node.getPotentials().getFirst().getPotentialRole()` (línea 127). Si el nodo no tiene potenciales, `getFirst()` lanza `NoSuchElementException`.

Por qué el daño es mayor que una excepción: `MultiStepEdit.doEdit()` solo revierte los pasos ya ejecutados si captura `DoEditException`; una excepción de tiempo de ejecución la atraviesa sin revertir nada. El primer paso de `VariableTypeEdit` (`RawSetType`, que pone el tipo de variable nuevo y cambia los estados) y el `SetNodeTypeEdit` de fuera ya se han aplicado, y como la edición no termina, no entra en el historial.

Medido ejecutando esa misma secuencia de ediciones sin ventana: salta `NoSuchElementException` en `VariableTypeEdit.java:127`; después el nodo queda con tipo `event`, variable de tipo `Event` y un solo estado, y el historial tiene las mismas tres entradas que antes. En la aplicación, la excepción llega desde el botón del diálogo al manejador global (el diálogo de «error no previsto»).

**El código.** core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:114-130

```
if (originalVariableType == VariableType.NUMERIC) {
    // Build the list of variables (node and its parents)
    List<Variable> variables = new ArrayList<>();
    ...
    // Create and assign uniform potential
    UniformPotential uniformPotential = new UniformPotential(
            variables,
            node.getPotentials().getFirst().getPotentialRole()
    );
    stepExecuter.execute(new RawSetPotentialEdit(node, uniformPotential));
```

core/src/main/java/org/openmarkov/core/action/base/MultiStepEdit.java:36-48

```
@Override protected final void doEdit() throws DoEditException {
    StepExecuter stepExecuter = new StepExecuter();
```

```
    try {
        this.edits.clear();
        this.doMultiStepEdit(stepExecuter);
        this.edits.addAll(stepExecuter.executedEdits);
    } catch (DoEditException e) {
        for (PNEdit editToUndo : stepExecuter.executedEdits.reversed()) {
            editToUndo.undo();
        }
        throw e;
    }
}
```

**Cómo arreglarlo.** Hay dos piezas.

La local: el papel del potencial no debe salir de un potencial que puede no existir. Se puede deducir del tipo de nodo (como hace `PotentialOperations.getUniformPotential`, que usa POLICY para decisiones y CONDITIONAL_PROBABILITY en otro caso) o comprobar la lista vacía. Hay que decidir también qué potencial debe tener un nodo de suceso (probablemente el que genere `PotentialUtils.generateDefaultPotential`, que `ChangeNodeTypeEdit` ya aplica después).

La de la maquinaria: que la reversión de `MultiStepEdit` se haga ante cualquier excepción (`catch` de `RuntimeException` además de `DoEditException`, relanzando la original), y que `StepExecuter.execute` apunte la edición antes de ejecutarla para que un paso que falle a medias también se revierta. La misma reversión falta en `MultiEdit.doEdit()`. El problema general de diseño, que una edición que modifica la red y después falla no se revierte, está descrito en nota 142; allí no hay nada que corregir.

Pruebas:

- Red de simulación de eventos discretos, nodo de decisión numérico sin potenciales, `ChangeNodeTypeEdit` a EVENT: exigir que termine (o lance una excepción del dominio) y que, si falla, el nodo siga siendo de decisión y numérico.
- De la maquinaria: un `MultiStepEdit` cuyo segundo paso lance una excepción no comprobada debe dejar la red como estaba.

### Error 29 · [Corregido] Mover un estado de un nodo cuyo hijo no tiene potencial revienta y deja la tabla del nodo permutada con los estados en el orden viejo

| | |
|---|---|
| **Estado** | Corregido. La parte del nodo sin potencial, desde `2c4fca6` (ajustado en `72d422b`); el resto, en el commit `6d143ee` (29-09-2026): `VariableStateOperations` reordena todos los potenciales del nodo y de sus hijos, no solo el primero, y los instala todos cuando ya están calculados. También los de un nodo de decisión, que se saltaban: una política impuesta conservaba el orden viejo de los estados de su padre y cambiaba de sentido sin aviso (medido en `io/src/test/resources/ImposedPolicy.pgmx`), consecuencia que esta ficha no recogía. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:200-236`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:154-163`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:215-220` |

| | |
|---|---|
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 30, error 70 |

**Qué pasa.** Subir o bajar un estado de un nodo cuyo hijo (o él mismo) es de azar o de utilidad y no tiene potencial termina en un error inesperado. Lo grave es cómo queda la red: la tabla del nodo ya se ha reordenado, pero sus estados siguen en el orden viejo, así que las probabilidades quedan intercambiadas en silencio. La edición no entra en el historial, así que ni deshacer ni «Cancelar» en el diálogo la revierten.

Camino de llegada de la parte ya corregida: un nodo de azar o de utilidad sin potencial. Desde los commits `710de2c` y `7776e28`, un nodo recién creado no recibe potencial hasta que se abre su relación; además, el lector de `.pgmx` acepta un nodo al que le falta su `<Potential>` sin añadir nada: en el repositorio lo hace `io/src/test/resources/ImposedPolicy.pgmx` (nodo de utilidad U). Basta abrir un fichero así y mover un estado del nodo o de uno de sus padres en la pestaña de dominio. Lo que pasa con un nodo numérico de azar sin potencial al propagar o evaluar está en error 43.

Por qué pasa: `VariableStateOperations.reorderStatesInNodeAndChildren` (módulo `core`) primero pregunta a todos los potenciales si aceptan el reordenado (`refuseIfAnyPotentialCannotBeReordered`: esa parte ya es transaccional), luego llama a `setPotentialAfterReorderingFirstPotential` para el nodo y para cada hijo, y al final cambia los estados de la variable. Ese método, para nodos de azar o de utilidad, hace `auxNode.getPotentials().getFirst()` sin comprobar que haya alguno. `NodeStateEdit.doEdit` solo traduce `NotSupportedOperationException`, así que en la ventana la `NoSuchElementException` sale como error inesperado.

Medido, con A → B, A con tabla [0,3; 0,7] y B (de azar) sin potencial: `NodeStateEdit(A, DOWN, 0, "")` lanza `NoSuchElementException` desde la línea 230. La tabla de A ya se ha reordenado e instalado (queda [0,7; 0,3]), pero los estados de A siguen siendo «a0, a1» porque `setStates` no llegó a ejecutarse; la excepción salta antes de `addEdit`.

Segundo efecto del mismo método: `auxNode.setPotential(newPotential)` vacía la lista antes de añadir, así que un nodo con dos potenciales se queda solo con el primero reordenado; los demás no se quedan con los números viejos, se eliminan. Medido en `0_miscellaneous/networks/bn/BN-prostanet.pgmx` («Cáncer de próstata» pasa de 2 potenciales a 1) y en `0_miscellaneous/networks/mid/MID-Cochlear.pgmx` («Processor 2 age [1]»). En esas dos redes el segundo potencial parece un duplicado o una tabla uniforme, así que el cambio numérico es nulo o pequeño, pero el modelo cambia sin aviso.

**El código.** core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:200-236

```
private static void reorderStatesInNodeAndChildren(Node node, Variable variable, State[]
↪  newStates) {
    refuseIfAnyPotentialCannotBeReordered(node, variable);
    setPotentialAfterReorderingFirstPotential(node, variable, newStates);
    for (Node child : node.getChildren()) {
        setPotentialAfterReorderingFirstPotential(child, variable, newStates);
    }
    variable.setStates(newStates);
    resetLink(node);
}
...
private static void setPotentialAfterReorderingFirstPotential(Node auxNode, Variable
↪  variable, State[] newStates) {
```

```
    if (auxNode.getNodeType() == NodeType.CHANCE || auxNode.getNodeType() ==
    ↪  NodeType.UTILITY) {
        Potential oldPotential = auxNode.getPotentials().getFirst();
        Potential newPotential = oldPotential.reorder(variable, newStates);
        if (newPotential != null) {
            auxNode.setPotential(newPotential);
        }
    }
}
```

**Cómo arreglarlo.** Reordenar todos los potenciales del nodo (y de cada hijo) y no tocar un nodo sin potenciales; instalar los resultados con `setPotentials(lista)` en vez de `setPotential(uno)`. Para no dejar la red a medias, calcular primero todos los potenciales reordenados y solo después instalarlos y cambiar los estados: la misma idea que ya sigue `refuseIfAnyPotentialCannotBeReordered`, que pregunta a todos antes de tocar nada.

Conviene revisar a la vez si el lector debe aceptar nodos de azar sin potencial (hoy los acepta en silencio).

Prueba: A → B con B sin potencial, mover un estado de A: no debe lanzar, y los estados y la tabla de A deben quedar permutados juntos; y un nodo con dos potenciales debe conservar los dos, ambos reordenados.

### Error 30 · [Corregido] Deshacer o cancelar un cambio de estados copia las tablas de los hijos en el nodo y en sus padres, y la red da otras probabilidades y se guarda así

| | |
|---|---|
| **Estado** | Corregido en el commit `c5328e1` (29-09-2026): para poder deshacer, `NodeStateEdit` guarda la lista de potenciales propia del nodo y la de cada vecino, como las demás ediciones, en vez de lo que devuelve `ProbNet.getPotentials(variable)`, que no se ha tocado. Deshacer o cancelar añadir, quitar, subir o bajar un estado deja cada nodo con sus potenciales de antes; en `BN-prostanet.pgmx`, la red vuelve a tener 50 potenciales y las mismas probabilidades. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:137-145,203-207`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:576-600`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:241-248`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:399-402`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TablePotentialPanel.java:170-171`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_1_0.java:140-153` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 24, error 29, nota 5, nota 150 |

**Qué pasa.** En la pestaña de dominio del diálogo de propiedades de un nodo, añadir, quitar, subir o bajar un estado y después pulsar «Cancel» (o aceptar y deshacer con el menú de edición) deja la red con tablas repetidas. El nodo recupera sus estados, pero su lista de potenciales incluye ahora

también las tablas de sus hijos, y cada padre recibe a su vez las tablas de los hijos de ese padre. Nada lo avisa.

Consecuencias medidas en `0_miscellaneous/networks/bn/BN-prostanet.pgmx` con el nodo «Cáncer de próstata» (2 padres y 10 hijos), añadiendo un estado y cancelando (la misma secuencia que el diálogo: sub-historia abierta, `NodeStateEdit` y `cancelLastSubEditHistory`):

- la red pasa de 50 potenciales a 82: «Cáncer de próstata» tiene 12 (tenía 2), «Metástasis» 13, «Cistitis» 6;
- la propagación sin hallazgos da otros números: la probabilidad del primer estado de «PSA aux» pasa de 0,674 a 0,958 (la mayor diferencia en toda la red es 0,28);
- guardar escribe los 82 potenciales; al reabrir, cada tabla repetida se cuelga del nodo de su primera variable («Cáncer de próstata» con 6, «TAC» con 3…) y las posteriores siguen mal (diferencia máxima 0,30), así que el daño queda en el fichero;
- el primer potencial de «Cáncer de próstata» pasa a ser la tabla de «Biopsia». La pestaña de la relación enseña el primer potencial del nodo (`TablePotentialPanel`, líneas 170-171), así que, por lectura, enseñaría la tabla de «Biopsia» como si fuera la del nodo; no se ha construido el panel.

Lo mismo al quitar un estado, al bajar uno y deshacer, e incluso con una edición que no cambia nada (bajar el primer estado, medido por programa). Con `0_miscellaneous/networks/bn/BN-asia.pgmx` y el nodo «TuberculosisOrCancer» la red pasa de 8 potenciales a 12 y la probabilidad del estado «yes» de «X-ray» pasa de 0,110 a 0,071. Renombrar un estado no lo provoca, porque su deshacer no toca los potenciales.

Por qué pasa: el constructor de `NodeStateEdit` (módulo `core`) fotografía, para poder deshacer, `probNet.getPotentials(variable)` del nodo y de cada vecino. Ese método no devuelve los potenciales guardados en el nodo, sino todos los potenciales de la red que contienen la variable, buscados en el nodo, en sus vecinos y en los otros padres de sus hijos; entre ellos está la tabla de cada hijo. `undo()` instala esas listas con `setPotentials` en el nodo y en cada vecino, de modo que las tablas de los hijos quedan en dos o tres nodos a la vez. Además, `getPotentials(variable)` recorre los vecinos antes que el propio nodo, y por eso la tabla de un hijo queda la primera de la lista.

Llegan a esta edición todos los botones de estados de la pestaña de dominio (`DiscretizeTablePanel`, líneas 746, 784, 830 y 848, y el arrastre de filas de la línea 183; `PrefixedKeyTablePanel`, líneas 154, 174, 207 y 231), también cuando el cambio se propaga a los nodos gemelos de otros ciclos. «Cancel» llega a `undo()` porque el diálogo del nodo cancela su historial provisional (`NodePropertiesDialog.doCancelClickBeforeHide`).

**El código.** core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:136-145

```
// Save node potentials and neighbours
this.oldPotentials = probNet.getPotentials(variable);
this.listOldPotentials = new HashMap<>();
for (Node nodeNeighbour : probNet.getNeighbors(node)) {
    this.listOldPotentials.put(
            nodeNeighbour.getVariable(),
            probNet.getPotentials(nodeNeighbour.getVariable())
    );
}
```

core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:203-207

```
// We restore the potentials
node.setPotentials(oldPotentials);
for (Variable variable : listOldPotentials.keySet()) {
```

```
    probNet.getNode(variable).setPotentials(listOldPotentials.get(variable));
}
```

core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:586-599

```
Set<Node> semiNeighbors = new LinkedHashSet<>(getNeighbors(node));
semiNeighbors.add(node);
...
for (Node neighbor : semiNeighbors) {
    List<Potential> nodePotentials = neighbor.getPotentials();
    for (Potential potential : nodePotentials) {
        if (potential.contains(variable)) {
            potentials.add(potential);
        }
    }
}
```

**Cómo arreglarlo.** Fotografiar la lista propia de cada nodo, copiada: `new ArrayList<>(node.getPotentials())` para el nodo y lo mismo para cada nodo cuya lista cambie la edición (la copia hace falta porque `setPotentials` vacía la lista del nodo antes de rellenarla). `VariableStateOperations` solo sustituye los potenciales del nodo y de sus hijos (`setUniformPotential` al añadir o quitar, `reorderStatesInNodeAndChildren` al mover), así que bastan el nodo y sus hijos; guardar también los padres con su propia lista no hace daño.

`ProbNet.getPotentials(Variable)` hace lo que dice su documentación y tiene otros llamadores que solo leen (`ProbNetOperations`, `DANOperations`, `CENetworkBuilder`); no hay que cambiarlo, sino dejar de usarlo para fotografiar.

Si la fotografía se comparte con `NodeReplaceStatesEdit`, como propone error 24, hay que compartir la corregida.

Prueba de regresión: sobre `0_miscellaneous/networks/bn/BN-prostanet.pgmx`, dentro de una sub-historia, ejecutar `NodeStateEdit` con `ADD`, `REMOVE`, `UP` y `DOWN` sobre «Cáncer de próstata» y cancelar: el número de potenciales de cada nodo, el primero de cada lista y las posteriores sin hallazgos deben ser los de antes.

### Error 31 · Pegar en una red que ya incumple una restricción se rechaza siempre, con un mensaje sobre la violación que ya estaba

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.base |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/MultiStepEdit.java:36-37`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:151-161`<br>`core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:77`<br>`gui/src/main/java/org/openmarkov/gui/action/ChangeNodeTypeEdit.java:45,77-86`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/InvertLinkEdit.java:131-134` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué estrategia rige para las ediciones de varios pasos («comprobar antes» o «probar y revertir») y si una violación previa debe bloquear ediciones que |

| | |
|---|---|
| | no la empeoran |
| **Corregir antes** | error 37: si se decide que una red que incumple sus restricciones no pueda entrar al abrir un fichero, este error deja de alcanzarse. |
| **Grafo** |   |
| **Relacionados** | error 28, nota 184, error 37, nota 142, nota 158 |

**Qué pasa.** En una red que ya incumple una restricción (por ejemplo, una red bayesiana con un ciclo leída de un fichero), todo pegado se rechaza y se revierte, y el mensaje habla de esa violación que ya estaba, no de lo pegado. Lo mismo ocurrirá al cambiar el tipo de cualquier nodo o al invertir cualquier enlace en esa red, porque las tres operaciones comprueban la red entera.

Hoy solo se llega con una red inválida, y la única puerta hacia una red inválida es abrir un fichero, que no comprueba ninguna restricción: eso es error 37, y los dos defectos van juntos.

Por qué pasa: `PNEdit.checkConstraintsWillBeMet` se documenta como la promesa de que ninguna restricción quedará violada tras la edición, y `MultiEdit` la implementa recorriendo sus sub-ediciones. `MultiStepEdit` la anula con un cuerpo vacío, así que sus tres descendientes (`PasteEdit`, `VariableTypeEdit`, `ChangeNodeTypeEdit`) no comprueban nada por adelantado: ejecutan los pasos y revierten si uno falla. `VariableTypeEdit` y `ChangeNodeTypeEdit` terminan su comprobación con `super.checkConstraintsWillBeMet(...)`, que parece delegar en el recorrido de sub-ediciones y resuelve al cuerpo vacío: esas líneas no hacen nada.

Para comprobar el resultado, `PasteEdit` añade al final una edición anónima cuyo `doEdit()` no hace nada y cuya comprobación previa es `probNet.checkConstraintsIn(...)`: la red entera, después de pegar. `ChangeNodeTypeEdit` hace lo mismo dentro de `SetNodeTypeEdit` (`probNet.checkConstraints()` tras cambiar el tipo) e `InvertLinkEdit` tras invertir. Esa comprobación posterior no distingue lo que la edición ha roto de lo que ya estaba roto.

Medido: se abrió con el lector de `.pgmx` una red bayesiana con el ciclo A→B→A (se carga sin aviso) y se pegó en ella un nodo suelto Z copiado de otra red. Un `AddNodeEdit` solo para Z pasa su comprobación, pero el pegado se rechaza con «Could not met the constraint: No cycles paths are allowed. - There is a cycle traversing between … A … and … B …» y la red queda sin Z.

**El código.** gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:151-161

```
PNEdit verifier = new PNEdit(probNet) {

    @Override public void checkConstraintsWillBeMet(ConstraintChecker constraintChecker) {
        probNet.checkConstraintsIn(constraintChecker);
    }

    @Override protected void doEdit() {

    }
};
stepExecuter.execute(verifier);
```

core/src/main/java/org/openmarkov/core/action/base/MultiStepEdit.java:33-34

```
@Override public void checkConstraintsWillBeMet(ConstraintChecker constraintChecker) {
}
```

**Cómo arreglarlo.** Hay que decidir qué hacer con una red que ya llega inválida (ver `decision-de-equipo`). Una opción es comparar las violaciones antes y después y rechazar solo las nuevas; otra, impedir que entre inválida al abrir el fichero, que es el arreglo de error 37.

A más largo plazo, la anulación a vacío y la edición verificadora sobran cuando cada restricción sepa juzgar una edición; esa propuesta de diseño está en nota 153 y nota 158, donde no hay nada que corregir.

Sitios con la misma comprobación posterior: `PasteEdit` (verificador), `ChangeNodeTypeEdit.SetNodeTypeEdit.doEdit` e `InvertLinkEdit.doEdit` (líneas 131-134); y las dos llamadas `super.checkConstraintsWillBeMet` que no hacen nada en `VariableTypeEdit` y `ChangeNodeTypeEdit`.

Prueba de regresión: abrir una red con un ciclo, pegar un nodo aislado y exigir que se pegue (o, si se decide lo contrario, que el mensaje diga que la red ya era inválida).

### Error 33 · [Jorge Rico] Deshacer o cancelar la casilla «mostrar el comentario al abrir» no la devuelve a su valor

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/NetworkCommentEdit.java:34-50`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkDefinitionPanel.java:301-316`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkDefinitionPanel.java:351-358`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:281-284`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:356` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** El usuario marca la casilla «mostrar el comentario al abrir» en las propiedades de la red, pulsa «Cancelar», y la red queda con la opción cambiada. Al guardar se escribe en el fichero (`PGMXWriter_0_2`, atributo del comentario) y al volver a abrir la red se enseña el comentario. Deshacer tampoco la devuelve.

Camino de llegada: Red → Propiedades → pestaña de definición. La casilla ejecuta, cada vez que se marca o desmarca, un `NetworkCommentEdit` con el comentario actual y el valor nuevo de la casilla (`NetworkDefinitionPanel.getCheckBoxShowCommentOnOpening`); editar el comentario también lo construye, pasando el estado de la casilla. El diálogo abre un historial parcial y «Cancelar» deshace lo hecho en él.

Por qué pasa: `NetworkCommentEdit` (módulo `core`) cambia dos cosas en `doEdit()`: el comentario de la red y la opción `showCommentWhenOpening` (si el comentario se enseña al abrir el fichero). El constructor solo guarda el comentario anterior (`currentComment`), y `undo()` solo repone el comentario.

Medido: con la opción a falso, ejecutar la edición con la casilla marcada y deshacer deja la opción a `true`; dentro de un historial parcial, cancelar también la deja a `true`.

**El código.** core/src/main/java/org/openmarkov/core/action/core/NetworkCommentEdit.java:34-50

```
public NetworkCommentEdit(ProbNet probNet, String newComment, boolean
↪  showCommentWhenOpening) {
```

```
    super(probNet);
    this.newComment = newComment;
    this.currentComment = probNet.getComment();
    this.showCommentWhenOpening = showCommentWhenOpening;
}

@Override protected void doEdit() {
    probNet.setComment(newComment);
    probNet.setShowCommentWhenOpening(showCommentWhenOpening);
}

@Override public void undo() {
    super.undo();
    probNet.setComment(currentComment);
}
```

**Cómo arreglarlo.** Guardar en el constructor el valor anterior de `probNet.getShowCommentWhenOpening()` y reponerlo en `undo()` junto al comentario.

Prueba: con la opción a falso, ejecutar `NetworkCommentEdit(red, comentario, true)` y deshacer debe dejarla a falso; y lo mismo con `openNewSubEditHistory` + `cancelLastSubEditHistory`.

### Error 34 · [Corregido] Deshacer o cancelar el cambio de «Always append» de un suceso no devuelve el valor que tenía

| | |
|---|---|
| **Estado** | Corregido en el commit `8c56ebc` (14-10-2026): `EventNodeAlwaysAppendEdit` guarda `node.isAlwaysAppend()` como valor anterior. `EditUndoCases` tiene ya dos casos para esta edición (los dos que antes salían mal: quitar «Always append» a un suceso sin «Always observed», y ponérselo a uno con «Always observed»); con el código anterior fallaban los dos. Copiar una red ya conservaba «Always append» desde el commit `db16fd4` (`Node.copyPropertiesTo`). El constructor de copia `Node(Node)` sigue sin copiarlo; nadie lo usa con sucesos. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/EventNodeAlwaysAppendEdit.java:35-56`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:674-690,735-744,1001-1002`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:406-409`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:141-149,733-743` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 78 |

**Qué pasa.** Un nodo de suceso (los nodos de las redes de simulación de eventos discretos) tiene dos campos booleanos distintos en `Node`: `alwaysObserved` («su valor se conoce siempre») y `alwaysAppend` («cuando otro suceso lo dispara, se encola siempre una instancia nueva» frente a «solo hay una en la cola y se le cambia la hora»). El segundo decide cómo se simula el suceso.

Lo que ve el usuario: un suceso que tenía «Always append» marcado, se desmarca y se cancela, **se queda desmarcado**, y la simulación lo trata desde entonces como un suceso que no se encola. No hay aviso.

Camino del usuario: en una red de simulación de eventos discretos, abrir las propiedades de un nodo de suceso. El diálogo enseña la casilla «Always append» en la fila de agentes/criterio/observado (`NodeDefinitionPanel.getAgentsOrDecisionCriteriaOrObserved`, líneas 674-689). Marcarla o desmarcarla ejecuta la edición al momento (líneas 996-998). Si el usuario pulsa **Cancelar**, el diálogo deshace todo lo hecho (`NodePropertiesDialog.doCancelClickBeforeHide` → `cancelLastSubEditHistory`, líneas 389-392); si pulsa Aceptar y luego Ctrl+Z, igual. En los dos casos se ejecuta el `undo()` de la edición.

Por qué pasa: la edición que cambia `alwaysAppend`, `EventNodeAlwaysAppendEdit` (módulo `core`), guarda en su constructor como valor anterior el **otro** campo, `node.isAlwaysObserved()`, y su `undo()` escribe ese valor en `alwaysAppend`. Deshacer, por tanto, no devuelve lo que había: deja en «encolar siempre» lo que valiera «siempre observado».

Como la casilla «siempre observado» no se ofrece para nodos de suceso (`AlwaysObservedPropertyValidator` solo la admite en nodos de azar), en la práctica `alwaysObserved` vale `false` en un suceso, y de ahí sale el resultado descrito arriba.

Medido (red `DESNetworkType`, historial de deshacer activado):

```
antes alwaysAppend=true  alwaysObserved=false -> tras la edición=false -> tras
↪  deshacer=false   (debía ser true)
antes alwaysAppend=false alwaysObserved=false -> tras la edición=true  -> tras
↪  deshacer=false   (correcto por casualidad)
antes alwaysAppend=true  alwaysObserved=true  -> tras la edición=false -> tras
↪  deshacer=true    (correcto por casualidad)
antes alwaysAppend=false alwaysObserved=true  -> tras la edición=true  -> tras
↪  deshacer=true    (debía ser false)
```

La prueba de ediciones del proyecto (`integrationTests/src/test/java/org/openmarkov/integrationTests/action/EditUndoCases.java`, línea 98) deja esta edición sin caso («needs an event node of a simulation network; pending»), por eso no se detecta.

**El código.** core/src/main/java/org/openmarkov/core/action/core/EventNodeAlwaysAppendEdit.java:35-56

```java
public EventNodeAlwaysAppendEdit(Node node, boolean newAlwaysAppend) {
    super(node.getProbNet());
    this.lastAlwaysAppend = node.isAlwaysObserved();
    this.newAlwaysAppend = newAlwaysAppend;
    this.node = node;
}
...
@Override public void undo() {
    super.undo();
    node.setAlwaysAppend(lastAlwaysAppend);
}
```

**Cómo arreglarlo.** Guardar `node.isAlwaysAppend()` en el constructor (línea 37). Una edición genérica de propiedades que lea y escriba el mismo campo evitaría este tipo de error de raíz, pero es un problema de diseño de las ediciones que no hace falta resolver para esto.

Por coherencia, el constructor de copia `Node(Node)` (líneas 141-149) puede copiar también `alwaysAppend` (y lo que copia `copyPropertiesTo`). Hoy no lo copia, pero su único llamador de producción es `ICIOptionListenerAssistant` (línea 110), que trabaja con nodos de azar de modelos canónicos, nunca con sucesos, así que nadie lo alcanza con un suceso.

Prueba de regresión: rellenar el caso pendiente de `EditUndoCases` con un nodo de suceso de una red `DESNetworkType` que empiece con `alwaysAppend=true` y `alwaysObserved=false`; ejecutar la

edición con `false`, deshacer y comprobar que vuelve a `true` (y el caso simétrico con `always Observed=true`).

### Error 35 · [Corregido] La ventana de una red expandida o aprendida a partir de otra no se marca nunca como modificada, y cerrarla no pide guardar

| | |
|---|---|
| **Estado** | Corregido en el commit `4d1921e` (29-09-2026): `ProbNetCopier` ya no sustituye los oyentes de la copia por los del original, en ninguna de las dos copias. La copia se marca como modificada tras una edición, que es lo que su pestaña lee para pedir que se guarde (medido con la red expandida de `MID-Chancellor-corrected.pgmx` y con las copias de `BN-asia.pgmx`), y los oyentes del original dejan de recibir sus ediciones. Al aprender partiendo de una red modelo, la pestaña de la red aprendida queda marcada como modificada al terminar, como ya pasaba sin red modelo. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetCopier.java:39-67,119`<br>`core/src/main/java/org/openmarkov/core/action/base/PNESupport.java:55-62,92-95,286-295`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:101-108,451-469`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:431-432`<br>`learning.core/src/main/java/org/openmarkov/learning/core/LearningManager.java:185-186`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:161-186`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:1152-1155`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:368-388`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:899-904,1019-1023` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 36, error 49, error 75, error 174, nota 38, nota 177 |

**Qué pasa.** Cuando una red expandida o aprendida a partir de otra se abre en su propia pestaña, lo que el usuario edite después en esa pestaña no la pone nunca como modificada: el título no cambia de color y, al cerrarla (o al cerrar la aplicación), no se pregunta si se quiere guardar, así que los cambios se pierden sin aviso.

Hay dos caminos en los que una copia acaba abierta en su propia pestaña del editor:

1. **Expandir una red temporal.** Clic derecho sobre el lienzo vacío de una red con nodos temporales (un MID, diagrama de influencia de Markov) → «Expand network» (`EditorInputHandler.java:667-670`, `InferenceHandler.expandNetwork`, líneas 164-185). La red expandida sale de `TemporalNetOperations.expandNetwork`, que hace `deepCopy` y luego `copy()` (líneas 457 y 102), y `ProbNetOperations.convertNumericalVariablesToFS` hace otra `copy()` (línea 432). Se abre con `createNewFrame`.
2. **Aprender partiendo de una red modelo.** Herramientas de aprendizaje con la opción de usar como red modelo la red abierta (`LearningDialog.java:1019-1023`) y «empezar desde

la red modelo»: `LearningManager.applyModelNet` hace `modelNet.copy()` (línea 186) y esa red aprendida se abre en una pestaña nueva (`LearningDialog.java:899-904`). Además, todas las ediciones que el algoritmo de aprendizaje ejecuta sobre la red aprendida avisan a la ventana de la red modelo.

No se ha arrancado la aplicación: la cadena desde la edición hasta el indicador está leída en el código, y el comportamiento del indicador está medido.

**Qué es un oyente aquí.** Cada red (`ProbNet`) tiene un `PNESupport`, que guarda la historia de deshacer/rehacer y una lista de oyentes (`PNEditListener`). Cada vez que se ejecuta una edición sobre la red (`PNEdit.executeEdit`), se avisa a todos los oyentes de esa lista. Uno de esos oyentes es el propio `PNESupport` de la red: es él quien, al recibir el aviso, pone a verdadero su indicador `networkIsModified` (líneas 286-295). La ventana del editor (`NetworkEditorPanel.afterEditExecutes`, líneas 1158-1161) lee ese indicador después de cada edición y lo copia a su `modified`; de `modified` dependen el color del título de la pestaña (`MainPanel.java:519-537`) y que al cerrar la pestaña se pregunte si se quiere guardar (`NetworkFileHandler.networkCanBeClosed`, líneas 372-392).

**Por qué pasa.** Las dos copias de red de `ProbNetCopier` (la superficial, `shallowCopy`, que es `ProbNet.copy()`, y la profunda, `deepCopy`) terminan con `dest.getPNESupport().setListeners(source.getPNESupport().getListeners())`. `setListeners` vacía la lista de la copia y mete en ella los oyentes del original. Resultado:

- la copia deja de oírse a sí misma: su `PNESupport` ya no está en su propia lista, así que su indicador de modificación no cambia nunca;
- la copia avisa, en cada edición, a los oyentes del original: al `PNESupport` del original y a la ventana del original (su `NetworkEditorPanel`, su `VisualNetwork`, su `PNEditEventHandler`), que se redibujan y reconstruyen su información visual por una edición que no ocurrió en su red.

Medido sobre `0_miscellaneous/networks/bn/BN-asia.pgmx`: la copia no contiene su propio `PNESupport` entre sus oyentes y sí el del original y el oyente que se añadió al original; tras añadir un enlace en la copia (con deshacer activado) se avisa una vez al oyente del original, la copia sigue con `networkIsModified() = false` y el original no tiene el enlace. La misma edición sobre una red leída de fichero da `networkIsModified() = true`. Sobre `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`, la red que devuelve `TemporalNetOperations.expandNetwork(red, evidencia, nombre)` tampoco se oye a sí misma y oye al `PNESupport` y a la ventana del MID original.

En esas dos pestañas, además, el deshacer está apagado, porque `createNewFrame` no lo enciende (sí lo hacen `openNetwork` y `createNewNetwork`); eso está descrito en error 36. Aun encendiéndolo, el indicador seguiría sin cambiar, como muestra la medida. Otro defecto de las mismas copias, que comparten con el original las opciones de inferencia, está en error 49.

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetCopier.java:39-66

```
static ProbNet shallowCopy(ProbNet source) {
    ProbNet dest = new ProbNet(source.getNetworkType());
    ...
    copyLinks(source, dest, false);
    copyMetadata(source, dest, false);
    dest.getPNESupport().setListeners(source.getPNESupport().getListeners());
    ...
    // Shared like the potentials: a shallow copy carries the very options of the original.
    dest.setInferenceOptions(source.getInferenceOptions());
    return dest;
}
```

core/src/main/java/org/openmarkov/core/action/base/PNESupport.java:92-95 y 286-295

```
public void setListeners(Collection<? extends PNEditListener> listeners) {
    this.listeners.clear();
    this.listeners.addAll(listeners);
}
...
private void onEditChanges(PNEdit edit) {
    ...
    this.networkIsModified = (!this.withUndo ||
    ↪  this.editsHistoryStacker.getCurrentUndoManager()
                                                .nextEditToUndo() !=
                                                ↪  lastDoneEditWhenSave);
}
```

**Cómo arreglarlo.** Que ninguna de las dos copias transfiera oyentes por omisión: la copia se queda con la lista que le da su propio constructor (`PNESupport(ProbNet)` se añade a sí mismo, línea 61). Si algún llamador necesita de verdad que la copia avise a otros, que lo pida con una llamada explícita.

Hay que quitarlo en los dos sitios: `ProbNetCopier.shallowCopy` (línea 56) y `ProbNetCopier.deepCopy` (línea 119). Conviene revisar los llamadores de `copy()`/`deepCopy()` que registran oyentes después (por ejemplo `HuginForest`, líneas 108 y 114, que añade y quita la heurística sobre su red de Markov) para confirmar que ninguno cuenta con heredar los del original.

Pruebas de regresión (en `core`, sin ventana):

- Leer o construir una red, añadirle un oyente contador, copiarla con `copy()` y con `deepCopy()`; en cada copia comprobar que su propio `PNESupport` está entre sus oyentes, que el contador del original no está, y que tras ejecutar una edición sobre la copia `networkIsModified()` pasa a verdadero y el contador del original sigue a cero.
- Otra sobre `TemporalNetOperations.expandNetwork(red, evidencia, nombre)` con un MID del repositorio, con las mismas comprobaciones.

### Error 36 · [Corregido] En la pestaña de una red expandida o aprendida, deshacer no recuerda nada hasta que se crea un nodo o se abre ciertos diálogos

| | |
|---|---|
| **Estado** | Corregido. En las redes que el usuario crea o abre (fichero, dirección de internet, sesión anterior, línea de órdenes), `NetworkFileHandler` ya encendía el deshacer desde el principio; el resto, en el commit `10bbe95` (29-09-2026): `NetworkFileHandler.createNewFrame`, por donde pasan todas las pestañas, enciende el deshacer, así que en la red expandida, en la aprendida y en la que se abre desde el árbol de decisión las ediciones se pueden deshacer desde el principio. Por eso el aprendizaje automático deja sus pasos en el historial, como ya hacía el interactivo: 14 con `asia10K.csv` y 67 con `alarm10k.csv`, medidos con escalada, y todos se deshacen sin errores. |
| **Módulo** | core, gui, learning.gui |
| **Paquete** | org.openmarkov.core.action.base; org.openmarkov.gui.window |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/PNESupport.java:55-58,196-212,223-232`<br>`core/src/main/java/org/openmarkov/core/action/base/PNEdit.java:94-96`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:116`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.jav` |

| | |
|---|---|
| | a:97,103-118,126-129,178<br>gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:161-186<br>learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:903-905<br>gui/src/main/java/org/openmarkov/gui/window/decisiontree/DecisionTreeEditor.java:270-281<br>gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EditorInputHandler.java:714 |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 35, nota 177 |

**Qué pasa.** Hay pestañas de red que se abren con `NetworkFileHandler.createNewFrame` o `openNetwork(ProbNet)` sin encender el deshacer. En ellas, mover o borrar nodos, añadir o quitar enlaces, etc. no se apunta y la opción Deshacer queda deshabilitada, hasta que el usuario hace alguna de las acciones que lo encienden (por ejemplo, crear un nodo o abrir las propiedades de la red); a partir de ahí funciona el resto de la sesión. La consecuencia en la ventana (Deshacer deshabilitado) se ha deducido de la lectura y de la medida sobre `PNESupport`, no se ha visto con la aplicación.

Esas pestañas son:

- «Expand network» (menú contextual del lienzo en modo inferencia sobre una red temporal): `InferenceHandler.expandNetwork` abre en una pestaña nueva el resultado de `TemporalNetOperations.expandNetwork`, que parte de `probNet.copy()`.
- El aprendizaje automático: `LearningDialog` (línea 904) abre la red que devuelve `LearningManager.getLearnedNet()`, creada con `new ProbNet()`; solo el aprendizaje interactivo enciende el deshacer.
- La opción de abrir la red asociada a un nodo del árbol de decisión (`DecisionTreeEditor.openAssociatedNetwork`); no se comprobó de dónde sale esa red.

Por qué pasa: cada red (`ProbNet`) tiene su gestor de ediciones, `PNESupport` (módulo `core`), que nace con `withUndo = false`; `PNEdit.executeEdit` solo apunta una edición en el historial de deshacer si ese indicador está encendido (líneas 94-96). Como `ProbNet` crea siempre un `PNESupport` nuevo (línea 116), también una copia de una red nace con el deshacer apagado.

Hoy encienden el deshacer: `NetworkFileHandler` al crear una red nueva (línea 105) y al abrir un fichero o una dirección de internet (URL, línea 182, que también usa la reapertura de la sesión anterior y la apertura desde la línea de órdenes); y además `EditorInputHandler.createNode` (línea 722, al crear un nodo con doble clic o desde el menú del lienzo), `PotentialEditPanel` (línea 157, al editar un potencial), `NetworkPropertiesDialog` (línea 82), `ReorderVariablesDialog` (línea 31), `AddFindingDialog` (línea 90) e `InteractiveLearningDialog` (línea 86).

Medido: tras encender el deshacer en una red y copiarla con `copy()`, la copia tiene `withUndo=false`; añadir un nodo a la copia deja `getCanUndo()=false`, y en la original `true`.

En las pestañas de red expandida o aprendida hay además otro defecto, el indicador de modificación que no cambia nunca, descrito en error 35.

**El código.** gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:134-137

```
    void openNetwork(ProbNet probNet) {
        NetworkEditorPanel newNetworkEditorPanel = this.createNewFrame(probNet, null);
        networkPanels.add(newNetworkEditorPanel);
```

```
    }
```

**Cómo arreglarlo.** Encender el deshacer en un único sitio por el que pasen todas las pestañas de red: `NetworkFileHandler.createNewFrame` (líneas 111-126), que usan la creación, la apertura, la expansión, el aprendizaje y el árbol de decisión. Así sobran las llamadas de las líneas 105 y 182. Si el aprendizaje automático no debe dejar deshacer sus pasos, encenderlo después de `learningManager.learn()`.

Las llamadas repartidas por diálogos (`PotentialEditPanel`, `NetworkPropertiesDialog`, `ReorderVariablesDialog`, `AddFindingDialog`, `EditorInputHandler`) quedarían redundantes; quitarlas es una limpieza aparte. Si se hace, la propuesta de diseño de nota 177 (extraer de la ventana un documento de red que lleve, entre otras cosas, el estado de modificado) es el lugar natural; allí no hay nada que corregir.

Prueba: abrir en una pestaña una red copiada con `copy()`, ejecutar una edición y comprobar que `getCanUndo()` es verdadero.

### Error 239 · [Corregido] Quitar un enlace hacia un nodo de azar definido por una fórmula cambia la fórmula por una relación uniforme, aunque la fórmula no use ese padre, y la uniforme sigue nombrando al padre quitado

| | |
|---|---|
| **Estado** | Corregido en el commit `03ea0d9` (15-10-2026): `RemoveLinkEdit` solo cambia por una uniforme una tabla que sin el padre suma cero, y cualquier otra relación queda como la deja su propio `removeVariable`; la fórmula que no usa al padre queda sobre las variables que quedan. Arregla también las regresiones lineales: «A + 2 · B» quedaba uniforme al quitar B, y ahora queda «A» |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.base.linkEdits; org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:118-166`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/FunctionPotential.java:198-216` |
| **Comprobado** | Ejecutándolo por programa; no en la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 99 |

**Qué pasa.** Medido el 15-10-2026 con un pequeño programa de prueba: una red bayesiana con tres nodos de azar numéricos, `A`, `B` y `N`, los enlaces A → N y B → N, y en `N` la fórmula «A + 1». Quitar el enlace B → N con `RemoveLinkEdit`, que es lo que hace el editor al borrar un enlace, deja en `N` un `UniformPotential` sobre `N`, `A` y `B`. Se pierde la fórmula, sin aviso, aunque no usaba `B`, y la nueva relación nombra a `B`, que ya no es padre de `N`. Deshacer devuelve la fórmula.

Por qué pasa: `FunctionPotential.removeVariable` (líneas 198-216) dice que conserva la fórmula si no usa la variable quitada, pero en ese caso devuelve una copia con las mismas variables, la quitada incluida (línea 215). Después `RemoveLinkEdit.doEdit` (líneas 152-163, un «Temporal patch») cambia por una uniforme toda relación que no se pueda pasar a tabla o cuya tabla sume cero. Una fórmula sobre variables numéricas no se puede pasar a tabla, así que acaba siempre uniforme, con las variables que traía. A las demás relaciones que no se pueden pasar a tabla les pasa lo mismo; en las redes de sucesos eso es parte del error 99.

**Cómo arreglarlo.** Que `FunctionPotential.removeVariable` devuelva la fórmula sin la variable quitada, y que `RemoveLinkEdit` no cambie por uniforme lo que la propia relación ya ha resuelto: por ejemplo, aplicar el parche solo a las tablas.

Prueba: el caso medido deja en `N` la fórmula «A + 1» sobre `N` y `A`; si la fórmula usa `B`, queda la fórmula por omisión sobre `N` y `A`, que es lo que dice el comentario de `removeVariable`; y deshacer y rehacer devuelven cada estado.

### Error 243 · [Corregido] Elegir un dominio estándar con otro número de estados da un error inesperado y no cambia nada cuando el nodo, o uno de sus hijos, no tiene relación, como pasa con toda decisión sin política impuesta y con el nodo que se observa antes de ella

| | |
|---|---|
| **Estado** | Corregido en el commit `19792a8` (17-10-2026): `NodeReplaceStatesEdit` guarda la lista entera de relaciones del nodo y de cada hijo, que puede estar vacía, con una copia de cada una, y al deshacer la repone con `setPotentials`; con eso un nodo con varias relaciones también las recupera todas. Medido con la prueba nueva, un diagrama de influencia hecho a mano con dos decisiones sin relación (Disease → Result ← Do test?, Result → Therapy): la edición con un estado más sobre «Therapy», sobre «Result» y sobre «Do test?» se hace, y al deshacerla el nodo recupera sus estados, las decisiones siguen sin relación y «Disease» y «Result» tienen sus tablas de antes. Que las dos ediciones compartan la fotografía de relaciones sigue en el error 24. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/NodeReplaceStatesEdit.java:43-59,66-80`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:313-318`<br>`core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:80,92`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:136-141,200-204`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:952-962,1121-1131` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 24, error 29, error 210, error 260 |

**Qué pasa.** En la pestaña del dominio del diálogo de un nodo, elegir un dominio estándar con otro número de estados («Standard domains», o la lista de dominios estándar) acaba en la ventana de error inesperado, y el dominio se queda como estaba. Pasa siempre que el nodo, o alguno de sus hijos, no tiene ninguna relación. En un diagrama de influencia eso es lo corriente: una decisión sin política impuesta no tiene relación, así que falla con cualquier decisión y con cualquier nodo del que cuelgue una decisión, como el resultado de una prueba que se conoce antes de decidir el tratamiento.

Medido el 15-10-2026 sin ventana, ejecutando la edición que lanza el diálogo (`NodeReplaceStatesEdit`) con un dominio de otro tamaño sobre cada nodo de estados finitos de `0_miscellaneous/networks/id/ID-decide-test.pgmx`:

```
Disease (chance, 2 states before) | without relation: none | done, now 3 states
Result of test (chance, 3 states before) | without relation: [Therapy] | THROWS
↪  NullPointerException at NodeReplaceStatesEdit.lambda$new$0:53, states still 3
```

```
Therapy (decision, 2 states before) | without relation: [itself] | THROWS
↪  NullPointerException at NodeReplaceStatesEdit.<init>:51, states still 2
Do test? (decision, 2 states before) | without relation: [itself, Therapy] | THROWS
↪  NullPointerException at NodeReplaceStatesEdit.<init>:51, states still 2
```

Tres de sus cuatro nodos de estados finitos no admiten el cambio. Contado en las 148 redes que se leen de `0_miscellaneous/networks` (sin ejecutar la edición en cada una, solo mirando qué nodos no tienen relación): en 128 hay al menos un nodo así, y son 605 de sus 1987 nodos de estados finitos (432 decisiones y 173 nodos de azar).

Que la excepción llega a `OMExceptionHandler` y sale como error inesperado está leído: el botón y la lista llaman a la edición desde `NodeDomainValuesTablePanel` (líneas 952-962 y 1121-1131), que solo captura `DoEditException`. En un modelo temporal, el mismo método cambia después los nodos gemelos de los otros ciclos; si falla uno de ellos, los anteriores ya han cambiado. Esto último está leído y no medido.

Por qué pasa: lo trae el arreglo parcial del error 24 (`461cd46`, 12-10-2026). El constructor de `NodeReplaceStatesEdit` copia ahora, para poder deshacer, la relación del nodo y la de cada hijo con `getPotential().copy()`, sin mirar si la hay. `Node.getPotential` devuelve `null` cuando la lista de relaciones está vacía (líneas 313-318). Antes de ese commit el constructor no guardaba nada y el cambio funcionaba, porque `VariableStateOperations.replaceStates` sí mira si la lista está vacía, para el nodo (línea 80) y para cada hijo (línea 92); esto está leído en el código anterior. La prueba del arreglo, sobre `NodeStatesReplaceTest.pgmx`, no lo ve porque allí todos los nodos tienen relación.

Además, `undo` repone solo una relación por nodo (`setPotential`), así que un nodo con varias se quedaría con la primera; no se ha buscado ninguna red que lo tenga.

**El código.** core/src/main/java/org/openmarkov/core/action/core/NodeReplaceStatesEdit.java:49-54

```
if (this.numberOfStatesChanged) {
    this.potentialsByNode = new HashMap<>();
    this.potentialsByNode.put(this.node, node.getPotential().copy());
    node.getChildren().forEach(child ->
                                       this.potentialsByNode.put(child,
                                       ↪  child.getPotential().copy()));
    this.oldLinks =
    ↪  this.node.getProbNet().getLinks(node).stream().map(CloneUtils::safeClone).toList();
```

**Cómo arreglarlo.** Guardar la lista entera de relaciones de cada nodo, que puede estar vacía, con una copia de cada una, y reponerla con `setPotentials`, como ya hace `NodeStateEdit` (líneas 136-141 al construirse y 200-204 al deshacer), que tuvo el mismo fallo con un hijo sin relación y lo tiene corregido (error 29). Lo más limpio es que las dos ediciones compartan esa fotografía, como propone el error 24.

La copia que se guarde conserva el comentario de la relación en todas las clases desde `52033cb` (error 260).

Prueba: sobre `ID-decide-test.pgmx`, ejecutar `NodeReplaceStatesEdit` con un estado más sobre «Therapy» y sobre «Result of test»; las dos ediciones se hacen, y al deshacerlas la decisión sigue sin relación y los demás nodos tienen las suyas de antes.

### Error 258 · [Corregido] Pegar en una red que ya tiene un nodo `X` un trozo que trae `X` y `X'` junta las dos variables en las fórmulas del trozo: una fórmula pasa a nombrar dos veces la misma, y con una distribución por tabla el pegado se cae y deja nodos sueltos que no se pueden deshacer

| | |
|---|---|
| **Estado** | Corregido en el commit `aa4bcfb` (15-10-2026): `PasteEdit` cambia las variables de cada relación empezando por los nombres más largos, de modo que ninguna coincide a medio camino con otra; la fórmula pegada es `{X'}+2*{X''}` y la distribución por tabla se pega sin caer. No se ha cambiado que `MultiStepEdit` deshaga los pasos dados ante cualquier excepción (error 28) |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.action; org.openmarkov.core.expression |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:70-78,114-128`<br>`core/src/main/java/org/openmarkov/core/expression/VariableExpression.java:17-24,32-49`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithFunctions.java:255-265`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java:341-349`<br>`core/src/main/java/org/openmarkov/core/action/base/MultiStepEdit.java:38-55` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición de pegar, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 234, error 21, error 28, error 257 |

**Qué pasa.** `X` y `X'` conviven en una red en cuanto se pega `X` en su propia red: el pegado recibe un apóstrofo. Si después se copian los dos, junto con un nodo cuya fórmula los nombra a ambos, y se pegan en otra red que ya tiene un `X`, el nodo pegado queda mal:

- una fórmula como `{X}+2*{X'}` sale como `{X''}+2*{X''}`: el nodo vale tres veces `X''` en vez de `X' + 2·X''`, sin ningún aviso;
- una distribución por tabla (`DistributionTablePotential`, la que da en una red de sucesos el tiempo hasta un suceso) escrita con la misma fórmula hace caer el pegado con `IllegalStateException: Duplicate key X'`. Lo ya pegado se queda en la red: los nodos y los enlaces, con el nodo de la fórmula sin ninguna relación, y deshacer no lo alcanza.

Medido el 15-10-2026 sin ventana, con `PasteEdit`, pegando `X`, `X'` y `Chain` (con la fórmula `{X}+2*{X'}`) en una red que solo tiene `X`, con la lista de variables de la relación en el orden `Chain, X, X'` y en el inverso:

```
Chain, formula: nodes [X, X', X'', Chain], says {X''}+2*{X''}
Chain reversed, formula: nodes [X, X', X'', Chain], says {X'}+2*{X''}
Chain, DTP: THROWS java.lang.IllegalStateException / root java.lang.IllegalStateException:
↪  Duplicate key X' (...)
Chain, DTP: nodes left [X, X', X'', Chain], can undo false, Chain says [], links 2
Chain reversed, DTP: nodes [X, X', X'', Chain], says [{X'}+2*{X''}]
```

En la ventana, la excepción llega a `OMExceptionHandler` y sale como error inesperado; esto está leído.

Por qué pasa: `PasteEdit` da a cada nodo pegado un nombre libre añadiendo apóstrofos (líneas 70-78), así que `X` baja como `X'` y `X'` como `X''`. Luego, en cada relación, cambia las variables

por las pegadas **una a una y en el orden de su lista** (líneas 120-128), y desde `0883462` (error 234) cada cambio reescribe las fórmulas con `VariableExpression.replacing` (líneas 32-49), que cambia el texto `{X}` por `{X'}` y se queda con una sola variable por nombre. El primer cambio deja `{X'}+2*{X'}`, donde las dos ya no se distinguen, y el segundo, `X'` por `X''`, cambia las dos. En la distribución por tabla, el segundo cambio construye antes una fórmula sobre la lista de padres numéricos (`TableWithFunctions.replaceVariableInFunctions`, líneas 255-265), que en ese momento tiene dos variables llamadas `X'`, y el constructor de `VariableExpression` (línea 19) las mete en un mapa por nombre con `Collectors.toMap`, que se niega. Como la excepción no es una `DoEditException`, `MultiStepEdit` no deshace los pasos ya dados (líneas 38-55): es la misma maquinaria del error 28. Con el orden inverso sale bien, porque `X'` se cambia antes de que nadie más se llame así.

**El código.** gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:120-128

```
for (int i = 0; i < potential.getNumVariables(); ++i) {
    String variableName = potential.getVariable(i).getName();
    if (PasteEdit.this.newVariables.containsKey(variableName)) {
        Variable variable =
↪   this.probNet.getVariable(PasteEdit.this.newVariables.get(variableName));
        potential.replaceVariable(i, variable);
    } else {
        externalVars.add(potential.getVariable(i));
    }
}
```

**Cómo arreglarlo.** Cambiar las variables en un orden en que ningún nombre nuevo coincida con uno viejo que aún no se ha cambiado. Basta con empezar por los nombres más largos: un nombre nuevo solo gana apóstrofos, así que es siempre más largo que el viejo, y quien se llama como otro va a llamarse tiene un nombre más largo y ya se ha movido. La alternativa más general es cambiarlas todas de una vez, con un mapa de viejas a nuevas, en las relaciones que guardan fórmulas. Y que `MultiStepEdit` deshaga los pasos dados ante cualquier excepción, como propone el error 28.

Prueba: pegar con `PasteEdit` los nodos `X`, `X'` y uno con la fórmula `{X}+2*{X'}` en una red que ya tiene `X`, y comprobar que la fórmula pegada es `{X'}+2*{X''}`; lo mismo con una distribución por tabla escrita con esa fórmula, que además no debe lanzar ninguna excepción.

## Error 271 · Copiar un nodo temporal, como «Health [1]», y pegarlo en la misma red se rechaza diciendo que ya hay una variable con ese nombre, porque el apóstrofo que debía distinguir la copia se pierde al ponerle el nombre

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.action; org.openmarkov.core.model.network |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:70-77`<br>`core/src/main/java/org/openmarkov/core/model/network/Variable.java:400-402,594-628` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición de pegar, sin ventana); el mensaje que se ve está leído en el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 21, error 234, error 257, error 258 |

**Qué pasa.** En un modelo temporal, como un diagrama de influencia de Markov (MID), copiar un nodo de un ciclo —«Health [1]»— y pegarlo en la misma red no pega nada: la edición se rechaza, y por lo leído en el código la ventana dice «There is already a variable named Health [1] in the network.». Con un nodo que no es temporal, «Health», el pegado sale bien y la copia se llama «`Health'`». En un modelo temporal casi todos los nodos son temporales, así que copiar y pegar dentro de la misma red no sirve allí.

Por qué pasa: `PasteEdit` (líneas 70-77) busca un nombre libre añadiendo apóstrofos al del nodo copiado —«`Health [1]'`»— y se lo pone a la variable nueva con `Variable.setName`. Pero `setName` (líneas 400-402) no guarda el texto tal cual: lo parte en nombre base y ciclo (`parseNameIntoParts`, líneas 594-615), tomando como ciclo el número que hay entre el último « [» y el último «]», y vuelve a escribir el nombre a partir de esas dos partes (`rebuildFullName`, líneas 622-628). Lo que queda detrás del «]» se pierde: «`Health [1]'`» pasa a ser «Health [1]», el nombre del original, y la restricción de nombres distintos rechaza el nodo.

Medido el 15-10-2026 con un pequeño programa sobre el jar del 14 de octubre (`PasteEdit` y `Variable` no han cambiado después), con un MID de un solo nodo:

```
Health [1] | setName("Health [1]'") gives «Health [1]»
Health [1] | THROWS DoEditException$CannotDoEditException, caused by
↪  ConstraintViolatedException$VariableNameIsAlreadyPresent
Health [1] | nodes after [Health [1]]
Health     | setName("Health'") gives «`Health'`»
Health     | pasted; nodes [Health, Health']
```

**Cómo arreglarlo.** Dar a la copia un nombre que `setName` conserve. Para un nodo temporal, lo natural es añadir el apóstrofo al nombre base y no al nombre entero: «`Health' [1]`», que se lee como el nombre base «`Health'`» en el ciclo 1. Así la copia sigue siendo del mismo ciclo y el nombre que el usuario ve es el que se ha buscado libre. Conviene buscar el nombre libre con la misma regla con que se va a escribir (comprobando `containsVariable` sobre el nombre ya reconstruido), para que no vuelva a pasar con otro nombre que `setName` cambie. Las fórmulas que nombran la variable pegada se reescriben con el nombre nuevo, así que el cambio debe llegar también allí (error 258).

Prueba: en un MID con «Health [1]», pegar con `PasteEdit` ese nodo en la misma red: la edición se hace, la red tiene dos nodos y la copia está en el ciclo 1 con un nombre distinto del original.

### Error 275 · [Corregido] Al borrar el enlace de un padre, la relación del hijo pierde el comentario que le escribió el usuario si es una tabla, una regresión o algo que se vuelve uniforme, y lo conserva si es una fórmula o un modelo canónico

| | |
|---|---|
| **Estado** | Corregido en el commit `ecbd410` (17-10-2026): toda relación que fabrica `removeVariable` (la general de `Potential`, `TablePotential`, `LinearCombinationPotential`, `FunctionPotential`, `TreeADDPotential` y `ExactDistrPotential`), y toda la que fabrica `RemoveLinkEdit` por su cuenta (la uniforme de una tabla que suma cero, la relación por omisión de una utilidad, la suma de utilidades), se lleva el comentario, el criterio y las propiedades de la vieja con `copyAttributesFrom`, como ya hacían los modelos canónicos. Con eso también los conserva un nodo pegado sin su padre. Medido con doce clases de relación, en nodos de azar y de utilidad, al quitar el enlace y al pegar: todas conservan el comentario y las propiedades; antes solo los modelos canónicos y la suma de utilidades, y la fórmula que no nombra al padre conservaba el comentario. En las redes del repositorio la relación de una utilidad no lleva criterio propio (lo lleva su variable), así que copiarlo no cambia la inferencia. Con prueba. No se ha tocado `addVariable`. |
| **Módulo** | core |

| | |
|---|---|
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.core.action.base.linkEdits |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:137`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:161-175`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/LinearCombinationPotential.java:183-212`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:726-736`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/FunctionPotential.java:207-221` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 239, error 260 |

**Qué pasa.** Cuando se borra un enlace (o el nodo padre), la relación del hijo pierde a ese padre con su propio `removeVariable` (`RemoveLinkEdit`, línea 137), y lo que sale es una relación nueva. Que conserve el comentario que el usuario le escribió depende de su clase:

- una tabla lo pierde: `TablePotential.removeVariable` (líneas 161-175) es una proyección, que hace una tabla nueva sin comentario;
- una regresión lo pierde: `LinearCombinationPotential.removeVariable` (líneas 183-212) construye una regresión nueva con sus covariables y coeficientes, y nada más;
- lo que se vuelve uniforme lo pierde (un árbol, una distribución, una regresión exponencial): `Potential.removeVariable` (líneas 726-736) hace un `UniformPotential` nuevo;
- una fórmula que no nombra al padre lo conserva desde `03ea0d9` (error 239), que se lo pone a mano (`FunctionPotential.removeVariable`, línea 217), pero no el criterio ni las propiedades; una que lo nombra pasa a la fórmula por omisión y lo pierde;
- un modelo canónico lo conserva todo, con `copyAttributesFrom`.

Medido el 15-10-2026 con las clases de `03ea0d9` delante del jar del 14 de octubre: una tabla con el comentario «a note», al quitarle un padre, sale con el comentario vacío; una regresión, igual; una fórmula que no nombra al padre sale con «a note». Deshacer lo devuelve. El comentario se pierde sin aviso, y el fichero que se guarde después ya no lo lleva. Es de la familia del error 260, en el que lo perdían las copias de cinco clases de relación (corregido en `52033cb`).

**Cómo arreglarlo.** Que toda relación que sale de quitar una variable se lleve el comentario, el criterio y las propiedades de la que había, como ya hacen los modelos canónicos: `copyAttributesFrom(this)` en `Potential.removeVariable`, en `TablePotential.removeVariable` (sobre la tabla proyectada), en `LinearCombinationPotential.removeVariable` y en `FunctionPotential.removeVariable`, en lugar de solo el comentario. Lo mismo conviene en `addVariable`.

Prueba: en `RemovingAParentKeepsAFormulaOrARegressionTest`, poner un comentario a una tabla y a una regresión, quitarles un padre con `RemoveLinkEdit` y comprobar que lo conservan.

### Error 280 · [Corregido] Añadir o quitar un estado a una decisión con política impuesta, o a un padre de esa decisión, deja la tabla de la política con las casillas de antes: evaluar la red acaba en la ventana de error no previsto y la red guardada después ya no se abre

| | |
|---|---|
| **Estado** | Corregido en el commit `ab4261e` (16-10-2026), con la opción que eligió Manuel Arias ese día: al añadir o quitar un estado a una decisión con política impuesta, o a un padre suyo, la decisión pierde la política, como con «Remove policy», y la evaluación la optimiza; una decisión sin política no cambia. La regla está en `VariableStateOperations.setUniformPotential`, por donde pasan las dos tablas de estados de la pestaña de dominio y los gemelos de otros ciclos. Medido con el arreglo sobre el diagrama de abajo: el fichero guardado se vuelve a abrir en todos los casos; con U dependiendo solo de D, añadir un estado a X deja D sin política y la utilidad esperada pasa de 1,7 (la de la política impuesta) a 2,0 (la de la decisión óptima), también en el fichero reabierto. Deshacer devuelve la política. «Standard domains» (`replaceStates`) sigue poniendo una política uniforme: la regla común es la que pide el error 243. Con prueba. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network; org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:116-128,130-166,251-272`<br>`core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:69-110`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:151-160`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:83-90`<br>`gui/src/main/java/org/openmarkov/gui/component/DiscretizeTablePanel.java:735,773`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/PrefixedKeyTablePanel.java:154,174` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición, la evaluación, y el guardado y la lectura del fichero, sin ventana; el diálogo de propiedades de la decisión se abrió de verdad y acepta sin aviso) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 243, error 29, error 30, error 24, error 56, error 38, error 288 |

**Qué pasa.** Una decisión puede tener una política impuesta: una tabla que dice qué opción se elige en cada caso, y que la evaluación usa en vez de optimizar. Si el usuario añade o quita un estado a esa decisión en la pestaña de dominio de sus propiedades (botones de añadir y quitar de la tabla de estados), la tabla de la política se queda como estaba, con las casillas de los estados de antes. Lo mismo si el estado se añade o se quita a un padre de la decisión, porque la política también depende de él. Nada avisa: el diálogo acepta. Después, evaluar la red (utilidad esperada, políticas óptimas, árbol de decisión) acaba en un `ArrayIndexOutOfBoundsException` —la ventana de error no previsto—, y si la red se guarda, el fichero no se vuelve a abrir, porque el lector rechaza una tabla cuyo tamaño no es el producto del número de estados de sus variables.

Por qué pasa. `NodeStateEdit.doEdit` (líneas 151-160) llama a `VariableStateOperations.modifyState`; para añadir y quitar (`addStateAt`, líneas 116-128, y `removeStateAt`, líneas 130-166) la relación del nodo y las de sus hijos se sustituyen por uniformes en `setUniformPotential` (líneas 251-272), pero ese método se salta los nodos de decisión (líneas 256 y 264), tanto el propio nodo como los hijos: una decisión sin política no tiene relación y no hay nada que cambiar, pero una con política impuesta se queda con la tabla antigua. El camino de los dominios estándar, `replace`

`States` (líneas 69-110), no hace esa distinción: pone uniforme toda relación que exista, también la política.

Medido el 15-10-2026 con el código de `3a93386`, sobre un diagrama de influencia X → D, D → U, X → U con la política impuesta D = X en D:

```
ADD estado a D      | D pasa a [c, 0, 1]; la política sigue con dims [2, 2] y valores [1,
↪  0, 0, 1]
                    | evaluar: ArrayIndexOutOfBoundsException: Index 4 out of bounds for
                    ↪  length 4
                    | guardar y abrir: «Argument table (4) is invalid because its length
                    ↪  must be 6, the product of the numbers of states of the variables»
REMOVE estado de D  | D pasa a [0]; la política sigue con dims [2, 2]
ADD estado a X      | X pasa a [c, 0, 1]; la política de D sigue con dims [2, 2]; evaluar:
↪  ArrayIndexOutOfBoundsException
D sin política      | la decisión sigue sin relación: nada que corregir
```

El diálogo de propiedades de la decisión con la política desfasada se abre y acepta con «OK» sin avisar (medido abriéndolo de verdad). El rechazo al abrir el fichero lo da el constructor de `TablePotential` (líneas 83-90). Mover un estado arriba o abajo no tiene el fallo: `reorderStatesInNodeAndChildren` sí reordena las políticas.

**Cómo arreglarlo.** Que `setUniformPotential` trate a una decisión que tiene relación como a los demás nodos: ponerle una política uniforme sobre los estados nuevos (lo que hace `replaceStates`) o quitarle la política impuesta; y lo mismo para los hijos que son decisiones con política. Deshacer ya repone las relaciones guardadas del nodo y de sus vecinos (`NodeStateEdit.undo`), así que no necesita cambios. Conviene que la `NodeStateEdit` y `NodeReplaceStatesEdit` compartan la regla, como pide el error 243.

Prueba: en el diagrama de arriba, `NodeStateEdit` con `ADD` y con `REMOVE` sobre D y sobre X; después de cada una, la tabla de la política de D tiene el tamaño de sus estados y los de X, `VEEvaluation` termina, y la red escrita con `PGMXWriter_1_0` se vuelve a leer.

### Error 281 · [Corregido] Al cambiar un nodo de azar a decisión en sus propiedades, la tabla de probabilidad condicionada se queda como relación del nodo y todo el programa la toma por una política impuesta: la decisión no se optimiza y la utilidad esperada es la de elegir al azar con esas probabilidades

| | |
|---|---|
| **Estado** | Corregido en el commit `164f2e2` (16-10-2026): un nodo que pasa a ser decisión se queda sin relación, como una decisión creada como tal, y la evaluación lo optimiza hasta que se le imponga una política. Visto en la ventana el 16-10-2026 sobre el diagrama de abajo, abriendo las propiedades de C, pulsando el botón de decisión y aceptando: sin el arreglo C queda con política impuesta y la utilidad esperada es 15,5; con él, C queda sin política, la evaluación calcula su política óptima y la utilidad esperada es 20. Deshacer devuelve el nodo de azar con su tabla. Con prueba. |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.action; org.openmarkov.core.model.network.potential; org.openmarkov.core.inference.tasks |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/action/ChangeNodeTypeEdit.java:48-63`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:145-152`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:118-135,149-157` |

| | |
|---|---|
| | `core/src/main/java/org/openmarkov/core/inference/tasks/TaskUtilities.java:214-238`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:574-576`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:331-347` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición, la evaluación, y el guardado y la lectura del fichero, sin ventana); lo que enseña la pestaña de la política está leído en el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 166, error 282, error 56, error 243, nota 4 |

**Qué pasa.** En la pestaña de definición de las propiedades de un nodo, los botones del tipo de nodo cambian un nodo de azar a decisión (`NodeDefinitionPanel`, líneas 331-347, que ejecutan `ChangeNodeTypeEdit`). El nodo pasa a ser una decisión, pero conserva su tabla de probabilidad condicionada como relación. Para el programa, una decisión con relación es una decisión con política impuesta: `Node.hasPolicy` (líneas 574-576) y `TaskUtilities.hasImposedPolicy` (líneas 214-238) solo miran si hay relación. Así que la evaluación no calcula la política óptima de esa decisión, sino que la trata como si el usuario hubiera impuesto «elegir al azar con las probabilidades de antes», y la pestaña de la política enseña esa tabla como política impuesta por el usuario (`TablePotentialPanel`, líneas 907-927; leído). Nada avisa.

Por qué pasa. `ChangeNodeTypeEdit.doMultiStepEdit` (líneas 48-63) cambia el tipo de nodo, cambia el tipo de variable solo si el de antes no vale para el nuevo tipo de nodo (estados finitos vale para una decisión) y conserva la relación si su clase pasa `validate` para el nodo (líneas 56-62, con `PotentialUtils.getFilteredPotentialClasses`, líneas 118-135). `TablePotential.validate` (líneas 145-152) acepta una tabla para cualquier nodo que no sea un suceso, sin mirar el papel de la tabla (`conditionalProbability`) ni el tipo de nodo. Una decisión creada como tal no tiene relación: `generateDefaultPotential` devuelve `null` para `DECISION` (líneas 149-157).

Medido el 15-10-2026 con el código de `3a93386`, sobre un diagrama de influencia A → C → U, con P(C | A) = (0,3; 0,7; 0,6; 0,4) y U(C) = (10, 20):

```
ChangeNodeTypeEdit(C, DECISION) | C sigue con TablePotential role=conditionalProbability
↪  vars=[C, A] valores [0.3, 0.7, 0.6, 0.4]
                                | hasPolicy=true, hasImposedPolicy=true
                                | VEEvaluation: no calcula ninguna política óptima
                                ↪  (optimalPolicies=[])
                                | guardada y vuelta a abrir: utilidad esperada 15,5 =
                                ↪  10·0,45 + 20·0,55, la de usar P(C|A) como política
Lo mismo quitando antes la relación (una decisión creada como tal) | optimalPolicies=[C];
↪  la utilidad óptima es 20 (elegir siempre C=1)
```

El paso contrario, de una decisión con política impuesta a nodo de azar, conserva la tabla con el papel «policy»; se guarda como probabilidad, se vuelve a leer bien y evalúa igual que un nodo de azar normal (medido), así que no es un fallo. Pasar de azar a utilidad sí sustituye la tabla por la relación por omisión de una utilidad, porque la variable pasa a numérica y la tabla deja de valer.

**Cómo arreglarlo.** Que al pasar a decisión la relación de probabilidad se trate como no válida: `ChangeNodeTypeEdit` debe dejar la decisión sin relación, que es lo que `generateDefaultPotential` da para una decisión y lo que ya hace cuando la clase de la relación no vale (`SetPotentialEdit` con la relación nueva). Si se quisiera conservar algo, solo una relación cuyo papel ya sea `POLICY`. La otra opción, que `TablePotential.validate` mire el papel de la tabla y el tipo de nodo, toca a todos los sitios que eligen clases de relación para un nodo y no hace falta.

Prueba: sobre el diagrama de arriba, `ChangeNodeTypeEdit`(C, `DECISION`): `hasPolicy` da falso y `VE Evaluation` devuelve una política óptima para C; deshacer devuelve el nodo de azar con su tabla.

### Error 282 · [Corregido] Cambiar en la pestaña de dominio el tipo de variable de una decisión sin política impuesta (a numérica, y si se quiere de vuelta a estados finitos) le deja una relación uniforme que el programa toma por política impuesta: la decisión deja de optimizarse y nada lo avisa

| | |
|---|---|
| **Estado** | Corregido en el commit `46aed4e` (16-10-2026), con la regla que eligió Manuel Arias para el error 280: el cambio de tipo de variable no da relación a una decisión, y una decisión con política impuesta la pierde, también cuando el que cambia es su padre. Medido sobre el diagrama de abajo: tras pasar D, o X, a numérica y de vuelta a estados finitos, D queda sin política; con la tabla de U escrita de nuevo (el cambio la deja uniforme: error 288), la utilidad esperada es 3 y la evaluación calcula la política óptima de D, donde antes daba 2,5 y ninguna política óptima. Deshacer devuelve la política. Con prueba. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core; org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:79-134,156-160,186-189`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/PotentialOperations.java:281-293`<br>`core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:251-272`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:906-931` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición, la evaluación, y el guardado y la lectura del fichero, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 281, error 28, error 243, error 56, error 288 |

**Qué pasa.** En la pestaña de dominio de las propiedades de un nodo, el desplegable del tipo de variable ejecuta `VariableTypeEdit`(nodo, tipo, `true`) (`NodeDomainValuesTablePanel`, líneas 906-931). Si el nodo es una decisión sin política impuesta y el cambio pasa por «numérica» —de estados finitos a numérica, o de vuelta—, la decisión sale con una relación uniforme. Como para el programa una decisión con relación tiene política impuesta (error 281), la evaluación deja de optimizar esa decisión y usa «elegir al azar»; el resultado cambia sin ningún aviso, y la pestaña de la política enseña una tabla uniforme como impuesta por el usuario (leído).

Por qué pasa. `VariableTypeEdit.doMultiStepEdit` (líneas 79-134) llama a `setPotentialsNode AndChildren` (líneas 156-160) y, al salir de numérica, al bloque de las líneas 113-133; los dos ponen una relación uniforme al nodo con `setUniformPotentialToNode` (líneas 186-189) siempre que `updatePotential` es cierto, sin mirar el tipo de nodo. Para una decisión, `PotentialOperations.getUniformPotential` (líneas 281-293) da un `UniformPotential` con papel `POLICY`: una política impuesta. El camino de los estados, `VariableStateOperations.setUniformPotential` (líneas 251-272), se salta las decisiones justo para no hacer esto.

Medido el 15-10-2026 con el código de `3a93386`, sobre un diagrama de influencia X → D, D → U, X → U con U = (1, 2, 3, 4) y D sin política:

```
antes                                  | D sin relación; VEEvaluation calcula la política
↪  óptima de D (optimalPolicies=[D])
```

```
VariableTypeEdit(D, NUMERIC)           | D: UniformPotential role=policy vars=[D, X];
↪ hasImposedPolicy=true
VariableTypeEdit(D, FINITE_STATES)     | D: estados [absent, present], sigue con la
↪ política uniforme
guardada y vuelta a abrir              | utilidad esperada 2,5 (la media de las cuatro
↪ utilidades) y ninguna política óptima; optimizando D sería 3
```

El cambio a numérica deja además a los hijos con relaciones uniformes, como corresponde; eso no es un fallo.

**Cómo arreglarlo.** Que `VariableTypeEdit` no dé relación a una decisión que no la tenía: `setUniformPotentialToNode` debe saltarse las decisiones sin relación, como hace `VariableStateOperations.setUniformPotential`; si la decisión tenía una política impuesta, sí hay que sustituirla por una uniforme sobre los estados nuevos (hoy, al pasar por numérica, se sustituye; al pasar entre estados finitos y discretizada no se toca, y los estados tampoco cambian).

Prueba: sobre el diagrama de arriba, `VariableTypeEdit` a numérica y de vuelta a estados finitos: `hasPolicy` sigue siendo falso y `VEEvaluation` calcula la política óptima de D.

### Error 283 · [Corregido] Copiar o cortar guarda en el portapapeles los propios nodos de la red, no una copia, y lo que se pega es lo que esos nodos tengan en ese momento: tras cortar dos nodos enlazados, el hijo pegado tiene el enlace pero su tabla ya no depende del padre, y la red se guarda así; copiar un nodo, editar su tabla y pegar pega la tabla editada

| | |
|---|---|
| **Estado** | Corregido en el commit `3d87c0d` (16-10-2026): «Copy» y «Cut» guardan en el portapapeles los nodos y enlaces de una copia profunda de la red, hecha al copiar (`VisualNetwork.exportToClipboard`); `PasteEdit` no cambia. Medido con las mismas llamadas que hacen los menús, sobre la red de abajo: tras cortar A y C y pegar, la relación de C nombra a A y vale [0.3, 0.7, 0.6, 0.4]; tras copiar A, cambiar su tabla o añadirle un estado y pegar, A' tiene dos estados y [0.3, 0.7]. La copia profunda funciona en las 148 redes legibles del repositorio y tarda como mucho 74 ms. Con prueba, que corre sin pantalla. |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.window.edition; org.openmarkov.gui.graphic; org.openmarkov.gui.action; org.openmarkov.core.action.base.linkEdits |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/edition/EditorPanelClipboardAssistant.java:16-41`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/SelectedContent.java:20-29`<br>`gui/src/main/java/org/openmarkov/gui/graphic/VisualNetwork.java:1153-1167`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:71-78, 115-151`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:124-135`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:117-168` |
| **Comprobado** | Leyendo el código y ejecutándolo (lo que hacen «Cut», «Copy» y «Paste» con las ediciones, y el guardado y la lectura del fichero, sin ventana; el diálogo de propiedades del nodo pegado se abrió de verdad y no avisa) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 284, error 21, error 234, error 257, error 258, error 260, error 271, error 31, |

| | |
|---|---|
| | nota 184 |

**Qué pasa.** «Edit → Copy» y «Edit → Cut» (`MainPanelListenerAssistant`, líneas 124-135) llaman a `VisualNetwork.exportToClipboard` (líneas 1153-1167), que mete en un `SelectedContent` los propios objetos `Node` y `Link` de la red. `EditorPanelClipboardAssistant` los guarda tal cual (líneas 22-24) y `paste` (líneas 29-31) devuelve una copia de las listas, no de los nodos. `PasteEdit` lee en el momento de pegar el nombre (línea 72), la variable, el tipo, la posición y las relaciones (líneas 115-151) de esos nodos, que son los de la red.

Con «Cut» las consecuencias se ven enseguida. Tras copiar, los nodos se borran con `Remove SelectedEdit` y `CRemoveNodeEdit`, que quita primero los enlaces con `RemoveLinkEdit`; esa edición sustituye la tabla del hijo por su proyección sin el padre (líneas 117-168: la tabla con el padre en su primer estado). Los nodos borrados siguen en el portapapeles con esas tablas recortadas. Al pegar, el hijo pegado tiene el enlace desde el padre pegado, pero su tabla ya no nombra al padre; el diálogo de propiedades del nodo pegado no avisa de nada, y la red se guarda con el enlace y una relación que no lo refleja. Con «Copy» el defecto es más silencioso: lo que se pega es el estado que tenga el original al pegar, no al copiar.

Medido el 15-10-2026 con el código de `3a93386` (las mismas ediciones que ejecutan los menús, sin ventana):

```
Red bayesiana A -> C, P(C|A) = [0.3, 0.7, 0.6, 0.4]
Cut de A y C      | el objeto C queda con TablePotential vars=[C] valores [0.3, 0.7]
Paste             | nodo C pegado: padres [A], relación TablePotential vars=[C] valores
↪  [0.3, 0.7] (no nombra a A)
guardar y abrir   | igual: C con padre A y una relación sin A
Copy de A ([0.3, 0.7]), editar la tabla de A a [0.9, 0.1], Paste | A' sale con [0.9, 0.1]
```

Por lo leído, pasa lo mismo si tras copiar se borra el original, se le cambian los estados o se le cambia el nombre (la copia toma el nombre nuevo con el apóstrofo). Qué hace la propagación con una red cuyo enlace no está en la relación no se ha medido.

**Cómo arreglarlo.** Copiar en el momento de copiar: `exportToClipboard` (o el propio `EditorPanelClipboardAssistant`) debe guardar copias de los nodos, con copias de sus relaciones (`Potential.copy`), y copias de los enlaces (`CloneUtils.safeClone`, como hace `NodeReplaceStatesEdit` con los suyos), de modo que lo que haya en el portapapeles no cambie con lo que le pase a la red. El constructor de copia de `Node` comparte las relaciones con el original, así que no basta por sí solo. `PasteEdit` sigue igual: ya trabaja con lo que le dan. Lo que falta copiar al pegar (restricciones, revelaciones, «always observed») está en el error 284.

Prueba: cortar A y C y pegar: la relación de C' nombra a A' y tiene los valores originales; copiar A, cambiar su tabla y pegar: A' tiene la tabla que había al copiar.

## Error 284 · [Corregido] Al pegar un trozo de red, los enlaces pegados salen sin sus restricciones ni sus condiciones de revelación (en las redes de análisis de decisiones, que son las que las admiten), y los nodos pegados sin la marca «always observed» ni, en los sucesos, «always append»

| | |
|---|---|
| **Estado** | Corregido en el commit `e5f2ac5` (16-10-2026): `PasteEdit` copia en cada enlace pegado la tabla de restricciones, con las variables pegadas, y los estados e intervalos reveladores, cuando la red de destino los admite; y en cada nodo, «always observed», «always append» y, a petición de Manuel Arias, también la marca «input» y el tipo de política. Medido con las mismas llamadas que hacen los menús, en la red de abajo: el enlace pegado tiene la restricción y |

| | |
|---|---|
| | el estado revelador, y X' «always observed», también tras guardar y abrir; pegado en una red bayesiana, que no admite restricciones ni revelaciones, se pega sin ellas. Con prueba, que corre sin pantalla. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.action |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:90-97,152-156`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:143-152`<br>`core/src/main/java/org/openmarkov/core/model/graph/Link.java:154,286,306-364`<br>`core/src/main/java/org/openmarkov/core/model/network/type/DecisionAnalysisNetworkType.java:22-23`<br>`core/src/main/java/org/openmarkov/core/model/network/type/MarkovDANType.java:20-21`<br>`core/src/main/java/org/openmarkov/core/model/network/type/TuningNetworkType.java:21` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición de pegar, sin ventana); lo de «always append» está leído |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 283, error 21, error 27, error 25, error 260 |

**Qué pasa.** `PasteEdit` crea los enlaces pegados con `AddLinkEdit` (líneas 90-97), que hace un enlace sin más, y de cada nodo copia solo el comentario, la relevancia, el propósito y las propiedades adicionales (líneas 152-156). Se quedan por el camino las restricciones del enlace (la tabla de compatibilidad entre los estados de los dos nodos), sus condiciones de revelación (los estados o intervalos del padre que revelan al hijo) y la marca «always observed» del nodo; en un nodo de suceso, también «always append» (leído). El constructor de copia de `Node` (líneas 143-152) sí copia «always observed», pero pegar no pasa por él.

Las restricciones y las revelaciones solo existen en las redes de análisis de decisiones (DAN), en las DAN de Markov y en las redes de ajuste, que son los tipos que quitan las restricciones `NoLinkRestriction` y `NoRevelationArc` (líneas 22-23, 20-21 y 21 de sus clases); en los demás tipos no hay nada que perder.

Medido el 15-10-2026 con el código de `3a93386`, en una DAN X → Y con una restricción (X = 0 incompatible con Y = 0), el estado X = 1 como revelador e X marcado «always observed», pegando X e Y con su enlace en la misma red:

```
original  | enlace X -> Y: hasRestrictions=true, revealing=[state 1]; X alwaysObserved=true
pegado    | enlace X' -> Y': hasRestrictions=false, revealing=[]; X' alwaysObserved=false;
↪  la tabla de Y' sí se copia
```

**Cómo arreglarlo.** Después de crear los enlaces, copiar en cada uno la tabla de restricciones (`getRestrictionsPotential`, con las variables cambiadas por las pegadas, como ya se hace con las relaciones), los estados reveladores (los de la variable pegada con los mismos índices) y los intervalos reveladores; y en cada nodo, `setAlwaysObserved` y `setAlwaysAppend`. La comprobación final de la red (`PasteEdit`, líneas 161-171) ya se encarga de rechazar lo que el tipo de red no admita. Si el error 283 se arregla copiando al copiar, esas copias deben llevarse también todo esto.

Prueba: en una DAN, pegar X → Y con una restricción, un estado revelador y «always observed»: el enlace y el nodo pegados los tienen.

### Error 285 · [Corregido] Borrar el enlace de un padre de azar o de decisión a un nodo de utilidad, o borrar ese padre, cambia la tabla de la utilidad por una relación uniforme; desde entonces ninguna evaluación funciona, con un mensaje que pide poner un hallazgo en la utilidad, hasta que se vuelve a escribir su tabla. Un nodo de azar conserva su tabla proyectada

| | |
|---|---|
| **Estado** | Corregido en el commit `7fe37a0` (16-10-2026), con la opción que eligió Manuel Arias ese día: la utilidad conserva sus valores con el padre quitado en su primer estado, como un nodo de azar. Lo que dice abajo esta ficha de `ExactDistrPotential` no es cierto: no redefine `removeVariable` y daba una uniforme; en las utilidades se proyecta sobre ese estado, y una relación que aun así no pueda quitar al padre toma la de una utilidad nueva. El cambio se limita a las utilidades; lo mismo en los nodos de azar numéricos es el error 289. Medido sobre el diagrama de abajo: quitar A → U deja U(B) = (0, 20), con tabla y con relación exacta, y la utilidad esperada en 10, también tras guardar y abrir; en `ID-decide-test.pgmx`, borrar «Disease» deja «Health state» en (10, 9) y la utilidad esperada en 10, y quitar Therapy → Cost of therapy deja una red que se evalúa (9,44). Deshacer devuelve la relación. La prueba `RemoveLinkEditPropertyTest`, que exigía la uniforme, exige ahora que no lo sea. Con prueba. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.base.linkEdits |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:91-116,117-168`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UniformPotential.java:92-104`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:149-157`<br>`core/src/main/java/org/openmarkov/core/action/core/CRemoveNodeEdit.java:62-81` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición, la evaluación, y el guardado y la lectura del fichero, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 26, error 239, error 275, error 229, error 99, error 288 |

**Qué pasa.** Al borrar un enlace que llega a un nodo de utilidad (herramienta de borrar sobre el enlace, o borrar el nodo padre, que pasa por la misma edición: `CRemoveNodeEdit`, líneas 62-81), `RemoveLinkEdit.doEdit` decide la relación nueva de la utilidad en las líneas 91-116: si todos los padres que quedan son numéricos, un nodo de supervalor (una suma); si queda algún padre de azar o de decisión, o no queda ninguno, un `UniformPotential` (línea 111). La tabla de utilidades se tira. Un nodo de azar, en cambio, conserva su tabla con el padre fijado en su primer estado (líneas 117-168, `removeVariable`).

Una relación uniforme sobre una variable numérica, que es lo que es una utilidad, no se puede convertir en tabla (`UniformPotential`, líneas 92-104): desde ese momento la utilidad esperada, las políticas óptimas, el árbol de decisión y el coste-efectividad fallan con «The numeric variable U has a uniform distribution, so it has no value to compute with. Set a finding on it in edition mode and try again.», el mensaje que el error 229 puso para los nodos numéricos de azar, y que aquí pide un hallazgo sobre una utilidad. El usuario tiene que abrir el nodo, volver a elegir «Table» y escribir los valores. El fichero guardado conserva la uniforme y falla igual al abrirlo.

Medido el 15-10-2026 con el código de `3a93386`, sobre un diagrama de influencia A → U, B → U con U = (0, 10, 20, 30):

```
quitar A -> U        | U: UniformPotential vars=[U, B]; VEEvaluation y getCPT lanzan «The
↪  numeric variable U has a uniform distribution, so it has no value to compute with. Set
↪  a finding on it in edition mode and try again.»
guardar y abrir      | igual
quitar también B -> U| U: UniformPotential vars=[U]; igual
utilidad nueva sin padres | ExactDistrPotential con valor 0: evalúa
nodo de azar C que pierde a su padre A | TablePotential vars=[C] con los valores de A en su
↪  primer estado
```

Añadir un padre a una utilidad hace lo contrario: `AddLinkEdit` extiende la tabla con `addVariable` y conserva los valores. El error 239 y el error 275 son de la misma familia: lo que una relación pierde al quitarle una variable.

**Cómo arreglarlo.** Hacer con las utilidades lo que ya se hace con los nodos de azar: `removeVariable` sobre la relación que había (una tabla o una `ExactDistrPotential` conservan los valores con el padre en su primer estado). Si se prefiere no inventar valores, la relación por omisión de una utilidad nueva (`generateDefaultPotential`, líneas 149-157: una `ExactDistrPotential` a cero), que al menos evalúa. En ningún caso una uniforme, que ninguna utilidad puede usar. El caso de supervalor (solo padres de utilidad) se queda como está.

Prueba: en el diagrama de arriba, quitar A → U y después B → U: tras cada edición la relación de U es una tabla sobre sus padres y `VEEvaluation` termina; deshacer devuelve la tabla original.

### Error 286 · [Corregido] En una red de análisis de decisiones, cambiar el tipo de variable de un nodo entre «estados finitos» y «discretizada», que conserva sus estados y sus relaciones, borra sin aviso las restricciones y las condiciones de revelación de todos sus enlaces

| | |
|---|---|
| **Estado** | Corregido en el commit `1b7535a` (16-10-2026): `VariableTypeEdit` borra las restricciones y las revelaciones solo cuando cambian los estados, es decir, cuando el cambio pasa por «numérica» o cuando un único estado se sustituye por los estados por omisión. Medido con la edición sobre la red de abajo: de estados finitos a discretizada y vuelta, el enlace conserva la restricción y el estado revelador; a numérica, la restricción se quita y deshacer la devuelve. Con prueba. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:79-110,137-152,270-323`<br>`core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:301-329`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:906-931`<br>`core/src/main/java/org/openmarkov/core/model/network/type/DecisionAnalysisNetworkType.java:22-23` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 24, error 25, error 27, error 284 |

**Qué pasa.** `VariableTypeEdit.doMultiStepEdit` solo toca las relaciones cuando el cambio pasa por «numérica» (líneas 88-134); entre «estados finitos» y «discretizada» los estados se conservan (`RawSetType` repone los de antes) y las tablas siguen valiendo. Pero las líneas 137-152 se ejecutan con cualquier cambio: vacían las condiciones de revelación de los enlaces hacia los hijos (`RawSetRevelation`) y quitan la tabla de restricciones de todos los enlaces del nodo (`RawSetRestriction`),

aunque estén definidas sobre los mismos estados que siguen ahí. Donde sí tiene sentido borrarlas es cuando cambia el número de estados, que es lo que hace `VariableStateOperations.resetLink` (líneas 301-329) al añadir o quitar uno.

Solo afecta a los tipos de red que admiten restricciones y revelaciones: las redes de análisis de decisiones (DAN), las DAN de Markov y las redes de ajuste (error 284). El camino es la pestaña de dominio de las propiedades del nodo, desplegable del tipo de variable (`NodeDomainValuesTable Panel`, líneas 906-931).

Medido el 15-10-2026 con el código de `3a93386`, en una DAN X → Y con una restricción y un estado revelador en el enlace, y X con la tabla (0,3; 0,7):

```
antes                                    | X: estados [0, 1], Finite states; enlace:
↪  hasRestrictions=true, revealing=[state 1]
VariableTypeEdit(X, DISCRETIZED)         | X: estados [0, 1], Discretized; tabla de X
↪  [0.3, 0.7] y tabla de Y intactas; enlace: hasRestrictions=false, revealing=[]
VariableTypeEdit(X, FINITE_STATES)       | X: estados [0, 1], Finite states; el enlace
↪  sigue sin restricciones
```

**Cómo arreglarlo.** Borrar las restricciones y las revelaciones solo cuando cambian los estados —es decir, cuando el cambio pasa por «numérica», que es cuando también cambian las relaciones—, y conservarlas entre «estados finitos» y «discretizada». Las dos ediciones internas ya saben deshacerse, así que basta con no ejecutarlas en ese caso.

Prueba: en una DAN, `VariableTypeEdit`(X, `DISCRETIZED`) conserva la restricción y el estado revelador del enlace X → Y; a numérica los quita y deshacer los devuelve.

### Error 287 · [Corregido] Añadir, quitar o mover un criterio de decisión en las propiedades de la red, deshacerlo, rehacerlo y volver a deshacerlo deja el cambio puesto: la lista guardada para deshacer es la misma que la edición modifica al rehacer

| | |
|---|---|
| **Estado** | Corregido en el commit `fda22af` (16-10-2026): `undo` instala una copia de la lista guardada. Medido con la edición: añadir, quitar o bajar un criterio, deshacer, rehacer y deshacer deja los criterios del principio, «Cost» y «Effectiveness». `NetworkAgentEdit` no tiene el fallo, por lo leído: toma su copia de los agentes en cada `doEdit`, también al rehacer. Con prueba. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/DecisionCriteriaEdit.java:33-46,69-85,99-105`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:1110-1116`<br>`core/src/main/java/org/openmarkov/core/model/network/NetworkMetadata.java:118-124`<br>`core/src/main/java/org/openmarkov/core/action/base/PNEdit.java:443-453`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/DecisionCriteriaTablePanel.java:92-123`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:271-290` |
| **Comprobado** | Leyendo el código y ejecutándolo (la edición y deshacer y rehacer, sin ventana); quitar y mover están leídos |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 57, error 58, error 174, nota 36, nota 81 |

**Qué pasa.** Los criterios de decisión se editan en la pestaña de criterios de las propiedades de la red (`DecisionCriteriaTablePanel`, líneas 92-123, que ejecuta `DecisionCriteriaEdit`); «OK» cierra el diálogo y deja lo hecho como un solo paso de deshacer (`NetworkPropertiesDialog`, líneas 271-290). «Edit → Undo» lo quita y «Edit → Redo» lo vuelve a poner; pero un segundo «Undo» ya no lo quita: el criterio añadido sigue en la red, y lo mismo con uno quitado (que no vuelve) o movido (que no vuelve a su sitio). Renombrar sí se deshace bien.

Por qué pasa. El constructor guarda una copia de la lista de criterios (`lastCriteria`, línea 46) y `doEdit` (líneas 69-85) modifica la lista viva de la red (`getDecisionCriteria` devuelve la lista misma, `ProbNet`, líneas 1110-1116, y `NetworkMetadata`, líneas 118-124). `undo` (líneas 99-105) no vuelve a copiar: instala `lastCriteria` como lista de la red con `setDecisionCriteria`. Desde ese momento la copia guardada *es* la lista viva. Rehacer vuelve a ejecutar `doEdit` (`PNEdit.redo`, líneas 443-453), que añade el criterio a esa lista, y la fotografía de antes se pierde; el siguiente deshacer instala la misma lista y no cambia nada.

Medido el 15-10-2026 con el código de `3a93386`, sobre un diagrama de influencia nuevo:

```
criterios al empezar        | [Cost, Effectiveness]
añadir «new»                | [Cost, Effectiveness, new]
deshacer                    | [Cost, Effectiveness]
rehacer                     | [Cost, Effectiveness, new]
deshacer otra vez           | [Cost, Effectiveness, new]   <- debía ser [Cost,
↪  Effectiveness]
```

Quitar y mover (`REMOVE`, `UP`, `DOWN`) modifican la lista viva del mismo modo, así que les pasa lo mismo (leído). `NetworkAgentEdit` guarda y repone la lista de agentes con el mismo patrón y conviene mirarlo con la misma prueba (leído).

**Cómo arreglarlo.** Que ninguna edición comparta su lista con la red: `undo` debe instalar una copia (`new ArrayList<>(lastCriteria)`), o `doEdit` trabajar sobre una copia e instalarla con `setDecisionCriteria`. La nota 66 cuenta ediciones de opciones de la red que rehacen dos veces; aquí el problema es el contrario, pero la prueba es la misma: hacer, deshacer, rehacer y deshacer, y comparar con el principio.

Prueba: añadir un criterio, deshacer, rehacer y deshacer: la lista es la del principio; lo mismo con quitar, subir y bajar.

### Error 288 · [Corregido] Añadir o quitar un estado a un nodo de azar o de decisión, o pasar su tipo de variable por «numérica», cambia la tabla de sus hijos de utilidad por una relación uniforme; desde entonces ninguna evaluación funciona, con un mensaje que pide poner un hallazgo en la utilidad, hasta que se vuelve a escribir su tabla

| | |
|---|---|
| **Estado** | Corregido en el commit `8856458` (16-10-2026), con la opción que eligió Manuel Arias ese día: al añadir o quitar un estado a un padre, una tabla o una relación exacta de la utilidad conserva los valores de las configuraciones que siguen, con 0 en las del estado nuevo; con un dominio estándar de otro tamaño, al pasar por «numérica» y con cualquier otra relación, la utilidad toma la relación de una utilidad nueva. Nunca una uniforme. Medido en `ID-decide-test.pgmx`: añadir «maybe» a «Therapy» deja «Health state» en (0, 0, 10, 4, 9, 8) y la utilidad esperada en 9,3289, como antes, también tras guardar y abrir; quitar un estado de «Disease» deja (10, 9) y 10; un dominio estándar de tres estados en «Disease», o pasarla a numérica y de vuelta, deja «Health state» a 0 y la red se evalúa. Deshacer devuelve los valores. Con prueba. |
| **Módulo** | core |

| | |
|---|---|
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/VariableStateOperations.java:251-276,91-100`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/PotentialOperations.java:281-293`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UniformPotential.java:91-104`<br>`core/src/main/java/org/openmarkov/core/action/core/VariableTypeEdit.java:170-176` |
| **Comprobado** | Ejecutándolo (la edición, la evaluación, y el guardado y la lectura del fichero, sin ventana); el camino de los dominios estándar, leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 285, error 280, error 243, error 229 |

**Qué pasa.** Al añadir o quitar un estado a un nodo de azar o de decisión en la pestaña de dominio de sus propiedades (y en sus gemelos de otros ciclos, que pasan por la misma edición), `VariableStateOperations.setUniformPotential` (líneas 251-276) pone una relación uniforme a todos sus hijos que no son decisiones. Para un hijo de azar es la tabla uniforme de siempre, que se puede usar; un hijo de utilidad recibe también un `UniformPotential` (`PotentialOperations.getUniformPotential`, líneas 281-293), y una relación uniforme sobre una variable numérica, que es lo que es una utilidad, no se puede convertir en tabla (`UniformPotential`, líneas 91-104). Desde ese momento la utilidad esperada, las políticas óptimas, el árbol de decisión y el coste-efectividad fallan con «The numeric variable … has a uniform distribution, so it has no value to compute with. Set a finding on it in edition mode and try again.», que pide un hallazgo sobre una utilidad. El usuario tiene que abrir cada utilidad afectada, volver a elegir «Table» y escribir todos sus valores. El fichero guardado conserva la uniforme y falla igual al abrirlo y evaluarlo.

Visto al corregir el error 280. Medido el 16-10-2026 con el código de `ab4261e`:

```
ID-decide-test.pgmx, antes                  | utilidad esperada 9,3289
añadir un estado a Therapy (decisión)       | Health state y Cost of therapy pasan a
↪ UniformPotential
                                            | evaluar: «The numeric variable Cost of
                                            ↪ therapy has a uniform distribution, so it
                                            ↪ has no value to compute with. Set a finding
                                            ↪ on it in edition mode and try again.»
añadir un estado a Disease (azar)           | Health state pasa a UniformPotential;
↪ evaluar: lo mismo con «Health state»
X → D, D → U, X → U, U = (1, 2, 3, 4)       | utilidad esperada 3,4
añadir un estado a D, o a X, o quitar uno a D | U pasa a UniformPotential; evaluar falla
↪ igual; guardado y vuelto a abrir, falla igual
X → D, D → U (U no depende de X)            | añadir un estado a X no toca U y la red se
↪ evalúa
```

El camino de los dominios estándar, `VariableStateOperations.replaceStates` (líneas 91-100), hace lo mismo con los hijos que tienen relación: les pone un `UniformPotential` con sus variables (leído). Lo mismo pasa al cambiar en esa pestaña el tipo de variable de un padre de la utilidad cuando pasa por «numérica»: `VariableTypeEdit.setPotentialsNodeAndChildren` (líneas 170-176) pone un `UniformPotential` a la utilidad hija si alguno de sus padres no es numérico. Medido el 16-10-2026 con el código de `46aed4e` (visto al corregir el error 282), en X → D, D → U, X → U: pasar D, o X, a numérica y de vuelta a estados finitos deja U uniforme, y evaluar da «The utility of the node U is not defined: its relation is still the uniform one it was created with.». Es de la misma familia que el error 285, que deja una utilidad uniforme al quitarle un padre.

**Cómo arreglarlo.** Lo que pide el error 285: que una utilidad no reciba nunca una uniforme. Al cambiar el número de estados de un padre, la utilidad puede tomar la relación por omisión de una utilidad nueva (`generateDefaultPotential`: una `ExactDistrPotential` a cero) sobre sus padres, que al menos evalúa, o conservar los valores de los estados que siguen y poner cero en los nuevos. Conviene que la regla sea una sola para `setUniformPotential`, `replaceStates`, `VariableTypeEdit` y `RemoveLinkEdit`.

Prueba: en `ID-decide-test.pgmx`, `NodeStateEdit` con `ADD` sobre «Therapy» y sobre «Disease»: tras cada una, las utilidades se pueden convertir en tabla y `VEEvaluation` termina; deshacer devuelve las tablas originales. Lo mismo con `VariableTypeEdit` a numérica y de vuelta a estados finitos sobre «Therapy».

### Error 289 · [Corregido] Un nodo de azar numérico cuya relación es «Exact» —un valor exacto para cada configuración de sus padres— pierde todos sus valores al quitarle un padre o al pegarlo sin él: queda con una relación uniforme, y desde entonces ninguna evaluación funciona, con un mensaje que pide poner un hallazgo en el nodo

| | |
|---|---|
| **Estado** | Corregido en el commit `bfd7a34` (17-10-2026): `ExactDistrPotential` tiene su propio `removeVariable`, que conserva los valores con el padre quitado en su primer estado, como una tabla; `RemoveLinkEdit` lo usa también en las utilidades, sin el caso aparte del error 285. Hacía falta además que la regla provisional de `RemoveLinkEdit` que vuelve uniforme una relación cuya tabla suma cero no se aplique a la relación exacta: sus números son valores, y una relación exacta recién elegida vale cero en todas sus casillas, así que sin eso el caso más corriente seguía dando la uniforme. Medido sobre A → N con N numérica: con (5, 7), quitar A → N deja la relación exacta 5; con (0, 7), la exacta 0; pegar N solo da N' con la exacta 5; deshacer devuelve (5, 7), y la red se evalúa. Con prueba. Queda sin estudiar la utilidad esperada de 2,0 que se cuenta abajo, que sale igual antes de editar. |
| **Módulo** | core, gui |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.core.action.base.linkEdits; org.openmarkov.gui.action |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:726-736`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExactDistrPotential.java`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:139`<br>`gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:140` |
| **Comprobado** | Ejecutándolo (las ediciones y la evaluación, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 285, error 288, error 229 |

**Qué pasa.** Quitar una variable de una relación es lo que hacen las ediciones cuando un nodo pierde un padre: al borrar el enlace o el nodo padre (`RemoveLinkEdit`, línea 139) y al pegar un nodo sin su padre (`PasteEdit`, línea 140). Una tabla, un árbol, una fórmula o una regresión conservan sus valores con la variable quitada en su primer estado, porque redefinen `removeVariable`. `ExactDistrPotential` no lo redefine, y la versión general de `Potential` (líneas 726-736) devuelve una `UniformPotential`. En un nodo de azar numérico, una relación uniforme no se puede convertir en tabla: la utilidad esperada, las políticas óptimas y la propagación fallan con «The numeric variable N has a uniform distribution, so it has no value to compute with. Set a finding on it in edition mode and try again.», y el usuario tiene que volver a escribir la relación.

Visto al corregir el error 285, que hace lo mismo con las utilidades y se corrigió proyectando la relación exacta solo en ellas, por decisión de Manuel Arias. Medido el 16-10-2026 con el código de `7fe37a0`, sobre un diagrama de influencia A → N → U, con A finita, N numérica con la relación exacta (5, 7) y U la suma de N:

```
quitar A → N              | N: UniformPotential vars=[N]
                          | evaluar: «The numeric variable N has a uniform distribution, so
                          ↪  it has no value to compute with. Set a finding on it in
                          ↪  edition mode and try again.»
copiar N sin A y pegar    | N': UniformPotential vars=[N']; su relación no se puede
↪  convertir en tabla, con el mismo mensaje
```

En los ficheros del repositorio ningún nodo de azar tiene la relación «Exact»: se llega a ella eligiéndola en el panel de la relación de un nodo numérico. `InvertLinkEdit` también llama a `removeVariable` (línea 116), pero ningún camino de usuario lo ejecuta: la ventana invierte los enlaces con `InvertLinkAndUpdatePotentialsEdit`, que no la llama, y `InvertLinkEdit` solo se usa para comprobar las restricciones. En la red de la medida, la utilidad esperada antes de editar sale 2,0 y no 6; no se ha estudiado si es cosa de cómo se construyó la variable numérica.

**Cómo arreglarlo.** Dar a `ExactDistrPotential` un `removeVariable` que conserve los valores con la variable en su primer estado, proyectando su tabla como hace `TablePotential`. Con eso, la rama de las utilidades de `RemoveLinkEdit` puede usar `removeVariable` sin el caso aparte que le puso el error 285.

Prueba: en el diagrama de arriba, quitar A → N deja a N con la relación exacta 5 y la red se evalúa; copiar N solo y pegarlo da N' con la relación exacta 5.

# 5 Restricciones de red

Una restricción es una regla que dice qué permite cada tipo de red (por ejemplo, que no haya ciclos). Se imponen dos veces: al comprobar una red entera (PNConstraint.checkProbNet) y antes de cada edición (PNEdit.checkConstraintsWillBeMet).

### Error 37 · [Jorge Rico] Abrir un fichero no comprueba ninguna restricción: una red bayesiana con un ciclo se abre sin aviso

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.reader |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:134-150`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:272-286`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:164-191`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:103-124`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:190-216`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:339-356` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |

| | |
|---|---|
| **Decisión de equipo** | **Sí.** Qué hacer al abrir una red que no cumple sus restricciones: rechazar el fichero o abrirlo avisando (abrir avisando no deja fuera ficheros antiguos) |
| **Relacionados** | error 5, error 31, error 38, error 39, error 111, nota 153 |

**Qué pasa.** Un fichero `.pgmx` con una red que incumple sus restricciones se abre sin ningún aviso. Medido: se escribió con `PGMXWriter_1_0` una red bayesiana A→B, se añadió a mano en el XML el enlace B→A, y `PGMXReader.read` la devolvió sin error, con los dos enlaces y con `getUnsatisfiedConstraints() = [NoCycle]`.

A partir de ahí la red inválida se edita y se guarda como si nada. Además, pegar, cambiar el tipo de un nodo o invertir un enlace se rechazan con un mensaje sobre el ciclo que ya estaba (medido para pegar: es error 31). No se ha medido qué hace cada algoritmo de inferencia con una red así.

Las restricciones de una red se comprueban por dos vías: la de cada edición antes de ejecutarse y la de red completa (`PNConstraint.checkProbNet`, que se llama desde `ProbNet.checkConstraints`, `checkConstraintsIn` y `getUnsatisfiedConstraints`). En producción, la de red completa solo la llaman `InvertLinkEdit`, `ChangeNodeTypeEdit` y el verificador de `PasteEdit`, las tres después de modificar la red; `InferenceAlgorithm.checkConstraints`, con las restricciones del algoritmo y no las de la red; y `ProbNet.setNetworkType`, para las restricciones del tipo nuevo.

En el módulo `io` no hay ninguna llamada. `PGMXReader_0_2.read` construye la red con `new ProbNet(tipo)` (cuyo javadoc dice expresamente que instala las restricciones sin comprobarlas), añade nodos, enlaces y potenciales directamente, y la devuelve. `NetworkFileHandler.openNetworkSource` la pone en una pestaña sin preguntar nada. Tampoco comprueba nada `resttemplate` (hoy es solo un esqueleto) ni el aprendizaje, que construye la red con ediciones.

**El código.** io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:134-150

```
public PGMXReader.NetworkAndEvidence read(URL networkSource) throws ProbNetParserException
↪  {
    FormatManager formatManager = FormatManager.getInstance();
    try {
        formatManager.checkVersion(networkSource);
        formatManager.checkStructure(networkSource);
    } catch (SAXException | IOException e) {
        throw new ProbNetParserException.PGMXInvalid(e.getMessage());
    }
    Element root = getRootElement(networkSource);
    ProbNet probNet = this.getProbNet(root, networkSource.getFile());
    this.getInferenceOptions(root, probNet);
    List<EvidenceCase> evidence = this.getEvidence(root, probNet);
    this.getPolicies(root, probNet);
    this.setVariableType(root, probNet);
    this.setDefaultStates(root, probNet);
    return new PGMXReader.NetworkAndEvidence(probNet, evidence);
}
```

**Cómo arreglarlo.** Llamar a `probNet.getUnsatisfiedConstraints()` (o `checkConstraints()`) al terminar de leer.

Hay que elegir dónde. El sitio común a todos los formatos es `NetsIO.openNetworkURL` o `NetworkFileHandler.openNetworkSource`, pero entonces los que abren redes sin ventana quedan fuera. Si se pone en cada lector, hay que hacerlo en PGMX (las versiones 0.2 y 1.0 comparten `read`), en XMLBIF y en los demás lectores registrados (el formato Elvira no se mantiene).

Qué hacer con el resultado es la decisión de producto: lanzar una excepción de análisis con la lista

de violaciones, o devolver la red con avisos que la ventana muestre. Las redes de ejemplo y de pruebas del repositorio podrían no pasar todas; conviene recorrerlas antes de elegir.

Prueba de regresión: el fichero con A→B y B→A debe dar la excepción o el aviso que se decida.

### Error 38 · [Jorge Rico] La comprobación de nombres de estado no se ejecuta nunca: se puede dejar un estado con el nombre vacío, o con uno repetido por programa

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.constraint |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/constraint/ValidState.java:23-83`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:123-124`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:147-152`<br>`core/src/main/java/org/openmarkov/core/model/network/Variable.java:292-307`<br>`gui/src/main/java/org/openmarkov/gui/component/DiscretizeTablePanel.java:733-746`<br>`gui/src/main/java/org/openmarkov/gui/component/DiscretizeTablePanel.java:907-940` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si «dos estados con el mismo nombre» se decide ignorando mayúsculas o no, y si la regla vive en una restricción activa o directamente en la variable y en las ediciones |
| **Relacionados** | error 37, error 39, error 58, error 72, nota 143, nota 147 |

**Qué pasa.** `ValidState` es la regla «un estado no puede tener el nombre vacío ni repetir el de otro estado de la misma variable». Tiene las dos vías: `checkState` para una edición (la usa `NodeStateEdit.checkConstraintsWillBeMet`) y `checkProbNet` para la red entera. Pero ninguna de las dos se ejecuta.

Desde la ventana, los estados se añaden y renombran en la pestaña de dominio de las propiedades del nodo (`DiscretizeTablePanel`). Añadir inventa un nombre libre («… 1», «… 2»), así que por ahí no entran repetidos. Renombrar ejecuta `NodeStateEdit(RENAME)`, que llama a `Variable.renameState`: esta rechaza un nombre exacto que ya tenga otro estado, pero **no** rechaza el nombre vacío, y compara con igualdad exacta, así que acepta «Yes» junto a «yes».

Medido con una variable de estados «yes» y «no»: renombrar «no» a “ ” se acepta; renombrar después «yes» a “ ” se rechaza por repetido; la red escrita con `PGMXWriter_1_0` y leída con `PGMXReader` vuelve con los estados `[yes]` y `[]`. Como tampoco se comprueba nada al abrir un fichero (error 37), un fichero con estados vacíos o repetidos también entra sin aviso.

Por programa también se alcanza el duplicado por alta, porque `NodeStateEdit` es pública. Medido directamente sobre `NodeStateEdit`: añadir un estado llamado igual que otro («a0» a una variable con «a0, a1») se acepta; renombrar un estado a la cadena vacía se acepta; la red se escribe en `.pgmx` con `<State name="a0"/>` repetido o `<State name=""/>` y se vuelve a leer igual. Con un nombre repetido, `Variable.getStateIndex("a0")` devuelve 0 también para el tercer estado, de modo que toda búsqueda por nombre (lector, escritor, evidencia) encuentra el primero.

Por qué pasa: `NodeStateEdit.checkConstraintsWillBeMet` (módulo `core`) solo hace algo si `probNet.getConstraintOfClass(ValidState.class)` devuelve una restricción. `ValidState` no lleva la anotación `@Constraint`, así que `ConstraintManager` no la descubre, ningún tipo de red la incluye y nadie hace `new ValidState()`. Todo el contenido de `NodeStateEdit.checkConstraintsWillBeMet` es código muerto. Comprobado ejecutando: en redes bayesianas, diagramas de influencia, MID (diagramas de influencia de Markov) y DAN (redes de análisis de decisiones) recién creadas, `getConstraintOfClass(ValidState.class)` es `null`; en cambio `ValidCriterionName`, que sí está anotada, aparece en todas.

Las copias de la regla ya no coinciden entre sí, aunque hoy no se note porque la clase está inactiva: `ValidState.existState` compara con `equalsIgnoreCase`, mientras que `ValidState.checkProbNet` y `Variable.renameState` comparan exacto. Si se activara la restricción tal como está, añadir «Yes» junto a «yes» se rechazaría por la vía de la edición y pasaría por la de la red.

Hay además una trampa para el arreglo: para `RENAME`, `NodeStateEdit` guarda en `newState` el estado **seleccionado** (línea 124), no uno con el nombre nuevo, y la comprobación usa `getNewState().getName()`. Si se activara `ValidState` tal cual, cada renombrado preguntaría por el nombre actual del propio estado, que existe siempre, y **todos** los renombrados se rechazarían.

**El código.** core/src/main/java/org/openmarkov/core/action/core/NodeStateEdit.java:123-124, 147-152

```
State selectedState = oldStates[selectedStateIndex];
this.newState = (stateAction != StateAction.RENAME) ? new State(newName) : selectedState;
...
@Override public void checkConstraintsWillBeMet(ConstraintChecker constraintChecker) {
    if (probNet.getConstraintOfClass(ValidState.class) instanceof ValidState constraint) {
        constraint.checkState(constraintChecker, this.getNewState()
                                                    .getName(), this.getNode(),
                                                    ↪  this.getStateAction());
    }
}
```

core/src/main/java/org/openmarkov/core/model/network/constraint/ValidState.java:23, 57-60

```
public class ValidState extends PNConstraint {      // sin @Constraint
...
private static boolean existState(String state, Node node) {
    for (State states : node.getVariable().getStates()) {
        if (states.getName().equalsIgnoreCase(state)) {
```

core/src/main/java/org/openmarkov/core/model/network/Variable.java:292-299

```
public void renameState(State state, String newName) {
    for (State s : states) {
        // The state being renamed is skipped: renaming it to its own current name is a
        ↪  no-op,
        // not a collision.
        if ((s != state) && s.getName().contentEquals(newName)) {
            throw new InvalidArgumentException(newName, "new name", "that name already
            ↪  belongs to another state");
        }
    }
```

**Cómo arreglarlo.** Hay dos opciones, y elegir es cosa del equipo:

- (a) Anotar `ValidState` como obligatoria, dejarle una sola regla y corregir `NodeStateEdit` para que compruebe `newName` (no el nombre del estado seleccionado) y, al renombrar,

excluya al propio estado.
- (b) Borrar `ValidState` y su consulta, y hacer que la regla viva en un único sitio del dominio: `Variable.renameState`, donde ya está la de repetidos, y el alta de estados de `VariableStateOperations.addStateAt`, donde falta; en los dos hay que añadir el rechazo del nombre vacío.

En cualquier caso hay que decidir antes si «Yes» y «yes» chocan.

Sitios donde la regla debe cumplirse con el mismo criterio:

- la comprobación de red completa, `ValidState.checkProbNet`, y la previa de la edición, `NodeStateEdit.checkConstraintsWillBeMet` con `ValidState.checkState`, si se conserva la clase;
- `Variable.renameState`;
- el alta en `VariableStateOperations.addStateAt`;
- la sustitución de todos los estados, `VariableStateOperations.replaceStates`, que usa `NodeReplaceStatesEdit` (dominios estándar);
- la propagación a los nodos gemelos de otros ciclos, `propagateNodeStateEditRelatedVariables` en `DiscretizeTablePanel` y en `PrefixedKeyTablePanel`;
- el lector de ficheros, si se decide comprobar las restricciones al abrir (error 37).

Pruebas: añadir un estado con nombre repetido y renombrar un estado a “ ” deben rechazarse con un mensaje, y el nombre original debe seguir en la tabla; renombrar un estado a su propio nombre y a un nombre libre debe aceptarse.

### Error 39 · Las restricciones opcionales no entran en ninguna red: ocho reglas escritas no defienden a nadie

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.constraint |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/constraint/ConstraintManager.java:57-106`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:115-124`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:311-331`<br>`core/src/test/java/org/openmarkov/core/model/network/constraint/EveryConstraintReachesANetworkTest.java:34-52`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:99-130`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VariableElimination.java:127-132` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué significa OPTIONAL: la activa el tipo de red; la activa el usuario y el formato la guarda; o el nivel desaparece |
| **Relacionados** | error 37, error 38, nota 89, nota 146 |

**Qué pasa.** Cada restricción declara en su anotación `@Constraint` un comportamiento por omisión: YES (obligatoria), NO u OPTIONAL. Ocho restricciones marcadas OPTIONAL no entran en ninguna red: `NoLoops`, `NoMixedParents`, `NoSuperValueNode`, `OnlyFiniteStatesVariables`, `OnlyNumericVariables`, `OnlyOneUtilityNode`, `ProperUtilityPotentials` y `OnlyUnlabeledLinks`.

Las ediciones que consultan esas restricciones reciben `null` y no comprueban nada: `AddLinkEdit` con `NoLoops`, `NoMixedParents` y `NoSuperValueNode`; `AddNodeEdit` y `ChangeNodeTypeEdit` con `OnlyOneUtilityNode`; `VariableTypeEdit` con `OnlyFiniteStatesVariables` y `OnlyNumericVariables`; `RemoveNodeEdit` con `ProperUtilityPotentials`; `InvertLinkEdit` con `NoMixedParents`. Sus pruebas pasan porque añaden la restricción a mano.

Un caso visible: la eliminación de variables exige `NoMixedParents` como restricción del algoritmo. El editor deja dar a un nodo de utilidad padres de utilidad y de azar a la vez sin quejarse, y el error solo aparece al evaluar (red no evaluable), como quedó documentado en el commit `098cf85`.

Hay otra vía por la que una opcional puede entrar sin que nadie lo decida: la sección `Additional Constraints` del lector PGMX instancia cualquier clase cuyo nombre venga en el fichero (ver nota 89; allí no hay nada que corregir hoy).

Por qué pasa: `ConstraintManager.buildConstraintList(tipo, includeOptionals)` incluye las opcionales solo si se le pasa `true`, pero en producción solo se llama la versión de un argumento, que pasa `false` (desde el constructor de `ProbNet`, `setNetworkType` y `getAdditionalConstraints`). Una restricción opcional solo entra en una red si un tipo la sube a YES o si alguien la añade con `addConstraint`.

La prueba `EveryConstraintReachesANetworkTest` (commit `e2158e7`) fija la situación de hoy a la espera de la decisión: cuatro restricciones se añaden a mano (`MaxNumParents` y `ModelNetworkConstraint` desde el aprendizaje; `OnlyDiscreteVariables` y `OnlyContinuousVariables` desde el diálogo de propiedades de la red), dos las sube el tipo MDP (proceso de decisión de Markov), y las ocho de arriba no entran en ninguna red.

**El código.** core/src/main/java/org/openmarkov/core/model/network/constraint/ConstraintManager.java:57-66

```
public ArrayList<PNConstraint> buildConstraintList(NetworkType type, boolean
↪  includeOptionals) {
    // Init the list with those constraints that have the default value set to YES
    ArrayList<PNConstraint> constraints = new ArrayList<>();
    for (Class<? extends PNConstraint> constraintClass :
    ↪  defaultConstraintBehaviors.keySet()) {
        boolean isMandatoryConstraint = getDefaultBehavior(constraintClass) ==
        ↪  ConstraintBehavior.YES;
        boolean isOptionalConstraintToUse = includeOptionals &&
        ↪  getDefaultBehavior(constraintClass) == ConstraintBehavior.OPTIONAL;
        if (isMandatoryConstraint || isOptionalConstraintToUse) {
            addIfInstantiable(constraints, constraintClass);
        }
    }
```

core/src/main/java/org/openmarkov/core/model/network/constraint/ConstraintManager.java:104-106

```
public final ArrayList<PNConstraint> buildConstraintList(NetworkType type) {
    return buildConstraintList(type, false);
}
```

**Cómo arreglarlo.** Hay tres salidas, y elegir es cosa del equipo:

- (1) Cada tipo de red declara YES las opcionales que quiere, y las que no quiera ningún tipo se borran.
- (2) Las activa el usuario desde las propiedades de la red y el formato `.pgmx` las guarda. Esto exige que la sección de restricciones adicionales del fichero escriba y lea lo mismo

(ver nota 89).
- (3) Desaparece OPTIONAL y todo es YES o NO por tipo.

Detalle latente que hay que resolver a la vez: las anulaciones de un tipo solo tratan YES y NO (`ConstraintManager`, líneas 69-84); un tipo que declarase OPTIONAL no conseguiría nada.

Tras decidir, vaciar la lista `WITHOUT_A_NETWORK` de `EveryConstraintReachesANetworkTest`, que es la prueba de regresión. Si una restricción pasa a una red, revisar que sus dos vías (la de `check ProbNet` y la copia en cada edición) coincidan.

# 6 Evidencia, inferencia y variables numéricas

La propagación de la evidencia, la entrada en modo inferencia y la discretización de las variables numéricas que se hace antes de resolver con un algoritmo exacto.

### Error 40 · [Corregido] Una variable numérica de azar sin precisión se discretiza a un único valor cero y el modelo responde con otros números

| | |
|---|---|
| **Estado** | Corregido en el commit `d6579f7` (10-10-2026): `Variable.round` devuelve el valor sin redondear cuando la precisión no es positiva, con prueba (P → N con precisión 0 y valores 3 y 7 da dos estados). El lector y el constructor `Variable(String)` siguen dejando la precisión en cero. Queda sin medir lo que hace una precisión cero donde se usa como paso: el selector numérico al poner un hallazgo, los límites por omisión al discretizar y el valor mínimo de una relación delta. |
| **Módulo** | core, io |
| **Paquete** | org.openmarkov.core.model.network; org.openmarkov.io.probmodel.reader |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/Variable.java:170-175`<br>`core/src/main/java/org/openmarkov/core/model/network/Variable.java:660-662`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:495`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:528-535`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1006-1010` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir qué significa una precisión cero. Manuel Arias lo decidió el 10-10-2026: que el valor no se redondea. Es lo que ya significaba para las utilidades, que el programa escribe con precisión cero (1490 variables en las redes del repositorio) |
| **Relacionados** | error 44, error 45, error 169 |

**Qué pasa.** Si se abre un `.pgmx` con una variable numérica de azar que no trae `<Precision>` y cuyo valor depende de sus padres, al evaluar o propagar la variable se trata como si valiera siempre cero. Lo que el modelo decía que valía se pierde sin aviso ni excepción, y los resultados cambian.

Medido con un diagrama de influencia de tres nodos escrito a mano: P (azar, dos estados al 50 %) → N (numérica de azar, vale 3 si P=p0 y 7 si P=p1) → U (utilidad en árbol: 0 si N ≤ 5, 10 si N > 5).

```
precision de N = 0.01 -> estados de N discretizada: [3, 7] -> utilidad esperada = 5.0
precision de N = 0.0  -> estados de N discretizada: [0]    -> utilidad esperada = 0.0
```

Y directamente: con `new Variable("X")`, `round(12.5)=0.0`, `round(-3.7)=-0.0`, `round(0)=0.0`; con valores 3, 7 y −2 la variable convertida sale con los estados «-0» y «0».

Si la variable numérica tiene hallazgo (por ejemplo, porque un potencial delta lo induce, como el «Time in treatment» de los modelos de Markov), no pasa por esta rama y no le afecta: quitar `<Precision>` de `0_miscellaneous/networks/mid/MID-Chancellor.pgmx` no cambia su utilidad esperada (50608,78 en los dos casos).

Por qué pasa. Antes de resolver con un algoritmo exacto (eliminación de variables: propagación, evaluación, análisis de coste-efectividad, evolución temporal), `ProbNetOperations.convertNumericalVariablesToFS` convierte cada variable numérica de azar que no tenga hallazgo en una variable de estados finitos. Para ello recorre todas las configuraciones de los padres, calcula el valor que toma la variable en cada una y lo redondea a la precisión de la variable con `oldVariable.round(...)` (línea 495). Cada valor distinto tras el redondeo se convierte en un estado.

`Variable.round` calcula `Math.round(value / precision) * precision`. Si la precisión es 0, la división da infinito (o `NaN` si el valor también es 0), `Math.round` devuelve el mayor (o menor) entero largo, o 0, y al multiplicar por 0 el resultado es siempre 0,0 (o −0,0 para valores negativos). Es decir: todos los valores se redondean a cero. La variable discretizada sale con un único estado «0» (o con dos, «-0» y «0», si había valores negativos y positivos, porque la lista de estados distingue 0,0 de −0,0), y la tabla que la define apunta todas las configuraciones a ese estado.

La precisión cero sí aparece en la práctica:

- El lector del formato propio (`PGMXReader_0_2.getXMLContinuousVariable`, líneas 1006-1010) pone 0,0 cuando al elemento `<Variable>` de una variable numérica le falta el hijo `<Precision>` (y también, claro, si trae `<Precision>0.0</Precision>`). Un fichero escrito por OpenMarkov siempre lleva la etiqueta, pero uno escrito a mano o por otra herramienta puede no llevarla; ninguna de las redes de ejemplo del repositorio la omite.
- El constructor `new Variable(String name)` (líneas 170-175), el de variables numéricas sobre toda la recta, deja la precisión en 0,0, mientras que el resto de constructores la dejan en 0,01. Hoy en producción solo se usa para nodos de utilidad y variables auxiliares que no pasan por la discretización, así que el camino real es el del fichero.
- Desde la ventana de propiedades del nodo no se puede poner cero: el desplegable de precisión solo ofrece 1, 0,1, 0,01, 0,001 y 0,0001.

**El código.** core/src/main/java/org/openmarkov/core/model/network/Variable.java:170-175, 660-662

```
    public Variable(String name) {

        this(name, new State[]{new State("0")},
             new PartitionedInterval(false, Double.NEGATIVE_INFINITY,
             ↪  Double.POSITIVE_INFINITY, false), 0.0);
        this.variableType = VariableType.NUMERIC;
    }
...
    public double round(double value) {
        return Math.round(value / precision) * precision;
    }
```

io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1006-1010

```
        // Precision
        Element xMLPrecision = variableElement.getChild(XMLTags.PRECISION.toString());
        double precision = xMLPrecision != null ?
        ↪  Double.parseDouble(xMLPrecision.getText()) : 0.0;
```

```
        return new Variable(variableName, leftClosed, min, max, rightClosed, precision);
```

core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:494-499

```
                        scalarValue = oldPotential.tableProject(configuration,
↪  inferenceOptions).getValues()[0];
                        scalarValue = oldVariable.round(scalarValue);
                        projectedValues[index++] = scalarValue;
                        if (!newStates.contains(scalarValue)) {
                            newStates.add(scalarValue);
                        }
```

**Cómo arreglarlo.** Hay que decidir qué significa una precisión cero. Dos opciones razonables:

- (a) «No redondear»: `Variable.round` devuelve el valor tal cual cuando la precisión es menor o igual que cero. Con esta opción conviene además normalizar −0,0 a 0,0 al construir la lista de estados, para no fabricar dos estados «-0» y «0».
- (b) «Valor no válido»: el lector y el constructor `Variable(String)` usan el valor por defecto de la clase (0,01) cuando no hay precisión, y `setPrecision` rechaza el cero.

Sitios donde la regla tiene que cumplirse:

- `Variable.round`, usado en `ProbNetOperations.convertNumericalVariablesToFS` (líneas 495 y 582) y en `Variable.getStateIndex(double)` (línea 343);
- el valor por defecto del lector `PGMXReader_0_2.getXMLContinuousVariable` (línea 1008);
- el constructor `Variable(String)`;
- los usos de la precisión como paso: `Variable.getDefaultInterval` (suma la precisión para construir intervalos) y el selector numérico de `AddFindingDialog` (usa la precisión como incremento).

La forma en que se escriben los límites de los tramos con una precisión como 0,0 era el error 169, corregido el 09-10-2026 sin cambiar lo que significa esa precisión.

Prueba de regresión: una red P → N (numérica, precisión 0, valores 3 y 7) convertida con `convertNumericalVariablesToFS` debe dar dos estados; y leer un `.pgmx` sin `<Precision>` como el descrito arriba y evaluarlo con `VEEvaluation` debe dar 5,0.

## Error 41 · [Corregido] Poner un hallazgo, en modo inferencia, sobre un nodo numérico de azar hace fallar la propagación con un NullPointerException

| | |
|---|---|
| **Estado** | Corregido en el commit `b8e0c8a` (30-09-2026): `VEPropagation` discretiza las variables numéricas con toda la evidencia, previa y posterior a la resolución, y pasa los hallazgos de las dos a las variables nuevas, sin tocar la evidencia que recibe. En `MID-Colorectal.pgmx`, con la política de cribado impuesta, un hallazgo posterior sobre «Age [0]» da las mismas probabilidades que el mismo hallazgo previo, y la edad cambia el resultado; sin política impuesta sigue fallando al calcular la política óptima, que tiene que discretizar la edad sin observarla (error 42). En `MID-dmhee-2.5.pgmx`, un hallazgo posterior de valor 0 sobre «Duration of treatment [0]» se propaga; con otro valor, que contradice el delta del nodo, se rechaza como evidencia incompatible (error 194). |
| **Módulo** | core, inference |
| **Paquete** | org.openmarkov.core.model.network; org.openmarkov.inference.algorithm.variableElimination.tasks |

| | |
|---|---|
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:63-70,203-225,324-333,431-459` `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:117-157,228-242` `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VariableElimination.java:95-100` `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:581-600` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 42, error 44, error 45, error 46, error 194, nota 55 |

**Qué pasa.** El usuario abre una red con nodos numéricos de azar, pasa a modo inferencia y añade un hallazgo a uno de ellos (el diálogo pide un valor numérico). La propagación falla con `NullPointerException`: la excepción no es de las que `EvidenceManager.doPropagation` recoge y llega al gestor global, que muestra la ventana de error inesperado; el panel no enseña probabilidades para ese caso de evidencia.

Medido sobre `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-2.5.pgmx` (un modelo de Markov con decisiones): con un hallazgo posterior de valor 1 sobre el nodo numérico de azar «Duration of treatment [0]», `getPosteriorValues` lanza `NullPointerException: Cannot invoke "Node.getNeighbors()" because "node" is null` en `ProbNetOperations.removeUnreachableNodes(ProbNetOperations.java:218)`, llamado desde `VEPropagation.pruneNetwork(VEPropagation.java:241)`. Con un hallazgo sobre un nodo de estados finitos («State [1]») la propagación termina bien.

Por qué pasa. En modo inferencia, cada vez que cambia la evidencia, `EvidenceManager.doPropagation` (módulo `gui`) calcula las probabilidades a posteriori con `VEPropagation` (eliminación de variables, módulo `inference`), pasando como evidencia posterior a la resolución los hallazgos que el usuario ha puesto.

`VEPropagation.resolve` prepara la red: la expande si es temporal y luego `exactAlgorithmsPreprocessing` (`VariableElimination`, líneas 95-100) llama a `TaskUtilities.discretizeNonObservedNumericVariables(probNet, getPreResolutionEvidence())`, es decir, a `ProbNetOperations.convertNumericalVariablesToFS`. Esa conversión solo tiene en cuenta la evidencia **previa** a la resolución: un nodo numérico de azar sin hallazgo previo se convierte en uno de estados finitos poniéndole un objeto `Variable` nuevo con el mismo nombre (`node.setVariable(newVariable)`). El hallazgo posterior sigue apuntando a la variable numérica original.

Después, para cada variable de interés, `pruneNetwork` (líneas 228-242) mete en la lista de variables a conservar las de todos los hallazgos y llama a `ProbNetOperations.getPruned`. En `removeUnreachableNodes` (líneas 210-212) se hace `nodesToKeep.add(probNet.getNode(variable))` sin comprobar el resultado: para la variable original `getNode` devuelve `null`, el `null` entra en el conjunto y en la línea 218 `node.getNeighbors()` lanza la excepción. La función hermana `getEvidenceNodes` (líneas 324-333) sí descarta los `null`.

Comprobado aparte que, tras la expansión, `getNode` con la variable original encuentra el nodo y, tras la discretización, ya no (buscando por nombre sí).

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:209-218

```
        // Store nodes of variablesOfInterest in nodesToKeep
        for (Variable variable : variablesOfInterest) {
            nodesToKeep.add(probNet.getNode(variable));
```

```
        }

        // Add neighbors of variablesOfInterest as nodesToKeep and store them in
        // nodesToExplore
        HashSet<Node> nodesToKeepClon = new HashSet<>(nodesToKeep);
        for (Node node : nodesToKeepClon) {
            List<Node> neighbors = node.getNeighbors();
```

**Cómo arreglarlo.** Saltar el `null` en `removeUnreachableNodes` evita la excepción pero no basta: el hallazgo sobre la variable numérica se ignoraría sin decir nada, la misma pérdida silenciosa que describe nota 55 (allí no hay nada que corregir).

El arreglo completo es que la preparación de la red tenga en cuenta toda la evidencia (previa y posterior) al discretizar y al absorber nodos numéricos, o que tras la discretización los hallazgos se traduzcan a las variables nuevas (buscando por nombre) antes de podar.

Sitios afectados:

- `VariableElimination.exactAlgorithmsPreprocessing`;
- `ProbNetOperations.convertNumericalVariablesToFS`;
- `VEPropagation.pruneNetwork` y `getAllEvidence`;
- `removeUnreachableNodes`, que debería tratar una variable ausente igual que `getEvidenceNodes`, o fallar con una excepción del dominio con mensaje.

Prueba: `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-2.5.pgmx` con un hallazgo posterior sobre «Duration of treatment [0]»; la propagación debe terminar y el valor observado debe influir en el resultado.

### Error 42 · [Corregido] Entrar en modo inferencia en una red con nodos de utilidad que otro nodo de utilidad suma o multiplica (ID-arthronet, ID-mediastinet, ID-used-car-buyer…), o con un nodo numérico de azar sin observar, falla con NullPointerException

| | |
|---|---|
| **Estado** | Corregido en el commit `fcc9888` (09-10-2026), con la opción (a): el valor de un nodo de utilidad absorbido se calcula en `VEPropagation` sobre una copia de la red sin sus descendientes, preparada como la red entera y con las políticas óptimas de la red completa ya fijadas. Un nodo numérico de azar sin hallazgo no se calcula, y ya no impide calcular los demás. Con prueba: en `ID-arthronet`, `ID-mediastinet`, `ID-used-car-buyer` e `ID-mediastinet-1-0` todo nodo de utilidad tiene valor, cada nodo de suma vale la suma de sus sumandos y el nodo final da la utilidad global; `ID-arthronet` da lo mismo que `ID-arthronet-ce` en los nodos que comparten. Medido en las redes bayesianas y diagramas de influencia de hasta 40 nodos del repositorio, pidiendo todas las variables: las 14 que daban esta excepción terminan y las otras 123 siguen igual; dos fallan antes y después por otra causa (`ID-delayed-result-of-test`, una relación uniforme que no se convierte en tabla); 25 no se probaron por tamaño. Visto en la aplicación real, arrancada desde un guion: `ID-arthronet` e `ID-used-car-buyer` entran en modo inferencia sin error. No se han tocado `getNodeAncestors` ni `removeUnreachableNodes`; no se han probado hallazgos previos a la resolución. |
| **Módulo** | inference, core |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.tasks; org.openmarkov.core.model.network |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:141-157,175-217,228-242` |

| | |
|---|---|
| | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VariableElimination.java:95-97`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:146-178`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:431-459,1050-1054`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:584-588`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/InferencePresenter.java:60-69,118-126`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:146-164` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No. Pedía decidir qué enseña el modo inferencia en un nodo de utilidad que otro suma o multiplica: su utilidad esperada, calculada aparte, o nada. Manuel Arias eligió lo primero el 09-10-2026: la ventana pinta un valor en todo nodo de utilidad, y el código de la propagación ya decía que había que calcularlos |
| **Tapado por** | error 46: en nueve redes, entre ellas MID-HPV, el cálculo de rangos falla antes. |
| **Relacionados** | error 8, error 41, error 44, error 45, error 46, error 108, error 112, error 113, nota 109 |

**Qué pasa.** Con `0_miscellaneous/networks/id/ID-arthronet.pgmx` abierto y sin evidencia, pulsar el botón de modo inferencia termina en la ventana de error no previsto con un `NullPointerException`, y la red vuelve a modo edición: el modo inferencia no se puede usar en esa red. El camino es `MainPanelListenerAssistant` (líneas 155-171) → `InferenceHandler.setWorkingMode` → `EvidenceManager.updateIndividualProbabilitiesAndUtilities` → `doPropagation`, que construye un `VEPropagation` (propagación por eliminación de variables) y le pide los resultados de **todas** las variables de la red (líneas 585-589). `MainPanelListenerAssistant` recoge la excepción, vuelve al modo edición y la relanza envuelta; `OMExceptionHandler`, al encontrar debajo un `NullPointerException`, abre el diálogo de error no previsto.

Medido con un pequeño programa de prueba que hace las mismas llamadas que `doPropagation` (primero el cálculo de rangos de las utilidades, después la propagación) sobre todas las redes que se pueden leer del repositorio. Terminan con este `NullPointerException`, con el cálculo de rangos sin fallar antes:

- en `0_miscellaneous/networks`: `id/ID-arthronet.pgmx`, `id/ID-mediastinet.pgmx`, `id/ID-used-car-buyer.pgmx`, `dan/DAN-mediastinet.pgmx`, `dan/DAN-qale-mediastinet.pgmx` y `dan/DAN-used-car-buyer.pgmx` (DAN: red de análisis de decisiones);
- en las redes de prueba: once de `integrationTests/src/main/resources/networks` (`id/ID-decide-test-sv.pgmx` y diez de `dan`, casi todas con «-sv» en el nombre) y `core/src/test/resources/nets/DiscretePotentialOperations/IDDecideTestSymptom.pgmx`.

En otras nueve (entre ellas `0_miscellaneous/networks/mid/MID-HPV.pgmx`) el cálculo de rangos falla antes (error 46); arreglado ese, llegarían a este. Las versiones con dos criterios (`ID-arthronet-ce.pgmx`, `ID-mediastinet-ce.pgmx`) no tienen nodos de utilidad intermedios y se propagan bien.

Añadido el 09-10-2026: le pasa también a `0_miscellaneous/networks/id/1-0/ID-mediastinet-1-0.pgmx`, la misma red en formato 1.0. Medido sin ventana, pidiendo todas las variables a `VEPropagation`: `NullPointerException` en `ProbNetOperations.getNodeAncestors` (línea 1059), llamado desde `VEPropagation.pruneNetwork` (línea 235).

Hay una segunda forma de llegar, sin ninguna red del repositorio que la tenga: un nodo numérico de azar sin hallazgo cuyo valor se pueda enumerar a partir de sus padres. Medido con una red

bayesiana construida por programa, Disease (estados finitos) → N (numérico, relación exacta 3 o 7): pedir todas las variables lanza la misma excepción; pedir solo Disease devuelve [0,9, 0,1]. Si la red tiene además un nodo de utilidad que depende de N, el cálculo de rangos falla antes (error 46).

Por qué pasa. `VEPropagation.resolve` trabaja sobre una copia de la red que `VariableElimination.exactAlgorithmsPreprocessing` (líneas 95-97) modifica de dos maneras que dejan fuera variables de la red del usuario:

- `TaskUtilities.absorbAllIntermediateNumericNodes` (`BasicOperations`, líneas 144-170) absorbe en sus hijos los nodos numéricos que tienen hijos, y los quita de la red. Entre ellos están los nodos de utilidad cuyo valor suma o multiplica otro nodo de utilidad (los antiguos nodos de «supervalor»). Medido: en `ID-arthronet.pgmx` desaparecen 12 de sus 13 nodos de utilidad, en `ID-mediastinet.pgmx` 19 y en `ID-used-car-buyer.pgmx` 5; solo queda el nodo final.
- `TaskUtilities.discretizeNonObservedNumericVariables` (`ProbNetOperations.convertNumericalVariablesToFS`) sustituye cada nodo numérico de azar sin hallazgo por uno de estados finitos con un objeto `Variable` nuevo del mismo nombre, así que buscar el nodo con la variable original ya no lo encuentra.

Después, para cada variable de interés, `resolve` busca la variable por nombre, pero solo para ver si tiene hallazgo (línea 147). Luego llama a `pruneNetwork` con el objeto que le dio el llamador, y `pruneNetwork` hace `getNodeAncestors(preprocessedNetwork.getNode(variableOfInterest))` (línea 233). `getNode` devuelve `null` para una variable que ya no está, y `ProbNetOperations.getNodeAncestors` falla al meter ese `null` en un `ArrayDeque` (línea 1054). Si se pasara de ahí, `InvokeVariableEliminationCore` volvería a fallar en `probNet.getNode(variableOfInterest).getNodeType()` (línea 192).

Es de la misma familia que error 41 (la red preparada ya no contiene una variable de la red del usuario), pero allí la variable perdida es la de un hallazgo posterior y la excepción sale en `removeUnreachableNodes`; arreglar uno no arregla el otro.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/-tasks/VEPropagation.java:145-155, 228-235

```
        if (variablesOfInterest != null) {
            for (Variable variableOfInterest : variablesOfInterest) {
                Variable variableOfInterestInProbnet =
↪   probNet.getVariable(variableOfInterest.getName());
                if (evidenceVariables.contains(variableOfInterestInProbnet)) {

↪   variablesOfInterestBelongingToEvidence.add(variableOfInterestInProbnet);
                } else {
                    ProbNet preprocessedNetwork = pruneNetwork(probNet.copy(),
↪   variableOfInterest);
                    ...
    private ProbNet pruneNetwork(ProbNet preprocessedNetwork, Variable variableOfInterest)
            throws IncompatibleEvidenceException.EvidenceIsIncompatibleWithOther {
        //Prune all the nodes except the variable of interest and its ancestors (and the
        ↪  corresponding findings).
        List<Variable> variablesNotToBePruned = new ArrayList<>();
        variablesNotToBePruned.add(variableOfInterest);
        for (Node node : ProbNetOperations.getNodeAncestors(preprocessedNetwork.getNode(
        ↪  variableOfInterest))) {
```

core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:1050-1054

```
    public static Set<Node> getNodeAncestors(Node node) {
        Set<Node> ancestors = new HashSet<>();

        Deque<Node> noExploredNodes = new ArrayDeque<>();
        noExploredNodes.add(node);
```

**Cómo arreglarlo.** `VEPropagation.resolve` tiene que traducir cada variable de interés a la red preparada antes de usarla, y decidir qué hace con las que ya no están:

- Nodo numérico de azar discretizado: buscarlo por nombre (como ya hace la línea 147) en `pruneNetwork` y en `InvokeVariableEliminationCore`, y guardar el resultado con la variable del llamador como clave, que es la que busca la ventana. La ventana no usa la tabla de un nodo numérico de azar (`InferencePresenter`, líneas 118-126, pinta el valor del hallazgo o nada), así que también vale no calcularlo; lo que no puede es lanzar una excepción.
- Nodo de utilidad absorbido: no queda nodo al que preguntar. Hay que decidir entre (a) calcular su utilidad esperada aparte, sobre una copia en la que ese nodo sea el último (quitando antes sus descendientes para que no se absorba), o (b) no calcularla y no pintar nada en ese nodo. La opción (b) obliga a tocar también `InferencePresenter.paintInferenceResults UtilityNode` (línea 66, que hace `individualProbabilities.get(variable).getValues( )[0]` y daría otro `NullPointerException`).

Otros sitios de la misma regla: `ProbNetOperations.getNodeAncestors` y `removeUnreachable Nodes` deberían rechazar una variable ausente con una excepción que la nombre, en vez de dejar pasar un `null`; y el arreglo de error 41 (los hallazgos) debe usar la misma traducción de variables.

Prueba de regresión: `VEPropagation` sobre `0_miscellaneous/networks/id/ID-arthronet.pgmx` y `0_miscellaneous/networks/id/ID-mediastinet.pgmx` con todas las variables de interés y evidencia vacía debe terminar y devolver, para los nodos de utilidad intermedios, lo que se decida; y la red bayesiana Disease → N con todas las variables no debe lanzar excepción.

### Error 43 · [Corregido] Propagar o evaluar una red con un nodo numérico de azar sin potencial muere con IndexOutOfBoundsException al discretizar

| | |
|---|---|
| **Estado** | Corregido. La comprobación de la ventana principal, desde `83ff8ab`; el resto, en el commit `8b173d5` (30-09-2026): `ProbNetOperations.convertNumerical VariablesToFS` rechaza un nodo numérico de azar sin potencial con `There IsNoPotentialInNodeException`, envuelta en `UnrecoverableException`, y el mensaje nombra el nodo. Así lo reciben también el análisis de coste-efectividad del menú Herramientas, el análisis de sensibilidad y quien llame a la API. Dejar el nodo sin convertir solo trasladaba el fallo: la propagación moría después, en `VEPropagation`, con el mismo `IndexOutOfBoundsException`. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:431-443`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:561-566`<br>`core/src/main/java/org/openmarkov/core/inference/tasks/TaskUtilities.java:88-90`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VariableElimination.java:95-97` |
| **Comprobado** | Leyendo el código y ejecutándolo |

| | |
|---|---|
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 70 |

**Qué pasa.** Si en un fichero `.pgmx` a un nodo numérico de azar le falta su potencial (el lector lo acepta en silencio), el usuario ve un error inesperado al propagar o evaluar, sin indicación de qué nodo está mal. Es el mismo camino de llegada que error 29; desde la ventana no se crea un nodo así.

Comprobado ejecutando con una red bayesiana A → X, con X numérica de azar sin potencial: `ProbNetOperations.convertNumericalVariablesToFS(net)` lanza `IndexOutOfBoundsException: Index 0 out of bounds for length 0` en la línea 443, y `VEPropagation` (propagación por eliminación de variables, módulo `inference`) sobre la misma red muere igual al pedir los posteriores, vía `VariableElimination.exactAlgorithmsPreprocessing` → `TaskUtilities.discretizeNonObservedNumericVariables`.

Por qué pasa: `ProbNetOperations.convertNumericalVariablesToFS` (módulo `core`) convierte en variables de estados finitos las variables numéricas de azar antes de resolver con un algoritmo exacto. Para cada nodo numérico de azar empieza con `Potential oldPotential = node.getPotentials().get(0);`. Unas líneas más abajo, para los nodos no numéricos, sí comprueba `!node.getPotentials().isEmpty()`.

Llaman a este método también la evaluación temporal (`TemporalEvaluation`), la evolución temporal (`MIDTemporalEvolution`) y la expansión de redes temporales (`TemporalNetOperations`).

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:438-443

```
for (Node node : sortedNodes) {
    Variable oldVariable = node.getVariable();
    // Should the node be converted
    if (oldVariable.getVariableType() == VariableType.NUMERIC && node.getNodeType() ==
    ↪  NodeType.CHANCE) {
        EvidenceCase configuration = new EvidenceCase(evidence);
        Potential oldPotential = node.getPotentials().get(0);
```

**Cómo arreglarlo.** Dejar sin convertir un nodo numérico de azar que no tiene potencial (no hay valores que enumerar), o rechazarlo con una excepción del dominio que nombre el nodo. Qué conviene es cosa del algoritmo, pero no un `IndexOutOfBoundsException`.

El mismo supuesto («todo nodo de azar tiene potencial») está en error 29 y en el lector, que es donde podría imponerse de una vez.

Prueba: con la red A → X anterior, `convertNumericalVariablesToFS` no debe lanzar esa excepción y `VEPropagation` debe dar el posterior de A o un error con mensaje.

### Error 44 · [Corregido] El hallazgo sobre una variable numérica observada con un valor más fino que una centésima desaparece de la evidencia al discretizar

| | |
|---|---|
| **Estado** | Corregido en el commit `0fa7195` (15-10-2026), al corregir el error 45, con el arreglo descrito abajo: el hallazgo pasa siempre al único estado de la variable nueva. Con el 45 corregido, el nodo observado lleva la tabla que informa a sus padres, y perder el hallazgo dejaba de ser inocuo. Antes, cerrado sin arreglo el 30-09-2026. El defecto sigue en el código: de los 1001 valores 0,00; 0,01; …; 10,00 pierden el hallazgo 129. No se ha encontrado nada que un usuario |

| | |
|---|---|
| | pueda ver por su causa. Medido comparando el código de `d3a0ad0` con el arreglo descrito abajo (apuntar el hallazgo al estado 0): la propagación como la hace la ventana da las mismas probabilidades en `MID-Colorectal.pgmx`, con la política de cribado impuesta, la precisión de «Age [0]» puesta a 0,01 y un hallazgo de 50,05, tanto previo como posterior a la resolución; en `MID-Chancellor.pgmx` con duración de ciclo 0,1, la evolución temporal de «State [0]» y de «Life years [0]» da los mismos valores, y la de «Cost lamivudine [0]» difiere en el último dígito (1012,3756706466048 frente a 1012,3756706466047). El único fallo visible es pedir la evolución temporal de la propia variable numérica, «Time in treatment [0]»: con ciclo 0,1 muere con `NoSuchElementException` en `MIDTemporalEvolution.computePosteriorProbability`, y con el arreglo funciona. Pero la ventana no ofrece la evolución temporal para un nodo numérico de azar (`MainPanelMenuAssistant`), y el único código del producto que lanza esa tarea es ese diálogo. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:444-459`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:575-588`<br>`core/src/main/java/org/openmarkov/core/model/network/Variable.java:75` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 40, error 41, error 42, error 45 |

**Qué pasa.** Cualquier tarea exacta con evidencia previa a la resolución sobre una variable numérica de azar puede perder ese hallazgo sin aviso. La evidencia llega por el diálogo de añadir hallazgo en modo edición (cuyo selector usa la precisión de la variable como paso), desde el fichero, o inducida por potenciales delta y de desplazamiento de ciclo.

Las consecuencias visibles medidas son menores:

- La utilidad esperada (`VEEvaluation`) y las probabilidades de los demás nodos no cambian (N=7,0 y N=7,123 dan 10,0 en la red descrita más abajo). El valor observado ya queda recogido en la variable de un solo estado cuya tabla vale 1, y los hijos toman la rama que corresponde leyendo el nombre del estado; el hallazgo perdido sobre una variable de un solo estado no añade información. Por eso, hacia los hijos, el modelo no se resuelve como si nadie hubiera observado nada; hacia los padres sí, con cualquier valor observado (error 45).
- La propagación tal como la hace la ventana (`EvidenceManager.doPropagation`, con evidencia posterior a la resolución vacía pero no nula) tampoco cambia: al extender la evidencia posterior, la tabla de un solo estado induce de nuevo el hallazgo.
- Si se llama a `VEPropagation` sin evidencia posterior (como hace `VETemporalEvolution`) y se pregunta por esa misma variable numérica, con 7,0 devuelve [1,0] y con 7,123 lanza `NullPointerException` en `ProbNetOperations.getNodeAncestors`, llamado desde `VEPropagation.pruneNetwork`. Ningún llamador de producción pregunta hoy por una variable numérica de azar por ese camino (la exportación de la evolución temporal las salta).
- En `MIDTemporalEvolution` sobre `0_miscellaneous/networks/mid/MID-Chancellor.pgmx` con la duración de ciclo cambiada a 0,1, la probabilidad de «Time in treatment [3]» sale 1,0000000002241949 en lugar de 1,0: al perderse el hallazgo, el valor se calcula eliminando variables en lugar de salir directamente de la evidencia.

Por qué pasa. En `ProbNetOperations.convertNumericalVariablesToFS` (la discretización previa

a todo algoritmo exacto), una variable numérica de azar **con hallazgo** se sustituye por una variable de un solo estado cuyo nombre es el valor observado escrito con `String.valueOf(value)` (línea 450), y su potencial pasa a ser una tabla con un único 1. Al final del método (líneas 575-588) se recorren los hallazgos para volver a apuntarlos sobre la variable nueva: se redondea el valor con la precisión de la **variable nueva** y se busca el estado cuyo nombre sea `String.valueOf` de ese redondeo. Si no se encuentra, el `if` se salta la línea que vuelve a añadir el hallazgo, y el hallazgo desaparece de la evidencia de la tarea sin aviso.

Las dos cosas no encajan por dos razones:

1. La variable nueva se construye con el constructor de estados, que deja la precisión por defecto, 0,01 (línea 75 de `Variable`), en vez de heredar la de la variable numérica. Cualquier valor con más de dos decimales (7,123; 0,001) se redondea a otro número y su nombre no coincide.
2. Incluso con dos decimales, `Math.round(v / 0.01) * 0.01` no siempre devuelve exactamente `v` en coma flotante: para 0,35 da 0,35000000000000003. Se comprobó que 129 de los 1001 valores 0,00; 0,01; …; 10,00 pierden así su hallazgo. Un caso realista es la variable de tiempo de un modelo de Markov con duración de ciclo 0,1: a partir del tercer ciclo el valor inducido es 0,30000000000000004 y su hallazgo se pierde.

Medido con un diagrama de influencia de tres nodos, P (azar, dos estados al 50 %) → N (numérica de azar, vale 3 si P=p0 y 7 si P=p1) → U (utilidad: 0 si N ≤ 5, 10 si N > 5), el mismo de error 40, con un hallazgo previo a la resolución sobre N:

```
valor 7.0   -> estado "7.0",   round 7.0                -> hallazgo sobre N tras
↪  convertir: true
valor 0.35  -> estado "0.35",  round 0.35000000000000003 -> hallazgo sobre N tras
↪  convertir: false
valor 7.123 -> estado "7.123", round 7.12               -> hallazgo sobre N tras
↪  convertir: false
valor 0.001 -> estado "0.001", round 0.0                -> hallazgo sobre N tras
↪  convertir: false
```

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:447-451, 575-588

```
                    Finding finding = configuration.removeFinding(oldVariable);
                    if (finding != null) {
                        double value = finding.numericalValue;
                        Variable newVariable = new Variable(oldVariable.getName(),
↪  String.valueOf(value));
                        node.setVariable(newVariable);
...
        // Update evidence, replacing references to old variables with new ones
        List<Finding> findings = evidence.getFindings();
        for (Finding finding : findings) {
            Variable originalVariable = finding.getVariable();
            if (convertedVariables.containsKey(originalVariable)) {
                evidence.removeFinding(originalVariable);
                Variable convertedVariable = convertedVariables.get(originalVariable);
                double numericalValue =
                ↪  convertedVariable.round(finding.getNumericalValue());
                int stateIndex =
                ↪  convertedVariable.getStateIndex(String.valueOf(numericalValue));
                if (stateIndex != -1) {
                    evidence.addFinding(new Finding(convertedVariable, stateIndex));
                }
            }
```

```
        }
```

**Cómo arreglarlo.** En la rama de las variables observadas el estado es único y se sabe cuál es: el hallazgo se puede volver a apuntar con el índice 0 directamente, sin redondear ni buscar por nombre (o, si se prefiere buscar, nombrar y buscar con la misma función). Si se decide redondear el valor observado, debe hacerse una sola vez, con la precisión de la variable original y antes de nombrar el estado. Conviene también que la variable nueva herede la precisión de la original.

Otros sitios de la misma regla, «el nombre de un estado discretizado y su búsqueda usan la misma escritura del número»:

- la rama de variables sin hallazgo del mismo método nombra con `DecimalFormat("#.#####")` (línea 531), y `TreeADDPotential.replaceNumericVariable` vuelve a leer esos nombres con `Double.parseDouble`;
- `Variable.getStateIndex(double)` (línea 343) busca por `String.valueOf(round(value))`, que tiene el mismo problema para variables de estados finitos con nombres numéricos.

El `NullPointerException` de `VEPropagation.pruneNetwork` al preguntar por una variable numérica que la discretización ha sustituido es un defecto distinto, error 42; arreglar este error no lo arregla.

Prueba de regresión: convertir la red P → N con un hallazgo N=7,123 y otro N=0,35 y comprobar que la evidencia sigue conteniendo un hallazgo sobre la N convertida.

### Error 194 · Un hallazgo sobre un nodo definido por un potencial delta, con otro valor que el del delta, se sustituye sin aviso por ese valor si es previo a la resolución, y se rechaza como evidencia incompatible si es posterior

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network; org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/EvidenceCase.java:317-330`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/DeltaPotential.java:177-185`<br>`core/src/main/java/org/openmarkov/core/inference/tasks/TaskUtilities.java:47-54`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:262-283` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Qué significa un hallazgo sobre un nodo cuyo potencial es un delta: un valor que sustituye al del delta, como si este fuera un valor por omisión que el usuario cambia para un caso, o evidencia imposible, que hay que rechazar con un mensaje |
| **Relacionados** | error 41, error 44, error 45, error 48 |

**Qué pasa.** Visto al corregir el error 41. En `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-2.5.pgmx`, «Duration of treatment [0]» es un nodo numérico de azar con un potencial delta de valor 0. Con un hallazgo previo a la resolución de valor 2 (el que se pone en modo edición), al extender la evidencia el hallazgo pasa a valer 0, y la propagación da lo mismo que sin hallazgo: la política óptima elige «combination therapy» y la probabilidad de «State [1]» es [0,005; 0,0341; 0,1027;

0,8582]. Con el delta cambiado a 2, que es lo que diría el hallazgo si sustituyera al valor del delta, la política óptima elige «monotherapy» y «State [1]» es [0,0098; 0,0669; 0,2018; 0,7215]. El usuario no recibe ningún aviso.

Desde el commit `b8e0c8a`, que corrige el error 41, el mismo hallazgo puesto después de la resolución (en modo inferencia) no se sustituye: choca con el valor que el delta ha puesto en la evidencia previa y la propagación lo rechaza como evidencia incompatible; la ventana quita el hallazgo y lo comunica como describe el error 48. Antes de ese commit fallaba con `NullPointerException`. Los dos modos tratan hoy de forma distinta el mismo gesto.

En las redes del repositorio hay 15 nodos numéricos de azar con potencial delta: nueve son edades («Age», «Age at entry», «Age at diagnosis») y seis contadores de tiempo («Time in treatment», «Time in state», «Duration of treatment»). En `MID-dmhee-3.5.pgmx`, un hallazgo previo de 40 u 80 sobre «Age [0]», cuyo delta vale 60, también pasa a valer 60; en ese modelo la edad no cambia la utilidad esperada (579,833 con el delta en 40, 60 u 80), así que allí no se nota.

Por qué pasa: `EvidenceCase.extendInPlace`, que usan `TaskUtilities.extendPreResolutionEvidence` y `extendPostResolutionEvidence`, recorre todos los potenciales de la red y guarda los hallazgos que inducen con `findings.put`, que pisa el que ya hubiera sobre la misma variable (línea 328). `DeltaPotential.getInducedFindings` devuelve siempre su propio valor, sin mirar la evidencia que recibe.

**Cómo arreglarlo.** Depende de la decisión.

- Si el hallazgo sustituye al valor del delta, `extendInPlace` no debe pisar un hallazgo que ya está, y el valor que el delta pone en la evidencia previa no debe chocar con un hallazgo posterior sobre el mismo nodo.
- Si es evidencia imposible, `extendInPlace` debe rechazar el hallazgo del usuario con `IncompatibleEvidenceException` cuando el valor inducido es otro, como ya pasa con el hallazgo posterior, y la ventana decir por qué.

Prueba: en `MID-dmhee-2.5.pgmx`, un hallazgo de valor 2 sobre «Duration of treatment [0]» debe tratarse igual antes y después de la resolución, como diga la decisión.

### Error 45 · [Corregido] Observar un nodo numérico de azar no cambia la probabilidad de sus padres, y la utilidad esperada se calcula como si esa observación no dijera nada de ellos

| | |
|---|---|
| **Estado** | Corregido en el commit `0fa7195` (15-10-2026), con la opción A que eligió Manuel Arias: usar la densidad, y avisar solo donde no se pueda calcular. En `ProbNetOperations.convertNumericalVariablesToFS`, el nodo observado conserva a sus padres: su tabla dice, para cada configuración de ellos, cuánto de probable es el valor observado. Si la relación da un único valor (tabla, «Exact», función), es 1 o 0 según coincida con el observado, redondeando con la precisión de la variable; si es una distribución, su densidad en ese valor (`ProbDensFunction.getDensity`, nuevo, con la biblioteca `commons-math3` para la normal, la lognormal, las gamma, las beta, la triangular y «Range»); si es uniforme, la misma para todas. Dirichlet y complemento no tienen densidad sobre un valor numérico: el hallazgo se rechaza con `NonProjectablePotentialException.DensityCannotBeCalculated`, que nombra el nodo y la distribución. Un valor que ninguna configuración da es evidencia imposible, con el aviso de siempre. El recorrido de las configuraciones de los padres se sacó a un método que usan las dos ramas. Medido con la red de la ficha: relación exacta y N = 7 dan P(Disease) = [0, 1] y utilidad esperada 10; normal y N = 8, P(present) = 0,909 y utilidad 9,09, tanto con el hallazgo antes de resolver como después |

| | |
|---|---|
| | (la política se calcula antes y solo con la evidencia previa, así que el hallazgo posterior no la cambia). Corrige también el error 44. Con pruebas. No se ha probado en la ventana. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:420-459` `core/src/main/java/org/openmarkov/core/inference/tasks/TaskUtilities.java:88-90` `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VariableElimination.java:95-97` `gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:701-705` `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/ProbDensFunction.java:24-64` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué hacer con un hallazgo sobre un nodo cuya relación es una distribución de probabilidad (normal, gamma…): usar la densidad en el valor observado, que hoy no se puede calcular, o rechazar el hallazgo con un mensaje |
| **Relacionados** | error 40, error 41, error 42, error 44, error 48, nota 109 |

**Qué pasa.** En una red con un nodo numérico de azar que depende de un nodo de estados finitos (por ejemplo, una prueba cuyo resultado numérico depende de si hay enfermedad), el usuario pone en modo edición un hallazgo sobre el nodo numérico: menú contextual del nodo, «Add finding» (la opción está activa para todo nodo que no sea de utilidad, salvo los temporales con un padre del mismo nombre). Al entrar en modo inferencia, la probabilidad del padre sigue siendo la previa, como si no se hubiera observado nada. Como la causa está en la discretización que hacen todas las tareas exactas, lo mismo vale para las que reciben evidencia previa a la resolución: evaluación del diagrama de influencia, coste-efectividad, sensibilidad y evolución temporal (medido en la propagación y en la evaluación). No hay ningún aviso.

Hacia los hijos el hallazgo sí llega: el nodo queda fijado en el valor observado, y los nodos que dependen de él se calculan con ese valor (es lo que se mide en error 44). Lo que se pierde es lo que el valor observado dice sobre los padres.

Medido con redes de tres nodos construidas por programa: Disease (absent 0,9 / present 0,1) → N (numérico de azar, precisión 0,01) y, en el diagrama de influencia, además Disease → U (utilidad 0 si absent, 10 si present). Con un hallazgo previo a la resolución sobre N, propagando como la ventana (`VEPropagation` con todas las variables, igual que `EvidenceManager.doPropagation`) y evaluando con `VEEvaluation`:

```
N de estados finitos «3»/«7», tabla determinista, N = «7»         -> P(Disease) = [0.0,
↪  1.0]  utilidad esperada = 10.0
N numérico, relación exacta (3 si absent, 7 si present), N = 7    -> P(Disease) = [0.9,
↪  0.1]  utilidad esperada = 1.0
N numérico, normal (media 5 o 8, desviación 1), N = 8             -> P(Disease) = [0.9,
↪  0.1]  utilidad esperada = 1.0
```

La primera fila es la misma red con N de estados finitos, que da la respuesta correcta. En la tercera, la respuesta correcta es P(present) ≈ 0,909 (el cociente de las dos densidades normales en 8 multiplica por 90 la razón de probabilidades previa). Sin hallazgo, la red con la normal ni siquiera se puede propagar («Potential P(N | Disease) cannot be converted to a table»); con hallazgo se propaga sin quejarse y sin usar la distribución.

Por qué pasa. Antes de resolver con eliminación de variables, `VariableElimination.exact`

`AlgorithmsPreprocessing` llama a `TaskUtilities.discretizeNonObservedNumericVariables`, que es `ProbNetOperations.convertNumericalVariablesToFS`. Para cada nodo numérico de azar con hallazgo previo, ese método (líneas 444-459) pone en el nodo una variable nueva de un solo estado, cuyo nombre es el valor observado, y cambia su potencial por una tabla sobre esa única variable con el valor 1. La tabla nueva no tiene a los padres: desaparece el factor que decía cuánto de probable es el valor observado en cada configuración de los padres, y los padres se quedan con su distribución previa.

El javadoc del método solo habla de convertir las variables sin hallazgo. Tratar las observadas así es correcto cuando su valor lo fija un potencial delta y sus padres también están fijados, que es el caso de los modelos de Markov del repositorio («Time in treatment», «Age»): ahí perder el factor no cambia nada.

Camino de llegada. Ninguna red del repositorio lo alcanza tal cual: los nodos numéricos de azar de `0_miscellaneous/networks` y de `integrationTests/src/main/resources/networks` no tienen padres de estados finitos, o son temporales con un padre del mismo nombre (y entonces «Add finding» está desactivado), o su red falla antes por otros motivos. Se llega construyendo la red en el editor: un nodo numérico de azar admite las relaciones «Exact» y «UnivariateDistr» con padres de estados finitos.

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:441-459

```
            if (oldVariable.getVariableType() == VariableType.NUMERIC && node.getNodeType()
            ↪  == NodeType.CHANCE) {
                EvidenceCase configuration = new EvidenceCase(evidence);
                Potential oldPotential = node.getPotentials().get(0);
                if (configuration.contains(oldVariable)) {
                    // Convert numerical variables with evidence to one-state
                    // variables
                    Finding finding = configuration.removeFinding(oldVariable);
                    if (finding != null) {
                        double value = finding.numericalValue;
                        Variable newVariable = new Variable(oldVariable.getName(),
↪  String.valueOf(value));
                        node.setVariable(newVariable);
                        ...
                        TablePotential potential = new
↪  TablePotential(Arrays.asList(newVariable),

↪  oldPotential.getPotentialRole());
                        potential.getValues()[0] = 1;
                        node.setPotential(potential);
                    }
```

**Cómo arreglarlo.** El potencial del nodo observado tiene que conservar a sus padres: una tabla sobre la variable de un solo estado y los padres que diga, para cada configuración de los padres, cuánto de probable es haber observado ese valor.

- Si la relación es determinista (tabla, «Exact», función, suma o producto), basta con el recorrido de configuraciones que ya hace la rama de las variables sin hallazgo (líneas 461-526): proyectar el potencial en cada configuración, redondear con la precisión de la variable y poner 1 donde coincide con el valor observado y 0 donde no. Si ninguna configuración da ese valor, la evidencia es imposible y debe tratarse como tal (error 48).
- Si la relación es una distribución (`UnivariateDistrPotential`), lo que corresponde es la densidad en el valor observado para cada configuración. `ProbDensFunction` no tiene hoy ningún método que la calcule (tiene media, varianza, muestreo, intervalos…). Hay que decidir

si se añade ese cálculo a las distribuciones o si se rechaza el hallazgo con un mensaje que diga que no se puede usar.

Sitios de la misma regla: la rama de hallazgos de `ProbNetOperations.convertNumericalVariablesToFS` es el único sitio, pero la usan todas las tareas exactas (`VariableElimination.exactAlgorithmsPreprocessing`; `TemporalEvaluation`, línea 105; `MIDTemporalEvolution`, línea 339; `TemporalNetOperations`, línea 467), así que el arreglo llega a todas. Conviene hacerlo a la vez que error 44, que toca el mismo bloque (el nombre del estado nuevo y la búsqueda del hallazgo), y que error 40 (precisión cero). Hugin y el muestreo no pasan por aquí; cómo tratan ellos los nodos numéricos está en nota 109, donde no hay nada que corregir.

Prueba de regresión: la red Disease → N con relación exacta y hallazgo N = 7 debe dar P(Disease) = [0, 1] con `VEPropagation` y utilidad esperada 10 con `VEEvaluation`, igual que la misma red con N de estados finitos; y la utilidad esperada de `0_miscellaneous/networks/mid/MID-Chancellor.pgmx` no debe cambiar.

### Error 46 · [Corregido] Un nodo de utilidad cuyo rango no se puede calcular impide entrar en modo inferencia aunque la propagación funcionaría

| | |
|---|---|
| **Estado** | Corregido en el commit `2abb859` (30-09-2026): `EvidenceManager.computeUtilityRanges` calcula el rango nodo a nodo y deja sin él al nodo que falla; `InferencePresenter` solo lo aplica si existe, así que ese nodo conserva los límites del dominio de su variable. Comprobado con `MID-dmhee-4.7.pgmx`: se entra en modo inferencia y «Drug cost [0]» queda sin barra escalada. |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.window.edition.networkEditorPanel; org.openmarkov.core.model.network |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:558-598`<br>`core/src/main/java/org/openmarkov/core/model/network/UtilityFunctionComputer.java:82-119`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/InferencePresenter.java:60-69`<br>`gui/src/main/java/org/openmarkov/gui/graphic/NumericVariableBox.java:101-123`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:70-125`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:145-163` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 9, error 41, error 42, error 50, error 71, error 112, error 113 |

**Qué pasa.** Al entrar en modo inferencia en una red con un nodo de utilidad cuyo rango no se puede calcular, la red vuelve a modo edición y sale la ventana de error. El usuario no puede usar el modo inferencia en esa red, aunque la propagación en sí funcionaría y aunque los demás nodos de utilidad sí tengan rango.

El camino es `InferenceHandler.setWorkingMode` → `updateIndividualProbabilitiesAndUtilities` → `EvidenceManager.doPropagation`; la excepción llega a `MainPanelListenerAssistant` (líneas 155-171), que devuelve la red a modo edición y muestra la ventana de error.

Por qué pasa. En modo inferencia, los nodos de utilidad se dibujan con una barra cuyo largo se escala entre un mínimo y un máximo aproximados. `EvidenceManager.doPropagation` (módulo

`gui`, líneas 581-600) calcula esos límites al principio, con `calculateMinAndMaxUtilityRanges` (líneas 559-572), que para cada nodo de utilidad llama a `UtilityFunctionComputer.approximate MinUtility` y `approximateMaxUtility` (módulo `core`).

Estas funciones convierten el potencial del nodo en tabla (`tableProject`), y eso falla para varias clases de potencial: una distribución sobre un número (`UnivariateDistrPotential`), un `Uniform Potential` sobre una variable numérica, un árbol (`TreeADDPotential`) o una suma o producto (`SumPotential`, `ProductPotential`) cuyos sumandos fallan, o un potencial que depende de una variable numérica sin hallazgo (`MissingEvidenceInVariable`). La excepción (`NonProjectable PotentialException`) no se recoge en `doPropagation` (su `try` solo recoge `OutOfMemoryError`), así que no se propaga nada.

Medido con un pequeño programa de prueba sobre las redes de `0_miscellaneous/networks` (148 se leen, 2 no): 779 nodos de utilidad, 414 sin rango, en 73 redes. Para esas 73 se lanzó además `VEPropagation` sin evidencia y sin calcular rangos:

- En 4 la propagación termina bien y el cálculo de rangos es lo único que bloquea el modo inferencia, con un nodo sin rango en cada una: `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-2.5.pgmx` («Cost [0]»), `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-4.7.pgmx` («Drug cost [0]», error «There is required evidence to be present in variable Numeric variable Time in treatment [0]…»), `0_miscellaneous/networks/mid/MID-Chancellor.pgmx` y `0_miscellaneous/networks/mid/MID-Cochlear.pgmx`.
- En las otras 69 la propagación falla también por sí sola (sobre todo `PotentialCannot BeConvertedToATable`, y en algunas `UnreachableException`, `CannotResolveVariable`, `Not ApplicableNetwork` o `NullPointerException`, que es error 42); 60 de ellas son las copias en formato 1.0 de las carpetas `1-0`. Arreglar este error no hace funcionar esas 69; son otros defectos (en las copias en formato 1.0, sobre todo error 112).

Es decir, casi la mitad de las redes tiene algún nodo de utilidad sin rango, pero las que quedan bloqueadas solo por esta causa son 4.

**El código.** gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:559-570,581-590

```
    private void calculateMinAndMaxUtilityRanges() throws NonProjectablePotentialException
    ↪  {
        List<Variable> utilityVariables = this.networkEditorPanel.getVisualNetwork().
        ↪  getProbNet().getVariables(NodeType.UTILITY);
        for (Variable utility : utilityVariables) {
            ProbNet newNet =
↪   this.networkEditorPanel.getVisualNetwork().getProbNet().copy();
            ...
            this.minUtilityRange.put(utility,
            ↪  UtilityFunctionComputer.approximateMinUtility(node));
            this.maxUtilityRange.put(utility,
            ↪  UtilityFunctionComputer.approximateMaxUtility(node));
        }
    }
    ...
        try {
            this.calculateMinAndMaxUtilityRanges();
            Propagation vePosteriorValues = new
↪   VEPropagation(this.networkEditorPanel.getVisualNetwork().getProbNet());
```

**Cómo arreglarlo.** Calcular el rango nodo a nodo recogiendo el fallo de cada uno: el nodo sin rango se queda sin límites (o con los del dominio de su variable, que `NumericVariableBox` ya usa

al construirse, líneas 83-84) y los demás siguen.

Hay que tocar a la vez `InferencePresenter.paintInferenceResultsUtilityNode` (líneas 67-68): hoy pasa el `Double` del mapa a `setMinValue(double)`, y un rango ausente daría `NullPointerException` al desempaquetar.

De paso, copiar la red una sola vez fuera del bucle (hoy se copia una vez por nodo de utilidad).

Prueba: entrar en modo inferencia en `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-4.7.pgmx` (o llamar a `doPropagation` sobre su panel) debe terminar con las probabilidades calculadas y el nodo «Drug cost [0]» sin barra escalada.

### Error 47 · [Corregido] El diálogo de añadir hallazgo marca siempre el primer estado, de modo que aceptar sin tocar cambia el hallazgo que ya había, y deshacer ese cambio no devuelve el anterior

| | |
|---|---|
| **Estado** | Corregido en el commit `9ca2afa` (30-09-2026): `AddFindingDialog` pasa a `setNewFinding` el hallazgo que tenía el nodo, así que deshacer lo devuelve, y elige el estado inicial en un solo sitio (`initialState`). Al ejecutarlo se vio que la primera consecuencia de abajo no se daba: `ButtonGroup.add` desmarca el botón que llega marcado si el grupo ya tiene otro, y el del hallazgo se añadía antes que el del estado 0, así que aceptar sin tocar conservaba el hallazgo. La segunda sí se daba. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/AddFindingDialog.java:119-141,172-186`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:123-136,489-509`<br>`gui/src/main/java/org/openmarkov/gui/action/AddFindingEdit.java:29-58` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** Para una variable de estados finitos, el diálogo de añadir hallazgo se abre siempre con el estado 0 marcado (el que aparece último en la lista), sea cual sea el hallazgo que ya tenga el nodo. Consecuencias, deducidas leyendo el código (no se ha abierto el diálogo):

1. Si el nodo ya tiene un hallazgo en otro estado y el usuario abre el diálogo (menú contextual del nodo, «Add finding») y acepta sin tocar nada, `doOkClickBeforeHide` crea el hallazgo con el estado marcado, el 0, y `EvidenceManager.setNewFinding` sustituye el que había.
2. En modo edición, si el nodo ya tenía hallazgo previo a la resolución, `doOkClickBeforeHide` pasa `previousFinding` a `setNewFinding`, que crea `AddFindingEdit(visualNode, evidenceCase, previousFinding, finding)`. Al deshacer, `AddFindingEdit.undo` pone `previousFinding`, es decir, el estado 0, en lugar del hallazgo real anterior.

Las variables numéricas no están afectadas: el diálogo inicia el selector con el valor del hallazgo existente.

Por qué pasa. `EvidenceManager.addFinding` (líneas 130-137) abre `AddFindingDialog` para el nodo seleccionado pasándole el hallazgo que ya tenga en el caso de evidencia actual (en modo inferencia) o en la evidencia previa a la resolución (en modo edición). Para una variable de estados finitos, `getPrincipalPanel` crea un botón de opción por estado, recorriendo los estados del último al primero, todos en el mismo grupo:

- si hay hallazgo, marca el botón cuyo nombre coincide con el estado del hallazgo, y además, en todas las vueltas, pone `previousFinding` al estado de esa vuelta;
- en la vuelta `i == 0` marca el botón del estado 0 y pone `previousFinding` al estado 0, sin mirar si había hallazgo.

Como marcar un botón del grupo desmarca los demás, la última vuelta pisa la marca del hallazgo existente, y `previousFinding` vale siempre el estado 0.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/AddFindingDialog.java:122-138

```
            for (int i = states.length - 1; i >= 0; i--) {
                String stateName = states[i].getName();
                JRadioButton jRadioButton = new JRadioButton(stateName);
                if (finding != null) {
                    jRadioButton.setSelected(finding.getState().equals(stateName));
                    previousFinding = new Finding(this.variable, new State(stateName));
                }
                radioButtonsPanel.add(jRadioButton);
                jRadioButton.setActionCommand(stateName);
                if (i == 0) {
                    jRadioButton.setSelected(true);
                    previousFinding = new Finding(this.variable, new State(stateName));
                }
```

**Cómo arreglarlo.** Marcar el estado del hallazgo existente si lo hay, y el estado 0 solo cuando no hay hallazgo (`if (i == 0 && finding == null)`); y fijar `previousFinding = finding` una sola vez fuera del bucle (`null` si no hay hallazgo).

Prueba: construir el diálogo sin mostrarlo, o extraer la elección del estado inicial a un método, para una variable con tres estados y un hallazgo en el estado 2; debe quedar marcado el 2 y `previousFinding` debe ser ese hallazgo. En modo edición, cambiar un hallazgo previo de 2 a 1 y deshacer debe devolver el 2.

### Error 48 · [Corregido] La evidencia imposible recibe un tratamiento distinto según el gesto, y en el más común se presenta como un error del programa

| | |
|---|---|
| **Estado** | Corregido en el commit `dd3a7bc` (09-10-2026). Comprobado en la aplicación con `BN-asia.pgmx` («Tuberculosis = yes» y «TuberculosisOrCancer = no»): salía el diálogo de error no previsto con «All the values are 0.0 in potential P(X-ray)». Ahora `VEPropagation` lanza `IncompatibleEvidenceException.EvidenceIsImpossible` («These findings are impossible in this network: together they have probability zero.»), y el doble clic en un estado y quitar un hallazgo la enseñan en el diálogo de error normal. La propagación automática se deja como estaba: el hallazgo que no se puede añadir se deshace y los gestos que no tienen nada que deshacer la apagan. Queda sin tocar: el mensaje de la propagación estocástica («Samples weigth is 0»), el mensaje de dos hallazgos sobre la misma variable (error 186, del que solo se corrige la errata «evidente»), que la propagación se apague sin avisar, y el algoritmo de Hugin, que la ventana no usa |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window.edition.networkEditorPanel |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EditorCommandHandlerGenerator.java:269-277`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:183-190,527-541,394-401,633-825` |

| | |
|---|---|
| | `gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:146-164`<br>`gui/src/main/java/org/openmarkov/gui/dialog/PropagationOptionsDialogListener.java:67-72`<br>`stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/StochasticPropagationOutputFrame.java:194-201`<br>`gui/src/main/java/org/openmarkov/gui/dialog/OMExceptionHandler.java:17-59`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialTransform.java:45-48`<br>`core/src/main/resources/core/localize/core_class_localizations_en.xml:96-97`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:534-536` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No. Pedía decidir qué respuesta única dar y si al fallar la propagación debe apagarse la propagación automática. Manuel Arias lo decidió el 09-10-2026: una excepción propia con su mensaje, en el diálogo de error normal, y la propagación automática como estaba |
| **Relacionados** | error 45, error 54, error 67, error 85, error 86, error 108, error 147, error 186, nota 91, nota 105, nota 167 |

**Qué pasa.** La evidencia imposible es una situación normal de uso: el usuario introduce observaciones que el modelo considera imposibles. Hay dos formas:

- dos hallazgos contradictorios sobre la misma variable, que detecta `EvidenceCase.addFinding` (`IncompatibleEvidenceException.EvidenceIsIncompatibleWithOther`). Uno de los dos puede no haberlo puesto el usuario: la extensión de la evidencia añade el hallazgo que fija una tabla con un 1. Medido con `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-2.5.pgmx`, cuyo «State [0]» tiene probabilidad 1 en «State A»: un hallazgo en cualquier otro estado, previo o posterior a la resolución, hace fallar `VEPropagation.getAllEvidence` (línea 263) con el texto «The new evidence … is not compatible with an old evidente: …», que habla de una evidencia que el usuario no ha puesto y tiene una errata («evidente» por «evidence», `core/src/main/resources/core/localize/core_class_localizations_en.xml`, línea 220);
- un conjunto de hallazgos con probabilidad cero en la red: la probabilidad posterior sale toda a ceros y la normalización lanza `CannotNormalizePotentialException` (`TablePotentialTransform.normalize`, líneas 45-48); en la propagación estocástica el equivalente es `SamplesWeightIsZero`.

Todo acaba en el manejador global `OMExceptionHandler`: lo envuelto en `UnreachableException` (o una excepción no comprobada) abre el diálogo de «algo que un desarrollador no previó»; lo envuelto en `UnrecoverableException` abre un diálogo de error normal. Con la propagación automática activa (la opción por omisión), en modo inferencia, cada gesto acaba así:

- Doble clic en un estado de un nodo (el gesto principal): `EditorCommandHandlerGenerator`, líneas 269-277, envuelve todo en `UnreachableException` → diálogo de error no previsto.
- Quitar un hallazgo dejando una evidencia restante imposible: `EvidenceManager.removeFinding`, líneas 191-194, apaga la propagación y envuelve en `UnreachableException` → diálogo de error no previsto.
- Añadir un hallazgo por el diálogo del menú contextual: `setNewFinding` (líneas 529-540) deshace el hallazgo y relanza; el botón Aceptar de `OkCancelDialog` lo envuelve en `UnrecoverableException` → error normal.

- Entrar en modo inferencia: `MainPanelListenerAssistant`, líneas 158-171, vuelve al modo anterior y lanza `UnrecoverableException` → error normal.
- Crear un caso de evidencia, ir al primero, anterior, siguiente o último: `EvidenceManager`, líneas 656-822, apagan la propagación (`setPropagationActive(false)`) y relanzan; `GUIUtils.executeUIAction` lo convierte en error normal.
- Quitar todos los hallazgos (líneas 394-401): apaga la propagación y relanza.
- Aceptar el diálogo de opciones de propagación: `PropagationOptionsDialogListener`, líneas 68-72 → error normal.
- Propagación estocástica: `StochasticPropagationOutputFrame`, líneas 201-208 → error normal.

Además, el texto de `CannotNormalizePotentialException` es «All the values are 0.0 in potential {potential}», que no dice al usuario que su evidencia es imposible; y apagar la propagación cambia el estado de la ventana sin avisar. El único sitio que trata la contradicción como un problema de los datos, con contexto, es el lector PGMX (`PGMXReader_0_2`, líneas 534-536, `EvidenceIncompatibleInFile`).

No se ha ejecutado la ventana; la cadena se ha seguido leyendo el código.

**El código.** gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EditorCommandHandlerGenerator.java:269-277

```
try {
    this.networkEditorPanel.getEvidenceManager()
                           .toggleFinding(visualNode, visualState);
} catch (IncompatibleEvidenceException | NotEvaluableNetworkException |
         NonProjectablePotentialException | NotEnoughMemoryException |
         DoEditException | CannotNormalizePotentialException |
         ConstraintViolatedException | ThereIsNoPotentialInNodeException ex) {
    throw new UnreachableException(ex);
}
```

gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:191-194

```
} catch (NotEvaluableNetworkException | NonProjectablePotentialException |
↪  NotEnoughMemoryException |
         IncompatibleEvidenceException | CannotNormalizePotentialException |
↪  ConstraintViolatedException e) {
    this.networkEditorPanel.setPropagationActive(false);
    throw new UnreachableException(e);
```

**Cómo arreglarlo.** Una salida posible: una excepción del dominio con mensaje («esta evidencia es imposible en esta red», «estos dos hallazgos se contradicen»), lanzada donde ya se detecta —la normalización de la posterior en la propagación (`VEPropagation`), los pesos de las muestras (`StochasticPropagation`) y `EvidenceCase.addFinding`— y capturada una sola vez en los embudos que ya existen (`GUIUtils.executeUIAction`, `JMenuItemBuilder`, `OkCancelDialog`, `OMExceptionHandler`). Así sobran los envoltorios «inalcanzable» de `EditorCommandHandlerGenerator` y de `EvidenceManager.removeFinding`.

Hay que decidir a la vez qué pasa con la propagación automática al fallar (hoy unos sitios la apagan y otros no).

Cuidado: la normalización por columnas del aprendizaje y la normalización condicionada que necesita la evolución temporal (error 67) no son evidencia imposible y no deben recibir ese mensaje.

Pruebas: con una red de dos nodos y una tabla con un cero, introducir por cada gesto la evidencia

de probabilidad cero y comprobar que el mensaje y el tipo de diálogo son los mismos. Como la ventana no se prueba sin pantalla, al menos una prueba del núcleo que afirme el tipo y el mensaje de la excepción de `VEPropagation`.

### Error 49 · [Corregido] Construir una propagación o una evolución temporal cambia el tipo de análisis guardado en la red del usuario, sin pasar por deshacer ni marcarla como modificada

| | |
|---|---|
| **Estado** | Corregido en el commit `8806ba4` (30-09-2026): `ProbNetCopier.shallowCopy` da a la copia su propio objeto de opciones (`new InferenceOptions(...)`), así que las escrituras de `VEPropagation` y `MIDTemporalEvolution.forceUnicriterion` quedan en la copia del algoritmo. `ProbNetPotentialQueries.buildMarkovDecisionNetwork` no se ha cambiado: nada escribe en las opciones de la red de Markov que construye. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetCopier.java:64-65`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetPotentialQueries.java:133-142`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:83-94`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:173,545-551`<br>`core/src/main/java/org/openmarkov/core/inference/InferenceAlgorithm.java:35-38,73-84`<br>`core/src/main/java/org/openmarkov/core/action/core/MulticriteriaEdit.java:56,70`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:585`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:319,373-376` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 35, error 75, nota 6, nota 48 |

**Qué pasa.** Construir una propagación por eliminación de variables (`VEPropagation`) o una evolución temporal (`MIDTemporalEvolution`) sobre una red pone el tipo de análisis de esa red a un solo criterio. El cambio no pasa por el sistema de ediciones: no se deshace y no marca la red como modificada. La otra cara del mismo defecto de las copias de red, la lista de oyentes, es error 35.

Medido: se lee `0_miscellaneous/networks/id/ID-CEA-test-2therapies.pgmx` (dos criterios), se pone su tipo a `COST_EFFECTIVENESS` y se construye `new VEPropagation(red)` sin llamar a nada más; después, la red del llamador dice `UNICRITERION`, no está marcada como modificada y no hay nada que deshacer.

En la ventana se ve menos de lo que parece. La ventana construye `VEPropagation` sobre la red del usuario cada vez que propaga evidencia en modo inferencia (`EvidenceManager.java:585`), pero para entrar en modo inferencia con una red temporal o con más de un criterio pasa antes por el diálogo de opciones con el tipo forzado a `UNICRITERION` (`InferenceHandler.java:84-85`), que ya lo pone mediante `MulticriteriaEdit`. En los recorridos seguidos, la escritura del constructor coincide con lo que la ventana ya había dejado, así que no se ha encontrado un gesto en el que el usuario vea un número distinto por ella. Lo comprobado es:

- el tipo de análisis de la red del usuario cambia fuera del sistema de ediciones: no se deshace y no marca la red como modificada; si luego se guarda por otro motivo, el fichero lleva el tipo que dejó el algoritmo;
- cualquier llamador sin ventana (una prueba, el módulo `resttemplate`, un programa que use OpenMarkov como biblioteca) que construya una propagación sobre un modelo de coste-efectividad se queda con el modelo pasado a un criterio;
- el orden de construcción importa: `MIDTemporalEvolution(red, utilidades)` lee el tipo de las opciones compartidas (líneas 189-191) justo después de que otro constructor lo haya podido pisar.

**Contexto.** Cada red guarda unas opciones de inferencia (`InferenceOptions`): entre ellas, el tipo de análisis multicriterio (`MulticriteriaOptions.Type`: `UNICRITERION`, un solo criterio, o `COST_EFFECTIVENESS`, coste-efectividad), las opciones temporales y las de Monte Carlo. Esas opciones las elige el usuario en el diálogo de opciones de inferencia y se guardan en el fichero `.pgmx`. Cuando el diálogo las cambia, lo hace con una edición (`MulticriteriaEdit`), que se puede deshacer y marca la red como modificada.

**Por qué pasa.** Todo algoritmo de inferencia (`InferenceAlgorithm`) documenta su campo `probNet` como «una copia de la red recibida» y hace `network.copy()` (líneas 35-38 y 84). Pero la copia superficial **no copia** las opciones: `ProbNetCopier.shallowCopy` le pone a la copia el mismo objeto (`dest.setInferenceOptions(source.getInferenceOptions())`, línea 65). La red de Markov que construye `ProbNetPotentialQueries.buildMarkovDecisionNetwork` hace lo mismo (línea 140). La copia profunda, en cambio, sí las copia (`new InferenceOptions(...)`, línea 83).

Dos constructores escriben en las opciones de «su copia», que son las de la red del llamador:

- `VEPropagation` (los dos constructores, líneas 85-86 y 91-92) pone el tipo a `UNICRITERION`.
- `MIDTemporalEvolution.forceUnicriterion` (líneas 545-551), llamado desde el constructor de una variable (línea 173) y desde el diálogo de evolución temporal por criterio (`TraceTemporalEvolutionDialog.java:376`), hace lo mismo.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/-tasks/VEPropagation.java:83-87

```
public VEPropagation(ProbNet network) throws ... {
    super(network);
    probNet.getInferenceOptions().getMultiCriteriaOptions()
           .setMulticriteriaType(MulticriteriaOptions.Type.UNICRITERION);
}
```

core/src/main/java/org/openmarkov/core/model/network/ProbNetCopier.java:64-65

```
// Shared like the potentials: a shallow copy carries the very options of the original.
dest.setInferenceOptions(source.getInferenceOptions());
```

**Cómo arreglarlo.** Que la copia superficial copie las opciones (`new InferenceOptions(source.getInferenceOptions())`, como ya hace la profunda en la línea 83). Es un objeto pequeño, y compartir las tablas de números, que es lo que hace barata la copia, no cambia. El mismo cambio hace falta en `ProbNetPotentialQueries.buildMarkovDecisionNetwork` (línea 140), que también reparte el mismo objeto. Con eso las escrituras de `VEPropagation` y `MIDTemporalEvolution.forceUnicriterion` quedan en la copia del algoritmo.

Hay que revisar a la vez si algún llamador depende hoy de leer, en la red original, lo que el algoritmo escribió. Hay que buscar lecturas de `getMulticriteriaType()` posteriores a construir un algoritmo: `MIDTemporalEvolution.java:189-191`, `TemporalEvaluation.java:81`,

`DecompositionGenerateDecisionTree.java:58`, `SensitivityAnalysisController.java:97`, `DecisionTreeEditor.java:310`.

Prueba de regresión: leer un modelo con dos criterios, poner su tipo a coste-efectividad, construir `VEPropagation` y `MIDTemporalEvolution` (constructor de una variable) sobre él y comprobar que el tipo de la red sigue siendo coste-efectividad; y comprobar que `copy()` devuelve un objeto de opciones distinto con los mismos valores.

### Error 227 · [Corregido] En una red de análisis de decisiones con restricciones, la propagación sin evidencia responde que los hallazgos son imposibles

| | |
|---|---|
| **Estado** | Corregido en el commit `ac0cc81` (09-10-2026): la propagación deja a cero en todos sus estados la variable que no existe, y solo responde que los hallazgos son imposibles cuando la probabilidad de los propios hallazgos es cero (la calcula aparte, sobre los hallazgos y sus antepasados). En modo inferencia ese nodo dice «does not exist» y no tiene valores. Con esto el modo inferencia se ofrece ya a las redes de análisis de decisiones: comprobado en la aplicación que las 34 de `0_miscellaneous/networks/dan` entran sin ningún diálogo de error, y visto el nodo «Result of test» de `DAN-reactor`. **Mejora posible, sin hacer:** el artículo (Díez, Luque, König y Bermejo, «Decision analysis networks», informe técnico CISIAD-14-01, UNED, 2014, sección II.B) dice que una restricción total «implies that in some scenarios the variable Y does not exist», pero no qué enseñar cuando la variable existe solo en algunos escenarios. Hoy se enseña su probabilidad dado que existe (las barras suman 1). La alternativa es enseñar la probabilidad total, con las barras sumando la probabilidad de que exista y el resto como «no existe» |
| **Módulo** | inference, core |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.tasks |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:225-233`<br>`0_miscellaneous/networks/dan/DAN-reactor.pgmx` |
| **Comprobado** | Ejecutando la propagación del modo inferencia por programa, sin evidencia; no en la ventana, porque el botón no deja entrar |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir qué enseñar como probabilidad de una variable en una DAN con restricciones. Manuel Arias lo decidió el 09-10-2026: la que no existe, sin valores; la que existe solo en algunos escenarios, su probabilidad dado que existe, al menos por el momento |
| **Relacionados** | error 201, error 48 |

**Qué pasa.** Este error es un paso del plan para ofrecer el modo inferencia a las redes de análisis de decisiones (DAN) y a los diagramas de influencia de Markov (MID), decidido el 09-10-2026 (error 201). Hoy un usuario no llega a él, porque el botón del modo inferencia está apagado para esos tipos de red; llegará en cuanto se encienda.

Medido: en `DAN-reactor.pgmx` y en su versión en formato 1.0, `VEPropagation.getPosteriorValues`, con todas las variables y sin ningún hallazgo, lanza `IncompatibleEvidenceException.EvidenceIsImpossible` («These findings are impossible in this network: together they have probability zero»). El mensaje es falso: no hay hallazgos. Las otras 66 redes de análisis de decisiones del repositorio propagan.

La red del reactor tiene cuatro restricciones en sus enlaces. El artículo que define estas redes (Díez, Luque, König y Bermejo, «Decision analysis networks», sección II.B) dice que una restricción $(x, y)$ sobre una variable de azar equivale a poner a cero la probabilidad correspondiente. La probabilidad de alguna variable sale así toda a ceros, y la propagación lo toma por evidencia imposible desde el

commit `dd3a7bc` (antes lanzaba «All the values are 0.0 in potential …»).

Medido después, pidiendo las variables una a una: fallan «Result of test» y «Result of advanced reactor»; las demás propagan. La propagación resuelve antes las decisiones con su política óptima, que aquí es no hacer la prueba («Test decision» = [0, 1]) y construir el reactor convencional («Build decision» = [0, 1, 0]). Con esa estrategia la prueba no se hace y el reactor avanzado no se construye, así que sus resultados no tienen ningún valor posible: son restricciones totales. El artículo lo dice así: una restricción total $(x, Y)$ «implies that in some scenarios the variable Y does not exist». Lo que la propagación llama evidencia imposible es una variable que, con esa estrategia, no existe.

**Cómo arreglarlo.** Hay dos cosas distintas que resolver: que la propagación no llame evidencia imposible a lo que no lo es, y qué debe enseñar el modo inferencia en ese nodo. El artículo define el significado de una DAN por su árbol de decisión equivalente (sección III) y no se ha encontrado que diga qué es la probabilidad de una variable suelta; esa es la decisión.

Prueba: la propagación de `DAN-reactor.pgmx` sin evidencia termina, y con dos hallazgos imposibles sigue dando el mensaje del error 48.

### Error 228 · [Corregido] La propagación de `MID-HPV` termina con NullPointerException al podar la red

| | |
|---|---|
| **Estado** | Corregido en el commit `3a978f8` (09-10-2026). La causa no era la que suponía la ficha: fallaban solo «AR QoL [0]» y «HSR QoL [0]», los dos nodos de utilidad que suma «QoL [0]». Para calcular un nodo así, el arreglo del error 42 volvía a discretizar las variables numéricas en una copia de la red; eso les daba variables distintas de las que nombran los hallazgos (la edad de cada ciclo, que se conoce con certeza), la poda no las reconocía como protegidas y las quitaba. Ahora la copia se toma con las variables ya discretizadas. `MID-HPV` propaga entera en unos 4 segundos; los valores de «QoL [0]», «Cost of cytology [0]» y «Health state» no cambian. Los dos nodos salen con utilidad 0, como las demás utilidades del ciclo 0 de esa red; no se ha comprobado por qué, y es parte de la duda sobre qué debe enseñar el modo inferencia en un MID (error 229) |
| **Módulo** | inference, core |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.tasks |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:70,220`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:278`<br>`0_miscellaneous/networks/mid/MID-HPV.pgmx` |
| **Comprobado** | Ejecutando la propagación del modo inferencia por programa, sin evidencia; no en la ventana, porque el botón no deja entrar |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 201 |

**Qué pasa.** Este error es un paso del plan para ofrecer el modo inferencia a las redes de análisis de decisiones (DAN) y a los diagramas de influencia de Markov (MID), decidido el 09-10-2026 (error 201). Hoy un usuario no llega a él, porque el botón del modo inferencia está apagado para esos tipos de red; llegará en cuanto se encienda.

Medido: en `MID-HPV.pgmx` y en su versión en formato 1.0, `VEPropagation.getPosteriorValues`, con todas las variables y sin evidencia, lanza `NullPointerException`: «Cannot invoke "Node.getNeighbors()" because "node" is null», en `ProbNetOperations.removeUnreachableNodes` (línea 220), llamado desde `ProbNetOperations.getPruned` (línea 70) y este desde `VEPropagation.pruneNetwork`

(línea 278). Tarda 3,4 segundos en llegar al fallo.

La poda recibe una variable de interés que no tiene nodo en la red que se poda. No se ha mirado cuál es ni en qué paso anterior desaparece.

**Cómo arreglarlo.** Localizar la variable y el paso que la quita de la red (la discretización o la absorción de nodos numéricos son los candidatos, por lectura). Según lo que salga, o la poda no debe recibirla, o el paso anterior no debe quitarla. Que `removeUnreachableNodes` acepte un nodo nulo sin decir nada tapa el problema.

Prueba: la propagación de `MID-HPV.pgmx` sin evidencia termina.

### Error 229 · [Corregido] Tres MID no propagan porque la relación uniforme de una variable numérica sin hallazgo no se convierte en tabla

| | |
|---|---|
| **Estado** | Corregido en el commit `d868666` (10-10-2026), como se decidió: la evaluación pide un hallazgo para la variable numérica sin valor («The numeric variable Age [0] has a uniform distribution, so it has no value to compute with. Set a finding on it in edition mode and try again.»). Con el hallazgo, `MID-mammography` se evalúa y propaga (su otro fallo era el del error 230); `MID-Colorectal` también, pero tarda minutos (error 231). `MID-CHD-Walker` no puede evaluarse por otra razón, que no es un fallo del programa: en el fichero, sus siete nodos de utilidad llevan la relación uniforme con que se crea un nodo, sin números; el mensaje lo dice ahora («The utility of the node C: ETT is not defined …»). El hallazgo tiene que ponerse en modo edición: puesto en modo inferencia, la conversión de las variables numéricas no lo usa. Lo que decía antes esta ficha: Medido el 09-10-2026, después de la decisión: **poner el hallazgo no basta, y el fallo no es del modo inferencia.** Estas tres redes tampoco se evalúan con las demás herramientas: el análisis de coste-efectividad (`VECEAnalysis`) y la evolución temporal (`MIDTemporalEvolution`) dan el mismo «Potential P(Age [0]) cannot be converted to a table», y a eso un usuario sí llega hoy. Con un hallazgo previo a la resolución «Age [0] = 50», la propagación sigue sin terminar bien y cada red falla en otro sitio: `MID-CHD-Walker`, en otra relación uniforme («Potential 2 Variables: C: ETT, Dec: ETT cannot be converted to a table»); `MID-Colorectal` no acaba en 60 segundos; `MID-mammography` da «An expression cannot be resolved as there is no value resolved for variable Age [0]». Con el mismo hallazgo puesto después de la resolución, las tres siguen dando el fallo original: la discretización no lo usa |
| **Módulo** | inference, core |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.tasks |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/UniformPotential.java:98`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:499` |
| **Comprobado** | Ejecutando la propagación del modo inferencia por programa, sin evidencia; no en la ventana, porque el botón no deja entrar |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No. Manuel Arias lo decidió el 09-10-2026: la propagación pide un hallazgo para la variable numérica sin valor; y el modo inferencia de un MID enseña por omisión la red de dos ciclos, y la desplegada cuando el usuario la pida |
| **Relacionados** | error 201, error 40, error 45 |

**Qué pasa.** Este error es un paso del plan para ofrecer el modo inferencia a las redes de análisis de decisiones (DAN) y a los diagramas de influencia de Markov (MID), decidido el 09-10-2026 (error 201). Hoy un usuario no llega a él, porque el botón del modo inferencia está apagado para esos tipos de red; llegará en cuanto se encienda.

Medido: en `MID-CHD-Walker`, `MID-Colorectal` y `MID-mammography`, en sus dos formatos, `VEPropagation.getPosteriorValues` sin evidencia lanza `PotentialCannotBeConvertedToATable`: «Potential P(Age [0]) cannot be converted to a table», en `UniformPotential.tableProject` (línea 98), llamado desde `ProbNetOperations.convertNumericalVariablesToFS` (línea 499).

«Age [0]» es una variable numérica con una relación uniforme y sin hallazgo. El artículo que define los MID (Díez, Yebra, Bermejo y otros, «Markov influence diagrams», *Medical Decision Making*, 2017, sección de evaluación) dice que la evaluación exacta vale para MID con variables discretas «and numeric variables whose value is known with certainty», y que «some MIDs having numerical variables may require patient-level simulation». En estas redes la edad inicial no se conoce con certeza.

El mismo fallo se vio el 09-10-2026 en un diagrama de influencia, `ID-delayed-result-of-test`, al corregir el error 42; no entró en esta medida porque esa red propaga sin evidencia.

**Cómo arreglarlo.** Depende de la decisión, que es de modelo y conviene consultar con Javier Díez. Hay además una duda previa que afecta a todos los MID: el artículo describe su evaluación «by expanding it to the desired horizon», y la propagación medida trabaja sobre la red sin desplegar, con sus dos ciclos. No se ha encontrado en el artículo qué debe enseñar el modo inferencia en un MID.

Prueba: según la decisión, las tres redes propagan, o el modo inferencia explica por qué no se ofrece.

### Error 230 · [Corregido] `MID-CHAP-Ryan-Griffin` no propaga porque una expresión usa una variable numérica sin valor

| | |
|---|---|
| **Estado** | Corregido en el commit `971f541` (10-10-2026). Eran dos cosas. Al copiar un nodo al ciclo siguiente, una regresión desplazaba sus variables pero no las de las fórmulas de sus covariables (`GLMPotential.shift` solo llamaba al de la clase base), así que en el ciclo 1 la fórmula seguía pidiendo «Age [0]». Y la tabla que sale de una utilidad dada por un árbol con un modelo exponencial conservaba la variable de utilidad, con lo que la política óptima de la terapia dependía de los costes y la propagación tenía un ciclo; ese segundo defecto solo aparece en esta red, en cuatro nodos. Ahora el análisis de coste-efectividad da un resultado y la propagación da valor a 17 de sus 18 nodos. No hay resultado de referencia de esta red con que contrastar los números. Lo que decía antes esta ficha: Medido el 09-10-2026: **la ficha se equivocaba al decir que la edad no tiene valor.** En esta red «Age [0]» tiene una relación delta, es decir, un valor fijo del modelo, y aun así la expresión no lo encuentra; con un hallazgo «Age [0] = 50», previo o posterior a la resolución, el fallo es el mismo. Tampoco es del modo inferencia: el análisis de coste-efectividad y la evolución temporal de esta red dan el mismo mensaje. No depende, por tanto, del error 229 |
| **Módulo** | inference, core |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.tasks |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/expression/ReferencedExpression.java:60,89,117` |
| **Comprobado** | Ejecutando la propagación del modo inferencia por programa, sin evidencia; no en la ventana, porque el botón no deja entrar |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. No es la decisión del error 229: aquí la variable sí tiene valor |
| **Relacionados** | error 201, error 229 |

**Qué pasa.** Este error es un paso del plan para ofrecer el modo inferencia a las redes de análisis de decisiones (DAN) y a los diagramas de influencia de Markov (MID), decidido el 09-10-2026 (error 201). Hoy un usuario no llega a él, porque el botón del modo inferencia está apagado para esos tipos de red; llegará en cuanto se encienda.

Medido: en `MID-CHAP-Ryan-Griffin`, en sus dos formatos, `VEPropagation.getPosteriorValues` sin evidencia lanza `CannotResolveVariable`: «An expression cannot be resolved as there is no value resolved for variable Age [0]», en `ReferencedExpression` (líneas 89, 60 y 117).

Es la otra cara del error 229: aquí la edad no falla al convertirse en tabla, sino al evaluar la expresión de otro nodo que la usa.

**Cómo arreglarlo.** Lo que se decida en el error 229. Si la propagación pide un hallazgo en la edad, este error desaparece con él; si la discretiza, la expresión tiene que evaluarse para cada valor.

Prueba: la propagación de `MID-CHAP-Ryan-Griffin.pgmx` termina, o el modo inferencia explica por qué no se ofrece.

### Error 231 · [Corregido] La propagación de `MID-Cochlear` no termina en un minuto

| | |
|---|---|
| **Estado** | Corregido en el commit `271c669` (16-10-2026), con la opción que eligió Manuel Arias ese día. La causa: al desplegar el modelo, el nodo de utilidad de cada ciclo recibía una copia del criterio (`ProbNet.addShiftedNode` clona la variable, y con ella su criterio), y la eliminación solo suma las utilidades de un criterio cuando comparten el objeto (`TablePotentialArithmetic.sumByCriterion`): en los 100 ciclos de `MID-Cochlear` había 603 objetos para 3 criterios, las tablas de utilidad no se sumaban y cada una crecía hasta el tamaño de la probabilidad conjunta (al eliminar «Alive [23]» entraban 462 tablas con 5,3 millones de casillas y salían 458 millones). Ahora el nodo del ciclo siguiente comparte el criterio del nodo del que sale. Medido: la evaluación de `MID-Cochlear` baja de 138 a 6,6 segundos, su propagación de 152 a 9,4 y su análisis de coste-efectividad de 118,6 a 5,8, con los mismos resultados; la propagación de `MID-Colorectal` con la edad en 50 baja de 188 a 29 segundos (medido con la batería de pruebas corriendo a la vez); las otras 27 redes de diagramas de influencia y MID dan las mismas políticas y utilidades. Con prueba. Antes: desde el commit `1e8b521` la red ya no deja la aplicación parada: la propagación se puede seguir y parar. El tiempo de la primera propagación sigue igual. Desde el commit `124a9ba` (10-10-2026) las políticas óptimas se calculan una vez al entrar en modo inferencia y no en cada hallazgo: con un hallazgo, `MID-HPV` propaga en 3,3 segundos en vez de 5,4, y `MID-hip-Briggs` en 1,1 en vez de 1,7. Lo que queda en cada hallazgo es la segunda etapa, que calcula cada nodo con una eliminación aparte. **Medido con la simulación de las eliminaciones, sin cambiar el algoritmo:** sumar entre sí las tablas de utilidad que tienen las mismas variables no ahorra nada (0 % en `MID-HPV` y en `MID-Cochlear`: casi ninguna las comparte); eliminar siempre la variable más barata en vez de seguir el orden actual deja el trabajo en el 16 % en `MID-HPV` y lo multiplica por 45 en `MID-Cochlear`, así que no sirve como regla general. Medido el 09-10-2026: el tiempo no se va en la propagación, sino en lo que hace antes. La red tiene una decisión sin política («Intervention decided») y horizonte 100, y la propagación empieza por calcular la política óptima evaluando la red desplegada (`VEPropagation.calculateOptimalPolicies` → `VEEvaluation.getOptimalPolicies`). Esa evaluación, con un solo criterio, no había terminado a los 120 segundos; el análisis de coste-efectividad de la misma red (`VECEAnalysis`) termina en 116 segundos. Por eso no termina ninguna variable, ni siquiera «Gender», que no tiene padres. Es un problema de la evaluación de esta red, no del modo inferencia. Medido después, eliminación |

| | |
|---|---|
| | a eliminación: la evaluación con un criterio sí termina, en 143 segundos, tras eliminar 1414 variables de la red desplegada, del último ciclo hacia el primero. Las 1000 primeras eliminaciones tardan 1,5 segundos; las 400 restantes, 141. Los nodos de utilidad se mantienen separados, que es la condición del artículo, pero eso hace que cada eliminación recorra todas las tablas de utilidad acumuladas hasta entonces: 59 al llegar al ciclo 92, 388 en el ciclo 36 y 604 al final, con tablas de hasta 991 232 valores. La eliminación más lenta tardó 4 segundos. En `MID-Chancellor` son 21 variables y 13 milisegundos |
| **Módulo** | inference, core |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.tasks |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java` `0_miscellaneous/networks/mid/MID-Cochlear.pgmx` |
| **Comprobado** | Ejecutando la propagación del modo inferencia por programa, sin evidencia; no en la ventana, porque el botón no deja entrar |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 201 |

**Qué pasa.** Este error es un paso del plan para ofrecer el modo inferencia a las redes de análisis de decisiones (DAN) y a los diagramas de influencia de Markov (MID), decidido el 09-10-2026 (error 201). Hoy un usuario no llega a él, porque el botón del modo inferencia está apagado para esos tipos de red; llegará en cuanto se encienda.

Medido: en `MID-Cochlear`, en sus dos formatos (34 nodos), `VEPropagation.getPosteriorValues` con todas las variables y sin evidencia no había terminado a los 60 segundos, que es donde se cortó la medida. No se sabe si termina más tarde ni dónde se va el tiempo. Las otras redes MID que propagan lo hacen en menos de 5 segundos.

El artículo que define los MID dice que la complejidad de la evaluación crece linealmente con el horizonte cuando todas las decisiones son atemporales, y que sus redes se evalúan con OpenMarkov en menos de 10 segundos. Esa frase habla de la evaluación de la red desplegada, no de esta propagación.

**Cómo arreglarlo.** Lo que hay que mirar es la evaluación de la red desplegada a 100 ciclos: qué tamaño alcanzan las tablas al eliminar y si los nodos de utilidad se mantienen separados hasta eliminar la decisión, que es la condición que pone el artículo para que el coste no crezca exponencialmente con el horizonte («value nodes should be kept separate until the moment of eliminating the first decision»). No se ha mirado todavía.

Prueba: la propagación de `MID-Cochlear.pgmx` sin evidencia termina en un tiempo que se pueda esperar en la ventana.

### Error 232 · [Corregido] La red que abre «Expand network» no entra en modo inferencia: sale el diálogo de error no previsto

| | |
|---|---|
| **Estado** | Corregido en el commit `08a3c4d` (09-10-2026). Al convertir una variable numérica en discreta, un árbol la sustituía en la raíz de sus ramas pero no en la lista de variables padre que guarda cada rama (`TreeADDPotential.replaceNumericVariable`); como «Expand network» renombra las variables, la antigua ya no se encontraba al copiar el árbol. Visto en la aplicación: la red desplegada de `MID-Chancellor` entra en modo inferencia sin ningún diálogo |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.window; org.openmarkov.core.model.network |

| | |
|---|---|
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:176-200`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:440-500` |
| **Comprobado** | En la aplicación, con un guion que lanza las acciones de menú |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 201 |

**Qué pasa.** Camino del usuario: abrir `0_miscellaneous/networks/mid/MID-Chancellor.pgmx`, clic derecho sobre el fondo de la red → «Expand network», aceptar las opciones de inferencia. Se abre en otra pestaña la red desplegada, convertida en diagrama de influencia, con 148 nodos. Al pulsar en ella el botón de modo inferencia sale el diálogo «Unexpected error … Please, send this to the developers of OpenMarkov», con el texto «Argument is invalid: Argument copyNet (Time in treatment [0]) is invalid because network MID-Chancellor_extended …», y la red se queda en modo edición.

Visto el 09-10-2026 con un guion, no con el ratón. El mensaje se cortó en la medida y no se ha mirado dónde se lanza. Solo se ha probado con esa red.

Importa para el error 201: se decidió que el modo inferencia de un MID enseñe la red de dos ciclos por omisión y la desplegada cuando el usuario la pida, y esta es la acción que ya existe para pedirla.

**Cómo arreglarlo.** Primero, localizar la excepción: el texto apunta a una operación que copia un nodo («copyNet») y encuentra una variable numérica, «Time in treatment [0]», que no espera en esa red. Sin esa lectura no se puede proponer nada.

Prueba: la red desplegada de `MID-Chancellor` propaga.

### Error 233 · Con la opción de transiciones a medio ciclo, la evaluación falla si alguna utilidad temporal está dada por un árbol

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:296-330` |
| **Comprobado** | Ejecutándolo por programa; no en la ventana |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué hace la opción de medio ciclo cuando la utilidad de un estado no es constante: el artículo de Pérez-Martín, Bermejo y Díez (2019) dice que la corrección de medio ciclo no se puede aplicar a ese modelo y propone otra cosa |
| **Relacionados** | error 201 |

**Qué pasa.** Medido el 09-10-2026: en `MID-Chancellor.pgmx`, con la opción de transición puesta a medio ciclo («Half»), calcular las políticas óptimas (`VEEvaluation.getOptimalPolicies`) termina con `ClassCastException`: «TreeADDPotential cannot be cast to ExactDistrPotential», en `TemporalNetOperations.applyTransitionTime` (línea 311). Con «Beginning» y con «End» la misma red se evalúa. El usuario llega eligiendo «Half» en las opciones de inferencia y pidiendo cualquier análisis de esa red; no se ha hecho en la ventana.

Para el medio ciclo, ese método pone en cada nodo de utilidad la media de su tabla y la del ciclo anterior, y para eso convierte la relación a `ExactDistrPotential` sin mirar de qué clase es. En esa red el coste de la lamivudina está dado por un árbol, porque depende del tiempo en tratamiento.

El código lleva un comentario de 2022 que ya lo dice: «cast to ExactDistrPotential but not every utility potential is ExactDistrPotential».

El artículo de Pérez-Martín, Bermejo y Díez («Evaluation of Markov models with discontinuities», *Medical Decision Making*, 2019), cuyo ejemplo es esta misma red, dice que la corrección de medio ciclo «cannot be applied to this model because it assumes that the cost function for each state is constant», y propone, para los modelos con discontinuidades, aplicar la regla del trapecio a un modelo en el que el coste en el punto de discontinuidad es la media de sus límites por la izquierda y por la derecha.

**Cómo arreglarlo.** Como mínimo, que la opción no termine en una excepción de conversión: o se hace la media también con árboles, o se responde que esa opción no vale para esa red. Qué debe hacer es la decisión, y el artículo citado es la referencia.

Prueba: `MID-Chancellor.pgmx` con «Half» se evalúa, o da el mensaje que se decida.

### Error 267 · [Corregido] Al discretizar un nodo numérico de azar con dos o más padres, algunas configuraciones de los padres no se preguntan nunca y otras se preguntan dos veces: el nodo pierde valores que el modelo da, un hallazgo posible se rechaza como imposible y otro no informa de lo que debería

| | |
|---|---|
| **Estado** | Corregido en el commit `3f91087` (15-10-2026): cuando un padre vuelve a su primer estado, `forEachConfigurationOfParents` lo pone también en la configuración, igual que al empezar; las tres escrituras del estado de un padre pasan por un solo método. La red de la ficha da los estados 1\|2\|3\|4, ver 3 deja A = a0 seguro y ver 4, A = a1 |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:463-465,489-498,572-626` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 45, error 44 |

**Qué pasa.** Antes de resolver con un algoritmo exacto (propagación del modo inferencia, evaluación, coste-efectividad…), `ProbNetOperations.convertNumericalVariablesToFS` da estados a cada nodo numérico de azar. Para eso recorre todas las configuraciones de sus padres, el primero cambiando más deprisa, y en cada una pregunta lo que vale el nodo (si nadie lo ha observado) o cuánto de probable es el valor observado (si tiene hallazgo, rama nueva de `0fa7195`). Con dos o más padres, el recorrido pregunta mal: desde el segundo padre, cada vuelta pregunta con los padres anteriores en su último estado, y no en el primero. Con dos padres A y B de dos estados, las cuatro preguntas son (a0, b0), (a1, b0), (a1, b1) y (a1, b1): la configuración (a0, b1) no se pregunta nunca, y su casilla de la tabla recibe lo que vale (a1, b1).

Lo que ve el usuario, con un nodo numérico N hijo de A (0,5 / 0,5) y de B (0,7 / 0,3) cuya relación «Exact» da 1, 2, 3 o 4 según (a0, b0), (a1, b0), (a0, b1) y (a1, b1), y una utilidad que vale 0 o 10 según B:

```
sin ver nada   N con estados 1|2|4, tabla de certezas 1 0 0 | 0 1 0 | 0 0 1 | 0 0 1
N visto en 3   «These findings are impossible in this network: together they have
↪  probability zero.»
N visto en 4   P(A) = 0,5 / 0,5 y P(B) = 0 / 1        (debería ser A = a1 seguro)
```

El valor 3, que el modelo da con probabilidad 0,15, no existe para el programa; verlo se rechaza como evidencia imposible; y ver 4 no dice nada de A. Medido el 15-10-2026 con las clases de `0fa7195` delante del jar del 14 de octubre, con `VEPropagation` y el hallazgo antes y después de resolver (dan lo mismo). La rama sin hallazgo es anterior a `0fa7195`: con el jar del 14 de octubre solo, los estados son también 1|2|4. La de los hallazgos llega con `0fa7195`, que sacó el recorrido a un método aparte para usarlo en las dos ramas (líneas 463-465 y 489-498).

Por qué pasa. En `forEachConfigurationOfParents` (líneas 572-626), cuando un padre ha pasado por todos sus estados y le toca volver al primero, el contador vuelve a cero, pero la configuración no se toca (líneas 620-623): sigue diciendo que ese padre está en su último estado. Igual con un padre numérico ya convertido, que se escribe en la configuración como un hallazgo numérico.

Con qué frecuencia se llega: hace falta que, al cambiar un padre que no es el primero, el valor del nodo dependa todavía de los padres anteriores. En las redes de `0_miscellaneous/networks` no se ha encontrado ningún caso. Medido con el mismo recorrido corregido en una copia de la clase (abajo): las tablas de los nodos numéricos de azar de `MID-CHAP-Ryan-Griffin.pgmx` y `MID-mammography.pgmx` desplegadas a uno y a tres ciclos, y las probabilidades de sus transiciones y diagnósticos con varios hallazgos, salen iguales con y sin el arreglo. En esos nodos, el padre que se pregunta mal no cambia el valor cuando el último ya ha dejado su primer estado. Se llega construyendo la red en el editor, como en el error 45.

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:609-623

```
                int nextStateIndex = ++parentIndices[parentIndex];
                if (nextStateIndex < parent.getVariable().getNumStates()) {
                    nextConfiguration = true;
                    if (originalVariables.containsKey(parentVariable)) {
                        ...
                        configuration.changeFinding(new Finding(originalVariable,
↪  numericalValue));
                    } else if (parentVariable.getVariableType() ==
                    ↪  VariableType.FINITE_STATES) {
                        configuration.changeFinding(new Finding(findingVariable,
↪  nextStateIndex));
                    }
                } else {
                    parentIndices[parentIndex] = 0;
                    parentIndex++;
                }
```

**Cómo arreglarlo.** En la rama `else`, antes de pasar al padre siguiente, devolver también la configuración al primer estado de ese padre, igual que se hace al empezar el recorrido (líneas 579-594): `changeFinding(new Finding(findingVariable, 0))` para un padre de estados, y el valor de su primer estado para uno numérico ya convertido. Medido con ese cambio en una copia de la clase: la red de arriba da los estados 1|2|3|4; ver 3 deja A = a0 seguro y ver 4, A = a1 seguro; y nada más cambia en las demás preguntas del generador.

Prueba: en `AFindingOnANumericNodeTellsAboutItsParentsTest`, la red de arriba: sin hallazgo, N tiene cuatro estados, 1, 2, 3 y 4, y la casilla de (a0, b1) es la del 3; con N visto en 3, P(A = a0) = 1; con N visto en 4, P(A = a1) = 1.

### Error 268 · [Corregido] Al observar un nodo numérico de azar cuya distribución tiene un parámetro imposible en alguna configuración de sus padres (una desviación 0, una forma 0, un intervalo al revés…), o una beta de parámetro menor que 1 vista en 0 o en 1, la propagación y la evaluación mueren con una excepción de Commons Math y la ventana enseña un error no previsto

| | |
|---|---|
| **Estado** | Corregido en el commit `0841101` (15-10-2026): si Commons Math se niega a construir la distribución con los parámetros de una configuración, el hallazgo se rechaza con un aviso que dice dónde y cuáles (`NonProjectablePotentialException.ParametersAreNotADistribution`), por ejemplo «with Disease = absent, the parameters of its Normal distribution, mu = 5 and sigma = 0, do not make a distribution»; la beta de parámetro menor que 1 vista en 0 o en 1 da densidad infinita y acaba como el error 270. No se ha añadido la comprobación al aceptar el panel de la relación |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.modelUncertainty; org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/NormalFunction.java:81-83`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/GammaAbstract.java:28-30`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/LogNormalFunction.java:65-67`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/BetaFunction.java:87-89`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/RangeFunction.java:64-66`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/TriangularFunction.java:80-82`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:635-658`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:379-382` |
| **Comprobado** | Leyendo el código y ejecutándolo; lo que se ve en la ventana está leído en el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 45, error 269, error 270 |

**Qué pasa.** Desde `0fa7195` (error 45), un hallazgo sobre un nodo numérico de azar cuya relación es una distribución («UnivariateDistr»: normal, gamma, beta, lognormal, triangular o «Range») pesa cada configuración de los padres con la densidad de la distribución en el valor observado. Para calcularla, `ProbDensFunction.getDensity` construye en cada llamada la distribución de Apache Commons Math (la biblioteca de matemáticas del programa, `commons-math3` 3.6.1) con los parámetros de esa configuración: `new NormalDistribution(null, mu, sigma)`, `new GammaDistribution(null, k, theta)`… Esos constructores se niegan cuando los parámetros no hacen una distribución, y la densidad de la beta se niega en 0 y en 1 cuando allí es infinita.

Medido el 15-10-2026 con las clases de `0fa7195` delante del jar del 14 de octubre, sobre redes hechas por programa como la de la prueba de ese commit (Disease → N, Disease → U), con N observado, propagando con `VEPropagation` y evaluando con `VEEvaluation`; los parámetros, primero los de «absent» y después los de «present»:

```
normal 5 0 / 8 1, N visto en 8          NotStrictlyPositiveException
```

| | |
|---|---|
| `gamma 0 3 / 3 2, visto en 3` | `NotStrictlyPositiveException` |
| `range 6 2 / 4 12, visto en 5` | `NumberIsTooLargeException` |
| `triangular 0 4 5 / 2 10 6, visto en 3` | `NumberIsTooLargeException` |
| `beta 0,5 0,5 / 2 2, visto en 0 o en 1` | `NumberIsTooSmallException` |

Basta un parámetro imposible en una sola configuración de los padres. Son excepciones de Commons Math, de tiempo de ejecución, sin texto en los ficheros de idioma del programa; no las recoge la conversión, ni la propagación, ni `EvidenceManager.doPropagation`, que solo recoge excepciones de OpenMarkov, así que el modo inferencia acaba en la ventana de error no previsto (leído en el código; no se ha visto en la ventana). Lo mismo les pasa a la evaluación y a las demás tareas exactas, que discretizan igual. Con el jar del 14 de octubre, sin `0fa7195`, estos cinco casos contestaban: el hallazgo no decía nada a los padres, que se quedaban en 0,9 / 0,1.

Ninguna red de `0_miscellaneous/networks` tiene un nodo numérico de azar con relación «UnivariateDistr»: se llega construyendo uno en el editor, como en el error 45, y nada impide escribir esos parámetros. Cada distribución sabe comprobar los suyos (`NormalFunction.verifyParameters` exige sigma $> 0$, y está `verifyParametersDomain`), y `UnivariateDistrPotential.checkDistributionValues` (líneas 379-382) los comprobaría, pero ningún código del programa llama a ese método. Y un parámetro calculado con una fórmula sobre un padre numérico puede salir imposible solo en algunas configuraciones.

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:643-652

```
        if (potential instanceof UnivariateDistrPotential distribution) {
            ProbDensFunction function = distribution.probDensFunction(configuration);
            if (!(function instanceof ExactFunction)) {
                double density = function.getDensity(observedValue);
                if (Double.isNaN(density)) {
                    throw new NonProjectablePotentialException.DensityCannotBeCalculated(
                            variable, distribution.getProbDensFunctionName());
                }
                return density;
            }
```

**Cómo arreglarlo.** En `likelihoodOfObservedValue`, antes de pedir la densidad, comprobar los parámetros de la configuración con la comprobación de la propia distribución (`verifyParameters`), y convertir lo que no valga —y cualquier excepción de Commons Math que aún salga de `getDensity`— en una `NonProjectablePotentialException` con mensaje, como `DensityCannotBeCalculated`, que diga qué nodo, qué parámetro y por qué. Así el usuario recibe un aviso como el que ya recibe con Dirichlet o complemento, y no un error no previsto. Aparte, conviene que el panel de la relación compruebe los parámetros al aceptarla, con `checkDistributionValues`, para que la mayoría de estos casos no lleguen a la resolución. La densidad infinita de la beta en 0 y en 1 es el mismo caso que el del error 270, y debe acabar igual que él.

Prueba: en `AFindingOnANumericNodeTellsAboutItsParentsTest`, la red Disease → N con normal (5, 0) / (8, 1) y N visto en 8: `VEPropagation` lanza una `NonProjectablePotentialException` cuyo mensaje nombra N y sigma, y no una excepción de Commons Math; lo mismo con la beta (0,5, 0,5) / (2, 2) vista en 0.

### Error 269 · [Corregido] Un hallazgo sobre un nodo numérico de azar cuya relación es «UnivariateDistr» hace morir la propagación y la evaluación con ArrayIndexOutOfBoundsException en cuanto la red tiene otro hallazgo numérico o el nodo tiene un padre numérico

| | |
|---|---|
| **Estado** | Corregido en el commit `781a45a` (15-10-2026): `EvidenceCase.getFindingsMap` escribe el número observado para una variable numérica, como lo lee una fórmula, y el nombre del estado para las demás; `UnivariateDistrPotential.probDensFunction` solo hace el mapa si algún parámetro es una fórmula. Las dos redes de la ficha y la tabla de fórmulas con A = 40 dan lo esperado. La tabla de fórmulas calcula sus casillas en precisión simple (`AugmentedProbTableInference`, `Float.valueOf`): con A = 40 da 0,6000000238 en vez de 0,6 |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:429-446`<br>`core/src/main/java/org/openmarkov/core/model/network/EvidenceCase.java:217-221`<br>`core/src/main/java/org/openmarkov/core/model/network/Finding.java:82-91,129-131`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:451-465,585-588,612-616`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/AugmentedProbTablePotential.java:124-125` |
| **Comprobado** | Leyendo el código y ejecutándolo; lo que se ve en la ventana está leído en el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 45, error 98, error 259, error 268 |

**Qué pasa.** Desde `0fa7195` (error 45), un hallazgo sobre un nodo numérico de azar cuya relación es «UnivariateDistr» —una distribución (normal, gamma…) o «Exact», un valor por configuración de los padres— hace morir la propagación del modo inferencia, la evaluación y las demás tareas exactas con `ArrayIndexOutOfBoundsException` en dos casos: si la evidencia tiene otro hallazgo sobre una variable numérica, o si el nodo tiene un padre numérico. La excepción no es de OpenMarkov y nadie la recoge: en la ventana sale el error no previsto (leído en el código, como en el error 268).

El caso más claro: una red con un nodo numérico N2 de relación uniforme. Sin hallazgo en N2, el programa se niega a resolver con un mensaje que pide justamente uno: «The numeric variable N2 has a uniform distribution, so it has no value to compute with. Set a finding on it in edition mode and try again.» Si el usuario lo pone, y la red tiene además observado un nodo numérico con relación «UnivariateDistr», la resolución muere. Por lo mismo (deducido leyendo, no medido), en los modelos de Markov cuya edad inicial es uniforme y necesita un hallazgo (`MID-Colorectal`, `MID-CHD-Walker`, `MID-mammography`) morirá la resolución en cuanto se observe además un nodo numérico con esa relación. Ninguna red de `0_miscellaneous/networks` tiene hoy un nodo numérico de azar con relación «UnivariateDistr», así que se llega construyéndolo en el editor.

Medido el 15-10-2026 con las clases de `0fa7195` delante del jar del 14 de octubre, sobre redes hechas por programa (Disease, ausente 0,9 / presente 0,1, padre de N):

```
N «Exact» 3 / 7 visto en 7, solo                                P(Disease) = 0 / 1
N «Exact» 3 / 7 visto en 7, y N2 uniforme visto en 5
↪  ArrayIndexOutOfBoundsException
```

```
N normal 5 1 / 8 1 visto en 8, y N2 normal 5 1 / 3 1 visto en 5
↪  ArrayIndexOutOfBoundsException
N normal de media {M}, con M numérico, 3 o 7 según Disease
↪  ArrayIndexOutOfBoundsException

java.lang.ArrayIndexOutOfBoundsException: Index 2147483647 out of bounds for length 1
    at org.openmarkov.core.model.network.Variable.getStateName(Variable.java:471)
    at org.openmarkov.core.model.network.Finding.getState(Finding.java:130)
    at org.openmarkov.core.model.network.EvidenceCase.lambda$getFindingsMap$0(EvidenceCase
    ↪  .java:220)
```

Con el jar del 14 de octubre, sin `0fa7195`, los tres últimos contestaban 0,9 / 0,1 (el hallazgo no decía nada a los padres: el error 45); desde el arreglo, mueren.

Depende también del formato del fichero: la misma red con la relación «Exact» hecha como `Exact DistrPotential` contesta bien (P(Disease) = 0 / 1 con N = 7 y N2 = 5), porque esa relación no pasa por aquí; guardada con `PGMXWriter_1_0`, que escribe esa relación como `UnivariateDistr` de distribución «Exact», y vuelta a leer, muere.

Por qué pasa. `UnivariateDistrPotential.probDensFunction` (líneas 429-446), nuevo en `0fa7195`, calcula los parámetros de la distribución en una configuración, y para las fórmulas pide los valores de lo que se ha visto con `configuration.getFindingsMap()` (línea 437). Ese mapa se hace siempre, con todos los hallazgos, aunque ningún parámetro sea una fórmula. `EvidenceCase.getFindingsMap` (líneas 217-221) escribe cada hallazgo como el nombre de su estado (`Finding.getState`, líneas 129-131), y un hallazgo sobre una variable numérica no tiene estado: `new Finding(variable, double)` le pone `Integer.MAX_VALUE` como índice (líneas 82-91). La configuración que recibe lleva toda la evidencia (`ProbNetOperations`, línea 451) y, por cada padre numérico ya convertido, un hallazgo numérico (líneas 585-588 y 612-616). Aunque no muriera, el mapa no serviría para las fórmulas: daría el nombre de un estado donde la fórmula necesita un número.

El mismo `getFindingsMap` está en `AugmentedProbTablePotential.tableProject` (línea 125), la tabla de probabilidad cuyas casillas son fórmulas sobre padres numéricos. Medido llamándolo directamente: una tabla sobre S y su padre numérico A, con las casillas `1-{A}/100` y `{A}/100`, proyectada con A = 40, lanza la misma `Index 2147483647 out of bounds for length 1`. No se ha buscado por dónde llega a eso un usuario.

Es de la familia del error 98: allí es una tabla la que usa como índice el `Integer.MAX_VALUE` de un hallazgo sin estado.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:437-444

```
        Map<Variable, String> values = configuration.getFindingsMap();
        double[] parameters = new double[pseudoVariableDistribution.getNumStates()];
        for (int i = 0; i < parameters.length; i++) {
            VariableExpression expression = distributionTable.getFunctionValues()[column +
↪  i];
            parameters[i] = expression == null || expression.references().isEmpty()
                    ? distributionTable.getValues()[column + i]
                    : Double.parseDouble(expression.evaluateWith(values));
        }
```

**Cómo arreglarlo.** Que el mapa de valores escriba cada hallazgo como lo necesita una fórmula: el número, para una variable numérica (`finding.getNumericalValue()`, escrito como lo lee la expresión), y el nombre del estado para las demás. Puede hacerse en `getFindingsMap` o, si otros llamadores necesitan el nombre del estado, en un método aparte que usen `probDensFunction` y `AugmentedProbTablePotential.tableProject`. Con eso deja de morir, y además las fórmulas

sobre un padre numérico reciben su número. De paso, `probDensFunction` puede no construir el mapa cuando ningún parámetro de la columna es una fórmula.

Prueba: en `AFindingOnANumericNodeTellsAboutItsParentsTest`, la red Disease → N con «Exact» 3 / 7 y un N2 uniforme sin padres, vistos N = 7 y N2 = 5: P(Disease) = 0 / 1. Y la normal de media {M}, con M numérico que vale 3 o 7 según Disease, vista en 7: P(Disease = present) es la que dan las dos densidades.

### Error 270 · [Corregido] Observar un nodo numérico de azar en un valor donde la densidad de su distribución es infinita (una gamma de forma menor que 1 vista en 0) deja la probabilidad de sus padres en NaN, sin ningún aviso

| | |
|---|---|
| **Estado** | Corregido en el commit `d9016fc` (15-10-2026), con la opción A elegida por Manuel Arias: una densidad infinita en el valor observado rechaza el hallazgo con el aviso «The finding on N cannot be used: at 0, the density of its Gamma distribution is infinite.» (`NonProjectablePotentialException.DensityIsInfinite`), antes y después de resolver; donde la densidad es finita, el hallazgo se usa como antes |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:635-658` `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/GammaAbstract.java:28-30` `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/InferencePresenter.java:108-113` `gui/src/main/java/org/openmarkov/gui/graphic/VisualState.java:340,346-352` |
| **Comprobado** | Leyendo el código y ejecutándolo; lo que enseña la ventana está leído en el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 45, error 268 |

**Qué pasa.** Desde `0fa7195` (error 45), un hallazgo sobre un nodo numérico de azar con una distribución pesa cada configuración de los padres con la densidad en el valor observado. Hay distribuciones cuya densidad es infinita en un punto: la gamma de forma menor que 1 en 0, por ejemplo, para la que Commons Math contesta `Infinity`. Si el nodo se observa justo ahí, la configuración con esa densidad pesa infinito, y al normalizar sale infinito entre infinito: la probabilidad del padre es NaN, sin ningún aviso.

Medido el 15-10-2026 con las clases de `0fa7195` delante del jar del 14 de octubre, sobre la red de la prueba de ese commit (Disease, ausente 0,9 / presente 0,1, padre de N, numérico de azar, y de U, que vale 0 si ausente y 10 si presente), con N gamma (0,5; 1) si ausente y (2; 1) si presente, visto en 0, antes o después de resolver:

```
P(Disease) = [NaN, 0.0]        utilidad esperada 0.0
```

Ninguna red de `0_miscellaneous/networks` tiene un nodo numérico de azar con una distribución: se llega construyéndolo en el editor, como en el error 45. El límite tiene respuesta: la densidad de «ausente» en 0 es infinita y la de «presente» es 0, así que el hallazgo deja «ausente» seguro. La utilidad esperada coincide con ese límite; la probabilidad, no.

Lo que vería el usuario, leído en el código y no visto en la ventana: `InferencePresenter` (líneas

108-113) solo trata como inexistente una variable cuyos valores son todos cero, y aquí no lo son, así que pasa el NaN a la caja; `VisualState.paint` pinta un NaN sin barra (línea 340) y con un guion en lugar del número (líneas 346-352), que es como el modo inferencia enseña una variable que no existe. La fila de «absent» sale con un guion y la de «present» con 0.0000: ninguna de las dos dice que el hallazgo hace seguro «absent».

Por qué pasa. `ProbNetOperations.likelihoodOfObservedValue` (líneas 635-658) solo rechaza una densidad NaN, con `DensityCannotBeCalculated` (líneas 646-651); una infinita pasa tal cual a la tabla del nodo observado.

**El código.** core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:646-651

```
double density = function.getDensity(observedValue);
if (Double.isNaN(density)) {
    throw new NonProjectablePotentialException.DensityCannotBeCalculated(
            variable, distribution.getProbDensFunctionName());
}
return density;
```

**Cómo arreglarlo.** Lo más simple, y de acuerdo con lo que se decidió para el error 45 (usar la densidad y avisar donde no se pueda calcular): tratar una densidad infinita como una que no se puede calcular, `!Double.isFinite(density)` en lugar de `Double.isNaN(density)`, y rechazar el hallazgo con el aviso de siempre. La otra vía es dar el límite: si alguna configuración tiene densidad infinita, el hallazgo deja seguras esas configuraciones y descarta las demás. Pero si son varias, cuánto pesa cada una depende de lo deprisa que crece su densidad cerca del punto, que no se obtiene de `getDensity`. La beta de parámetro menor que 1 vista en 0 o en 1 es el mismo caso, aunque hoy acaba en una excepción de Commons Math (error 268); conviene que las dos terminen igual.

Prueba: en `AFindingOnANumericNodeTellsAboutItsParentsTest`, la red de arriba con N visto en 0: `VEPropagation` no devuelve un NaN; rechaza el hallazgo con `DensityCannotBeCalculated` (o, si se elige el límite, P(Disease = absent) = 1).

## Error 272 · Observar un nodo numérico cuya beta tiene un parámetro que no es positivo se rechaza diciendo que los parámetros no hacen una distribución si se ve en el extremo, pero diciendo que la beta no sabe decir cómo de probable es el valor si se ve en cualquier otro punto

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.modelUncertainty; org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/BetaFunction.java:87-93`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:639-670` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 268, error 270, error 45 |

**Qué pasa.** Desde `0841101` (error 268), un hallazgo sobre un nodo numérico de azar cuya distribución tiene parámetros imposibles se rechaza con un mensaje que dice dónde y cuáles. Con

la beta no siempre: si un parámetro es 0 (o negativo), el mensaje depende de dónde se vea el número. En el extremo se rechaza bien; en cualquier otro punto, el mensaje dice que la beta es una distribución que no sabe decir cómo de probable es un valor, como se dice de una Dirichlet, cosa que no es verdad: lo que está mal es el parámetro, y el mensaje no lo nombra.

Medido el 15-10-2026 con las clases de `0841101` delante del jar del 14 de octubre, sobre la red de la prueba de ese commit (Disease, padre de N), con N beta (0, 2) si ausente y (5, 1) si presente:

```
visto en 0     ParametersAreNotADistribution: «The finding on N cannot be used: with
↪  Disease = absent, the
               parameters of its Beta distribution, alpha = 0 and beta = 2, do not make a
               ↪  distribution.»
visto en 0,5   DensityCannotBeCalculated: «The finding on N cannot be used: its relation is
↪  a Beta
               distribution, which cannot say how likely the observed value is.»
```

Por qué pasa: la `BetaDistribution` de Commons Math no comprueba sus parámetros al construirse; solo mira alpha cuando se le pide la densidad en 0, y beta en 1, y entonces se niega, lo que `likelihoodOfObservedValue` (líneas 639-670) convierte en «parámetros imposibles». `BetaFunction.getDensity` (líneas 87-93) contesta infinito en un extremo solo si los dos parámetros son positivos; con uno que no lo es, y fuera de los extremos, Commons Math devuelve «no es un número» —el logaritmo de su función gamma lo da para todo número que no es positivo—, y `likelihoodOfObservedValue` lo toma por una densidad que no se puede calcular. Por lectura, igual con un parámetro negativo. No se ha mirado por qué caminos llega una beta así desde la ventana (un parámetro escrito con una fórmula, un fichero).

**El código.** core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/BetaFunction.java:87-93

```java
    @Override public double getDensity(double x) {
        ...
        if (alpha > 0 && beta > 0 && ((x == 0 && alpha < 1) || (x == 1 && beta < 1))) {
            return Double.POSITIVE_INFINITY;
        }
```

**Cómo arreglarlo.** Que `BetaFunction.getDensity` se niegue antes de llamar a Commons Math cuando alpha o beta no son positivos, con una `NotStrictlyPositiveException` (que `likelihoodOfObservedValue` ya convierte en «parámetros imposibles») o directamente con `ParametersAreNotADistribution`. La misma regla conviene en las demás distribuciones que Commons Math no comprueba al construirse.

Prueba: en `AFindingOnANumericNodeTellsAboutItsParentsTest`, la red de arriba vista en 0,5: `VEPropagation` lanza `ParametersAreNotADistribution` y no `DensityCannotBeCalculated`.

# 7 Decisiones: políticas, criterios, agentes y árbol de decisión

Todo lo que tiene que ver con los nodos de decisión: la utilidad esperada y la política óptima que se enseñan desde el menú, las políticas impuestas, los criterios de decisión, los agentes, las redes de análisis de decisiones y el árbol de decisión.

### Error 50 · [Corregido] El árbol de decisión no se puede construir en casi ninguna red guardada en formato 1.0 (Mediastinet, decide-test…): sale un error inesperado

| | |
|---|---|
| **Estado** | Corregido. Lo propio del formato 1.0 lo resolvió el commit `551e715` (23-09-2026), que convierte en tabla las distribuciones «Exact»: el 30-09-2026, de las 100 redes de decisión de `0_miscellaneous/networks` (formatos 0.2 y 1.0), 82 daban árbol y las demás fallaban igual en los dos formatos. El resto lo corrige el commit `44312ac` (30-09-2026): `DANOperations.projectPotentials` ya no se traga `NonProjectablePotentialException`, así que un potencial que no se puede proyectar se dice con el mismo mensaje que la estrategia óptima («cannot be converted to a table») en vez de con un error no previsto más adelante. Los árboles de las 83 redes que dan árbol salen idénticos, en estructura y en valores, antes y después de ese commit. Las cuatro versiones de `delayed-result-of-test` y las dos de `DAN-king` tenían utilidades uniformes, que no dan ningún número; el commit `19124a0` las pone a cero y las seis se evalúan y dan árbol. Siguen sin árbol `DAN-arthronet` (error 63) y, por tamaño, las redes de prueba con seis a ocho decisiones, que no terminan en 60 segundos. |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.decompositionIntoSymmetricDANs.core |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:60-171`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:396-446`<br>`core/src/main/java/org/openmarkov/core/model/network/UtilityOperations.java:28-50`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:193-205` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 3, error 9, error 46, error 112, error 166, nota 180 |

**Qué pasa.** Con una red abierta de tipo diagrama de influencia o red de análisis de decisiones, el botón «Decision tree» de la barra de herramientas llama a `InferenceHandler.showDecisionTree`, que construye un `DecisionTreeEditor`; este pide el árbol a `DecisionTreeManagerImpl.buildDecisionTree(probNet, 5)`. Con `0_miscellaneous/networks/id/1-0/ID-mediastinet-1-0.pgmx` la construcción muere con

```
java.lang.NullPointerException: Cannot invoke "Variable.getVariableType()" because
↪  "variable" is null
  at TableWithEvents.setEventAsStates(TableWithEvents.java:154)
  at TableWithEvents.deepCopy(TableWithEvents.java:447)
  at UnivariateDistrPotential.deepCopy(UnivariateDistrPotential.java:402)
  at UtilityOperations.transformToUnicriterion(UtilityOperations.java:41)
  at TaskUtilities.scaleUtilitiesUnicriterion(TaskUtilities.java:79)
  at VariableElimination.unicriterionPreprocess(VariableElimination.java:107)
  at VEEvaluation.resolve(VEEvaluation.java:68)
  at DANConditionalSymmetricInference.<init>(DANConditionalSymmetricInference.java:54)
  ... DANDecisionTreeInference / IDDecisionTreeInference (recursión por ramas)
```

Es una excepción de tiempo de ejecución, así que el manejador global (`OMExceptionHandler`) la enseña en la ventana de «error inesperado» y la pestaña del árbol no se abre. La versión antigua de la misma red, `0_miscellaneous/networks/id/ID-mediastinet.pgmx` (con tablas en vez de distribuciones), sí da árbol.

No es un caso raro de Mediastinet. Medido construyendo el árbol de todas las redes de `0_miscellaneous/networks/id/1-0/` y `0_miscellaneous/networks/dan/1-0/` (formato ProbModelXML 1.0, en el que las utilidades y muchas probabilidades se guardan como potenciales `UnivariateDistr`, distribución «exacta» sobre un número):

- 43 de las 50 redes dan este mismo `NullPointerException`, entre ellas `ID-decide-test-1-0`, `ID-arthronet-1-0`, `ID-CEA-minimal-1-0`, `DAN-decide-test-1-0` y las cuatro de Mediastinet;
- tres fallan con «Potential … cannot be converted to a table» (`ID-Monty-Hall-spanish-1-0`, `DAN-dating-1-0`, `DAN-used-car-buyer-1-0`);
- una falla con otro puntero nulo en `DANOperations.instantiate` (`DAN-arthronet-1-0`, que también falla en su versión 0.2; es error 63);
- dos no terminan en 60 segundos.

Las mismas redes en formato 0.2 (`0_miscellaneous/networks/id/*.pgmx`, `0_miscellaneous/networks/dan/*.pgmx`, con tablas) dan árbol salvo `DAN-arthronet` y `DAN-king`.

Por qué ocurre. El árbol se construye rama a rama: para cada valor de una decisión o de una variable observada, `DANOperations.instantiate` copia la red y proyecta sobre ese valor los potenciales que dependen de la variable (`projectPotentials`), y si la variable es una decisión la quita de la copia (línea 165). `projectPotentials` sabe proyectar `TablePotential` y `ExactDistrPotential`; para el resto intenta `tableProject` y, si el potencial no se puede convertir en tabla, **se traga la excepción** y deja el potencial como estaba, todavía con la variable que se va a quitar. Un `UnivariateDistrPotential` no se convierte en tabla (`PotentialCannotBeConvertedToATable`), así que queda colgando de una variable que ya no está en la red.

Medido llamando a `instantiate` sobre cada decisión de `ID-mediastinet-1-0`: tras instanciar `Decision_TBNA` los potenciales de `TBNA_Morbidity` y `Economic_Cost_TBNA` siguen nombrando `Decision_TBNA`; tras `Treatment`, los de `Survivors_QALE`, `Inmediate_Survival` y `Economic_Cost_Treatment` siguen nombrando `Treatment`, y lo mismo con `Decision_PET`, `Decision_MED` y `Decision_EBUS_EUS`.

Después, la evaluación de la rama escala las utilidades a un solo criterio (`UtilityOperations.transformToUnicriterion`), que hace `potential.deepCopy(probNet)` de cada utilidad sobre la red de la rama. `Potential.deepCopy` busca cada variable por nombre y mete `null` por la que falta (eso es error 3), y `TableWithEvents.deepCopy` (clase madre de `UnivariateDistrPotential`) recorre en la línea siguiente esa lista y muere.

La evaluación de cada rama (`DANConditionalSymmetricInference`) solo trata como rama de valor cero la evidencia imposible; cualquier otro error sube hasta el usuario, y por eso este fallo se ve como excepción en vez de como un árbol con valores falsos. Aunque el `null` de `deepCopy` se convirtiera en una excepción con mensaje (error 3), el árbol seguiría sin salir: lo que falta es que la rama no deje potenciales apuntando a variables quitadas.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:410-425

```
for (Potential potential : probNet.getPotentials(variable)) {
    // Project all potentials except the potential of the node
    if (nodePotential == null || (!nodePotential.equals(potential))) {
        Variable conditionedVariable = potential.getConditionedVariable();
        Node node = probNet.getNode(conditionedVariable);
        if (potential instanceof TablePotential || potential instanceof
        ↪  ExactDistrPotential) {
            // Maintain the type of potential.
            node.setPotential(potential.project(decisionEvidence));
        } else {
```

```
            ...
            TablePotential potentialProjectedFromTreeADD = null;
            try {
                potentialProjectedFromTreeADD = potential.tableProject(decisionEvidence,
↪  null);
            } catch (NonProjectablePotentialException ignored) {
            }
```

core/src/main/java/org/openmarkov/core/model/network/UtilityOperations.java:39-42

```
List<Potential> scaledPotentials = new ArrayList<>();
for (Potential potential : utilityPotentials) {
        Potential scaledPotential = potential.deepCopy(probNet);
```

**Cómo arreglarlo.** El sitio donde se decide es `DANOperations.projectPotentials`: un potencial que depende de la variable instanciada no puede quedarse sin proyectar. Opciones:

- (a) Proyectarlo por un camino que no exija convertirlo en tabla: los `UnivariateDistrPotential` y demás potenciales con eventos necesitarían su propio `project` sobre una evidencia de decisión.
- (b) Si no hay forma de proyectarlo, no tragarse `NonProjectablePotentialException`: dejarla subir para que `buildDecisionTree` falle con un mensaje comprensible («el modelo contiene un potencial de tipo X que el árbol de decisión no sabe tratar») en vez de con un puntero nulo lejos de allí.

La opción (a) es la que da árbol; la (b) solo cambia el mensaje.

Hay que revisar también `updatePotentialsWithNewVariable` y el borrado de nodos por restricciones de `instantiate` (líneas 100-160), que también quitan variables de la copia sin mirar si queda algún potencial que las nombre, y `pruneZeroProbabilityChanceNodes`.

Error 46 describe la misma incapacidad (un nodo de utilidad que es una distribución no se deja convertir en tabla) en el modo inferencia. Por qué las redes en formato 1.0 tienen distribuciones donde las 0.2 tienen tablas, y cómo guardar una red en ese formato la deja así, está en error 112; si allí se decide que una distribución exacta se pueda convertir en tabla, `projectPotentials` la proyectaría por su rama de `tableProject` (por lectura).

Prueba: construir con `DecisionTreeManagerImpl.buildDecisionTree(net, 5)` el árbol de `0_miscellaneous/networks/id/1-0/ID-mediastinet-1-0.pgmx`, `0_miscellaneous/networks/id/1-0/ID-decide-test-1-0.pgmx` y `0_miscellaneous/networks/dan/1-0/DAN-decide-test-1-0.pgmx` y comprobar que sale (o, si se elige la opción b, que falla con la excepción declarada y su mensaje, nunca con `NullPointerException`); y una prueba unitaria de `DANOperations.instantiate` que compruebe que ningún potencial de la red devuelta nombra una variable ausente de ella.

### Error 51 · [Corregido] «Show optimal policy» y «Show expected utility» de una decisión fallan con NullPointerException porque el panel del tipo de política nunca se construye

| | |
|---|---|
| **Estado** | Corregido en el commit `295a7a3` (15-10-2026), de Jorge Rico: `setEnabledDecisionOptions` ya no llama al panel del tipo de política, y un panel de solo lectura no copia la relación a la red del nodo, que era lo que hacía fallar «Show expected utility». Visto en la ventana el 16-10-2026 sobre `ID-decide-test.pgmx` en modo inferencia, con «Therapy» señalada: las dos ventanas se abren con su tabla y sin error. Queda sin hacer el selector del tipo de |

| | |
|---|---|
| | política que Manuel Arias decidió incorporar el 14-10-2026; hoy su falta no tiene consecuencias visibles. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:478-486,617-632,299-300`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditDialog.java:95-102`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:567-607`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/NodeContextualMenu.java:191-203`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:277-279`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/PolicyTypePanel.java`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:95` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si el tipo de política (óptima, determinista, probabilista) debe poder elegirse en la interfaz o si se retira `PolicyTypePanel` |
| **Relacionados** | error 52, error 55, error 56, nota 32, error 210 |

**Qué pasa.** En modo inferencia, con la evidencia compilada, el menú contextual de un nodo de decisión ofrece «Show expected utility» y «Show optimal policy» (`NodeContextualMenu`, líneas 191-203). Pedir cualquiera de las dos saca el diálogo de «error inesperado» con un `NullPointerException`, y la ventana de la política o de la utilidad esperada no llega a mostrarse.

El camino desde el menú, seguido en el código: las acciones (`MainPanelListenerAssistant`, líneas 284-286) llaman a `NetworkEditorPanel.showExpectedUtilityOfNode` y `showOptimalPolicyOfNode` (líneas 568-609). Las dos construyen un nodo auxiliar de tipo decisión (en la segunda, con `setPolicyType(PolicyType.OPTIMAL)`; en la primera, con el tipo de política por omisión de `Node`, que también es `OPTIMAL`, línea 95 de `Node`) y abren un `PotentialEditDialog` de solo lectura con `requestValues()`.

`PotentialEditDialog.requestValues` (líneas 94-100), cuando el nodo es de decisión, de política óptima y de solo lectura, llama a `PotentialEditPanel.setEnabledDecisionOptions` antes de hacer visible la ventana. Ese método (líneas 609-621) termina con `this.getPoliticyTypePanel().setEnabledDecisionOptions(true)`. Pero `getPoliticyTypePanel` (líneas 466-474) tiene el cuerpo que construiría el panel dentro de un comentario, bajo un `//TODO`, y el campo `pnlPolicyType` no se asigna en ningún otro sitio (tampoco se añade al panel: líneas 287-288 comentadas). Devuelve siempre `null`.

Medido con un pequeño programa de prueba: sobre un `PotentialEditPanel` con un nodo de decisión de política óptima (instancia creada sin ejecutar el constructor, porque el constructor crea un editor de comentarios que es un `JDialog` y no se puede construir sin pantalla), `setEnabledDecisionOptions` lanza `NullPointerException: Cannot invoke "PolicyTypePanel.setEnabledDecisionOptions(boolean)" because the return value of "PotentialEditPanel.getPoliticyTypePanel()" is null`, en la línea 620. En la aplicación, la excepción no está envuelta y llega al manejador global.

No se ha ejecutado el camino completo desde el menú (la evaluación previa y la construcción de la ventana no se pueden probar sin pantalla), así que en alguna red la petición puede fallar antes por otro motivo: por ejemplo, error 52 salta durante el cálculo de la utilidad esperada, antes de abrir la ventana.

Añadido el 09-10-2026. El 08-10-2026 se pidieron las dos entradas en la ventana con un guion automático, sobre `0_miscellaneous/networks/id/1-0/ID-mediastinet-1-0.pgmx` en modo inferencia y con la decisión `Decision_TBNA` señalada, y las dos acabaron en la ventana de error inesperado. «Show optimal policy» falla como se describe arriba, con ese mismo `NullPointerException`. «Show expected utility» falla antes y por otra causa: `showExpectedUtilityOfNode` pone la utilidad calculada en un nodo auxiliar de una red recién creada y vacía (`NetworkEditorPanel`, líneas 574-575), y el constructor de `PotentialEditPanel` empieza copiando ese potencial a la red del nodo con `deepCopy` (línea 150). Desde el commit `38e9782` (error 3) esa copia se niega cuando a la red de destino le falta una variable, en vez de dejar nulos, y lanza `InvalidArgumentException: Argument copyNet (Decision_TBNA) is invalid because network null has no such variable`. Quitar la última línea de `setEnabledDecisionOptions` no basta, por tanto, para que se abra «Show expected utility»: hay que dar al nodo auxiliar una red con las variables de la utilidad, como hace `showOptimalPolicyOfNode` con la suya, o no copiar el potencial en un diálogo de solo lectura. No se ha comprobado volviendo al commit anterior a `38e9782`.

El tipo de política no se puede elegir en ninguna parte de la interfaz, pero hoy eso no tiene otra consecuencia: fuera de la interfaz, el tipo de política solo lo lee `RemovePolicyEdit` (ver error 56); ni la inferencia ni el formato `.pgmx` lo usan.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:466-474

```
private PolicyTypePanel getPoliticyTypePanel() {
    //TODO
    /*
    if (this.pnlPolicyType == null) {
        this.pnlPolicyType = new PolicyTypePanel(this, this.node);
    }
     */
    return this.pnlPolicyType;
}
```

gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:609-621

```
void setEnabledDecisionOptions() {
    switch (this.node.getPolicyType()) {
        case OPTIMAL, DETERMINISTIC -> this.getPotentialTypeJCombobox().setEnabled(false);
        case PROBABILISTIC -> {
            ...
        }
    }
    this.getPoliticyTypePanel().setEnabledDecisionOptions(true);
}
```

**Cómo arreglarlo.** Arreglo inmediato: que `setEnabledDecisionOptions` no dependa de un panel que no existe (quitar la última línea o comprobar `null`).

La decisión de fondo es si el selector del tipo de política se incorpora de verdad o se retiran `PolicyTypePanel`, `getPoliticyTypePanel` y el campo. El constructor comentado no compila tal cual: `PolicyTypePanel` espera un `PotentialEditDialog` y se le pasaba el panel. Si se incorpora, el cambio del tipo de política debería ser una edición deshacible y no una asignación directa (ver error 56), y `ImposePolicyPanel.initializePotential` dejaría de fijar `OPTIMAL` a mano.

Al corregirlo, estas dos ventanas empezarán a abrirse, y con ellas se verán error 53 (la utilidad esperada ignora la evidencia) y error 55 (el título enseña el nombre de la clave).

Prueba: una prueba de interfaz que, sobre `0_miscellaneous/networks/id/ID-decide-test.pgmx`

en modo inferencia, pida «Show optimal policy» y «Show expected utility» de «Therapy» y compruebe que se abre la ventana sin excepción. El panel no se puede construir sin pantalla, así que la prueba necesita una.

### Error 52 · [Corregido] «Show expected utility» sobre una decisión muere con un índice fuera de rango cuando la utilidad que resulta no tiene variables, y rellena con ceros cuando le faltan variables

| | |
|---|---|
| **Estado** | Corregido en el commit `d592f80` (08-10-2026): `VEExpectedUtilityDecision.resolve` añade a la utilidad, antes de reordenarla, las variables de las que no depende, repitiendo su valor para cada estado (`TablePotential.addVariable`). Fijando cada opción de la decisión como evidencia sale el mismo número en las redes medidas. Lo comprueba `TheExpectedUtilityOfADecisionHasAValueForEveryConfigurationTest`. No se han tocado `TablePotential.reorder` ni `getAccumulatedOffsets`: siguen suponiendo que el orden pedido es una permutación de las variables del potencial, y no se ha establecido que otro llamador pueda pedir otra cosa. La ventana sigue sin abrirse por el error 51. |
| **Módulo** | core, inference, gui |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.inference.algorithm.variableElimination.tasks |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEExpectedUtilityDecision.java:55-66`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/AbstractIndexedPotential.java:183-208`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:742-771`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/StrategicTablePotential.java:107-110`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:575-589` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Tapado por** | error 51: la tabla con ceros no se ve porque la ventana no llega a abrirse. |
| **Relacionados** | error 51, error 53, error 67 |

**Qué pasa.** Con una red con decisiones en modo inferencia y el caso de evidencia compilado, clic derecho sobre una decisión sin política impuesta → «Show expected utility» (`ActionCommands.DECISION_SHOW_EXPECTED_UTILITY` → `NetworkEditorPanel.showExpectedUtilityOfNode`). Ese método crea `VEExpectedUtilityDecision(red, decisión)` y pide `getExpectedUtility()`.

`VEExpectedUtilityDecision.resolve` evalúa la red con `VEEvaluation`, condicionando a los predecesores informativos de la decisión, y **reordena** la utilidad resultante para poner la decisión delante: `veEvaluation.getUtility().reorder(orderedVariables)`. La lista `orderedVariables` es la decisión seguida de sus predecesores informativos, y no tiene por qué coincidir con las variables de la utilidad que devuelve la evaluación. `TablePotential.reorder(List<Variable>)` supone que el orden nuevo es una permutación de las variables del potencial. Dos casos rompen esa suposición:

1. **La utilidad es una constante (cero variables).** Pasa en un modelo sin nodos de utilidad o donde la decisión no influye en ninguna utilidad. El reordenamiento llama a `getAccumulatedOffsets(this.variables, otherVariables)` con `this.variables` vacía: el método crea `ordering = new int[0]` y lee `ordering[0]` (línea 201), y lanza `ArrayIndexOutOfBoundsException`. Es una excepción de tiempo de ejecución que el manejador global

enseña en la ventana de «error inesperado». Salta durante el cálculo, antes de abrir la ventana de la utilidad esperada, así que en este caso no se llega a error 51.

2. **La utilidad tiene menos variables que el orden pedido.** No hay excepción: el recorrido copia los valores en las casillas del primer estado de las variables que faltan y deja el resto a cero, así que la ventana enseñaría una tabla con ceros que nadie ha calculado, como si fueran utilidades. Hoy esa ventana no llega a abrirse por error 51; esta tabla con ceros aparecerá en cuanto se corrija.

Medido ejecutando `VEExpectedUtilityDecision` sobre `integrationTests/src/main/resources/networks/id/ID-only-decision-no-utility.pgmx`, `integrationTests/src/main/resources/networks/id/ID-three-dec-two-util.pgmx`, `integrationTests/src/main/resources/networks/id/ID-only-imposed-decision-and-chance-no-utility.pgmx`, `integrationTests/src/main/resources/networks/dan/DAN-only-decision-no-utility.pgmx` y `DAN-dating.pgmx` (está en `integrationTests/src/main/resources/networks/dan/` y en `0_miscellaneous/networks/dan/`):

```
ID-only-decision-no-utility.pgmx / D   : ArrayIndexOutOfBoundsException: Index 0 out of
↪  bounds for length 0
ID-three-dec-two-util.pgmx / D         : ArrayIndexOutOfBoundsException
ID-three-dec-two-util.pgmx / D1        : [D1, D] [4.0, 5.0, 0.0, 0.0]
ID-three-dec-two-util.pgmx / D2        : [D2, D, D1] [4.0, 1.0, 0.0, 0.0, 5.0, 2.0, 0.0,
↪  0.0]
DAN-dating.pgmx / Ask?                 : ArrayIndexOutOfBoundsException
DAN-dating.pgmx / NClub?               : [NClub?, Ask?, Accept] [0.52, 16.82, 0.0, 0.0,
↪  0.0, 0.0, 0.0, 0.0]
DAN-only-decision-no-utility.pgmx / D  : ArrayIndexOutOfBoundsException
```

La pila de las excepciones:

```
DAN-dating.pgmx, Ask?  → ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
   at AbstractIndexedPotential.getAccumulatedOffsets(AbstractIndexedPotential.java:201)
   at AbstractIndexedPotential.getAccumulatedOffsets(AbstractIndexedPotential.java:308)
   at StrategicTablePotential.reorder(StrategicTablePotential.java:109)
ID-only-imposed-decision-and-chance-no-utility.pgmx, D → la misma excepción, vía
↪  TablePotential.reorder(TablePotential.java:744)
   y VEExpectedUtilityDecision.resolve(VEExpectedUtilityDecision.java:65)
```

Y directamente sobre `TablePotential`: una tabla sin variables reordenada a [D] lanza la misma excepción; una tabla sobre X con valores [1, 2] reordenada a [D, X] devuelve [1, 0, 2, 0].

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/-tasks/VEExpectedUtilityDecision.java:55-66

```
private void resolve() throws ... {
    List<Variable> informationalPredecesors =
    ↪  ProbNetOperations.getInformationalPredecessors(probNet, decision);
    List<Variable> orderedVariables = new ArrayList<>(informationalPredecesors);
    orderedVariables.remove(decision);
    orderedVariables.add(0, decision);

    VEEvaluation veEvaluation = new VEEvaluation(probNet);
    veEvaluation.setConditioningVariables(informationalPredecesors);

    result = veEvaluation.getUtility().reorder(orderedVariables);
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/AbstractIndexedPotential.java:183-203

```
public static int[] getAccumulatedOffsets(List<Variable> variables, List<Variable>
↪  otherVariables) {
    int otherSize = otherVariables.size();
    int thisSize  = variables.size();
    int[] accOffsetXY = new int[thisSize];
    if (otherSize == 0) {
        return accOffsetXY; // Initialized to 0
    }
    int[] ordering = new int[thisSize];
    ...
    int ordering_0 = ordering[0];
```

**Cómo arreglarlo.** Hay dos sitios, que se complementan.

En el cálculo de desplazamientos y el reordenamiento:

- `AbstractIndexedPotential.getAccumulatedOffsets` debe devolver el array vacío también cuando `variables` está vacía (hoy solo se protege `otherSize == 0`). Los `reorder` de `TablePotential` y `StrategicTablePotential` usan el mismo cálculo y quedan cubiertos por este cambio.
- `TablePotential.reorder` debe comprobar que el orden nuevo contiene exactamente las mismas variables que el potencial y lanzar una excepción con mensaje si no; y tratar el potencial sin variables (con orden vacío, devolver una copia).

En `VEExpectedUtilityDecision.resolve`, no pedir un reordenamiento que no es una permutación. Lo más directo es extender antes la utilidad a todas las variables de `orderedVariables`: una utilidad que no depende de una variable tiene el mismo valor para todos sus estados, así que se repiten valores y no se ponen ceros. También vale reordenar solo sobre las variables que la utilidad tiene y extender después. Qué debe enseñar la ventana cuando la utilidad no depende de la decisión no es del todo obvio: además de repetir el valor constante, cabe un aviso de que la utilidad esperada no depende de la decisión; en cualquier caso, de forma explícita.

Otros llamadores de `reorder` que pueden recibir potenciales constantes u órdenes que no son permutaciones, y deben revisarse con la misma regla: `VESensAnTornadoSpider` (línea 83), `VESensAnMap` (76), `VESensAnPlot` (74), `VEPropagation.InvokeVariableEliminationCore` (208), `MIDTemporalEvolution.computePosteriorProbability` (610, que además puede recibir una lista con un nulo, ver error 67) y `TablePotentialMaximization` (367).

De paso, `showExpectedUtilityOfNode` no pasa la evidencia a la tarea (error 53).

Pruebas de regresión: `new VEExpectedUtilityDecision(red, Ask?).getExpectedUtility()` sobre `DAN-dating.pgmx` no lanza `ArrayIndexOutOfBoundsException`; `VEExpectedUtilityDecision` sobre `ID-only-decision-no-utility.pgmx` no lanza y devuelve una tabla sobre D con ceros legítimos (no hay utilidad); sobre `ID-three-dec-two-util.pgmx` para D1, la tabla no tiene casillas a cero fabricadas; un `TablePotential` sin variables `.reorder(List.of(v))` no se sale del array; y una prueba unitaria de `reorder` con un orden que no es permutación que exija la excepción.

### Error 53 · [Corregido] La utilidad esperada de una decisión ignora la evidencia previa a la resolución

| | |
|---|---|
| **Estado** | Corregido en el commit `0bb37e3` (08-10-2026): `VEExpectedUtilityDecision.resolve` pasa la evidencia previa a la resolución a su evaluación, y `NetworkEditorPanel` se la da a la tarea en «Show expected utility» y a la evaluación en «Show optimal policy». Con Disease=present, «Do test?» da [7,25; 7,05]; lo comprueba un caso de `TheExpectedUtilityOfADecisionHasAValueForEvery` |

| | |
|---|---|
| | `ConfigurationTest`. La parte de la ventana no tiene prueba, y las dos ventanas siguen sin abrirse por el error 51. Los dos métodos vacíos de `VEOptimalIntervention` no se han tocado: nadie los llama. |
| **Módulo** | inference, gui |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.tasks; org.openmarkov.gui.window.edition.networkEditorPanel |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEExpectedUtilityDecision.java:55-66` `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEOptimalIntervention.java:120-126` `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:575-587` `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:593-597` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Tapado por** | error 51: la ventana no llega a abrirse. |
| **Relacionados** | error 52, error 54 |

**Qué pasa.** La utilidad esperada de cada opción de una decisión se calcula como si no se hubiera observado nada, aunque haya hallazgos previos a la resolución.

Medido con `0_miscellaneous/networks/id/ID-decide-test.pgmx`:

```
Do test? sin evidencia:                          [Do test?] [9.16, 9.3289]
Do test? con Disease=present en la tarea:        [Do test?] [9.16, 9.3289]
Do test? VEEvaluation con Disease=present:       [Do test?] [7.25, 7.05]
```

Con la evidencia, la mejor opción cambia (sin evidencia conviene hacer la prueba; con Disease=present, no hacerla), y la tarea no lo refleja.

Camino del usuario: en modo inferencia, el menú contextual de un nodo de decisión sin política impuesta ofrece «Show expected utility», que llama a `NetworkEditorPanel.showExpectedUtilityOfNode`. Ese método crea la tarea **sin** llamar a `setPreResolutionEvidence`, así que aunque la clase pasara la evidencia, la ventana seguiría enseñando la utilidad sin los hallazgos previos a la resolución que el usuario haya puesto en modo edición. Lo mismo ocurre con «Show optimal policy» (`showOptimalPolicyOfNode`, que crea `VEEvaluation` sin evidencia). En cambio, la propagación que pinta el modo inferencia sí usa esa evidencia para calcular las políticas (`VEPropagation.calculateOptimalPolicies`), de modo que las dos ventanas pueden contradecir lo que se ve en los nodos. Esto último se ha comprobado leyendo el código; no se ha ejecutado la ventana.

Hoy esas dos ventanas no llegan a abrirse por error 51; la contradicción descrita aparecerá en cuanto se corrija.

Por qué pasa: `VEExpectedUtilityDecision` (la tarea que calcula la utilidad esperada de cada opción de una decisión en función de lo que se sabe al tomarla) hereda de `VariableElimination`, así que tiene `setPreResolutionEvidence`. Pero su `resolve()` crea un `VEEvaluation` nuevo y nunca le pasa esa evidencia.

`VEOptimalIntervention` tiene `setPreResolutionEvidence` y `setConditioningVariables` con el cuerpo vacío, pero su constructor sí recibe la evidencia y se la pasa a su evaluación, y el único llamador de producción (`InferenceHandler`, línea 226, «Mostrar estrategia óptima») usa ese constructor. Nadie llama a los métodos vacíos, así que por ahí hoy no se llega a ningún fallo.

**El código.** gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:575-580

```
public void showExpectedUtilityOfNode() throws
↪ IncompatibleEvidenceException.EvidenceIsIncompatibleWithOther,
↪ NonProjectablePotentialException,
↪ NotEvaluableNetworkException.NotApplicableNetwork, ConstraintViolatedException {
    VisualNode visualNode = this.visualNetwork.getLastSelectedNode();
    Node node = visualNode.getNode();
    VEExpectedUtilityDecision veExpectedUtilityDecision
            = new VEExpectedUtilityDecision(this.visualNetwork.getProbNet(),
            ↪  node.getVariable());
    Potential expectedUtility = veExpectedUtilityDecision.getExpectedUtility();
```

inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEOptimalIntervention.java:120-126

```
@Override public void setPreResolutionEvidence(EvidenceCase preresolutionEvidence) {

}

@Override public void setConditioningVariables(List<Variable> conditioningVariables) {

}
```

**Cómo arreglarlo.** En `VEExpectedUtilityDecision.resolve()`, pasar `getPreResolutionEvidence()` a la `VEEvaluation` interna.

Para que el usuario lo vea, `NetworkEditorPanel.showExpectedUtilityOfNode` y `showOptimalPolicyOfNode` tienen que pasar la evidencia previa a la resolución del `EvidenceManager` (la misma que usa `doPropagation`).

En `VEOptimalIntervention`, los dos métodos vacíos deben delegar en `veEvaluation` (o lanzar `UnsupportedOperationException`), para que un llamador futuro no reciba silencio.

Prueba de regresión: `VEExpectedUtilityDecision` sobre `0_miscellaneous/networks/id/ID-decide-test.pgmx` para «Do test?» con Disease=present debe dar [7,25; 7,05].

## Error 54 · [Corregido] Un hallazgo previo posible que la primera opción de una decisión hace imposible se toma por evidencia imposible: la evaluación da utilidad 0 y el análisis de coste-efectividad devuelve la partición cero y revienta

| | |
|---|---|
| **Estado** | Corregido en el commit `1cebd6d` (08-10-2026), junto con el error 85 (`af673f0`): cuando la probabilidad depende de la decisión que se elimina, `DecisionVariableElimination` deja fuera de la comparación las opciones con probabilidad cero y se queda con la probabilidad de la opción elegida. Medido con `ID-CEA-test-2therapies-new-test.pgmx`: «New test = negative» da utilidad 295 308,34 y probabilidad 0,8138; «positive», 166 921,02 y 0,1862; «Test = positive», probabilidad 0,1754; y el análisis de coste-efectividad global da los mismos tramos que fijando además «Dec: New test = yes». En `ID-arthronet-ce.pgmx`, «Isquemia = Menor 1 hora y media» da 71 007,47, lo mismo que fijando «Realizar Implante = si». Si el hallazgo es posible con varias opciones con distinta probabilidad, con un solo criterio se elige la de mayor utilidad y se toma su probabilidad; en coste-efectividad se toma la mayor probabilidad, que es una aproximación, porque la opción óptima cambia de un tramo a otro. No se han tocado `DecisionTreeExpansion` ni `DANInference`, que toman |

| | |
|---|---|
| | la probabilidad de la primera rama: la ventana del árbol lo construye sin los hallazgos, y en las redes de análisis de decisiones probadas (`DAN-decide-test.pgmx`, `DAN-decide-test-with-symptom.pgmx`) no se ha obtenido un resultado equivocado. |
| **Módulo** | inference, core |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination; org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/DecisionVariableElimination.java:50-57`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialTransform.java:99-104`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/DiscretePotentialOperations.java:386-388`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/VariableEliminationCore.java:160-173`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEAlgebra.java:181-203` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 48, error 53, error 85, error 86, error 87 |

**Qué pasa.** Un hallazgo previo a la resolución (el que se pone en modo edición con «Add finding») sobre una variable de azar que depende de una decisión puede ser perfectamente posible y, aun así, la evaluación y el análisis de coste-efectividad lo tratan como si fuera imposible. Pasa cuando el valor observado es imposible con la **primera** opción de la decisión y posible con otra, y antes de esa decisión hay otras variables en el modelo (una decisión anterior, o algo que se conoce al tomarla). El caso típico es el resultado de una prueba: «negative» o «positive» solo pueden darse si se decidió hacerla, y «no» suele ser la primera opción.

Lo que ve el usuario:

- Menú Tools → «Cost-effectiveness analysis», ámbito global: la tarea devuelve la partición de probabilidad cero y la ventana `CEPDialog` revienta con `NullPointerException` (error 86).
- Ámbito de decisión, con una decisión anterior a la afectada: todas sus opciones salen con la partición cero y la ventana revienta igual (error 86).
- Evaluación de un solo criterio: la probabilidad de la evidencia sale 0 y la utilidad no se calcula (sale 0 en las dos redes medidas). «Show optimal strategy» del modo inferencia (`InferenceHandler.showOptimalStrategy`, que pasa la evidencia previa a `VEOptimalIntervention`) recibe una estrategia que sale de un empate entre ceros, o ninguna (`null`).
- La propagación que pinta el modo inferencia no se ve afectada.

Camino del usuario: abrir `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx` (decisiones «Dec: Test», «Dec: New test» y «Therapy», en ese orden); en modo edición, clic derecho sobre «New test» → «Add finding» → «negative»; menú Tools → «Cost-effectiveness analysis», aceptar las opciones y elegir el ámbito global.

Por qué pasa. La eliminación de variables resuelve el modelo desde la última decisión hacia atrás. Al eliminar una decisión, `DecisionVariableElimination` maximiza la utilidad y quita la decisión de la probabilidad con `DiscretePotentialOperations.projectOutVariable`, que la fija en su primera opción: `TablePotentialTransform.projectOutVariable` pone un hallazgo en el estado 0 y proyecta. Eso supone que la probabilidad no depende de la decisión, lo que es cierto sin evidencia (una decisión no cambia la probabilidad de lo que pasó antes), pero deja de serlo en cuanto se ha observado algo que depende de ella. Con la primera opción, la evidencia tiene probabilidad

cero; al eliminar después las variables de azar anteriores, `CEAlgebra.multiplyAndMarginalize` ve probabilidades cero y pone la partición cero, y en un solo criterio la utilidad queda multiplicada por cero.

Si antes de la decisión afectada no hay ninguna variable (como «Dec: Test» en esa red), no queda nada que multiplicar por cero: la utilidad sale bien, aunque la probabilidad de la evidencia se quede en 0; en coste-efectividad ese caso falla antes, en `optimalCEP` (error 85). En `ID-arthronet-ce.pgmx`, en cambio, «Realizar Implante» es la primera decisión pero se toma conociendo «IMC», «Diabetes» y «Alergia ATB», que se eliminan después.

Medido, siguiendo la eliminación con `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx` y «New test = negative»: antes de eliminar «Dec: New test», la probabilidad sobre «Dec: New test», «Test» y «Dec: Test» vale 0,8138, 0,732 y 0,0818 en tres casillas con «Dec: New test = yes» y 0 en todas las de «no»; tras eliminar la decisión, la probabilidad sobre «Test» y «Dec: Test» es cero entera, y al eliminar «Test» todas las particiones pasan a ser la partición cero.

Resultados medidos, comparados con el mismo cálculo imponiendo como política la única opción compatible con el hallazgo (lo que debería dar):

```
ID-CEA-test-2therapies-new-test.pgmx
  New test = negative   coste-efectividad global: partición cero (CEPDialog:
  ↪  NullPointerException)
                        evaluación: utilidad 0, probabilidad 0; estrategia óptima «Dec:
                        ↪  Test = no», de un empate
                        con «Dec: New test = yes» impuesta: 1 tramo, coste 150, efectividad
                        ↪  9,8486; utilidad 295 308,34
  New test = positive   igual; con la política impuesta: 3 tramos; utilidad 157 075,56
  New test = not done   (compatible con la primera opción) 4 tramos; utilidad 267 513,8:
  ↪  sale bien
  Test = positive       («Dec: Test» no tiene nada antes) evaluación 194 271,92, igual que
  ↪  con la política impuesta
ID-arthronet-ce.pgmx
  Isquemia = Menor 1 hora y media   partición cero; evaluación: utilidad 0; estrategia
  ↪  óptima nula
                                    con «Realizar Implante = si» impuesta: utilidad 69
                                    ↪  423,50
```

En `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx`, el análisis global devuelve también la partición cero con «Isquemia = Mayor 1 hora y media» y con los otros dos valores de «CC_Drenaje»; en `0_miscellaneous/networks/id/ID-mediastinet-ce.pgmx`, con «MED = negative» y «MED = positive». Para «CC_Drenaje = menor 800 cc o mayor 1000 cc» y «MED = negative» se ha comprobado que la propagación del modo inferencia acepta el hallazgo y pinta la decisión afectada en su segunda opción con probabilidad 1; con «New test = negative», «Dec: New test = yes».

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/-DecisionVariableElimination.java:50-57

```
TablePotential jointProbability;

if (!probPotentials.isEmpty()) {
    jointProbability = DiscretePotentialOperations.multiply(probPotentials);
    projectedProbability =
↪  DiscretePotentialOperations.projectOutVariable(variableToDelete, jointProbability);
} else {
    projectedProbability = DiscretePotentialOperations.createUnityProbabilityPotential();
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotential-

Transform.java:99-104

```
static TablePotential projectOutVariable(Variable variable, TablePotential inputPotential)
        throws IncompatibleEvidenceException.EvidenceIsIncompatibleWithOther,
        ↪  NonProjectablePotentialException {
    EvidenceCase evi = new EvidenceCase();
    evi.addFinding(new Finding(variable, variable.getStates()[0]));
    return inputPotential.tableProject(evi, null);
}
```

**Cómo arreglarlo.** Al eliminar una decisión, la probabilidad que queda no puede tomarse en la primera opción cuando depende de la decisión. Opciones técnicas:

- multiplicar la utilidad por esa probabilidad antes de maximizar, trabajando con utilidades sin normalizar, y dividir al final por la probabilidad de la evidencia, que es la forma habitual de la eliminación de variables en diagramas de influencia con evidencia; para coste-efectividad ya existe la multiplicación de una partición por una probabilidad (`CEAlgebra.multiply`);
- tomar la probabilidad en la opción óptima: en un solo criterio es la política que calcula `MaxOutVariable`, pero en coste-efectividad la opción óptima cambia de un tramo a otro de la disposición a pagar, así que ahí no basta.

En cualquier caso, `projectOutVariable` no debería proyectar en silencio una probabilidad que depende de la variable: al menos, comprobarlo.

La misma regla vale en los tres sitios que usan `DecisionVariableElimination`: `VariableEliminationCore` (evaluación, coste-efectividad y políticas de la propagación), `DecisionTreeExpansion` (línea 389) y `DANInference` (línea 113). Este arreglo y el de error 85 se necesitan los dos: arreglado solo `optimalCEP`, «New test = negative» seguiría dando la partición cero; arreglado solo esto, «Test = positive» seguiría fallando en `optimalCEP`.

Prueba de regresión: con `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx` y «New test = negative», `VEEvaluation` debe dar 295 308,34 y `VECEAnalysis` global un tramo con coste 150 y efectividad 9,8486, lo mismo que imponiendo «Dec: New test = yes»; y la probabilidad de la evidencia debe ser la misma en los dos cálculos.

### Error 55 · [Corregido] Las ventanas «utilidad esperada» y «política óptima» de una decisión se titulan con el nombre de la clave en bruto

| | |
|---|---|
| **Estado** | Corregido en el commit `d0504fe` (16-10-2026): las dos ventanas piden su título a la base de textos y le añaden el nombre de la decisión, como las demás ventanas de relaciones: «Expected utility: Therapy» y «Optimal policy: Therapy». Con prueba, que abre las dos ventanas de verdad y lee su título; necesita pantalla, y sin ella se salta. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window.edition.networkEditorPanel |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:575-577,601-604`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditDialog.java:76-85`<br>`gui/src/main/resources/gui/localize/Dialogs_en.xml:824-829` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 51, nota 32 |

**Qué pasa.** En modo inferencia, clic derecho sobre una decisión sin política impuesta → «Show expected utility» o «Show optimal policy». Las dos opciones abren un `PotentialEditDialog` de solo lectura cuya barra de título dice `ExpectedUtilityDialog.Title` y `OptimalPolicyDialog.Title`, sin el nombre de la decisión.

Desde el commit `295a7a3`, que corrige el error 51, las dos ventanas se abren y el título con la clave en bruto se ve: comprobado en la ventana el 16-10-2026.

Por qué pasa: el constructor de `PotentialEditDialog` pone un título correcto sacado de la base de textos, `getBaseTitle() + ": " + nombreDelNodo`, pero `NetworkEditorPanel.showExpectedUtilityOfNode` y `showOptimalPolicyOfNode` lo sustituyen a continuación por la cadena literal de la clave.

Comprobado consultando la base de textos: `ExpectedUtilityDialog.Title` → «Expected utility» y `OptimalPolicyDialog.Title` → «Optimal policy» existen en `Dialogs_en.xml`. Es un texto que se ve mal en inglés, así que la decisión de tener la aplicación solo en inglés no lo deja fuera.

**El código.** gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:583-586

```
PotentialEditDialog expectedUtilityDialog = new
↪  PotentialEditDialog(ComponentUtilities.getOwner(this), dummyNode, true);
expectedUtilityDialog.setTitle("ExpectedUtilityDialog.Title");
expectedUtilityDialog.requestValues();
```

gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:609-612

```
PotentialEditDialog optimalPolicyDialog =
        new PotentialEditDialog(ComponentUtilities.getOwner(this), dummyNode, true);
optimalPolicyDialog.setTitle("OptimalPolicyDialog.Title");
optimalPolicyDialog.requestValues();
```

**Cómo arreglarlo.** Pedir el texto a la base: `StringDatabase.getUniqueInstance().getString("ExpectedUtilityDialog.Title") + ": " + node.getName()` (y lo mismo con `OptimalPolicyDialog.Title`). Otra opción es dar a `PotentialEditDialog` el título base por parámetro, o por una subclase que redefina `getBaseTitle()`, como ya hace `ImposePolicyDialog`, para que el título lleve el nombre del nodo como las demás ventanas de potencial. Los dos sitios están en el mismo fichero.

Prueba: construir el diálogo por el mismo camino (sin mostrarlo) y comprobar que `getTitle()` empieza por «Expected utility» o «Optimal policy» y contiene el nombre de la decisión; o, en general, una prueba que busque `setTitle("` seguido de una clave con punto en el código de la interfaz.

### Error 56 · [Corregido] Deshacer «quitar la política impuesta» no devuelve la política: se pierde la tabla

| | |
|---|---|
| **Estado** | Corregido en el commit `96186e3` (08-10-2026): `RemovePolicyEdit` guarda los potenciales que tenga el nodo, sea cual sea el tipo de política, y `undo()` los repone; ya no hay `get(0)` sin guarda. El caso «removing the imposed policy of a decision» de `EditUndoCases` impone la política {0, 1}, la quita y deshace. El texto de la edición nombra ahora `oldPotentials`. El tipo de política no se ha tocado. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |

| | |
|---|---|
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/RemovePolicyEdit.java:27-49`<br>`core/src/main/java/org/openmarkov/core/model/network/Node.java:95`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/ImposePolicyPanel.java:31-43`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:553-564`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:659-665`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:464-473` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 51 |

**Qué pasa.** En modo edición, con una decisión que tiene política impuesta, el usuario pide en el menú contextual del nodo «quitar política» (`NetworkEditorPanel.removePolicyFromNode`, habilitado cuando la decisión tiene potenciales) y luego Edición → Deshacer. La política impuesta, que el usuario escribió a mano, se pierde; la entrada del menú «Deshacer» aparenta haber funcionado.

Comprobado con un pequeño programa que reproduce esos pasos (política {0, 1} puesta como hace `ImposePolicyEdit`, `RemovePolicyEdit`, deshacer): tras deshacer, la decisión tiene 0 potenciales.

Por qué pasa: `RemovePolicyEdit` (módulo `core`) quita la política impuesta a una decisión. En el constructor guarda el potencial solo si `node.getPolicyType() != PolicyType.OPTIMAL`; en `doEdit()` vacía los potenciales del nodo siempre; en `undo()`, si no guardó nada, vuelve a vaciarlos.

El tipo de política nunca es otro que `OPTIMAL` en producción:

- el campo `Node.policyType` nace en `OPTIMAL`;
- `ImposePolicyPanel.initializePotential`, que es lo que corre al imponer una política desde la ventana, hace `node.setPolicyType(PolicyType.OPTIMAL)` antes de crear la tabla;
- el lector de `.pgmx` no escribe el tipo;
- el panel que permitiría elegir «determinista» o «probabilista» (`PolicyTypePanel`) está desactivado: su construcción está comentada en `PotentialEditPanel` (ver error 51).

Recorriendo las redes del repositorio se encontraron decisiones con política impuesta y tipo «óptima» en `io/src/test/resources/ImposedPolicy.pgmx`, `integrationTests/src/main/resources/networks/mid/21-gene.pgmx` y otras.

Además, el `getPotentials().get(0)` del constructor lanza `IndexOutOfBoundsException` si una decisión marcada como no óptima no tiene tabla (comprobado construyendo el caso por programa), pero no se alcanza, porque nada marca hoy una decisión como no óptima. Tampoco es un cambio registrado el tipo de política: `setPolicyType` se llama directamente, sin edición.

**El código.** core/src/main/java/org/openmarkov/core/action/core/RemovePolicyEdit.java:27-49

```
public RemovePolicyEdit(Node node) {
    super(node.getProbNet());
    this.node = node;
    if (node.getNodeType() == NodeType.DECISION && node.getPolicyType() !=
    ↪  PolicyType.OPTIMAL) {
        oldPotential = node.getPotentials().get(0);
    } else {
        oldPotential = null;
```

```
    }
}

@Override protected void doEdit() {
    node.clearPotentials();
}

@Override public void undo() {
    super.undo();
    if (oldPotential != null) {
        node.setPotential(oldPotential);
    } else {
        node.clearPotentials();
    }
}
```

**Cómo arreglarlo.** Guardar lo que el nodo tenga, sea cual sea el tipo de política: una copia de la lista `node.getPotentials()` (vacía si no hay), y reponerla entera en `undo()` con `setPotentials`. Eso elimina también el `get(0)` sin guarda.

Si además se quiere que el tipo de política signifique algo, `ImposePolicyPanel` no debería fijarlo en `OPTIMAL` al imponer, y cambiarlo debería ser una edición; eso es un cambio aparte.

Prueba: imponer una política $\{0, 1\}$ con `ImposePolicyEdit`, ejecutar `RemovePolicyEdit` y deshacer: la decisión debe volver a tener exactamente esa tabla. Y construir `RemovePolicyEdit` sobre una decisión sin tabla no debe lanzar.

### Error 57 · [Corregido] Renombrar un criterio de decisión no se deshace, ni con Deshacer ni con «Cancelar» en las propiedades de la red

| | |
|---|---|
| **Estado** | Corregido en el commit `871dc06` (08-10-2026): `DecisionCriteriaEdit` guarda el nombre anterior del criterio y, al deshacer un renombrado, se lo devuelve al mismo objeto. El caso «renaming a decision criterion» de `EditUndoCases` lo comprueba. El «Cancelar» del diálogo no se ha probado aparte: deshace con este mismo `undo()`. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/DecisionCriteriaEdit.java:44`<br>`core/src/main/java/org/openmarkov/core/action/core/DecisionCriteriaEdit.java:67-83`<br>`core/src/main/java/org/openmarkov/core/action/core/DecisionCriteriaEdit.java:97-101`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/DecisionCriteriaTablePanel.java:56-70`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:281-284` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 58, nota 36, nota 81 |

**Qué pasa.** En Red → Propiedades → pestaña de criterios de decisión, el usuario edita el nombre de un criterio en la tabla (`DecisionCriteriaTablePanel.tableChanged` construye la edición `RENAME`). El diálogo abre un historial parcial al entrar y, al pulsar «Cancelar», deshace lo hecho (`cancelLast`

`SubEditHistory`). Pero el renombrado no se deshace: **«Cancelar» no cancela el renombrado**, y tampoco lo deshace Edición → Deshacer tras «Aceptar».

Comprobado con un pequeño programa de prueba: renombrar «cost» a «coste» y deshacer deja «coste»; lo mismo dentro de un historial parcial cancelado.

Por qué pasa: `DecisionCriteriaEdit` (módulo `core`) hace en el constructor `lastCriteria = new ArrayList<>(probnet.getDecisionCriteria())`, una lista nueva con los mismos objetos `Criterion`. Para `RENAME`, `doEdit()` cambia el nombre del objeto (`modifiedCriterion.setCriterionName(newName)`), y `undo()` repone la lista guardada, que contiene ese mismo objeto ya renombrado. Deshacer no cambia nada. Añadir, quitar, subir y bajar sí se deshacen, porque ahí lo que cambia es la lista.

Los criterios se comparten por referencia: la variable de cada nodo de utilidad apunta a uno (`Variable.getDecisionCriterion`), así que es el propio objeto el que hay que devolver a su nombre.

**El código.** core/src/main/java/org/openmarkov/core/action/core/DecisionCriteriaEdit.java:44, 79, 97-101

```
this.lastCriteria = new ArrayList<>(probnet.getDecisionCriteria());
...
case RENAME -> this.modifiedCriterion.setCriterionName(newName);
...
@Override public void undo() {
    super.undo();
    probNet.setDecisionCriteria(lastCriteria);
}
```

**Cómo arreglarlo.** Guardar el nombre anterior del criterio en el constructor (o en `doEdit`) y, para `RENAME`, devolvérselo al mismo objeto en `undo()`; para las demás acciones, reponer la lista como ahora.

Prueba: renombrar un criterio y deshacer debe devolver el nombre viejo; la misma prueba dentro de `openNewSubEditHistory` / `cancelLastSubEditHistory` reproduce el «Cancelar» del diálogo. Conviene comprobar también que las variables de utilidad que apuntan al criterio muestran el nombre viejo tras deshacer.

### Error 58 · Confirmar el nombre de un criterio sin cambiarlo, o cambiar solo sus mayúsculas, se rechaza como si chocara consigo mismo

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/DecisionCriteriaEdit.java:47-65`<br>`core/src/main/java/org/openmarkov/core/model/network/constraint/ValidCriterionName.java:31-54`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/DecisionCriteriaTablePanel.java:56-84` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si dos criterios pueden llamarse igual salvo mayúsculas (la comprobación de la edición las ignora y la de red completa no) |
| **Relacionados** | error 38, error 57, nota 143 |

**Qué pasa.** En Red → Propiedades → criterios de decisión, terminar de editar la celda del nombre de un criterio sin cambiar el texto, o cambiando solo sus mayúsculas («cost» → «Cost»), se rechaza como si el nombre ya lo tuviera otro criterio. En la aplicación, el gestor global de excepciones muestra un diálogo de error.

Comprobado construyendo el panel sin pantalla: escribir en la celda el mismo nombre que ya tiene lanza `UnrecoverableException` con causa `CriterionNameIsAlreadyPresent`. Medido también directamente sobre la edición: «cost» → «cost» y «cost» → «Cost» se rechazan ambos.

`DecisionCriteriaTablePanel.tableChanged` construye un `RENAME` con cada evento de actualización de la columna del nombre, y una tabla Swing emite ese evento al terminar de editar una celda aunque el texto no haya cambiado (el modelo de la tabla no filtra valores iguales).

Por qué pasa: `DecisionCriteriaEdit.checkConstraintsWillBeMet` (módulo `core`), para `ADD` y `RENAME`, pasa el nombre nuevo a minúsculas y sin espacios y lo compara con **todos** los criterios de `lastCriteria`, incluido `modifiedCriterion`. Al renombrar un criterio a su propio nombre, o a su nombre con otras mayúsculas, encuentra el propio criterio y añade `CriterionNameIsAlreadyPresent`. La restricción `ValidCriterionName` lleva `@Constraint(defaultBehavior = YES)`, así que está en todas las redes (comprobado en redes bayesianas, diagramas de influencia, MID —diagramas de influencia de Markov— y DAN —redes de análisis de decisiones—), y la comprobación corre siempre.

Las dos copias de la regla difieren: la comprobación de red completa, `ValidCriterionName.checkProbNet`, compara con igualdad exacta y sí admitiría «Cost» junto a «cost».

Subir el primer criterio o bajar el último, en la misma pestaña, es otro asunto, que la ventana no deja pedir y donde no hay nada que corregir (nota 36).

**El código.** core/src/main/java/org/openmarkov/core/action/core/DecisionCriteriaEdit.java:52-61

```
case ADD, RENAME -> {
    String name = this.newName.trim().toLowerCase();
    if (name.isEmpty()) {
        constraintChecker.addException(new
↪   ConstraintViolatedException.CriterionNameIsEmpty(constraint, modifiedCriterion));
    }
    for (Criterion criterion : this.lastCriteria) {
        if (criterion.getCriterionName().trim().toLowerCase().equals(name)) {
            constraintChecker.addException(new
↪   ConstraintViolatedException.CriterionNameIsAlreadyPresent(constraint,
↪   modifiedCriterion, name));
        }
    }
}
```

**Cómo arreglarlo.** Al renombrar, saltarse `modifiedCriterion` en el bucle (comparación por identidad), y en la tabla no construir la edición si el nombre no ha cambiado.

Decidir si las mayúsculas cuentan y aplicar la misma regla en los dos sitios: la comprobación previa de la edición (`DecisionCriteriaEdit.checkConstraintsWillBeMet`) y la de red completa (`ValidCriterionName.checkProbNet`). La comprobación de la edición también la usa el diálogo de criterios estándar (`StandardCriteriaDialog`, que añade criterios con `ADD`).

Prueba: renombrar un criterio a su propio nombre no debe lanzar; renombrarlo al nombre de otro criterio sí.

### Error 59 · [Corregido] Evaluar una red de decisiones que no tiene ningún criterio de decisión muere con IndexOutOfBoundsException en vez de explicar qué falta

| | |
|---|---|
| **Estado** | Corregido en el commit `fff73e1` (08-10-2026) para el camino del fichero: `PGMXReader_0_2.getDecisionCriterion` da el criterio por omisión también cuando `<DecisionCriteria>` está vacío, salvo en redes solo de azar. `DAN-dating.pgmx` con el elemento vacío se lee con un criterio y se evalúa con el mismo resultado que el fichero original. Los sitios que suponen que hay un criterio no se han tocado; el otro camino hacia una red sin criterios, el diálogo de criterios estándar, es el error 174. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialFactory.java:41-47`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANInference.java:64-69`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:338-362`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:714-727`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/StandardCriteriaDialog.java:82-110`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/StandardCriteriaDialog.java:126-136` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 174 |

**Qué pasa.** Un criterio de decisión es la unidad en la que se mide una utilidad (por ejemplo, coste en euros o efectividad en años de vida ajustados por calidad). Cada red guarda una lista de criterios y cada nodo de utilidad apunta a uno. En OpenMarkov, «ser una utilidad» se reconoce precisamente porque el potencial lleva criterio.

Evaluar una red de análisis de decisiones con la lista de criterios vacía termina en un error interno sin relación aparente con los criterios. Medido con copias de `0_miscellaneous/networks/dan/DAN-dating.pgmx` y `0_miscellaneous/networks/dan/DAN-decide-test.pgmx` a las que se dejó el elemento `<DecisionCriteria></DecisionCriteria>` vacío: el lector las abre sin queja con cero criterios, y la evaluación por descomposición en redes simétricas (`DANDecompositionIntoSymmetricDANsEvaluation`, la que usa la ventana para la estrategia óptima de una red de análisis de decisiones) muere en las dos con `IndexOutOfBoundsException: Index 0 out of bounds for length 0` en `DANInference.setUtility(DANInference.java:66)`, llamado desde `maximizeAndSetUtility` (línea 121).

Cómo se llega a una red sin criterios:

- Abriendo un `.pgmx` cuyo elemento `<DecisionCriteria>` esté vacío. El escritor de OpenMarkov no lo escribe así (si la lista está vacía omite el elemento, y al leer un fichero sin ese elemento el lector crea un criterio por omisión salvo en redes solo de azar), así que el fichero tiene que venir de fuera o estar editado a mano.
- Desde la ventana, medido ejecutando el método de aceptar del diálogo sin construir la ventana (error 174): el botón de criterios estándar de las propiedades de la red abre `StandardCriteriaDialog`, que no preselecciona ninguna opción; al aceptar, primero borra todos

los criterios con ediciones y después, si no se eligió ninguna opción, `getDefaultCriteria` hace `selectedDefaultCriteria.equals(...)` sobre un nulo (línea 136) y lanza `NullPointerException`, dejando la red sin criterios si después se aceptan las propiedades de la red. El botón de borrar un criterio de la tabla, en cambio, solo se habilita con dos o más filas, así que por ahí no se vacía la lista.

Por qué pasa: `TablePotentialFactory.createZeroUtilityPotential(red)` fabrica una utilidad que vale cero y le pone `red.getDecisionCriteria().get(0)`: comprueba que la red no sea nula, pero no que la lista tenga algún criterio. La llaman cinco sitios de la evaluación de redes de análisis de decisiones y de árboles de decisión (`DecisionTreeExpansion:395`, `DANInference:118`, `DANOperations:566` y `:579`, `DANConditionalSymmetricInference:44`) cuando una rama se queda sin utilidad.

El mismo supuesto aparece en `DANInference.setUtility` (línea 66), que da el primer criterio de la red a toda utilidad que llegue sin criterio; en el lector de ficheros (`PGMXReader_0_2`, línea 726: un nodo de utilidad sin elemento `<Criterion>` recibe el primero de la red); y en `StandardCriteriaDialog` (línea 104).

Con una red de ese tipo sin nodos de utilidad la evaluación contestó 0 sin fallar, así que no se consiguió hacer pasar la ejecución por `createZeroUtilityPotential` con una red real; leyendo el código, falla igual (`get(0)` sobre lista vacía).

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialFactory.java:41-47

```
public static TablePotential createZeroUtilityPotential(ProbNet dan) {
    TablePotential newPotential = createOneValuePotential(PotentialRole.UNSPECIFIED, 0.0);
    if (dan != null) {
        newPotential.setCriterion(dan.getDecisionCriteria().get(0));
    }
    return newPotential;
}
```

inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANInference.java:64-69

```
protected void setUtility(Potential util) {
    if (util.getCriterion() == null) {
        util.setCriterion(this.probNet.getDecisionCriteria().get(0));
    }
    this.utility = util;
}
```

**Cómo arreglarlo.** Hay que decidir qué significa una red de decisiones sin criterios y aplicarlo en todos los sitios que suponen que hay uno: `TablePotentialFactory.createZeroUtilityPotential`, `DANInference.setUtility`, `PGMXReader_0_2` (línea 726) y `StandardCriteriaDialog` (línea 104). Opciones:

- rechazar la red al evaluarla con una excepción del dominio que diga «la red no tiene criterios de decisión» (por ejemplo `NotEvaluableNetworkException`);
- crear un criterio por omisión al leer un `<DecisionCriteria>` vacío, como ya se hace cuando el elemento falta.

Que `StandardCriteriaDialog` borre los criterios antes de saber que hay una opción elegida es un defecto propio, error 174.

Prueba de regresión: leer `DAN-dating.pgmx` con `<DecisionCriteria></DecisionCriteria>` y eva-

luarla con `DANDecompositionIntoSymmetricDANsEvaluation`: debe dar el mensaje elegido (o evaluar con el criterio por omisión), no `IndexOutOfBoundsException`; y `createZeroUtilityPotential` con una red sin criterios no debe lanzar ese error.

### Error 60 · [Corregido] El agente asignado a cada decisión se pierde al guardar y no se lee al abrir

| | |
|---|---|
| **Estado** | Corregido en el commit `341e1ed` (08-10-2026): el escritor pone `<Agent>` en la variable que tiene agente, y el lector lo busca por nombre en la lista de la red y asigna ese objeto. Para eso el lector lee ahora la lista de agentes antes que las variables. Si el nombre no está en la lista, el fichero se rechaza con `PGMXParserException.UnknownAgent`. Medido con `test-decpomdp-manual.pgmx`: sus cuatro decisiones se leen con su agente, y también tras guardar con `PGMXWriter_1_0` y releer. No se ha comprobado `OnlyOneAgent` al abrir: abrir un fichero no comprueba restricciones (error 37). El escritor pone `<Agent>` tras `<Criterion>`, donde ya escribía el criterio, que no es el orden del esquema. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.reader; org.openmarkov.io.probmodel.writer |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:318-332,669-789`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:235-266,410-432,503-508`<br>`core/src/main/resources/val_v4.xsd:58-61`<br>`io/src/test/resources/test-decpomdp-manual.pgmx:55,68,81,93`<br>`0_miscellaneous/networks/pomdp/Dec-POMDP-wireless-network.pgmx:153,163`<br>`io/src/test/java/org/openmarkov/io/probmodel/AgentsTest.java` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 61, error 62, nota 72 |

**Qué pasa.** Abrir un Dec-POMDP (proceso de decisión de Markov parcialmente observable descentralizado: varios decisores que no comparten lo que ven) deja todas sus decisiones sin agente, y guardar una red después de asignar agentes a mano los borra del fichero. La asignación solo vive mientras la red está abierta. Hoy ningún algoritmo de inferencia lee `Variable.getAgent()` (solo la interfaz, la restricción `OnlyOneAgent` y `AddNodeEdit`), así que la pérdida no cambia ningún resultado numérico: lo que se pierde es el modelo que el usuario escribió.

Un Dec-POMDP necesita dos datos: la lista de agentes de la red y, para cada nodo de decisión, a qué agente pertenece. En OpenMarkov la lista está en `ProbNet.getAgents()` y la pertenencia en `Variable.getAgent()` (módulo `core`). El usuario asigna el agente de una decisión en el diálogo de propiedades del nodo, con el selector «Agent» (`NodeDefinitionPanel.getJComboBoxNetworkAgents`), que aparece en las redes cuyo tipo tiene agentes (por ejemplo, el tipo `DEC_POMDP`).

El formato propio lo contempla: el esquema `core/src/main/resources/val_v4.xsd` (líneas 58-61) admite un `<Agent>` dentro de cada `<Variable>`, junto a `<Criterion>`, y los dos ficheros Dec-POMDP del repositorio lo usan (`io/src/test/resources/test-decpomdp-manual.pgmx`, cuatro decisiones; `0_miscellaneous/networks/pomdp/Dec-POMDP-wireless-network.pgmx`, dos).

Medido con el lector `PGMXReader` y el escritor `PGMXWriter_1_0`:

```
leer test-decpomdp-manual.pgmx  agentes=[Agent 1, Agent 2]  D1 [0]=none D2 [0]=none D1
↪  [1]=none D2 [1]=none
asignar Agent 1 a D1 [0], guardar: el fichero solo contiene <Agents><Agent name="Agent
↪  1"/><Agent name="Agent 2"/></Agents>
```

```
releer                          D1 [0]=none ...
leer Dec-POMDP-wireless-network  agentes=[A, B]  Action A [0]=none Action B [0]=none   (el
↪  fichero dice A y B)
```

Por qué pasa: el lector y el escritor (módulo `io`) solo tratan la lista de la red.

- `PGMXReader_0_2.getAgents` (líneas 318-332) lee `<Agents>` bajo `<ProbNet>`; el método que lee cada variable (`loadVariableAdvancedInformation`, líneas 669-789) lee el criterio de los nodos de utilidad y `<AlwaysObserved>`, y no mira `<Agent>`. `PGMXReader_1_0` hereda ese método.
- `PGMXWriter_0_2.getAgents` (líneas 235-266) escribe la lista; `getVariableChildren` (líneas 410-432) escribe el criterio (`getDecisionCriterion`, líneas 503-508) y no el agente. `PGMXWriter_1_0` no lo cambia.

La prueba `io/src/test/java/org/openmarkov/io/probmodel/AgentsTest.java` solo comprueba que se leen los dos nombres de la lista, y por eso no lo detecta.

**El código.** io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:426-432

```
getAlwaysObservedAttribute(variableElement, node);
NodeType nodeType = node.getNodeType();
if (nodeType == NodeType.UTILITY) {
    Element decisionCriterionElement = new Element(XMLTags.CRITERION.toString());
    getDecisionCriterion(variableElement, decisionCriterionElement, node);
}
// (no hay nada equivalente para XMLTags.AGENT en una variable de decisión)
```

**Cómo arreglarlo.** Escribir `<Agent name="..."/>` dentro de la variable cuando `node.getVariable().getAgent() != null`, en el orden que fija el esquema (después de `<Criterion>`, antes de `<AlwaysObserved>`). Y leerlo en el método de lectura de variables: buscar en `probNet.getAgents()` el agente con ese nombre (comparando texto con `equals`) y asignar **ese objeto** a la variable, igual que se hace con el criterio.

Hay que decidir qué hacer si el nombre no está en la lista: lanzar la excepción de fichero mal formado del lector, o avisar. Lo que no conviene es inventar un agente nuevo que la lista no tiene.

Hay que arreglarlo **a la vez que error 62**: en cuanto el lector asigne agentes, abrir las propiedades de una decisión de un fichero leído puede lanzar `IllegalArgumentException` si el objeto cadena no es el mismo que el del selector. Asignar el mismo objeto de la lista, como se propone arriba, evita la excepción, pero el selector debe arreglarse igualmente.

Comprobar también que la restricción `OnlyOneAgent` (redes de un solo agente) sigue rechazando un fichero que asigne agentes en una red que no los admite, una vez el lector los lea.

Prueba de regresión: en `AgentsTest`, leer `test-decpomdp-manual.pgmx` y comprobar que `D1 [0]` y `D1 [1]` tienen «Agent 1» y `D2 [0]`, `D2 [1]` «Agent 2»; guardar con `PGMXWriter_1_0`, releer y comprobar lo mismo.

### Error 61 · [Corregido] Quitar, renombrar o mover un agente de la red deja las decisiones con agentes que la lista ya no tiene

| | |
|---|---|
| **Estado** | Corregido en el commit `cc841e8` (08-10-2026): `ProbNetAgentManager.modifyAgent` monta la lista de RENAME, UP y DOWN con los agentes que ya había, buscados por nombre; el renombrado se crea con las propiedades del antiguo y se asigna a las decisiones que lo tenían. `NetworkAgentEdit` guarda el agente de cada nodo y lo repone al deshacer. Un agente se identifica por su nombre. |

| | |
|---|---|
| | Medido con dos agentes (uno con una propiedad) y una decisión de cada uno: quitar, renombrar, subir y bajar, y el deshacer de las tres primeras, dejan la lista, las decisiones y la propiedad coherentes. No se han tocado el selector de agente del nodo, que sigue asignando un objeto nuevo con el nombre, ni la copia de redes. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.action; org.openmarkov.core.model.network |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/action/NetworkAgentEdit.java:48-59`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetAgentManager.java:47-86`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkAgentsTablePanel.java:44-147`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:80-91,270-283` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 60, error 62 |

**Qué pasa.** Las redes con varios decisores (tipo Dec-POMDP, proceso de decisión de Markov parcialmente observable descentralizado) tienen una lista de agentes, editable en el menú File → «Network properties…» (Ctrl+D) → pestaña de agentes (`NetworkAgentsTablePanel`), y cada decisión puede pertenecer a uno (`Variable.getAgent()`). Cada operación de la pestaña ejecuta una `NetworkAgentEdit` (módulo `gui`, paquete `org.openmarkov.gui.action`), que delega en `ProbNetAgentManager.modifyAgent` (módulo `core`). Tres cosas dejan la lista y los nodos diciendo cosas distintas.

**1. Deshacer un borrado no devuelve el agente a sus decisiones.** Quitar un agente (REMOVE, líneas 58-77 de `ProbNetAgentManager`) lo quita de la lista y pone a `null` el agente de las decisiones que eran suyas, lo cual es correcto. Pero `NetworkAgentEdit.undo()` (líneas 55-59) solo repone la lista; el propio código lo dice con un `//TODO restore agents in nodes`. Esto se alcanza también sin pulsar Ctrl+Z: el botón **Cancelar** del diálogo de propiedades de la red deshace todo lo hecho en él (`doCancelClickBeforeHide` → `cancelLastSubEditHistory`, líneas 280-282). Un usuario que quita un agente por error y cancela ve la lista como estaba, pero sus decisiones se han quedado sin dueño, sin aviso.

**2. Renombrar, subir o bajar un agente borra sus propiedades.** Un agente es un `StringWithProperties`: un nombre y unas propiedades adicionales que el formato guarda. Para RENAME, UP y DOWN (líneas 78-84) la lista se reconstruye con `new StringWithProperties((String) objects[0])`, es decir, solo con los nombres. Subir y bajar no cambian ningún nombre y aun así pierden las propiedades. Hoy la interfaz no permite escribir propiedades de agentes, así que solo se pierden las que vengan de un fichero (`PGMXReader_0_2.getAgents` las lee); ninguna red del repositorio las tiene. Al guardar después, se pierden del fichero.

**3. Renombrar un agente no renombra a sus decisiones.** Las decisiones que eran de «Agent 1» siguen diciendo «Agent 1» cuando la lista ya dice otra cosa. Ese nombre huérfano es el que hace que el diálogo de propiedades de esas decisiones no se abra (error 62).

Medido con una red `DECPOMDPType` con historial de deshacer, con «Agent 1» con una propiedad, D1 de «Agent 1» y D2 de «Agent 2»:

```
inicio                      agentes=Agent 1(1 prop.) Agent 2(0)   D1=Agent 1 D2=Agent 2
quitar Agent 1              agentes=Agent 2(0)                    D1=none    D2=Agent 2
deshacer                    agentes=Agent 1(1 prop.) Agent 2(0)   D1=none    D2=Agent 2
```

```
subir Agent 2                 agentes=Agent 2(0) Agent 1(0 prop.)   D1=Agent 1 D2=Agent 2
renombrar Agent 1 -> Doctor   agentes=Doctor(0) Agent 2(0)          D1=Agent 1 D2=Agent 2
```

El commit `8055530` corrigió que se pudieran crear agentes con el mismo nombre (issue 577) y las excepciones del panel al añadir, quitar o mover filas (issue 646); ninguno de los tres defectos de arriba cambia con eso. El deshacer del cambio de tipo de red que borraba los agentes sí está corregido y tiene prueba (`ChangeNetworkTypeEditAgentsTest`); es otro camino.

**El código.** gui/src/main/java/org/openmarkov/gui/action/NetworkAgentEdit.java:48-59

```java
@Override protected void doEdit() {
    this.lastAgents = super.getProbNet().getAgents().stream()
            .collect(Collectors.toList());
    probNet.modifyAgent(stateAction,agentName,dataTable);
}

@Override public void undo() {
    super.undo();
    probNet.setAgents(lastAgents);
    //TODO restore agents in nodes
}
```

core/src/main/java/org/openmarkov/core/model/network/ProbNetAgentManager.java:78-84

```java
case DOWN, RENAME, UP:
    ArrayList<StringWithProperties> modifiedAgent = new ArrayList<>();
    for (Object[] objects : dataTable) {
        modifiedAgent.add(new StringWithProperties((String) objects[0]));
    }
    probNet.setAgents(modifiedAgent);
    break;
```

**Cómo arreglarlo.**

- Deshacer: guardar en `doEdit`, además de la lista, qué agente tenía cada decisión (o solo las que cambian), y reponerlo en `undo()`.
- Mover: reordenar los objetos `StringWithProperties` existentes en vez de crear otros con el nombre.
- Renombrar: crear el agente nuevo copiando las propiedades del viejo y cambiar el agente de las decisiones que tenían el viejo; deshacerlo debe devolver las dos cosas.

Hay que decidir de paso cómo se identifica un agente dentro de la red (por objeto o por nombre) y aplicarlo igual en `ProbNetAgentManager`, en el selector de agente del nodo (error 62), en el lector y el escritor de ficheros (error 60) y en la copia de redes (`Variable` clona el agente con `CloneUtils.safeClone`). Hoy `REMOVE` compara por texto y el selector por objeto.

Prueba de regresión: sobre una red `DECPOMDPType` con dos decisiones de agentes distintos y un agente con una propiedad, comprobar para cada operación (quitar, renombrar, subir, bajar) y su deshacer que la lista y el agente de cada decisión quedan coherentes y que las propiedades se conservan.

### Error 62 · [Corregido] Las propiedades de una decisión no se abren si su agente ya no coincide, como objeto, con uno de la lista

| | |
|---|---|
| **Estado** | Corregido en el commit `25dd552` (08-10-2026): la búsqueda del agente está en `NodeDefinitionPanel.indexOfAgent`, que compara los nombres con `equals` y devuelve el hueco vacío (puesto 0) si el nombre no está en la lista. Probado sobre ese método, no abriendo el diálogo. El nombre huérfano en el nodo sigue siendo el error 61; no se han buscado otros `==` entre cadenas en `gui`. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:129-138,622-671,674-677`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:73-88,337-345`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetAgentManager.java:78-84`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkAgentsTablePanel.java:44-78` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 60, error 61, nota 30, nota 72 |

**Qué pasa.** El diálogo de propiedades de una decisión deja de abrirse después de renombrar su agente en las propiedades de la red. Camino de hoy:

1. Red de tipo Dec-POMDP (proceso de decisión de Markov parcialmente observable descentralizado; tiene «Agent 1» y «Agent 2»). Abrir las propiedades de una decisión y elegir «Agent 1». El nodo guarda la misma cadena que tenía el selector, así que de momento todo cuadra.
2. Abrir File → «Network properties…», pestaña de agentes, y renombrar «Agent 1» a «Doctor» (`NetworkAgentsTablePanel.tableChanged` → `NetworkAgentEdit` RENAME). La lista pasa a decir «Doctor», pero el nodo sigue diciendo «Agent 1» (ver error 61).
3. Abrir otra vez las propiedades de esa decisión: el nombre «Agent 1» no está en la lista y el diálogo no se abre.

Aunque el usuario vuelva a renombrar «Doctor» a «Agent 1», la cadena tecleada es otro objeto y la comparación sigue fallando.

Medido llamando al método real del panel sobre una red `DECPOMDPType`:

```
mismo objeto que en la lista                    -> se abre, marcado el puesto 1
mismo texto, otro objeto                        -> IllegalArgumentException:
↪  setSelectedIndex: 3 out of bounds
tras renombrar Agent 1 -> Doctor, decisión de Agent 1 -> IllegalArgumentException:
↪  setSelectedIndex: 3 out of bounds
tras renombrar Agent 1 -> Doctor, decisión de Agent 2 -> se abre, marcado el puesto 2
```

Por qué pasa. Cuando la red tiene lista de agentes, la fila «Agent» del diálogo es un selector con un hueco vacío y los nombres de los agentes de la red. Lo construye `NodeDefinitionPanel.getJComboBoxNetworkAgents` (módulo `gui`, líneas 622-671), que busca cuál marcar comparando el nombre del agente del nodo con cada nombre de la lista mediante `==`. Entre dos `String`, `==` pregunta si son el mismo objeto en memoria, no si dicen lo mismo. Si no hay coincidencia de objeto, el bucle termina con `i` igual al número de elementos y `setSelectedIndex(i)` lanza `IllegalArgumentException`.

Ese método se ejecuta dentro del constructor del panel (líneas 129-138 y, a través de `getAgentsOr`

`DecisionCriteriaOrObserved`, líneas 674-677), y el panel se construye dentro del constructor de `NodePropertiesDialog` (líneas 66-81, `reinitialize` → `getNodeDefinitionPanel`, líneas 330-338). Así que la excepción impide que el diálogo de propiedades se abra. Además, leyendo el código, el constructor del diálogo ya ha abierto un grupo de ediciones (`openNewSubEditHistory`, línea 75) que nadie cierra al fallar; no se ha medido qué efecto tiene eso después.

Cuando se corrija error 60 (leer el agente de cada decisión desde el fichero), habrá un segundo camino: si el lector crea cadenas nuevas, abrir las propiedades de cualquier decisión de un Dec-POMDP recién leído lanzará la misma excepción.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:655-666

```
if (node.getVariable().getAgent() != null && agents != null) {
    String name = node.getVariable().getAgent().getString();
    int i;
    for (i = 0; i < agentNames.length; i++) {
        if (name == agentNames[i]) {
            break;
        }
    }
    jComboBoxNetworkAgents.setSelectedIndex(i);
} else {
    jComboBoxNetworkAgents.setSelectedIndex(0);
}
```

**Cómo arreglarlo.** Comparar con `name.equals(agentNames[i])` y decidir qué se marca si el nombre no está en la lista (lo natural es el hueco vacío, puesto 0, y no una excepción). Con eso el diálogo se abre siempre; el nombre huérfano sigue siendo un defecto de la red, que corrige error 61 (renombrar debe renombrar también en los nodos).

Es el mismo patrón que nota 30 (comparar nombres con `==` en el editor de árboles), donde hoy no hay nada que corregir; conviene buscar otros `==` entre cadenas en `gui`.

Prueba de regresión: construir el selector para una decisión cuyo agente sea `new StringWithProperties(new String("Agent 1"))` en una red con «Agent 1» en la lista y comprobar que queda marcado ese agente; y con un nombre que no está en la lista, que queda marcado el hueco vacío.

### Error 63 · [Corregido] Evaluar DAN-arthronet (y cualquier red de análisis de decisiones con la misma forma) termina con NullPointerException al instanciar

| | |
|---|---|
| **Estado** | Corregido en el commit `2ec95a5` (08-10-2026): `DANOperations.instantiate` pasa al enlace siguiente cuando el nodo de destino ya no está en la copia. `DAN-arthronet.pgmx` se evalúa (0,0825). Con las cuatro decisiones fijadas en sí, no, no, no da −2,3212, lo mismo que la cuenta a mano. No coincide con `ID-arthronet.pgmx` (0,4961) porque los ficheros no son el mismo modelo: la tabla de «Infeccion PTR» da una probabilidad de infección con implante de 0,6455 en el diagrama de influencia y de 0,4675 en la red de análisis de decisiones. Los otros bucles del método que nombra «Cómo arreglarlo» no se han revisado. Al medirlo apareció el error 200. |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.decompositionIntoSymmetricDANs.core |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:71-79` |

| | |
|---|---|
| | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:100-146` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 64, error 113 |

**Qué pasa.** El usuario abre la red y pide la estrategia óptima o la evaluación (`InferenceHandler` usa `DANDecompositionIntoSymmetricDANsEvaluation` para las redes de análisis de decisiones), o el árbol de decisión (`DANDecisionTreeInference` y `DecisionTreeExpansion`, que también llaman a `instantiate`). La red no se puede evaluar: sale un `NullPointerException`.

Medido con `0_miscellaneous/networks/dan/DAN-arthronet.pgmx`:

```
instanciar «Realizar Implante» = no:
  NullPointerException: Cannot invoke "Node.getNodeType()" because "destinationNode" is
  ↪  null
    at DANOperations.instantiate(DANOperations.java:79)
evaluación completa (DANDecompositionIntoSymmetricDANsEvaluation): la misma excepción
```

Por qué pasa: `DANOperations.instantiate` recorre los enlaces que salen de la variable instanciada en la red **original** y, para cada uno, busca el nodo de destino en la **copia** (`instantiatedNet.getNode(...)`, línea 75) y lo usa sin mirar si es nulo (`destinationNode.getNodeType()`, línea 79; y más abajo `destinationNode.getVariable()`, línea 101).

El propio bucle quita nodos de la copia: cuando un enlace con restricciones prohíbe todos los estados de su destino para el estado instanciado (líneas 106-146), se eliminan el destino y sus descendientes. Si otro enlace de la misma variable, que llega más tarde en el recorrido, apuntaba a uno de esos nodos ya eliminados, `getNode` devuelve nulo y la línea 79 lanza. Si el enlace eliminado llega antes o después depende del orden en que la red original devuelve sus enlaces.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:71-79

```
        for (Link<Node> link : originalDAN.getLinks(originalNode)) {
            if (!link.getFrom().equals(originalNode)) {
                continue;
            }
            Node destinationNode = instantiatedNet.getNode(link.getTo().getVariable());
            // Remove link between restricting originalNode and restricted originalNode
            instantiatedNet.removeLink(link.getFrom().getVariable(),
↪   link.getTo().getVariable(), true);

            if (destinationNode.getNodeType() == NodeType.CHANCE) {
```

**Cómo arreglarlo.** Cuando `destinationNode` es nulo, pasar al enlace siguiente (`continue`): el nodo ya se quitó porque en esta rama no cabe, y no queda nada que revelar ni restringir en él. Conviene hacerlo antes de `removeLink`, que no falla con nodos ausentes pero no tiene sentido.

Revisar a la vez los demás bucles del mismo método que usan nodos de la copia después de haber podido quitarlos: la cascada de borrado (líneas 110-144) y `pruneZeroProbabilityChanceNodes`.

Prueba de regresión: evaluar `0_miscellaneous/networks/dan/DAN-arthronet.pgmx` con `DANDecompositionIntoSymmetricDANsEvaluation` sin excepción, e instanciar «Realizar Implante» = no.

### Error 64 · [Cerrado sin arreglo] Al instanciar una variable de una red de análisis de decisiones se crea un segundo enlace entre dos nodos ya enlazados, y la red resultante incumple su propia restricción de enlaces distintos

| | |
|---|---|
| **Estado** | Cerrado sin arreglo el 08-10-2026. El defecto sigue en el código: en `DAN-delayed-result-of-test.pgmx`, `DAN-king.pgmx` y `DAN-used-car-buyer.pgmx` la red instanciada queda con dos enlaces entre el mismo par de nodos (tres en la última). No se ha encontrado nada que un usuario pueda ver por su causa. Medido comparando el código de `9841b95` con el arreglo descrito abajo (reutilizar el enlace que ya existe): `DANDecompositionIntoSymmetricDANsEvaluation` da la misma utilidad y la misma estrategia óptima en 31 redes de `0_miscellaneous/networks/dan` (todas las que se leen menos `DAN-7-test-problem` y `DAN-CE-8-test-problem`, que se dejaron fuera por su duración), y el árbol de decisión de las tres redes afectadas es idéntico nodo a nodo. Las tres se evalúan hoy enteras. |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.decompositionIntoSymmetricDANs.core |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:79-97`<br>`core/src/main/java/org/openmarkov/core/model/graph/Graph.java:186-205`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNetOperations.java:815-835`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:413-425` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 63 |

**Qué pasa.** Toda evaluación de una red de análisis de decisiones (DAN, del inglés *decision analysis network*) por descomposición (`DANDecompositionIntoSymmetricDANsEvaluation`, que usa la ventana para «Mostrar estrategia óptima» en `InferenceHandler`) y la construcción del árbol de decisión (`DANDecisionTreeInference`, `DecisionTreeExpansion`) pasan por `DANOperations.instantiate`. En algunas redes, la red intermedia que devuelve queda con dos enlaces entre el mismo par de nodos e incumple una restricción de su propio tipo.

Medido instanciando cada variable en cada estado en las 34 redes que se pueden leer de `0_miscellaneous/networks/dan`:

```
DAN-delayed-result-of-test.pgmx -> Therapy today=no:  Do test? -> Result of test (2
↪  enlaces)
                                   Do test?=yes:      Therapy today -> Result of test (2
                                   ↪  enlaces)
DAN-king.pgmx                   -> War=no / War=yes:  Marriage -> Wealth (2 enlaces)
DAN-used-car-buyer.pgmx         -> Dec: Second Test=differential: Dec: First Test -> Second
↪  result (2 enlaces)
```

Las tres redes tienen la restricción `DistinctLinks`, que la copia hereda, así que la red intermedia incumple una restricción de su propio tipo.

Qué puede cambiar: quienes leen si un nodo queda revelado toman el primer enlace que encuentran entre los dos nodos. `ProbNet.getLink` (usado en `BasicOperations.getVariablesRevealedTransitivelyByVariable`, que da el orden de eliminación de una DAN) y el bucle de `ProbNetOperations.getObservableVariables` se quedan con el primero, que es el original; el revelador

añadido puede no contar, o contar con las restricciones del otro enlace. En `DAN-delayed-result-of-test.pgmx` instanciando «Therapy today=no», el primer enlace es el original (revelador solo para «yes» y con restricciones) y el añadido revela para «no» y «yes».

No se ha podido medir si esto cambia la utilidad o la estrategia final: dos de las tres redes no se evalúan enteras por otros motivos (`PotentialCannotBeConvertedToATable`), y la tercera (`DAN-used-car-buyer`, utilidad 32,96) no tiene con qué compararse sin modificar el código.

Por qué pasa. Una DAN se evalúa descomponiéndola: `DANOperations.instantiate` fija una variable en uno de sus estados y devuelve una copia de la red simplificada para esa rama. Cuando un enlace sale de la variable instanciada con estados reveladores (el destino pasa a conocerse si la variable toma ese estado) y el destino tiene decisiones predecesoras todavía por tomar, el código no lo marca como observado, sino que traza un enlace revelador desde cada una de esas decisiones hasta el destino (líneas 89-95), con todos los estados de la decisión como reveladores.

`ProbNet.addLink` delega en `Graph.addLink`, que por diseño crea siempre un enlace nuevo (su javadoc dice que impedir duplicados es trabajo de la restricción `DistinctLinks`). Si la decisión predecesora ya era padre directo del destino —lo corriente: la decisión que revela algo suele además influir en ello—, la copia queda con dos enlaces entre el mismo par de nodos: el original (sin esos estados reveladores, y a veces con restricciones) y el nuevo.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:82-95

```
                        List<Node> predecessorDecisions = ProbNetOperations
                                .getPredecessorDecisions(destinationNode, instantiatedNet);
                        // If it has predecessor decisions, do not reveal it yet, but add
                        ↪  revealing links
                        // from every predecessor decision to the originalNode
                        if (predecessorDecisions.isEmpty()) {
                            destinationNode.setAlwaysObserved(true);
                        } else {
                            for (Node predecessorDecision : predecessorDecisions) {
                                Link<Node> revealingArc = instantiatedNet
                                        .addLink(predecessorDecision, destinationNode,
                                        ↪  true);
                                State[] predecessorDecisionStates =
                                ↪  predecessorDecision.getVariable().getStates();
                                for (State predecessorDecisionState :
                                ↪  predecessorDecisionStates)

↪  revealingArc.addRevealingState(predecessorDecisionState);
                                }
                        }
```

**Cómo arreglarlo.** Antes de añadir, buscar con `instantiatedNet.getLink(predecessorDecision, destinationNode, true)`: si ya existe, añadirle los estados reveladores que le falten en lugar de crear otro enlace; si no, crearlo como hoy. Hay que decidir qué hacer si el enlace existente tiene restricciones; lo natural es conservarlas, porque siguen siendo ciertas.

Otros sitios que deben respetar la misma regla:

- los que crean enlaces en redes intermedias de la inferencia: todos los `addLink` de `DANOperations` y de `DecisionTreeExpansion`;
- los que leen suponiendo un único enlace por par: `ProbNet.getLink`, `ProbNetOperations.getObservableVariables`, `BasicOperations.getVariablesRevealedTransitivelyByVariable`.

Prueba de regresión: instanciar «Therapy today=no» en `0_miscellaneous/networks/dan/DAN-delayed-result-of-test.pgmx` y comprobar que hay un solo enlace «Do test?» → «Result of test», que revela para los dos estados, y que la copia cumple `checkConstraints()`.

### Error 65 · [Corregido] Pedir la estrategia óptima de un diagrama de influencia o una red de análisis de decisiones sin decisiones lanza una excepción inesperada

| | |
|---|---|
| **Estado** | Corregido en el commit `65498cd` (08-10-2026): el botón se enciende, en los dos modos, solo si la red admite decisiones y tiene al menos un nodo de decisión (`MainPanelMenuAssistant.canShowOptimalStrategy`), y se recalcula tras cada edición, deshacer y rehacer. La regla tiene prueba; el recálculo no, porque necesita la ventana principal. Manuel Arias probó el botón en la ventana el 08-10-2026 y funciona correctamente. El cálculo (`VEOptimal Intervention`, `InferenceHandler.showOptimalStrategy`) no se ha tocado: con el botón apagado no se llega a él desde la ventana. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:386-391,406-411`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:222-258`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEOptimalIntervention.java:64-90`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/OptimalStrategyDialog.java:31-37`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/toolbar/StandardToolBar.java:260-263` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 66, error 92, error 223 |

**Qué pasa.** El botón de la barra de herramientas «Show optimal strategy» (`DECISION_SHOW_OPTIMAL_STRATEGY`) está activo en los diagramas de influencia, las redes de análisis de decisiones, los MID (diagramas de influencia de Markov) y demás tipos con decisiones aunque la red no tenga ninguna decisión, incluido un diagrama de influencia recién creado y vacío. Al pulsarlo, el gestor global muestra el diálogo de «excepción inesperada» pidiendo que se envíe a los desarrolladores; el programa no se cierra. En las redes del repositorio no hay diagramas de influencia ni redes de análisis de decisiones sin decisiones, pero cualquier modelo a medio construir lo es.

Medido con una red de prueba (un nodo de azar «Disease» con dos estados y un nodo de utilidad «U» hijo suyo, escrita como diagrama de influencia y como red de análisis de decisiones): el diagrama de influencia lanza `NullPointerException: Cannot load from object array because the return value of "StrategicTablePotential.getStrategyTrees()" is null`; un diagrama de influencia vacío construido en código lanza la misma; la red de análisis de decisiones lanza `ClassCastException: TablePotential cannot be cast to StrategyCarrier`. Las dos son excepciones de ejecución.

Por qué pasa. El botón se activa en `MainPanelMenuAssistant`: en modo edición con `!currentProbNet.hasConstraintOfClass(OnlyChanceNodes.class)` y en modo inferencia con la misma condición. Esos tipos de red no llevan esa restricción, así que el botón queda activo sin mirar si hay decisiones.

Al pulsarlo, `InferenceHandler.showOptimalStrategy` sigue uno de dos caminos:

- Red de análisis de decisiones: `DANDecompositionIntoSymmetricDANsEvaluation(...).getUtility()` se convierte a `StrategyCarrier` y se toma `getStrategyTrees()[0]` (línea 218). Sin decisiones la utilidad es un `TablePotential` normal y la conversión lanza `ClassCastException`.
- Resto (diagramas de influencia, MID…): `new OptimalStrategyDialog(..., new VEOptimalIntervention(...))`, cuyo constructor llama a `getOptimalIntervention()`. Si la utilidad no trae árboles de estrategia, el método intenta construir uno a partir de `getAnAdmissibleOrderOfDecisions`, pero ese relleno está dentro de `if (!decisions.isEmpty())`; sin decisiones no hace nada y la última línea, `strategic.getStrategyTrees()[0]`, lanza `NullPointerException`.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/-tasks/VEOptimalIntervention.java:74-89

```
StrategyTree[] strategyTrees = strategic.getStrategyTrees();
// Create strategy tree if it is empty
if ((strategyTrees==null)||(strategyTrees.length==0)) {
    List<Variable> decisions = BasicOperations.getAnAdmissibleOrderOfDecisions(probNet);
    if (!decisions.isEmpty()) {
        Variable lastVar = decisions.getLast();
        StrategyTree auxST = new StrategyTree(lastVar,lastVar.getStates());
        ...
        strategic.strategyTrees = new StrategyTree[1];
        strategic.strategyTrees[0] = auxST;
    }
}
return strategic.getStrategyTrees()[0];
```

gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:386-391

```
boolean isOnlyChanceNodes = currentProbNet.hasConstraintOfClass(OnlyChanceNodes.class);
setOptionEnabled(ActionCommands.DECISION_CREATION, !isOnlyChanceNodes);
setOptionEnabled(ActionCommands.UTILITY_CREATION, !isOnlyChanceNodes);
...
setOptionEnabled(ActionCommands.DECISION_SHOW_OPTIMAL_STRATEGY, !isOnlyChanceNodes);
```

**Cómo arreglarlo.** Hay que fijar el comportamiento en dos sitios coherentes.

En la interfaz: activar el botón solo si la red tiene al menos un nodo de decisión, en los dos modos (`MainPanelMenuAssistant`, ramas `EDITION` e `INFERENCE`), y comprobar que se recalcula cuando se añade o se quita la última decisión.

En el cálculo, para que un llamador que no pase por el botón no reciba una excepción de ejecución: `VEOptimalIntervention.getOptimalIntervention` debería lanzar una excepción declarada con mensaje claro («la red no tiene decisiones») o devolver un resultado vacío documentado; y `InferenceHandler.showOptimalStrategy` debería comprobar el tipo antes de convertir a `StrategyCarrier` en la rama de las redes de análisis de decisiones.

El botón del árbol de decisión tiene una regla de activación parecida (error 66).

Prueba de regresión: sobre las redes de prueba sin decisiones (diagrama de influencia y red de análisis de decisiones), comprobar que el botón queda desactivado tras cargar la red en los dos modos, y que `VEOptimalIntervention.getOptimalIntervention` falla con la excepción declarada y no con `NullPointerException`.

### Error 66 · [Corregido] El botón del árbol de decisión se enciende en modo inferencia para redes que no admiten árbol (MID, POMDP, LIMID…), y al pulsarlo sale un error

| | |
|---|---|
| **Estado** | Corregido en el commit `e640410` (08-10-2026), que solo unifica la regla: los dos modos usan `MainPanelMenuAssistant.canShowDecisionTree` (la red admite decisiones y es un diagrama de influencia o una red de análisis de decisiones). **El fallo que describe esta ficha no se alcanzaba desde la ventana**: el modo inferencia solo se ofrece a diagramas de influencia y redes bayesianas (error 201), así que con un MID, un POMDP o un LIMID el usuario no llegaba a ver el botón encendido. Comprobado por Manuel Arias en la ventana el 08-10-2026, después del commit: con un MID no se puede entrar en modo inferencia. En los dos tipos que sí entran, el botón queda como estaba. La regla no se ha llevado a `DecisionTreeManager`. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:378-411`<br>`inference/src/main/java/org/openmarkov/inference/decisiontree/operation/DecisionTreeManagerImpl.java:55-74`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/toolbar/StandardToolBar.java:275-287` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 65 |

**Qué pasa.** Un MID (diagrama de influencia de Markov), un POMDP (proceso de decisión de Markov parcialmente observable), un LIMID (diagrama de influencia con memoria limitada) o una red de sucesos discretos tienen el botón «Decision tree» apagado mientras se editan y encendido mientras se hace inferencia. Al pulsarlo en inferencia, el usuario recibe un error que dice que la opción no vale para esa red.

Medido sobre `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`: `DecisionTreeManagerImpl.buildDecisionTree` rechaza la red con `NotEvaluableNetworkException.NotApplicableNetwork` y el mensaje «This algorithm cannot evaluate Markov Influence Diagram network named MID-Chancellor-corrected.pgmx, as it can only evaluate networks of type: [Influence diagram, Decision analysis]». En la aplicación esa excepción comprobada llega envuelta en `UnrecoverableException` al manejador global, que la enseña en la ventana de error.

Por qué pasa. El árbol de decisión solo se ofrece desde el botón «Decision tree» de la barra de herramientas estándar (`StandardToolBar.getDecisionTreeButton`). `MainPanelMenuAssistant` decide si ese botón está activo, y lo decide con dos reglas distintas según el modo de trabajo:

- En modo edición (línea 379): activo si la red no tiene la restricción `OnlyChanceNodes` y es un diagrama de influencia (`InfluenceDiagramType`) o una red de análisis de decisiones (`DecisionAnalysisNetworkType`).
- En modo inferencia (líneas 395-397): activo si la red no tiene la restricción `OnlyChanceNodes`, sin mirar el tipo.

Ninguna de las dos reglas mira si hay decisiones, pero para el árbol eso no produce fallo: con diagramas de influencia y redes de análisis de decisiones sin ninguna decisión (`integrationTests/src/main/resources/networks/id/ID-only-utility.pgmx`, `integrationTests/src/main/resources/networks/id/ID-one-chance.pgmx`, `integrationTests/src/main/resources/`

`networks/dan/DAN-one-chance.pgmx`) el árbol se construye (un árbol con solo nodos de azar).

**El código.** gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:390-393, 404-410

```
setOptionEnabled(ActionCommands.DECISION_SHOW_OPTIMAL_STRATEGY, !isOnlyChanceNodes);
setOptionEnabled(ActionCommands.DECISION_TREE, !isOnlyChanceNodes &&
↪  ((currentProbNet.getNetworkType() instanceof InfluenceDiagramType)
        || (currentProbNet.getNetworkType() instanceof DecisionAnalysisNetworkType)));
...
case INFERENCE -> {
    ...
    if (!currentNetworkEditorPanel.getProbNet().hasConstraintOfClass(OnlyChanceNodes.class
    ↪  )) {
        setOptionEnabled(ActionCommands.DECISION_TREE, true);
        setOptionEnabled(ActionCommands.DECISION_SHOW_OPTIMAL_STRATEGY, true);
    }
}
```

**Cómo arreglarlo.** Usar una sola regla para los dos modos, la misma que aplica quien construye el árbol: tipo de red diagrama de influencia o red de análisis de decisiones (y sin `OnlyChanceNodes`). Lo más robusto es sacar esa condición a un único método (por ejemplo en `DecisionTreeManager`, «¿admite esta red un árbol?») y usarlo en `MainPanelMenuAssistant` en las dos ramas y en `DecisionTreeManagerImpl.buildDecisionTree`, para que no vuelvan a divergir.

El botón de estrategia óptima de las mismas líneas tiene su propio problema (error 65), que no se arregla con esto.

Prueba: con una red MID abierta en modo inferencia, comprobar que la opción `DECISION_TREE` queda desactivada; con un diagrama de influencia, activada en los dos modos.

### Error 200 · [Corregido] Al instanciar una variable de una red de análisis de decisiones, un nodo de utilidad de producto puede borrarse y dejar suelto uno de sus factores, que la rama suma como si fuera una utilidad

| | |
|---|---|
| **Estado** | Corregido en el commit `c38679e` (14-10-2026), con el primero de los dos caminos de «Cómo arreglarlo»: un nodo que la rama quita vale cero en los nodos que combinan utilidades. El borrado ya no se lleva por delante una suma: le quita ese término y conserva los demás. Un producto que pierde un factor vale cero, así que se quita, y con él los factores que solo lo alimentaban. Los dos bucles de borrado de `DANOperations` (el de las restricciones y el de las variables de probabilidad cero) comparten ahora ese código. Medido: con «Realizar Implante = no», `DAN-arthronet.pgmx` vale 0 y conserva «Beneficio neto», «EVAC Total», «Coste total», «Coste ajustado» y «C2E»; aquí el producto no pierde ningún factor, porque «Coste total» sigue siendo una suma. El caso del producto se prueba con una red pequeña hecha en la prueba |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.decompositionIntoSymmetricDANs.core |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:111-142` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 63, error 64 |

**Qué pasa.** Visto al corregir el error 63. En `0_miscellaneous/networks/dan/DAN-arthronet.pgmx`, la rama en la que no se hace el implante vale −0,0000333 y debería valer 0: no hay implante, ni pruebas, ni tratamiento, y por tanto ni coste ni efectividad. El diagrama de influencia con las mismas variables, `ID-arthronet.pgmx`, da 0 con «Realizar Implante = no».

Lo ve el usuario al pedir la estrategia óptima o el árbol de decisión de esa red, que desde el commit `2ec95a5` se evalúa: la utilidad de la rama «no» entra en la comparación con la de «si» desplazada en esa cantidad. En esta red solo puede cambiar la decisión en un empate casi exacto.

Medido instanciando «Realizar Implante = no» con `DANOperations.instantiate` y evaluando lo que queda con `DANDecompositionIntoSymmetricDANsEvaluation`:

```
nodos que quedan: IMC, Diabetes, Alergia ATB, EVAC Implante, Coste Implante, C2E
utilidad: -3.33E-5
```

Por qué pasa. La utilidad de la red es «Beneficio neto», que suma «EVAC Total» y «Coste ajustado»; «Coste ajustado» es un nodo de producto que multiplica «Coste total» por «C2E», un factor constante de valor $-3{,}33 \cdot 10^{-5}$ que pasa el coste a unidades de efectividad. Al instanciar «no», `instantiate` quita los nodos que esa opción hace imposibles y todos sus descendientes (líneas 111-142), entre ellos «Coste total», «Coste ajustado», «EVAC Total» y «Beneficio neto». «C2E», «EVAC Implante» y «Coste Implante» no descienden de ninguno y se quedan. Sin los nodos que los combinaban, la evaluación los trata como utilidades sueltas y las suma: «EVAC Implante» y «Coste Implante» valen 0 con «no», y queda el factor.

El borrado deshace la estructura de los nodos que combinan utilidades. Aquí lo que sobra es un factor; con otra red podría quedar un coste sin pasar por su factor, sumado a la efectividad en otra unidad. No se ha medido en ninguna otra red.

**Cómo arreglarlo.** Hecho con el primero de los dos caminos, como dice la fila *Estado*. Hay que decidir qué vale un nodo que combina utilidades (suma o producto) cuando la rama borra alguno de sus padres, y aplicarlo en la cascada de borrado. Dos opciones:

- conservar el nodo que combina y dar valor cero a los padres que la rama borra, como ya se hace con la última utilidad de la red (`replaceUtilityPotentialWithZero`); en un producto, eso deja el producto en cero;
- al borrar un nodo que combina, borrar también los padres de utilidad que no tengan otro hijo.

Prueba de regresión: instanciar «Realizar Implante = no» en `DAN-arthronet.pgmx` y evaluar lo que queda debe dar 0.

### Error 210 · El gesto que abre la relación de un nodo (Alt+clic) da un error inesperado sobre una decisión sin política impuesta, que no tiene relación que enseñar

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node; org.openmarkov.gui.window.edition.networkEditorPanel |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:157-160,228-233,476-484,621-633,692-703`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EditorCommandHandlerGenerator.java:437-452`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:450-457` |

| | |
|---|---|
| | gui/src/main/java/org/openmarkov/gui/bindings/Bindings.java:227-237<br>gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:653-663<br>core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:149-157<br>gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditDialog.java:83-85,95-102<br>gui/src/main/java/org/openmarkov/gui/dialog/node/ImposePolicyDialog.java:16-26<br>core/src/main/java/org/openmarkov/core/action/base/PNESupport.java:142-154,197-201 |
| **Comprobado** | Leyendo el código y ejecutándolo (llamando al método que abre el diálogo; el gesto no se ha hecho en la ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 51, error 28 |

**Qué pasa.** El gesto «Edit relation» (por omisión Alt+clic sobre un nodo; se puede reasignar) abre el diálogo de la relación del nodo. Sobre una decisión a la que no se ha impuesto ninguna política, que es como está casi toda decisión, termina en un `NullPointerException` y sale el diálogo de error inesperado.

Los menús no tienen el problema: para una decisión sin política no encienden «Edit probability» sino «Impose policy» (`MainPanelMenuAssistant`, líneas 653-663). El gesto, en cambio, solo comprueba que haya un nodo bajo el ratón (`EditorCommandHandlerGenerator`, líneas 437-439) y llama a `NetworkEditorPanel.showPotentialDialog`.

Medido el 8-10-2026 con el guion de las capturas automáticas, que llama a ese mismo `showPotentialDialog` con la decisión `Decision_TBNA` de `ID-mediastinet-1-0.pgmx` señalada:

```
NullPointerException: Cannot invoke "Object.getClass()" because the return value of
↪  "org.openmarkov.core.model.network.Node.getPotential()" is null
    at PotentialEditPanel.getPotentialTypeJCombobox(PotentialEditPanel.java:233)
    at PotentialEditPanel.getPotentialTypePanel(PotentialEditPanel.java:290)
    at PotentialEditPanel.configureComponentsPanel(PotentialEditPanel.java:183)
    at PotentialEditPanel.<init>(PotentialEditPanel.java:169)
```

Por qué pasa: `getPotentialTypeJCombobox` da por hecho que, si el nodo no tiene relación, se le puede poner la relación por omisión de su tipo (líneas 230-232). Para una decisión, `PotentialUtils.generateDefaultPotential` devuelve nulo (línea 152), y la línea 233 usa ese nulo. Medido aparte: para la decisión `Therapy` de `ID-decide-test.pgmx`, `getPotential()` y `generateDefaultPotential` devuelven nulo.

El gesto en sí no se ha hecho en la ventana; que llega a ese método se ha establecido leyendo.

Cuando la red se está editando, el daño no acaba en la ventana de error. El panel abre una sub-historia de ediciones (`PotentialEditPanel`, línea 159: `openNewSubEditHistory`) antes de llegar a la línea que falla, y como el panel no llega a construirse, nadie la cierra. Desde ese momento «deshacer» trabaja sobre esa sub-historia vacía, y lo hecho antes en la red ya no se puede deshacer. En modo inferencia el panel es de solo lectura y no abre la sub-historia. Medido el 15-10-2026 sin ventana sobre `0_miscellaneous/networks/id/ID-decide-test.pgmx`, construyendo solo el panel, como hace `PotentialEditDialog.initialize`, después de cambiar el dominio de «Disease» a tres estados:

```
after an edit: Disease has 3 states, can undo true
```

```
panel THROWS NullPointerException at org.openmarkov.gui.dialog.node.PotentialEditPanel.get⌋
↪  PotentialTypeJCombobox(PotentialEditPanel.java:233)
after the throw: can undo false, main history is the current one false
undo: 0 edits undone, Disease has 3 states
```

Con una política impuesta el gesto tampoco hace lo que el menú, aunque ya no falle al editar. En modo edición, el menú del nodo ofrece para esa decisión «Edit policy», que abre `ImposePolicyDialog`; el gesto abre siempre `PotentialEditDialog` (`NetworkEditorPanel.requestPotentialValues`, líneas 483-495), con el título «Edit probability: Therapy» en vez de «Edit policy: Therapy», y lo que se acepte se guarda con `SetPotentialEdit` a través de `setPotentialWithRestrictions` en vez de con `ImposePolicyEdit` (`PotentialEditPanel`, líneas 692-703; leído, sin haber encontrado una diferencia que se vea en la red). La lista de clases de relación es la misma en las dos ventanas. En modo inferencia el menú no ofrece nada para una decisión con política, y el gesto abre la ventana de solo lectura: `PotentialEditDialog.requestValues` (líneas 95-102) llama antes de enseñarla a `setEnabledDecisionOptions`, porque la decisión tiene la política «óptima» —`ImposePolicyPanel.initializePotential` la fija así al imponerla—, y ese método termina en el `NullPointerException` del error 51, porque `getPoliticyTypePanel` devuelve siempre `null`. Medido el 12-10-2026 con el commit `175a3ad`, construyendo la ventana igual que el gesto sin enseñarla, sobre «Therapy» de `ID-decide-test.pgmx` con una política impuesta: la ventana se construye con el título «Edit probability: Therapy», y el paso de `setEnabledDecisionOptions`, llamado suelto, lanza `NullPointerException`. El 15-10-2026 se ha vuelto a leer el código: lo que interviene no ha cambiado desde entonces, y sin pantalla no se puede repetir la medida, porque el panel construye un editor de comentarios que es un `JDialog`.

**Cómo arreglarlo.** Que el gesto siga la misma regla que el menú, en todos los casos de una decisión: en modo edición, sin política, no hacer nada o abrir «Impose policy»; con política, abrir «Edit policy» (`ImposePolicyDialog`); en modo inferencia, no abrir la ventana de la relación, que es lo que hace el menú para una decisión con política (y para una sin política ofrece «Show optimal policy» y «Show expected utility», que tienen su propio fallo en el error 51). La regla de qué se puede abrir para cada nodo está hoy repetida en `MainPanelMenuAssistant.objectsSelected` y en `NodeContextualMenu`; conviene que el gesto pregunte al mismo sitio.

Aparte de la regla del gesto, el panel no debería dejar la historia de ediciones a medias si falla al construirse: o abre la sub-historia cuando ya ha construido todo, o la cancela (`cancelLastSubEditHistory`) si algo falla antes.

Prueba: con la regla extraída a un método, comprobar que para una decisión sin política la acción de editar la relación no está permitida, que para una con política en modo edición es la de «Edit policy», y que en modo inferencia no se permite. Y, tras una edición cualquiera, construir el panel sobre una decisión sin política y comprobar que, falle o no, `getCanUndo()` sigue siendo verdadero y deshacer devuelve lo anterior.

### Error 222 · [Corregido] En una red de análisis de decisiones, un hallazgo previo sobre una decisión no cuenta: la utilidad y la estrategia óptima salen como si no se supiera, y la estrategia puede contradecirlo

| | |
|---|---|
| **Estado** | Corregido en el commit `2c1c801` (09-10-2026): en `DANDecompositionIntoSymmetricDANsInference`, una decisión con hallazgo se fija en esa opción (`DANOperations.instantiate`) cuando le llega el turno, es decir, cuando está entre las que se pueden tomar. Fijarla al empezar daba, con `NClub? = yes`, una utilidad de 10,48, mayor que la de elegir libremente (9,41), porque esa decisión solo se toma en algunos escenarios. Medido con `DAN-dating.pgmx`: `Ask? = yes` da 1,1782, la utilidad de la red fijada en esa opción, y la estrategia empieza en `Accept`; `Ask? = no` da 9,4076; `NClub? = yes` da 9,3030 y `NClub? =` |

| | |
|---|---|
| | `no` 2,5015. Con prueba. Visto en la aplicación real, arrancada desde un guion, con el hallazgo puesto donde lo guarda «Add finding». Para `NClub?` no hay un valor independiente con que comparar; solo se ha medido esta red; el árbol de decisión no se ha tocado. |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.decompositionIntoSymmetricDANs.core |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANDecompositionIntoSymmetricDANsInference.java:54-68`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:328-355`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/DecisionTreeExpansion.java:276`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:229-237` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 53, error 54, error 223, error 224, error 201 |

**Qué pasa.** En una red de análisis de decisiones (DAN, del inglés *decision analysis network*), «Show optimal strategy» evalúa la red descomponiéndola en redes simétricas (`DANDecomposition IntoSymmetricDANsEvaluation`) y le pasa los hallazgos que el usuario haya puesto en modo edición, que son la evidencia previa a la resolución. Si uno de esos hallazgos es sobre una decisión, es decir, se sabe qué opción se eligió, no cuenta: la utilidad y la estrategia son las mismas que sin él.

Medido con un pequeño programa de prueba que hace lo que hace la ventana, con `0_miscellaneous/networks/dan/DAN-dating.pgmx`:

```
sabido de antemano   utilidad    lo que la estrategia dice de Ask?
nada                 9.407584    no
Ask? = yes           9.407584    no
Ask? = no            9.407584    no
NClub? = yes         9.407584    (la misma estrategia que sin saber nada)
```

La segunda línea es la que más se nota: la ventana dice «Ask? → no» a quien acaba de decirle que la respuesta fue «yes». En un diagrama de influencia no pasa. Medido igual con `0_miscellaneous/networks/id/ID-decide-test.pgmx`: con `Do test? = no` sabido, la estrategia ya no pregunta por esa decisión y empieza en `Therapy`.

Por qué pasa: cuando la descomposición busca qué decisiones se pueden tomar primero llama a `DANOperations.getNextDecisions(dan)` (línea 56), sin la evidencia. Existe la otra versión, `getNextDecisions(probNet, evidence)` (líneas 345-355), que quita de las candidatas las decisiones sobre las que hay un hallazgo, y la usa el árbol de decisión (`DecisionTreeExpansion`, línea 276). Aquí la decisión se abre en todas sus opciones y se maximiza sobre ellas.

En la ventana, una red de este tipo no entra en modo inferencia (error 201), así que estos hallazgos solo pueden ser los del modo edición.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANDecompositionIntoSymmetricDANsInference.java:54-58

```
} else {// If O_A is empty, then some variable introduces asymmetries
    Variable rootDecision;
    List<Node> nextDecisions = DANOperations.getNextDecisions(dan);
    if (nextDecisions.size() == 1) { // If exactly one decision D can be made first
        rootDecision = nextDecisions.get(0).getVariable();
```

**Cómo arreglarlo.** Pasar la evidencia a `getNextDecisions`, y tratar la decisión con hallazgo como lo que es, una variable de valor conocido: instanciarla en esa opción en vez de abrirla. Hay que mirar junto con esto los otros dos sitios en que la descomposición no mira la evidencia (errores 223 y 224).

Prueba de regresión: con `DAN-dating.pgmx` y `Ask? = yes` sabido, la utilidad debe ser la de la rama «yes» (la que da el árbol de decisión para esa rama) y la estrategia no debe proponer otra opción para `Ask?`.

### Error 223 · [Corregido] «Show optimal strategy» de una red de análisis de decisiones lanza ClassCastException cuando, con lo que se sabe de antemano, no queda nada que decidir

| | |
|---|---|
| **Estado** | Corregido en el commit `4e316fd` (09-10-2026): `DANEvaluation.getOptimalStrategyTree`, que solo lanzaba «not implemented», devuelve la estrategia o nada, y la ventana lo usa; sin estrategia enseña «With what is already known, there is nothing left to decide.». Con prueba del cálculo. Visto en la aplicación real, arrancada desde un guion, con `DAN-one-decision.pgmx`: sin nada sabido se abre la estrategia; con `D = yes` sabido, antes salía «An unexpected error has occurred» y ahora el aviso. El hallazgo se puso desde el guion donde lo guarda «Add finding», no con el diálogo. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:229-237`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEOptimalIntervention.java:64-90` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 65, error 222, error 224 |

**Qué pasa.** Con una red de análisis de decisiones (DAN) y un hallazgo previo que deja la red sin nada que decidir, «Show optimal strategy» no abre la ventana: lanza un `ClassCastException`, que sale como error inesperado.

Medido con un pequeño programa de prueba que hace lo que hace la ventana, con `integrationTests/src/main/resources/networks/dan/DAN-one-decision.pgmx`, que tiene una sola decisión, `D`: sin hallazgos, la estrategia sale («D → no»); con `D = yes` o con `D = no` sabido de antemano, `ClassCastException`.

Por qué pasa: la rama de las DAN de `InferenceHandler.showOptimalStrategy` convierte la utilidad a `StrategyCarrier` (un potencial que lleva la estrategia al lado de cada número) y toma su primer árbol de estrategia (línea 233), sin comprobar nada. Cuando no se ha maximizado sobre ninguna decisión, la utilidad que devuelve la evaluación es un potencial corriente, sin estrategia. La otra rama, la de las redes que no son DAN, sí contempla el caso: `VEOptimalIntervention` envuelve la utilidad en un potencial con estrategia y, si no trae ninguna, la construye.

Es la conversión que el error 65 describía para una red sin decisiones. Aquel se corrigió apagando el botón cuando la red no tiene decisiones, sin tocar el cálculo; esta red tiene una, el botón está encendido y el fallo se alcanza.

**El código.** gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:229-234

```
if (networkPanel.getProbNet().getNetworkType().equals(DecisionAnalysisNetworkType.
↪  getUniqueInstance())) {
    DANEvaluation eval = new DANDecompositionIntoSymmetricDANsEvaluation(probNet,
↪  ...getPreResolutionEvidence());
    StrategyTree strategyTree = ((StrategyCarrier)
↪  eval.getUtility()).getStrategyTrees()[0];
    strategyTree.pruneAndGraftNode("OD");
```

**Cómo arreglarlo.** Comprobar el tipo antes de convertir y, si la utilidad no trae estrategia, abrir la ventana diciendo que no queda nada que decidir. Lo mismo si `getStrategyTrees()` viene vacío.

Prueba de regresión: con `DAN-one-decision.pgmx` y `D = yes` sabido, el cálculo que alimenta la ventana no lanza ninguna excepción.

### Error 224 · [Corregido] En una red de análisis de decisiones, un hallazgo previo sobre una variable que decide qué puede pasar después hace fallar la evaluación con «evidencia incompatible», sea cual sea el estado visto

| | |
|---|---|
| **Estado** | Corregido en el commit `9b379b1` (17-10-2026): en `DANDecompositionIntoSymmetricDANsInference`, una variable con hallazgo que restringe a otras se fija en ese estado (`DANOperations.instantiate`) cuando le llega el turno, como ya se hacía con una decisión con hallazgo (error 222). El hallazgo cuenta solo donde la variable existe, por decisión de Manuel Arias: en `DAN-dating.pgmx`, `Accept = yes` y `Accept = no` dan 9,4076, lo mismo que sin hallazgo, porque `Accept` solo existe si se pregunta y lo mejor sigue siendo no preguntar; leerlo como que se preguntó daría 3,1006 con `Accept = yes`. Medido en todas las redes de `0_miscellaneous/networks/dan`: fallaban `DAN-dating`, `DAN-reactor` y `DAN-decide-test-with-restrictive-symptom`, en los dos formatos, con cualquier estado de cualquier variable de azar que restringe a otras (18 casos); ahora se evalúan y dan la misma utilidad que el árbol de decisión con el mismo hallazgo. Con prueba. |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.decompositionIntoSymmetricDANs.core |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANDecompositionIntoSymmetricDANsInference.java:34-53`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANOperations.java:482,548-553,614`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:229-237` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 222, error 223, error 186, error 48 |

**Qué pasa.** En `0_miscellaneous/networks/dan/DAN-dating.pgmx`, la variable `Accept` (si la otra persona acepta la invitación) se conoce al preguntar y decide qué se puede hacer después. Si el usuario la pone como hallazgo previo, con cualquiera de sus dos estados, la red no se puede evaluar: «Show optimal strategy» termina en `EvidenceIsIncompatibleWithOther`, el aviso de evidencia incompatible, aunque el hallazgo no es incompatible con nada.

Medido con un pequeño programa de prueba que hace lo que hace la ventana: con `Accept = yes` sabido de antemano y con `Accept = no`, la evaluación lanza esa excepción; sin hallazgos, o con hallazgos sobre otras variables, termina.

Por qué pasa: la descomposición trata aparte las variables que se ven desde el principio y además restringen a otras (las que introducen asimetrías). La lista de las que se ven desde el principio se calcula quitando las que ya tienen hallazgo (`getVariablesObservedFromTheBegginning`), pero la de las que restringen se pide sin mirar la evidencia (`DANOperations.getAsymmetricObservableVariables(dan)`, línea 36). Si una de ellas tiene hallazgo, la descomposición la abre igualmente en todos sus estados y, para cada uno, añade «vale este estado» a lo que se sabe (`extendEvidenceCase`, línea 51). En cuanto llega a un estado distinto del que se sabía, `EvidenceCase.addFinding` rechaza el segundo hallazgo sobre la misma variable y la evaluación entera se acaba ahí.

**Cómo arreglarlo.** Quitar de las variables que restringen las que ya tienen hallazgo, e instanciar cada una de esas en el estado que se sabe en vez de abrirla en todos. Conviene hacerlo a la vez que el error 222, que es el mismo descuido con las decisiones.

Prueba de regresión: con `DAN-dating.pgmx`, `Accept = yes` y `Accept = no` sabidos de antemano deben evaluarse sin excepción, y dar la utilidad de la rama correspondiente del árbol de decisión.

### Error 290 · Al evaluar una red de análisis de decisiones con un hallazgo previo cuya probabilidad depende de una decisión, la descomposición y el árbol toman la probabilidad de la primera opción: «Show optimal strategy» da otra utilidad y, a veces, otra estrategia, y el árbol puede dar utilidad 0

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.decompositionIntoSymmetricDANs.core |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANInference.java:133-138`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANDecompositionIntoSymmetricDANsInference.java:78`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/decompositionIntoSymmetricDANs/core/DANDecisionTreeInference.java:142`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/ChanceVariableElimination.java:58-63`<br>`inference/src/main/java/org/openmarkov/inference/decisiontree/operation/DecisionTreeManagerImpl.java:52` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | Sí: cómo se comparan, al eliminar una decisión, un escenario en el que el hallazgo no cuenta porque su variable no existe (regla del error 224) y otro en el que cuenta con una probabilidad menor que uno |
| **Relacionados** | error 54, error 222, error 224, error 194 |

**Qué pasa.** Al eliminar una decisión que la descomposición o el árbol abren rama a rama, `DANInference.conditionMaximizeAndSetProbabilityAndUtility` toma como probabilidad la de la primera rama («as it should be equal») y se la pasa a `DecisionVariableElimination` sin la decisión. Sin evidencia es cierto; con un hallazgo previo que depende de la decisión, no. Es lo que el error 54 corrigió en `DecisionVariableElimination`, que aquí no llega a actuar porque la probabilidad que recibe ya no depende de la decisión.

Con `0_miscellaneous/networks/dan/DAN-king.pgmx` y `Wedding = yes`: `Wedding` copia la decisión `Marriage`, y la primera opción, `no`, hace imposible el hallazgo. El árbol (`DANDecisionTreeEvaluation`) se queda con probabilidad 0; al eliminar después una variable de azar, `ChanceVariableElimination` divide 0 entre 0, pone a 0 lo que no es un número, y la utilidad sale 0. Lo

bueno es 7,73: sin hallazgos la estrategia óptima ya casa a la princesa, y con `Marriage = yes` las dos vías dan 7,73. La estrategia óptima da 7,73 porque elimina `Marriage` con la eliminación de variables. Igual con `Offspring = yes`: 8,85 con probabilidad 0,6 en la estrategia, 0 en el árbol.

El árbol con hallazgos no lo alcanza ningún usuario: la ventana lo construye sin evidencia (`DecisionTreeManagerImpl`, línea 52). La descomposición sí, y con ella «Show optimal strategy» y el análisis de coste-efectividad. Medido el 17-10-2026 en las DAN de `0_miscellaneous/networks/dan/1-0`, comparando con la misma descomposición cuando se le pasa la probabilidad de cada opción, que es la regla del error 54 (con ella la descomposición y el árbol coinciden en los 19 casos probados):

```
DAN-king                          Result 2 = pass        hoy 8,0226; regla del 54 7,9831;
↪  cambia la estrategia (con Noble descent = yes, Task 3 primero en vez de Task 1)
DAN-king                          Result 3 = pass        hoy 7,7760; regla del 54 7,7940;
↪  cambia la estrategia
DAN-king                          Result 1 = fail        utilidad 7,45 en los dos;
↪  probabilidad 0,86 (la de «kill dragon») en vez de 0,5 (la de «kill unicorn», la
↪  elegida)
DAN-diabetes                      Blood test result = negative   hoy 987,8125; regla del 54
↪  991,3459
DAN-decide-test-with-symptom      Result of test = negative      hoy 9,5961; regla del 54
↪  9,7061
DAN-dating                        Club = negative        hoy 7,3072; regla del 54 8,2421
DAN-dating                        TVExp = positive       hoy 10 (P 0,15); regla del 54
↪  9,4076 (P 1)
```

La fila *Estado* del error 54 dice que en DAN-decide-test-with-symptom no se obtuvo un resultado equivocado; con el hallazgo de la tabla, sí cambia la utilidad. La prueba del error 224, que compara la descomposición con el árbol, no ve este error, porque las dos vías pasan por el mismo método.

Con DAN-dating la regla del 54 no basta: `TVExp`, `Club` y `MeetFr` solo existen en algunos escenarios, y el hallazgo pesa con su probabilidad en unos y no cuenta en otros. En `DAN-arthronet` las dos vías tampoco coinciden (`Cortes Congelados`, `Ga67 Tc99`); allí no se ha medido la regla del 54. Aparte, el árbol no fija una decisión con hallazgo como lo hace la descomposición desde el error 222: con `Marriage = no` da 7,73, y la estrategia 6,05; ningún usuario llega al árbol con hallazgos.

**Cómo arreglarlo.** Al eliminar una decisión, juntar las probabilidades de las ramas sobre la decisión, como ya hace `conditionEliminateChanceAndSetProbabilityAndUtility` con las variables de azar, y pasarlas a `DecisionVariableElimination`, que deja fuera las opciones imposibles y se queda con la probabilidad de la elegida. Antes hay que decidir qué peso tiene un escenario en el que el hallazgo no cuenta. El mismo método sirve a la descomposición y al árbol: con un solo arreglo se corrigen los dos.

Prueba de regresión: con `DAN-king.pgmx`, `Wedding = yes` debe dar 7,73 y `Offspring = yes` 8,85 con probabilidad 0,6 en el árbol, y la descomposición y el árbol deben dar la misma utilidad con cualquier hallazgo de un estado sobre las variables de azar de DAN-king.

# 8 Modelos temporales y evolución temporal

Las redes con variables que se repiten ciclo a ciclo (por ejemplo, los diagramas de influencia de Markov) y la ventana de evolución temporal.

### Error 67 · [Corregido] Con evidencia, las curvas de la evolución temporal no suman uno: se muestran probabilidades conjuntas en vez de condicionadas

| | |
|---|---|
| **Estado** | Corregido en el commit `4e20cf9` (09-10-2026): `MIDTemporalEvolution` normaliza por separado cada opción de la decisión, de modo que cada una suma uno. Una opción con la que la evidencia es imposible se queda a cero en el resultado de la tarea, y la ventana no le da valor: la tabla muestra «-» y su curva no se dibuja. Queda sin tocar: la evidencia de un ciclo no cambia las probabilidades de los ciclos anteriores; una evidencia imposible con todas las opciones deja la ventana entera a «-» sin mensaje; y la exportación a Excel con `VETemporalEvolution`, a la que ya no llega ningún usuario |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.temporalevaluation.tasks; org.openmarkov.inference.algorithm.variableElimination.tasks |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:273-283`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:291-295`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:602-619`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:210-213`<br>`core/src/main/java/org/openmarkov/core/inference/InferenceAlgorithm.java:200-204`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:316-335` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No. Pedía decidir si las curvas deben normalizarse por cada opción de la decisión y qué significa una columna que suma cero. Manuel Arias lo decidió el 09-10-2026: se normaliza cada opción, y la opción con la que la evidencia es imposible se muestra sin valor |
| **Relacionados** | error 48, error 52, error 69, error 74 |

**Qué pasa.** En modo inferencia de un modelo de Markov con decisiones (MID, diagrama de influencia de Markov), la evolución temporal de un nodo de azar temporal dibuja, ciclo a ciclo, la probabilidad de cada estado de la variable para cada opción de la decisión elegida. Para llegar al fallo basta con tener hallazgos previos a la resolución (puestos en modo edición) o elegir un escenario en el diálogo de la evolución temporal (`TemporalEvolutionDialog` añade esos hallazgos a la evidencia), seleccionar un nodo de azar temporal y pedir su evolución temporal.

Con evidencia, la gráfica y la tabla muestran la probabilidad conjunta P(V, e) y no la condicionada P(V | e). Son números que no suman uno, presentados como probabilidades y escalados de forma distinta para cada opción de la decisión, lo que distorsiona la comparación entre ellas. Sin evidencia no se nota, porque la tabla ya suma uno por columna.

Al final de `MIDTemporalEvolution.computePosteriorProbability` (líneas 612-615) la normalización está protegida por `if (getConditioningVariables() == null || getConditioningVariables().isEmpty())`, con un «TODO - Realizar la normalización condicionada» encima.

Esa lista no está vacía nunca: `commonPreprocessing` (línea 294) la fija a `Collections.singletonList(conditioningDecision)`, y `InferenceAlgorithm.setConditioningVariables` guarda cualquier lista no nula. Con decisión, es una lista de un elemento; sin decisión (`conditioningDecision` nulo), es una lista de un elemento que es nulo. En los dos casos la condición es falsa y la

normalización no se ejecuta.

La misma lista tiene dos consecuencias más:

- El brazo `CannotNormalizePotentialException` del `catch` de `resolve()` (líneas 278-281, declarado inalcanzable con un «TODO: Maybe this can actually happen») no puede saltar mientras la normalización no se ejecute: con evidencia imposible, las curvas saldrían todas a cero sin aviso. Si se implementa la normalización, ese `catch` convierte la excepción en `UnreachableException`.
- En la línea 609 se construye `Arrays.asList(probabilityVariable, conditioningDecision)` para reordenar; sin decisión, se reordena sobre una lista con un nulo dentro. No se ha visto dispararse en las pruebas hechas.

La misma guarda está copiada en `VEPropagation.InvokeVariableEliminationCore` (líneas 210-213). Allí sí normaliza cuando no hay variables condicionantes (la propagación normal), pero no cuando `VETemporalEvolution` le pasa la decisión, que es lo que hace la exportación a Excel de la evolución temporal.

Medido con `0_miscellaneous/networks/mid/MID-Chancellor.pgmx`, variable «State», decisión «Therapy type» (suma de cada columna, una por opción):

```
sin evidencia:                State [0..4]: 1.000000 1.000000
State [1] = State A:          State [0], [1]: 1.000000 1.000000 ; State [2..4]: 0.721453
↪  0.858220
State [1] = State B:          State [2..4]: 0.201845 0.102739
State [1] = State C:          State [2..4]: 0.066897 0.034051
```

Los ciclos 0 y 1 suman 1 porque la variable observada se trata aparte con un potencial determinista. Llamando a la tarea sin decisión, cosa que la ventana no permite, las columnas suman 2,0: la decisión sin política se suma como si fuera una variable de azar.

Un primer intento de medida con `0_miscellaneous/networks/mid/MID-Chancellor-new.pgmx` (horizonte 1, variable `State [1]`, hallazgo en `State [0]`) dio las mismas sumas con y sin evidencia (1,0 en el ciclo 0 y 0,0 en el ciclo 1), quizá porque el programa de prueba no preparaba la red como lo hace la ventana. Ese resultado no se ha sabido interpretar; la medida que vale es la de `MID-Chancellor.pgmx`.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/-tasks/MIDTemporalEvolution.java:292-294

```
    private void commonPreprocessing() throws NotAllNodesHavePoliciesException,
    ↪  IncompatibleEvidenceException.EvidenceIsIncompatibleWithOther,
    ↪  NonProjectablePotentialException {
        checkDecision(probNet, probNet.getNode(conditioningDecision));
        setConditioningVariables(Collections.singletonList(conditioningDecision));
```

inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MID-TemporalEvolution.java:605-617

```
        TablePotential posteriorValue = variableEliminationCore.getProbability();
        //copied from
        ↪  org.openmarkov.inference.variableElimination.tasks.VEPropagation.InvokeVariableEliminationCor
        ↪  lines 220-236
        if (posteriorValue.getVariables().getFirst() != probabilityVariable) {
            // TODO - Comprobar este código
            List<Variable> orderedVariables = new
            ↪  ArrayList<>(Arrays.asList(probabilityVariable, conditioningDecision));
            posteriorValue = posteriorValue.reorder(orderedVariables);
```

```
        }
        // TODO - Realizar la normalización condicionada
        if (getConditioningVariables() == null || getConditioningVariables().isEmpty()) {
            DiscretePotentialOperations.normalize(posteriorValue);
        }
        //end copied
        return posteriorValue;
```

**Cómo arreglarlo.** Antes de tocar el código hay que tomar la decisión de equipo.

Si se normaliza:

- Sin decisión condicionante, `commonPreprocessing` debe dejar la lista vacía (no una lista con un nulo) y normalizar la tabla de una variable.
- Con decisión, normalizar P(V, D, e) columna a columna, una columna por opción de D.
- Una columna que suma cero significa que la evidencia es imposible bajo esa alternativa, aunque no lo sea bajo otras; no puede tratarse como error. La normalización por columnas se niega hoy a dividir por una columna que suma cero y lanza una excepción (commit `87129d6`), así que esa columna hay que marcarla como alternativa imposible en vez de dejar que la excepción salte.
- Ajustar a la vez el `catch` de `resolve()`, que convertiría esa excepción en `UnreachableException`, y el hilo del monitor de progreso de `TraceTemporalEvolutionDialog` (líneas 315-334), que espera un aviso que no llega si hay una excepción.

Si no se normaliza: la ventana y el javadoc deben decir que los valores son P(V, e) y no probabilidades condicionadas.

En cualquier caso, la línea 609 no debería construir listas con nulos.

Sitios donde la misma regla tiene que cumplirse: `MIDTemporalEvolution.computePosteriorProbability` y `VEPropagation.InvokeVariableEliminationCore` (usado por `VETemporalEvolution` desde la exportación a Excel). Los potenciales deterministas de las variables observadas (`createDeterministicPotential`) ya suman uno.

Pruebas de regresión:

- Evolución temporal de «State» en `0_miscellaneous/networks/mid/MID-Chancellor.pgmx` con el hallazgo State [1] = State A: si se decide normalizar, cada columna de los ciclos 2 en adelante debe sumar 1.
- Un MID pequeño con evidencia en una variable del ciclo 0: la evolución de otra variable debe sumar uno en cada ciclo o, si se decide no normalizar, sumar P(e).

### Error 68 · [Corregido] La evolución temporal de una variable que no empieza en el ciclo cero termina con NullPointerException

| | |
|---|---|
| **Estado** | Corregido en el commit `d02a7e7` (09-10-2026): `getFirstSlice` devuelve el primer ciclo en que la red tiene la variable; la gráfica (`TraceTemporalEvolutionDialog.createSeries`) se salta los ciclos en que no existe, que también la hacían fallar, y «Sum» conserva el número de ciclo; la tabla de una variable de azar pone «-» en esos ciclos en vez de «0.0». Con prueba del cálculo (las siete variables dan valor del ciclo 1 al 20) y de la tabla. Se llega desde el modo edición, no desde el modo inferencia, al que una red de este tipo no entra (error 201). Visto en la ventana real de resultados, abierta desde un guion, para «Cost AZT [1]», «Therapy applied [1]» y «State [0]»; para verla hubo que dejar fuera la biblioteca de gráficas antigua, porque con el jar que se repartía |

| | |
|---|---|
| | esa ventana no se abría para ninguna variable (error 74); corregido ese, se volvió a abrir para las tres sin tocar nada. |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.temporalevaluation.tasks |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:161-174`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:440-451`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:553-565` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 69 |

**Qué pasa.** En modo inferencia de un modelo de Markov con decisiones (MID, diagrama de influencia de Markov), la evolución temporal de una variable que el modelo solo define a partir del ciclo 1 no se puede pedir: sale un error inesperado y la ventana con la gráfica no se abre. Son variables frecuentes: costes y utilidades que se cuentan desde el primer ciclo, variables de tratamiento aplicado… No se saltan ciclos en silencio: no se obtiene nada.

Camino de llegada: abrir `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`, pasar a modo inferencia, clic derecho sobre un nodo temporal cuyo primer ciclo es el 1 (por ejemplo «Cost AZT [1]» o «Therapy applied [1]») → «Temporal Evolution» → aceptar. `NetworkEditorPanel` abre `TemporalEvolutionDialog`, que al aceptar crea `TraceTemporalEvolutionDialog`, y este lanza `new MIDTemporalEvolution(red, variable)` en un hilo aparte. La excepción no está entre las que ese hilo captura, así que llega al manejador general de excepciones de la aplicación, que la enseña como error inesperado, y la ventana de resultados no se abre (esto último, por lectura del código).

`MIDTemporalEvolution` calcula el valor de una variable ciclo a ciclo. Su constructor averigua en qué ciclo empieza la variable pedida con `getFirstSlice` y busca la variable de ese ciclo: `firstSlice = getFirstSlice(temporalVariable, probNet); this.temporalVariable = probNet.getVariable(temporalVariable.getBaseName(), firstSlice);`

`getFirstSlice` busca la variable en el ciclo 0, descarta el resultado y devuelve 0 siempre. La cabecera de la clase promete contemplar variables que no empiezan en el ciclo cero, y el javadoc del propio método lleva un «FIXME» que reconoce no saber hacerlo.

Para una variable que solo existe desde el ciclo 1, `probNet.getVariable(nombre, 0)` devuelve nulo, `this.temporalVariable` queda nulo y la evaluación muere en `evaluateSlice` (línea 450) al pedirle el nombre base.

Medido sobre `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx` recorriendo todos sus nodos temporales, llamando a la tarea como lo hace la ventana, sin evidencia, con la decisión condicionante que pone el diálogo y también sin ella. La decisión de esa red se llama «Therapy choice» («Therapy type» es la de `MID-Chancellor.pgmx`). Siete de sus nueve variables temporales no se pueden pedir:

```
Community care cost, Cost AZT, Cost lamivudine, Direct medical cost, Life years (utilidad,
↪  primer ciclo 1),
Therapy applied, Transition inhibited (azar, primer ciclo 1):
  java.lang.NullPointerException: Cannot invoke "Variable.getBaseName()" because
  ↪  "this.temporalVariable" is null
     at MIDTemporalEvolution.evaluateSlice(MIDTemporalEvolution.java:450)
     at MIDTemporalEvolution.evaluate(MIDTemporalEvolution.java:522)
```

```
    at MIDTemporalEvolution.resolve(MIDTemporalEvolution.java:277)
State, Time in treatment (primer ciclo 0): 21 resultados, ciclos 0..20
```

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/-tasks/MIDTemporalEvolution.java:166-168

```
        //Gets the first slice in which temporalVariable sequence is defined
        firstSlice = getFirstSlice(temporalVariable, probNet);
        this.temporalVariable = probNet.getVariable(temporalVariable.getBaseName(),
        ↪  firstSlice);
```

inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MID-TemporalEvolution.java:561-565

```
    private static int getFirstSlice(Variable temporalVariable, ProbNet probNet) {
        int timeSlice = 0;
        probNet.getVariable(temporalVariable.getBaseName(), timeSlice);
        return timeSlice;
    }
```

inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MID-TemporalEvolution.java:449-451

```
} else if (firstSlice <= slice) {
    temporalVariableSlice = probNet.getVariable(temporalVariable.getBaseName(), slice);
}
```

**Cómo arreglarlo.** Hacer que `getFirstSlice` devuelva el menor ciclo en que la red tiene una variable con ese nombre base. Dos maneras: recorrer los ciclos desde 0 hasta el horizonte hasta que `getVariable(baseName, k)` no sea `null`, o recorrer `probNet.getVariables()` y quedarse con el mínimo `getTimeSlice()` de las que comparten `getBaseName()`. Si no hay ninguna, lanzar una excepción con mensaje.

Con eso `this.temporalVariable` deja de ser `null` y la condición `firstSlice <= slice` de `evaluateSlice` (líneas 449 y 467), que ya compara con el ciclo en curso, hace lo que promete la cabecera de la clase («contempla la posibilidad de que la secuencia no empiece en el ciclo cero»).

Hay que comprobar que la tabla de resultados (`TemporalEvolutionTablePane`) y la gráfica aceptan una evolución sin ciclo 0: el constructor de tabla por criterio ya escribe «-» antes del primer ciclo; el constructor de tabla por variable recorre ciclos desde 0 y rellena con 0,0 los que faltan.

Prueba de regresión: sobre `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`, `new MIDTemporalEvolution(red, variable).getTemporalEvolution()` debe devolver resultados para los ciclos 1 a 20, sin excepción, para cada uno de los siete nodos citados (en particular «Cost AZT [1]» y «Therapy applied [1]»).

### Error 69 · [Corregido] Pedir la evolución temporal de una decisión temporal muere con NoSuchElementException

| | |
|---|---|
| **Estado** | Corregido en el commit `6522c8e` (09-10-2026): cuando la decisión no tiene política, `MIDTemporalEvolution` calcula antes las políticas óptimas de la red y se las pone a las decisiones que no tienen, de modo que el resultado es la probabilidad de cada opción en cada ciclo bajo la estrategia óptima, como ya lo era con una política impuesta. De las dos salidas de la decisión de equipo se tomó la primera. Con la transición al principio del ciclo, la utilidad del ciclo 0 no cuenta y la decisión de ese ciclo sale con política uniforme. La evolución de una variable de azar numérica (`Age [0]` en `MID-hip-Briggs`) sigue dando la |

| | |
|---|---|
| | misma excepción por programa; el menú no la ofrece |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.temporalevaluation.tasks |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:602-619` `inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:440-481` `gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:642-656` `gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/NodeContextualMenu.java:168-169` `gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:312-341` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No. Pedía decidir si la evolución temporal de una decisión tiene sentido (probabilidad de elegir cada opción en cada ciclo) o si la opción no debe ofrecerse sobre decisiones. Manuel Arias lo decidió el 09-10-2026: tiene sentido, y sin política se calcula bajo la estrategia óptima |
| **Relacionados** | error 67, error 68 |

**Qué pasa.** En un MID (diagrama de influencia de Markov), el diálogo de propiedades de cualquier nodo ofrece el selector de ciclo temporal (`NodeDefinitionPanel`, visible cuando `variables CouldBeTemporal()`), así que una decisión puede ser temporal («Therapy [0]»). Sobre un nodo temporal, el menú contextual añade «Temporal Evolution» (`NodeContextualMenu`, líneas 168-169) y `MainPanelMenuAssistant` la activa para todo nodo temporal salvo los de azar que no son de estados finitos (líneas 629-642): decisiones incluidas. Al aceptar el diálogo de ciclos, `TraceTemporalEvolutionDialog` lanza en un hilo aparte `new MIDTemporalEvolution(red, variable).getTemporalEvolution()`.

Sobre una decisión, el cálculo muere con `NoSuchElementException`. Nadie la recoge en el hilo (el `catch` del diálogo solo recoge `IndexOutOfBoundsException` y excepciones comprobadas), así que llega al manejador general, que la enseña como error inesperado, y la ventana de resultados no se abre.

En cada ciclo, `evaluateSlice` quita la variable de interés de la lista de variables a eliminar y pide a `VariableEliminationCore` la probabilidad. Para una decisión el resultado es un potencial **sin variables**, y `computePosteriorProbability` hace `posteriorValue.getVariables().getFirst()` sobre una lista vacía.

Medido con un pequeño programa de prueba sobre un MID mínimo: `Illness [0]` (azar, estados healthy/ill), `Therapy [0]` (decisión, no/yes) y `Cost [0]` (utilidad que depende de las dos), horizonte 5. Se probó con la decisión condicionante que pone el diálogo por omisión (la única decisión sin política, `Therapy [0]`) y también sin decisión condicionante:

```
decision Therapy [0] EXC java.util.NoSuchElementException
   at java.util.ArrayList.getFirst(ArrayList.java:440)
   at MIDTemporalEvolution.computePosteriorProbability(MIDTemporalEvolution.java:607)
   at MIDTemporalEvolution.evaluateSlice(MIDTemporalEvolution.java:475)
```

`Illness [0]` y `Cost [0]` sí se calculan.

Ninguna red de `0_miscellaneous/networks/mid` ni de `integrationTests` tiene decisiones temporales, así que para reproducirlo hace falta construir un MID como el descrito.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/-tasks/MIDTemporalEvolution.java:602-612

```
private TablePotential computePosteriorProbability(VariableEliminationCore
↪  variableEliminationCore, Variable
        probabilityVariable) throws CannotNormalizePotentialException {
    // 13/10/2022 only one conditioning decision variable
    TablePotential posteriorValue = variableEliminationCore.getProbability();
    //copied from
    ↪  org.openmarkov.inference.variableElimination.tasks.VEPropagation.InvokeVariableEliminationCore
    ↪  lines 220-236
    if (posteriorValue.getVariables().getFirst() != probabilityVariable) {
        // TODO - Comprobar este código
        List<Variable> orderedVariables = new
        ↪  ArrayList<>(Arrays.asList(probabilityVariable, conditioningDecision));
        posteriorValue = posteriorValue.reorder(orderedVariables);
    }
```

gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:642-655

```
if (visualNode.getNode().getVariable().isTemporal()) {
    ...
    if (!(
            visualNode.getNode().getNodeType() == NodeType.CHANCE &&
                    visualNode.getNode()
                              .getVariable()
                              .getVariableType() != VariableType.FINITE_STATES
    )) {
        canLog = true;
        canTemporalEvolution = true;
    }
}
```

**Cómo arreglarlo.** Hay que decidir qué significa la evolución temporal de una decisión.

- Si se quiere contestar (probabilidad de tomar cada opción en cada ciclo según la política óptima o impuesta), `MIDTemporalEvolution` necesita un camino propio para decisiones: hoy `evaluateSlice` las trata como variables de probabilidad y la eliminación no produce un potencial sobre ellas.
- Si no se quiere, la opción no debe activarse sobre decisiones: hay que cambiar la condición de `MainPanelMenuAssistant` (líneas 634-642) y la construcción del menú en `NodeContextualMenu` (líneas 168-169). Además, `MIDTemporalEvolution` debería rechazar la variable con una excepción declarada en su constructor, como ya hace con `VariableIsNotTemporal`.

En ambos casos, `computePosteriorProbability` no debe llamar a `getFirst()` sin comprobar que la lista no esté vacía.

Prueba: construir el MID mínimo descrito y comprobar que `new MIDTemporalEvolution(red, Therapy[0]).getTemporalEvolution()` o bien devuelve una tabla por ciclo, o bien el constructor lanza la excepción declarada; y que la opción de menú queda desactivada si se elige no ofrecerla.

### Error 70 · [Corregido] «Igual que el ciclo anterior» revienta si el ciclo anterior no tiene esa variable o su nodo no tiene potencial

| | |
|---|---|
| **Estado** | Corregido en el commit `6bac458` (14-10-2026). `SameAsPrevious.getOriginalPotential` retrocede ciclo a ciclo saltando los que no tienen la variable o cuyo nodo no tiene relación, y devuelve `null` si llega al principio sin encontrar una que no sea «Same as previous». `MainPanelMenuAssistant` toma ese `null` como «sin incertidumbre»; `TemporalNetOperations.compactNetwork` lanza `NotSupportedOperationException` con el nombre del nodo que no tiene de dónde copiar. No se toca `validate`: el editor sigue ofreciendo «Same as previous» aunque el ciclo anterior no tenga la variable. Pruebas: los cuatro casos de la medida, uno más (un ciclo sin la variable se salta y se copia el de antes) y la expansión; con el código anterior fallaban cinco de los seis. Visto de paso, fuera de este error: `classifyNodesbySlices` supone que el primer ciclo con variables es el 0. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/SameAsPrevious.java:61-63`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/SameAsPrevious.java:85-103`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:66-76`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:250-278` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 29, error 43, error 71 |

**Qué pasa.** En un modelo temporal (red bayesiana dinámica, diagrama de influencia de Markov, etc.) cada variable existe en varios ciclos o cortes de tiempo (`Edad [0]`, `Edad [1]`…). El potencial `SameAsPrevious` («Same as previous», igual que el anterior) de un nodo del ciclo $t$ no guarda números: indica que su distribución es la del mismo nodo en el ciclo anterior.

Basta con abrir un fichero en el que un nodo con «Same as previous» no tiene la variable en el ciclo de antes, o con elegir «Same as previous» para un nodo así, para que salte una excepción al refrescar los menús, y vuelva a saltar con cada edición posterior. Una variable que aparece a mitad del modelo —un tratamiento que empieza en el ciclo 1 o en el 3— tiene exactamente esa forma.

El camino se ha leído en el código, no ejecutado en la ventana: `MainPanelMenuAssistant.updateInferenceButtons` recorre todos los potenciales de la red y llama a `getOriginalPotential` para cada `SameAsPrevious`, y se ejecuta al abrir una red (`updateOptionsNewNetworkOpen`) y tras cada modificación (`updateOptionsNetworkModified`). En la evaluación, `TemporalNetOperations.compactNetwork` (usada por `expandNetwork`) llega al caso 3 de la medida de abajo cuando un nodo `SameAsPrevious` cuelga de otro `SameAsPrevious` sin variable en el ciclo de antes.

Nada impide llegar a esas formas. `SameAsPrevious.validate`, que decide si el editor de potenciales ofrece este tipo para un nodo, solo exige que la variable sea temporal y de ciclo mayor que cero; no mira si el ciclo anterior tiene la variable. El lector de `.pgmx` (`PGMXPotentialParsers.getSameAsPrevious`) solo exige que la variable sea temporal.

`getOriginalPotential` busca el potencial retrocediendo ciclo a ciclo hasta encontrar uno que no sea también `SameAsPrevious`. El bucle hace tres cosas seguidas sin comprobar ninguna: `probNet.getVariable(nombreBase, --ciclo)` (que devuelve `null` si ese ciclo no tiene la variable), `prob`

`Net.getNode(variable)` (que devuelve `null` si recibe `null`) y `getPotentials().getFirst()` (que lanza si la lista está vacía).

Además, cuando el método devuelve `null` (si el retroceso llega al ciclo 0 sin encontrar un potencial que no sea `SameAsPrevious`), sus dos llamadores lo usan sin comprobar: `TemporalNetOperations.compactNetwork` hace `.getOriginalPotential(probNet).copy()` y `MainPanelMenuAssistant.updateInferenceButtons` hace `originalPotential.isUncertain()`.

Medido con un pequeño programa de prueba sobre redes construidas en código:

```
1. solo existe Treatment [1] (SameAsPrevious)          -> NullPointerException
↪  (getNode(...) es null)
2. A [0] sin potenciales, A [1] SameAsPrevious          -> NoSuchElementException
3. B [2] y B [3] SameAsPrevious, no existe B [1]        -> NullPointerException
4. C [0] uniforme, C [1] SameAsPrevious (caso normal)   -> devuelve P(C [0]) = Uniform
```

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/SameAsPrevious.java:85-103

```
private static @Nullable Potential getOriginalPotential(ProbNet probNet, Variable variable)
↪ {
    Potential previousPotential = null;
    if (!variable.isTemporal()) {
        return null;
    }
    int timeSlice = variable.getTimeSlice();
    @Nullable Variable previousVariable = null;
    while (timeSlice > 0 && previousVariable == null) {
        previousVariable = probNet.getVariable(variable.getBaseName(), --timeSlice);
        previousPotential = probNet.getNode(previousVariable).getPotentials().getFirst();
        if (previousPotential instanceof SameAsPrevious) {
            previousVariable = null;
        }
    }
    if (previousVariable == null) {// There is no previous variable
        return null;
    }
    return previousPotential;
}
```

**Cómo arreglarlo.** Dentro del bucle, si `getVariable` devuelve `null` o el nodo no tiene potenciales, pasar al ciclo anterior sin tocar el nodo; al terminar sin encontrar nada, devolver `null` (ya documentado con `@Nullable`).

Después, los dos llamadores tienen que tratar ese `null`:

- `TemporalNetOperations.compactNetwork` (líneas 70-76): lanzar una excepción del dominio que diga qué nodo tiene «Same as previous» sin ciclo anterior del que copiar, en vez de un `NullPointerException`.
- `MainPanelMenuAssistant.updateInferenceButtons` (líneas 253-258): tratarlo como potencial sin incertidumbre.

La regla puede imponerse también antes: `SameAsPrevious.validate` (líneas 61-63) podría exigir que exista la variable en el ciclo anterior, para que el editor no ofrezca este tipo donde no tiene sentido, y el lector podría avisar. Si se endurece `validate`, un fichero que ya tenga esa forma debe seguir abriéndose sin reventar, así que el arreglo del bucle es necesario en cualquier caso.

Prueba de regresión: en `core/src/test`, los cuatro casos de arriba (variable sin ciclo anterior, ciclo anterior sin potencial, cadena de dos `SameAsPrevious` sin ciclo anterior, caso normal), comprobando

que los tres primeros devuelven `null` sin excepción y el cuarto devuelve el potencial del ciclo 0.

### Error 71 · [Corregido en parte] Un modelo de Markov con regresiones que nombran variables del ciclo (CHAP) no se puede evaluar: al expandirlo, las fórmulas de los ciclos siguientes siguen nombrando las variables del ciclo 0

| | |
|---|---|
| **Estado** | Presente en parte. El commit `971f541` (10-10-2026), el mismo que corrige el error 230, hace que `GLMPotential.shift` desplace también las fórmulas de las covariables, que es el primer punto de *Cómo arreglarlo*. Medido el 15-10-2026 sin ventana: en las cuatro redes CHAP, el análisis de coste-efectividad, la evaluación, la propagación y la evolución temporal ya terminan, y el coste-efectividad da los números de la medida de abajo (placebo, 183,2 de coste y 2,629 de efectividad; cotrimoxazol, 188,8 y 4,082); `MID-mammography-1-0.pgmx`, con un hallazgo en «Age [0]», también se resuelve. Sigue presente lo de «Expand network»: el despliegue termina, pero la red que abre no se puede propagar, ni evaluar, ni analizar, y sale el mismo mensaje («… no value resolved for variable Age [0]»), en las cuatro redes CHAP. Está medido con la llamada que hace la ventana, `TemporalNetOperations.expandNetwork(red, evidencia, nombre)`, y no visto en ella. Es lo que avisaba el segundo punto de *Cómo arreglarlo*: las fórmulas de la red desplegada siguen apuntando a variables de copias anteriores de la red, y al final del despliegue `transformToInfluenceDiagram` (líneas 513-529) renombra «Age [0]» a «Age \|0\|», así que ya no la encuentran tampoco por el nombre. Con `MID-Chancellor.pgmx`, que no tiene regresiones, la red desplegada sí se resuelve. Tampoco se han vuelto a poner `testCHAP` ni `testCHAPSV` en `CEAGlobalAnalysisTest`; la prueba de `971f541` mide `MID-CHAP-Ryan-Griffin.pgmx` con el coste-efectividad y la propagación. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:152-158,320-322`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:320-325`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:500-502`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:51-93,100-107,180-197`<br>`core/src/main/java/org/openmarkov/core/expression/VariableExpression.java:11-18`<br>`core/src/main/java/org/openmarkov/core/expression/ReferencedExpression.java:76-88`<br>`integrationTests/src/test/java/org/openmarkov/integrationTests/costeffectiveness/CEAGlobalAnalysisTest.java:75-78` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 10, error 13, error 14, error 15, error 46, error 70, error 114 |

**Qué pasa.** Con `0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx` abierto (un diagrama de influencia de Markov, MID, sobre profilaxis con cotrimoxazol), la herramienta de análisis de coste-efectividad (`CostEffectivenessPlugin`), con ámbito global y tras aceptar sus dos diálogos de opciones, termina en un diálogo de error con el texto «An expression cannot be resolved as there is no value resolved for variable Age [0]» (`NonProjectablePotentialException$CannotResolve`

`Variable`). El mensaje no dice qué potencial ni que el modelo esté mal; de hecho el modelo está bien. Con el mismo mensaje fallan, medido con un pequeño programa de prueba que hace las mismas llamadas:

- la evaluación (`VEEvaluation`) y la propagación del modo inferencia (esta, detrás del error de rangos de error 46);
- «Expand network» (`TemporalNetOperations.expandNetwork(red, evidencia, nombre)`);
- la evolución temporal (`MIDTemporalEvolution`), aquí envuelta en `UnreachableException`, que la ventana enseña como error no previsto.

Afecta a las cuatro redes CHAP del repositorio: la anterior, su copia `0_miscellaneous/networks/mid/1-0/MID-CHAP-Ryan-Griffin-1-0.pgmx`, e `integrationTests/src/main/resources/networks/mid/chap.pgmx` y `chap-sv.pgmx`. Es la causa por la que se quitaron las pruebas `testCHAP` y `testCHAPSV` de `CEAGlobalAnalysisTest` (el comentario de las líneas 75-78 lo dice). `0_miscellaneous/networks/mid/1-0/MID-mammography-1-0.pgmx` tiene el mismo problema en las regresiones de «Invasive cancer», pero falla antes por otro motivo.

Por qué pasa. Un MID se describe con los ciclos 0 y 1, y antes de resolver se expande hasta el horizonte. `TemporalNetOperations.compactNetwork` crea el ciclo 1 de los nodos que solo existen en el ciclo 0, y `expandNetwork` (líneas 100-107) va copiando el último ciclo al siguiente. En los dos casos, `expandPotentialAndLinks` (líneas 180-197) copia el potencial y llama a `Potential.shift`, que cambia su lista de variables por las del ciclo nuevo (líneas 500-502).

Las regresiones (la familia `GLMPotential`: combinación lineal, exponencial, función, riesgos de Weibull y exponencial) guardan cada covariable como un `VariableExpression`, una fórmula que apunta a objetos `Variable` concretos. `GLMPotential.shift` (líneas 320-322) solo llama a `super.shift`, y `WeibullHazardPotential.shift` desplaza además la variable de tiempo, pero ninguno toca las fórmulas. Así, tras expandir CHAP:

```
Age at state entry [1]: variables [Age at state entry [1], Age [1], Time in state [1]]
                        fórmula   {Age [0]} + -1.0*{Time in state [0]}
```

Al calcular, `GLMPotential.tableProject` da valor solo a las variables del potencial (`Age [1]`, `Time in state [1]`), y `ReferencedExpression` busca `Age [0]` por objeto y por nombre sin encontrarlo (líneas 76-88). En CHAP expandida a 20 ciclos hay 426 referencias así: las de «Age at state entry [k]» y las de las exponenciales de los árboles de «Inpatient costs [k]» y «Outpatient costs [k]».

Medido: expandiendo CHAP y reescribiendo a mano cada fórmula para que nombre la variable del mismo nombre base de su potencial (350 fórmulas), `VECEAnalysis` sobre la red expandida termina: placebo, coste 183,2 y efectividad 2,629; cotrimoxazol, 188,8 y 4,082; el umbral entre ambas está en 3,87. Con esa reescritura se ve también el efecto de error 10 sobre CHAP (allí se describe la medida).

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:320-322

```
    @Override public void shift(ProbNet probNet, int timeDifference) {
        super.shift(probNet, timeDifference);
    }
```

core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:186-194

```
            Potential newPotential;
            if (oldPotential instanceof CycleLengthShift) {
                newPotential = new
↪   CycleLengthShift(oldPotential.getShiftedVariables(probNet, timeDifference),
                                                       probNet.getCycleLength());
```

```
            } else {
                newPotential = oldPotential.copy();
                newPotential.shift(probNet, timeDifference);
            }
```

**Cómo arreglarlo.** `GLMPotential.shift` tiene que desplazar también las covariables: para cada fórmula, sustituir cada variable temporal a la que apunta por `probNet.getShiftedVariable(variable, timeDifference)` (las atemporales se quedan) y construir un `VariableExpression` nuevo sobre la lista de variables desplazada, porque la fórmula guarda las variables por objeto y no basta con cambiar el texto. `WeibullHazardPotential.shift` y `ExponentialHazardPotential` heredan de ahí; `TreeADDPotential.shift` ya reenvía el desplazamiento a las ramas, así que los árboles quedan cubiertos.

Otros sitios de la misma regla, «un potencial y sus fórmulas nombran las mismas variables»:

- `UnivariateDistrPotential` y `AugmentedProbTable`, cuyos parámetros también pueden ser fórmulas y que heredan `Potential.shift` sin tocarlas (no se ha encontrado ninguna red del repositorio con ese caso tras expandir);
- `GLMPotential.deepCopy` (línea 343) y el constructor de copia (línea 84) copian el array de covariables pero no las fórmulas, que siguen apuntando a las variables de la red original; hoy funciona porque la búsqueda cae al nombre, y conviene comprobarlo al tocar esto.

Prueba de regresión: una prueba de `core` que expanda un MID de dos ciclos con un nodo `Y [0]` definido como combinación lineal de `{X [0]}` y compruebe que la fórmula de `Y [1]` nombra `X [1]`; y volver a poner `testCHAP` en `CEAGlobalAnalysisTest`, con `VECEAnalysis` sobre `integrationTests/src/main/resources/networks/mid/chap.pgmx` terminando sin excepción.

### Error 72 · [Corregido] Pasar a atemporal una variable temporal puede dejar dos variables con el mismo nombre, y la red se guarda y se reabre así

| | |
|---|---|
| **Estado** | Corregido en el commit `492d1cd` (14-10-2026). `TimeSliceEdit.getNewName` devuelve el nombre base cuando el instante nuevo es `Variable.noTemporalTimeSlice`, como `rebuildFullName`, y `getPreviousName` devuelve el nombre que la variable tiene. El choque se rechaza con `VariableNameIsAlreadyPresent`; el diálogo ya trataba ese rechazo (vuelve a seleccionar el instante anterior y enseña el mensaje), como cuando se elige un instante que ya existe. `NodeBaseNameEdit` no tiene el hueco: compara nombres base, lo que es más estricto. Pruebas: el choque se rechaza; sin choque se acepta y deshacer devuelve «Therapy [0]»; dar un instante que ya tiene otra variable se sigue rechazando. Con el código anterior fallaba la primera. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/TimeSliceEdit.java:49-57`<br>`core/src/main/java/org/openmarkov/core/action/core/TimeSliceEdit.java:94-105`<br>`core/src/main/java/org/openmarkov/core/model/network/Variable.java:580-628`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:845-866` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |

| | |
|---|---|
| **Decisión de equipo** | No |
| **Relacionados** | error 38 |

**Qué pasa.** Una variable de un modelo temporal se llama, por ejemplo, «Therapy [2]»: nombre base e instante. Si la red ya tiene una variable atemporal llamada «Therapy» y el usuario pasa a atemporal la variable «Therapy [0]», el cambio se acepta y la red queda con dos variables llamadas «Therapy».

Camino de llegada: diálogo de propiedades del nodo, pestaña de definición, desplegable del instante → «Atemporal» (`NodeDefinitionPanel`, que construye `TimeSliceEdit(node, Integer.MIN_VALUE)`), en una red donde ya existe una variable atemporal con el nombre base.

`TimeSliceEdit` (módulo `core`) cambia el instante de una variable. Si la red tiene la restricción `DistinctVariableNames` (obligatoria por omisión), su comprobación previa quita de la lista de nombres `getPreviousName()` y mira si queda `getNewName()`. Los dos métodos componen siempre `nombreBase + " [" + instante + "]"`.

Cuando el instante nuevo es `Integer.MIN_VALUE`, que significa «atemporal», el nombre que la variable tendrá de verdad es el nombre base («Therapy», así lo construye `Variable.rebuildFullName`). Pero la comprobación pregunta por «Therapy [-2147483648]», que no existe nunca, y siempre pasa.

Comprobado ejecutando, en una red MID (diagrama de influencia de Markov) con «Therapy [0]» y «Therapy»:

- `getNewName()` devuelve «Therapy [-2147483648]»; la edición se ejecuta y la red queda con los nombres [Therapy, Therapy].
- `probNet.getVariable("Therapy")` devuelve la primera (estados «no, yes»), de modo que la otra queda oculta a toda búsqueda por nombre.
- Deshacer devuelve «Therapy [0]» y rehacer vuelve a duplicar.
- La comprobación de red completa (`probNet.checkConstraints()`) sí detecta `VariableNameIsAlreadyPresent`, pero la edición no la usa.
- La red se escribe en `.pgmx` y se vuelve a leer con dos variables llamadas «Therapy»; como el fichero enlaza potenciales y enlaces por nombre, no vuelve como se guardó.

El paso contrario (de atemporal a un instante) no tiene el mismo hueco: la comprobación pregunta por el nombre correcto («Therapy [0]»).

**El código.** core/src/main/java/org/openmarkov/core/action/core/TimeSliceEdit.java:49-57, 94-96

```
@Override public void checkConstraintsWillBeMet(ConstraintChecker constraintChecker) {
    if (probNet.getConstraintOfClass(DistinctVariableNames.class) instanceof
    ↪  DistinctVariableNames constraint) {
        List<String> variablesProbNetNames = probNet.getVariablesNames();
        variablesProbNetNames.remove(this.getPreviousName());
        if (variablesProbNetNames.contains(this.getNewName())) {
            constraintChecker.addException(new
↪   ConstraintViolatedException.VariableNameIsAlreadyPresent(constraint,
↪   this.getNewName()));
        }
    }
}
...
public String getNewName() {
    return lastBaseName + " " + "[" + String.valueOf(newTimeSlice) + "]";
}
```

**Cómo arreglarlo.** Que `getNewName()` y `getPreviousName()` devuelvan el nombre base solo cuando el instante es `Integer.MIN_VALUE` (lo mismo que hace `Variable.rebuildFullName`), o mejor, calcular el nombre con la propia regla de `Variable` para que no haya dos copias.

La regla «nombres distintos» vive en `DistinctVariableNames.checkProbNet` (red completa) y en cada edición que cambia un nombre: en esta, `TimeSliceEdit.checkConstraintsWillBeMet`, y en la edición del nombre de nodo (`NodeBaseNameEdit`), que conviene revisar con la misma prueba.

Prueba: en una red con «Therapy [0]» y «Therapy», pasar la primera a atemporal debe rechazarse; pasar a atemporal sin conflicto debe aceptarse.

### Error 73 · [Corregido] Cambiar la duración del ciclo en la pestaña de opciones temporales de las propiedades de la red no cambia la red

| | |
|---|---|
| **Estado** | Corregido en el commit `e2c168b` (14-10-2026). Al pulsar «OK», `NetworkPropertiesDialog` pide a la pestaña que aplique la duración (`NetworkTemporalOptionsPanel.applyCycleLength`): si el valor o la unidad han cambiado, lo hace con `CycleLengthEdit`, que se deshace como las demás ediciones; si el valor no es un número positivo, el diálogo no se cierra y lo dice. Abrir la pestaña ya no da duración a una red que no la tiene: enseña la de omisión. `CycleLengthEdit` acepta una red sin duración, y deshacerla la deja otra vez sin ella. Comprobado con pruebas sobre la edición y sobre la pestaña, no en la ventana |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.network |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkTemporalOptionsPanel.java:44-114`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:131,214-219,270-273`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkDefinitionPanel.java:347-350`<br>`core/src/main/java/org/openmarkov/core/action/core/CycleLengthEdit.java:28-49` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 66 |

**Qué pasa.** En una red temporal (las que no llevan la restricción `OnlyAtemporalVariables`: modelos de Markov con decisiones o MID —diagramas de influencia de Markov—, POMDP —procesos de decisión de Markov parcialmente observables—, redes bayesianas dinámicas, LIMID —diagramas de influencia con memoria limitada— dinámicos, DAN —redes de análisis de decisiones— de Markov…), el diálogo «File → Network properties…» tiene la pestaña «Temporal options», con un cuadro de texto para el valor de la duración del ciclo y un desplegable para la unidad (día, mes, año…).

Escribir «6», elegir «months» y aceptar deja la duración del ciclo como estaba. Al reabrir el diálogo se ve el valor antiguo, al guardar se escribe el antiguo (`PGMXWriter_0_2.java:197-202`) y los cálculos que dependen de él (el ajuste del descuento por unidad de tiempo en `MIDTemporalEvolution.java:631-632`, el potencial `CycleLengthShift`) usan el antiguo. No hay mensaje: el usuario cree que lo ha cambiado.

Ningún otro diálogo de la interfaz llama a `setCycleLength`, así que la duración del ciclo de una red existente no se puede cambiar desde el programa.

`NetworkTemporalOptionsPanel` pone los dos controles y los rellena con lo que dice la red. Eso es todo lo que hace: no registra ningún oyente en ellos, los guarda en campos privados sin métodos de acceso y ningún otro código los lee.

Al pulsar «OK», `NetworkPropertiesDialog.doOkClickBeforeHide` cierra la sub-historia de ediciones y devuelve `NetworkDefinitionPanel.checkName()`, que siempre devuelve `true`; no mira esa pestaña.

La edición necesaria existe (`CycleLengthEdit`, con su deshacer), pero no la crea ningún código de producción.

Además, si la red no tiene duración de ciclo, `initialize()` le asigna una nueva con `probNet.setCycleLength(...)` directamente, fuera del sistema de ediciones, solo por abrir la pestaña.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkTemporalOptionsPanel.java:55-95

```
private void initialize() {
    add(getUnitScale());
    add(getTemporalUnits());
    CycleLength temporalUnit;
    if (probNet.getCycleLength() != null) {
        temporalUnit = probNet.getCycleLength();
    } else {
        temporalUnit = new CycleLength();
        probNet.setCycleLength(temporalUnit);
    }
    probNetUnit = temporalUnit.getUnit();
    probNetScale = temporalUnit.getValue();
    temporalUnits.setSelectedItem(
            stringDatabase.getString("NetworkTemporalOptionsPanel.TemporalOptions.Unit." +
↪   probNetUnit.toString()));
    unitScale.setText(String.valueOf(probNetScale));
}
...
private Component getUnitScale() {
    if (unitScale == null) {
        unitScale = new JTextField();
    }
    return unitScale;
}
```

gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:270-273

```
@Override protected boolean doOkClickBeforeHide() {
    probNet.getPNESupport().closeSubEditHistory();
    return NetworkDefinitionPanel.checkName();
}
```

**Cómo arreglarlo.** Hacer que el cambio llegue a la red mediante `CycleLengthEdit`, dentro de la sub-historia que el diálogo ya abre, para que «Cancel» lo revierta (`cancelLastSubEditHistory`) y deshacer funcione.

Dos opciones razonables:

- ejecutar la edición cuando cambia cualquiera de los dos controles (oyente de acción en el desplegable y de pérdida de foco o «Enter» en el cuadro de texto);
- leer los dos controles en `doOkClickBeforeHide`.

En ambos casos hay que validar el número (positivo, bien escrito) y avisar si no lo es, en vez de lanzar una excepción. Conviene también que la asignación por defecto de `initialize()` no cambie la red fuera de las ediciones.

`CycleLengthEdit` es una de las ediciones de opciones de la red que ejecutan su cambio dos veces al rehacer (nota 66); allí no hay nada que corregir mientras nadie la use, pero antes de usarla desde la interfaz hay que revisar su `redo()`.

Prueba de regresión: construir el panel sin pantalla sobre una red MID, cambiar el texto y la unidad, aceptar y comprobar `probNet.getCycleLength()`; y comprobar que cancelar y deshacer lo devuelven.

### Error 211 · En la evolución temporal, con «Sum» y una decisión temporal, el nombre de cada curva sale cortado en el corchete del ciclo: «Therapy [0» en vez de «Therapy [0] = yes»

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.inference.temporalevolution |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:1397-1412,1495-1512` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 67, error 69 |

**Qué pasa.** En la ventana de la evolución temporal de una variable de azar, la opción «Sum» reúne en una sola curva las de los estados marcados, una por cada opción de la decisión con la que se mira. Cuando esa decisión es temporal, y su nombre lleva por tanto el ciclo entre corchetes, el nombre de cada curva sumada sale cortado: «Therapy [0», sin el cierre del corchete y sin la opción. Las curvas de las distintas opciones se quedan con el mismo nombre y no se sabe cuál es cuál.

Por qué pasa: cada curva se llama `estado [Decisión = opción]` (líneas 1495-1512), por ejemplo `healthy [Therapy [0] = yes]`. Para nombrar la suma, `sumSeries` se queda con lo que hay entre el primer `[` y el primer `]` (línea 1400), y el primer `]` es el del ciclo, no el que cierra. Evaluada aparte, la expresión da «Therapy [0» para ese nombre, y «Therapy = yes» cuando el nombre de la decisión no lleva corchetes.

Comprobado solo leyendo: la ventana no se ha abierto en ese caso.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:1397-1401

```
private XYSeries sumSeries(ArrayList<XYSeries> arraySeries) {
    String seriesName = (String) arraySeries.get(0).getKey();
    if (!conditioningVariables.isEmpty()) {
        seriesName = seriesName.substring(seriesName.indexOf('[') + 1,
↪   seriesName.indexOf(']'));
    }
```

**Cómo arreglarlo.** Cortar en el último `]` (`lastIndexOf`), o mejor no desmontar un texto: construir el nombre de la suma a partir de la decisión y la opción, que el método que crea las curvas ya conoce.

Prueba: con el cálculo del nombre extraído a un método, comprobar que para `healthy [Therapy [0] = yes]` devuelve `Therapy [0] = yes`.

### Error 259 · [Corregido] Al desplegar un modelo temporal, una tabla de fórmulas copiada al ciclo siguiente se sigue calculando sobre las variables del ciclo del que se copió, y la red desplegada no se puede propagar

| | |
|---|---|
| **Estado** | Corregido en el commit `64f666f` (17-10-2026): `AugmentedProbTablePotential.shift` desplaza, con la lista de variables, las dos listas que saca de ella, la tabla interna y los nombres de las fórmulas, por el mismo `replaceVariable` que ya usaba al sustituir una variable. Medido con las dos pruebas nuevas: en el MID de la prueba desplegado a tres ciclos, la relación de «State [2]» queda sobre «State [2]» y «State [1]», sus fórmulas nombran «Age [2]» y con «Age [2]» = 40 proyecta [0,6; 0,4; 0,4; 0,6]; y con números en las casillas, la propagación de la red desplegada a cuatro ciclos da lo mismo que con la tabla corriente ([0,6095; 0,3905] en «State [3]»), donde antes moría con el índice fuera de rango. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/AugmentedProbTablePotential.java:37-68,124-140,156-162,173-178`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:293-295,500-502`<br>`core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:186-202`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VariableElimination.java:79-85` |
| **Comprobado** | Leyendo el código y ejecutándolo (el despliegue y la propagación, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 71, error 234, error 269 |

**Qué pasa.** Una tabla de fórmulas (`AugmentedProbTablePotential`, que el selector de relaciones llama «AugmentedProbTable») es una tabla de probabilidad cuyas casillas son fórmulas sobre los padres numéricos del nodo. Por dentro guarda, además de su lista de variables, tres cosas sacadas de ella: la lista de las variables de estados finitos, la de los padres numéricos y una tabla interna escrita sobre las primeras, que es la que lleva las fórmulas. Al desplegar un modelo temporal (con «Expand network», o antes de resolverlo: `VariableElimination` lo despliega siempre que tiene variables temporales), la relación de cada nodo se copia al ciclo siguiente con `copy()` y `shift`, y en esta clase `shift` es el de `Potential`, que solo cambia la lista de variables. Las otras tres cosas se quedan en el ciclo de antes, y al calcular, `tableProject` construye la tabla sobre ellas (línea 127). El nodo del ciclo nuevo da entonces una tabla sobre las variables del ciclo anterior.

Medido el 15-10-2026 sin ventana. Una tabla de fórmulas `[S [0], A [0]]` con las casillas `1-{A [0]}/100` y `{A [0]}/100`, copiada y desplazada un ciclo:

```
A shift: variables [S [1], A [1]], finite-state [S [0]], parameters [A [0]], inner table
↪  over [S [0]], cells 1-{A [0]}/100 | {A [0]}/100
```

Y un diagrama de influencia de Markov (MID) con `S [0] → S [1]`, donde la relación de `S [1]` es una tabla de fórmulas sin ningún padre numérico, con números en las casillas (0,9 y el resto; 0,2 y el resto), desplegado a tres ciclos con `TemporalNetOperations.expandNetwork` y propagado con `VEPropagation`:

```
B plain table: S [2] says TablePotential over [S [2], S [1]], projected over [S [2], S [1]]
B plain table: P(S [1]) = [0.55, 0.45] over [S [1]]
B plain table: P(S [2]) = [0.5850000000000001, 0.41500000000000004] over [S [2]]
B plain table: P(S [3]) = [0.6095, 0.39049999999999996] over [S [3]]
B table of formulas: S [2] says AugmentedProbTablePotential over [S [2], S [1]], projected
↪  over [S [1], S [0]]
```

```
B table of formulas: THROWS java.lang.IndexOutOfBoundsException: Index 0 out of bounds for
↪  length 0
    at ...VEPropagation.InvokeVariableEliminationCore(VEPropagation.java:252)
```

Con una tabla corriente las probabilidades salen bien; con la de fórmulas, la relación de `S [2]` se calcula sobre `S [1]` y `S [0]`, ninguna de las tablas calculadas habla ya de `S [3]`, y la propagación muere con un índice fuera de rango que no dice nada de la causa. Si las fórmulas nombran un padre numérico, siguen nombrando el del ciclo de antes. (Proyectar una tabla de fórmulas con un hallazgo sobre un padre numérico falla además por otra causa, desplazada o no: error 269.) En el repositorio la usa una sola red, `integrationTests/src/main/resources/networks_for_every_potential/Dynamic-LIMID-every-potential.pgmx`, en el ciclo 0; al desplegarla, el ciclo 1 que se le crea sale igual (medido: la relación de «AugmentedProbTable [1]» se calcula sobre las variables del ciclo 0). Su prueba, `TheFormulaTableOfTheTestNetworkIsProjectedTest`, solo proyecta la del ciclo 0.

Por qué pasa: `AugmentedProbTablePotential` redefine `replaceVariable` (líneas 156-162) y `deepCopy` (líneas 173-178), que rehacen las dos listas y llevan la tabla interna a las variables nuevas, pero no `shift` ni `setVariables` (`Potential`, líneas 500-502 y 293-295), que solo cambian `variables`. `TemporalNetOperations.expandPotentialAndLinks` copia y desplaza (líneas 199-200).

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/AugmentedProbTablePotential.java:124-127

```
public @NotNull TablePotential tableProject(EvidenceCase evidenceCase, InferenceOptions
↪  inferenceOptions, List<TablePotential> alreadyProjectedPotentials) throws
↪  NonProjectablePotentialException.CannotEvaluate,
↪  NonProjectablePotentialException.CannotResolveVariable {
    Map<Variable, String> findingsMap = evidenceCase.getFindingsMap();
    VariableExpression[] expressions = this.augmentedProbTable.getFunctionValues();
    var resolvedTablePotential = new TablePotential(this.finiteStatesVariables, role);
```

**Cómo arreglarlo.** Redefinir `shift` en `AugmentedProbTablePotential` para que, además de la lista de variables, desplace las dos listas que saca de ella y la tabla interna, y reescriba las fórmulas con los nombres del ciclo nuevo, como hace `GLMPotential.shift` con sus covariables desde `971f541`. Lo mismo con `setVariables`, o que la clase no dependa de copias de su lista y las saque de `variables` cuando las necesite. El «Cómo arreglarlo» del error 71 ya apunta a esta clase por sus fórmulas; esto va más allá, porque la tabla se queda en el ciclo de antes aunque las fórmulas no nombren ninguna variable. Conviene mirar a la vez las demás relaciones que guardan por dentro una tabla o una lista sacada de sus variables.

Prueba: desplegar a tres ciclos un MID de dos ciclos en el que `S [1]` es una tabla de fórmulas sobre `S [0]` y el padre numérico `A [1]`, y comprobar que la relación de `S [2]` se calcula sobre `S [2]` y `S [1]`, que sus fórmulas nombran `A [2]`, y que la propagación de la red desplegada da las mismas probabilidades que con la tabla equivalente.

### Error 261 · [Corregido] Una variable cuyo primer ciclo dice «Same as previous», sin ningún ciclo anterior que copiar, pasa la escritura del modelo sin aviso, y la evolución temporal se para después con un error inesperado que solo dice que el potencial de un nodo no se puede convertir en una tabla, a menudo de un nodo que el usuario no ha dibujado

| | |
|---|---|
| **Estado** | Corregido en el commit `5ab5aa1` (17-10-2026): antes de escribir el modelo, `TemporalNetOperations.compactNetwork` mira todos los nodos con «Same as previous» y, si alguno no tiene nada que copiar, se niega nombrándolos a todos, en el orden de los ciclos; la comprobación de `6bac458`, que solo miraba el ciclo siguiente de un nodo, se quita. Manuel Arias eligió que la negativa sea un aviso y no la ventana de error inesperado: la excepción es `NotEvaluableNetworkException.NothingToCopy`, y la ventana, con el título «Nothing to copy», dice «The relation of these nodes is «Same as previous», but no earlier cycle has a relation for them to copy: «Treatment [1]».». Medido con la red de abajo y horizontes 1, 2 y 3: la escritura del modelo y la evolución de «Age» se niegan nombrando «Treatment [1]». Ninguna de las 148 redes del repositorio que se leen usa «Same as previous». Con prueba. `SameAsPrevious.validate` sigue ofreciéndolo. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network; org.openmarkov.inference.algorithm.temporalevaluation.tasks |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:51-100,107-114`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/SameAsPrevious.java:62-70`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/temporalevaluation/tasks/MIDTemporalEvolution.java:273-282,394-403`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:316-339` |
| **Comprobado** | Leyendo el código y ejecutándolo (la escritura del modelo, las dos evoluciones temporales y la propagación, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 70, nota 199 |

**Qué pasa.** En un diagrama de influencia de Markov (MID), un tratamiento que empieza en el ciclo 1 —«Treatment [1]» sin «Treatment [0]»— cuya relación es «Same as previous» no tiene nada que copiar. El editor de relaciones ofrece «Same as previous» a cualquier nodo de ciclo mayor que cero (`SameAsPrevious.validate`, líneas 62-64), como ya decía el error 70, y ésa es justo la forma del ejemplo de aquella ficha. El usuario no recibe ningún aviso de que el modelo está mal: lo que ve, por lectura, es que pedir la evolución temporal de un nodo, el que sea (`NetworkEditorPanel.temporalEvolution` → `TemporalEvolutionDialog` → `TraceTemporalEvolutionDialog`), acaba en la ventana de error inesperado con un texto que solo dice que el potencial de «Treatment [n]» no se puede convertir en una tabla, sin decir que no tiene nada que copiar; y con un horizonte de 2 o más, ese «Treatment [n]» es un nodo que el usuario no ha dibujado.

Por qué pasa: desde `6bac458` (arreglo del error 70), `TemporalNetOperations.compactNetwork` se niega a escribir el modelo, nombrando el nodo, cuando un nodo dice «Same as previous» y ningún ciclo anterior tiene una relación que copiar. Pero lo mira **desde el ciclo de antes**: recorre los nodos de cada ciclo y, para cada uno que ya tiene gemelo en el ciclo siguiente, mira qué dice ese gemelo. Una variable cuyo **primer** ciclo dice «Same as previous» no tiene ciclo de antes, así que nadie la mira. La escritura pasa en silencio, el «Same as previous» se queda donde estaba

y se copia tal cual a los ciclos que se añaden hasta el horizonte. Lo que pase después depende de quién proyecte esa relación: `SameAsPrevious.tableProject` lanza siempre `PotentialCannotBeConvertedToATable` (líneas 68-70), sin mensaje propio.

Medido el 15-10-2026 con el Java de hoy, sin ventana, con «Age [0]» con una tabla (0,3; 0,7) y «Treatment [1]» con «Same as previous»:

```
horizonte 1, expandNetwork:                    Age [0] y Age [1] con tabla; Treatment [1]
↪  sigue siendo SameAsPrevious
horizonte 1, evolución de Age, VETemporalEvolution:   Age [0] = Age [1] = [0.3, 0.7]
horizonte 1, evolución de Age, MIDTemporalEvolution:  UnreachableException «Potential
↪  P(Treatment [1]) cannot be converted to a table»
horizonte 2, evolución de Age, MIDTemporalEvolution:  UnreachableException «Potential
↪  P(Treatment [2]) cannot be converted to a table»
horizonte 3, evolución de Age, MIDTemporalEvolution:  UnreachableException «Potential
↪  P(Treatment [3]) cannot be converted to a table»
horizonte 1, evolución de Treatment, VETemporalEvolution:
↪  PotentialCannotBeConvertedToATable, sin mensaje
horizontes 1 a 3, VEPropagation de todas las variables:
↪  PotentialCannotBeConvertedToATable, sin mensaje
horizonte 2, evolución de Age, VETemporalEvolution:   «Treatment [2]» is «Same as
↪  previous», but no earlier cycle has a relation for it to copy
```

Son tres cosas distintas:

- Lo que el tratamiento no alcanza **se contesta**: la evolución de la edad con `VETemporalEvolution` poda lo que no llega a la edad y el «Same as previous» no se proyecta nunca.
- La ventana de la evolución temporal usa `MIDTemporalEvolution` (`TraceTemporalEvolutionDialog`, línea 320), que proyecta todas las relaciones del modelo, así que se para aunque se le pregunte por la edad; envuelve la excepción en una `UnreachableException` (línea 281), y el hilo de la ventana no la recoge (líneas 332-338), así que llega al manejador global como error inesperado. Nombra el último ciclo del tratamiento, que con horizonte 2 o más es una copia hecha al escribir el modelo.
- `VETemporalEvolution`, con un horizonte que pasa del primer ciclo de la variable, se para con la negativa de `6bac458`, pero nombrando «Treatment [2]», porque escribe el modelo dos veces y la segunda ve como «Same as previous» la copia que hizo la primera (nota 199). Hoy ninguna ventana la usa.

**El código.** core/src/main/java/org/openmarkov/core/model/network/TemporalNetOperations.java:59-74

```
            for (Node generatingNode : classifiedNodes.get(slice)) {
                if (!probNet.containsShiftedVariable(generatingNode.getVariable(), 1)) {
                    ...
                } else {
                    // Replace all SameAsPrevious potentials
                    Variable variable =
↪  probNet.getShiftedVariable(generatingNode.getVariable(), 1);
                    Node node = probNet.getNode(variable);
                    if (!node.getPotentials().isEmpty() && node.getPotentials().get(0)
                    ↪  instanceof SameAsPrevious) {
                        Potential originalPotential = ((SameAsPrevious)
↪  node.getPotentials().get(0))
                                .getOriginalPotential(probNet);
                        if (originalPotential == null) {
                            throw new NotSupportedOperationException(
```

**Cómo arreglarlo.** Que la comprobación de `6bac458` mire todos los nodos con «Same as previous» y no solo los gemelos de un nodo del ciclo de antes: antes de escribir el modelo (al principio de `compactNetwork` o de `expandNetwork`), recorrer los nodos de la red y, para cada uno cuya relación sea «Same as previous», negarse con las mismas palabras si `getOriginalPotential(probNet)` es nulo, nombrando ese nodo («Treatment [1]»). Con eso, la evolución de la edad con horizonte 1, que hoy se contesta, también se negaría, como ya se niega con horizonte 2; es coherente, porque el modelo está mal. Si además se quiere evitar el modelo mal hecho desde el principio, `SameAsPrevious.validate` podría no ofrecer «Same as previous» cuando ningún ciclo anterior tiene relación; la ficha del error 70 dejó `validate` como estaba a conciencia.

Prueba: con la red de arriba y horizonte 1, que `TemporalNetOperations.expandNetwork` lance `NotSupportedOperationException` cuyo motivo nombre «Treatment [1]», y que `MIDTemporalEvolution` de «Age» se niegue con ese mismo motivo y no con `PotentialCannotBeConvertedToATable`.

# 9 Coste-efectividad y análisis de sensibilidad

El análisis de coste-efectividad, el análisis de sensibilidad determinista (tornado, araña, mapa de dos parámetros) y el probabilístico, con sus ventanas de resultados.

### Error 74 · [Corregido] El jar ejecutable que se publica lleva mezcladas dos versiones de JFreeChart, y la evolución temporal, el coste-efectividad por decisión y casi todas las gráficas del análisis de sensibilidad fallan con NoSuchMethodError

| | |
|---|---|
| **Estado** | Corregido en el commit `6ae9b16` (09-10-2026): `inference.des/pom.xml` excluye `jfree:jfreechart` de SSJ, y con ella se va `jfree:jcommon`; las dos salen también de la lista de licencias de terceros. Medido construyendo el jar: los ficheros de `org/jfree/ui/` pasan de 117 a 0; seis de los métodos de la lista de abajo, llamados de verdad con `java -cp`, fallan los seis con el jar del 8 de octubre y funcionan los seis con el nuevo; el generador `MRG31k3p` de SSJ sigue funcionando. Con prueba: el diálogo del mapa en ámbito global, que fallaba con las dos bibliotecas, se construye (necesita pantalla). Se hizo después del error 83 (`6fa4e26`). No se ha lanzado el jar con `java -jar` para abrir sus ventanas, ni se ha añadido la comprobación de clases duplicadas en la construcción. |
| **Módulo** | full, inference.des |
| **Ficheros** | `inference.des/pom.xml:24-28`<br>`pom.xml:322-326`<br>`full/pom.xml:608-654`<br>`full/pom.xml:178-196`<br>`.github/workflows/maven.yml:51-57`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:389-390`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:655,681,1057`<br>`costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:865,877`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:462,470,635,643,664`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/PlotDialog.java:294`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CEProbabilisticDialog.java:545,557,580,590,593` |

| | |
|---|---|
| | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CESpiderDialog.java:377,388,477,496` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 15, error 67, error 80, error 81, error 82, error 83, error 84, error 86, error 88, error 89, error 90, error 91, nota 37, nota 58, nota 169 |

**Qué pasa.** Con el `OpenMarkov.jar` que se reparte, varias ventanas de resultados no llegan a abrirse. Es el jar que construye la integración continua en `.github/workflows/maven.yml` (paso «Build OpenMarkov.jar»: `mvn package -pl full -am -DskipTests --batch-mode -PGenerateFullJar`) y publica en GitHub Pages (`https://openmarkov.github.io/OpenMarkov/OpenMarkov.jar`). Al construir su gráfica, cada una de esas ventanas llama a un método de JFreeChart (la biblioteca de gráficas) que no existe en las clases que lleva el jar, y Java lanza `java.lang.NoSuchMethodError`. Como no es una `RuntimeException`, el gestor global (`OMExceptionHandler`) la trata como error previsto y `ExceptionDialog` enseña un aviso titulado «Internal error», con «An internal error has happened:», el nombre del método y la pila de llamadas (esto último, por lectura).

Ventanas que fallan con ese jar:

- **Evolución temporal** (modo inferencia, clic derecho sobre un nodo → «Temporal Evolution» → aceptar, y también la de la red entera): falla siempre, por variable y por criterio (`Trace TemporalEvolutionDialog.getChartsPanel`, línea 1056, y `getChartsByCriterionPanel`, líneas 654 y 680).
- **Menú Tools → «Cost-effectiveness analysis» con ámbito de decisión**: el constructor de `CEDecisionResults` arma la pestaña del plano coste-efectividad, que falla en la línea 844, y la ventana no se abre. El ámbito global usa otra ventana (`CEPDialog`), que no está afectada.
- **Menú Tools → «Sensitivity analysis»**: «Tornado/Spider» (la pestaña de araña falla siempre, `TornadoSpiderDialog` línea 652; la de tornado ya en la línea 450 con ámbito global), «Plot» (`PlotDialog`, línea 294), «Map» con ámbito global (`MapDialog`, línea 389), el plano y la curva de aceptabilidad del análisis probabilístico (`CEProbabilisticDialog`, líneas 523 y 571) y «Spider CE» (`CESpiderDialog`, línea 377). El mapa con ámbito de decisión sí se abre.
- `CEDESDialog`, de la simulación de sucesos discretos, tiene cinco llamadas de este tipo, pero solo se construye con el análisis probabilístico de la simulación, que hoy no se puede activar (nota 58).

Mientras dure, en el jar no se llega a ver lo que describen error 67 (evolución temporal), error 84 (coste-efectividad por decisión), error 88 y error 91 (tornado), ni el mapa de ámbito global de error 82.

Por qué pasa. `inference.des/pom.xml` declara SSJ 3.3.1, la biblioteca de simulación estocástica de la que OpenMarkov solo usa los generadores de números aleatorios (`umontreal.ssj.rng`). El `pom` de SSJ declara a su vez, en ámbito de ejecución, `jfree:jfreechart:1.0.12`, que trae `jfree:jcommon:1.0.15`. El proyecto usa `org.jfree:jfreechart:1.5.6`, declarada en el `pom.xml` raíz. Entre esas versiones JFreeChart cambió de grupo (`jfree` → `org.jfree`), así que para Maven son dos bibliotecas distintas y no elige una: las dos entran en las dependencias de `full`.

El perfil `GenerateFullJar` de `full/pom.xml` usa `maven-assembly-plugin` con el descriptor predefinido `jar-with-dependencies`, que desempaqueta todas las dependencias en un solo jar y guarda una sola copia de cada fichero. Medido sobre un jar construido con ese perfil a partir del código actual y sobre el publicado (fechado el 8 de septiembre de 2026), comparando cada clase con las de los dos jar de JFreeChart:

- de las 692 clases de la 1.5.6, las 536 que también existen con el mismo nombre en la 1.0.12 están con la copia de la 1.0.12;
- las 156 que solo tiene la 1.5.6 también están, por ejemplo `org.jfree.chart.ui.RectangleEdge` (en la 1.0.12 esa clase era `org.jfree.ui.RectangleEdge`, de JCommon, que también va dentro).

Como cada clase está una sola vez, no hay orden de carga que decida nada: con `java -jar` esas 536 clases son siempre las de la 1.0.12. El código de OpenMarkov, compilado contra la 1.5.6, encuentra las clases pero no algunos métodos: `LegendTitle.setPosition` existe en la 1.0.12 solo con un `org.jfree.ui.RectangleEdge`, y `setDefaultToolTipGenerator` se llamaba allí `setBaseToolTipGenerator`.

Medido:

- Recorriendo en las clases de OpenMarkov del jar todas las llamadas, accesos a campos y creaciones de objetos de bibliotecas de terceros (4 215), solo 8 métodos no existen dentro del jar, todos de JFreeChart, usados en 27 sitios (lista abajo). Contra la 1.5.6 sola no falta ninguno.
- Construyendo las gráficas sin pantalla con los métodos reales: con las clases del jar, `MapDialog.getMapChart` en ámbito global lanza `NoSuchMethodError` en la línea 389; `TraceTemporalEvolutionDialog.getChartsPanel`, en la 1056; y `CEDecisionResults`, con `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx`, construye la pestaña de análisis y falla en la del plano, línea 844. Con la 1.5.6 por delante de las clases del jar, el mapa global y la evolución temporal se construyen.

Desde el IDE, con el sistema de módulos de Java, no pasa. Los `module-info.java` piden el módulo `org.jfree.jfreechart`, que es el nombre que declara el manifiesto de la 1.5.6; nadie pide `jfreechart`, que sería el nombre de la 1.0.12. Las dos no pueden estar a la vez en la ruta de módulos: medido, al resolver los dos jar de JFreeChart junto con el de SSJ, Java se niega con `ResolutionException: Modules org.jfree.jfreechart and jfreechart export package org.jfree.data.time to module ssj`, así que la aplicación ni arrancaría. Y si la 1.0.12 queda en la ruta de clases, un paquete que pertenece a un módulo con nombre se carga siempre del módulo. El IDE no se ha arrancado: esto sale de los descriptores y de esa medida.

Los instaladores del perfil `Installer`, que la integración continua no publica, tienen la misma mezcla por otro camino: copian todas las dependencias a `full/target/libs`, las dos versiones incluidas, y `jpackage` pone en la ruta de clases los jar de esa carpeta ordenados por nombre, con `jfreechart-1.0.12.jar` antes que `jfreechart-1.5.6.jar` (comprobado en las clases de `jpackage` de Java 25; no se ha construido ningún instalador).

**El código.** inference.des/pom.xml:24-28

```
<dependency>
    <groupId>ca.umontreal.iro.simul</groupId>
    <artifactId>ssj</artifactId>
    <version>3.3.1</version>
</dependency>
```

full/pom.xml:621-626 (perfil `GenerateFullJar`)

```
<finalName>OpenMarkov</finalName>
<appendAssemblyId>false</appendAssemblyId>
<attach>false</attach>
<descriptorRefs>
    <descriptorRef>jar-with-dependencies</descriptorRef>
</descriptorRefs>
```

Métodos de JFreeChart 1.5.6 que no existen dentro de `OpenMarkov.jar`, y dónde se llaman:

```
LegendTitle.setPosition(RectangleEdge)          CEDecisionResults:844
↪ CEProbabilisticDialog:523,571
                                                CESpiderDialog:377  TornadoSpiderDialog:652
                                                ↪ CEDESDialog:495,545
PaintScaleLegend.setPosition(RectangleEdge)     MapDialog:389
PaintScaleLegend.setMargin(RectangleInsets)     MapDialog:390
ValueMarker.setLabelTextAnchor(TextAnchor)      PlotDialog:294
↪ TornadoSpiderDialog:450,458,623,631
Marker.setLabelTextAnchor(TextAnchor)           CEProbabilisticDialog:558  CEDESDialog:532
XYItemRenderer.setDefaultToolTipGenerator       CEDecisionResults:856
↪ CEProbabilisticDialog:535,568
                                                CEDESDialog:505,542
XYLineAndShapeRenderer.setDefaultToolTipGenerator
                                                TraceTemporalEvolutionDialog:654,680,1056
                                                ↪ CESpiderDialog:388
XYSeriesCollection.getSeriesIndex(Comparable)   CESpiderDialog:477,496
```

**Cómo arreglarlo.** Quitar la 1.0.12 del árbol de dependencias: en `inference.des/pom.xml`, añadir a la dependencia de SSJ una exclusión de `jfree:jfreechart`; con ella se va también `jfree:jcommon`, que solo llega por ahí. De SSJ, OpenMarkov solo usa `umontreal.ssj.rng.MRG31k3p` y `RandomStream` (en `DESRandomProvider` y `NotRandomStream`), y ninguna clase de ese paquete hace referencia a JFreeChart (comprobado en el jar de SSJ 3.3.1); la usa solo su paquete de gráficas, `umontreal.ssj.charts`.

Con la exclusión, la regla se cumple a la vez en los tres sitios que reúnen las dependencias: el jar del perfil `GenerateFullJar`, la carpeta `full/target/libs` de la que tiran los instaladores del perfil `Installer`, y la ruta de módulos del IDE y de las pruebas. Para que no vuelva a pasar con otra biblioteca que cambie de grupo, una opción es una comprobación en la construcción que falle cuando dos dependencias traen la misma clase (por ejemplo, la regla `banDuplicateClasses` de `extra-enforcer-rules` para `maven-enforcer-plugin`).

Al quitar la 1.0.12, el jar pasa a comportarse como el IDE también en error 83, que hoy solo se ve desde el IDE.

Prueba: tras construir el jar, comprobar que no contiene `org/jfree/ui/` (JCommon) y que `org/jfree/chart/title/LegendTitle.class` es la de la 1.5.6; y lanzar con `java -cp OpenMarkov.jar` un pequeño programa sin pantalla que construya la gráfica de `TraceTemporalEvolutionDialog.getChartsPanel` o la del mapa en ámbito global sin error. La integración continua puede ejecutarlo justo después del paso «Build OpenMarkov.jar».

### Error 75 · [Corregido] El diagrama de araña de coste-efectividad reescribe los números de la red abierta con la media de cada distribución

| | |
|---|---|
| **Estado** | Corregido en el commit `4f22f63` (14-10-2026): `networkSample` modifica una copia del potencial y la pone en la copia de la red, como `sampleNetwork`; desaparecen el caso `interval > numIntervals` y la vuelta de más de `VECESensAnSpider.resolve`. Medido con la tarea y los parámetros con nombre, que son los que ofrece la ventana: sin el arreglo, la tabla de `Disease` de `ID-CEA-test-2therapies` puesta a [0,8, 0,2] quedaba en [0,86, 0,14] y la de `Result of test` de `ID-decide-test` con 0,030000000000000023; con él, ninguna tabla cambia. Con tablas que coinciden con la media, los resultados de la araña son los mismos antes y después. Cuando no coinciden, sin el arreglo los parámetros que se barrían después del que la pisaba se calculaban con la media y no con la tabla de la red, y ese orden cambia en cada apertura (error 93); con él, todos se calculan con la tabla de la red. Con prueba de los dos casos; sin el arreglo |

| | |
|---|---|
| | fallan los dos. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.modelUncertainty |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/SystematicSampling.java:252-321,382-387`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/UncertainParameter.java:36-38`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VECESensAnSpider.java:59-86`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/model/SensitivityAnalysisController.java:88-92`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CESpiderDialog.java:158-168` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 35, error 49, nota 103, nota 104 |

**Qué pasa.** Ejecutar el diagrama de araña de coste-efectividad cambia las tablas de la red abierta en el editor. Si una tabla no coincide con la media de la distribución de su parámetro incierto, al terminar la araña la tabla tiene la media. Si coinciden, queda un residuo de redondeo. Si el análisis lanza una excepción a mitad del barrido, la red se queda con el punto barrido, porque no hay `finally`. En todos los casos la red cambia sin pasar por una edición, así que no se puede deshacer ni se marca como modificada.

La ventana de incertidumbre escribe en la tabla las medias al aceptar (`UncertainValuesDialog.calculateReferenceValues`), así que en un modelo editado solo desde la aplicación la media y la tabla suelen coincidir y el usuario ve solo el residuo de redondeo; el cambio visible aparece con un fichero en el que no coinciden.

La red que llega al cálculo es la red abierta: `SensitivityAnalysisController` toma `getCurrentNetworkEditorPanel().getProbNet()` y `CESpiderDialog` la pasa a `new VECESensAnSpider(probNet)`, cuyas copias (`InferenceAlgorithm` y la línea 63) son superficiales.

`SystematicSampling.networkSample(red, parámetro, min, max, numIntervalos, nombre, intervalo)` debería devolver una red en la que el parámetro incierto vale el punto `intervalo` de su barrido. Empieza con `ProbNet net = originalNet.copy()`, pero `ProbNet.copy()` es una copia superficial que comparte los potenciales. Además, el método toma como tabla a modificar `originalSubPotentialTable` (línea 282: `TablePotential newSubPotentialTable = originalSubPotentialTable;`) y escribe la columna en su vector de valores (`copyInArray`, línea 307). Así que la red que recibe queda cambiada.

Su propio comentario (líneas 296-298) lo reconoce, y para devolver la red a su estado añade un caso especial: si `interval > numIntervals`, escribe `uncertainParameter.getBaseLineValue()`, que es la **media** de la distribución (`UncertainParameter.getBaseLineValue` devuelve `getProbDensFunction().getMean()`), no el número que tenía la tabla.

El único llamador de producción es `VECESensAnSpider.resolve`, que recorre `for (int i = 0; i <= (iterations+1); ++i)` y descarta el resultado de la última vuelta (`if(i<=iterations)`). Esa vuelta solo sirve para reponer el valor, y cuesta un análisis de coste-efectividad completo más por parámetro.

`sampleNetwork`, el método hermano que usan el tornado y la araña de un solo criterio, no tiene este problema: construye un potencial nuevo y lo sustituye en la copia. Comprobado sobre `0_miscellaneous/networks/id/ID-decide-test.pgmx`: la red original queda idéntica.

Medido con pequeños programas de prueba (deterministas, una ejecución de cada uno) sobre `0_miscellaneous/networks/id/ID-decide-test.pgmx` y `0_miscellaneous/networks/id/ID-CEA-test-2therapies.pgmx`, con una variación de ±20 % sobre el valor de referencia (la opción «Percentage over reference value», PORV en el código) y 4 intervalos:

- Una llamada a `networkSample` con `interval = 0` para «prevalence» cambia la tabla de `Disease` de la red recibida de [0,86, 0,14] a [0,888, 0,112]; para «specificity», la de `Result of test` pasa de 0,97/0,03 a 0,776/0,224.
- Tras la llamada de reposición, los valores vuelven, pero con residuo de redondeo: después de la araña completa, la tabla de `Result of test` de la red recibida queda con 0,030000000000000023 y 0,08999999999999997 donde tenía 0,03 y 0,09.
- Si la tabla no coincide con la media de su distribución, la araña la pisa: con la tabla de `Disease` puesta a [0,8, 0,2] y la distribución de «prevalence» Beta(14, 86) (media 0,14), tras ejecutar la araña la red recibida tiene [0,86, 0,14].

**El código.** core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/SystematicSampling.java:282-308

```
TablePotential newSubPotentialTable = originalSubPotentialTable;
...
double auxValueToAssign = min+(pointsDistance*interval);
//Returns the net to its original potentials (after giving the last net with the last
↪  intervals potentials)
//for the next uncertainParameter. This is necessary because even if you operate on a copy
↪  of the original network
//with "net.copy()", the changes are transferred to the original network.
if(interval > numIntervals) {
    auxValueToAssign = uncertainParameter.getBaseLineValue();
}
double[] newSubpotentialTableValues = newSubPotentialTable.getValues();
System.arraycopy(sampledConfigurationValues, 0, auxSampledConfigurationValues, 0,
↪  numStates);
replaceValueAndRedistributeComplements(auxSampledConfigurationValues, sampler,
↪  posUncertainInColumn,
            auxValueToAssign);
copyInArray(newSubpotentialTableValues, configurationBasePositionInitColumn,
            auxSampledConfigurationValues);
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*. `networkSample` debe construir un potencial nuevo (con una copia del vector de valores) y sustituirlo en la copia de la red, como ya hace `sampleNetwork` (líneas 242-243: `net.removePotential(originalPotential); net.addPotential(newPotential);`), en vez de escribir en `originalSubPotentialTable`.

Con eso sobran el caso `interval > numIntervals` y la vuelta extra de `VECESensAnSpider.resolve`: `i <= iterations+1` pasa a `i <= iterations`, y desaparece el `if(i<=iterations)`.

El arreglo tiene que estar en `networkSample`: la copia superficial de la red seguirá compartiendo las tablas.

Pruebas:

- Tras `VECESensAnSpider.getCEPotentials()` sobre una red cuya tabla no coincide con la media de la distribución del parámetro, los vectores de valores de todos los potenciales de la red recibida deben ser idénticos (con `Arrays.equals`) a los de antes.
- Una llamada suelta a `networkSample` no debe modificar la red que recibe.

### Error 76 · [Corregido] En el análisis probabilístico, una columna con incertidumbre en solo algunas casillas no se sortea o pierde sus números fijos

| | |
|---|---|
| **Estado** | Corregido el 09-10-2026 en cuatro commits. `4eef20f`: `TablePotential.getUncertainColumn` da la columna con la distribución «Exact» en el número de cada casilla sin distribución, y de ahí la toman los dos sorteos, la tabla de la ventana y el diálogo de incertidumbre; vale también para una tabla construida por programa. `1a8cfe7`: las reglas de familia pasan del diálogo a `DistributionsOfAColumn`, en el módulo `core`, con mensajes que nombran el estado que las incumple. `61da645`: al abrir una red se avisa de las columnas que las incumplen, con una ventana como la de las columnas que no suman uno. `059e816`: los cinco análisis que sortean (`VECEPSA`, `VECESensAnSpider`, `VESensAnMap`, `VESensAnPlot` y `VESensAnTornadoSpider`) se niegan a ejecutarse con una columna así, también dentro de las ramas de un árbol. Cada commit lleva su prueba. Visto en la aplicación real, arrancada desde un guion: el aviso al abrir y la negativa del plano de coste-efectividad y del análisis de araña, lanzados llamando al método que llama el botón. Medido: de los 458 ficheros `.pgmx` del repositorio que se leen, la comprobación solo rechaza el que se escribió para la prueba. La corrección por el usuario se hace en el diálogo de incertidumbre de la columna. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.modelUncertainty |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/TablePotentialSampler.java:60-77`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/Sampler.java:56-82,126-179`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1324-1356`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VECEPSA.java:105-113`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CEProbabilisticDialog.java:152-164` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir qué hacer con una columna en la que unas casillas llevan distribución y otras un número fijo. Manuel Arias lo decidió el 09-10-2026: el fichero se abre avisando, los análisis que sortean no se ejecutan mientras quede una columna que incumpla las reglas de familia, y la columna se corrige en la ventana. Es lo que ya se hace con las columnas que no suman uno (commit `9dc49bb`). La misma pregunta, en general, es el error 37, que sigue esperando decisión de equipo |
| **Relacionados** | error 77, error 116, error 167, nota 58, nota 110 |

**Qué pasa.** Una tabla de probabilidad con incertidumbre lleva en sus casillas, además del número, una distribución (Beta, Dirichlet, «Complement»…). Si en una columna (una configuración de los padres) solo algunas casillas llevan distribución, el análisis probabilístico de sensibilidad de coste-efectividad la sortea mal, sin ningún aviso:

- si la casilla del primer estado no tiene distribución, la columna no se sortea nunca y las distribuciones de las demás casillas se ignoran;
- si la tiene, la columna se sortea, pero las casillas sin distribución quedan a cero y su número fijo se pierde.

Camino de llegada: el análisis probabilístico de sensibilidad de coste-efectividad (`CEProbabilistic`

`Dialog`, que construye `VECEPSA` y en cada simulación llama a `potential.sample()` sobre todos los potenciales de la red). También `TreeADDPotential.sample` y `ExactDistrPotential.sample` delegan en este código.

La ventana de incertidumbre (`UncertainValuesDialog.readDataFromTable`) siempre escribe una distribución en todas las casillas de la columna, así que desde la edición no se produce una columna a medias. Sí se produce leyendo un `.pgmx`: el formato admite `<Value />` vacío en cualquier casilla de `UncertainValues`, y `PGMXReader_0_2.getUncertainValue` lo convierte en `null`.

Se leyeron los 77 `.pgmx` del repositorio que contienen `UncertainValues` (bajo `0_miscellaneous/networks` y los `src/*/resources` de los módulos): 63 se leen, 51 tienen incertidumbre y ninguno tiene una columna a medias (las columnas son o todas nulas o todas con distribución). Para llegar al fallo hace falta un fichero hecho a mano o por otra herramienta.

`TablePotentialSampler.sample` recorre la tabla columna a columna y decide si sortearla con `hasUncertainty = uncertainValues.get(0) != null`, es decir, mirando solo la casilla del primer estado. Si esa casilla no tiene distribución, la columna entera se copia tal cual. Si la tiene, `generateSample` reparte las casillas entre la familia de distribuciones independientes, la de Dirichlet y la de complemento; las casillas sin distribución no están en ninguna (los métodos de `Sampler` saltan los nulos), así que quedan a cero en la columna sorteada.

Medido con un pequeño programa de prueba (5 sorteos, con el generador del propio programa, que no admite semilla), sobre una tabla P(X | Y) con X de tres estados:

- Columna y0 = [0,2 fijo, Beta(2,5), Complement]: los cinco sorteos devuelven [0,2 0,5 0,3], la columna original, sin sortear nunca.
- Columna y1 = [Beta(2,5), 0,3 fijo, Complement]: los cinco sorteos devuelven la casilla fija a 0,0000 (por ejemplo [0,4084 0,0000 0,5916]); la columna suma 1 pero ya no dice que P(x1 | y1) = 0,3.

**El código.** core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/TablePotentialSampler.java:60-76

```
for (int configurationIndex = 0; configurationIndex < numConfigurations;
↪  configurationIndex++) {
    int configurationBasePosition = numStates * configurationIndex;
    uncertainValues = getUncertainValuesChance(uTable, configurationBasePosition,
↪  numStates);
    hasUncertainty = uncertainValues.get(0) != null;
    if (hasUncertainty) {
        sampledConfigurationValues = generateSample(uncertainValues, numStates,
↪  functionTypes);
        copyInArray(sampledValues, configurationBasePosition, sampledConfigurationValues);
    } else {
        for (int stateIndex = 0; stateIndex < numStates; stateIndex++) {
            sampledValues[configurationBasePosition + stateIndex] =
↪  originalValues[configurationBasePosition
                    + stateIndex];
        }
    }
}
```

**Cómo arreglarlo.** Un número fijo en una columna con incertidumbre es la distribución «Exact» en ese número; el diálogo de incertidumbre ya lo hacía así (`createExactUncertainValuesFromDouble`), y es lo que hace el commit `4eef20f`.

Pero no toda mezcla vale. El diálogo aplica al aceptar unas reglas de familia (`UncertainValues`

`Dialog.doVerifyRule1`, `2` y `3`): con un «Exact» distinto de cero, las demás casillas solo pueden ser Exact, Range, Triangular o Complement; con una Beta, solo Exact en cero o Complement. Las dos columnas medidas arriba las incumplen: la Beta puede salir por encima de 0,8 y dejar el complemento negativo. El lector no aplica esas reglas, así que un fichero puede traer una columna que el diálogo no habría aceptado. Visto en la ventana el 09-10-2026: con el commit `4eef20f`, el doble clic sobre una de esas columnas abre el diálogo con «Exact 0.2», y aceptarlo responde «Family rule nº1 has been broken».

Lo que se hizo, según lo decidido el 09-10-2026:

- llevar las reglas de familia del diálogo (módulo `gui`) al módulo `core`, para que haya una sola comprobación;
- al abrir un fichero, avisar de las columnas que las incumplen, como se avisa de las que no suman uno (`NetworkFileHandler`, `ColumnsThatDoNotAddUpToOne`);
- que los análisis que sortean se nieguen a ejecutarse mientras quede alguna, nombrándolas.

La corrección por el usuario ya es posible en la ventana: la columna está bloqueada para la edición a mano, el diálogo de incertidumbre se abre con sus distribuciones y exige las reglas al aceptar, y «Remove uncertainty» la deja sin incertidumbre.

Prueba: un fichero con una columna que incumple una regla debe abrirse dando el aviso, y el sorteo debe negarse.

### Error 77 · [Corregido] El análisis probabilístico de sensibilidad no se puede repetir: cada columna se sortea con un generador nuevo sembrado con el reloj

| | |
|---|---|
| **Estado** | Corregido en el commit `a990fac` (09-10-2026): los potenciales se sortean con el generador que reciben (`Potential.sample(Random)`), el análisis crea uno para todas sus simulaciones y `VECEPSA.setSeed` fija su semilla. El diálogo del análisis de sensibilidad tiene un campo «Seed» con una ayuda emergente; mientras hay semilla escrita, la casilla de varios hilos se desmarca y se desactiva. Medido antes del arreglo: correlación de 0,0125, 0,0155 y 0,0073 entre dos columnas independientes en 200.000 sorteos (límite de tres errores típicos: 0,0067); con el arreglo queda dentro del límite. Queda sin tocar: con varios hilos el análisis sigue sin poder repetirse (nota 103), y la ventana del análisis probabilístico no usa varios hilos aunque la casilla esté marcada |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.modelUncertainty |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/Sampler.java:162-179`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/TablePotentialSampler.java:85-88`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/XORShiftRandom.java`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:566-575`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VECEPSA.java:105-113`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CEProbabilisticDialog.java:152-164` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |

| **Relacionados** | error 76, error 136, nota 58, nota 103, nota 104, nota 110 |
|---|---|

**Qué pasa.** Dos ejecuciones del análisis probabilístico de coste-efectividad sobre el mismo modelo nunca dan los mismos resultados, y no hay forma de volver a obtener una ejecución concreta ni de escribir una prueba de regresión con números. Además, las columnas sorteadas una tras otra salen ligeramente correlacionadas, aunque el modelo declare sus parámetros independientes.

Camino de llegada: menú de análisis de sensibilidad, análisis probabilístico sobre una red de coste-efectividad con parámetros inciertos (`CEProbabilisticDialog` → `VECEPSA.resolve` → `sample NetworkPotentials` → `TablePotential.sample`).

`Sampler.generateSample` pide el generador de números aleatorios dentro del método (`Random randomGenerator = createRandomGenerator();`), y `TablePotentialSampler.createRandom Generator` devuelve `new XORShiftRandom()`, cuyo estado inicial es `System.nanoTime()`. Es decir: cada columna de cada tabla de cada simulación usa un generador nuevo sembrado con el reloj.

`TablePotential.sample()` no recibe generador ni semilla, `VECEPSA` no tiene ningún método para fijarla y `CEProbabilisticDialog` no la pide. La propagación estocástica (`StochasticPropagation`) sí admite semilla, así que el proyecto ya tiene el patrón.

La correlación viene de sembrar con el reloj generadores que se crean con nanosegundos de diferencia: XORShift es lineal, y estados iniciales cercanos dan salidas emparentadas.

Medido con pequeños programas de prueba:

- Repetición: 3 ejecuciones de 3 sorteos de una Beta(3,7); las nueve extracciones son distintas entre ejecuciones.
- Correlación: una tabla de cuatro columnas, cada una Beta(3,7) más complemento, 200.000 sorteos del potencial entero, repetido 3 veces. Las medias (0,299-0,301) y desviaciones (0,137-0,138) coinciden con las de la Beta, pero la correlación entre columnas contiguas sale entre 0,011 y 0,024 en las nueve medidas (el error típico con independencia es 0,0022, así que son de 5 a 11 errores típicos, siempre positivas), y 0,009-0,011 entre la primera y la cuarta.
- Control: la misma Beta extraída de un único generador sembrado una vez da correlaciones de 0,0007, 0,0009 y -0,0031.

Es un efecto pequeño, pero introduce correlación entre parámetros que el modelo declara independientes.

**El código.** core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/Sampler.java:162-168

```
protected double[] generateSample(FamilyDistribution otherFamily, DirichletFamily
↪  dirFamily,
        ComplementFamily complementFamily, int[] indexesOther, int[] indexesDirichlet,
↪  int[] indexesComplement,
        int numStates) {
    Random randomGenerator = createRandomGenerator();
    double[] sampledConfigurationValues = new double[numStates];
```

core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/TablePotentialSampler.java:85-88

```
@Override
protected Random createRandomGenerator() {
    return new XORShiftRandom();
}
```

**Cómo arreglarlo.** Un único generador por análisis, creado una vez con una semilla que se pueda fijar (y mostrar), pasado hacia abajo:

- `Potential.sample(Random)` en lugar de `sample()`, y sus redefiniciones en `TablePotential`, `ExactDistrPotential`, `TreeADDPotential` y cualquier otra subclase que sortee;
- `Sampler.generateSample` recibiéndolo como parámetro.

`SystematicSampling.createRandomGenerator` devuelve hoy `null` y no lo usa, así que el cambio de firma no le afecta.

Si se usan varios hilos (`VECEPSA.setUseMultithreading`), cada hilo necesita su propio generador derivado de la semilla, además de su propia red; es el problema de varios hilos que describe nota 103, donde hoy no hay nada que corregir.

Pruebas:

- Dos análisis con la misma semilla devuelven exactamente los mismos resultados.
- La correlación entre dos columnas independientes sorteadas muchas veces queda dentro de tres errores típicos de cero.

### Error 78 · [Corregido] El diálogo de incertidumbre rechaza cualquier Dirichlet sobre dos o más estados, incluidas las que traen las redes del repositorio

| | |
|---|---|
| **Estado** | Corregido en el commit `8c46ebe` (09-10-2026), que el documento no recogió hasta el 17-10-2026: la regla 3 exige al menos dos Dirichlet. El mismo día, el commit `1a8cfe7` llevó las reglas de familia a `core` (`DistributionsOfAColumn`, ver error 79), y su prueba cubre los casos de abajo: la columna de `MID-Chancellor-corrected` (tres Dirichlet y un 0) se acepta; una Dirichlet sola con ceros, una Dirichlet junto a un complemento y una junto a una beta se rechazan. Medido el 17-10-2026 con el diálogo construido de verdad sobre `MID-Chancellor-corrected.pgmx` y «OK» pulsado desde el código: las dos columnas con Dirichlet de «State [1]» se aceptan; con el código anterior a `8c46ebe`, las dos se rechazan con `Rule3Broken`. `doVerifyRule4` sigue sin que nadie la llame. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/UncertainValuesDialog.java:521-549,578-583,649-674,676-684`<br>`gui/src/main/resources/gui/localize/gui_class_localizations_en.xml:53-56`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TablePotentialPanel.java:701-724`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/OkCancelDialog.java:68-79` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 79 |

**Qué pasa.** En OpenMarkov una distribución de Dirichlet sobre una columna de una tabla de probabilidad se escribe poniendo en cada casilla una `DirichletFunction` con su parámetro alfa: una Dirichlet sobre tres estados son tres casillas Dirichlet. Así la escriben las redes del repositorio: `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx` tiene columnas `Dirichlet 15, Dirichlet 512, Dirichlet 731, Exact 0` y `Dirichlet 17, 116, 350, 1251`.

El diálogo de incertidumbre no acepta ninguna de esas columnas. Camino del usuario: abrir `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`, abrir la tabla del nodo que lleva esa incertidumbre, clic derecho sobre una de esas columnas → «Edit uncertainty» y pulsar «OK» sin tocar nada. `OkCancelDialog` envuelve la excepción en `UnrecoverableException`, el gestor global muestra el mensaje de la regla 3 y el diálogo no se cierra; la única salida es «Cancel». Tampoco se puede crear desde cero una Dirichlet sobre varios estados con «Assign uncertainty».

Al pulsar «OK», `UncertainValuesDialog.doOkClickBeforeHide` comprueba las reglas de familia 1, 2 y 3. La regla 3 cuenta las casillas Dirichlet: con cero no hace nada; con una exige que todas las demás sean exactas con valor 0; con dos o más **lanza siempre** `Rule3Broken`.

El mensaje que ve el usuario dice lo contrario: «If one of the distributions is a Dirichlet, then: all the others must be Exact with v = 0 or Dirichlet. At least one of the others must also be a Dirichlet». Es decir, el mensaje describe justamente la columna que el código rechaza, y el código solo acepta una Dirichlet de una sola casilla (que siempre vale 1 y no tiene incertidumbre).

Medido con un pequeño programa de prueba que llama a la verificación real: 3 Dirichlet → `Rule3Broken`; 2 Dirichlet → `Rule3Broken`; 2 Dirichlet + exacta a 0 → `Rule3Broken`; 1 Dirichlet + 2 exactas a 0 → aceptada; beta + complemento → aceptada.

En la misma clase, `doVerifyRule4` (rechaza una columna formada solo por complementos) no la llama nadie, cosa que el propio código anota con `@ToCheck`; no se ha comprobado qué consecuencia tiene aceptar esa columna.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/UncertainValuesDialog.java:649-674

```
private static boolean doVerifyRule3(FamilyDistribution family) throws
↪  FamilyDistributionRuleBrokenException.Rule3Broken {
    List<UncertainValue> uncertainFamily = family.getFamily();
    int totalSizeFamily = uncertainFamily.size();
    List<UncertainValue> dirUncertain = getUncertainValuesOfClass(uncertainFamily,
    ↪  DirichletFunction.class);
    int numDirichlet = dirUncertain.size();
    switch (numDirichlet) {
        case 0 -> {
        }
        case 1 -> {
            List<UncertainValue> exactUncertain =
            ↪  getUncertainValuesOfClass(uncertainFamily, ExactFunction.class);
            int numExact = exactUncertain.size();
            boolean verify = (
                    (numExact + numDirichlet == totalSizeFamily) && areAllZero(
                            new FamilyDistribution(exactUncertain).getMean())
            );
            if (!verify) {
                throw new FamilyDistributionRuleBrokenException.Rule3Broken(family);
            }
        }
        default -> {
            throw new FamilyDistributionRuleBrokenException.Rule3Broken(family);
        }
    }
    return true;
}
```

**Cómo arreglarlo.** Reescribir la regla 3 según su mensaje: si hay alguna casilla Dirichlet, todas las demás deben ser Dirichlet o exactas con valor 0, y debe haber al menos dos Dirichlet (`num`

`Dirichlet >= 2 && numDirichlet + numExactasACero == total`). El caso de una sola Dirichlet pasa a ser inválido, como dice el texto. El mensaje en `gui_class_localizations_en.xml:53-54` ya describe esa regla.

Si las reglas de familia se mueven a `core` para aplicarlas también al leer o al muestrear (ver error 79), la regla corregida es la que debe moverse.

Queda por ver, de paso, si `doVerifyRule4` debe llamarse o borrarse.

Pruebas de regresión:

- Una prueba de la verificación de familia con las columnas «3 Dirichlet» y «2 Dirichlet + exacta a 0» (aceptadas), y «1 Dirichlet + exactas a 0», «Dirichlet + complemento» y «Dirichlet + beta» (rechazadas).
- Otra que abra las columnas de `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx` y compruebe que pasan la verificación.

### Error 79 · [Corregido] El editor de incertidumbre y el muestreador reparten de forma distinta la probabilidad del complemento cuando una columna mezcla Dirichlet y complemento

| | |
|---|---|
| **Estado** | Corregido por los commits `61da656` y `059e816` (09-10-2026), con la primera opción, que Manuel Arias confirmó el 15-10-2026: una columna que mezcla Dirichlet y complemento no es válida. Las reglas de familia están ahora en `core` (`DistributionsOfAColumn`); al abrir una red, una ventana enumera las columnas cuyas distribuciones no pueden ir juntas, y los cinco análisis que muestrean una red (`VECEPSA`, `VESensAnTornadoSpider`, `VECESensAnSpider`, `VESensAnPlot` y `VESensAnMap`) se niegan a empezar con ellas (`DistributionsOfAColumn.mustGoTogetherIn`). Comprobado el 15-10-2026 que no hay otro código del producto que muestree la incertidumbre de una red. Las dos fórmulas que describe la ficha siguen siendo distintas, pero ya no llega a ninguna de ellas una columna así. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.modelUncertainty |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/Sampler.java:162-179`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/UncertainValuesDialog.java:253-289`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/TablePotentialSampler.java:38-80`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/SystematicSampling.java:216-217,292-293` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si una columna que mezcla Dirichlet con complemento es válida (y entonces qué masa recibe el complemento) o se rechaza también al leer el fichero |
| **Relacionados** | error 78 |

**Qué pasa.** Una columna de una tabla de probabilidad condicionada puede llevar incertidumbre: cada casilla tiene una distribución (exacta, beta, Dirichlet, complemento…). Si una columna mezcla Dirichlet y complemento, el análisis de sensibilidad probabilístico muestrea una columna que suma más de 1, sin aviso.

El editor no deja escribir esa columna en un nodo de azar, porque su regla 3 la rechaza (ver error 78). Pero el muestreador no mira las reglas del editor ni las valida nadie al leer: un `.pgmx` escrito a mano o por otro programa con una columna así llega al análisis. En las redes del repositorio no

hay ninguna columna con esa mezcla: las de `0_miscellaneous/networks/mid/MID-Chancellor-*.pgmx` tienen Dirichlet y complemento en el mismo potencial, pero en columnas distintas.

De una columna incierta salen números de dos maneras:

- El editor (`UncertainValuesDialog.calculateReferenceValues`, al aceptar el diálogo de incertidumbre) calcula los valores de referencia de la tabla: cada distribución en su media, las casillas Dirichlet normalizadas entre sí, y los complementos repartiéndose `1 - (suma de las medias de las «otras» + suma de las Dirichlet)`.
- El muestreador que usan los análisis de sensibilidad (`Sampler.generateSample`, llamado desde `TablePotentialSampler.sample` en el análisis probabilístico y desde `Systematic Sampling` en tornado y araña) reparte entre los complementos `1 - (suma de las «otras»)`, sin descontar lo que se llevan las Dirichlet.

Solo difieren en columnas que mezclan Dirichlet y complemento. Como la regla 3 del editor rechaza esa columna, la fórmula del editor nunca se aplica a ese caso.

Medido con un pequeño programa de prueba sobre la columna [Dirichlet(2), Complemento(1)]: el editor da `[1.0, 0.0]` (suma 1) y el muestreador da `[1.0, 1.0]` (suma 2). Con una columna solo de Dirichlet los dos coinciden en forma.

**El código.** core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/Sampler.java:170-176

```
double[] sampleDir = getSample(dirFamily, randomGenerator);
placeInArray(sampledConfigurationValues, indexesDirichlet, sampleDir);
// Process complements
double massForComp = 1.0 - (Tools.sum(sampleOther));
complementFamily.setProbMass(massForComp);
double[] sampleComp = complementFamily.getSample();
```

gui/src/main/java/org/openmarkov/gui/dialog/node/UncertainValuesDialog.java:275-281

```
double[] meanDir = dir.getMean();
placeInArray(refValues, dirichletIndexes, meanDir);
// Process complements
double massForComp = 1.0 - (Tools.sum(meanOther) + Tools.sum(meanDir));
comp.setProbMass(massForComp);
double[] meanComp = comp.getMean();
```

**Cómo arreglarlo.** Lo primero es la decisión de equipo:

- Si esa mezcla es inválida (que es lo que dice hoy la regla 3 del editor), hay que comprobar las mismas reglas al leer o antes de muestrear, y avisar.
- Si es válida, las dos fórmulas deben ser la misma, y lo natural es que vivan en un único sitio de `core` (por ejemplo, en `Sampler` o `FamilyDistribution`) al que llamen tanto el editor como el muestreador, de modo que no puedan volver a divergir.

Las reglas de familia hoy viven solo en el diálogo (`gui`), así que moverlas a `core` es lo que permite aplicarlas en el muestreo y en la lectura.

Prueba de regresión: para un conjunto de columnas (exacta+complemento, beta+complemento, solo Dirichlet, Dirichlet+exactas a 0 y, según lo decidido, Dirichlet+complemento), exigir que la media que calcula el editor y la esperanza del muestreador coincidan y que ninguna columna aceptada sume más de 1.

### Error 80 · [Corregido] El mapa de dos parámetros pinta como mejor una opción que no lo es cuando la decisión tiene tres o más opciones

| | |
|---|---|
| **Estado** | Corregido en el commit `1d4189b` (14-10-2026): la opción ganadora de cada casilla la elige `MapDialog.bestOption`, que lleva la mejor utilidad vista hasta el momento y declara empate solo cuando otra opción la iguala. Con prueba, sin pantalla, de los casos de esta ficha y de otros cuatro: con la lógica anterior fallan tres de los nueve. Comparación con `==`, como antes; no se ha introducido tolerancia. |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:270-298` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 81, error 82, error 83 |

**Qué pasa.** El análisis de sensibilidad de tipo «Map (two-way analysis)» (análisis de dos parámetros a la vez) dibuja una rejilla: cada casilla es una combinación de valores de los dos parámetros inciertos elegidos. Cuando el ámbito del análisis es una decisión («Scope: decision»), cada casilla se pinta del color de la opción de esa decisión que da mayor utilidad esperada en ese punto, o del color de «draw» (empate) si hay empate entre las mejores.

Con dos opciones el resultado es correcto. Con tres o más, gana la última opción que supere a la primera, aunque no sea la mejor, y se declara empate en cuanto cualquier opción iguala a la primera, aunque otra las supere a las dos.

Camino del usuario: con una red de decisión abierta que tenga dos parámetros inciertos con nombre, menú de herramientas → «Sensitivity analysis», aceptar las opciones de inferencia, elegir el tipo «Map (two-way analysis)», ámbito de decisión y una decisión con tres o más opciones.

En las redes de ejemplo del repositorio las decisiones con dos parámetros inciertos con nombre son de dos opciones, así que no se ha visto con una red real; la consecuencia (color de una opción equivocada) se ha comprobado sobre los datos que la gráfica recibe. Hasta los commits `2a88410` y `f2898ff` la consecuencia visible quedaba además tapada por otros dos defectos del mismo mapa: toda la zona de datos salía de un solo color (error 82) y las opciones tercera y siguientes salían en gris (error 81).

El método privado `MapDialog.getMapChart()` busca la opción ganadora así: guarda en `zAxisValue` la utilidad de la primera opción y recorre las demás. Pero `zAxisValue` no se actualiza cuando aparece una opción mejor, de modo que cada opción se compara con la primera y no con la mejor hasta ese momento.

Medido con un pequeño programa de prueba que construye la gráfica del mapa (sin abrir la ventana) con una decisión de tres opciones `a`, `b`, `c` y una casilla con utilidades escritas a mano. El valor que se pinta es el índice de la opción ganadora (−1 es empate):

```
utilidades [1, 5, 3] -> se pinta 2 (opción c)     debería ser 1 (opción b)
utilidades [1, 5, 1] -> se pinta -1 (empate)      debería ser 1 (opción b)
utilidades [1, 1, 5] -> se pinta 2 (opción c)     correcto
utilidades [5, 5, 3] -> se pinta -1 (empate)      correcto
```

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:274-297

```
double zAxisValue = uncertainParameterPotential.getValues()[dataPosition];
...
if (decisionVariable != null) {
    int greatUtilityStateIndex = 0;
    for (int decisionVariableStateIndex = 1;
         decisionVariableStateIndex < decisionVariableStates; decisionVariableStateIndex++)
↪ {
        int dataPositionForThatDecision = dataPosition + (
                decisionVariableStateIndex * totalIterations * totalIterations
        );
        double zAxisValueForThatDecision = uncertainParameterPotential
                .getValues()[dataPositionForThatDecision];
        if (zAxisValueForThatDecision > zAxisValue) {
            greatUtilityStateIndex = decisionVariableStateIndex;
        } else if (zAxisValueForThatDecision == zAxisValue) {
            greatUtilityStateIndex = -1;
        }
    }
    zAxisValue = greatUtilityStateIndex;
}
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*, que es lo que se proponía. Llevar en el bucle el mejor valor visto hasta el momento (actualizarlo cuando una opción lo supera) y declarar empate solo cuando una opción iguala a ese mejor valor; si luego otra lo supera, el empate desaparece.

Conviene separar la búsqueda de la opción ganadora en un método estático que reciba el vector de utilidades de una casilla y devuelva el índice o $-1$, para poder probarlo sin la ventana.

Es el único sitio del módulo que decide el ganador de una casilla del mapa (los demás diálogos —tornado, gráfica de un parámetro, araña— no pintan ganadores por casilla). Si se decide comparar con tolerancia en vez de con `==`, debe hacerse igual en la comparación de «mayor» y en la de «igual».

Prueba de regresión: el método extraído con `{1,5,3}` → 1, `{1,5,1}` → 1, `{1,1,5}` → 2, `{5,5,3}` → $-1$, `{3,5,5}` → $-1$.

### Error 81 · [Corregido] En el mapa de dos parámetros el empate y las opciones tercera y siguientes salen en gris, no del color que enseña la leyenda

| | |
|---|---|
| **Estado** | Corregido en el commit `f2898ff` (14-10-2026): la escala se construye con sus límites, `new LookupPaintScale(-1.0, decisionVariableStates - 1, ...)`, con el blanco del fondo del gráfico para lo que quede fuera, que no se da. Con prueba que abre el mapa de `ID-decide-test` con ámbito de decisión y exige que cada entrada de la leyenda tenga en la escala su mismo color (necesita pantalla). Sin el arreglo, el empate da 192,192,192 en vez de 255,85,85. La tercera opción no se ha probado con una red, porque ninguna del repositorio tiene una decisión de tres opciones con dos parámetros inciertos con nombre; pasa por la misma escala. |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:352-381` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

| **Relacionados** | error 74, error 80, error 82, error 83 |
|---|---|

**Qué pasa.** En el mapa de dos parámetros con ámbito de decisión, la leyenda dice que el empate es rojo y que la tercera opción es amarilla, pero en el mapa los dos salen del mismo gris claro, indistinguibles entre sí. El empate gris afecta también a decisiones de dos opciones, que son las de las redes de ejemplo; la tercera opción gris, solo a decisiones de tres o más.

Camino del usuario: menú de herramientas → «Sensitivity analysis», tipo «Map (two-way analysis)», ámbito de decisión. Hasta el commit `2a88410` la consecuencia visible quedaba además tapada porque la zona de datos salía entera de un color (error 82).

Cada casilla del mapa recibe un número: el índice de la opción ganadora (0, 1, 2…) o −1 si hay empate (ver error 80). Para convertir ese número en color, `MapDialog.getMapChart()` crea una `LookupPaintScale` de JFreeChart (la biblioteca de gráficas) y le añade un color por cada valor, el mismo que pone en la leyenda.

El constructor sin argumentos de `LookupPaintScale` fija una escala que solo cubre de 0,0 a 1,0 y usa gris claro (192,192,192) para todo lo que cae fuera. Por eso las entradas añadidas para −1 (empate) y para 2, 3… (tercera opción en adelante) quedan fuera del rango y se pintan en gris claro.

Medido con un pequeño programa de prueba que construye la gráfica del mapa (sin abrir la ventana) con una decisión de tres opciones, y le pregunta a la escala de qué color pinta cada valor:

```
escala de 0.0 a 1.0, color por omisión 192,192,192
leyenda «draw»   255,85,85     escala para -1.0: 192,192,192
leyenda «D = a»   85,85,255    escala para  0.0:  85,85,255
leyenda «D = b»   85,255,85    escala para  1.0:  85,255,85
leyenda «D = c»  255,255,85    escala para  2.0: 192,192,192
```

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:355-381

```
DefaultDrawingSupplier defaultDrawingSupplier = new DefaultDrawingSupplier();
LegendItemCollection chartLegend = new LegendItemCollection();
LookupPaintScale paintScale = new LookupPaintScale();
...
chartLegend.add(drawLegendItem);
paintScale.add(-1.0, drawLegendItem.getFillPaint());

for (int stateIndex = 0; stateIndex < decisionVariableStates; stateIndex++) {
    LegendItem legendItem = new LegendItem(
            decisionVariable.getName() + " = " + decisionVariable.getStateName(stateIndex),
↪   // Legend label
            ...
            defaultDrawingSupplier.getNextPaint()); // Fill paint
    paintScale.add(stateIndex, legendItem.getFillPaint());
    chartLegend.add(legendItem);
}
plot.setFixedLegendItems(chartLegend);
renderer.setPaintScale(paintScale);
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*, que es lo que se proponía. Construir la escala con sus límites reales: `new LookupPaintScale(-1.0, decisionVariableStates - 1, color)`, de modo que cubra desde el empate (−1) hasta el índice de la última opción (los dos límites están incluidos en la escala). El color por omisión debería ser uno que no se confunda con ninguna entrada de la leyenda.

Es el único sitio que crea esta escala (la rama de ámbito global usa `GrayPaintScale` con los límites

mínimo y máximo, que no tiene este problema). `DefaultDrawingSupplier` repite colores a partir de cierto número de opciones; no es parte de este fallo.

Prueba de regresión: construir la gráfica con una decisión de tres opciones (se puede hacer fabricando el diálogo sin ejecutar su constructor) y comprobar que `getPaint(z)` coincide con el color del elemento de la leyenda correspondiente para z = −1, 0, 1, 2.

### Error 82 · [Corregido] El mapa de dos parámetros pinta toda la zona de datos de un solo color porque cada casilla mide una unidad

| | |
|---|---|
| **Estado** | Corregido en el commit `2a88410` (14-10-2026): el cuadrado de cada casilla mide la distancia entre dos casillas (`setBlockWidth` y `setBlockHeight` con `hVariationInterval` y `vVariationInterval`). Con prueba que abre el diálogo sobre `ID-decide-test` (prevalence y sensitivity, 25 % y 50 puntos), pinta el mapa en una imagen y mira el color del centro de cada casilla (necesita pantalla). Sin el arreglo, 15 de las 2601 casillas salen de otro color en ámbito de decisión (las 15 en que gana «no») y 2573 en el global; con él, ninguna, y los ejes van de −0,255 a 0,255. |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:211-246`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:339-341` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 80, error 81, error 83, error 90 |

**Qué pasa.** El análisis de sensibilidad «Map (two-way analysis)» dibuja la zona de datos entera del color de una sola casilla, así que el mapa no enseña nada. Además, los ejes quedan de −0,75 a 0,75, con media unidad vacía a cada lado de los datos. Afecta a los dos ámbitos, el de decisión y el global (el global pinta una escala de grises con el mismo renderizador).

Camino del usuario: menú de herramientas → «Sensitivity analysis», tipo «Map (two-way analysis)», con cualquier red que tenga dos parámetros inciertos con nombre.

`MapDialog.getMapChart()` dibuja el mapa con un `XYBlockRenderer` de JFreeChart, que pinta un rectángulo («bloque») centrado en cada punto de datos. El tamaño del bloque se da en unidades de los datos y por omisión es 1 × 1. `MapDialog` no lo cambia nunca.

Las coordenadas del mapa son la variación de cada parámetro. Para las variaciones en porcentaje o en razón (todas menos «UDIN», intervalo definido por el usuario) van de −v/100 a +v/100: con la variación por omisión del 25 %, de −0,25 a 0,25, y con 50 puntos por parámetro (valor por omisión) las casillas están a 0,01 unas de otras.

Cada bloque mide entonces 1 × 1, cien veces más que la separación, y cualquier bloque centrado en un punto entre −0,25 y 0,25 cubre por completo el cuadrado de datos. Como los bloques se pintan en orden, el último (la esquina superior derecha) tapa a todos. JFreeChart estira los ejes para que quepa el bloque, y de ahí el rango de −0,75 a 0,75.

Con el tipo «UDIN» la separación depende de los límites que escriba el usuario; con los de omisión (0 a 1) los bloques también se solapan mucho.

Medido con un pequeño programa de prueba que construye la gráfica del mapa sin abrir la ventana, con 5 × 5 casillas (4 intervalos, variación del 25 %) y una decisión de dos opciones: en las tres

columnas de la izquierda gana «no» y en las dos de la derecha gana «yes». Pintada en una imagen de 800 × 800 y leyendo el color en el centro de cada casilla:

```
anchura del bloque 1.0, altura 1.0
eje horizontal de -0.75 a 0.75, eje vertical de -0.75 a 0.75
las 25 casillas, esperado «no» (azul 85,85,255) en x = -0.25, -0.125, 0 y «yes» (verde) en
↪  x = 0.125, 0.25:
todas leen 85,255,85 (verde)
```

Da lo mismo con la versión 1.5.6 de JFreeChart que declara el `pom.xml` y, en ámbito de decisión, con el jar ejecutable publicado; con ese jar el mapa de ámbito global no llega a dibujarse (error 74).

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:221-222, 245-246, 339-341

```
// Size of every horizontal interval
double hVariationInterval = (hMaxVariationValue - hMinVariationValue) / iterations;
...
// Size of every vertical interval
double vVariationInterval = (vMaxVariationValue - vMinVariationValue) / iterations;
...
XYBlockRenderer renderer = new XYBlockRenderer();

XYPlot plot = new XYPlot(dataset, xAxis, yAxis, renderer);
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*, que es lo que se proponía. Tras crear el renderizador, darle el tamaño de una casilla: `renderer.setBlockWidth(hVariationInterval)` y `renderer.setBlockHeight(vVariationInterval)`; las dos variables ya están calculadas. Con eso los bloques se tocan sin solaparse y los ejes llegan medio intervalo más allá del primer y el último punto. Sirve para los dos ámbitos, porque el renderizador es común.

No hay otro sitio del módulo que use `XYBlockRenderer`.

Prueba de regresión: construir la gráfica con un patrón conocido (media rejilla gana una opción, media otra), pintarla en una `BufferedImage` sin pantalla y comprobar el color en el centro de cada casilla; comprobar también que el rango del eje horizontal es [mín − intervalo/2, máx + intervalo/2].

### Error 83 · [Corregido] El mapa de dos parámetros no se abre cuando gana la misma opción en todas las casillas y no es la primera, o cuando la utilidad es la misma en todo el mapa

| | |
|---|---|
| **Estado** | Corregido en el commit `6fa4e26` (09-10-2026): el eje de la escala se crea solo en el ámbito global, `MapDialog.scaleRange` abre la escala a los dos lados cuando la utilidad es la misma en todo el mapa, y el máximo arranca en menos infinito. Con prueba del rango y, desde el commit `6ae9b16`, del diálogo entero en los cuatro casos de esta ficha (necesita pantalla). Medido con JFreeChart 1.5.6 sola: sin el arreglo, «Do test?» da `Range[1.0,1.0]`; con él, los cuatro se construyen. |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:248-249`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:300-308`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:349-350` |

| | |
|---|---|
| | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:384` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 80, error 81, error 82, error 88, error 90 |

**Qué pasa.** El análisis de sensibilidad «Map (two-way analysis)» con ámbito de decisión no se abre cuando en todas las casillas del mapa gana la misma opción y esa opción no es la primera de la decisión: sale el aviso de error inesperado con `IllegalArgumentException: A positive range length is required: Range[1.0,1.0]`. Es un caso corriente: basta con que la mejor opción no cambie dentro de los intervalos de los dos parámetros y que no sea la primera (que suele ser «no»).

Camino del usuario: abrir `0_miscellaneous/networks/id/ID-decide-test.pgmx`, menú Tools → «Sensitivity analysis», aceptar las opciones de inferencia, tipo «Map (two-way analysis)», parámetros «cost of test» y «cost of therapy» con la variación y los puntos por omisión (25 % y 50 puntos), ámbito de decisión, decisión «Do test?».

En ámbito global pasa lo mismo cuando la utilidad esperada es igual y positiva en todas las casillas, por ejemplo con dos parámetros que no influyen en el resultado.

Así se comporta con la versión de JFreeChart que declara el proyecto (1.5.6), que es la que se usa desde el IDE. Con el jar ejecutable publicado, que lleva dentro las clases de JFreeChart 1.0.12 (error 74), el ámbito de decisión no falla, porque esa versión no comprueba la longitud del rango; el ámbito global falla igual, antes: con utilidades iguales y positivas, en `new GrayPaintScale` (línea 384, `Requires lowerBound < upperBound`), y con cualquier otra, por el `NoSuchMethodError` de error 74. Al arreglar error 74, el jar pasa a fallar también en ámbito de decisión.

Por qué pasa: `MapDialog.getMapChart` guarda en cada casilla, en ámbito de decisión, el índice de la opción ganadora (−1 si hay empate), y en ámbito global la utilidad. Mientras recorre las casillas lleva el menor y el mayor valor (`minRangeUtility`, `maxRangeUtility`), y después crea el eje de la escala y le da ese rango con `scaleAxis.setRange(minRangeUtility, maxRangeUtility)` (línea 350). Lo hace también en ámbito de decisión, donde ese eje no se usa. JFreeChart 1.5.6 rechaza un rango de longitud cero.

Si en todas las casillas gana la opción de índice k ≥ 1, el mínimo y el máximo valen k. Con la primera opción (0) o con empate (−1) en todo el mapa no falla, pero solo porque el máximo arranca en `Double.MIN_VALUE`, que es un positivo diminuto y no el número más negativo (el mismo inicio de error 88 y error 90): el rango queda de 0, o de −1, a ese número. Por lo mismo, en ámbito global una utilidad constante negativa o cero no falla, y una positiva sí.

Medido construyendo la gráfica sin pantalla con los datos reales de la tarea `VESensAnMap` sobre `0_miscellaneous/networks/id/ID-decide-test.pgmx` (51 × 51 casillas, variación del 25 %) y JFreeChart 1.5.6:

```
cost of test x cost of therapy, decisión Do test?: «yes» en 2601 casillas     ->
↪  IllegalArgumentException (MapDialog.java:350)
cost of test x cost of therapy, decisión Therapy:  «no» en 2601 casillas      -> se
↪  construye
prevalence x sensitivity, decisión Do test?:       «no» en 15, «yes» en 2586  -> se
↪  construye
```

Con datos escritos a mano: tres opciones con la tercera ganadora en todas las casillas → `Range[2.0,2.0]`; la primera o empate en todas → se construye; ámbito global con todas las utilidades a 7 → `Range[7.0,7.0]`; a −7 o a 0 → se construye.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:248-249, 300-308, 349-350, 384

```
double minRangeUtility = Double.MAX_VALUE;
double maxRangeUtility = Double.MIN_VALUE;
...
                // Update minimum utility found
                if (zAxisValue < minRangeUtility) {
                    minRangeUtility = zAxisValue;
                }
                // Update maximum utility found
                if (zAxisValue > maxRangeUtility) {
                    maxRangeUtility = zAxisValue;
                }
...
NumberAxis scaleAxis = new
↪  NumberAxis(stringDatabase.getString("SensitivityAnalysis.General.Scale"));
scaleAxis.setRange(minRangeUtility, maxRangeUtility);
...
    PaintScale paintScale = new GrayPaintScale(minRangeUtility, maxRangeUtility);
```

**Cómo arreglarlo.** Crear el eje de la escala y darle rango solo en la rama global, que es donde se usa (para `PaintScaleLegend`). Allí, si el mínimo y el máximo coinciden, abrir el rango un poco a los dos lados antes de dárselo al eje y a `GrayPaintScale`, que tampoco admite un rango vacío.

Este arreglo debe ir junto con el del inicio del máximo: error 88 propone arrancarlo con `Double.NEGATIVE_INFINITY` también en `MapDialog` (línea 249), y con ese cambio solo, un mapa donde gana la primera opción o hay empate en todas las casillas, o una utilidad global constante negativa o cero, pasarían a fallar igual.

En ámbito de decisión la escala de colores es una `LookupPaintScale`, que no necesita el eje; su rango propio es el de error 81.

Prueba de regresión: construir la gráfica sin pantalla (o extraer el cálculo del rango a un método sin interfaz) con una rejilla en la que gane siempre la segunda opción, otra en la que gane siempre la primera, otra con empate en todas las casillas y, en ámbito global, con utilidad constante positiva, cero y negativa: en todos los casos debe construirse.

## Error 84 · [Corregido] La tabla de intervenciones fronterizas pone el nombre de otra opción cuando dos opciones tienen el mismo coste y la misma efectividad

| | |
|---|---|
| **Estado** | Corregido en el commit `8f87658` (14-10-2026): el cálculo de la frontera (`CEDecisionResults.frontierOptions`) devuelve la posición de cada opción elegida en la decisión, y la tabla la usa para escribir el nombre en vez de buscar la partición con `indexOf`. La gráfica del plano coste-efectividad usa el mismo cálculo y solo toma los números. Comprobado con una prueba de dos particiones iguales, no en la ventana |
| **Módulo** | costEffectiveness |
| **Paquete** | org.openmarkov.costEffectiveness |
| **Ficheros** | `costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:500-508,568-611`<br>`core/src/main/java/org/openmarkov/core/model/network/CEP.java:480-485`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEBaseOperations.java:395-409` |

| | |
|---|---|
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 85, error 86 |

**Qué pasa.** La ventana de coste-efectividad con ámbito de decisión (`CEDecisionResults`) tiene una pestaña de intervenciones fronterizas: las opciones de la decisión que forman la frontera eficiente, con su coste, su efectividad y la razón coste-efectividad incremental (ICER). Cuando dos opciones dan exactamente el mismo coste y la misma efectividad, la frontera contiene la segunda y la fila se titula con el nombre de la primera. Los números son los mismos, pero la tabla nombra una opción que el análisis no eligió.

Camino del usuario: diagrama de influencia o modelo de Markov con criterio de coste-efectividad, menú de herramientas → «Cost-effectiveness analysis», ámbito de decisión, pestaña de intervenciones fronterizas, cuando dos opciones de la decisión dan exactamente el mismo coste y la misma efectividad (por ejemplo, opciones que no afectan a ninguna utilidad). Es un caso poco frecuente y la consecuencia es un nombre equivocado, no un número equivocado. No se ha buscado una red de ejemplo del repositorio que lo muestre.

`calculateFrontierInterventions` calcula la frontera con `CEBaseOperations.deterministicCEA` y devuelve, por identidad (`==`), los objetos `CEP` (las particiones de coste-efectividad) de las opciones elegidas. Después, `getFrontierInterventionsTable` tiene que saber de qué opción es cada uno para escribir su nombre, y lo busca con `Arrays.asList(cepsForDecision).indexOf(...)`, que compara con `equals`.

`CEP.equals` compara por valor (umbrales, costes y efectividades; es una redefinición real desde el commit `325e5c6`). Si dos opciones tienen particiones iguales, `indexOf` devuelve siempre la primera. Y `deterministicCEA`, ante costes iguales, se queda con la última opción (condición `i > 0` de la línea 403).

Medido con un pequeño programa de prueba sobre dos particiones construidas a mano (coste 0 y efectividad 0 para «no» y para «yes»), usando los métodos reales de la ventana fabricada sin constructor:

```
mismo objeto: false  equals: true  indexOf(segunda): 0
deterministicCEA elige: yes (índice 1)
la partición de la frontera es el objeto de «yes»: true
tabla de la frontera: 1 fila, nombre de la fila 0: no
```

**El código.** costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:500-508

```
public JTable getFrontierInterventionsTable() {

    List<CEP> frontierInterventions =
    ↪  calculateFrontierInterventions(AnalysisTab.FRONTIER_INTERVENTIONS);
    String[] interventionsNames = new String[frontierInterventions.size()];
    List<CEP> cepsForDecisionList = Arrays.asList(cepsForDecision);
    for (int i = 0; i < frontierInterventions.size(); i++) {
        int stateIndex = cepsForDecisionList.indexOf(frontierInterventions.get(i));
        interventionsNames[i] = decisionVariable.getStateName(stateIndex);
    }
```

**Cómo arreglarlo.** Que `calculateFrontierInterventions` devuelva también el índice del estado de cada intervención (ya lo tiene: `selectedIndices.get(j)`, línea 584), por ejemplo devolviendo índices en vez de particiones, y que la tabla use ese índice en lugar de buscarlo.

Hay otro llamador de `calculateFrontierInterventions` (línea 930, la línea de la frontera en el plano), que solo usa costes y efectividades y habría que adaptar si cambia el tipo devuelto. No hay más búsquedas por valor de particiones en el módulo.

Prueba de regresión: dos particiones iguales, como en la medida de arriba, comprobando que la fila se llama «yes».

### Error 85 · [Corregido] El análisis de coste-efectividad de un diagrama de influencia falla con NullPointerException cuando un hallazgo previo hace imposible una opción de una decisión y no las demás

| | |
|---|---|
| **Estado** | Corregido en el commit `af673f0` (08-10-2026): `CEBaseOperations.optimalCEP` compara solo las opciones cuya partición no es la de probabilidad cero, y conserva el índice de estado de cada una para su intervención. Con `ID-CEA-test-2therapies-new-test.pgmx` y un hallazgo en cada estado de «Test», el análisis global termina y da los mismos umbrales, costes y efectividades que fijando además la única opción posible de «Dec: Test». Los otros lectores de umbrales que nombra «Cómo arreglarlo» no se han tocado. Que una opción imposible cuya partición no es la cero siga compitiendo es el error 54. |
| **Módulo** | inference |
| **Paquete** | org.openmarkov.inference.algorithm.variableElimination.operation |
| **Ficheros** | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEBaseOperations.java:64-120`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEBaseOperations.java:609-661`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEBaseOperations.java:543-556`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEAlgebra.java:195-203`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEAlgebra.java:333-358`<br>`core/src/main/java/org/openmarkov/core/model/network/CEP.java:146-151` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 48, error 54, error 84, error 86, error 87, nota 46 |

**Qué pasa.** Con un hallazgo previo a la resolución sobre una variable de azar cuyo valor depende de una decisión, el análisis de coste-efectividad no se completa: sale el aviso de error inesperado con `NullPointerException: Cannot read the array length because "thresholds[i]" is null`. El caso típico es el resultado de una prueba, que vale «not done» cuando se decide no hacerla: anotar el resultado como ya conocido basta.

Camino del usuario: abrir `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx`; en modo edición, clic derecho sobre «Test» → «Add finding» y elegir cualquier estado; menú Tools → «Cost-effectiveness analysis», aceptar las opciones de inferencia y elegir el ámbito global. Con ámbito de decisión falla cuando la decisión elegida es anterior a aquella cuyas opciones deja imposibles el hallazgo y el escenario del diálogo no fija esa otra decisión: por ejemplo, en `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx` con un hallazgo en «Ga67 Tc99» y la decisión «Realizar Implante». En `ID-CEA-test-2therapies-new-test.pgmx`, el ámbito de «Dec: New test» no falla, porque su escenario fija «Dec: Test»; y con el de «Dec: Test» el análisis termina, pero la ventana recibe la partición de probabilidad cero de una opción y revienta al pintarla (error 86).

Por qué pasa. El análisis por eliminación de variables representa el resultado de cada configuración con una partición de coste-efectividad (`CEP`: para cada tramo de la disposición a pagar, la intervención óptima con su coste y su efectividad). Cuando una configuración tiene probabilidad cero, `CEAlgebra.multiplyAndMarginalize` pone en su lugar la partición de probabilidad cero (`CEP.getZeroPartition()`), un objeto sin tramos cuyos umbrales, costes, efectividades e intervenciones son `null`.

Al eliminar una decisión, `CEAlgebra.ceMaximize` pasa a `CEBaseOperations.optimalCEP` la partición de cada opción. `optimalCEP` solo contempla dos casos: que todas sean de probabilidad cero (devuelve la partición cero) o que no lo sea ninguna. Si alguna no es cero, llama a `getUnionThresholds(partitions, null)`, que lee los umbrales de todas, también los de la partición cero, y `getMinThreshold` falla al pedir `thresholds[i].length` (líneas 649-650). Más abajo haría lo mismo con `getIntervention`, `getCost` y `getEffectiveness`, y el umbral máximo lo toma de `partitions.get(0)`, que en la partición cero vale 0.

Un hallazgo sobre una variable que depende de una decisión deja con probabilidad cero las opciones incompatibles con él y no las demás: «Test = not done» hace imposible «Dec: Test = yes», y «Test = negative» o «positive» hacen imposible «Dec: Test = no». Esa mezcla es la que llega a `optimalCEP`.

Medido:

- `optimalCEP` con las particiones [normal, cero] y [cero, normal] lanza `NullPointerException` en `getMinThreshold`; con [cero, cero] devuelve la partición cero.
- Con `ID-CEA-test-2therapies-new-test.pgmx`, poniendo un hallazgo en cada estado de cada variable de azar y lanzando `VECEAnalysis` en ámbito global como lo hace el menú: los tres estados de «Test» fallan en `optimalCEP`; los demás hallazgos, no. En el ámbito de «Dec: Test» las particiones salen `[cep, zero]` («not done») o `[zero, cep]` («negative», «positive»). Sin hallazgos, la red se analiza bien.
- Con `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx`: los tres estados de «Ga67 Tc99» (la gammagrafía, que vale «Prueba no realizada» si no se hace) fallan igual en ámbito global, y también en el ámbito de «Realizar Implante» con un escenario para sus tres predecesores informativos («IMC», «Diabetes», «Alergia ATB»), como el que añade el diálogo.

**El código.** inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEBaseOperations.java:64-72

```
public static CEP optimalCEP(Variable decisionVariable, List<CEP> partitions) {
    CEP result;
    int i;
    for (i = 0; i < partitions.size() && partitions.get(i).isZero(); i++)
        ;
    if (i == partitions.size()) {
        result = CEP.getZeroPartition();
    } else {
        double[] allThresholds = getUnionThresholds(partitions, null);
```

inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/operation/CEBaseOperations.java:612-616, 648-650

```
for (int i = 0; i < indexes.length; i++) {
    // When probabilities = null, all partitions must be taken into account.
    if (probabilities == null || probabilities[i] != 0.0) {
        thresholds[i] = partitions.get(i).getThresholds();
...
```

```
for (int i = 0; i < indexes.length; i++) {
    if ((probabilities == null || probabilities[i] != 0.0)
            && indexes[i] < thresholds[i].length) {// Threshold list not empty
```

**Cómo arreglarlo.** Que las opciones cuya partición es de probabilidad cero no participen en la maximización: una opción incompatible con lo observado no se puede elegir. En `optimalCEP`, calcular los umbrales solo con las particiones que no son cero, hacer el análisis determinista de cada tramo (`deterministicCEA`) solo con esas opciones, conservando el índice de estado de cada una para construir su `StrategyTree`, y tomar el umbral máximo de una partición que no sea cero.

La misma regla (una partición cero no aporta umbrales ni valores) debe cumplirse en los otros lectores de umbrales: `getUnionThresholds` hoy solo se salta las particiones cuya probabilidad es cero, no las particiones cero; `weightedAverage`, al que llama `CEAlgebra.multiplyAndMarginalize`, lee los umbrales y los valores de toda partición con probabilidad distinta de cero; y `CEAlgebra.addMarginalize` (hoy sin llamadores) calcula qué particiones tener en cuenta y luego no lo usa. La ventana del ámbito de decisión tiene su propio problema con la partición cero, descrito en error 86.

Prueba de regresión: en `inference/src/test`, `optimalCEP` con [normal, cero] y con [cero, normal] debe devolver la partición normal con la intervención de su opción; y, con `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx` y el hallazgo «Test = positive», `VECEAnalysis` en ámbito global debe terminar con una partición que no sea cero.

### Error 86 · [Corregido] Las ventanas de resultados del análisis de coste-efectividad fallan con NullPointerException cuando una opción de la decisión, o la evidencia entera, tiene probabilidad cero

| | |
|---|---|
| **Estado** | Corregido en el commit `6238a4b` (09-10-2026): en `CEDecisionResults`, la opción imposible con la evidencia lo dice en su fila («impossible with the evidence»), no tiene coste ni efectividad y queda fuera de los tramos, del plano, de la frontera y del selector «Relative to»; si todas las opciones son imposibles, o lo es el resultado global, se responde con el mensaje del error 48 en vez de abrir la ventana. Comprobado abriendo la ventana con «Test = not done» y con «Test = positive»; el aviso del ámbito global no se ha visto en pantalla. Quedan sin tocar `CESpiderDialog` y `CEProbabilisticDialog`, del análisis de sensibilidad |
| **Módulo** | costEffectiveness, gui |
| **Paquete** | org.openmarkov.costEffectiveness; org.openmarkov.gui.dialog.costeffectiveness |
| **Ficheros** | `costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:294-312`<br>`costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:401-418`<br>`costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:568-597`<br>`costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:887-923`<br>`core/src/main/java/org/openmarkov/core/model/network/CEP.java:149-151`<br>`core/src/main/java/org/openmarkov/core/model/network/CEP.java:167-173`<br>`core/src/main/java/org/openmarkov/core/model/network/CEP.java:227-229`<br>`gui/src/main/java/org/openmarkov/gui/dialog/costeffectiveness/CEPDialog.java:204-208`<br>`costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CostEffectivenessPlugin.java:83-110` |

| | |
|---|---|
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 48, error 54, error 74, error 84, error 85, error 87, nota 46 |

**Qué pasa.** El análisis de coste-efectividad con ámbito de decisión no llega a abrir su ventana cuando alguna opción de la decisión es imposible con la evidencia: sale el aviso de error inesperado con `NullPointerException: Cannot read the array length because "this.strategyTrees" is null`. La ventana del ámbito global hace lo mismo cuando es el resultado entero el que tiene probabilidad cero.

Camino del usuario: abrir `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx`; en modo edición, clic derecho sobre «Test» → «Add finding» y elegir cualquier estado, por ejemplo «not done»; menú Tools → «Cost-effectiveness analysis» y aceptar las opciones de inferencia; en el diálogo de ámbito, que propone el ámbito de decisión con «Dec: New test», elegir la decisión «Dec: Test» (no tiene escenario que fijar) y aceptar.

El resultado de ese análisis es una partición de coste-efectividad por opción (`CEP`: para cada tramo de la disposición a pagar, el coste, la efectividad y la intervención óptima). A una opción incompatible con la evidencia le corresponde la partición de probabilidad cero (`CEP.getZeroPartition()`), un objeto sin tramos cuyos vectores son `null`. Con «Test = not done», «Dec: Test = yes» es imposible y las particiones son `[normal, cero]`; con «negative» o «positive», `[cero, normal]`.

Por qué pasa. El constructor de `CEDecisionResults` monta primero la pestaña de análisis, y `getIntervalsPanel` pide a cada partición `getNumIntervals()`, que devuelve `strategyTrees.length` sin mirar si es la partición cero. No es el único sitio: `getAnalysisTable` pide después a cada opción `getCost`, `getEffectiveness` y `getIntervention`, y `CEP.index` lee `costs.length`; lo mismo hacen `calculateFrontierInterventions` (pestaña de intervenciones fronterizas y línea del plano), `getScatterPlotData` y `getLineFrontierInterventionsData` (plano). Ningún método de la ventana consulta `isZero()`.

La ventana global, `CEPDialog`, construye su tabla con `getDataFromCEP`, que lee `cep.getCosts().length`, y con la partición cero lanza `NullPointerException: Cannot read the array length because "costs" is null`. Le llega esa partición cuando la evidencia previa a la resolución es imposible, por ejemplo en `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx` con «Isquemia = Implante no realizado» y «CC_Drenaje = mayor 800 cc y menor 1000 cc».

Otro camino que deja particiones cero en la ventana por decisión: un escenario imposible en el diálogo de ámbito (error 87), que deja todas las opciones a cero. Con ámbito global, un hallazgo de este tipo ya no hace fallar el análisis ni le hace devolver la partición cero entera (errores 85 y 54, corregidos). En las redes de análisis de decisiones el selector desactiva el ámbito de decisión, así que por ahí no se llega a esta ventana.

Con el jar ejecutable publicado la ventana por decisión tampoco se abre por error 74, pero este error sale antes: la pestaña de análisis se construye antes que la del plano.

Medido, reproduciendo el camino del menú con el panel de ámbito real (`ScopeSelectorPanel`, construido sin pantalla), la tarea `VECEAnalysis` y los métodos de la ventana:

- `ID-CEA-test-2therapies-new-test.pgmx`, hallazgo «Test = not done», decisión «Dec: Test»: particiones `[normal, cero]`; `getIntervalsPanel` lanza la excepción en `CEP.getNumIntervals` (línea 228) y `getAnalysisTable` en `CEP.index` (línea 168). Con «Test = positive», `[cero, normal]` y lo mismo. Sin hallazgo, o con «Test = not done» y las decisiones «Dec: New test» o «Therapy», las particiones son normales y los dos métodos construyen su panel.

- Recorriendo todas las decisiones que ofrece el diálogo, sin hallazgo y con cada hallazgo suelto sobre cada variable de azar, con el escenario por omisión, en las redes de coste-efectividad de `0_miscellaneous/networks/id/` (las cuatro `ID-CEA-*`, `ID-arthronet-ce.pgmx` y las tres `ID-mediastinet*-ce.pgmx`): la ventana recibe alguna partición cero, pero no todas, en 3 casos de `ID-CEA-test-2therapies-new-test.pgmx` y en 12 de cada `ID-mediastinet*-ce.pgmx`; y todas cero en 4, 2, 27, 37 y 33 casos de `ID-CEA-test-2therapies-new-test.pgmx`, `ID-arthronet-ce.pgmx`, `ID-mediastinet-ce.pgmx`, `ID-mediastinet-first-ebus-ce.pgmx` e `ID-mediastinet-first-eus-ce.pgmx`. Sin hallazgo no hay ninguno.
- `CEPDialog.getDataFromCEP` con la partición global de `ID-arthronet-ce.pgmx` y la evidencia imposible de arriba: `NullPointerException` en la línea 207.

**El código.** costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:303-312

```
for (int i = 0; i < gtablePotentialResult.elementTable.size(); i++) {
    CEP cep = (CEP) gtablePotentialResult.elementTable.get(i);
    cepsForDecision[i] = cep;
    if (cep.getNumIntervals() > 1) {
        moreThanOneInterval = true;
        for (double threshold : cep.getThresholds()) {
            thresholds.add(threshold);
        }
    }
}
```

core/src/main/java/org/openmarkov/core/model/network/CEP.java:149-151, 227-229

```
private CEP() {
    zeroProbability = true;
}
...
public int getNumIntervals() {
    return strategyTrees.length;
}
```

gui/src/main/java/org/openmarkov/gui/dialog/costeffectiveness/CEPDialog.java:204-207

```
private static Object[][] getDataFromCEP(CEP cep) {
    double[] costs = cep.getCosts();
    double[] effectiveness = cep.getEffectivities();
    int numRows = costs.length;
```

**Cómo arreglarlo.** La ventana por decisión debe tratar la partición cero como una opción imposible con la evidencia:

- en `getIntervalsPanel`, no pedirle tramos ni umbrales (comprobar `isZero()` antes);
- en `getAnalysisTable`, enseñar su fila sin coste ni efectividad, con un texto que diga que es imposible con la evidencia (el texto exacto es cosa de la interfaz);
- dejarla fuera del plano (`getScatterPlotData`), de la frontera (`calculateFrontierInterventions`, que alimenta la pestaña de intervenciones fronterizas y `getLineFrontierInterventionsData`) y del selector «relative to» de `getAbsoluteRelativePanel`, porque `getScatterPlotData` resta el coste y la efectividad de la opción de referencia.

Si todas las particiones son cero, o la global es la partición cero, no hay nada que enseñar: `CostEffectivenessPlugin.onClick` debería avisar, en sus dos ramas y antes de construir `CEDecisionResults` o `CEPDialog`, de que la evidencia (con el escenario elegido) es imposible, con el mismo mensaje que se decida en error 48.

Otros lectores de particiones que no miran `isZero()` y reciben la evidencia previa del editor con su escenario: `CESpiderDialog` (líneas 443 y 451) y `CEProbabilisticDialog` (líneas 148, 488 y 498), del análisis de sensibilidad (por lectura, no medido). `CEP.clone` tampoco la mira, pero hoy no se llega (nota 46). Que `CEP.getNumIntervals` devuelva 0 para la partición cero evitaría la primera excepción, no las de `getCost`, así que no sustituye a lo anterior.

Prueba de regresión: sacar de la ventana el cálculo de los umbrales y de las filas de la tabla a métodos sin interfaz y probarlos con `[normal, cero]`, `[cero, normal]` y `[cero, cero]`; y comprobar que `VECEAnalysis` sobre `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx`, con el hallazgo «Test = not done» y la decisión «Dec: Test», da `[normal, cero]`, que es el caso que la ventana debe cubrir.

### Error 87 · [Corregido] El ámbito de decisión del análisis de coste-efectividad propone por omisión, y deja elegir, escenarios imposibles con la evidencia: la ventana revienta o enseña efectividades 0

| | |
|---|---|
| **Estado** | Corregido en el commit `b0458cf` (15-10-2026), con la primera opción, que eligió Manuel Arias: ofrecer solo los escenarios posibles. La clase nueva `Possible Scenario` (`gui`) arma los desplegables del escenario para `ScopeSelectorPanel` y para `ScopePanel` del análisis de sensibilidad: cada desplegable ofrece solo los estados posibles con los hallazgos y con lo elegido en los desplegables anteriores, y conserva su elección mientras siga siendo posible; las decisiones aún no tomadas cuentan como si eligieran al azar. Así se llega a cualquier escenario posible y a ninguno imposible. La probabilidad se calcula con `VEPropagation.findingsArePossible`, nuevo, sobre los hallazgos y sus antecesores. Medido: con «Test = negative» y la decisión «Dec: New test», «Dec: Test» solo ofrece «yes»; con «TBNA = negative» en `ID-mediastinet-ce.pgmx` y la decisión «Dec:PET», el escenario propone «Dec:TBNA = yes»; armar el escenario más largo de esa red (once predecesores) tarda unos 30 ms. Con prueba de integración. No se ha probado en la ventana. |
| **Módulo** | gui, costEffectiveness |
| **Paquete** | org.openmarkov.gui.dialog.inference.common; org.openmarkov.costEffectiveness |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/inference/common/ScopeSelectorPanel.java:227-285`<br>`costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CostEffectivenessPlugin.java:84-91`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:264-300`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:534-543` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si el diálogo debe proponer y ofrecer solo escenarios posibles con la evidencia, o dejarlos elegir y avisar al aceptar |
| **Relacionados** | error 54, error 85, error 86 |

**Qué pasa.** En el análisis de coste-efectividad con ámbito de decisión, el diálogo pide un escenario: un valor para cada predecesor informativo de la decisión elegida, es decir, las decisiones anteriores y las variables que se conocen al tomarla. Cada desplegable empieza en el primer estado; si la variable tiene un hallazgo previo, se fija en ese valor y se bloquea. Nada comprueba que el escenario sea posible.

Así, con un hallazgo sobre el resultado de una prueba, el escenario enseña el resultado bloqueado pero deja la decisión de hacer la prueba en su primera opción, «no», aunque el hallazgo diga que

se hizo. El análisis recibe una evidencia imposible, todas las opciones salen con la partición de probabilidad cero y la ventana revienta con `NullPointerException` (error 86).

Camino del usuario: abrir `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx`; en modo edición, «Add finding» sobre «Test» → «negative»; menú Tools → «Cost-effectiveness analysis», aceptar las opciones; en el diálogo de ámbito, con ámbito de decisión, elegir «Dec: New test». El escenario enseña «Test = negative» bloqueado y «Dec: Test = no»; al aceptar sale el aviso de error inesperado.

El usuario también puede elegir a mano, sin ningún hallazgo, una combinación imposible. Según a qué variables afecte, la ventana revienta igual o el análisis termina con números que no corresponden a nada: en la misma red, para «Therapy» con «Dec: Test = yes» y «Test = not done», los costes salen 18, 20 018 y 70 018 y la efectividad 0 en las tres opciones; con el escenario posible «Dec: Test = no», la efectividad es 8,768, 9,074 y 8,908.

Por qué pasa: `ScopeSelectorPanel.getDecisionScenarioPanel` crea un desplegable por predecesor con sus estados en orden, fija y bloquea los que tienen hallazgo, y `updateSelectedScenario` convierte lo elegido en hallazgos sin mirar si son compatibles entre sí ni con la evidencia. `CostEffectivenessPlugin.onClick` añade esos hallazgos a la evidencia previa y construye la ventana. El panel de ámbito del análisis de sensibilidad (`ScopePanel.getDecisionScenarioPanel` y `SensitivityAnalysisDialog`, líneas 534-543) hace lo mismo (por lectura).

Medido con el panel real (`ScopeSelectorPanel`, construido sin pantalla), la tarea `VECEAnalysis` y la evidencia que arma el menú:

```
ID-CEA-test-2therapies-new-test.pgmx, Test = negative, decisión Dec: New test
  escenario por omisión: Dec: Test = no           -> particiones [cero, cero]
  eligiendo Dec: Test = yes                       -> [normal, normal]
ID-mediastinet-ce.pgmx, TBNA = negative, decisión Dec:PET
  escenario por omisión: CT_scan = negative, Dec:TBNA = no   -> [cero, cero]
  eligiendo Dec:TBNA = yes                                   -> [normal, normal]
```

Recorriendo todas las decisiones de las redes de coste-efectividad de `0_miscellaneous/networks/id/` sin hallazgos, el escenario por omisión es siempre posible: el defecto aparece con un hallazgo o eligiendo a mano.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/inference/common/ScopeSelectorPanel.java:244-252

```java
JComboBox<String> stateSelector = new JComboBox<>();
for (State state : variable.getStates()) {
    stateSelector.addItem(state.getName());
}
if (preResolutionEvidence.contains(variable)) {
    stateSelector.setSelectedItem(preResolutionEvidence.getFinding(variable).getState());
    stateSelector.setEnabled(false);
}
selectedScenario.put(stateSelector, variable);
```

costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CostEffectivenessPlugin.java:84-90

```java
case DECISION -> {
    EvidenceCase newPreResolutionEvidence = new EvidenceCase(preResolutionEvidence);
    for (Finding finding : scopeSelectorPanel.getSelectedFindings()) {
        newPreResolutionEvidence.addFinding(finding);
    }
    yield new CEDecisionResults(parent, probNet, newPreResolutionEvidence,
                                scopeSelectorPanel.getDecisionSelected());
```

**Cómo arreglarlo.** Hay que decidir el comportamiento:

- proponer por omisión un escenario compatible con la evidencia (para cada decisión del escenario, una opción con la que los hallazgos sean posibles) y ofrecer en cada desplegable solo los valores posibles con lo ya elegido; exige calcular probabilidades cada vez que cambia un desplegable;
- dejar elegir cualquier combinación y comprobar al aceptar que la evidencia con el escenario tiene probabilidad mayor que cero, avisando si no.

Para esa comprobación está la probabilidad que devuelve `VECEAnalysis.getProbability`, que ya no sale 0 con evidencia posible (error 54, corregido). Aun así, la ventana debe resistir un resultado con todas las opciones a cero (error 86).

La misma regla debe cumplirse en el panel de escenario del análisis de sensibilidad (`ScopePanel`) y en `SensitivityAnalysisDialog`, que arma allí la evidencia.

Prueba de regresión: con `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx`, el hallazgo «Test = negative» y la decisión «Dec: New test», el escenario que propone `ScopeSelectorPanel` debe tener «Dec: Test = yes», o el diálogo debe rechazar «Dec: Test = no» con un aviso.

### Error 88 · [Corregido] En el diagrama de tornado, un modelo cuyos valores son todos negativos dibuja barras que llegan hasta cero y ordena mal los parámetros

| | |
|---|---|
| **Estado** | Corregido en el commit `696a06d` (14-10-2026). Arreglados a la vez los errores 88 y 90, como pedían sus fichas: el máximo arranca en menos infinito en el tornado (`TornadoSpiderDialog.range`), en la araña y en «Plot»; los ejes de «Plot» y de la araña se ensanchan a cada lado una milésima del mayor valor absoluto (`PlotDialog.axisRange`), o una unidad si todo vale 0, de modo que con valores positivos quedan prácticamente como antes; y la barra de un parámetro que no hace variar nada se ensancha con el valor absoluto, porque con un valor negativo salía al revés (visto al corregir, no estaba en la ficha). El mapa ya lo había arreglado el commit `6fa4e26`. Con prueba sin pantalla de la barra (−1010 a −990) y del eje en cinco casos (todo negativo, constante negativa, cero, cruzando el cero y todo positivo); con la lógica anterior fallan todos menos el positivo. |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:333-383`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:544-545`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/PlotDialog.java:213`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:249`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/model/TornadoBar.java:54-60` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 83, error 89, error 90, error 91 |

**Qué pasa.** El diagrama de tornado del análisis de sensibilidad dibuja, para cada parámetro incierto, una barra horizontal entre el menor y el mayor valor que toma el resultado del modelo (la utilidad esperada, o la diferencia entre dos opciones de una decisión) cuando ese parámetro se barre por su intervalo. Las barras se ordenan de más ancha a más estrecha (`TornadoBar.compareTo`), de modo que arriba quedan los parámetros que más influyen.

Camino de llegada: análisis de sensibilidad → tornado, sobre una red con parámetros inciertos cuyo resultado es negativo en todo el barrido (por ejemplo, un modelo de costes expresados como utilidades negativas).

Consecuencias, deducidas del código (el diálogo no se ha ejecutado porque es una ventana):

- Sin decisión: la barra va del valor mínimo real hasta 0 en lugar de hasta el máximo real. Por ejemplo, si la utilidad varía entre −1 010 y −990, la barra mide 1 010 en vez de 20.
- Con decisión (comparar una opción con otra): `maxInterventionUtility` y `maxReferenceUtility` también arrancan en `Double.MIN_VALUE`, y con valores negativos las dos quedan en ~0, así que los extremos de la barra salen mal.
- Como el orden depende de la anchura, un parámetro poco influyente sobre un resultado muy negativo aparece arriba del todo, que es lo contrario de lo que el diagrama debe comunicar.

`TornadoSpiderDialog.getTornadoBars` inicia cada mínimo con `Double.MAX_VALUE` y cada máximo con `Double.MIN_VALUE`. `Double.MIN_VALUE` no es el número más negativo representable sino el positivo más pequeño (unos $4{,}9 \cdot 10^{-324}$). Si todos los valores del barrido son negativos, `Math.max(Double.MIN_VALUE, valor)` se queda en ese número positivo, prácticamente cero.

El mismo inicio aparece en el eje vertical del diagrama de araña de la misma clase (línea 533), en `PlotDialog` (línea 213) y en `MapDialog` (línea 249), con otra consecuencia que describe error 90.

Aparte, en la rama con decisión la barra no sale de la diferencia punto a punto entre las dos opciones; ese es otro defecto, error 89.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:333-377

```
private List<TornadoBar> getTornadoBars() {
    ...
        double minValue = Double.MAX_VALUE;
        double maxValue = Double.MIN_VALUE;
        double minInterventionUtility = Double.MAX_VALUE;
        double maxInterventionUtility = Double.MIN_VALUE;
        double minReferenceUtility = Double.MAX_VALUE;
        double maxReferenceUtility = Double.MIN_VALUE;
        if (decisionVariable != null) {
            ...
            minValue = Math.min(minInterventionUtility - minReferenceUtility,
↪   maxInterventionUtility - maxReferenceUtility);
            maxValue = Math.max(maxInterventionUtility - maxReferenceUtility,
↪   minInterventionUtility - minReferenceUtility);
        } else {
            for (double value : potential.getValues()) {
                minValue = Math.min(minValue, value);
                maxValue = Math.max(maxValue, value);
            }
        }
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*. Iniciar los máximos con `Double.NEGATIVE_INFINITY` (o `-Double.MAX_VALUE`, como ya hace `getTornadoChart` en la línea 384), o arrancar mínimo y máximo con el primer valor del barrido.

Sitios donde debe cambiarse lo mismo: `TornadoSpiderDialog` líneas 328, 330, 332 y 533, `PlotDialog` línea 213 y `MapDialog` línea 249.

Conviene revisar a la vez el cálculo de la rama con decisión (error 89) y el rango del eje `minRangeUtility * 0.999, maxRangeUtility * 1.001`, que con valores negativos recorta los extremos en vez de ampliarlos; los dos cambios de esas líneas tienen que ir juntos (error 90).

Prueba de regresión: extraer el cálculo de mínimos y máximos a un método sin interfaz y comprobar que, con valores todos negativos (−1 010 … −990), la barra va de −1 010 a −990.

### Error 89 · [Corregido] En el diagrama de tornado con ámbito de decisión, la barra de un parámetro no abarca lo que varía la diferencia entre las dos opciones, y puede quedar reducida a un punto

| | |
|---|---|
| **Estado** | Corregido en el commit `21dc602` (14-10-2026): `TornadoSpiderDialog.differences` calcula la diferencia entre la intervención y la referencia en cada punto del barrido, y la usan el tornado y la araña. Con prueba del barrido escrito a mano (de 0 a 2, antes un punto) y de la tarea real sobre `ID-decide-test`, «utility not treated», «Do test? = yes» respecto a «no»: de −0,2000 a 0,1991. Los máximos que arrancaban en `Double.MIN_VALUE` eran el error 88, corregido después en `696a06d`. |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:346-362`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:573-583`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:295-301` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 88, error 90, error 91 |

**Qué pasa.** En el análisis de sensibilidad «Tornado/Spider» con ámbito de decisión, el diagrama de tornado debería dibujar, para cada parámetro, una barra que vaya del menor al mayor valor que toma la diferencia de utilidad esperada entre la opción elegida como intervención y la de referencia al barrer el parámetro. La barra que dibuja no sale de esa diferencia, sino de restar por separado los extremos de cada opción, y puede ser más corta que el rango real, o no ser más que un punto, de modo que el parámetro baja en el orden del diagrama. El diagrama de araña de la misma ventana sí resta punto a punto, así que las dos pestañas no cuentan lo mismo.

Camino del usuario: menú Tools → «Sensitivity analysis», aceptar las opciones de inferencia, tipo «Tornado/Spider», ámbito de decisión, elegir la decisión; en la pestaña del tornado, «Intervention:» y «relative to:» son por omisión la segunda y la primera opción. Con el jar ejecutable publicado esta ventana hoy no llega a abrirse (error 74); desde el IDE sí.

Por qué pasa: `TornadoSpiderDialog.getTornadoBars` recorre el barrido de la intervención y guarda su mínimo y su máximo; hace lo mismo con la referencia; y toma como extremos de la barra el menor y el mayor de `minIntervención − minReferencia` y `maxIntervención − maxReferencia`. Eso solo coincide con el rango de la diferencia cuando los extremos de las dos opciones caen en los mismos puntos del barrido y la diferencia no tiene su extremo en medio. Si una opción sube con el parámetro y la otra baja, los mínimos están en puntos opuestos y la resta mezcla valores de puntos distintos. En cambio, `getSpiderChart` calcula en cada punto `valueIntervention - valueReference`.

Aparte de esto, los máximos arrancan en `Double.MIN_VALUE`, que falla con utilidades negativas; eso es error 88 y se arregla por separado.

Medido:

- Con `0_miscellaneous/networks/id/ID-decide-test.pgmx`, la tarea real `VESensAnTornadoSpider` (variación del 25 %, 50 puntos) y `getTornadoBars` con las opciones por omisión, en la decisión «Do test?» («yes» respecto a «no») la barra de «utility not treated» va de −0,2000 a 0,1109, y la diferencia punto a punto va de −0,2000 a 0,1991: la barra mide 0,31 en vez de 0,40. Las otras siete barras de esa decisión, y las ocho de «Therapy», coinciden.
- Con barridos escritos a mano de cinco puntos: si «no» va de 0 a −1 y «yes» de 0 a 1, la barra sale [1, 1], un punto, cuando la diferencia va de 0 a 2. Si la referencia es constante, o las dos opciones suben a la vez, la barra es correcta.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:350-362

```
for (int i = 0; i <= iterations; i++) {
    double value = potential.getValues()[i + (interventionDecisionIndex * (iterations +
    ↪  1))];
    minInterventionUtility = Math.min(minInterventionUtility, value);
    maxInterventionUtility = Math.max(maxInterventionUtility, value);
}
// For reference utility values
for (int j = 0; j <= iterations; j++) {
    double value = potential.getValues()[j + (relativeDecisionIndex * (iterations + 1))];
    minReferenceUtility = Math.min(minReferenceUtility, value);
    maxReferenceUtility = Math.max(maxReferenceUtility, value);
}
minValue = Math.min(minInterventionUtility - minReferenceUtility, maxInterventionUtility -
↪  maxReferenceUtility);
maxValue = Math.max(maxInterventionUtility - maxReferenceUtility, minInterventionUtility -
↪  minReferenceUtility);
```

sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:565-567 (araña)

```
double valueIntervention = potentials[i +
↪  (interventionSpiderDecisionSelector.getSelectedIndex() * (iterations + 1))];
double valueReference = potentials[i + (relativeSpiderDecisionSelector.getSelectedIndex() *
↪  (iterations + 1))];
value = valueIntervention - valueReference;
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*. Calcular en cada punto del barrido la diferencia `intervención − referencia` y tomar el mínimo y el máximo de esas diferencias, como hace la araña. Conviene sacar ese cálculo (los valores de la curva de un parámetro para una pareja de opciones) a un método sin interfaz que usen `getTornadoBars` y `getSpiderChart`, para que las dos pestañas no puedan volver a separarse.

Al tocarlo, arrancar el mínimo y el máximo con infinito o con el primer valor, que es el arreglo de error 88 para las mismas líneas; el cálculo sin decisión de `getTornadoBars` no tiene este defecto, solo el de error 88.

Prueba de regresión: con el método extraído, un barrido en que «no» baja de 0 a −1 y «yes» sube de 0 a 1 debe dar [0, 2]; y con `0_miscellaneous/networks/id/ID-decide-test.pgmx`, decisión «Do test?», «yes» respecto a «no», la barra de «utility not treated» debe ir de −0,2000 a 0,1991.

### Error 90 · [Corregido] Con utilidades todas negativas, las gráficas «Plot» y de araña del análisis de sensibilidad aplastan las curvas contra el borde inferior y recortan su punto más bajo, y una utilidad constante negativa no se ve

| | |
|---|---|
| **Estado** | Corregido en el commit `696a06d` (14-10-2026). Arreglados a la vez los errores 88 y 90, como pedían sus fichas: el máximo arranca en menos infinito en el tornado (`TornadoSpiderDialog.range`), en la araña y en «Plot»; los ejes de «Plot» y de la araña se ensanchan a cada lado una milésima del mayor valor absoluto (`PlotDialog.axisRange`), o una unidad si todo vale 0, de modo que con valores positivos quedan prácticamente como antes; y la barra de un parámetro que no hace variar nada se ensancha con el valor absoluto, porque con un valor negativo salía al revés (visto al corregir, no estaba en la ficha). El mapa ya lo había arreglado el commit `6fa4e26`. Con prueba sin pantalla de la barra (−1010 a −990) y del eje en cinco casos (todo negativo, constante negativa, cero, cruzando el cero y todo positivo); con la lógica anterior fallan todos menos el positivo. |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/PlotDialog.java:212-213`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/PlotDialog.java:279-281`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:544-545`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:627-629`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:248-249`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:349-350`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:384` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 82, error 83, error 88, error 89 |

**Qué pasa.** En el análisis de sensibilidad, cuando todos los valores que se dibujan son negativos, la gráfica de un parámetro («Plot») y el diagrama de araña («Tornado/Spider», pestaña de araña) ponen el eje vertical desde algo por encima del valor más bajo hasta 0. Las curvas quedan aplastadas contra el borde inferior, casi planas, y el punto más bajo de cada una queda fuera del eje y no se dibuja. Si la utilidad es constante y negativa, la curva entera queda fuera y la gráfica sale vacía. Los valores son todos negativos, por ejemplo, en un modelo cuyas utilidades son costes con signo negativo, o, con ámbito de decisión, cuando la opción elegida como intervención es peor que la de referencia en todo el barrido (la araña dibuja la diferencia).

En el mapa de dos parámetros con ámbito global, la escala de grises iba del valor más bajo a 0, así que todas las casillas salían casi negras y la leyenda llegaba hasta 0. Eso ya no pasa: desde el commit `6fa4e26` (error 83) el máximo del mapa arranca en `Double.NEGATIVE_INFINITY` (comprobado leyendo el código el 14-10-2026). Lo que sigue de este error es lo de «Plot» y la araña.

Camino del usuario: menú Tools → «Sensitivity analysis», tipo «Plot», «Tornado/Spider» o «Map (two-way analysis)», sobre una red cuya utilidad esperada sea negativa en todo el intervalo de

los parámetros. Desde el IDE; con el jar ejecutable publicado estas ventanas no llegan a abrirse (error 74).

Por qué pasa: `PlotDialog.getPlotChart` (línea 213), `TornadoSpiderDialog.getSpiderChart` (línea 533) y `MapDialog.getMapChart` (línea 249) arrancan el máximo con `Double.MIN_VALUE`, que no es el número más negativo sino el positivo más pequeño (unos $4{,}9 \cdot 10^{-324}$). Con todos los valores negativos, el máximo se queda en ese número, prácticamente 0. Es el mismo inicio que en las barras del tornado (error 88), pero aquí la consecuencia es el eje o la escala de color.

En «Plot» y en la araña se suma otra cuenta: el eje se fija en `minRangeUtility * 0.999, maxRangeUtility * 1.001`, que ensancha el rango con valores positivos pero lo estrecha con negativos: el mínimo multiplicado por 0,999 queda por encima del valor más bajo, que se sale.

Medido construyendo las gráficas sin pantalla con JFreeChart 1.5.6, barrido de cinco puntos:

```
Plot y araña, utilidades de -1010 a -990: eje [-1009, 4.9e-324], 1 de 5 puntos fuera, la
↪  curva ocupa el 1,9 % de la altura
Plot y araña, utilidad constante -7:      eje [-6.993, 4.9e-324], 5 de 5 puntos fuera
Plot y araña, utilidades de 990 a 1010:   eje [989, 1011], ningún punto fuera, la curva
↪  ocupa el 90,9 %
Mapa global, utilidades de -1010 a -990:  escala [-1010, 4.9e-324], gris del valor más bajo
↪  0 y del más alto 5 (de 255)
Mapa global, utilidades de 990 a 1010:    escala [990, 1010], gris 0 y 255
```

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/PlotDialog.java:212-213, 279-281 (lo mismo en `TornadoSpiderDialog.java`, líneas 532-533 y 615-617)

```java
double minRangeUtility = Double.MAX_VALUE;
double maxRangeUtility = Double.MIN_VALUE;
...
// Set the range axis
NumberAxis rangeAxis = (NumberAxis) plot.getRangeAxis();
rangeAxis.setRange(minRangeUtility * 0.999, maxRangeUtility * 1.001);
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*. Arrancar el máximo con `Double.NEGATIVE_INFINITY` (o mínimo y máximo con el primer valor) en los tres sitios, que es el mismo cambio que error 88 pide para el tornado, y a la vez cambiar la cuenta del eje de «Plot» y de la araña por un margen que no dependa del signo, por ejemplo `min - 0.001 * |max - min|` y `max + 0.001 * |max - min|`, con un margen fijo cuando `max == min`.

Las dos cosas tienen que ir juntas. Con solo el cambio del inicio, una utilidad constante negativa daría un límite inferior mayor que el superior y JFreeChart lanzaría `IllegalArgumentException` (`Range(double, double): require lower <= upper`, comprobado leyendo `org.jfree.data.Range`), y una constante 0 daría un rango de longitud cero, que también rechaza. En `MapDialog` pasa lo mismo con la escala y el eje de la leyenda; allí el caso de rango vacío es error 83, y su arreglo debe ir junto con este.

La araña con ámbito de decisión y la casilla de la línea del 0 marcada ya ensancha el eje hasta incluir el 0 (líneas 633-641); el arreglo no debe romper ese ajuste.

Prueba de regresión: construir sin pantalla la gráfica de `PlotDialog` y la de la araña con un barrido de −1 010 a −990, otro constante en −7, otro constante en 0 y otro positivo, y comprobar que todos los puntos quedan dentro del eje y que ninguna lanza excepción; y, para el mapa global, que el valor más alto de un barrido negativo se pinta blanco.

### Error 91 · El diagrama de tornado con dieciséis parámetros o más dibuja barras fuera del recuadro, y con veinte algunas no se ven

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:385-388`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:432-435` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 74, error 88, error 89 |

**Qué pasa.** El análisis de sensibilidad determinista de tipo «tornado/araña» (menú Tools → Sensitivity analysis, aceptar las opciones de inferencia, elegir el tipo de análisis «Tornado/Spider», marcar los parámetros inciertos y pulsar OK) abre `TornadoSpiderDialog`, que dibuja un diagrama de tornado: una barra horizontal por cada parámetro incierto marcado. No hay límite al número de parámetros que se pueden marcar: son las casillas de la lista de parámetros del diálogo.

Con 16 parámetros o más, las barras de los extremos se salen del área de dibujo, y con 20 algunas quedan enteras fuera y no se ven, sin que el diagrama avise de que faltan. Como las barras se ordenan de mayor a menor efecto (`Collections.sort(tornadoBars)`), las que desaparecen por arriba son precisamente las de los parámetros más influyentes.

El método privado `getTornadoChart` reparte el espacio del eje de categorías (el eje vertical, donde van las barras) con una cuenta fija: el margen superior y el inferior valen `0.475 - bars * 0.03`, donde `bars` es el número de parámetros. Con 15 parámetros el margen vale 0,025; con 16 vale −0,005 y a partir de ahí es negativo.

JFreeChart (la biblioteca de gráficas) no comprueba el signo del margen: con un margen negativo coloca la primera y la última categoría por encima y por debajo del área de dibujo, que recorta lo que se sale.

Medido con un pequeño programa de prueba sin ventana que construye el mismo `CategoryPlot` (mismos márgenes, `setCategoryMargin(0.01)`, `setMaximumBarWidth(0.05)`, orientación horizontal, título) en una imagen de 600×400 píxeles cuya área de datos va del píxel 30,5 al 378,4:

- 5 parámetros: la primera barra empieza en 143,6 y la última acaba en 259,1; todas dentro.
- 15 parámetros: de 39,2 a 365,3; todas dentro.
- 16 parámetros: de 28,8 a 375,8; la primera barra ya asoma por encima del área.
- 20 parámetros: de −13,0 a 417,7; 4 de las 20 barras quedan enteras fuera del área.
- 25 parámetros: de −65,1 a 470,0; 8 barras enteras fuera.

Los píxeles exactos dependen del tamaño de la ventana y de los rótulos, pero el signo del margen no: con 16 o más parámetros el reparto deja barras fuera.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:385-388, 432-435

```
private JFreeChart getTornadoChart(List<TornadoBar> tornadoBars) {
    // JFreeChart attributes definition. Number of bars in the chart, one per parameter
    int bars = uncertainParameters.size();
...
```

```
    double space = 0.475 - (bars * 0.03);
    categoryAxis.setLowerMargin(space);
    categoryAxis.setUpperMargin(space);
    categoryAxis.setCategoryMargin(0.01);
```

**Cómo arreglarlo.** Que el margen nunca sea negativo: acotarlo por abajo (por ejemplo `Math.max(0.0, ...)` o un mínimo pequeño positivo) o calcularlo de forma que el espacio total de márgenes más categorías no supere el área. Si con muchas barras no caben con una altura legible, la alternativa es hacer crecer la altura del panel con el número de barras (con barra de desplazamiento).

Es el único sitio del módulo con esta cuenta; `MapDialog` pone márgenes a cero y no se ve afectado.

Prueba: construir el `JFreeChart` que devuelve `getTornadoChart` con 16, 20 y 30 barras, dibujarlo con `createBufferedImage(600, 400, info)` y comprobar con `ChartRenderingInfo` que todas las `CategoryItemEntity` quedan dentro de `getPlotInfo().getDataArea()`.

## Error 92 · El análisis de sensibilidad no se abre sobre una red sin decisiones: sale un error inesperado

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:114-132`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:213-246`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:154-181`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:320-384`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/SensitivityAnalysisPlugin.java:31-40`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisFrameGenerator.java:33-47` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 65 |

**Qué pasa.** Sobre una red sin nodos de decisión que tenga parámetros inciertos con nombre, el análisis de sensibilidad no llega a abrirse: sale un error inesperado. Si la red no tiene parámetros inciertos con nombre, el diálogo sí se abre (con el aviso de pocos parámetros), porque en ese caso no se construye el panel donde está el fallo.

Camino exacto desde el menú:

- Abrir una red sin nodos de decisión que tenga parámetros inciertos con nombre, por ejemplo `integrationTests/src/main/resources/networks/id/ID-one-chance-sa.pgmx` (diagrama de influencia con un nodo de azar, una utilidad, ninguna decisión y dos parámetros inciertos con nombre).
- Menú Tools → «Sensitivity analysis». La entrada la añade `SensitivityAnalysisPlugin` y está activa en cuanto hay una red abierta.
- Su acción llama a `SensitivityAnalysisFrameGenerator.create`, que construye `SensitivityAnalysisController` (recoge los parámetros inciertos con nombre) y

muestra el diálogo modal de opciones de inferencia (`InferenceOptionsDialog`).
- Pulsar OK en ese diálogo. `create` construye `SensitivityAnalysisDialog`; su constructor llama a `getContentFrame()`, que arma el selector de tipo de análisis (`getAnalysisTypePanel`: selecciona el primer tipo, «Tornado/Spider» en un análisis de un criterio o «Spider CE» en coste-efectividad, llama a `updateFlags()` y vuelve a llamar a `getContentFrame()`).
- Como hay parámetros inciertos, `getContentFrame` añade `ParametersPanel`, `AxisVariationPanel` y `getScopePanel()` → `new ScopePanel(controller)`.
- Todos los tipos que se ofrecen ponen `setCanBeDecision(true)`, así que `ScopePanel.getMainPanel` llama a `getDecisionSelectorPanel()`, que llena el desplegable con las decisiones de la red (ninguna) y hace `decisionSelector.setSelectedIndex(0)`: `IllegalArgumentException`.

La comprobación de que no hay decisiones (líneas 122-127) llega después y solo desactiva controles. La excepción sube por el constructor del diálogo hasta la acción del menú; el manejador global la enseña como error inesperado y el diálogo del análisis de sensibilidad no llega a abrirse.

Medido con un pequeño programa de prueba que construye `ScopePanel` con la configuración que pone «Tornado/Spider» (`canBeGlobal` y `canBeDecision` verdaderos). Con `integrationTests/src/main/resources/networks/id/ID-one-chance-sa.pgmx` (0 decisiones, 2 parámetros con nombre):

```
java.lang.IllegalArgumentException: setSelectedIndex: 0 out of bounds
   at javax.swing.JComboBox.setSelectedIndex(JComboBox.java:652)
   at ScopePanel.getDecisionSelectorPanel(ScopePanel.java:233)
   at ScopePanel.getMainPanel(ScopePanel.java:120)
   at ScopePanel.<init>(ScopePanel.java:106)
```

Con `integrationTests/src/main/resources/networks/id/ID-decide-test-sv.pgmx` (2 decisiones) el panel se construye bien.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:119-127

```
if (configuration.isCanBeDecision()) {
    mainPanel.add(getDecisionSelectorPanel());

    List<Node> decisionNodes = probNet.getNodes(NodeType.DECISION);
    if (decisionNodes == null || decisionNodes.isEmpty()) {
        for (Component component : scopeTypePanel.getComponents()) {
            component.setEnabled(false);
        }
    }
```

sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:220-243

```
decisionSelector = new JComboBox<>();
for (Node node : probNet.getNodes(NodeType.DECISION)) {
    ...
    decisionSelector.addItem(node.getName());
}
...
decisionSelector.setSelectedIndex(0);
decisionSelectorPanel.add(decisionSelector);

if (scopeType == ScopeType.GLOBAL) {
    ...
} else {
    decisionSelected = probNet.getVariable(decisionSelector.getSelectedItem().toString());
```

```
}
```

**Cómo arreglarlo.** Cuando la red no tiene decisiones, el ámbito debe quedarse en «global» y no ofrecer el de decisión. Sitios:

- `ScopePanel.getDecisionSelectorPanel`, línea 233: seleccionar solo si `decisionSelector.getItemCount() > 0`, que es exactamente la guarda que ya tiene el panel equivalente de la interfaz general, `gui/src/main/java/org/openmarkov/gui/dialog/inference/common/ScopeSelectorPanel.java`, líneas 210-212.
- Las lecturas de `decisionSelector.getSelectedItem().toString()` de las líneas 166 y 242, que darían `NullPointerException` con el desplegable vacío si el ámbito queda en «decisión».
- Mejor aún, en `SensitivityAnalysisDialog.updateFlags` (líneas 320-384), poner `setCanBeDecision(false)` cuando la red no tenga decisiones, para que la comprobación se haga antes de construir el panel y no después.

Prueba: construir `ScopePanel` (o el diálogo, con una red abierta simulada) sobre `integrationTests/src/main/resources/networks/id/ID-one-chance-sa.pgmx` y comprobar que no lanza y que el ámbito queda en `ScopeType.GLOBAL`.

### Error 93 · [Corregido] Los parámetros inciertos del análisis de sensibilidad salen en un orden distinto cada vez que se abre, y cambia el parámetro que el diálogo propone por omisión

| | |
|---|---|
| **Estado** | Corregido en el commit `be8b38f` (14-10-2026). `SystematicSampling` reúne los parámetros con `LinkedHashSet` y `LinkedHashMap`, en el orden de las casillas de cada tabla y de las ramas de cada árbol. El orden entre relaciones ya era estable: los nodos de la red se guardan en una lista. La prueba construye 50 veces una red con dos nodos de diez parámetros con nombre; con el código anterior salían, por ejemplo, `A2, A5, A3, A1, A0…` en vez de `A0, A1, A2…` |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.modelUncertainty |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/SystematicSampling.java:46-96`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/model/SensitivityAnalysisController.java:100-113`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ParametersPanel.java:96-166` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 107 |

**Qué pasa.** La lista de parámetros inciertos del diálogo del análisis de sensibilidad aparece en un orden distinto cada vez que se abre, y con ella cambia el parámetro que el diálogo marca por omisión.

Camino del usuario: menú Tools → «Sensitivity analysis» con una red con parámetros inciertos con nombre. `SensitivityAnalysisController` pide `SystematicSampling.getUncertainParameters(probNet)` y guarda los nombres en `orderedUncertainParametersKeys` en el orden recibido; `ParametersPanel` crea una casilla o un botón de opción por parámetro en ese orden. En los análisis de un parámetro («Plot») marca el primero de la lista; en el mapa de dos parámetros («Map») marca el primero para el eje horizontal y el segundo para el vertical.

`getUncertainParameters(ProbNet)` recorre los potenciales de la red y, para cada uno, llama a

la versión privada que mete los parámetros en un `HashSet<UncertainParameter>` a partir de las claves de un `HashMap<UncertainValue, …>`. Ni `UncertainValue` ni `UncertainParameter` redefinen `hashCode`, así que el orden dentro de cada potencial depende del código de identidad de los objetos, que cambia entre ejecuciones.

Medido con un pequeño programa de prueba sobre `integrationTests/src/main/resources/networks/id/ID-decide-test-sv.pgmx`, seis ejecuciones (los parámetros sin nombre, que el diálogo descarta, salen como `null`):

```
null | prevalence | null | null | sensitivity | specificity | null | null | utility
↪  non-treated disease | utility treated disease | utility not treated | cost of test |
↪  cost of therapy
null | prevalence | null | null | specificity | sensitivity | null | null | utility not
↪  treated | utility non-treated disease | utility treated disease | cost of test | cost
↪  of therapy
null | prevalence | null | sensitivity | specificity | null | null | null | utility treated
↪  disease | utility not treated | utility non-treated disease | cost of test | cost of
↪  therapy
null | prevalence | null | null | sensitivity | null | null | specificity | utility treated
↪  disease | utility non-treated disease | utility not treated | cost of test | cost of
↪  therapy
null | prevalence | null | null | sensitivity | null | specificity | null | utility not
↪  treated | utility non-treated disease | utility treated disease | cost of test | cost
↪  of therapy
null | prevalence | null | specificity | null | null | sensitivity | null | utility
↪  non-treated disease | utility not treated | utility treated disease | cost of test |
↪  cost of therapy
```

Los ocho parámetros con nombre son siempre los mismos; lo que cambia es su orden dentro de cada potencial. En esta red, el parámetro que el mapa de dos parámetros propone para el eje vertical es unas veces «sensitivity» y otras «specificity».

**El código.** core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/SystematicSampling.java:62-81

```java
private static Set<UncertainParameter> getUncertainParameters(Potential potential) {
    Set<UncertainParameter> uncertainParams = new HashSet<>();

    Map<UncertainValue, SubPotentialAndPositionInTablePotential> uncertainValues =
    ↪  getUncertainValues(potential);

    for (UncertainValue auxUncertainValue : uncertainValues.keySet()) {
        SubPotentialAndPositionInTablePotential subPotentialAndPosition =
↪  uncertainValues.get(auxUncertainValue);
        uncertainParams
                .add(new UncertainParameter(potential, auxUncertainValue,
                ↪  subPotentialAndPosition.getSubPotential(),
                        subPotentialAndPosition.getPosition()));
    }
    return uncertainParams;
}

private static Map<UncertainValue, SubPotentialAndPositionInTablePotential>
↪  getUncertainValues(Potential potential) {
    Map<UncertainValue, SubPotentialAndPositionInTablePotential> uncertainValuesHash = new
    ↪  HashMap<>();
```

**Cómo arreglarlo.** Usar `LinkedHashMap` en `getUncertainValues` (línea 81) y `LinkedHashSet` en la versión privada de `getUncertainParameters` (línea 63), para conservar el orden en que se

recorren las casillas (por índice) y las ramas de los árboles.

El orden entre potenciales lo da `ProbNet.getPotentials()`; conviene comprobar que también es estable (en la medida, el orden entre potenciales no cambió).

Es el mismo patrón (conjuntos y mapas sin orden sobre objetos sin `hashCode`) que causa error 4 en las operaciones de potenciales, pero en otro sitio: arreglar aquél no arregla éste.

El otro llamador de `getUncertainParameters(ProbNet)` es `SystematicSampling` línea 413 (búsqueda por nombre), al que el orden no le afecta.

Prueba: llamar varias veces a `getUncertainParameters` sobre `integrationTests/src/main/resources/networks/id/ID-decide-test-sv.pgmx` en procesos distintos (o con varias copias de la red) y comprobar que la lista de nombres es siempre la misma y sigue el orden de las casillas de cada tabla.

### Error 94 · En el análisis probabilístico de coste-efectividad, ocultar una opción en una pestaña la oculta en las dos, pero la casilla de la otra pestaña sigue marcada

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CEProbabilisticDialog.java:121,148,229-264,360-368,434-479` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 132 |

**Qué pasa.** El análisis de sensibilidad probabilístico de coste-efectividad (menú de herramientas → «Sensitivity analysis» sobre una red con análisis de coste-efectividad, tipo «Cost-effectiveness plot») abre `CEProbabilisticDialog`, con dos pestañas: el plano de coste-efectividad y la curva de aceptabilidad. Cada una tiene a la derecha un recuadro «Show/hide interventions» con una casilla por opción de la decisión.

Desmarcar una opción en la pestaña del plano la quita también de la curva, pero la casilla de esa opción en la pestaña de la curva sigue marcada: esa pestaña enseña una casilla marcada para una opción que no dibuja. Si el usuario vuelve a pulsar esa casilla, la desmarca (y su pulsación deja la opción en «oculto»), con lo que tiene que pulsar dos veces para volver a ver la opción.

`getShowHidePanel()` fabrica casillas nuevas cada vez que se llama, y se llama dos veces: desde `getAbsRelShowHidePanel()` para la pestaña del plano y desde `getAcceptabilityCurvePanel()` para la de la curva. Cada casilla, al pulsarla, escribe su estado en el vector compartido `selectedStates` y llama a `refreshChartPanels()`, que rehace las dos gráficas. Ninguno de los dos juegos actualiza las casillas del otro.

No es el mismo defecto que el ya corregido en el commit `dce0e1b` (el escenario del diálogo de configuración del análisis frente a la evidencia de la red): éste está en la ventana de resultados.

Comprobado leyendo; no se ejecutó porque construir la ventana exige ejecutar la simulación probabilística (`VECEPSA`) en su constructor.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CEProbabilisticDialog.java:434-459

```
public JScrollPane getShowHidePanel() {
    JPanel showHidePanel = new JPanel();
    // TODO - LOCALIZE
    showHidePanel.setBorder(new TitledBorder("Show/hide interventions"));
    ...
    List<JCheckBox> showHideCheckBoxes = new ArrayList();
    for (int stateIndex = 0; stateIndex < decisionVariable.getNumStates(); stateIndex++) {
        JCheckBox stateCheckbox = new JCheckBox(decisionVariable.getStateName(stateIndex));
        stateCheckbox.addActionListener(new ActionListener() {
            @Override public void actionPerformed(ActionEvent e) {
                JCheckBox checkBox = (JCheckBox) e.getSource();
                for (int stateIndex = 0; stateIndex < decisionVariable.getNumStates();
                ↪  stateIndex++) {
                    if (decisionVariable.getStateName(stateIndex).equals(checkBox.getText
                    ↪  ())) {
                        selectedStates[stateIndex] = checkBox.isSelected();
                    }
                }
                refreshChartPanels();
            }
        });
        stateCheckbox.setSelected(true);
        selectedStates[stateIndex] = true;
```

**Cómo arreglarlo.** Dos opciones:

- Un solo juego de casillas compartido por las dos pestañas. Swing no permite un mismo componente en dos contenedores, así que habría que moverlo a la pestaña visible al cambiar de pestaña.
- Mantener dos juegos y, al pulsar una casilla, poner la casilla de la misma opción del otro juego en el mismo estado, guardando los dos juegos en campos, como hace `CEDecisionResults` con `cePlaneShowHideCheckBoxes`.

Si se prefiere que cada pestaña filtre por su cuenta, entonces cada juego necesita su propio vector de estados y `refreshChartPanels` debe rehacer solo su gráfica; es una decisión de interfaz.

La misma estructura aparece en `gui/src/main/java/org/openmarkov/gui/desnets/CEDESDialog.java` (ventana del plano de la simulación de sucesos discretos), aunque allí la pestaña de la curva de aceptabilidad está comentada en `getTabbedPane`, `getShowHidePanel` se usa una sola vez y no se produce el desajuste.

Prueba: construir el diálogo sin su constructor con resultados escritos a mano, desmarcar una casilla del plano y comprobar que la de la curva queda desmarcada.

### Error 255 · Con un intervalo definido por el usuario que no pasa por el cero, los ejes del mapa de dos parámetros llegan hasta el cero y dejan, entre el cero y las casillas, una franja vacía pintada de azul

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:321-355`<br>`gui/src/main/java/org/openmarkov/gui/configuration/GUIColors.java:235` |

| | |
|---|---|
| **Comprobado** | Leyendo el código y ejecutándolo (los ejes y el pintado, armados con JFreeChart como los arma el mapa, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 82, error 256 |

**Qué pasa.** Menú Tools → «Sensitivity analysis», tipo «Map (two-way analysis)», con la variación «User defined interval» en los dos parámetros, por ejemplo de 0,25 a 0,75. Cada eje del mapa empieza en el cero y no en la primera casilla: entre el cero y el borde de las casillas (0,1875) queda una franja vacía, pintada del color de fondo del recuadro (`GUIColors.SensitivityAnalysis.PLOT_BACKGROUND`, 0, 0, 255: azul), en forma de L a lo largo del borde izquierdo y del de abajo, que ocupa casi la cuarta parte de cada eje. Con un intervalo de 2 a 3, las casillas se quedan en el último tercio del eje; con uno negativo, de −3 a −2, el eje se alarga por la derecha hasta el cero. Con una variación en tanto por ciento el recorrido pasa siempre por el cero y no se nota.

Por qué pasa: `MapDialog.getMapChart` arma los dos ejes como `NumberAxis` de JFreeChart, la biblioteca con la que el programa dibuja sus gráficas, y no les fija los extremos: deja que los saquen de las casillas. Les quita el margen (`setLowerMargin(0)`, `setUpperMargin(0)`), pero no lo otro que un `NumberAxis` hace por omisión: **incluir siempre el cero** (`autoRangeIncludesZero` vale `true`).

Desde `2a88410` (arreglo del error 82) cada casilla mide lo que hay de un punto al siguiente, así que el eje debería acabar media casilla más allá de las casillas de los bordes, que es justo lo que pedía la prueba propuesta en aquella ficha («el rango del eje horizontal es [mín − intervalo/2, máx + intervalo/2]»). Antes, como cada casilla medía una unidad, la franja solo aparecía con intervalos a más de media unidad del cero.

Medido el 15-10-2026 armando los ejes, el renderizador y el recuadro con las mismas llamadas que las líneas 321-354, con la JFreeChart del jar y una rejilla de 5 × 5 casillas:

```
intervalo de 0,25 a 0,75:  ejes de 0.0 a 0.8125   (sin incluir el cero, de 0.1875 a 0.8125)
intervalo de 2 a 3:        ejes de 0.0 a 3.125
intervalo de -3 a -2:      ejes de -3.125 a 0.0
intervalo de 0 a 1:        ejes de -0.125 a 1.125
variación de ±25 %:        ejes de -0.3125 a 0.3125
```

Pintando el primero en una imagen de 800 × 800, el color en (0,09; 0,5) y en (0,5; 0,09) es 0, 0, 255, y en las casillas, 192, 192, 192.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:337-348

```
        NumberAxis xAxis = new NumberAxis(hUncertainParameter.getName() + " - " +
↪  stringDatabase
                .getString(hAxisVariation.getVariationType().toString()));
        xAxis.setLowerMargin(0);
        xAxis.setUpperMargin(0);
        xAxis.setNumberFormatOverride(hDecimalFormat);

        // Set the y axis format
        NumberAxis yAxis = new NumberAxis(vUncertainParameter.getName() + " - " +
↪  stringDatabase
                .getString(vAxisVariation.getVariationType().toString()));
        yAxis.setLowerMargin(0);
        yAxis.setUpperMargin(0);
        yAxis.setNumberFormatOverride(vDecimalFormat);
```

**Cómo arreglarlo.** `xAxis.setAutoRangeIncludesZero(false)` y lo mismo para `yAxis`; o fijar los extremos a mano, de mín − intervalo/2 a máx + intervalo/2, que es lo que hacen las otras gráficas del análisis con su eje horizontal (`PlotDialog`, línea 298; la araña, `TornadoSpiderDialog`, línea 614). Las gráficas armadas con `ChartFactory.createXYLineChart` no tienen el problema en su eje horizontal: la fábrica ya les quita el cero (comprobado en la JFreeChart del jar).

Prueba: armar el mapa con un intervalo de 0,25 a 0,75 y cuatro tramos en los dos ejes y comprobar que los dos van de 0,1875 a 0,8125; y con ±25 %, que siguen yendo de −0,3125 a 0,3125.

### Error 256 · En el análisis de sensibilidad, pedir una variación del 0 %, o un intervalo con los dos extremos iguales o del revés, acaba en un error inesperado y no se abre ni «Plot», ni la ventana del tornado y la araña, ni el mapa de dos parámetros

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/AxisVariationPanel.java:270-291,344-385`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/PlotDialog.java:212-222,296-298`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:240-247,527-539,614`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:237-246,350-352`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:572-599` |
| **Comprobado** | Leyendo el código y ejecutándolo (las llamadas a JFreeChart de las tres ventanas, repetidas sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 82, error 83, error 90, error 255 |

**Qué pasa.** En el análisis de sensibilidad (menú Tools → «Sensitivity analysis»), cuánto se mueve cada parámetro se escribe a mano: el tanto por ciento es un desplegable que deja escribir lo que se quiera (`AxisVariationPanel`, líneas 271-291) y el intervalo, dos casillas de texto libre (líneas 344-385). Nadie mira que lo escrito tenga anchura, así que se puede pedir un 0 %, o un intervalo con los dos extremos iguales o del revés (el de abajo mayor que el de arriba). Con eso, al pulsar el botón de aceptar no se abre ninguna de estas ventanas: JFreeChart, la biblioteca de gráficas, rechaza el eje, la excepción sale del constructor de la ventana y, por lectura, llega al manejador global, que enseña la ventana de error inesperado.

- **«Plot»** fija el eje de abajo con `setRange(mínimo, máximo)` (`PlotDialog.getPlotChart`, línea 298), y JFreeChart no admite un eje sin longitud ni uno del revés.
- **«Tornado/Spider»**: la araña hace lo mismo con un margen del 2 % (`TornadoSpiderDialog`, línea 614). El tornado, por sí solo, aguanta un recorrido sin anchura (líneas 402-411), pero las dos pestañas se arman en el mismo constructor (`getTabbedPane`, líneas 240-247), así que con la araña se pierde también el tornado; esto último está leído.
- **El mapa** se abría, con casillas de una unidad, hasta `2a88410` (arreglo del error 82); desde entonces cada casilla mide lo que hay de un punto al siguiente (líneas 246 y 270), y `XYBlockRenderer.setBlockWidth` no admite cero ni menos (líneas 351-352).

Medido el 15-10-2026 repitiendo con la JFreeChart del jar las llamadas exactas de las tres ventanas,

con los extremos que ellas calculan:

```
mapa, 0 %:              IllegalArgumentException The 'width' argument must be > 0.0
mapa, de 0,5 a 0,5:     IllegalArgumentException The 'width' argument must be > 0.0
mapa, de 1 a 0:         IllegalArgumentException The 'width' argument must be > 0.0
Plot, 0 %:              IllegalArgumentException A positive range length is required:
↪  Range[-0.0,0.0]
Plot, de 0,5 a 0,5:     IllegalArgumentException A positive range length is required:
↪  Range[0.5,0.5]
Plot, de 1 a 0:         IllegalArgumentException Range(double, double): require lower
↪  (1.0) <= upper (0.0).
araña, 0 %:             IllegalArgumentException A positive range length is required:
↪  Range[-0.0,0.0]
araña, de 1 a 0:        IllegalArgumentException Range(double, double): require lower
↪  (1.02) <= upper (-0.02).
```

Las mismas excepciones salieron el 14-10-2026 llamando sin pantalla a los métodos que arman las tres gráficas, sobre `ID-decide-test-sv.pgmx`, con un 0 % y con el intervalo de 1 a 0; ninguna de esas clases ha cambiado desde entonces.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/PlotDialog.java:296-298

```
        // Set the domain axis
        NumberAxis domainAxis = (NumberAxis) plot.getDomainAxis();
        domainAxis.setRange(minVariationValue, maxVariationValue);
```

sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/MapDialog.java:350-352

```
        XYBlockRenderer renderer = new XYBlockRenderer();
        renderer.setBlockWidth(hVariationInterval);
        renderer.setBlockHeight(vVariationInterval);
```

**Cómo arreglarlo.** Que el diálogo no deje lanzar un análisis con un recorrido sin anchura: antes de `controller.runAnalysis()` en `SensitivityAnalysisDialog.doOkClick`, o al salir de los controles de `AxisVariationPanel`, comprobar que el tanto por ciento es mayor que cero y que el extremo de abajo del intervalo es menor que el de arriba (y que los dos son números), decirlo con un mensaje y no lanzar nada. Vale para los dos ejes del mapa. Como defensa, las tres ventanas no deberían llamar a `setRange` ni a `setBlockWidth` con una anchura que no sea positiva.

Prueba: sobre un método estático de comprobación, que un 0 %, un intervalo de 0,5 a 0,5 y uno de 1 a 0 se rechacen, y que un 25 % y un intervalo de 0 a 1 se acepten. Y, con pantalla, que pedir «Plot» con un 0 % no abra la ventana de error inesperado.

### Error 262 · [Corregido] En el análisis de sensibilidad con ámbito de una decisión, pulsar el botón de aceptar vuelve a armar los desplegables del escenario, que vuelven a su primer estado posible, y la gráfica se hace con ese escenario y no con el que eligió el usuario

| | |
|---|---|
| **Estado** | Corregido en el commit `7ebcb51` (15-10-2026): `PossibleScenario` puede empezar cada desplegable en el estado elegido en un escenario anterior y lo conserva mientras sea posible con la evidencia; `ScopePanel` le pasa las elecciones de ese momento al rehacer el escenario al aceptar. Cambiar de decisión o de ámbito sigue armando uno nuevo. La prueba va sobre `PossibleScenario`; el cambio de `ScopePanel` está comprobado leyendo el código, porque el panel necesita la ventana principal |

| | |
|---|---|
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog; org.openmarkov.gui.dialog.inference.common |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:572-599` `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:264-318` `gui/src/main/java/org/openmarkov/gui/dialog/inference/common/PossibleScenario.java:54-106` |
| **Comprobado** | Leyendo el código y ejecutándolo (el panel del ámbito, sin ventana, con el Java de hoy y con el del 14 de octubre) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 87 |

**Qué pasa.** Menú Tools → «Sensitivity analysis», ámbito «One decision», una decisión, y en el recuadro «Scenario» un estado para cada predecesor informativo de la decisión (las decisiones anteriores y lo que se sabe al tomarla). El diálogo no es modal y su botón de aceptar no lo cierra, para poder lanzar varios análisis. Al pulsarlo, los desplegables del escenario vuelven a su primer estado posible, y el análisis se hace con ese escenario por omisión y no con el elegido, sin ningún aviso: la gráfica que sale no es la del escenario que el usuario ve que eligió un instante antes.

Por qué pasa: para que el escenario no contradiga una evidencia puesta en la red con el diálogo abierto, `dce0e1b` (10-08-2026) hizo que aceptar, antes de nada, **rehiciera** el escenario con la evidencia de ese momento (`doOkClick`, líneas 575-578). `ScopePanel.refreshScenarioFromEvidence` llama a `refreshScenario`, que quita el panel del escenario y pone uno nuevo armado por `getDecisionScenarioPanel`, con un `PossibleScenario` nuevo y **desplegables nuevos**, cada uno en su primer estado; desde `b0458cf` (arreglo del error 87), en el primero de los posibles. `PossibleScenario` conserva la elección de un desplegable mientras siga siendo posible, pero estos desplegables acaban de nacer y no tienen ninguna. Después `doOkClick` añade a la evidencia los hallazgos de ese escenario nuevo (líneas 587-597).

Medido el 15-10-2026 armando `ScopePanel` (es un `JPanel`, no necesita ventana) sobre un controlador hecho sin su constructor —que busca la ventana principal— y con sus campos puestos por reflexión; se elige el ámbito de una decisión, se cambia una fila y se llama a `refreshScenarioFromEvidence`, como hace `doOkClick`. En `ID-CEA-test-2therapies-new-test.pgmx`, decisión «Therapy», eligiendo «Dec: Test = yes» (el desplegable de «Test» pasa solo a «negative», que es lo posible):

```
escenario antes de aceptar:  [Dec: New test=no, Dec: Test=yes, New test=not done,
↪  Test=negative]
escenario después:           [Dec: New test=no, Dec: Test=no, New test=not done, Test=not
↪  done]
```

En `ID-mediastinet.pgmx`, decisión «Treatment», «CT_scan = positive» vuelve a «negative»; con el jar del 14 de octubre, anterior a `b0458cf`, pasa igual.

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:572-578

```
    protected void doOkClick() throws NonProjectablePotentialException,
    ↪  IncompatibleEvidenceException, NotSupportedOperationException,
    ↪  NotEvaluableNetworkException.NotApplicableNetwork,
    ↪  NotEvaluableNetworkException.UnsatisfiedConstraints, ConstraintViolatedException {
        // This dialog stays open between analyses, so the user may have set evidence in
        ↪  the network
```

```
        // since the scenario was built. Both are read again before they are mixed.
        controller.readPreResolutionEvidenceFromNetwork();
        if (scopePanel != null) {
            scopePanel.refreshScenarioFromEvidence();
        }
```

sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:311-318

```
    private void refreshScenario() {
        mainPanel.setVisible(false);
        if (decisionScenarioPanel != null) {
            mainPanel.remove(decisionScenarioPanel);
            mainPanel.add(getDecisionScenarioPanel());
        }
        mainPanel.setVisible(true);
    }
```

**Cómo arreglarlo.** Rehacer el escenario **conservando lo elegido mientras siga siendo posible**, que es lo que `PossibleScenario` ya sabe hacer desplegable a desplegable. Dos caminos: darle a `PossibleScenario` un método que reciba la evidencia nueva y vuelva a ofrecer los estados posibles sobre los mismos desplegables (fijando y bloqueando los que hayan ganado un hallazgo, y liberando los que lo hayan perdido); o, al rehacer el panel, poner cada desplegable nuevo en el estado que tenía el viejo antes de llamar a `offerPossibleStates`. El cambio de decisión o de ámbito sí debe seguir armando un escenario nuevo. El diálogo del coste-efectividad (`ScopeSelectorPanel`) no rehace su escenario al aceptar, así que no tiene esto.

Prueba: con `ID-CEA-test-2therapies-new-test`, ámbito de «Therapy», elegir «Dec: Test = yes», llamar a `refreshScenarioFromEvidence` y comprobar que el escenario sigue diciendo «Dec: Test = yes» y «Test = negative»; y que, si entretanto la evidencia gana «Test = not done», «Dec: Test» pasa a «no».

### Error 277 · En el análisis de sensibilidad, después de un análisis con el ámbito de una decisión, elegir otra decisión, o pasar al modelo entero y volver, enseña el escenario con las filas fijas en lo que se eligió para el análisis anterior, y no se pueden cambiar hasta volver a aceptar

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:572-597`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/ScopePanel.java:264-280` |
| **Comprobado** | Leyendo el código y ejecutándolo (el panel del ámbito, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 262, error 87 |

**Qué pasa.** Menú Tools → «Sensitivity analysis», ámbito «One decision». El diálogo se queda abierto entre análisis. Después de aceptar uno, si el usuario elige otra decisión, o pasa al modelo entero y vuelve a «One decision», el recuadro «Scenario» sale con las filas fijas y apagadas en lo que se eligió para el análisis anterior: no puede elegir el escenario de la decisión nueva. Solo al

volver a aceptar se relee la evidencia de la red y las filas se sueltan; pero ese análisis ya se hace con los estados que se veían fijos. Para elegir otro escenario hay que aceptar una vez, con un análisis que no se quería, elegir y aceptar otra vez.

Por qué pasa: al aceptar, `doOkClick` (líneas 591-595) le añade al controlador el escenario elegido, como si fuera evidencia, y lo deja ahí. La próxima vez que se arma un escenario, `ScopePanel.getDecisionScenarioPanel` (línea 278) lo arma con `controller.getPreResolutionEvidence()`, que ahora lleva el escenario del análisis anterior, y `PossibleScenario` fija toda fila que esa evidencia nombra.

Medido el 15-10-2026 ejecutando `ScopePanel` sin pantalla, con las clases de `7ebcb51` delante del jar del 14 de octubre (y con el jar solo, igual), en `ID-CEA-test-2therapies-new-test.pgmx`: decisión «Therapy», eligiendo «Dec: Test = yes» y «Test = positive», aceptando y eligiendo después «Dec: New test» y otra vez «Therapy» (`!` es una fila fija):

```
elegido (Therapy):        Dec: New test=no,yes@0  Dec: Test=no,yes@1  New test=not done@0
↪  Test=negative,positive@1
evidencia del controlador tras aceptar: [Dec: New test:no, Dec: Test:yes, New test:not
↪  done, Test:positive]
otra decisión (Dec: New test):  Dec: Test=no,yes@1!  Test=not done,negative,positive@2!
otra vez Therapy:         Dec: New test=no,yes@0!  Dec: Test=no,yes@1!  New test=not
↪  done,negative,positive@0!  Test=not done,negative,positive@2!
```

**El código.** sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:591-595

```
            EvidenceCase newPreResolutionEvidence = new
↪  EvidenceCase(controller.getPreResolutionEvidence());
            for (Finding finding :
            ↪  controller.getSensitivityAnalysisModel().getSelectedScenario()) {
                newPreResolutionEvidence.addFinding(finding);
            }
            controller.setPreResolutionEvidence(newPreResolutionEvidence);
```

**Cómo arreglarlo.** No tocar la evidencia del controlador en `doOkClick` y pasarle al análisis una copia con el escenario añadido; o, si el modelo necesita guardarla, que `ScopePanel` arme el escenario con la evidencia de la red (`readPreResolutionEvidenceFromNetwork`) y no con la del controlador.

Prueba: con el panel del ámbito sobre esa red, elegir un escenario de «Therapy», hacer lo que hace `doOkClick` con la evidencia, elegir «Dec: New test» y comprobar que ninguna de sus filas sale fija.

## 10 Simulación de sucesos discretos

Las redes de simulación de eventos discretos (DES), en las que el tiempo avanza de suceso en suceso, y la simulación de Monte Carlo y la propagación estocástica.

### Error 95 · [Corregido] En la simulación de sucesos, una utilidad acumulativa que depende de un nodo de azar vale cero hasta que llega el primer suceso, y el resultado sale más bajo

| | |
|---|---|
| **Estado** | Corregido en el commit `508dd20` (15-10-2026), con la opción que eligió Manuel Arias: un nodo de azar con un suceso entre sus antecesores empieza en su primer estado, o en 0 si es numérico, como ya hacía sin decirlo (el valor que deja `clear()`); su relación solo dice qué pasa cuando ocurre un suceso. La regla queda escrita en `ChanceEvaluation.startSimulation`. `UtilityEvaluation.` |

| | |
|---|---|
| | `startSimulation` da valor al empezar a todas las utilidades acumulativas (`valueCumulativeUtilities`, antes `updateOrphanNodes`, que solo recorría las huérfanas), con los valores de sus padres en el instante 0. Prueba: una red de sucesos con horizonte de 10 años en la que el suceso que enfermaría llega en el año 20; la efectividad sale 10 y el coste 1100, donde antes salían 0 y 100. Para simular un individuo desde una prueba, `DESInference` tiene ahora un constructor que solo prepara la simulación y `simulateOneIndividual`, visibles solo en su paquete. La segunda prueba de regresión que propone la ficha (los estados de «Ill» en orden inverso dan el mismo resultado) no se cumple con la opción elegida: con ese orden el paciente empieza enfermo. No se ha probado en la ventana. |
| **Módulo** | gui, inference.des |
| **Paquete** | org.openmarkov.gui.desnets; org.openmarkov.inference.DES |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/desnets/UtilityEvaluation.java:50-72`<br>`gui/src/main/java/org/openmarkov/gui/desnets/UtilityEvaluation.java:80-103`<br>`gui/src/main/java/org/openmarkov/gui/desnets/UtilityRecord.java:87-96`<br>`gui/src/main/java/org/openmarkov/gui/desnets/ChanceEvaluation.java:51-89`<br>`gui/src/main/java/org/openmarkov/gui/desnets/GenericEvaluation.java:84-87`<br>`inference.des/src/main/java/org/openmarkov/inference/DES/DESRecord.java:120-123`<br>`inference.des/src/main/java/org/openmarkov/inference/DES/DESRecord.java:195-197`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:317-335` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué valor tiene en el instante cero un nodo de azar que tiene un suceso como antecesor (hoy, implícitamente, su primer estado o 0,0, sin muestrearlo), porque de ese valor depende la utilidad del primer tramo |
| **Relacionados** | error 96 |

**Qué pasa.** Una utilidad acumulativa en una red DES (simulación de eventos discretos) es una tasa —coste por año, calidad de vida— que se acumula multiplicando su valor instantáneo por el tiempo que lo mantiene. En `UtilityRecord` es acumulativa toda utilidad sin sucesos padres.

Si una utilidad acumulativa depende de un nodo de azar que un suceso puede cambiar —el caso normal: la calidad de vida depende de estar enfermo, y se enferma cuando ocurre el suceso—, desde el instante 0 hasta el primer suceso que la alcanza vale 0. La simulación no avisa: da una efectividad y un coste más bajos.

Camino de llegada: cualquier red DES con una utilidad acumulativa que cuelgue de un nodo de azar con un suceso antecesor, simulada desde el botón o menú de análisis de coste-efectividad (en una `DESNet`, `MainPanelListenerAssistant` desvía `COST_EFFECTIVENESS_DETERMINISTIC` a `monteCarloSimulation`).

El valor instantáneo de una utilidad se fija en dos momentos:

- al empezar cada individuo, `UtilityEvaluation.startSimulation` llama a `updateOrphanNodes`, que solo recorre `orphanRecords`: las utilidades «huérfanas», las que no tienen ningún suceso como antecesor (`DESRecord.isOrphanNode`);
- cuando ocurre un suceso, `UtilityEvaluation.update` recalcula las utilidades que ese suceso

alcanza.

Una utilidad que depende de un nodo de azar con un suceso antecesor no es huérfana, así que nadie le da valor al empezar: conserva el 0 que le pone `clear()`. El tramo desde el instante 0 hasta el primer suceso que la alcanza se acumula multiplicado por 0.

Los nodos de azar tienen el mismo origen: `ChanceEvaluation.startSimulation` solo evalúa los nodos de azar huérfanos (`addValuesOrphanNodes`). Un nodo de azar con un suceso antecesor empieza con `variableValue` = 0 de `clear()`, es decir, en su primer estado (o 0,0 si es numérico), sin muestrear su potencial. Para dar valor a la utilidad en el instante 0 hay que usar el valor de ese nodo en el instante 0, y ese valor hoy no lo define el algoritmo: de ahí la decisión de equipo.

El repositorio no tiene ninguna red de sucesos. La medida se hizo con una red de sucesos de prueba con:

- una decisión `Therapy`;
- un suceso `Onset` que hace pasar el nodo de azar `Ill` de «no» a «yes», y un suceso terminal `Death`;
- tres utilidades acumulativas: `QALY` = 1,0 si `Ill`=no y 0,6 si yes; `CareCost` = 100 si no y 2000 si yes; `Monitoring` = 10, que solo depende de la decisión;
- horizonte de 10 años, una simulación con la traza textual activada.

Para `Therapy`=none, `Onset` ocurre en el año 3 y `Death` en el 6. La traza dice:

```
Accrue cumulative utilities/payoffs from clock=0 to clock=3 (interval length 3)
    Monitoring   Instantaneous value in the interval: 10.0
    QALY         Instantaneous value in the interval: 0.0
    CareCost     Instantaneous value in the interval: 0.0
```

`Monitoring` es huérfana y vale lo que debe. `QALY` y `CareCost` valen 0 durante tres años en los que el paciente está sano. El resumen da efectividad media 1,8 (solo 0,6 × 3 años enfermo) cuando el modelo dice 4,8 (3 × 1,0 + 3 × 0,6), y coste 6120 cuando serían 6420 (faltan 3 × 100).

En esa red, el valor inicial de `Ill` coincide con lo correcto porque «no» es su primer estado; con los estados en otro orden el paciente empezaría enfermo.

Para reproducirlo hay que construir en el editor una red de sucesos como la descrita.

**El código.** gui/src/main/java/org/openmarkov/gui/desnets/UtilityEvaluation.java:50-72

```java
    @Override
    public void startSimulation(Finding decisionFinding) throws OpenMarkovException {
        desRecordHashMap.values().forEach(UtilityRecord::clear);
        this.decisionFinding = decisionFinding;
        updateOrphanNodes();
        previousEvaluationTime = 0;
    }

    /**
     * Update orphan UTILITY nodes
     */
    void updateOrphanNodes() throws OpenMarkovException {
        for (var utilityRecord : orphanRecords) {
                Configuration configuration = new Configuration();
                if (utilityRecord.hasDecisionParent()) {
                    configuration.addFinding(decisionFinding);
                }
                for (Node chanceParent : utilityRecord.getParentsByType(NodeType.CHANCE)) {
                    Variable parentVariable = chanceParent.getVariable();
```

```
                    configuration.addFinding(parentVariable,
↪  desInference.getChanceEvaluation().getDESRecord(chanceParent).getVariableValue());
                }
                utilityRecord.setVariableValue(configuration);
            }
        }
```

gui/src/main/java/org/openmarkov/gui/desnets/UtilityRecord.java:87-90

```
    public void accrueCumulativeUtility(double previousEvaluationTime, double clock) {
        //Cumulative Utility
        double utility = variableValue * (clock - previousEvaluationTime);
        accruedUtility += utility;
```

**Cómo arreglarlo.** Dar valor al empezar a todas las utilidades acumulativas, no solo a las huérfanas, con los valores de sus padres en el instante 0. Para eso hace falta antes que todos los nodos de azar tengan en el instante 0 un valor que el modelo respalde, que es la decisión de equipo: hoy los no huérfanos tienen el 0 implícito de `clear()`.

Sitios que tienen que cumplir la misma regla («en el instante 0 todo nodo con valor instantáneo tiene el valor que dicta el modelo»):

- `ChanceEvaluation.startSimulation` / `addValuesOrphanNodes` (nodos de azar).
- `UtilityEvaluation.startSimulation` / `updateOrphanNodes` (utilidades).
- `EventEvaluation.updateOrphanEvents`, que calcula los tiempos de los sucesos huérfanos con los valores de sus padres de azar en el instante 0.
- `GenericEvaluation.setOrphanNodes` y `DESRecord.isOrphanNode`, que definen hoy quién se evalúa al empezar.

`UtilityEvaluation.accrueImmediateUtility` construye una configuración que no usa (líneas 116-124); no es este defecto, pero conviene no copiar ese patrón.

Pruebas de regresión: la red descrita (sano al empezar, suceso en el año 3, muerte en el 6, con números exactos) debe dar efectividad 4,8 y coste 6420 para `Therapy`=none; y otra con los estados de `Ill` en orden inverso debe dar el mismo resultado.

### Error 96 · La simulación de sucesos se cae cuando un suceso actualiza un nodo de azar cuyo padre de azar depende de otro suceso que aún no ha ocurrido

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.desnets |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/desnets/ChanceEvaluation.java:63-89,97-113`<br>`gui/src/main/java/org/openmarkov/gui/desnets/ChanceRecord.java:91-124`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:260-312` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 95, error 97, error 98 |

**Qué pasa.** En una red DES (simulación de eventos discretos), si un suceso E1 actualiza un nodo de azar A que tiene como padre de azar un nodo B que depende de otro suceso E2 aún no ocurrido, la simulación se cae con `IndexOutOfBoundsException` en cuanto ocurre E1. El mensaje no dice qué nodos se esperan. No hace falta ningún ciclo en la red.

Camino de llegada: construir en el editor una red DES así (los enlaces y la tabla de transiciones con dos padres se crean desde la interfaz) y lanzar la simulación con el botón o menú de coste-efectividad. La excepción no la captura nadie en el hilo que lanza `MainPanelListenerAssistant.monteCarloSimulation`, así que llega al manejador global `OMExceptionHandler`, que abre el diálogo de error inesperado (esto último, por lectura).

Cuando ocurre un suceso, `ChanceEvaluation.update` recalcula los nodos de azar descendientes de ese suceso. Para respetar el orden (un nodo después de sus padres de azar) repite: busca el primer pendiente cuyos padres de azar estén todos marcados como evaluados (`ChanceRecord.isUpdatable`), lo calcula y lo quita de la lista. La búsqueda es `while (!remaining.get(i).isUpdatable()) ++i;`, sin tope: si ningún pendiente está listo, `i` llega al tamaño de la lista y `get(i)` lanza la excepción.

`isUpdatable` exige que **todos** los padres de azar estén marcados como evaluados, también los que no están entre los pendientes. Un nodo de azar con un suceso antecesor no se evalúa al empezar la simulación (ver error 95) y su marca `evaluated` queda a `false` desde `clear()` hasta que ocurre su propio suceso. Por eso A nunca está «listo» y la lista revienta.

El otro bucle igual, en `addValuesOrphanNodes` (línea 72), solo recorre nodos huérfanos, cuyos padres de azar son también huérfanos y están en la misma lista; allí solo se llegaría con un ciclo entre nodos de azar, y ese caso se cae antes en otro sitio (error 97).

El repositorio no tiene ninguna red de sucesos. La medida se hizo con una red de sucesos de prueba con dos sucesos: `Onset` (año 3), que hace pasar `Ill` a «yes», y `Accident` (año 5), que hace pasar `Injured` a «yes»; y un enlace `Injured → Ill`, con la tabla de `Ill` sobre `[Ill, Onset, Injured]`. Al ocurrir `Onset` la simulación termina con `IndexOutOfBoundsException: Index 1 out of bounds for length 1` en `ChanceEvaluation.update` (línea 107).

**El código.** gui/src/main/java/org/openmarkov/gui/desnets/ChanceEvaluation.java:103-110

```
        remainingDescendants.forEach(ChanceRecord::resetEvaluationStatus);

        while (!remainingDescendants.isEmpty()) {
            int i = 0;
            while (!remainingDescendants.get(i).isUpdatable())
                ++i;
            ChanceRecord chanceRecord = remainingDescendants.remove(i);
            computeChanceValue(chanceRecord, eventHappened);
        }
```

gui/src/main/java/org/openmarkov/gui/desnets/ChanceRecord.java:121-124

```
    public boolean isUpdatable() {
        //08/03/2023; Self-loop-->first extracted parents different from node
        return chanceRecordParents.stream().filter(chanceRecord ->
        ↪  !chanceRecord.equals(this)).allMatch(ChanceRecord::isEvaluated);
    }
```

**Cómo arreglarlo.** Dos cambios que van juntos:

- Que un padre de azar solo haga esperar si está entre los nodos que se van a recalcular en esta pasada; los demás ya tienen su valor vigente. Hoy la marca `evaluated` mezcla «calculado en esta pasada» con «calculado alguna vez».

- Poner tope a la búsqueda y, si no queda ningún pendiente listo, lanzar una excepción que nombre los nodos que se esperan entre sí (solo posible con un ciclo).

Sitios con la misma regla: `ChanceEvaluation.update` (líneas 105-110), `ChanceEvaluation.addValuesOrphanNodes` (líneas 70-73), `ChanceRecord.isUpdatable` / `resetEvaluationStatus` / `clear`.

El valor que tiene un nodo de azar no huérfano antes de su primer suceso es la decisión de equipo de error 95. Este arreglo no la necesita (basta con no esperar al padre), pero la simulación usará ese valor.

Prueba de regresión: la red descrita (dos sucesos independientes, el nodo actualizado por el primero con un padre de azar que depende del segundo) debe simular sin excepción, y `Ill` debe pasar a «yes» en el año 3.

### Error 97 · [Corregido] Un ciclo entre nodos de azar en una red de sucesos impide simular: hoy desborda la pila y, detrás, hay un bucle infinito

| | |
|---|---|
| **Estado** | Corregido en el commit `0cb2697` (09-10-2026): restricción nueva `EveryCycleHasAnEvent`, activa en las redes de sucesos, que rechaza el enlace o la inversión que cerraría un ciclo sin ningún nodo de suceso y lo señala en una red que ya lo tenga; la simulación rechaza esa red con el mismo mensaje («Cannot simulate»), visto en la ventana. El lazo de un nodo de azar sobre sí mismo sigue permitido. Los otros dos recorridos sin fin de la ficha (`EventRecord.findDescendants` y los bucles de `ChanceEvaluation`) quedan sin tocar: con la restricción no se llega a ellos. El `StackOverflowError` no se volvió a medir al corregir. No se ha contrastado el resto de las restricciones del apéndice A del artículo con el programa |
| **Módulo** | gui, inference.des |
| **Paquete** | org.openmarkov.gui.desnets; org.openmarkov.inference.DES |
| **Ficheros** | `inference.des/src/main/java/org/openmarkov/inference/DES/DESRecord.java:153-178`<br>`gui/src/main/java/org/openmarkov/gui/desnets/EventRecord.java:84-126`<br>`gui/src/main/java/org/openmarkov/gui/desnets/GenericEvaluation.java:64-81`<br>`gui/src/main/java/org/openmarkov/gui/desnets/ChanceEvaluation.java:70-73`<br>`core/src/main/java/org/openmarkov/core/model/network/type/DESNetworkType.java:30-43` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No. Pedía decidir si una red DES puede tener ciclos entre nodos de azar. Lo resuelve el artículo que define estas redes (Yago y Díez, «DESnets: A Graphical Representation for Discrete Event Simulation and Cost-Effectiveness Analysis», *Mathematics* 11(7):1602, 2023, apéndice A): «Every directed cycle involving more than one node contains at least one event». Manuel Arias decidió el 09-10-2026 aplicar esa restricción |
| **Relacionados** | error 96, error 106 |

**Qué pasa.** El tipo de red `DESNet` (red de simulación de eventos discretos, DES) desactiva a propósito las restricciones `NoCycle`, `NoLoops` y `NoSelfLoop`, para que un suceso pueda volver a citarse a sí mismo. Por eso el editor deja crear dos nodos de azar que dependen el uno del otro: medido, en una red DES, `AddLinkEdit` acepta `Ill → Mood` y después `Mood → Ill`.

Con esa red, la simulación no llega a empezar: sale un `StackOverflowError`, que en la aplicación

llega al manejador global `OMExceptionHandler` y se muestra como error inesperado (por lectura), sin decir que el problema es un ciclo.

Al construir los registros de los nodos de azar, el constructor de `DESRecord` llama a `setEvent Ancestors`, y `addEventAncestors` sube por los padres recursivamente sin llevar cuenta de los ya visitados; solo se salta el propio nodo (lazo sobre sí mismo). Con `Ill ↔ Mood` va de uno a otro hasta desbordar la pila.

Medido: `StackOverflowError` en `DESRecord.addEventAncestors` (líneas 166/174), que `Generic Evaluation` recibe envuelto en `InvocationTargetException` y relanza como `Unreachable Exception`.

Detrás hay otros dos puntos con el mismo defecto, que hoy no se ejecutan porque la construcción ya ha fallado (comprobados leyendo):

- `EventRecord.findDescendants` baja por los nodos de azar hijos con una lista de pendientes y solo comprueba si el hijo **ya está en la lista**, no si ya se visitó: con `A → B → A` saca A y mete B, saca B y vuelve a meter A, y así sin fin. La aplicación se quedaría colgada en el hilo de la simulación, sin excepción.
- Los bucles sin tope de `ChanceEvaluation` (error 96) lanzarían `IndexOutOfBoundsException` porque ninguno de los dos nodos está listo antes que el otro.

Un ciclo que pase por un nodo de suceso (A → E → B → A) no provoca la recursión, porque `add EventAncestors` se detiene en los sucesos.

El repositorio no tiene ninguna red de sucesos: para reproducirlo basta con construir una red de sucesos con dos nodos de azar enlazados en los dos sentidos (`Ill → Mood` y `Mood → Ill`) y simularla.

**El código.** inference.des/src/main/java/org/openmarkov/inference/DES/DESRecord.java:165-178

```
    protected void addEventAncestors(Node node) {
        List<Node> parents = node.getParents();
        if (parents.isEmpty()) {
            return;
        }
        for (Node parent : parents) {
            if (parent.getNodeType() == NodeType.EVENT) {
                eventAncestors.add(parent);
            } else if (!parent.equals(node)) {
                addEventAncestors(parent);
            }
        }

    }
```

gui/src/main/java/org/openmarkov/gui/desnets/EventRecord.java:107-118

```
        while (!pending.isEmpty()) {
            Node chanceNode = pending.remove(0);
            chanceDescendants.add((ChanceRecord)
↪   desInference.getChanceEvaluation().getDESRecord(chanceNode));
            for (Node child : chanceNode.getChildren()) {
                //08/03/2023 - self-loop
                if (child.equals(chanceNode)) continue;
                switch (child.getNodeType()) {
                    case CHANCE:
```

```
                    if (!pending.contains(child)) {
                        pending.add(child);
                    }
                    break;
```

**Cómo arreglarlo.** Primero hay que tomar la decisión de equipo: si esos ciclos tienen sentido en un modelo DES.

Si no lo tienen: una restricción del tipo de red que prohíba ciclos que no pasen por un nodo de suceso. Como toda restricción, debe imponerse en los dos sitios: `PNConstraint.checkProbNet` (red completa, por ejemplo al abrir un fichero) y `PNEdit.checkConstraintsWillBeMet` (la edición que añadiría el enlace, `AddLinkEdit`, y las que lo crean de otra forma, como invertir enlaces o pegar). Aun así conviene que la simulación rechace la red con un mensaje claro.

Si lo tienen: hay que definir el orden de evaluación y hacer que los tres recorridos terminen:

- `DESRecord.addEventAncestors`: llevar un conjunto de nodos visitados.
- `EventRecord.findDescendants`: llevar un conjunto de visitados, no solo mirar la lista de pendientes.
- `ChanceEvaluation.addValuesOrphanNodes` y `ChanceEvaluation.update`: tope y mensaje que nombre los nodos que se esperan (ver error 96).

Prueba de regresión: una red DES con `Ill ↔ Mood` produce el mensaje decidido (rechazo del enlace o de la red), o simula y termina, según la decisión; en ningún caso `StackOverflowError` ni cuelgue (con tiempo límite en la prueba).

### Error 98 · En una red de sucesos, un nodo cuya tabla o valor exacto incluye un suceso padre hace caer la simulación en cuanto ocurre ese suceso

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/Finding.java:82-91`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:238-268`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:721-738`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExactDistrPotential.java:158-160`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ExactDistrPotential.java:219-223`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:224-229`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:149-161`<br>`gui/src/main/java/org/openmarkov/gui/action/ChangeNodeTypeEdit.java:48-63`<br>`gui/src/main/java/org/openmarkov/gui/desnets/ChanceEvaluation.java:136-138`<br>`gui/src/main/java/org/openmarkov/gui/desnets/UtilityEvaluation.java:90-92`<br>`gui/src/main/java/org/openmarkov/gui/desnets/EventEvaluation.java:96-101` |
| **Comprobado** | Leyendo el código y ejecutándolo |

| | |
|---|---|
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Cómo deben tratar las variables de suceso los potenciales que no las entienden (tabla, valor exacto): prohibirlas y convertir el potencial al enlazar, ignorarlas al leer la casilla, o dar al hallazgo de un suceso un estado válido |
| **Relacionados** | error 96, error 99 |

**Qué pasa.** En una red DES (simulación de eventos discretos), un nodo cuyo potencial es una tabla (`TablePotential`) o un valor exacto (`ExactDistrPotential`) y que tiene un suceso entre sus variables hace caer la simulación con `ArrayIndexOutOfBoundsException` en cuanto ocurre ese suceso. En los tres caminos medidos la excepción no la captura el hilo de la simulación y termina, por lectura, en el diálogo de error inesperado de `OMExceptionHandler`.

Nada en la interfaz dice que un nodo con un suceso padre necesita un potencial de sucesos; se descubre al simular. La ventana de edición de potenciales ya evita crear estos potenciales: para un nodo cuya lista de variables incluye un suceso no ofrece ni «Table» ni «Exact» (medido con `PotentialUtils.getFilteredPotentialClasses`; `ExactDistrPotential.validate` rechaza explícitamente las variables de suceso en una `DESNet`). Pero no retira el potencial que ya tiene el nodo, y ni el enlace (`AddLinkEdit`), ni el cambio de tipo (`ChangeNodeTypeEdit`), ni el lector de ficheros comprueban esa misma regla.

Cuando ocurre un suceso, los nodos hijos de ese suceso se recalculan con una configuración que incluye el suceso ocurrido: `ChanceEvaluation.computeChanceValue`, `UtilityEvaluation.update` y `EventEvaluation.updateChildEvent` llaman a `configuration.addFinding(variableDelSuceso, instante)`. Para una variable de suceso, `Finding(Variable, double)` guarda como índice de estado `Integer.MAX_VALUE`.

Los potenciales que saben de sucesos (`TransitionTablePotential`, `DistributionTablePotential`, `UnivariateDistrPotential`, los árboles de sucesos) traducen ese hallazgo a su variable interna `Events`. Pero una `TablePotential` o un `ExactDistrPotential` cuya lista de variables incluya el suceso calculan la casilla como `desplazamiento × índice`, y el entero desborda: la lectura cae fuera de la tabla.

El repositorio no tiene ninguna red de sucesos. Las medidas se hicieron con una red de sucesos de prueba con un suceso `Onset` que ocurre en el año 3, por tres caminos:

- **Nodo de azar enlazado desde un suceso sin cambiar su potencial.** Se crea un nodo de azar `Pain` (su potencial por omisión es una tabla) y se enlaza `Onset → Pain` con `AddLinkEdit`, como hace el editor. `TablePotential.addVariable` mantiene la tabla, ahora sobre `[Pain, Onset]`. Al simular, cuando ocurre `Onset`: `ArrayIndexOutOfBoundsException: Index -2 out of bounds for length 2` en `TablePotential.sampleConditionedVariable`.
- **Nodo cambiado a utilidad en la ventana del nodo.** Un nodo de azar enlazado desde `Onset` se cambia a utilidad con `ChangeNodeTypeEdit` (lo que ejecuta el panel de definición del nodo); como su tabla no vale para una utilidad, recibe el potencial por omisión de `PotentialUtils.generateDefaultPotential`, un `ExactDistrPotential` sobre la variable y **todos** sus padres, suceso incluido. Al simular: `ArrayIndexOutOfBoundsException: Index 2147483647 out of bounds for length 1` en `TablePotential.getValue`, llamado desde `UtilityEvaluation.update`.
- **Fichero.** La misma red con el potencial exacto de la utilidad `TreatmentCost` escrito sobre `[TreatmentCost, Onset, Therapy]`: `Index 2147483647 out of bounds for length 2`. La tabla sin `Onset` contesta lo que debe.

Para reproducir los dos primeros basta con construir en el editor una red de sucesos con un suceso y enlazarlo a un nodo de azar nuevo, o a uno que después se cambia a utilidad.

**El código.** core/src/main/java/org/openmarkov/core/model/network/Finding.java:82-91

```
public Finding(Variable variable, double numericalValue) {
    // TODO Throw exception if numerical values is outside the domain of variable
    this.variable = variable;
    this.numericalValue = numericalValue;
    if (variable.getVariableType() == VariableType.DISCRETIZED) {
        stateIndex = this.variable.getStateIndex(this.numericalValue);
    } else {
        this.stateIndex = Integer.MAX_VALUE;
    }
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:721-729

```
public double sampleConditionedVariable(double[] randomNumbers, EvidenceCase parents) {
    int index = 0;
    int sampleIndex = 0;
    // find index of first position for the given configuration
    for (int i = 1; i < variables.size(); ++i) {
        index += parents.getState(variables.get(i)) * offsets[i];
    }
    double accumulatedProbability = values[index + sampleIndex];
```

core/src/main/java/org/openmarkov/core/model/network/potential/ExactDistrPotential.java:219-223

```
public static boolean validate(Node node, List<Variable> variables, PotentialRole role)
↪ {
    //11/01/2023 FIXME Provisional; validate for DESnets
    if ((node.getProbNet().getNetworkType() instanceof DESNetworkType)){
        return !variables.stream().anyMatch(variable ->
        ↪  variable.getVariableType()==VariableType.EVENT) &&
        ↪  node.getVariable().getVariableType() == VariableType.NUMERIC;
    }
```

**Cómo arreglarlo.** Hace falta la decisión de equipo sobre el tratamiento. Opciones, sin elegir:

- La regla que ya aplica `validate` —un potencial que no entiende sucesos no puede tener variables de suceso— se impone también al enlazar (sustituyendo el potencial por uno de sucesos o por uno válido), al cambiar el tipo de nodo (que `generateDefaultPotential` no incluya los sucesos o elija un potencial de sucesos en una `DESNet`) y al leer un fichero (error de lectura con el nombre del nodo).
- Que `TablePotential` y `ExactDistrPotential` ignoren las variables de suceso al calcular la casilla.
- Que el hallazgo de un suceso tenga un índice de estado válido.

Sitios donde tendría que cumplirse la regla elegida:

- `AddLinkEdit.doEdit` (rama general, `oldPotential.addVariable`), y las otras ediciones que añaden padres (invertir enlace, pegar).
- `ChangeNodeTypeEdit.doMultiStepEdit` y `PotentialUtils.generateDefaultPotential`.
- `PGMXReader_1_0` al construir potenciales de una `DESNet`.
- `ExactDistrPotential.validate` y la validación de `TablePotential` para `DESNet`.
- En la simulación, `ChanceEvaluation.computeChanceValue`, `UtilityEvaluation.update` y `EventEvaluation.updateChildEvent`, que construyen el hallazgo del suceso; y, para no leer fuera de la tabla sin decirlo, `TablePotential.getValue` / `sampleConditionedVariable` con

un índice de estado fuera de rango.

Pruebas de regresión: los tres casos medidos (nodo de azar con tabla por omisión enlazado desde un suceso, nodo cambiado a utilidad, fichero con valor exacto sobre el suceso) deben o bien ser rechazados con un mensaje en el momento de la edición o de la lectura, o bien simular sin excepción, según la decisión.

### Error 99 · En una red de sucesos, añadir o quitar un enlace cambia la relación del nodo hijo por una uniforme que la simulación no admite, y simular termina en un error inesperado que no dice qué nodo es

| | |
|---|---|
| **Estado** | Presente en parte. Con un nodo recién creado al que aún no se le ha abierto la relación, enlazarlo ya no le deja una relación uniforme sino ninguna, y antes de simular sale un mensaje normal que nombra el nodo sin potencial (commits `710de2c` y `83ff8ab`). Sigue sin corregir para todo nodo que ya tuviera relación: añadir o quitar un enlace, o un bucle, la cambia por `UniformPotential` perdiendo sus números, y la simulación sigue terminando en la `ClassCast Exception` de `DESRecord` como error inesperado sin decir el nodo. |
| **Módulo** | core, gui, inference.des |
| **Paquete** | org.openmarkov.core.action.base.linkEdits; org.openmarkov.core.model.network.potential; org.openmarkov.gui.desnets; org.openmarkov.inference.DES |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:189-234`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/RemoveLinkEdit.java:92-116`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:700-736`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:149-156`<br>`inference.des/src/main/java/org/openmarkov/inference/DES/DESRecord.java:107`<br>`gui/src/main/java/org/openmarkov/gui/desnets/GenericEvaluation.java:64-81`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:394-443`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:289-312`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/EmptyPotentialPanel.java:19`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/DistributionTablePotentialPanel.java:66-72`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TransitionTablePotentialPanel.java:24-25` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué relación debe quedar en un nodo de una red de sucesos cuando se le añade o se le quita un padre: una que la simulación admita y conserve los números que se puedan (como hace una tabla corriente), o la uniforme de hoy, obligando a editarla antes de simular |
| **Relacionados** | error 21, error 98, error 101, error 106 |

**Qué pasa.** En una red DES (simulación de eventos discretos), el usuario enlaza en el editor un nodo con una utilidad o con un suceso. Por ejemplo, traza un enlace de un suceso a una utilidad nueva, o de un nodo de azar a la tabla de un suceso que ya tenía sus parámetros escritos. Si después lanza la simulación (botón o menú de coste-efectividad, que en una `DESNet` llama a `monte`

`CarloSimulation`) sin haber abierto y cambiado la relación de ese nodo, la simulación no empieza. Sale el diálogo de error inesperado con una `ClassCastException` que no dice qué nodo tiene la relación que no vale. Si el nodo tenía números (los parámetros de un suceso, los valores de una utilidad), se han perdido sin aviso.

Por qué pasa, en dos piezas:

**1. Las ediciones de enlaces dejan una relación uniforme.** `AddLinkEdit.doEdit` (líneas 189-234) actualiza la relación del hijo con `oldPotential.addVariable(padre)`. `Potential.addVariable` (líneas 700-717), que es la versión de la clase base, devuelve un `UniformPotential` nuevo con todas las variables y ningún número. Solo algunas clases lo redefinen: `TablePotential` amplía su tabla, pero ninguna de las relaciones de sucesos lo hace. Es lo que les pasa a:

- las utilidades, cuya relación por omisión en una red de sucesos es `ExactDistrPotential` (`PotentialUtils.generateDefaultPotential`, líneas 152-160);
- los sucesos, que empiezan con `DeltaPotential` y suelen tener `DistributionTablePotential`.

Lo mismo hacen otras dos ramas:

- el bucle sobre sí mismo de un suceso o de un nodo de azar, en `AddLinkEdit`, crea a propósito un `UniformPotential` (líneas 213-221);
- `RemoveLinkEdit.doEdit`, al quitar un enlace hacia una utilidad (líneas 92-116).

Y por último `Potential.removeVariable` (líneas 726-736), que usa `PasteEdit` al pegar un nodo sin sus padres (error 21).

**2. La simulación no comprueba las relaciones.** `DESInference.initialize` (líneas 392-441) construye un registro por nodo, y el constructor de `DESRecord` (línea 107) hace `(DESSimulable Potential) recordNode.getPotentials().getFirst()`. `UniformPotential` no implementa `DESSimulablePotential`, así que el molde falla. `GenericEvaluation` (líneas 63-80) envuelve la excepción en `UnreachableException` sin decir el nodo (guarda `lastNode`, pero no lo usa). En la aplicación, el hilo que crea `monteCarloSimulation` (líneas 435-459) solo captura `IOException` y `OpenMarkovException`, así que la excepción llega al manejador global y se muestra como error inesperado (esto último, por lectura).

La interfaz lo sabe a medias. Si el usuario abre el diálogo del nodo, ve la relación «Uniform» con un panel vacío (`EmptyPotentialPanel`) y puede elegir otra. Los comentarios de `DistributionTablePotentialPanel` (líneas 66-72, con un `FIXME`) y de `TransitionTablePotentialPanel` (líneas 24-25) dicen que al añadir o quitar un enlace el nodo se queda con una relación uniforme. Pero nada avisa antes de simular.

El repositorio no tiene redes de sucesos. Medido con una red de sucesos de prueba que simula sin problemas (una decisión `Therapy`, un nodo de azar `Sex`, un suceso `Onset` con tabla de distribución sobre `Therapy` y `Sex`, un suceso terminal `Death` con tabla de distribución sobre `Onset`, y las utilidades `QALY` y `Monitoring`), aplicando las mismas ediciones que el editor y lanzando la simulación:

```
utilidad nueva U, sin enlaces:          ExactDistrPotential  -> simula
enlace Onset -> U (utilidad nueva):     UniformPotential [U, Onset]                    ->
↪  falla
enlace Sex -> Monitoring:               UniformPotential [Monitoring, Therapy, Sex]    ->
↪  falla
suceso nuevo Relapse, enlace Onset -> Relapse: DeltaPotential pasa a UniformPotential  ->
↪  falla
enlace Sex -> Death:                    UniformPotential [Death, Onset, Sex] (se pierde el
↪  4,0) -> falla
quitar el enlace Therapy -> Monitoring: UniformPotential [Monitoring]                   ->
↪  falla
```

```
bucle Onset -> Onset:                    UniformPotential [Onset, Therapy, Sex, Onset]   ->
↪  falla
nodo de azar nuevo, enlace Onset -> nodo: TablePotential (ampliada), no uniforme
```

El último caso no queda uniforme, pero es el de error 98: esa tabla con un suceso hace caer la simulación cuando el suceso ocurre.

En todos los casos que fallan, el error es el mismo:

```
UnreachableException: InvocationTargetException
  causa: ClassCastException: class UniformPotential cannot be cast to class
  ↪  DESSimulablePotential
    at DESRecord.<init>(DESRecord.java:107)
    at UtilityRecord.<init>  /  EventRecord.<init>
    at GenericEvaluation.<init>(GenericEvaluation.java:72)
    at DESInference.initialize(DESInference.java:439 / 441)
```

**El código.** core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:225-231

```
            } else {
                for (Potential oldPotential : oldPotentials) {
                    // Update potential
                    Potential newPotential =
↪  oldPotential.addVariable(nodeFrom.getVariable());
                    newPotentials.add(newPotential);
                }
            }
```

core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:700-706

```
    public Potential addVariable(Variable variable) {
        Potential newPotential;
        if (!variables.contains(variable)) {
            List<Variable> newVariables = new ArrayList<>(variables);
            newVariables.add(variable);
            newPotential = new UniformPotential(newVariables, role);
        } else {
```

inference.des/src/main/java/org/openmarkov/inference/DES/DESRecord.java:107

```
        recordPotential = (DESSimulablePotential) recordNode.getPotentials().getFirst();
```

**Cómo arreglarlo.** Dos partes.

**La simulación debe rechazar la red con un mensaje.** Esta parte no necesita decisión. Antes de construir los registros, `DESInference.initialize` debe comprobar que la relación de cada nodo de azar, suceso y utilidad es simulable (`DESSimulablePotential`) y válida para sus padres. Si no lo es, debe avisar nombrando los nodos, igual que ya avisa cuando faltan criterios o hay más de una decisión. `GenericEvaluation` no debería convertir en `UnreachableException` un fallo que depende de la red del usuario.

**Qué relación dejan las ediciones.** Esta parte necesita la decisión de equipo. Opciones:

- Que las relaciones de sucesos (`ExactDistrPotential`, `DistributionTablePotential`, `DeltaPotential`…) redefinan `addVariable` y `removeVariable` para ampliar o reducir su tabla conservando los números, como hace `TablePotential`.
- Que en una `DESNet` la edición ponga la relación por omisión válida para los padres nuevos (la que ofrezca primero `PotentialUtils.getFilteredPotentialClasses`).
- Dejar la uniforme y confiar en el aviso de la simulación.

Sitios que producen hoy la relación uniforme y deben seguir la regla que se elija:

- `AddLinkEdit.doEdit`, en la rama general y en la del bucle sobre sí mismo (líneas 213-221).
- `RemoveLinkEdit.doEdit`, en las ramas de utilidad (líneas 97-116) y en la general.
- `Potential.addVariable` y `Potential.removeVariable`.
- `PasteEdit`, al quitar los padres que no se copian (error 21).
- Las ediciones que cambian los padres de otra forma (invertir un enlace).

Cuando `SumPotential` sustituye a la relación de una utilidad cuyos padres son todos numéricos, en `AddLinkEdit` (líneas 195-205) y en `RemoveLinkEdit` (líneas 99-106), tampoco es una relación simulable, por lectura.

Pruebas de regresión:

- Con la red de prueba descrita, cada una de las ediciones medidas debe dejar una relación que simula o, según lo decidido, hacer que la simulación se niegue con un mensaje que nombre el nodo; nunca `ClassCastException`.
- Añadir un padre a un suceso con tabla de distribución conserva sus parámetros, si se elige esa opción.

### Error 100 · Una tabla de vida marcada «con tasas» se simula como si fueran probabilidades

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/PiecewiseExponentialPotential.java:56-59,238-277,369-375`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/PiecewiseExponentialPanel.java:84-86,114`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:535-536`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_1_0.java:432-434` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si se implementa la variante con tasas (y se enciende la casilla) o se retira el campo, que hoy promete algo que no hace |
| **Relacionados** | error 101 |

**Qué pasa.** Un fichero con `<Rates>true</Rates>` en una tabla de vida se abre, se enseña con la casilla «Use rates» marcada, se guarda igual y se simula tratando las tasas como probabilidades, sin aviso. Ninguna red del repositorio lleva `<Rates>`, y desde la interfaz el campo siempre queda en `false`, así que hoy solo se llega con un fichero escrito a mano o por otra herramienta.

`PiecewiseExponentialPotential` (módulo `core`) es la relación «exponencial por tramos» o tabla de vida de la simulación de eventos discretos (DES): una tabla de pares (inicio del tramo, valor) con la que se sortea cuándo ocurre un suceso. Tiene un campo `useRates` cuyo comentario (líneas 56-59) dice que es cierto «si la distribución está definida por tasas en vez de por probabilidades por unidad de tiempo (como una probabilidad anual o mensual)». Son dos cosas distintas: una tasa $\lambda$ y una probabilidad por unidad de tiempo p se relacionan por $p = 1 - e^{\wedge}(-\lambda)$.

El sorteo (`sampleConditionedVariable`, líneas 238-277) y el precálculo de la tabla (`setPiecewiseTable`) usan siempre la fórmula de probabilidades, `Math.log(1 - pI)` y `Math.pow(1 - p, …)`, y no leen `useRates` en ningún sitio.

El campo solo se copia, se escribe en el fichero como `<Rates>` (`PGMXWriter_1_0`, líneas 432-434), se lee de vuelta (`PGMXReader_1_0`, líneas 535-536) y se enseña en el panel con una casilla «Use rates» que está apagada (`PiecewiseExponentialPanel`, líneas 86-88), así que el usuario no puede cambiarlo.

Medido con un pequeño programa de prueba: con la misma tabla y los mismos números aleatorios, el sorteo da `7.094838467442498` con `useRates=false` y con `useRates=true`.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/PiecewiseExponentialPotential.java:275

```
double sample = tI + (Math.log(u) - Math.log(prodAi))/Math.log(1 - pI);
```

gui/src/main/java/org/openmarkov/gui/dialog/common/PiecewiseExponentialPanel.java:86-88

```
ratesCheck = new JCheckBox("Use rates");
ratesCheck.setSelected(piecewiseExponentialPotential.isUseRates());
ratesCheck.setEnabled(false);
```

**Cómo arreglarlo.** Exige decidir entre:

- Implementar las tasas: convertir cada valor con p = 1 − e^(−λ) cuando `useRates` es cierto, en `setPiecewiseTable` (productos acumulados) y en `sampleConditionedVariable`, y encender la casilla. El campo debe tenerse en cuenta también en `equals`/copia si los hay, y la conversión debe hacerse en un solo sitio para que las dos cuentas no diverjan.
- Retirar la casilla y dejar de aceptar `true` en el lector (o avisar al leerlo), para que el fichero no diga algo que el programa no hace.

Prueba: con una tabla de un tramo de tasa λ, comprobar que la media de muchos sorteos con `useRates=true` se acerca a 1/λ, y con `false` y p se acerca a −1/ln(1 − p).

### Error 101 · La relación «External» se ofrece en el editor, elegirla lanza una excepción, y no trae ningún valor de fuera

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/ExternalPotential.java:17-24,67-70,137-144`<br>`gui/src/main/java/org/openmarkov/gui/desnets/ChanceEvaluation.java:63-89,123-161`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:117-134`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:228-253,488-522,584-587`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/PotentialPanelManager.java:95-117`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:117,515-519`<br>`integrationTests/src/test/java/org/openmarkov/integrationTests/gui/AllPotentialsHaveAPanelTest.java:13-16` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Si la relación «External» se completa (ligada al fichero de datos de Monte Carlo, con su panel) o se deja de ofrecer como tipo de relación |

| **Relacionados** | error 23, error 99, error 100, nota 71, nota 90, nota 96, nota 98, nota 162 |
|---|---|

**Qué pasa.** En una red de simulación de eventos discretos (DES), el selector de clase de relación del editor ofrece «External» para cualquier nodo de azar. Si el usuario la elige, sale un error no previsto (`NullPointerException`) porque no tiene panel, y el nodo no puede editarse. Y un nodo con esta relación (por ejemplo, leído de un fichero) no trae ningún valor de fuera: cuando el fichero de datos de la simulación no le da valor, la simulación sigue con un valor sin sentido en vez de avisar.

`ExternalPotential` (módulo `core`) se declara como tipo de relación con `@PotentialType(names = "External")` y su comentario dice que representa «un valor que se toma de una fuente externa, como un fichero». La clase no guarda ninguna fuente (ni fichero, ni columna, ni nombre) y su único método de simulación devuelve siempre `Double.MAX_VALUE` (líneas 137-144), que el comentario de la interfaz `DESSimulablePotential` usa como «no hay muestra». Medido: `sampleConditionedVariable` devuelve `1.7976931348623157E308`. El lector de ProbModelXML sí la reconoce (`PGMXReader_1_0`, líneas 117 y 515-519), así que un fichero puede traerla.

La simulación DES sí tiene un fichero de datos de entrada, elegido en las opciones de Monte Carlo (`MonteCarloOptions.getInputFilePath`, `DESInference`, línea 404). Al empezar cada individuo, `ChanceEvaluation.addValuesOrphanNodes` (módulo `gui`, líneas 63-89) toma del fichero el valor de cada nodo de azar **sin sucesos antecesores** cuyo nombre sea una columna del fichero, y solo si no lo encuentra muestrea su relación.

Esa lectura no depende de la clase de relación: vale igual para una tabla que para «External». Por tanto, «External» funciona de hecho como una etiqueta que no comprueba nada. Si no hay fichero, si el nombre del nodo no es una columna, o si el nodo tiene un suceso antecesor (entonces se recalcula con `computeChanceValue` sin mirar el fichero), la simulación usa `Double.MAX_VALUE` como valor del nodo (para una variable de estados, `(int) Double.MAX_VALUE` como índice de estado) sin avisar. Esto último es lectura del código; no se ha lanzado una simulación.

El selector de clase de relación del editor (`PotentialEditPanel.getPotentialTypeJCombobox`, líneas 227-241) ofrece todo tipo cuyo `validate` acepte el nodo (`PotentialUtils.getFilteredPotentialClasses`, líneas 121-138), sin mirar si tiene panel, y `ExternalPotential.validate` (líneas 67-70) acepta cualquier nodo de azar de una red DES. Medido con un pequeño programa de prueba en una red `DESNetworkType` con un nodo de azar: los tipos ofrecidos son `[ExternalPotential, DeltaPotential, WeibullHazardPotential, TablePotential]`.

Si el usuario la elige, `potentialTypeChanged` (líneas 476-507) instala la relación nueva en el nodo y recarga el panel; `PotentialPanelManager.createPotentialPanel` (líneas 95-114) busca el panel de esa clase, obtiene `null` y lanza `NullPointerException`. Medido llamando a ese método con un nodo que tiene `ExternalPotential`:

```
panel THREW java.lang.NullPointerException: Cannot invoke
↪  "java.lang.Class.getConstructor(java.lang.Class[])" because "potentialPanelClass" is
↪  null
```

La excepción no la caza nadie y llega al manejador global como error no previsto.

La prueba `AllPotentialsHaveAPanelTest` está desactivada precisamente porque este tipo, `IncrementPotential` y `TableWithEvents` no tienen panel (líneas 13-16).

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/ExternalPotential.java:67-70,141-144

```java
public static boolean validate(Node node, List<Variable> variables, PotentialRole role) {
    //28/08/2023 FIXME Provisional; only for DESnets
    return (node.getProbNet().getNetworkType() instanceof DESNetworkType) &&
    ↪  (node.getNodeType() == NodeType.CHANCE);
```

```
}
...
@Override
public double sampleConditionedVariable(double[] randomNumbers, EvidenceCase parents) {
    return Double.MAX_VALUE;
}
```

**Cómo arreglarlo.** Necesita la decisión de equipo: completar la relación o retirarla como tipo ofrecido.

Completarla supone decidir su relación con el fichero de datos de Monte Carlo (por ejemplo, que la simulación exija una columna para todo nodo «External» y avise antes de empezar si falta, o si el nodo tiene sucesos antecesores), qué guarda el fichero `.pgmx` y qué panel la edita.

Mientras no se decida, lo mínimo para que el usuario no tropiece es que `validate` devuelva `false` (o que el selector no ofrezca tipos sin panel) y que `createPotentialPanel` lance un error con mensaje en vez de `NullPointerException`.

La regla «no ofrecer lo que no se puede editar» afecta a los tres tipos sin panel y debe cumplirse en `PotentialUtils.getFilteredPotentialClasses` o en `PotentialEditPanel.getPotentialTypeJCombobox`. El lector de ficheros seguirá pudiendo traerlos, así que el diálogo también debe abrirse sin error para un nodo que ya la tenga.

Reactivar `AllPotentialsHaveAPanelTest` es la prueba natural.

### Error 102 · [Corregido] La tabla de resultados de la simulación de sucesos pone la columna de media descontada al criterio que no tiene descuento, y la media descontada del coste no se ve

| | |
|---|---|
| **Estado** | Corregido en el commit `765eb0a` (14-10-2026): `CriteriaValues.toString` recorre la lista de criterios, en el orden de la red, en vez del mapa. Medido simulando `io/src/test/resources/DES-illness.pgmx` (criterios «cost», con descuento del 3 %, y «effectiveness», sin él) y leyendo la tabla de la ventana de resultados: sin el arreglo, «effectiveness» lleva las columnas «Mean» y «Disc. Mean» y «cost» una sola (6113,33), sin su media descontada; con él, «cost» lleva 6113,33 y 5461,95 y «effectiveness» una columna. Con prueba de que el texto nombra los criterios en el orden de la lista con cuatro parejas de nombres; sin el arreglo fallan dos. Se eligió el primer arreglo propuesto; el de que la ventana reciba los objetos de resultados no se hizo. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.desnets |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/desnets/CriteriaValues.java:20,36-42,195-209`<br>`gui/src/main/java/org/openmarkov/gui/desnets/SimulationSummaryResults.java:188-216`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:375-388`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESResultsWindow.java:144-227` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 103 |

**Qué pasa.** En la tabla de resultados de una simulación de sucesos discretos (DES —simulación de eventos discretos—, redes de tipo «DESNet») con dos criterios, uno con descuento y otro

sin él, las columnas pueden asignarse al criterio equivocado. Con criterios llamados «Cost» (con descuento) y «Effectiveness» (sin él), la efectividad sale con dos columnas y el coste con una: la media descontada del coste, que es el resultado que se busca al poner descuento, no aparece en la tabla, y bajo «Disc. Mean» se enseña un valor del criterio que no tiene descuento. Con los nombres por omisión de una red nueva («Cost (€)», «Effectiveness (QALY)») no se nota.

Camino del usuario: red de tipo DESNet con dos criterios de decisión, uno con descuento y otro sin él, cuyos nombres caigan en orden distinto en el `HashMap` (por ejemplo «Cost» y «Effectiveness»); botón de coste-efectividad de la barra de herramientas (en una DESNet lanza la simulación, `MainPanelListenerAssistant`, líneas 308-311) → aceptar las opciones → tabla de resultados. En el repositorio no hay redes DESNet de ejemplo, así que no se ha ejecutado una simulación completa.

Al terminar la simulación, `DESInference.showResultsWindow` llama a `DESResultsWindow.presentResults` pasándole dos cosas por separado: el texto `simulationSummaryResults.toString()` y un vector `usingDiscount` que dice, criterio por criterio, si tiene descuento. La ventana no recibe objetos: trocea ese texto por puntos y comas (`generateTable`) y recorre a la vez los trozos y `usingDiscount`, poniendo dos columnas («Mean» y «Disc. Mean») a los criterios con descuento y una a los demás.

El problema es el orden. `usingDiscount` se construye recorriendo la lista de criterios de la red (`criteria.get(criterionNum)`). El texto lo escribe `CriteriaValues.toString()` recorriendo `criteriaValuesHashMap.keySet()`, un `HashMap<EqualCriterion, SimulationResults>` cuya clave se dispersa por el nombre del criterio: el orden depende de los nombres. Cuando los dos órdenes no coinciden, cada columna se asigna al criterio equivocado.

Medido con un pequeño programa de prueba que arma un texto con el formato real (criterios en el orden que da el `HashMap`) y llama por reflexión a `DESResultsWindow.generateTable`, con la red declarando «Cost» (con descuento: media 100, media descontada 90) y «Effectiveness» (sin descuento: media 2):

```
orden de la red [Cost (€), Effectiveness (QALY)] -> orden del texto [Cost (€),
↪  Effectiveness (QALY)]
orden de la red [cost, effectiveness]            -> orden del texto [effectiveness, cost]
orden de la red [Cost, Effectiveness]            -> orden del texto [Effectiveness, Cost]
con [Cost, Effectiveness]: cabeceras Effectiveness (dos columnas: Mean, Disc. Mean) | Cost
↪  (una columna)
fila de la serie 0, opción «none»: 2 | 2 | 100
```

**El código.** gui/src/main/java/org/openmarkov/gui/desnets/CriteriaValues.java:202-207

```java
String result = "";
for (EqualCriterion key : criteriaValuesHashMap.keySet()) {
    result += key.getCriterionName() + ";" + criteriaValuesHashMap.get(key).toString() +
↪  ";";
}
return result;
```

gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:375-378

```java
private void showResultsWindow(double simulationTime, JPanel psaPanel) {
    boolean[] usingDiscount = new boolean[criteria.size()];
    for (int criterionNum = 0; criterionNum < criteria.size(); criterionNum++)
        usingDiscount[criterionNum] = criteria.get(criterionNum).getDiscount() > 0;
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*, con la primera de estas opciones. Que el texto recorra los criterios en el orden de la lista (`CriteriaValues.criteria`, que ya existe) en vez de `keySet()` del mapa; o cambiar el mapa por un `LinkedHashMap` rellenado en el orden de la lista (constructor, líneas 36-42, y `setCriteriaValuesHashMap`, que puede sustituirlo). La solución de

fondo sería que la ventana reciba los objetos de resultados en vez de trocear un texto, pero es más trabajo.

Sitios afectados por el mismo orden: `generateTable` (tabla) y `saveCsv` (exportación). `saveCsv` toma los nombres de criterios de lo que `generateTable` dejó en el campo estático `criteria`, así que su cabecera y sus datos siguen el orden del texto y no se desalinean entre sí por esto.

`SimulationSummaryResults.toString` recorre también un `HashMap<String, CriteriaValues>` por nombre de opción, pero la ventana toma los nombres de las opciones del propio texto, así que ese orden no causa errores.

Prueba de regresión: la situación medida arriba, con criterios «Cost» (con descuento) y «Effectiveness» (sin él), comprobando que la cabecera con dos columnas es la de «Cost» y que la media descontada 90 aparece.

### Error 103 · [Corregido] El CSV exportado de la simulación de sucesos tiene la fila de títulos con una columna menos que las de datos, y una hoja de cálculo desalinea todas las columnas

| | |
|---|---|
| **Estado** | Corregido en el commit `d08082e` (14-10-2026): el texto del CSV lo construye `DESResultsWindow.csv`, sin selector de ficheros; la fila de títulos empieza por «SERIES» seguido de coma y ninguna fila termina en coma. Con prueba que construye los resultados con las clases reales (dos opciones, dos criterios, una serie) y exige que el primer título sea «SERIES» y que las dos filas tengan nueve columnas; con la lógica anterior falla. La fragilidad de borrar con `replaceAll` lo que empieza por una letra sigue como estaba. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.desnets |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/desnets/DESResultsWindow.java:97-126` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 102 |

**Qué pasa.** El botón «Export as CSV» (CSV: valores separados por comas) de la ventana de resultados de la simulación de sucesos discretos escribe un fichero cuya fila de títulos tiene una celda menos que las de datos. Al abrirlo en una hoja de cálculo, cada título queda encima de la columna de su vecina (el de «Mean cost for none» sobre la serie, etc.).

Camino del usuario: simular una red DESNet (botón de coste-efectividad de la barra de herramientas) → en la ventana de resultados, «Export as CSV» → abrir el fichero en una hoja de cálculo.

`DESResultsWindow.saveCsv` escribe un fichero separado por comas. La fila de títulos empieza con `sb.append("SERIES;")` y sigue con los títulos separados por comas. El punto y coma no separa nada en ese fichero, así que la primera celda es «SERIES;Mean cost for none» y la columna del número de serie queda sin título. Las filas de datos, en cambio, empiezan por el número de serie seguido de coma.

Comprobado con un pequeño programa de prueba que copia literalmente las líneas 89-117 (el método no se puede llamar sin pantalla porque abre un selector de ficheros) y las aplica a un texto con el formato que produce `SimulationSummaryResults.toString()`:

```
SERIES;Mean cost for none,Disc. Mean cost for none,Mean effectiveness for none,...,Disc.
↪  Mean effectiveness for therapy,
```

```
0,100.75,90.675,200.75,180.675,300.5,270.45,400.5,360.45,
fila de títulos: 9 campos, el primero «SERIES;Mean cost for none»
fila de datos:   10 campos, el primero «0»
```

**El código.** gui/src/main/java/org/openmarkov/gui/desnets/DESResultsWindow.java:97-116

```
private static void saveCsv(String simulationResults) {
    Scanner scanner = new Scanner(simulationResults);
    StringBuilder sb = new StringBuilder();
    sb.append("SERIES;");
    // TODO: skip discounted values when there is no discount.
    interventions.forEach(intervention -> {
        criteria.forEach(criterion -> {
            sb.append("Mean ")
                    .append(criterion)
                    .append(" for ")
                    .append(intervention)
                    .append(",");
            ...
        });
    });
    sb.append("\n");
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*. Cambiar `"SERIES;"` por `"SERIES,"` en la línea 91. Las dos filas terminan en una coma final que crea un campo vacío; conviene quitarla en las dos a la vez para que sigan teniendo el mismo número de campos.

Hay otra fragilidad en la misma función: las filas de datos se obtienen borrando con `replaceAll("[a-zA-Z].*?;", "")` todo lo que empiece por una letra hasta el siguiente punto y coma; un nombre de opción o de criterio que empiece por un número o un símbolo (por ejemplo «10 mg») dejaría restos en la fila. Si se reescribe la exportación a partir de los objetos de resultados (ver error 102), desaparecen las dos cosas.

Prueba de regresión: extraer la construcción del texto del CSV a un método sin selector de ficheros y comprobar, con un texto de dos opciones y dos criterios, que la fila de títulos y las de datos tienen el mismo número de campos y que el primer título es «SERIES».

## Error 104 · Simular con la traza textual activada una red de sucesos que aún no se ha guardado falla antes de empezar

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.desnets |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/desnets/DESLogTextWriter.java:53-61`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESLogTextWriter.java:221-238`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:435`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:89-111`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/common/MonteCarloOptionsPanel.java:313-319`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:289-312` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |

| | |
|---|---|
| **Decisión de equipo** | No |
| **Relacionados** | nota 111 |

**Qué pasa.** La simulación DES (simulación de eventos discretos) puede escribir una traza del algoritmo en un fichero de texto: casilla «Generate evaluation algorithm trace» del panel de opciones de Monte Carlo, que aparece al lanzar la simulación. Con esa casilla marcada, una red de sucesos que aún no se ha guardado no se simula: sale un error inesperado.

Camino de llegada: Archivo, Nuevo, tipo DESNet; construir el modelo; pulsar el botón o menú de coste-efectividad (en una `DESNet` lanza `monteCarloSimulation`); marcar la traza textual y aceptar. La excepción sale del constructor de `DESInference` dentro del hilo que crea `monteCarlo Simulation`, que solo captura `IOException` y `OpenMarkovException`, así que (por lectura) llega al manejador global y se muestra como error inesperado; no se simula nada.

`DESInference.initialize` construye siempre un `DESLogTextWriter`; si la traza está activada, su constructor llama a `createFilename(probNet.getName())`, que hace `probNetName.substring(0, probNetName.indexOf('.'))`. Si el nombre no tiene punto, `indexOf` devuelve -1 y `substring(0, -1)` lanza `StringIndexOutOfBoundsException`.

Una red abierta desde fichero se llama como el fichero (`asia.pgmx`), pero una red creada con Archivo, Nuevo recibe el nombre del texto `InternalFrame.Title`, que es «Untitled» (`Network FileHandler.createNewNetwork`), y lo conserva hasta que se guarda con otro nombre.

Medido con una red de sucesos de prueba (no está en el repositorio): con el nombre cambiado a «Untitled» y la traza activada, `initialize` termina con `StringIndexOutOfBoundsException: Range [0, -1) out of bounds for length 8`. Con un nombre con extensión (`nombre.pgmx`), escribe la traza en `DESNetFiles/TextualLog/nombre-AAAA-MM-DD-HH-mm.log`. Para reproducirlo basta cualquier red de sucesos nueva, sin guardar, siguiendo el camino de arriba.

Detalles menores del mismo método:

- Un nombre con varios puntos (`modelo.v2.pgmx`) se corta en el primero (`modelo`).
- El nombre lleva la fecha con resolución de minutos, así que dos simulaciones de la misma red en el mismo minuto escriben en el mismo fichero y la segunda sobrescribe la primera.
- Como el fichero se abre en `initialize` antes de construir los registros, una simulación que falla después (por ejemplo, la de error 97) deja un fichero de traza vacío.

**El código.** gui/src/main/java/org/openmarkov/gui/desnets/DESLogTextWriter.java:221-227

```
    private File createFilename(String probNetName) {

        String filename = probNetName.substring(0, probNetName.indexOf('.'));
        filename += "-";
        filename += new SimpleDateFormat("yyyy-MM-dd-HH-mm").format(new Date());
        filename += ".log";
```

**Cómo arreglarlo.** Quitar la extensión solo si la hay (último punto, no el primero) y usar el nombre entero si no la hay.

Sitios con la misma suposición sobre el nombre de la red: `DESLogTextWriter.createFilename` es el único que corta en `indexOf('.')` entre las clases de la simulación de sucesos (paquete `org.openmarkov.gui.desnets`); `DESResultsExcelWriter.createFilename` (líneas 181-183) repite exactamente el mismo corte, aunque hoy esa clase no tiene ninguna referencia; si se vuelve a usar, necesita el mismo arreglo.

Prueba de regresión: construir `DESLogTextWriter` con la traza activada para redes llamadas «Untitled», «a.pgmx» y «a.b.pgmx» no debe lanzar y debe producir nombres que empiecen por

«Untitled», «a» y «a.b».

### Error 105 · [Corregido] Un fallo al muestrear un «Tree with Excluded Events» abre una ventana de aviso desde el núcleo, y sin pantalla la tapa con otra excepción

| | |
|---|---|
| **Estado** | Corregido en el commit `294b6fc` (15-10-2026): `sampleConditionedVariable` quita de la configuración los sucesos que haya, si hay alguno, y pregunta al árbol sin sucesos; el `try/catch` con la ventana y su `import` se han quitado. Comprobado con una simulación de verdad: un nodo X con esta relación sobre el suceso E1 y el nodo Y se vuelve a sortear cuando otro suceso, E2, cambia Y, sin ningún suceso padre en la configuración; sin el arreglo la simulación se detenía ahí (sin pantalla, con `HeadlessException`), y con él termina y X sigue a Y. Con prueba en `gui` (paquete `desnets`) que simula esa red diez años. La relación hermana, `TreeWithEventsPotential`, ya no abría ninguna ventana desde el commit `e3a4b1e`. Lo que propone la nota 170 para que `core` deje de depender del escritorio no se ha tocado. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.treeadd |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:28,125-143`<br>`core/src/main/java/module-info.java:20`<br>`core/src/main/java/org/openmarkov/core/action/core/PurposeEdit.java:18`<br>`core/src/main/java/org/openmarkov/core/action/core/MonteCarloOptionsEdit.java:8,32`<br>`gui/src/main/java/org/openmarkov/gui/desnets/ChanceEvaluation.java:85,123-161`<br>`inference.des/src/main/java/org/openmarkov/inference/DES/DESRecord.java:228`<br>`gui/src/main/java/org/openmarkov/gui/desnets/EventRecord.java:135` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 21, error 22, error 23, error 106, nota 68, nota 74, nota 170 |

**Qué pasa.** `TreeWithExcludedEventsPotential` es la relación «Tree with Excluded Events» de la simulación de eventos discretos (DES): un árbol sobre los padres del nodo del que se excluye el suceso. Si algo falla al sortear el valor del nodo, abre una ventana de aviso desde `core`, el módulo del modelo, que no debería mostrar ventanas:

- Con pantalla (la aplicación), la simulación se detiene en una ventana modal con el texto fijo «TreeWithExcludeEventsPotential:Getting sample exception: », con el nombre de la clase en lugar de una explicación, y queda parada hasta que el usuario pulsa Aceptar. Después llega la excepción original por el camino normal, así que el error se comunica dos veces y la primera sin sentido para el usuario.
- Sin pantalla (una prueba, el módulo `resttemplate`, un servidor, cualquier ejecución `headless`), `JOptionPane` lanza `HeadlessException`, que **sustituye** a la excepción original y oculta la causa.

Su método `sampleConditionedVariable` (líneas 125-143), que usa la simulación para sortear el valor del nodo (`DESRecord.java:228`, `EventRecord.java:134`), busca entre los padres de la configuración las variables de tipo suceso, quita el hallazgo del primero y pregunta a su árbol sin sucesos. Si algo falla, su `catch` (líneas 138-142, con un `//FIXME when completed this method coding`

`this catch will be removed`) abre un `JOptionPane.showMessageDialog` y después relanza la excepción.

El fallo que dispara la ventana no es artificial. El propio método tiene un comentario (líneas 131-133) que dice que esta relación se usa también con nodos de azar y que algunas configuraciones no llevan sucesos, y `ChanceEvaluation` calcula los nodos de azar sin sucesos antecesores con una configuración sin suceso (`computeChanceValue(chanceRecord, null)`, línea 85). En ese caso `eventVariables.get(0)` (línea 135) sobre una lista vacía lanza `IndexOutOfBoundsException`, que es la que abre la ventana. Ese acceso a una lista vacía es un defecto aparte, que conviene arreglar a la vez.

Medido con un pequeño programa de prueba, sin pantalla, sobre una red de sucesos construida en código con un nodo Y cuyos padres son un suceso E y un nodo Age: muestrear Y con una configuración sin sucesos termina en `java.awt.HeadlessException`, y la causa real no aparece. Se llamó directamente al sorteo; no se ha ejecutado una simulación completa que llegue a ese `catch`, así que lo que ve el usuario en la aplicación se deduce de la lectura.

Además de esta llamada, en `core` quedan otros dos restos de escritorio: `MonteCarloOptionsEdit`, que declara `javax.swing.undo.CannotUndoException` en su `undo()` (líneas 8 y 32), y un `import javax.swing.JOptionPane` sin uso en `PurposeEdit` (línea 18). Por eso `core/src/main/java/module-info.java` sigue con `requires java.desktop` (línea 20).

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:125-143

```
@Override
public double sampleConditionedVariable(double[] randomNumbers, EvidenceCase parents)
↪  throws OpenMarkovException {
    List<Variable> eventVariables = parents.getVariables()
                                           .stream()
                                           .filter(variable -> variable.getVariableType()
                                           ↪  == VariableType.EVENT)
                                           .toList();
    ...
    try {
        parents.removeFinding(eventVariables.get(0));
        return noEventTree.sampleConditionedVariable(randomNumbers, parents);
    } catch (OpenMarkovException | RuntimeException e) {
        //FIXME when completed this method coding this catch will be removed
        JOptionPane.showMessageDialog(null, "TreeWithExcludeEventsPotential:Getting sample
        ↪  exception: " + this.getConditionedVariable());
        throw e;
    }
}
```

**Cómo arreglarlo.** Quitar el `try/catch` con el diálogo y dejar salir la excepción, o envolverla en una excepción del dominio con un mensaje que diga qué nodo, qué configuración y qué falta; quien lanza la simulación en la interfaz decide cómo enseñarla. Es lo que ya hace `TreeWithEventsPotential.sampleConditionedVariable` (líneas 188-190), que comprueba antes cuántos sucesos hay en la configuración. Quitar también el import de la línea 28.

Quitar el hallazgo del suceso solo si hay alguno (`eventVariables.isEmpty()`), coherente con el comentario que admite configuraciones sin sucesos.

Ese mismo método modifica la configuración que recibe (`removeFinding`), cosa que describe nota 74; allí no hay nada que corregir, pero el cambio cae en el mismo sitio.

Para que `core` deje de depender del escritorio: sustituir `CannotUndoException` en `MonteCarlo`

`OptionsEdit` por una excepción propia o ninguna, borrar el import sin uso de `PurposeEdit` y retirar `requires java.desktop` de `core/src/main/java/module-info.java`, de modo que cualquier diálogo nuevo en el núcleo sea un error de compilación. La propuesta de conjunto está en nota 170, que es una nota: allí no hay ningún error que corregir.

Prueba: construir un `TreeWithExcludedEventsPotential`, pedirle un sorteo con una configuración sin suceso bajo `-Djava.awt.headless=true` y comprobar que no sale `HeadlessException`: debe salir la excepción del dominio o, si se decide admitir configuraciones sin sucesos, la muestra del árbol.

### Error 106 · Un nodo de azar discretizado en una red de sucesos impide simular con un error inesperado, y el mensaje previsto diría que ese nodo «debe ser de azar»

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui, inference.des |
| **Paquete** | org.openmarkov.gui.desnets; org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/desnets/ChanceRecord.java:77-85`<br>`gui/src/main/java/org/openmarkov/gui/desnets/GenericEvaluation.java:64-81`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:107-115,439`<br>`inference.des/src/main/java/org/openmarkov/inference/DES/exception/NodeMustBeChance.java:6-13`<br>`inference.des/src/main/resources/inference_des/localize/inference_des_class_localizations_en.xml:5-6`<br>`core/src/main/java/org/openmarkov/core/exception/OpenMarkovException.java:27-35`<br>`core/src/main/java/org/openmarkov/core/model/network/VariableType.java:58-65`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:856-868,886-894`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:289-312` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si una red de sucesos debe admitir nodos de azar discretizados (y entonces la simulación los trata como de estados) o si el editor no debe ofrecer ese tipo en una red de sucesos |
| **Relacionados** | error 97, error 99, error 105, nota 55, error 202 |

**Qué pasa.** En una red DES (simulación de eventos discretos), el usuario abre las propiedades de un nodo de azar y, en la pestaña del dominio, cambia el tipo de variable a «Discretized» (el selector ofrece a todo nodo de azar los tipos de `VariableType.of(CHANCE)`: estados finitos, discretizada y numérica, en cualquier tipo de red). La edición se acepta. Al lanzar la simulación (botón o menú de coste-efectividad), no se simula nada y sale el diálogo de error inesperado.

La simulación no admite nodos de azar discretizados, y lo comprueba a propósito: el constructor de `ChanceRecord` (líneas 76-84) lanza `NodeMustBeChance` si la variable del nodo no es numérica ni de estados finitos. Pero ese aviso previsto no llega al usuario, por tres motivos que se encadenan:

- `GenericEvaluation` (líneas 63-80) construye los registros por reflexión y convierte cualquier `InvocationTargetException` en `UnreachableException`, así que la excepción de la red del usuario se presenta como fallo del programa. En la aplicación, el hilo que crea `monteCarlo`

`Simulation` no la captura y llega al manejador global, que abre el diálogo de error inesperado (por lectura).

- Si llegara al `catch (OpenMarkovException e)` del constructor de `DESInference` (líneas 105-113), la ventana «Cannot simulate» enseñaría `e.getMessage()`, y en las excepciones de OpenMarkov ese mensaje es nulo: el texto está en `toString()` (`OpenMarkovException`, líneas 27-35).
- El texto de `NodeMustBeChance` es «{node} must be chance.», que para un nodo que sí es de azar no explica nada.

Medido con una red de sucesos de prueba que simula sin problemas, cambiando el nodo de azar `Sex` a discretizado con `VariableTypeEdit` (la edición que ejecuta el selector del diálogo). Las restricciones de la red no lo impiden, y el nodo conserva su tabla. Al simular:

```
DESInference.initialize -> UnreachableException:
↪  java.lang.reflect.InvocationTargetException
  causa: chance node of Discretized variable Sex with states [state male, state female]
  ↪  must be chance.
    at ChanceRecord.<init>(ChanceRecord.java:81)
    at GenericEvaluation.<init>(GenericEvaluation.java:72)
    at ChanceEvaluation.<init>(ChanceEvaluation.java:40)
    at DESInference.initialize(DESInference.java:437)
NodeMustBeChance.getMessage() = null
```

**El código.** gui/src/main/java/org/openmarkov/gui/desnets/ChanceRecord.java:77-82

```
    public ChanceRecord(Node chanceNode, double variableValue) throws OpenMarkovException {
        super(chanceNode);
        if ((chanceNode.getNodeType() != NodeType.CHANCE)
                || ((chanceNode.getVariable().getVariableType() != VariableType.NUMERIC)
                && (chanceNode.getVariable().getVariableType() !=
                ↪  VariableType.FINITE_STATES)))
            throw new NodeMustBeChance(chanceNode);
```

gui/src/main/java/org/openmarkov/gui/desnets/GenericEvaluation.java:69-79

```
        try {
            Constructor<T> recordConstructor =
            ↪  recordClass.getDeclaredConstructor(Node.class);
            for (Node node : nodeList) {
                lastNode = node;
                desRecordHashMap.put(node, recordConstructor.newInstance(node));
            }

        } catch (NoSuchMethodException | InvocationTargetException | InstantiationException
        ↪  |
                 IllegalAccessException e) {
            throw new UnreachableException(e);
        }
```

gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:109-115

```
        try {
            if (!initialize(probNet))
                return;
        } catch (OpenMarkovException e) {
            JOptionPane.showMessageDialog(null, e.getMessage(), "Cannot simulate",
            ↪  JOptionPane.ERROR_MESSAGE);
            return;
        }
```

**Cómo arreglarlo.** Lo que no necesita decisión:

- En `GenericEvaluation`, si la causa de la `InvocationTargetException` es una `OpenMarkovException`, relanzar esa causa en lugar de envolverla en `UnreachableException` (el constructor ya declara `throws OpenMarkovException`). Solo los errores de reflexión propiamente dichos son inalcanzables.
- En `DESInference`, enseñar la excepción con su texto localizado (`toString()`, o `ExceptionDialog`) en vez de `getMessage()`.
- Dar a este caso un mensaje que diga lo que pasa: por ejemplo, que la variable del nodo de azar es discretizada y la simulación solo admite nodos de azar numéricos o de estados finitos. `NodeMustBeChance` se usa también para el caso de tipo de nodo, que hoy no se alcanza porque `GenericEvaluation` elige los nodos por su tipo.

Lo que necesita decisión: si la red de sucesos admite nodos de azar discretizados. Si no los admite, el selector de tipo de variable no debería ofrecer «Discretized» en una red de sucesos (`NodeDomainValuesTablePanel`, líneas 855-867, que hoy usa `VariableType.of` sin mirar el tipo de red). La regla tendría que cumplirse también en `VariableTypeEdit.checkConstraintsWillBeMet` y al abrir un fichero. Si los admite, `ChanceRecord` y el muestreo deben tratarlos como variables de estados.

Los mismos constructores por reflexión lanzan `NodeMustBeEvent`, `NodeMustBeUtility` y `EventIsNotParentOf`, y un fallo del potencial (como el de error 99) o un desbordamiento de pila (como el de error 97) acaba en el mismo `UnreachableException`. El cambio en `GenericEvaluation` los afecta a todos.

Prueba de regresión: con una red de sucesos cuyo nodo de azar es discretizado, `new DESInference(red, monitor)` (o `initialize`) no lanza `UnreachableException`: o bien rechaza con una `OpenMarkovException` cuyo texto nombra el nodo y el tipo de variable, o bien simula, según lo decidido.

### Error 107 · [Corregido] La hoja exportada de la propagación estocástica escribe los bloques de los hallazgos en un orden distinto en cada ejecución

| | |
|---|---|
| **Estado** | Corregido en el commit `c7b7f8f` (14-10-2026). `EvidenceCase` guarda los hallazgos en el orden en que se añaden (`LinkedHashMap` en los constructores que crean el mapa, también en el de copia); el constructor que recibe un mapa de fuera lo sigue usando tal cual. `changeFinding` sustituye el hallazgo en su sitio en vez de quitarlo y añadirlo al final, así que `fuse` con sobrescritura (lo que hace `StochasticPropagation` al juntar las evidencias) conserva el orden. No se toca `XlsxWrite`. De paso, el peso de cada muestra del muestreo ponderado por verosimilitud, que se multiplica en el orden de los hallazgos, ya no puede cambiar en la última cifra entre dos ejecuciones con la misma semilla. La prueba repite cada caso 50 veces con 20 variables nuevas; con el código anterior fallaban sus cinco casos. |
| **Módulo** | stochasticPropagationOutput |
| **Paquete** | org.openmarkov.stochasticPropagationOutput |
| **Ficheros** | `stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/XlsxWrite.java:123-125`<br>`stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/XlsxWrite.java:362-364`<br>`core/src/main/java/org/openmarkov/core/model/network/EvidenceCase.java:37`<br>`core/src/main/java/org/openmarkov/core/model/network/EvidenceCase.java:196-198` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |

| | |
|---|---|
| **Decisión de equipo** | No |
| **Relacionados** | error 93, error 108 |

**Qué pasa.** Exportar dos veces la misma red con la misma semilla y los mismos hallazgos produce ficheros con los bloques de hallazgos en orden distinto, lo que impide compararlos. Con un solo hallazgo no se nota.

Camino del usuario: con una red abierta y hallazgos introducidos, menú Tools → «Export stochastic propagation data» (`StochasticPropagationOutputPlugin`), que abre `StochasticPropagationOutputFrame`; al exportar, un hilo con `XlsxWrite` escribe un fichero `.xlsx` con las muestras y un resumen por variable.

En el resumen, para cada variable con hallazgo que no se ha muestreado (en el muestreo ponderado por verosimilitud, las variables observadas no se muestrean), `XlsxWrite` escribe un bloque (nombre, estados, en cuál se observó). Recorre esas variables en el orden de `algorithm.getFusedEvidence().getVariables()`. Las variables muestreadas (el grueso de la hoja) sí van en un orden estable (`getVariablesToSample()`).

`EvidenceCase` guarda los hallazgos en un `HashMap<Variable, Finding>` y `getVariables()` devuelve `new ArrayList<>(findings.keySet())`: el orden es el de la tabla de dispersión. Como `Variable` no redefine `hashCode` ni `equals`, se usa el código de identidad del objeto, que la máquina virtual asigna de forma distinta en cada ejecución.

La causa está en `EvidenceCase`, no en el exportador: `getVariables()` tiene muchos llamadores en el proyecto, y cualquiera que dependa del orden hereda el mismo comportamiento.

Medido con un pequeño programa de prueba que lee `inference/src/test/resources/asia.pgmx`, crea un `EvidenceCase` con hallazgos en «Positive X-ray?», «Has bronchitis» y «Smoker?» (en ese orden) y escribe `getVariables()`, seis ejecuciones:

```
[Smoker?, Positive X-ray?, Has bronchitis]
[Smoker?, Has bronchitis, Positive X-ray?]
[Has bronchitis, Smoker?, Positive X-ray?]
[Has bronchitis, Smoker?, Positive X-ray?]
[Has bronchitis, Smoker?, Positive X-ray?]
[Positive X-ray?, Smoker?, Has bronchitis]
```

**El código.** core/src/main/java/org/openmarkov/core/model/network/EvidenceCase.java:37, 196-198

```java
protected final HashMap<Variable, Finding> findings;
...
public List<Variable> getVariables() {
    return new ArrayList<>(findings.keySet());
}
```

stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/XlsxWrite.java:123-125, 362-364

```java
// Evidence variables
EvidenceCase evidence = algorithm.getFusedEvidence();
List<Variable> evidenceVariables = evidence.getVariables();
...
// Write about evidence variables that have not been sampled
for (Variable evidenceVariable : evidenceVariables) {
    if (!sampledVariables.contains(evidenceVariable)) {
```

**Cómo arreglarlo.** Cambiar el mapa de `EvidenceCase` a `LinkedHashMap` (orden de inserción), en la declaración del campo y en los constructores que lo crean (líneas 61 y 68; el constructor de la línea 46 recibe un `HashMap` de fuera y habría que copiarlo a un `LinkedHashMap` o cambiar su tipo).

Hay que revisar que los constructores de copia de `EvidenceCase` y `StochasticPropagation` (línea 114, `new EvidenceCase(postResolutionEvidence)`) conserven el orden.

Es el mismo patrón (mapas sin orden sobre objetos sin `hashCode`) que causa error 4 en las operaciones de potenciales, pero en otro sitio: arreglar aquél no arregla éste.

Alternativa local, peor: ordenar en `XlsxWrite` por el orden de las variables de la red.

Prueba: crear un `EvidenceCase` con hallazgos en tres variables en un orden dado y comprobar que `getVariables()` los devuelve en ese orden. Como el defecto depende de la dispersión por identidad, la prueba debe repetirse con varias instancias o en varios procesos para que tenga valor.

### Error 108 · «Export stochastic propagation data» sobre una red bayesiana con un nodo numérico de azar sale como error no previsto, o con un mensaje técnico, en vez de decir que la herramienta no admite nodos numéricos

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | stochasticPropagationOutput, inference |
| **Paquete** | org.openmarkov.stochasticPropagationOutput; org.openmarkov.inference.algorithm.likelihoodWeighting; org.openmarkov.inference.algorithm.huginPropagation |
| **Ficheros** | `stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/StochasticPropagationOutputFrame.java:175-189,194-244`<br>`stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/StochasticPropagationOutputPlugin.java:36-49`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/likelihoodWeighting/StochasticPropagation.java:77-86`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/huginPropagation/HuginPropagation.java:56-65`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:672-675`<br>`core/src/main/java/org/openmarkov/core/inference/InferenceAlgorithm.java:94-97,154-171` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 42, error 48, error 107, nota 109, nota 168 |

**Qué pasa.** Una red bayesiana puede tener nodos numéricos de azar: por omisión, las propiedades de la red permiten variables discretas y continuas, y en la pestaña de dominio del diálogo de un nodo se puede cambiar su tipo a numérico. Ninguna red bayesiana del repositorio los tiene; se llega construyéndola en el editor.

Con una red así abierta, el menú Tools → «Export stochastic propagation data» abre la ventana «Stochastic propagation output». La ventana solo comprueba que haya una red, que sea bayesiana y que tenga nodos (`boundError`, líneas 182-196). Al pulsar «Export to a .xlsx file», `onActionRequest` construye el algoritmo de muestreo elegido y el algoritmo exacto de Hugin (propagación por agrupamiento), compila Hugin, muestrea y después calcula las probabilidades exactas. Ninguno de los dos algoritmos declara que no admite variables numéricas: solo exigen que la red sea

bayesiana (`StochasticPropagation.getAdditionalConstraints` devuelve una lista vacía y `HuginPropagation.getAdditionalConstraints`, `null`). El fallo llega después, y según la relación del nodo:

- Si la relación no se puede convertir en tabla, falla la compilación de Hugin con `NonProjectablePotentialException`. `actionPerformed` la envuelve y sale un error normal: «Potential P(N | P) cannot be converted to a table». No dice qué hacer ni que el problema es el tipo del nodo.
- Si se puede convertir en tabla, Hugin compila y falla el muestreo: `Potential.sampleConditionedVariable` lanza `UnsupportedOperationException` para toda clase de potencial que no lo redefine. Es una excepción de tiempo de ejecución que `actionPerformed` no recoge, y sale la ventana de error no previsto que pide enviar el fallo a los desarrolladores.

Medido con un pequeño programa de prueba que construye la red P → N con las ediciones del editor (añadir dos nodos de azar, enlazarlos, `VariableTypeEdit` a numérico), pone en N cada relación que el diálogo del nodo ofrece, instanciada como la instancia el diálogo, y repite las llamadas de `onActionRequest` en su orden:

- sin tocar la relación (el editor deja un `UniformPotential`), y con «Binomial», «DistributionTable», «Function», «Uniform» y «UnivariateDistr»: error normal, «Potential P(N | P) cannot be converted to a table»;
- con «Delta», «Exact» (también con valores 3 y 7), «Exponential», «Linear combination», «Product», «Sum» y «Tree/ADD»: error no previsto, `UnsupportedOperationException: sampleConditionedVariable not implemented for …`.

Da lo mismo con «Likelihood weighting», el algoritmo marcado por omisión, que con «Logic sampling», y con la red P → N → C, en la que el nodo numérico es además padre de otro. Con la relación «Exact», la misma red tampoco se puede propagar en el modo inferencia de la ventana principal, por otra causa (error 42).

**El código.** stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/StochasticPropagationOutputFrame.java:182-196

```
public Exception boundError() {
    // Stop if there is no net opened yet
    if (probNet == null) {
        return new NoNetOpenedException();
    }
    // Stop if other than bayesian net
    if (!(probNet.getNetworkType() instanceof BayesianNetworkType)) {
        return new InvalidNetworkTypeException.NotAllowedType(probNet,
        ↪  Arrays.asList(BayesianNetworkType.getUniqueInstance()));
    }
    // Stop if no nodes in the net
    if (probNet.getNumNodes() == 0) {
        return new NetworkHasNoNodesException(probNet);
    }
    return null;
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:672-675

```
public int sampleConditionedVariable(Random randomGenerator, Map<Variable, Integer>
↪  sampledParents) {
    throw new UnsupportedOperationException(
            "sampleConditionedVariable not implemented for " + getClass().getSimpleName());
```

```
}
```

**Cómo arreglarlo.** Rechazar la red antes de calcular, con un mensaje que diga que la herramienta solo admite variables discretas (de estados finitos o discretizadas). Hay dos sitios, y conviene usar los dos:

- En los algoritmos, para que cualquier llamador reciba la negativa: que `StochasticPropagation.getAdditionalConstraints` y `HuginPropagation.getAdditionalConstraints` exijan que todas las variables de azar sean discretas. Existe ya la restricción `OnlyDiscreteVariables`, con su texto en inglés; `InferenceAlgorithm.checkConstraintsApplicability` la comprobaría al construir el algoritmo, y la ventana ya convierte `ConstraintViolatedException` en un error normal. El otro llamador de producción de Hugin, la maximización de la esperanza del aprendizaje (`EMAlgorithm`), trabaja con redes aprendidas de bases de casos, de estados finitos, y no le afectaría.
- En la ventana, en `boundError`, para avisar al abrir la herramienta en vez de al pulsar el botón, igual que ya se hace con el tipo de red.

Que el muestreo admita nodos numéricos sería otra funcionalidad, no el arreglo de este fallo.

Prueba: construir `LikelihoodWeighting`, `LogicSampling` y `HuginPropagation` sobre una red bayesiana con un nodo numérico de azar con relación «Exact» debe lanzar la excepción declarada con el nombre del nodo, nunca `UnsupportedOperationException`; y con `0_miscellaneous/networks/bn/BN-asia.pgmx` deben seguir construyéndose.

### Error 109 · Las cuatro casillas de estadísticos del panel de Monte Carlo no reflejan el modelo ni cambian nada

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.inference.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/inference/common/MonteCarloOptionsPanel.java:189-201`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/common/MonteCarloOptionsPanel.java:343-409`<br>`core/src/main/java/org/openmarkov/core/inference/MonteCarloOptions.java:58-72`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:1069-1071` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si los estadísticos de la simulación deben poder elegirse (y entonces conectarlos al cálculo y al fichero) o si las cuatro casillas sobran |
| **Relacionados** | error 164, nota 111, nota 131 |

**Qué pasa.** Con una red de sucesos discretos (DES —simulación de eventos discretos—) abierta, el diálogo de opciones de inferencia (`InferenceOptionsDialog`) incluye `MonteCarloOptionsPanel` y, al aceptar, copia sus opciones a la red (`setMonteCarloOptions(monteCarloOptionsPanel.getMonteCarloOptions())`).

El recuadro «DES Inference Statistics» tiene cuatro casillas: Mean, Sum, Trimmed Mean y Median. Lo que el usuario ve es una casilla activa («Mean») que puede desmarcar sin efecto alguno y tres que nunca puede tocar; ninguna refleja lo que dice el modelo.

Las cuatro se construyen con el valor de `monteCarloOptions.isMean()` (copiado y pegado; debería

ser `isSum`, `isTrimmedMean`, `isMedian`) y a continuación se les fuerza un estado fijo: Mean marcada y activa; las otras tres desmarcadas y desactivadas.

`getMonteCarloOptions()` escribe después los cuatro estados en el objeto de opciones, de modo que aceptar deja siempre mean=true, trimmedMean=false, median=false y sum=false (el valor por omisión de `trimmedMean` en `MonteCarloOptions` es `true`).

Esos cuatro indicadores no los lee nadie. Buscando `isMean()`, `isSum()`, `isTrimmedMean()` e `isMedian()` en todo el código de producción solo aparecen en la propia clase `MonteCarloOptions` (constructor de copia) y en este panel; los escritores y lectores de ProbModelXML (`PGMXWriter_1_0`, `PGMXReader_1_0`) solo guardan el número de simulaciones, el de series y el fichero de entrada. Por tanto pisar los valores no cambia ningún resultado: la consecuencia es un control que no hace nada, no un resultado cambiado.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/inference/common/MonteCarloOptionsPanel.java:359-381

```
private JCheckBox getJCheckBoxMean() {
    if (meanCheckBox == null) {
        //TODO use stringDatabase
        meanCheckBox = new JCheckBox("Mean", monteCarloOptions.isMean());
        meanCheckBox.setEnabled(true);
        meanCheckBox.setSelected(true);
    }
    return meanCheckBox;
}
...
private JCheckBox getJCheckBoxSum() {
    if (sumCheckBox == null) {
        //TODO use stringDatabase
        sumCheckBox = new JCheckBox("Sum", monteCarloOptions.isMean());
        sumCheckBox.setEnabled(false);
        sumCheckBox.setSelected(false);
    }
    return sumCheckBox;
}
```

gui/src/main/java/org/openmarkov/gui/dialog/inference/common/MonteCarloOptionsPanel.java:195-198

```
this.monteCarloOptions.setMean(meanCheckBox.isSelected());
this.monteCarloOptions.setTrimmedMean(trimmedMeanCheckBox.isSelected());
this.monteCarloOptions.setMedian(medianCheckBox.isSelected());
this.monteCarloOptions.setSum(sumCheckBox.isSelected());
```

**Cómo arreglarlo.** Depende de la decisión de equipo:

- Si los estadísticos deben poder elegirse: inicializar cada casilla con su propio indicador (`isMean`, `isSum`, `isTrimmedMean`, `isMedian`) sin forzar el estado, activarlas, hacer que la simulación (`DESInference` y la ventana de resultados, en el módulo `gui`) use los indicadores, y guardarlos y leerlos en `PGMXWriter_1_0`/`PGMXReader_1_0`.
- Si no: quitar el recuadro «DES Inference Statistics» y las cuatro escrituras de `getMonteCarloOptions()`, y los cuatro campos de `MonteCarloOptions` si nadie más los necesita (hay una prueba, `OptionsAreCopiedWholeTest`, que los copia).

Prueba para la primera opción: con `MonteCarloOptions` con sum=true y mean=false, construir el panel y comprobar que las casillas lo reflejan y que `getMonteCarloOptions()` devuelve lo mismo sin tocar nada.

### Error 110 · «Sort probnet» falla con una excepción en un modelo de sucesos discretos que tiene un enlace de un nodo a sí mismo

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | full |
| **Paquete** | org.openmarkov.full |
| **Ficheros** | `full/src/main/java/org/openmarkov/full/SortProbnetPlugin.java:52-63`<br>`full/src/main/java/org/openmarkov/full/SortProbnetPlugin.java:95-120`<br>`core/src/main/java/org/openmarkov/core/model/network/type/DESNetworkType.java:34-39` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** `SortProbnetPlugin` (módulo `full`) añade al menú Herramientas la entrada «Sort probnet», que recoloca los nodos de la red abierta. En cualquier red con un enlace de un nodo a sí mismo, «Sort probnet» sale por una excepción no controlada desde la acción del menú y no mueve ningún nodo (la edición `MoveNodeEdit` va después).

En las redes de simulación de eventos discretos (DES, tipo `DESNetworkType`) un enlace de un nodo a sí mismo está permitido: el tipo desactiva `NoSelfLoop` y activa `OnlySelfLoopsWithEventAndChanceNodes` (líneas 34-39), es decir, se admite en nodos de suceso y de azar. Se usa, por ejemplo, para un suceso que vuelve a programarse a sí mismo (una revisión que programa la siguiente), y el editor sabe dibujarlo (`VisualLink`, `VisualArrow.isSelfLoop`).

En las redes de ejemplo del proyecto no hay ahora ninguna red DES con un enlace de un nodo a sí mismo, pero el editor permite crearlas. Para reproducirlo en la aplicación hay que construir una red de sucesos con un enlace de un suceso a sí mismo y pedir «Sort probnet».

El complemento usa el algoritmo de disposición por fuerzas de Fruchterman-Reingold de la biblioteca JGraphT. En `getSortedCoordinates` (líneas 95-120) mete los nodos en un `SimpleDirectedGraph` y, por cada enlace dibujado, llama a `graph.addEdge(origen, destino)`. `SimpleDirectedGraph` es un grafo «simple»: no admite un enlace de un vértice a sí mismo y lanza `IllegalArgumentException: loops not allowed`.

Medido por partes con un pequeño programa de prueba: en una red DES construida en código con dos nodos de suceso, `new AddLinkEdit(net, checkup, checkup, true).executeEdit()` se acepta; y `new SimpleDirectedGraph<>(DefaultEdge.class).addEdge(v, v)` lanza `java.lang.IllegalArgumentException: loops not allowed` con el JGraphT del jar. No se ha ejecutado el complemento completo (necesita el panel del editor con la ventana principal), pero no hay nada entre el recorrido de enlaces y `addEdge` que filtre esos enlaces.

**El código.** full/src/main/java/org/openmarkov/full/SortProbnetPlugin.java:97-113

```
Graph<VisualNode, DefaultEdge> graph = new SimpleDirectedGraph<>(DefaultEdge.class);
...
visualNetwork.getAllNodes().forEach(graph::addVertex);
for (var link : visualNetwork.getVisualLinks()) {
    graph.addEdge(link.getSourceNode(), link.getDestinationNode());
    if (!link.isDirected()) {
        graph.addEdge(link.getDestinationNode(), link.getSourceNode());
    }
}
layoutAlgorithm.layout(graph, model);
```

**Cómo arreglarlo.** Saltarse en el bucle los enlaces cuyo origen y destino son el mismo nodo (no aportan nada a una disposición por fuerzas: la fuerza de un nodo sobre sí mismo no está definida). Alternativa: usar un tipo de grafo de JGraphT que admita lazos (`DirectedPseudograph`), comprobando antes que `FRLayoutAlgorithm2D` los tolera.

El mismo método lo usan `graphicallySortNetwork` y `graphicallySortNetworkAsMoveEdit`, así que el arreglo en `getSortedCoordinates` cubre las dos.

Prueba: construir el grafo con la lista de enlaces de una red DES con un enlace de un nodo a sí mismo (conviene extraer la construcción del grafo a un método que reciba nodos y enlaces, para probarlo sin ventana) y comprobar que la disposición se calcula y da una posición a cada nodo.

### Error 214 · Con una red de sucesos abierta, «Cost effectiveness analysis» del menú Tools pide sus opciones y acaba en «This algorithm cannot evaluate DESNet», mientras que el botón de la barra lanza la simulación

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | costEffectiveness |
| **Paquete** | org.openmarkov.costEffectiveness; org.openmarkov.gui.window |
| **Ficheros** | `costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CostEffectivenessPlugin.java:53-61,64-110` `gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:260-266` |
| **Comprobado** | Leyendo el código y ejecutándolo (los dos algoritmos, sin ventana) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 92, error 102, error 201 |

**Qué pasa.** Con una red de simulación de sucesos discretos abierta, hay dos maneras de pedir el coste-efectividad y hacen cosas distintas:

- el botón de la barra lanza la simulación de Monte Carlo, que es lo que una red de sucesos admite (`InferenceHandler.showCostEffectivenessDeterministic`, líneas 263-266);
- la entrada Tools → «Cost effectiveness analysis» no mira el tipo de red: abre las opciones de inferencia, luego el diálogo del ámbito y, al aceptar, intenta evaluar la red con los algoritmos exactos, que la rechazan.

Medido con un pequeño programa de prueba y una red de sucesos pequeña hecha para la ocasión, `DES-illness.pgmx` (el repositorio no guarda ninguna red de este tipo), construyendo sobre ella los dos algoritmos que usa la entrada, `VECEAnalysis` (ámbito de una decisión) y `DANDecompositionIntoSymmetricDANsCEA` (ámbito global). Los dos lanzan `NotEvaluableNetwork Exception.NotApplicableNetwork`:

```
This algorithm cannot evaluate DESNet network named DES-illness.pgmx, as it can only
↪  evaluate networks of type: [Bayesian, Influence diagram, Markov Influence Diagram,
↪  Decision analysis].
```

Por qué pasa: `CostEffectivenessPlugin` enciende su entrada en cuanto hay una red abierta, sea del tipo que sea (líneas 53-61), y `onClick` solo distingue diagramas de influencia y MID; todo lo demás va a la descomposición de redes de análisis de decisiones (línea 103).

Por lo mismo la entrada está encendida con una red bayesiana. En una captura automática del 8-10-2026 con `BN-asia.pgmx`, aceptar los dos diálogos abre una tabla «Cost-effectiveness intervals» con una sola fila: coste 0, efectividad 0, intervención «---».

**Cómo arreglarlo.** Que la entrada del menú y el botón de la barra vayan por el mismo camino: lo más simple es que `CostEffectivenessPlugin.onClick` llame a lo mismo que el botón, que ya distingue la red de sucesos. Y que la entrada se encienda con la misma regla que el botón, no con «hay una red abierta».

Prueba: con una red de sucesos, comprobar que la entrada del menú no llega a construir `VECEAnalysis`; y con una red bayesiana sin criterios de coste y efectividad, que está apagada.

### Error 252 · El CSV que exporta la ventana de resultados de una simulación de sucesos pega varios números en una sola celda, y la fila se queda con menos columnas que títulos, cuando una media vale diez millones o más, o menos de una milésima

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.desnets |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/desnets/DESResultsWindow.java:117-147,234-240`<br>`gui/src/main/java/org/openmarkov/gui/desnets/SimulationResults.java:270-283`<br>`gui/src/main/java/org/openmarkov/gui/desnets/SimulationSummaryResults.java:200-211`<br>`gui/src/main/java/org/openmarkov/gui/desnets/DESInference.java:392-404` |
| **Comprobado** | Leyendo el código y ejecutándolo (la tabla y el CSV, armados sin ventana con resultados escritos a mano) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 103, error 102, error 12, nota 198 |

**Qué pasa.** En una red de simulación de sucesos discretos, el botón de coste-efectividad de la barra lanza la simulación y abre la ventana de resultados, con el botón «Export as CSV» (CSV: valores separados por comas). Si alguna media, o media descontada, vale diez millones o más, o menos de una milésima (sin ser cero), la fila de esa serie sale con menos columnas que la de títulos, y en sus celdas hay varios números pegados que ya no son ningún número. Una hoja de cálculo pone cada valor bajo un título que no es el suyo, o no lo lee.

Por qué pasa: desde `d08082e` (arreglo del error 103) la fila de títulos está bien, pero las filas de números siguen saliendo como salían. `DESResultsWindow.csv` parte del volcado de texto de los resultados (`SimulationSummaryResults.toString`) y borra con `replaceAll("[a-zA-Z].*?;", "")` todo lo que empieza por una letra hasta el siguiente punto y coma; así se van «SERIES», los nombres de las opciones y de los criterios, «Mean» y «DiscountedMean». Pero los números los escribe `SimulationResults.toString` concatenando el `double` (`"Mean;" + mean`, línea 273), es decir, con `Double.toString`, que usa **exponente** para todo valor de diez millones o más, o menor que una milésima: 12 345 678,9 se escribe «1.23456789E7». La «E» es una letra, así que la expresión se lleva «E7;» entero, **con el punto y coma que separaba ese número del siguiente**, y los dos quedan pegados.

Medido el 15-10-2026 con un pequeño programa de prueba que arma los resultados con las clases reales (decisión con «none» y «drug»; criterios «cost», con descuento, y «effectiveness», sin él; una serie, un individuo) y llama sin ventana a `generateTable` y a `csv`, como hace el botón. Las medias de «none» son 12 345 678,9 (descontada 11 111 111,01) en el coste y 0,0005 en la efectividad:

```
volcado:  SERIES;0;none;cost;Mean;1.23456789E7;DiscountedMean;1.111111101E7;effectiveness;⌋
↪  Mean;5.0E-4;DiscountedMean;5.0E-4;;drug;cost;Mean;100.0;...
```

```
títulos:  SERIES,Mean cost for none,Disc. Mean cost for none,Mean effectiveness for
↪  none,...   (9 columnas)
fila:     0,1.234567891.1111111015.05.0,100.0,90.0,2.0,2.0
↪  (6 columnas)
```

Los cuatro números de «none» acaban en una sola celda, y los de «drug» quedan bajo los títulos de «none». La tabla de la propia ventana no sufre esto: `generateTable` parte el volcado por puntos y comas y lee cada número con `Double.parseDouble` (`formatDouble`, líneas 234-240), que entiende el exponente; en la misma medida enseña bien 12 345 678,9, 11 111 111,01 y 0,0005. La prueba que acompañó a `d08082e` usa medias de 100, 90 y 2 y no lo ve.

Diez millones es mucho para la media de un individuo en euros, pero no en una moneda pequeña, y una efectividad media por debajo de una milésima (un suceso raro, una probabilidad) no tiene nada de raro.

Con la misma causa, y ya apuntado en el «Cómo arreglarlo» del error 103: un nombre de opción que empieza por una cifra. Con «10 mg», la expresión empieza a borrar en la «m» y deja «10 » pegado al número siguiente; medido en el mismo programa, la fila conserva sus nueve columnas pero la segunda dice «10 150.0».

**El código.** gui/src/main/java/org/openmarkov/gui/desnets/DESResultsWindow.java:136-145

```
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            line = line.replaceAll(";ICER.+", "")
                    .replaceAll("[a-zA-Z].*?;", "")
                    .replaceAll(",", ".")
                    .replaceAll(";+", ";")
                    .replaceAll(";$", "")
                    .replaceAll(";", ",");
            sb.append(line).append("\n");
        }
```

**Cómo arreglarlo.** Sacar cada fila de los objetos de resultados y no del texto: para cada serie de `SimulationSummaryResults.getSeriesList()`, para cada opción en el mismo orden en que se escribieron los títulos y para cada criterio, `getSimulationResults(criterio).getMean()` y `getDiscountedMean()`, escritos número a número (con `Double.toString`, que las hojas de cálculo leen con exponente, o con un formato fijo). Es la solución de fondo que ya proponían los errores 102 y 103: que la ventana reciba los resultados y no un volcado. Si se mantiene el volcado, por lo menos partirlo por puntos y comas y tomar los campos por su posición, como ya hace `generateTable`, en vez de borrar lo que empieza por una letra. Los títulos y las filas deben salir de la misma lista de opciones (ver nota 198, sobre su orden).

Prueba: como `TheExportedResultsHaveATitleOverEachColumnTest`, con medias de 12 345 678,9 y 11 111 111,01 en el coste y 0,0005 en la efectividad de «none»: cada fila tiene tantas columnas como títulos y `Double.parseDouble` de cada celda da la media de su título (12345678.9 bajo «Mean cost for none»). Y otro caso con una opción llamada «10 mg».

## 11 Abrir y guardar redes

La lectura y escritura de redes: el formato propio ProbModelXML (.pgmx) y la puerta de entrada de los demás formatos.

### Error 111 · [Corregido] Un `.pgmx` con un elemento mal escrito se abre sin ningún aviso y con todas sus tablas convertidas en uniformes

| | |
|---|---|
| **Estado** | Corregido en el commit `24ef74b` (10-10-2026): al abrir un fichero, la ventana lo valida contra `val_v4.xsd` (`FormatManager.schemaProblems`) y, si no lo cumple, enseña un aviso con la línea y el motivo de cada incumplimiento; la red se abre igual. Visto en la aplicación con `BN-asia.pgmx` y `<Valores>`: «Line 142: Invalid content was found starting with element 'Valores'…». Medido ese día sobre los 492 `.pgmx` del repositorio: 27 no cumplen el esquema; 22 son de versiones que el programa ya rechaza y 5 se abren (dos copias de `Dec-POMDP-wireless-network`, dos de `POMDP-coffee-robot` y `io/src/test/resources/HPV-model-0.2.0.pgmx`, con `DecisionCriteria` donde no se espera). Lo que el programa escribe (las 458 redes legibles, en los dos formatos) cumple el esquema salvo `POMDP-coffee-robot`, cuya rama sin estados se vuelve a escribir igual. Validar tarda unos 2,4 ms por fichero, lo mismo que leerlo. Sigue igual: fuera de la ventana nada avisa (es la pregunta del error 37), y `checkStructure(URL)` sigue vacío. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.core.io.format.annotation |
| **Ficheros** | `io/src/main/java/org/openmarkov/core/io/format/annotation/FormatManager.java:119-132,226-238,240-258`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:134-150`<br>`gui/src/main/java/org/openmarkov/gui/dialog/io/NetsIO.java:168-180` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir entre rechazar el fichero que no cumple el esquema o abrirlo avisando. Manuel Arias lo decidió el 10-10-2026: abrirlo avisando, como las columnas que no suman uno |
| **Relacionados** | error 37, error 114, error 117, nota 50, nota 181 |

**Qué pasa.** Un fichero `.pgmx` (el formato propio ProbModelXML) tiene un esquema XML (lenguaje de marcado extensible), `val_v4.xsd`, que dice qué elementos lleva y en qué orden. Al abrir un fichero, el programa debería comprobarlo contra ese esquema y, si no lo cumple, decir en qué línea está el problema.

El usuario llega con Archivo → Abrir sobre un `.pgmx` editado a mano, generado por otro programa o dañado. La red aparece normal y el usuario trabaja con números que no son los del fichero, sin ninguna señal.

Los dos caminos de apertura llaman a `FormatManager.checkStructure(URL)`: la ventana, a través de `NetsIO.openNetworkURL`/`openNetworkFile` → `FormatManager.getProbNetReader` (línea 124), y el lector, en `PGMXReader_0_2.read` (línea 138). Ese método tiene todo su cuerpo dentro de un comentario (líneas 240-258), así que no comprueba nada.

Existe otra variante, `checkStructure(String, InputStream)` (líneas 226-238), que sí valida contra el esquema y lanza `ProbNetParserException.BadlyStructuredFile` con la línea y la columna del error, pero nadie la llama. `checkVersion`, que sí está activo, solo mira el atributo de versión.

Como el lector busca los elementos por nombre y, si no los encuentra, sigue con valores por defecto, un fichero con la estructura mal se lee «bien» con otro contenido.

Medido con un pequeño programa de prueba: se copió `0_miscellaneous/networks/bn/BN-asia.pgmx` cambiando `<Values>` por `<Valores>` en las tablas. Por el mismo camino que usa la ventana (`getProbNetReader(url).read(url)`) se abre sin error, con sus 8 nodos, y **todas** las tablas salen uniformes (por ejemplo, X-ray `[0.5, 0.5, 0.5, 0.5]` en lugar de `[0.95, 0.05, 0.02, 0.98]`).

La variante con flujo, sobre el mismo fichero, lo rechaza: «An error occurred at line 142, column 18: cvc-complex-type.2.4.a … 'Valores' …».

Hace falta una decisión porque, validando con la variante que funciona las 150 redes `.pgmx` de `0_miscellaneous/networks`, 148 pasan y 2 no:

- `0_miscellaneous/networks/pomdp/Dec-POMDP-wireless-network.pgmx`: elemento `Agents` en un sitio que el esquema no admite, línea 349.
- `0_miscellaneous/networks/pomdp/POMDP-coffee-robot.pgmx`: un `States` vacío, línea 627.

Las dos son POMDP (procesos de decisión de Markov parcialmente observables) y hoy se abren. Si la validación rechaza, dejarán de abrirse; si avisa, se abrirán con un aviso. Es la misma pregunta que plantea error 37 para las restricciones de la red.

**El código.** io/src/main/java/org/openmarkov/core/io/format/annotation/FormatManager.java:240-258

```
public void checkStructure(URL url) throws ProbNetParserException.BadlyStructuredFile {
    /*
    InputStream xsd = getClass().getClassLoader().getResourceAsStream("val_v4.xsd");
    SchemaFactory factory = SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);
    ...
    try {
        schema.newValidator().validate(new StreamSource(url.openStream()));
    } catch (SAXException e) {
        throw new UnrecoverableException(new ProbNetParserException.CannotParseFile(e,
↪   url));
    } ...
    */
}
```

**Cómo arreglarlo.** Hacer que `checkStructure(URL)` valide apoyándose en la variante con flujo: abrir `url.openStream()` y delegar en `checkStructure(String, InputStream)` (líneas 226-238), que ya produce `BadlyStructuredFile` con línea y columna. El bloque comentado convertía el error de validación en `UnrecoverableException`, que no es lo adecuado para un error del fichero del usuario.

Hay dos llamadores y los dos quedan cubiertos con ese único cambio: `FormatManager.getProbNetReader` (línea 124) y `PGMXReader_0_2.read` (línea 138). Conviene que solo valide uno de los dos para no leer el fichero dos veces.

Según lo que decida el equipo, `NetsIO` rechaza el fichero con el mensaje o lo abre y muestra el aviso. Si se elige rechazar, hay que corregir antes las dos redes POMDP citadas.

Prueba de regresión: el fichero de prueba descrito arriba (`BN-asia.pgmx` con las tablas en `<Valores>`) debe dar `BadlyStructuredFile` (o el aviso) al abrirse por `getProbNetReader(url).read(url)`, y las 148 redes que hoy cumplen el esquema deben seguir abriéndose.

### Error 112 · [Corregido] Guardar en formato ProbModelXML 1.0, el que la ventana propone para una red nueva, convierte las utilidades en distribuciones y la red ya no se puede evaluar al reabrirla

| | |
|---|---|
| **Estado** | Corregido en dos commits. `551e715` (23-09-2026): una distribución «Exact» con números, que es como el formato 1.0 guarda las relaciones exactas, se convierte en tabla como el `ExactDistrPotential` que es, así que las redes guardadas en 1.0 vuelven a evaluarse y dan lo mismo que en 0.2. `4eba97f` (10-10-2026): `PGMXWriter_0_2` escribe esa distribución como la relación exacta, |

| | |
|---|---|
| | así que una red abierta desde un fichero 1.0 y guardada en 0.2 se vuelve a abrir (`ID-decide-test-1-0.pgmx` sigue dando 9,3289). Medido ese día con las 161 redes en formato 1.0 del repositorio, guardadas en 0.2 y reabiertas: antes no se reabrían 120 por esta causa; ahora se reabren 119. La que queda es `Dynamic-LIMID-every-potential.pgmx`, que lleva una distribución Normal: el formato 0.2 no tiene distribuciones y el escritor la sigue escribiendo de una forma que su lector rechaza. Otras dos (`BN-conditional-gaussian.pgmx` y `BN-discretized-cauchy.pgmx`, de `io/src/test/resources`) no se reabren por otra causa, sin mirar: un `NullPointerException` en el lector. |
| **Módulo** | io, core, gui |
| **Paquete** | org.openmarkov.io.probmodel.writer; org.openmarkov.io.probmodel.reader; org.openmarkov.core.model.network.potential; org.openmarkov.gui.window |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_1_0.java:191-199`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:126-166`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:356-364`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:149-156`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader.java:32-42`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:89-111,136-139,214-215,256-290,490-520`<br>`gui/src/main/java/org/openmarkov/gui/dialog/io/NetworkOMFileChooser.java:73-82,107-124`<br>`gui/src/main/java/org/openmarkov/gui/configuration/UserPreferences.java:119-121` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No. Pedía decidir dónde se resuelve. Quedó decidido con el commit `551e715`: en el potencial, que arregla también los ficheros 1.0 ya guardados. Queda sin decidir qué hace el escritor 0.2 con una distribución que no es «Exact» con números |
| **Relacionados** | error 8, error 9, error 42, error 46, error 50, error 113, error 114, error 153 |

**Qué pasa.** Un usuario crea un diagrama de influencia en el editor, lo evalúa sin problemas y lo guarda. Al reabrirlo no puede evaluarlo: entrar en modo inferencia, pedir la estrategia óptima o lanzar el análisis de coste-efectividad terminan en una ventana de error con «Potential P(U | …) cannot be converted to a table»; si la red tiene nodos de utilidad que suman o multiplican otros, la estrategia óptima sale en la ventana de error no previsto. Guardar otra vez no lo arregla.

Qué formato usa cada gesto de guardar:

- Al abrir un `.pgmx`, `PGMXReader.read` (líneas 37-41) elige el escritor según la versión del fichero, y la ventana lo apunta en la pestaña de la red (`NetworkFileHandler`, líneas 208-209). «Save» usa ese escritor (líneas 255-257): una red abierta desde un fichero 0.2 se guarda en 0.2, y una abierta desde un 1.0, en 1.0.
- Una red que no tiene fichero no tiene escritor: una red nueva (línea 107), una red expandida (`InferenceHandler`, línea 181), una aprendida (`LearningDialog`, línea 904) o una abierta desde el árbol de decisión. Para ella «Save» hace «Save network as…».
- «Save network as…» abre la ventana de ficheros con los formatos «Elvira», «OpenMarkov.0.2» y «OpenMarkov.1.0». Marca de entrada el formato del escritor de la red, si lo tiene (líneas 491-500); si no, el último que el usuario eligió al guardar, y la primera vez `PGMXWriter_1_0` (`UserPreferences`, líneas 107-109; `NetworkOMFileChooser`, líneas 76-85).

Así, llegan al formato 1.0 una red nueva (o expandida, o aprendida) guardada tal como propone la ventana, cualquier red guardada con «Save network as…» eligiendo «OpenMarkov.1.0», y cualquier red abierta desde un fichero 1.0.

Por qué pasa. El editor da a todo nodo de utilidad nuevo una relación «Exact», un `ExactDistrPotential` (`PotentialUtils.generateDefaultPotential`, línea 157), y es la relación que el diálogo del nodo llama «Exact»: una tabla con el valor del nodo para cada combinación de sus padres. Los ficheros 0.2 la guardan como tabla y el lector 0.2 la reconstruye como `ExactDistrPotential`. `PGMXWriter_1_0` la escribe en cambio como `<Potential type="UnivariateDistr" distribution="Exact">` (líneas 191-199), y `PGMXReader_1_0.getUnivariateDistrPotential` la lee como `UnivariateDistrPotential` (líneas 142-165), salvo en las redes de simulación de sucesos, donde ya la devuelve como `ExactDistrPotential` (líneas 136-141). `UnivariateDistrPotential.tableProject` lanza siempre `PotentialCannotBeConvertedToATable` (líneas 356-364), y todos los cálculos exactos necesitan convertir en tabla cada potencial: la evaluación, la propagación, el coste-efectividad, el cálculo de rangos del modo inferencia (error 46), «Absorb parents» (error 9) y el árbol de decisión (error 50).

Medido con un pequeño programa de prueba:

- Un diagrama de influencia construido con las mismas ediciones que el editor (Disease → U ← Therapy, U con relación «Exact» y valores 10, 3, 8 y 9) da una utilidad esperada de 9,02. Guardado con `PGMXWriter_0_2` y reabierto, sigue dando 9,02. Guardado con `PGMXWriter_1_0` y reabierto, U es un `UnivariateDistrPotential` y la evaluación y la propagación del modo inferencia fallan con «Potential P(U | Disease, Therapy) cannot be converted to a table». Guardado otra vez con «Save» (1.0) y reabierto, igual; el fichero lleva además `<Functions>"1" "1" "1" "1"</Functions>` junto a los números, de modo que el editor de la relación enseña «1» en cada casilla (error 153).
- Guardando y reabriendo cada red en formato 0.2 de `0_miscellaneous/networks/id`, `dan`, `mid`, `limids` y `pomdp`: de las 53 cuya evaluación (`VEEvaluation`) termina, tras el formato 0.2 se siguen evaluando 44 (las otras nueve fallan por defectos distintos de este: siete, las que tienen nodos de producto, por error 113, corregido después de esta medida, y dos no llegan a guardarse por error 114) y tras el formato 1.0 ninguna. De las 41 en las que el cálculo del modo inferencia termina, tras el formato 1.0 ninguna. En `0_miscellaneous/networks/id` son 14 redes evaluables que dejan de serlo; el análisis de coste-efectividad (`VECEAnalysis`) de `ID-CEA-minimal.pgmx` y de `ID-arthronet-ce.pgmx` también deja de funcionar.
- Las 15 redes de `0_miscellaneous/networks/id/1-0/` fallan al evaluarse; sus versiones 0.2 de `0_miscellaneous/networks/id/` se evalúan, salvo `ID-delayed-result-of-test.pgmx`, que falla en los dos formatos por tener utilidades uniformes.
- No hay vuelta atrás desde la ventana: guardar con «Save network as…» en «OpenMarkov.0.2» una red reabierta desde un fichero 1.0 escribe el `UnivariateDistrPotential` tal cual, y al abrir ese fichero el lector 0.2 lo rechaza (`PGMXParserException$PotentialTypeNotSupported`). Queda volver a elegir «Exact» en cada nodo y teclear otra vez sus números (por lectura: el diálogo crea la relación nueva vacía).

**El código.** io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_1_0.java:191-199

```
String potentialType = PotentialUtils.getPotentialName(potential.getClass());
if (potential.getClass() == ExactDistrPotential.class) {
    potentialType = PotentialUtils.getPotentialName(UnivariateDistrPotential.class);
    String distribution = "Exact";
    potentialElement.setAttribute(XMLAttributes.TYPE.toString(), potentialType);
    potentialElement.setAttribute(XMLAttributes.DISTRIBUTION.toString(), distribution);
} else {
```

```
    potentialElement.setAttribute(XMLAttributes.TYPE.toString(), potentialType);
}
```

io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:136-144

```
if (probNet.getNetworkType() instanceof DESNetworkType){
    if (Objects.requireNonNull(probNet.getNode(variables.getFirst())).getNodeType() ==
    ↪  NodeType.CHANCE){
        return
        ↪  PGMXPotentialParsers.getTablePotential(xmlPotential,probNet,xmlRole,variables);
    }
    return PGMXPotentialParsers.getExactDistrPotential(xmlPotential,probNet,xmlRole,
    ↪  variables);
}
UnivariateDistrPotential potential;
if (parametrization != null) {
    potential = new UnivariateDistrPotential(variables, univariateName, parametrization,
↪   xmlRole);
```

core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:356-359

```
@Override
public @NotNull TablePotential tableProject(EvidenceCase evidenceCase, InferenceOptions
↪   inferenceOptions) throws
↪   NonProjectablePotentialException.PotentialCannotBeConvertedToATable {
    throw new NonProjectablePotentialException.PotentialCannotBeConvertedToATable(this);
}
```

gui/src/main/java/org/openmarkov/gui/configuration/UserPreferences.java:113-115

```
public static final UserPreference<Class<? extends ProbNetWriter>>
↪   LATEST_SAVED_NETWORK_WRITER_CLASS = UserPreference
        .of("formats/latest_saved_network_writer", () -> PGMXWriter_1_0.class, null, new
        ↪   TypeToken<>() {
        });
```

**Cómo arreglarlo.** Una relación «Exact» tiene que volver de un fichero 1.0 como algo que los cálculos exactos puedan convertir en tabla. Hay tres sitios posibles, y elegir entre ellos es una decisión:

- Al escribir: que `PGMXWriter_1_0` guarde un `ExactDistrPotential` de forma que el lector 1.0 lo reconstruya como `ExactDistrPotential`, como ya ocurre entre el escritor y el lector 0.2. No arregla los ficheros 1.0 ya guardados, entre ellos las copias de `0_miscellaneous/networks/*/1-0/`.
- Al leer: que `PGMXReader_1_0.getUnivariateDistrPotential` devuelva un `ExactDistrPotential` cuando la distribución es «Exact» y sus parámetros son números (sin fórmulas en `<Functions>` que dependan de otras variables), como ya hace para las redes de sucesos (líneas 136-141). Arregla también los ficheros existentes; el editor enseñaría «Exact» en lugar de «UnivariateDistr».
- En el potencial: que `UnivariateDistrPotential.tableProject` (las dos versiones, líneas 356-364) construya la tabla cuando la distribución es «Exact» con parámetros numéricos, igual que `ExactDistrPotential`. Arregla los ficheros existentes y todos los llamadores a la vez, sin cambiar la relación que ve el usuario.

Con cualquiera de las tres, lo que hoy falla por esta causa en otras fichas deja de fallar por ella: los

rangos del modo inferencia de las copias 1.0 (error 46), «Absorb parents» en sus nodos de suma y producto (error 9; las versiones 0.2 de esas redes, con `ExactDistrPotential`, ya se absorben) y la proyección de `DANOperations.projectPotentials` del árbol de decisión (error 50, por lectura: pasaría por su rama de `tableProject`). Conviene comprobar esas tres fichas después del arreglo.

Aparte de dónde se arregle: `PGMXWriter_0_2` no debería escribir un tipo de potencial que su propio lector rechaza.

Prueba de regresión: para cada red de `0_miscellaneous/networks/id/*.pgmx` en formato 0.2, guardar con `PGMXWriter_1_0`, reabrir con `PGMXReader` y comprobar que `VEEvaluation.getUtility()` da lo mismo que antes de guardar; lo mismo con un diagrama de influencia creado con `AddNodeEdit` y `AddLinkEdit` y un nodo de utilidad «Exact». Si se arregla al leer o en el potencial, además, `0_miscellaneous/networks/id/1-0/ID-decide-test-1-0.pgmx` debe dar la misma utilidad esperada que `0_miscellaneous/networks/id/ID-decide-test.pgmx` (9,3289).

### Error 113 · [Corregido] Guardar en formato ProbModelXML 0.2, el que usa «Save» con una red abierta desde un fichero 0.2, convierte en uniforme todo nodo de utilidad de producto, y al reabrirla la red ya no se evalúa

| | |
|---|---|
| **Estado** | Corregido en el commit `e2f5ac1` (09-10-2026): `PGMXWriter_0_2.getPotentialAttributesAndVariables` escribe la variable del nodo una sola vez, conserva el tipo (`Sum` o `Product`) y pone todos los padres dentro de `<Variables>`, sin tocar la red que se guarda (guardar quitaba además la variable del nodo de suma en la red abierta). Con prueba sobre las siete redes citadas: tras guardar y reabrir, cada nodo de utilidad conserva su clase y sus variables y la utilidad esperada es la misma. El lector no se ha tocado: los ficheros ya guardados con el defecto siguen abriéndose mal, y en ellos hay que volver a elegir «Product». No se ha probado en la ventana. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.writer; org.openmarkov.io.probmodel.reader |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:770-798,833-851`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:726-736`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/SumPotential.java:141-144`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/ProductPotential.java:68-77`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1397-1409`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader.java:32-41`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:256-264`<br>`core/src/main/java/org/openmarkov/core/inference/BasicOperations.java:42-60` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 8, error 42, error 46, error 63, error 112, error 114 |

**Qué pasa.** Un nodo de utilidad de producto multiplica las utilidades de sus padres: es la relación «Product» del diálogo del nodo, que se ofrece a los nodos de utilidad cuyos padres son otros nodos de utilidad (`ProductPotential.validate`, líneas 68-77). Siete redes de `0_miscellaneous/`

`networks` en formato 0.2 los tienen: `id/ID-arthronet.pgmx`, `id/ID-mediastinet.pgmx`, `dan/DAN-arthronet.pgmx`, `dan/DAN-economic-mediastinet.pgmx`, `dan/DAN-mediastinet.pgmx`, `dan/DAN-qale-mediastinet.pgmx` y `mid/MID-HPV.pgmx`.

Camino del usuario: abrir `0_miscellaneous/networks/id/ID-arthronet.pgmx`; en modo edición, «Show optimal strategy» enseña la estrategia óptima. Hacer cualquier cambio («Save» solo se enciende con la red modificada), pulsar «Save», cerrar la red y volver a abrir el fichero. Ahora «Show optimal strategy» termina en la ventana de error no previsto con `ClassCast Exception: UniformPotential cannot be cast to FunctionPotential`, y la relación del nodo «Coste ajustado» aparece como «Uniform». Guardar otra vez no lo arregla: el fichero ya no dice que el nodo era un producto.

«Save» usa el formato del fichero abierto: `PGMXReader.read` (líneas 37-41) devuelve el escritor de su versión y `NetworkFileHandler.saveNetwork` (líneas 250-258) lo usa, así que toda red abierta desde un fichero 0.2 se guarda con `PGMXWriter_0_2`. Llega también «Save network as…» eligiendo el formato «OpenMarkov.0.2», con cualquier red. El formato 1.0 escribe bien el producto; su problema con las utilidades es otro (error 112).

Por qué pasa. `PGMXWriter_0_2.getPotentialAttributesAndVariables` (módulo `io`, líneas 770-798) trata juntos los potenciales de suma y de producto de un nodo de utilidad: escribe la variable del nodo como `<UtilityVariable>` y, para quitarla de la lista, llama a `potential.removeVariable(utilityVariable)` y sigue con lo que devuelve. `SumPotential` redefine `removeVariable` y devuelve el mismo potencial; `ProductPotential` no, y la versión de `Potential` (líneas 726-736) crea un `UniformPotential` con las variables que quedan. El tipo que se escribe sale de la clase de ese objeto: `type="Uniform"`.

En el mismo método hay un segundo fallo, que afecta a sumas y productos. Después, `getPotentialVariables` (líneas 833-851) mira si la primera variable que queda es de un nodo de utilidad y, si lo es, la escribe como otra `<UtilityVariable>` en lugar de dentro de `<Variables>`. En un nodo de suma o de producto esa variable es su primer padre, que es otro nodo de utilidad. El lector (`PGMXReader_0_2.getPotential`, líneas 1403-1409) solo lee la primera `<UtilityVariable>`, así que ese padre desaparece de las variables del potencial al reabrir.

Al calcular, el preproceso de los algoritmos exactos absorbe el nodo uniforme como si fuera una fórmula y falla con el `ClassCastException` que describe error 8 (`BasicOperations.absorbParentPotentials`, línea 54). Arreglado ese error, el nodo seguiría sin ser un producto.

Medido con un pequeño programa de prueba que guarda cada red con el escritor que usaría «Save», la reabre y repite los cálculos de «Show optimal strategy» (`VEOptimalIntervention` en diagramas de influencia y en MID —diagramas de influencia con memoria—, `DANDecompositionIntoSymmetricDANsEvaluation` en las DAN —redes de análisis de decisiones—) y, en `MID-HPV.pgmx`, el análisis de coste-efectividad global:

- `ID-arthronet.pgmx`: utilidad esperada 0,4961 antes de guardar; tras reabrir, `ClassCast Exception`. «Coste ajustado» pasa de producto de «Coste total» y «C2E» a uniforme sobre «C2E».
- `ID-mediastinet.pgmx`: 3,9844 → `ClassCastException`; se estropean «Net_QALE» y «Weighted_Economic_Cost».
- `DAN-economic-mediastinet.pgmx` (−0,1223), `DAN-mediastinet.pgmx` (1,4710) y `DAN-qale-mediastinet.pgmx` (2,1154): → `ClassCastException`.
- `MID-HPV.pgmx`: utilidad 3480,73 y coste-efectividad global con tres tramos → `ClassCast Exception` en los dos; se estropea «QoL [0]».
- `DAN-arthronet.pgmx`: su estrategia ya falla antes de guardar (error 63); tras guardar, «Coste ajustado» queda también uniforme.

Para ver qué exige el arreglo se corrigieron a mano los ficheros guardados. Cambiando solo `type="Uniform"` por `type="Product"`, los dos diagramas de influencia y las tres DAN vuelven a dar sus números, pero `MID-HPV.pgmx` da otros sin avisar: utilidad 3563,98 y efectividad 143,5 en cada tramo, en vez de 3480,73 y 60,16, porque a «QoL [0]» le falta el factor «HSR QoL [0]». Pasando además el primer padre a `<Variables>`, las seis dan lo mismo que antes de guardar. En las sumas, la segunda `<UtilityVariable>` no ha cambiado ningún resultado medido (la absorción toma los sumandos de los enlaces del nodo), pero el potencial reabierto tampoco tiene todas sus variables.

**El código.** io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:777-789

```
} else if (potential instanceof SumPotential || potential instanceof ProductPotential) {
    if (potential.getPotentialRole() != PotentialRole.CONDITIONAL_PROBABILITY) {
        Variable utilityVariable = potential.getVariable(0); // it could be null in
↪  Branches potentials
        potential = potential.removeVariable(utilityVariable);
        getUtilityElement(potentialElement, utilityVariable);
    }
}

String potentialType = PotentialUtils.getPotentialName(potential.getClass());
...
potentialElement.setAttribute(XMLAttributes.TYPE.toString(), potentialType);
```

io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:833-845

```
protected static void getPotentialVariables(Potential potential, Element potentialElement,
↪  ProbNet probNet) {
    List<Variable> potentialVariables = potential.getVariables();
    if (!potentialVariables.isEmpty()
            && probNet.getNode(potentialVariables.get(0)).getNodeType() ==
            ↪  NodeType.UTILITY) {
        Variable utilityVariable = potential.getVariable(0);
        ...
            Element utilityElement = new Element(XMLTags.UTILITY_VARIABLE.toString());
            writeVariableName(utilityVariable, utilityElement);
            potentialElement.addContent(utilityElement);
        ...
        potentialVariables.remove(utilityVariable);
```

core/src/main/java/org/openmarkov/core/model/network/potential/Potential.java:726-736

```
public Potential removeVariable(Variable variable) {
    Potential newPotential;
    if (variables.contains(variable)) {
        List<Variable> newVariables = new ArrayList<>(variables);
        newVariables.remove(variable);
        newPotential = new UniformPotential(newVariables, role);
    } else {
        newPotential = this;
    }
    return newPotential;
}
```

0_miscellaneous/networks/id/ID-arthronet.pgmx:734-740 («Coste ajustado» en el fichero original)

```
<Potential type="Product" role="utility">
  <UtilityVariable name="Coste ajustado" />
```

```
  <Variables>
    <Variable name="Coste total" />
    <Variable name="C2E" />
  </Variables>
</Potential>
```

El mismo nodo en el fichero que escribe «Save»:

```
<Potential type="Uniform" role="utility">
  <UtilityVariable name="Coste ajustado" />
  <UtilityVariable name="Coste total" />
  <Variables>
    <Variable name="C2E" />
  </Variables>
</Potential>
```

**Cómo arreglarlo.** En `PGMXWriter_0_2.getPotentialAttributesAndVariables`, para la suma o el producto de un nodo de utilidad: escribir la variable del nodo una sola vez como `<UtilityVariable>`, el tipo de la clase original (`Sum` o `Product`) y todos los padres dentro de `<Variables>`. Sin llamar a `removeVariable`, que en `Potential` devuelve un `UniformPotential` y en `SumPotential` quita la variable del potencial de la propia red (por lectura), y sin que `getPotentialVariables` vuelva a sacar el primer padre; por ejemplo, trabajando con una copia de la lista de variables sin la del nodo.

Hacen falta las dos partes: con solo el tipo, `MID-HPV.pgmx` da otros números sin avisar (medido arriba).

Otros sitios: las ramas de un árbol se escriben con el mismo `getPotential` (`getTreeADDBranch`), así que el cambio les vale también; `PGMXWriter_1_0` redefine este método y no tiene el problema. Los ficheros ya guardados no conservan que el nodo era un producto, y habrá que volver a elegir «Product» en su diálogo. Si se quiere que el lector recupere al menos el padre perdido de esos ficheros, `PGMXReader_0_2.getPotential` puede tomar una segunda `<UtilityVariable>` como una variable más del potencial.

Prueba de regresión: para las siete redes citadas, guardar con `PGMXWriter_0_2`, reabrir y comprobar que cada nodo de utilidad conserva su clase y sus variables, y que la utilidad esperada (y, en `0_miscellaneous/networks/mid/MID-HPV.pgmx`, el análisis de coste-efectividad global) es la misma que antes de guardar.

### Error 114 · [Jorge Rico] «Save» falla con NullPointerException en ID-CEA-test-2therapies-new-test, MID-CHAP-Ryan-Griffin y MID-Cochlear: sus ficheros traen un papel de potencial mal escrito que el lector acepta sin avisar

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.reader; org.openmarkov.io.probmodel.writer |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1375-1387,1563-1571`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:70-97,733-753`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/PotentialRole.java:31-42`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.jav` |

| | |
|---|---|
| | `a:256-264,318-334,378-398`<br>`0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx:148`<br>`0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx:808`<br>`0_miscellaneous/networks/mid/MID-Cochlear.pgmx:4854`<br>`integrationTests/src/main/resources/networks/mid/MID-Cochlear.pgmx` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si el lector rechaza un papel de potencial desconocido (corrigiendo antes los ficheros que lo traen) o lo acepta como sinónimo avisando |
| **Relacionados** | error 71, error 111, error 112, error 113 |

**Qué pasa.** Abrir `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx`, `0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx` o `0_miscellaneous/networks/mid/MID-Cochlear.pgmx`, cambiar algo y pulsar «Save» termina en la ventana de error no previsto con `NullPointerException: Cannot invoke "PotentialRole.toString()" because "potentialRole" is null`. La red no se guarda y sigue marcada como modificada. Al cerrarla contestando que sí a guardar, falla igual y la pestaña no se cierra (por lectura: `NetworkFileHandler.networkCanBeClosed`, líneas 372-392). «Save network as…» con el formato «OpenMarkov.0.2» falla también; con «OpenMarkov.1.0» guarda, pero las utilidades quedan como describe error 112.

El fichero no se estropea: `PGMXWriter_0_2.write` (líneas 70-97) construye el documento entero antes de abrir el fichero, y la excepción salta al construirlo. Antes de escribir, «Save» ya ha copiado el fichero a uno `.bak` junto a él (`createBackUpNetworkFile`, líneas 312-328).

Por qué pasa. Los tres ficheros, en formato 0.2, traen un potencial con `role="joinProbability"`, y el nombre que usa el programa es `jointProbability` (`PotentialRole.JOINT_PROBABILITY` escrito con su `toString`). Es la tabla de probabilidad condicionada de un nodo de azar: «New test» en la primera red (línea 148 del fichero), «AIDS [0]» en la segunda (línea 808) e «Intervention applied» en la tercera (línea 4854). La copia `integrationTests/src/main/resources/networks/mid/MID-Cochlear.pgmx` tiene la misma errata.

`PGMXReader_0_2.getPotentialRole` (módulo `io`, líneas 1375-1387) rechaza un potencial sin papel con `PotentialWithoutRole`, pero un papel que no reconoce lo pasa a `getPotentialRolByLabel` (líneas 1563-1571), que devuelve `null` sin avisar. Los cálculos no usan ese papel, así que las redes se abren y se evalúan con normalidad. Al guardar, `PGMXWriter_0_2.setPotentialRole` (líneas 746-753) llama a `potentialRole.toString()` y falla. `PGMXWriter_1_0` no escribe el papel dentro del potencial, y por eso ese formato sí guarda. Validar el fichero contra su esquema (error 111) tampoco lo detectaría: el esquema no comprueba los atributos del potencial.

Medido con un pequeño programa de prueba:

- Recorriendo todas las redes de `0_miscellaneous/networks`, estas tres son las únicas con un potencial de papel nulo y las únicas que `PGMXWriter_0_2` no puede guardar.
- «Save» sobre una copia de cada una, como lo hace la ventana (el escritor que devuelve el lector, sobre el mismo fichero): `NullPointerException` en `setPotentialRole`, y el fichero con el mismo tamaño que antes.
- Cambiando en otra copia `joinProbability` por `conditionalProbability`, o por `jointProbability`, la red se guarda, se reabre y la evaluación sin hallazgos da lo mismo que la original: 271 387,976 en `ID-CEA-test-2therapies-new-test.pgmx` y 156 890,28 en `MID-Cochlear.pgmx`. `MID-CHAP-Ryan-Griffin.pgmx` no se evalúa ni antes ni después, por error 71.

Hay otro sitio de la ventana que decide por el papel, leído en el código y no ejecutado. En la

ventana del nodo «New test», el botón «Reorder variables» se ofrece para una tabla de probabilidad de más de dos variables, como esta, que tiene tres (`PotentialEditPanel.enableReorderVariableButton`, líneas 659-670). `ReorderVariablesPanel` quita de la lista la primera variable, que es la del propio nodo, solo si el papel es `CONDITIONAL_PROBABILITY` (`isConditionalProbabilityNotExactDistrPotential`, líneas 81-84, 93-95 y 146-148). Con el papel nulo no la quita: la lista ofrece reordenar «New test» junto con sus padres, y al aceptar no la vuelve a poner la primera. En las otras dos redes no pasa: las tablas de «AIDS [0]» y de «Intervention applied» también salen con el papel nulo, pero tienen dos variables, y el botón no se ofrece (los papeles y el número de variables, medidos el 15-10-2026 abriendo las tres redes con el lector del programa).

**El código.** io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1375-1386

```
protected PotentialRole getPotentialRole(Element xmlPotential) throws PGMXParserException {
    String xmlPotentialRole =
    ↪  xmlPotential.getAttributeValue(XMLAttributes.ROLE.toString());
    if (xmlPotentialRole == null) {
        throw new PGMXParserException.PotentialWithoutRole(xmlPotential);
    }
    PotentialRole xmlRole;
    if (xmlPotentialRole.equalsIgnoreCase("utility")) {
        xmlRole = PotentialRole.UNSPECIFIED;
    } else {
        xmlRole = PGMXReader_0_2.getPotentialRolByLabel(xmlPotentialRole);
    }
    return xmlRole;
```

io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1563-1571

```
protected static PotentialRole getPotentialRolByLabel(String auxLabel) {
    for (PotentialRole role : PotentialRole.values()) {
        String u = role.toString();
        if (u.equals(auxLabel)) {
            return role;
        }
    }
    return null;
}
```

io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:746-748

```
protected static void setPotentialRole(Element potentialElement, Potential potential) {
    PotentialRole potentialRole = potential.getPotentialRole();
    String oldPotentialRoleString = potentialRole.toString();
```

**Cómo arreglarlo.** Hay tres cosas que hacer, y la primera exige decidir:

- Qué hace el lector con un papel desconocido. Una opción es rechazar el fichero con una excepción que nombre el papel y el nodo, como ya hace con un potencial sin papel; entonces hay que corregir antes los cuatro ficheros citados, o dejarán de abrirse. La otra es aceptar `joinProbability` como sinónimo y avisar al abrir. En los dos casos, lo que no puede hacer es dejar el papel en `null` en silencio. Para estas tablas de nodos de azar, el papel que corresponde es el de probabilidad condicionada, que es también el que hace que la ventana de reordenar variables deje fija la del nodo.
- El escritor no debería fallar con un papel nulo: o deduce el papel del tipo de nodo, o lanza `WriterException` con un mensaje que nombre el nodo, que la ventana enseñaría como error normal.

- Corregir la errata en los cuatro ficheros.

Prueba: leer las tres redes, guardarlas con `PGMXWriter_0_2` en un fichero temporal, reabrirlas y comprobar que `0_miscellaneous/networks/id/ID-CEA-test-2therapies-new-test.pgmx` sigue dando una utilidad esperada de 271 387,976; y una prueba del lector con un papel inventado que dé el comportamiento que se decida.

### Error 115 · [Corregido] Abrir una red de sucesos con un nombre de distribución o parametrización que no existe falla con un NullPointerException lejos de la causa

| | |
|---|---|
| **Estado** | Corregido en el commit `e349a63` (10-10-2026): `PGMXReader_1_0.getDistributionTablePotential` comprueba la pareja antes de construir la relación y, si el programa no la conoce o falta la parametrización, lanza `PGMXParserException.UnknownDistribution` («The relation of Onset cannot be read: the program does not know the distribution Exact with the parametrization Alpha / Beta.», o «…without a parametrization.»). Con prueba de los dos casos sobre `io/src/test/resources/DES-illness.pgmx`. No se ha visto en la ventana cómo sale el mensaje. Sigue igual `DistributionTablePotential.changeDistribution`, que se traga el fallo si se construye la relación desde código con una pareja desconocida; desde la ventana no se llega. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java:93-96`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java:141-165`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:310-324`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/DistributionTablePotentialPanel.java:217-236` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir si un fichero sin el atributo `parametrization` se lee con una parametrización por omisión o se rechaza con un mensaje. Manuel Arias lo decidió el 10-10-2026: se rechaza con un mensaje; el programa no tiene una parametrización por omisión de cada distribución |
| **Relacionados** | error 116 |

**Qué pasa.** El usuario abre un fichero `.pgmx` de una red DES (simulación de eventos discretos) en la que el potencial de un suceso nombra una distribución o una parametrización que el programa no conoce, o no trae el atributo `parametrization`. La lectura termina con un `NullPointerException` cuyo mensaje no dice qué nodo ni qué nombre no se reconoció.

Lo que ve el usuario es el aviso de fichero estropeado: `NetsIO.openNetworkURL` (módulo `gui`, líneas 173-178) convierte cualquier excepción de tiempo de ejecución de la lectura en `CorruptNetworkFile`, cuyo mensaje es «Network from … is corrupted and could not be loaded.». El aviso no dice qué nodo ni qué nombre no se reconoció, y la causa solo queda en la traza de la consola.

`DistributionTablePotential` es el potencial de un nodo de suceso: una distribución de probabilidad cuyos parámetros se leen de una tabla. Construye su mitad interna en `changeDistribution`: pide al gestor `ParametrizedFunctionManager` la lista de parámetros de la pareja (distribución, parametrización), fabrica la variable de parámetros y la tabla `TableWithEvents`. Todo va dentro de un `try` cuyo `catch (Exception e)` solo imprime la traza.

Si el gestor no conoce la pareja, `getParameters` devuelve `null`, la línea siguiente lanza `NullPointerException`, el `catch` lo imprime por la consola y el constructor termina con normalidad, dejando `distributionParameters`, `distributionVariable` y `tableWithEvents` a `null`. El siguiente que use el potencial falla lejos de la causa.

Al abrir, `PGMXReader_1_0.getDistributionTablePotential` toma los nombres de los atributos `distribution` y `parametrization` del elemento `<Value>` sin validarlos, construye el potencial y en la línea siguiente llama a `potential.getTableWithEvents().setValues(table)`.

Medido:

- `new DistributionTablePotential(vars, rol, "Beta", "Alpha / Beta")` imprime la traza y devuelve un objeto con `getTableWithEvents()` y `getDistributionVariable()` a `null`.
- Con dos variantes de una red DES de prueba, (a) con `parametrization="Alpha / Beta"` en el potencial del suceso `Death` y (b) sin atributo `parametrization`, la consola muestra la primera traza y la lectura termina con `NullPointerException: Cannot invoke "TableWithEvents.setValues(double[])" because the return value of "DistributionTablePotential.getTableWithEvents()" is null`.

Desde la ventana del potencial (`DistributionTablePotentialPanel`) los desplegables solo ofrecen distribuciones y parametrizaciones que el gestor conoce, así que por ahí no se ha encontrado manera de llegar (comprobado leyendo, no ejecutando la ventana).

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/DistributionTablePotential.java:141-165

```
    public void changeDistribution(String distributionName, String
    ↪  distributionParametrization) {
        try {
            this.distributionName = distributionName;
            this.parametrizationName = distributionParametrization;
            List<String> distributionParametersList = ParametrizedFunctionManager.
            ↪  getUniqueInstance().getParameters(distributionName,
            ↪  distributionParametrization);
            distributionParameters = distributionParametersList.toArray(new
↪   String[distributionParametersList.size()]);
            ...
            this.tableWithEvents = new TableWithEvents(tableParents, role,
            ↪  (numericVariables.size() > 0));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:313-324

```
        String distributionName =
        ↪  distribution.getAttributeValue(XMLAttributes.DISTRIBUTION.toString());
        //08/10/2020 -parametrization added
        String distributionParametrization =
        ↪  distribution.getAttributeValue(XMLAttributes.PARAMETRIZATION.toString());
        ...
        potential = new DistributionTablePotential(variables, xmlRole, distributionName,
↪   distributionParametrization);
        potential.getTableWithEvents().setValues(table);
```

**Cómo arreglarlo.** Quitar el `catch` genérico de `changeDistribution` y rechazar en el sitio una pareja desconocida con una excepción declarada que nombre la distribución y la parametrización.

Sitios donde la regla «la pareja distribución/parametrización debe existir» tiene que cumplirse:

- `DistributionTablePotential.changeDistribution`, que llaman el constructor de cuatro argumentos, el de dos, el de copia y la ventana en `DistributionTablePotentialPanel.itemStateChanged` (línea 229).
- `PGMXReader_1_0.getDistributionTablePotential`: traducir esa excepción a la excepción de lectura del formato (`ProbNetParserException` o la que use el lector) con el nombre del nodo, para que el usuario vea un mensaje de fichero inválido.
- `DistributionTablePotential.sampleConditionedVariable` (línea 291) vuelve a pedir la clase al gestor con los mismos nombres; con la validación en la construcción ya no puede recibir una pareja desconocida.

Queda por decidir qué hacer con los ficheros sin atributo `parametrization`. Opciones: rechazarlos con un mensaje, o elegir una parametrización por omisión para la distribución nombrada.

Prueba de regresión: leer un `.pgmx` con una parametrización inexistente produce la excepción de lectura con el nombre del nodo y de la pareja, sin trazas en la consola; construir el potencial con esa pareja lanza la excepción declarada.

### Error 116 · [Corregido] Una red de sucesos cuyo fichero trae incertidumbre en la tabla de parámetros de un suceso no se puede abrir, y el aviso solo dice que el fichero está dañado

| | |
|---|---|
| **Estado** | Corregido en el commit `aef361a` (10-10-2026): `PGMXReader_1_0` rechaza `<UncertainValues>` en la tabla de un suceso y en una tabla de transiciones con `PGMXParserException.UncertaintyInAnEventTable` («The relation of Onset cannot be read: the table of an event or of a transition cannot carry uncertainty.»). Medido antes del arreglo: en la tabla de transiciones el fichero se abría y la incertidumbre se perdía sin aviso; ahora ese fichero no se abre. Con prueba de los dos casos sobre `io/src/test/resources/DES-illness.pgmx`. No se ha visto en la ventana cómo sale el mensaje. En `getDistributionTablePotential` queda, sin alcanzarse ya, el tramo que leía esa incertidumbre. |
| **Módulo** | io, core |
| **Paquete** | org.openmarkov.io.probmodel.reader; org.openmarkov.core.model.network.potential |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:311-341`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:297-316`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/TableWithEvents.java:71,114,233-239`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXPotentialParsers.java:97-120`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_1_0.java:317-324`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:893-902`<br>`core/src/main/java/org/openmarkov/core/action/core/UncertainValuesEdit.java:151-174`<br>`gui/src/main/java/org/openmarkov/gui/dialog/io/NetsIO.java:168-180` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir si las tablas de parámetros de los sucesos pueden llevar |

| | |
|---|---|
| | incertidumbre, o si el lector la rechaza o la ignora avisando. Manuel Arias lo decidió el 10-10-2026: el lector la rechaza con un mensaje que nombra el nodo, que es lo que ya hacen el editor (la rechaza), el escritor (no la escribe) y la simulación (no la usa) |
| **Relacionados** | error 76, error 115, nota 58 |

**Qué pasa.** En una red DES (simulación de eventos discretos), el tiempo hasta que ocurre un suceso lo da un `DistributionTablePotential`: una distribución (exacta, exponencial…) cuyos parámetros se leen de una tabla interna, un `TableWithEvents` que a su vez guarda un `TablePotential`.

El usuario abre con Archivo → Abrir un `.pgmx` de una red de sucesos en el que el potencial de un suceso trae, además de `<Values>`, un elemento `<UncertainValues>` con distribuciones para sus parámetros. La red no se abre. La ventana dice «Network from file:… is corrupted and could not be loaded.», sin decir qué nodo ni qué elemento del fichero es el problema.

El formato lo admite: el esquema `val_v4.xsd` permite `<UncertainValues>` en cualquier `<Potential>`, y el lector de la versión 1.0 lo busca a propósito en estos potenciales. `PGMXReader_1_0.getDistributionTablePotential` (líneas 326-331) lo lee y lo pasa a `potential.getTableWithEvents().setUncertainValues(...)`.

Por qué falla: la tabla interna es siempre un `TablePotential` a secas (los constructores de `TableWithEvents` la crean con `new TablePotential(...)`, líneas 71 y 114), y `TablePotential.setUncertainValues` (líneas 310-316) lanza `UnsupportedOperationException` con cualquier lista que no sea nula: solo su subclase `UncertainTablePotential` puede guardar incertidumbre. Para las tablas corrientes el lector ya elige `UncertainTablePotential` cuando el fichero trae incertidumbre (`PGMXPotentialParsers.getTablePotential`, líneas 97-120); para las tablas de sucesos nadie lo hace.

`NetsIO.openNetworkURL` convierte la excepción en `CorruptNetworkFile` sin guardar la causa, y la ventana la presenta con su texto fijo (esto último, por lectura).

Medido con una red de sucesos de prueba que se abre sin problemas (una decisión `Therapy`, un suceso `Onset`, un nodo de azar `Ill` y un suceso terminal `Death` con un `DistributionTable` exacto sobre `[Death, Onset, Ill]`), añadiendo al potencial de `Death` un `<UncertainValues>` con dos distribuciones normales:

```
FormatManager.getProbNetReader(url).read(url) -> UnsupportedOperationException: Cannot set
↪  uncertain values on a plain TablePotential. Use UncertainTablePotential instead.
     at TablePotential.setUncertainValues(TablePotential.java:312)
     at TableWithEvents.setUncertainValues(TableWithEvents.java:238)
     at PGMXReader_1_0.getDistributionTablePotential(PGMXReader_1_0.java:330)
NetsIO.openNetworkURL(url) -> CorruptNetworkFile: Network from file:…/red.pgmx is corrupted
↪  and could not be loaded. (causa: null)
```

Como comparación, el mismo `<UncertainValues>` (una Beta y un complemento) puesto en la tabla corriente del nodo de azar `Sex` de esa red se abre bien, y el potencial queda como `UncertainTablePotential`.

Camino de llegada: solo un fichero escrito a mano o por otra herramienta. OpenMarkov no escribe nunca esa incertidumbre: `PGMXWriter_1_0.getDistributionTablePotential` escribe la tabla interna con `getValuesTablePotential` (`PGMXWriter_0_2`, líneas 893-902), que solo añade `<UncertainValues>` si `getUncertainValues()` no es nulo, y en esta tabla lo es siempre. Tampoco se puede poner desde la interfaz: el menú contextual de la tabla de un suceso (`TableWithEventsPanel`) solo ofrece «Add function», y `UncertainValuesEdit.prepareUncertainPotential` (líneas 151-174) rechaza expresamente una tabla que va dentro de otro potencial. El repositorio no tiene redes de sucesos.

Aunque se leyera, esa incertidumbre no tendría hoy ningún efecto: la rama de análisis probabilístico de sensibilidad de la simulación no se puede activar, como se explica en nota 58, donde no hay nada que corregir.

**El código.** io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:320-331

```
        Element xmlUncertain = xmlPotential.getChild(XMLTags.UNCERTAIN_VALUES.toString());
        ...
        potential = new DistributionTablePotential(variables, xmlRole, distributionName,
↪  distributionParametrization);
        potential.getTableWithEvents().setValues(table);
        // 20/05/2024 Begin PSA
        UncertainValue[] uncertainValues;
        if (xmlUncertain != null) {
            uncertainValues = getUncertainValues(xmlUncertain);
            potential.getTableWithEvents().setUncertainValues(uncertainValues);
        }
```

core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:310-316

```
    public void setUncertainValues(UncertainValue[] uv) {
        if (uv != null) {
            throw new UnsupportedOperationException(
                    "Cannot set uncertain values on a plain TablePotential. Use
                    ↪  UncertainTablePotential instead.");
        }
        // null is a no-op: removing uncertainty from a non-uncertain potential is harmless
    }
```

**Cómo arreglarlo.** Hace falta decidir si las tablas de parámetros de los sucesos admiten incertidumbre. Opciones, sin elegir:

- **Admitirla.** Que la tabla interna de `TableWithEvents` sea un `UncertainTablePotential` cuando tenga que llevar incertidumbre, o que `TableWithEvents.setUncertainValues` la sustituya por uno con las mismas variables y valores, como hace `UncertainValuesEdit.upgradeToUncertain` con la tabla de un nodo. Entonces el constructor de copia de `TableWithEvents` (línea 114, `new TablePotential(...)`) tiene que conservar la clase y la incertidumbre, y el escritor la volverá a escribir sin más cambios (`getValuesTablePotential`). La simulación seguiría sin usarla.
- **Rechazarla.** En `PGMXReader_1_0.getDistributionTablePotential`, lanzar la excepción de lectura del formato (`PGMXParserException` o la que use el lector) con el nombre del nodo, diciendo que las tablas de sucesos no admiten incertidumbre.
- **Ignorarla avisando.** Leer el resto del potencial y avisar de que se ha descartado `<UncertainValues>`.

Sitios donde tiene que cumplirse la regla que se elija:

- `PGMXReader_1_0.getDistributionTablePotential`, el único que hoy lee incertidumbre para una tabla de sucesos.
- `PGMXReader_1_0.getTransitionTablePotential` (líneas 288-298), que no mira `<UncertainValues>` y, por lectura, la pierde sin avisar; debería tratarla igual.
- `TableWithEvents.setUncertainValues` y el constructor de copia de `TableWithEvents`.
- `UncertainValuesEdit.prepareUncertainPotential`, que ya rechaza la incertidumbre en tablas internas desde la interfaz; si se decide admitirla, es el otro sitio que cambia.

Prueba de regresión: en `io/src/test`, leer un `.pgmx` de sucesos con `<UncertainValues>` en un `DistributionTable`. Según lo decidido, se abre y conserva la incertidumbre (y al guardar y releer sigue ahí), o lanza una `ProbNetParserException` con el nombre del nodo; en ningún caso `UnsupportedOperationException`.

### Error 117 · [Corregido] Abrir un fichero `.xml` que no es ProbModelXML (por ejemplo un XMLBIF) desde «Abrir» falla con un `StringIndexOutOfBoundsException` sin decir que el formato no se reconoce

| | |
|---|---|
| **Estado** | Corregido en el commit `2c3fb1e` (09-10-2026): `PGMXReader.read` rechaza el fichero cuyo elemento raíz no se llama `ProbModelXML` con la excepción nueva `PGMXParserException.NotAProbModelXMLFile` («File … is not a ProbModelXML network: its root element is BIF.»); `FormatManager.getFileVersion` ya no corta una versión sin punto, y `ReaderFactory.getReader` lleva el fichero sin versión a `WrongVersion`, cuyo mensaje queda como «Version null is not supported.». Con prueba de los dos casos. No se ha probado en la ventana cómo presenta el diálogo el mensaje. |
| **Módulo** | io, gui |
| **Paquete** | org.openmarkov.core.io.format.annotation; org.openmarkov.io.probmodel.reader; org.openmarkov.gui.dialog.io |
| **Ficheros** | `io/src/main/java/org/openmarkov/core/io/format/annotation/FormatManager.java:119-160`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader.java:29-41`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/ReaderFactory.java:24-35`<br>`gui/src/main/java/org/openmarkov/gui/dialog/io/NetsIO.java:168-180`<br>`gui/src/main/java/org/openmarkov/gui/dialog/io/NetworkOMFileChooser.java:47-59` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 111, nota 173 |

**Qué pasa.** El usuario elige en el diálogo de abrir red un fichero `.xml` que no es ProbModelXML, por ejemplo uno en XMLBIF (un formato de intercambio de redes bayesianas en XML, lenguaje de marcado extensible), que usa precisamente esa extensión. Recibe una excepción de índice sin relación con el problema, en vez de un mensaje del tipo «este fichero no es ProbModelXML» (o «XMLBIF no está soportado»). No se ha comprobado cómo presenta la ventana esta excepción no controlada. Lo mismo ocurre con cualquier `.xml` sin atributo `formatVersion`, o con un valor sin punto.

El diálogo (`NetworkOMFileChooser`) construye sus filtros a partir de los lectores dados de alta. El de ProbModelXML (`PGMXReader`) declara las extensiones `pgmx` y `xml`, así que el usuario puede elegir cualquier fichero `.xml`.

Al abrir, `NetsIO.openNetworkURL` llama a `FormatManager.getProbNetReader(url)`, que antes de elegir lector calcula la versión del fichero en `getFileVersion`: lee el atributo `formatVersion` del elemento raíz y le quita la última parte con `substring(0, lastIndexOf('.'))`. En un fichero sin ese atributo, `getAttribute` devuelve la cadena vacía, `lastIndexOf('.')` es $-1$ y `substring(0, -1)` lanza. Esa llamada está fuera del `try` de `openNetworkURL`, así que la excepción no se convierte en `CorruptNetworkFile` y sube tal cual.

Ejecutado con `io/src/test/resources/netAB.xml` (un XMLBIF correcto):

```
FormatManager.getProbNetReader -> java.lang.StringIndexOutOfBoundsException: Range [0, -1)
↪  out of bounds for length 0
PGMXReader.read               -> java.lang.NullPointerException: Cannot invoke
↪  "String.startsWith(String)" because "strVersion" is null
                                  at ReaderFactory.lambda$getReader$0(ReaderFactory.java:27)
```

La segunda línea es lo que pasaría si se llegara al lector: `PGMXReader.read` pasa a `ReaderFactory.getReader` el atributo `formatVersion` leído con la biblioteca JDOM, que es `null`, y `strVersion.startsWith` revienta antes de llegar al `case null` que lanzaría la excepción prevista `ProbNetParserException.WrongVersion`.

El estado del propio lector de XMLBIF se describe aparte, en nota 50, donde no hay nada que corregir.

**El código.** io/src/main/java/org/openmarkov/core/io/format/annotation/FormatManager.java:134-159

```java
private static @NotNull String getFileVersion(URL url, String fileExtension) throws
↪  ProbNetParserException.BadlyStructuredFile {
    String fileVersion;
    if (fileExtension.equals("elv")) {
        fileVersion = "";
    } else {
        ...
        fileVersion = doc.getDocumentElement().getAttribute("formatVersion");
        //Removing the last index of the version
        fileVersion = fileVersion.substring(0, fileVersion.lastIndexOf('.'));
    }
    return fileVersion;
}
```

io/src/main/java/org/openmarkov/io/probmodel/reader/ReaderFactory.java:24-34

```java
public static PGMXReader_0_2 getReader(String strVersion) throws
↪  ProbNetParserException.WrongVersion {
    Version version = Arrays.stream(Version.values())
                            .filter(
                                    iteratorVersion ->
↪  strVersion.startsWith(iteratorVersion.toString()))
                            .findFirst()
                            .orElse(null);
    return switch (version) {
        case V02 -> new PGMXReader_0_2();
        case V10 -> new PGMXReader_1_0();
        case null -> throw new ProbNetParserException.WrongVersion(strVersion);
    };
}
```

**Cómo arreglarlo.** Hacer que un fichero sin versión (o que no es ProbModelXML) produzca una excepción de lectura con mensaje, en los dos sitios donde se interpreta la versión:

- `FormatManager.getFileVersion`: si el atributo está vacío o no tiene punto, lanzar `ProbNetParserException.WrongVersion` (o `BadlyStructuredFile`) en vez de cortar la cadena; mejor aún, comprobar primero que el elemento raíz se llama `ProbModelXML`. La versión que calcula ni siquiera se usa para elegir lector: solo va en el mensaje de `NoReaderForFileException`.
- `ReaderFactory.getReader`: tratar `strVersion == null` antes del `stream`, para que llegue al `WrongVersion` previsto.

La solución de fondo, reconocer el formato por el contenido y no solo por la extensión, es la que describe nota 173; este arreglo local es independiente y no la bloquea.

Prueba de regresión: en `io/src/test`, abrir `netAB.xml` con `FormatManager.getInstance().getProbNetReader(url)` y con `new PGMXReader().read(url)` y comprobar que ambos lanzan una `ProbNetParserException` (no una excepción de índice ni de nulo).

### Error 202 · [Corregido] Al abrir una red temporal en la que una variable nombra un agente que la red no tiene, el aviso nombra la variable sin su ciclo y no se sabe cuál de ellas es

| | |
|---|---|
| **Estado** | Corregido en el commit `38ab605` (09-10-2026): `loadVariableAdvancedInformation` guarda el nombre con su ciclo antes de recortarlo y se lo da a las dos excepciones; el aviso dice «Variable D1 [1] has the agent…». Con prueba del aviso del agente; el de la variable sin estados lleva el mismo cambio, sin prueba propia. No se ha probado en la ventana. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.reader |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:666-677,687-690,708,753-756`<br>`io/src/main/java/org/openmarkov/io/probmodel/exception/PGMXParserException.java:63-72`<br>`io/src/main/resources/io/localize/io_class_localizations_en.xml:17` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 60, error 37, error 106 |

**Qué pasa.** Desde el commit `341e1ed` (error 60) el lector de `.pgmx` rechaza un fichero en el que una variable nombra, en su `<Agent>`, un agente que no está en la lista de agentes de la red. El aviso dice qué variable es, pero en una red temporal lo dice sin el ciclo (el instante al que pertenece la variable, que va entre corchetes al final de su nombre): hay una «State [0]» y una «State [1]», y el texto no deja saber cuál de las dos hay que corregir.

Medido con un pequeño programa de prueba: en una copia de `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx` se añadió `<Agent name="Nadie" />` a la variable `State` del ciclo 1. El lector la rechaza con `PGMXParserException.UnknownAgent` y este texto:

```
Variable State has the agent Nadie, which is not in the list of agents of the network.
```

Por qué pasa: `PGMXReader_0_2.loadVariableAdvancedInformation` quita el ciclo del nombre nada más empezar (líneas 687-690), porque lo necesita así para construir la variable, y ese nombre recortado es el que pasa después a `agentOfTheNetwork` (línea 755), que lo mete en la excepción (línea 676). Le pasa lo mismo, por lectura, al aviso de una variable sin estados (`VariableHasNoStates`, línea 708).

Un fichero así sale de escribirlo a mano o con otra herramienta: el programa escribe en cada decisión un agente de la lista.

**Cómo arreglarlo.** Guardar el ciclo antes de recortar el nombre y pasar a las dos excepciones el nombre completo (el nombre base más `[ciclo]` cuando el elemento trae `timeSlice`).

Prueba de regresión: en `AgentsTest`, leer una red temporal con un agente desconocido en la variable de un ciclo distinto de cero y comprobar que `variableName` de la excepción lleva el ciclo.

### Error 235 · [Corregido] Guardar una red de sucesos en el formato 0.2 la deja sin las relaciones de los sucesos y sin las opciones de Monte Carlo, sin avisar, y el fichero ya no se puede abrir

| | |
|---|---|
| **Estado** | Corregido en el commit `208605c` (15-10-2026): antes de guardar, los escritores de `.pgmx` comprueban que el lector de su formato sabe leer el tipo de cada relación del fichero; si no, no guardan nada y el mensaje nombra el nodo, el tipo y el formato. El formato 0.2 rechaza además las redes de sucesos. Si «Save as» se rechaza, la pestaña conserva su fichero y su formato |
| **Módulo** | io; gui |
| **Paquete** | org.openmarkov.io.probmodel.writer; org.openmarkov.gui.dialog.io |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:50,872-899`<br>`gui/src/main/java/org/openmarkov/gui/dialog/io/NetworkOMFileChooser.java:52-58` |
| **Comprobado** | Ejecutándolo por programa; no en la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 113, error 236, error 237 |

**Qué pasa.** Al guardar con «Save as», el selector de ficheros ofrece los dos escritores de `.pgmx`, «OpenMarkov.0.2» y «OpenMarkov.1.0» (`NetworkOMFileChooser`, líneas 52-58, lista todos los escritores). El de 0.2 no sabe escribir las relaciones propias de las redes de sucesos: `PGMXWriter_0_2.getPotentialBody` (líneas 872-899) solo escribe el contenido de las tablas, los árboles, los modelos canónicos, las fórmulas, las regresiones, la delta y la binomial; de cualquier otra relación escribe el tipo y las variables, y nada más. Tampoco escribe las opciones de Monte Carlo ni la casilla «Always append», que solo escribe `PGMXWriter_1_0`.

Medido el 15-10-2026 con un pequeño programa de prueba sobre `io/src/test/resources/DES-illness.pgmx`, la red de sucesos de las pruebas: guardada con `PGMXWriter_0_2`, el fichero sale con `formatVersion="0.2.0"`, sin `<MonteCarlo>`, y con las tres tablas de distribuciones (`Onset`, `Checkup` y `Death`) y la tabla de transición de `Ill` reducidas a la lista de sus variables, sin distribuciones ni valores. No sale ningún aviso. Al abrirlo, el lector termina con «Potential type DistributionTable is not supported.» (`PGMXParserException.PotentialTypeNotSupported`); y aunque aceptara ese tipo, los datos ya no están en el fichero.

La rama `DES-development` de Carmen Yago, de la que viene la simulación de sucesos, escondía ese escritor (con el comentario «FIXME remove; for evaluation jar»); al traer su trabajo no se trajo esa parte.

**Cómo arreglarlo.** Que el escritor 0.2 se niegue, con un mensaje que diga qué nodo y qué relación no sabe guardar, a escribir una red con algo que caiga fuera de `getPotentialBody` o con opciones que no sabe escribir, en vez de escribir el tipo sin contenido. Así quedan cubiertas también las otras relaciones que hoy no escribe (`Indicator`, `PiecewiseExponential`, `Increment`, `External` y los dos árboles con sucesos) y quien lo llame por programa. Dejar de ofrecer el formato 0.2 para las redes de sucesos sería un complemento, no un sustituto.

Prueba: guardar `DES-illness.pgmx` con `PGMXWriter_0_2` da el mensaje y no deja un fichero a medias; con `PGMXWriter_1_0` se guarda y se vuelve a abrir con las mismas relaciones y opciones.

### Error 236 · [Corregido] Al abrir un .pgmx, un error dentro de un árbol con sucesos («Tree with Events» o «Tree with Excluded Events») solo se escribe en la consola: la red se abre sin aviso con ese árbol vacío o a medias

| | |
|---|---|
| **Estado** | Corregido en el commit `cc166da` (15-10-2026): el lector ya no se traga los fallos de estos árboles; un árbol dividido por una variable que no le corresponde se rechaza con un mensaje que nombra el nodo y la variable, y cada árbol y sus ramas se buscan por su nombre. Además, el árbol con sucesos excluidos se lee con sus variables, sin repetir la variable por la que se divide |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.reader |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:436-471,475-508` |
| **Comprobado** | Ejecutándolo por programa; no en la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 111, error 235, error 105 |

**Qué pasa.** `PGMXReader_1_0.getTreeWithEventsPotential` y `getTreeWithExcludedEventsPotential` envuelven toda la lectura del árbol en un `try` que captura cualquier excepción, la escribe en la consola con `printStackTrace` (líneas 467 y 504) y devuelve la relación tal como esté: la que crea el constructor, con un árbol por omisión por cada suceso padre, o un árbol a medio llenar. El resto del lector, desde el arreglo del error 111, rechaza un fichero mal escrito con un mensaje.

Medido el 15-10-2026 con un pequeño programa de prueba: en `io/src/test/resources/DES-illness.pgmx` se puso en `Death` un «Tree with Events» sobre `Death`, `Onset` e `Ill`, se guardó con `PGMXWriter_1_0` y se volvió a abrir bien. Después se abrieron dos copias dañadas de ese fichero:

- con `<TopVariable name="Onsett"/>`, una variable que no existe: en la consola sale un `NullPointerException` de `PGMXReader_0_2.getTreeADDBranches` (línea 1237), y la red se abre con el árbol por omisión en `Death`;
- con «abc» en vez de un número en una rama: en la consola sale un `NumberFormatException`, y la red se abre igual.

En los dos casos el usuario no ve ningún mensaje. Además, según el código, el lector va quitando hijos del elemento por su posición (`removeContent(1)`), así que depende del orden exacto de los elementos; esto no se ha medido. Ninguna red del repositorio tiene estos árboles, así que ninguna prueba recorre este código.

**Cómo arreglarlo.** Quitar los dos `try` y dejar que el fallo salga como una `PGMXParserException` que nombre el nodo, como las demás relaciones; y leer `TopVariable` y `Branches` por su nombre en vez de por su posición.

Prueba: las dos copias dañadas se rechazan con un mensaje que nombra el nodo, y un árbol de cada uno de los dos tipos, guardado con `PGMXWriter_1_0`, se vuelve a abrir igual.

### Error 237 · Los .pgmx que guardaba la versión de la simulación de sucesos de Carmen Yago (formato 0.5.0) no se abren: el programa no conoce esa versión, ni el nombre con que guardaba un tipo de árbol, ni la distribución exponencial de un parámetro incierto

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io; core |
| **Paquete** | org.openmarkov.io.probmodel.reader; org.openmarkov.core.model.network.potential.treeadd; org.openmarkov.core.model.network.modelUncertainty |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/ReaderFactory.java:24-35`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/Version.java:7-13`<br>`core/src/main/resources/version.xsd:11-12`<br>`io/src/test/resources/version.xsd:11-12`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeWithExcludedEventsPotential.java:39`<br>`core/src/main/java/org/openmarkov/core/model/network/modelUncertainty/ExponentialFunction.java:17` |
| **Comprobado** | Ejecutándolo por programa (la versión) y leyendo el código (el nombre del árbol y la exponencial); no hay redes reales de esa versión |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si el programa debe abrir los ficheros de esa versión. Para comprobarlo hacen falta las redes de Carmen Yago (Osteo, Cochlear, TSD-15…), que no están en el repositorio |
| **Relacionados** | error 235, error 236 |

**Qué pasa.** Carmen Yago desarrolló la simulación de sucesos entre 2018 y 2026 en la rama `DES-development` de los antiguos repositorios por módulos, que nunca se fusionó; Jorge Rico trajo su trabajo a mano el 20-05-2026. Su escritor guardaba `formatVersion="0.5.0"`: en su rama se llamaba aún `PGMXWriter_0_5`; en `development` pasó a llamarse 1.0 en 2021 (commit `61e8fbf` del antiguo repositorio `io.probmodelxml`), y la rama nunca recibió ese cambio.

Hoy `Version` solo conoce 0.2 y 1.0, `ReaderFactory.getReader` rechaza cualquier otra con «Version 0.5.0 is not supported.», y los dos `version.xsd` solo admiten 0.2.0 y 1.0.0. Medido el 15-10-2026: `DES-illness.pgmx` con la versión cambiada a 0.5.0 da `ProbNetParserException.WrongVersion`.

Aunque se aceptara la versión, quedan dos diferencias, vistas leyendo el código:

- Su rama guardaba el árbol de sucesos excluidos con el nombre «Tree with Exclued Events», con una errata, que al traerlo se corrigió a «Tree with Excluded Events» (línea 39). El lector busca la clase por ese nombre, así que esos ficheros darían «Potential type Tree with Exclued Events is not supported.».
- Su rama registraba la distribución exponencial poniendo en la clase `ExponentialFunction` la anotación `@ProbDensFunctionType`, con el nombre «Exponential» y el parámetro `lambda`, y al traer la clase no se trajo la anotación. Sin ella, un parámetro incierto con distribución exponencial no se puede leer (`PGMXReader_0_2`, línea 1367, busca la distribución por su nombre), y la exponencial tampoco se ofrece en el diálogo de incertidumbre.

Según el código, el resto de lo que escribía su versión lo entiende el lector 1.0: usa las mismas etiquetas, y las tablas simples que guardaba como `UnivariateDistr` «exact» se traducen hoy igual. No hay en el repositorio ninguna red de sucesos de esa versión; la única 0.5.0, `0_miscellaneous/networks/id/ID-decide-test-0.5.0.pgmx`, no es de sucesos, y `NetsIOTest` la excluye («only for 'Augmented bayesian networks' branch»).

**Cómo arreglarlo.** Si se decide abrirlos: añadir 0.5 a `Version` y enviarlo a `PGMXReader_1_0` en `ReaderFactory`; añadir 0.5.0 a los dos `version.xsd`; añadir «Tree with Exclued Events» como segundo nombre en la anotación de `TreeWithExcludedEventsPotential`; y poner a `Exponential Function` la anotación de su rama, comprobando antes si entonces sale también en la lista de distribuciones de `UnivariateDistr`, que se indexa por nombre, y si debe salir.

Prueba: abrir las redes de Carmen Yago y comparar los resultados de su simulación con los que daba su versión.

### Error 273 · Cuando un escritor de .pgmx se niega a guardar una relación de un nodo temporal, el mensaje nombra el nodo con el instante dos veces: «UnivariateDistribution [0] [0]»

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.writer |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:133-149,1244-1246` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 235 |

**Qué pasa.** Desde `208605c` (error 235), un escritor de `.pgmx` no guarda una relación que el lector de su formato no sabría leer, y lo dice con un mensaje que nombra el nodo. Si el nodo es de un ciclo, el nombre sale con el instante dos veces. Medido el 15-10-2026 guardando `Dynamic-LIMID-every-potential.pgmx` con el escritor de 0.2 de ese commit:

```
The network is not saved: the relation of UnivariateDistribution [0] [0], of type
↪  UnivariateDistr,
cannot be saved in format 0.2.0, which could not read it back. Save the network in another
↪  format.
```

Con un nodo que no es de un ciclo sale bien («the relation of Cost»).

Por qué pasa: `PGMXWriter_0_2.nodeOf` (líneas 133-149) saca el nombre del fichero ya escrito: el atributo `name` de la primera variable de la relación y, si tiene el atributo `timeSlice`, el instante entre corchetes. Pero el escritor ya pone el instante en el nombre: una variable de un ciclo se escribe con `name` igual a su nombre completo, «Health [1]», y además `timeSlice="1"` (líneas 1244-1246).

**Cómo arreglarlo.** En `nodeOf`, tomar el atributo `name` tal cual, sin añadirle el instante.

Prueba: en `WhatAFormatCannotReadBackIsNotSavedTest`, guardar en 0.2 una red con una relación que ese formato no lee en un nodo de un ciclo, y comprobar que el mensaje nombra el nodo una sola vez con su instante.

### Error 274 · [Corregido] Una tabla de fórmulas guardada en formato 0.2 se escribe con el tipo «Table», así que la comprobación de lo que el formato puede leer no la ve, se guarda sin aviso y al abrir el fichero es una tabla de ceros

| | |
|---|---|
| **Estado** | Corregido en el commit `70bc81c` (16-10-2026): `PGMXWriter_0_2` escribe la tabla de fórmulas con su tipo, `AugmentedProbTable`, y la comprobación de `208605c` se niega a guardarla en 0.2 con el mensaje «The network is not saved: the relation of Smoker, of type AugmentedProbTable, cannot be saved in format |

| | |
|---|---|
| | 0.2.0, which could not read it back. Save the network in another format.», sin dejar fichero; en 1.0 se guarda y se vuelve a leer como tabla de fórmulas. El escritor de 1.0 no cambia. Con prueba, en `WhatAFormatCannotReadBackIsNotSavedTest`. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.probmodel.writer |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:848-849,949-950,1095-1107` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 235, error 259 |

**Qué pasa.** Una tabla de fórmulas (`AugmentedProbTablePotential`, la tabla cuyas casillas son fórmulas sobre padres numéricos) guardada con el formato 0.2 se pierde: el fichero se guarda sin ningún aviso, pero al abrirlo la relación es una tabla con incertidumbre y sin números, y las fórmulas no están. Es el mismo error 235 por otro camino: desde `208605c` el escritor no guarda lo que el lector de su formato no sabría leer, pero esta relación se escribe con un nombre de tipo que sí sabe leer.

Medido el 15-10-2026 con los escritores y lectores de `208605c`, con una tabla de fórmulas sobre «Age» con las casillas `1-{Age}/100` y `{Age}/100`:

```
guardada en 0.2 y abierta:  UncertainTablePotential values [0.0, 0.0]; proyectarla con Age
↪  = 30:
                            ArrayIndexOutOfBoundsException: Index -2 out of bounds for
                            ↪  length 2
guardada en 1.0 y abierta:  AugmentedProbTablePotential formulas [1-{Age}/100, {Age}/100]
```

Por qué pasa: `PGMXWriter_0_2` escribe una `AugmentedProbTablePotential` con el tipo `Table` (líneas 848-849, a propósito: `potential instanceof AugmentedProbTablePotential || …`), y dentro sus fórmulas como parámetros inciertos de tipo función (líneas 949-950 y 1095-1107). La comprobación de `208605c` mira el tipo, y `Table` lo lee el lector de 0.2, que lo toma por una tabla con incertidumbre.

**Cómo arreglarlo.** Escribir la tabla de fórmulas con su propio tipo, `AugmentedProbTable`, que el lector de 0.2 no lee: así la comprobación de `208605c` la ve y se niega a guardar en 0.2, como con las demás relaciones que ese formato no sabe leer. Si se quiere que 0.2 la guarde, el lector de 0.2 tendría que leerla.

Prueba: en `WhatAFormatCannotReadBackIsNotSavedTest`, guardar en 0.2 una red con una tabla de fórmulas: no se guarda, y el mensaje nombra el nodo y el tipo `AugmentedProbTable`.

### Error 276 · Un .pgmx cuyo indicador de un suceso no lleva la etiqueta del tiempo hasta el suceso o la de la probabilidad no se abre por un puntero nulo, y el mensaje solo dice que la red está corrompida, sin decir qué nodo ni qué falta

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io, gui |
| **Paquete** | org.openmarkov.io.probmodel.reader; org.openmarkov.gui.dialog.io |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:518-545` |
| **Comprobado** | Leyendo el código y ejecutándolo (el lector, sin ventana); el mensaje de la ventana, leído en el código |

| | |
|---|---|
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 266, error 265 |

**Qué pasa.** Desde `109ee86` (error 266), un indicador con un número fuera de rango no se abre, con un mensaje que dice el nodo y el número. Pero si al indicador le falta una de sus dos etiquetas, `<TTE>` o `<ProbabilityOccurrence>`, el lector muere con un `NullPointerException`. Según el código —no se ha visto en la ventana—, `NetsIO` convierte cualquier `RuntimeException` del lector en `CorruptNetworkFile`, cuyo texto es «Network from … is corrupted and could not be loaded.»: el fichero no se abre, pero el mensaje no dice qué nodo ni qué le falta, que es justo lo que el error 266 quería para los números fuera de rango. El escritor del Java escribe siempre las dos etiquetas, así que solo pasa con un fichero escrito a mano o por otro programa.

Por qué pasa: `PGMXReader_1_0.getIndicatorPotential` (líneas 518-534) busca las dos etiquetas con `getChild`, que contesta `null` cuando una falta, y `numberIn` (líneas 537-545) lee su texto sin mirarlo.

Medido el 15-10-2026 con las clases de `109ee86` delante del jar del 14 de octubre, sobre la red de sucesos de `AnIndicatorOutOfRangeIsNotReadTest` con cada etiqueta quitada:

```
tte          missing   NullPointerException   Cannot invoke "org.jdom2.Element.getText()"
↪  because "<parameter1>" is null
probability  missing   NullPointerException   Cannot invoke "org.jdom2.Element.getText()"
↪  because "<parameter1>" is null
```

Con el jar del 14 de octubre, sin el arreglo, el mismo puntero nulo («`because "xmlTte" is null`»).

**Cómo arreglarlo.** Leer una etiqueta que falta como una escrita vacía: `numberIn` puede devolver NaN si el elemento es nulo, y entonces la regla de `IndicatorPotential` la rechaza con el mensaje de siempre, que nombra el nodo. O un mensaje propio que diga qué etiqueta falta.

Prueba: en `AnIndicatorOutOfRangeIsNotReadTest`, un fichero sin `<TTE>` y otro sin `<ProbabilityOccurrence>`: los dos se rechazan con una `PGMXParserException` que nombra Onset, y no con un `NullPointerException`.

## 12 Bases de casos

Los ficheros de casos (CSV y Weka ARFF) con los que se aprende y se evalúa una red.

### Error 118 · [Corregido] Una base de casos de Weka con un valor desconocido («?») se lee con filas descuadradas y casos inventados, sin ningún aviso

| | |
|---|---|
| **Estado** | Corregido en el commit `739b5f9` (14-10-2026): el lector de ANTLR se sustituye por uno escrito a mano, `ArffReader`, y se borran `ArffLexer`, `ArffParser`, sus restos y `arff.g`. Lee un caso por línea; «?» es un valor desconocido y da a su columna un estado `?`, como el lector de Excel; un valor que su columna no declara, una línea con más o menos valores que columnas o una cabecera que no se entiende hacen que se rechace el fichero con `CouldNotParseSourceException`, cuyo mensaje dice la línea. La ventana de aprender enseña ese rechazo, como el de cualquier base de casos, en el diálogo de error inesperado. Comprobado con las pruebas de `WekaFilesAreReadLineByLineTest`, entre ellas el fichero de ejemplo de Weka `weather.nominal.arff` |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |

| | |
|---|---|
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:181-205`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffParser.java:517-550`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffParser.java:586-596`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:53-70`<br>`io/src/main/java/org/openmarkov/io/database/weka/arff.g:15,242-263` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 119, error 120, error 121, error 122, error 123 |

**Qué pasa.** El formato ARFF (Attribute-Relation File Format, el de la herramienta de aprendizaje automático Weka) marca un valor que no se conoce con una interrogación: `?`. Una base de casos con algún `?` se carga sin ningún error: `ArffDataBaseIO.load` devuelve una base con filas descuadradas y casos que no están en el fichero, la ventana la acepta y se aprende o se evalúa con otros datos. El único aviso sale por la salida de errores.

OpenMarkov lee estos ficheros con `ArffDataBaseIO` (módulo `io`), registrado como formato de base de casos «WekaDB» con la extensión `arff`. El usuario llega a él desde la ventana de aprender una red (botón de abrir la base de casos, que ofrece el filtro «WekaDB») y desde las herramientas de evaluación de redes del módulo `bnEvaluation` (panel `DBOpenerPanel`, que usan la validación cruzada, la evaluación de una red y partir un banco de casos).

El lector está generado con ANTLR 2 (un generador de analizadores sintácticos) a partir de `arff.g`. El analizador léxico (`ArffLexer`) está declarado como filtro (`options {filter=true;}`): todo carácter que no empieza ninguna de sus reglas se descarta en silencio (rama `default` de `nextToken`, que hace `consume(); continue tryAgain;`). La interrogación no está en ninguna regla (`IDENT` solo admite letras, dígitos y unos pocos signos), así que desaparece.

Además, los finales de línea se descartan también (regla `SEPARATOR` con `Token.SKIP`), de modo que el analizador sintáctico no sabe dónde acaba una fila: decide que un caso termina cuando detrás de un valor no viene una coma (`example()`, línea 527).

Con `a,?,e` el analizador recibe `a , , e`: la segunda coma no encaja, `reportError` escribe «line 7:4: unexpected token: ,» en la salida de errores, `recover` se salta elementos y la lectura continúa.

Medido con un pequeño programa de prueba, sobre tres columnas X{a,b}, Y{c,d}, Z{e,f}:

- Fichero: `a,c,e` / `a,?,e` / `b,d,f` / `b,d,e` (cuatro filas).
- Base leída: **cinco casos**, `[0,0,0] [0,0,0] [0,0,0] [1,1,1] [1,1,0]`. La segunda fila del fichero (`a,d,e` sin el «?» sería `[0,1,0]`) no aparece y hay un caso inventado.
- Con el «?» en la última columna (`a,c,e` / `a,c,?` / `b,d,f`): **dos casos**, `[0,0,0] [0,0,0]`; la tercera fila se pierde.

Del mismo mecanismo sale un segundo efecto, también silencioso: en `caseData()` (líneas 586-596), si un valor de la sección de datos no es ninguno de los estados declarados, `getStateIndex` devuelve -1 y el caso se queda con un 0, es decir, con el primer estado de esa columna.

El «?» es además la marca de valor ausente que usa el resto del proyecto: el preprocesado de aprendizaje (`learning.core`, `Discretization`) busca un estado llamado `?` para tratar los valores ausentes, y el lector de CSV (valores separados por comas) trata `?` como cualquier otro valor y lo convierte en un estado de la columna. El lector de Weka es el único que lo pierde.

El commit `c5f5014` añadió a `load` la comprobación de columnas repetidas (`CaseDatabaseReader.withDistinctVariableNames`); no toca el análisis de los datos y no cambia este fallo.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:181-205

```
default:
    if ((LA(1) == '"' || LA(1) == '\'') && (_tokenSet_0.member(LA(2)))) {
        mSTRING(true);
        ...
    } else {
        if (LA(1) == EOF_CHAR) {
            uponEOF();
            _returnToken = makeToken(Token.EOF_TYPE);
        } else {
            consume();
            continue tryAgain;
        }
    }
```

io/src/main/java/org/openmarkov/io/database/weka/ArffParser.java:586-596

```
try {
    Token lcommon = l1 != null ? l1 : l2;
    int stateIndex = fsVariables.get(index).getStateIndex(lcommon.getText());
    if (stateIndex != -1) {
        example[index] = stateIndex;
    }
    index++;
} catch (IndexOutOfBoundsException ignored) {
}
```

**Cómo arreglarlo.** El fichero `arff.g` no corresponde ya al Java generado: declara `package org.openmarkov.learning.io`, llama a `addProbNode` y usa `probNode.properties`, mientras que `ArffParser.java` tiene `addNode` y `setAdditionalProperties`. Regenerar desde la gramática no es directo.

Hay dos caminos: corregir a mano el código generado, o sustituir el analizador de ANTLR por un lector escrito a mano. El formato es de líneas (cabecera `@relation`, `@attribute`, `@data`, y una fila por línea separada por comas), y un lector a mano resuelve a la vez este error, error 119, error 120 y error 121.

Lo que el arreglo debe cumplir:

- Una fila del fichero es un caso: los finales de línea cuentan.
- `?` se reconoce como valor ausente y se trata como en el resto del proyecto: un estado `?` en esa columna, como hace `CSVDataBaseIO`.
- Un valor no declarado en la cabecera no se convierte en el estado 0 en silencio: o se añade como estado, o se rechaza el fichero con un mensaje que lo nombre.
- Un error de análisis no se escribe en la salida de errores para seguir leyendo, sino que llega al usuario como excepción de análisis (`ParsingSourceException.CouldNotParseSourceException`, que `load` ya declara).

Prueba de regresión en `io/src/test/java/org/openmarkov/io/database/` (junto a `RepeatedColumnNamesAreRefusedTest`, que ya usa ficheros ARFF temporales): las cuatro filas del ejemplo dan cuatro casos, con el «?» en la columna que toca; un «?» en la última columna no pierde filas; un valor no declarado no da el estado 0.

### Error 119 · [Corregido] Un fichero de Weka en UTF-8 con acentos o con «ñ» pierde estados y lee todos sus casos con ceros

| | |
|---|---|
| **Estado** | Corregido en el commit `739b5f9` (14-10-2026), con el del error 118: el lector de ANTLR se sustituye por uno escrito a mano, `ArffReader`, y se borran `ArffLexer`, `ArffParser`, sus restos y `arff.g`. Lee el fichero en UTF-8 y, si no lo es, en ISO-8859-1, como estaban escritos los ficheros que el lector anterior sí leía; un nombre o un valor admiten cualquier carácter. `save` escribe en UTF-8 |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:36`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:432-685`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:687-1000`<br>`io/src/main/java/org/openmarkov/io/database/weka/arff.g:36-45` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 118, error 120, error 121, error 122, error 123 |

**Qué pasa.** Un fichero de Weka (formato ARFF, Attribute-Relation File Format, extensión `arff`) guardado en UTF-8, la codificación de texto habitual hoy, cuyos estados llevan tildes o «ñ» se carga sin ningún error, pero con una columna que no tiene sus estados y con todos los casos en el estado 0. Se aprende o se evalúa con esos datos. Se llega desde la ventana de aprender (abrir base de casos, filtro «WekaDB») y desde las herramientas de `bnEvaluation` que abren un banco de casos.

El lector de bases de casos de Weka (`ArffDataBaseIO`) lee el fichero **byte a byte**: el analizador léxico se construye con `new ByteBuffer(in)` (línea 36), que convierte cada byte en un carácter.

Sus reglas de palabras (`IDENT`) y de cadenas entre comillas (`STRING`) enumeran los caracteres aceptados, y entre ellos solo hay estas letras que no son ASCII (el juego básico de 128 caracteres del inglés): `á é í ó ú ü` y sus mayúsculas, escritas en el código como `á`, `é`, `í`, `ó`, `ú`, `ü`, `Á`… `Ü`.

Cada una de esas letras ocupa un byte en Latin-1 (ISO-8859-1) y dos en UTF-8; en UTF-8 ninguno de los dos bytes está en la lista. Como el analizador léxico es de filtro, los bytes que no reconoce se descartan en silencio (ver error 118) y la palabra se corta.

Medido con pequeños programas de prueba:

- UTF-8, `@ATTRIBUTE W {camión,seco}` y datos `camión,a` / `seco,b` / `camión,b`: aviso «expecting RIGHTB, found 'n'» por la salida de errores; la variable W queda con **un solo estado, `cami`**; salen **cinco casos** (en vez de tres), todos con W en el estado 0.
- Latin-1, `camión`: se lee bien.
- Latin-1 y UTF-8, `@ATTRIBUTE W {niño,adulto}`: W queda con el estado `ni`, tres casos en vez de dos, todos a 0. La «ñ» no está en la lista, de modo que falla en cualquier codificación; lo mismo cualquier otra letra fuera de la lista (`ç`, `à`…).

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:35-37

```
public ArffLexer(InputStream in) {
    this(new ByteBuffer(in));
}
```

io/src/main/java/org/openmarkov/io/database/weka/arff.g:36-39 (la misma lista está expandida

en `mIDENT` y `mSTRING` de `ArffLexer.java`)

```
IDENT
    options {testLiterals=true;}
    :   ('a'..'z'|'A'..'Z'|'_'|'$'|'0'..'9')
 ('a'..'z'|'á'|'é'|'í'|'ó'|'ú'|'ü'|'A'..'Z'|'Á'|'É'|'Í'|'Ó'|'Ú'|'Ü'|'_'|'$'|'&'|'-'|'0'..'
 ↪  9'|'%'|'.')*;
```

**Cómo arreglarlo.** Leer el fichero como texto en UTF-8 (un `Reader` con `StandardCharsets.UTF_8`, no bytes sueltos) y no enumerar letras: un nombre es todo lo que no sea uno de los separadores del formato, que son todos ASCII (coma, llaves, comillas, espacio, tabulador, fin de línea, `%` de comentario). Si se reescribe el lector a mano (ver error 118), esto va incluido.

El escritor `ArffDataBaseIO.save` usa `new OutputStreamWriter(stream)` sin codificación explícita; conviene fijarla también a UTF-8 para que lectura y escritura casen en cualquier máquina.

Prueba de regresión en `io/src/test/java/org/openmarkov/io/database/`: un fichero UTF-8 con `camión` y `niño` como estados conserva los nombres completos y asigna a cada caso su estado.

## Error 120 · Un fichero de Weka con una columna numérica, de texto o de fecha no se puede abrir: falla con NullPointerException

| | |
|---|---|
| **Estado** | Presente. Desde el commit `739b5f9` (14-10-2026, error 118) ya no sale `NullPointerException`: el lector nuevo rechaza la columna con un mensaje que la nombra. Falta la decisión de qué hacer con estas columnas |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffParser.java:148-159`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffParser.java:340-470`<br>`io/src/main/java/org/openmarkov/io/database/weka/arff.g:192,233` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué hacer con una columna numérica (tomar como estados los valores vistos, discretizarla, o rechazar el fichero con un mensaje claro) |
| **Relacionados** | error 118, error 119, error 121, error 122, error 123 |

**Qué pasa.** El formato ARFF (Attribute-Relation File Format) de Weka admite cuatro clases de columna: `numeric` (también `real` e `integer`), `string`, `date` y la lista de valores entre llaves (`{a,b,c}`). Un fichero con una columna de cualquiera de las tres primeras clases no se abre: en la ventana de aprender sale una excepción no controlada y la base no se carga.

Es el caso de muchos ficheros ARFF reales: los conjuntos de datos de ejemplo de Weka con variables continuas (por ejemplo, edad o presión) declaran `numeric` o `real`.

La gramática del lector de OpenMarkov (`ArffDataBaseIO`, analizador generado `ArffParser`) solo acepta la lista entre llaves: la regla `attribute` exige detrás del nombre la regla `states`, que empieza por `match(LEFTB)` (la llave de apertura).

Con cualquiera de las otras tres clases, `states()` lanza un error de reconocimiento que se imprime por la salida de errores («line 2:14: expecting LEFTB, found 'numeric'») y se «recupera»; la lista de estados queda vacía. `addNode` (línea 148) solo crea la variable si hay estados (`if (states.length != 0)`), así que llama a `probNet.addNode(null, …)` y el constructor de `Node` falla con `NullPointerException: Cannot invoke "Object.hashCode()" because "variable" is null`. La excepción es de ejecución y no es de las que `load` declara.

Medido con `@ATTRIBUTE N numeric` y `@ATTRIBUTE S string`: el aviso «expecting LEFTB, found 'numeric'» y la `NullPointerException`, con la pila `Node.<init>` ← `ProbNet.addNode` ← `ArffParser.addNode` ← `ArffParser.attribute`.

Las columnas numéricas son además las que la ventana de aprender sabe tratar: su panel de variables ofrece discretizar las variables cuyos estados son números. Con el lector de CSV (valores separados por comas), donde ninguna columna declara nada, los valores vistos se convierten en estados y esa discretización puede aplicarse; con el de Weka el fichero no llega a abrirse.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffParser.java:148-159

```
public Node addNode(LinkedHashMap<String, String> infoNode, State[] states, NodeType
↪  nodeType, String nodeName) {
    Variable fsVariable = null;
    ...
    if (states.length != 0) {
        fsVariable = new Variable(nodeName, states);
    }
    Node node = probNet.addNode(fsVariable, nodeType);
    node.setAdditionalProperties(infoNode);
    return node;
}
```

io/src/main/java/org/openmarkov/io/database/weka/arff.g:192,233

```
n:ATTRIBUTE^ (id1:IDENT! | id2:STRING!) states {
...
states : lb:LEFTB! state! (c:COMMA! state!)* rb:RIGHTB! {};
```

**Cómo arreglarlo.** Hay que decidir qué significa en OpenMarkov una columna `numeric`, `string` o `date`. Opciones:

- (a) Tomar como estados los valores que aparecen en la sección de datos, que es lo que el lector de CSV hace con todas las columnas (`CSVDataBaseIO.load`, que además los ordena alfabéticamente) y deja la columna lista para la discretización de la ventana de aprender.
- (b) Rechazar el fichero con una `ParsingSourceException` que nombre la columna y su clase.

En ningún caso debe salir una `NullPointerException`. En el mismo cambio, `addNode` no debe llamar a `probNet.addNode` con una variable nula.

Si se reescribe el lector a mano (ver error 118), el tratamiento de las cuatro clases entra en la lectura de la cabecera. Conviene aceptar también `real` e `integer`, sinónimos de `numeric` en el formato.

Prueba de regresión: un fichero con una columna `numeric` y otra `string` se abre (o se rechaza con la excepción declarada, según lo que se decida) y nunca lanza `NullPointerException`.

### Error 121 · [Corregido] Un fichero de Weka que escribe `@Relation`, `@Attribute` o `@Data` con mayúsculas y minúsculas mezcladas no se abre

| | |
|---|---|
| **Estado** | Corregido en el commit `739b5f9` (14-10-2026), con el del error 118: el lector de ANTLR se sustituye por uno escrito a mano, `ArffReader`, y se borran `ArffLexer`, `ArffParser`, sus restos y `arff.g`, que reconoce `@relation`, `@attribute` y `@data` en mayúsculas o minúsculas, y rechaza una cabecera que no entiende con la excepción declarada, nombrando la línea |

| | |
|---|---|
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:1020-1097`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:186-194`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:56-59`<br>`io/src/main/java/org/openmarkov/io/database/weka/arff.g:47-51` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 118, error 119, error 120, error 122, error 123 |

**Qué pasa.** En el formato ARFF (Attribute-Relation File Format) de Weka las palabras de cabecera `@relation`, `@attribute` y `@data` no distinguen mayúsculas de minúsculas; en la práctica se ven escritas `@relation`, `@RELATION`, `@Relation`… Un fichero que escriba `@Relation`, `@Attribute` o `@Data` no se abre: en la ventana de aprender o en las herramientas de evaluación de `bnEvaluation` sale una excepción no controlada y la base no se carga.

El analizador léxico del lector de OpenMarkov (`ArffLexer`, generado desde `arff.g`) solo acepta dos formas de cada palabra, todo en minúsculas o todo en mayúsculas: `mRELATION` mira la segunda letra y, si es `r`, exige `"@relation"`; si es `R`, exige `"@RELATION"`.

Con `@Relation`, `match("@RELATION")` falla en la tercera letra y el error se imprime por la salida de errores. El analizador sintáctico abandona la regla `relation()` sin rellenar su resultado, `parser.relation()` devuelve `null`, y `ArffDataBaseIO.load` falla en la línea 59 al hacer `ioNet.get("ProbNet")`. La excepción es de ejecución y `load` no la declara.

Medido con `@Relation r`: «line 1:4: expecting RELATION, found 'lation'» y `NullPointerException: Cannot invoke "java.util.HashMap.get(Object)" because "this.ioNet" is null`. Igual con `@Attribute` o `@Data`.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffLexer.java:1028-1037

```
if ((LA(1) == '@') && (LA(2) == 'r')) {
    match("@relation");
} else if ((LA(1) == '@') && (LA(2) == 'R')) {
    match("@RELATION");
} else {
    throw new NoViableAltForCharException((char) LA(1), getFilename(), getLine(),
    ↪  getColumn());
}
```

io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:56-59

```
try (FileInputStream fileStream = new FileInputStream(file)) {
    ArffParser parser = new ArffParser(new ArffLexer(fileStream));
    ioNet = parser.relation();
    ProbNet probNet = (ProbNet) ioNet.get("ProbNet");
```

**Cómo arreglarlo.** Reconocer las tres palabras sin distinguir mayúsculas: en ANTLR 2 (el generador con que está hecho el analizador), con `caseSensitive=false` limitado a esas reglas; si se reescribe el lector a mano (ver error 118), comparando en minúsculas.

Además, `load` debe comprobar si `parser.relation()` devuelve `null` y lanzar `ParsingSourceException.CouldNotParseSourceException` en vez de dejar que salga una `NullPointerException`;

eso protege también de cualquier otro error de cabecera.

Prueba de regresión: `@Relation`, `@Attribute` y `@Data` en un mismo fichero se leen igual que en minúsculas; una cabecera ilegible produce la excepción declarada.

### Error 122 · [Corregido] Una base de casos con una columna de números, guardada en formato Weka, ya no la puede abrir el propio OpenMarkov

| | |
|---|---|
| **Estado** | Corregido en el commit `e22a9a1` (14-10-2026): `ArffDataBaseIO.save` escribe siempre los estados entre llaves, sin `numeric`; no declara `?` como estado y lo escribe como valor desconocido, que el lector nuevo del error 118 vuelve a convertir en el estado `?`; y pone entre comillas, escapando comillas y barras como Weka, todo nombre o valor con espacios, comas, comillas, llaves o `%`. Comprobado guardando y releyendo bases con estados numerados, con un caso en `?`, con una columna toda en `?` y con esos caracteres. Antes del arreglo, la otra parte revisada no es un fallo: la cuenta de comas con que el escritor evita declarar `?` como estado (líneas 114-129) da `{a,b}` con el `?` al principio, en medio o al final (medido), y solo fallaría con dos estados `?` en una misma variable, que no pueden existir. El nombre de relación con la ruta completa se trata en error 123. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:80-132`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:137-156` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 118, error 119, error 120, error 121 |

**Qué pasa.** Una base de casos que tenga una variable cuyos estados se llaman con números (`0`, `1`, `2`…, cosa habitual), guardada en formato «WekaDB», no la puede volver a abrir el propio OpenMarkov.

Se llega desde las ventanas que guardan bases de casos y ofrecen el formato «WekaDB» en su selector (`DBWriterOMFileChooser`): el generador de bases de casos (`DBGeneratorGUI`, módulo `db Generator`), el preprocesado de datos (`DataPreprocessingDialog`) y partir un banco de casos (`SplitDatasetDialog`), estos dos últimos en `bnEvaluation`.

`ArffDataBaseIO.save` (módulo `io`) escribe la base en formato ARFF (Attribute-Relation File Format, el de la herramienta Weka). Antes de escribir los estados de cada variable mira si todos sus nombres (salvo `?`) son números enteros; si lo son, decide que la columna es numérica y escribe **a la vez** la palabra `numeric` y la lista de valores: `@ATTRIBUTE N numeric {1,2}`. En el formato ARFF una columna es `numeric` o es una lista de valores entre llaves, no las dos cosas.

El lector del propio OpenMarkov rechaza esa línea, porque su gramática exige una llave detrás del nombre (ver error 120): «expecting LEFTB, found ‘numeric’» y `NullPointerException` al crear el nodo.

Medido: una base con N{1,2} y X{a,b}, guardada con `save` y vuelta a abrir con `load`, no se abre.

Un segundo efecto del mismo escritor: si un caso tiene en alguna columna el estado `?` (la marca de valor ausente del proyecto), la sección de datos escribe `?` y el lector descuadra las filas (ver error 118). Medido: tres casos escritos, dos leídos.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:94-112

```
boolean numeric = true;
for (int i = 0; i < states.length; i++) {
    try {
        if (!states[i].getName().equals("?")) {
            Integer.parseInt(states[i].getName());
        }
    } catch (NumberFormatException e) {
        numeric = false;
    }
}
if (numeric) {
    output.write("numeric {");
} else {
    output.write("{");
}
```

**Cómo arreglarlo.** Escribir siempre la lista de valores entre llaves, sin `numeric`: los estados de una variable de OpenMarkov son una lista finita, y eso es lo que representa `{…}`. Aunque el lector llegue a aceptar columnas `numeric` (ver error 120), la línea `numeric {…}` seguiría siendo inválida para cualquier otro programa que lea ARFF, incluido Weka.

El valor ausente debe escribirse y leerse con el mismo convenio en los dos lados: si el escritor no declara `?` como estado y escribe `?` en los datos, el lector tiene que reconocer `?` como ausente (ver error 118).

Prueba de regresión de ida y vuelta en `io/src/test/java/org/openmarkov/io/database/`: una base con una variable de estados `1,2` y otra con un caso en `?`, guardada y vuelta a leer, da las mismas variables, los mismos estados y los mismos casos.

### Error 123 · [Corregido] Un fichero de Weka guardado en una carpeta cuyo nombre lleva acentos, «ñ» o «+» no lo puede volver a abrir OpenMarkov

| | |
|---|---|
| **Estado** | Corregido en dos commits. En el `739b5f9` (14-10-2026), con el del error 118, el lector nuevo, `ArffReader`, acepta cualquier carácter en un nombre entre comillas, así que una base guardada en una carpeta con tilde, eñe o `+` se vuelve a leer. En el `e22a9a1`, con el del error 122, `save` nombra la relación con el nombre del fichero sin extensión, y el fichero ya no lleva la ruta de la carpeta |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:83-85`<br>`io/src/main/java/org/openmarkov/io/database/weka/arff.g:41-45` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 118, error 119, error 120, error 121 |

**Qué pasa.** Una base de casos guardada en formato «WekaDB» en una carpeta cuyo nombre lleva tilde, «ñ», `+` u otros signos no se puede volver a abrir: la lectura termina con `NullPointerException`. Se llega guardando la base desde el generador de bases de casos, el preprocesado de datos o partir un banco de casos, en cualquier carpeta con uno de esos caracteres (en castellano, cualquier carpeta con tilde o eñe).

Es otro defecto del mismo escritor que error 122, esta vez en el nombre de la relación que escribe `ArffDataBaseIO.save`.

La primera línea de un fichero ARFF (Attribute-Relation File Format, el formato de Weka), `@RELATION nombre`, da nombre al conjunto de datos. El escritor usa la **ruta absoluta** del fichero: `@RELATION "/home/usuario/Investigación/casos.arff"`. Además de dejar escrita en el fichero la estructura de carpetas del usuario, eso hace que el nombre dependa de caracteres que el lector no admite.

La regla de cadenas entre comillas del lector (`STRING`) solo acepta letras ASCII (el juego básico de 128 caracteres del inglés), `á é í ó ú ü` en un byte (codificación Latin-1) y una lista corta de signos, entre los que no están `+`, `~`, `#`, `ñ` ni ninguna letra acentuada escrita en UTF-8 (ver error 119).

Medido guardando la misma base de dos variables en tres carpetas:

- `…/plain/casos.arff`: se vuelve a leer (aparte del defecto del `?` de error 118).
- `…/Investigación/casos.arff`: «unexpected token: casos.arff» y `NullPointerException: … because "this.casesAux" is null`.
- `…/a+b/casos.arff`: «unexpected token: “» y la misma `NullPointerException`.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:80-85

```
@Override public void save(File file, CaseDatabase database) throws IOException {
    FileOutputStream stream = new FileOutputStream(file);
    OutputStreamWriter output = new OutputStreamWriter(stream);
    /* Relation name */
    String relationName = file.getAbsolutePath();
    output.write("\n@RELATION \"" + relationName + "\"\n");
```

**Cómo arreglarlo.** Usar como nombre de relación el nombre del fichero sin extensión (o un nombre neutro), escapando comillas si las lleva, y no la ruta absoluta.

Del lado del lector, aceptar en una cadena entre comillas cualquier carácter salvo la comilla de cierre (ver error 119). Cualquiera de las dos cosas por separado ya evita el fallo; juntas hacen que la ida y vuelta no dependa de la carpeta.

Prueba de regresión: guardar una base en una carpeta temporal cuyo nombre lleve tilde, eñe y `+`, y volver a leerla con el mismo número de variables y casos; y comprobar que el fichero no contiene la ruta de la carpeta.

### Error 124 · [Jorge Rico] Una línea de un CSV con campos de menos, de más o en blanco se lee como un caso inventado o tumba la lectura

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.excel |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/excel/CSVDataBaseIO.java:106-165`<br>`io/src/main/java/org/openmarkov/io/database/excel/CSVDataBaseIO.java:233-261` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** Una base de casos en formato CSV (valores separados por comas) con alguna línea que no trae tantos campos como columnas tiene la cabecera, o con una línea en blanco, se lee mal. Si a la línea le faltan campos o está en blanco, el usuario aprende una red (o carga evidencia) con casos que el fichero no dice, sin ningún aviso. Si le sobran campos, o si una columna no la rellena ninguna línea, recibe una excepción genérica que no señala la línea culpable.

El lector, `CSVDataBaseIO`, se da de alta con `@CaseDatabaseFormat(extension = "csv")` y lo usan todos los sitios que abren una base de casos a través de `CaseDatabaseManager`:

- el diálogo de aprendizaje (`LearningDialog.loadCaseFile`, que se abre desde el menú de herramientas de aprendizaje);
- «cargar evidencia» del editor (`NetworkFileHandler.loadEvidence`);
- el generador de bases de casos (`DBGeneratorGUI`);
- los diálogos de evaluación de redes (`DBOpenerPanel`, `SplitDatasetDialog`, `DataPreprocessingDialog`).

El lector toma la primera línea como la lista de nombres de las columnas y cada línea siguiente como un caso. Para cada línea, `getDataLine` reserva una fila de enteros del ancho de la cabecera (un entero por columna, que es el número de estado) y recorre **los campos que traiga la línea**, sin compararlos con el número de columnas. Luego `load` ordena alfabéticamente los estados de cada columna y renumera los casos. Nada avisa de que una línea no tiene la forma esperada.

Las cuatro consecuencias, ejecutadas con un pequeño programa de prueba:

1. **Línea con campos de menos.** Las columnas a las que la línea no llega se quedan con el valor inicial del array, 0. Ese 0 no significa «falta el dato»: es el número del primer estado que se vio en esa columna. Con el fichero `A,B,C / yes,no,yes / no,no / yes,yes,no`, el segundo caso sale `no no yes`: la C que la línea no traía se lee como «yes», porque «yes» fue lo primero que apareció en la columna C. Qué valor se inventa depende del orden de las líneas del fichero.

2. **Línea con campos de más.** `variablesStates.get(numVariable)` se sale de la lista y la lectura muere con `IndexOutOfBoundsException: Index 3 out of bounds for length 3`, sin decir qué línea del fichero es ni cuántos campos traía.

3. **Línea en blanco.** No se salta: cuenta como un caso. Su único campo, la cadena vacía, se da de alta como un estado más de la primera columna, y las demás columnas se rellenan con el 0 del punto 1. Con `A,B / yes,no / (línea vacía) / no,yes` salen tres casos, la variable A queda con tres estados (`""`, `no`, `yes`) y el caso inventado es `"" no`. Basta con que el fichero termine con una línea vacía de más para que ocurra.

4. **Columna que ninguna línea llega a rellenar** (caso extremo del punto 1: todas las líneas traen menos campos que la cabecera). Esa columna se queda sin ningún estado y la renumeración de `load` indexa un array de longitud cero: `ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0` en `CSVDataBaseIO.java:149`.

**El código.** io/src/main/java/org/openmarkov/io/database/excel/CSVDataBaseIO.java:128-131

```
while (scanner.hasNextLine()) {
    data.add(getDataLine(scanner.nextLine(), variablesStates, variablesStatesNames));
    numRows++;
}
```

io/src/main/java/org/openmarkov/io/database/excel/CSVDataBaseIO.java:233-259

```
private static int[] getDataLine(String line, List<Map<String, Integer>> variablesStates,
                                 List<List<String>> variablesStatesNames) {
    int[] statesLines = new int[variablesStates.size()];
    ...
    String[] fields = line.split("[,;]", -1);
    for (int numVariable = 0; numVariable < fields.length; numVariable++) {
        String stateVariable = fields[numVariable];
        ...
        Map<String, Integer> variableStates = variablesStates.get(numVariable);
        Integer stateNumber = variableStates.get(stateVariable);
        if (stateNumber == null) {
            stateNumber = variableStates.size();
            variableStates.put(stateVariable, stateNumber);
            variablesStatesNames.get(numVariable).add(stateVariable);
        }
        statesLines[numVariable] = stateNumber;
    }
    return statesLines;
}
```

**Cómo arreglarlo.** En `load`, antes de llamar a `getDataLine`, decidir qué hacer con cada línea según su número de campos, llevando la cuenta del número de línea del fichero para los mensajes:

- línea vacía (o solo espacios): saltarla sin contarla como caso;
- número de campos distinto del de la cabecera: rechazar el fichero con una `ParsingSourceException` (o la excepción de lectura que ya declara el método) cuyo mensaje diga el número de línea, cuántos campos traía y cuántos esperaba.

Rellenar los campos que faltan con un estado de «valor desconocido» sería otra opción, pero cambia lo que el aprendizaje recibe y conviene no inventarlo sin avisar.

Es el único sitio: los lectores de Excel del mismo paquete (`ExcelDataBaseIO`, `OldExcelDataBaseIO`) recorren las columnas de la cabecera y no los campos de la fila, así que no tienen esta forma del defecto.

Prueba de regresión: ampliar `CSVDataBaseIOEmptyFieldsTest` (`io/src/test/java/org/openmarkov/io/database/excel/`) con cuatro ficheros —línea corta, línea larga, línea en blanco intermedia y línea en blanco final— y comprobar que las líneas en blanco no crean casos ni estados y que las otras dos producen una excepción cuyo mensaje contiene el número de línea.

### Error 218 · En «Generate dataset randomly», elegir «Load Net from file» abre el selector de ficheros sin pedirlo; si se cancela, «Open» queda apagado y «Generate» sigue usando la red abierta

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | dbGenerator |
| **Paquete** | org.openmarkov.dbgenerator.gui |
| **Ficheros** | `dbGenerator/src/main/java/org/openmarkov/dbgenerator/gui/DBGeneratorGUI.java:105-113,157-177,192-206` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 33 |

**Qué pasa.** La ventana «Generate dataset randomly» genera una base de casos a partir de una red, que puede ser la que está abierta o una que se carga de un fichero. Con una red abierta, la ventana se abre con «Use network already open» elegido y el botón «Open» apagado. Al elegir «Load Net from file»:

- se abre de inmediato el selector de ficheros, sin que el usuario haya pulsado «Open»;
- si lo cancela, «Load Net from file» queda elegido, «Open» sigue apagado y «Generate» sigue encendido; pulsarlo genera los casos con la red que estaba abierta, no con una cargada de un fichero. Para volver a intentarlo hay que pulsar otra vez sobre la opción ya elegida.

Por qué pasa: el oyente de «Load Net from file» (líneas 157-177) pide el fichero él mismo cuando todavía no hay ninguno, y solo enciende «Open» si se ha elegido uno. Si se cancela no toca nada: ni «Open», ni la red que la ventana tenía apuntada (la abierta, líneas 105-109).

Comprobado solo leyendo: la ventana no se puede construir sin pantalla.

**Cómo arreglarlo.** Que elegir «Load Net from file» solo encienda «Open» y apague «Generate» hasta que haya una red cargada, y que el selector de ficheros se abra únicamente con «Open».

Prueba: con pantalla, elegir «Load Net from file», cancelar y comprobar que «Open» está encendido y «Generate» apagado.

### Error 246 · Una base de casos leída de un .csv con alguna casilla en blanco y guardada en formato Weka no se puede volver a abrir: el escritor pone el estado vacío entre comillas, como Weka, y el lector rechaza un valor vacío en la cabecera

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:80-83,101-107`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:120-130`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/SplitDatasetDialog.java:305-307` |
| **Comprobado** | Leyendo el código y ejecutándolo; el fichero, abierto también con Weka 3.8.6 |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 122, error 118, error 124 |

**Qué pasa.** Una base de casos en `.csv` con alguna casilla en blanco en medio de una línea (`x,,w`) se lee con un estado vacío, de nombre `""`, en esa columna. Si esa base se guarda en formato Weka —por ejemplo, con Tools → «Split Dataset» dando a los dos ficheros la extensión `.arff`, que elige el escritor por la extensión (`SplitDatasetDialog`, líneas 305-307)—, el fichero que sale no se puede volver a abrir: al cargarlo en la ventana de aprender se rechaza con «Line 4: the attribute «B» has an empty or a repeated value». El commit `e22a9a1` («A case database saved in Weka format can be opened again», arreglo del error 122) no cubre este caso.

Por qué pasa: `ArffDataBaseIO.save` escribe cada estado con `quoted`, que pone entre comillas un texto vacío (líneas 101-107): el estado vacío sale como `''`, que es lo mismo que escribe Weka para un valor vacío. Pero `ArffReader.readAttribute`, al leer la lista de valores de la cabecera, rechaza cualquier valor que quede vacío después de quitarle las comillas (líneas 124-128), sin distinguir `''` de dos comas sin nada entre ellas.

Medido el 15-10-2026 con un pequeño programa que lee con `CSVDataBaseIO`, guarda con `ArffDataBaseIO` y vuelve a leer:

```
blank-middle.csv:  A,B,C / x,,w / y,z,v
  leído:    B con los estados «» y «z»
  guardado: @ATTRIBUTE B {'',z}   ...   x,'',w
  releído:  CouldNotParseSourceException: Line 4: the attribute «B» has an empty or a
  ↪  repeated value
```

Weka 3.8.6 abre ese mismo fichero sin problema: lee la columna con los valores `''` (vacío) y `z`, y el primer caso con el valor vacío.

No se ha ejecutado «Split Dataset» desde la ventana; que guarda en Weka con la extensión `.arff` está leído en el código.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:120-130

```
        List<String> values = new ArrayList<>();
        String list = type.substring(1, type.length() - 1);
        // No values: every case of the column is unknown
        for (String field : list.isBlank() ? List.<String>of() : fields(list, lineNumber))
        ↪ {
            String value = unquote(field);
            if (value.isEmpty() || values.contains(value)) {
                throw new ParseException("Line " + lineNumber + ": the attribute «" + name
                ↪  + "» has an empty or a "
                                         + "repeated value", lineNumber);
            }
            values.add(value);
        }
```

**Cómo arreglarlo.** En `readAttribute`, rechazar solo el valor vacío **sin comillas** (el de dos comas seguidas, `{a,,b}`) y aceptar `''` y `""` como el estado vacío. `readCase` ya lo encuentra después, porque `unquote("''")` da `""`. La regla tiene que ser la misma que la del escritor: todo lo que `save` escribe, `read` lo tiene que leer.

Prueba: leer un `.csv` con una casilla en blanco, guardarlo con `ArffDataBaseIO.save` y volver a leerlo: la columna tiene el estado vacío y los casos son los mismos. Y que `{a,,b}` siga rechazándose.

## Error 247 · Un fichero de Weka con un comentario («%») detrás de la lista de valores de una columna o detrás de un caso ya no se abre, aunque el lector anterior y Weka lo leen

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java: 74-99,108-119,148-176,182-194` |
| **Comprobado** | Leyendo el código y ejecutándolo; el mismo fichero, leído con el lector anterior (el de ANTLR, compilado de 739b5f9^) y con Weka 3.8.6 |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 118, error 121 |

**Qué pasa.** En el formato de Weka, `%` empieza un comentario que llega hasta el final de la línea. Un fichero `.arff` con un comentario detrás de lo que dice una línea, como

```
@attribute X {a,b} % why
```

```
...
a % why
```

se rechaza al cargarlo en la ventana de aprender, con uno de estos dos mensajes:

```
Line 2: the attribute «X» is of type «{a,b} % why»; only attributes that list their values
↪  between braces can be read
Line 4: the value «a % why» is not one of those of the attribute «X»: [a, b]
```

Es una regresión del commit `739b5f9` (arreglo del error 118): el lector anterior, el de ANTLR, abría esos ficheros, porque su regla `REMARK` empezaba en cualquier `%` que abriera una palabra y tiraba hasta el final de la línea. Weka 3.8.6 también los abre.

Por qué pasa: `ArffReader.readLines` solo reconoce un comentario **al principio de la línea** (`line.startsWith("%")`, línea 79). Detrás de algo, el `%` y lo que le sigue quedan pegados a lo anterior: en la cabecera, el tipo deja de acabar en `}` y `readAttribute` lo rechaza (líneas 115-119); en un caso, `fields` lo deja dentro del último valor (líneas 182-194), y `readCase` no lo encuentra entre los declarados (líneas 168-173).

Medido el 15-10-2026 sobre dos ficheros de dos casos, uno con el comentario en la línea de `@attribute` y otro en la de un caso:

```
lector de hoy:              Line 2: ... is of type «{a,b} % why» ...  /  Line 4: the value
↪  «a % why» ...
lector anterior (ANTLR):    los dos se leen: X con los estados a y b, dos casos
Weka 3.8.6:                 los dos se leen igual
```

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:76-85

```
        for (int index = 0; index < lines.size(); index++) {
            int lineNumber = index + 1;
            String line = lines.get(index).strip();
            if (line.isEmpty() || line.startsWith("%")) {
                continue;
            }
            if (inData) {
                readCase(line, lineNumber);
                continue;
            }
```

**Cómo arreglarlo.** Quitar el comentario de cada línea antes de interpretarla: buscar el primer `%` que esté fuera de unas comillas y cortar ahí. Hace falta decidir qué pasa con un `%` pegado a una palabra (`Rate50%`): hoy forma parte del nombre, y Weka lo trata como el principio de un comentario (medido: rechaza `@attribute Rate50% {low%,high}` por «premature end of line»); lo más prudente es tomarlo como comentario solo cuando va detrás de un blanco o de la llave que cierra los valores, para no cambiar cómo se leen los nombres que hoy se leen. El escritor ya pone entre comillas todo valor con `%`, así que lo que OpenMarkov escribe no cambia.

Prueba: leer un fichero con un comentario detrás de `@relation`, de un `@attribute` y de un caso, y comprobar que da las mismas columnas y los mismos casos que sin los comentarios; y que un `%` entre comillas (`'50%'`) siga siendo parte del valor.

### Error 248 · Un valor que Weka escribe escapado con una barra inversa, por llevar «%», comillas dobles, un tabulador o un final de línea, se lee con la barra y la letra en el nombre: «`50\% \"rate\"`» en vez de «`50% "rate"`»

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:196-228`<br>`io/src/main/java/org/openmarkov/io/database/weka/ArffDataBaseIO.java:97-107` |
| **Comprobado** | Leyendo el código y ejecutándolo; lo que escribe Weka, con weka.core.Utils.quote de Weka 3.8.6 |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 118, error 122 |

**Qué pasa.** Weka pone entre comillas y escapa con una barra inversa, además de las comillas simples y de la propia barra, las comillas dobles, `%`, el tabulador, los finales de línea (`\n`, `\r`) y el carácter U+001E (`weka.core.Utils.backQuoteChars`). Al guardar un `.arff`, para el valor `50% "rate"` escribe `'50\% \"rate\"'`, y para `a` + tabulador + `b`, `'a\tb'`.

El lector de OpenMarkov solo deshace `\'` y `\\`; lo demás se queda tal cual. Un fichero guardado por Weka con alguno de esos caracteres en un nombre o en un valor se abre sin ningún aviso, con todos sus casos, pero con otros nombres: `50\% \"rate\"` y `a\tb` (una barra y una te) en lugar de `50% "rate"` y un tabulador. La red que se aprenda de él tendrá esos estados, y no casará con otra base, o con una red modelo, que use los nombres de verdad.

El mensaje del commit `e22a9a1` dice que los nombres se escapan «as Weka does» y que «the reader undoes the escapes»; el escritor de OpenMarkov, en cambio, solo escapa `\` y `'` (`ArffDataBaseIO.quoted`, líneas 101-107), así que lo que él escribe sí lo lee bien, y Weka lo lee también. El fallo está solo en leer lo que escribe Weka.

Por qué pasa: `ArffReader.unquote` solo trata como escape la barra seguida de la comilla que abre el valor o de otra barra (líneas 220-224); `closingQuote` aplica la misma regla para encontrar la comilla de cierre (líneas 201-203).

Medido el 15-10-2026 con `weka.core.Utils.quote` de Weka 3.8.6 y con el lector del programa:

```
Weka escribe:           '50\% \"rate\"'   'a\tb'   c
OpenMarkov lee:         variable X   50\% \"rate\"   a\tb   c
Weka 3.8.6 lee:         [50% "rate"]  [a<tabulador>b]  [c]
```

El lector anterior, el de ANTLR, tampoco los leía bien (con el mismo fichero se descolocaba y daba cinco casos), así que no es una regresión del commit `739b5f9`.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:216-227

```
        char quote = field.charAt(0);
        StringBuilder text = new StringBuilder();
        for (int position = 1; position < field.length() - 1; position++) {
            char character = field.charAt(position);
            if (character == '\\' && position + 2 < field.length()
                && (field.charAt(position + 1) == quote || field.charAt(position + 1) ==
                ↪  '\\')) {
                position++;
```

```
                character = field.charAt(position);
            }
            text.append(character);
        }
        return text.toString();
```

**Cómo arreglarlo.** Deshacer en `unquote` los mismos escapes que escribe Weka, que son los que deshace su `Utils.unbackQuoteChars`: `\\`, `\'`, `\"`, `\%`, `\t`, `\n`, `\r` y `\u001E`. Conviene, de paso, que el escritor escape lo mismo que Weka, para que lo que escribe OpenMarkov y lo que escribe Weka sea igual byte a byte.

Prueba: leer un fichero con la cabecera `@attribute X {'50\% \"rate\"','a\tb',c}` y esos tres valores en los casos, y comprobar que los estados son `50% "rate"`, `a` + tabulador + `b`, y `c`.

### Error 249 · En un fichero de Weka, un valor «'?'» entre comillas que su columna no declara hace que se rechace el fichero, salvo que antes, en esa columna, haya aparecido un «?» sin comillas: entonces se acepta y se lee como valor desconocido

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:148-176,178-181` |
| **Comprobado** | Leyendo el código y ejecutándolo; los dos ficheros, abiertos también con Weka 3.8.6 |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 118 |

**Qué pasa.** El lector de Weka (desde el commit `739b5f9`, arreglo del error 118) toma por valor desconocido una `?` **sin comillas** y le da a su columna un estado `?`. Una `'?'` entre comillas, en cambio, es un valor que se llama así, y la columna tiene que declararlo: el comentario de `fields` dice que las comillas se guardan precisamente «so that a quoted ? is not taken for an unknown value». Pero esa regla solo se cumple si la columna no ha visto antes una `?` sin comillas: esa `?` ya ha creado el estado `?`, y una `'?'` posterior lo encuentra y se acepta como ese mismo valor desconocido. El mismo valor en la misma columna hace que el fichero se rechace o se abra según el orden de las líneas:

```
a,'?',e                    → Line 7: the value «?» is not one of those of the attribute
↪ «Y»: [c, d]
a,?,e  y después  a,'?',e   → se abre: Y con los estados c, d, ?, y los dos casos en ?
```

Weka 3.8.6 rechaza los dos ficheros («nominal value not declared in header»), que es lo que dice la regla.

Por qué pasa: `ArffReader.readCase` trata aparte la `?` sin comillas (líneas 161-167), que añade a la columna el estado `?` si no lo tiene; para cualquier otro valor quita las comillas y lo busca entre los estados de la columna (líneas 168-173), sin distinguir los que declaró la cabecera del `?` que añadió una línea anterior.

Medido el 15-10-2026 con dos ficheros de tres columnas (`X {a,b}`, `Y {c,d}`, `Z {e,f}`): uno con la línea `a,'?',e`, y otro con `a,?,e` seguida de `a,'?',e`. Los resultados son los de arriba.

**El código.** io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:158-171

```
        for (int column = 0; column < example.length; column++) {
            String field = fields.get(column);
            List<String> values = states.get(column);
            if (field.equals(MISSING)) {
                if (!values.contains(MISSING)) {
                    values.add(MISSING);
                }
                example[column] = values.indexOf(MISSING);
                continue;
            }
            String value = unquote(field);
            example[column] = values.indexOf(value);
            if (example[column] == -1) {
                throw new ParseException("Line " + lineNumber + ": the value «" + value +
                ↪  "» is not one of those of the "
```

**Cómo arreglarlo.** Al buscar un valor entre comillas, buscarlo solo entre los que declaró la cabecera, no entre los estados que el lector ha ido añadiendo (hoy, solo el `?` de los valores desconocidos): por ejemplo, guardando aparte cuántos valores declaró cada columna, o anotando si el `?` lo trajo la cabecera o un caso.

Prueba: un fichero con `a,?,e` seguida de `a,'?',e`, en una columna que no declara `?`, se rechaza en la segunda línea, como el que solo trae `a,'?',e`; y uno cuya columna declara `'?'` se sigue leyendo.

### Error 250 · Un fichero de Weka guardado en UTF-8 con la marca de orden de bytes, que el lector anterior abría, se rechaza con un mensaje que dice haber encontrado «@relation r», que es justo lo que espera: el carácter que sobra no se ve

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.database.weka; org.openmarkov.io.database.excel |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/database/weka/ArffReader.java:63-72,74-99`<br>`io/src/main/java/org/openmarkov/io/database/excel/CSVDataBaseIO.java:110-116,216-219` |
| **Comprobado** | Leyendo el código y ejecutándolo; el mismo fichero, leído con el lector anterior (el de ANTLR, compilado de 739b5f9^) y con Weka 3.8.6 |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | Sí: si la marca se acepta, como hacía el lector anterior, o se rechaza, como hace Weka, y en ese caso con un mensaje que la nombre |
| **Relacionados** | error 119, error 118 |

**Qué pasa.** Algunos programas, como versiones antiguas del Bloc de notas de Windows, empiezan los ficheros en UTF-8 con tres bytes que no son texto, `EF BB BF`: la «marca de orden de bytes» (BOM, *byte order mark*). Un `.arff` así se abría hasta el commit `739b5f9` (arreglo de los errores 118 y 119), y ahora se rechaza al cargarlo en la ventana de aprender con:

```
Line 1: expected @relation, @attribute or @data, found «\uFEFF@relation r»
```

(donde `\uFEFF` está por un carácter invisible, que en el mensaje no se ve). El mensaje es desconcertante: lo que dice haber encontrado parece justo lo que esperaba.

Por qué pasa: `ArffReader.text` decodifica el fichero como UTF-8 (líneas 63-72), y los tres bytes se convierten en el carácter invisible U+FEFF, que `strip` no quita (línea 78). La primera palabra de la primera línea pasa a ser `\uFEFF@relation`, que no es ninguna de las tres palabras clave (líneas

86-97). El lector anterior, el de ANTLR, abría estos ficheros porque tiraba sin decir nada todo carácter que no conocía, y esos tres también.

Medido el 15-10-2026 con un fichero de una columna `X {a,b}` y un caso, precedido de los tres bytes:

```
lector de hoy:              Line 1: expected @relation, @attribute or @data, found
↪  «\uFEFF@relation r»
lector anterior (ANTLR):    se lee: X con los estados a y b, un caso
Weka 3.8.6:                 Unable to determine structure as arff (Reason:
↪  java.io.IOException:
                            keyword @relation expected, read Token[\uFEFF@relation], line
                            ↪  1).
```

Weka tampoco lo abre, y el formato no dice nada de esa marca; por eso hace falta decidir. Pero es un fichero que OpenMarkov abría y ya no abre.

**Cómo arreglarlo.** Si se decide aceptarla: quitar un U+FEFF al principio del texto en `ArffReader.text`, después de decodificarlo. Si se decide rechazarla: comprobarlo ahí mismo y rechazar el fichero con un mensaje que diga que empieza con la marca de orden de bytes.

La misma regla conviene en los demás lectores de texto. El de `.csv` tampoco quita la marca: medido el 15-10-2026, `CSVDataBaseIO.load` lee `EF BB BF` + `A,B` como dos columnas (líneas 110-116 y 216-219), y la primera se llama U+FEFF seguido de `A`, un nombre que se ve como «A» pero no es igual a «A» (por ejemplo, el de la variable de una red modelo).

Prueba: leer un `.arff` que empieza con `EF BB BF` y comprobar lo que se decida: que da las mismas columnas y casos que sin la marca, o que el mensaje del rechazo nombra la marca.

# 13 Aprendizaje y evaluación de redes

El aprendizaje de estructura y parámetros (PC, maximización de la esperanza, clasificadores) y las herramientas que dividen, discretizan y evalúan bases de casos.

### Error 125 · [Corregido] La exactitud del Bayes ingenuo selectivo se calcula con el caso evaluado dentro de los datos de entrenamiento, y con muchas variables predice siempre la primera clase

| | |
|---|---|
| **Estado** | Corregido en el commit `511dd10` (05-10-2026): `predictClassValue` recibe la iteración del caso y usa solo su conjunto de entrenamiento, del que toma también la frecuencia y la probabilidad a priori de cada clase, y suma logaritmos en vez de multiplicar. De las dos pruebas de regresión propuestas abajo, la de la característica que identifica cada caso ya pasaba antes del arreglo: el conjunto de entrenamiento de la propia ventana, que no contiene el caso, anulaba el producto de todas las clases. El suavizado y el solapamiento de las ventanas siguen como estaban (error 126 y error 131). |
| **Módulo** | learning.metric |
| **Paquete** | org.openmarkov.learning.metric.cmi.accuracy |
| **Ficheros** | `learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:130-142`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:172-185`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/util/MetricUtils.java:95-107` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |

| | |
|---|---|
| **Decisión de equipo** | No |
| **Relacionados** | error 126, error 128, error 131, error 135 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Selective naive bayes» (hacia delante o hacia atrás). La exactitud (proporción de casos cuya clase se acierta) es la que decide qué característica se añade o se quita, y se calcula mal: el clasificador se evalúa con casos que ha visto al entrenar, y con muchas características predice siempre la primera clase.

En modo Bayes ingenuo simple, el que usa «Selective naive bayes», `Accuracy.computeNBNet` `Accuracy` recorre las diez ventanas de prueba y para cada caso llama a `predictClassValue(row, variables)`. Ese método no recibe la iteración a la que pertenece el caso: para cada característica multiplica un factor por **cada uno de los diez conjuntos de entrenamiento** (`crossTabs.get(variable)[it]` con `it` de 0 a 9).

El caso evaluado solo está excluido del entrenamiento de su propia ventana, así que está dentro de los demás. Es justo lo que la división en pliegues pretendía evitar.

Además, el producto no es una probabilidad. Cada factor es una cuenta de un conjunto de entrenamiento (unos nueve décimos de los casos) dividida por la frecuencia de la clase en **toda** la base, y se multiplican diez por característica, con lo que las características pesan como si estuvieran elevadas a la décima potencia frente a la probabilidad a priori de la clase.

Y no se trabaja con logaritmos: con muchas características el producto baja por debajo del menor `double` positivo para todas las clases, y `MetricUtils.getIndexOfMaxValue`, que empieza con máximo 0,0 y exige «mayor que», devuelve siempre el índice 0.

Medido:

- Sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv`: de las 10 000 evaluaciones, 2 073 casos están en 7 de los 10 conjuntos de entrenamiento, 4 618 en 8 y 3 309 en 9 (por el solapamiento de las ventanas que describe error 131).
- Con una base sintética de 2 000 casos, clase binaria equilibrada y características binarias que dependen de la clase (probabilidad 0,7 frente a 0,3): con 20 características la exactitud calculada es 0,9625; con 60, 0,9995; con 100 baja a 0,902; con 150 predice la clase 0 en los 2 000 casos y la exactitud es 0,4825 (la proporción de la clase 0).
- Sobre `BN-asia10k.csv`, con sus 7 características, el efecto es pequeño: la exactitud calculada es 0,843 frente a 0,8445 de una evaluación correcta con las mismas ventanas, y difieren 49 predicciones de 10 000.

La otra rama de la clase, `predictClassValueAugmentedNB` (la del superpadre), usa solo el conjunto de entrenamiento de la iteración que corresponde (`it`), así que no tiene este fallo (tiene otros: ver error 127 y error 128).

**El código.** learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:172-185

```
public int predictClassValue(int[] row, Collection<Variable> variables){
    double[] probs = getProbRootNode();
    for (String variable : variables.stream().map(Variable::getName).toList()) {
        int index = getIndexVariable(variable);
        for (int it=0; it<KFOLD; it++){
            for(int j=0; j<freqRootNode.length;j++){
                probs[j]*= ((crossTabs.get(variable)[it][row[index]][j] +
↪   alpha)/(freqRootNode[j] + alpha* variables.size()));
            }
        }
    }
```

```
    return getIndexOfMaxValue(probs);
}
```

**Cómo arreglarlo.** Pasar a `predictClassValue` la iteración del caso (como ya hace `predictClassValueAugmentedNB`) y usar solo `crossTabs.get(variable)[it]`. Dividir por la frecuencia de la clase en ese mismo conjunto de entrenamiento, y tomar la probabilidad a priori también de ese conjunto (hoy `getProbRootNode` usa toda la base).

Sumar logaritmos en vez de multiplicar probabilidades, o normalizar tras cada factor, para que el producto no se anule.

El suavizado usa `alpha * variables.size()` en el denominador; lo habitual es `alpha` por el número de estados de la característica, pero eso es otra cuestión (y hoy `alpha` vale 0 en esta rama: ver error 126).

Prueba de regresión: con la base sintética de 150 características descrita arriba, la exactitud debe ser próxima a la de un Bayes ingenuo evaluado por separado, no la proporción de la clase 0; y con una característica que identifica cada caso (un estado distinto por caso), la exactitud no debe superar la de adivinar.

### Error 126 · [Corregido] El Bayes ingenuo selectivo elige las características sin el suavizado que el usuario pone en el diálogo: la métrica trabaja siempre con alfa 0

| | |
|---|---|
| **Estado** | Corregido en el commit `d426114` (05-10-2026): `SelectiveNBAlgorithm.init` pasa el alfa a la métrica, y `predictClassValue` lo suma en el denominador una vez por estado de la característica, no una vez por característica. La prueba de regresión usa una base sintética (cinco características de diez estados, 150 casos) en vez de `Cataract-RRDdatasetPreProc3.csv`, que no está al alcance de las pruebas de `learning.algorithm`: hacia atrás conserva tres características con alfa 0 y cuatro con alfa 0,5. Las medidas de abajo son anteriores a este arreglo y al del error 125, y no se han repetido. La rama del superpadre conserva el denominador antiguo (errores 127 a 130). |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.snb |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/snb/SelectiveNBAlgorithm.java:40-50`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:42`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:172-185`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:59-65`<br>`learning.gui/src/main/java/org/openmarkov/learning/algorithm/nbderived/snb/gui/SelectiveNBParametersDialog.java:45-51` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 125, error 128, error 135 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Selective naive bayes» → opciones, donde el diálogo ofrece «Alpha parameter (Laplace-like correction)» con 0,5 por omisión, y también por la validación cruzada de `bnEvaluation`. Ese valor no interviene en qué características conserva o quita el algoritmo: se eligen con alfa 0. El alfa solo se aplica después, al calcular las tablas de la red resultante.

El alfa es el suavizado de Laplace: se suma a cada cuenta para que una combinación de valores que no aparece en los datos no dé probabilidad cero.

«Selective naive bayes» elige las características con la métrica `Accuracy` (exactitud: proporción de casos cuya clase se acierta), que tiene su propio campo `alpha`, inicializado a 0 y que solo cambia con `setAlpha`. `SelectiveNBParametersDialog.getInstance` pasa el alfa al constructor del algoritmo, que lo guarda en `LearningAlgorithm.alpha` para el aprendizaje de parámetros. `SelectiveNBAlgorithm.init` le pasa a la métrica la variable clase, pero no el alfa. `SuperParentNBAlgorithm.init`, que usa la misma métrica, sí llama a `setAlpha(alpha)`.

Con alfa 0, en `Accuracy.predictClassValue` cada factor es cuenta / frecuencia de la clase: un valor de una característica que no aparece con una clase en uno de los diez conjuntos de entrenamiento anula esa clase para el caso. Eso cambia qué características parecen útiles.

Medido ejecutando «Selective naive bayes» hacia atrás (quitando características desde el Bayes ingenuo completo; hacia delante el resultado no es repetible, ver error 135), con alfa 0,5 en el algoritmo, primero como hoy y después llamando también a `setAlpha(0,5)` en la métrica:

- `core/src/test/resources/nets/Cataract-RRDdatasetPreProc3.csv` (537 casos), clase `success`: hoy quita las seis características y deja la clase sola; con el alfa en la métrica quita solo `loc57` y conserva `breakType`, `loc48`, `loc48rrd`, `loc57rrd` y `opType`.
- `integrationTests/src/main/resources/networks/learning/alarm500.csv` (500 casos, 36 características): con clase `HYPOVOLEMIA` hoy conserva 8 y con el alfa en la métrica 28; con clase `INTUBATION`, 7 frente a 12; con clase `BP`, 21 frente a 29.
- `learning.algorithm/src/test/resources/network/BN-asia10k.csv` (clase `Bronchitis`), `integrationTests/src/main/resources/networks/learning/learnTestDataBase.csv` (clase `A`) y `learning.algorithm/src/test/resources/network/HeadToHead2.csv` (clase `C`): el mismo resultado en los dos casos.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/snb/SelectiveNBAlgorithm.java:40-50

```
@Override public void init(ModelNetUse modelNetUse) {
    if (metric instanceof Accuracy) {
        ((Accuracy) metric).setClassVariable(this.classVariableName);
    }
    if (!forward) {
        setRelationsForRootVariable();
    }
    ...
}
```

learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:59-65

```
@Override public void init(ModelNetUse modelNetUse) {
    if (metric instanceof Accuracy) {
        ((Accuracy) metric).setClassVariable(this.classVariableName);
        ((Accuracy) metric).setAugmentedNet(true);
        ((Accuracy) metric).setAlpha(alpha);
    }
```

**Cómo arreglarlo.** Llamar a `((Accuracy) metric).setAlpha(alpha)` en `SelectiveNBAlgorithm.init`, como ya hace `SuperParentNBAlgorithm.init`.

Al activarse el suavizado empieza a contar la forma del denominador de `predictClassValue`, `freqRootNode[j] + alpha * variables.size()`, que multiplica el alfa por el número de características

en vez de por el número de estados de la característica (ver error 125); conviene revisarlo a la vez.

Los únicos algoritmos que usan `Accuracy` son estos dos, y en `SuperParentNBAlgorithm` la regla ya se cumple.

Prueba de regresión: tras `init`, el alfa de la métrica debe ser el del algoritmo; y con `CataractRRDdatasetPreProc3.csv`, clase `success`, hacia atrás, el modelo aprendido con alfa 0 y con alfa 0,5 no debe ser el mismo (hoy, con los dos, la clase sola).

### Error 127 · [Corregido] El clasificador de superpadre pesa sus enlaces con cuentas leídas en casillas equivocadas de la tabla de tres variables

| | |
|---|---|
| **Estado** | Corregido en el commit `935dbd1` (05-10-2026): `build2ndLevelProbDistribution` lee cada casilla en la posición en que la guarda `buildAbsoluteFreqCrossTab`. La prueba de regresión, con variables de 3, 4 y 2 estados, compara todas las casillas de las diez iteraciones con las cuentas directas. Cómo se usan después esas cuentas sigue como estaba (errores 128 a 130). |
| **Módulo** | learning.metric |
| **Paquete** | org.openmarkov.learning.metric.cmi.accuracy |
| **Ficheros** | `learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:292-310`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:193-222`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/util/MetricUtils.java:173-211` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 128, error 129, error 130, error 131, error 132, error 133, nota 8 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Superparent naive bayes» (con la casilla del mismo superpadre marcada o no), y también desde la validación cruzada de `bnEvaluation`. El algoritmo elige el superpadre y los enlaces que cuelgan de él con probabilidades que no corresponden a los datos.

En modo «red aumentada» (el que activa `SuperParentNBAlgorithm.init` con `setAugmentedNet(true)`), la métrica `Accuracy` necesita, para cada pareja de características (n1, n2), cuántos casos de entrenamiento tienen cada combinación (valor de n1, valor de n2, valor de la clase).

`build2ndLevelProbDistribution` pide esas cuentas a `MetricUtils.buildAbsoluteFreqCrossTab`, que devuelve un `TablePotential` con variables (n1, clase, n2), rellenado según la convención de OpenMarkov: la primera variable cambia en el paso más corto, es decir, casilla = x1 + s1 · (c + sc · x2) (s = número de estados).

Después lo reparte en `crossTab[it][i][j][k]` cortando el array como si la casilla fuera k + sc · (j + s2 · i): n1 en el paso más largo, n2 en medio y la clase en el más corto. Las dimensiones cuadran, así que no hay error de índice, pero cada `crossTab[it][i][j][k]` contiene la cuenta de otra combinación.

Esas cuentas son las que `getProb(tail, head, row, it)` usa para estimar P(tail | head, clase), y con ellas `predictClassValueAugmentedNB` calcula la exactitud con la que `SuperParentNBAlgorithm` elige el superpadre y sus enlaces.

Medido sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv`, primer conjunto de entrenamiento, n1 = `Dyspnea`, n2 = `Smoker`, clase = `Bronchitis`, comparando lo leído con las cuentas hechas directamente sobre los casos: 6 de las 8 casillas no coinciden. Por ejemplo,

(0,0,1) lee 340 y son 248; (0,1,0) lee 248 y son 1 489; (1,0,0) lee 1 489 y son 340. Solo coinciden (0,0,0) y (1,1,1).

`predictClassValueAugmentedNB`, el método que consume estas cuentas, tiene además otro defecto, descrito en error 128.

**El código.** learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:292-310

```
protected double[][][][] build2ndLevelProbDistribution(Node n1, Node n2){
    int rootStates = getRootNode().getVariable().getNumStates();
    int fstStates = n1.getVariable().getNumStates();
    int sndStates = n2.getVariable().getNumStates();
    double[][][][] crossTab = new double[KFOLD][fstStates][sndStates][rootStates];
    for(int it=0; it<KFOLD; it++){
        TablePotential potential = buildAbsoluteFreqCrossTab(probNet, caseDatabase, n1, n2,
↪   getRootNode(),  dataset.getTraining()[it]);
        for(int i=0; i<fstStates;i++){
            double[][] tmp = new double[sndStates][rootStates];
            for(int j=0; j<sndStates; j++){
                tmp[j] = Arrays.copyOfRange(potential.getValues(), (i * sndStates + j) *
↪   rootStates, (i * sndStates + j + 1) * rootStates);
            }
           crossTab[it][i] = tmp;
        }
    }
    ...
```

**Cómo arreglarlo.** Leer cada casilla con el orden real de la tabla: `crossTab[it][i][j][k] = values[i + fstStates * (k + rootStates * j)]`, o pedir la tabla con las variables en el orden que el corte espera.

La otra tabla de la clase, `buildProbDistributionForNode`, sí respeta el orden de `getAbsolute Frequencies` y no necesita cambio.

Conviene hacerlo junto con error 128, que corrige cómo `predictClassValueAugmentedNB` usa estas cuentas.

Prueba de regresión: con variables de números de estados distintos (por ejemplo 3, 2 y 4), comprobar que `crossTab[0][i][j][k]` coincide con las cuentas directas sobre `dataset.getTraining()[0]` en todas las casillas.

### Error 128 · [Corregido] El clasificador de superpadre mide cada enlace penalizando solo la clase verdadera del caso, así que ningún enlace puede mejorar la exactitud y elige enlaces que la empeoran

| | |
|---|---|
| **Estado** | Corregido en el commit `5a850f9` (05-10-2026): `getProb` recibe la clase como parámetro y `predictClassValueAugmentedNB` multiplica cada clase por el factor del enlace calculado con sus cuentas. La prueba de regresión no usa `BN-asia10k.csv`, donde la mejora es de milésimas y depende de los errores 129 y 130, sino una base sintética en la que la clase vale 1 cuando dos características binarias difieren: con el enlace entre ellas la exactitud pasa de 0,95 (antes del arreglo, 0,4025). Con «Unique superparent» marcada sigue sin aprenderse ningún enlace (error 132). |
| **Módulo** | learning.metric |
| **Paquete** | org.openmarkov.learning.metric.cmi.accuracy |
| **Ficheros** | `learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/a` |

| | |
|---|---|
| | `ccuracy/Accuracy.java:193-222` `learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:150-163` `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:84-142` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Tapado por** | error 132: con «Unique superparent» marcada, que es la opción de fábrica, no se aprende ningún enlace. |
| **Relacionados** | error 125, error 126, error 127, error 129, error 130, error 131, error 132, error 133 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Superparent naive bayes», y también por la validación cruzada de `bnEvaluation`. Con la opción «Unique superparent» marcada, que es la de fábrica, hoy no se aprende ningún enlace por otro motivo (error 132); desmarcándola, el algoritmo añade enlaces que, según su propia medida, empeoran el clasificador, y no puede encontrar ninguno que lo mejore.

El clasificador de superpadre parte del Bayes ingenuo y cuelga enlaces entre características de una característica elegida («superpadre»). Debe quedarse con un enlace solo si mejora la exactitud (la proporción de casos cuya clase se acierta). Esa exactitud la calcula la métrica `Accuracy` en modo «red aumentada» con `computeAugmentedNetAccuracy`, que para cada caso de prueba llama a `predictClassValueAugmentedNB`.

`predictClassValueAugmentedNB` calcula para cada clase c la probabilidad a priori por el producto de P(característica | c), y después, para cada enlace, multiplica **solo la casilla de la clase verdadera del caso** (`probs[row[getClassVariableIndex()]]`) por `getProb`, que además lee la cuenta de esa misma clase verdadera. Lo correcto es multiplicar cada clase c por la probabilidad del enlace calculada con c. El programa usa la respuesta que tiene que adivinar.

El efecto es sistemático, no aleatorio. `getProb` devuelve (cuenta + $\alpha$)/(total + $\alpha \cdot$ n), que nunca pasa de 1, así que multiplicar solo la clase verdadera solo puede bajarla frente a las demás: un caso bien clasificado puede pasar a mal clasificado y nunca al revés. La exactitud con cualquier conjunto de enlaces es, por tanto, menor o igual que sin enlaces. Con $\alpha = 0$ y una combinación no vista, `getProb` da 0/0 y la clase verdadera no puede ganar nunca.

En `SuperParentNBAlgorithm.getOptimalEdit` eso hace que el superpadre y cada enlace se elijan por cuánto **menos** empeoran la exactitud, y que ningún enlace pueda mejorar de verdad al Bayes ingenuo (el algoritmo tampoco lo comprueba: ver error 133).

Medido sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv`, clase `Bronchitis`, $\alpha = 0{,}5$, con las mismas ventanas de prueba que usa la métrica:

- Con el código de hoy, la exactitud sin enlaces es 0,8444, y cada uno de los 42 enlaces posibles (parejas ordenadas de características) la baja, a valores entre 0,4264 y 0,8376.
- Repitiendo la cuenta con cuentas hechas directamente sobre los casos (para apartar el defecto de error 127): multiplicando solo la clase verdadera, ningún enlace pasa de 0,8444 (el mejor, 0,8402); multiplicando cada clase con su propia cuenta, 7 de los 42 enlaces la mejoran (el mejor, el del factor P(Dyspnea | X-ray, clase), con 0,8458). Esta cuenta conserva la doble cuenta de la característica hija (error 130); sin ella, los enlaces que mejoran son 14.
- El algoritmo de superpadre con «Unique superparent» desmarcada añade el enlace X-ray → Smoker con una exactitud medida de 0,8253, por debajo de la del Bayes ingenuo sin enlaces, y se detiene; que lo acepte aunque quede por debajo es otro defecto, descrito en error 133.

**El código.** learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:193-222

```
public int predictClassValueAugmentedNB(int[] row, List<Variable[]> van, int it){
    double[] probs = getProbRootNode();
    for (String variable :
    ↪  getNonRootNodes().stream().map(Node::getVariable).map(Variable::getName).toList())
    ↪  {
        int index = getIndexVariable(variable);
        for(int i=0; i<freqRootNode.length;i++){
            probs[i]*= ((crossTabs.get(variable)[it][row[index]][i]+
↪   alpha)/(freqRootNode[i]+ alpha*getNonRootNodes().size()))
            ;
        }
    }
    van.forEach(arc->{
        probs[row[getClassVariableIndex()]] *= getProb(arc[0], arc[1], row, it);
    });
    return getIndexOfMaxValue(probs);
}

protected double getProb(Variable tail, Variable head, int[] row, int it){
    double[][][][] ct = _2ndLevelCrosstab.get(tail.getName()+"-"+head.getName());
    double count = 0;
    double value = ct[it][row[getIndexVariable(tail)]][row[getIndexVariable(head)]][row[
    ↪  getClassVariableIndex()]];
    for(int i=0; i < tail.getNumStates(); i++){
        count+=ct[it][i][row[getIndexVariable(head)]][row[getClassVariableIndex()]];
    }
    return (value + alpha) / (count + alpha * getNonRootNodes().size());
}
```

**Cómo arreglarlo.** Pasar a `getProb` la clase como parámetro en vez de leerla del caso, y en `predictClassValueAugmentedNB` multiplicar cada `probs[c]` por `getProb(arc[0], arc[1], row, it, c)` para todas las clases c.

`predictClassValueAugmentedNB` es el único sitio que usa `getProb`; `computeAugmentedNetAccuracy` es su único llamador, y a él llegan `scoreAugmentedNet` y `getScore` en modo aumentado.

Conviene hacerlo junto con error 127, que da a `getProb` cuentas de casillas equivocadas, y con error 131 y error 132, porque los cuatro cambian los enlaces que aprende el clasificador de superpadre.

Prueba de regresión: sobre `BN-asia10k.csv` con clase `Bronchitis`, la exactitud de `computeAugmentedNetAccuracy` con un enlace debe poder superar a la de sin enlaces (con cuentas correctas lo hace el enlace del factor P(LungCancer | Smoker, clase), con y sin la doble cuenta de error 130); y, en general, multiplicar el factor de un enlace por una misma constante en todas las clases no debe cambiar la predicción.

### Error 129 · [Corregido] El clasificador de superpadre puntúa el superpadre y cada enlace candidato con la probabilidad condicionada al revés, P(padre | hijo, clase), y a los enlaces ya añadidos les aplica la del sentido correcto, así que decide con una exactitud que no es la de la red que resulta

| | |
|---|---|
| **Estado** | Corregido en el commit `a6e3ef0` (05-10-2026): `scoreAugmentedNet` construye todas las parejas con la hija primero, y los parámetros de `getProb` se llaman `child` y `parent`. La prueba de regresión usa una base sintética con características de 3, 2 y 4 estados que dependen unas de otras, en vez de `BN-asia10k.csv`: un enlace candidato y un superpadre candidato puntúan lo mismo que la red una vez añadidos sus enlaces (antes del arreglo, 0,9133 frente a 0,9167 y 0,9183 frente a 0,9417). Con «Unique superparent» marcada sigue sin aprenderse ningún enlace (error 132). |
| **Módulo** | learning.metric |
| **Paquete** | org.openmarkov.learning.metric.cmi.accuracy |
| **Ficheros** | `learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:64-97`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:193-222`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:84-139` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Tapado por** | error 132: con «Unique superparent» marcada, que es la opción de fábrica, no se aprende ningún enlace. |
| **Relacionados** | error 127, error 128, error 130, error 131, error 132, error 133 |

**Qué pasa.** El usuario llega por Herramientas («Tools») → «Learning» → tipo «Discriminative» → «Superparent naive bayes», y también por la validación cruzada de `bnEvaluation`, que ofrece el mismo algoritmo. El algoritmo elige el superpadre y cada enlace por la exactitud (proporción de casos cuya clase se acierta) que la métrica `Accuracy` da a la red con el cambio propuesto; esa exactitud se calcula con un factor distinto del que tendrá la red una vez hecho el cambio, así que acepta o rechaza enlaces por un número que no es el de la red resultante.

El clasificador de superpadre parte del Bayes ingenuo, elige una característica como «superpadre» y le cuelga como hijas otras características. En modo «red aumentada», `Accuracy.scoreAugmentedNet` reúne una lista de parejas de variables y `predictClassValueAugmentedNB` multiplica, por cada pareja, `getProb(pareja[0], pareja[1], …)`. `getProb(tail, head, …)` divide la cuenta de (tail, head, clase) entre la de (head, clase): es P(pareja[0] | pareja[1], clase). Para que sea la probabilidad de la hija dado su padre, la pareja tiene que ir como {hija, padre}.

`scoreAugmentedNet` no ordena igual todas las parejas:

- Enlaces ya añadidos (líneas 83-86): para cada característica con un segundo padre pone {característica, padre}, y el factor es P(hija | padre, clase), el correcto.
- Enlace candidato entre dos características, X→Y (línea 89): pone la pareja tal como viene en el cambio, {X, Y}, y el factor es P(X | Y, clase): el del padre dado la hija.
- Elección del superpadre S, que se puntúa con el cambio clase→S (líneas 91-93): pone {S, característica} para cada característica sin segundo padre, y el factor es P(S | característica, clase), también al revés.

`SuperParentNBAlgorithm.getOptimalEdit` elige el superpadre con la mejor de esas puntuaciones

y acepta cada enlace S→hija si su puntuación supera a la anterior. En el paso siguiente, el enlace ya añadido se puntúa en el otro sentido, con otro número.

Medido sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv`, clase `Bronchitis`, alfa 0,5, con «Unique superparent» desmarcada (con la opción marcada, que es la de fábrica, hoy no se aprende nada por error 132):

- Con el código de hoy, el algoritmo añade X-ray→Smoker con una exactitud de 0,8253. La misma métrica, sobre la red con ese enlace tal como lo puntúa a partir del paso siguiente (la pareja {Smoker, X-ray}), da 0,6735.
- Repitiendo la cuenta con cuentas hechas directamente sobre los casos y multiplicando cada clase por su factor (para apartar error 127 y error 128), y con el resto del producto tal como lo escribe hoy el método, que cuenta dos veces la característica hija (error 130): con el factor al revés el superpadre elegido sería Dyspnea (0,8458), y con el correcto, TuberculosisOrCancer, empatado con Smoker a 0,8456 (en un empate el algoritmo se queda con el último). Los seis enlaces que salen de Dyspnea, que con el factor al revés dan 0,8458, por encima del Bayes ingenuo (0,8444), con el factor correcto quedan todos por debajo (entre 0,8426 y 0,8443). De los 42 enlaces posibles entre dos características, 40 dan distinta exactitud según el sentido del factor.

**El código.** learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:80-97

```
private double scoreAugmentedNet(Variable[] v){
    LinkedList<Variable[]> augmentedNet = new LinkedList<>();

    getNonRootNodes().stream().filter(n -> n.getNumParents() > 1).toList().forEach(node ->
    ↪ {
        augmentedNet.add(new Variable[]{node.getVariable(),
                node.getParents().stream().filter(n -> n !=
↪   getRootNode()).toList().get(0).getVariable()});
    });

    if(!v[0].getName().equals(getRootNode().getName()) &&
    ↪  !v[1].getName().equals(getRootNode().getName())){
        augmentedNet.add(v);
    }else{
        getNonRootNodes().stream().filter(n->n.getNumParents()<2 &&
        ↪  !n.getName().equals(v[1].getName())).forEach(tail ->{
            augmentedNet.add(new Variable[]{v[1], tail.getVariable()});
        });
    }

    return computeAugmentedNetAccuracy(augmentedNet);
}
```

learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:212-222

```
protected double getProb(Variable tail, Variable head, int[] row, int it){
    double[][][][] ct = _2ndLevelCrosstab.get(tail.getName()+"-"+head.getName());
    double count = 0;

    double value = ct[it][row[getIndexVariable(tail)]][row[getIndexVariable(head)]][row[
    ↪  getClassVariableIndex()]];
    for(int i=0; i < tail.getNumStates(); i++){
        count+=ct[it][i][row[getIndexVariable(head)]][row[getClassVariableIndex()]];
```

```
    }

    return (value + alpha) / (count + alpha * getNonRootNodes().size());
}
```

**Cómo arreglarlo.** Usar un solo convenio para todas las parejas. Lo más corto es construir las del candidato con la hija primero, como ya se hace con los enlaces añadidos: `new Variable[]{v[1], v[0]}` para un enlace entre características y `new Variable[]{tail.getVariable(), v[1]}` para el superpadre. La otra opción es que `getProb` reciba (padre, hija) y dar la vuelta a las líneas 83-86. En los dos casos conviene renombrar los parámetros de `getProb`: `tail` y `head` suelen nombrar el origen y el destino de un enlace, y aquí se usan al contrario.

`scoreAugmentedNet` es el único sitio que construye las parejas; las consumen `computeAugmentedNetAccuracy`, `predictClassValueAugmentedNB` y `getProb`. `SuperParentNBAlgorithm` es el único algoritmo que activa el modo aumentado.

Conviene hacerlo a la vez que error 127 (cuentas leídas de casillas equivocadas en la misma tabla), error 128 (el factor solo se aplica a la clase verdadera) y error 133 (el algoritmo no compara con el Bayes ingenuo de partida): los cuatro cambian qué enlaces aprende el clasificador de superpadre.

Prueba de regresión: sobre `BN-asia10k.csv` con clase `Bronchitis`, la puntuación que `getScore` da a `AddLinkEdit(X-ray → Smoker)` sin enlaces entre características debe ser igual a la que la métrica da a la red una vez añadido ese enlace; y, en general, un enlace debe puntuarse igual como candidato que como enlace ya añadido.

### Error 130 · [Corregido] El clasificador de superpadre cuenta dos veces cada característica que recibe un segundo padre: al medir la exactitud multiplica P(hija | clase) y además P(hija | padre, clase), así que elige enlaces con un clasificador que no es el que construye

| | |
|---|---|
| **Estado** | Corregido en el commit `271f3ee` (05-10-2026): `predictClassValueAugmentedNB` no multiplica P(hija \| clase) para la hija de cada pareja, que solo aporta el factor del enlace. La prueba de regresión es la primera de las dos propuestas abajo, con una base sintética en la que una característica es copia exacta de otra: con el enlace entre las dos, la exactitud es la del clasificador sin la copia (antes del arreglo, 0,7565 frente a 0,799). La segunda, sobre `BN-asia10k.csv`, no se ha añadido. Con «Unique superparent» marcada sigue sin aprenderse ningún enlace (error 132). |
| **Módulo** | learning.metric |
| **Paquete** | org.openmarkov.learning.metric.cmi.accuracy |
| **Ficheros** | `learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:193-209`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:80-97`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:212-222`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:84-142`<br>`learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java:172-183` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Tapado por** | error 132: con «Unique superparent» marcada, que es la opción de fábrica, no |

| | |
|---|---|
| | se aprende ningún enlace. |
| **Relacionados** | error 127, error 128, error 129, error 132, error 133 |

**Qué pasa.** El usuario llega por Herramientas («Tools») → «Learning» → tipo «Discriminative» → «Superparent naive bayes», y también por la validación cruzada de `bnEvaluation`, que ofrece el mismo algoritmo. Para elegir el superpadre y los enlaces, el algoritmo mide la exactitud (la proporción de casos cuya clase se acierta) de un clasificador en el que cada característica que recibe un segundo padre cuenta dos veces. La red que devuelve no es así, de modo que los enlaces se eligen por un número que no corresponde a la red resultante. Con «Unique superparent» marcada, que es la opción de fábrica, hoy no se aprende ningún enlace por otro motivo (error 132); desmarcándola, quitar solo esta doble cuenta cambia el enlace aprendido en tres de las cuatro bases probadas.

En un Bayes ingenuo aumentado, la probabilidad de cada clase c es P(c) por el producto, para cada característica X, de su probabilidad dada su familia: P(X | c) si solo tiene la clase como padre, y P(X | padre, c) si tiene además otro padre. La red que resulta del aprendizaje es así: el aprendizaje de parámetros (`LearningAlgorithm.parametricLearning`) da a cada característica una sola tabla, condicionada a todos sus padres.

`Accuracy.predictClassValueAugmentedNB` multiplica primero P(X | c) para **todas** las características (`getNonRootNodes`) y después, por cada pareja de la lista de enlaces, el factor de `getProb`, que es la probabilidad de un miembro de la pareja dado el otro y la clase. No quita el factor P(X | c) de la característica que en ese enlace hace de hija, así que esa característica aporta dos factores, P(hija | c) · P(hija | padre, c). Pasa con todas las parejas que construye `scoreAugmentedNet`: las de los enlaces ya añadidos, la del enlace candidato y las del superpadre que se está puntuando con todas sus hijas.

Hoy el cálculo tiene además otros tres defectos: cuentas leídas en casillas equivocadas (error 127), el factor de cada enlace aplicado solo a la clase verdadera del caso (error 128) y la probabilidad de las parejas candidatas tomada al revés (error 129). Las cuentas con que error 128 y error 129 apartan esos defectos conservan esta doble cuenta.

Medido sobre cuatro bases de casos, con alfa 0,5 y «Unique superparent» desmarcada, ejecutando el algoritmo con la métrica de hoy y con una copia de la métrica que solo deja fuera P(hija | c) de la hija de cada pareja (la hija de un enlace ya añadido y la del enlace o del superpadre que se puntúa), sin tocar nada más:

- `learning.algorithm/src/test/resources/network/BN-asia10k.csv`, clase `Bronchitis`: hoy aprende X-ray→Smoker (exactitud medida 0,8253); sin la doble cuenta, Dyspnea→Smoker (0,8394).
- `integrationTests/src/main/resources/networks/learning/learnTestDataBase.csv`, clase `A`: B→E (0,7220) frente a D→B (0,7290).
- `learning.algorithm/src/test/resources/network/HeadToHead2.csv`, clase `C`: B→F (0,4800) frente a B→E (0,4116).
- `core/src/test/resources/nets/Cataract-RRDdatasetPreProc3.csv`, clase `success`: opType→breakType en los dos casos (0,8302 frente a 0,8453).

Medido también con los otros tres defectos apartados, con un cálculo aparte sobre las mismas ventanas de prueba y el mismo suavizado que la métrica (cuentas hechas directamente sobre los casos, el factor de cada enlace en cada clase y la probabilidad de la hija dado el padre), con y sin la doble cuenta:

- `BN-asia10k.csv`: el Bayes ingenuo acierta 0,8444. Con la doble cuenta, 7 de los 42 enlaces posibles entre dos características lo mejoran, entre ellos los seis que llegan a Dyspnea; sin ella, 14, y ninguno de esos seis. De los 42 enlaces, 36 dan distinta exactitud con y sin la

doble cuenta. Siguiendo el algoritmo publicado desde el Bayes ingenuo, con la doble cuenta se añade Smoker→Dyspnea (0,8458), que sin ella da 0,8438, por debajo del Bayes ingenuo; sin ella se añaden TuberculosisOrCancer→Smoker y TuberculosisOrCancer→X-ray (0,8458).

- `learnTestDataBase.csv`: con la doble cuenta, E→C, E→B y B→D (0,8400, y 0,8070 sin la doble cuenta); sin ella, D→B, D→C y D→E (0,8360).
- `HeadToHead2.csv`: con la doble cuenta, F→E y B→A (0,7452); sin ella, F→E y A→B (0,7480).
- `Cataract-RRDdatasetPreProc3.csv`: en los dos casos, ningún enlace mejora el Bayes ingenuo (0,9642).

**El código.** learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:193-209

```
public int predictClassValueAugmentedNB(int[] row, List<Variable[]> van, int it){
    double[] probs = getProbRootNode();

    for (String variable :
    ↪  getNonRootNodes().stream().map(Node::getVariable).map(Variable::getName).toList())
    ↪  {
        int index = getIndexVariable(variable);
        for(int i=0; i<freqRootNode.length;i++){
            probs[i]*= ((crossTabs.get(variable)[it][row[index]][i]+
↪   alpha)/(freqRootNode[i]+ alpha*getNonRootNodes().size()))
            ;
        }
    }

    van.forEach(arc->{
        probs[row[getClassVariableIndex()]] *= getProb(arc[0], arc[1], row, it);
    });

    return getIndexOfMaxValue(probs);
}
```

**Cómo arreglarlo.** En `predictClassValueAugmentedNB`, reunir antes del primer bucle las hijas de las parejas de `van` y no multiplicar P(X | c) para ellas; su única contribución debe ser el factor del enlace.

Saber cuál de los dos miembros de la pareja es la hija depende de un convenio que hoy no es uniforme: `scoreAugmentedNet` pone la hija primero en los enlaces ya añadidos y la segunda en los candidatos (error 129). Con el arreglo de esa ficha (todas las parejas como {hija, padre}), la hija es siempre `arc[0]`; por eso conviene hacer los dos cambios a la vez.

`predictClassValueAugmentedNB` es el único sitio que combina los dos tipos de factor; `predictClassValue`, que usa el Bayes ingenuo selectivo, no tiene enlaces entre características y no está afectado. Conviene arreglarlo junto con error 127, error 128, error 129 y error 133, porque todos cambian qué enlaces aprende el clasificador de superpadre.

Prueba de regresión: con un enlace cuyo factor P(hija | padre, c) valga lo mismo para todas las clases, la predicción de `predictClassValueAugmentedNB` debe ser la del Bayes ingenuo sin esa característica; y sobre `BN-asia10k.csv` con clase `Bronchitis`, la exactitud con el enlace Smoker→Dyspnea debe ser la del clasificador en que Dyspnea solo aporta P(Dyspnea | Smoker, c).

### Error 131 · [Corregido] La «validación cruzada de diez pliegues» con que los clasificadores selectivo y de superpadre eligen enlaces aparta diez ventanas al azar que se solapan y deja sin evaluar más de un tercio de los casos

| | |
|---|---|
| **Estado** | Corregido en el commit `144e329` (05-10-2026): `Dataset` baraja los casos una vez, con la semilla fija, y los reparte en diez partes disjuntas que los cubren todos (los que sobran de la división, uno a cada una de las primeras), y `Accuracy` divide los aciertos entre el número de casos. Con menos de diez casos se mantienen las diez partes y quedan vacías las que no reciben ninguno; no se avisa. Las pruebas de regresión comprueban, con 1 003 casos, que cada uno se prueba una vez y entrena las otras nueve partes, que con tres casos no falla y que el reparto es repetible. |
| **Módulo** | learning.metric |
| **Paquete** | org.openmarkov.learning.metric.cmi.accuracy |
| **Ficheros** | `learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Dataset.java:42-68`<br>`learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Accuracy.java:130-162` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 125, error 127, error 128, error 129, error 132, error 135 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Selective naive bayes» o «Superparent naive bayes», y también por la validación cruzada de `bnEvaluation` (`CrossValidationDialog`), que ofrece los mismos algoritmos. Los enlaces que se añaden o se quitan se eligen con una exactitud que no es la de una validación cruzada: más de un tercio de los casos no se prueba nunca y otros se prueban varias veces.

La métrica `Accuracy` (exactitud: proporción de casos cuya clase se acierta) es la que usan esos dos clasificadores para decidir qué enlaces añadir o quitar. Para medir la exactitud sin examinar al clasificador con los mismos casos con que se entrena, construye un `Dataset` que, según su javadoc, divide la base de casos en k pliegues (k = `Accuracy.KFOLD` = 10). En una validación cruzada de k pliegues cada caso cae en exactamente una de las k partes de prueba.

`Dataset.initializeDatasets` no hace eso. Para cada una de las diez iteraciones elige un inicio aleatorio `maxCase` (generador con semilla fija 42) y toma como prueba la ventana contigua `[maxCase, maxCase + n/10)`; el entrenamiento es el resto.

Cada ventana se sortea independientemente de las demás, así que se solapan: hay casos que caen en varias y casos que no caen en ninguna. Además, `nextInt(n - sampleSize - 1)` impide que la ventana llegue a los dos últimos casos, y al ser ventanas contiguas, si la base viene ordenada (por ejemplo, por clase) cada ventana es una muestra sesgada.

`computeNBNetAccuracy` y `computeAugmentedNetAccuracy` dividen los aciertos entre `test[0].length * KFOLD`. Como todas las ventanas tienen el mismo tamaño, el número resultante es la media de diez exactitudes medidas en ventanas aleatorias (un caso probado dos veces cuenta dos veces); no es la exactitud de una validación cruzada. Con menos de diez casos la ventana tiene tamaño cero y la división da `NaN`.

Medido sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv` (10 000 casos): las ventanas empiezan en 6752, 808, 5289, 324, 4641, 4337, 7625, 803, 7342 y 2886; 3 691 casos no se prueban nunca, 3 309 una vez y 3 000 dos o más veces.

**El código.** learning.metric/src/main/java/org/openmarkov/learning/metric/cmi/accuracy/Dataset.java:53-68

```
private void initializeDatasets(){
    IntStream.range(0, numOfSamples).forEach(it ->{
        int maxCase = random.nextInt(cases.length-sampleSize-1);
        test[it]= Arrays.copyOfRange(cases, maxCase, maxCase+sampleSize);
        training[it] = new int[cases.length-sampleSize][];
        for(int i = 0; i<maxCase;i++){
            training[it][i]=cases[i];
        }
        for(int i=maxCase+sampleSize; i<cases.length;i++){
            training[it][i-sampleSize]=cases[i];
        }
    });
}
```

**Cómo arreglarlo.** Construir una partición de verdad: barajar los índices de los casos una vez (con semilla fija, para que sea repetible) y repartirlos en diez partes disjuntas que cubran todos los casos, repartiendo el resto de la división entre las primeras partes. El entrenamiento de cada iteración es la unión de las otras nueve.

Entonces la exactitud total es aciertos / número de casos, y no `test[0].length * KFOLD`, que ya no sería igual al total si las partes no tienen el mismo tamaño. Hay que cambiarlo en `computeNBNetAccuracy` y en `computeAugmentedNetAccuracy`.

Hay que decidir también qué hacer con bases de menos de diez casos (menos pliegues o un aviso).

El cambio alterará los modelos que aprenden los dos clasificadores; conviene hacerlo junto con error 125 y error 127, que usan las mismas partes.

Prueba: con una base de 1 003 casos, cada índice aparece en exactamente una parte de prueba y en nueve de entrenamiento.

## Error 132 · [Corregido] El clasificador de superpadre, con su opción por defecto, no aprende nada: propone un enlace del superpadre a sí mismo y el cambio entero se rechaza

| | |
|---|---|
| **Estado** | Corregido en el commit `87b167f` (05-10-2026): `getOptimalEdit` deja fuera al superpadre de las huérfanas a las que lo enlaza, en los dos modos, y en el modo «Unique superparent» comprueba que el cambio compuesto está permitido antes de proponerlo y de vaciar `orphans`, cuando se piden solo cambios permitidos. La prueba de regresión del modo «Unique superparent» usa una base sintética en la que la clase vale 1 cuando dos características difieren, en vez de `BN-asia10k.csv`. En el modo de una en una, un superpadre sigue en `orphans` y más tarde podría proponerse como hija de una de sus hijas; ejecutado con solo cambios permitidos, ese enlace no se propone. Con «solo cambios permitidos» desmarcada no se ha comprobado. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.spnb |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:59-142`<br>`learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java`<br>`learning.gui/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/gui/SuperParentNaiveBayesParametersDialog.java:26-52` |
| **Comprobado** | Leyendo el código y ejecutándolo |

| | |
|---|---|
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 127, error 128, error 129, error 130, error 131, error 133, error 134 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Superparent naive bayes» con los parámetros por defecto, y también desde la validación cruzada de `bn Evaluation`. En el aprendizaje automático recibe el Bayes ingenuo de partida, sin ningún enlace nuevo y sin aviso.

«Superparent naive bayes» parte del Bayes ingenuo, elige una característica como «superpadre» (la que más exactitud da) y cuelga de ella otras características que todavía no tienen más padre que la clase (las «huérfanas»). En `SuperParentNBAlgorithm.init` la lista `orphans` se llena con **todas** las características, y en `getOptimalEdit`, al elegir el superpadre, se añade a `superParents` pero no se quita de `orphans`.

Con la opción «Unique superparent» (`sameSP`), que es la que el diálogo de parámetros trae marcada por defecto, se construye un único cambio compuesto (`ListPNEdit`) con un `AddLink Edit` del superpadre a cada huérfana, incluida ella misma. Una red no admite un enlace de un nodo consigo mismo, así que el cambio entero se rechaza. Como `orphans` ya se ha vaciado, en la siguiente llamada no hay nada que proponer y el aprendizaje termina con el Bayes ingenuo inicial.

En el aprendizaje automático (`LearningAlgorithm.run`) el rechazo se captura y solo se escribe en el registro a nivel de depuración. En el aprendizaje interactivo, aplicar esa propuesta hace que `InteractiveLearningDialog` convierta el `DoEditException` en `UnrecoverableException` (esto último comprobado solo leyendo).

Medido sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv` (clase `Bronchitis`, alfa 0,5, `sameSP = true`):

```
step 0 [Add link ... X-ray ... to ... X-ray ..., Add link ... X-ray ... to ... Dyspnea ...,
↪  ...] {0.432300} REFUSED CannotDoEditException
final links(7) [Bronchitis->Dyspnea, Bronchitis->LungCancer, Bronchitis->Smoker,
↪  Bronchitis->Tuberculosis, Bronchitis->TuberculosisOrCancer, Bronchitis->VisitToAsia,
↪  Bronchitis->X-ray]
```

Con `sameSP = false` (de una en una) el enlace del superpadre consigo mismo también se puntúa como candidato. En el aprendizaje automático lo descarta la comprobación de restricciones (`isAllowed`), y en la misma base el algoritmo aprendió `X-ray -> Smoker` sin problema. Podría aparecer como propuesta en el interactivo si el usuario desmarca «solo cambios permitidos».

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:104-117

```java
if (bestParent != null && !orphans.isEmpty()) {
    superParents.add(bestParent);
    final Node selectedParent = bestParent;
    if (this.sameSP) {
        // sameSP mode: connect bestParent to ALL orphans via a compound edit
        List<PNEdit> linkEdits = orphans.stream()
                .map(orphan -> (PNEdit) new AddLinkEdit(probNet,
                ↪  selectedParent.getVariable(),
                        orphan.getVariable(), true))
                .toList();
        ListPNEdit compoundEdit = new ListPNEdit(probNet, linkEdits);
        bestEditProposal = new LearningEditProposal(compoundEdit,
                new ScoreEditMotivation(bestPartialScore));
        orphans.clear();
        currentAccuracy = bestPartialScore;
```

**Cómo arreglarlo.** Quitar el superpadre de `orphans` al elegirlo (antes de construir el cambio compuesto y antes del bucle de una en una), o filtrar `orphan != selectedParent` en los dos modos.

Revisar a la vez qué pasa si el cambio compuesto se rechaza por otra razón: hoy `orphans.clear()` se hace antes de saber si se aplicó.

Prueba de regresión: ejecutar el algoritmo con `sameSP = true` sobre `BN-asia10k.csv` y comprobar que la red final tiene enlaces entre características y ninguno de un nodo consigo mismo; y con `sameSP = false`, que la lista de candidatas nunca contiene el enlace superpadre → superpadre.

### Error 133 · [Corregido] El clasificador de superpadre no mide nunca el Bayes ingenuo de partida: acepta el primer superpadre y el primer enlace aunque den menos exactitud que no añadir nada

| | |
|---|---|
| **Estado** | Corregido en el commit `3ce9c93` (05-10-2026): `init` guarda en `currentAccuracy` la exactitud del Bayes ingenuo, que la métrica da ahora como puntuación de la red (`Accuracy.getScore()` en modo aumentado: la exactitud con los enlaces entre características que la red ya tiene). El superpadre se elige por su exactitud, sin umbral; un enlace S→hija, o el cambio con todos los enlaces en el modo «Unique superparent», solo se propone si supera a `currentAccuracy`, salvo que se pidan también los cambios que no mejoran. En la elección del superpadre se ha quitado la condición `\|\| !onlyPositiveEdits`, que sin umbral solo hacía que ganara el último candidato en vez del mejor. Con «Unique superparent» marcada sigue sin aprenderse ningún enlace (error 132). |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.spnb |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:31-47`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:59-74`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:84-142` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Tapado por** | error 132: con «Unique superparent» marcada, que es la opción de fábrica, no se aprende ningún enlace. |
| **Relacionados** | error 127, error 128, error 129, error 130, error 132, error 134, error 141 |

**Qué pasa.** El usuario llega por Herramientas («Tools») → «Learning» → tipo «Discriminative» → «Superparent naive bayes», y también por la validación cruzada de `bnEvaluation`. Con «Unique superparent» desmarcada, el algoritmo añade un enlace aunque, según su propia medida, el clasificador con ese enlace acierte menos casos que el Bayes ingenuo del que partió. Con la opción marcada, que es la de fábrica, hoy no se aprende nada por otro motivo (error 132); arreglado eso, el superpadre con todas sus hijas se aplicaría también sin compararlo con el Bayes ingenuo.

El javadoc de la clase describe el algoritmo publicado: partir del Bayes ingenuo, **evaluarlo**, elegir el superpadre que más aumente la exactitud (la proporción de casos cuya clase se acierta), probar un enlace de él a cada característica huérfana y quedarse con el mejor solo si mejora la exactitud; si no, devolver el clasificador. El paso de evaluar el clasificador de partida no está en el código: `currentAccuracy` empieza en 0,0 (línea 47) y nada le asigna la exactitud del Bayes ingenuo;

`init` tampoco la pone a cero, así que una segunda llamada a `init` (por ejemplo, «Finish» en el aprendizaje interactivo, error 141) conserva la de antes, que en el aprendizaje interactivo ya ha cambiado solo con llenar la tabla (error 134).

En `getOptimalEdit`, `bestPartialScore` empieza valiendo `currentAccuracy`:

- En la primera llamada cualquier superpadre cumple `addScore >= bestPartialScore`, porque la exactitud no es negativa, y se elige el mejor aunque quede por debajo del Bayes ingenuo. `bestPartialScore` pasa a ser la exactitud de la red con ese superpadre como padre de todas las características que aún no tienen segundo padre.
- Con «Unique superparent» se propone el cambio con todos esos enlaces y `currentAccuracy` toma esa exactitud.
- Sin ella, cada enlace S→hija se acepta si su exactitud supera a `bestPartialScore`, es decir, a la de la red con el superpadre como padre de todas ellas, no a la del Bayes ingenuo.

En los pasos siguientes la comparación es con la exactitud del paso anterior, así que lo aceptado de más en el primero ya no se corrige.

Medido con el código de hoy, alfa 0,5 y «Unique superparent» desmarcada, calculando con la misma métrica la exactitud del Bayes ingenuo sin enlaces (`Accuracy.computeAugmentedNetAccuracy` con la lista de enlaces vacía):

- `learning.algorithm/src/test/resources/network/BN-asia10k.csv`, clase `Bronchitis`: Bayes ingenuo 0,8444; el mejor superpadre, X-ray, 0,4323; añade X-ray→Smoker con 0,8253.
- `integrationTests/src/main/resources/networks/learning/learnTestDataBase.csv`, clase `A`: Bayes ingenuo 0,7860; superpadre B, 0,3670; añade B→E con 0,7220.
- `learning.algorithm/src/test/resources/network/HeadToHead2.csv`, clase `C`: Bayes ingenuo 0,6756; superpadre B, 0,0290; añade B→F con 0,4800.
- `core/src/test/resources/nets/Cataract-RRDdatasetPreProc3.csv`, clase `success`: Bayes ingenuo 0,9642; superpadre opType, 0,0321; añade opType→breakType con 0,8302.

En las cuatro el algoritmo se detiene después de ese enlace. Hoy todos los enlaces bajan la exactitud medida por error 128, así que, partiendo de la exactitud del Bayes ingenuo, en estas bases no se añadiría ninguno. El defecto no depende de eso: con la métrica corregida, un primer superpadre o un primer enlace que no mejore el Bayes ingenuo se seguiría aceptando (esto último está leído en las comparaciones de `getOptimalEdit`).

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:31-47

```
/**
 *         SUPERPARENT ALGORITHM
 *             0. Initialize network to naive Bayes.
 *             1. Evaluate the current classifier.
 *             2. Consider making each node a SuperParent.  Let ASP be the SuperParent
 ↪  which increases accuracy the most.
 *             3. Consider an arc from ASP to each orphan. If the best such arc improves
 ↪  accuracy, keep it and go to 2.
 *             Else: Return the current classifier.
 */
...
private double currentAccuracy = 0.0;
```

learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:86-129

```
double bestPartialScore = currentAccuracy;
...
for (Node nodeParent : subtractListFromNonRootNode(superParents)) {
    AddLinkEdit addLink = new AddLinkEdit(probNet, getRootNode().getVariable(),
↪   nodeParent.getVariable(), true);
    double addScore = metric.getScore(addLink);
    if (... && (addScore >= bestPartialScore || !onlyPositiveEdits)) {
        bestPartialScore = addScore;
        bestParent = nodeParent;
    }
}
...
    for (Node nodeChild : orphans) {
        AddLinkEdit addLink = new AddLinkEdit(probNet, bestParent.getVariable(),
↪   nodeChild.getVariable(), true);
        double addScore = metric.getScore(addLink);
        if (... && (addScore > bestPartialScore || !onlyPositiveEdits)) {
            bestEdit = addLink;
            bestPartialScore = addScore;
        }
    }
```

**Cómo arreglarlo.** Medir el Bayes ingenuo al empezar y guardarlo en `currentAccuracy`, en `init` (después de `setRelationsForRootVariable` y de configurar la métrica), por ejemplo con `Accuracy.computeAugmentedNetAccuracy` sobre la lista vacía de enlaces; así también se repone en cada `init`.

Separar la puntuación del superpadre de la del clasificador actual: el superpadre se elige por su exactitud, pero un enlace S→hija solo debe aceptarse si su exactitud supera a `currentAccuracy`, y no a la del superpadre con todas sus hijas. En el modo «Unique superparent», que añade todos los enlaces de una vez, el cambio compuesto solo debe proponerse si su exactitud supera a `currentAccuracy`. Hay que mantener la condición `|| !onlyPositiveEdits`, que en el aprendizaje interactivo deja ver también los cambios que no mejoran.

Conviene hacerlo con error 128, error 129 y error 127, que cambian las exactitudes con las que se compara, y con error 132, que hoy impide aprender nada con la opción de fábrica.

Prueba de regresión: con una métrica de prueba (una subclase de `Accuracy`) que dé al Bayes ingenuo una exactitud mayor que la de cualquier superpadre y cualquier enlace, el algoritmo, en los dos modos, debe devolver el Bayes ingenuo sin enlaces entre características.

### Error 134 · En el aprendizaje interactivo con el clasificador de superpadre, llenar la tabla hace avanzar el algoritmo como si se hubieran aplicado las propuestas: «Block edit» o volver a marcar «Only positive edits» dejan la tabla vacía y dan el aprendizaje por terminado, y aplicando las propuestas una a una se aprenden menos enlaces que con el aprendizaje automático

| | |
|---|---|
| **Estado** | Presente en parte. «Finish» nada más abrir la ventana ya aprende lo mismo que el aprendizaje automático: `init` repone `currentAccuracy` desde el commit `3ce9c93` (05-10-2026, error 133). Lo demás sigue presente. La ficha se volvió a medir el 7-10-2026 sobre `2b4fc61`, con los errores 127 a 133 ya corregidos, y sus números son los de esa medida. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.spnb |

| | |
|---|---|
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:41-47`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:59-79`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:88-153`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/common/DiscriminativeAlgorithm.java:81-90`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:306-350`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:407-431`<br>`learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java:88-103` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Si la tabla del aprendizaje interactivo con el superpadre debe enseñar solo alternativas para el paso en curso o también los pasos siguientes |
| **Relacionados** | error 132, error 133, error 140, error 141 |

**Qué pasa.** El usuario llega por Herramientas («Tools») → «Learning» → tipo «Discriminative» → «Superparent naive bayes», deja «Interactive learning», que es la opción marcada por omisión, y pulsa «Learn». La ventana «Interactive learning» se abre con una propuesta. Con «Unique superparent» desmarcada (en «Options») la propuesta es un enlace; con la casilla marcada, que es la opción de fábrica, es un cambio compuesto con todos los enlaces que salen del superpadre elegido. A partir de ahí:

- Si pulsa «Block edit» sobre esa propuesta, la tabla se queda vacía y la ventana da el aprendizaje por terminado: desactiva «Apply edit» y «Complete phase», y «Finish» ya no aprende. Con «Unique superparent» desmarcada hay otros enlaces que el algoritmo ofrecería.
- Si desmarca «Only positive edits», la tabla enseña varios enlaces, cada uno desde un superpadre distinto y ninguno desde el de la propuesta inicial; si la vuelve a marcar, la tabla se queda vacía y la ventana da el aprendizaje por terminado. Con «Unique superparent» marcada la tabla queda vacía ya al desmarcarla.
- Si aplica las propuestas una a una con «Apply edit» hasta que la tabla se vacía, la red puede quedarse con menos enlaces entre características que la que da el aprendizaje automático sobre la misma base.

El clasificador de superpadre guarda el estado de su búsqueda en tres campos: `superParents` (las características ya usadas como superpadre), `orphans` (las que aún no tienen un segundo padre) y `currentAccuracy` (la exactitud que hay que superar). `getOptimalEdit` los cambia mientras busca, antes de que se aplique nada: añade el superpadre elegido a `superParents` (aunque después ningún enlace desde él pase la comparación), quita la hija propuesta de `orphans` (o vacía la lista en el modo «Unique superparent») y pone en `currentAccuracy` la exactitud de la propuesta.

El aprendizaje automático (`LearningAlgorithm.run`) aplica cada propuesta nada más pedirla, y no lo nota. La ventana, en cambio, cada vez que rehace la tabla (`InteractiveLearningDialog.updateEditionsTable`: al abrirse, tras «Apply edit», «Block edit» o «Show blocked» y al cambiar cada casilla) llama a `getBestEdit` y después a `getNextEdit` hasta tener 50 propuestas. `DiscriminativeAlgorithm.getBestEdit` solo vacía el historial de propuestas ya consideradas, y `getNextEdit` vuelve a llamar a `getOptimalEdit`, así que cada llamada avanza el estado como si la propuesta anterior se hubiera aplicado. El refresco siguiente parte de ese estado: deja fuera los superpadres ya propuestos, no ofrece las hijas ya propuestas y exige superar la exactitud de la última propuesta. Lo mismo pasa con el llenado que sigue a cada «Apply edit»: gasta superpadres que el aprendizaje

automático habría probado en los pasos siguientes, y por eso aplicar una a una puede acabar antes.

«Finish» nada más abrir la ventana sí da hoy la red del aprendizaje automático: pasa por `init` (error 141), que rehace los tres campos. Después de un bloqueo o de un cambio de casilla que deje la tabla vacía, «Finish» no hace nada, porque la ventana ya ha dado el aprendizaje por terminado.

Es el mismo patrón que el del aprendizaje interactivo con PC (error 140), con otro estado y en otra clase.

Medido con una prueba temporal que construye el algoritmo como la ventana de aprendizaje (métrica de exactitud, alfa 0,5) y repite sus llamadas: llenar la tabla; bloquear con `blockEdit` y volver a llenarla; llenarla con la casilla desmarcada y otra vez marcada; `learn` como «Finish»; y aplicar la primera fila y volver a llenar la tabla hasta que se vacía. Sobre cuatro bases, con «Unique superparent» desmarcada:

- `learning.algorithm/src/test/resources/network/BN-asia10k.csv`, clase `Bronchitis`: el aprendizaje automático añade cuatro enlaces (X-ray→Tuberculosis, LungCancer→TuberculosisOrCancer, Tuberculosis→Dyspnea y TuberculosisOrCancer→Smoker). La tabla se abre con TuberculosisOrCancer→Smoker (0,8428). Bloqueándolo, la tabla queda vacía; el mismo algoritmo, con esa propuesta bloqueada antes de llenar la tabla, ofrece TuberculosisOrCancer→LungCancer (0,8428). Desmarcando «Only positive edits», la tabla enseña cinco enlaces desde cinco superpadres distintos (Tuberculosis→TuberculosisOrCancer, Smoker→Tuberculosis, VisitToAsia→LungCancer, X-ray→VisitToAsia y Dyspnea→X-ray) y ninguno desde TuberculosisOrCancer; volviéndola a marcar, la tabla queda vacía. Aplicando la primera fila hasta vaciar la tabla quedan tres enlaces: falta LungCancer→TuberculosisOrCancer.
- `integrationTests/src/main/resources/networks/learning/learnTestDataBase.csv`, clase `A`: el automático añade cuatro enlaces (D→B, B→E, C→D y F→C). La tabla se abre con D→B (0,808); bloqueándolo, tabla vacía (sin el llenado previo se ofrecerían D→E y B→D); con la casilla desmarcada, tres enlaces desde otros superpadres; al volver a marcarla, vacía. Aplicando la primera fila hasta vaciar la tabla quedan dos enlaces (D→B y F→C).
- `learning.algorithm/src/test/resources/network/HeadToHead2.csv`, clase `C`: el automático añade A→B y F→E. La tabla se abre con dos filas, F→E (0,7114) y su inverso E→F (0,7126), que es la propuesta del paso siguiente; bloqueando la primera, vacía (aquí también sin el llenado previo); con la casilla desmarcada, un enlace (A→B); marcada otra vez, vacía. Aplicando la primera fila hasta vaciar la tabla quedan los dos enlaces del automático.
- `core/src/test/resources/nets/Cataract-RRDdatasetPreProc3.csv`, clase `success`: el automático no añade ningún enlace y la tabla se abre vacía. Un segundo llenado con las mismas casillas ofrece loc48rrd→loc48 (0,9646), que supera la exactitud de partida; es el llenado que, por lectura, hace «Show blocked».

En las tres primeras bases, «Finish» nada más abrir da la misma red que el aprendizaje automático.

Con «Unique superparent» marcada, en las tres primeras bases la tabla se abre con el cambio compuesto (en `BN-asia10k.csv`, los seis enlaces desde TuberculosisOrCancer, 0,8454) y cualquier refresco la deja vacía, porque la primera propuesta ya vació `orphans`: al bloquear, al desmarcar «Only positive edits» y al volver a marcarla. «Finish» nada más abrir y «Apply edit» dan la red del aprendizaje automático.

La ventana es un `JDialog` y no se ha abierto: lo que hace con la tabla vacía (botones desactivados, «Finish» sin efecto) está leído en `updateEditionsTable` y `btnRunActionPerformed`.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/SuperParentNBAlgorithm.java:111-150

```
if (bestParent != null && !orphans.isEmpty()) {
    superParents.add(bestParent);
    ...
    if (this.sameSP && (bestParentScore > currentAccuracy || !onlyPositiveEdits)) {
        ...
        if (!linkEdits.isEmpty() && (!onlyAllowedEdits || isAllowed(compoundEdit))) {
            bestEditProposal = new LearningEditProposal(compoundEdit,
                    new ScoreEditMotivation(bestParentScore));
            orphans.clear();
            currentAccuracy = bestParentScore;
        }
    } else if (!this.sameSP) {
        for (Node nodeChild : children) {
            ...
        }
    }
}

if (bestEdit != null) {
    bestEditProposal = LearningEditProposal.scored(bestEdit, bestPartialScore);
    markEditAsConsidered(bestEdit);
    orphans.remove(probNet.getNode(bestEdit.getVariableTo()));
    currentAccuracy = bestPartialScore;
}
```

learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/common/DiscriminativeAlgorithm.java:81-90

```
@Override
public LearningEditProposal getBestEdit(boolean onlyAllowedEdits, boolean
↪  onlyPositiveEdits) {
    resetHistory();
    return getNextEdit(onlyAllowedEdits, onlyPositiveEdits);
}

@Override
public LearningEditProposal getNextEdit(boolean onlyAllowedEdits, boolean
↪  onlyPositiveEdits) {
    return getOptimalEdit(onlyAllowedEdits, onlyPositiveEdits);
}
```

**Cómo arreglarlo.** Buscar propuestas no debe cambiar el estado del algoritmo; `superParents`, `orphans` y `currentAccuracy` deben cambiar cuando se aplica (o se deshace) una edición. Dos caminos:

- Actualizarlos al ejecutarse o deshacerse una edición, escuchando las ediciones de la red, y dejar `getOptimalEdit` como una búsqueda que solo lee.
- Deducirlos de la red en cada búsqueda: huérfanas son las características cuyo único padre es la clase, superpadres las que ya tienen hijas entre las características, y la exactitud actual la de la red tal como está.

Qué debe enseñar la tabla depende de lo que decida el equipo: solo alternativas para el paso en curso (otros superpadres y otras hijas, con el historial de propuestas ya consideradas para no repetirlas), o también los pasos siguientes, en cuyo caso el llenado debe hacerse sobre una copia del estado y separar en la tabla lo que depende de aplicar antes otra fila.

Hay que mantener el resultado del aprendizaje automático y de la validación cruzada de `bn`

`Evaluation`, que usan `run`. Si una edición se rechaza al ejecutarla, el estado no debe cambiar.

Pruebas de regresión, repitiendo las llamadas de la ventana sobre `BN-asia10k.csv` con clase `Bronchitis` y «Unique superparent» desmarcada: tras llenar la tabla, bloquear la primera propuesta y volver a llenarla no debe dejarla vacía (bloqueada antes del primer llenado, se ofrece TuberculosisOrCancer→LungCancer); desmarcar y volver a marcar «Only positive edits» debe dar la misma tabla que al abrir; y aplicar la primera fila hasta vaciar la tabla debe dar la misma red que el aprendizaje automático.

### Error 135 · [Corregido] El Bayes ingenuo selectivo hacia delante aprende modelos distintos en ejecuciones distintas sobre la misma base

| | |
|---|---|
| **Estado** | Corregido en el commit `1bd8c38` (05-10-2026): `getOptimalEdit` recorre las candidatas en el orden de la red también hacia delante, y entre las de igual exactitud gana la primera; un paso que iguala la exactitud actual se sigue aceptando. Cuando se piden también los cambios que no mejoran, se propone la mejor candidata y no la última recorrida. La prueba de regresión usa una métrica de prueba con veinte características empatadas y comprueba que se propone la primera de la red; no ejecuta el algoritmo dos veces sobre una base, como proponía la ficha. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.snb |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/snb/SelectiveNBAlgorithm.java:59-96` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 125, error 126, error 131, error 136 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Selective naive bayes» con la casilla «Forward» marcada, que es el valor por defecto del diálogo de parámetros; también desde la validación cruzada de `bnEvaluation`. Sobre la misma base de casos, cada ejecución puede dar un modelo final distinto.

«Selective naive bayes» elige qué características conserva: hacia delante parte de la clase sola y en cada paso añade la característica que más exactitud da; hacia atrás parte del Bayes ingenuo completo y quita.

En `SelectiveNBAlgorithm.getOptimalEdit`, hacia delante, las candidatas se reúnen con `Collectors.toSet()` (un `HashSet`), y el método se queda con la última que cumpla `addScore >= bestPartialScore`. El orden de un `HashSet` de nodos depende de `Node.hashCode`, que depende de `Variable`, que no redefine `hashCode` (usa la identidad del objeto, es decir, las señas de memoria).

Como la exactitud es un número de aciertos sobre un número fijo de casos, los empates son frecuentes, y cuál gana depende de ese orden.

Medido ejecutando el algoritmo hacia delante sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv` (clase `Bronchitis`) en 11 procesos Java distintos. El orden de los pasos cambia de una ejecución a otra (todos con exactitud 0,845799), por ejemplo:

```
ejecución 1: Dyspnea, Tuberculosis, TuberculosisOrCancer, LungCancer, VisitToAsia, X-ray
ejecución 2: Dyspnea, VisitToAsia, TuberculosisOrCancer, X-ray, Tuberculosis, LungCancer
```

Y el modelo final también cambia. En 8 ejecuciones seguidas salieron cuatro modelos:

```
5 veces: Dyspnea, LungCancer, Tuberculosis, TuberculosisOrCancer, VisitToAsia, X-ray   (sin
↪  Smoker)
```

```
1 vez:   Dyspnea, Smoker, TuberculosisOrCancer, VisitToAsia, X-ray
1 vez:   Dyspnea, LungCancer, Smoker, Tuberculosis, VisitToAsia
1 vez:   Dyspnea, Smoker, Tuberculosis, VisitToAsia, X-ray
```

Hacia atrás las candidatas son `getRootNode().getChildren()`, una lista con el orden de inserción, y en tres ejecuciones salió siempre lo mismo: quita Smoker, TuberculosisOrCancer, Tuberculosis, LungCancer, VisitToAsia y X-ray, y deja solo Dyspnea.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/snb/SelectiveNBAlgorithm.java:67-89

```
Collection<Node> candidates = (!forward) ? getRootNode().getChildren() :
        getNonRootNodes().stream()
                         .filter(n -> !getRootNode().getChildren().contains(n))
                         .collect(Collectors.toSet());

for (Node n1 : candidates) {
    ...
    double addScore = metric.getScore(edit);
    if (!isEditAlreadyConsidered(edit) && !isBlocked(edit)
            && (!onlyAllowedEdits || isAllowed(edit))
            && (addScore >= bestPartialScore || !onlyPositiveEdits)
    ) {
        bestEdit = edit;
        bestPartialScore = addScore;
    }
}
```

**Cómo arreglarlo.** Recorrer las candidatas en un orden determinista (una lista en el orden de la base de casos, o ordenada por nombre) en las dos direcciones, y fijar una regla de desempate explícita: por ejemplo, la primera en ese orden, con `>` en vez de `>=`. Hay que tener en cuenta que `>=` también sirve hoy para aceptar pasos que no empeoran la exactitud; si se cambia, hay que conservar ese criterio con otra condición.

El mismo tipo de dependencia del orden de un `HashMap` está en `MaximumWeightSpanningTree.build` (ver error 136).

Prueba de regresión: ejecutar el algoritmo dos veces sobre la misma base y comprobar que los pasos y el modelo final coinciden. Como el orden de un `HashSet` puede coincidir dentro de un mismo proceso, conviene crear las variables en órdenes distintos en las dos ejecuciones.

### Error 136 · [Corregido] El clasificador de árbol aumentado devuelve un modelo con los enlaces dirigidos de otra manera cada vez que se aprende

| | |
|---|---|
| **Estado** | Corregido en el commit `9e9d4aa` (09-10-2026): la raíz del árbol es la primera característica de la base de casos (`getRootOfTheTree`), y `MaximumWeightSpanningTree.build` toma las parejas con el mismo peso en el orden de las características. La segunda fuente de variación, que la ficha daba sin medir, se midió antes del arreglo: con la raíz fija y cuatro características con el mismo peso entre todas salieron 16 árboles distintos en 200 aprendizajes. No se ha aprendido desde la ventana |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.treeaugmentednb |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/treeaugmentednb/TreeAugmentedNBAlgorithm.java:34-56`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/common/MaximumWeightSpanningTree.java:38-100` |

| | |
|---|---|
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir qué regla fija la raíz del árbol. Manuel Arias lo decidió el 09-10-2026: la primera característica de la base de casos |
| **Relacionados** | error 4, error 77, error 135 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → pestaña general → tipo de algoritmo «discriminativo» → algoritmo «Tree augmented naive bayes», con aprendizaje automático o interactivo, y también desde la validación cruzada de `bnEvaluation` (`CrossValidationDialog` ofrece el mismo algoritmo). Cada vez que aprende sobre la misma base, la red tiene las mismas parejas unidas pero los enlaces dirigidos de otra manera, y nada lo advierte.

Lo que el clasificador responde sobre la clase apenas cambia: todas las orientaciones de un mismo árbol son equivalentes en lo que dicen sobre la clase, y con el suavizado de los parámetros puede haber diferencias mínimas en los números. Pero la red que el usuario ve, guarda y compara es distinta en cada aprendizaje.

El clasificador TAN (Tree Augmented Naive Bayes, Bayes ingenuo aumentado con un árbol) parte del Bayes ingenuo —la variable clase es padre de todas las demás, que se llaman características— y añade entre las características un árbol de peso máximo, donde el peso de cada pareja es su información mutua condicionada a la clase. Ese árbol se construye sin dirección y después hay que orientarlo: se elige una característica como raíz y todos los enlaces se dirigen alejándose de ella.

La raíz se elige al azar con un generador sembrado con el reloj, en `TreeAugmentedNBAlgorithm.getRandomVariable()`, llamado desde `init`. El algoritmo publicado (Friedman, Geiger y Goldszmidt, 1997) permite cualquier raíz; el problema no es la regla sino que no sea repetible.

Medido aprendiendo el TAN tres veces seguidas sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv` con `Bronchitis` como clase (enlaces entre características, ordenados):

```
run 0 [Dyspnea->TuberculosisOrCancer, Dyspnea->VisitToAsia, LungCancer->Smoker,
↪  TuberculosisOrCancer->LungCancer, TuberculosisOrCancer->Tuberculosis,
↪  TuberculosisOrCancer->X-ray]
run 1 [Dyspnea->VisitToAsia, LungCancer->Smoker, LungCancer->TuberculosisOrCancer,
↪  TuberculosisOrCancer->Dyspnea, TuberculosisOrCancer->Tuberculosis,
↪  TuberculosisOrCancer->X-ray]
run 2 [Dyspnea->VisitToAsia, LungCancer->Smoker, TuberculosisOrCancer->Dyspnea,
↪  TuberculosisOrCancer->LungCancer, TuberculosisOrCancer->Tuberculosis,
↪  TuberculosisOrCancer->X-ray]
```

Hay una segunda fuente posible de variación, no medida: `MaximumWeightSpanningTree.build` guarda los pesos en un `HashMap` cuyas claves son conjuntos de nodos, y `Node.hashCode` depende de `Variable`, que no redefine `hashCode` (usa la identidad del objeto). Si dos parejas empatan en peso, el orden en que el algoritmo de Kruskal las considera depende de las señas de memoria, y con él qué árbol sale. Con `BN-asia10k.csv` el árbol sin dirigir salió igual en las tres ejecuciones.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/-treeaugmentednb/TreeAugmentedNBAlgorithm.java:34-56

```java
@Override public void init(ModelNetUse modelNetUse) {
    ...
    if (!mwst.isBuilt()) {
        mwst.build(probNet, metric, getNonRootNodes());
        Variable randomRoot = getRandomVariable();
        mwst.redirect(randomRoot, probNet);
    }
    ...
}
```

```
protected Variable getRandomVariable() {
    return getNonRootVariables().get(new Random().nextInt(getNonRootVariables().size()));
}
```

**Cómo arreglarlo.** Sustituir el sorteo por una regla determinista (la que decida el equipo) en `getRandomVariable`, y renombrar el método si deja de sortear. Las pruebas `TreeAugmentedNBAlgorithmTest` y `GetRandomVariableRegressionTest` redefinen o llaman a ese método y habrá que ajustarlas.

Para que el resultado sea repetible del todo, `MaximumWeightSpanningTree.build` también debería desempatar de forma determinista: por ejemplo, ordenando por peso y, a igualdad, por los nombres de la pareja, y usando un `LinkedHashMap` o una lista en vez de un `HashMap` con claves `Set<Node>`.

La misma familia de problema aparece en `SelectiveNBAlgorithm` (ver error 135).

Prueba de regresión: aprender dos veces el TAN sobre `BN-asia10k.csv` con la misma clase y comprobar que los enlaces dirigidos coinciden.

### Error 137 · [Corregido] La maximización de la esperanza no devuelve lo aprendido para los nodos cuyo potencial es uniforme: siguen uniformes en la red resultante

| | |
|---|---|
| **Estado** | Corregido en el commit `3a9a269` (05-10-2026): `adaptNetwork` guarda las tablas creadas para sustituir a un `UniformPotential` y `parametricLearning`, al terminar, pone cada una en su nodo de `probNet`. Se pone directamente con `setPotential`, sin una edición que se pueda deshacer, como el resto del aprendizaje de parámetros. La prueba de regresión es la propuesta abajo: tras aprender, B tiene una tabla con las frecuencias de los casos. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.em |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/em/EMAlgorithm.java:119-235`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/em/EMAlgorithm.java:246-290`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:396-398` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** El usuario llega por el menú de aprendizaje → algoritmo EM → pestaña de red modelo → partir de la red modelo, con una red modelo en la que algún nodo tiene potencial uniforme. Recibe la red aprendida con esos nodos todavía uniformes, sin aviso.

El algoritmo EM (Expectation Maximization, maximización de la esperanza; «Expectation maximization (EM)» en el menú de aprendizaje) aprende los parámetros de una red con estructura fija, aunque falten datos.

`EMAlgorithm.parametricLearning` llama a `adaptNetwork`, que hace `probNet.copy()` —una copia superficial: nodos nuevos que comparten los mismos objetos potencial— y recoge en la lista `potentials` las tablas a aprender. En cada iteración escribe los números nuevos en `potential.getValues()`. Como para un `TablePotential` ese array es el mismo objeto que tiene la red original, lo aprendido llega a la red que se devuelve (`probNet`).

Un `UniformPotential` es el potencial «uniforme»: no es una tabla sino una clase hermana, y se usa cuando nadie ha escrito los números; lo crean, por ejemplo, `Potential.addVariable` al añadir un enlace y los ficheros `.pgmx` con `type="Uniform"`. Para un nodo con ese potencial, `adaptNetwork` construye un `TablePotential` nuevo, lo añade a `potentials` y lo pone en el nodo de **la copia** (`expandedNet`). El EM lo aprende, pero el nodo de `probNet` sigue con su `UniformPotential`, y `parametricLearning` devuelve `probNet`.

Medido con un pequeño programa de prueba: red A → B, A con una tabla y B con `UniformPotential`, 100 casos en los que B depende claramente de A, alfa 0. Tras `parametricLearning`, A tiene los valores aprendidos `[0.8, 0.2]` (el mismo objeto, actualizado) y B sigue siendo un `UniformPotential`.

La prueba existente `EMAlgorithmTests.parametricLearningWithUniformPotentialCptDoesNotThrow` solo comprueba que no se lanza excepción y dice expresamente que no mira los parámetros aprendidos.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/em/EMAlgorithm.java:246-261

```
private static ProbNet adaptNetwork(ProbNet probNet, List<TablePotential> potentials,
                                    Map<ICIPotential, List<TablePotential>>
                                    ↪  iciSubpotentials) {
    ProbNet expandedNet = probNet.copy();
    for (Potential potential : expandedNet.getPotentials()) {
        if (potential.getPotentialRole() == PotentialRole.CONDITIONAL_PROBABILITY) {
            switch (potential) {
                case UniformPotential uniformPotential -> {
                    ...
                    TablePotential newPotential = new
↪   TablePotential(uniformPotential.getVariables(),

↪   uniformPotential.getPotentialRole());
                    potentials.add(newPotential);

↪   expandedNet.getNode(potential.getVariable(0)).setPotential(newPotential);
                }
```

**Cómo arreglarlo.** Al terminar `parametricLearning`, poner en el nodo correspondiente de `probNet` cada tabla creada para sustituir a un `UniformPotential`. Para ello, guardar en `adaptNetwork` la correspondencia variable → tabla nueva, igual que ya se guarda `iciSubpotentials` para los modelos canónicos y se reinstala al final.

Si la instalación en la red devuelta debe poder deshacerse, hacerla con una edición.

Prueba de regresión: la misma red A → B con B uniforme; tras aprender, B debe tener un `TablePotential` con P(B | A) próximo a las frecuencias de los casos.

### Error 138 · [Corregido] El esqueleto que aprende el algoritmo PC depende de los nombres de las variables: cambiar el nombre de una columna de la base de casos añade o quita enlaces

| | |
|---|---|
| **Estado** | Corregido en el commit `efc5f82` (05-10-2026): `separationSetsLogic` trata cada pareja una sola vez en cada profundidad, y `evaluateSeparationSets` prueba los subconjuntos de los vecinos de X y los de los vecinos de Y, sin repetir los comunes, y guarda el mejor. Un empate exacto entre dos pruebas se sigue resolviendo por el orden de los nodos. La prueba de regresión es la de la red sintética de cinco variables (20 000 casos muestreados en la propia prueba): |

| | |
|---|---|
| | el esqueleto es el correcto con el nodo llamado «A» o «G»; antes del arreglo, con «A» sobraba A–B. Las medidas de abajo sobre `alarm10k.csv` y `alarm500.csv` no se han repetido, ni se ha medido cuántas pruebas de independencia más se hacen. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.pc |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/SkeletonDiscovery.java:18-27`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/SkeletonDiscovery.java:173-216`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/util/NodePair.java:38-51`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:412-425` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 139, error 140, nota 9 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → algoritmo PC (el que aprende la estructura de la red con pruebas de independencia condicional), en aprendizaje automático o interactivo, y también por la validación cruzada de `bnEvaluation` (`CrossValidationDialog`), que ofrece el mismo algoritmo. Con la misma base de casos, basta cambiar el nombre de una columna para que la red aprendida tenga enlaces de más o de menos. Nada lo advierte.

La primera fase del PC construye el esqueleto (los enlaces sin dirección). Parte del grafo completo y quita el enlace X–Y cuando encuentra un conjunto de variables que separa X de Y (una prueba de independencia condicional con p mayor que el nivel de significación). En el algoritmo publicado, en la profundidad d (el tamaño de los conjuntos que se prueban) se prueban los subconjuntos de d vecinos de X **y** los de d vecinos de Y. El javadoc de `SkeletonDiscovery` dice que sigue la variante PC-Stable y que el esqueleto no depende del orden.

`SkeletonDiscovery.separationSetsLogic` recorre los nodos por orden alfabético (`PCAlgorithm.sortedNodes`) y, para cada nodo X, sus vecinos Y, así que visita cada pareja dos veces: una desde cada extremo. La primera visita, desde el nodo cuyo nombre va antes, prueba los subconjuntos de vecinos de ese nodo y guarda el mejor resultado en la caché con la llave `NodePair`, que ordena los dos nodos por nombre y es la misma para (X, Y) y (Y, X).

En la segunda visita, desde el otro extremo, la condición `motivation.getSeparationSet().size() < adjacencySize` ya es falsa, porque lo guardado tiene tamaño d, y no se prueba nada. Los vecinos del segundo nodo solo se prueban en una profundidad en la que el primero no tiene bastantes vecinos. Si el conjunto que separa la pareja solo está entre los vecinos del segundo, el enlace falso se queda.

Medido llamando a las mismas clases que la ventana de aprendizaje con sus opciones por defecto (prueba de entropía cruzada, nivel de significación 0,05, alfa 0,5, prueba de dirección causal activada, sin red modelo, valores ausentes conservados y sin discretizar), y comparando los esqueletos por los nombres originales:

- Con una red sintética F→A, A→C, E→C, C→B, E→B y 20 000 casos, A y B quedan separados por {C, E} (p = 0,187), que son vecinos de B. Con el nodo llamado «A» ese conjunto no se prueba nunca (desde A solo se prueba {F, C}, con $p = 2{,}6 \cdot 10^{-38}$) y sobra el enlace A–B. Con el mismo nodo llamado «G», el esqueleto sale bien.
- `integrationTests/src/main/resources/networks/learning/alarm10k.csv` (37 variables,

10 000 casos): con los nombres del fichero el esqueleto tiene 44 enlaces; invirtiendo el orden alfabético de todos los nombres salen 48, con cinco de más (BP–STROKEVOLUME, DISCONNECT–KINKEDTUBE, HISTORY–PAP, MINVOL–TPR, MINVOL–VENTTUBE) y uno de menos (TPR–VENTMACH). De 74 cambios de nombre de una sola variable (anteponer «zz» o «AA»), 11 cambian el esqueleto.

- En esa misma base, con los nombres del fichero se queda TPR→VENTMACH: el mejor conjunto entre los vecinos de TPR, {ANAPHYLAXIS, BP, CATECHOL}, da $p = 3{,}5 \cdot 10^{-6}$, pero VENTTUBE, vecino de VENTMACH, los separa con $p = 0{,}146$ y no se prueba porque «TPR» va antes que «VENTMACH». Renombrando TPR como «zzTPR», el enlace desaparece.
- `integrationTests/src/main/resources/networks/learning/alarm500.csv` (500 casos): 35 enlaces con los nombres del fichero; invirtiendo el orden, 36 (sobra VENTLUNG–VENTTUBE); 6 de los 74 cambios de nombre de una variable cambian el esqueleto. Con VENTTUBE renombrada «AAVENTTUBE» en la cabecera del fichero, VENTLUNG–VENTTUBE se queda: desde los vecinos de AAVENTTUBE el mejor p es $2{,}6 \cdot 10^{-11}$, y {EXPCO2, VENTALV}, vecinos de VENTLUNG, da $p = 0{,}30$ pero no se prueba.
- En las otras 13 bases probadas (`learning.algorithm/src/test/resources/networks/BN-asia10k.csv`, `HeadToHead1.csv`, `HeadToHead2.csv`, `HeadToHead2-10k.csv`, `ThreeNodesAtoBtoC.csv`, `TwoNodesAtoB-1000.csv`; `integrationTests/src/main/resources/networks/learning/asia10K.csv`, `BN-A-B-C-E.csv`, `learnTestDataBase.csv`, `lineAndStar.xlsx`; `core/src/test/resources/nets/Cataract-RRDdatasetPreProc3.csv`, `Cataract-RRDdatasetPreProc4.csv`; `0_miscellaneous/networks/bn/BN-asia-5k.xls`) ningún cambio de nombre cambió el esqueleto. No se probaron las bases de `Cataract` con columnas numéricas sin discretizar ni la de Elvira (`.dbc`), que la ventana no abre.

El resultado es repetible: dos ejecuciones con los mismos nombres dan la misma red. Cambiar el nombre en la cabecera del fichero da lo mismo que cambiarlo después de leer la base.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/SkeletonDiscovery.java:173-192

```
private void separationSetsLogic(int adjacencySize) {
    for (Node nodeX : pc.sortedNodes()) {
        for (Node nodeY : PCAlgorithm.sorted(nodeX.getSiblings())) {
            List<Node> snapshotNeighbors = stableAdjSnapshot.getOrDefault(nodeX,
            ↪  Collections.emptyList());
            List<Node> adjacencySubset = new ArrayList<>(snapshotNeighbors);
            adjacencySubset.remove(nodeY);
            ...
            if (!pc.alreadyConsidered(removeLinkEdit, lastRemovedEdits)) {
                PCEditMotivation motivation = pc.cache.get(new NodePair(nodeX, nodeY));
                if (motivation == null || (motivation.getScore() != ALREADY_DONE
                        && motivation.getSeparationSet().size() < adjacencySize)) {
                    evaluateSeparationSets(nodeX, nodeY, adjacencySubset, adjacencySize);
                }
            }
        }
    }
}
```

learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/SkeletonDiscovery.java:199-216

```
private void evaluateSeparationSets(Node nodeX, Node nodeY,
                                    List<Node> adjacencySubset, int adjacencySize) {
    double bestScore = 0.0;
```

```
    List<Node> bestScoreSeparationSet = null;
    for (List<Node> separationSet : PCAlgorithm.subSetsOfSize(adjacencySubset,
    ↪  adjacencySize)) {
        double linkScore = pc.independenceTester.test(pc.database(), nodeX, nodeY,
        ↪  separationSet);
        ...
    }
    if (bestScoreSeparationSet != null) {
        pc.cache.put(new NodePair(nodeX, nodeY),
                new PCEditMotivation(bestScore, bestScoreSeparationSet));
    }
}
```

**Cómo arreglarlo.** En cada profundidad d, evaluar cada pareja una sola vez con los subconjuntos de tamaño d de los vecinos de X (sin Y) **y** de los vecinos de Y (sin X), tomados de la misma instantánea de vecinos, y guardar en la caché el mejor de los dos lados. Por ejemplo, recorrer solo las parejas con X antes que Y y pasar a `evaluateSeparationSets` las dos listas.

La caché tiene que saber que en la profundidad d se probaron los dos lados, para que la condición de `separationSetsLogic` no deje fuera a ninguno; hoy el tamaño del conjunto guardado hace ese papel y no basta.

El conjunto separador guardado es el que después leen `ColliderOrientation.findEdit` y `creates ContradictoryCollider` para orientar las figuras en cabeza, `onEditUndone` para repetir la prueba al deshacer e `invalidateNeighborCache` para descartar pruebas tras un borrado; todos deben seguir recibiendo el conjunto que separó la pareja, sea del lado que sea.

Probar los dos lados puede duplicar el número de pruebas de independencia en redes con muchos vecinos.

Pruebas de regresión: con la red sintética de cinco variables descrita arriba, el esqueleto debe ser el mismo con el nodo llamado «A» o «G»; con `alarm10k.csv`, renombrar TPR como «zzTPR» no debe cambiar el esqueleto; y, en general, aplicar a los nombres una permutación (por ejemplo, invertir su orden alfabético) no debe cambiar el esqueleto.

### Error 139 · El aprendizaje interactivo con PC ofrece las dos orientaciones de un mismo enlace, incluida la contraria a la que obliga una regla de Meek

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.pc |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:544-548`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/MeekOrientation.java:177-245`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/BaseLinkEdit.java:71-85`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:403-410` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si la lista interactiva debe ofrecer las dos direcciones de un enlace que ninguna regla orienta (hoy lo hace) o solo una |
| **Relacionados** | error 138, error 140, error 141, error 142 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → algoritmo PC (el que aprende la estructura de la red con pruebas de independencia condicional) → aprendizaje interactivo → pasar a la fase de orientación. La tabla de propuestas ofrece las dos direcciones de un mismo enlace.

Entre ellas aparece a veces, junto a una orientación obligada por una regla de Meek (las reglas que orientan los enlaces que quedan sin dirección), su contraria con el motivo inocuo «Do not create cycles». Si el usuario elige esa fila, la red aprendida contradice la regla sin aviso.

`PCAlgorithm.alreadyConsidered(edit, consideredEdits)` sirve para no proponer dos veces el mismo cambio sobre un enlace: pregunta si el cambio ya está en el conjunto de los considerados y si está «el mismo enlace al revés». Pero el cambio al revés lo construye siempre como `RemoveLinkEdit`, sea cual sea el tipo del cambio recibido, y `BaseLinkEdit.equals` exige que las dos clases sean iguales.

En la fase del esqueleto (`SkeletonDiscovery`), donde todo son borrados, funciona. En la fase de Meek (`MeekOrientation`), los cambios son `OrientLinkEdit` y el conjunto está lleno de orientaciones, así que la segunda mitad de la pregunta no puede dar «sí» nunca.

En aprendizaje automático no se nota, porque `run` pide cada vez solo la mejor propuesta y el historial se vacía. Sí se nota en el aprendizaje interactivo: `InteractiveLearningDialog.updateEditionsTable` llena la tabla llamando a `getBestEdit` y luego a `getNextEdit` hasta 50 veces, y cada llamada añade la propuesta al historial.

Para un enlace X–Z sin dirección, la regla de reserva (`tryOrientUnorientedWithoutPath`) propone primero X→Z; en la llamada siguiente X→Z ya está considerada, pero Z→X no se reconoce como «el mismo enlace al revés» y se propone también. Y lo mismo ocurre después de una orientación obligada por una regla de Meek: se ofrece la contraria como si fuera neutra.

La regla R1 orienta B→C cuando hay A→B y A no es vecino de C, porque la dirección contraria crearía una figura en cabeza que los datos no apoyan.

Medido con un pequeño programa de prueba que reproduce el llenado de la tabla (`getBestEdit(true, true)` y `getNextEdit(true, true)` hasta 50) sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv` con el PC, prueba de entropía cruzada y nivel de significación 0,05, ejecutando cada vez la primera propuesta. Casi todas las listas de la fase de orientación contienen las dos direcciones de cada enlace; en dos pasos aparecen juntas una orientación obligada y su contraria:

```
Orient link: TuberculosisOrCancer --> X-ray  {Meek R1}
```

```
...
Orient link: X-ray --> TuberculosisOrCancer  {Do not create cycles}

Orient link: Smoker --> LungCancer  {Meek R1}
Orient link: LungCancer --> Smoker  {Do not create cycles}
```

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:544-548

```
public boolean alreadyConsidered(BaseLinkEdit edit, Set<PNEdit> consideredEdits) {
    BaseLinkEdit inverseEdit = new RemoveLinkEdit(probNet, edit.getVariableTo(),
↪   edit.getVariableFrom(),
                                                  edit.isDirected());
    return consideredEdits.contains(edit) || consideredEdits.contains(inverseEdit);
}
```

core/src/main/java/org/openmarkov/core/action/base/linkEdits/BaseLinkEdit.java:75-79

```
@Override public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if ((obj == null) || (obj.getClass() != this.getClass()))
        return false;
    ...
```

**Cómo arreglarlo.** Si el equipo decide que no deben ofrecerse las dos direcciones del mismo enlace, construir el cambio al revés con la misma clase que el recibido (un `OrientLinkEdit` para una orientación, un `RemoveLinkEdit` para un borrado); por ejemplo, con un método por tipo en vez de uno genérico.

Los sitios que usan la comprobación son `SkeletonDiscovery.separationSetsLogic` e `isValidEdit` (borrados) y `MeekOrientation.buildOrientProposal` y `tryOrientDirection` (orientaciones).

Hay que revisar a la vez la reserva de `tryOrientUnorientedWithoutPath`, que intenta la dirección contraria cuando la preferida ya se propuso: con la comprobación arreglada, esa segunda llamada solo serviría cuando la preferida crea un ciclo o está bloqueada, que es lo que su motivo dice.

Si el equipo decide que las dos direcciones deben ofrecerse cuando ninguna regla obliga, al menos no debe ofrecerse la contraria de una orientación de Meek.

Prueba de regresión: llenar la lista como hace el diálogo sobre una red con A→B–C (A no vecino de C) y comprobar que no aparece C→B.

### Error 140 · [Corregido] En el aprendizaje interactivo con PC, llenar la tabla adelanta la fase del algoritmo: la tabla mezcla orientaciones con enlaces aún por quitar, «Complete phase» orienta sin quitar ningún enlace y marcar una casilla deja la tabla sin borrados o vacía

| | |
|---|---|
| **Estado** | Corregido en el commit `fc17e1e` (14-10-2026), con la decisión que tomó Manuel Arias ese día: la tabla enseña solo las propuestas de la fase en curso. `getNextEdit` no deja avanzar la fase (si la búsqueda la cambia, la repone y devuelve `null`), y `getBestEdit` arranca el esqueleto desde la profundidad 0, como ya se hacía tras aplicar un borrado; las pruebas de independencia siguen en la caché. Medido repitiendo las llamadas de la ventana: en `BN-asia10k.csv` la tabla inicial son 22 borrados, sin «Only positive edits» 27 y al volver a marcarla 22, siempre en la fase inicial; «Complete phase» tras llenar la tabla quita enlaces hasta dejar 7 y se para en la orientación de colisionadores (sin el arreglo orientaba los 28 del grafo completo); en `HeadToHead2.csv`, 5, 8 y 5 borrados, y |

| | |
|---|---|
| | «Complete phase» deja 5 enlaces (sin el arreglo no hacía nada). El aprendizaje automático da la misma red antes y después en las dos bases. Con prueba de esos casos en `learning.algorithm`; sin el arreglo fallan los cuatro. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.pc |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:151-239`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:288-387`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:92`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:298-312`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:354-361`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:403-427`<br>`learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java:108-122` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Si la tabla del aprendizaje interactivo debe enseñar solo las propuestas de la fase en curso o también las de las fases siguientes |
| **Relacionados** | error 134, error 138, error 139, error 141, error 142 |

**Qué pasa.** El usuario llega por el menú de aprendizaje → algoritmo PC (el que aprende la estructura de la red con pruebas de independencia condicional) → «Interactive learning» → «Learn». Se abre la ventana «Interactive learning» con la tabla de propuestas. En cuanto quedan menos de 50 enlaces por quitar (desde el primer momento en una base de pocas variables, y hacia el final del esqueleto en las demás):

- Debajo de los enlaces por quitar, la tabla ofrece orientaciones de enlaces que todavía no se sabe si se quedan, incluidos los que la misma tabla propone quitar más arriba.
- Si el usuario aplica una de esas orientaciones, ya no se le ofrece ningún borrado más, aunque quedaban.
- Si desmarca o vuelve a marcar «Only positive edits» u «Only allowed edits», o pulsa «Block edit» o «Show blocked», la tabla pierde todos los borrados; a veces se queda vacía, y entonces la ventana da el aprendizaje por terminado y desactiva «Apply edit» y «Complete phase».
- Si pulsa «Complete phase», el algoritmo orienta todos los enlaces que haya sin quitar ninguno, o no hace nada y da el aprendizaje por terminado.

El PC trabaja en fases: primero quita enlaces (esqueleto), después orienta figuras en cabeza (colisionadores) y por último orienta el resto con las reglas de Meek. `PCAlgorithm` guarda la fase en curso en el campo `phase`.

`InteractiveLearningDialog.updateEditionsTable`, al abrirse la ventana y cada vez que se actualiza, llama a `getBestEdit` y después a `getNextEdit` hasta tener 50 propuestas. Cada llamada va a `getOptimalEdit`, y cuando la fase en curso no tiene más propuestas, `transitionToNextPhase` **cambia `phase`** y sigue buscando en la siguiente, sin haber aplicado nada a la red. Al terminar de llenar la tabla, el algoritmo se ha quedado en la fase de orientación o en `ORIENTATION_FINISHED`.

La fase solo vuelve al principio al aplicar o deshacer un borrado, al añadir un enlace o al deshacer la orientación de un colisionador (`afterEditExecutes` y `afterUndoingEdit`); `getBestEdit` solo vacía el historial de propuestas, no la fase. Por eso todo lo que vuelve a llenar la tabla sin pasar

por esas ediciones (las dos casillas, «Block edit», «Show blocked») empieza desde la fase en que se quedó el llenado anterior. «Complete phase» llama a `LearningAlgorithm.runTillNextPhase`, que toma como fase de partida la adelantada y aplica propuestas mientras no cambie.

Aplicar una orientación de las reglas de Meek pone la fase en `REMAINING_LINKS_ORIENTATION`, y deshacerla la deja en `HEAD_TO_HEAD_ORIENTATION`, así que los borrados pendientes no vuelven a ofrecerse.

«Finish» no arrastra la fase adelantada, porque `learn` pasa por `init`, que la devuelve al principio (y que además descarta lo que el usuario había aplicado: ver error 141); pero con la tabla vacía la ventana ya no llama a `learn`.

Medido con un pequeño programa de prueba que construye el algoritmo como la ventana de aprendizaje (prueba de entropía cruzada, nivel de significación 0,05, alfa 0,5, prueba de dirección causal activada) y repite las llamadas de la ventana:

- `learning.algorithm/src/test/resources/network/BN-asia10k.csv` (8 variables): la tabla inicial tiene 50 filas: 22 borrados y, desde la fila 23, 28 orientaciones de enlaces del grafo completo; al terminar de llenarla la fase es la de las reglas de Meek. Desmarcar «Only positive edits» deja 50 orientaciones y ningún borrado, y volver a marcarla, lo mismo. Aplicar la fila 23 («Orient link: Dyspnea –> Bronchitis», motivo «Causal direction test (ANM)») deja la tabla siguiente con 50 orientaciones y ningún borrado. «Complete phase» nada más abrir deja la red con sus 28 enlaces, todos orientados, y la tabla vacía.
- `learning.algorithm/src/test/resources/network/HeadToHead2.csv` (5 variables): la tabla inicial tiene 25 filas (5 borrados y 20 orientaciones) y el llenado termina en `ORIENTATION_FINISHED`. Desmarcar «Only positive edits» deja la tabla vacía. «Complete phase» nada más abrir no hace nada y la tabla queda vacía, con la red en el grafo completo sin dirección (10 enlaces). Con la tabla vacía, la ventana marca el aprendizaje como terminado y «Finish» ya no aprende.
- `integrationTests/src/main/resources/networks/learning/alarm500.csv` (37 variables): la tabla inicial son 50 borrados y la fase no se mueve; ahí el adelanto aparece cuando quedan menos de 50 enlaces por quitar.

Solo se midió la casilla «Only positive edits»; «Only allowed edits», «Block edit» y «Show blocked» llaman al mismo `updateUI`. La ventana es un `JDialog` y no se ha abierto; lo que hace con la tabla vacía (botones desactivados, «Finish» sin efecto) está leído en `updateEditionsTable` y `btnRunActionPerformed`.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:193-200

```
public LearningEditProposal getOptimalEdit(boolean onlyAllowedEdits, boolean
↪  onlyPositiveEdits) {
    if (phase == Phase.INITIAL_PHASE) {
        LearningEditProposal proposal = skeleton.findEdit(onlyAllowedEdits,
↪  onlyPositiveEdits);
        if (proposal != null) {
            return proposal;
        }
        return transitionToNextPhase(onlyAllowedEdits);
    }
```

learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:367-372

```
private LearningEditProposal transitionToNextPhase(boolean onlyAllowedEdits) {
    return switch (phase) {
        case INITIAL_PHASE -> {
            phase = Phase.HEAD_TO_HEAD_ORIENTATION;
            yield getOrientationEdit(onlyAllowedEdits);
        }
```

learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:403-419

```
private void updateEditionsTable(boolean onlyAllowed, boolean onlyPositive,
                                 List<LearningEditProposal> blockedEdits) {
    ArrayList<LearningEditProposal> showableEditProposals = new ArrayList<>();
    LearningEditProposal bestEdition =
↪   learningManager.getLearningAlgorithm().getBestEdit(onlyAllowed, onlyPositive);
    while (showableEditProposals.size() < SHOWING_EDIT_NUM && bestEdition != null) {
        showableEditProposals.add(bestEdition);
        bestEdition = learningManager.getLearningAlgorithm().getNextEdit(onlyAllowed,
↪   onlyPositive);
    }
    editionsTable.fill(showableEditProposals);
    if (showableEditProposals.isEmpty()) {
        ...
        learningFinished = true;
    }
```

**Cómo arreglarlo.** Hecho con el primero de los dos caminos, como dice la fila *Estado*. Hay que separar la fase en que está la red (la que marcan las ediciones aplicadas) de la fase hasta la que se ha asomado el llenado de la tabla. Dos caminos, según lo que decida el equipo sobre qué debe enseñar la tabla:

- Si la tabla solo debe enseñar la fase en curso: que `getNextEdit` no pase de fase (o que `updateEditionsTable` deje de pedir cuando `getPhase()` cambie), y que la fase avance solo cuando la fase en curso se quede sin propuestas al pedir la mejor.
- Si debe enseñar también las fases siguientes: que el llenado no deje rastro, guardando `phase` (y el estado de `SkeletonDiscovery`: profundidad e instantánea de vecinos) antes de pedir la primera propuesta y reponiéndolo al terminar; por ejemplo, con un método del algoritmo que devuelva de una vez la lista de propuestas. Las orientaciones que se ofrezcan con borrados pendientes deben quedar claramente separadas, porque se calculan sobre un esqueleto que aún no es el final.

Los sitios que asignan `phase` son `getOptimalEdit`, `getOrientationEdit`, `findBestEditInCurrentPhase`, `transitionToNextPhase`, `init`, `afterEditExecutes` y `afterUndoingEdit`. Los comentarios de `init` y de `afterEditExecutes` ya describen el adelanto y lo compensan solo tras una edición; con el arreglo esas compensaciones deben revisarse. `LearningAlgorithm.runTillNextPhase` («Complete phase») debe partir de la fase real.

Pruebas de regresión, repitiendo las llamadas de la ventana sobre `BN-asia10k.csv`: tras llenar la tabla, un segundo llenado con otras casillas sigue ofreciendo los 22 borrados; «Complete phase» nada más abrir quita enlaces y no orienta ninguno; y con `HeadToHead2.csv` un segundo llenado no sale vacío.

### Error 141 · En el aprendizaje interactivo con PC, «Finish» vuelve a aprender desde el grafo completo y descarta los borrados y orientaciones que el usuario ya había aplicado, aunque su ayuda dice que sigue desde el estado actual

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.pc |
| **Ficheros** | `learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:208-219`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:343-352`<br>`learning.gui/src/main/resources/learning/gui/localize/Learning_en.xml:67-69`<br>`learning.core/src/main/java/org/openmarkov/learning/core/LearningManager.java:107-109`<br>`learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java:88-121`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:127-139`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/IndependenceRelationsAlgorithm.java:51-77`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningController.java:83`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:919-926` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Si «Finish» debe seguir desde la red tal como la ha dejado el usuario, como dice su ayuda emergente, o volver a aprender desde el principio |
| **Relacionados** | error 133, error 134, error 139, error 140, error 142 |

**Qué pasa.** El usuario llega por Herramientas («Tools») → «Learning» → algoritmo PC (el que aprende la estructura de la red con pruebas de independencia condicional) → «Interactive learning», que es la opción marcada por omisión → «Learn». En la ventana «Interactive learning» aplica propuestas de la tabla con «Apply edit»: quita enlaces o elige la orientación de otros. La ventana no es modal y la red aprendida está abierta en el editor, así que también puede borrar un enlace en el lienzo. Al final pulsa «Finish», cuya ayuda emergente dice «Launches the algorithm from the current state». La ventana se cierra y la red que queda es la del aprendizaje automático hecho desde el principio: lo que el usuario decidió a mano se pierde, sin aviso.

`InteractiveLearningDialog.btnRunActionPerformed` llama a `LearningManager.learn`, que llama a `LearningAlgorithm.run`. `run` empieza por `init(modelNetUse)`, después aplica la mejor propuesta mientras haya y termina con el aprendizaje de parámetros.

Para el PC, `init` es `PCAlgorithm.init`: devuelve la fase al principio, vacía la caché de pruebas de independencia, el estado del esqueleto y el historial de propuestas, y antes llama a `IndependenceRelationsAlgorithm.init`. Este quita todos los enlaces de la red y une todos los nodos entre sí (`probNet.marry`), es decir, vuelve al grafo completo sin dirección. Lo hace sin red modelo, y con red modelo cuando están marcadas «Use the information of the nodes» (queda marcada al cargar la red modelo) o «Allow link addition». Si con la red modelo solo se permite borrar o invertir enlaces, conserva los enlaces que haya pero les quita la dirección, con lo que se pierden las orientaciones.

Ese mismo `init` ya se ejecutó antes de abrir la ventana (`LearningController.initLearning` llama

a `LearningManager.init`), así que la segunda llamada no prepara nada nuevo: solo deshace lo hecho. Lo único que no toca es la lista de propuestas bloqueadas. «Complete phase» (`LearningAlgorithm.runTillNextPhase`) no pasa por `init`.

El comentario de `PCAlgorithm.init` presenta «Finish» como una ejecución nueva que no debe arrastrar la fase adelantada por el llenado de la tabla (error 140); la ayuda del botón promete lo contrario.

Medido con un pequeño programa de prueba que construye el PC como la ventana de aprendizaje con sus opciones por defecto (prueba de entropía cruzada, nivel de significación 0,05, alfa 0,5, prueba de dirección causal activada), repite las llamadas de la ventana (llenar la tabla tras cada cambio, aplicar con `LearningManager.applyEdit`) y termina llamando a `learn` como «Finish», sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv`:

- El aprendizaje automático da 7 enlaces: Bronchitis→Dyspnea, Bronchitis→Smoker, LungCancer→TuberculosisOrCancer, Smoker→LungCancer, Tuberculosis→TuberculosisOrCancer, TuberculosisOrCancer→Dyspnea y TuberculosisOrCancer→X-ray.
- Aplicando a mano los 21 borrados que ofrece la tabla (siempre la primera fila) y después la fila 3, «Orient link: Dyspnea –> Bronchitis» (motivo «Causal direction test (ANM)»), la red queda con el esqueleto de 7 enlaces y Dyspnea→Bronchitis. Tras «Finish» la red es exactamente la del aprendizaje automático, con Bronchitis→Dyspnea.
- Borrando nada más abrir el enlace Smoker–LungCancer, con la misma edición `RemoveLinkEdit` que construye el editor al borrar un enlace, tras «Finish» el enlace vuelve (Smoker→LungCancer) y la red es otra vez la del aprendizaje automático.

La ventana es un `JDialog` y no se ha abierto: que el botón llama a `learn` y después se cierra está leído en `btnRunActionPerformed`.

**El código.** learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:343-352

```
private void btnRunActionPerformed(ActionEvent evt) throws ... {
    try {
        if (!learningFinished) {
            learningManager.learn();
            this.setVisible(false);
        }
    } finally {
        this.setVisible(false);
    }
}
```

learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java:88-92

```
public void run(ModelNetUse modelNetUse) throws ... {
    init(modelNetUse);
    /* Main loop */
    LearningEditProposal bestEdition = getBestEdit(true, true);
    while (bestEdition != null) {
```

learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/IndependenceRelationsAlgorithm.java:51-75

```
@Override public void init(ModelNetUse modelNetUse) {
    if (modelNetUse != null && (modelNetUse.isLinkAdditionAllowed() ||
    ↪  modelNetUse.isUseNodePositions())) {
```

```
        for (Link<Node> link : probNet.getLinks()) {
            probNet.removeLink(link);
        }
        probNet.marry(probNet.getNodes());
    }
    else if (modelNetUse != null && (modelNetUse.isLinkRemovalAllowed() ||
    ↪  modelNetUse.isLinkInversionAllowed())) {
        ...
    } else {
        for (Link<Node> link : probNet.getLinks()) {
            probNet.removeLink(link);
        }
        probNet.marry(probNet.getNodes());
    }
```

learning.gui/src/main/resources/learning/gui/localize/Learning_en.xml:67-69

```
<Run value="Finish">
    <ToolTip value="Launches the algorithm from the current state"/>
</Run>
```

**Cómo arreglarlo.** Depende de lo que decida el equipo sobre «Finish»:

- Si debe seguir desde la red tal como está, como dice su ayuda: que el botón no pase por `init`. Por ejemplo, un método de `LearningAlgorithm` que haga el bucle de `run` y el aprendizaje de parámetros sin llamar a `init`, y que `LearningManager` lo ofrezca para el aprendizaje interactivo. Para el PC hay que partir entonces de la fase real y no de la adelantada por el llenado de la tabla: hoy es `init` quien la corrige, así que este arreglo va junto con el de error 140. La caché de pruebas sigue valiendo, porque los borrados aplicados quedan marcados como hechos.
- Si debe volver a empezar: que el rótulo y la ayuda lo digan, y que la ventana avise antes de descartar lo aplicado.

`LearningManager.learn` lo usan también el aprendizaje automático de `LearningDialog` y la validación cruzada de `bnEvaluation` (`LearningEvaluator`), donde la red está recién creada y `init` no descarta nada; el cambio no debe alterarlos.

Los demás algoritmos que ofrecen aprendizaje interactivo también vuelven a ejecutar su `init` al pulsar «Finish» (por ejemplo, `SuperParentNBAlgorithm.init` vuelve a llenar su lista de características huérfanas y `HillClimbingAlgorithm.init` vuelve a añadir la restricción de número máximo de padres, si se fijó). No se ha medido qué efecto tiene en cada uno, pero la regla que se elija debe cumplirse para todos.

Prueba de regresión, repitiendo las llamadas de la ventana sobre `BN-asia10k.csv`: aplicar los 21 borrados y la orientación Dyspnea→Bronchitis y llamar al método que use «Finish». Si se decide seguir desde el estado actual, la red final debe conservar Dyspnea→Bronchitis; y, borrando a mano Smoker–LungCancer, el enlace no debe volver.

### Error 142 · [Corregido] En el aprendizaje interactivo con PC, después de bloquear una propuesta de borrado, «Finish» termina con un NullPointerException y deja la red a medio aprender; en bases con muchas variables fallan también «Complete phase» y «Apply edit»

| | |
|---|---|
| **Estado** | Corregido en el commit `63f3ebc` (05-10-2026): `SkeletonDiscovery.isValidEdit` pregunta por la edición sola, con `isBlocked(PNEdit)`, y también por la inversa: con solo la primera, el borrado bloqueado X–Y se proponía al recorrer la pareja desde Y, porque ya no entraba en la lista de borrados considerados que antes lo tapaba. `LearningEditProposal.equals` admite un motivo nulo, y `btnNextPhaseActionPerformed` vuelve a añadir la ventana como oyente en un `finally`; esto último solo está comprobado leyendo. Las orientaciones (`ColliderOrientation`, `MeekOrientation` y `PCAlgorithm.getNextEdit`) siguen comparando edición y motivo. La prueba de regresión usa la red sintética de cinco variables del error 138, no `BN-asia10k.csv` ni `alarm500.csv`: llena la tabla, bloquea cada borrado por turno, reinicia el algoritmo como «Finish» y lo ejecuta hasta el final, que no falla y conserva el enlace. «Complete phase» y «Apply edit» no tienen prueba propia. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.pc |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/SkeletonDiscovery.java:221-269`<br>`learning.core/src/main/java/org/openmarkov/learning/core/util/LearningEditProposal.java:70-83`<br>`learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java:88-122`<br>`learning.core/src/main/java/org/openmarkov/learning/core/algorithm/LearningAlgorithm.java:190-233`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:170-180`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:405`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:208-218`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:327-361` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 139, error 140, error 141 |

**Qué pasa.** El usuario llega por Herramientas («Tools») → «Learning» → algoritmo PC (el que aprende la estructura de la red con pruebas de independencia condicional) → «Interactive learning», que es la opción marcada por omisión → «Learn». En la ventana «Interactive learning» selecciona una propuesta de borrado («Remove link between …») y pulsa «Block edit», para que el algoritmo no quite ese enlace. Si después pulsa «Finish», sale el diálogo de «error que un desarrollador no previó» con un `NullPointerException`, la ventana interactiva se cierra y la red queda en el editor a medio aprender: con parte de los enlaces del grafo completo, ninguno orientado y sin el aprendizaje de parámetros del final.

En una base con muchas variables falla igual «Complete phase», y también seguir aplicando propuestas con «Apply edit», en cuanto se han quitado los enlaces que el algoritmo mira antes que el bloqueado. Bloquear una propuesta de orientación no falla.

Por qué pasa: para buscar el siguiente borrado, `SkeletonDiscovery.getOptimalEditFromCache` recorre las parejas de nodos por orden alfabético y, para cada candidata, `isValidEdit` pregunta si está bloqueada construyendo `new LearningEditProposal(removeLinkEdit, bestMotivation)`. El motivo que le pasa no es el de esa pareja, sino el de la mejor candidata encontrada hasta ese momento en el recorrido, que vale `null` para la primera. `LearningAlgorithm.isBlocked(LearningEditProposal)` la busca con `blockedEdits.contains`, que llama al `equals` de la propuesta nueva contra cada bloqueada; `LearningEditProposal.equals` compara primero las ediciones y, si son la misma, llama a `this.motivation.equals(...)` con el motivo `null`.

Falla, por tanto, cuando el borrado bloqueado es la primera candidata de un recorrido, lo que ocurre en cuanto las parejas que van antes por orden alfabético ya se han quitado o no cumplen las condiciones. Por eso no falla al bloquear, porque llenar la tabla no quita enlaces. «Finish» vuelve a aprender desde el grafo completo (error 141) y va quitando enlaces hasta llegar al bloqueado; «Complete phase» (`runTillNextPhase`) y «Apply edit» también llegan, si el algoritmo sigue en la fase de quitar enlaces. En las bases pequeñas llenar la tabla ya ha llevado el algoritmo a la fase de orientación (error 140), así que «Complete phase» no pasa por los borrados; y con `HeadToHead2.csv` la tabla se queda vacía al bloquear y la ventana ya no llama a `learn`.

Cuando `bestMotivation` no es `null`, la misma pregunta compara el borrado bloqueado con el valor p de otra pareja y no lo reconoce; lo que evita que se proponga es la segunda comprobación, en `PCAlgorithm.getNextEdit`, que sí usa el motivo de la propuesta.

Tras el fallo, `btnRunActionPerformed` («Finish») esconde la ventana en su `finally`, y el manejador global (`OMExceptionHandler`) presenta cualquier `RuntimeException` como error no previsto. En «Complete phase», `btnNextPhaseActionPerformed` quita la ventana de la lista de oyentes de las ediciones de la red antes de llamar a `runTillNextPhase` y la vuelve a añadir después, sin `finally`: por lectura, tras el fallo la ventana sigue abierta con la tabla vieja y ya no se actualiza.

Medido con un pequeño programa de prueba que construye el PC como la ventana de aprendizaje con sus opciones por defecto (prueba de entropía cruzada, nivel de significación 0,05, alfa 0,5, prueba de dirección causal activada), llena la tabla como la ventana (`getBestEdit` y después `getNextEdit` hasta 50 filas), bloquea una propuesta con `blockEdit`, vuelve a llenar la tabla como hace la ventana tras «Block edit», y después llama a lo mismo que «Finish» (`LearningManager.learn`) o «Complete phase» (`runTillNextPhase`), o aplica cada vez la primera propuesta con `LearningManager.applyEdit`:

- `learning.algorithm/src/test/resources/network/BN-asia10k.csv` (8 variables; la tabla inicial tiene 22 borrados y 28 orientaciones): bloqueando cualquiera de 21 de los 22 borrados, «Finish» lanza `NullPointerException: Cannot invoke "Object.equals(Object)" because "this.motivation" is null` en `LearningEditProposal.equals`, llamado desde `SkeletonDiscovery.isValidEdit`, y la red se queda con entre 8 y 28 enlaces sin dirección (el grafo completo tiene 28; el aprendizaje automático da 7, todos orientados). El otro borrado, LungCancer–Smoker, es un enlace que el aprendizaje automático no quita, y con él «Finish» termina bien. Bloqueando cualquiera de las 28 orientaciones, «Finish» termina bien.
- `integrationTests/src/main/resources/networks/learning/alarm500.csv` (37 variables; la tabla inicial son 50 borrados): bloqueando cualquiera de los 50, «Finish» y «Complete phase» lanzan la misma excepción; tras «Finish» la red se queda con 220 o 235 enlaces sin dirección de los 666 del grafo completo. Bloqueando el primero y aplicando cada vez la primera propuesta, la excepción salta al volver a llenar la tabla tras 396 borrados aplicados.
- `learning.algorithm/src/test/resources/network/HeadToHead2.csv`: tras bloquear, la tabla queda vacía y la ventana ya no llama a `learn`, así que no falla.

La ventana es un `JDialog` y no se ha abierto: lo que enseña tras la excepción está leído en su código y en `OMExceptionHandler`.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/SkeletonDiscovery.java:232-238

```
RemoveLinkEdit removeLinkEdit =
        new RemoveLinkEdit(pc.net(), nodeX.getVariable(), nodeY.getVariable(), false);

if (isValidEdit(removeLinkEdit, bestMotivation, onlyAllowedEdits)) {
    bestMotivation = motivation;
    bestEditProposal = new LearningEditProposal(removeLinkEdit, motivation);
}
```

learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/SkeletonDiscovery.java:264-269

```
private boolean isValidEdit(RemoveLinkEdit removeLinkEdit, PCEditMotivation bestMotivation,
                            boolean onlyAllowedEdits) {
    return !pc.checkBlocked(new LearningEditProposal(removeLinkEdit, bestMotivation))
            && !pc.alreadyConsidered(removeLinkEdit, lastRemovedEdits)
            && (!onlyAllowedEdits || pc.checkAllowed(removeLinkEdit));
}
```

learning.core/src/main/java/org/openmarkov/learning/core/util/LearningEditProposal.java:70-78

```
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if ((obj == null) || (obj.getClass() != this.getClass()))
        return false;
    LearningEditProposal other = (LearningEditProposal) obj;
    return this.edit.equals(other.edit) && this.motivation.equals(other.motivation);
}
```

**Cómo arreglarlo.** En `SkeletonDiscovery.isValidEdit`, preguntar si el borrado está bloqueado por la edición sola, con `pc.isBlocked(removeLinkEdit)`: la variante `LearningAlgorithm.isBlocked(PNEdit)` (líneas 226-233) compara solo las ediciones y es la que ya usan el ascenso de colinas y los clasificadores. Así no se construye una propuesta con el motivo de otra pareja.

Hacer que `LearningEditProposal.equals` admita un motivo nulo (con `Objects.equals`, como ya hace `hashCode`) quitaría la excepción, pero dejaría la pregunta comparando con el motivo de otra pareja.

Los sitios del PC que preguntan si una propuesta está bloqueada son `SkeletonDiscovery.isValidEdit`, `PCAlgorithm.getNextEdit` (línea 178), `ColliderOrientation` (línea 141) y `MeekOrientation` (líneas 220 y 239); los cuatro últimos comparan edición y motivo a la vez. Conviene que todos usen el mismo criterio.

En `InteractiveLearningDialog.btnNextPhaseActionPerformed`, volver a añadir la ventana como oyente en un `finally`, para que un fallo no la deje sin actualizar.

Pruebas de regresión, repitiendo las llamadas de la ventana: con `BN-asia10k.csv`, llenar la tabla, bloquear el borrado Tuberculosis–VisitToAsia y llamar a `learn`, que no debe lanzar y debe dejar ese enlace en la red; con `alarm500.csv`, bloquear el primer borrado de la tabla y llamar a `runTillNextPhase`, que tampoco debe lanzar ni quitar el enlace bloqueado.

### Error 143 · [Corregido] El clasificador de bosque aumentado lanza un puntero nulo cuando ninguna característica tiene información mutua positiva con la clase (por ejemplo, con una base de una sola columna)

| | |
|---|---|
| **Estado** | Corregido en el commit `3b2da4f` (05-10-2026): `getOptimalEdit` devuelve `null` si `getBestRootForSubtree` no encuentra raíz, sin asignar `subtreeRoot`, y `computeAveragedConditionalMutualInformation` da 0 con menos de dos características. Las dos pruebas de regresión construyen la base en la propia prueba (solo la clase, y la clase con una característica constante), en vez de leer `OneNode-1000.csv`. |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.fanb |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/fanb/ForestAugmentedNBAlgorithm.java:85-157` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** El usuario llega por el menú de aprendizaje → tipo discriminativo → «Forest augmented naive bayes», con una base cuyas características no tienen relación con la clase, o marcando como variables solo la clase. Al pulsar el botón de aprender, después de haberse abierto ya la ventana de la red nueva, sale una `NullPointerException` que recoge el manejador general de excepciones de la aplicación.

El clasificador FAN (Forest Augmented Naive Bayes, Bayes ingenuo aumentado con un bosque) empieza eligiendo la característica de la que colgar el primer árbol. `getBestRootForSubtree` busca la de mayor información mutua con la clase, pero parte de `bestScore = 0.0` y exige `addScore > bestScore`, así que devuelve `null` si no hay características o si todas tienen información mutua exactamente cero con la clase. `getOptimalEdit` hace a continuación `bestEdit.getVariableTo()` sin comprobarlo.

`KDBAlgorithm.getOptimalEdit`, con una estructura parecida, sí comprueba `bestEdit != null`.

Medido con un pequeño programa de prueba:

```
OneNode-1000.csv (una sola columna, que es la clase)
  -> java.lang.NullPointerException: Cannot invoke "...BaseLinkEdit.getVariableTo()"
  ↪  because "bestEdit" is null
     at ...ForestAugmentedNBAlgorithm.getOptimalEdit(ForestAugmentedNBAlgorithm.java:95)
base sintética de 100 casos: clase binaria + una característica con un único estado
  -> la misma excepción en la misma línea
```

El fichero `OneNode-1000.csv` es `learning.algorithm/src/test/resources/network/OneNode-1000.csv`. El segundo caso muestra que no hace falta una base de una columna: basta con que ninguna característica esté relacionada con la clase en la muestra (por ejemplo, columnas constantes).

Además, con menos de dos características, `computeAveragedConditionalMutualInformation` divide por `n·(n−1) = 0` y el umbral sale `NaN`, aunque eso por sí solo no produce error (ninguna comparación con `NaN` es cierta).

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/fanb/ForestAugmentedNBAlgorithm.java:93-97

```java
if (subtreeRoot == null) {
    bestEdit = getBestRootForSubtree();
    subtreeRoot = probNet.getNode(bestEdit.getVariableTo());
```

```
    mwst.redirect(subtreeRoot.getVariable(), probNet);
    bestPartialScore = unconditionedMetric.getScore(bestEdit);
```

**Cómo arreglarlo.** Si `getBestRootForSubtree` devuelve `null`, no proponer nada (devolver `null`, como hacen los demás clasificadores de la familia cuando no tienen propuesta, por ejemplo `KDBAlgorithm`), sin asignar `subtreeRoot`.

Proteger también la división de `computeAveragedConditionalMutualInformation` para menos de dos características.

Prueba de regresión: aprender FAN con `OneNode-1000.csv` y con una base cuya única característica es constante; el resultado debe ser el Bayes ingenuo (o la clase sola) sin excepción.

### Error 144 · [Corregido] Discretizar por frecuencias iguales una columna numérica escrita con decimales o notación científica lanza una excepción o cuenta mal los casos

| | |
|---|---|
| **Estado** | Corregido en el commit `0e77ce6` (05-10-2026): `discretizeEqualFreq` guarda cada valor numérico con su número de casos en un mapa ordenado y ya no busca el estado por su nombre; dos nombres del mismo número suman sus casos. La primera prueba de regresión no lee `Cataract-Data2.csv`, que las pruebas de `learning.core` no alcanzan, sino una variable con estados escritos como los que fallaban («1000000000.0», «1.00E+09», «1.000291517E9»); la segunda es la de «1», «1.50» y «3», que da el corte en 1,5. |
| **Módulo** | learning.core |
| **Paquete** | org.openmarkov.learning.core.preprocess |
| **Ficheros** | `learning.core/src/main/java/org/openmarkov/learning/core/preprocess/Discretization.java:271-358`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1213-1214`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/DataPreprocessor.java:78-83` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 54 |

**Qué pasa.** El usuario llega por el preprocesado de datos de `bnEvaluation` (`DataPreprocessingDialog` → `DataPreprocessor.process`), eligiendo la discretización «Equal frequency intervals» para una variable numérica de un CSV (valores separados por comas) con valores como los de más abajo; el diálogo solo habilita la discretización para variables cuyos estados son todos números, y estas lo son. La ventana de aprender una red tiene los mismos controles, pero su pestaña de preprocesado no se añade a la ventana (`LearningDialog.java`, línea 544, comentada), así que por ahí no se llega. Sale una `ArrayIndexOutOfBoundsException` al guardar el preprocesado, o, sin ningún aviso, unos intervalos distintos de los que corresponden a los datos.

La discretización por «frecuencias iguales» convierte una variable numérica (sus estados son los valores leídos del fichero, por ejemplo «1000000000.0») en otra de intervalos que contienen aproximadamente el mismo número de casos.

`Discretization.discretizeEqualFreq` convierte los nombres de los estados en números y los ordena, pero al ordenarlos pierde a qué estado pertenece cada número. Para saber cuántos casos tiene cada valor vuelve a buscar el estado por su nombre escribiendo el número: con `Double.toString` o, si contiene «E», con un `DecimalFormat("###.########")`, y si no lo encuentra, con la

parte entera.

Eso solo funciona si alguna de esas escrituras reproduce exactamente el texto del fichero. Si ninguna lo hace, `stateIndex` vale −1 y `histogram[stateIndex]` lanza `ArrayIndexOutOfBounds Exception`.

Medido aplicando la discretización por frecuencias iguales (3 intervalos) a cada variable numérica de las bases CSV de los recursos de prueba, en `core/src/test/resources/nets/`:

```
Cataract-Data2.csv               var=id  -> ArrayIndexOutOfBoundsException: Index -1 out of
↪  bounds for length 537  (estado «1000000000.0», toString «1.0E9»)
Cataract-RRDdatasetPreProc1.csv  var=ID  -> la misma excepción (estado «1.000291517E9»)
Cataract-RRDdatasetPreProc2.csv  var=ID  -> la misma excepción (estado «1.00E+09»)
```

Hay además un caso silencioso: si la escritura no coincide pero la parte entera sí coincide con **otro** estado, se usa la frecuencia de ese otro estado. Medido con una variable de estados «1», «1.50» y «3» con 10, 80 y 10 casos, pidiendo 2 intervalos: al buscar 1,5 se encuentra «1» y se suma 10 en vez de 80, no se alcanza la frecuencia objetivo y sale un único intervalo `(-Infinity , Infinity)` con los 100 casos, cuando con las cuentas correctas habría un corte en 1,5.

**El código.** learning.core/src/main/java/org/openmarkov/learning/core/preprocess/Discretization.java:280-321

```
List<Double> orderedStates = new ArrayList<Double>();
for (int i = 0; i < states.length; i++) {
    if (!states[i].getName().equals("?"))
        orderedStates.add(Double.parseDouble(states[i].getName()));
}
Collections.sort(orderedStates);
...
for (Double state : orderedStates) {
    stateName = state.toString();
    String stateToSearch = stateName.contains("E") ?
            decimalFormat.format(state.doubleValue()) :
            state.toString();
    stateIndex = variable.getStateIndex(stateToSearch);
    if (stateIndex == -1) {
        stateIndex = variable.getStateIndex("" + state.intValue());
    }
    stateFreq = histogram[stateIndex];
```

**Cómo arreglarlo.** No volver a buscar el estado: construir pares (valor numérico, índice del estado) al leer los nombres, ordenar los pares por valor y tomar la frecuencia con el índice guardado. Si dos nombres distintos representan el mismo número («2» y «2.0»), sus frecuencias deben sumarse en el mismo valor.

Las demás discretizaciones (`discretizeMDLP`, `discretizeChiMerge`, `discretizeKMeans`) recorren los casos y leen el nombre del estado de cada caso, así que no tienen este problema.

Pruebas de regresión: discretizar por frecuencias iguales la columna `id` de `Cataract-Data2.csv` sin excepción; y la variable sintética de estados «1», «1.50», «3» (10, 80, 10 casos, 2 intervalos) debe dar un corte en 1,5.

### Error 145 · [Corregido] Dividir un banco de casos en entrenamiento y prueba desde «Split Dataset» lanza siempre una excepción, elija el usuario la opción que elija

| | |
|---|---|
| **Estado** | Corregido en el commit `8750f28` (05-10-2026): los tres métodos de `SplitSetManager` declaran `List<Variable>` sin conversión; ninguno modifica la lista. Las pruebas de regresión son las propuestas abajo: los primeros y los últimos casos, el reparto al azar y `SplitSet.toTable`. El resto de `SplitDatasetDialog.doOkClickBeforeHide` (el guardado de los dos ficheros y la ventana de resultados), que hasta ahora no se había podido ejecutar nunca, solo se ha leído: no se ha visto nada que vaya a fallar, pero sigue sin ejecutarse. |
| **Módulo** | bnEvaluation |
| **Paquete** | org.openmarkov.bnEvaluation |
| **Ficheros** | `bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/SplitSetManager.java:53-55`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/SplitSetManager.java:164-165`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/SplitSetManager.java:196-197`<br>`core/src/main/java/org/openmarkov/core/model/database/CaseDatabase.java:103-105`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/SplitDatasetDialog.java:265-314`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/ResultsDialog.java:180,276`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/SplitSet.java:49` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** El usuario abre el menú Herramientas («Tools») → «Split Dataset» (`SplitDatasetPlugin`), elige una base de casos y cualquiera de las tres opciones, y pulsa Aceptar. La ventana no puede terminar con ninguna opción: la excepción sale como error no controlado, no se piden los ficheros de destino ni se guarda nada.

La ventana es `SplitDatasetDialog` (módulo `bnEvaluation`), que parte un banco de casos en uno de prueba y otro de entrenamiento. Ofrece tres maneras de elegir los casos de prueba: al azar, los primeros o los últimos. Al pulsar Aceptar, `doOkClickBeforeHide` (líneas 265-284) llama, según la opción, a `SplitSetManager.generateRandomTestSet`, `generateFirstTestSet` o `generateLastTestSet`.

Los tres métodos empiezan convirtiendo la lista de variables a `ArrayList`: `ArrayList<Variable> listaVariables = (ArrayList<Variable>) caseDatabase.getVariables();`. `CaseDatabase.getVariables()` devuelve siempre `Collections.unmodifiableList(variables)`, que no es un `ArrayList`, así que la conversión falla con `ClassCastException` antes de hacer nada.

Los otros dos métodos de `SplitSetManager`, `crossValidation` y `multipleSamples`, declaran `List<Variable>` y funcionan. Son los que usa la validación cruzada de `bnEvaluation` (`CrossValidationController`), que no se ve afectada.

Tampoco se llega nunca a la ventana de resultados de una partición (`ResultsDialog` con un `SplitSet`) ni a `SplitSet.toTable`, que solo se usan desde esta ventana.

Medido:

- Sobre `learning.algorithm/src/test/resources/network/BN-asia10k.csv`: `generateRandomTestSet(100)`, `generateFirstTestSet(100)` y `generateLastTestSet(100)` lanzan `java.lang.ClassCastException`; `crossValidation(10)` termina bien.
- Sobre una base de 10 casos y 2 variables: los tres métodos lanzan `java.lang.ClassCastException: class java.util.Collections$UnmodifiableRandomAccessList cannot be cast to class java.util.ArrayList`, en las líneas 55, 165 y 197 de `SplitSetManager`.

**El código.** bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/SplitSetManager.java:53-56

```
public SplitSet generateRandomTestSet(int numTest) {
    //database information
    ArrayList<Variable> listaVariables = (ArrayList<Variable>) caseDatabase.getVariables();
    int numVariables = listaVariables.size();
```

core/src/main/java/org/openmarkov/core/model/database/CaseDatabase.java:103-105

```
public List<Variable> getVariables() {
    return Collections.unmodifiableList(variables);
}
```

bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/SplitDatasetDialog.java:274-284

```
if (randomCasesButton.isSelected()) {
    sets = splitSetManager.generateRandomTestSet(numTest);
    sets.setTitle(title + "Random selected.\n");
} else {
    if (firstCasesButton.isSelected()) {
        sets = splitSetManager.generateFirstTestSet(numTest);
        ...
    } else {
        sets = splitSetManager.generateLastTestSet(numTest);
```

**Cómo arreglarlo.** En los tres métodos, declarar `List<Variable> listaVariables = caseDatabase.getVariables();` sin conversión, como ya hacen `crossValidation` y `multipleSamples`, y comprobar que después no se modifica la lista; si hace falta una lista propia, copiarla con `new ArrayList<>(...)`. Los tres sitios son las líneas 55, 165 y 197 de `SplitSetManager`.

Revisar después el resto de la ventana, que hoy no se ha podido ejecutar nunca: el guardado de los dos ficheros (`CaseDatabaseManager.getWriter(...).save`) y la ventana de resultados con `SplitSet.toTable`.

Prueba de regresión en `bnEvaluation/src/test/java/org/openmarkov/bnEvaluation/`: para una base de N casos, los tres métodos devuelven un `SplitSet` cuyo banco de prueba tiene `numTest` casos y el de entrenamiento N − `numTest`, de modo que entre los dos suman todos los casos; los primeros y los últimos casos coinciden con las filas esperadas; y `SplitSet.toTable()` se construye sin error.

### Error 146 · [Corregido] En la validación cruzada, si el número de casos no es múltiplo del número de pliegues, la ventana de resultados dice que las puntuaciones se calcularon con menos casos y la pérdida por caso de la log-verosimilitud sale más alta

| | |
|---|---|
| **Estado** | Corregido en el commit `0a04b34` (05-10-2026): `MeasureValue.averageValue` conserva el total de casos de todas las iteraciones y guarda cuántas son, y la pérdida por caso (`getLossPerCase`) es la suma de las iteraciones dividida por ese total. Con varias iteraciones, la ventana y la hoja de Excel dicen «Scores are averages of k iterations, with N cases in all» (`MeasuresSet.getScores CasesNote`); con una sola, lo mismo que antes. En Excel esa frase sustituye a las dos notas anteriores. El texto también cambia en el muestreo repetido, donde N es ahora el total de las muestras. La prueba de regresión acumula cuatro pliegues de 3, 3, 2 y 2 casos en un `MeasuresSet` y comprueba la fila «LOGLIKELIHOOD Loss» y el texto; no ejecuta una validación cruzada entera. |
| **Módulo** | bnEvaluation |
| **Paquete** | org.openmarkov.bnEvaluation.measures |
| **Ficheros** | `bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/measures/MeasureValue.java:48-58`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/measures/MeasuresSet.java:115-161`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/measures/MeasuresSet.java:168-187`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/LearningEvaluator.java:114-136`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/SplitSetManager.java:229-286`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/export/ExcelExporter.java:221-251`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/ResultsDialog.java:325-332` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 20 |

**Qué pasa.** El usuario llega por el menú Herramientas («Tools») → «Cross-validation» (`Cross ValidationDialog`): elige una base de casos, un algoritmo de aprendizaje, el número de pliegues («Folds», 10 por omisión) y la medida «Goodness of fit Log-Likelihood measures». La ventana de resultados (`ResultsDialog`) dice «Cross validation with k=… folders. Scores are calculated with N cases.», y en la pestaña «Scores» enseña «LOGLIKELIHOOD score» y «LOGLIKELIHOOD Loss», cuya ayuda dice «average negative log-likelihood per case». La exportación a Excel escribe lo mismo.

En la validación cruzada de k pliegues, `LearningEvaluator.runEvaluator` aprende una red con cada conjunto de entrenamiento y la puntúa con su pliegue de prueba; la puntuación de un pliegue es una suma sobre sus casos. `MeasuresSet.accumulateMeasureSet` suma las puntuaciones y los números de casos de los k pliegues, y `setAveraged` los divide por k. Lo que se muestra es, por tanto, la puntuación media de un pliegue, y N el número medio de casos por pliegue; la hoja de Excel lo dice («Measures calculated as an average of k iterations»). La pérdida es −puntuación / N.

Cuando el número de casos n es múltiplo de k, todo cuadra: con 12 casos y 4 pliegues sale «3 cases» y la pérdida es la media por caso.

`MeasureValue.averageValue` divide el número de casos con división entera (`getNumCases() / numIterations`). Si n no es múltiplo de k, `SplitSetManager.crossValidation` hace pliegues de ⌊n/k⌋ + 1 y ⌊n/k⌋ casos, la media verdadera tiene decimales y se trunca. La ventana dice menos casos de los que tiene un pliegue medio, y la pérdida se divide por ese número truncado: sale multiplicada por n / (k · ⌊n/k⌋).

Medido con un algoritmo que solo aprende parámetros (el factor no depende del algoritmo), comparando la pérdida mostrada con la suma de las log-verosimilitudes de los pliegues dividida por n:

- 12 casos, 4 pliegues (3, 3, 3, 3): «3 cases», pérdida 1,7019, igual a la verdadera.
- 10 casos, 4 pliegues (3, 3, 2, 2): «2 cases», pérdida 2,0864 frente a 1,6692 (un 25 % más).
- 13 casos, 4 pliegues (4, 3, 3, 3): «3 cases», pérdida 1,6879 frente a 1,5580 (un 8,3 % más).
- `core/src/test/resources/nets/Cataract-RRDdatasetPreProc3.csv` (537 casos), 10 pliegues (siete de 54 y tres de 53): «53 cases», pérdida 2,4318 frente a 2,4001 (un 1,3 % más).

Las bases en CSV (valores separados por comas) de `learning.algorithm/src/test/resources/network/` e `integrationTests/src/main/resources/networks/learning/` tienen 500, 1 000, 5 000 o 10 000 casos, así que con los 10 pliegues por omisión no se nota; las de `Cataract` en `core/src/test/resources/nets/` (537 casos) sí. La evaluación de una red con una sola base de prueba y el muestreo repetido («multiple samples», pruebas del mismo tamaño) no se ven afectados.

**El código.** bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/measures/MeasureValue.java:48-58

```
public void averageValue(int numIterations) {
    value = value / (double) numIterations;
    super.setNumCases(super.getNumCases() / numIterations);
}

@Override
public void accumulate(Measure measure) {
    double valueToAdd = ((MeasureValue) measure).getValue();
    value = value + valueToAdd;
    super.setNumCases(super.getNumCases() + measure.getNumCases());
}
```

bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/measures/MeasuresSet.java:176-180

```
for (MeasureValue measure : measures) {
    rows.add(new ScoresRow.Data(measure.getMeasureType() + " score", measure.getValue()));
    if (measure.getMeasureType() == LOGLIKELIHOOD) {
        double loss = -measure.getValue() / measure.getNumCases();
        rows.add(new ScoresRow.Data(measure.getMeasureType() + " Loss", loss));
```

**Cómo arreglarlo.** No truncar: guardar el total de casos de prueba de los k pliegues y calcular la pérdida por caso como −(suma de las log-verosimilitudes de los pliegues) / (total de casos), o, lo que es lo mismo, −puntuación media · k / total. El número medio de casos, si se quiere enseñar, debe ser un número con decimales.

Los sitios que usan ese número son `MeasuresSet.getMeasureInformation` (el texto de la ventana), `MeasuresSet.buildScoresRows` (la fila «Loss», que también usa la exportación) y `ExcelExporter.writeScores` (la nota «Scores are calculated with N cases»). Conviene que el texto diga a la vez que las puntuaciones son medias de k pliegues y cuántos casos hay en total, porque en la misma ventana la matriz de confusión se da con el total de casos (`MeasureMatrix.accumulate` no divide) y las puntuaciones con la media por pliegue.

Prueba de regresión: validación cruzada con 10 casos y 4 pliegues; la fila «LOGLIKELIHOOD Loss» debe ser igual a −(suma de las log-verosimilitudes de los cuatro pliegues) / 10.

### Error 147 · [Corregido] Quedarse sin memoria al aprender una red se presenta como un error que el programa no había previsto

| | |
|---|---|
| **Estado** | Corregido en el commit `7aa7fc9` (07-10-2026): `LearningDialog.learnButtonActionPerformed` envuelve la `NotEnoughMemoryException` en `UnrecoverableException`, que el manejador global enseña como error normal con el texto «The system has run out of memory». Sin prueba de regresión: haría falta agotar la memoria o sacar esa captura de la ventana. La normalización imposible ya se mostraba bien desde el commit `87129d6`. |
| **Módulo** | learning.gui |
| **Paquete** | org.openmarkov.learning.gui |
| **Ficheros** | `learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:936-945`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:210-218`<br>`gui/src/main/java/org/openmarkov/gui/dialog/OMExceptionHandler.java:17-58` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 48, error 148, error 149, nota 174 |

**Qué pasa.** Quedarse sin memoria al aprender una red (bases de casos grandes, muchas variables o muchos padres) es algo a lo que el usuario puede llegar desde el botón de aprender de `LearningDialog`. En ese caso se abre el diálogo de «error que un desarrollador no previó», en vez de un mensaje de error normal. No se ha provocado el agotamiento de memoria.

En `LearningDialog.learnButtonActionPerformed`, el mismo `try` que ya trata bien la normalización imposible tiene `catch (OutOfMemoryError e1) { throw new UnreachableException(new NotEnoughMemoryException(e1)); }`.

`NotEnoughMemoryException` es la excepción del dominio para este caso, y en la propagación de evidencia se muestra como error normal. Aquí, al ir envuelta en `UnreachableException`, el manejador global (`OMExceptionHandler`) la presenta como un error no previsto.

En la misma línea del bloque se declaran inalcanzables otras excepciones; la de las variables no observadas sí se alcanza y se describe en error 148.

**El código.** learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:936-945

```
} catch (OutOfMemoryError e1) {
    throw new UnreachableException(new NotEnoughMemoryException(e1));
} catch (CannotNormalizePotentialException e) {
    JOptionPane.showMessageDialog(this,
            e.getExceptionMessage() + "\n" +
↪   stringDatabase.getString("Learning.Alpha.SmoothingAdvice"),
            e.getExceptionTitle(), JOptionPane.ERROR_MESSAGE);
} catch (UnobservedVariablesException | EmptyModelNetException | DoEditException e) {
    throw new UnreachableException(e);
}
```

**Cómo arreglarlo.** Envolver la `NotEnoughMemoryException` en `UnrecoverableException` (error normal), como hacen los demás sitios que la capturan, o mostrarla con un diálogo propio.

El último `catch` tiene el mismo problema con la excepción de variables no observadas (error 148); conviene arreglarlos a la vez.

Prueba: difícil de automatizar para la memoria.

### Error 148 · [Corregido] Aprender con una red modelo que tiene una variable que no está en la base de casos sale como un error que el programa no había previsto, aunque la excepción trae su propio mensaje

| | |
|---|---|
| **Estado** | Corregido en el commit `728b158` (07-10-2026): `LearningDialog.learnButton ActionPerformed` separa `UnobservedVariablesException` de las otras dos excepciones del bloque y la envuelve en `UnrecoverableException`, que el manejador global enseña como error normal. El mensaje lista las variables que faltan solo por su nombre («…which are: B, C»); lo comprueba un caso nuevo de `LearningManagerTest`. La parte de la ventana no tiene prueba. La clave `Learning.Error.UnobservedVariables` de `Learning_en.xml` sigue sin usarse. |
| **Módulo** | learning.gui |
| **Paquete** | org.openmarkov.learning.gui |
| **Ficheros** | `learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:865-895`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:936-944`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:544`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1031-1041`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/LearningEvaluator.java:139-144`<br>`learning.core/src/main/java/org/openmarkov/learning/core/LearningManager.java:75-93`<br>`learning.core/src/main/java/org/openmarkov/learning/core/LearningManager.java:165-176`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningController.java:66-85`<br>`core/src/main/resources/core/localize/core_class_localizations_en.xml:446-447`<br>`learning.gui/src/main/resources/learning/gui/localize/Learning_en.xml:122-127` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 147, error 149, nota 10 |

**Qué pasa.** En la ventana de aprender una red (`LearningDialog`), el usuario abre una base de casos y, en la pestaña de la red modelo, carga una red desde fichero; al cargarla queda marcada «Use the information of the nodes». Si la red modelo tiene alguna variable que no es una columna de la base de casos, al pulsar «Learn» con cualquier algoritmo salvo «Expectation maximization (EM)» sale el diálogo de «error que un desarrollador no previó», en vez de un mensaje que diga qué variable falta. Pasa en aprendizaje automático y en interactivo.

La ventana tiene código para elegir qué variables usar, pero ese panel no se muestra (la línea que lo añade a las pestañas está comentada), así que el usuario no puede desmarcar una variable; se llega con una base a la que le falta una columna.

`LearningController.initLearning` construye el `LearningManager`, y su constructor, cuando se usa la red modelo, llama a `applyModelNet`. Si el algoritmo no admite variables no observadas (`supportsUnobservedVariables = false` en `@LearningAlgorithmType`, que es el caso de todos salvo EM) y a la base le falta alguna variable de la red modelo, lanza `UnobservedVariables Exception`.

`LearningDialog.learnButtonActionPerformed` la captura en `catch (UnobservedVariables Exception | EmptyModelNetException | DoEditException e) { throw new Unreachable Exception(e); }`, y el manejador global (`OMExceptionHandler`) presenta cualquier `Unreachable Exception` como error no previsto.

La excepción ya tiene un texto para el usuario en `core_class_localizations_en.xml` («Algorithm {algorithmType.name} does not support unobserved variables, which are: {unobservedVariables}»), que es el que se vería si llegara como error normal; y `Learning_en.xml` tiene otro, `Learning.Error. UnobservedVariables`, que ninguna clase usa.

Medido llamando a las mismas clases que la ventana con `integrationTests/src/main/resources/networks/learning/asia10K.csv` sin la variable `Smoker` y la red modelo `integrationTests/src/main/resources/networks/learning/BN-asia.pgmx`, con «Use the information of the nodes»: el `LearningManager` lanza, con el PC y con «Hill climbing», `UnobservedVariablesException` con el texto «Algorithm PC does not support unobserved variables, which are: [Finite states variable Smoker with states [state no, state yes]]». La ventana no se ha abierto: que acaba en el diálogo de error no previsto está leído en el `catch` y en `OMExceptionHandler`.

Las otras dos excepciones del mismo `catch` sí son inalcanzables desde esta ventana. `EmptyModel NetException` solo se lanza si se pide usar la red modelo sin haberla, y la ventana solo construye `ModelNetUse` cuando hay red modelo. `DoEditException` solo podría venir de `AutoArrangeEdit`, que no comprueba ninguna restricción.

**El código.** learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:936-944

```
} catch (OutOfMemoryError e1) {
    throw new UnreachableException(new NotEnoughMemoryException(e1));
} catch (CannotNormalizePotentialException e) {
    JOptionPane.showMessageDialog(this,
            e.getExceptionMessage() + "\n" +
↪   stringDatabase.getString("Learning.Alpha.SmoothingAdvice"),
            e.getExceptionTitle(), JOptionPane.ERROR_MESSAGE);
} catch (UnobservedVariablesException | EmptyModelNetException | DoEditException e) {
    throw new UnreachableException(e);
}
```

learning.core/src/main/java/org/openmarkov/learning/core/LearningManager.java:169-176

```
List<Variable> missingVariables = getMissingVariables(database.getVariables(),
↪   modelNet.getVariables());
if (!algorithmClass.getAnnotation(LearningAlgorithmType.class)
                   .supportsUnobservedVariables() && !missingVariables.isEmpty()) {
    ...
    throw new UnobservedVariablesException(algorithmClass, missingVariables);
}
```

**Cómo arreglarlo.** Separar `UnobservedVariablesException` del `catch` que la declara inalcanzable y mostrarla como error de entrada del usuario: envolverla en `UnrecoverableException` (el manejador global la enseña entonces con su texto) o con un diálogo propio, como ya se hace con la normalización imposible en el mismo `try`. Si se quiere, la ventana puede además avisar antes, al

cargar la base de casos o la red modelo.

El texto de la excepción enseña cada variable con `Variable.toString` («Finite states variable Smoker with states […]»); conviene que liste solo los nombres. Hay que decidir también qué hacer con la clave `Learning.Error.UnobservedVariables`, que hoy no se usa.

La validación cruzada de `bnEvaluation` (`LearningEvaluator.learnTrainNet`) también declara inalcanzable esta excepción, y ahí sí lo es, porque no usa red modelo.

Es el mismo `catch` que el de error 147, y conviene arreglarlos a la vez.

Prueba de regresión: construir el `LearningManager` con una base sin la variable `Smoker`, la red modelo `BN-asia.pgmx` y el PC, comprobar que lanza `UnobservedVariablesException`, y que la ventana la trata como error de entrada (no como `UnreachableException`).

### Error 149 · [Corregido] Usar en la ventana de aprendizaje una red modelo que no tiene todas las columnas de la base de casos saca un error no previsto al cargarla y deja la ventana a medio preparar

| | |
|---|---|
| **Estado** | Corregido en el commit `cd8d624` (08-10-2026): la ventana pregunta si la red modelo discretiza la columna con `LearningController.isDiscretizedInModelNet`, que responde que no cuando la red modelo no tiene esa columna. Lo comprueba `LearningControllerTest`, la primera prueba del módulo `learning.gui`. La ventana entera no se ha probado, porque no se construye sin pantalla. |
| **Módulo** | learning.gui |
| **Paquete** | org.openmarkov.learning.gui |
| **Ficheros** | `learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1154-1241`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:736-761`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:773-790`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1019-1041`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:544`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1243-1248`<br>`core/src/main/java/org/openmarkov/core/model/network/ProbNet.java:807-809` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 147, error 148 |

**Qué pasa.** El usuario abre Herramientas («Tools») → «Learning» (`LearningDialog`), carga una base de casos y, en la pestaña «Model network», carga una red desde fichero («Load model network from file» → «Open») o toma la red abierta («Use the already open network»). Si la base tiene alguna columna que no es una variable de la red modelo, sale el diálogo de «error que un desarrollador no previó» con un `NullPointerException`. Si carga primero la red modelo y después la base, el mismo error sale al cargar la base.

Aprender con una red modelo que solo tiene parte de las columnas es un uso previsto: al cargarla, la ventana elige por código «Use only the variables in the model network» y deja sin marcar las columnas que la red no tiene; y `LearningManager.applyModelNet`, cuando se parte de la red modelo, le añade las variables de la base que le faltan.

`updateVariableSelectionPanel` rehace el panel de preprocesado: para cada columna de la base crea su casilla y sus listas de opciones. Con red modelo, las líneas 1216-1223 buscan la columna en la red modelo para proponer «Use model network» si allí está discretizada, con `modelNet.getNode(variable.getName()).getVariable()`. `ProbNet.getNode(String)` devuelve `null` cuando la red no tiene esa variable, y la llamada lanza `NullPointerException`. El comentario de encima dice «if the variable is in the model net», y más arriba (línea 1186) la casilla ya pregunta `modelNet.containsVariable(...)` para la misma columna; aquí no se comprueba.

La pestaña de preprocesado no se añade a la ventana (la línea 544 está comentada), pero el método se ejecuta igual: lo llaman `loadCaseFile`, al terminar de leer la base; `modelNetSelected`, al cargar la red modelo o tomar la abierta; y el botón de variables de la red modelo, que está en ese mismo panel. El usuario ve el error sin ver el panel.

Lo que queda después depende del orden:

- Base de casos y después red modelo: la excepción sale de `modelNetSelected`, que se queda a medias. La red modelo queda cargada y el panel invisible solo tiene las columnas anteriores a la primera que falta en la red modelo. `getSelectedVariables` lee ese panel, así que «Learn» aprende solo con esas columnas. Si las que faltan van todas detrás, el resultado es el que se buscaba; si alguna va antes, las columnas siguientes se pierden: con PC o ascenso de colinas («Hill climbing») sale `UnobservedVariablesException`, que también se enseña como error no previsto (error 148), y con EM (maximización de la esperanza) se aprende una red con solo las primeras columnas, sin más aviso.
- Red modelo y después base de casos: `loadCaseFile` captura la excepción, borra el nombre del fichero, desactiva «Learn» (`resetCaseDatabaseFile`) y la vuelve a lanzar. Esa base no se puede usar con esa red modelo.

Medido con un pequeño programa de prueba que ejecuta, en el mismo orden, la parte del bucle de `updateVariableSelectionPanel` que usa la red modelo, y después construye el `LearningManager` como lo hace «Learn» con las columnas que quedan (red modelo con «Use the information of the nodes» marcada y lo demás sin marcar):

- `learning.algorithm/src/test/resources/network/HeadToHead2.csv` (columnas A, B, C, E, F) con la red modelo `learning.algorithm/src/test/resources/network/ThreeNodesAtoBtoC.pgmx` (A, B, C): `NullPointerException` en la columna E («Cannot invoke "…Node.getVariable()" because the return value of "…ProbNet.getNode(String)" is null»). Quedan A, B y C, y con PC, ascenso de colinas y EM el `LearningManager` se construye con esas tres variables.
- `integrationTests/src/main/resources/networks/learning/asia10K.csv` (primera columna Smoker, segunda LungCancer) con `integrationTests/src/main/resources/networks/learning/BN-asia.pgmx` sin el nodo LungCancer: la excepción salta en LungCancer y solo queda Smoker. Con PC y ascenso de colinas, `UnobservedVariablesException`; con EM, el `LearningManager` se construye con una red de una sola variable, Smoker.

La ventana es un `JDialog` y no se ha abierto: qué diálogo enseña el manejador global y en qué estado quedan el nombre del fichero y «Learn» está leído.

**El código.** learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1184-1187

```
boolean select = true;
if (modelNetVariablesRadioButton.isSelected()) {
    select = modelNet.containsVariable(variable.getName());
}
```

learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1214-1223

```
/* If there is a model net and the variable is in the model net
 * and is discretized, then the default option should be 'As in model'*/
if (modelNet != null) {
    VariableType variableTypeInModel = modelNet.getNode(variable.getName()).getVariable()
                                                .getVariableType();
    if (variableTypeInModel == VariableType.DISCRETIZED) {
        discretizeOptions.setSelectedItem(stringDatabase.getString(
↪   "Learning.Discretize.ModelNet"));
    }
}
```

learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:1031-1037

```
private void modelNetSelected() {
    modelNetVariablesRadioButton.setSelected(true);
    modelNetTabPanel.useNodePositionsCheckBox.setEnabled(true);
    modelNetTabPanel.useNodePositionsCheckBox.setSelected(true);
    modelNetTabPanel.startFromModelNetCheckBox.setEnabled(true);

    updateVariableSelectionPanel();
```

**Cómo arreglarlo.** Comprobar que la red modelo tiene la columna antes de pedir su tipo, como dice el comentario: tomar `Node modelNode = modelNet.getNode(variable.getName())` y proponer «Use model network» solo si `modelNode != null` y su variable está discretizada.

Es el único sitio de la ventana que pide a la red modelo el nodo de una columna sin comprobarlo. Lo que viene después no necesita cambio: con «Use only the variables in the model network» las columnas que la red no tiene quedan sin marcar, `FilterDatabase` las quita antes de discretizar, y `LearningManager.adaptDatabaseToModelNet` ya comprueba que la variable exista en la base.

Prueba: una prueba de interfaz con pantalla (AssertJ-Swing) que cargue `HeadToHead2.csv` y después `ThreeNodesAtoBtoC.pgmx` sin excepción y compruebe que `getSelectedVariables` da A, B y C; y en el orden contrario, que la base queda cargada y «Learn» activo. Si se prefiere una prueba sin pantalla, sacar el cuerpo del bucle a un método que no dependa de Swing.

### Error 215 · En «Data preprocessing» no se ve la lista que acompaña a «Outliers:», y en «Cross validation» salen cortadas la casilla «Show individual probabilities» y la última fila de medidas

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | bnEvaluation |
| **Paquete** | org.openmarkov.bnEvaluation.component; org.openmarkov.bnEvaluation.dialog |
| **Ficheros** | `bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/component/PreprocessingOptionsPanel.java:84-87,329-334`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/DataPreprocessingDialog.java:94`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/component/MeasuresPanel.java:47,52`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/CrossValidationDialog.java:105` |
| **Comprobado** | Leyendo el código; visto en capturas automáticas de las dos ventanas |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

| | |
|---|---|
| **Relacionados** | nota 140 |

**Qué pasa.** Dos ventanas del menú Tools enseñan controles que no caben en ellas. Visto el 8-10-2026 en capturas automáticas, sin base de casos cargada:

- «Data preprocessing»: en el primer renglón se lee el rótulo «Outliers:» y a su derecha no hay nada. La lista en la que se elige qué hacer con los valores anómalos (los que se apartan mucho del resto) no se ve, así que no se puede cambiar esa opción.
- «Cross validation»: en el recuadro «Measures», la casilla «Show individual probabilities» sale cortada («Show individ») y la última fila de las puntuaciones (AIC y MDL) queda por debajo del borde.

Por qué pasa, por lectura. En `PreprocessingOptionsPanel` (líneas 329-334), el rótulo de la variable de clase, su lista, el rótulo «Outliers:» y su lista van en un `FlowLayout`, la disposición de Swing que pone un componente detrás de otro y pasa al renglón siguiente lo que no cabe; el renglón tiene sitio para una sola fila, y la lista, que es ancha (359 píxeles de ancho preferido, medido construyendo el panel sin ventana), cae a una segunda fila que no se ve. En «Cross validation», `MeasuresPanel` pide 840×220 píxeles (línea 47) y la ventana se fija en 720×700 (`CrossValidationDialog`, línea 105).

No se ha medido con una base cargada ni con otra escala de la interfaz.

**Cómo arreglarlo.** En «Data preprocessing», dar a «Outliers:» y su lista un renglón propio, o usar una disposición que no parta el renglón. En «Cross validation», no fijar el tamaño de la ventana por debajo del que pide su contenido (quitar el `setSize` y dejar el `pack()`), o meter el panel en uno con barra de desplazamiento.

Prueba: construir cada ventana con pantalla y comprobar que todos los componentes visibles quedan dentro del recuadro de su contenedor.

### Error 216 · En «Cross validation» y «Bayesian network evaluation», el rótulo «Classification variable» se queda apagado para siempre, también cuando la lista de al lado ya se puede usar

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | bnEvaluation |
| **Paquete** | org.openmarkov.bnEvaluation.component |
| **Ficheros** | `bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/component/MeasuresPanel.java:51,246-255`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/CrossValidationDialog.java:318,344`<br>`bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/dialog/BNEvaluationDialog.java:103,294-299` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 215 |

**Qué pasa.** Las dos ventanas comparten el recuadro «Measures» (`MeasuresPanel`), que nace apagado y se enciende al cargar la base de casos. Sus casillas se encienden; el rótulo «Classification variable» sigue en gris, como si la lista de variables que tiene al lado no se pudiera usar, también después de marcar «Predictive capacity», que es lo que enciende esa lista.

Medido construyendo el panel sin ventana:

```
recién construido:        rótulo encendido
tras setEnabled(false):   rótulo apagado
tras setEnabled(true):    rótulo apagado
```

Por qué pasa: `MeasuresPanel.setEnabled` apaga todos sus componentes cuando se le pide apagar, y cuando se le pide encender solo enciende las casillas.

**El código.** bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/component/MeasuresPanel.java:246-255

```
@Override public void setEnabled(boolean enabled) {
    super.setEnabled(enabled);
    if (enabled) {
        this.allMeasureRelatedCheckboxes().forEach(checkbox -> checkbox.setEnabled(true));
    } else {
        for (Component component : this.getComponents()) {
            component.setEnabled(false);
        }
    }
}
```

**Cómo arreglarlo.** Encender también el rótulo en la rama de encender. La lista de variables y «Show individual probabilities» deben seguir dependiendo de «Predictive capacity».

Prueba de regresión: la medida de arriba; tras `setEnabled(false)` y `setEnabled(true)` el rótulo está encendido.

### Error 217 · «Reset values» de la ventana de aprender borra el nombre de la base de casos y la lista de variables, pero deja «Learn» encendido; pulsarlo acaba en «No dataset is present»

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | learning.gui |
| **Paquete** | org.openmarkov.learning.gui |
| **Ficheros** | `learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:762-770,864-868,991-1000,1138-1153`<br>`learning.gui/src/main/resources/learning/gui/localize/learning_gui_class_localizations_en.xml:7-8` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 148, error 149 |

**Qué pasa.** En la ventana «Learning», con una base de casos cargada, «Reset values» vacía el campo con el nombre del fichero y la lista de variables, y deja el botón «Learn» encendido. Pulsar «Learn» abre un aviso, «No dataset is present. Please, select a dataset.», y no aprende nada. La ventana ofrece una acción que no puede hacer.

Por qué pasa: `resetButtonActionPerformed` (líneas 991-1000) pone a nulo la ruta y el nombre de la base y limpia los paneles, pero no apaga `learnButton`. `learnButtonActionPerformed` empieza comprobando la ruta (líneas 866-868) y lanza `LearningException.NoDatabasePresent`. El método que sí deja la ventana como si no hubiera base, `resetCaseDatabaseFile` (líneas 762-770), apaga «Learn» y los selectores de variables, y no se llama desde aquí.

Comprobado solo leyendo: la ventana no se puede construir sin pantalla.

**Cómo arreglarlo.** Que «Reset values» llame a `resetCaseDatabaseFile`, además de limpiar lo que ya limpia. Otra opción, si se prefiere que «Reset values» devuelva las opciones a su valor inicial sin descargar la base, es que no borre la ruta ni el nombre; hay que elegir una de las dos.

Prueba: con pantalla, cargar una base, pulsar «Reset values» y comprobar que «Learn» queda apagado (o que la base sigue nombrada, según lo que se elija).

### Error 219 · En «Data preprocessing», con «Selected variables» elegido, cargar otra base de casos deja apagadas las casillas de todas las variables: no se pueden marcar ni desmarcar

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | bnEvaluation |
| **Paquete** | org.openmarkov.bnEvaluation.component |
| **Ficheros** | `bnEvaluation/src/main/java/org/openmarkov/bnEvaluation/component/PreprocessingOptionsPanel.java:359-362,451-466,477-507` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 215 |

**Qué pasa.** La ventana enseña una fila por variable, con una casilla para elegir cuáles se conservan. Las casillas solo se pueden tocar con «Selected variables» elegido. Si con esa opción elegida se carga otra base de casos (o la misma otra vez), las filas nuevas salen con la casilla apagada aunque «Selected variables» siga elegido. Para poder tocarlas hay que pasar a «All variables» y volver.

Medido construyendo el panel sin ventana y cargándole dos veces una base de ocho variables:

```
primera carga («All variables»):          casillas apagadas (correcto)
se elige «Selected variables»:            casillas encendidas
segunda carga:                            «Selected variables» sigue elegido, casillas
↪  apagadas
```

Por qué pasa: `rebuildVariableRows` crea cada casilla apagada (línea 507) sin mirar cuál de las dos opciones está elegida; las casillas solo se encienden en el oyente de «Selected variables» (líneas 461-466), que no se ejecuta al cargar.

**Cómo arreglarlo.** Crear cada casilla con `setEnabled(selectedVariablesRadioButton.isSelected())`, o devolver la ventana a «All variables» al cargar una base.

Prueba de regresión: la medida de arriba; tras la segunda carga, las casillas están encendidas si «Selected variables» sigue elegido.

### Error 240 · Aprender el clasificador de árbol aumentado con una base de casos en la que solo queda la variable clase termina en NoSuchElementException, sin aprender nada, mientras que los demás clasificadores de la familia devuelven la clase sola

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.treeaugmentednb |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/treeaugmentednb/TreeAugmentedNBAlgorithm.java:33-51`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/naivebayes/IDiscriminativeBayes.java:43-45` |

| | |
|---|---|
| | `learning.gui/src/main/java/org/openmarkov/learning/gui/LearningDialog.java:554-561,891-893` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 136, error 143 |

**Qué pasa.** El usuario llega por Tools → «Learning» → tipo de algoritmo «Discriminative» → «Tree augmented naive bayes», con una base de casos de una sola columna, o desmarcando en el apartado «Preprocessing» de la ventana todas las variables menos la que elige como clase. Al pulsar «Learn», en vez de una red con la clase sola sale la ventana de error no previsto con `java.util.NoSuchElementException`, y no se abre ninguna red. Lo mismo pasa con ese algoritmo en la validación cruzada de `bnEvaluation`, que llama al mismo `init` (`LearningEvaluator`, línea 152); esto último solo está leído.

Por qué pasa: desde el arreglo del error 136 (commit `9e9d4aa`), `TreeAugmentedNBAlgorithm.init` orienta el árbol a partir de `getRootOfTheTree`, que toma la primera característica con `getNonRootVariables().getFirst()` (línea 50). Con solo la clase no hay ninguna característica: la lista está vacía y `getFirst()` lanza la excepción. La llamada viene de `LearningController.initLearning`, que `LearningDialog.learnButtonActionPerformed` hace antes de crear la ventana de la red (líneas 891-893), y nadie la recoge (líneas 554-561). Antes de ese arreglo moría en el mismo sitio de otra manera: el sorteo de la raíz pedía `new Random().nextInt(0)`, que lanza `IllegalArgumentException` (esto último leído en el código, no ejecutado).

Es el mismo caso que el error 143 resolvió para el bosque aumentado (FAN): sin características no hay árbol que colgar, y lo que corresponde es no proponer nada.

Medido el 15-10-2026 con un pequeño programa que construye el algoritmo como la ventana, sobre `learning.algorithm/src/test/resources/network/OneNode-1000.csv` (una sola columna, `A`, que es la clase):

```
init threw java.util.NoSuchElementException
    at java.base/java.util.List.getFirst(List.java:823)
    at ...TreeAugmentedNBAlgorithm.getRootOfTheTree(TreeAugmentedNBAlgorithm.java:50)
    at ...TreeAugmentedNBAlgorithm.init(TreeAugmentedNBAlgorithm.java:39)
```

Con la misma base, los otros ocho clasificadores de la familia (Bayes ingenuo, FAN, KDB con k = 0 y k = 2, Bayes ingenuo selectivo hacia atrás y hacia delante, superpadre en sus dos modos) terminan sin proponer ningún enlace. Con dos columnas (`TwoNodesAtoB-1000.csv`) el TAN tampoco falla: no propone nada.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/-treeaugmentednb/TreeAugmentedNBAlgorithm.java:37-51

```
        if (!mwst.isBuilt()) {
            mwst.build(probNet, metric, getNonRootNodes());
            mwst.redirect(getRootOfTheTree(), probNet);
        }
        setRelationsForRootVariable();

        MaxNumParents maxNumParentsConstraint = new MaxNumParents(2);
        this.probNet.addConstraint(new NoCycle());
        this.probNet.addConstraint(maxNumParentsConstraint);
    }

    /** @return the feature the links of the tree are directed away from: the first one of
    ↪  the network */
```

```
    protected Variable getRootOfTheTree() {
        return getNonRootVariables().getFirst();
    }
```

**Cómo arreglarlo.** En `init`, no orientar el árbol cuando no hay ninguna característica (`getNonRootVariables().isEmpty()`): el árbol está vacío y `getOptimalEdit` ya no encuentra nada que proponer, así que el resultado es la clase sola, como en el resto de la familia. Conviene que `getRootOfTheTree` no se llame nunca con la lista vacía, o que devuelva `null` y `init` lo compruebe.

Prueba: aprender el TAN sobre `OneNode-1000.csv` con `A` como clase: `init` no lanza ninguna excepción, `getBestEdit(true, true)` devuelve `null` y la red aprendida tiene un solo nodo y ningún enlace.

### Error 241 · En el aprendizaje interactivo del árbol aumentado (TAN), con «Only allowed edits» desmarcada y más de 50 enlaces posibles, la tabla enseña los 50 últimos en el orden de las variables y no los 50 mejores, porque la búsqueda de la mejor propuesta se queda con la última que encuentra

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.treeaugmentednb; org.openmarkov.learning.gui.interactive |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/treeaugmentednb/TreeAugmentedNBAlgorithm.java:60-90`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:56,407-414`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/EditProposalTable.java:52-53,80-89`<br>`learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/fanb/ForestAugmentedNBAlgorithm.java:111-112` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 136, error 134, error 140, nota 189 |

**Qué pasa.** El usuario llega por Tools → «Learning» → tipo «Discriminative» → «Tree augmented naive bayes» → «Interactive learning» → «Learn», y en la ventana «Interactive learning» desmarca «Only allowed edits». La tabla debería ofrecer los 50 enlaces entre características que más valen; con una base de ocho características o más (56 parejas ordenadas o más) ofrece en cambio los 50 que quedan **al final** en el orden de las variables de la red, y el mejor enlace puede no aparecer. La tabla los ordena después por su puntuación (`EditProposalTable.fill`, columna 1, de mayor a menor), así que nada indica que faltan los mejores.

Por qué pasa: `TreeAugmentedNBAlgorithm.getOptimalEdit` recorre todas las parejas de características y, para cada una que pasa el filtro, asigna `bestEdit` y `bestPartialScore` **sin compararla** con la mejor que llevaba (líneas 76-82): devuelve la última que pasa, no la mejor. `InteractiveLearningDialog.updateEditionsTable` llena la tabla pidiendo la mejor y luego la siguiente hasta tener `SHOWING_EDIT_NUM = 50` (líneas 56 y 407-414), y cada propuesta devuelta queda marcada como ya considerada; así que las 50 que entran son las 50 últimas parejas del recorrido. El FAN y el KDB, con la misma estructura, sí comparan (`addScore >= bestPartialScore`, `ForestAugmentedNBAlgorithm`, línea 111; `KDBAlgorithm`, línea 116).

Con «Only allowed edits» marcada solo pasan los enlaces del árbol de peso máximo, uno menos

que características, y la tabla los enseña todos mientras sean 50 o menos; el aprendizaje automático y «Complete phase» aplican todos los del árbol, en otro orden, y la red aprendida es la misma. Ahí el defecto no se ve, salvo con más de 51 características.

Medido el 15-10-2026 con un pequeño programa que construye el TAN como la ventana y repite las llamadas de `updateEditionsTable`:

```
alarm500.csv (37 columnas, clase HISTORY), sin «Only allowed edits»:
  1260 enlaces posibles entre características; se enseñan 50: 35 que salen de BP
  y 15 que salen de CO, las dos últimas columnas
  mejor enlace posible: MINVOL -> VENTALV, 0,554733; el mejor de los enseñados: 0,388181
alarm500.csv, con «Only allowed edits»: se enseñan los 35 del árbol, y el mejor (0,554733)
↪  está entre ellos
BN-asia10k.csv (clase Bronchitis), con «Only allowed edits»: la primera propuesta,
  TuberculosisOrCancer -> Tuberculosis, vale 0,026307; la mejor, TuberculosisOrCancer ->
  ↪  LungCancer, 0,187305
```

(`alarm500.csv` es `integrationTests/src/main/resources/networks/learning/alarm500.csv`; `BN-asia10k.csv`, el de `learning.algorithm/src/test/resources/network`.)

Hay además una pregunta en el mismo filtro: la pertenencia al árbol de peso máximo (`mwst.contains`) está dentro de la condición de «only allowed» (línea 77), de modo que al desmarcar la casilla la tabla ofrece cualquier enlace entre dos características, también los que el TAN nunca añadiría. El FAN hace lo mismo (línea 112), así que puede ser intencionado.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/nbderived/-treeaugmentednb/TreeAugmentedNBAlgorithm.java:68-84

```
        for (Node n1 : nodes) {
            for (Node n2 : nodes) {
                if (n2 == n1) {
                    continue;
                }
                AddLinkEdit addLink = new AddLinkEdit(probNet, n1.getVariable(),
↪  n2.getVariable(), true);
                double addScore = metric.getScore(addLink);

                if (!isEditAlreadyConsidered(addLink)
                        && ((!onlyAllowedEdits || isAllowed(addLink) &&
                        ↪  mwst.contains(n1.getVariable(), n2.getVariable()))
                        && (!onlyPositiveEdits || addScore > 0) && !isBlocked(addLink))
                ) {
                    bestEdit = addLink;
                    bestPartialScore = addScore;
                }
            }
        }
```

**Cómo arreglarlo.** Añadir a la condición `addScore > bestPartialScore` (o `>=`, como el FAN y el KDB; con `>` gana el primero de dos empatados, con `>=` el último), para que cada llamada devuelva la mejor de las que quedan. Conviene decidir de paso si un enlace fuera del árbol de peso máximo debe ofrecerse alguna vez en el TAN y en el FAN, o si `mwst.contains` debe aplicarse con la casilla desmarcada también.

Prueba: con `BN-asia10k.csv` y `Bronchitis` como clase, `getBestEdit(true, true)` devuelve `TuberculosisOrCancer -> LungCancer` (0,187305) y las sucesivas `getNextEdit(true, true)` devuelven puntuaciones que no crecen; y con `alarm500.csv` y `HISTORY` como clase, `getBestEdit(false, true)` devuelve un enlace de puntuación 0,554733 (`MINVOL -> VENTALV` o su inverso).

### Error 251 · En el aprendizaje interactivo con PC, aplicar el último borrado que mejora la red con «Only positive edits» marcada hace pasar el algoritmo a orientar colisionadores sin haber aplicado nada más, y desmarcar después la casilla ya no ofrece los borrados que quedaban; desmarcarla un borrado antes, sí

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | learning.algorithm |
| **Paquete** | org.openmarkov.learning.algorithm.pc |
| **Ficheros** | `learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:150-189,202-223,371-390`<br>`learning.gui/src/main/java/org/openmarkov/learning/gui/interactive/InteractiveLearningDialog.java:378-393,407-414` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | Sí: cuándo da por terminada una fase el PC interactivo: cuando se queda sin propuestas con las casillas que haya marcadas, como ahora, o solo al aplicar una propuesta de la fase siguiente (o con «Complete phase») |
| **Relacionados** | error 140, error 141, error 134 |

**Qué pasa.** El usuario llega por Tools → «Learning» → algoritmo PC → «Interactive learning» → «Learn», y en la ventana «Interactive learning» aplica uno a uno, con «Apply edit», los borrados de enlaces que la tabla le ofrece con «Only positive edits» marcada. Al aplicar el último, la tabla pasa a ofrecer orientaciones de colisionadores. Si entonces desmarca «Only positive edits» para ver también los borrados que no mejoran la red, la tabla no los enseña: solo orientaciones. Si la hubiera desmarcado un borrado antes, los habría visto. Lo que ofrece la tabla depende del orden en que se marcó la casilla.

Es lo que queda del error 140: su arreglo (commit `fc17e1e`) hace que pedir la **siguiente** propuesta no cambie la fase, pero pedir la **mejor** sí la cambia todavía. Y la fase en curso se da por agotada mirando con las casillas que haya marcadas en ese momento, así que avanza porque se ha mirado con «Only positive edits» marcada, no porque se haya hecho algo. La decisión que tomó ese arreglo —que la tabla enseñe solo las propuestas de la fase en curso— no se cumple aquí, porque cuál es la fase en curso lo ha decidido la casilla.

Por qué pasa: tras cada edición aplicada, la ventana vuelve a llenar la tabla (`applySelected` y `updateUI`, líneas 378-393), que empieza por `getBestEdit` (líneas 407-414). `PCAlgorithm.getBestEdit` llama a `nextEdit` y de ahí a `getOptimalEdit` (líneas 150-189); en la fase inicial, si `skeleton.findEdit` no encuentra ningún borrado que cumpla las casillas, `getOptimalEdit` llama a `transitionToNextPhase` (líneas 203-209), que cambia `phase` y busca en la fase siguiente (líneas 371-390). `getNextEdit` repone la fase si la búsqueda la ha cambiado (líneas 174-178); `getBestEdit` no.

Medido el 15-10-2026 con un pequeño programa que construye el algoritmo como la prueba del Java de ese arreglo (`TheInteractivePCOffersOnlyTheEditsOfTheCurrentPhaseTest`: prueba de entropía cruzada, alfa 0,5, significación 0,05, sin prueba de dirección causal), aplica los borrados pidiendo cada vez la mejor propuesta con las dos casillas marcadas, y después llena la tabla sin «Only positive edits»:

```
HeadToHead2.csv: aplicados 5 borrados → fase 1; sin la casilla: 3 filas, 0 borrados
                 aplicados 4 (queda uno que mejora) → fase 0; sin la casilla: 4 filas, 4
                 ↪  borrados
                 aplicados 5, con la casilla quitada antes del último → fase 0; 3 filas, 3
                 ↪  borrados
```

```
BN-asia10k.csv:  aplicados 21 borrados → fase 1; sin la casilla: 2 filas, 0 borrados
                 aplicados 20 (queda uno que mejora) → fase 0; sin la casilla: 7 filas, 7
                 ↪  borrados
                 aplicados 21, con la casilla quitada antes del último → fase 0; 6 filas, 6
                 ↪  borrados
```

La fase 0 es la de quitar enlaces, y la 1, la de orientar colisionadores. La primera y la tercera línea de cada base son la misma red (5 enlaces en `HeadToHead2.csv` y 7 en `BN-asia10k.csv`) con la casilla desmarcada: la tabla ofrece 0 borrados o 3 (6 en `BN-asia10k.csv`) según la casilla se quitara antes o después de aplicar el último borrado que mejora la red.

(Las dos bases son las de `learning.algorithm/src/test/resources/network`.) La ventana es un `JDialog` y no se ha abierto: lo que hace al aplicar una propuesta y al cambiar la casilla está leído en `InteractiveLearningDialog`.

**El código.** learning.algorithm/src/main/java/org/openmarkov/learning/algorithm/pc/PCAlgorithm.java:202-209

```
    public LearningEditProposal getOptimalEdit(boolean onlyAllowedEdits, boolean
    ↪  onlyPositiveEdits) {
        if (phase == Phase.INITIAL_PHASE) {
            LearningEditProposal proposal = skeleton.findEdit(onlyAllowedEdits,
↪  onlyPositiveEdits);
            if (proposal != null) {
                return proposal;
            }
            return transitionToNextPhase(onlyAllowedEdits);
        }
```

**Cómo arreglarlo.** Hace falta decidir cuándo termina una fase en el modo interactivo. Si basta con que se quede sin propuestas con las casillas marcadas, como hoy, al menos desmarcar una casilla debería poder volver a la fase anterior cuando esta todavía tiene propuestas. Lo más simple, y coherente con el arreglo del error 140, es que mirar no cambie nunca la fase: que `getBestEdit` devuelva las propuestas de la fase siguiente cuando la en curso se ha agotado, pero sin dejar `phase` cambiada, y que la fase avance al aplicar una propuesta de la fase siguiente (en `afterEdit Executes`, que ya pone la fase según la edición aplicada) o con «Complete phase». El aprendizaje automático no cambia, porque pide siempre con las dos casillas marcadas.

Prueba: con `HeadToHead2.csv`, aplicar los 5 borrados pidiendo la mejor propuesta con las dos casillas marcadas, y llenar después la tabla sin «Only positive edits»: debe dar la misma tabla que si la casilla se quita antes de pedir la mejor propuesta tras el último borrado (hoy, 3 borrados); lo mismo con `BN-asia10k.csv` y sus 21 borrados (6).

## 14 Interfaz: editor de red y paneles de las relaciones

El lienzo donde se dibuja la red, el diálogo de propiedades de un nodo y los paneles con que se edita cada tipo de relación.

### Error 150 · [Corregido] Volver a elegir en el diálogo del nodo la clase de relación que ya tiene le borra todos sus números

| | |
|---|---|
| **Estado** | Corregido en el commit `dd38f65` (14-10-2026): `potentialTypeChanged` no hace nada si la clase elegida es exactamente la de la relación actual del nodo. Con prueba que construye el panel real, vuelve a elegir en su selector la clase que ya está elegida y exige que el nodo conserve su relación y sus números, sobre `ggtp` de `BN-hepar` (sin el arreglo, sus 384 valores pasaban a 0,25 y cambiaba el |

| | |
|---|---|
| | orden de sus padres) y sobre el Weibull de `Failure [1]` de `MID-hip-Briggs`; sin el arreglo fallan los dos. Necesita pantalla. `setEnabledDecisionOptions`, que vuelve a elegir por programa la clase de una política probabilista, deja de dejarla uniforme; a ese camino no se llega hoy (error 51). |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:228-256,488-522,658-669,675-683` `gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:243-269,314-339,406-437` `core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:55-59` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** En modo edición, al abrir las propiedades de un nodo (doble clic sobre él), la pestaña «Probability» (o «Utility») tiene el selector «Relation Type:», que ofrece las clases de relación aplicables al nodo («Table», «Tree/ADD», «Hazard (Weibull)»…). Si el usuario despliega el selector y pulsa la opción que ya aparece seleccionada, la tabla del nodo pasa a ser uniforme; en las demás clases, los parámetros vuelven a sus valores por omisión. Es un gesto fácil de hacer sin querer: abrir la lista para mirar qué opciones hay y cerrarla pinchando en la actual.

Qué pasa luego depende de lo que haga el usuario en el diálogo (`NodePropertiesDialog`):

- Con «OK», y también con solo **cambiar a otra pestaña** (`handleChangeTab` → `accept PotentialEditChanges`), `commitChanges` compara con el potencial original, ve que ha cambiado y ejecuta un `SetPotentialEdit`. Los números quedan perdidos en la red; la edición se puede deshacer desde el menú de edición si el usuario se da cuenta.
- Para no perderlos, tiene que pulsar el botón «Reset Probability» de la pestaña (que llama a `uncommitChanges` y repone el potencial original) o «Cancel» en el diálogo (que, según el código, deshace las ediciones hechas dentro de él con `cancelLastSubEditHistory`; esto no se ha ejecutado).

La subclase `ImposePolicyPanel` (imponer una política a una decisión) hereda el mismo selector y el mismo método.

La pestaña contiene un `PotentialEditPanel`, cuyo selector (`potentialTypeComboBox`) se suscribe con `addActionListener(evt -> this.potentialTypeChanged())`. Un `JComboBox` de Swing lanza el evento de acción cada vez que se elige un elemento, **también cuando es el que ya estaba elegido** (medido: dos eventos al volver a elegir dos veces el mismo elemento).

`potentialTypeChanged` no mira si la clase elegida es la que el nodo ya tiene. Si es «Table» y el potencial actual es una tabla (o uniforme), construye con `instanciatePotential` un `Table Potential` nuevo, cuyo constructor lo deja uniforme (`setUniform()`). Para cualquier otra clase, la última rama hace lo mismo con el constructor de esa clase, que deja los parámetros por omisión. Después pone ese potencial en el nodo (`setPotentialInNode`) y redibuja el panel: los números que tenía el nodo desaparecen de la vista y del nodo.

Medido con un pequeño programa de prueba sobre la clase real del panel. El programa fabrica el `PotentialEditPanel` sin ejecutar su constructor (el constructor crea un editor de comentarios que es un `JDialog`, y eso no se puede hacer sin pantalla), le da el nodo y elige en su selector real la clase que el nodo ya tiene:

```
0_miscellaneous/networks/bn/BN-hepar.pgmx, nodo ggtp: TablePotential, 384 valores, 341
↪  distintos, primeros [0.68, 0.144, 0.096, 0.08, 0.40357143, 0.13214286]
el selector muestra TablePotential; se vuelve a elegir TablePotential
después: TablePotential, 1 valor distinto, primeros [0.25, 0.25, 0.25, 0.25, 0.25, 0.25]
potentialHasChanged (lo que decide si se guarda al aceptar): true

0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx, nodo Failure [1]: coeficientes
↪  [0.3740968, -5.490935, -0.0367, 0.7685, -1.344474], con matriz de covarianzas
se vuelve a elegir WeibullHazardPotential
después: coeficientes [0, 0, 0, 0, 0], sin matriz de covarianzas
```

El redibujado posterior falla en el programa de prueba por no tener pantalla; el cambio del nodo ya se ha hecho antes.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:476-506

```
private void potentialTypeChanged() {
    Class<? extends Potential> potentialType = (Class<? extends Potential>)
    ↪  this.potentialTypeComboBox.getSelectedItem();
    Potential newPotential = null;
    var currentPotential = this.node.getPotential();
    if (potentialType == TablePotential.class && (currentPotential instanceof
    ↪  TablePotential || currentPotential instanceof UniformPotential)) {
        var variables = this.node.getParents()
                                 .stream()
                                 .map(Node::getVariable)
                                 .collect(Collectors.toCollection(ArrayList::new));
        variables.addFirst(this.node.getVariable());
        newPotential = this.instanciatePotential(potentialType, variables);
    }
    ...
    if (newPotential == null) {
        newPotential = this.instanciatePotential(potentialType,
↪   currentPotential.getVariables());
    }
    this.setPotentialInNode(newPotential);
    reloadPotentialPanel();
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*. Al principio de `potentialTypeChanged`, salir sin hacer nada si la clase elegida es exactamente la del potencial actual del nodo (`potentialType == currentPotential.getClass()`). El caso «Table» con un `UniformPotential` actual sí es un cambio de clase (esa rama convierte el uniforme en tabla), así que debe seguir haciéndose.

La comprobación debe estar en `potentialTypeChanged` y no en el oyente, para que cubra también a `ImposePolicyPanel`, que hereda el método, y a cualquier llamada programática a `setSelectedItem` sobre el selector (hoy `setEnabledDecisionOptions`, línea 616, lo hace para políticas probabilistas). Como alternativa, escuchar con un `ItemListener` que solo actúe en `ItemEvent.SELECTED`, que Swing no lanza si el elemento no cambia; pero la comprobación explícita protege también de llamadas directas.

Prueba de regresión: construir el panel sobre el nodo `ggtp` de `0_miscellaneous/networks/bn/BN-hepar.pgmx`, elegir otra vez «Table» y comprobar que los 384 valores no cambian y que `potentialHasChanged()` devuelve `false`; y lo mismo con un nodo «Hazard (Weibull)», como el de la medida.

### Error 151 · [Corregido] Cambiar en el panel de una regresión de la matriz de covarianzas a su descomposición de Cholesky no borra la matriz vieja, y al guardar la red se pierde lo escrito

| | |
|---|---|
| **Estado** | Corregido en el commit `ee2121c` (09-10-2026): `GLMPotential.setCholesky Decomposition` borra la matriz de covarianzas (opción (b), en el modelo, para que valga para cualquier llamador; el lector ProbModelXML y el constructor de copia llaman a uno solo de los dos métodos y hacen lo mismo que antes). Con prueba de las tres consecuencias, construyendo el panel sin mostrarlo: el nodo se queda solo con la descomposición, el panel reabierto dice «Cholesky decomposition» con los números escritos, el fichero guardado los lleva, y desmarcar «Uncertainty» quita la incertidumbre. No se ha probado en la ventana. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/GLMPotentialPan el.java:123-127,141-173` `core/src/main/java/org/openmarkov/core/model/network/potential/GLM Potential.java:180-191` `io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2 .java:1107-1115` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** Camino del usuario: abrir las propiedades de un nodo con potencial «Linear combination» o «Exponential» que tenga matriz de covarianzas → marcar «Uncertainty» si no lo está → cambiar el selector a «Cholesky decomposition» → escribir los números → aceptar (y guardar la red). Consecuencias:

1. Al volver a abrir el panel, el selector vuelve a «Covariance matrix» con los números viejos. Lo que se ve y lo que el modelo usa para muestrear dejan de coincidir.
2. Al guardar la red, el fichero lleva la matriz vieja; al volver a abrirlo, la descomposición se recalcula a partir de ella. **Lo que escribió el usuario se pierde.**
3. Con el selector en «Cholesky decomposition», desmarcar «Uncertainty» no quita la incertidumbre: la matriz de covarianzas vieja sobrevive.

Un potencial de regresión (`GLMPotential`) puede llevar incertidumbre sobre sus coeficientes dada de dos maneras: la matriz de covarianzas (`covarianceMatrix`) o su descomposición de Cholesky (`choleskyDecomposition`, una matriz triangular L tal que $L \cdot L^T$ es la de covarianzas). El muestreo usa siempre la de Cholesky. `setCovarianceMatrix` guarda la matriz y recalcula la descomposición; `setCholeskyDecomposition` guarda la descomposición pero **no toca** la matriz de covarianzas.

`GLMPotentialPanel` (el panel de «Linear combination»/«Linear regression» y «Exponential») ofrece el selector «Covariance matrix» / «Cholesky decomposition». Al aceptar, `saveChanges` hace una copia del potencial actual —que conserva las dos matrices— y escribe solo la del tipo elegido. Si el nodo tenía matriz de covarianzas y el usuario cambia a «Cholesky decomposition» y escribe la descomposición, la copia se queda con la matriz de covarianzas vieja y la descomposición nueva. De ahí salen las tres consecuencias:

1. Al reabrir el panel, `setData` mira primero la matriz de covarianzas, y por eso el selector vuelve a «Covariance matrix».
2. El escritor ProbModelXML escribe la matriz de covarianzas si existe, y solo si no existe, la descomposición.

3. Desmarcar «Uncertainty» con el selector en «Cholesky decomposition» escribe `setCholesky Decomposition(null)`, que deja intacta la matriz de covarianzas.

Medido de extremo a extremo con un pequeño programa de prueba sobre el nodo «Age at state entry [0]» de `0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx` (potencial «Linear combination», tres covariables), al que se le pone antes una matriz de covarianzas identidad; se construye el panel sin pantalla, se cambia el selector, se escribe la descomposición y se acepta:

```
abierto: selector=Covariance matrix  celda(1,1)=1.0
tras aceptar: covarianzas=[1,0,1,0,0,1]  Cholesky=[2,0.5,3,0.25,0.125,4]
reabierto: selector=Covariance matrix  triángulo=1 0 1 0 0 1
guardado con PGMXWriter_1_0 y leído: covarianzas=[1,0,1,0,0,1]  Cholesky=[1,0,1,0,0,1]
```

`WeibullPotentialPanel` (paneles de «Hazard (Weibull)» y «Hazard (Exponential)») no tiene este defecto: construye un potencial nuevo con el constructor que recibe el tipo de matriz, y así solo existe la que se eligió.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/GLMPotentialPanel.java:141-169

```java
public boolean saveChanges() throws DoEditException {
    GLMPotential newPotential = (GLMPotential) this.potential.copy();
    ...
    String matrixTypeName = matrixTypeComboBox.getSelectedItem().toString();
    newPotential.setCovariates(covariates);
    newPotential.setCoefficients(coefficients);
    if (matrixTypeName.equals(MATRIX_TYPE_COVARIANCE)) {
        newPotential.setCovarianceMatrix(uncertaintyMatrix);
    } else {
        newPotential.setCholeskyDecomposition(uncertaintyMatrix);
    }
```

core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:180-191

```java
public void setCovarianceMatrix(double[] covarianceMatrix) {
    this.covarianceMatrix = covarianceMatrix;
    this.choleskyDecomposition = calculateCholesky(covarianceMatrix);
}
...
public void setCholeskyDecomposition(double[] choleskyDecomposition) {
    this.choleskyDecomposition = choleskyDecomposition;
}
```

**Cómo arreglarlo.** Que dar la incertidumbre de una manera borre la otra. Hay dos lugares posibles, con consecuencias distintas:

- (a) En el panel: poner a `null` la matriz de covarianzas antes de `setCholeskyDecomposition`, o construir el potencial nuevo con el constructor que recibe `MatrixType`, como ya hace `WeibullPotentialPanel`.
- (b) En el modelo: que `setCholeskyDecomposition` ponga `covarianceMatrix = null`. Esta opción cubre a cualquier llamador, pero el lector ProbModelXML (`PGMXPotential Parsers.java:219-224`) y el constructor de copia de `GLMPotential` (líneas 82-90) llaman a esos métodos y hay que confirmar que siguen haciendo lo mismo (hoy solo llaman a uno de los dos, así que no cambian).

Prueba de regresión: la misma secuencia de la medida —potencial con covarianzas, cambiar a Cholesky, aceptar— y comprobar que `getCovarianceMatrix()` es `null`, que al reabrir el selector

dice «Cholesky decomposition» con los números escritos, y que tras escribir y leer el fichero la descomposición es la escrita.

### Error 152 · [Corregido] Añadir una fila a la tabla de vida y aceptar sin tocarla borra la probabilidad del primer tramo

| | |
|---|---|
| **Estado** | Corregido en el commit `0dd53c0` (14-10-2026). «Add» crea la fila con un tiempo una unidad mayor que el mayor de la tabla (0 si está vacía) y probabilidad 0, así que nunca pisa otro tramo. Escribir en una celda un tiempo que ya tiene otra fila se rechaza: la celda conserva su valor y un mensaje («Tiempo repetido») lo explica; se comparan los números, así que «5» y «5.0» son el mismo tiempo. Se quitan los botones de subir y bajar, porque los tramos se guardan siempre en el orden de sus tiempos; el de cargar un fichero de datos se queda. Lo que no es un número sigue rompiendo el guardado, como en el error 164. Medido con la prueba sobre el panel real: con la tabla `{0.0=0.1, 5.0=0.2}`, añadir una fila y guardar da `{0.0=0.1, 5.0=0.2, 6.0=0.0}`. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/PiecewiseExponentialTablePanel.java:32,70-90,145-147`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/PiecewiseExponentialPanel.java:103-121`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTablePanel.java:400-413` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 164, error 184, nota 13 |

**Qué pasa.** El panel de la tabla de vida (`PiecewiseExponentialPanel` con su tabla `PiecewiseExponentialTablePanel`, módulo `gui`) enseña una fila por tramo con dos columnas visibles, «Time» y «Probability», y botones para añadir, quitar, subir y bajar filas.

- **Añadir** una fila y aceptar sin editarla deja a 0 la probabilidad del primer tramo, sin aviso. Lo mismo ocurre si se añaden dos filas y se edita solo una.
- **Subir y bajar** cambian el orden de las filas en la tabla, pero ese orden no se conserva: al reabrir el diálogo las filas vuelven a salir ordenadas por tiempo. Son dos botones que no hacen nada que se conserve.

Al aceptar, `saveChanges` (líneas 103-121) recorre las filas y las mete en un `TreeMap<Double, Double>` con el tiempo como clave.

`actionPerformedAddValue` (líneas 145-147) añade la fila `{0, 0, 0}`, es decir, tiempo 0 y probabilidad 0. Al volcar las filas en el mapa, la fila nueva (que va la última) sustituye a la que ya empezaba en el tiempo 0, que es lo habitual en una tabla de vida.

Los botones de subir y bajar se heredan de `KeyTablePanel` (líneas 400-413); el orden que dan se pierde al volcar las filas en el mapa ordenado por tiempo.

Medido con un pequeño programa de prueba sobre el panel real, con un nodo de suceso de una red de simulación de eventos discretos (DES) cuya tabla es `{0.0=0.1, 5.0=0.2}`: tras añadir una fila y guardar, la tabla queda `{0.0=0.0, 5.0=0.2}`.

Los textos del panel («Use rates», «Use first interval», «Life Table», «Time», «Probability», «Load data file», «Choose a CSV data file») están escritos en el código, pero eso no forma parte de este

error.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/PiecewiseExponentialTablePanel.java:145-147

```
protected void actionPerformedAddValue(ActionEvent e) {
    tableModel.addRow(new Object[]{0, 0, 0});
}
```

gui/src/main/java/org/openmarkov/gui/dialog/common/PiecewiseExponentialPanel.java:109-115

```
TreeMap<Double, Double> treeMapTable = new TreeMap<>();
for (int i = 0; i < piecewiseExponentialTablePanel.valuesTable.getRowCount(); i++) {
    treeMapTable.put(Double.parseDouble(... getValueAt(i, 1, null).toString()),
                     Double.parseDouble(... getValueAt(i, 2, null).toString()));
}
```

**Cómo arreglarlo.** Que la fila nueva proponga un tiempo que no exista (por ejemplo, el último tiempo más uno) y que `saveChanges` rechace con un mensaje dos filas con el mismo tiempo en vez de quedarse con la última.

Los botones de subir y bajar pueden quitarse del panel (pasando `reorderable=false` al constructor de `KeyTablePanel`, línea 32), o la tabla puede ordenarse sola por tiempo.

Como en error 164, `saveChanges` usa `Double.parseDouble` sin comprobar el texto.

Prueba: construir el panel sobre una tabla que empiece en 0, añadir una fila, guardar y comprobar que la probabilidad del tramo 0 no cambia (o que el guardado se rechaza por tiempo repetido).

### Error 153 · [Corregido] El editor de una distribución univariante enseña «1» en cada parámetro cuando el fichero solo trae los números, y editarlo separa las dos copias de los parámetros

| | |
|---|---|
| **Estado** | Corregido en el commit `9ad0a39` (09-10-2026). La regla es una sola y vive en `UnivariateDistrPotential`: la expresión de un parámetro que es un número es el número con que se calcula. Se cumple al leer (`setParameters`: los números del fichero pasan a las expresiones, salvo en las casillas con fórmula), al editar (`setParameter`: si lo escrito es un número, cambia también el número) y al escalar. Si un fichero trae las dos copias y no coinciden, manda el número, que es con lo que ya se calculaba. Medido antes del arreglo: escribir «2» en la primera casilla de «Effectiveness» cambiaba solo la expresión y la utilidad esperada seguía en 53000; ahora pasa a 60000. El escalado se ve en «Expand network»: en `MID-Chancellor-1-0.pgmx`, «Direct medical cost» del ciclo 1 se calcula con 6554,7 y el diálogo enseñaba «1»; ahora enseña 6554,7. Visto en la ventana el 09-10-2026, abriendo el diálogo de la relación con un guion (no desde el menú): «Effectiveness» enseña 0.8 y 1.3. Escribir en una casilla desde la ventana no se podía, por el error 225, corregido después. |
| **Módulo** | core, gui, io |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.gui.dialog.common; org.openmarkov.gui.action; org.openmarkov.io.probmodel.reader |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/UnivariateDistrPotential.java:48,330-343`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:126-166`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/UnivariateDistrPotentialPanel.java:231-248`<br>`gui/src/main/java/org/openmarkov/gui/action/AugmentedPotentialValu` |

| | |
|---|---|
| | eEdit.java:97-118,236-238<br>io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_1_0.java:266-289 |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | error 9, error 112 |

**Qué pasa.** Al abrir el editor de la relación de un nodo cuya distribución univariante se leyó de un fichero que no trae `<Functions>`, el usuario ve «1» en cada casilla de parámetro en lugar de los números del fichero. Por ejemplo, en `0_miscellaneous/networks/id/1-0/ID-CEA-minimal-1-0.pgmx`, la relación de «Effectiveness» enseña «1» en las dos casillas en lugar de 0.8 y 1.3. No se ha abierto el diálogo: la conclusión sale de la lectura del panel y de la medida sobre el potencial leído.

Añadido el 09-10-2026: visto el 08-10-2026 en una captura automática del diálogo. En `0_miscellaneous/networks/id/1-0/ID-mediastinet-1-0.pgmx`, la relación del nodo de utilidad `Survivors_QALE` enseña «1» en sus seis casillas, y el fichero trae 2.0833, 0.4167, 2.64, 1.25, 5.75 y 1.1667.

Además, si el usuario escribe en una casilla, cambia solo una de las dos copias de los parámetros, y el fichero guardado puede acabar diciendo cosas distintas en cada una.

Un `UnivariateDistrPotential` (módulo `core`) es una relación cuyo valor sigue una distribución de probabilidad sobre un número (por ejemplo «Exact», un valor fijo), con los parámetros por cada combinación de estados de los padres. Los parámetros se guardan dos veces en su tabla aumentada (`AugmentedProbTable`): como números (`getValues()`) y como expresiones (`getFunctionValues()`), que permiten escribir fórmulas que nombran otras variables. Al crear la tabla, `initializeAugmentedProbTable` pone todas las expresiones a «1» (`INITIALIZATION_VALUE`, línea 48).

El lector (`PGMXReader_1_0.getUnivariateDistrPotential`, líneas 126-166) solo rellena las expresiones si el fichero trae `<Functions>`, y siempre pone en la tabla los números leídos de `<Parameters>` o `<Values>` (el orden de estas dos cosas se corrigió en el commit `51a0cd6`). Si el fichero no trae `<Functions>`, las expresiones siguen valiendo «1».

`UnivariateDistrPotentialPanel` (el panel registrado para esta clase) llena la tabla con `getTablePotential().getFunctionValues()` (línea 237), es decir, con las expresiones.

Al escribir en una casilla, `AugmentedPotentialValueEdit` modifica solo las expresiones (`newAugmentedValues = newAugmentedProbTable.getFunctionValues()`, líneas 108-116 y 238) y deja los números como estaban. El escritor (`PGMXWriter_1_0.getUnivariateDistrPotential`, líneas 266-274) guarda las dos copias: `<Functions>` con las expresiones y `<Parameters>` con los números. Qué copia usa cada consumidor (inferencia, simulación, análisis de sensibilidad) no se ha revisado.

Medido con un pequeño programa de prueba sobre `0_miscellaneous/networks/id/1-0/ID-CEA-minimal-1-0.pgmx`, cuyo potencial de «Effectiveness» es `<Potential type="UnivariateDistr" distribution="Exact">` con `<Values>0.8 1.3</Values>`: tras leer, números `[0.8, 1.3]` y expresiones `[1, 1]`; en «Cost», números `[0.0, 14000.0]` y expresiones `[1, 1]`.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/UnivariateDistrPotentialPanel.java:231-246

```
    @Override protected Object[][] setPotentialDataInCentreArea(Object[][] oldValues) {
        Object[][] values = oldValues;
        int numColumns = values[0].length;
        // rounding initial values
        VariableExpression[] initialValues = getTablePotential().getFunctionValues();
        for (int j = 1; j <= numColumns - 1; j++) {
```

```
            for (int i = getLastEditableRow(); i >= getFirstEditableRow(); i--) {
                int potentialIndex = PotentialsTablePanelOperations.getPotentialIndex(i, j,
                ↪  getTablePotential());
                VariableExpression value = initialValues[potentialIndex];
                values[i][j] = value;
            }
        }
        return values;
    }
```

**Cómo arreglarlo.** Hay que fijar una sola regla para las dos copias y cumplirla en los tres sitios:

- Al leer (`PGMXReader_1_0.getUnivariateDistrPotential`): sin `<Functions>`, construir las expresiones a partir de los números leídos, en lugar de dejar «1».
- Al enseñar (`UnivariateDistrPotentialPanel.setPotentialDataInCentreArea`).
- Al editar (`AugmentedPotentialValueEdit`): cuando la expresión escrita es un número, actualizar también el número de esa casilla.

Conviene revisar `AugmentedProbTablePotentialPanel` (línea 279), que lee las expresiones igual.

Prueba: leer `0_miscellaneous/networks/id/1-0/ID-CEA-minimal-1-0.pgmx` y comprobar que expresiones y números de «Effectiveness» coinciden (0.8 y 1.3); editar una casilla con «2» y comprobar que las dos copias valen 2 y que el fichero guardado las escribe iguales.

### Error 154 · [Jorge Rico] En la tabla de un nodo de utilidad, una restricción de enlace bloquea las casillas equivocadas y deja editables las prohibidas

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core, gui |
| **Paquete** | org.openmarkov.core.model.network.potential.operation; org.openmarkov.gui.dialog.common |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/operation/LinkRestrictionPotentialOperations.java:70-113,403-416`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TablePotentialPanel.java:579-602,616-640`<br>`gui/src/main/java/org/openmarkov/gui/validator/LinkRestrictionValidator.java:35-58` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 25, error 155 |

**Qué pasa.** Una restricción de enlace declara qué combinaciones de estados de un padre y del hijo son incompatibles. En la tabla del hijo, las casillas de esas combinaciones salen en rojo y bloqueadas. Cuando el hijo es un nodo de utilidad, el usuario ve en rojo y bloqueadas casillas que no tienen nada que ver, y puede escribir en las que la restricción prohíbe.

Camino de llegada: `LinkRestrictionValidator.validate` permite la restricción en un enlace de un padre de estados finitos a un nodo de utilidad. La restricción `NoLinkRestriction` está activa por defecto y solo la desactivan las redes de análisis de decisiones (DAN), las DAN de Markov y las redes de ajuste. En esas redes: botón derecho sobre el enlace → editar la restricción, marcar una combinación incompatible, y abrir la tabla del nodo de utilidad. No se ha buscado si alguna

red del repositorio trae ya una restricción así.

El panel de la tabla (`TablePotentialPanel.getNotEditablePositions`, líneas 597-611) pide a `LinkRestrictionPotentialOperations.getStateCombinationsWithLinkRestriction` (módulo `core`) las combinaciones prohibidas, las traduce a fila y columna con `getRowAndColumnForStateCombination` (líneas 560-583), las bloquea y el pintor las pone en rojo.

En un nodo de utilidad la tabla es un `ExactDistrPotential`, y la función toma su tabla interior, cuyas variables son solo los padres (la variable de utilidad no está). Así lo tienen todos los nodos de utilidad de tabla de las redes del repositorio que se miraron.

`getStateCombinationsWithLinkRestriction` busca los dos extremos del enlace en esa lista con índices que empiezan en 0 y solo se asignan si la variable aparece. La variable de utilidad (`var2`) no aparece, así que `var2Index` se queda en 0 (el primer padre), y `getStateCombinations` hace `combination[var1Index] = var1Value; combination[var2Index] = var2Value;`, que escribe el estado de la utilidad (siempre 0) encima del primer padre, en un vector con una posición de más.

Además, `getRowAndColumnForStateCombination` calcula la columna recorriendo las variables desde la segunda, porque supone que la primera es la del nodo; en la tabla interior eso salta el primer padre.

Medido con un pequeño programa de prueba sobre `0_miscellaneous/networks/dan/DAN-3-test-problem.pgmx`, nodo de utilidad «Quality of life», padres Disease (absent, present) y Therapy (no, yes); las columnas de la tabla son (absent,no), (absent,yes), (present,no), (present,yes):

- restricción «Therapy = yes incompatible»: se bloquea solo (present,no); deberían bloquearse (absent,yes) y (present,yes);
- restricción «Disease = present incompatible»: se bloquean (absent,no) y (present,no); deberían bloquearse (present,no) y (present,yes);
- restricción «Disease = absent incompatible»: sale lo mismo que con «present»; el estado del padre se pierde.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/operation/LinkRestrictionPotentialOperations.java:89-107

```
            Map<Integer, Integer> independentVariables = new HashMap<>();
            int var1Index = 0, var2Index = 0;
            for (int i = 0; i < nodeVariables.size(); i++) {
                Variable variable = nodeVariables.get(i);
                if (variable.equals(var1)) {
                    var1Index = i;
                } else {
                    if (variable.equals(var2)) {
                        var2Index = i;
                    } else {
                        independentVariables.put(i, variable.getNumStates());
                    }
                }
            }
            ...
                        stateList.addAll(LinkRestrictionPotentialOperations
                                .getStateCombinations(independentVariables, nodeVariables,
                                ↪  i, var1Index, j, var2Index));
```

**Cómo arreglarlo.** Hay que corregir los dos sitios a la vez:

- En `getStateCombinationsWithLinkRestriction`/`getStateCombinations`, construir las combinaciones con tantas coordenadas como variables tenga la tabla que se mira, y fijar solo las

de las variables que están en ella (la de utilidad no aporta coordenada, así que cada estado prohibido del padre bloquea la columna entera).
- En `TablePotentialPanel.getRowAndColumnForStateCombination`, calcular la columna a partir de las variables de los padres de la tabla que se mira, sin suponer que la primera es la del nodo cuando la tabla es la interior de un `ExactDistrPotential` (la fila, en utilidad, es siempre la única).

Prueba: el caso medido arriba, comprobando que «Therapy = yes» bloquea exactamente las columnas (absent,yes) y (present,yes), y una prueba equivalente con un nodo de azar para no romper el caso que hoy funciona.

### Error 155 · [Jorge Rico] Pulsar una casilla de la ventana de restricción de un enlace edita la tabla del padre: cambia en silencio una de sus probabilidades o lanza una excepción, y si el padre es una decisión la ventana ni siquiera se abre

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.component; org.openmarkov.gui.action; org.openmarkov.gui.dialog.link |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/component/LinkRestrictionValuesTable.java:48-87`<br>`gui/src/main/java/org/openmarkov/gui/component/ValuesTable.java:142-149,170-207`<br>`gui/src/main/java/org/openmarkov/gui/action/TablePotentialValueEdit.java:144-157`<br>`gui/src/main/java/org/openmarkov/gui/component/PotentialsTablePanelOperations.java:48-75,204-207`<br>`gui/src/main/java/org/openmarkov/gui/dialog/link/LinkRestrictionPanel.java:85-91,140-154`<br>`gui/src/main/java/org/openmarkov/gui/dialog/link/LinkRestrictionEditDialog.java:39-48,99-108`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:252-264`<br>`gui/src/main/java/org/openmarkov/gui/validator/LinkRestrictionValidator.java:35-58` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 25, error 27, error 154, nota 23 |

**Qué pasa.** En el menú contextual de un enlace, «Add restrictions» (o «Edit restrictions», si ya tiene) abre la ventana «Link Restriction: Link between … and …»: una casilla por cada pareja de un estado del padre (columnas) y un estado del hijo (filas), que se marca como incompatible (0) o compatible (1) al pulsarla. La ventana se ofrece para enlaces entre nodos de azar o de decisión de estados finitos y para enlaces hacia un nodo de utilidad (`LinkRestrictionValidator.validate`). Pulsar una casilla, además de cambiar la restricción, edita la tabla del **padre**, en una casilla que depende de dónde se pulsó. Lo que ve el usuario:

- Si el padre es una decisión sin política impuesta, la ventana no se abre: sale la ventana de error no previsto con `NoSuchElementException`. Los nodos de decisión no tienen potencial, ni los que se crean en el editor ni los que se leen de un fichero. Pasa, por ejemplo, con

«Do test?» → «Result of test» en `0_miscellaneous/networks/id/ID-decide-test.pgmx` y, con «Edit restrictions», en el mismo enlace de `0_miscellaneous/networks/dan/DAN-decide-test.pgmx`. En las redes en formato 0.2 de `0_miscellaneous/networks/dan` hay 107 enlaces con restricción que salen de una decisión, repartidos en 31 redes, y no se puede abrir la ventana de ninguno.
- Si el padre tiene tabla, según la casilla pulsada, una probabilidad del padre pasa a 0 o a 1 sin ningún aviso, o sale la ventana de error no previsto con `ArrayIndexOutOfBoundsException`. En este segundo caso la restricción ya ha cambiado, pero la tabla del hijo no llega a tocarse (lo que sí pasa en las demás casillas es error 25).

El cambio de la tabla del padre es una edición normal dentro de la sub-historia de ediciones que abre la ventana: «Cancel», o deshacer después de «OK», lo devuelven. Con «OK» se queda, la red calcula con él y se guarda así.

Medido con un pequeño programa de prueba que construye el panel y la tabla reales de la ventana (`LinkRestrictionPanel`, sin construir la ventana) y pulsa cada casilla como lo hace el panel:

- `0_miscellaneous/networks/bn/BN-asia.pgmx`, VisitToAsia → Tuberculosis. En la fila de «Tuberculosis = yes», con cualquiera de las dos columnas, la tabla de VisitToAsia pasa de `[0.99, 0.01]` a `[0.0, 1.0]`; tras «OK», la propagación da P(VisitToAsia = yes) = 1; tras «Cancel», o tras «OK» y deshacer, vuelve a `[0.99, 0.01]`. En la fila de «Tuberculosis = no», las dos casillas lanzan `ArrayIndexOutOfBoundsException: Index -1 out of bounds for length 2`.
- La misma red, Tuberculosis → TuberculosisOrCancer. Cada casilla pone a 0 o a 1 una columna de la tabla de Tuberculosis, `[0.99, 0.01, 0.95, 0.05]` (P(Tuberculosis | VisitToAsia)). Por ejemplo, pulsar «Tuberculosis = no, TuberculosisOrCancer = no» la deja en `[0.0, 1.0, 0.95, 0.05]`: quien no ha estado en Asia tiene tuberculosis con probabilidad 1.
- `0_miscellaneous/networks/id/ID-decide-test.pgmx`, Disease → Health state (un nodo de utilidad): las dos casillas dejan Disease en `[0.0, 1.0]` en lugar de `[0.86, 0.14]`.
- La misma red, Result of test → Therapy (el hijo es una decisión): cuatro de las seis casillas no cambian nada; «positive, yes» deja a cero una columna entera de la tabla de Result of test (de `[1.0, 0.0, 0.0]` a `[0.0, 0.0, 0.0]`) y «positive, no» cambia otra de `[0.0, 0.97, 0.03]` a `[0.03, 0.97, 0.0]`.
- «Do test?» → «Result of test» en `ID-decide-test.pgmx` (sin restricción) y en `DAN-decide-test.pgmx` (con restricción): construir el panel lanza `NoSuchElementException` en `ValuesTable`, línea 149.

Por qué pasa. `LinkRestrictionValuesTable` (módulo `gui`) es una subclase de `ValuesTable`, la tabla con que se editan los valores de un nodo, y le pasa como nodo el padre del enlace (`super(link.getFrom(), tableModel, modifiable)`, línea 49):

- El constructor de `ValuesTable` toma el primer potencial de ese nodo (`node.getPotentials().getFirst()`, línea 149); con un padre sin potencial lanza `NoSuchElementException`. `LinkRestrictionPanel.getLinkRestrictionValuesTable` (línea 87) construye la tabla al montar la ventana, así que la ventana no llega a verse.
- `LinkRestrictionValuesTable.setValueAt` (líneas 54-81) ejecuta la edición de la restricción y luego llama a `super.setValueAt(newNumericValue, row, column, source)` (línea 64). Es la versión de `ValuesTable` (líneas 170-207), que crea un `TablePotentialValueEdit` sobre `this.node`, el padre, con la fila y la columna de la ventana de restricción. `TablePotentialValueEdit` (líneas 144-157) las traduce a una posición de la tabla del padre como si vinieran de la ventana de la tabla del propio padre (`PotentialsTablePanelOperations.getPotentialIndex`, líneas 67-75: el comienzo de la columna más la última fila editable del padre menos la fila pulsada). El resultado es una casilla cualquiera del padre, o −1, y `redistribute`

`Probabilities` pone ahí el valor pulsado y reparte la columna, o falla en su línea 207.

La casilla de la ventana no necesita esa llamada para actualizarse: la tabla escucha las ediciones de la red (`ValuesTable`, línea 146) y `afterEditExecutes` (líneas 83-87) ya pone el valor nuevo en el modelo (por lectura).

**El código.** gui/src/main/java/org/openmarkov/gui/component/LinkRestrictionValuesTable.java:48-64

```
public LinkRestrictionValuesTable(Link<Node> link, ValuesTableModel tableModel, final
↪  boolean modifiable) {
    super(link.getFrom(), tableModel, modifiable);
    this.link = link;
    node2 = link.getTo();
}

@Override public void setValueAt(Object newValue, int row, int column, Object source) {
    ...
    LinkRestrictionPotentialValueEdit linkPotentialEdit =
            new LinkRestrictionPotentialValueEdit(link, newNumericValue, row, column);
    try {
        linkPotentialEdit.executeEdit();
        super.setValueAt(newNumericValue, row, column, source);
```

gui/src/main/java/org/openmarkov/gui/component/ValuesTable.java:142-149, 192-195

```
public ValuesTable(Node node, ValuesTableModel tableModel, final boolean modifiable) {
    super(tableModel, modifiable, false, true);
    this.tableModel = tableModel;
    if (node != null) {
        node.getProbNet().getPNESupport().addListener(this);
        this.node = node;
        this.probNet = node.getProbNet();
        this.potential = node.getPotentials().getFirst();
    ...
        new TablePotentialValueEdit(this.node, (Double) newValue, row, column,
        ↪  this.priorityList,
                                    this.getTableModel().getNotEditablePositions())
                .executeEdit();
```

**Cómo arreglarlo.** La tabla de la ventana de restricción no debe estar ligada al padre como si enseñara sus valores:

- No pasar el padre al constructor de `ValuesTable`. Con `null`, el constructor ya no toma potenciales, pero tampoco registra la tabla como oyente de las ediciones, así que habría que registrarla en la red del enlace para que siga recibiendo `afterEditExecutes` y `afterUndoing Edit`. La otra forma es que `LinkRestrictionValuesTable` herede de una tabla que no sea la de los valores de un nodo.
- Sustituir `super.setValueAt(...)` (línea 64) por poner el valor en el modelo de la tabla, como ya hace `afterEditExecutes`.

`LinkRestrictionValuesTable` es la única subclase de `ValuesTable` que recibe un nodo distinto del que enseña. En el mismo método está error 25 (la tabla del hijo, cambiada fuera del historial): conviene arreglar los dos a la vez.

Prueba de regresión: construir `LinkRestrictionPanel` sobre «Do test?» → «Result of test» de `0_miscellaneous/networks/dan/DAN-decide-test.pgmx` sin excepción; sobre VisitToAsia → Tuberculosis de `0_miscellaneous/networks/bn/BN-asia.pgmx`, pulsar las cuatro casillas: ninguna

debe lanzar excepción y la tabla de VisitToAsia debe seguir en `[0.99, 0.01]`; y sobre Disease → Health state de `0_miscellaneous/networks/id/ID-decide-test.pgmx`, Disease debe seguir en `[0.86, 0.14]`.

### Error 156 · [Jorge Rico] La tabla de un nodo de utilidad enseña 922 337 203,685478 en lugar de cualquier valor mayor, en valor absoluto, que unos 922 millones

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core, gui |
| **Paquete** | org.openmarkov.core.model.network; org.openmarkov.gui.dialog.common |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/Util.java:288-303,318-327`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TablePotentialPanel.java:521-533`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TableWithEventsPanel.java:495` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** Camino de llegada: propiedades de un nodo de utilidad → pestaña de la tabla, en una red cuyo valor de utilidad supere unos 922 millones en valor absoluto (un coste en euros de un programa, por ejemplo). Todas esas casillas enseñan 922337203.685478 (o −922337203.685478 si el valor es negativo).

El modelo no cambia: al teclear 2000000000 en una casilla, el potencial guarda $2{,}0 \cdot 10^9$ y la casilla lo enseña bien hasta que la tabla se vuelve a construir (por ejemplo, al reabrir el diálogo del nodo), cuando vuelve a salir 922337203.685478.

`Util.roundWithPrecision(double x, int numDecimals)` (módulo `core`) redondea calculando `Math.round(x * 10^numDecimals) / 10^numDecimals`. `Math.round(double)` devuelve un `long` y, cuando el producto pasa de 9 223 372 036 854 775 807, devuelve ese máximo en lugar de desbordar. `Util.roundAndReduce` empieza probando con 10 decimales, así que todo valor cuyo valor absoluto supere unos 922 337 203 ($9{,}22 \cdot 10^{18}$ / $10^{10}$) sale convertido en ±922 337 203,6854776.

`TablePotentialPanel.setPotentialDataInCentreArea` (líneas 502-514) pasa por `roundAndReduce(valor, 10⁻¹², 10)` todos los valores antes de ponerlos en la tabla. Esto afecta a la tabla de cualquier nodo que admita números grandes; en la práctica, a los nodos de utilidad, cuya tabla es un `ExactDistrPotential` que este mismo panel enseña.

`TableWithEventsPanel` (línea 495) redondea igual los valores de las tablas con sucesos. Los demás usos de `roundAndReduce` (`PotentialsTablePanelOperations.redistributeProbabilities`, `EventTablePotentialValueEdit`, `AugmentedPotentialValueEdit`) trabajan con probabilidades, que no llegan a esos tamaños.

Medido con un pequeño programa de prueba sobre `0_miscellaneous/networks/dan/DAN-3-test-problem.pgmx`, nodo de utilidad «Quality of life», poniendo en su tabla 1 000 000 000, 1 234 567 890, $10^{15}$ y −1 000 000 000: el panel enseña 922337203.685478 en las tres primeras casillas y −922337203.685478 en la cuarta.

**El código.** core/src/main/java/org/openmarkov/core/model/network/Util.java:318-327

```java
public static double roundWithPrecision(double x, int numDecimals) {
    double xRounded;
    double scale = Math.pow(10, numDecimals);
    xRounded = Math.round(x * scale);
    xRounded = xRounded / scale;
    return xRounded;
}
```

**Cómo arreglarlo.** Arreglarlo en `Util.roundWithPrecision`, que es donde está el defecto y lo heredan todos sus usos. Dos formas:

- Si `|x * scale|` no cabe en un `long` (o el valor no tiene parte decimal representable a esa escala), devolver `x` sin tocar.
- Redondear con `BigDecimal.valueOf(x).setScale(numDecimals, RoundingMode.HALF_UP)`, como ya hace `roundWithSignificantFigures`.

Sitios que dependen de ello: `TablePotentialPanel.setPotentialDataInCentreArea` y `TableWithEventsPanel` (línea 495).

Prueba: `roundAndReduce(1e9, 1e-12, 10) == 1e9`, `roundAndReduce(-1e15, 1e-12, 10) == -1e15`, y que los redondeos pequeños de siempre no cambian (por ejemplo `roundAndReduce(0.30000000000000004, 1e-12, 10) == 0.3`).

### Error 157 · [Corregido en parte] [Jorge Rico] El panel de un riesgo de Weibull deja borrar la covariable «Gamma», y la relación se queda sin forma y revienta al calcular

| | |
|---|---|
| **Estado** | Presente en parte. El commit `522d926` (23-09-2026) hizo que el panel pregunte al potencial que edita por sus covariables obligatorias, así que ya no deja quitar ni reescribir `Gamma` en un riesgo de Weibull; sigue apagándose el botón de añadir al elegir una fila obligatoria. |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/GLMPanel.java:130-143,158-178`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/GLMPotential.java:128-130`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:40-43,122-124,137,176,278-281` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 14, error 158 |

**Qué pasa.** Camino del usuario: propiedades del nodo «Failure» de `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx` (o de cualquier nodo con «Hazard (Weibull)») → elegir la fila «Gamma» → quitar → aceptar → evaluar la red. Pasa lo siguiente:

1. Con la fila «Gamma» elegida, el botón de quitar está encendido. Si el usuario la quita y acepta, `WeibullPotentialPanel.saveChanges` crea un Weibull sin «Gamma». Después, `getGammaIndex` devuelve −1: `getGamma()` y `tableProject` (línea 176, `Math.exp(coefficients[gammaIndex])`) lanzan `ArrayIndexOutOfBoundsException` al evaluar la red, y el muestreo lanza `InvalidArgumentException` («no covariate declares the shape parameter gamma»).

2. Con una covariable obligatoria elegida («Constant»), además del botón de quitar se apaga el de **añadir**, que no tiene relación con la fila elegida: añadir una covariable no le hace nada a la que esté seleccionada.

Cada clase de regresión tiene covariables que no pueden faltar. Para `GLMPotential` es «Constant» (método estático `getMandatoryCovariates`). Para `WeibullHazardPotential` («Hazard (Weibull)») son «Gamma» y «Constant» (campo estático `MANDATORY_COVARIATES`): el coeficiente de «Gamma» es el logaritmo del parámetro de forma de la distribución de Weibull.

`GLMPanel.valueChanged` decide, al elegir una fila, si se pueden usar los botones, y pregunta siempre a `GLMPotential.getMandatoryCovariates()`, aunque el potencial que edita sea un Weibull.

El oyente de doble clic sobre la columna de covariables (`CovariatesTableMouseListener`, que permite reescribir una covariable) usa la misma lista, así que también dejaría reescribir «Gamma»; hoy esa columna no se ve (error 158).

Medido con un pequeño programa de prueba que construye sin pantalla el `WeibullPotential Panel` del nodo «Failure [1]» de `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx` y elige cada fila:

```
fila 0 Gamma            -> añadir=on  quitar=on  subir=off bajar=on
fila 1 Constant         -> añadir=off quitar=off subir=on  bajar=on
fila 2 {Age at entry}   -> añadir=on  quitar=on  subir=on  bajar=on
getGamma de un Weibull sin Gamma: ArrayIndexOutOfBoundsException: Index -1 out of bounds
↪  for length 4
```

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/GLMPanel.java:130-143

```java
@Override public void valueChanged(ListSelectionEvent e) {
    super.valueChanged(e);
    int row = valuesTable.getSelectedRow();
    if (row >= 0 && row < tableModel.getRowCount()) {
        VariableExpression covariate = (VariableExpression) tableModel.getValueAt(row, 0);
        boolean isMandatory = false;
        VariableExpression[] mandatoryCovariates = GLMPotential.getMandatoryCovariates();
        for (VariableExpression mandatoryCovariate : mandatoryCovariates) {
            isMandatory |=
↪   mandatoryCovariate.asStringExpression().equals(covariate.asStringExpression());
        }
        setEnabledRemoveValue(!isMandatory);
        setEnabledAddValue(!isMandatory);
    }
}
```

core/src/main/java/org/openmarkov/core/model/network/potential/WeibullHazardPotential.java:40-43

```java
protected static final VariableExpression[] MANDATORY_COVARIATES = new
↪   VariableExpression[]{
        VariableExpression.Common.GAMMA,
        VariableExpression.Common.CONSTANT
};
```

**Cómo arreglarlo.** Que la lista de covariables obligatorias dependa del potencial que se edita (el campo `potential` de `GLMPanel` ya lo tiene): por ejemplo, un método de instancia en `GLMPotential`, redefinido en `WeibullHazardPotential` y en `ExponentialHazardPotential` (esta última hereda de Weibull, pero su única obligatoria es «Constant», línea 32 de su fichero). Un método estático no sirve, porque no se puede redefinir.

Sitios donde la regla tiene que cumplirse: `GLMPanel.valueChanged` (botón de quitar) y `GLMPanel.CovariatesTableMouseListener.mouseClicked` (reescribir la covariable). Además, quitar la línea `setEnabledAddValue(!isMandatory)`.

Si se quiere que el modelo también se defienda, el constructor de `WeibullHazardPotential` que recibe covariables podría rechazar una lista sin «Gamma»; es una decisión aparte.

Prueba de regresión: construir `WeibullPotentialPanel` sin pantalla sobre el nodo «Failure [1]» de `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx`, elegir la fila «Gamma» y comprobar que quitar está apagado y añadir encendido; con un «Hazard (Exponential)», que la fila «Constant» apaga quitar y ninguna otra lo hace.

### Error 158 · [Jorge Rico] En el panel de una regresión la columna de las covariables no se ve: solo aparecen los coeficientes, sin decir de qué son

| | |
|---|---|
| **Estado** | Presente en parte. El diálogo «Reorder variables» (`ReorderVariablesPanel`) ya no oculta su primera columna desde el commit `8055530`; queda el panel de las regresiones. |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/GLMPanel.java:36-42`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTablePanel.java:132-140,190-197`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTable.java:176-203`<br>`gui/src/main/java/org/openmarkov/gui/component/StickyColumnsTablePane.java:111-114,124-146`<br>`gui/src/main/java/org/openmarkov/gui/component/OMTableModel.java:21-43` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 157, error 165 |

**Qué pasa.** Camino del usuario: abrir las propiedades de un nodo cuyo potencial sea de regresión (por ejemplo «Failure» en `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx`, o «Age at state entry» en `0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx`) y mirar la pestaña de la relación.

El usuario ve una columna de números sueltos sin saber a qué covariable corresponde cada uno. Además, no puede usar la edición de una covariable con doble clic en su celda (`CovariatesTableMouseListener`, que abre la ventanilla de expresiones), porque la columna no está en pantalla.

Los potenciales de regresión —«Linear combination»/«Linear regression», «Exponential», «Hazard (Weibull)» y «Hazard (Exponential)»— se editan en el diálogo del nodo con dos paneles, `GLMPotentialPanel` y `WeibullPotentialPanel`. Los dos meten un `GLMPanel`: una tabla de dos columnas, «Covariate» (la covariable: una variable o una expresión) y «Coefficient» (su coeficiente), con los botones de añadir, quitar, subir y bajar.

`GLMPanel` hereda de `KeyTablePanel`, pensado para listas cuya primera columna es una clave interna que no se enseña. Su último argumento, `firstColumnHidden`, hace que `KeyTable.defaultConfiguration()` ponga la primera columna a ancho cero y además quite de la vista la tabla fija

de la izquierda (`removeStickyTableFromView`), que es la que contiene esa columna. `GLMPanel` pasa `true`, así que la columna que desaparece es precisamente «Covariate». Otros paneles que usan esta clase (`PiecewiseExponentialTablePanel`) declaran una columna extra «Hidden» para que se oculte ésa; `GLMPanel` no.

Medido con un pequeño programa de prueba que abre `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx`, construye sin pantalla el `WeibullPotentialPanel` del nodo «Failure [1]» y mira su tabla:

```
columnas del modelo: 2: Covariate | Coefficient
tabla fija (columna 0) en la vista: false
tabla visible: 1 columna: Coefficient
filas: Gamma 0.3740968 / Constant -5.490935 / {Age at entry} -0.0367 / {Sex} 0.7685 /
↪ {Prosthesis type} -1.344474
```

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/GLMPanel.java:36-37

```
public GLMPanel() {
    super(new String[]{"Covariate", "Coefficient"}, new Object[0][2], true, true, true,
    ↪  true);
```

gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTable.java:176-199

```
if (this.firstColumnHidden && this.getColumnCount() > 0) {
    TableColumn column = this.getColumn(0);
    column.setMinWidth(0);
    column.setMaxWidth(0);
    column.setWidth(0);
    column.setPreferredWidth(0);
    column.setResizable(false);
    ...
    this.removeStickyTableFromView();
    this.getHeaderColumn(0).setMinWidth(0);
```

**Cómo arreglarlo.** Pasar `false` como `firstColumnHidden` en el constructor de `GLMPanel` **no basta**. `KeyTablePanel.getTableModel` (línea 206) pasa el mismo indicador a `OMTableModel` como `firstColumnIsHeader`, y `OMTableModel.split` (líneas 26-43), que es lo que usa `StickyColumnsTablePane.setModel` para repartir las columnas entre la tabla fija y la desplazable, solo conserva los títulos de las columnas cuando ese indicador es `true`. Medido sobre un `KeyTablePanel` de dos columnas «Covariate»/«Coefficient»:

```
firstColumnHidden=true:  tabla fija en la vista=false  títulos: Covariate | Coefficient
firstColumnHidden=false: tabla fija en la vista=true   títulos: A | A   (se pierden)
```

Opciones:

- (a) Separar en `KeyTablePanel` los dos significados —ocultar la primera columna y conservar los títulos—, por ejemplo pasando siempre `true` al modelo. Hay que revisar entonces a los otros llamadores que pasan `false` (hoy solo `ReorderVariablesPanel`, que no enseña cabecera).
- (b) Hacer como `PiecewiseExponentialTablePanel` y declarar una columna oculta extra. Esto obliga a desplazar todos los índices de columna que leen `GLMPanel` (`getCoefficients`, `getCovariates`, `valueChanged`, el oyente de doble clic), `GLMPotentialPanel.actionPerformed` y `WeibullPotentialPanel.itemStateChanged`, que leen el modelo por posición.

Al hacerse visible la columna, el doble clic sobre una covariable vuelve a estar disponible, y con él la posibilidad de reescribir una covariable obligatoria: la comprobación de ese oyente usa la misma

lista equivocada que describe error 157, así que conviene arreglar los dos juntos.

Prueba de regresión: construir `GLMPanel` (o `WeibullPotentialPanel` sobre el nodo «Failure [1]» de `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx`) sin pantalla y comprobar que la tabla visible enseña la columna «Covariate» con los nombres de las covariables.

### Error 159 · [Corregido] «Add variables to potential» en una hoja del árbol no añade nada salvo que se elija la primera variable del árbol, que es la condicionada

| | |
|---|---|
| **Estado** | Corregido en el commit `8b10140` (14-10-2026): cada casilla de `AddVariablesCheckBoxPanel` lleva su variable (`getSelectedVariables`) y el editor ya no la busca por el nombre. Con prueba sin pantalla sobre la hoja «no» de `ID-Monty-Hall-spanish`: el diálogo ofrece solo «Puerta escogida por el presentador» (desde el arreglo del error 18) y, marcada, la hoja pasa a tener las tres variables y 27 casillas; con la búsqueda anterior no se añadía nada. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.treeadd |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:769-797`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/AddVariablesCheckBoxPanel.java:36-43` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 18, nota 30 |

**Qué pasa.** En el editor de árboles (`TreeADDEditorPanel`, módulo `gui`), el botón derecho sobre una hoja (el potencial que cuelga de una rama) ofrece «Add variables to potential». Abre `AddVariablesDialog`, con una casilla por cada variable que se puede añadir (las devuelve `TreeADDBranch.getAddableVariables`).

- Si el usuario marca cualquier variable que no sea la condicionada (un padre), no se añade nada. El diálogo se cierra, no hay aviso y la hoja queda igual.
- Si marca la variable condicionada (que el diálogo sí ofrece, ver error 18), es la única que se encuentra y se añade, y la relación de la hoja queda con esa variable repetida.

Es decir: la operación nunca sirve para lo que dice (añadir un padre a la relación de la hoja), y la única elección que hace algo produce una relación mal formada.

Al aceptar, `addVariablesToPotential` busca, por cada casilla marcada, la variable con ese nombre entre las del árbol. El `break` está fuera del `if`, al final del cuerpo del bucle, así que el bucle compara **solo con la primera variable** de `parentTreeADD.getVariables()` y sale. En un árbol de probabilidad condicionada la primera variable es siempre la condicionada (la del propio nodo).

Medido con un pequeño programa de prueba sobre `0_miscellaneous/networks/id/ID-Monty-Hall-spanish.pgmx`, nodo «Puerta escogida 2», hoja de la rama «no» (potencial sobre «Puerta escogida 2» y «Puerta escogida 1»). El menú contextual de la hoja ofrece «Add variables to potential»; el diálogo ofrece «Puerta escogida por el presentador» y «Puerta escogida 2». Reproduciendo la búsqueda del método:

- elegir «Puerta escogida por el presentador» añade `[]` y la hoja sigue igual;
- elegir «Puerta escogida 2» la añade y la hoja queda sobre `[Puerta escogida 2, Puerta escogida 1, Puerta escogida 2]`.

La búsqueda se reprodujo copiando sus líneas en el programa de prueba, porque el diálogo no se puede abrir sin pantalla; la lista de casillas y el menú sí se obtuvieron de las clases reales.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:771-787

```
List<Variable> newVariables = new ArrayList<Variable>();
for (JCheckBox checkBox : checkBoxes) {
    if (checkBox.isSelected()) {
        String variableName = checkBox.getText();
        for (Variable variable : parentTreeADD.getVariables()) {
            if (variable.getName().equals(variableName)) {
                newVariables.add(variable);
            }
            break;
        }
    }
}
Potential potential = branch.getPotential();
for (Variable newVariable : newVariables) {
    potential = potential.addVariable(newVariable);
}
```

**Cómo arreglarlo.** Hecho con la segunda opción: cada casilla lleva su variable. Meter el `break` dentro del `if` (o quitarlo y buscar con un `stream().filter(...).findFirst()`). Mejor aún, asociar a cada casilla su objeto `Variable` y no buscar por nombre; las búsquedas por nombre de este editor se describen en nota 30, donde no hay nada que corregir.

Este arreglo debe ir junto con el de error 18: en cuanto las demás variables se puedan añadir, la variable condicionada seguirá ofrecida y seguirá duplicándose si no se corrige `getAddableVariables`.

Prueba: sobre la hoja «no» de `0_miscellaneous/networks/id/ID-Monty-Hall-spanish.pgmx`, marcar «Puerta escogida por el presentador» y comprobar que la hoja pasa a tener tres variables distintas, con la tabla del tamaño correcto (27 casillas, 3×3×3) y los valores originales replicados.

### Error 160 · [Corregido] Bajo una rama de una variable discretizada el editor vuelve a ofrecer esa variable para un subárbol, y elegirla lanza NullPointerException

| | |
|---|---|
| **Estado** | Corregido en el commit `5373602` (15-10-2026): la regla está en un solo método, `TreeADDEditorPanel.variablesLeftWithASingleState`, que usan los menús de la rama y de la hoja y que aparta las variables de estados o discretizadas que una rama de arriba ha dejado con un solo estado; `addSubtree` toma los estados del subárbol nuevo de la rama más cercana de esa variable, tenga los que tenga (`statesLeftAbove`), y ya no se cae. Con A de tres estados, ramas {a1} y {a2, a3}, después B y otra vez A, se ofrece A bajo {a2, a3} con esos dos estados, y no bajo {a1} |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.treeadd |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:301-306,352-357,1045-1071` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 17 |

**Qué pasa.** Se llega desde el diálogo de propiedades de un nodo cuyo potencial es un árbol que ramifica por una variable discretizada (variable numérica troceada en intervalos con nombre): botón derecho sobre una rama de un subárbol colgado de una rama de esa variable que se ha quedado con un solo estado. El menú ofrece «Add subtree» con esa misma variable, y al elegirlo sale el diálogo de «error inesperado» que pide enviar una captura: la excepción no controlada (`NullPointerException`) llega al manejador global (`OMExceptionHandler`).

En el menú de una hoja, la misma condición hace que se ofrezca añadir esa variable a la relación de la hoja, lo que por error 159 no tiene efecto salvo en el caso de la variable condicionada.

En un árbol, una rama que ya ha preguntado por una variable y se ha quedado con **un solo estado** suyo no gana nada volviendo a preguntar por ella más abajo. El editor de árboles (`TreeADDEditorPanel`, módulo `gui`) lo tiene en cuenta: al montar el menú contextual sube por el camino de ramas y quita de las opciones la variable de cada rama de un solo estado. Lo hace en dos sitios:

- `possibleRootVariables` (líneas 351-356), usado por el menú de una rama, opción «Add subtree»;
- `setContextualMenuPotential` (líneas 300-305), menú de una hoja, opción «Add variables to potential».

En los dos, la condición compara el tipo de variable con `FINITE_STATES` dos veces; la segunda mitad debía ser `DISCRETIZED`, que también ramifica por estados, como hacen `setContextualMenuBranch` en la línea 240 y `addSubtree` en la línea 1043.

Al elegir «Add subtree» con la variable ya usada, `addSubtree` lo detecta (`isRootVariableUsedBefore`), busca hacia arriba una rama de esa variable con más de un estado para restringir los estados del subárbol nuevo, no la encuentra (todas tienen uno) y deja `groupedStates` a `null`; la línea 1063 (`groupedStates.size()`) lanza `NullPointerException`.

Medido con un pequeño programa de prueba sobre dos redes gemelas construidas a mano: nodo X con padres A (de estados) y D, árbol de X que pregunta primero por D, subárbol de A colgado de la rama «D = high» (un solo estado).

- Con D **de estados finitos**: bajo la rama «A = …» el menú no ofrece «Add subtree» con D. Correcto.
- Con D **discretizada**: el menú de esa rama ofrece «Add subtree: D», y al elegirlo `addSubtree` muere con `NullPointerException`.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:347-359

```
TreePath parentPath = branchPath.getParentPath(); // treeADD
while (parentPath.getLastPathComponent() != rootTreeADDPotential) {
    TreePath grandParentPath = parentPath.getParentPath();// branch
    if (grandParentPath.getLastPathComponent() instanceof TreeADDBranch treeBranch) {
        if ((
                treeBranch.getRootVariable().getVariableType() ==
↪  VariableType.FINITE_STATES
                        || treeBranch.getRootVariable().getVariableType() ==
                        ↪  VariableType.FINITE_STATES
        ) && treeBranch.getBranchStates().size() == 1) {
            possibleRootVariables.remove(treeBranch.getRootVariable());
        }
    }
    parentPath = grandParentPath;
}
```

**Cómo arreglarlo.** Cambiar la segunda mitad por `VariableType.DISCRETIZED` en los dos sitios (líneas 302 y 353); mejor, sacar el recorrido a un método único que usen los dos menús, porque es la misma regla escrita dos veces.

Además, `addSubtree` (líneas 1045-1066) no debería depender de que el menú haya filtrado bien: si la variable ya se usó y no hay rama agrupada, no debe usar `groupedStates` nulo (rechazar con un mensaje o crear el subárbol completo, según lo que se quiera).

Prueba: los dos árboles gemelos descritos; comprobar que con D discretizada el menú de la rama interior no ofrece D, y que `addSubtree` invocado con D en esa situación no lanza `NullPointerException`.

### Error 161 · [Corregido] En el editor de árboles, aceptar «Split interval» con el campo vacío (tal como se abre) o con un texto que no es un número sale como error inesperado, y lo mismo en «Change interval»

| | |
|---|---|
| **Estado** | Corregido en el commit `8f4d3a8` (15-10-2026): `doOkClickBeforeHide` de `SplitIntervalDialog` y de `ChangeIntervalDialog` comprueban que lo escrito sea un número; si no lo es (vacío, «abc», «1,5» o «NaN»), avisan de que tiene que serlo y la ventanilla sigue abierta con lo escrito. «Infinity», que es lo que enseña «Change interval» en el borde de un dominio sin límite, se sigue aceptando. La coma decimal no se acepta. Con prueba en `gui` que abre las dos ventanillas, y que se salta sin pantalla. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.treeadd |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:411-425,450-455,636-648`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/SplitIntervalDialog.java:79-81`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/SplitIntervalPanel.java:31-32`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/ChangeIntervalDialog.java:87-89`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/ChangeIntervalPanel.java:41-45` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 164, error 181, error 182, nota 17 |

**Qué pasa.** En el editor de una relación de tipo árbol (Tree/ADD), cada rama de una variable numérica tiene en su menú contextual «Split interval» (partir el intervalo de la rama por un umbral) y, si la rama no cubre todo el dominio, «Change interval». Las dos abren una ventanilla con campos de texto.

«Split interval» se abre con el campo del umbral vacío. Pulsar «OK» (o Intro) sin escribir nada, o después de escribir algo que no se lee como número —una letra, o un decimal con coma como «1,5»—, termina en el diálogo de error inesperado que pide enviar el fallo a los desarrolladores. Lo mismo ocurre en «Change interval» si se vacía o se estropea uno de sus dos límites. El árbol no cambia, porque la conversión es lo primero que se hace tras cerrar la ventanilla.

Camino del usuario: abrir `0_miscellaneous/networks/mid/MID-Chancellor.pgmx`, abrir la relación de «State [1]» o de «Cost lamivudine [0]» (dos árboles con ramas de «Time in treatment [0]», una variable numérica; la primera de esas ramas va de 0 a 2), clic derecho en una rama numérica, «Split interval» y «OK».

Por qué pasa: `TreeADDEditorPanel.splitInterval` (módulo `gui`, línea 643) convierte el texto con `Float.parseFloat`, y `changeInterval` (líneas 449-450) con `Double.parseDouble`. Ninguno captura `NumberFormatException`. `actionPerformed` solo envuelve en `UnrecoverableException` —que la ventana enseña con su mensaje, como se ve en nota 17— las excepciones propias del árbol; `NumberFormatException` es de tiempo de ejecución y llega al manejador global como error no previsto. Las ventanillas no validan: `SplitIntervalDialog.doOkClickBeforeHide` solo comprueba `getText() != null`, que se cumple siempre, y `ChangeIntervalDialog.doOkClickBeforeHide` devuelve `true`.

Medido ejecutando por separado lo que no depende de la ventana:

- `SplitIntervalPanel` real: el campo del umbral empieza vacío;
- el `doOkClickBeforeHide` real de `SplitIntervalDialog` devuelve `true` (la ventanilla se cerraría) con «», «abc», «1,5» y «2.5»;
- `Float.parseFloat` lanza `NumberFormatException` con los tres primeros («empty String», `For input string: "abc"`, `For input string: "1,5"`) y acepta «2.5»;
- en la rama de 0 a 2 de «Time in treatment [0]», `ChangeIntervalPanel` rellena los límites con «0.0» (no editable, es el borde del dominio) y «2.0» (editable); `Double.parseDouble` con el campo vaciado lanza la misma excepción.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:406-420

```
case ActionCommands.SPLIT_INTERVAL -> {
    // node must be a branch
    try {
        splitInterval(ae, (TreeADDBranch) node, path);
    } catch (InvalidLimitInTreeADDException | TriedToSplitIntervalOutsideBoundsException e)
    ↪ {
        throw new UnrecoverableException(e);
    }
}
case ActionCommands.CHANGE_INTERVAL -> {
    try {
        changeInterval(ae, (TreeADDBranch) node, path);
    } catch (InvalidArgumentException | ChangeDomainOfTreeADDIsNotAllowedException e) {
        throw new UnrecoverableException(e);
    }
}
```

gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:643

```
float introducedLimit = Float.parseFloat(panel.getLimit().getText());
```

gui/src/main/java/org/openmarkov/gui/dialog/treeadd/SplitIntervalDialog.java:79-81

```
@Override protected boolean doOkClickBeforeHide() {
    return getJPanelSplitInterval().getLimit().getText() != null;
}
```

**Cómo arreglarlo.** Validar en la ventanilla antes de cerrarla: que `doOkClickBeforeHide` de `SplitIntervalDialog` y de `ChangeIntervalDialog` intente convertir sus textos y, si alguno no es un número, avise y devuelva `false`, para que el usuario corrija sin perder la ventanilla. La otra opción es capturar `NumberFormatException` en `splitInterval` y `changeInterval` y lanzar una excepción del árbol con mensaje, que `actionPerformed` ya envuelve en `UnrecoverableException`.

Las tres conversiones están en `TreeADDEditorPanel`: línea 643 (`splitInterval`) y líneas 449-450 (`changeInterval`). Si se quiere aceptar la coma decimal, hay que hacerlo en las tres.

Prueba de regresión: con `SplitIntervalPanel` con el texto vacío y con «abc», aceptar no debe cerrar la ventanilla (o no debe salir ninguna excepción que no sea del árbol); con «1» en la rama de 0 a 2 de «Time in treatment [0]», la rama debe partirse en dos.

### Error 162 · [Jorge Rico] La pestaña «Parents» del diálogo de un nodo ofrece como posibles padres nodos que las restricciones de la red rechazan, y elegir uno termina en un diálogo de error técnico

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/PrefixedDataTablePanel.java:165-190`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/PrefixedDataTablePanel.java:115-143`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTablePanel.java:362-380`<br>`core/src/main/java/org/openmarkov/core/action/base/linkEdits/AddLinkEdit.java:69-187`<br>`core/src/main/java/org/openmarkov/core/action/base/PNEdit.java:37-56`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/KeyListSelectionDialog.java:139` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 23, nota 17, nota 24, nota 25, nota 153 |

**Qué pasa.** El diálogo de propiedades de un nodo (`NodePropertiesDialog`, módulo `gui`) tiene una pestaña «Parents» (`NodeParentsPanel`) con una tabla de padres (`PrefixedDataTablePanel`) y un botón para añadir. El botón abre una lista con los candidatos, y en ella aparecen nodos que la red no permite enlazar: por ejemplo, a «VisitToAsia» de `0_miscellaneous/networks/bn/BN-asia.pgmx` se le ofrecen «X-ray», «Dyspnea», «Tuberculosis» y «TuberculosisOrCancer», que son descendientes suyos y cerrarían un ciclo.

Si el usuario elige uno, el programa no se cae: sale un diálogo de error con un texto poco legible y el enlace no se añade. Eligiendo «X-ray» para «VisitToAsia», el diálogo tiene el título «There is a cycle» y el mensaje «Could not met the constraint: No cycles paths are allowed. There is a cycle traversing between chance node of Finite states variable X-ray with states [state no, state yes] and chance node of Finite states variable VisitToAsia with states [state no, state yes].»

Además, la lista admite selección múltiple: si se eligen varias filas y una falla, las anteriores ya se han añadido, y la lista de candidatos (`absentData`) no se recalcula antes de salir por la excepción.

La lista es un `KeyListSelectionDialog` con los candidatos que calcula `absentPrefixedData` (líneas 165-190): todos los nodos de la red que no son ya padres y no son el propio nodo. Por cada uno prepara una edición `AddLinkEdit` sin ejecutarla. No consulta ninguna restricción de la red; la de ciclos (`NoCycle`), por ejemplo, solo actúa al ejecutar la edición.

Al aceptar, `actionPerformedAddValue` (líneas 115-143) ejecuta la edición de cada fila elegida. `PNEdit.executeEdit` comprueba entonces las restricciones (`AddLinkEdit.checkConstraintsWillBeMet`, líneas 69-187: enlace repetido, máximo de padres, enlace hacia un corte temporal anterior, ciclo, bucle, padres mezclados, padres de utilidad…) y, si alguna se incumple, lanza `DoEdit`

`Exception.CannotDoEditException`. `KeyTablePanel.actionPerformed` (líneas 362-380) la envuelve en `UnrecoverableException`, que el manejador global convierte en un diálogo de error (no cierra el programa).

Medido con un pequeño programa de prueba que construye el panel real para cada nodo y pregunta a cada edición preparada si cumpliría las restricciones (`PNEdit.tryConstraintsWillBeMet`):

- `0_miscellaneous/networks/bn/BN-asia.pgmx`: se ofrecen 48 candidatos en total y 18 los rechaza la restricción de ciclos (todos descendientes).
- `0_miscellaneous/networks/id/ID-decide-test.pgmx`: 34 ofrecidos, 17 rechazados (ciclo, padre de utilidad no permitido y combinaciones).
- `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`: 120 ofrecidos, 52 rechazados (incluye enlaces a un corte temporal anterior).

El diálogo de error citado arriba es el que el manejador global enseñaría al ejecutar la edición «X-ray → VisitToAsia».

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/PrefixedDataTablePanel.java:165-181

```
private Object[][] absentPrefixedData() {
    List<Node> allNodes = node.getProbNet().getNodes();
    List<Node> nodes = new ArrayList<Node>();
    edits.clear();

    for (Node otherNode : allNodes) {
        if (!node.getParents().contains(otherNode) && otherNode != node) {
            AddLinkEdit linkEdit = new AddLinkEdit(node.getProbNet(),
↪   otherNode.getVariable(), node.getVariable(),
                                                   true);
            ...
            edits.add(linkEdit);
            nodes.add(otherNode);
```

gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTablePanel.java:377-379

```
} catch (DoEditException | ThereIsNoNodeInDataException ex) {
    throw new UnrecoverableException(ex);
}
```

**Cómo arreglarlo.** Filtrar la lista con la misma regla que aplicará la edición: añadir el candidato solo si `linkEdit.constraintsWillBeMet()` es verdadero (el método ya existe en `PNEdit`). Así la regla sigue escrita en un solo sitio (`AddLinkEdit.checkConstraintsWillBeMet`) y no se crea otra copia en la interfaz. La lista debe recalcularse tras cada alta o baja (hoy ya se recalcula en los caminos normales).

Sitios que añaden enlaces desde la interfaz y deben comportarse igual:

- la pestaña «Parents» (`PrefixedDataTablePanel`);
- la creación de enlaces con el ratón en el lienzo (`VisualNetwork`, que ya consulta las restricciones para colorear la flecha; su coste se describe en nota 25, donde no hay nada que corregir);
- las opciones de crear enlace padre/hijo del menú contextual de un nodo;
- `MultiAddLinkEdit`, que descarta en silencio los pares imposibles (ver nota 24, donde tampoco hay nada que corregir).

Conviene también que, si a pesar de todo la edición falla, se ejecuten todas o ninguna de las filas

elegidas, y que el mensaje de `ThereIsACycle` nombre los nodos por su nombre y no por `toString()`.

Prueba: sobre `0_miscellaneous/networks/bn/BN-asia.pgmx`, construir la pestaña de padres de «VisitToAsia» y comprobar que la lista no contiene «X-ray», «Dyspnea», «Tuberculosis» ni «TuberculosisOrCancer», y que todos los candidatos ofrecidos se pueden añadir sin excepción.

### Error 163 · [Jorge Rico] Escribir el límite superior del último intervalo de las condiciones de revelación de un enlace lanza una excepción inesperada y deja la tabla diciendo algo que el enlace no tiene

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.component |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/component/RevelationArcDiscretizeTablePanel.java:97-141`<br>`gui/src/main/java/org/openmarkov/gui/action/RevelationIntervalEdit.java:144-150`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:260-265`<br>`gui/src/main/java/org/openmarkov/gui/validator/RevelationArcValidator.java` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** En una red de análisis de decisiones (DAN), un enlace que sale de un nodo de azar numérico puede llevar condiciones de revelación: los intervalos de valores del padre que revelan el valor del hijo. Se editan con clic derecho sobre el enlace → «Edit revelation conditions», que abre `RevelationArcEditDialog` con una tabla `RevelationArcDiscretizeTablePanel` (una fila por intervalo, con límite inferior y superior). El botón de añadir pone un intervalo nuevo al final (`RevelationIntervalEdit.getNewPartitionedInterval`: el primero es (-∞, ∞) y los siguientes empiezan donde acaba el anterior y terminan en ∞).

Si el usuario escribe un número en el límite superior del último intervalo, sale el diálogo de «excepción inesperada» que pide enviarla a los desarrolladores: la excepción no se captura en el panel y llega al gestor global (`OMExceptionHandler`). La celda se queda con el número tecleado, pero el intervalo del enlace no cambia. Es el caso corriente: el último intervalo es el que se acaba de añadir y el que el usuario va a acotar. En la práctica, el límite superior del último intervalo no se puede cambiar desde la tabla.

Al escribir un número en una celda de límite, `tableChanged` comprueba que no se monte sobre el intervalo vecino. Para el límite inferior mira el intervalo anterior con `if (row > 0)`, bien. Para el límite superior mira el siguiente con `if (row < numRows)`, que se cumple también en la última fila (`row = numRows - 1`), y entonces `table.getValueAt(row + 1, …)` lee una fila que no existe.

Medido con un pequeño programa de prueba que construye el panel real sin pantalla sobre una red de análisis de decisiones con un nodo numérico A y un enlace A → B:

- con un único intervalo (-∞, ∞), escribir 5 en su límite superior lanza `ArrayIndexOutOfBoundsException: 1 >= 1`;
- con dos intervalos (-∞, 5) y [5, ∞), escribir 3 en el límite superior del primero funciona, y escribir 10 en el del último lanza `ArrayIndexOutOfBoundsException: 2 >= 2`; después de

la excepción la celda muestra 10 pero el intervalo del enlace sigue siendo [5, ∞), porque la edición no llegó a ejecutarse.

**El código.** gui/src/main/java/org/openmarkov/gui/component/RevelationArcDiscretizeTablePanel.java:110-132

```
if (lower) {
    double upperLimit = (Double) table.getValueAt(row,
    ↪  DiscretizeTablePanel.UPPER_BOUND_VALUE_COLUMN_INDEX);
    ...
    if (row > 0) {
        double previousLimit = (Double) table.getValueAt(row - 1,
        ↪  DiscretizeTablePanel.UPPER_BOUND_VALUE_COLUMN_INDEX);
        ...
    }
} else {
    double lowerLimit = (Double) table.getValueAt(row,
    ↪  DiscretizeTablePanel.LOWER_BOUND_VALUE_COLUMN_INDEX);
    ...
    if (row < numRows) {
        double nextLimit = (Double) table.getValueAt(row + 1,
        ↪  DiscretizeTablePanel.LOWER_BOUND_VALUE_COLUMN_INDEX);
        if (nextLimit < newValue) {
            throw new UnrecoverableException(new InvalidArgumentException(newValue, "New
            ↪  value", "it is greater than the next bound's lower limit (" + nextLimit +
            ↪  ")"));
        }
    }
}
```

**Cómo arreglarlo.** Cambiar la condición a `if (row + 1 < numRows)`, simétrica de `row > 0`. Es el único sitio con esta comprobación de vecinos para las condiciones de revelación; la tabla de tramos del nodo (`DiscretizeTablePanel.tableChanged`) recoloca los límites de otra manera y no tiene esta lectura.

Conviene además que un valor rechazado devuelva la celda al valor del enlace (hoy la celda se queda con el número tecleado cuando algo lanza una excepción).

Prueba de regresión: construir `RevelationArcDiscretizeTablePanel` sin pantalla sobre un enlace con uno y con dos intervalos, escribir el límite superior de la última fila y comprobar que no lanza ninguna excepción y que el intervalo del enlace cambia.

### Error 164 · [Corregido] Escribir un número no válido en el panel del indicador lanza una excepción no prevista y vacía la pestaña de la relación

| | |
|---|---|
| **Estado** | Corregido en el commit `852ede1` (15-10-2026). Los dos campos son ahora casillas numéricas (`JSpinner`) limitadas a valores válidos: tiempo desde 0 y probabilidad de 0 a 1. Un texto que no es un número, un valor fuera de rango o un número solo en parte («1,5», «3.5abc», con el análisis estricto de `DecimalFormat`) no se acepta, y la casilla vuelve al último valor válido; el guardado ya no lanza ninguna excepción. Las casillas enseñan todos los decimales (la de por defecto enseñaría 0.0001 como 0). Con prueba sobre el panel. Sigue presente lo que se dice al final de «Cómo arreglarlo»: `acceptPotentialEditChanges` quita el panel antes de saber si el guardado ha ido bien, lo que afecta a cualquier panel de relación que lance una excepción de tiempo de ejecución. No se ha probado en la ventana. |
| **Módulo** | gui |

| | |
|---|---|
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/IndicatorPotentialPanel.java:80-89`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/IndicatorPotential.java:155-178`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:243-269,314-331,430-437`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditDialog.java:110-113`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/OkCancelDialog.java:68-79` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 109, error 152, error 161, nota 13 |

**Qué pasa.** El panel del indicador tiene dos campos de texto libre, el tiempo hasta el suceso y la probabilidad de que ocurra. Si el usuario escribe algo que no es un número, un tiempo negativo o una probabilidad fuera de [0, 1], al guardar sale la ventana de «error no previsto» (`UnexpectedThrowableDialog`).

El guardado se ejecuta al pulsar OK del diálogo de propiedades del nodo (`NodePropertiesDialog.doOkClickBeforeHide`, línea 416) y también al cambiar de pestaña dejando la de la relación (`handleChangeTab`, línea 248). En los dos casos el diálogo no se cierra (OK no llega a `dispose`, `OkCancelDialog`, líneas 67-78), pero la pestaña de la relación se queda sin panel y lo escrito se pierde.

Desde el diálogo propio de editar la relación (`PotentialEditDialog`, líneas 109-111) el panel no se quita: sale la misma ventana de error y el usuario puede corregir y volver a pulsar OK.

No se ha abierto ninguno de los dos diálogos; el camino y el efecto sobre la pestaña salen de la lectura del código.

Al guardar, `saveChanges` (líneas 80-89) hace `Double.parseDouble` sobre cada campo y llama a `IndicatorPotential.setTte` y `setpOccurrence` (módulo `core`, líneas 155-178), que lanzan `InvalidArgumentException` (subclase de `IllegalArgumentException`) si el tiempo es negativo o la probabilidad no está entre 0 y 1. No hay ninguna comprobación mientras se escribe, y ninguna de esas excepciones se caza.

Los dos caminos de guardado del diálogo del nodo pasan por `acceptPotentialEditChanges` (líneas 297-314), que **primero quita el panel de la pestaña** (línea 302) y después guarda, cazando solo tres excepciones comprobadas. Las de arriba son de tiempo de ejecución, así que suben hasta el manejador global.

Medido con un pequeño programa de prueba sobre el panel real:

```
tte=abc p=0.5  -> NumberFormatException: For input string: "abc"
tte=-1  p=0.5  -> InvalidArgumentException: Time to event  out of range 0..
tte=2   p=1.5  -> InvalidArgumentException: Occurrence out of range 0<=pOccurrence <=1
tte=2   p=0.25 -> guardado
```

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/IndicatorPotentialPanel.java:80-89

```
@Override public boolean saveChanges() throws DoEditException, ... {
    boolean result = super.saveChanges();
    IndicatorPotential oldPotential = (IndicatorPotential) node.getPotentials().get(0);
    IndicatorPotential newPotential =  new IndicatorPotential(oldPotential);
```

```
        newPotential.setTte(Double.parseDouble(tteTextField.getText()));
        newPotential.setpOccurrence(Double.parseDouble(pOcurrenceTextField.getText()));
    new PotentialChangeEdit(node, oldPotential, newPotential).executeEdit();
    return result;
}
```

**Cómo arreglarlo.** Dos caminos:

- Pedir los dos números con controles que solo admitan valores válidos (un `JSpinner` o un campo con formato: tiempo $\geq 0$, probabilidad en $[0, 1]$).
- Validar en `saveChanges` y comunicar el error con una excepción comprobada que el diálogo sepa enseñar sin perder lo escrito.

La regla de rango debe seguir en los métodos de `IndicatorPotential`, que son los que protegen también la lectura de ficheros.

Aparte, `acceptPotentialEditChanges` quita el panel antes de saber si el guardado ha ido bien; eso afecta a cualquier panel de relación que lance una excepción de tiempo de ejecución, no solo a este.

Prueba: construir el panel sobre un nodo de suceso, escribir «abc» y «-1» y comprobar que el guardado no deja escapar `NumberFormatException` ni `IllegalArgumentException` y que la relación del nodo no cambia.

### Error 165 · [Jorge Rico] La ventanilla de expresiones mete dos veces la variable o la función cuando se hace el doble clic que pide su ayuda

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/ArithmeticExpressionDialog.java:110-145`<br>`gui/src/main/resources/gui/localize/ArithmeticExpressionEvaluator_en.xml:5-6` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 158 |

**Qué pasa.** `ArithmeticExpressionDialog` («Enter an expression») es la ventanilla para escribir una expresión: un campo de texto y dos listas, la de variables y la de funciones. La ayuda emergente de cada lista dice «Double click to introduce the selected variable in the expression» y «Double click to introduce the call to the selected function in the expression».

Si el usuario hace ese doble clic, el texto entra dos veces (`{Age}{Age}`, o `pow()pow()`), y la expresión que queda no es válida y hay que corregirla a mano. Además, un solo clic para elegir una variable de la lista ya modifica la expresión.

Se usa desde muchos sitios: `GLMPanel` (añadir o reescribir una covariable), `FunctionPotentialPanel`, `UnivariateDistrPotentialPanel`, `AugmentedProbTablePotentialPanel` y `TableWithEventsPanel`. Camino del usuario, por ejemplo: propiedades de un nodo con potencial «Linear combination» → botón de añadir covariable → doble clic en una variable de la lista.

Los dos oyentes de ratón salen solo si `e.getClickCount() < 1`. Swing entrega `mouseClicked` con

`getClickCount()` igual a 1 en el primer clic y 2 en el segundo de un doble clic; nunca vale menos de 1. Por eso **cada clic** inserta el texto: un clic simple ya mete `{variable}`, y el doble clic lo mete dos veces.

Comprobado leyendo; no se ejecutó porque la ventanilla es un `JDialog` y no se puede construir sin pantalla.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/ArithmeticExpressionDialog.java:110-121

```
variableList.addMouseListener(new MouseAdapter() {
    @Override public void mouseClicked(MouseEvent e) {
        super.mouseClicked(e);
        if (e.getClickCount() < 1) {
            return;
        }
        try {
            insertTextInExpression("{" + variableList.getSelectedValue() + "}");
        } catch (BadLocationException ex) {
            throw new UnrecoverableException(ex);
        }
    }
});
```

**Cómo arreglarlo.** Cambiar la condición a `e.getClickCount() != 2` (o `< 2`) en los dos oyentes: el de `variableList` (línea 113) y el de `functionList` (línea 135). Con `!= 2` un triple clic no inserta una tercera vez.

Prueba de regresión: si se extrae el cuerpo de los oyentes a un método que reciba el número de clics, se puede probar sin pantalla que con 1 no inserta y con 2 inserta una vez; si no, prueba manual con doble clic en cada lista.

## Error 166 · [Corregido] Tras cambiar el tipo de un nodo, una copia del nodo ya no encuentra sus padres en la red, y la vista de tabla de un modelo canónico se construye sin ellos

| | |
|---|---|
| **Estado** | Corregido en el commit `86389d6` (14-10-2026), con la opción (a): `Node.equals` compara la variable y la red, no el tipo, y el código de dispersión se calcula solo con la variable que el nodo tiene al crearse. Para `setVariable`, el constructor de copia `Node(Node)` copia el código del original en vez de calcularlo, así que una copia hecha tras cambiar la variable también se encuentra. Pruebas: la de la medida, tras cambiar el tipo y tras cambiar la variable (mismo código, `getParents` de la copia da [P], un `HashSet` con X contiene la copia); con el código anterior fallaban las dos. La batería completa pasa sin cambios en los demás usos de `Map<Node,…>` y `Set<Node>`. No se ha abierto la vista TPC en la ventana. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/Node.java:80,133,141-149,272-283,339-341,413-424`<br>`core/src/main/java/org/openmarkov/core/model/graph/Graph.java:49-61,69-71`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/ICIOptionListenerAssistant.java:62-70,101-115`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TablePotentialPanel.java:360-362`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPan` |

| | |
|---|---|
| | el.java:332-341 |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 3, error 50, nota 180 |

**Qué pasa.** Camino del usuario (leído, no ejecutado): el tipo de un nodo se cambia desde el diálogo de propiedades del nodo, con los botones de tipo (`NodeDefinitionPanel.java:332-341` → `ChangeNodeTypeEdit`, que llama a `setNodeType`). El único sitio de la aplicación que construye una copia de un nodo real es la vista «TPC» (tabla de probabilidad condicional) del panel de un modelo canónico: al pulsar ese botón, `ICIOptionListenerAssistant.itemStateChangedTPC` hace `new Node(panel.getNode())` y construye con esa copia un `CPTablePanel` (líneas 101-115). `TablePotentialPanel.howManyRows` calcula las filas de la tabla con `n.getParents().size()` (líneas 341-343).

Así, si el nodo se creó con otro tipo (decisión, utilidad), luego se pasó a azar y se le puso un modelo canónico (OR/MAX/MIN ruidoso), la vista TPC se construye como si el nodo no tuviera padres: le faltan las filas de cabecera de los padres mientras la tabla expandida sí los tiene. No se ha abierto el diálogo, así que no está medido qué ve exactamente el usuario: una tabla mal dibujada o una excepción.

En Java, dos objetos iguales según `equals` deben tener el mismo `hashCode`; si no, las tablas de dispersión (`HashMap`, `HashSet`) no los encuentran. `Node.equals` (líneas 426-433) considera iguales dos nodos con la misma variable (por identidad), la misma red y **el mismo tipo actual**. `Node.hashCode` devuelve un campo `final` calculado en el constructor con la variable y el tipo que tenía entonces (líneas 80, 133 y 148). `setNodeType` (líneas 272-283) cambia el tipo pero no ese campo.

Resultado: tras cambiar el tipo de un nodo, un nodo igual construido después (por ejemplo con el constructor de copia `Node(Node)`, líneas 141-149, que calcula el código con el tipo nuevo) tiene otro código de dispersión.

Con el propio objeto no pasa nada: su código no cambia y es igual a sí mismo, así que el grafo (que guarda padres e hijos en `HashMap<Node, …>`, `Graph.java:49-61`) lo sigue encontrando. Cambiar el **nombre** de una variable tampoco afecta, porque `Variable` no redefine `equals` ni `hashCode` y se compara por identidad. El problema aparece solo cuando se consulta la red con **otro objeto** `Node` igual.

Medido con un pequeño programa de prueba sobre un diagrama de influencia con el enlace P → X, con X creado como decisión. Antes de cambiar nada, `getParents(X)` y `getParents(new Node(X))` dan `[P]`. Tras `X.setNodeType(CHANCE)`: `new Node(X).equals(X)` es verdadero pero los códigos difieren; `getParents(X)` sigue dando `[P]` y `getParents(copia)` da `[]`; un `HashSet` que contiene X dice que no contiene la copia.

**El código.** core/src/main/java/org/openmarkov/core/model/network/Node.java:272-283 y 426-437

```
public void setNodeType(NodeType newNodeType) {
    NodeType oldNodeType = this.nodeType;
    if(oldNodeType == newNodeType){
        return;
    }
    this.probNet.nodeDepot.removeNode(this);
    this.nodeType = newNodeType;
    this.probNet.nodeDepot.addNode(this);
}
...
```

```
@Override public boolean equals(Object obj) {
    boolean equals = obj instanceof Node;
    if (equals) {
        Node otherNode = (Node) obj;
        equals = variable.equals(otherNode.variable) && probNet.equals(otherNode.probNet)
↪   && nodeType == otherNode.nodeType;
    }
    return equals;
}

@Override public int hashCode() {
    return hashCode;
}
```

**Cómo arreglarlo.** Hacer que `equals` y `hashCode` usen los mismos datos y que ninguno de ellos cambie mientras el nodo está en el grafo. Opciones:

- (a) Quitar el tipo de `equals` y del código de dispersión: un nodo se identifica por su variable y su red, y el tipo es un atributo que cambia.
- (b) No redefinir ninguno de los dos y comparar nodos por identidad, que es lo que el grafo necesita.

Con la variable pasa lo mismo: `setVariable` (líneas 166-170) cambia la variable pero no el código, así que la regla afecta también a ese método (llamado desde `ProbNetOperations.convertNumericalVariablesToFS`, líneas 451 y 538, y `DANOperations.java:159`). Tras el cambio hay que revisar además los 72 usos de `Map<Node,…>`/`Set<Node>` del código de producción.

Prueba de regresión en `core`: la de la medida (cambiar el tipo y comprobar que `getParents(new Node(nodo))` devuelve los padres y que `new Node(nodo).hashCode() == nodo.hashCode()` cuando son iguales), repetida con `setVariable`.

### Error 167 · [Corregido] Una columna con incertidumbre en un estado que no es el primero se deja editar, y una con incertidumbre solo en el primero se bloquea entera

| | |
|---|---|
| **Estado** | Corregido en el commit `4eef20f` (09-10-2026), el mismo del error 76: `TablePotential.hasUncertainty` mira todas las casillas de la columna, a través de `getUncertainColumn`, y de él dependen los tres usos del panel. Con prueba de la tabla. Visto en la ventana, con un guion, sobre un fichero con columnas a medias: la columna con el primer estado fijo, que antes se dejaba editar, queda bloqueada y el doble clic abre el diálogo de incertidumbre; la que tenía incertidumbre solo en el primer estado, que daba «Unexpected error» al hacer doble clic, lo abre también; la columna sin incertidumbre sigue editable. |
| **Módulo** | core, gui |
| **Paquete** | org.openmarkov.core.model.network.potential; org.openmarkov.gui.dialog.common |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:413-420`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/AbstractIndexedPotential.java:262-283`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/TablePotentialPanel.java:337-354,616-641,823-842,849-866`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1324-1355` |
| **Comprobado** | Leyendo el código y ejecutándolo |

| | |
|---|---|
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 76 |

**Qué pasa.** En una tabla de probabilidad condicionada con incertidumbre (un `UncertainTablePotential`, que guarda para cada casilla una distribución de segundo orden o nada), el panel de la tabla (`TablePotentialPanel`, módulo `gui`) decide qué columnas llevan incertidumbre para tres cosas:

- bloquear la edición a mano de esa columna y pintarla con el icono de incertidumbre (`getUncertaintyInColumns`, líneas 318-335, que alimenta `getNotEditablePositions`, líneas 614-620);
- habilitar «Assign/Edit/Remove uncertainty» en el menú contextual (`updateContextualMenuOptions`, línea 810);
- abrir el diálogo de incertidumbre con doble clic (`doubleClickEvent`, línea 842).

Si una columna tiene incertidumbre en la casilla de un estado del nodo que no es el primero, pero no en la del primero, la columna queda editable entera y sin icono, y el menú contextual ofrece «asignar» incertidumbre donde ya la hay. Editarla cambia los números y deja la distribución de segundo orden como estaba, sin aviso, de modo que los valores y la incertidumbre de la columna dejan de corresponderse. Al revés, una columna con incertidumbre solo en el primer estado queda bloqueada entera.

Camino de llegada: el diálogo de incertidumbre (`UncertainValuesDialog` con `UncertainValuesEdit`) siempre escribe la columna completa, así que desde la interfaz no se produce. Sí se produce abriendo un `.pgmx` cuyo `<UncertainValues>` tenga huecos (`<Value />`) en algunas casillas de una columna: `PGMXReader_0_2.getUncertainValues` lo acepta casilla a casilla.

Las tres llamadas del panel usan `TablePotential.hasUncertainty(configuration)`, donde la configuración tiene un hallazgo por cada padre (el nodo no está). `hasUncertainty` traduce esa configuración a una posición con `AbstractIndexedPotential.getPosition(EvidenceCase)`, que, al faltar la variable del nodo, pone su coordenada a 0. Resultado: solo se consulta la casilla del primer estado del nodo en esa columna.

Medido con un pequeño programa de prueba sobre una red A→B (B con estados b0, b1, b2): con incertidumbre solo en la casilla (A=a0, B=b1), `hasUncertainty(A=a0)` da `false` y la columna a0 queda editable entera y sin icono; con incertidumbre solo en (A=a1, B=b0), la columna a1 queda bloqueada entera. Editar la columna a0 cambia los números con `TablePotentialValueEdit` y deja la distribución de b1 como estaba.

También medido: en los 167 ficheros `.pgmx` legibles de `0_miscellaneous/networks` y de los recursos de prueba de `core`, `io`, `gui` e `inference` hay 102 tablas con incertidumbre y ninguna columna con incertidumbre incompleta.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/AbstractIndexedPotential.java:262-283

```
    public int getPosition(EvidenceCase configuration) {
        ...
        if (varsTable.size() != sizeEvi) {
            sizeCoordinates = sizeEvi + 1;
            startLoop       = 1;
            coordinates     = new int[sizeCoordinates];
            coordinates[0]  = 0;
        } else {
        ...
```

**Cómo arreglarlo.** Hay que fijar el significado y aplicarlo en un solo sitio: que una columna «tiene incertidumbre» si alguna de sus casillas la tiene (lo más prudente para bloquear la edición), recorriendo los estados del nodo en `hasUncertainty` o en un método nuevo de columna. Los tres usos de `TablePotentialPanel` (líneas 329, 810 y 842) deben pasar por esa misma comprobación.

Alternativa: rechazar al leer el fichero una columna con incertidumbre incompleta; hoy el lector no hace ninguna comprobación de ese tipo (el método `checkUncertainTable`, que no comprobaba nada, se quitó en el commit `13a2b89`).

Prueba: tabla con incertidumbre solo en el segundo estado de una columna; la columna debe quedar no editable y con las opciones «Edit/Remove uncertainty».

### Error 168 · [Jorge Rico] Tras alejar el zoom en modo inferencia, los nombres de estado recortados siguen recortados al volver a acercarlo

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.graphic |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/graphic/VisualState.java:37,317-320`<br>`gui/src/main/java/org/openmarkov/gui/graphic/VisualElement.java:132-146`<br>`gui/src/main/java/org/openmarkov/gui/graphic/FSVariableBox.java:64-82,100-106`<br>`gui/src/main/java/org/openmarkov/gui/graphic/NumericVariableBox.java:134-135`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:248-258` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** En modo inferencia, cada nodo desplegado dibuja sus estados con una barra de probabilidad y, delante, el nombre del estado. Si el usuario aleja el zoom (por ejemplo al 50 %), algunos nombres que caben enteros al 100 % se recortan; al volver al 100 %, siguen dibujándose recortados, como «ne…ve» en lugar de «negative». Vuelven a salir enteros cuando se recrean los estados visuales (`FSVariableBox.updateNumCases`, por ejemplo al cambiar el número de casos de evidencia). Es una consecuencia solo visual y menor.

`VisualState.paint` (módulo `gui`, paquete `org.openmarkov.gui.graphic`) acorta el nombre con `VisualElement.adjustText` para que quepa en 52 unidades (`InnerBox.BAR_HORIZONTAL_POSITION`) delante de la barra: si no cabe, deja el principio del nombre, «…» y las dos últimas letras. La línea 319 guarda el resultado en el campo `stateName` en lugar de usarlo solo para dibujar, y el nombre original se pierde en ese objeto.

El ancho disponible es una constante, así que agrandar la ventana o dibujar el nodo más ancho no cambia nada. Pero el ancho medido del texto sí depende del zoom: `NetworkEditorPanel.doPaint` escala el `Graphics2D` con el zoom (línea 251) y `adjustText` mide con ese contexto. Con el zoom alejado, un nombre se recorta, queda guardado recortado y ya no recupera su forma completa.

Otros usos del nombre guardado: `FSVariableBox.getVisualState(String)` busca por nombre, pero no tiene llamadores; `NumericVariableBox.updateNumCases` (líneas 134-135) recrea su estado

visual con el nombre guardado, que en un nodo numérico es el texto de su único estado.

Medido con la letra de los estados (11 puntos): «negative» mide 48 a escala 1 y 56 a escala 0,5; «Therapy», 46 y 52. Es decir, con el zoom al 50 % esos nombres se recortan (≥ 52). No se ha visto en la aplicación; se ha deducido de la lectura y de la medida de anchos.

**El código.** gui/src/main/java/org/openmarkov/gui/graphic/VisualState.java:317-320

```
        g.setColor(GUIColors.Inference.BOX_TEXT.getColor());
        g.setFont(STATES_FONT);
        this.stateName = adjustText(this.stateName, InnerBox.BAR_HORIZONTAL_POSITION, 2,
        ↪  STATES_FONT, g);
        g.drawString(this.stateName, (int) xName, (int) yText);
```

**Cómo arreglarlo.** Dibujar el texto acortado en una variable local y no tocar `stateName`. Buscar si hay otros `adjustText` que asignen a un campo (por ejemplo, en `VisualNode` para el nombre o el título del nodo) y aplicar la misma regla.

Prueba: pintar un `VisualState` con escala 0,5 y luego con escala 1 sobre una imagen en memoria y comprobar que `getStateName()` sigue devolviendo el nombre completo.

### Error 169 · [Corregido] Con una precisión que no contiene un «1» (0.0, 2.0, 0.5), los límites de los tramos se muestran con un punto final colgando

| | |
|---|---|
| **Estado** | Corregido en el commit `b58677a` (09-10-2026): el número de decimales sale del valor de la precisión (`Util.numDecimalsOf`), y lo usan `convertToString LimitValues` y `Util.roundWithPrecision`. Con precisión 0.0 o 2.0 los límites salen sin punto («5», «1234»); con 10.0 o 100.0 ya no se pierden cifras enteras ni se lanza una excepción; con 0.5 se enseña un decimal. Con 0.01, 0.1 y 1.0 no cambia nada. Con prueba de las dos funciones. Con precisión 0.0 o 2.0 los decimales de un límite se siguen cortando al enseñarlo (3,14 sale «3»): qué significa esa precisión es el error 40. No se ha probado en la ventana. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.component |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/component/DiscretizeTablePanel.java:596-639`<br>`gui/src/main/java/org/openmarkov/gui/component/DiscretizeTablePanel.java:563-568`<br>`gui/src/main/java/org/openmarkov/gui/component/DiscretizeTablePanel.java:1011`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:300-316`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:1007-1008`<br>`core/src/main/java/org/openmarkov/core/model/network/Util.java:178-202` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 40, error 207 |

**Qué pasa.** En la pestaña de dominio del diálogo de propiedades de un nodo numérico o discretizado, la tabla de tramos muestra los límites como texto. Si la precisión de la variable no contiene un «1» (por ejemplo 0.0, 2.0 o 0.5), cada límite finito sale con un punto final colgando: `5.`, `10.`, `1234.`.

Camino del usuario: abrir `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx`, abrir las propiedades del nodo de utilidad «EVAC Implante» (precisión 2.0), pestaña de dominio, y escribir 5 en el límite inferior (que era -∞). La tabla pasa a mostrar `(, 5., ,, ∞, )`. Lo mismo con «Reward for coffee [0]» de `0_miscellaneous/networks/pomdp/POMDP-coffee-robot.pgmx` (precisión 0.0).

Esas precisiones no son rebuscadas. El lector de `.pgmx` pone precisión 0.0 a una variable numérica cuyo elemento no trae `<Precision>` (`PGMXReader_0_2.java:1008`), y `new Variable(String)` también la deja en 0.0. En las redes de ejemplo del repositorio (`0_miscellaneous/networks`) hay 1251 nodos de utilidad con `<Precision>0.0</Precision>` y 327 con `<Precision>2.0</Precision>` (todos los de `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx`, por ejemplo). La lista desplegable de precisión del diálogo solo ofrece 1, 0.1, 0.01, 0.001 y 0.0001, que se muestran bien, así que el defecto aparece con precisiones que vienen de fichero.

`DiscretizeTablePanel.setDataFromPartitionedInterval` convierte los límites con `convertToStringLimitValues(limites, Double.toString(precision))`, que decide cuántos decimales mostrar buscando en el texto de la precisión la posición del punto y la del primer «1»: con «0.01» salen dos decimales y con «0.001» tres. Si la precisión no contiene un «1», el número de decimales queda en 0, y 0 no significa «sin decimales» sino «cortar justo después del punto».

Hay casos peores con la misma causa, que solo llegan desde un fichero: con precisión 10.0 el número de decimales sale negativo y el texto se corta antes del punto, de modo que 1234 se muestra como `123` y 3.14 como una cadena vacía; con precisión 1.0E-10 la rama de notación científica lee solo la primera cifra del exponente y muestra un decimal. Con esas precisiones el daño no se queda en lo que se ve: `Util.roundWithPrecision`, que se aplica al límite tecleado antes de guardarlo (`DiscretizeTablePanel`, línea 962), convierte con precisión 10.0 el 1234 en 123 y el 12345.678 en 1234, y lanza `NumberFormatException` con 5 o con 3.14; con precisión 100.0 convierte 1234 en 12, y `convertToStringLimitValues` lanza `StringIndexOutOfBoundsException`, de modo que, por lectura, la tabla de tramos no se puede ni rellenar (medido con las dos funciones aisladas). En las redes del repositorio no hay ninguna variable con esas precisiones. `Util.roundWithPrecision` (el redondeo que se aplica al número tecleado) interpreta el texto de la precisión con la misma lógica.

Medido con un pequeño programa de prueba que construye el panel real sin pantalla (el camino del usuario de arriba) y con la función aislada: precisión 0.0 → `[-∞, 0., 10., 3., 2., 1234., ∞]`; precisión 0.01 → `[-∞, 0.00, 10.00, 3.14, 2.50, 1234.00, ∞]`.

**El código.** gui/src/main/java/org/openmarkov/gui/component/DiscretizeTablePanel.java:596-630

```
public static String[] convertToStringLimitValues(double[] limits, String precision) {
    ...
    int indexE = precision.indexOf('E');
    if (indexE != -1) {
        numDecimals = Integer.parseInt(precision.substring(indexE + 2, indexE + 3));
    } else {
        int decimalPoint = precision.indexOf('.');
        int one = precision.indexOf('1');
        if (decimalPoint != -1 && one != -1) {
            numDecimals = one - decimalPoint;
        } else {
            numDecimals = 0;
        }
    }
    ...
            roundedStringDecimalPlace = rounded.indexOf('.');
            int finalLength = roundedStringDecimalPlace + numDecimals + 1;
            if (finalLength <= rounded.length()) {
                rounded = rounded.substring(0, finalLength);
```

**Cómo arreglarlo.** Calcular el número de decimales a partir del valor numérico de la precisión y no de su texto (por ejemplo, con `BigDecimal.valueOf(precision).stripTrailingZeros().scale()`, acotado a 0 por abajo), y cuando sea 0 no dejar el punto (cortar en `roundedString DecimalPlace`, sin el `+ 1`). Nunca truncar la parte entera.

La misma lectura del texto de la precisión está en `Util.roundWithPrecision(double, String)` (módulo `core`, `Util.java:178-202`), que debería usar la misma regla para que el valor redondeado y el mostrado coincidan.

Prueba de regresión: una prueba unitaria de `convertToStringLimitValues` con precisiones 0.0, 2.0, 0.5, 1.0, 10.0, 100.0, 0.01 y 1.0E-10 que exija que ningún texto termine en punto, que la parte entera nunca se recorte y que no se lance ninguna excepción; y otra de `Util.roundWithPrecision` con las mismas precisiones que exija que no quite cifras enteras (1234 no puede pasar a 123) ni lance.

### Error 170 · [Corregido] Con los nodos A, B y C ya creados, el siguiente nodo de azar se llama «E», la letra reservada para los nodos de suceso

| | |
|---|---|
| **Estado** | Corregido en el commit `0292789` (14-10-2026). `getNextChanceNodeName` comprueba antes de probar cada letra si está reservada, así que salta todas las reservadas vengan donde vengan. Por decisión de Manuel, la E solo se reserva en las redes que pueden tener nodos de suceso: `Util.getNextNodeName` recibe si la red los admite, y los dos llamadores lo calculan con la misma regla que usa el editor para ofrecer el nodo de suceso (`NodeTypeInfo.of(EVENT).canBeUsedInProbnet`). En las demás redes la E es una letra más: tras A, B y C viene la E. Pruebas: con sucesos, A, B y C dan «F»; sin ellos, «E»; vacía, «A»; de la A a la T sin las reservadas, «V»; con todas las libres ocupadas, «A1» (o «E» sin sucesos). Con el código anterior fallaban tres. |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/Util.java:370-405,429-445`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EditorCommandHandlerGenerator.java:772`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EditorInputHandler.java:698` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** Al crear un nodo de azar con doble clic en el lienzo o desde el menú contextual, en una red que ya tiene los nodos A, B y C, el nombre propuesto es «E», y el siguiente nodo de suceso pasaría a llamarse «E1». No hay error, pero rompe la convención de que la E es de los nodos de suceso, que `getNextEventNodeName` sigue.

El doble clic crea el nodo con `EditorCommandHandlerGenerator.createNode` y el menú contextual con `EditorInputHandler.createNode`; los dos piden el nombre a `Util.getNextNodeName` (módulo `core`). Para un nodo de azar, `getNextChanceNodeName` recorre las letras desde la A buscando la primera libre y quiere saltarse la D (decisiones), la U (utilidades) y la E (sucesos). Pero el salto es un `if` que avanza una sola letra: desde la C avanza a la D, la reconoce y avanza a la E, y en la siguiente vuelta prueba la E como candidata sin volver a mirar si está reservada.

La U y la D sí se saltan bien (tras la U viene la V, que no está reservada, y a la D solo se llega desde la C). El javadoc dice que las únicas letras que nunca devuelve son la D y la U, lo que tampoco coincide con la intención de saltar la E.

Medido con un pequeño programa de prueba sobre `Util.getNextNodeName`: con `{A, B, C}` el nodo de azar se llama «E»; con `{A, B, C, E}` se llama «F».

**El código.** core/src/main/java/org/openmarkov/core/model/network/Util.java:390-400

```
            while (!found && (letter <= 'Z')) {
                name = letter + ((index > 0) ? Integer.toString(index) : "");
                if (!existingNames.contains(name)) {
                    found = true;
                } else {
                    letter++;

                    if ((letter == 'D') || (letter == 'U') /*Events added*/ || (letter ==
                    ↪  'E')) {
                        letter++;
                    }
                }
            }
```

**Cómo arreglarlo.** Cambiar el `if` por un `while` que avance mientras la letra sea D, E o U (o comprobar la reserva antes de probar cada candidata, lo que además cubre el caso de empezar por una letra reservada), y corregir el javadoc para que diga las tres letras.

Prueba: `getNextNodeName(CHANCE, {A, B, C})` debe dar «F»; `{}` debe dar «A»; `{A…T}` sin D ni E debe dar «V».

### Error 207 · [Corregido] En la pestaña Domain de un nodo numérico, el selector de precisión enseña «1» cuando la precisión de la variable es 0.0001 o un valor que no está en su lista (2, 0.5, 0…)

| | |
|---|---|
| **Estado** | Corregido en el commit `fc06e94` (15-10-2026). El selector se elige comparando números (`selectPrecision`), y una precisión que no está en la lista se añade en su sitio: 0.0001 enseña «0.0001», 2.0 enseña «2» y 0.0 enseña «0». Además, colocar el selector al abrir la ventana ya no cuenta como una elección del usuario: antes redondeaba los límites de los tramos a la precisión (0.25 pasaba a 0.2 con precisión 0.1) y, con 2.0 o 0.0 en la lista, los habría dejado en números enteros. Con pruebas que abren el panel. No se ha probado en la ventana. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:296,506-516,959-963` |
| **Comprobado** | Leyendo el código y ejecutándolo; visto en una captura automática de la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 40, error 169 |

**Qué pasa.** El selector «Precision» de la pestaña Domain ofrece cinco valores: 1, 0.1, 0.01, 0.001 y 0.0001. Para una variable cuya precisión es 0.0001, la última de la lista, enseña «1». Lo mismo para cualquier precisión que no esté en la lista. El usuario ve una precisión que la variable no tiene, y no hay otro sitio de la ventana donde ver la verdadera.

Visto el 8-10-2026 en una captura automática de la ventana: el nodo de utilidad `Survivors_QALE` de `0_miscellaneous/networks/id/1-0/ID-mediastinet-1-0.pgmx`, cuyo fichero dice `<Precision>2.0</Precision>`, enseña «1».

Por qué pasa: la línea 296 elige en la lista el texto `String.valueOf(precision)`, y `setSelectedItem` no hace nada cuando ese texto no es uno de los de la lista. Medido aparte con una lista igual:

```
precisión 1.0    -> "1.0"     -> la lista enseña 1   (no coincide; acierta porque 1 es la
↪  primera)
precisión 0.1    -> "0.1"     -> 0.1
precisión 0.01   -> "0.01"    -> 0.01
precisión 0.001  -> "0.001"   -> 0.001
precisión 0.0001 -> "1.0E-4"  -> 1
precisión 2.0    -> "2.0"     -> 1
precisión 0.0    -> "0.0"     -> 1
precisión 0.5    -> "0.5"     -> 1
```

`String.valueOf` de un número menor que 0,001 lo escribe en notación científica (es lo mismo que rompe las regresiones en el error 12). Las precisiones que no están en la lista no son raras: muchos nodos de utilidad de las redes de ejemplo tienen 0.0 (error 169).

La precisión de la variable no cambia por abrir la ventana: el cambio solo se guarda cuando el usuario elige otro valor (líneas 959-963).

**Cómo arreglarlo.** Elegir comparando números y no textos: recorrer la lista y señalar la entrada cuyo valor numérico es el de la variable. Si la precisión no está en la lista, añadirla como una entrada más para que se vea, en vez de enseñar otra.

Prueba: con el cálculo de la entrada elegida extraído a un método, comprobar que 0.0001 elige «0.0001» y que 2.0 no elige «1».

### Error 209 · [Corregido] En el panel de la relación de un nodo, los rótulos «Distribution» y «Parametrization» se quedan a la vista aunque la relación sea una tabla y sus listas estén escondidas

| | |
|---|---|
| **Estado** | Corregido en el commit `295a7a3` (15-10-2026), de Jorge Rico: el constructor de `PotentialEditPanel` termina llamando a `updatePotentialPanel`, que esconde juntos las dos listas y sus rótulos. Visto en la ventana el 16-10-2026: en la relación de «Dyspnea» de `BN-asia.pgmx` los dos rótulos ya no se ven. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:179-198,285-296,582-600` |
| **Comprobado** | Leyendo el código; visto en una captura automática de la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 179, error 150 |

**Qué pasa.** Al abrir la relación de un nodo cuya relación es una tabla (pestaña Probability de sus propiedades, o «Edit probability»), junto al selector «Relation Type» se leen dos palabras sueltas, «Distribution» y «Parametrization», sin nada al lado. Son los rótulos de dos listas que solo tienen sentido cuando la relación es una distribución con parámetros (`UnivariateDistr`), y que en una tabla están escondidas.

Visto el 8-10-2026 en una captura automática de la pestaña Probability del nodo `Dyspnea` de `BN-asia.pgmx`.

Por qué pasa: al construirse, `PotentialEditPanel.configureComponentsPanel` (líneas 186-198) esconde las dos listas y no sus rótulos. `updatePotentialPanel` (líneas 587-600), que se ejecuta cuando el usuario cambia el tipo de relación, sí esconde las cuatro cosas, así que los rótulos

desaparecen en cuanto se cambia de tipo y se vuelve.

**Cómo arreglarlo.** Que `configureComponentsPanel` y `updatePotentialPanel` llamen a un mismo método que enseñe o esconda juntos los dos rótulos y las dos listas.

Prueba: con un nodo de tabla, comprobar tras construir el panel que los dos rótulos no son visibles. El panel necesita pantalla, porque crea un editor de comentarios que es una ventana.

### Error 225 · [Corregido] En el panel de una distribución univariante, escribir un parámetro en su casilla acaba en la ventana «Unexpected error» y el valor no cambia

| | |
|---|---|
| **Estado** | Corregido en el commit `652a0c8` (09-10-2026): la casilla de un parámetro se edita con el campo de expresiones, `AugmentedValuesTable.setValueAt` toma la tabla de la relación que tenga el nodo, y las casillas con los estados del padre ya no entran en edición (entraban, y acababan en el mismo error). Con prueba, que corre sin pantalla. Visto en la ventana, con el mismo guion: tras el doble clic la casilla se edita con el campo de expresiones y, al aceptar «2», el número y la expresión del parámetro valen 2 y no sale ninguna ventana de error. El diálogo de expresiones que no se abre sigue igual; no se ha probado con el ratón ni desde el menú. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.component; org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/component/AugmentedValuesTable.java:96-121`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/UnivariateDistrPotentialPanel.java:331-371,380-386`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/AugmentedProbTablePotentialPanel.java:357-385` |
| **Comprobado** | Leyendo el código y en la ventana, con un guion |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 153 |

**Qué pasa.** Camino del usuario: abrir `0_miscellaneous/networks/id/1-0/ID-CEA-minimal-1-0.pgmx`, abrir la relación de «Effectiveness» («Edit probability»), que es una distribución univariante («UnivariateDistr», «Exact»), hacer doble clic en la casilla de un parámetro o seleccionarla y empezar a escribir, poner «2» y pulsar Intro. Sale la ventana «Unexpected error» con `ClassCast Exception: String cannot be cast to VariableExpression`, y el parámetro sigue valiendo lo mismo. No hay otra manera de escribir un parámetro en este panel.

Visto el 09-10-2026 en la ventana, abriendo el diálogo de la relación con un guion que le envía el doble clic y el texto (no con el ratón ni desde el menú), con el gestor de errores de la aplicación puesto: la casilla entra en edición con un campo de texto corriente, y al terminar aparece «Unexpected error». El número y la expresión del parámetro quedan como estaban (0.8).

Por qué pasa. La tabla del panel es una `AugmentedValuesTable`, y su `setValueAt` (líneas 96-121) espera recibir una expresión: la toma del campo de expresiones si ese es el editor de la casilla y, si no, convierte sin más el valor recibido (línea 109). `AugmentedProbTablePotentialPanel`, el otro panel que usa esa tabla, le pone como editor el campo de expresiones (líneas 357-385); `UnivariateDistrPotentialPanel` no pone ninguno, así que la casilla se edita con el campo de texto por omisión y lo que llega es un texto.

En el mismo método hay un segundo fallo, que hoy no se alcanza porque el primero salta antes: después de aplicar la edición, la línea 115 convierte la relación del nodo a `AugmentedProbTable`

`Potential`, y la de este panel es un `UnivariateDistrPotential`. Medido llamando al método con una expresión: el valor se aplica y a continuación salta `ClassCastException`.

El panel tiene además un diálogo para escribir la expresión (`doubleClickEvent`, líneas 357-371), que no se abre: el oyente del ratón se añade al panel que contiene las tablas (línea 349) y no a las tablas, que son las que reciben el clic. Con el doble clic enviado a la casilla, el diálogo no apareció.

**Cómo arreglarlo.** Que la casilla se edite con el campo de expresiones, como en `AugmentedProbTablePotentialPanel`, y que `AugmentedValuesTable.setValueAt` tome la tabla de la relación que tenga el nodo, sea de una clase o de la otra. Hay que decidir qué se hace con el diálogo de expresiones que no se abre: engancharlo a las tablas o quitarlo.

Prueba: construir el panel de «Effectiveness», editar la casilla como lo hace el teclado, escribir «2» y comprobar que no hay excepción y que el número y la expresión del parámetro valen 2.

### Error 226 · [Corregido] El diálogo de incertidumbre rechaza toda columna con dos o más distribuciones de Dirichlet, que es justo la que su regla pide, y acepta una Dirichlet sola

| | |
|---|---|
| **Estado** | Corregido en el commit `8c46ebe` (09-10-2026): `doVerifyRule3` rechaza una Dirichlet sola y comprueba las columnas con dos o más. Con prueba de la regla. Visto en la ventana, con el mismo guion: aceptar sin cambios el diálogo de la columna de «State [1]» lo cierra sin ningún mensaje. Al aceptarlo, los números de la columna pasan a ser las medias exactas de las distribuciones; el fichero las traía redondeadas a diez decimales. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/UncertainValuesDialog.java:649-672`<br>`gui/src/main/resources/gui/localize/gui_class_localizations_en.xml:53-54` |
| **Comprobado** | Leyendo el código, ejecutándolo y en la ventana, con un guion |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 76, error 167 |

**Qué pasa.** Camino del usuario: abrir `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`, abrir la tabla del nodo «State [1]», hacer doble clic en una columna cuya incertidumbre es Dirichlet(15), Dirichlet(512), Dirichlet(731) y un cero exacto, y pulsar OK sin cambiar nada. El diálogo responde «Family rule nº3 has been broken» y no se cierra. No se puede aceptar en el diálogo ninguna columna con distribuciones de Dirichlet, así que tampoco editarla.

Visto el 09-10-2026 en la ventana, abriendo el diálogo con un guion que envía el doble clic y pulsa OK (no con el ratón ni desde el menú).

El diálogo comprueba al aceptar tres reglas sobre las distribuciones de una columna. La tercera, tal como la enseña el mensaje, dice: si una de las distribuciones es una Dirichlet, las demás deben ser «Exact» en cero o Dirichlet, y al menos otra debe ser también una Dirichlet. `doVerifyRule3` hacía lo contrario: con una sola Dirichlet comprobaba que el resto fueran ceros exactos y la aceptaba, y con dos o más lanzaba siempre.

Medido el 09-10-2026 pasando por las reglas del diálogo las 86 columnas con incertidumbre de las 148 redes legibles de `0_miscellaneous/networks`: 8 no pasaban, todas por esta regla y todas con esa forma, en el nodo «State [1]» de `MID-Chancellor-new` y `MID-Chancellor-corrected` (formatos 0.2 y 1.0).

La regla se ha juzgado por su propio texto, que es el que ve el usuario; no se ha contrastado con otra fuente.

**Cómo arreglarlo.** Intercambiar los dos casos de `doVerifyRule3`: rechazar una Dirichlet sola y, con dos o más, comprobar que el resto son ceros exactos.

El mensaje de las tres reglas no dice qué estado de la columna las incumple ni qué cambiar, y habla de «v = 0» cuando el parámetro se llama «nu»; eso se hizo al pasar las reglas al módulo `core` (commit `1a8cfe7`, error 76).

Prueba: la columna de «State [1]» se acepta, una Dirichlet sola se rechaza, dos Dirichlet con otra distribución se rechazan y una Beta con complemento se sigue aceptando.

### Error 238 · En el panel de la relación exponencial por tramos, marcar «Use first interval» cambia por dentro la fórmula del instante de entrada, pero el cuadro sigue enseñando la anterior, y desmarcarla no deshace el cambio

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/PiecewiseExponentialPanel.java:80-82,118`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/FunctionPotentialPanel.java:125-142` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 152 |

**Qué pasa.** Al marcar la casilla, `PiecewiseExponentialPanel` (líneas 81-82) llama a `FunctionPotentialPanel.setFunction` con el tiempo del primer tramo. `setFunction` (líneas 140-142) solo cambia el campo `function`; no toca el cuadro de texto, que sigue enseñando la fórmula anterior. Al aceptar se guarda la nueva (`saveChanges` usa `getFunction`, línea 118), así que se guarda algo distinto de lo que se veía. Desmarcar la casilla no hace nada, porque la acción solo actúa si está marcada, así que la fórmula no vuelve a la anterior.

Además, `setFunction` construye la expresión sobre todas las variables de la relación, incluida la del propio nodo, mientras que el diálogo de la fórmula, que se abre con doble clic en el cuadro, solo admite a los padres (líneas 128-129).

En la rama `DES-development` de Carmen Yago, `FunctionPanel.setFunction` cambiaba también el texto; se perdió al juntar `FunctionPanel` con `FunctionPotentialPanel`.

**Cómo arreglarlo.** Que `setFunction` ponga también el texto en el cuadro y construya la expresión solo sobre los padres, como el diálogo; y que al desmarcar vuelva la fórmula que había antes de marcar.

Prueba: en el panel de una relación exponencial por tramos, marcar la casilla enseña el tiempo del primer tramo; desmarcarla devuelve la fórmula anterior; y lo que se guarda al aceptar es lo que se ve.

### Error 242 · En modo inferencia, el valor de un nodo de utilidad cuyo rango no se puede calcular se escribe más ancho que la columna que tiene reservada y se sale por la derecha de la caja del nodo

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.graphic; org.openmarkov.gui.window.edition.networkEditorPanel |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/graphic/NumericVariableBox.java:79-85,169-175`<br>`gui/src/main/java/org/openmarkov/gui/graphic/InnerBox.java:61,123-125`<br>`gui/src/main/java/org/openmarkov/gui/graphic/VisualState.java:347`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/InferencePresenter.java:61-75`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:586-607` |
| **Comprobado** | Leyendo el código y ejecutándolo (la propagación del modo inferencia, el cálculo de rangos y el sitio donde acaba cada número, medido con las clases de la caja y la fuente del programa, sin ventana); no se ha visto en la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 46 |

**Qué pasa.** En modo inferencia, la caja de un nodo de utilidad escribe a la derecha de la barra lo que se espera que valga. Cuando el rango de esa utilidad no se puede calcular —por ejemplo, porque su valor depende de una variable numérica sin hallazgo—, el número es más ancho que la columna que se le reserva y se pinta por fuera del borde derecho de la caja: con un número de cuatro cifras enteras, 16 píxeles fuera. Se llega abriendo una de estas redes y pulsando el botón del modo inferencia, sin poner ningún hallazgo:

- `0_miscellaneous/networks/mid/MID-Chancellor.pgmx`: «Cost lamivudine [0]», que vale 2085.5044;
- `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-4.7.pgmx`: «Drug cost [0]», 4364.5000 (la red de la prueba del error 46);
- `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-2.5.pgmx`: «Cost [0]», 7120.5000;
- `0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx`: «Inpatient costs [0]», 12.0598, y «Outpatient costs [0]», 19.2826, que se salen 2 píxeles.

Por qué pasa. Desde `0697d849` (24-09-2026, incidencia 677) cada caja mide sus columnas con su fuente. La del número de una utilidad mide lo que ocupen los dos extremos de su rango, y nunca menos que «0.0000» (`NumericVariableBox.getValueColumnWidth`, líneas 169-175: «Wide enough for the value at either end of the range, so for every value in between»; `InnerBox.VALUE_COLUMN_WIDTH`, línea 61). El número se escribe empezando donde empieza esa columna (`InnerBox.getValueX`, líneas 123-125; `VisualState.paint`, línea 347), así que uno más ancho que ella se sale por la derecha. Desde el arreglo del error 46, un nodo cuyo rango no se calcula se queda sin él (`EvidenceManager.computeUtilityRanges`, líneas 586-607) y conserva los límites del dominio de su variable (`InferencePresenter.paintInferenceResultsUtilityNode`, líneas 68-74; el constructor de la caja, líneas 79-85), que para estas utilidades son -∞ e ∞. Escritos con el formato de la caja, «-∞» y «∞» son más estrechos que «0.0000», y la columna se queda con el ancho de «0.0000»: 39 píxeles. Lo que el número vale de verdad no cuenta.

Medido el 15-10-2026, sin ventana, con el jar del 14 de octubre y delante las clases que cambiaron

el 15 (las de la caja y de la ventana que se nombran aquí no han cambiado): en las redes de `0_miscellaneous/networks` (sin las copias de `1-0`), los rangos con `EvidenceManager.computeUtilityRanges` y los valores con `VEPropagation` sobre todas las variables y sin evidencia, como `EvidenceManager.doPropagation`. De los ocho nodos de utilidad sin rango de las redes que se pueden propagar, cinco tienen un valor que no cabe. Dónde acaba cada número, con una `NumericVariableBox` de una utilidad recién creada (dominio (-∞, ∞)) armada sobre un `VisualNetwork`, como en `ValuesFitInTheInnerBoxTest`, y la fuente del programa (DejaVu Sans de 11 puntos):

```
valor       columna   acaba en   debería acabar en (BOX_WIDTH - STATES_INDENT)   borde de
↪  la caja
2085.5044   39        214        193                                              198
7120.5000   39        214        193                                              198
12.0598     39        200        193                                              198
0.9300      39        193        193                                              198
```

La última fila es «QoL [0]» de `MID-HPV.pgmx`, otra utilidad sin rango cuyo número sí cabe.

**El código.** gui/src/main/java/org/openmarkov/gui/graphic/NumericVariableBox.java:169-175

```
    /**
     * Wide enough for the value at either end of the range, so for every value in between.
     */
    @Override protected double getValueColumnWidth() {
        return Math.max(VALUE_COLUMN_WIDTH,
        ↪  Math.max(getTextWidth(VisualState.formatValue(minValue)),
                                                    getTextWidth(VisualState.formatValue(
                                                    ↪  maxValue))));
    }
```

**Cómo arreglarlo.** Que la columna del número mida también los valores que la caja va a escribir: en `getValueColumnWidth`, el máximo entre lo de ahora y el ancho de cada valor de su `VisualState`. Conviene medir los de todos los casos de evidencia y no solo el del caso actual, para que la barra no cambie de largo al pasar de un caso a otro. Quitarle esos píxeles a la barra no cambia nada en estos nodos: con un rango infinito, el largo de la barra sale NaN y no se pinta.

Prueba: en `ValuesFitInTheInnerBoxTest`, una caja de utilidad sin límites (la variable recién creada tiene dominio (-∞, ∞)) con 2085.5044 puesto en su `VisualState` con `setStateValue(0, 2085.5044)`; `assertValueFits(box, 2085.5044)` tiene que pasar. Hoy el número acaba en 214 y el límite está en 193.

## Error 245 · [Corregido] En el editor del árbol de un nodo que es padre de sí mismo, el menú de una rama ofrece colgar un subárbol que pregunte por la variable del propio nodo, y el menú del árbol no ofrece ponerla arriba, porque cada menú la quita de la lista a su manera

| | |
|---|---|
| **Estado** | Corregido en el commit `44116cb` (15-10-2026), con la decisión que tomó Manuel Arias ese día: el árbol puede preguntar por el valor que el nodo tenía, y lo ofrecen los tres menús. La regla está escrita una sola vez, en `TreeADDPotential.variablesToAskFor` (todas las variables del árbol menos una aparición de la que define), y la usan «Change root variable», «Add subtree» y `TreeADDBranch.getAddableVariables`. Para el usuario solo cambia «Change root variable», que ahora ofrece la variable del nodo; los otros dos menús dan lo mismo que antes, comprobado caso por caso, también en los árboles de utilidad. Medido con una simulación de diez años: un árbol de exclusión de sucesos de «Health», padre de sí mismo, que pregunta primero por «Health» («h0» pasa a «h1» y «h1» a «h0», con un suceso en el año 3) da 14300 (tres años con «h0» y |

| | |
|---|---|
| | siete con «h1»), y lo mismo tras guardarlo y volverlo a abrir. Cómo se elige al empezar la simulación el valor de un nodo padre de sí mismo no se ha mirado. Las variables que una rama de más arriba ha dejado con un solo estado ya se quitaban en todas sus apariciones. Con prueba en `gui` de los tres menús, con y sin el bucle. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.treeadd |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:180-205,289-305,334-363` `core/src/main/java/org/openmarkov/core/model/network/potential/treeadd/TreeADDBranch.java:239-254` |
| **Comprobado** | Leyendo el código y ejecutándolo (los menús del editor, construido sin pantalla) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | Sí: si el árbol de un nodo padre de sí mismo puede preguntar por el valor que el nodo tenía, como ya pueden añadirlo las hojas desde el arreglo del error 18 |
| **Relacionados** | error 18, error 159, error 160, nota 164 |

**Qué pasa.** En una red de simulación de sucesos discretos (DES) un nodo puede ser padre de sí mismo: su relación depende del valor que el propio nodo tenía. Si esa relación es un árbol, la variable del nodo está dos veces entre las del árbol: una es la que el árbol define y la otra es la de ese padre. Desde `5555f2a` (error 18), el menú de una hoja y el diálogo «Add variables to potential» ofrecen a propósito esa segunda aparición: `TreeADDBranch.getAddableVariables` cuenta cuántas veces está la variable para reconocer el bucle sobre sí mismo. Los otros dos menús que eligen por qué variable preguntar la tratan de forma distinta entre sí:

- el menú del árbol entero, «Change root variable» (`setContextualMenuTreeADD`, líneas 180-205), recorre las variables y se salta **todas** las que son la del nodo, porque compara por identidad: no la ofrece nunca;
- el menú de una rama, «Add subtree» (`possibleRootVariables`, líneas 334-363), quita de la lista la variable de la rama y la del nodo con `List.remove(Object)`, que solo quita **la primera vez** que aparece: la otra se queda y se ofrece.

Así que se puede colgar bajo una rama un subárbol que pregunte por la variable del nodo, pero no se puede poner esa misma pregunta arriba del árbol. Medido el 15-10-2026 sin ventana, con un árbol escrito a mano para un nodo «Health» con los padres «Stage», «Sex» y él mismo (variables del árbol `Health, Stage, Sex, Health`), que pregunta primero por «Stage»; los menús que arma el editor, dichos por sus mandos:

```
menu=hand-made-self-loop|Health|tree||ChangeRootVariable>Sex
menu=hand-made-self-loop|Health|branch|0|AddSubTree>Sex;Health,-,JoinBranches>late,-,SetLa⌋
↪  bel
menu=hand-made-self-loop|Health|leaf|0|EditPotential,-,AddVariables>Sex;Health
```

Colgar el subárbol funciona en el editor: medido el 14-10-2026 llamando a la operación del panel, la rama pasa a preguntar por «Health», con sus dos estados. No se ha mirado si un árbol así se guarda, se vuelve a abrir y se simula bien.

No está claro cuál de los dos menús dice lo que se quiere. Que «Add subtree» la ofrezca es una casualidad de `List.remove`, que nadie ha escrito a propósito; pero va en la línea de lo que se decidió para las hojas en el error 18. Lo mismo pasa con las demás variables que esos métodos quitan de la lista (la de una rama de un solo estado más arriba, en `possibleRootVariables` y en `setContextualMenuPotential`, líneas 289-305): se quitan una vez cada una.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:189-190 y 344-345

```
for (Variable variable : treeADD.getVariables()) {
    if (variable != currentRootVariable && variable != conditionedVariable) {
```

```
possibleRootVariables.remove(branch.getRootVariable());
possibleRootVariables.remove(branch.getPotential().getConditionedVariable());
```

**Cómo arreglarlo.** Decidir primero si el árbol de un nodo padre de sí mismo puede preguntar por el valor que el nodo tenía. Si puede, como ya admiten las hojas, «Change root variable» debe saltarse solo la aparición que el árbol define, y no todas: contando apariciones, como hace `getAddableVariables`. Si no puede, `possibleRootVariables` debe quitarlas todas (`removeIf`) y las hojas dejar de ofrecerla. En los dos casos conviene un único método con la regla que usen los tres menús, porque hoy está escrita tres veces de tres maneras, y que las variables de una rama de un solo estado se quiten también todas o una a propósito, no por el método de la lista que se use.

Prueba: con el árbol escrito a mano descrito arriba, comprobar que los tres menús ofrecen la variable del nodo, o que no la ofrece ninguno, según lo que se decida; y que en un árbol corriente no la ofrece ninguno.

### Error 263 · En la pestaña Domain de las propiedades de un nodo de utilidad, el selector «Precision» enseña siempre «1», tenga la variable la precisión que tenga, y elegir «1» en él no cambia nada

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:126-151`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:183-195,291-297,507-517,527-536,979-983` |
| **Comprobado** | Leyendo el código y ejecutándolo (el panel de la pestaña, construido sin ventana como lo construye el diálogo); no se ha abierto el diálogo en la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 207, error 169, error 264 |

**Qué pasa.** Con un nodo de utilidad, menú contextual → «Node properties» → pestaña «Domain»: el selector «Precision» enseña «1» sea cual sea la precisión de la variable. El usuario ve una precisión que la variable no tiene, y no hay otro sitio de la ventana donde ver la verdadera. Es lo que el commit `fc06e94` corrigió para los nodos de azar, de decisión y de suceso (error 207); en los de utilidad, ese arreglo no llega a ejecutarse.

Además, como «1» ya está elegido, elegirlo en la lista no hace nada: para dar precisión 1 a una utilidad que tiene otra hay que elegir antes cualquier otro valor y después «1».

Afecta a casi todos los nodos de utilidad. En las 148 redes de `0_miscellaneous/networks` que se leen hay 779, y solo 10 tienen precisión 1: 318 tienen 0.01, 316 tienen 2, 127 tienen 0 (error 169) y 8 tienen 0.1.

Por qué pasa. `NodePropertiesDialog.setFieldsFromProperties` (líneas 126-151) solo rellena la pestaña Domain si el nodo es de azar, de decisión o de suceso (línea 130). Para un nodo de utilidad, el panel `NodeDomainValuesTablePanel` se construye —`initialize` (líneas 183-195) escribe la precisión en `jFormattedTextFieldPrecision`, un campo que no está en la disposición de la pestaña—, pero no se llama a su `setFieldsFromProperties`, que es donde `selectPrecision` (línea 297) coloca el selector. El selector, que nada esconde para un nodo de utilidad, se queda en

su primera entrada, «1» (líneas 507-517). Y el oyente del selector (líneas 979-983) solo cambia la precisión cuando recibe un `ItemEvent` de selección, que un `JComboBox` no lanza si se elige la entrada que ya estaba elegida.

Medido el 15-10-2026 con las clases de `fc06e94` delante del jar del 14 de octubre, construyendo el panel como lo construye el diálogo para un nodo de utilidad («construido») y rellenándolo después como lo hace con un nodo de azar («rellenado»):

```
Survivors_QALE (utilidad, precisión 2)      construido:  [*1] [0.1] [0.01] [0.001] [0.0001]
                                            rellenado:   [*2] [1] [0.1] [0.01] [0.001]
                                            ↪  [0.0001]
Net_QALE       (utilidad, precisión 2)      construido:  [*1] [0.1] [0.01] [0.001] [0.0001]
Survivors_QALE (utilidad, precisión 0.01)   construido:  [*1] [0.1] [0.01] [0.001] [0.0001]
                                            rellenado:   [1] [0.1] [*0.01] [0.001] [0.0001]
```

(`0_miscellaneous/networks/id/1-0/ID-mediastinet-1-0.pgmx`; la última, con la precisión puesta a 0.01 por programa; el asterisco señala la entrada elegida, y el selector está visible en todos.) Y en «Cost lamivudine [0]» de `0_miscellaneous/networks/mid/MID-Chancellor.pgmx`, de precisión 0.01, elegir «1» en el selector la deja en 0.01; elegir «0.1» y luego «1», en 1.

`Survivors_QALE` era el ejemplo de la ficha del error 207: con `fc06e94` sigue enseñando «1». Esa ficha achacaba el «1» a que `String.valueOf` escribe «2.0», que es cierto para los demás nodos; en los de utilidad, el selector no llega a buscar nada.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:126-131

```
    private void setFieldsFromProperties(Node node) throws
    ↪  ThereIsNoPotentialInNodeException {
        this.node = node;
        this.setTitle(this.stringDatabase.getString("NodePropertiesDialog.Title") + ": " +
        ↪  node.getName());
        this.nodeDefinitionPanel.setNodeProperties(node);
        if (node.getNodeType() == NodeType.CHANCE || node.getNodeType() ==
        ↪  NodeType.DECISION || node.getNodeType() == NodeType.EVENT) {
            this.getNodeDomainValuesTablePanel().setFieldsFromProperties(node);
```

**Cómo arreglarlo.** Colocar el selector también para los nodos de utilidad. Lo más directo es hacerlo en el propio panel: que `initialize`, en su rama de utilidad, llame a `selectPrecision(getJComboBoxPrecision(), node.getVariable().getPrecision())` en lugar de escribir la precisión en `jFormattedTextFieldPrecision`. La otra vía, que el diálogo llame a `setFieldsFromProperties` del panel también con `UTILITY`, obliga a revisar que su rama `NUMERIC` no enseñe a un nodo de utilidad nada que no deba tener. De paso conviene quitar `jFormattedTextFieldPrecision`, que no se ve. Lo que se haga debe respetar lo que `fc06e94` ya cuidó: colocar el selector no cuenta como una elección del usuario.

Prueba: en `NodeDomainValuesTablePanelTest`, construir el panel sobre un nodo de utilidad de precisión 0.01 y, sin llamar a `setFieldsFromProperties` —como hace el diálogo—, comprobar que el selector elige «0.01»; y que elegir después «1» deja la variable con precisión 1.

### Error 264 · Si un .pgmx da a una variable la precisión «NaN», «Infinity» o «-Infinity», el fichero se abre sin aviso, pero «Node properties» sobre ese nodo acaba en la ventana de error no previsto y el diálogo no se abre

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui, io |
| **Paquete** | org.openmarkov.gui.dialog.node; org.openmarkov.io.probmodel.reader |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:291-297,527-536`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:698,801,821-828`<br>`core/src/main/java/org/openmarkov/core/model/network/Variable.java:514-517,661-663`<br>`core/src/main/resources/val_v4.xsd:62` |
| **Comprobado** | Leyendo el código y ejecutándolo (el lector y el panel de la pestaña Domain, sin ventana); que el diálogo no llega a abrirse está leído en su código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 207, error 111, error 263 |

**Qué pasa.** Un fichero `.pgmx` escrito fuera del programa, con `<Precision>NaN</Precision>`, `<Precision>Infinity</Precision>` o `<Precision>-Infinity</Precision>` en una variable, se abre sin ningún aviso. Pero «Node properties» sobre un nodo de azar, de decisión o de suceso con esa variable saca la ventana de error no previsto con `java.lang.NumberFormatException: Infinite or NaN`, y el diálogo del nodo no se abre. Nada del programa escribe una precisión así: solo se llega con un fichero hecho a mano o por otro programa. Antes de `fc06e94` (15-10-2026) el diálogo se abría, con el selector en «1».

Por qué pasa. Desde `fc06e94` (error 207), `NodeDomainValuesTablePanel.selectPrecision` (líneas 527-536) añade a la lista la precisión que no está en ella, escrita con `BigDecimal.valueOf(precision)` (línea 533), y `BigDecimal` no admite ni infinitos ni NaN. El diálogo llama a ese método al rellenar la pestaña Domain (`setFieldsFromProperties`, línea 297) antes de hacerse visible. Y nadie para antes esos números:

- el lector los deja pasar: `PGMXReader_0_2.getXMLPrecision` (líneas 821-828), que usa también el formato 1.0, lee `<Precision>` con `Double.valueOf`, que entiende «NaN» e «Infinity», y la línea 801 se la pone a la variable;
- `Variable.setPrecision` (líneas 514-517) no mira el número;
- el esquema tampoco: `val_v4.xsd` no le da tipo al elemento `<Precision>` (línea 62), así que la comprobación que hace la ventana al abrir un fichero (error 111) no avisa (leído en el esquema, no comprobado con la ventana).

Medido el 15-10-2026 con las clases de `fc06e94` delante del jar del 14 de octubre, sobre una red bayesiana de dos nodos, A → B, con B discretizada y su `<Precision>` cambiada a mano; el fichero se lee con `PGMXReader` y la pestaña se rellena como lo hace el diálogo:

```
<Precision>NaN</Precision>          leída NaN          NumberFormatException: Infinite or NaN
<Precision>Infinity</Precision>     leída Infinity     NumberFormatException: Infinite or NaN
<Precision>-Infinity</Precision>    leída -Infinity    NumberFormatException: Infinite or NaN
<Precision>2</Precision>            leída 2.0          [*2] [1] [0.1] [0.01] [0.001] [0.0001]
```

```
    at java.base/java.math.BigDecimal.valueOf(BigDecimal.java:1388)
    at org.openmarkov.gui.dialog.node.NodeDomainValuesTablePanel.selectPrecision(NodeDomai⌋
    ↪  nValuesTablePanel.java:533)
```

```
    at org.openmarkov.gui.dialog.node.NodeDomainValuesTablePanel.setFieldsFromProperties(N
    ↪  odeDomainValuesTablePanel.java:297)
```

Con un nodo de utilidad no pasa, porque el diálogo no rellena su pestaña Domain (error 263); al corregir aquél, pasará también con ellos.

Una precisión así hace daño también fuera del diálogo (leído en el código, no medido): con precisión infinita, `Variable.round` (líneas 661-663) devuelve NaN para cualquier valor (cero por infinito), y ese redondeo es el que usa la discretización de las variables numéricas antes de resolver.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:527-536

```
    static void selectPrecision(JComboBox<String> comboBox, double precision) {
        int index = 0;
        while (index < comboBox.getItemCount() &&
        ↪  Double.parseDouble(comboBox.getItemAt(index)) > precision) {
            index++;
        }
        if (index == comboBox.getItemCount() ||
        ↪  Double.parseDouble(comboBox.getItemAt(index)) != precision) {
            comboBox.insertItemAt(BigDecimal.valueOf(precision).stripTrailingZeros().
↪  toPlainString(), index);
        }
        comboBox.setSelectedIndex(index);
    }
```

**Cómo arreglarlo.** Dos cosas, que se pueden hacer por separado:

- que `selectPrecision` no pase por `BigDecimal` un número que no es finito: escribirlo con su propio texto («NaN», «Infinity», como hace `Double.toString`) y ponerlo en la lista elegido, para que el usuario vea qué precisión tiene la variable y pueda cambiarla;
- que el lector no acepte una precisión que no sea un número finito mayor o igual que cero, que es lo único que `Variable.round` sabe usar: rechazar el fichero con un mensaje que nombre la variable, o avisar y dejarle la precisión por omisión, como se decida para los demás números mal escritos.

Prueba: en `NodeDomainValuesTablePanelTest`, `selectPrecision` con `Double.NaN` y con `Double.POSITIVE_INFINITY` no lanza y deja una entrada elegida; y, si se toca el lector, leer un `.pgmx` con `<Precision>NaN</Precision>` y comprobar que no da a la variable esa precisión sin avisar.

### Error 265 · [Corregido] En el panel del indicador de un nodo de suceso, la casilla del tiempo hasta el suceso acepta «NaN», «∞» y «1E400», y con cualquiera de ellos el suceso no llega a ocurrir nunca en la simulación, sin ningún aviso

| | |
|---|---|
| **Estado** | Corregido en el commit `c7b9de2` (15-10-2026): la regla entera está en `IndicatorPotential` (`isValidTte`: número finito no menor que 0; `isValidPOccurrence`: número entre 0 y 1, sin dejar pasar NaN), también en el constructor de cuatro argumentos; las casillas rechazan lo que no sea un número finito y vuelven al último valor válido |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.dialog.common; org.openmarkov.core.model.network.potential |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/IndicatorPotentialPanel.java:59-60,86-102,104-113` |

| | |
|---|---|
| | `core/src/main/java/org/openmarkov/core/model/network/potential/IndicatorPotential.java:79-84,155-177` `gui/src/main/java/org/openmarkov/gui/desnets/EventEvaluation.java:120,157` |
| **Comprobado** | Leyendo el código y ejecutándolo (las casillas del panel y la lectura de un fichero, sin ventana); lo que pasa en la simulación está leído en el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 164, error 266 |

**Qué pasa.** Desde `852ede1` (error 164), las dos casillas del panel del indicador —la relación «Indicator» de un nodo de suceso en una red de simulación de sucesos discretos (DES)— son `JSpinner` que solo admiten números válidos. La del tiempo hasta el suceso admite, sin embargo, tres textos que no dan un tiempo, y el panel los guarda sin queja:

```
read  time  NaN    accepted  NaN
read  time  ∞      accepted  Infinity
read  time  1E400  accepted  Infinity
```

«NaN» y «∞» son los símbolos con que `DecimalFormat` escribe en inglés un no-número y el infinito; «1E400» no cabe en un `double` y se lee como infinito. La casilla de la probabilidad rechaza los tres, porque tiene tope.

Con un tiempo NaN o infinito, el suceso no ocurre nunca en la simulación, sea cual sea su probabilidad: `EventEvaluation` solo pone en la cola un suceso cuyo instante no pasa del horizonte (líneas 120 y 157), y ni NaN ni infinito lo cumplen. Además, NaN es justo lo que `IndicatorPotential.sampleConditionedVariable` devuelve cuando el suceso no ocurre (líneas 79-84), así que, con «NaN» escrito, un suceso que ocurre y uno que no se confunden. Esto último está leído en el código; no se ha simulado ninguna red.

Por qué pasa. La casilla del tiempo es un `SpinnerNumberModel(0.0, 0.0, null, 1.0)` (líneas 59-60): desde cero y sin tope. Su formato, `0.################` en modo estricto (`numberSpinner`, líneas 86-92), lee esos tres textos como NaN o infinito. El modelo compara con su mínimo con `compareTo`, para el que un NaN es mayor que cualquier número, así que ninguno de los tres queda por debajo de cero, y no hay tope que los pare. Al guardar, `saveChanges` (líneas 104-113) pasa el valor a `IndicatorPotential.setTte` (líneas 171-177), que solo comprueba `tte < 0`, falso para NaN; `setpOccurrence` (líneas 155-162) deja pasar NaN del mismo modo.

La misma puerta está abierta desde un fichero, y también para la probabilidad: el lector acepta `<TTE>NaN</TTE>`, `<TTE>Infinity</TTE>` y `<ProbabilityOccurrence>NaN</ProbabilityOccurrence>`, el panel los enseña y los guarda sin queja, y el escritor los vuelve a escribir con `String.valueOf`, así que se quedan en el fichero. Con una probabilidad NaN el suceso tampoco ocurre (`randomNumbers[0] <= NaN` es falso).

Medido el 15-10-2026 con las clases de `852ede1` delante del jar del 14 de octubre. Los textos, escribiéndolos en el editor de cada casilla y haciendo `commitEdit`, que es lo que hace el panel al guardar (`valueOf`, líneas 94-102). Lo del fichero, con una red de sucesos de un nodo escrita con `PGMXWriter_1_0`, retocada a mano, leída con `PGMXReader` y guardada con el panel:

```
<TTE>NaN</TTE>                                    casillas «NaN» y «0.25»   guardado: tte =
↪  NaN
<TTE>Infinity</TTE>                               casillas «∞» y «0.25»     guardado: tte =
↪  Infinity
<ProbabilityOccurrence>NaN</ProbabilityOccurrence> casillas «2» y «NaN»      guardado: p =
↪  NaN
```

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/IndicatorPotential.java:171-177

```
    public void setTte(double tte) {
        if (tte < 0) {
            throw new InvalidArgumentException("Time to event  out of range 0..");
        }
        ;
        this.tte = tte;
    }
```

**Cómo arreglarlo.** Que la regla esté entera en `IndicatorPotential`: `setTte` debe rechazar todo lo que no sea un número finito mayor o igual que cero (`!(tte >= 0) || Double.isInfinite(tte)`), y `setpOccurrence` todo lo que no esté en [0, 1] escrito de modo que NaN no pase (`!(pOccurrence >= 0 && pOccurrence <= 1)`). Y que las casillas tampoco los acepten: en `valueOf`, si lo leído no es un número finito, volver al último valor válido, como ya se hace con un texto fuera de rango. Un tiempo infinito no hace falta para decir «no ocurre nunca»: para eso está la probabilidad 0.

Conviene hacerlo a la vez que el error 266, que es la otra puerta por la que entran en esta relación números que no valen.

Prueba: en `AnInvalidValueInTheIndicatorPanelIsNotSavedTest`, escribir «NaN», «∞» y «1E400» en la casilla del tiempo, guardar, y comprobar que el tiempo de la relación sigue siendo el de antes; y que `setTte(Double.NaN)`, `setTte(Double.POSITIVE_INFINITY)` y `setpOccurrence(Double.NaN)` lanzan `InvalidArgumentException`.

### Error 266 · [Corregido] Si un .pgmx trae en el indicador de un suceso un tiempo negativo o una probabilidad fuera de [0, 1], la red se abre sin aviso, el panel enseña esos números y aceptar el diálogo del nodo, sin haber tocado nada, acaba en la ventana de error no previsto

| | |
|---|---|
| **Estado** | Corregido en el commit `109ee86` (15-10-2026), con la opción A elegida por Manuel Arias: el lector comprueba los dos números con la regla de `IndicatorPotential` y, si uno no la cumple (también NaN o un texto que no es un número), no abre el fichero y dice qué nodo y qué número, por ejemplo «The relation of Onset cannot be read: its time to event, -1, must be a finite number not below 0.» |
| **Módulo** | io, core, gui |
| **Paquete** | org.openmarkov.io.probmodel.reader; org.openmarkov.core.model.network.potential; org.openmarkov.gui.dialog.common |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:520-526`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/IndicatorPotential.java:42-46,155-177`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/IndicatorPotentialPanel.java:76-80,94-113`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:314-331,430-437` |
| **Comprobado** | Leyendo el código y ejecutándolo (fichero escrito por el programa y retocado, leído, y el panel construido y guardado sin ventana); lo que se ve en la ventana está leído en el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | Sí: si un fichero con un indicador fuera de rango se rechaza, diciendo qué nodo |

| | |
|---|---|
| | y qué número, o se abre avisando, como se decidió en el error 111 para el fichero que no cumple el esquema, y en ese caso qué valor toma la relación |
| **Relacionados** | error 164, error 111, error 265 |

**Qué pasa.** Un `.pgmx` de una red de sucesos con `<TTE>-1</TTE>` o `<ProbabilityOccurrence>2</ProbabilityOccurrence>` en la relación «Indicator» de un nodo de suceso se abre sin aviso. Al abrir las propiedades del nodo y su pestaña de la relación, las casillas enseñan «-1» y «2», fuera de los límites que ellas mismas tienen. Al aceptar el diálogo, o al cambiar de pestaña, sin haber tocado nada, el panel intenta guardar esos mismos números, la relación los rechaza y sale la ventana de error no previsto, con la pestaña de la relación vacía: lo que la ficha del error 164 describía para un número mal escrito. Solo se llega con un fichero escrito fuera del programa: ni el panel ni los métodos de `IndicatorPotential` dejan poner esos números.

Por qué pasa. `PGMXReader_1_0.getIndicatorPotential` (líneas 520-526) hace la relación con el constructor de cuatro argumentos (`IndicatorPotential`, líneas 42-46), que copia los dos números sin mirarlos; las comprobaciones están solo en `setTte` y `setpOccurrence` (líneas 155-177). El panel pasa los números a sus casillas con `setValue` (`setData`, líneas 76-80), que no mira los límites del modelo. Al guardar, `valueOf` (líneas 94-102) intenta leer lo escrito, el modelo lo rechaza por estar fuera de rango y la casilla se queda con el valor que tiene, que es el de fuera de rango; `saveChanges` (líneas 104-113) se lo pasa a `setTte` o `setpOccurrence`, que lanzan `InvalidArgumentException`. Es una excepción de tiempo de ejecución, y `NodePropertiesDialog.acceptPotentialEditChanges` (líneas 314-331), que quita el panel antes de guardar, no la recoge.

La ficha del error 164 dice que «la regla de rango debe seguir en los métodos de `IndicatorPotential`, que son los que protegen también la lectura de ficheros»; pero la lectura no pasa por ellos. Y no es lo que esa ficha deja presente —que `acceptPotentialEditChanges` quite el panel antes de saber si el guardado ha ido bien—: aquello es qué pasa cuando un panel lanza; esto es que este panel, ya corregido, todavía lanza.

Medido el 15-10-2026 con las clases de `852ede1` delante del jar del 14 de octubre: una red de sucesos de un nodo escrita con `PGMXWriter_1_0` (`<TTE>2.0</TTE>`, `<ProbabilityOccurrence>0.25</ProbabilityOccurrence>`), retocada a mano y leída con `PGMXReader`; el panel, construido sobre el nodo leído y guardado con `saveChanges`, como hace el diálogo al aceptar:

```
<TTE>-1</TTE>                    leída tte=-1.0 p=0.25   casillas «-1» y «0.25»
↪  InvalidArgumentException: Time to event  out of range 0..
<ProbabilityOccurrence>2</...>    leída tte=2.0 p=2.0     casillas «2» y «2»
↪  InvalidArgumentException: Occurrence out of range 0<=pOccurrence <=1
```

No se ha abierto la ventana. Si el usuario no abre el panel, la simulación usa esos números tal cual (leído; no se ha simulado).

**El código.** io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_1_0.java:520-526

```
    protected Potential getIndicatorPotential(Element xmlPotential, ProbNet probNet,
    ↪  PotentialRole role,
                                              List<Variable> variables) {
        Element xmlTte = xmlPotential.getChild(XMLTags.TTE.toString());
        Element xmlPOcurrence = xmlPotential.getChild(XMLTags.P_OCCURRENCE.toString());
        IndicatorPotential indicatorPotential = new IndicatorPotential(variables, role,
↪   Double.parseDouble(xmlTte.getText()), Double.parseDouble(xmlPOcurrence.getText()));
        return indicatorPotential;
    }
```

**Cómo arreglarlo.** Que la lectura pase por la misma regla que el panel: construir la relación con el constructor de dos argumentos y poner los números con `setTte` y `setpOccurrence`, con la regla

ampliada del error 265, que rechaza también NaN e infinitos. La `InvalidArgumentException` que salga se convierte en lo que el equipo decida: un error de lectura que diga qué nodo y qué número están mal, o un aviso al abrir, como el de las columnas que no suman uno, con un valor válido en su lugar. Conviene, además, que el constructor de cuatro argumentos compruebe lo mismo, o quitarlo: solo lo usa el lector.

Prueba: leer un `.pgmx` de una red de sucesos con `<TTE>-1</TTE>` y comprobar que no se obtiene sin aviso una relación con ese tiempo (el lector lo rechaza nombrando el nodo, o avisa y pone uno válido); lo mismo con `<ProbabilityOccurrence>2</ProbabilityOccurrence>`.

### Error 279 · [Corregido] En un modelo temporal, las propiedades de un nodo que tiene gemelo en otro ciclo —como «Age [0]» junto a «Age [1]»— no se pueden aceptar: «OK» da «There is already a variable named Age in the network.» sin que se haya tocado el nombre, y «Cancel» deshace todo lo editado; salir de la casilla del nombre da el mismo error

| | |
|---|---|
| **Estado** | Corregido en el commit `ab11a43` (16-10-2026): `ValidName.nameIsAlreadyPresent` compara el nombre completo que tendría el nodo (nombre base y ciclo, compuesto por `Variable.nameWithBaseName`) con los nombres completos de las demás variables, como `DistinctVariableNames.checkProbNet`; y `focusLost` compara la casilla con el nombre base. Visto en la ventana el 16-10-2026 sobre `MID-HPV.pgmx`: en «Age [0]» y en «Age [1]», «OK» cierra el diálogo, y salir de la casilla del nombre no da error ni deja ningún paso de deshacer; un nombre de otra variable del mismo ciclo se sigue rechazando. Renombrar un nodo con el nombre base de una variable que solo está en otros ciclos ya no se rechaza, como en la comprobación de la red entera. Con prueba; la del diálogo necesita pantalla. |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.dialog.node; org.openmarkov.core.action.core; org.openmarkov.core.model.network.constraint |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:914-928,946-947,984-986`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodePropertiesDialog.java:406-409,430-437`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/OkCancelDialog.java:68-79`<br>`core/src/main/java/org/openmarkov/core/action/core/NodeBaseNameEdit.java:49-68`<br>`core/src/main/java/org/openmarkov/core/model/network/constraint/ValidName.java:34-39`<br>`core/src/main/java/org/openmarkov/core/model/network/Variable.java:400-402,414-417,594-628` |
| **Comprobado** | Leyendo el código y ejecutándolo, también en la ventana: el diálogo de propiedades abierto de verdad sobre `MID-HPV.pgmx` y sobre una red hecha en código, y «OK» pulsado; lo de salir de la casilla del nombre está leído en el código y medido solo en la edición |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 72, error 62, error 38, nota 147 |

**Qué pasa.** En un diagrama de influencia de Markov (MID), y en cualquier otro modelo temporal, casi todos los nodos tienen un gemelo en otro ciclo: «Age [0]» y «Age [1]» son la misma variable en dos instantes y comparten el nombre base «Age». Al abrir las propiedades de uno de ellos (doble clic sobre el nodo, o su menú contextual) y pulsar «OK», aunque no se haya tocado nada, sale el

aviso «There is already a variable named Age in the network.» y el diálogo sigue abierto. La única salida es «Cancel», que deshace todo lo que se hubiera editado en el diálogo (`NodeProperties Dialog.doCancelClickBeforeHide`, líneas 406-409). En la práctica, en un modelo temporal no se puede cambiar ninguna propiedad de esos nodos por este diálogo; solo se salva quien cambia el nombre base por uno nuevo, y entonces el nodo deja de ser gemelo de los demás ciclos.

Por qué pasa. La casilla del nombre enseña el nombre base (`setFieldsFromProperties`, líneas 946-947). Al pulsar «OK», `doOkClickBeforeHide` (líneas 430-437) llama a `checkNameConstraints` (líneas 984-986), que construye un `NodeBaseNameEdit` con el texto de la casilla y le pide `try ConstraintsWillBeMet`. Esa comprobación (`NodeBaseNameEdit.checkConstraintsWillBeMet`, líneas 49-68) pregunta a `ValidName.nameIsAlreadyPresent` (líneas 34-39), que compara el nombre nuevo con los nombres *base* de todas las demás variables: el gemelo del otro ciclo tiene el mismo nombre base, así que el nombre que el nodo ya tiene cuenta como repetido. La excepción llega al botón (`OkCancelDialog`, líneas 68-79), que la envuelve en `UnrecoverableException` y no cierra el diálogo.

La casilla del nombre tiene además un oyente de foco (`focusLost`, líneas 914-928): cuando el foco sale de ella compara el nombre *completo* del nodo, «Age [0]», con el texto de la casilla, «Age», que nunca coinciden en un nodo temporal, y ejecuta el mismo `NodeBaseNameEdit`. Con gemelo, el error anterior sale ya al pulsar en cualquier otra casilla (leído); sin gemelo, la edición se hace, no cambia nada y queda apuntada para deshacer como «Change node name … from B to B» (medido en la edición).

Medido el 15-10-2026 con el código de `3a93386`, abriendo de verdad el diálogo de propiedades (el mismo que abre el editor) y pulsando su «OK»:

```
MID-HPV.pgmx (34 nodos), «Age [0]»     | OK lanza VariableNameIsAlreadyPresent; el diálogo
↪  sigue abierto
Red en código con «A [0]» y «A [1]», «A [0]» | OK lanza VariableNameIsAlreadyPresent; el
↪  diálogo sigue abierto
Red en código con «B [0]» solo              | OK cierra el diálogo sin error
NodeBaseNameEdit(«A [0]», "A").tryConstraintsWillBeMet()  | lanza «Could not met the
↪  constraint: Variables have distinct names»
NodeBaseNameEdit(«A [0]», "B").tryConstraintsWillBeMet()  | pasa
NodeBaseNameEdit(«B [0]», "B").executeEdit()              | se hace, el nombre sigue «B
↪  [0]», queda un paso de deshacer «from B to B»
```

El mensaje que ve el usuario es el de `VariableNameIsAlreadyPresent`: «There is already a variable named Age in the network.». El error 72 corrigió el hueco contrario en `TimeSliceEdit` (el instante se comprobaba con un nombre que no existía); aquí el nombre se comprueba contra los gemelos del propio nodo.

**Cómo arreglarlo.** Dos cosas, en el diálogo y en la regla:

- Que el diálogo no edite ni compruebe un nombre que no ha cambiado: `focusLost` debe comparar el texto con el nombre base (`getBaseName`), que es lo que enseña, y `checkName Constraints` debe saltarse la comprobación cuando el nombre base es el que ya tiene.
- Que la regla no cuente a los gemelos del nodo: `ValidName.nameIsAlreadyPresent` debe comparar el nombre completo que resultará (nombre base más ciclo, como lo compone `Variable.rebuildFullName`) con los nombres completos de las demás variables, que es lo que mira `DistinctVariableNames.checkProbNet` y lo que hace `TimeSliceEdit` desde el error 72.

Queda por decidir, pero no hace falta para esto, si renombrar un nodo temporal debe renombrar también a sus gemelos de los otros ciclos, como la pestaña de dominio propaga los estados, el tipo y la precisión (`NodeDomainValuesTablePanel`).

Prueba: en un MID con «A [0]» y «A [1]», abrir el diálogo de propiedades de «A [0]» y pulsar «OK»: se cierra con `Ok`; `NodeBaseNameEdit`(«A [0]», "A") pasa las restricciones; y al salir de la casilla del nombre sin cambiarlo no queda ningún paso de deshacer.

## 15 Interfaz: menús, diálogos y herramientas

Los menús, los diálogos generales y las herramientas del menú Tools.

### Error 171 · [Jorge Rico] Abrir una red desde una dirección de internet apaga «Guardar» para todas las redes durante el resto de la sesión, y a la vez deja esa red apuntando a un fichero local inventado

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:189-196,213,229-232,256-264,378-392,401-412`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:103,291-298,426,866-870`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanel.java:556-564` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 177 |

**Qué pasa.** Menú Archivo → «Open network from URL…» (`ActionCommands.OPEN_NETWORK_URL`, atajo Ctrl+Alt+O) abre una red escribiendo su dirección de internet (URL), por ejemplo `https://www.cisiad.uned.es/redes/asia.pgmx`. El código está en `NetworkFileHandler.openNetworkSource` (módulo `gui`). Hay dos defectos enlazados.

**1. La red abierta por URL recibe como fichero local el camino de la dirección.** Para una dirección, `networkFile` es `url.getFile()` (línea 189), es decir `/redes/asia.pgmx`, y se pasa a `createNewFrame(netReadFrom, networkFile)`, que lo guarda con `setNetworkFile`. El lector devuelve además un escritor (`ProbNetInfo.writer()`, no nulo para `.pgmx` y Elvira).

Con eso, `saveNetwork` (línea 250) considera que la red ya tiene fichero y formato: intenta primero una copia de seguridad en `/redes/asia.bak` (si falla, solo escribe un aviso por consola) y después escribe la red en `/redes/asia.pgmx`, en la raíz del disco local. Si esa carpeta no existe, el guardado falla con un error; si existe y se puede escribir, se sobrescribe lo que haya. Hay caminos que llegan a `saveNetwork` sin pasar por el botón «Guardar» del menú:

- el menú contextual de la pestaña (clic derecho → «Save», `MainPanel`, línea 557), que no mira la bandera del punto 2;
- cerrar la pestaña con cambios y contestar «Sí» a guardar (`networkCanBeClosed`, línea 384);
- «Guardar», cuando vuelve a estar encendido al cambiar de pestaña (ver punto 2).

Además, al salir con la opción de reabrir la última sesión, `closeApplication` guarda `/redes/asia.pgmx` en la lista de ficheros de la sesión, y el siguiente arranque intenta abrirlo como fichero local.

El filtro de «la red ya está abierta» de `openNetworkSource` compara `networkFile.equals(editor.getNetworkFile())`, que ya tolera redes sin fichero, y la pregunta de recargar solo aparece si alguna copia está modificada; si no lo está, la pestaña anterior se quita sin preguntar (commit

`a5da86c`). Por el punto 1, dos direcciones distintas con el mismo camino (`http://a/redes/asia.pgmx` y `http://b/redes/asia.pgmx`) se consideran la misma red, y abrir la segunda cierra la primera sin preguntar si no tenía cambios.

**2. Para tapar lo anterior se apaga «Guardar», pero la bandera es de toda la ventana y no se baja nunca.** Tras abrir por URL se llama a `updateOptionsNetworkOpenedURL(true)` (línea 225), que pone `networkOpenedURL = true` en `MainPanelMenuAssistant` (el ayudante que enciende y apaga opciones de menú; hay uno para toda la ventana principal) y apaga «Guardar» y «Guardar y reabrir». No existe ninguna llamada con `false` en todo el proyecto.

Cada vez que se hace, deshace o rehace una edición en **cualquier** red abierta, `updateOptionsNetworkModified` (líneas 279-286) vuelve a aplicar `setOptionEnabled(SAVE_NETWORK, !networkOpenedURL)`, es decir, apaga «Guardar». Resultado: desde que se abre una red por URL, en ninguna red (ni las abiertas después desde disco, ni las nuevas) se puede guardar con el menú, el botón de la barra o Ctrl+S justo después de modificarla, que es cuando se quiere guardar.

Al cambiar de pestaña, `updateOptionsNetworkDependent` (línea 415) enciende «Guardar» según `getModified()` de la red seleccionada, sin mirar la bandera, y la siguiente edición lo vuelve a apagar. «Guardar como» no está afectado. «Guardar y reabrir» vuelve a encenderse al abrir o crear otra red (`updateOptionsNewNetworkOpen` enciende `FILING_ACTION_COMMANDS`).

Todo esto sale de la lectura del código; no se ha ejecutado.

**El código.** gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:197-200, 217, 233-236

```
String networkFile = switch (source) {
    case NetworkSource.SourceFile sourceFile -> sourceFile.file.getAbsolutePath();
    case NetworkSource.SourceURL sourceURL -> sourceURL.url.getFile();
};
...
NetworkEditorPanel networkPanel = this.createNewFrame(netReadFrom, networkFile);
...
case NetworkSource.SourceURL sourceURL -> {
    //LastOpenFiles.setLastFileName(sourceURL.url.getFile());
    mainPanel.getMainPanelMenuAssistant().updateOptionsNetworkOpenedURL(true);
}
```

gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:292-299, 872-876

```
public void updateOptionsNetworkModified(boolean canUndo, boolean canRedo) {
    ...
    // If the network has been opened from a URL the save button has to remain disabled
    setOptionEnabled(ActionCommands.SAVE_NETWORK, !networkOpenedURL);
}
...
public void updateOptionsNetworkOpenedURL(boolean networkOpenedURL) {
    this.networkOpenedURL = networkOpenedURL;
    setOptionEnabled(ActionCommands.SAVE_NETWORK, !networkOpenedURL);
    setOptionEnabled(ActionCommands.SAVE_OPEN_NETWORK, !networkOpenedURL);
}
```

**Cómo arreglarlo.** Una red traída de una dirección no tiene fichero local: `createNewFrame` debe recibir `null` como fichero para ella (el nombre de la red puede seguir saliendo del último tramo de la dirección, como hoy en la línea 185). Con eso `saveNetwork` cae en «Guardar como» y pregunta dónde, igual que con una red nueva, por cualquiera de los caminos (menú, barra, Ctrl+S, menú contextual de la pestaña, cerrar con cambios). Después se puede borrar la bandera `networkOpenedURL`, `updateOptionsNetworkOpenedURL` y la línea de `updateOptionsNetworkModified` que

la aplica.

Sitios donde la misma regla («una red sin fichero no se guarda en un camino inventado») tiene que cumplirse:

- `NetworkFileHandler.saveNetwork`;
- `saveOpenNetwork` (líneas 283-292, que usa `getNetworkFile()` para reabrir);
- `closeApplication`, que no debe guardar en la lista de la sesión un fichero nulo o inventado;
- el filtro de «la red ya está abierta» de `openNetworkSource`, que debería comparar la dirección completa para las redes traídas por URL (o no comparar).

Si algún día se separa de la ventana un documento de red con su fichero (ver nota 177, una propuesta de diseño sin nada que corregir), esta distinción cae ahí de forma natural.

Prueba: abrir una red por URL (se puede servir un `.pgmx` con un servidor local o un `file:` URL si el lector lo admite), comprobar que `getNetworkFile()` es nulo, que tras modificar otra red abierta desde disco «Guardar» está encendido, y que guardar la red traída pide un destino.

### Error 172 · La herramienta «Edits history» deja un núcleo del procesador al cien por cien hasta que se cierra el programa, y sus botones de deshacer y rehacer pueden colgar la aplicación

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | full |
| **Paquete** | org.openmarkov.full |
| **Ficheros** | `full/src/main/java/org/openmarkov/full/EditsHistoryPlugin.java:36-40`<br>`full/src/main/java/org/openmarkov/full/EditsHistoryPlugin.java:42-143`<br>`core/src/main/java/org/openmarkov/core/action/base/PNESupport.java:104-116`<br>`core/src/main/java/org/openmarkov/core/action/base/PNESupport.java:142-154` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Si esta herramienta, que su documentación llama «de desarrollo», debe aparecer en el menú Herramientas para todos los usuarios |

**Qué pasa.** `EditsHistoryPlugin` (módulo `full`) añade al menú Herramientas la entrada «Edits history»: abre una ventana no modal con un botón por cada edición del historial de la red activa (hechas y deshechas); pulsar uno deshace o rehace hasta esa edición. Su javadoc la llama herramienta de desarrollo, pero `ToolPluginManager` la carga como cualquier otro complemento y aparece para todos.

**1. Espera activa sin fin.** Desde que se abre la herramienta, un núcleo del procesador queda ocupado entero hasta que se sale del programa, aunque se cierre la ventana; cada vez que se vuelve a abrir, se ocupa otro núcleo más.

`action()` (líneas 42-143) lanza un `new Thread(() -> { while (true) { ... } })`. En cada vuelta pide la red activa, sus listas de ediciones hechas y deshechas, las compara con las de la vuelta anterior y, solo si cambian, rehace los botones. No hay ninguna espera entre vueltas (ni `sleep`, ni bloqueo, ni aviso por oyente). Tampoco comprueba si la ventana sigue abierta: al cerrarla (se oculta) el hilo continúa, y cada apertura lanza un hilo nuevo. Si no hay red abierta, la excepción se recoge y se vuelve a empezar igual de deprisa. Además, ese hilo modifica componentes Swing (quita y añade botones, `pack()`) fuera del hilo de la interfaz, lo que Swing no admite.

**2. Botones que pueden no terminar.** Si el botón pulsado ha quedado desfasado respecto al historial, la interfaz se queda congelada y hay que matar el programa; antes de colgarse, además, deshace todo el historial de esa red.

Cada botón de una edición hecha ejecuta, en el hilo de la interfaz, `while (!currentProbNet.getPNESupport().undo().contains(editAndDone.edit)) { }` (líneas 88-92), y los de ediciones deshechas lo mismo con `redo()` (líneas 110-114). `PNESupport.undo()` devuelve una lista vacía cuando ya no queda nada que deshacer. Si la edición del botón no está en el historial actual —porque el historial cambió y el botón es de la lista anterior (el refresco lo hace otro hilo sin coordinación con los clics), o porque la red capturada en `currentProbNet` ya no es la del historial que se muestra—, el bucle pide deshacer una y otra vez, recibe listas vacías y no sale nunca.

No se ha ejecutado (la ventana es un `JDialog` y no se puede construir sin pantalla). El consumo de procesador del punto 1 se sigue directamente del código; el cuelgue del punto 2 necesita que el botón quede desfasado respecto al historial.

**El código.** full/src/main/java/org/openmarkov/full/EditsHistoryPlugin.java:56-60, 74, 86-92

```
new Thread(() -> {
    while (true) {
        ProbNet currentProbNet = MainPanel.getCurrentProbNet();
        try {
            ...
            if (!newEdits.equals(edits)) {
                ...
                if (editAndDone.done) {
                    editButton.setIcon(IconBind.UNDO_ENABLED.icon());
                    editButton.addActionListener(e -> {
                        while (!currentProbNet.getPNESupport().undo().contains(editAndDone
                        ↪  .edit)) {

                        }
                    });
```

**Cómo arreglarlo.** Antes de arreglarla hay que decidir si la herramienta debe estar en el menú de todos los usuarios o solo en un modo de desarrollo.

Refresco: sustituir el hilo por un oyente de ediciones (`PNEditListener` registrado en el `PNESupport` de la red, más un oyente de cambio de pestaña para la red activa) que rehaga los botones en el hilo de la interfaz (`SwingUtilities.invokeLater`), y darlo de baja al cerrar la ventana. Si se prefiere un sondeo, un `javax.swing.Timer` con un intervalo y parado al cerrar.

Botones: antes de tocar el historial, buscar la posición de la edición en `getDoneEdits()` (o `getUndoneEdits()`) de la red actual; si no está, no hacer nada (o avisar); si está, llamar a `undo()`/`redo()` exactamente ese número de veces. Así el bucle termina siempre.

Prueba: sin ventana, extraer a un método el cálculo de «cuántos pasos hasta la edición X» sobre un `PNESupport` con tres ediciones y comprobar que deshace hasta la segunda en dos pasos y que con una edición que no está en el historial no hace nada.

### Error 173 · [Jorge Rico] Aceptar el diálogo de dominios estándar sin marcar ninguno lanza una excepción inesperada

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |

| | |
|---|---|
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/StandardDomainPanel.java:33-62`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/StandardDomainsDialog.java:56-76`<br>`gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:1070-1112`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/OkCancelDialog.java:67-80` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 24, error 174, error 180 |

**Qué pasa.** En el diálogo de propiedades de un nodo con variable de estados finitos o discretizada, la pestaña de dominio tiene el botón «Standard domains», que abre una ventana con un botón de radio por cada dominio. Ninguno sale marcado, y el botón «OK» está activo desde el principio (y es el botón por defecto, así que también responde a la tecla Intro).

Si el usuario pulsa «OK» sin marcar nada, sale el diálogo de «excepción inesperada» que pide enviarla a los desarrolladores. La red no cambia y el diálogo del nodo sigue abierto. Cerrar la ventana con la X equivale a «Cancel» y no falla.

`NodeDomainValuesTablePanel.actionPerformedStandardDomain` abre `StandardDomainsDialog`, cuyo `StandardDomainPanel` pone un botón de radio por cada dominio que ofrece `GUIDefaultStates.getAllDomains()` (los dominios propios del usuario en preferencias, seguidos de los seis de `StandardDomain`). `chosenStates` solo se asigna en el oyente de cada botón, así que vale `null` hasta que el usuario marca uno.

Al pulsar «OK» sin marcar nada, `requestValues()` devuelve `Ok` y `actionPerformedStandardDomain` llama a `getChosenStates()`, que ejecuta `Collections.unmodifiableList(null)` y lanza `NullPointerException`. Como no es `DoEditException`, `actionPerformed` no la captura y llega al gestor global.

Medido con un pequeño programa de prueba sobre el panel real: los seis botones (`absent - present`, `no - yes`, `negative - positive`, `absent - mild - moderate - severe`, `low - medium - high`, `not happened - happened`) salen con `selected=false`, y `getChosenStates()` lanza `NullPointerException: Cannot invoke "java.util.List.getClass()" because "list" is null`; tras marcar el segundo devuelve `[no, yes]`.

El diálogo hermano de criterios estándar de la red falla igual al aceptar sin marcar nada, y además borra antes los criterios: está en error 174.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/StandardDomainPanel.java:43-62

```
for (List<String> defaultState : states) {
    JRadioButton radioButton = new JRadioButton(GUIDefaultStates.getString(defaultState));
    radioButtons.add(radioButton);
    buttonGroup.add(radioButton);
    radioButton.addItemListener(e -> chosenStates = defaultState);
    add(radioButton, BorderLayout.CENTER);
}
...
public List<String> getChosenStates() {
    return Collections.unmodifiableList(this.chosenStates);
}
```

```
private List<String> chosenStates;
```

gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDomainValuesTablePanel.java:1088-1093

```
if (standardDomainDialog.requestValues() != OkCancelDialog.ChosenOption.Ok) {
    return;
}
var chosenStates = standardDomainDialog.getJPanelStandardDomains().getChosenStates();

State[] newStates = chosenStates.stream().map(State::new).toArray(State[]::new);
```

**Cómo arreglarlo.** Dos opciones equivalentes:

- desactivar «OK» hasta que se marque un botón (el oyente de cada botón lo activa);
- hacer que `StandardDomainsDialog.doOkClickBeforeHide` devuelva `false` si no hay nada marcado, para que el diálogo no se cierre.

Conviene que el oyente asigne `chosenStates` solo cuando el botón pasa a seleccionado (hoy el `ItemListener` también se dispara al deseleccionar, aunque asigna su propio dominio y el siguiente lo corrige).

Si se cambia el panel, conviene aplicar la misma regla a `StandardCriteriaDialog`/`StandardCriteriaPanel`, que ofrecen la misma elección con el mismo defecto (error 174).

Prueba de regresión: construir `StandardDomainsDialog` sin mostrarlo, comprobar que «OK» está desactivado (o que `doOkClickBeforeHide` devuelve `false`) mientras no hay selección, y que tras marcar un botón `getChosenStates()` devuelve sus estados.

### Error 174 · [Jorge Rico] Aceptar «Standard criteria» sin marcar ninguna opción borra los criterios de decisión de la red, sale como error inesperado y la red deja de marcarse como modificada

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.network |
| **Ficheros** | gui/src/main/java/org/openmarkov/gui/dialog/network/StandardCriteriaDialog.java:82-112<br>gui/src/main/java/org/openmarkov/gui/dialog/network/StandardCriteriaDialog.java:126-136<br>gui/src/main/java/org/openmarkov/gui/dialog/network/StandardCriteriaPanel.java:38-57<br>gui/src/main/java/org/openmarkov/gui/dialog/network/DecisionCriteriaTablePanel.java:178-195<br>gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:81-85<br>gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:270-283<br>gui/src/main/java/org/openmarkov/gui/dialog/common/OkCancelDialog.java:67-78<br>core/src/main/java/org/openmarkov/core/action/base/EditsHistoryStacker.java:48-60<br>core/src/main/java/org/openmarkov/core/action/base/PNESupport.ja |

| | |
|---|---|
| | `va:286-295` `gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:428` `gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:382-384` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 35, error 59, error 173, error 180 |

**Qué pasa.** En las propiedades de una red que no es solo de azar (un diagrama de influencia, una red de análisis de decisiones…), la pestaña de criterios de decisión tiene el botón «Standard criteria». Abre una ventana con seis botones de radio: «Cost-benefit» en €, £ o $, y «Cost-effectiveness» en €/QALY, £/QALY o $/QALY (QALY: años de vida ajustados por calidad). Ninguno sale marcado y «OK» está activo.

Si el usuario pulsa «OK» sin marcar nada, sale el diálogo de error inesperado, pero antes de fallar el diálogo ya ha quitado todos los criterios de la red. Lo que pasa después depende de lo que haga el usuario:

- La ventana de criterios estándar sigue abierta, porque la excepción impide cerrarla, y la tabla de la pestaña sigue enseñando los criterios viejos, porque solo se refresca cuando esa ventana termina con «OK» (por lectura).
- Si cierra esa ventana y acepta las propiedades de la red, la red se queda sin ningún criterio. Si se guarda y se reabre, el lector le pone el criterio por omisión «—» y sus nodos de utilidad se quedan sin criterio. Lo que ocurre al evaluar una red sin criterios está en error 59.
- Si cancela las propiedades de la red, los criterios vuelven.
- En los dos casos la red se queda, para el resto de la sesión, con una sub-historia de ediciones abierta que nadie cierra, y desde ese momento el indicador de «red modificada» deja de actualizarse. Si la red no estaba modificada, ninguna edición posterior la marca: «Save» sigue apagado (`MainPanelMenuAssistant`, línea 415) y cerrar la red no pregunta si se quiere guardar (`NetworkFileHandler.networkCanBeClosed`, líneas 372-374), así que el trabajo posterior se puede perder sin aviso (por lectura; el indicador congelado sí está medido).
- Si, tras la excepción, marca una opción y vuelve a pulsar «OK», se añaden los criterios elegidos, pero la sub-historia del primer intento sigue abierta.

Por qué pasa: `StandardCriteriaDialog.doOkClickBeforeHide` (módulo `gui`) abre una sub-historia, quita todos los criterios con `DecisionCriteriaEdit(REMOVE)` y solo después llama a `getDefaultCriteria`, que busca el botón marcado. Si no hay ninguno, `selectedDefaultCriteria` vale `null` y `selectedDefaultCriteria.equals(...)` (línea 136) lanza `NullPointerException`. `OkCancelDialog` vuelve a lanzar las excepciones de tiempo de ejecución, así que la ventana no se cierra y la excepción llega al manejador global. `closeSubEditHistory` (línea 109) no se ejecuta: la sub-historia queda encima de la que abrió el diálogo de propiedades, y al aceptar o cancelar este se cierra la de dentro y la suya queda abierta. `PNESupport.onEditChanges` no recalcula el indicador de modificada mientras la historia activa no sea la principal.

Medido invocando el `doOkClickBeforeHide` real con un `StandardCriteriaPanel` real y sin construir la ventana (un `JDialog` no se puede construir sin pantalla), sobre `0_miscellaneous/networks/id/ID-arthronet-ce.pgmx` (criterios «coste» y «efectividad») y con la sub-historia que abre el diálogo de propiedades:

- los seis botones salen sin marcar; el método lanza `NullPointerException: Cannot invoke "String.equals(Object)" because "selectedDefaultCriteria" is null` en la línea 136,

y la red queda con la lista de criterios vacía y dos sub-historias abiertas;
- tras aceptar las propiedades (`closeSubEditHistory`): sin criterios, una sub-historia abierta y la red sin marcar como modificada; el fichero guardado no lleva `<DecisionCriteria>` y al reabrirlo la red tiene el criterio «—» y sus nodos de utilidad ninguno;
- tras cancelar las propiedades (`cancelLastSubEditHistory`): vuelven «coste» y «efectividad», y sigue una sub-historia abierta;
- con esa sub-historia abierta, añadir después un criterio deja la red sin marcar como modificada; el mismo cambio con el historial bien cerrado la marca.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/network/StandardCriteriaDialog.java:82-109

```
@Override protected boolean doOkClickBeforeHide() throws DoEditException {
    // This must be a single operation
    probNet.getPNESupport().openNewSubEditHistory();
    // Remove all criterion in the probNet with edits
    while (!probNet.getDecisionCriteria().isEmpty()) {
        Criterion criterionToBeDeleted = probNet.getDecisionCriteria().get(0);
        DecisionCriteriaEdit edit = new DecisionCriteriaEdit(probNet, StateAction.REMOVE,
↪   criterionToBeDeleted, null);
        edit.executeEdit();
    }
    //Get the list with the new criteria
    List<Criterion> defaultCriteria = getDefaultCriteria();
    ...
    probNet.getPNESupport().closeSubEditHistory();
```

gui/src/main/java/org/openmarkov/gui/dialog/network/StandardCriteriaDialog.java:129-136

```
String selectedDefaultCriteria = null;
for (JRadioButton radioButton : standardCriteriaPanel.getRadioButtons()) {
    if (radioButton.isSelected()) {
        selectedDefaultCriteria = radioButton.getText();
    }
}

if (selectedDefaultCriteria.equals(stringDatabase.getString(
↪   "defaultCriteria.costBenefitEuros.Text"))) {
```

**Cómo arreglarlo.** Comprobar que hay una opción marcada antes de tocar la red. Dos maneras, equivalentes para el usuario:

- desactivar «OK» hasta que se marque un botón;
- que `doOkClickBeforeHide` devuelva `false`, sin abrir la sub-historia, si no hay nada marcado.

Además, conviene que la sub-historia que abre este método se cierre también si algo falla entre `openNewSubEditHistory` y `closeSubEditHistory` (con `try/finally`, cancelándola), para que un fallo del diálogo no deje la red a medias ni congele el indicador de modificada.

El diálogo hermano de dominios estándar tiene el mismo problema de selección (error 173); conviene resolver los dos con la misma regla. Los dos se abren además sin título (error 180).

Prueba de regresión: con el panel sin marcar, aceptar no debe cambiar la lista de criterios, y tras aceptar o cancelar las propiedades de la red la historia activa debe ser la principal; marcando «Cost-effectiveness (€/QALY)», la red debe quedar con los criterios «Cost» y «Effectiveness».

### Error 175 · «OpenMarkov configuration» enseña y edita un almacén de preferencias que el programa ya no usa

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.configuration |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/configuration/PreferencesDialog.java:95-103,161-196,302-384,424`<br>`gui/src/main/java/org/openmarkov/gui/dialog/configuration/PreferencesTableModel.java`<br>`gui/src/main/java/org/openmarkov/gui/dialog/configuration/PreferenceTreeNode.java`<br>`gui/src/main/java/org/openmarkov/gui/configuration/UserPreference.java:70-79`<br>`gui/src/main/java/org/openmarkov/gui/configuration/UserPreferenceResolveStrategy.java:10-14,128-138`<br>`gui/src/main/java/org/openmarkov/gui/window/EditAndViewHandler.java:106-108`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/MainMenu.java:250,374`<br>`gui/src/main/java/org/openmarkov/gui/window/settings/SettingsDialog.java` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Si la ventana antigua se retira del menú Tools (el diálogo «Settings…» del menú Edit ya hace copia, restauración y valores por omisión sobre el almacén bueno) o se rehace sobre `UserPreferences` |
| **Relacionados** | nota 68, nota 90, nota 98, nota 162 |

**Qué pasa.** El menú Tools termina en «OpenMarkov configuration» (`MainMenu`, líneas 250 y 374), que abre `PreferencesDialog` (`EditAndViewHandler.showUserConfigurationDialog`, líneas 106-108): un árbol de familias de preferencias a la izquierda, una tabla clave/valor a la derecha y cinco botones (Save, Reset user preferences, Export preferences, Import preferences, Cancel).

**Mira donde no están las preferencias.** Lo que enseña la ventana depende de la máquina:

- en una instalación nueva, el árbol sale con la raíz y nada debajo, y la tabla vacía;
- en una máquina donde corrió una versión antigua de OpenMarkov, sale lo que aquella dejó y ya nadie lee. Comprobado por lectura del disco en la máquina de desarrollo: `~/.java/.userPrefs/OPENMARKOV` tiene las familias `colors`, `directories`, `formats`, `languages` y `positions`, con fechas de 2025, mientras las preferencias vigentes están en `~/.openmarkov/preferences/{directories,formats,network,user_interface}`.

En los dos casos, editar un valor en la tabla no cambia nada del programa. «Export preferences» (líneas 346-356) escribe ese almacén viejo a un fichero XML, e «Import preferences» (líneas 361-373) lo mete de vuelta en él, donde nadie lo lee. «Reset user preferences» (líneas 378-384) es el único botón que actúa sobre las preferencias reales (llama a `clear` e `initialize` de todas las `UserPreferences`), y lo hace sin preguntar y sin que la tabla lo refleje.

Las preferencias del programa (`UserPreferences`: últimos ficheros, directorios, tema, escala, dominios propios, acciones de arranque…) se guardan con la primera estrategia de `UserPreferenceResolveStrategy` que sea capaz de escribir y borrar un valor de prueba (`UserPreference`, líneas 70-79). El orden es `USER_FOLDER` (ficheros bajo `~/.openmarkov/preferences`), `INSTALLED_LOCATION`, `BACKING_STORE` (la API `java.util.prefs`) y `SESSION`. En cualquier máquina con un directorio personal escribible gana la primera, así que `java.util.prefs` no se usa. El diálogo, en cambio,

construye su árbol sobre `Preferences.userRoot().node("OPENMARKOV")` (líneas 191-196), es decir, sobre `java.util.prefs`.

**Cancelar no cancela.** Lo escrito en la tabla se guarda al teclearlo (`PreferencesTableModel.setValueAt` hace `put` y `sync`). El «deshacer» que llama Cancelar (líneas 325-330) crea dos modelos nuevos sobre las raíces del sistema y del usuario y llama a `undo()`, que asigna a `pref` el campo `prefSaved`, que es la misma referencia (`this.prefSaved = pref` en el constructor): no restaura nada.

**Otros restos en la misma ventana.**

- `createTree` quita siempre el primer hijo del nodo raíz (líneas 179-180, comentario «not shown color preferences in tree»), suponiendo que es la familia de colores; el orden de `childrenNames()` no está garantizado, así que en una máquina con familias antiguas oculta una cualquiera.
- `ColorTableCellEditor` y `ColorTableCellRenderer` (144 líneas entre los dos) no los usa ninguna clase.
- «Export» e «Import» escriben «Export selected» / «Import selected» por la salida estándar (líneas 349 y 362).
- Los textos de la ventana están escritos en el código, pero eso no forma parte de este error.

Existe otro diálogo, `SettingsDialog` (menú Edit → «Settings…»), que sí trabaja sobre `User Preferences`: escala, tema, acciones de arranque, dominios propios, crear y restaurar una copia de la configuración y volver a los valores por omisión. La ventana antigua de Tools duplica esas funciones sobre el almacén equivocado.

No se ha abierto la ventana (es un `JDialog`); lo anterior es lectura del código y del disco.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/configuration/PreferencesDialog.java:191-196,325-330

```
private MutableTreeNode createUserRootNode(Object obj) throws BackingStoreException {
    if (obj == null) {
        return new PreferenceTreeNode(Preferences.userRoot());
    }
    return new PreferenceTreeNode(Preferences.userRoot().node((String) obj));   //
    ↪  "OPENMARKOV"
}
...
protected void actionPerformedCancel() throws BackingStoreException {
    new PreferencesTableModel(Preferences.systemRoot()).undo();
    new PreferencesTableModel(Preferences.userRoot()).undo();
    this.setVisible(false);
    this.dispose();
}
```

gui/src/main/java/org/openmarkov/gui/configuration/UserPreferenceResolveStrategy.java:10-14

```
enum UserPreferenceResolveStrategy {
    USER_FOLDER,
    INSTALLED_LOCATION,
    BACKING_STORE,
    SESSION;
```

**Cómo arreglarlo.** Exige una decisión de producto, con dos salidas:

1. Retirar la entrada «OpenMarkov configuration» del menú Tools (`MainMenu`, líneas 250 y 374, y `ActionCommands.CONFIGURATION` en `MainPanelListenerAssistant`), junto con `Preferences`

`Dialog`, `PreferencesTableModel`, `PreferenceTreeNode` y los dos controles de color sin uso, porque `SettingsDialog` ya cubre la función sobre el almacén real.

2. Rehacerla sobre `UserPreferences.getAllPreferences()` (el almacén que resuelve `UserPreference`), con un Cancelar que restaure los valores leídos al abrir, o sin Cancelar si los cambios se aplican al momento; la exportación e importación deberían usar el mismo formato que la copia de `SettingsDialog`.

En cualquiera de las dos, conviene que «Reset user preferences» pida confirmación, porque borra la lista de ficheros recientes y los dominios propios.

Prueba: con `UserPreference.IGNORE_STORAGE` o un directorio personal temporal, comprobar que lo que enseña la ventana coincide con `UserPreferences.getAllPreferences()` y que Cancelar deja los valores como estaban.

### Error 176 · [Corregido] El diálogo de opciones de inferencia guarda la escala de cada criterio redondeada a tres decimales, aunque solo se toque la celda: −1/30 000 queda en 0 y la evaluación deja de contar los costes

| | |
|---|---|
| **Estado** | Corregido en el commit `b6e5ece` (14-10-2026): `InferenceOptionsDialog.tableChanged` guarda el número tal como se escribe en los cuatro sitios (escala y descuento, en el análisis de un criterio y en el de coste-efectividad). Medido en el diálogo real con `ID-CEA-test-2therapies`: sin el arreglo, escribir $-3{,}33 \cdot 10^{-5}$ en la escala del coste, o entrar y salir de la celda cuando ya valía eso, dejaba −0,0 en la red, aunque la celda enseñaba el número entero. Con prueba que abre el diálogo, escribe en la celda y pulsa OK, en siete casos que cubren los cuatro sitios (los descuentos con `MID-Chancellor`); sin el arreglo fallan los siete. Necesita pantalla. Lo de rechazar con un mensaje un número que no se puede leer no se ha tocado. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.inference.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:606-620`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:642-651`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:682-691`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:725-728`<br>`core/src/main/java/org/openmarkov/core/model/network/UtilityOperations.java:67-86` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 11, error 12 |

**Qué pasa.** En una red con más de un criterio, el diálogo de opciones de inferencia (el que aparece al entrar en modo inferencia, al pedir el análisis de coste-efectividad, el de sensibilidad o el árbol de decisión, y desde el menú de opciones de inferencia) enseña una tabla con la escala de cada criterio. Lo que el usuario escribe en una escala se guarda redondeado a tres decimales: una escala menor que 0,0005 en valor absoluto queda en 0, y las cercanas pierden precisión (−0,0006 pasa a −0,001). Pasa también sin cambiar nada: basta con hacer clic en la celda y salir de ella, así que una escala más fina que venga del fichero se pierde en cuanto se toca.

Es un uso corriente. Para expresar el resultado en años de vida ajustados por calidad, la escala

del coste es −1 dividido por la disposición a pagar: con 30 000 por año, −0,0000333. Las redes del repositorio usan la convención contraria (coste −1, efectividad 30 000), que no se ve afectada. Al salir de la celda, la tabla se vuelve a construir y enseña la escala redondeada (−0.0), sin otro aviso.

La consecuencia es un resultado distinto sin aviso: con la escala del coste en 0, la evaluación de un solo criterio (la del modo inferencia, por ejemplo) ignora los costes. En el análisis de coste-efectividad, una escala 0 hace que `UtilityOperations.applyCEUtilityScaling` quite de la red los nodos de ese criterio (líneas 81-84, por lectura).

Por qué pasa: en la tabla de `InferenceOptionsDialog`, `tableChanged` toma el texto de la escala, lo formatea con `DecimalFormat` y el patrón `#.###` y guarda en la copia del criterio el número que resulta (líneas 647-650 para la escala de coste-efectividad y 687-690 para la de un solo criterio). Al terminar de editar una celda, `JTable.editingStopped` llama siempre a `setValueAt` con el texto del editor, aunque no haya cambiado, y eso dispara `tableChanged`. Las columnas de descuento usan el mismo patrón sobre el porcentaje (líneas 660-663 y 700-703), que ahí solo pierde cifras por debajo de 0,001 %.

Medido construyendo la tabla del diálogo sin ventana con `0_miscellaneous/networks/id/ID-CEA-test-2therapies.pgmx` (criterios «cost» y «effectiveness») y escribiendo en la escala de «cost»:

```
escrito -0.0000333                                  -> escala guardada -0.0
escrito -3.3333333333333335E-5                      -> -0.0
escrito -0.0005                                     -> -0.001
escrito -0.0006                                     -> -0.001
escrito 0.0004                                      -> 0.0
escrito -1                                          -> -1.0
escala ya en -3.33E-5, se sale de la celda sin tocar -> -0.0
escala de coste-efectividad, escrito 0.000001       -> 0.0
```

Con la misma red y la escala de la efectividad en 1, la evaluación da utilidad 8,9856 con la escala del coste en −1/30 000, y con prueba positiva recomienda «therapy 1»; con la escala redondeada (−0,0), 9,3937, y con prueba positiva recomienda «therapy 2», la de 70 000, y sin prueba «therapy 1» en lugar de no tratar.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:682-691

```java
if (column == UNICRITERIA_SCALE_COLUMN) {
    String scale = table.getValueAt(row, UNICRITERIA_SCALE_COLUMN).toString();
    if (scale.indexOf(' ') != -1) {
        scale = scale.substring(0, scale.indexOf(' '));
    }
    DecimalFormat format = (DecimalFormat) NumberFormat.getInstance(Locale.ENGLISH);
    format.applyLocalizedPattern("#.###");
    scale = format.format(Double.parseDouble(scale));
    decisionCriteria.get(row - 1).setUnicriterizationScale(Double.parseDouble(scale));
}
```

**Cómo arreglarlo.** Hecho como dice la fila *Estado*. Guardar el número tal como se escribe (`Double.parseDouble` del texto, sin pasar por `DecimalFormat`) y redondear, si se quiere, solo al enseñarlo, en el dibujante de la celda. Los cuatro sitios del mismo método deben cambiar a la vez: las dos escalas (líneas 642-651 y 682-691) y los dos descuentos (líneas 654-665 y 694-705).

Conviene también que un número que no se puede leer se rechace con un mensaje en la celda, en vez de la `NumberFormatException` que hoy lanza `setValueAt` (línea 613).

Prueba de regresión: sin ventana, con la tabla del diálogo o con el cálculo extraído a un método,

escribir `-3.3333333333333335E-5` en la escala de un criterio y comprobar que se guarda ese número; y que salir de la celda sin cambiarla no modifica la escala.

### Error 201 · [Corregido] El botón que pasa a modo inferencia está apagado en toda red que no sea un diagrama de influencia o una red bayesiana

| | |
|---|---|
| **Estado** | Corregido (16-10-2026): el modo inferencia se ofrece a los MID y a las redes de análisis de decisiones, como decidió Manuel Arias el 09-10-2026, y la última parte que quedaba, el tiempo de `MID-Cochlear`, la corrige el commit `271c669` (error 231). Medido en la ventana el 16-10-2026 con `MID-Cochlear`: desde «OK» en las opciones de inferencia hasta quedar en modo inferencia pasan 7,4 segundos, y 59 con el código de antes de ese commit; Manuel Arias lo comprobó en la ventana ese día. `MID-Colorectal`, con un hallazgo en «Age [0]», propaga en unos 30 segundos. Antes: desde el commit `1e8b521` (09-10-2026) el modo inferencia se ofrece también a los MID: la propagación corre en otro hilo y, si tarda más de dos segundos, un diálogo enseña una barra de progreso y un botón «Stop»; visto en la aplicación con `MID-Colorectal`, que se paró a los 32 segundos y volvió a edición sin más ventanas. Desde el commit `415a920` la barra mide el trabajo hecho y el diálogo da el tiempo que queda: antes de calcular la estrategia óptima se simulan las eliminaciones sobre una copia de la red en la que cada tabla es solo el conjunto de sus variables, y el trabajo de cada eliminación es el número de celdas de las tablas que construirá. En `MID-Cochlear` y `MID-Colorectal` la barra sigue al tiempo con pocos puntos de error; el trabajo previsto coincide con el hecho en 55 de 59 redes con decisiones y queda entre un 5 y un 11 % por debajo en cuatro redes de análisis de decisiones pequeñas. En la segunda etapa (el valor de cada nodo) la barra cuenta nodos y no se da tiempo. Quedan fuera las LIMID y los POMDP, que el algoritmo rechaza, y siguen sin evaluarse las redes de los errores 229 y 230. Probado el 09-10-2026 en la aplicación con el botón encendido para los MID, sin confirmarlo: de las 10 redes de `mid` (sin `MID-Cochlear`), entran en modo inferencia las tres `MID-Chancellor`, `MID-hip-Briggs` y las dos `MID-HPV`, en 3 a 5 segundos; `MID-Colorectal` entra pero deja la aplicación parada 163 segundos; `MID-CHAP-Ryan-Griffin`, `MID-CHD-Walker` y `MID-mammography` dan un diálogo de error normal y vuelven a edición. En `MID-Chancellor` todos los nodos de utilidad mostraban 0,0000, porque con transiciones al principio o a medio ciclo la propagación quitaba antes los nodos de utilidad del ciclo 0; desde el commit `654ea8f` (09-10-2026) cada nodo de utilidad recibe el valor descontado de su propio ciclo, sea cual sea la opción de transición (decidido por Manuel Arias a la vista de Pérez-Martín, Bermejo y Díez, «Evaluation of Markov models with discontinuities», *Medical Decision Making* 39(4), 2019: su ecuación 1 define el coste instantáneo en cada frontera entre ciclos, y las opciones de transición son formas de sumarlos). La red desplegada con «Expand network» entra ya en modo inferencia (error 232). Antes: desde el commit `ac0cc81` (09-10-2026) el modo inferencia se ofrece también a las redes de análisis de decisiones (error 227); faltan los MID. Desde el commit `a1e8859` (09-10-2026) el botón y la entrada de menú apagados lo explican en su ayuda emergente («Inference mode is offered only for Bayesian networks and influence diagrams»). El modo se irá ofreciendo a los MID y a las redes de análisis de decisiones a medida que se arregle su propagación. Medido ese día con la propagación del modo inferencia, sin evidencia, sobre las redes del repositorio: propagan 20 de 20 redes bayesianas, 30 de 30 diagramas de influencia, 66 de 68 redes de análisis de decisiones (falla `DAN-reactor` en sus dos formatos) y 14 de 26 MID (fallan `MID-CHD-Walker`, `MID-Colorectal` y `MID-mammography`, porque la relación uniforme de «Age [0]» no se convierte en tabla; `MID-CHAP-Ryan-Griffin`, por una expresión que usa «Age [0]» sin valor; `MID-HPV`, con `NullPointerException` al podar la red; y `MID-Cochlear`, que no termina en 60 segundos); el algoritmo rechaza las LIMID y los POMDP |

| | |
|---|---|
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:242-247,361-369` |
| **Comprobado** | Leyendo el código; visto en la ventana con un MID |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No. Pedía decidir si es intencionado y a qué tipos debe ofrecerse. Manuel Arias lo decidió el 09-10-2026: se ofrecerá también a los MID y a las redes de análisis de decisiones, que son los tipos que el algoritmo de propagación declara, pero no antes de arreglar las propagaciones que fallan |
| **Relacionados** | error 66 |

**Qué pasa.** Con un MID (diagrama de influencia de Markov) abierto, el usuario no puede pasar a modo inferencia: el botón de cambio de modo y las entradas de menú correspondientes están apagados. Lo vio Manuel Arias en la ventana el 08-10-2026, al comprobar el error 66.

Por qué pasa: `MainPanelMenuAssistant.getEnableWorkingModeButton` (líneas 242-247) devuelve verdadero solo si el tipo de la red es `InfluenceDiagramType` o `BayesianNetworkType`, y `updateOptionsNetworkDependent` (líneas 361-369) enciende con ese valor `CHANGE_WORKING_MODE`, `CHANGE_TO_INFERENCE_MODE` y `CHANGE_TO_EDITION_MODE`. Por lectura, quedan fuera todos los demás tipos: MID, POMDP, LIMID, redes de análisis de decisiones, redes de Markov, redes dinámicas y las demás. Solo se ha visto en la ventana con un MID.

La regla está en el código desde el commit `e1e38cc` (21-07-2026), con el que el proyecto entró en este repositorio. No se sabe si es intencionada.

Otras fichas de esta lista describen pasos «en modo inferencia» con redes de esos tipos (por ejemplo el error 66, que por eso no se alcanzaba); conviene releerlas con esto delante.

**Cómo arreglarlo.** Depende de la decisión. Si no es intencionado, la regla debería decir qué tipos de red pueden propagar evidencia, y no nombrar dos tipos concretos. Si lo es, conviene que la ventana lo explique, porque el botón apagado no dice por qué.

### Error 203 · La entrada «Select all» del menú Edit, y su atajo Ctrl+E, están apagados con una red abierta hasta que se pulsa una de las herramientas de edición de la barra

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:139-144,553-557`<br>`gui/src/main/java/org/openmarkov/gui/window/EditAndViewHandler.java:68-72`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/MainMenu.java:218,276` |
| **Comprobado** | Leyendo el código; visto en una captura automática del menú |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 201 |

**Qué pasa.** Con una red recién abierta, llena de nodos, la entrada «Select all» del menú Edit está apagada, y su atajo de teclado, Ctrl+E, no hace nada. Pasa a poderse usar en cuanto se pulsa en la barra una de las herramientas de edición (seleccionar, crear un nodo de azar, crear un enlace…),

que no tienen nada que ver con ella, y ya se queda encendida mientras haya alguna red abierta.

Visto el 8-10-2026 en una captura automática del menú Edit con `BN-asia.pgmx` abierta y sin haber tocado la barra: «Select all (off)».

Por qué pasa: el único sitio que apaga la entrada es `MainPanelMenuAssistant.updateOptionsAllNetworkClosed` (línea 144), que se ejecuta al arrancar y al cerrar la última red, y el único que la enciende es `setEditionOption` (línea 555), al que solo llama `EditAndViewHandler.activateEditionMode` cuando se elige una herramienta. Ni `updateOptionsNewNetworkOpen` ni `updateOptionsNetworkDependent`, que son los que ponen al día el menú al abrir una red o cambiar de pestaña, la nombran.

**El código.** gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:553-557

```
public void setEditionOption(String newEditionMode, boolean canPaste) {
    setOptionSelected(newEditionMode, true);
    setOptionEnabled(ActionCommands.SELECT_ALL, true);
    setOptionEnabled(ActionCommands.CLIPBOARD_PASTE, canPaste);
}
```

**Cómo arreglarlo.** Encender `SELECT_ALL` donde se encienden las demás entradas que dependen de que haya una red, en `updateOptionsNetworkDependent`, y quitar la línea de `setEditionOption`. Conviene decidir de paso si debe estar encendida en modo inferencia.

Prueba: con el ayudante del menú y una red abierta, sin elegir ninguna herramienta, comprobar que `SELECT_ALL` está encendida; y apagada sin redes abiertas.

### Error 204 · La ventana «About» del menú Help se abre con 24 píxeles de ancho, porque toma el ancho de un cartel que se carga encogido al tamaño de un icono

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog; org.openmarkov.gui.loader.element |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/AboutBox.java:33,71-101,130`<br>`gui/src/main/java/org/openmarkov/gui/loader/element/ImageLoader.java:61-63,99-112,118-126`<br>`gui/src/main/resources/gui/localize/Dialogs_en.xml:776-781` |
| **Comprobado** | Leyendo el código y ejecutándolo; visto en una captura automática de la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** Help → «About» abre una ventana de 24 píxeles de ancho y 224 de alto: una raya en la que no se lee nada (de cada rótulo se ve una letra y puntos suspensivos). Hay que ensancharla a mano para ver la versión y el resto.

Visto el 8-10-2026 en una captura automática de la ventana: mide 24×224. Medido después sin ventana: `ImageLoader.load("/images/OpenMarkovSplash.jpg")` devuelve un icono de 24×24, y el fichero mide 512×256.

Por qué pasa: `AboutBox` fija su tamaño con el del cartel (`width = openMarkovLogo.getIconWidth()`, `height = getIconHeight() + 200`, líneas 95-101) y lo carga con `ImageLoader.load`, que está pensado para los iconos de los botones: `createImage` reduce toda imagen a un cuadrado cuyo lado

depende de la escala de la interfaz (24 píxeles con la escala por omisión, líneas 99-112 y 118-126), salvo que se le pida `ImageOptions.SkipAutoScale`.

Otras tres cosas de la misma ventana, leídas en el código:

- la versión está escrita a mano, «0.4.0-SNAPSHOT» (línea 33), y el `pom.xml` dice 0.3.0-SNAPSHOT;
- el texto de los autores se lee de la base de textos (línea 79) y no se pone en ningún rótulo: `jLabelAuthors` se añade vacío (línea 130);
- el texto de derechos dice «(c) UNED - 2008-2019».

**Cómo arreglarlo.** Cargar el cartel sin encoger (`ImageLoader.load(url, ImageOptions.SkipAutoScale)`) o, mejor, no hacer depender el tamaño de la ventana de una imagen: componerla y llamar a `pack()`. Tomar la versión de donde la toma el resto del programa, y poner el texto de los autores en su rótulo o quitar la clave.

Prueba: construir el icono del cartel como lo hace la ventana y comprobar que mide lo que el fichero. La ventana entera necesita pantalla.

### Error 206 · En las listas de criterios, agentes y estados, «Delete» se enciende por el número de filas y no porque haya una señalada: tras borrar una fila sigue encendido, y pulsarlo otra vez lanza una excepción

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common; org.openmarkov.gui.dialog.network |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTablePanel.java:392-395,420-443`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/DecisionCriteriaTablePanel.java:101-108,135-142`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkAgentsTablePanel.java:107-116`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/PrefixedKeyTablePanel.java:171-174` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 173, error 174, nota 36, nota 81 |

**Qué pasa.** En la pestaña «Decision criteria» de las propiedades de la red, con tres criterios: se señala uno y se pulsa «Delete». Quedan dos criterios, ninguno señalado, y «Delete» sigue encendido. Pulsarlo de nuevo lanza una excepción de índice fuera de rango; como no es una excepción prevista, por lectura sale en la ventana de error inesperado.

Medido con un pequeño programa de prueba que construye el panel de criterios (`DecisionCriteriaTablePanel`) sin ventana, con `0_miscellaneous/networks/id/ID-CEA-test-2therapies-3criteria.pgmx`:

```
recién abierto, tres filas, ninguna señalada:   Delete apagado
se señala la fila 1:                            Delete encendido
se quita la señal:                              Delete encendido
se pulsa Delete sin fila señalada:              IndexOutOfBoundsException (índice -1)
se borra la fila 0; quedan dos, sin señalar:    Delete encendido
se pulsa Delete otra vez:                       IndexOutOfBoundsException (índice -1)
```

Por qué pasa: `KeyTablePanel.valueChanged`, que decide cuándo se pueden pulsar los botones,

calcula bien «Delete», «Up» y «Down» según la fila señalada (líneas 422-441) y termina pisando lo calculado para «Delete» con `removeValueButton.setEnabled(rowCount > 2)` (línea 442). `DecisionCriteriaTablePanel` lo enciende además siempre que haya dos filas (líneas 135-142). Los métodos que borran piden la fila señalada y la usan sin mirar si es −1.

`KeyTablePanel` es la base de la lista de criterios y de la de agentes de la red, y de la de estados de un nodo (`PrefixedKeyTablePanel`). Solo se ha ejecutado la de criterios; las otras dos comparten `valueChanged`. Por la misma línea 442, y por lectura, en la lista de agentes no se puede borrar un agente cuando solo quedan dos.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/KeyTablePanel.java:420-443

```
@Override public void valueChanged(ListSelectionEvent e) {
    int index = valuesTable.getSelectedRow();
    int rowCount = valuesTable.getRowCount();
    if ((rowCount == 0) || (index == -1)) {
        removeValueButton.setEnabled(false);
        ...
    } else {
        removeValueButton.setEnabled(true);
        ...
    }
    removeValueButton.setEnabled(rowCount > 2);
}
```

**Cómo arreglarlo.** Que «Delete» dependa de las dos cosas: hay una fila señalada y la lista admite perder una (el mínimo de filas es de cada lista: dos estados, un criterio; decidir el de los agentes). La regla del mínimo debería ser un método que cada subclase redefina, en vez de la última línea de `valueChanged` y el parche de `DecisionCriteriaTablePanel`. Tras borrar, dejar señalada la fila vecina, como ya hace la lista de estados.

Prueba de regresión: el programa de prueba de arriba, como prueba del panel: sin fila señalada «Delete» está apagado, y tras borrar una fila no queda encendido sin fila señalada.

## Error 220 · En el tema oscuro no se ven el rectángulo de selección ni el nombre de una pestaña con cambios, y no se leen las casillas con incertidumbre, el recuadro de una expresión ni los estados de las ramas de un árbol

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.configuration; org.openmarkov.gui.graphic; org.openmarkov.gui.window; org.openmarkov.gui.component; org.openmarkov.gui.dialog.common; org.openmarkov.gui.dialog.treeadd |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/graphic/SelectionRectangle.java:155-160`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanel.java:519-524`<br>`gui/src/main/java/org/openmarkov/gui/component/ValuesTableCellRenderer.java:279-292`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/ArithmeticExpressionDialog.java:222-225`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/VariableExpressionTextField.java:103-104`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDCellRenderer.java:196-245` |

| | |
|---|---|
| | `gui/src/main/java/org/openmarkov/gui/configuration/GUIColors.java:15-21,56-57,179`<br>`gui/src/main/java/org/openmarkov/gui/configuration/UserPreferences.java:92-93` |
| **Comprobado** | Leyendo el código y ejecutándolo (colores y contraste calculados sin ventana; el texto de las ramas, pintado sin ventana y visto en capturas automáticas) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 221, nota 11, nota 12 |

**Qué pasa.** El tema oscuro es el que el programa pone a quien tiene el sistema en oscuro, porque la preferencia por omisión es «Sync with OS» (`UserPreferences`, línea 93); también se puede elegir a mano, como «Dark (Beta)». En él, cinco cosas se pintan con un color pensado para un fondo claro. El commit `008cd39` (1-09-2026) dio colores propios del tema oscuro a los enlaces y a lo demás que se dibuja sobre el lienzo; estas cinco quedaron fuera.

El contraste entre dos colores va de 1 (el mismo color) a 21 (negro sobre blanco); por debajo de 3 una línea se pierde, y por debajo de 4,5 un texto pequeño cuesta de leer. Los de la lista están calculados con los colores que declara `GUIColors` y los que da el aspecto oscuro del programa, `FlatDarculaLaf`, en la versión 3.7.1 del `pom.xml` (texto 221, 221, 221; fondo de los paneles 60, 63, 65), leídos sin ventana:

- el rectángulo de trazos con que se señalan varios nodos se pinta en negro (`General.TEXT`) sobre el lienzo, que en oscuro es 69, 72, 74: contraste 2,3;
- el nombre de la pestaña de una red con cambios sin guardar se pinta en rojo, 212, 56, 56 (`General.ATTENTION_BG`, un color de fondo usado aquí como color de letra), sobre la pestaña, 60, 63, 65: contraste 2,2;
- una casilla de la tabla de probabilidades cuya columna lleva incertidumbre tiene fondo blanco (`UNCERTAINTY_BACKGROUND`) y la letra del tema, 221, 221, 221: contraste 1,4;
- el recuadro en que se escribe una expresión se pone verde, 0, 255, 0 (`General.CORRECT`), mientras la expresión es válida, y rojo, 255, 0, 0 (`General.WRONG`), cuando no lo es, con la letra del tema: contraste 1,0 sobre el verde y 2,9 sobre el rojo;
- los estados de cada rama de un árbol (estrategia óptima, intervenciones del coste-efectividad, pestaña del árbol de un nodo) salen en negro sobre el fondo del árbol, 70, 73, 75: contraste 2,3.

En el tema claro las cinco se leen: el rojo de la pestaña queda a 4,8 sobre un fondo blanco, y la letra negra a 15,3 sobre el verde y a 5,3 sobre el rojo.

El caso que más estorba es el del recuadro de una expresión: lo que se escribe deja de leerse justo cuando está bien escrito. `ArithmeticExpressionDialog.validateExpression` cambia el fondo y no toca el color de la letra (línea 225); el campo gemelo, `VariableExpressionTextField`, pone los dos colores juntos (líneas 103-104).

El texto de las ramas es distinto de los otros cuatro, porque nadie le pone el negro: `TreeADDCellRenderer.getBranchDescriptionHTML` lo mete en una celda de tabla de HTML (`<td align=center border=0>`), y a una celda no le llega el color de letra del rótulo que la contiene. Medido pintando sin ventana un rótulo con cada texto, en cada tema:

```
tema claro:   <html>yes</html>  letra (0,0,0)        <html><td ...>yes</td></html>  letra
↪  (2,2,2)
tema oscuro:  <html>yes</html>  letra (221,221,221)  <html><td ...>yes</td></html>  letra
↪  (0,0,1)
```

y visto el 8-10-2026 en capturas automáticas de la ventana de la estrategia óptima en oscuro, con

`ID-decide-test.pgmx`. Los otros cuatro no se han visto en una captura.

**Cómo arreglarlo.** Los cuatro primeros: dar a cada uso un color con valor para los dos temas, como hizo `008cd39` con `inDark`, y decir siempre el color de la letra junto con el del fondo. Para el rectángulo y la pestaña valen colores que el tema oscuro ya tiene (los de un enlace que se puede o no se puede trazar: 178, 188, 255 y 255, 143, 143). Para la casilla y el recuadro, que tienen un fondo fijo, basta fijar también la letra en negro. El texto de las ramas: quitar la celda de tabla, o escribir en ella el color del tema.

Conviene una prueba que recorra los pares de colores que se usan juntos y exija un contraste mínimo en los dos temas; `008cd39` ya midió así los del lienzo.

### Error 221 · Con el tema «Sync with OS», el programa puede quedarse en oscuro con el sistema en claro (o al revés) si el sistema cambió de tema mientras estaba elegido otro

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.configuration |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/configuration/Theme.java:52-71` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 220 |

**Qué pasa.** Con «Sync with OS» elegido, el programa debe ponerse claro u oscuro cuando lo hace el sistema. Hay una secuencia en la que deja de hacerlo:

1. el sistema está en claro al arrancar y el usuario tiene elegido «Light»;
2. el sistema pasa a oscuro;
3. el usuario elige «Sync with OS»: el programa se pone oscuro, bien;
4. el sistema vuelve a claro: el programa se queda en oscuro, hasta el siguiente cambio del sistema.

Por qué pasa: `Theme` recuerda en `lastWasDark` cómo estaba el sistema la última vez, para no repintar si no ha cambiado. Pero cuando la preferencia no es «Sync with OS» el oyente sale antes de apuntarlo (líneas 55-57), así que en el paso 2 no se entera. En el paso 3 se pregunta al sistema directamente y el tema sale bien, sin tocar `lastWasDark`. En el paso 4 el oyente compara «claro» con su `lastWasDark`, que sigue diciendo «claro» desde el arranque, y concluye que no ha cambiado nada.

Comprobado solo leyendo: hace falta un escritorio que cambie de tema.

**El código.** gui/src/main/java/org/openmarkov/gui/configuration/Theme.java:52-62

```
static {
    AtomicBoolean lastWasDark = new AtomicBoolean(Theme.OSisDark());
    com.jthemedetecor.OsThemeDetector.getDetector().registerListener(newIsDark -> {
        if (UserPreferences.PREFERRED_THEME.get() != Theme.SYNC_OS) {
            return;
        }
        if (lastWasDark.get() == newIsDark) {
            return;
        }
        lastWasDark.set(newIsDark);
```

```
...
```

**Cómo arreglarlo.** Apuntar el estado del sistema antes de mirar la preferencia; o no guardarlo, y comparar lo que dice el sistema con el tema que el programa tiene puesto, que es lo que importa.

Prueba: con la decisión extraída a un método sin interfaz (tema puesto, preferencia, lo que avisa el sistema), repetir la secuencia de arriba y comprobar que en el paso 4 se pide el cambio.

### Error 253 · [Corregido] En las opciones de la inferencia, un descuento como «1.4» se guarda como 0,013999999999999999 y no como 0,014, y la tabla lo enseña como «1.399999999999999900 %» y así se escribe en el fichero

| | |
|---|---|
| **Estado** | Corregido en el commit `c63a96e` (17-10-2026): `InferenceOptionsDialog` pasa el tanto por ciento a tanto por uno en decimal, con un único método para las dos tablas (`fractionOf`), y enseña el descuento sin ceros al final (`percentOf`). Medido el 17-10-2026 con el diálogo abierto de verdad, en la tabla de un criterio y en la de coste-efectividad: antes, «1.4» se guardaba 0,013999999999999999 y se enseñaba «1.399999999999999900 %», y entrar y salir de una casilla con 0,014 la dejaba igual; ahora se guarda 0,014 y se enseña «1.4 %», y los descuentos corrientes salen «3 %» en vez de «3.00 %». Con prueba sin ventana (los cien descuentos de 0,1 a 10 con un decimal se guardan y se enseñan como se escriben), y la del diálogo compara ya sin tolerancia. |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.inference.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:497-503,545-550,649-656,683-690`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_0_2.java:1368-1374`<br>`integrationTests/src/test/java/org/openmarkov/integrationTests/gui/TheInferenceOptionsKeepTheNumbersAsWrittenTest.java:130-139` |
| **Comprobado** | Leyendo el código y ejecutándolo (la misma aritmética y la escritura del fichero, sin ventana; el diálogo no se ha abierto) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 176 |

**Qué pasa.** En una red temporal (por ejemplo un MID, diagrama de influencia de Markov), la tabla de criterios del diálogo de las opciones de la inferencia tiene una columna de descuento en tanto por ciento. Si el usuario escribe «1.4», el criterio se queda con 0,013999999999999999, y no con 0,014, que es lo que daría leer «0.014». Al salir de la casilla la tabla se vuelve a armar y enseña «1.399999999999999900 %»; al aceptar, ese número llega a la red y se escribe así en el fichero:

```
<DiscountRate Criterion="cost" value="0.013999999999999999" unit="YEAR" />
```

Por el mismo camino que tenía el error 176, basta con entrar y salir de la casilla: un descuento de 0,014 leído del fichero se enseña «1.400 %», el editor quita « %» y deja «1.400», y al salir se guarda 0,013999999999999999. Esto está deducido del código y de la aritmética; no se ha visto en la ventana.

Por qué pasa: desde `b6e5ece` (arreglo del error 176) el diálogo guarda la escala tal como se escribe, pero el descuento se escribe en tanto por ciento y se pasa a tanto por uno dividiendo entre cien en binario (`tableChanged`, líneas 654-655 y 688-689). 1,4 no tiene escritura exacta en binario, y dividirlo entre cien añade otro redondeo. Para enseñarlo, la tabla lo multiplica por cien con `BigDecimal` a partir de `String.valueOf` (líneas 497-503 y 545-550), que conserva todas sus cifras.

La prueba del arreglo lo sabe: compara el descuento con una tolerancia de $10^{-12}$, y la escala sin ninguna.

Medido el 15-10-2026 con la misma aritmética, fuera del diálogo: de los 100 valores de 0,1 a 10,0 con un decimal, 27 se guardan distintos del número decimal escrito; por ejemplo, «1.4» se enseña «1.399999999999999900 %», «0.35» «0.349999999999999600 %», «0.7» «0.699999999999999900 %», «4.1» «4.099999999999999500 %» y «3.5555» «3.555499999999999600 %». Los corrientes (3; 3,5; 1,5; 5; 6) salen bien. Y escribiendo con `PGMXWriter_1_0` la red `MID-Chancellor.pgmx` con el descuento que da el diálogo para «1.4», el fichero lleva `value="0.013999999999999999"`.

El número guardado está siempre a un solo paso (el número representable vecino) del bueno, así que la diferencia en lo que se calcula con él es despreciable; lo que choca es lo que se ve en la tabla y en el fichero.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/inference/common/InferenceOptionsDialog.java:683-690

```
if (column == UNICRITERIA_DISCOUNT_COLUMN) {
    String discount = table.getValueAt(row,
    ↪  UNICRITERIA_DISCOUNT_COLUMN).toString();
    if (discount.indexOf(' ') != -1) {
        discount = discount.substring(0, discount.indexOf(' '));
    }
    double discountDouble = Double.parseDouble(discount);
    discountDouble /= 100;
    decisionCriteria.get(row - 1).setDiscount(discountDouble);
```

**Cómo arreglarlo.** Dividir en decimal, en los dos sitios (líneas 654-655 y 688-689): `new BigDecimal(discount).movePointLeft(2).doubleValue()` guarda 0,014 para «1.4», que es el número que se obtendría escribiendo el descuento en tanto por uno. Lo mejor es un único método que usen los dos. De paso, la tabla enseña los descuentos corrientes con ceros de más («3.00 %», «3.500 %») porque conserva la escala del `BigDecimal`; `stripTrailingZeros().toPlainString()` los quitaría.

Prueba: sobre ese método, sin ventana, que «1.4», «0.35», «4.1» y «3.5555» den exactamente `Double.parseDouble("0.014")`, `"0.0035"`, `"0.041"` y `"0.035555"`; y quitar la tolerancia de `TheInferenceOptionsKeepTheNumbersAsWrittenTest.theDiscount`.

### Error 254 · Si el fichero trae una duración del ciclo de cero o negativa, el diálogo de las propiedades de la red no se puede aceptar aunque nadie toque esa pestaña, y en modo inferencia, donde no se puede corregir, solo se cierra con «Cancel»

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.network; org.openmarkov.io.probmodel.reader |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkPropertiesDialog.java:81-101,271-280`<br>`gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkTemporalOptionsPanel.java:58-70,112-135`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:235-256`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:516-521`<br>`gui/src/main/java/org/openmarkov/java/swing/ComponentUtilities.java:152-163` |

| | |
|---|---|
| **Comprobado** | Leyendo el código y ejecutándolo (el lector y la pestaña de opciones temporales, sin ventana; el diálogo no se ha abierto) |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 73 |

**Qué pasa.** Desde `e2c168b` (arreglo del error 73), al pulsar «OK» en «Network properties» el diálogo comprueba que la duración del ciclo sea un número positivo, **la haya tocado alguien o no**. El lector de `.pgmx` no comprueba nada: el fichero de una red temporal (un MID, diagrama de influencia de Markov, por ejemplo) con `<TimeUnit unit="YEAR" Value="0" />` (o negativo) se abre sin ningún aviso y la red se queda con un ciclo que no dura nada. Quien abra después las propiedades de esa red para cambiar, por ejemplo, el comentario no puede aceptar: el diálogo dice «The cycle length must be a positive number.» y no se cierra hasta que se corrija la duración o se cancele.

Peor en modo inferencia: ahí `NetworkEditorPanel.changeNetworkProperties` abre el diálogo solo para leer (`workingMode != WorkingMode.EDITION`, línea 519), y su constructor desactiva todos los controles de todas las pestañas (`ComponentUtilities.removeInputsFor`, líneas 93-98). La duración no se puede corregir, «OK» sigue comprobándola y no cierra, y solo queda «Cancel». Un diálogo de solo lectura no debería aplicar nada al aceptar.

El programa no escribe por sí mismo una duración así (hasta `e2c168b` la casilla ni se leía, y ahora la rechaza): llega de un fichero escrito a mano o por otro programa. Por lectura, un «NaN» en el fichero tiene el mismo efecto; un texto que no es un número se queda en la duración por omisión, 1.

Por qué pasa: `NetworkPropertiesDialog.doOkClickBeforeHide` (líneas 271-277) llama siempre a `NetworkTemporalOptionsPanel.applyCycleLength`, que lee el texto de la casilla y devuelve `false` si no es positivo, sin mirar si es el mismo que la pestaña puso al abrirse (`probNetScale`, líneas 63-69). El panel se construye siempre, porque la lista de pestañas lo pide para decidir si la enseña. Y `PGMXReader_0_2.getTemporaUnit` (líneas 235-256) acepta cualquier número; el esquema `val_v4.xsd` no dice nada del atributo.

Medido el 15-10-2026 con `MID-Chancellor.pgmx` cambiando su `Value="1.0"` por `"0"` y por `"-1"`: el lector da una duración de 0,0 y de −1,0 años, `FormatManager.schemaProblems` no encuentra nada (así que la ventana tampoco avisaría al abrir), y un `NetworkTemporalOptionsPanel` construido sobre esa red (es un `JPanel`, no necesita ventana), sin tocarlo, contesta `applyCycleLength() = false`; con el fichero original, `true`. Lo que el diálogo hace con ese `false` está leído en `doOkClickBeforeHide`; el diálogo no se ha abierto.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/network/NetworkTemporalOptionsPanel.java:112-124

```
    public boolean applyCycleLength() {
        if (unitScale == null ||
        ↪  probNet.hasConstraintOfClass(OnlyAtemporalVariables.class)) {
            return true;
        }
        double value;
        try {
            value = Double.parseDouble(unitScale.getText().trim());
        } catch (NumberFormatException e) {
            value = Double.NaN;
        }
        if (!(value > 0) || Double.isInfinite(value)) {
            return false;
```

```
        }
```

**Cómo arreglarlo.** Que `applyCycleLength` solo compruebe (y aplique) la duración si el usuario ha cambiado el texto o la unidad respecto de lo que la pestaña enseñó al abrirse; una duración que llega del fichero y nadie toca pasa, como pasa hoy en el resto del programa. Y que `doOkClick BeforeHide`, con el diálogo en solo lectura, no aplique ninguna pestaña. Aparte, conviene decidir si el lector (o el esquema, con una restricción en `TimeUnit/@Value`) debe avisar de una duración no positiva al abrir el fichero, como ya avisa de lo que no cumple el esquema.

Prueba: con una red cuya duración del ciclo sea 0 (puesta en código o leída de un fichero), construir `NetworkTemporalOptionsPanel` sin tocarlo y comprobar que `applyCycleLength()` devuelve `true` y la red conserva su duración; escribir «0» en la casilla y comprobar que devuelve `false`. Lo del modo de solo lectura necesita pantalla: abrir el diálogo con `readOnly = true` sobre esa red, pulsar «OK» y comprobar que se cierra.

## 16 Textos y títulos que se ven mal

Textos que la aplicación, en inglés, enseña equivocados, vacíos o con el nombre de una clave.

### Error 177 · [Jorge Rico] La ayuda de atajos promete dos teclas equivocadas, y siguiendo una se cierra el programa

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog |
| **Ficheros** | `gui/src/main/resources/gui/localize/Dialogs_en.xml:702-709`<br>`gui/src/main/java/org/openmarkov/gui/dialog/ShortcutsBox.java:38-45,117-120`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/MainMenu.java:203,205,209,218,331,342`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanel.java:148-157`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:120`<br>`gui/src/main/java/org/openmarkov/gui/window/NetworkFileHandler.java:401-421` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 18, nota 127 |

**Qué pasa.** Help → «Shortcuts» abre `ShortcutsBox` (módulo `gui`), dos tablas de «tecla → función» cuyos textos salen de `Dialogs_en.xml` (claves `ShortcutsBox.Specific.<entrada>.Shortcut` y `.Function`, `ShortcutsBox`, líneas 38-45 y 117-120). Comparadas con los aceleradores que pone la barra de menús (`MainMenu.createAllItems`) y con el mapa de teclas de las pestañas (`MainPanel`, líneas 149-158), dos filas de la tabla de atajos específicos no dicen la verdad:

- **`Ctrl + W`**: la ayuda dice «Save and reopen» (`Dialogs_en.xml`, líneas 703-706). Esa tecla cierra la red de la pestaña actual («Close network», `CLOSE_TAB`, `MainMenu` línea 205). La tecla real de guardar y reabrir es `Ctrl + Alt + S` («Save network and reopen», línea 203).
- **`Ctrl + A/Q`**: la ayuda dice «Previous/Next OpenMarkov sub-window» (líneas 699-702). `Ctrl + Q` sale del programa («Exit», `EXIT_APPLICATION`, línea 209), y `Ctrl + A` no está atado a

nada en la barra de menús ni en el mapa de pestañas. Las teclas reales para pasar a la pestaña anterior o siguiente son `Ctrl + Shift + ←` / `Ctrl + Shift + →` (menú View, líneas 331 y 342, y `MainPanel`, líneas 149-158).

Consecuencia: quien quiera pasar a la pestaña siguiente siguiendo la ayuda pulsa `Ctrl + Q` y sale del programa (`MainPanelListenerAssistant`, línea 128 → `NetworkFileHandler.closeApplication`, líneas 395-405, que cierra todas las pestañas y llama a `System.exit(0)`). Por lectura, cada pestaña se cierra por el camino normal (`NetworkEditorPanel.close` → `networkCanBeClosed`), así que las redes modificadas deberían pedir confirmación antes de perderse; lo que no se evita es la salida. Y quien quiera guardar y reabrir con `Ctrl + W` cierra la red.

El resto de filas sí coincide con el mapa: `Ctrl + I` (cambiar modo), `Ctrl + N`, `Ctrl + O`, `Ctrl + S`, `Ctrl + E` (seleccionar todo, línea 218), `Ctrl + X/C/V`, `Ctrl + Z`, `Ctrl + Y`. Establecido leyendo el código; no se ha pulsado ninguna tecla.

**El código.** gui/src/main/resources/gui/localize/Dialogs_en.xml:699-706

```
<PreviousNext>
    <Shortcut value="Ctrl + A/Q"/>
    <Function value="Previous/Next OpenMarkov sub-window"/>
</PreviousNext>
<SaveReopen>
    <Shortcut value="Ctrl + W"/>
    <Function value="Save and reopen"/>
</SaveReopen>
```

gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/MainMenu.java:203-209

```
createItem(MenuItemNames.FILE_SAVE_OPEN_MENUITEM, ActionCommands.SAVE_OPEN_NETWORK,
↪  IconBind.SAVE_ENABLED, MainMenu.ctrlAlt(KeyEvent.VK_S));
...
createItem(MenuItemNames.FILE_CLOSE_MENUITEM, ActionCommands.CLOSE_TAB, null,
↪  MainMenu.ctrl(KeyEvent.VK_W));
...
createItem(MenuItemNames.FILE_EXIT_MENUITEM, ActionCommands.EXIT_APPLICATION, null,
↪  MainMenu.ctrl(KeyEvent.VK_Q));
```

**Cómo arreglarlo.** Corregir los dos textos en `Dialogs_en.xml`: `Ctrl + Alt + S` para «Save and reopen» y `Ctrl + Shift + ←/→` para «Previous/Next tab» (y, si se quiere, añadir filas para `Ctrl + W` «Close network» y `Ctrl + Q` «Exit»). El fichero `Dialogs_es.xml` tiene las mismas filas, pero no se mantiene.

Para que no vuelva a pasar, la ayuda podría construirse leyendo los aceleradores de los elementos de la barra de menús en vez de repetirlos a mano. Como mínimo, una prueba que compare cada fila de `ShortcutsBox.Common.*` y `ShortcutsBox.Specific.*` que nombre una tecla con `Ctrl` con el acelerador del elemento de menú correspondiente (`MainMenu.createAllItems` y `buildViewMenu`).

### Error 178 · [Jorge Rico] Al pegar un nodo de sucesos en una red que no los admite, el mensaje de error dice «»> eventNode is null «<»

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.action.core |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/action/core/AddNodeEdit.java:88-92,145-148` |

| | |
|---|---|
| | gui/src/main/java/org/openmarkov/gui/action/PasteEdit.java:85<br>gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:178<br>core/src/main/resources/core/localize/core_class_localizations_en.xml:456-457 |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | nota 153 |

**Qué pasa.** Crear directamente un nodo de sucesos (tipo EVENT, los nodos de las redes de simulación de eventos discretos) en una red que no los admite no es posible: la herramienta y el botón de tipo de nodo se deshabilitan si la red lleva la restricción `NoEventNodes`. Pero el portapapeles es único para todas las pestañas (`CLIPBOARD_ASSISTANT` es estático en `NetworkEditorPanel`), y pegar ejecuta un `AddNodeEdit` por nodo copiado (`PasteEdit`, línea 85) con el tipo del nodo original.

Copiar un nodo de sucesos en una red de simulación de eventos discretos y pegarlo en una red bayesiana o en un diagrama de influencia lanza la violación, que la ventana muestra en un diálogo de error. El texto de ese diálogo dice «»> eventNode is null «< is of type event.» en lugar del nombre del nodo. El pegado se rechaza entero y la red no cambia; el defecto es solo el texto.

`AddNodeEdit` añade un nodo a la red. Su comprobación previa (`checkConstraintsWillBeMet`) rechaza un nodo de tipo EVENT cuando la red lleva `NoEventNodes`, y para ello construye `CannotHaveEventNodeException(constraint, this.newNode)`. Pero `newNode` solo se asigna en `doEdit()`, que todavía no se ha ejecutado: en la comprobación vale `null`. El texto de esa excepción es «{eventNode} is of type event.», y con el nodo a `null` el sistema de textos lo resuelve como «»> eventNode is null «< is of type event.».

Medido con un pequeño programa de prueba que construye ese `PasteEdit` sin ventana: el mensaje que se entregaría es «Multiple constraints have been violated: - Could not met the constraint: No event nodes allowed — »> eventNode is null «< is of type event. - Could not met the constraint: Only chance nodes are allowed — Node for Event variable Ev with states [state event] must be a chance node.» En la red bayesiana salta además `OnlyChanceNodes`; en un diagrama de influencia solo saldría la primera línea, con el marcador.

**El código.** core/src/main/java/org/openmarkov/core/action/core/AddNodeEdit.java:88-92

```
if (probNet.getConstraintOfClass(NoEventNodes.class) instanceof NoEventNodes constraint) {
    if (this.nodeType == NodeType.EVENT) {
        constraintChecker.addException(new
↪   ConstraintViolatedException.CannotHaveEventNodeException(constraint, this.newNode));
    }
}
```

**Cómo arreglarlo.** La excepción necesita algo que exista antes de ejecutar: la variable (`this.variable`) en lugar del nodo. Eso obliga a cambiar el tipo del campo `eventNode` de `CannotHaveEventNodeException` (o añadir un constructor con `Variable`) y el texto de `core_class_localizations_en.xml` (líneas 456-457).

La misma regla vive en `NoEventNodes.checkProbNet` (que sí tiene el nodo); conviene que las dos vías produzcan el mismo texto.

Prueba: red bayesiana o diagrama de influencia, `AddNodeEdit` con tipo EVENT, y afirmar que el mensaje localizado contiene el nombre de la variable y no contiene «»>».

### Error 179 · [Jorge Rico] El selector del tipo de relación enseña nombres internos como «ProbTable» o «UnivariateDistr»

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui, core |
| **Paquete** | org.openmarkov.gui.dialog.node; org.openmarkov.core.model.network.potential.plugin |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:228-256`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/plugin/PotentialUtils.java:34-46`<br>`gui/src/main/resources/gui/localize/Selectables_en.xml:3-49`<br>`io/src/main/java/org/openmarkov/io/probmodel/writer/PGMXWriter_1_0.java:116,191` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | **Sí.** Qué nombre legible se enseña para cada tipo de relación |

**Qué pasa.** En el diálogo de editar el potencial de un nodo, el desplegable «tipo de relación» mezcla nombres legibles («Same as previous», «Linear combination», «OR / MAX») con identificadores pegados («ProbTable», «AugmentedProbTable», «DistributionTable», «UnivariateDistr») o poco claros («Exact»). `CycleLengthShift` también sale con su identificador, aunque no apareció en los nodos probados.

`PotentialEditPanel.getPotentialTypeJCombobox` (líneas 227-241) llena el desplegable con las clases de potencial permitidas para el nodo y las pinta con `PotentialUtils::getPotentialName`, que devuelve el primer valor de `names` de la anotación `@PotentialType`. Ese mismo nombre es el que los escritores de `.pgmx` guardan como tipo del potencial (`PGMXWriter_1_0`, líneas 116 y 191), así que es un identificador de formato.

`Selectables_en.xml` tiene un bloque `algorithmRelationType` con textos legibles, pero ningún código Java lo lee, y sus claves (`Table`, `SameAsPrevious`, `Tree_ADD`, `ICI-OR`, `ICI-MAXCausal`…) no coinciden con los nombres actuales de las anotaciones.

Medido con un pequeño programa de prueba que llama a `PotentialUtils.getFilteredPotentialClasses` sobre nodos reales (quitando las clases abstractas, como hace el panel):

- nodo de azar de estados finitos («Disease» de `0_miscellaneous/networks/id/ID-decide-test.pgmx`): AugmentedProbTable, Conditional Gaussian, Delta, Discretized Cauchy, Hazard (Exponential), ProbTable, Uniform;
- nodo de utilidad («Health state», de la misma red): DistributionTable, Exact, Exponential, Function, Linear combination, Tree/ADD, Uniform, UnivariateDistr;
- nodo de azar numérico («Duration of treatment [0]» de `0_miscellaneous/networks/mid/DMHEE/MID-dmhee-2.5.pgmx`): Binomial, Delta, DistributionTable, Exact, Exponential, Function, Linear combination, Product, Sum, Uniform, UnivariateDistr.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/node/PotentialEditPanel.java:235-237

```
↪  filteredPotentialNames.sort(Comparator.comparing(PotentialUtils::getPotentialName));
        this.potentialTypeComboBox = new JComboBox<Class<? extends
        ↪  Potential>>(filteredPotentialNames.toArray(new Class[0]));
        this.potentialTypeComboBox.setRenderer(new JComboBoxFunctionRender<Class<?
        ↪  extends Potential>>(PotentialUtils::getPotentialName));
```

**Cómo arreglarlo.** No cambiar `names` de las anotaciones, porque es lo que se escribe y se lee en los ficheros. Separar el nombre para mostrar:

- una clave de texto por clase de potencial en `Selectables_en.xml` (rehaciendo el bloque `algorithmRelationType` con claves que coincidan, o usando el nombre de la clase como clave);
- un método que devuelva el texto si existe y el identificador si no;
- usarlo en el pintor del desplegable y también en la ordenación de la línea 235, para que el orden sea el que ve el usuario.

Hay que decidir los textos.

Prueba: para cada clase concreta anotada con `@PotentialType`, el nombre para mostrar no debe ser la clave rodeada `>>> … <<<`.

### Error 180 · [Jorge Rico] El diálogo de dominios estándar se abre sin título

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.node |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/StandardDomainsDialog.java:24-37`<br>`gui/src/main/java/org/openmarkov/gui/dialog/common/DialogBase.java:49`<br>`gui/src/main/resources/gui/localize/Buttons_en.xml:3-7` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 173, error 174, error 181, error 212 |

**Qué pasa.** Al pulsar «Standard domains» en la pestaña de dominio del diálogo de un nodo, la ventana `StandardDomainsDialog` se abre con la barra de título vacía.

Su constructor llama a `super(owner)`, y ninguna clase de la cadena (`OkCancelDialog`, `BottomPanelButtonDialog`, `DialogBase`, que llama a `JDialog(owner)` sin título) llama a `setTitle`. El diálogo hermano `StandardCriteriaDialog` tampoco lo hace.

Comprobado leyendo; no se ha abierto la ventana.

**Cómo arreglarlo.** Añadir en el constructor `setTitle(stringDatabase.getString("StandardDomain.Text"))` (hoy «Standard domains», el texto del botón que la abre) o una clave propia en el fichero de textos. Lo mismo para `StandardCriteriaDialog` con `StandardCriteria.Text`.

Prueba: construir el diálogo sin mostrarlo y comprobar que `getTitle()` no está vacío.

### Error 181 · [Jorge Rico] Seis de las ventanillas del editor de árboles se abren sin título

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.treeadd |

| | |
|---|---|
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/treeadd/AddStatesToBranchDialog.java:48-61`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/RemoveStatesDialog.java:48-58`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/AddVariablesDialog.java:47-57`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/RemoveVariablesDialog.java:46-56`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/ChangeIntervalDialog.java:48-62`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/SetReferenceDialog.java`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/SplitIntervalDialog.java:33`<br>`gui/src/main/java/org/openmarkov/gui/dialog/treeadd/TreeADDEditorPanel.java:1245` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 161, error 180, error 182, nota 125, error 212 |

**Qué pasa.** Varias operaciones del menú contextual del editor de árboles (`TreeADDEditorPanel`, módulo `gui`) abren una ventanilla antes de actuar. En seis de ellas la barra de la ventana aparece vacía, y el usuario no ve qué operación está confirmando:

- `AddStatesToBranchDialog`, que abre «Add states to branch»;
- `RemoveStatesDialog`, que abre «Remove states from branch»;
- `AddVariablesDialog`, que abre «Add variables to potential»;
- `RemoveVariablesDialog`, que abre «Remove variables»;
- `ChangeIntervalDialog`, que abre «Change interval»;
- `SetReferenceDialog`, que abre «Set reference».

Todas heredan de `OkCancelDialog` → `BottomPanelButtonDialog` → `DialogBase` → `JDialog`, y ninguna de esas clases base pone un título. Las cinco primeras tienen su llamada a `setTitle` dentro de un bloque de comentario (restos de una API de textos antigua); `SetReferenceDialog` no tiene ni siquiera la llamada comentada.

La séptima, `SplitIntervalDialog`, sí tiene título: `setTitle("Split Interval")` en `initialize` (línea 33); el bloque comentado de más abajo no lo quita. La de poner etiqueta es un `JOptionPane.showInputDialog(null, "Enter label name: ", "Set label", 1)` (`TreeADDEditorPanel`, línea 1245), también con título.

Los rótulos escritos en inglés dentro del código («Threshold value: », «Included in first interval ](», «Split Interval», «Set label», «Enter label name: ») no forman parte de este error: la aplicación está solo en inglés y se leen bien.

No se ha abierto ninguna ventanilla (no se puede sin pantalla); la ausencia de título se deduce de que no hay ningún `setTitle` activo en la clase ni en sus bases.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/treeadd/AddVariablesDialog.java:47-57

```
private void configureComponentsPanel() {
    /*dialogStringResource =
            StringResourceLoader.getUniqueInstance().getBundleDialogs();
    messageStringResource =
```

```
            StringResourceLoader.getUniqueInstance().getBundleMessages();
    setTitle(dialogStringResource
            .getValuesInAString("NodePotentialDialog.Title"));*/
    getComponentsPanel().setLayout(new BorderLayout(5, 5));
    getComponentsPanel().add(getJPanelVariables(), BorderLayout.CENTER);
}
```

**Cómo arreglarlo.** Dar a cada ventanilla un título. Lo natural es el texto de la opción de menú que la abre (claves `TreeADD.JoinBranches`, `TreeADD.DissociateStates`, `TreeADD.AddVariables`, `TreeADD.RemoveVariables`, `TreeADD.ChangeInterval`, `TreeADD.SetReference` de `Menus_en.xml`), que es el criterio que ya siguen otros diálogos. Quitar de paso los bloques comentados. Si se quiere uniformidad, `SplitIntervalDialog` puede tomar también su título de `TreeADD.SplitInterval`, aunque no es necesario porque la aplicación está solo en inglés.

Es la misma familia que error 180 (diálogo de dominios estándar sin título); conviene revisar a la vez las demás subclases de `OkCancelDialog` que no llaman a `setTitle`.

Prueba: una prueba de interfaz (AssertJ-Swing, con pantalla) que abra cada operación del menú del árbol y compruebe que el título de la ventana no está vacío (estas ventanas no se pueden construir sin pantalla).

### Error 182 · [Jorge Rico] Los títulos de varias ventanas de error parten las siglas letra a letra: «Invalid limit in tree a d d», «X m l invalid»

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.exception |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/exception/IBundledOpenMarkovException.java:22-43`<br>`gui/src/main/java/org/openmarkov/gui/dialog/ExceptionDialog.java:52-69`<br>`gui/src/main/java/org/openmarkov/gui/dialog/ExceptionDialog.java:87-103`<br>`gui/src/main/java/org/openmarkov/gui/dialog/OMExceptionHandler.java:57`<br>`gui/src/main/java/org/openmarkov/gui/exception/InvalidLimitInTreeADDException.java`<br>`gui/src/main/java/org/openmarkov/gui/exception/ChangeDomainOfTreeADDIsNotAllowedException.java`<br>`core/src/main/java/org/openmarkov/core/exception/ProbNetParserException.java:147-160`<br>`io/src/main/java/org/openmarkov/io/probmodel/exception/PGMXParserException.java:92-103`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:136-141`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:176-180` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 161, error 181, error 183, nota 17 |

**Qué pasa.** Cuando una operación se rechaza con una excepción propia de OpenMarkov, la aplicación enseña una ventana de error cuyo mensaje sale del fichero de textos y cuyo título se

fabrica a partir del nombre de la clase de la excepción. Si el nombre lleva una sigla, el título la parte en letras sueltas y en minúscula. Casos que se ven en la aplicación:

- en el editor de árboles, partir un intervalo por un umbral que coincide con un límite de la rama y del mismo lado: «Invalid limit in tree a d d»; cambiar el intervalo de una rama saliéndose del dominio: «Change domain of tree a d d is not allowed»;
- abrir un `.pgmx` que no es XML bien formado, por ejemplo un fichero cortado: «X m l invalid»;
- por lectura, con la misma causa: un `.pgmx` que no pasa la comprobación de versión o de estructura, «P g m x invalid»; y uno con una rama de árbol sin potencial ni referencia o sin sus dos umbrales, «Tree a d d without potential or referenfe» (con la errata del nombre de la clase) y «Tree a d d without two thresholds».

Por qué pasa: `IBundledOpenMarkovException.getExceptionTitle` (módulo `core`) devuelve `titlifyExceptionName`, que quita el sufijo «Exception» y cambia cada letra mayúscula por un espacio seguido de esa letra en minúscula, sin tratar de otra forma las mayúsculas seguidas. `ExceptionDialog.show` usa ese título para toda excepción de OpenMarkov. Le llegan las que el manejador global recibe envueltas en `UnrecoverableException` (`OMExceptionHandler`, línea 57): las del editor de árboles (nota 17) y las de abrir un fichero, que `GUIUtils.executeUIAction` envuelve así.

Medido:

- títulos devueltos por las excepciones construidas: `InvalidLimitInTreeADDException` → «Invalid limit in tree a d d»; `ChangeDomainOfTreeADDIsNotAllowedException` → «Change domain of tree a d d is not allowed»; `TriedToSplitIntervalOutsideBoundsException` → «Tried to split interval outside bounds», que no tiene siglas y sale bien;
- leer con `PGMXReader` un `.pgmx` cortado a mitad lanza `ProbNetParserException$XMLInvalid` con título «X m l invalid»;
- recorriendo todas las clases de `org.openmarkov` que usan el título fabricado (170), nueve tienen siglas partidas: las seis citadas arriba (`InvalidLimitInTreeADDException`, `ChangeDomainOfTreeADDIsNotAllowedException`, `XMLInvalid`, `PGMXInvalid`, `TreeADDWithoutPotentialOrReferenfe`, `TreeADDWithoutTwoThresholds`), `OnlyDESNetsAllowedException` («Only d e s nets allowed», de la simulación de sucesos; no se ha buscado cómo llega a la ventana) y dos del formato Elvira (`SomeSubpotentialsArentLinkedToAnICIPotential` e `ICIModelNotSupportedByElvira`), que no se tocan porque ese formato está congelado; el arreglo de abajo las corrige de paso.

**El código.** core/src/main/java/org/openmarkov/core/exception/IBundledOpenMarkovException.java:22-43

```
@Override @Nullable default String getExceptionTitle() {
    return IBundledOpenMarkovException.titlifyExceptionName(this);
}
...
@NotNull static String titlifyExceptionName(@NotNull Object exception) {
    String simpleName = exception.getClass().getSimpleName();
    if (simpleName.endsWith("Exception")) {
        simpleName = simpleName.substring(0, simpleName.length() - "Exception".length());
    }
    for (var uppercaseLetter : IBundledOpenMarkovException.UPPERCASE_LETTERS) {
        simpleName = simpleName.replace(uppercaseLetter, " " +
↪   uppercaseLetter.toLowerCase());
    }
    ...
    return simpleName.replace("\\n", System.lineSeparator());
```

```
}
```

**Cómo arreglarlo.** Cambiar `titlifyExceptionName` para que una serie de mayúsculas se quede junta y en mayúsculas: cortar antes de una mayúscula que sigue a una minúscula, y antes de la última mayúscula de una serie cuando le sigue una minúscula («TreeADDWithout» → «Tree ADD without», «XMLInvalid» → «XML invalid», «OnlyDESNets» → «Only DES nets»). La otra opción es dar a estas clases un título propio en el fichero de textos; arreglar la función cubre también las clases que se creen después.

Aparte, dos nombres de clase llevan erratas que se ven en el título: `TreeADDWithoutPotentialOrReferenfe` («referenfe») y `SomeSubpotentialsArentLinkedToAnICIPotential` («arent»; es del formato Elvira).

Prueba de regresión: una prueba unitaria de `titlifyExceptionName` con `InvalidLimitInTreeADDException`, `ProbNetParserException.XMLInvalid` y `OnlyDESNetsAllowedException` que exija «Invalid limit in tree ADD», «XML invalid» y «Only DES nets allowed», y que un nombre sin siglas como `TriedToSplitIntervalOutsideBoundsException` siga dando «Tried to split interval outside bounds».

### Error 183 · [Jorge Rico] El aviso de zoom fuera de rango dice «Requested zoomManager» en vez de «Requested zoom»

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.menutoolbar.toolbar |
| **Ficheros** | `gui/src/main/resources/gui/localize/gui_class_localizations_en.xml:102-103`<br>`gui/src/main/java/org/openmarkov/gui/menutoolbar/toolbar/ZoomComboBox.java:125-143`<br>`gui/src/main/resources/gui/localize/Dialogs_en.xml:907-909` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 182, nota 70 |

**Qué pasa.** La caja de zoom de la barra de herramientas (`ZoomComboBox`) es editable. Si el usuario escribe un valor fuera del rango 10-500 % o que no es un número (por ejemplo «600»), sale un aviso que dice «Requested zoomManager (600 %) is out of the range [10..500].»: el usuario ve el nombre de un campo interno del programa donde debería leer «zoom».

`itemStateChanged` restaura el valor anterior y lanza `ZoomOutOfRangeException` envuelta en `UnrecoverableException`; el gestor global muestra su mensaje, que sale de `gui_class_localizations_en.xml`. Ese texto dice «Requested zoomManager ({requestedZoom}) is out of the range [{minZoom}..{maxZoom}].».

Medido con un pequeño programa de prueba: el mensaje construido es `Requested zoomManager (600 %) is out of the range [10..500].`

En `Dialogs_en.xml` hay otro resto igual, `SelectZoom.Title` = «Select zoomManager», pero ningún código usa esa clave, así que no se ve.

El mismo cambio que dejó `zoomManager` en estos textos estropeó también varios textos del fichero en castellano; están descritos en nota 123, donde no hay nada que corregir.

**El código.** gui/src/main/resources/gui/localize/gui_class_localizations_en.xml:102-103

```
<Localization class="org.openmarkov.gui.exception.ZoomOutOfRangeException"
              value="Requested zoomManager ({requestedZoom}) is out of the range
↪  [{minZoom}..{maxZoom}]."/>
```

**Cómo arreglarlo.** Cambiar el texto a «Requested zoom ({requestedZoom}) is out of the range [{minZoom}..{maxZoom}].» y, de paso, «Select zoomManager» por «Select zoom» (o borrar esa clave, que nadie usa). Buscar `zoomManager` en todos los ficheros `*_en.xml` de todos los módulos: hoy solo aparece en esos dos.

Prueba de regresión: una prueba que recorra los ficheros de textos en inglés y falle si algún valor contiene un identificador con forma de nombre de campo Java como `zoomManager`, o, más simple, que compruebe el mensaje de `ZoomOutOfRangeException`.

### Error 184 · [Jorge Rico] El rótulo del panel del indicador dice «Probability of ocurrence», con una errata

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.common |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/common/IndicatorPotentialPanel.java:60,65` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 152, error 186, error 187, error 188, nota 120, nota 140 |

**Qué pasa.** `IndicatorPotentialPanel` (módulo `gui`) es el panel con que se edita la relación «indicador» de un nodo de suceso en una red de simulación de eventos discretos (DES): cuándo ocurre el suceso (tiempo hasta el suceso) y con qué probabilidad.

El segundo de sus rótulos dice «Probability of ocurrence:» (línea 65), con una errata que se ve en la aplicación: en inglés se escribe «occurrence», con dos erres y dos ces.

Los dos rótulos, «Time-to-event:» (línea 60) y «Probability of ocurrence:», están escritos en el código. Que no salgan de un fichero de textos no tiene consecuencia en la aplicación en inglés y no forma parte de este error.

**El código.** gui/src/main/java/org/openmarkov/gui/dialog/common/IndicatorPotentialPanel.java:65

```
JLabel probabilityTextLabel = new JLabel("Probability of ocurrence:");
```

**Cómo arreglarlo.** Cambiar el texto a «Probability of occurrence:». Si se aprovecha para llevarlo a un fichero de textos, seguir el estilo del resto de paneles de relación. No hace falta prueba.

### Error 185 · [Jorge Rico] La ayuda emergente de «Add → Utility node», en el menú contextual del lienzo, dice «Create an utility node»

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |

| | |
|---|---|
| **Paquete** | org.openmarkov.gui.menutoolbar.menu |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/NetworkContextualMenu.java:106-133`<br>`gui/src/main/java/org/openmarkov/gui/componentBuilder/JMenuItemBuilder.java:166-167` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 188, nota 120 |

**Qué pasa.** Con una red abierta en modo edición, el clic derecho sobre una zona vacía del lienzo abre un menú con el submenú «Add», que ofrece «Chance node», «Decision node», «Utility node» y «Event node». Al dejar el ratón sobre «Utility node» aparece la ayuda emergente «Create an utility node»; en inglés es «a utility node», porque «utility» empieza por sonido de consonante. Las otras tres ayudas («Create a chance node», «Create a decision node», «Create an event node») están bien.

El texto está escrito en el código, en `NetworkContextualMenu.getCreateElementsMenuItem` (módulo `gui`, línea 114), y `JMenuItemBuilder.build` lo pone como ayuda emergente con `setToolTipText` (líneas 166-167). Este submenú es hoy una de las formas de crear nodos, además del doble clic en el lienzo; los botones de la barra de herramientas no están, por decisión.

Comprobado leyendo; no se ha abierto el menú. Fuera de este texto, «an utility» solo aparece en comentarios del código.

**El código.** gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/NetworkContextualMenu.java:114-116

```
new NodeMenuGenerator(NodeType.UTILITY, "Utility node",
↪  "jmenuItemCreateUtilityNode","Create an utility node",
                    IconBind.UTILITY_ENABLED.icon(),
                    ActionCommands.UTILITY_CREATION,
↪  !currentNetwork.hasConstraintOfClass(OnlyChanceNodes.class)),
```

**Cómo arreglarlo.** Cambiar el texto a «Create a utility node». Los rótulos y ayudas de este submenú están escritos en el código; pasarlos al fichero de textos es opcional y no hace falta para corregir la errata (nota 120).

Comprobación: tras el cambio, «an utility» no debe aparecer en ningún texto visible de la interfaz.

## Error 186 · [Jorge Rico] El mensaje de evidencia incompatible dice «old evidente» y describe cada hallazgo con su representación interna en lugar de nombrar la variable y el estado

| | |
|---|---|
| **Estado** | Presente. La errata «evidente» se corrigió en el commit `dd3a7bc` (09-10-2026), al corregir el error 48; lo demás sigue igual |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.exception |
| **Ficheros** | `core/src/main/resources/core/localize/core_class_localizations_en.xml:219-222,337-338`<br>`core/src/main/java/org/openmarkov/core/exception/IncompatibleEvidenceException.java:26-34`<br>`core/src/main/java/org/openmarkov/core/model/network/EvidenceCase.java:108-117` |

| | |
|---|---|
| | `inference/src/main/java/org/openmarkov/inference/algorithm/variableElimination/tasks/VEPropagation.java:260-266` `gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EditorInputHandler.java:260-269` `gui/src/main/java/org/openmarkov/gui/dialog/UnexpectedThrowableDialog.java:35-39,51,101-110` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 48, error 184, error 187 |

**Qué pasa.** Cuando un hallazgo contradice a otro sobre la misma variable, OpenMarkov lanza `IncompatibleEvidenceException$EvidenceIsIncompatibleWithOther` y la ventana de error enseña su mensaje, que sale del fichero de textos en inglés (`core_class_localizations_en.xml`, línea 220): «The new evidence {newFinding} is not compatible with an old evidente: {oldFinding}».

El usuario ve dos fallos en ese texto:

- «evidente» en lugar de «evidence»;
- los dos huecos no se rellenan con la variable y el estado, sino con la descripción genérica de un objeto `Finding` (línea 338: «Finding of variable {variable} with numerical value {numericalValue} and state index {stateIndex}»), que a su vez describe la variable con la lista entera de sus estados.

Camino del usuario: abrir `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx`, entrar en modo inferencia (funciona) y hacer doble clic en el estado «dead» de «State [0]». La tabla de «State [0]» da probabilidad 1 a «primary THR», y antes de propagar `VEPropagation` añade a la evidencia ese hallazgo forzado por el modelo; al juntarlo con el del usuario (`getAllEvidence`, líneas 260-266), `EvidenceCase.addFinding` lanza la excepción. El doble clic la envuelve como error no previsto (`EditorInputHandler`, líneas 270-277), y la ventana de error no previsto pone el mensaje al principio de su texto; por el diálogo «Add finding» sale en una ventana de error normal titulada «Evidence is incompatible with other». Pasa lo mismo con los otros estados de «State [0]» y, en `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`, `MID-Chancellor-new.pgmx`, `DMHEE/MID-dmhee-3.5.pgmx` y `DMHEE/MID-dmhee-4.8.pgmx`. Que la evidencia imposible acabe en una ventana de error, y en cuál, es error 48; esta ficha trata solo del texto.

Medido con un pequeño programa de prueba que hace la propagación del modo inferencia con ese hallazgo en `MID-hip-Briggs.pgmx` y pide a la excepción su título y su mensaje:

```
The new evidence Finding of variable Finite states variable State [0] with states [state
↪  dead,
state successful revised, state revision THR, state successful primary, state primary THR]
↪  with
numerical value NaN and state index 0 is not compatible with an old evidente: Finding of
↪  variable
Finite states variable State [0] with states [state dead, state successful revised, state
↪  revision
THR, state successful primary, state primary THR] with numerical value NaN and state index
↪  4
```

Con dos hallazgos contradictorios puestos directamente en un `EvidenceCase` sobre «X-ray» de `0_miscellaneous/networks/bn/BN-asia.pgmx` el formato es el mismo.

**El código.** core/src/main/resources/core/localize/core_class_localizations_en.xml:219-222

```
<Localization class=⌟
↪  "org.openmarkov.core.exception.IncompatibleEvidenceException$EvidenceIsIncompatibleWithOther"
              value="The new evidence {newFinding} is not compatible with an old evidente:
↪  {oldFinding}"/>
<Localization class=⌟
↪  "org.openmarkov.core.exception.IncompatibleEvidenceException$FindingVariableIsMissingAState"
              value="The variable {variable.getName} has no state {state}, so this finding
↪  cannot be translated."/>
```

core/src/main/resources/core/localize/core_class_localizations_en.xml:337-338

```
<Localization class="org.openmarkov.core.model.network.Finding"
              value="Finding of variable {variable} with numerical value {numericalValue}
↪  and state index {stateIndex}"/>
```

**Cómo arreglarlo.** Reescribir el texto de la línea 220 para que nombre la variable y los dos estados, con la misma forma de hueco encadenado que ya usa la línea 222 (`{variable.getName}`); por ejemplo, «The finding {newFinding.getVariable.getName} = {newFinding.getState} contradicts the finding {oldFinding.getVariable.getName} = {oldFinding.getState} already in the evidence». Para un hallazgo sobre una variable numérica, `getState` no da el valor observado: si ese caso importa, el texto necesita otro hueco para el número (`getNumericalValue`).

Es el único texto que usa `{newFinding}` y `{oldFinding}`; la descripción genérica de `Finding` (línea 338) no hace falta tocarla para esto.

Si antes se resuelve error 48 con una excepción nueva y un mensaje propio para la evidencia imposible, ese mensaje debe nombrar también variable y estado, y este texto dejará de verse en los casos que ella cubra.

Prueba: una prueba de `core` que construya `EvidenceIsIncompatibleWithOther` con dos hallazgos sobre «X-ray» y compruebe que `getExceptionMessage()` contiene «X-ray», «yes» y «no», y no contiene «evidente» ni «state index».

## Error 187 · Cuando ninguna muestra de la exportación de la propagación estocástica es compatible con la evidencia, la ventana de error dice «Samples weigth is 0»

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.exception |
| **Ficheros** | `core/src/main/resources/core/localize/core_class_localizations_en.xml:223-224,463-464`<br>`core/src/main/java/org/openmarkov/core/exception/IncompatibleEvidenceException.java:57-63`<br>`inference/src/main/java/org/openmarkov/inference/algorithm/likelihoodWeighting/StochasticPropagation.java:179-181`<br>`stochasticPropagationOutput/src/main/java/org/openmarkov/stochasticPropagationOutput/StochasticPropagationOutputFrame.java:194-202`<br>`gui/src/main/resources/gui/localize/Menus_en.xml:269-272` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 184, error 186 |

**Qué pasa.** El menú Tools → «Export stochastic propagation data» abre la ventana «Stochastic propagation output», que muestrea la red bayesiana abierta con la evidencia puesta en el modo inferencia y guarda las muestras en una hoja de cálculo. Si ninguna muestra es compatible con la evidencia, al pulsar «Export to a .xlsx file» sale una ventana de error titulada «Samples weight is zero» con el mensaje «Samples weigth is 0»: «weigth» en lugar de «weight».

Camino del usuario: abrir `0_miscellaneous/networks/bn/BN-asia.pgmx`, entrar en modo inferencia y poner los hallazgos VisitToAsia = yes, Tuberculosis = yes y X-ray = no (la propagación exacta funciona); abrir la herramienta, elegir «Logic sampling», dejar las 10 000 muestras de omisión y pulsar «Export to a .xlsx file». `StochasticPropagation.getPosteriorValues` lanza `IncompatibleEvidenceException.SamplesWeightIsZero` cuando el peso acumulado de las muestras es cero; `StochasticPropagationOutputFrame.actionPerformed` la envuelve en `UnrecoverableException`, y el manejador global la enseña en una ventana de error normal, con el título fabricado a partir del nombre de la clase y el mensaje del fichero de textos.

El fichero `core_class_localizations_en.xml` tiene dos entradas para esa excepción:

- líneas 463-464, con el nombre de clase correcto (`SamplesWeightIsZero`) y el texto «Samples weigth is 0»: es la que se ve;
- líneas 223-224, con el nombre de clase mal escrito (`SamplesWeigthIsZero`) y el texto «Samples weigth is zero»: no corresponde a ninguna clase y no se ve nunca.

Medido con un pequeño programa de prueba que repite las llamadas de la ventana con esos hallazgos y 10 000 muestras, veinte veces por algoritmo (la ventana no fija la semilla): con «Logic sampling», 17 de las 20 ejecuciones lanzan la excepción, con título «Samples weight is zero» y mensaje «Samples weigth is 0»; con «Likelihood weighting», que es el algoritmo marcado por omisión, ninguna.

En `gui/src/main/resources/gui/localize/Menus_en.xml` (línea 271) hay otra errata, «An error ocurred while trying to process command:» («ocurred» por «occurred»), en la clave `Tools.Plugin.Error`. Ningún código lee esa clave ni su vecina `Tools.Plugin.NotAvailable` (buscado en las fuentes y en las clases del jar completo), así que no se ve.

**El código.** core/src/main/resources/core/localize/core_class_localizations_en.xml:223-224

```
<Localization
↪  class="org.openmarkov.core.exception.IncompatibleEvidenceException$SamplesWeigthIsZero"
            value="Samples weigth is zero"/>
```

core/src/main/resources/core/localize/core_class_localizations_en.xml:463-464

```
<Localization
↪  class="org.openmarkov.core.exception.IncompatibleEvidenceException$SamplesWeightIsZero"
            value="Samples weigth is 0"/>
```

**Cómo arreglarlo.** Corregir el texto de las líneas 463-464 («Samples weight is 0») y borrar la entrada de las líneas 223-224, que no corresponde a ninguna clase. Si se aprovecha para mejorar el mensaje, puede decir que ninguna muestra es compatible con la evidencia y que se pruebe con más muestras o con «Likelihood weighting».

En `Menus_en.xml`, corregir «ocurred» o quitar el bloque `Tools.Plugin`, que nadie usa.

Prueba: una prueba de `core` que construya `SamplesWeightIsZero` y compruebe que `getExceptionMessage()` no contiene «weigth»; y, si se quiere cubrir las demás, una que busque «weigth» y «ocurred» en los ficheros `*_en.xml` de todos los módulos.

### Error 188 · [Jorge Rico] La ayuda emergente de «Unique superparent», en las opciones del clasificador de superpadre, dice «commont» en vez de «common»

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | learning.gui |
| **Paquete** | org.openmarkov.learning.algorithm.nbderived.spnb.gui |
| **Ficheros** | `learning.gui/src/main/resources/learning/gui/localize/Learning_en.xml:154-156`<br>`learning.gui/src/main/java/org/openmarkov/learning/algorithm/nbderived/spnb/gui/SuperParentNaiveBayesParametersDialog.java:30-31` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 184, error 185 |

**Qué pasa.** En Herramientas («Tools») → «Learning», con el tipo «Discriminative» y el algoritmo «Superparent naive bayes», el botón «Options» abre la ventana «SPNB Algorithm : Options». Al dejar el ratón sobre el rótulo «Unique superparent» aparece la ayuda emergente «If selected only the superparent would be commont to all features»: «commont» en lugar de «common».

`SuperParentNaiveBayesParametersDialog` (módulo `learning.gui`, líneas 30-31) pone como ayuda del rótulo el texto de la clave `Learning.SuperParentNaiveBayes.SameSP.Tooltip`, que está en `Learning_en.xml`, línea 155.

Comprobado leyendo el código y pidiendo la clave a `StringDatabase`, que devuelve ese texto; la ventana no se ha abierto. «commont» no aparece en ningún otro fichero de textos.

**El código.** learning.gui/src/main/resources/learning/gui/localize/Learning_en.xml:154-156

```
<SameSP value="Unique superparent ">
    <Tooltip value="If selected only the superparent would be commont to all features"/>
</SameSP>
```

**Cómo arreglarlo.** Cambiar «commont» por «common» en la línea 155 de `Learning_en.xml`. No hace falta prueba.

### Error 205 · Sin ninguna red abierta, la entrada del menú Edit que cambia de modo dice «Switch to edition mode», como si se estuviera en modo inferencia

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.menutoolbar.menu; org.openmarkov.gui.window |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/menutoolbar/menu/MainMenu.java:231,289,626-632`<br>`gui/src/main/java/org/openmarkov/gui/window/MainPanelMenuAssistant.java:361-367`<br>`gui/src/main/resources/gui/localize/Menus_en.xml:70,149` |
| **Comprobado** | Leyendo el código; visto en una captura automática del menú |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 201 |

**Qué pasa.** Con el programa recién arrancado y ninguna red abierta, la entrada del menú Edit que cambia de modo dice «Switch to edition mode» (apagada). No se está en ningún modo, y el modo en que se abre toda red es el de edición, así que lo que cabría ofrecer es pasar al de inferencia.

Visto el 8-10-2026 en capturas automáticas del menú: sin red, «Switch to edition mode (off)»; con `BN-asia.pgmx` abierta en modo edición, «Switch to inference mode».

Por qué pasa: el menú tiene una sola entrada para los dos sentidos del cambio, y `MainMenu` la crea con el texto de la clave `Inference.SwitchToEdition` (línea 231). El texto correcto para cada modo lo pone `MainPanelMenuAssistant.updateOptionsNetworkDependent` (líneas 361-367), que solo se ejecuta cuando hay una red.

**Cómo arreglarlo.** Crear la entrada con el texto de `Edit.SwitchToInference`, que es el del modo en que se abre una red.

Prueba: construir el menú sin redes y comprobar el texto de la entrada.

### Error 208 · El selector del criterio de un nodo de utilidad dice «Cost criterion named …», y los que eligen una opción en el análisis de coste-efectividad dicen «state no» y «state yes», en vez del nombre

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui, costEffectiveness, sensitivityAnalysis |
| **Paquete** | org.openmarkov.gui.dialog.node; org.openmarkov.costEffectiveness; org.openmarkov.sensitivityanalysis.dialog; org.openmarkov.gui.desnets |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/node/NodeDefinitionPanel.java:753-772`<br>`costEffectiveness/src/main/java/org/openmarkov/costEffectiveness/CEDecisionResults.java:698-701`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/CEProbabilisticDialog.java:412-415`<br>`sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/TornadoSpiderDialog.java:706-710`<br>`gui/src/main/java/org/openmarkov/gui/desnets/CEDESDialog.java:401-404`<br>`core/src/main/java/org/openmarkov/core/model/network/Criterion.java:171-173`<br>`core/src/main/java/org/openmarkov/core/model/network/State.java:85-87`<br>`core/src/main/resources/core/localize/core_class_localizations_en.xml:334-335,351` |
| **Comprobado** | Leyendo el código y ejecutándolo; visto en capturas automáticas de dos de las ventanas |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 179, nota 132 |

**Qué pasa.** Dos familias de selectores enseñan la descripción interna de un objeto donde debería ir su nombre:

- en la pestaña Definition de un nodo de utilidad, el selector «Decision criterion» dice «Cost criterion named ---» para el criterio que se llama «---» (en una red con costes y efectividad, «Cost criterion named Cost» y «Effectiveness criterion named Effectiveness»). Visto el 8-10-2026 en una captura automática, con `ID-mediastinet-1-0.pgmx`;

- en la ventana de resultados del análisis de coste-efectividad por decisión (`CEDecision Results`), el selector que acompaña a «Relative to:» dice «state no» y «state yes». Visto en otra captura, con `ID-CEA-test-2therapies-new-test.pgmx`. Por lectura, le pasa lo mismo a los selectores de opción de `CEProbabilisticDialog`, `TornadoSpiderDialog` y `CEDESDialog`, que se llenan igual.

Por qué pasa: esos selectores se llenan con los objetos `Criterion` y `State` y los pintan con su `toString()`, que devuelve el texto de la clase en el fichero de textos: `{ceCriterion} criterion named {criterionName}` y `state {name}`. Medido: `new Criterion("---").toString()` da «Cost criterion named ---» y `new State("no").toString()`, «state no».

`CESpiderDialog` no lo sufre: pone a su selector un pintor que pide el nombre (`new JComboBox FunctionRender<>(State::getName)`, línea 576), y en su captura se lee «yes» y «no».

**Cómo arreglarlo.** Poner el mismo pintor a los demás: `JComboBoxFunctionRender<>(State::get Name)` en los cuatro selectores de opción y `JComboBoxFunctionRender<>(Criterion::get CriterionName)` en el del criterio. No conviene cambiar `toString()`, que usan los mensajes de registro.

Prueba: construir cada selector sin ventana y comprobar que el componente que pinta la primera entrada lleva el nombre del estado o del criterio.

### Error 212 · El diálogo en que se pide un análisis de sensibilidad se abre con la barra de título vacía

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | sensitivityAnalysis |
| **Paquete** | org.openmarkov.sensitivityanalysis.dialog |
| **Ficheros** | `sensitivityAnalysis/src/main/java/org/openmarkov/sensitivityanalysis/dialog/SensitivityAnalysisDialog.java:99-101` `sensitivityAnalysis/src/main/resources/sensitivityanalysis/localize/sensitivityanalysis_en.xml:3-4` |
| **Comprobado** | Leyendo el código; visto en capturas automáticas de la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 180, error 181, error 92 |

**Qué pasa.** Tools → «Sensitivity analysis» (o el botón de la barra) abre, tras las opciones de inferencia, el diálogo en que se eligen el tipo de análisis y los parámetros. Su barra de título está vacía.

Visto el 8-10-2026 en las capturas automáticas de la ventana, con todas las redes con que se abrió (entre ellas `BN-asia.pgmx`, `ID-decide-test.pgmx` e `ID-CEA-test-2therapies-new-test.pgmx`): el título es siempre la cadena vacía.

Por qué pasa: el constructor de `SensitivityAnalysisDialog` llama a `super(owner)` y nadie llama a `setTitle`, ni en la clase ni en sus bases (es el mismo caso que los errores 180 y 181). Las ventanas de resultados sí se titulan, con «OpenMarkov - Sensitivity analysis - » y el nombre de la red.

**Cómo arreglarlo.** Añadir en el constructor `setTitle(StringDatabase.getUniqueInstance(). getString("SensitivityAnalysis.Title"))`, que hoy da «Sensitivity analysis» y es la clave que ya usan las ventanas de resultados.

Prueba: no se puede construir sin pantalla; con una, comprobar que `getTitle()` no está vacío.

### Error 213 · La ventana de opciones de la evolución temporal se titula «Temporal Evolution of:» seguido del nombre de la red sin espacio en medio

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.dialog.inference.temporalevolution |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TemporalEvolutionDialog.java:103-104`<br>`gui/src/main/java/org/openmarkov/gui/dialog/inference/temporalevolution/TraceTemporalEvolutionDialog.java:859-860`<br>`gui/src/main/resources/gui/localize/Dialogs_en.xml:8` |
| **Comprobado** | Leyendo el código; visto en una captura automática de la ventana |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |

**Qué pasa.** Al pedir la evolución temporal de un nodo se abre primero una ventana de opciones (número de ciclos). Su título es «Temporal Evolution of:MID-Chancellor-corrected.pgmx», con el nombre pegado a los dos puntos.

Visto el 8-10-2026 en capturas automáticas con `MID-Chancellor-corrected.pgmx` y con otra red temporal.

Por qué pasa: `TemporalEvolutionDialog` (línea 103) une el texto de la clave `TemporalEvolutionResultDialog.Title`, «Temporal Evolution of:», con el nombre de la red sin nada entre los dos. La ventana de resultados, `TraceTemporalEvolutionDialog` (línea 859), usa la misma clave y sí pone el espacio: «Temporal Evolution of: State».

Además, lo que la ventana de opciones nombra es la red, y lo que se va a calcular es la evolución de una variable.

**Cómo arreglarlo.** Poner el espacio, y decidir si la ventana de opciones debe nombrar la variable, como la de resultados.

Prueba: no se puede construir sin pantalla; con una, comprobar el título.

## 17 Rendimiento

Trabajo de más en el camino que más se recorre, y operaciones largas que bloquean la ventana.

### Error 189 · [Jorge Rico] Entrar en modo inferencia con una red grande deja la ventana entera congelada durante decenas de segundos, sin progreso ni forma de cancelar

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | gui |
| **Paquete** | org.openmarkov.gui.window.edition.networkEditorPanel |
| **Ficheros** | `gui/src/main/java/org/openmarkov/gui/window/MainPanelListenerAssistant.java:147-164,181-182`<br>`gui/src/main/java/org/openmarkov/gui/window/InferenceHandler.java:73-134`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:324-363,580-599` |

| | |
|---|---|
| | `gui/src/main/java/org/openmarkov/gui/util/GUIUtils.java:58-82`<br>`gui/src/main/java/org/openmarkov/gui/window/MainGUI.java:108-118`<br>`gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/NetworkEditorPanel.java:472-479` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Medio (días) |
| **Decisión de equipo** | **Sí.** Qué hacer con `MainGUI.freeze()/unfreeze()` (borrarlo o reactivarlo) |
| **Relacionados** | nota 105, nota 167, nota 176, nota 177 |

**Qué pasa.** Entrar en modo inferencia con `integrationTests/src/main/resources/networks/bn/BN-cpcs422b.pgmx` (422 nodos) deja la ventana sin responder unos 29 segundos en la máquina donde se midió (más en una lenta), y cada hallazgo nuevo otro tanto. No hay barra de progreso ni botón de cancelar, y el usuario no sabe si el programa se ha colgado. Lo mismo ocurre al propagar evidencia (`PROPAGATE_EVIDENCE`, líneas 188-189 de `MainPanelListenerAssistant`) y, con propagación automática, al añadir o quitar cada hallazgo, que repite la propagación para cada caso de evidencia en memoria.

Swing tiene un único hilo que pinta la ventana y atiende los clics (el EDT, *Event Dispatch Thread*). Todo lo que se ejecuta dentro de un `actionPerformed` ocupa ese hilo, y mientras dura la ventana no se repinta ni responde.

En OpenMarkov, pulsar el botón de modo inferencia llega a `MainPanelListenerAssistant.actionPerformed` (líneas 155-171) → `InferenceHandler.toggleWorkingMode/setWorkingMode` → `NetworkEditorPanel.updateIndividualProbabilitiesAndUtilities` → `EvidenceManager.updateIndividualProbabilitiesAndUtilities` (líneas 328-360), que llama a `doPropagation` para cada caso de evidencia (líneas 580-599), y esta construye un `VEPropagation` y le pide todas las probabilidades a posteriori. Todo en el mismo hilo: `GUIUtils.executeUIAction` (líneas 57-81) solo envuelve excepciones, no crea ningún hilo. `doPropagation` captura `OutOfMemoryError`, pero no hay forma de interrumpir un cálculo largo.

Medido con un pequeño programa de prueba que hace la misma llamada que `doPropagation` (un `VEPropagation` con todas las variables de interés, sin evidencia): tarda 330 ms en `integrationTests/src/main/resources/networks/bn/BN-cpcs179.pgmx` (179 nodos) y **28 688 ms** en `integrationTests/src/main/resources/networks/bn/BN-cpcs422b.pgmx` (422 nodos). Leer el fichero tarda 150-500 ms, así que abrir no es el problema. No se ha arrancado la aplicación: el tiempo es el del cálculo que la ventana ejecuta en su hilo, medido fuera de ella.

Aparte, `MainGUI.freeze()` y `unfreeze()` empiezan con `if (true) return;` (líneas 108-118), así que no hacen nada. Su único uso es alrededor del diálogo modal de propiedades del nodo (`NetworkEditorPanel.java:472-482`); no está relacionado con la propagación y desactivado no produce ningún efecto visible. Hay que decidir si se borra o se reactiva.

**El código.** gui/src/main/java/org/openmarkov/gui/window/edition/networkEditorPanel/EvidenceManager.java:580-588

```
public void doPropagation(EvidenceCase evidenceCase, int caseNumber) throws ... {
    Map<Variable, TablePotential> individualProbabilities = null;
    try {
        this.calculateMinAndMaxUtilityRanges();
        Propagation vePosteriorValues = new
↪   VEPropagation(this.networkEditorPanel.getVisualNetwork().getProbNet());
        vePosteriorValues.setVariablesOfInterest(this.networkEditorPanel.getVisualNetwork
↪   ().getProbNet().getVariables());
        vePosteriorValues.setPreResolutionEvidence(this.preResolutionEvidence);
        vePosteriorValues.setPostResolutionEvidence(evidenceCase);
        individualProbabilities = vePosteriorValues.getPosteriorValues();
```

gui/src/main/java/org/openmarkov/gui/window/MainGUI.java:108-118

```
public void freeze() {
    if (true) return;
    this.frameMirror.freeze();
}

//Conditionally disabled
public void unfreeze() {
    if (true) return;
    this.frameMirror.unfreeze();
}
```

**Cómo arreglarlo.** Ejecutar el cálculo fuera del hilo de la interfaz con un `SwingWorker` (el patrón que ya usa `SplashScreenLoader`): el trabajador construye la propagación y calcula; al terminar, `done()` pinta los resultados (`InferencePresenter.paintInferenceResults`) y actualiza menús en el hilo de la interfaz. Mientras dura, un diálogo de progreso con cancelar.

Conviene un único servicio para todos los caminos que llaman a `doPropagation`:

- entrar en modo inferencia (`InferenceHandler.setWorkingMode`);
- propagar evidencia (`NetworkEditorPanel.propagateEvidence`);
- añadir o quitar hallazgos con propagación automática;
- cambiar de caso de evidencia (`InferenceHandler.evidenceCasesNavigationOption`).

Cuidado: la red no debe poder editarse mientras el trabajador lee de ella, y el cálculo trabaja sobre la copia del algoritmo, que comparte potenciales con la red del usuario. La cancelación de verdad necesita que la tarea de inferencia la admita; esa propuesta de diseño está en nota 167, donde no hay nada que corregir.

Prueba: con `integrationTests/src/main/resources/networks/bn/BN-cpcs422b.pgmx`, comprobar (por ejemplo con AssertJ-Swing) que durante la propagación el hilo de la interfaz sigue atendiendo eventos.

### Error 190 · [Jorge Rico] Cada resultado de una operación sobre tablas reserva y rellena una tabla entera que tira acto seguido

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:55-75` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 191, error 192 |

**Qué pasa.** Toda operación de la inferencia sobre tablas gasta tiempo en preparar una tabla que no usa. Con tablas grandes, el desperdicio es del orden de una cuarta parte de la operación.

Las operaciones (`multiply`, `sum`, `multiplyAndMarginalize`, `divide`, `evaluateFunctionPotential`, la fusión…) calculan primero el array de resultados y luego construyen el potencial con `new TablePotential(variables, papel, tabla)`. Ese constructor empieza por `this(variables, role)`, que hace `values = new double[tableSize]` (la máquina virtual pone a cero todas las casillas) y

`setUniform()` (dos rellenos más si el papel es de probabilidad, ver error 191). Después comprueba la longitud de la tabla recibida y hace `this.values = table`, tirando el array recién preparado. El tamaño de esas tablas crece exponencialmente con el número de variables.

Medido con un pequeño programa de prueba (tabla de 4 194 304 casillas): construir con papel de probabilidad condicionada cuesta unos 11,8 ms, de los que 5,8 son la copia del array que el propio programa pasa; con papel de utilidad, 6,7 ms. Es decir, alrededor de 6 ms tirados por tabla de probabilidad (reserva más dos rellenos) y alrededor de 1 ms por tabla de utilidad (reserva). Una multiplicación que produce una tabla de ese tamaño tarda unos 23 ms.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:55-75

```
public TablePotential(List<Variable> variables, PotentialRole role) {
    super(variables, role); // AbstractIndexedPotential: computes dimensions, offsets,
    ↪  tableSize
    values = new double[tableSize];
    setUniform(); // Initializes the table as an uniform potential
}

public TablePotential(List<Variable> variables, PotentialRole role, double[] table) {
    this(variables, role);
    if (table.length != tableSize) {
        throw new InvalidArgumentException(table.length, "table",
                "its length must be " + tableSize + ", the product of the numbers of states
                ↪  of the variables");
    }
    this.values = table;
}
```

**Cómo arreglarlo.** Que el constructor de tres argumentos llame a `super(variables, role)` (que calcula dimensiones, desplazamientos y `tableSize`), compruebe la longitud y asigne `this.values = table`, sin reservar ni llamar a `setUniform`.

Revisar las tres subclases de `TablePotential` (`StrategicTablePotential` y `UncertainTablePotential`, que llaman a `super(variables, role, table)`, y `TableWithFunctions`) por si alguna dependía del relleno.

Verificar con el banco de referencia de las operaciones, `OperationsReferenceTimesTest` en `integrationTests` (red cpcs422b), antes y después; la batería entera de pruebas debe pasar sin cambios de valores.

### Error 191 · [Jorge Rico] Cada tabla de probabilidad nueva se rellena dos veces seguidas con el mismo valor

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:465-513` |
| **Comprobado** | Leyendo el código y ejecutándolo |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 190 |

**Qué pasa.** Cada tabla de probabilidad que se construye, incluidas las que produce cada operación de la inferencia, se rellena dos veces seguidas con el mismo valor uniforme. Con tablas grandes es tiempo perdido en el camino más frecuente de la inferencia.

`TablePotential.setUniform()` inicializa una tabla como distribución uniforme. Lo llama el constructor de dos argumentos, es decir, toda tabla que se construye (ver error 190). Cuando la tabla tiene variables, ninguna numérica, y el papel es probabilidad condicionada, política, probabilidad conjunta o restricción de enlace, calcula el valor uniforme y hace `Arrays.fill(values, value)` en la línea 501. Como en esa rama también pone `setValue = true`, al salir del `if` vuelve a hacer `Arrays.fill(values, value)` en la línea 510 sobre el mismo array con el mismo valor. Para tablas de utilidad (papel sin especificar) no se rellena ninguna vez.

Medido con un pequeño programa de prueba (tabla de 4 194 304 casillas, 22 variables binarias): construir `new TablePotential(variables, papel, tabla)` cuesta unos 6 ms más con papel de probabilidad condicionada que con papel de utilidad, y ese exceso son los dos rellenos de este error y la reserva de error 190, juntos. Como referencia, una multiplicación de dos tablas que produce una tabla de ese tamaño tarda unos 23 ms en total, así que el trabajo tirado ronda la cuarta parte. Quitar solo el relleno duplicado ahorra una de las dos pasadas.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/TablePotential.java:471-510

```
if (numVariables > 0 && noNumericVariables() && (
        role == PotentialRole.CONDITIONAL_PROBABILITY || role == PotentialRole.POLICY ||
                role == PotentialRole.JOINT_PROBABILITY || role ==
↪  PotentialRole.LINK_RESTRICTION)) {
    setValue = true;
    value = 0.0;
    switch (role) {
        ...
    } // When role = UTILITY -> value = 0.0 (default)
    Arrays.fill(values, value);
} else if (numVariables == 0) {
    setValue = true;
    ...
}
if (setValue) Arrays.fill(values, value);
```

**Cómo arreglarlo.** Borrar el `Arrays.fill` de la línea 501 (el de la 510 cubre las dos ramas). No cambia ningún valor.

Medirlo con el banco de referencia de las operaciones, `OperationsReferenceTimesTest` en `integrationTests`, antes y después. La batería de pruebas existente de `TablePotential` basta como red de seguridad; no hace falta prueba nueva de comportamiento.

### Error 192 · [Jorge Rico] La maximización reserva una tabla del tamaño de la unión de variables solo para calcular unos desplazamientos

| | |
|---|---|
| **Estado** | Presente |
| **Asignado a** | Jorge Rico |
| **Módulo** | core |
| **Paquete** | org.openmarkov.core.model.network.potential.operation |
| **Ficheros** | `core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialMaximization.java:86-105`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialMaximization.java:206-225` |

| | |
|---|---|
| | `core/src/main/java/org/openmarkov/core/model/network/potential/AbstractIndexedPotential.java:183-186`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/AbstractIndexedPotential.java:307-309`<br>`core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialElimination.java:113` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 190, nota 90, nota 93, nota 98, nota 162 |

**Qué pasa.** Absorber un nodo de decisión desde el menú (`AbsorbNodeEdit` → `NodeAbsorptionHandler`, línea 111) ejecuta `TablePotentialMaximization.multiplyAndMaximize` (línea 98), que reserva y pone a cero un array del tamaño de la unión de variables sin usarlo. El tamaño de la unión es el de la utilidad del hijo, así que es un gesto de edición, no el camino caliente de la inferencia. No se ha medido.

El algoritmo de desplazamientos acumulados recorre varias tablas a la vez sin construir la tabla conjunta: solo necesita, para cada operando, cuánto avanza su posición cuando cambia cada variable de la unión. `multiplyAndMaximize` y `multiplyAndMaximizeUniformly` obtienen esos desplazamientos construyendo `new TablePotential(unionVariables, null)` y llamando a su método de instancia `getAccumulatedOffsets(potential.getVariables())`, que se limita a llamar a la versión estática `AbstractIndexedPotential.getAccumulatedOffsets(this.variables, otras)`.

Construir ese potencial reserva un array `double[]` del tamaño de la unión (la variable que se maximiza por las que se conservan), que es justo la tabla que el algoritmo existe para no construir. Como el papel es `null`, `setUniform` no lo rellena: el coste es la reserva y puesta a cero de ese array. `TablePotentialElimination` (línea 113) ya usa la versión estática.

`multiplyAndMaximizeUniformly` (línea 218) no tiene llamadores de producción, solo pruebas.

**El código.** core/src/main/java/org/openmarkov/core/model/network/potential/operation/TablePotentialMaximization.java:98-105

```
TablePotential unionPotential = new TablePotential(unionVariables, null);
for (int i = 0; i < numProperPotentials; i++) {
    TablePotential potential = properPotentials.get(i);
    tables[i] = potential.getValues();
    initialPositions[i] = potential.getInitialPosition();
    currentPositions[i] = initialPositions[i];
    accumulatedOffsets[i] = unionPotential.getAccumulatedOffsets(potential.getVariables());
}
```

**Cómo arreglarlo.** Sustituir en los dos sitios por `accumulatedOffsets[i] = TablePotential.getAccumulatedOffsets(unionVariables, potential.getVariables());` y borrar `unionPotential`. No cambia valores; la prueba existente de maximización basta.

Si se borra `multiplyAndMaximizeUniformly` (el código sin llamadores se recoge en nota 162, donde no hay nada que corregir), queda un solo sitio.

## 18 Exportación a Amua

El exportador de árboles de decisión al formato del programa Amua (paquete `org.openmarkov.io.amua`, fachada `AmuaExporter`), al que se llega desde el menú contextual del árbol de decisión, «Export to Amua», desde el commit `76e6996`. Estos errores se comprobaron contra el código del

commit `4e21edf`; el 30 de septiembre se volvieron a comprobar contra `a0f0cd6`, sin cambios en el paquete entre los dos.

### Error 195 · [Corregido] Al exportar a Amua un árbol de coste-efectividad cuya red declara el criterio de efectividad antes que el de coste, el coste y la efectividad de cada rama van a la dimensión equivocada

| | |
|---|---|
| **Estado** | Corregido en el commit `635e0a7` (14-10-2026), con el primer camino de «Cómo arreglarlo»: `AmuaDTWriter` escribe el `<cost>` y el `<payoff>` de cada nodo poniendo el coste en la posición `costDim` y la efectividad en `effectDim`. La exportación de coste-efectividad solo acepta dos criterios, coste y efectividad (`AmuaDTValidator.hasCECriteria`), así que no hay más dimensiones que rellenar. Comprobado con pruebas del escritor en los dos órdenes, no abriendo el fichero en Amua |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.amua |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/amua/adatper/AmuaDTAssignDimensions.java:41-50`, `io/src/main/java/org/openmarkov/io/amua/writer/AmuaDTWriter.java:212-241` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 196 |

**Qué pasa.** El fichero de Amua lleva una lista de dimensiones y, en cada nodo, un `<cost>` y un `<payoff>` por dimensión, en el mismo orden. `AmuaDTAssignDimensions.assignDimensions` escribe las dimensiones en el orden de `getDecisionCriteria()` y fija `costDim` y `effectDim` según ese orden. En cambio, `AmuaDTWriter.writeNodeCost` y `writeNodePayoff` escriben siempre primero el coste y después la efectividad.

Si la red declara los criterios en el orden efectividad, coste, la dimensión que se llama como la efectividad recibe el coste y viceversa, y el análisis de coste-efectividad de Amua sale con los papeles cambiados, sin aviso. Las pruebas del paquete usan siempre el orden coste, efectividad, así que no lo ven.

**Cómo arreglarlo.** Escribir los valores de cada nodo en el orden de las dimensiones: el coste en la posición `costDim` y la efectividad en `effectDim`, pasando `AmuaDTDimensions` a los dos métodos del escritor. Una alternativa más sólida es guardar en el nodo un `double[]` por dimensión desde la conversión. Prueba: un árbol con los criterios en orden efectividad, coste, comprobando que el `<payoff>` de la posición `effectDim` de una hoja es su efectividad.

### Error 196 · [Corregido] Al exportar a Amua un árbol de coste-efectividad, el escenario base que se escribe es la primera opción de la decisión, no la de menor coste, salvo que las opciones sean hojas

| | |
|---|---|
| **Estado** | Corregido en el commit `d33fb70` (14-10-2026), como dice «Cómo arreglarlo»: `AmuaDTConverter.assignUtilities` pone como utilidad parcial de todo nodo su coste y su efectividad evaluados; el *payoff* sigue solo en las hojas, y la utilidad parcial no se escribe en el fichero, así que solo cambia el escenario base. Comprobado con la red nueva `integrationTests/src/main/resources/networks/id/ID-CEA-cheaper-option-second.pgmx`, cuya opción más barata es la segunda y lleva a un nodo de azar, exportando el árbol como la ventana; y con un empate en coste, que gana la opción más efectiva. Ninguna red del repositorio con coste-efectividad lo mostraba: las que tienen nodos de azar |

| | |
|---|---|
| | tras la decisión tienen más de una decisión, y Amua solo admite una |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.amua.adatper |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/amua/adatper/AmuaDTConverter.java:157-185`, `io/src/main/java/org/openmarkov/io/amua/adatper/AmuaDTAssignDimensions.java:119-149` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 195 |

**Qué pasa.** `calcBaseScenario` elige como escenario base la opción de menor coste (y, si empatan, la de mayor efectividad) mirando la utilidad parcial (`getPartialUtility()`) de cada hijo de la decisión. Pero `AmuaDTConverter.assignUtilities`, en los árboles de coste-efectividad, solo copia el coste y la efectividad evaluados en los nodos de utilidad (tipo 2); en los demás deja la utilidad parcial a (0, 0).

Las opciones de una decisión llevan normalmente a nodos de azar, así que todas empatan a (0, 0) y gana siempre la primera. En los árboles unicriterio, en cambio, la utilidad parcial se copia en todos los nodos. La prueba `assignDimensionsCETest` rellena a mano las utilidades parciales y solo comprueba que el escenario base no sea nulo.

**Cómo arreglarlo.** En `assignUtilities`, poner como utilidad parcial el coste y la efectividad evaluados de cualquier nodo, como ya se hace en el caso unicriterio, y dejar el *payoff* solo en las hojas. Prueba: un árbol convertido de verdad (no montado a mano) con dos opciones que llevan a nodos de azar de costes distintos, con la más barata en segundo lugar.

### Error 197 · Si las probabilidades de las ramas de un nodo de azar no suman uno, el exportador a Amua las sustituye todas por 1/n sin avisar, y el modelo exportado da otros resultados

| | |
|---|---|
| **Estado** | Presente |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.amua.adatper |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/amua/adatper/AmuaDTConverter.java:89-133` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | Sí: qué hacer con unas probabilidades que no suman uno (rechazar la exportación, o normalizar si el error es pequeño como se decidió en el error 5 y rechazar si es mayor) |
| **Relacionados** | error 5, error 198 |

**Qué pasa.** Después de convertir los hijos de un nodo de azar, `convertToAmuaTree` suma sus probabilidades y, si la suma se aparta de uno más de $10^{-4}$, pone todas a $1/n$. No lo avisa ni lo anota en ningún sitio. Además, una probabilidad `NaN` en una rama se cambia antes por 0 (líneas 93-96), así que una sola rama `NaN` basta para que todo el nodo pase a uniforme. El fichero de Amua describe entonces un modelo distinto del de OpenMarkov.

No se ha buscado una red del repositorio en la que la evaluación del árbol deje probabilidades que no sumen uno; la consecuencia se ha deducido leyendo el código.

**Cómo arreglarlo.** Llevar la comprobación a `AmuaDTValidator`, para que el usuario reciba el mensaje de árbol no válido con el nodo afectado, y aplicar lo que decida el equipo. En ningún

caso cambiar las probabilidades sin avisar.

### Error 198 · [Corregido] El exportador a Amua escribe las probabilidades, los costes y las efectividades redondeados a cuatro decimales: un valor por debajo de 0,00005 sale como 0 y tres ramas de 1/3 suman 0,9999

| | |
|---|---|
| **Estado** | Corregido en el commit `8032155` (14-10-2026), con el criterio que fijó Manuel Arias ese día: lo que se exporta a Amua tiene que poder leerlo Amua, sin suponer cómo funciona. Por eso se leyó el código de Amua (`github.com/zward/Amua`, commit `fcce8eb`). Lee cada número con `Double.parseDouble` (`math/Token.java`); exige que las probabilidades de las ramas de un nodo de azar sumen exactamente 1.0, sin tolerancia, salvo que una sea «C», el complemento, y entonces las demás tienen que sumar entre 0 y 1 (`tree/DecisionTree.java`, `checkModel`); y una rama de nodo de azar sin probabilidad le da una excepción al comprobar el modelo, porque el constructor que usa al leer el fichero no le da valor. Así que con cuatro decimales tres ramas de 1/3 ya no se podían leer, y con toda la precisión tampoco habría garantía. `AmuaDTWriter` escribe ahora todos los números con toda su precisión, con punto decimal y sin exponente (`BigDecimal.toPlainString`), así que Amua lee el mismo número, y lo de la nota 52 queda resuelto: las pruebas, que corren con el sistema en castellano, escribían `0,3333` antes del arreglo. En cada nodo de azar escribe «C» en la rama más probable, con lo que las demás suman claramente menos de uno; y escribe la probabilidad de toda rama de un nodo de azar, también cuando vale 0. Comprobado con pruebas del escritor, no abriendo el fichero en Amua |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.amua.writer |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/amua/writer/AmuaDTWriter.java:29`, `io/src/main/java/org/openmarkov/io/amua/writer/AmuaDTWriter.java:187-241` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 197, nota 52 |

**Qué pasa.** `AmuaDTWriter` formatea las probabilidades y, en los árboles de coste-efectividad, los costes y las efectividades con un `DecimalFormat("#.####")` estático. Todo valor se escribe con cuatro decimales como mucho: una probabilidad o una efectividad pequeñas se convierten en 0, y las ramas de un nodo de azar pueden dejar de sumar uno exactamente (tres ramas de 1/3 dan 0,3333 cada una). No se ha comprobado cómo reacciona Amua a una suma de 0,9999.

En los árboles unicriterio, en cambio, el coste y el *payoff* se escriben con `String.valueOf` y toda su precisión. Los cuatro decimales de `DEFAULT_DECIMALS` son en Amua solo los decimales con que se muestran los resultados, no una precisión de los datos.

**Cómo arreglarlo.** Escribir todos los números con `Double.toString`, que conserva la precisión y usa siempre el punto decimal, lo que resuelve también lo de la nota 52. Prueba: exportar un nodo de azar con tres ramas de 1/3 y una efectividad de $10^{-6}$ y releer los valores del XML.

### Error 199 · [Corregido] Un árbol de coste-efectividad cuya decisión no está en la raíz ni en el primer nivel pasa la validación del exportador a Amua, pero se rechaza con el mensaje «Decision node not found»

| | |
|---|---|
| **Estado** | Corregido en el commit `e5eb89c` (15-10-2026): en un árbol de coste-efectividad, `AmuaDTValidator` exige que la decisión sea el primer nodo, porque Amua toma como estrategias las opciones de la raíz, sea de la clase que sea (`AmuaModel.getStrategies`, en Amua 0.3.4). Si no lo es, la exportación se niega con un mensaje que nombra la decisión y el nodo con el que empieza el árbol; un árbol sin decisión se niega con un mensaje propio. Esto rechaza también la decisión en el primer nivel, que antes se exportaba sin aviso y con un escenario base que no era ninguna de las estrategias que vería Amua. Ninguna red del repositorio llega a estos casos: con un nodo de azar por encima, la decisión se repite en cada rama y el exportador ya lo rechazaba porque Amua admite una sola decisión; hace falta una red asimétrica con la decisión en una sola rama. Con prueba en `io` de los tres casos y del árbol que empieza por la decisión, que se sigue exportando. La excepción del escenario base sin opciones no se ha tocado: sigue sin ser alcanzable. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.amua.adatper |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/amua/adatper/AmuaDTAssignDimensions.java:119-172`, `io/src/main/java/org/openmarkov/io/amua/adatper/AmuaDTValidator.java:103-125`, `io/src/main/java/org/openmarkov/io/amua/AmuaExporter.java:80-100` |
| **Comprobado** | Leyendo el código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 196 |

**Qué pasa.** `AmuaDTValidator` acepta la única decisión del árbol a cualquier profundidad. Pero `getDecisionNode`, que usa el cálculo del escenario base, solo la busca en la raíz y en sus hijos directos. Si hay dos nodos de azar antes de la decisión, el árbol pasa la validación y después se rechaza con «Decision node not found», un mensaje que contradice al árbol que el usuario tiene delante.

Además, `calcBaseScenario` lanza `IllegalArgumentException` si la decisión no tiene opciones, y `AmuaExporter.isValidDTForAmua` solo captura `IllegalStateException`, así que ese caso saldría como error no previsto. Hoy no parece alcanzable, porque una decisión del árbol tiene siempre opciones.

**Cómo arreglarlo.** Comprobar en Amua dónde admite el nodo de decisión en un análisis de coste-efectividad y exigir eso mismo en `AmuaDTValidator`, con un mensaje que diga dónde está la decisión y dónde tendría que estar. Si Amua la admite a cualquier profundidad, que `getDecisionNode` la busque en todo el árbol. Hacer que el cálculo del escenario base lance la misma excepción que el resto de la validación.

### Error 244 · [Corregido] Al exportar a Amua un árbol de coste-efectividad cuya red no da unidad a uno de los criterios, el fichero lleva dos nombres de dimensión y un solo símbolo, y Amua, que cuenta las dimensiones por los símbolos, se queda sin sitio para la efectividad

| | |
|---|---|
| **Estado** | Corregido en el commit `3a93386` (15-10-2026): `AmuaDTAssignDimensions` da a cada dimensión como símbolo la unidad de su criterio, o un texto vacío si no la tiene, así que el fichero lleva siempre tantos `<dimSymbols>` como `<dimNames>`. Comprobado leyendo los ficheros con las clases de Amua 0.3.4, como los lee su ventana al abrirlos: con `ID-CEA-minimal.pgmx` y «Effectiveness» sin unidad, los símbolos eran `[€]` y ahora son `[€, ]`; la lectura no avisa de nada en ninguno de los dos casos. No se ha visto lo que Amua muestra al calcular ese árbol. Con prueba en `integrationTests` de ese mismo caso. |
| **Módulo** | io |
| **Paquete** | org.openmarkov.io.amua.writer; org.openmarkov.io.amua.adatper |
| **Ficheros** | `io/src/main/java/org/openmarkov/io/amua/writer/AmuaDTWriter.java:105-118,300-304`<br>`io/src/main/java/org/openmarkov/io/amua/adatper/AmuaDTAssignDimensions.java:41-43`<br>`io/src/main/java/org/openmarkov/io/probmodel/reader/PGMXReader_0_2.java:343-346` |
| **Comprobado** | Leyendo el código y ejecutándolo (el exportador); lo que hace Amua, leído en su código |
| **Esfuerzo** | Pequeño (horas) |
| **Decisión de equipo** | No |
| **Relacionados** | error 195, error 198 |

**Qué pasa.** El usuario abre una red de coste-efectividad con una sola decisión, abre la ventana del árbol de decisión y elige «Export to Amua» en el menú contextual del árbol. Si la red no da unidad a uno de sus dos criterios, el fichero `.amua` sale con un `<dimSymbols>` de menos:

```
<dimNames>Cost</dimNames>
<dimNames>Effectiveness</dimNames>
<dimSymbols>€</dimSymbols>
<decimals>4</decimals>
<decimals>4</decimals>
```

Amua lee las tres listas como vectores (`@XmlElement public String dimNames[], dimSymbols[]` en `main/DimInfo.java`) y cuenta las dimensiones del modelo **por los símbolos**: `DecisionTree.parseTree` empieza con `int numDim=myModel.dimInfo.dimSymbols.length;` y dimensiona con ese número los costes y los pagos de cada nodo. Así que recibe un modelo de coste-efectividad (`costDim` 0, `effectDim` 1) de una sola dimensión, y la efectividad de cada coste y de cada pago se queda sin sitio. Esto está leído en el código de Amua (`github.com/zward/Amua`, commit `fcce8eb`), no ejecutado.

Por qué pasa: `AmuaDTAssignDimensions.assignDimensions` toma como símbolo de cada dimensión la unidad de su criterio (`c.getCriterionUnit()`, línea 43), que puede ser nula, y `AmuaDTWriter.writeAmuaDimInfo` escribe cada símbolo con `addElement` (línea 114), que no escribe nada cuando el valor es nulo (líneas 300-304). Los nombres y los decimales nunca son nulos, así que las tres listas dejan de tener la misma longitud.

La unidad es nula cuando el `.pgmx` no trae el atributo `unit` en el `<Criterion>`: `PGMXReader_0_2.getDecisionCriterion` lo lee tal cual (líneas 343-346), y `PGMXWriter_0_2` tampoco lo escribe cuando es nulo. Un criterio creado desde la ventana nace con la unidad «—» (`Criterion`, línea 29), y vaciarla en la tabla de criterios deja, por lo leído en `DecisionCriteriaTablePanel`, un

texto vacío, que sí se escribe (`<dimSymbols></dimSymbols>`). Entre las redes de `0_miscellaneous/networks` hay nueve de coste-efectividad con criterios sin unidad, pero ninguna llega al exportador: con las ocho que son MID no se construye el árbol de decisión (`NotApplicableNetwork`), y `DAN-decide-test-ce-1-0.pgmx` se rechaza con «Amua supports only one decision node». Se llega con un fichero escrito a mano o por otro programa.

Medido el 15-10-2026 con el exportador del programa sobre `0_miscellaneous/networks/id/ID-CEA-minimal.pgmx`, con el árbol de la ventana (profundidad 5): el original da los dos símbolos, `€` y `QALY`; quitando el atributo `unit="QALY"` del criterio `Effectiveness`, aparte del nombre del modelo, la única diferencia en el fichero es que desaparece `<dimSymbols>QALY</dimSymbols>`, como arriba.

**El código.** io/src/main/java/org/openmarkov/io/amua/writer/AmuaDTWriter.java:110-118

```
        for (AmuaDimensionInfo d : dimensions) {
            addElement(dimInfo, "dimNames", d.getName());
        }
        for (AmuaDimensionInfo d : dimensions) {
            addElement(dimInfo, "dimSymbols", d.getSymbols());
        }
        for (AmuaDimensionInfo d : dimensions) {
            addElement(dimInfo, "decimals", d.getDecimals());
        }
```

**Cómo arreglarlo.** Escribir siempre un `<dimSymbols>` por dimensión, vacío si el criterio no tiene unidad: lo más limpio es que `AmuaDTAssignDimensions` ponga `""` cuando `getCriterionUnit()` es nulo, para que el modelo intermedio ya no lleve nulos. Las tres listas de `DimInfo` (nombres, símbolos y decimales) deben tener siempre la misma longitud, que es el número de criterios.

Prueba: exportar el árbol de `ID-CEA-minimal.pgmx` con el criterio `Effectiveness` sin unidad y comprobar que el fichero lleva dos `<dimSymbols>`, el segundo vacío, tantos como `<dimNames>`.

# A Lo que se comprobó y no entra en la lista

Estas notas recogen todo lo que se comprobó al preparar la lista y quedó fuera de ella. No hay nada que corregir en ellas; están para que nadie tenga que volver a mirarlo y para que, si algún día cambia algo (por ejemplo, un código que hoy no llama nadie empieza a usarse), se sepa qué había.

## Ya corregidos

**1** **Reparto de una columna que se queda a cero al restringir un enlace**
Corregido en el commit `fd17916`: `LinkRestrictionPotentialOperations.redistributeProbabilities` reparte ahora la columna a partes iguales entre los estados que siguen permitidos, sea cual sea su número.

**2** **Deshacer una absorción de nodo dejaba al hijo sin potenciales**
Corregido en el commit `ca779d4`: `AbsorbNodeEdit.doEdit` guarda ya los potenciales viejos y nuevos que usan `undo` y `redo`.

**184** **Pegar un nodo con un enlace a un nodo no copiado dejaba un nodo a medias**
Corregido en el commit `2c4fca6` (issue 663): `VisualNetwork.exportToClipboard` copia ya solo los enlaces cuyos dos extremos están entre los nodos seleccionados, y es el único sitio que llena el portapapeles de «Copy» y «Cut». Era el error 32 de la versión anterior de esta lista.

## No son errores

**3** **La poda de nodos estériles declara estéril a un padre con un solo hijo sin comprobarlo**

En `ProbNetOperations.removeBarrenNodes`, el único hijo de ese padre es, por construcción, el nodo estéril desde el que se llega a él, así que la comprobación que se omite daría siempre «sí». El atajo está dentro del método y ningún otro llamador puede romper esa condición.

**4** **Cambiar una decisión con política por un nodo de azar vacía potenciales que no existen**

`TaskUtilities.replaceDecisionsWithPoliciesByChanceNodes` recorre para quitarlos los potenciales de un nodo recién creado, que no tiene ninguno, así que el bucle no hace nada. El nodo queda con la política como único potencial, que es lo que se quiere; solo sobra una línea engañosa.

**5** **Los mapas locales de `resetLink` guardan restricciones y revelaciones que nadie lee**

`VariableStateOperations.resetLink` copia las restricciones de enlace y las condiciones de revelación en dos mapas locales que mueren al salir del método: es código muerto sin consecuencia propia, porque `NodeStateEdit` ya guarda y repone esos datos (commit `f7081df`). La pérdida al deshacer un dominio estándar es un defecto de `NodeReplaceStatesEdit`, descrito en error 24, y no se arregla rescatando estos mapas.

**6** **El tipo de análisis guardado en el fichero se lee al abrirlo**

`PGMXReader_0_2.getMulticriteriaOptions` lee `<SelectedAnalysisType>` y fija el tipo en las opciones de la red, y `PGMXReader_1_0` hereda ese método. Medido: `integrationTests/src/main/resources/networks/IDCEAnTherapies/DAN-1tests.pgmx`, `0_miscellaneous/networks/dan/DAN-CE-2-test-problem.pgmx` y `0_miscellaneous/networks/id/ID-CEA-minimal.pgmx` se abren como coste-efectividad.

**7** **La lognormal se muestrea con una inversa aproximada de la normal y se acota con la exacta**

`LogNormalFunction.getSample` usa la aproximación de Odeh y Evans y `getInterval` la inversa exacta de Apache Commons Math, pero las dos difieren como mucho en unas cienmillonésimas para las probabilidades que da el generador en la práctica. Medido con un millón de extracciones: la fracción de muestras dentro del intervalo del 95 % es 0,95007.

**8** **La tabla de cuentas de la información mutua condicionada no sigue su orden declarado**

`ConditionalMutualInformationMetric.buildAbsoluteFreqCrossTab` rellena la tabla con un orden de variables distinto del declarado, pero el único método que la lee, `score`, la recorre con ese mismo orden. El número resultante es la información mutua condicionada correcta; medido contra un cálculo independiente sobre los casos.

**9** **El algoritmo PC consulta dos veces la caché con la misma llave al orientar colisionadores**

`ColliderOrientation.createsContradictoryCollider` busca `new NodePair(from, existingParent)` y luego `new NodePair(existingParent, from)`, pero `NodePair` ordena sus dos nodos por nombre y las dos llaves son iguales. La segunda consulta sobra, pero el resultado del algoritmo PC de aprendizaje de estructura es correcto.

**10** **Pedir la red modelo sin marcar posiciones ni estructura no deja el aprendizaje sin red**

`LearningManager.applyModelNet` devuelve `null` en ese caso, pero solo se llama cuando `ModelNetUse.isUseModelNet()` es verdadero, y el constructor de `ModelNetUse` lo pone a falso si no se marca ninguna de las dos opciones. `LearningManager` crea entonces una red con un nodo por variable de la base de casos.

**11 La letra negrita de 30 puntos de la tabla de probabilidades nunca llega a la pantalla**
`ValuesTableCellRenderer.setCellFonts` fija esa letra, pero justo después `Default TableCellRenderer.getTableCellRendererComponent` la sustituye por la de la tabla. Medido: todas las casillas salen con la letra de la tabla y sin texto recortado; `setCell Fonts` es código sin efecto.

**12 Una columna con incertidumbre no se pinta de rojo en un nodo con restricciones de enlace**
`ValuesTableWithLinkRestrictionCellRenderer.getCellColors` pone fondo rojo a toda casilla no editable, pero para una columna con incertidumbre `ValuesTableCell Renderer.getTableCellRendererComponent` devuelve la etiqueta del icono de incertidumbre, con su propio fondo. Medido: solo sale roja la casilla que el enlace prohíbe.

**13 Un número mal escrito en la tabla de probabilidades se avisa con una ventana**
La excepción que lanza `ValuesTable.setValueAt` llega a `OMExceptionHandler`, que abre la ventana «Mismatched value» con lo tecleado; medido con «0,3», «abc» y «-0.3». Que no se admita la coma decimal queda fuera por la decisión de que la aplicación sea solo en inglés, y `OpenMarkov.main` fija el formato de números inglés.

**14 Los textos con huecos usan dos convenciones, tilde y llaves numeradas**
Las claves con huecos `~` se rellenan con `StringDatabase.getFormattedString` y las de huecos `{0}` con `MessageFormat`; comprobadas todas las claves inglesas con huecos, cada una va por su mecanismo y ningún texto sale sin rellenar. Tener dos convenciones es una incomodidad de mantenimiento, no un fallo.

**15 Propagar un dominio a otros ciclos crea varias ediciones, pero se deshacen de una vez**
`NodePropertiesDialog` abre una sub-historia de ediciones al construirse y, al aceptar, la cierra como una sola entrada `ListPNEdit`, así que un único «deshacer» devuelve todos los ciclos; comprobado ejecutando con `0_miscellaneous/networks/mid/MID-mammography.pgmx`. Lo que sí se pierde al deshacer es el contenido de las tablas, que es error 24.

**16 La suma de utilidades conserva los planes de decisión**
La suma de potenciales conserva los árboles de estrategia, así que no hay ningún defecto de OpenMarkov que corregir.

**17 El editor de árboles rechaza las operaciones imposibles con un mensaje claro**
`TreeADDEditorPanel.actionPerformed` envuelve esas excepciones en `Unrecoverable Exception`, y `OMExceptionHandler` enseña un diálogo con su mensaje sin cerrar la aplicación; las excepciones se lanzan antes de modificar el árbol. Medido: quitar todos los estados de una rama, partir un intervalo por un límite imposible y salir del dominio dan los tres un mensaje claro en inglés, aunque con un título mal formado (error 182). Un umbral que no es un número, en cambio, sí sale como error no previsto (error 161).

**18 La entrada «Settings…» del menú Edición sí tiene texto**
La clave `Edit.OpenSettings` está en `gui/src/main/resources/gui/localize/Tool Bars_en.xml` y `StringDatabase` junta todos los ficheros de textos, así que la opción del menú se lee «Settings…». Solo le falta la clave del mnemónico (la letra subrayada que permite elegirla con el teclado), igual que a otras entradas del mismo menú: no tiene atajo de letra, pero no hay texto ilegible.

**19 La herramienta de poner sucesos no enseña ninguna clave sin texto**
La ayuda del botón de sucesos pide `Edit.Mode.Event.ToolTip`, que existe en `ToolBars_ en.xml`, y la barra de edición y el menú contextual del lienzo escriben «Event» en el código. La clave `Edit.Mode.Event`, que no tiene texto, no se pide para enseñarla; la tabla `NodeModeDescriptor` que la usaba se retiró en el commit `8055530`.

**20 La coherencia «débil» pedida por la validación cruzada no llega a los resultados**

`CrossValidationDialog` pide la plantilla de medidas con `Coherence.WEAK`, pero `LearningEvaluator.runEvaluator` construye el resultado con el constructor de copia de `MeasuresSet`, que marca todas las variables como usadas. Medido: los resultados no muestran la frase sobre variables sin evidencia, que en la validación cruzada sería falsa; el `Coherence.WEAK` escrito a mano queda como código engañoso sin efecto.

**21 El paseo guiado «First tour» señala el nodo correcto en el segundo cambio de nombre**

Los pasos eligen el nodo con `getVisualNetwork().getAllNodes().get(1)`, y `Network EditorPanel.doPaint` reordena esa lista en cada repintado poniendo primero los nodos seleccionados: tras renombrar B, que queda seleccionado, la lista es [C, A] y `get(1)` es A. Medido el mecanismo con un pequeño programa de prueba; elegir el nodo por una posición que depende de la selección es frágil, pero hoy acierta.

**22 Rehacer un nodo añadido conserva lo que puso `addNodeConsistently`**

`AddNodeEdit.redo` solo vuelve a insertar el nodo, pero el potencial por omisión, la política y la posición viven en el propio objeto `Node`, que `undo` saca del grafo sin tocarlo. Medido en un diagrama de influencia: deshacer y rehacer devuelve los nodos idénticos.

**23 Deshacer un enlace añadido no tiene restricciones de enlace que reponer**

Un enlace recién añadido no tiene restricciones ni estados reveladores, y `DistinctLinks` y `NoMultipleLinks` impiden añadirlo encima de otro, así que `AddLinkEdit.undo` no tiene nada que reponer. Las restricciones que se pongan después las introducen ediciones propias, con su propio deshacer.

**24 Enlazar varios nodos a la vez omite los pares ya enlazados y los lazos prohibidos**

`MultiAddLinkEdit` descarta los pares que ya tienen ese enlace o que serían un lazo prohibido, que es justo el resultado que el usuario pide; durante el arrastre, `Visual Network.updateLinkCreation` pinta esas flechas en gris o transparentes. Avisarlo de forma más visible sería un cambio de interfaz, no la corrección de un fallo.

**25 Comprobar restricciones en cada movimiento del ratón al trazar enlaces cuesta muy poco**

`VisualNetwork.updateLinkCreation` comprueba las restricciones de cada enlace candidato en cada movimiento del ratón. Medido sobre `integrationTests/src/main/resources/networks/bn/BN-cpcs422b.pgmx` (422 nodos, 867 enlaces): 0,14 ms de media con diez enlaces candidatos y 2,6 ms en el peor caso, sin contar el repintado, muy por debajo de lo que se nota.

**26 El recolector de violaciones de restricciones no quita repetidos, pero el mensaje final sí**

`ConstraintChecker` guarda las excepciones en un `HashSet` que no las funde, pero `MultipleConstraintsViolateds` las agrupa por su texto, así que un mensaje idéntico sale una sola vez (medido con `NoMultipleLinks`). Que `NoCycle` nombre cada enlace de un ciclo es información, no un repetido, y hoy solo lo ve quien abre desde un fichero una red con ciclo (error 37).

**27 Varias operaciones copian la lista de variables de cada potencial dentro de sus bucles**

`Potential.getVariables()` devuelve una copia en cada llamada y varios bucles la piden una vez por potencial, no por casilla, así que el coste es pequeño y acotado. Es una observación de rendimiento, no un fallo; si algún día se busca velocidad, dar acceso de solo lectura a la lista interna sería una vía barata.

**28** **Tres modelos con «Age [0]» uniforme no se analizan sin darle un valor: lo exige la red**

En `0_miscellaneous/networks/mid/MID-CHD-Walker.pgmx`, `MID-Colorectal.pgmx` y `MID-mammography.pgmx`, «Age [0]» es un nodo numérico sin distribución que el modelo espera recibir como hallazgo, y sin él no se puede discretizar: `MID-Colorectal.pgmx` guarda ese hallazgo en el fichero y su coste-efectividad global termina (medido: un tramo, coste 1837 y efectividad 23,11). `MID-mammography.pgmx` no lo guarda, y con «Age [0] = 50» falla más adelante por error 13; `MID-CHD-Walker.pgmx` es un esqueleto cuyos siete nodos de utilidad son uniformes, sin números que calcular. — cambios —

**189** **El superpadre desempata por la última candidata, y el Bayes ingenuo selectivo por la primera**

`SuperParentNBAlgorithm.getOptimalEdit` elige el superpadre con `addScore >= best ParentScore` (línea 104), así que entre candidatas de igual exactitud gana la última; `SelectiveNBAlgorithm` hace ganar a la primera desde el commit `1bd8c38` (error 135). Las candidatas se recorren en el orden de los nodos de la red (`MetricUtils.getNonRoot Nodes`), que es fijo, así que el modelo aprendido es el mismo en cada ejecución: son dos reglas de desempate distintas, no un resultado que cambie. Comprobado leyendo el código.

**190** **La lista de unidades de las opciones de inferencia sale en el orden de un `HashMap`**

`InferenceOptionsDialog.getUnitsPanel` (líneas 795-802) llena la lista «Unit» recorriendo un `HashMap<String, String>` con las unidades de los criterios. Las claves son textos, cuyo `hashCode` es el mismo en cada ejecución, así que el orden es fijo, aunque no sea el de los criterios: medido en cuatro redes con el coste en € y la efectividad en QALY (años de vida ajustados por calidad), sale siempre «QALY, €». La unidad elegida al abrir es la que guarda la red (`MulticriteriaOptions.getMainUnit`); la primera de la lista solo se toma si la red no guarda ninguna.

**193** **«Distribution table» se ofrece también fuera de las redes de sucesos**

`DistributionTablePotential.validate` (líneas 128-131) la admite para cualquier nodo numérico de cualquier red desde el commit `d6cfb0e` del antiguo repositorio de `core`, de José Andrés (10-06-2026), que quitó la condición «solo en redes de sucesos», marcada como provisional en la rama de Carmen Yago. No es un error: una tabla con una distribución por cada configuración de los padres es una relación válida para un nodo numérico, y en la propagación exacta de una red bayesiana falla con el mensaje normal, «Potential P(N | P) cannot be converted to a table», igual que «Function», «Uniform» o «UnivariateDistr» (medido para el error 108).

**194** **En las redes de sucesos, la simulación se lanza desde «Cost-effectiveness analysis»**

`InferenceHandler.showCostEffectivenessDeterministic` (líneas 270-276) lanza la simulación si la red es de sucesos, así que se llega a ella por «Cost-effectiveness analysis → Deterministic analysis». La rama de Carmen Yago tenía un botón propio, «DES Simulation»; José Andrés lo unificó a propósito (commit `7913cc4`, «DES simulation button unified and bug fixed»). El texto «DES Simulation» (`MCSimulateNetwork` en `ToolBars_en.xml`) ya no lo usa nadie (nota 195).

**197** **Quitar de una suma de utilidades un término que no nombra la deja como estaba y contesta que lo ha quitado, y eso es lo que conviene**
`DANOperations.dropTermOfSum` (`c38679e`, líneas 225-238), que usa `removeWithDescendants` cuando una rama de una red de análisis de decisiones (DAN) descarta un nodo, quita de la lista de una suma de utilidades la variable del nodo que se va sin mirar si estaba, y contesta que lo ha hecho siempre que a la suma le queden otros términos, aunque su comentario dice que contesta «si lo hizo». Así, si el nodo que se va tiene un enlace hacia una suma que no lo nombra (el enlace está y la relación no lo usa), la suma se vuelve a escribir igual y se queda en la red. Eso es lo correcto, porque esa suma no depende del nodo que se va. Medido el 15-10-2026 sin ventana con una DAN pequeña (D decide, X solo es posible si D = «yes», U depende de X, V depende de D, y S suma solo V aunque U también apunta a S): con D = «no», U se va, S se queda diciendo `[S, V]` y la rama vale 1, lo mismo que cuando S suma también U. En las redes del repositorio hay tres enlaces así, todos en `integrationTests/src/main/resources/networks/dan/DAN-nested-sum-sv-non-tree.pgmx` (U1→U5, U4→U6 y U5→U0), y esa red solo tiene nodos de utilidad, sin decisiones ni restricciones, así que no hay ninguna rama que los alcance. Como mucho, el comentario debería decir que contesta si la suma se queda. Viene del arreglo del error 200.

**198** **Las opciones de la tabla y del CSV de una simulación salen en el orden de un `HashMap`, no en el que declara la decisión**
`SimulationSummaryResults` guarda cada serie de la simulación de sucesos en un `HashMap<String, CriteriaValues>` cuya clave es el nombre de la opción (`addEmptySeries`, líneas 67-74), y `toString` lo recorre con `entrySet` (líneas 202-208); la tabla de la ventana de resultados (`DESResultsWindow.generateTable`) y el CSV toman de ese texto qué opciones hay y en qué orden, así que las columnas salen en el orden que el mapa dé a los nombres y no en el que declara la decisión. Medido el 15-10-2026 armando los resultados con las clases reales y llamando sin ventana a `generateTable` y a `csv`: con «none, drug», «none, drug, surgery», «untreated, treated» y «watchful waiting, surgery» sale el orden declarado; con «therapy 1, therapy 2» sale primero «therapy 2»; con «Standard, New», primero «New»; con «no treatment, treatment», primero «treatment»; y con «placebo, drug A, drug B», «placebo» va la última. Cada número va bajo la opción que es —por eso la ficha del error 102 lo daba por inofensivo—, y como las claves son textos el orden es el mismo en cada ejecución: lo único que se ve es que la tabla puede no seguir el orden en que el modelo declara las opciones, y que ese orden cambia con solo renombrar una. El plano de coste-efectividad (`CEDESDialog`) no se ve afectado, porque recorre los estados de la decisión. Es de la familia de la nota 190. Bastaría con un `LinkedHashMap` rellenado en el orden de los estados de la decisión, o con que `toString` recorra esos estados.

## Defectos sin consecuencia visible hoy

**29** **El javadoc de la comparación de motivos del algoritmo PC dice lo contrario que el código**
`PCEditMotivation.compareTo` hace ganar al conjunto de separación más pequeño, que es el orden por tamaño creciente del algoritmo PC de aprendizaje de estructura publicado, mientras que su javadoc dice que gana el más grande. Solo el comentario está mal; el código hace lo correcto.

**30 Tres búsquedas del editor de árboles comparan nombres con «==» en vez de `equals`**

`TreeADDEditorPanel.addSubtree`, `associateStates` y `removeVariablesFromPotential` comparan con `==` el texto elegido por el usuario con el nombre de la variable o del estado, lo que compara la identidad de los objetos y no su contenido. Hoy funciona porque, medido, ese texto es el mismo objeto `String` que el nombre; es un defecto latente que aparecería si el texto llegara como otro objeto con el mismo contenido.

**31 El orden de las entradas empatadas del menú Tools depende del descubrimiento de clases**

`MainMenu.buildToolsMenu` ordena los complementos solo por prioridad, y las seis entradas empatadas quedan en el orden en que `PluginLoader` las descubre. En el jar ejecutable ese orden es fijo; solo al ejecutar desde directorios de clases depende de `File.listFiles`, cuyo orden no está definido, y no se ha observado ningún cambio real.

**32 La red auxiliar que enseña la política óptima de una decisión duplica cada enlace**

`NetworkEditorPanel.showOptimalPolicyOfNode` añade el potencial de la política con `ProbNet.addPotential`, que ya traza los enlaces, y después los vuelve a trazar con `addLink`, que no comprueba duplicados; medido con `0_miscellaneous/networks/dan/DAN-dating.pgmx` e `integrationTests/src/main/resources/networks/id/ID-decide-test-sv.pgmx`. Esa red no se dibuja y el diálogo solo enseña el potencial, así que no hay consecuencia visible, aunque cualquier código que recorriera sus enlaces los contaría de más.

**33 El generador de bases de casos propone 5000 casos y el comentario del código dice 1000**

`DBGeneratorGUI` hace `caseNumber.setSelectedIndex(2); // 1000 cases`, y el índice 2 de la lista es «5000». El número está a la vista en la ventana, así que el usuario no queda engañado; lo que hay es una contradicción entre comentario y código, y decidir cuál de los dos vale.

**185 Multiplicar deja fuera las variables de un solo estado de los potenciales constantes**

`TablePotentialArithmetic.multiply` trata como constante cualquier potencial de una sola casilla y deja fuera del producto sus variables, que tienen un estado cada una; por lo mismo, el árbol de estrategia de un potencial de una casilla con variables no llega al resultado. La suma, la marginalización, la maximización y la evaluación de fórmulas solo tratan como constante el potencial sin variables, y conservan esas variables. No cambia ningún número, porque una variable de un estado no añade ninguna dimensión a la tabla. Comprobado el 23 de septiembre en las 165 redes distintas del repositorio que tienen nodos numéricos de azar o son de tipo DAN (propagación, evaluación, políticas, árbol de estrategia, coste-efectividad y evaluación de DAN, con evidencia en los nodos numéricos y sin ella): hoy ningún resultado lleva una variable de un estado. Conservarlas rompe dos cosas que hoy funcionan: «Absorb parents» sobre un producto con dos padres de regresión sin padres propios (error 193) y la utilidad esperada de `DAN-dating.pgmx`, que sale con el mismo valor pero con la variable `U mExp` (nota 186). Era lo que quedaba del error 1.

**186 La evaluación de una DAN deja una utilidad a cero sin criterio, que cuenta como probabilidad**

Cuando una restricción deja un nodo sin estados posibles y la utilidad que habría que quitar es la última de la red, `DANOperations.replaceUtilityPotentialWithZero` (líneas 231-238, llamado en las líneas 136 y 198) la deja con una tabla a cero sobre su propia variable y sin criterio. Sin criterio, la eliminación de variables la cuenta como una probabilidad: `VariableEliminationCore.getProbability` (línea 322) la multiplica con las demás. Pasa al evaluar `DAN-dating.pgmx`, y el resultado es el correcto: `DAN EvaluationTest.testDANDatingAskNo` fija a mano 9,407584 y la evaluación lo da. No se ha comprobado si ese cero cambia el resultado de otra red. Hoy el producto deja fuera la variable de esa utilidad (nota 185); si la conservara, la utilidad esperada de `DAN-dating` saldría con la variable `U mExp`.

**187 Eliminar una decisión de la que no depende ninguna utilidad deja en cero la probabilidad de la evidencia**

Cuando ninguna utilidad depende de la decisión que se elimina, `DecisionVariable Elimination` suma una lista vacía, que `TablePotentialArithmetic.sum` convierte en una constante cero, y el máximo que sale es una constante cero sin criterio (medido: `StrategicTablePotential` de valor 0, no aditivo). Sin criterio, la eliminación de variables la cuenta como una probabilidad y la multiplica con las demás; es el mismo mecanismo de la nota 186. Medido el 9 de octubre sin ventana: `VEEvaluation.get Probability()` da 0, sin ningún hallazgo, en `DAN-dating.pgmx` y `DAN-reactor.pgmx`, y 1 en las otras 47 redes con decisiones de `0_miscellaneous/networks/id` y `0_ miscellaneous/networks/dan` que se leen. No se ha encontrado nada que un usuario vea por su causa: las utilidades no cambian, el árbol de decisión de las dos redes sale con probabilidad 1 en la raíz y probabilidades coherentes en las ramas, y ninguna ventana enseña esa probabilidad. No se ha comprobado si ese cero puede cambiar el peso de una rama en otra red.

**188 La exactitud del Bayes ingenuo suma sus logaritmos en un orden que depende de la memoria**

`Accuracy.scoreNB` reúne las características en un `HashSet<Variable>`, y `Variable` no redefine `hashCode`, así que `predictClassValue` las recorre en un orden que puede cambiar de una ejecución a otra. Lo que acumula al recorrerlas es una suma de logaritmos, y una suma en coma flotante depende del orden en su último decimal. Solo podría cambiar la clase predicha en un empate casi exacto entre dos clases. Comprobado leyendo el código; no se ha observado ningún cambio (la medida de 80 clasificadores hecha el 8 de octubre al portar OpenMarkov a C++/Qt dio los mismos números en dos ejecuciones). Es de la familia del error 135 y de la segunda fuente de variación del error 136.

**191 El orden de la lista de tipos de red es el del descubrimiento de clases**

`NetworkDefinitionPanel.getJComboBoxNetworkTypes` llena el selector del tipo de red con `NetworkTypeUtils.NETWORK_TYPE_CLASSES` sin ordenar, en el orden en que `Plugin Search` encuentra las clases; es lo mismo que la nota 31, en otra lista. Visto el 8 de octubre en una captura automática, ejecutando desde directorios de clases: los catorce tipos salen sin orden reconocible (LIMID, MDP, …, con la red bayesiana en el puesto doce). No se ha mirado el orden en el jar publicado. Nada falla: el tipo elegido al crear una red se fija aparte, y es la red bayesiana.

**196** **La copia de un enlace comparte con el original su tabla de restricciones y sus listas de revelación, y `Link` gana un `equals` sin `hashCode`**
`Link.clone`, nuevo en `461cd46` (líneas 377-387), construye una copia con su propia tabla de restricciones y sus propias listas de estados e intervalos reveladores y luego no la devuelve: devuelve `super.clone()`, la copia superficial de `Object.clone`, que apunta a la misma tabla y a las mismas listas que el original (medido el 15-10-2026 sin ventana sobre `NodeStatesReplaceTest.pgmx`: la tabla de restricciones de la copia es el mismo objeto que la del original). Su único llamador es `NodeReplaceStatesEdit` (línea 54), que guarda esas copias para deshacer un cambio de dominio, y hoy no se pierde nada: `VariableStateOperations.resetLink` no modifica la tabla ni las listas de un enlace, sino que las sustituye (`setRestrictionsPotential(null)` y listas nuevas), así que lo guardado queda intacto, y en la misma medida deshacer devuelve las restricciones de los cuatro enlaces de la red tal como estaban. Se notaría si algo modificara en el sitio la tabla o las listas de un enlace entre el cambio y su deshacer: `getRevealingStates` y `getRevealingIntervals` entregan la lista viva a propósito, para `RevelationIntervalEdit`. En el mismo commit `Link` gana un `equals` por contenido sin su `hashCode`, contra lo que dice el comentario de la clase (líneas 34-37): que la igualdad es por identidad a propósito, porque `Graph.removeLink` cuenta con ello. Leído en el código: no se ha encontrado nada que compare dos enlaces distintos con el mismo contenido, ni que guarde enlaces en una tabla de dispersión y los busque con otro objeto (los mapas de `NodeStateEdit` y de `resetLink`, nota 5, buscan con el mismo), así que tampoco esto tiene hoy consecuencia. Es de la familia del error 24.

## Defectos que hoy no alcanza nadie

**34** **Tras guardar, el indicador de red modificada de `PNESupport` sigue diciendo que sí**
`PNESupport.onSave` no recalcula `networkIsModified`, que sigue valiendo `true` hasta la edición siguiente. Su único lector, `NetworkEditorPanel`, lo consulta solo después de ejecutar, deshacer o rehacer una edición, cuando ya se ha recalculado; y el asterisco de la pestaña, el botón de guardar y la pregunta al cerrar dependen del indicador propio del panel, que se pone a falso al guardar.

**35** **`RemovePotentialsEdit` no se puede deshacer**
La edición hereda el `undo()` vacío de `PNEdit`, así que quitar potenciales con ella no se podría deshacer (el mismo problema en `PotentialChangeEdit` ya se corrigió en el commit `b81524f`). Ningún menú, diálogo ni algoritmo la construye: solo aparece en su propio fichero, en el fichero de textos y en el catálogo de pruebas de deshacer, que la aparta por no tener llamadores.

**36** **Subir el primer criterio de decisión o bajar el último se sale de la lista**
`DecisionCriteriaEdit.doEdit` no comprueba los extremos en las acciones `UP` y `DOWN`, y llamada así lanza `IndexOutOfBoundsException`. Desde la ventana no se llega: `DecisionCriteriaTablePanel` es el único llamador, `KeyTablePanel.valueChanged` apaga «subir» en la primera fila y «bajar» en la última, y tras cada movimiento se pierde la selección.

**37** **`TemporalEvaluation` no arranca desde el jar completo de la aplicación**
`TemporalEvaluation.resolve` usa `LogManager.getLogger()` sin argumentos, que desde `OpenMarkov.jar` lanza `UnsupportedOperationException` porque el manifiesto del jar completo no declara `Multi-Release: true`; con el jar normal de log4j funciona. Hoy nadie llega: `TemporalEvaluation` no tiene llamadores de producción y solo la usan pruebas, que corren con el jar normal.

**38 La copia superficial de una red recupera las restricciones de su tipo que se quitaron**

`ProbNetCopier.shallowCopy` crea la copia con las restricciones del tipo de red y le añade las del original, así que una restricción quitada vuelve (medido con `Distinct Links`: la copia no pasa `checkConstraints()` y el original sí). En producción solo `SuperParentNBAlgorithm` quita una restricción del tipo, y no se ha encontrado nada que lea después las restricciones de una copia de esa red.

**39 Una propagación de Hugin reutilizada con otra evidencia mezcla la evidencia vieja**

`ClusterPropagation.introduceEvidence` añade los potenciales de la evidencia nueva sin vaciar los anteriores, y `collectEvidence` reutiliza el mensaje guardado de la vez anterior; medido sobre `0_miscellaneous/networks/bn/BN-asia.pgmx`, la segunda pregunta no da la respuesta correcta, [0,99, 0,01]. Los dos llamadores de producción construyen un objeto nuevo en cada consulta o fijan toda la evidencia antes de la primera pregunta, así que ningún usuario ve un número equivocado.

**40 Convertir otra camarilla en raíz del árbol de Hugin lanzaría siempre una excepción**

`HuginForest.convertToRoot` espera `null` de `Graph.getParents` y de `Stack.pop`, que en realidad devuelven una lista vacía y lanzan excepción, así que acabaría siempre en `IndexOutOfBoundsException` o `EmptyStackException`; además empieza el recorrido por la raíz vieja y no por `newRoot`. El método no tiene ningún llamador en el proyecto, ni en producción ni en pruebas.

**41 `StrategyManager` lee la política de una decisión con las elecciones de otra**

`StrategyManager.getPotentialForm` calcula mal dónde empiezan las elecciones de cada decisión cuando las decisiones no tienen el mismo número de combinaciones de padres, y `evaluate` busca la política por número de ciclo y no por decisión. La clase no tiene ningún llamador en el proyecto; su paquete solo aparece exportado en `module-info.java`.

**42 La búsqueda de estrategias por fuerza bruta divide por cero con límites pequeños**

`StrategyManager.bruteForce` lanza `ArithmeticException` con cualquier límite menor que 20, evalúa dos estrategias más de las que promete y lanza `NullPointerException` si ninguna estrategia mejora a la base; `randomWalk` tiene la misma división. Nadie llama a estos métodos.

**43 La búsqueda de estrategias recorre elecciones del último ciclo que la evaluación no usa**

`StrategyManager.orderNodesBySlice` incluye las decisiones del ciclo `horizon`, pero `evaluate` solo recorre hasta `horizon - 1`, así que esas elecciones multiplican el número de estrategias sin cambiar ninguna utilidad. La clase no tiene ningún llamador en el proyecto.

**44 El texto de una partición de coste-efectividad usa la puntuación numérica del sistema**

`CEP` crea sus `DecimalFormat` con la puntuación por omisión del proceso, así que con el sistema en castellano escribe `Eff: 8,5`. La aplicación de escritorio fija el formato inglés en `OpenMarkov.main` antes de crear ninguna partición, de modo que solo lo vería quien use el núcleo como biblioteca sin fijarlo.

**45 La «lambda» de una partición de coste-efectividad cierra todos sus tramos por la derecha**

`CEP.getLambda` crea la lista de lados con un elemento de más, así que todos los tramos quedan abiertos por la izquierda y cerrados por la derecha, al revés de lo que dicen sus nombres de estado «λ in [a , b)». Solo la llama `CEP.getIntervention()` sin argumentos, que no tiene ningún llamador.

**46 Copiar la partición de coste-efectividad de probabilidad cero lanza una excepción**

`CEP.clone` recorre cuatro vectores que en la partición de probabilidad cero son `null`, así que `CEP.getZeroPartition().clone()` lanza `NullPointerException` (medido). La única copia de una partición es la de `CEBaseOperations.cutPartition`, que siempre recibe particiones construidas por `deterministicCEA`, con los vectores llenos.

**47 La heurística híbrida de eliminación no triangula: repite siempre la misma variable**

`HybridElimination.getDeletionSequence` elige la variable sobre una red y borra enlaces en otra sin quitar nunca la elegida, así que repite siempre la misma y el grafo que devuelve no está triangulado; además `isSuperValueNode` declara supervalor un nodo con un solo padre de utilidad. Ningún código de producción construye un `Hybrid Elimination`, y su prueba está comentada.

**48 La evolución temporal de varias utilidades falla si no se fuerza un solo criterio**

El constructor `MIDTemporalEvolution(ProbNet, List<Node>)` deja el objeto en modo de coste-efectividad y `evaluateSlice` lanza `ClassCastException` al tratar la partición como una tabla (medido sobre `0_miscellaneous/networks/mid/MID-Chancellor-corrected.pgmx`). Su único llamador, `TraceTemporalEvolution Dialog`, llama a `forceUnicriterion()` justo después de construirlo, y así el cálculo termina bien.

**49 La evaluación ciclo a ciclo traga la excepción de un potencial no proyectable**

`TemporalEvaluation.resolve` captura `NonProjectablePotentialException`, la registra y evalúa un modelo al que le faltan ese potencial y los siguientes, sin avisar a quien llama. Solo la construyen pruebas de `integrationTests`, y en la práctica ni siquiera llega a ese punto: falla antes al pedir el registro de mensajes (nota 37).

**50 El lector de XMLBIF no lee ningún fichero, ni pone enlaces ni posiciones**

`XMLBIFReader` falla con `NullPointerException` incluso con `io/src/test/resources/netAB.xml`, el fichero de su prueba, porque `PGMXReader_0_2.getVariable` es estático y no usa su forma de leer nombres; además `getLinks` está vacío y no lee las posiciones de los nodos. Su anotación de formato está comentada y ningún llamador lo construye, así que hoy no se alcanza; lo que sí le pasa a quien abre desde «Abrir» un `.xml` de ese formato es error 117.

**51 El exportador de árboles de decisión a Amua no lo usaba ningún menú ni llamador**

El paquete `org.openmarkov.io.amua` (fachada `AmuaExporter`) convierte un árbol de decisión en un fichero del programa Amua. Cuando se hizo la lista no tenía llamadores; desde el commit `76e6996` el menú contextual del árbol de decisión ofrece «Export to Amua», y sus defectos alcanzables pasan a la lista como errores 195 a 199.

**52 El exportador a Amua escribiría los decimales con coma si el sistema está en castellano**

`AmuaDTWriter` usa `new DecimalFormat("#.####")` con los símbolos del idioma del sistema, así que con `es_ES` escribiría `0,8`, que Amua no puede leer. Dentro de la aplicación de escritorio no se ve, porque `OpenMarkov.main` fija el formato numérico inglés al arrancar; solo lo sufriría quien usara el exportador como biblioteca. El redondeo a cuatro decimales del mismo formato sí se ve y es el error 198, cuyo arreglo resuelve también esto. Resuelto con él en el commit `8032155`.

**53 Pedir al algoritmo PC el motivo de una orientación de una sola flecha lanza una excepción**

`PCAlgorithm.getMotivation(PNEdit)` lee siempre la segunda flecha de un `COrient LinksEdit`, y con uno de una sola flecha, que `ColliderOrientation.tryOrient Collider` construye a veces, lanza `NoSuchElementException`. Solo lo llaman pruebas: el diálogo de aprendizaje interactivo toma el motivo que ya lleva cada propuesta, construido correctamente.

**54 La discretización por anchura igual lanza una excepción con una columna no numérica**

`Discretization.calculateVariableMin` y `calculateVariableMax` convierten los nombres de estado con `Double.parseDouble` sin protección, y `Discretization.process` no comprueba si la variable es numérica. Los dos diálogos que la llaman, el de aprendizaje y el de preprocesado de `bnEvaluation`, deshabilitan la discretización y la dejan en «ninguna» para las variables no numéricas.

**55 `Configuration.addFinding` descarta sin aviso el valor de una variable discretizada**

El `switch` de `Configuration.addFinding(Variable, double)` no contempla las variables discretizadas y vuelve sin hacer nada, como dice su javadoc. Sus únicos llamadores, en la simulación de eventos discretos, solo le pasan variables de sucesos y de nodos de azar, y `ChanceRecord` rechaza antes de simular los nodos de azar que no son numéricos ni de estados finitos (cómo llega ese rechazo al usuario se describe en error 106).

**56 Una tabla de funciones sin variables tendría una fórmula nula**

El constructor de `TableWithFunctions` deja la única casilla a `null` cuando la lista de variables está vacía, y evaluarla fallaría dentro del evaluador de expresiones. Su único constructor de producción es el de `TableWithEvents`, que siempre le pasa al menos la variable condicionada.

**57 `TableWithFunctions.equals` no compara las fórmulas**

`TableWithFunctions.equals` compara los números heredados y no las fórmulas, así que dos tablas que solo difieren en una fórmula salen iguales (medido). Ningún código de producción la compara: la tabla solo vive dentro de `TableWithEvents`, que hereda la igualdad de `Potential`, y la ventana de potenciales compara por reflexión campo a campo.

**58 El análisis probabilístico de sensibilidad de la simulación de sucesos no cambia nada**

La rama de análisis probabilístico de sensibilidad de `DESInference` escribe los valores sorteados donde el muestreo no los lee, y sobre tablas cuyo `getUncertainValues()` es siempre `null`, así que no tiene efecto. Solo corre con `MonteCarloOptions.isPsa()` verdadero, y nada en producción lo activa: la casilla «Perfom PSA» se crea desactivada y no se añade a la ventana, y el lector de ficheros no fija esa opción.

**59 La razón coste-efectividad incremental del resumen de la simulación vale siempre cero**

`SimulationSummaryResults` no calcula nunca la razón coste-efectividad incremental, porque el método que lo haría no tiene llamadores, y `toString` escribe `ICER;0.0`. Ese cero no llega al usuario: `DESResultsWindow` no enseña esa columna y la exportación a CSV (valores separados por punto y coma) la borra.

**60 `StatsProperties` pide la mediana y los cuartiles con el percentil en fracción**

`StatsProperties` llama a `StatUtils.percentile` con 0,5, 0,25 y 0,75, cuando ese método espera un valor de 0 a 100, así que la mediana y los cuartiles salen iguales al mínimo (medido con los valores 1 a 100). La clase no tiene ninguna referencia en el proyecto, ni en producción ni en pruebas.

**61 El diálogo que ofrece elegir entre inglés y español no se puede abrir desde ningún menú**

`LanguageDialog` muestra la lista fija «English» y «Spanish» de `Languages`, pero la opción que lo abre se crea en `MainMenu.createAllItems` y `buildHelpMenu` no la añade a ningún menú. Lo único alcanzable es la opción de arranque `-l/-language`, y `String Database.setLanguage` rechaza cualquier idioma distinto del inglés con un aviso en el registro, como pide la decisión de que la aplicación sea solo en inglés.

**62 El respaldo de `Languages.getString` para un idioma sin nombre no se ejecuta nunca**

`Languages.getString` espera una `MissingResourceException` que `StringDatabase.getString` nunca lanza (devuelve la clave rodeada de `>>>` y `<<<`), así que su respaldo no funciona; lo mismo pasa en `getShortString`. Solo se llama con «English» y «Spanish», cuyos nombres existen, y su único consumidor, `LanguageDialog`, no se puede abrir (nota 61).

**63 Con una clave repetida en dos ficheros de textos, gana uno según el orden de lectura**

`StringDatabase` junta los textos de todos los ficheros en un mapa y el último leído pisa al anterior, en un orden que nadie fija; y dos ficheros con el mismo nombre base en módulos distintos harían desaparecer uno entero. Medido en inglés: la única clave repetida, `BUNDLEFILE.Text`, tiene el mismo texto en todos sus ficheros y ningún nombre base se repite; las colisiones reales están en los ficheros en castellano, fuera por la decisión de que la aplicación sea solo en inglés.

**64 El constructor `PartitionedInterval(Object[][])` no acepta la tabla que produce su pareja**

El constructor lee siete columnas por tramo donde `convertToTableFormat` escribe seis, compara símbolos con `==` e interpreta el corchete al revés en la rama de varios tramos; medido con un intervalo de dos tramos, lanza `ClassCastException`. Ninguna construcción de `PartitionedInterval` del proyecto, ni en producción ni en pruebas, usa ese constructor.

**65 `DiscretizeTablePanel.setPartitionedInterval` empareja cada tramo con el estado equivocado**

El método pega los nombres de estado invertidos sobre un volcado que no está invertido, así que la fila $i$ mostraría el tramo $i$ con el nombre del estado *n-1-i*. Su único llamador de producción, `RevelationArcPanel.setFieldsFromProperties`, trabaja con la subclase `RevelationArcDiscretizeTablePanel`, que redefine el método, y el panel de dominio del nodo se llena con `setDataFromPartitionedInterval`, que empareja bien.

**66 Cuatro ediciones de opciones de la red ejecutan su cambio dos veces al rehacer**

`TemporalOptionsEdit`, `MulticriteriaEdit`, `MonteCarloOptionsEdit` y `CycleLength Edit` redefinen `redo()` como `super.redo(); doEdit();`, y `super.redo()` ya ejecuta `do Edit()`. Tres de ellas solo se crean en pruebas; `MulticriteriaEdit` sí se puede rehacer desde la aplicación, pero su `doEdit()` solo asigna valores y repetirlo deja la red igual. Es el mismo patrón que nota 84 y nota 85.

**67 El nombre de la fila de un nodo numérico se recorta con la medida de los nodos de estados**

`VisualState.paint` recorta el nombre de la fila a 52 unidades aunque en una fila numérica la barra empieza en la 32, así que un nombre más largo quedaría debajo de la barra. Las filas numéricas solo dibujan « EU» en los nodos de utilidad, que mide 23 píxeles, o una cadena vacía en los de azar numéricos, y nadie llama a `VisualState.set StateName`.

**68 La opción «Save evidence…» no está en ningún menú y su código no guarda nada**

`NetworkFileHandler.saveEvidence` solo imprime la ruta elegida por la salida estándar, y además duplicaría la evidencia previa a la resolución. La entrada se crea en `MainMenu. createAllItems`, pero `rechargeFileMenu` no la añade, no tiene atajo de teclado y ningún otro componente emite su mando; la evidencia sí se guarda al guardar la red.

**69 «Save evidence…», «Link properties» y cambiar idioma se crean y no están en ningún menú**

`MainMenu.createAllItems` crea esas tres entradas, pero ningún método que construye los menús las coloca y ningún otro componente emite sus mandos. Lo que hay detrás es código muerto sin consecuencia visible: un guardado que no guarda (nota 68), `Network EditorPanel.changeLinkProperties` con el cuerpo vacío y `LanguageDialog` (nota 61).

**70 Quedan once constantes y sus textos de un submenú de zoom del menú View que ya no existe**

Ningún código usa las constantes `VIEW_ZOOM_*` de `MenuItemNames`: `MainMenu.buildView Menu` solo pone las entradas de pestaña siguiente y anterior, y el zoom está en la barra de botones. Las constantes apuntan además a claves `View.ZoomManager.*` que no existen (los textos de `Menus_en.xml` están bajo `View.Zoom.*`); es limpieza, y el mismo renombrado sí estropea un texto visible, que es error 183.

**71 Seis botones de opciones del panel de un modelo canónico no tienen escuchador**

En `ICIOptionsPanel` solo la pareja «Canonical parameters» / «Whole table» tiene escuchador; las otras seis opciones tienen el `addItemListener` comentado. Pero se crean con `setEnabled(false)` y nada las habilita (medido), así que nadie puede pulsarlas; lo único visible son seis opciones en gris, una con la errata «Independant parameters» en `Dialogs_en.xml`.

**72 `StringWithProperties.put` falla con un agente leído de fichero sin propiedades**

`PGMXReader_0_2.getAgents` sustituye la tabla de propiedades del agente por `null` cuando el `<Agent>` no tiene propiedades, y un `put(clave, valor)` posterior lanzaría `NullPointerException`; un agente creado con `StringWithProperties(String)` sí tiene tabla. En producción nadie llama a `put(clave, valor)` sobre un agente, y el escritor comprueba que la tabla no sea nula.

**73** **Elegir el criterio de un nodo de utilidad apila cuatro ediciones iguales**
`NodeDefinitionPanel.getJComboBoxDecisionCriteria` añade el panel dos veces como escuchador y `itemStateChanged` no descarta la notificación de deselección, así que elegir otro criterio apila cuatro `NodeDecisionCriteriaEdit`, y elegir otro agente, dos `NodeAgentEdit` (medido). El diálogo de propiedades agrupa todas sus ediciones en una sola al aceptar y las deshace en orden inverso al cancelar, así que ni Ctrl+Z ni «Cancel» muestran diferencia.

**74** **Muestrear un árbol de relaciones vacía la configuración de padres que recibe**
`TreeADDPotential.sampleConditionedVariable` y `TreeWithExcludedEventsPotential` quitan hallazgos de la configuración del llamador y no la devuelven como estaba (medido: entra `[E, Age]` y sale vacía). Todos los llamadores de producción, en la simulación de eventos discretos, crean una configuración nueva para cada muestreo y no la vuelven a leer.

**75** **La tabla de la evolución temporal fallaría si se construyera antes que la gráfica**
`TemporalEvolutionTablePane` lee los valores por índice sin repetirlos cuando el resultado de un ciclo no depende de la decisión, y construida sola lanza `ArrayIndexOutOfBoundsException` (medido). En la ventana, `TraceTemporalEvolutionDialog.createSeries` construye antes la gráfica y deja guardados los valores ya repetidos, así que la tabla sale bien; queda un acoplamiento oculto entre las dos pestañas.

**76** **El informe de la evolución temporal fallaría con una tabla sin columna de nombres**
`TemporalEvolutionReport.write` convierte a `String` la primera columna de la tabla y lanzaría `ClassCastException` si esa columna tuviera números, como en la tabla por variable de una utilidad sin decisión. Pero `TraceTemporalEvolutionDialog.getTablePane` nunca construye esa tabla: para utilidades usa el constructor por criterio, cuya primera columna son siempre nombres; medido, el informe de una utilidad se escribe bien con y sin decisión.

**77** **El plano de coste-efectividad de la simulación de sucesos fallaría con un solo criterio**
`CEDESDialog.getCEPlaneChartPanel` usa `criteria.get(1)` sin comprobar, así que con un solo criterio lanzaría `IndexOutOfBoundsException` y tampoco se vería la tabla de resultados. Pero `DESInference.initialize`, por la que pasan los dos constructores, avisa y no simula con menos de dos criterios, y no hay otro llamador de `CEDESDialog`.

**78** **La casilla «Always append» lleva el nombre de componente de un campo antiguo**
`NodeDefinitionPanel.getJCheckboxAlwaysAppend` crea la casilla con el nombre de componente `jCheckboxOverrideTimeStamp`, que no dice lo que es; ese nombre solo sirve para encontrarla desde pruebas de interfaz o paseos guiados. Ningún código la busca por ese nombre ni por `jCheckboxAlwaysAppend`; si se corrige, conviene hacerlo junto con error 34, que es el defecto de deshacer de esta misma casilla.

**79** **Invertir un enlace recalculando los potenciales puede cerrar un ciclo**
`InvertLinkAndUpdatePotentialsEdit` no comprueba ninguna restricción y, ejecutada directamente, deja una red bayesiana que incumple `NoCycle` (medido), cuando su hermana `InvertLinkEdit` rechazaría esa inversión. Su único llamador de producción es la opción del menú contextual del enlace, que antes pregunta a `LinkInversionWithPotentialsUpdateValidator` y sale deshabilitada si la inversión cerraría un ciclo.

**80 El inverso de orientar un enlace es la propia orientación**
`OrientLinkEdit.getUndoEdit()` devuelve la misma edición en vez de una que devuelva el enlace a no dirigido. Su único consumidor, `Metric.afterUndoingEdit`, solo lo usan los algoritmos de puntuación y búsqueda, que nunca crean orientaciones, y el algoritmo PC, que sí las crea, no usa `Metric`.

**81 Subir el primer criterio de decisión o bajar el último: la ventana no lo permite**
`DecisionCriteriaEdit.doEdit` no comprueba que haya sitio al subir o bajar un criterio, pero sus únicos llamadores con `UP` o `DOWN` son los botones de `DecisionCriteriaTable Panel`, que `KeyTablePanel.valueChanged` deshabilita en la primera y la última fila y sin selección, y el criterio sale siempre de la propia lista. Es el mismo caso que nota 36; el defecto vecino de deshacer el renombrado de un criterio es error 57.

**82 Pegar una selección de solo enlaces revienta al calcular el recuadro**
`PasteEdit` calcula el máximo de las coordenadas de los nodos pegados, y con una lista vacía lanzaría `NoSuchElementException`. `MainPanelMenuAssistant` deshabilita Copiar y Cortar (en el menú, en sus atajos y en la barra) cuando solo hay enlaces seleccionados, así que el portapapeles siempre lleva al menos un nodo. Un nodo acompañado de un enlace cuyo otro extremo no se copió sí llega, y falla de otra manera: nota 184.

**83 Editar un parámetro de un modelo canónico deja dos ediciones y un deshacer vacío**
`ICITablePotentialValueEdit.doEdit` ejecuta dentro un `ICIPotentialEdit` como edición aparte, y su propio `undo()` no hace nada. Las dos entradas caen en el historial provisional de `PotentialEditPanel`: al aceptar se descarta y se sustituye por un único `SetPotentialEdit` que se deshace bien, y al cancelar se deshace entero y la edición interna repone los parámetros.

**84 Rehacer «quitar hallazgo» repite la retirada**
`RemoveFindingEdit.redo()` no apaga el interruptor `typicalRedo`, así que la clase base ejecuta `doEdit()` y después la clase vuelve a quitar el hallazgo. La segunda retirada no encuentra nada y no cambia nada; el cambio de contrato de las ediciones descrito en nota 154 lo haría desaparecer de raíz.

**85 Rehacer un cambio de potencial aplica el cambio dos veces**
`SetPotentialEdit.redo()` llama a la clase base sin apagar el interruptor de rehacer y después repite `setPotentialWithRestrictions`. Repetirlo no cambia ningún valor, porque las casillas prohibidas ya valen cero y cada columna ya suma uno (salvo, a lo sumo, redondeo en el último decimal); también desaparecería con el cambio de contrato de nota 154.

**86 El constructor de un argumento de `SetPotentialEdit` no comprueba la lista vacía**
`SetPotentialEdit(Node)` toma `getPotentials().getFirst()` sin comprobar que el nodo tenga potenciales, al contrario que el de dos argumentos. No tiene ningún llamador, ni en producción ni en pruebas; borrarlo o darle la misma guarda es limpieza.

**87 El conjunto de oyentes de ediciones promete ser seguro entre hilos y no lo es**
`PNESupport` guarda los oyentes en un conjunto sincronizado que `PNEdit.executeEdit`, `undo` y `redo` recorren sin tomar su cerrojo, y `setListeners` lo vacía y lo rellena en dos pasos, contra lo que dice su javadoc. No se ha encontrado ningún camino de producción que ejecute ediciones o cambie oyentes desde otro hilo mientras se avisa, ni que añada un oyente dentro de un aviso con otros oyentes detrás.

**88** **Tres restricciones convierten la red a `ProbNet` sin comprobar su clase**
`ProperUtilityPotentials`, `UtilityNodes` y `OnlyOneAgent` hacen un molde del `Graph Network` recibido a `ProbNet` o `PotentialNetwork` sin mirar, lo que fallaría con otra implementación de esas interfaces. Hoy `ProbNet` es la única; `ValidCriterionName` muestra la forma segura, con `instanceof`.

**89** **La sección de restricciones adicionales del fichero escribe frases y lee nombres de clase**
`PGMXWriter_0_2.getAdditionalConstraints` se salta la primera restricción, solo escribe las que el tipo de red prohíbe y las escribe como texto para el usuario, mientras que el lector espera nombres de clase, así que no podría reabrir ese fichero (medido). Ninguna red construida desde la aplicación lleva restricciones que su tipo prohíba, así que hoy el escritor no escribe nada en esa sección.

**90** **La rama de respaldo de `SetPotentialVariablesEdit` desordena la tabla, pero no se alcanza**
Si `reorder(List)` devuelve `null`, `SetPotentialVariablesEdit.doEdit` cambia el orden de las variables del potencial original sin mover sus valores, y la tabla quedaría mal. Su único constructor de producción es el botón de reordenar variables de `PotentialEdit Panel`, que solo llega con tablas y modelos canónicos, cuyos `reorder` devuelven siempre un potencial.

**91** **Coste-efectividad declara inalcanzable la evidencia incompatible, y hoy lo es**
En el ámbito global, la única vía hacia `EvidenceIsIncompatibleWithOther` era la discretización que pisaba la evidencia del usuario, corregida en el commit `66d32ab`, y la conversión que queda en `VECEAnalysis.getUtility` quita el hallazgo antes de añadir el de la variable convertida. En el ámbito por decisión, `ScopeSelectorPanel` fija y deshabilita el estado de las variables ya observadas. Los dos tratamientos distintos siguen escritos; unificarlos entra en error 48.

**92** **Las operaciones públicas de `PotentialOperations` rechazan subclases de `Table Potential`**
`AuxiliaryOperations.checkObjectsCollectionType` compara la clase exacta, así que `multiply`, `multiplyAndMaximize` y las dos `multiplyAndMarginalize` de `Potential Operations` rechazan una `StrategicTablePotential` o una `UncertainTablePotential` (medido), aunque `DiscretePotentialOperations` sabe tratarlas. Su único llamador de producción es `ClusterOfVariables.getDowngoingPotential`, que nadie llama; la propagación de Hugin usa directamente `DiscretePotentialOperations`.

**93** **Maximizar operandos que son todos constantes devuelve una tabla de elecciones vacía**
El atajo de `TablePotentialMaximization.multiplyAndMaximize` para operandos sin variables no añade ninguna elección, cuando debería dejar empatados todos los estados de la decisión, como hace `multiplyAndMaximizeUniformly` (medido: tabla de elecciones de tamaño 0). El único lector de esa tabla, `Strategy.Policy`, solo se construye desde `Strategy(StrategyUtilities)`, que nadie llama, y la absorción de un nodo de decisión solo usa la tabla de valores.

**94** **Multiplicar una tabla estratégica sin árboles de estrategia lanza `NullPointer Exception`**
`TablePotentialArithmetic.multiply` y `evaluateFunctionPotential` leen `strategy Trees` de una `StrategicTablePotential` sin comprobar que no sea nulo, cosa que sí hacen `sum` y otras operaciones del mismo paquete (medido). Todos los sitios de producción que construyen una `StrategicTablePotential` le asignan los árboles antes de soltarla, y el único que no lo hace, `VEOptimalIntervention` sin decisiones, no la multiplica.

**95 Sumar fuera una variable de una utilidad estratégica sin árboles lanza una excepción**

`TablePotentialElimination.multiplyAndMarginalize(probabilidad, utilidad, variable)` y el constructor de `SumOutVariable` leen `strategyTrees` sin comprobar que no sea nulo (medido en los dos). Como en nota 94, ninguna ruta de producción entrega a estas operaciones una `StrategicTablePotential` sin árboles.

**96 La clase abstracta `ICIPotential` figura entre los tipos de potencial aplicables a un nodo**

`PotentialUtils.getFilteredPotentialClasses` devuelve `ICIPotential` para cualquier nodo con padres, porque está anotada con `@PotentialType` y su `validate` la acepta, aunque construirla lanza `UnreachableException` (medido). De sus dos llamadores de producción, `PotentialEditPanel.getPotentialTypeJCombobox` quita las clases no concretas y `ChangeNodeTypeEdit.doMultiStepEdit` solo pregunta si la lista contiene la clase del potencial actual, que nunca es la abstracta.

**97 El MAX ruidoso marca su tabla como probabilidad conjunta y normalizarla la estropea**

`MaxPotential` construye sus tablas intermedias con papel de probabilidad conjunta y `MinPotential` con papel de condicionada, así que la tabla del MAX sale como conjunta y `TablePotentialTransform.normalize` la divide por la suma de toda la tabla (medido: cada columna queda sumando 0,25). La vía que llevaría a normalizarla, la evolución temporal de un diagrama de influencia de Markov, no normaliza nunca (es error 67), y no se encontró otro consumidor de producción cuyo resultado dependa de ese papel.

**98 Ordenar potenciales por el orden total de las decisiones devuelve cada potencial dos veces**

`TablePotentialMerge.orderPotentialsByTotalOrder` añade al final la lista de entrada entera en vez de solo los potenciales que quedan sin colocar (medido: entran dos potenciales y salen cuatro), así que multiplicar esa lista contaría cada factor dos veces. El método, expuesto también en `DiscretePotentialOperations`, no tiene ningún llamador, ni en producción ni en pruebas.

**99 `matrixPotential` con una lista de solo probabilidades da todo ceros**

`TablePotentialTransform.matrixPotential` multiplica el producto de las probabilidades por la suma de las utilidades, y sin utilidades esa suma es la constante 0 (medido: `[0.3, 0.7]` da `[0.0, 0.0]`). Ni el método ni su fachada en `DiscretePotentialOperations` tienen llamadores, tampoco en pruebas.

**100 La copia vieja de `Util` de las operaciones formatea mal negativos y usa coma decimal**

`org.openmarkov.core.model.network.potential.operation.Util`, copia antigua de `org.openmarkov.core.model.network.Util`, da `printInteger(-1510000L)` = `"0"` y, con la configuración regional española, `roundedString(3.4, "0.001")` = `"3,400"` (medido). Ningún código de producción la importa; solo la usa `UtilTest`, y lo razonable es borrarla con su prueba.

**101 Reordenar un modelo canónico da a la copia las variables auxiliares del original**

`ICIPotential.reorder` copia el potencial y después rellena el mapa de la copia con las variables auxiliares del original, así que los dos comparten esos objetos (medido con un modelo de ajuste); se llega desde el botón de reordenar variables del diálogo del nodo. No da ningún resultado distinto porque la edición pone la copia en el nodo y guarda el original solo para deshacer, de modo que nunca coinciden en una misma inferencia; si coincidieran, la eliminación los trataría como la misma variable.

**102 La tabla expandida que un modelo canónico guarda en caché no está protegida entre hilos**

`ICIPotential.getProbability` guarda la tabla completa en `expandedPotential`, un campo sin `volatile` ni sincronización, así que dos hilos a la vez podrían ver una tabla a medio publicar o anterior a un cambio. Su único llamador de producción, `LikelihoodWeighting`, corre en un solo hilo, y el único cálculo con varios hilos, `VECEPSA`, usa eliminación de variables, que no pasa por aquí. Es el mismo caso que nota 104.

**103 El análisis probabilístico de sensibilidad con varios hilos repite muestras**

Con `useMultithreading` activado, las tareas de `VECEPSA` sortean y copian la misma red compartida, y entre el sorteo de un hilo y su copia otro hilo puede volver a sortear (medido sobre `integrationTests/src/main/resources/networks/mid/MID-dmhee-4.7.pgmx`: 71 resultados repetidos de 300). El único llamador de producción, el diálogo del análisis probabilístico de coste-efectividad, pone `setUseMultithreading(false)`; la rama de varios hilos solo la activan las pruebas.

**104 La caché de la tabla expandida de un modelo canónico no se usa desde dos hilos**

`ICIPotential.getProbability` calcula y guarda `expandedPotential` sin sincronización, y los métodos que cambian parámetros la anulan, así que un hilo podría leer una tabla vieja. Su único llamador de producción, `LikelihoodWeighting`, corre en un hilo, y `VECEPSA` con varios hilos (nota 103) evalúa con eliminación de variables, que no pasa por esa caché. Es el mismo caso que nota 102.

**105 Una propagación reutilizada con otra evidencia contesta con la consulta anterior**

`VEPropagation.getPosteriorValues` no vuelve a calcular si ya tiene un resultado, `ClusterPropagation` acumula la evidencia vieja (nota 39) y `StochasticPropagation` quita para siempre de las variables de interés las que tuvieron hallazgo; medido sobre `0_miscellaneous/networks/bn/BN-asia.pgmx`, `VEPropagation` reutilizada devuelve exactamente la respuesta anterior. Todos los llamadores de producción construyen un objeto nuevo por consulta; fijar un ciclo de vida único para las tareas es la propuesta de nota 167.

**106 Un complemento de formato o herramienta que no se construye inutiliza su gestor**

`FormatManager` y `ToolPluginManager` construyen todos sus complementos en su inicializador estático, y si uno falla la primera llamada lanza `ExceptionInInitializerError` y las siguientes `NoClassDefFoundError`, sin decir cuál; `InferenceManager.build`, en cambio, deja fuera solo al culpable. Medido: hoy todos los lectores, escritores y herramientas se construyen sin error, así que solo lo dispararía un complemento defectuoso futuro (la política única para instanciar complementos es nota 171).

**107 El preprocesado del aprendizaje no devuelve a «no discretizar» una variable no numérica**

En `LearningDialog.discretizeComboBoxActionPerformed` (línea 814) se llama a `setSelectedItem(0)` donde se quería `setSelectedIndex(0)`, así que una variable no numérica que la red modelo tiene discretizada se quedaría con «Use model network». Pero la pestaña de preprocesado no se añade a la ventana de aprendizaje (la línea 544 que la añade está comentada), así que nadie puede cambiar esa opción; el panel equivalente de `bnEvaluation` usa `setSelectedIndex` y no tiene el defecto.

**108 Pedir variables al muestreo en otro orden cruza sus probabilidades, pero nadie lo hace**

`StochasticPropagation.getPosteriorValues` (líneas 164-172) suma cada muestra en la fila de la k-ésima variable de interés que encuentra al recorrer las variables en orden de muestreo, pero la guarda a nombre de la k-ésima de la lista que le dio el llamador; medido en `0_miscellaneous/networks/bn/BN-asia.pgmx` con las variables en el orden de `getVariables()`, P(Tuberculosis | X-ray = yes) sale [0,43, 0,57] en lugar de [0,91, 0,09]. Nadie llega: el único llamador de producción, `StochasticPropagationOutput Frame`, no llama a `setVariablesOfInterest` y usa la lista del constructor, ordenada topológicamente, y la ventana principal propaga siempre con `VEPropagation`. — cambios —

**109 Hugin usa como probabilidades la tabla de un nodo numérico, escrita a mano en el fichero**

En una red bayesiana P → N con N numérico de azar y un `TablePotential` con los valores 3 y 7, `HuginPropagation` devuelve P = [0,3, 0,7] (multiplica esos valores como si fueran probabilidades), el muestreo ≈ [0,5, 0,5] y `VEPropagation` falla con `NullPointer Exception` (error 42). Esa tabla solo la trae un `.pgmx` escrito a mano, porque el lector acepta `Table` sobre una variable numérica; el editor no la crea (pone una relación uniforme y `TablePotential.validate` excluye las variables numéricas), guardar con `PGMXWriter_0_2` o `PGMXWriter_1_0` y reabrir da `ExactDistrPotential` o `Univariate DistrPotential`, y ninguna red bayesiana del repositorio tiene nodos numéricos. Con las relaciones que ofrece el editor («Exact», «UnivariateDistr») la herramienta de propagación estocástica falla antes, sin dar números distintos ni decir que no admite nodos numéricos (error 108).

**110 Escalar una utilidad de regresión ya muestreada no la cambia, pero hoy ningún análisis llega**

`scalePotential` de `ExponentialPotential` y de `LinearCombinationPotential` solo cambia `coefficients`, y `GLMPotential.tableProject` usa `sampledCoefficients` cuando existen; como `VECEPSA` muestrea antes de que `VECEAnalysis` descuente y aplique la escala de coste-efectividad, esas dos operaciones se perderían (medido en el potencial: muestreado y escalado por 0,5 sigue valiendo 100). Hoy no se llega: las únicas redes con utilidades de regresión inciertas, `0_miscellaneous/networks/mid/MID-CHAP-Ryan-Griffin.pgmx` y sus copias, no pasan de la discretización, y una utilidad de regresión en un nodo hace fallar el análisis de coste-efectividad con `ClassCast Exception` antes de muestrear (medido con `0_miscellaneous/networks/id/ID-CEA-minimal.pgmx` y `0_miscellaneous/networks/mid/MID-hip-Briggs.pgmx`).

**111 La casilla «Results Per Series (.xlsx file)» no hace nada, pero está siempre desactivada**

`MonteCarloOptionsPanel.getJCheckBoxExcelResultsPerSeries` desactiva siempre la casilla (línea 333, «Provisional for paper .jar»), y `PGMXReader_1_0` no lee esa opción, así que en la ventana de la simulación de sucesos sale gris y desmarcada (medido construyendo el panel). Aunque se marcara, nadie lee `MonteCarloOptions.isResultsTo Excel()` y la clase que escribiría el Excel, `DESResultsExcelWriter`, no tiene ninguna referencia. Si se decide retirar controles del mismo panel (error 109), esta casilla y esa clase van con ellos.

**192 El análisis de sensibilidad probabilístico de una red de sucesos no se puede pedir, y el código que lo haría está mal**

La simulación sabe repetir las series sorteando los parámetros inciertos (`DESInference`, líneas 118-200, con la opción `psa` de `MonteCarloOptions`), pero nada permite activarlo: la casilla «Perfom PSA» existe (`MonteCarloOptionsPanel.getPsaCheckBox`, líneas 257-266), pero no se añade al panel; la opción no se guarda en el `.pgmx`, y los paneles de las tablas con sucesos no abren el diálogo de incertidumbre. Ya estaba a medias en la rama `DES-development` de Carmen Yago, que lo daba por preliminar. Si se activara tal como está: `DESInference.samplePSATablePotential` (líneas 191-200) cambia cada valor incierto por un valor fijo, en vez de poner cada muestra en la tabla, y la copia de la red comparte las relaciones con la red del editor (`ProbNet.copy` es `ProbNetCopier.shallowCopy`), así que todas las series usarían los mismos números y se borraría la incertidumbre de la red abierta; `UncertainValuesEdit.prepareUncertainPotential` (líneas 151-170) lanza `UnsupportedOperationException` con la tabla interna de una `TableWithEvents`; y el constructor para sucesos de `UncertainValuesDialog` (línea 97), que no llama nadie, ha perdido el filtro de las distribuciones válidas para sucesos (`getDESValidProbDensFunctions`). Hay que decidir si se termina o se borra.

**199 `VETemporalEvolution` escribe el modelo de Markov dos veces**

`VETemporalEvolution.resolve` escribe el modelo hasta el horizonte (`TaskUtilities.expandNetwork`, línea 43) y pasa el resultado a un `VEPropagation`, cuya preparación (`VEPropagation.prepare`, línea 174, que llama a `VariableElimination.generalPreprocessing`, línea 84) lo vuelve a escribir: la red escrita sigue siendo un MID, sin la restricción `OnlyAtemporalVariables`, así que la pregunta de si hay que escribirla contesta que sí. La segunda vez no añade ningún ciclo, porque ya están todos, pero copia la red entera y la recorre de nuevo; y si algo va mal, nombra nodos que hizo la primera. Medido el 15-10-2026 con «Age [0]» con una tabla, «Treatment [1]» con «Same as previous» sin nada que copiar y horizonte 2: `TemporalNetOperations.expandNetwork` escribe el modelo sin quejarse, y `VETemporalEvolution` se niega con ««Treatment [2]» is «Same as previous», but no earlier cycle has a relation for it to copy», desde `compactNetwork` llamada por `generalPreprocessing`; «Treatment [2]» es la copia que hizo la primera escritura (ver error 261). No cambia ningún número: en las redes medidas, `VETemporalEvolution` y `MIDTemporalEvolution`, que escribe el modelo una sola vez, dan los mismos valores bit a bit. Hoy no lo alcanza nadie desde la aplicación: la ventana de la evolución temporal usa `MIDTemporalEvolution`, y el único sitio de la interfaz que crea una `VETemporalEvolution` es `TraceTemporalEvolutionDialog.createExcel(ProbNet, EvidenceCase, Variable)` (líneas 722-760), al que no llama nadie; queda la ayuda de pruebas `InferenceTestsTools`. Si vuelve a usarse, bastaría con que `resolve` no escriba el modelo y deje que lo haga la propagación.

## Fuera de alcance por decisión de equipo

**112 Una relación de Elvira que ningún modelo canónico reclama hace perder la red entera**

`ElviraParser.addSubPotentials` lanza `SomeSubpotentialsArentLinkedToAnICIPotential` si queda alguna subrelación sin reclamar, y no se abre nada de la red. El formato Elvira está congelado por decisión de equipo; comprobado solo leyendo el código.

**113 El escritor de Elvira repite la variable de cada utilidad y crea un enlace a sí misma**

`ElviraWriter` escribe la variable de utilidad y después otra vez todas las variables de la relación (`relation U U A D2 E`), y la red releída tiene un enlace `U -> U`. El formato Elvira está congelado por decisión de equipo.

**114 El escritor de Elvira escribe `GeneralizedMin`, que su propio analizador no reconoce**

`ElviraWriter` escribe el modelo MIN generalizado como `GeneralizedMin`, pero `ReservedWordTokens` no tiene esa palabra, así que el fichero no se puede releer. El formato Elvira está congelado por decisión de equipo.

**115 El escritor de Elvira muere al escribir un nodo de supervalor y deja el fichero a medias**

`ElviraWriter` convierte las relaciones que no son tablas con `tableProject(null, null)`, y una suma o un producto de utilidades lanza porque necesita las tablas de sus hijos; el fichero queda sin cerrar. El formato Elvira está congelado por decisión de equipo.

**116 Guardar una red en Elvira pierde el título de todos los nodos**

`ElviraWriter` vuelve a intercambiar nombre y título antes de escribir, pero no escribe ninguna línea `title`, así que al releer los nodos se llaman por su nombre corto. El formato Elvira está congelado por decisión de equipo.

**117 Un nodo continuo de Elvira sin precisión, mínimo o máximo hace fallar la lectura**

`ElviraParser` convierte `Min`, `Max` y `Precision` con `Double.parseDouble` sin comprobar que existan, y `ElviraWriter` solo escribe la precisión si el nodo la tenía en sus propiedades, así que un fichero que se leía puede dejar de leerse tras guardarlo. El formato Elvira está congelado por decisión de equipo.

**118 Una tabla generalizada de Elvira con incertidumbre hace fallar la lectura**

`ElviraScanner.getNextToken` convierte los números de la tabla con `Double.parse Double` sin contemplar la barra que separa la incertidumbre ni la marca `#` de las casillas deducidas, lo que afecta a los ficheros `.elv` de análisis de sensibilidad. El formato Elvira está congelado por decisión de equipo.

**119 La pestaña de la tabla del nodo solo enseñaría el nombre de la clave en castellano**

Las claves `NodePropertiesDialog.EditPotentialTab.*` que usa `NodeProperties Dialog.updatePotentialTabTitleAndButton` están todas en `Dialogs_en.xml`, así que en inglés la pestaña y los botones salen bien. Solo faltan en el fichero en castellano, que no se mantiene porque la aplicación es solo en inglés.

**120 Varios rótulos y menús llevan su texto inglés escrito en el código**

«Numeric value:» en `AddFindingDialog`, los submenús «Add» de `NetworkContextual Menu` y «Alignment» de `NodeContextualMenu` y los rótulos de otros paneles siguen escritos en el código, pero están en inglés y la aplicación es solo en inglés, así que no hay nada visible. Aparte, la ayuda emergente «Create an utility node» sí es una errata visible: está en error 185.

**121 En castellano, la opción «Discretas y continuas» cuelga de una etiqueta mal escrita**

En `Dialogs_es.xml` el elemento que pide `NetworkVariablesPanel.getListOfTypes` está bajo `<items>` en minúscula, así que en castellano la clave no existiría. En `Dialogs_ en.xml` las tres entradas están bien y la aplicación solo sirve inglés (`StringDatabase. SUPPORTED_LANGUAGES`).

**122 Los mensajes asociados a excepciones solo existen en inglés**

Los textos de las clases de excepción viven en ocho ficheros `*_class_localizations_en. xml` y no hay versión en castellano. Como `StringDatabase` solo sirve inglés, por decisión de equipo, no hay consecuencia visible.

**123** **Varios textos de los ficheros en castellano están estropeados o faltan**
«Ampliar zoomManager», «Reducir zoomManager», «Proposito» y las claves que faltan están solo en los ficheros `*_es.xml`, como `Menus_es.xml`, que la aplicación no carga porque es solo en inglés. La misma sustitución de `zoom` por `zoomManager` sí dejó un texto visible en inglés, que es error 183.

**124** **Los estados del dominio «not happened - happened» no tienen texto en el fichero de textos**
Sin texto, `GUIDefaultStates.getString` devuelve el nombre interno del estado, que en inglés es justo lo que debe verse, como en el resto de dominios de `Selectables_en.xml`. Solo se notaría en castellano, que queda fuera porque la aplicación es solo en inglés.

**125** **El aviso de cambio de dominio al partir un intervalo del árbol no usa su clave de textos**
`TreeADDEditorPanel.splitInterval` muestra «Be careful, you changed variable domain» escrito en el código, y la clave `TreeADD.DomainChangeWarning` de `Menus_en.xml` no la usa nadie. En la aplicación, que es solo en inglés, el aviso se lee bien; la clave sin uso es texto muerto.

**126** **El título de la ventana de editar una relación solo tiene texto en el fichero inglés**
`PotentialEditDialog.getBaseTitle` pide la clave `NodePropertiesDialog.Edit PotentialTab.EditPotentialTitle`, que está en `Dialogs_en.xml` y se resuelve como «Edit probability» (medido). Solo falta en `Dialogs_es.xml`, que no se mantiene porque la aplicación es solo en inglés.

**127** **Las dos entradas del menú Ver no tienen texto en castellano**
Las claves `View.GoNextTab` y `View.GoPreviousTab` que usa `MainMenu.buildViewMenu` están en `Menus_en.xml` y dan «Go to next tab» y «Go to previous tab» (medido). Solo faltan en `Menus_es.xml`, que no se mantiene porque la aplicación es solo en inglés.

**128** **El título de la pestaña del árbol de decisión está escrito en el código**
`InferenceHandler.showDecisionTree` abre la pestaña con «Decision tree for » más el nombre de la red, escrito en el código y no en un fichero de textos. Se lee bien en inglés, y la aplicación es solo en inglés.

**129** **El menú del árbol de decisión dice en castellano «Expandir N niveles» y expande uno**
La entrada `ExpandNext` de `TreeContextualMenu` ejecuta `DecisionTreeEditor.inferenceExpandNextLevel`, que expande un nivel, y en inglés dice «Expand next level», que es lo que hace. Solo el texto castellano promete otra cosa, y los ficheros `*_es.xml` no se mantienen.

**130** **Dos claves de opciones temporales y de inferencia cuelgan de otro padre en castellano**
`TemporalEvolutionDialog.getSlicesPanel` pide `Inference.TemporalOptions` y el menú pide `Inference.InferenceOptions`; en `Menus_en.xml` las dos cuelgan de `<Inference>` y dan «Temporal options» e «Inference options» (medido). Solo en `Menus_es.xml` cuelgan de `<Edit>`, y ese fichero no se mantiene.

**131** **Los rótulos del panel de opciones de Monte Carlo están escritos en el código**
`MonteCarloOptionsPanel` crea sus rótulos con textos ingleses en el código, que se leen bien en la aplicación, que es solo en inglés. La errata «Perfom PSA» está en la casilla de `getPsaCheckBox()`, que nadie añade al panel (nota 58).

**132** **Seis rótulos del análisis probabilístico de coste-efectividad están en el código**
`CEProbabilisticDialog` escribe en el código «WTP for evaluation» (WTP: disposición a pagar), «Display:», «Relative to:» y otros, mientras que `CEDecisionResults` pide cuatro de ellos por clave a `costeffectiveness_en.xml`, con el mismo texto. En inglés las dos ventanas dicen lo mismo, y la aplicación es solo en inglés.

**133 Las claves «Decision» e «ICER» del análisis de sensibilidad faltan en castellano**

`SensitivityAnalysis.General.Decision` y `SensitivityAnalysis.General.ICER` (ICER: razón coste-efectividad incremental) están en `sensitivityanalysis_en.xml`, y `CESpiderDialog` muestra «Decision:» correctamente. Solo faltan en castellano, y la aplicación es solo en inglés.

**134 Los rótulos de los resultados de la simulación de sucesos están escritos en el código**

`DESResultsWindow`, `CEDESDialog` y el monitor «Running simulation» de `MainPanel` `ListenerAssistant` llevan sus textos en el código, con un aviso `FIXME use String` `Database`. Están en inglés y se ven bien; solo importaría con otro idioma, y la aplicación es solo en inglés.

**135 El formato de bases de casos de Elvira (`.dbc`) no está registrado ni se ofrece**

`ElviraDataBaseIO` lleva comentada la anotación `@CaseDatabaseFormat`, así que `Case` `DatabaseManager` no lo encuentra y ningún selector de ficheros lo ofrece. Es un formato de Elvira, congelado por decisión de equipo; además, las bases de aprendizaje de las pruebas ya se convirtieron a CSV (valores separados por comas) en el commit `9c36f04`.

**136 El escritor de bases de casos de Elvira (`.dbc`) no escribe los estados**

`ElviraDataBaseIO.writeVariables` no escribe `num-states` ni `states`, y `Elvira` `DBParser` falla al leer lo escrito porque esos estados le llegan nulos. Es código exclusivo del formato Elvira, congelado por decisión de equipo, y además no está registrado (nota 135).

**137 El lector de bases de casos de Elvira (`.dbc`) se fía del número de casos de la cabecera**

`ElviraDBParser` reserva la tabla de casos con el número que declara la cabecera, así que más casos dan `ArrayIndexOutOfBoundsException` y menos dejan filas a cero que pasan por casos. Es código exclusivo del formato Elvira, congelado por decisión de equipo y no registrado (nota 135); ningún otro lector comparte ese mecanismo.

**138 La lista de algoritmos de aprendizaje toma los nombres de las anotaciones**

`GeneralTabPanel` muestra el `name` de `@LearningAlgorithmType` y no usa las claves `Learning.<Algoritmo>.Algorithm` de los ficheros de textos. En inglés esos nombres equivalen a los de `Learning_en.xml`; la diferencia solo se vería con otro idioma, y la aplicación es solo en inglés.

**139 La casilla de prueba causal del algoritmo PC tiene su rótulo y su ayuda en el código**

`PCParametersDialog` crea la casilla «Use ANM causal direction test (phase 3)» (ANM: modelo de ruido aditivo) y su ayuda con textos escritos en el código. En inglés se leen bien y la aplicación es solo en inglés; que la ventana no nombre «phase 3» en otro sitio es cuestión de redacción.

**140 Los textos de las herramientas de evaluación de redes y de casos están en el código**

Las cuatro entradas del menú Herramientas del módulo `bnEvaluation` y los textos de sus ventanas, como `setTitle("Bayesian network evaluation")`, están escritos en inglés en el código, sin fichero de textos. En inglés se leen bien, y la aplicación es solo en inglés; la única diferencia visible es de estilo, las mayúsculas de «Split Dataset» frente a las otras tres entradas.

**141 Guardar en `.pgmx` un OR ruidoso o un MAX causal lo reabre como MAX general**

`PGMXWriter_0_2.getICIPotential` no escribe el tipo de modelo y `PGMXPotential Parsers.getICIPotential` construye siempre el tipo general, aunque la tabla de probabilidad es idéntica antes y después (medido). Solo lo alcanzan modelos importados de Elvira y solo se nota al volver a exportar a Elvira, formato congelado por decisión de equipo; el índice de parámetros por padre y el `equals` de `ICIPotential` ya se corrigieron en el commit `6260a4c`.

## Propuestas de diseño y limpieza

**142 Una edición que modifica la red y después falla no se revierte**

Propuesta: que `PNEdit.executeEdit()` revierta ante cualquier fallo de `doEdit()` y que `MultiStepEdit.StepExecuter.execute` apunte cada paso antes de ejecutarlo, en vez de después. No es un error por sí mismo, sino una garantía que falta en el diseño: los casos concretos que hoy llegan al usuario tienen su propia ficha: error 28 y nota 184.

**143 El comentario de `NoBackwardLink` describe una regla más estricta que la del código**

Hay que decidir cuál es la regla: el comentario exige que un nodo temporal con un hijo atemporal esté en el ciclo cero, y `NoBackwardLink.allowedLink` acepta el enlace en cualquier ciclo. No se ha encontrado un fallo observable, y la comprobación de la red entera y la de `AddLinkEdit` usan el mismo método, así que no divergen entre sí.

**144 Cada restricción se da de alta como oyente de ediciones sin escuchar nada**

Propuesta: quitar el `implements PNEditListener` de `PNConstraint` y su alta en `ProbNet.addConstraint` (ver nota 158). No es un error visible: ninguna restricción atiende ningún aviso, y aunque `setNetworkType` deja dados de alta oyentes de restricciones ya quitadas (medido: dos, tras pasar de red bayesiana a diagrama de influencia, a simulación de eventos discretos y otra vez a bayesiana), su único coste es recorrerlos en cada edición.

**145 El nombre de `@Constraint` no lo lee nadie y en seis casos no coincide con la clase**

Propuesta: si algún día se guardan las restricciones en el fichero por un nombre estable (nota 89), usar este atributo corrigiendo antes sus dos erratas, «InitialNodeConstrain» y «OnySelfLoops…». No es un error: `ConstraintManager` solo lee `defaultBehavior()`, así que el `name()` es un dato que nadie usa y que no coincida con la clase no tiene efecto observable.

**146 `OnlyUnlabeledLinks` no impone nada y no llega a ninguna red**

Hay que decidir si se implementa o se borra: su `checkProbNet` es un cuerpo vacío generado automáticamente y ninguna edición la consulta. No hay comportamiento que corregir, porque es opcional, nadie la añade y nunca está en una red; es una de las ocho restricciones opcionales de error 39.

**147 `ValidName` está inactiva, y su comprobación ya la hacen otras dos restricciones**

Propuesta de limpieza: borrar `ValidName` dejando su método estático `nameIsAlready Present` donde se use, o unificarla con `NoEmptyName`, cuyo `checkProbNet` es el mismo código. No es un agujero: en `NodeBaseNameEdit.checkConstraintsWillBeMet` las líneas que consultan `ValidName` son código muerto, pero justo antes `DistinctVariableNames` y `NoEmptyName` ya rechazan el nombre repetido y el vacío.

**148** **`ModelNetworkConstraint` solo trabaja por la vía de las ediciones, a propósito**
Propuesta: cuando cada restricción sepa juzgar una edición (nota 153), esta restricción encajaría allí de forma natural. No es un error: su `checkProbNet` está vacío a propósito, porque la regla (qué enlaces de la red modelo no se pueden tocar durante el aprendizaje) solo tiene sentido ante una edición, y la consultan `AddLinkEdit`, `RemoveLinkEdit`, `InvertLinkEdit` y `OrientLinkEdit`.

**149** **Deshacer tiene el cuerpo vacío en la clase base, y olvidarlo en una edición no avisa**
Propuesta: hacer obligatorio el contrato de edición (nota 154), para que olvidar `undo()` o `checkConstraintsWillBeMet()` en una subclase de `PNEdit` no pase inadvertido. No es un error por sí mismo: los casos graves ya se corrigieron, y las ediciones que hoy no redefinen `undo()` no llegan al usuario, como `RemovePotentialsEdit` (nota 35).

**150** **No está decidido dónde toma una edición la foto del estado anterior**
Propuesta: con el contrato de nota 154, tomar la foto siempre dentro de `doEdit()`, donde se construye la reversión, porque una foto tomada en el constructor caduca si la edición se ejecuta más tarde. No es un error hoy: las ediciones que fotografían en el constructor (`NodeStateEdit`, `TablePotentialValueEdit`) se ejecutan justo después de construirse, y `AddLinkEdit`, que la ventana construye sin ejecutar para colorear flechas, fotografía en `doEdit()`.

**151** **El protocolo de rehacer depende de un interruptor que cada clase usa a su manera**
Propuesta: eliminar el interruptor `typicalRedo`, de modo que rehacer vuelva a ejecutar `doEdit()` y guarde su nueva reversión (nota 154); hoy conviven al menos seis variantes. No es un error: las variantes comprobadas repiten operaciones que no cambian el resultado (nota 84, nota 85 y nota 66).

**152** **Ediciones que ejecutan otras fuera de la composición, y un `equals` que cambia el estado**
Propuesta: prohibir que una edición ejecute otra desde su `doEdit()` sin ser una edición compuesta, porque deja dos entradas en el historial por una sola acción. No es un error visible: el caso concreto (nota 83) no llega al usuario, y `COrientLinksEdit.equals` genera y marca sus sub-ediciones al compararse, pero son las mismas que marcaría al ejecutarse.

**153** **Cada restricción debería saber juzgar una edición, en vez de repetir su regla fuera**
Propuesta: un método `checkEdit` en `PNConstraint`, recorrido por `executeEdit()`, al que se migre restricción a restricción la lógica que hoy repiten las ediciones (`AddLinkEdit` nombra quince clases de restricción) y algunos validadores de la ventana, para que una restricción nueva descubierta como complemento se imponga también sobre las ediciones. Es un rediseño, no un error: los defectos concretos que causa la duplicación tienen sus propias fichas, como error 178, error 38, error 39 y nota 79.

**154** **Contrato de edición obligatorio: `doEdit` devuelve su propia reversión**
Propuesta: que `doEdit()` devuelva un objeto de reversión, que `undo()` y `redo()` sean finales en `PNEdit`, que desaparezca `typicalRedo`, que las modificaciones de `ProbNet` devuelvan su reversión y que `executeEdit()` se niegue a ejecutarse dentro del `doEdit()` de una edición no compuesta. Es un rediseño de fondo, sin defecto concreto propio.

**155** **Que `doEdit` devuelva cómo deshacerse y que cada restricción sepa juzgar una edición**
Son dos propuestas de rediseño ya descritas en nota 154 (la reversión devuelta por `doEdit()`) y en nota 153 (un método `checkEdit` en la restricción). No tienen defecto concreto propio.

**156 Que los cambios de `ProbNet` devuelvan su reversión y comprobar la red al abrir**

Propuestas: que cada modificación de `ProbNet` devuelva su reversión, que forma parte de nota 154, y comprobar la red al terminar de leer un fichero y decidir qué hacer con una red inválida. No tienen defecto propio; el defecto concreto de la segunda es error 37.

**157 Devolver la regla a la restricción y decidir qué significa una restricción opcional**

Devolver a la restricción la regla que hoy repiten las ediciones es la misma propuesta que nota 153, y decidir qué significa `OPTIONAL` es la decisión pendiente de error 39, donde está su defecto concreto. No hay defecto propio.

**158 Una sola estrategia para las ediciones de varios pasos y limpieza de las restricciones**

Propuestas: quitar la anulación vacía de `MultiStepEdit` y la edición verificadora de `PasteEdit`, apuntar cada paso antes de ejecutarlo y revertir en `executeEdit()` (nota 142); y limpiar en las restricciones el registro inerte de oyentes (nota 144), el `name()` de la anotación (nota 145), los tres moldes (nota 88) y las clases sin uso. Son reorganización y limpieza; lo que tienen de defecto concreto está en error 31, error 28 y nota 89.

**159 Prohibir que una edición ejecute otra fuera de una edición compuesta**

Propuesta: que `executeEdit()` se niegue a ejecutarse si se llama desde el `doEdit()` de una edición que no sea compuesta, para que una acción no pueda dejar dos entradas en el historial; es la misma propuesta que nota 152. No hay defecto que llegue al usuario: el caso de `ICITablePotentialValueEdit` (nota 83) queda en un historial provisional que se descarta.

**160 La suma y la multiplicación localizan potenciales con `List.remove` e `indexOf`**

Propuesta de limpieza de rendimiento menor: que `TablePotentialArithmetic.sum` aparte las constantes y `multiply` localice el factor con árboles de estrategia guardando su posición mientras recorren, en vez de buscarlos por igualdad. No es un error: `Potential.equals` descarta antes por clase y por lista de variables, así que no se comparan tablas enteras, y dos tablas iguales tienen las mismas variables y posiciones, así que el resultado no cambia.

**161 Código comentado y comentarios en castellano en las operaciones sobre potenciales**

Propuesta de limpieza: borrar los dos métodos comentados de `LinkRestriction PotentialOperations` (unas 83 líneas que ni siquiera se podrían descomentar) y las sentencias comentadas de `PotentialOperations`, y pasar a inglés los cuatro comentarios en castellano de `DiscretePotentialOperations`, `MaxOutVariable`, `MaxPotential` y `Min Potential`. No tiene consecuencia observable; en los modelos canónicos no hay hoy código comentado, solo dos javadoc huérfanos y comentarios que cuentan la historia del código.

**162 Código sin llamadores en operaciones, ediciones, restricciones y potenciales**

Propuesta de limpieza: borrar lo que sigue sin llamadores de producción, como las dos `multiplyAndMaximizeUniformly`, `maximize(Collection)`, `matrixPotential` (nota 99), `RemovePotentialsEdit` (nota 35), `PNEdit.setProbNet`, `ProbNet.removeConstraints`, `ProbNet.removeAllConstraints` y `NoMixedParents.parentNodeIsNotMixed`. No tiene consecuencia observable; varios de esos métodos ya tienen pruebas propias, que habría que borrar con ellos.

**163 `initialPosition` es un concepto a medias, y «constante» se decide de dos formas**

Propuesta: retirar o completar la posición inicial de `AbstractIndexedPotential`, que leen la eliminación y la maximización pero no `multiply` ni `sum`, y unificar el criterio de potencial constante (tabla de longitud 1 frente a lista de variables vacía). La primera parte no tiene defecto alcanzable, porque ningún potencial llega a tener una posición inicial distinta de cero; la consecuencia concreta de la segunda es error 1.

**164 Contratos sin escribir en las operaciones sobre potenciales**

Propuesta: documentar que `TablePotential.tableProject` puede devolver el propio argumento, que `DiscretePotentialOperations.normalize` modifica el potencial que recibe y que `addVariable` y `removeVariable` modifican el receptor en los árboles pero devuelven uno nuevo en las tablas, y unificar las dos definiciones de «tiene intervenciones». No se encontró un defecto alcanzable: los llamadores que ignoran el resultado de `addVariable` trabajan sobre árboles, y las dos definiciones de «tiene intervenciones» solo discrepan en código que nadie llama.

**165 Las interfaces de capacidad de los potenciales conviven con los métodos que sustituían**

Propuesta: completar la sustitución para la que se crearon `Projectable`, `Reorderable`, `Scalable`, `UncertaintyCarrier`, `StrategyCarrier` y `CEUtilityPotential`, en lugar de los métodos de `Potential` que devuelven nulo o lanzan «no soportado». No hay error observable: en producción nadie pregunta por las cuatro primeras y las operaciones siguen preguntando por la clase concreta; el riesgo es que quien lea las interfaces crea que el sistema de tipos protege lo que no protege.

**166 Decidir si el aprendizaje de parámetros debe aprender modelos canónicos**

Hay que decidir si la rama de `EMAlgorithm.adaptNetwork` que aprende modelos canónicos se mantiene como capacidad documentada o se retira; con muchos padres no escala, porque la función que combina crece exponencialmente. No es un error: la rama funciona, sus fallos anteriores ya se corrigieron y la cubre `EMWithCanonicalModelsTest`.

**167 Un único ciclo de vida para las tareas de inferencia, con aviso de progreso y cancelación**

Propuesta: documentar en `Task` un solo ciclo de vida (una consulta por objeto, o invalidar lo calculado cuando cambian la evidencia o las variables de interés) y añadir en `core` una interfaz pequeña de progreso y cancelación, que hoy se informa de cuatro formas distintas, entre ellas un `ProgressMonitor` obligatorio en `DESInference` que impide simular sin pantalla. Es una propuesta de contrato: los defectos concretos que cuelgan de ella son nota 105 y nota 39.

**168 Subir el preprocesamiento de la inferencia al marco común y dejar una sola proyección**

Propuesta: pasar el preprocesamiento (expansión temporal, políticas, evidencia, descuentos, discretización y absorción de nodos numéricos) de `VariableElimination` a `InferenceAlgorithm`, y dejar una sola de las tres copias de la proyección a red de Markov (`TaskUtilities`, `ClusterPropagation`, `HybridElimination`). Es una reorganización: la diferencia de resultados entre algoritmos solo aparece en redes bayesianas con variables numéricas (medido con una red hecha a mano, nota 109), y no se encontró un camino real de la aplicación donde se note.

**169 Declarar las versiones de terceros en un solo sitio y que cada módulo declare lo que usa**

Propuesta: mover las versiones de las bibliotecas de terceros a `<dependencyManagement>` del `pom.xml` raíz (el fichero que configura la construcción con Maven), que cada módulo declare lo que usa y verificarlo con `mvn dependency:analyze`. Hoy todas se declaran en `<dependencies>` del raíz y las heredan todos los módulos, incluido `core`; es una reorganización sin consecuencia visible en la aplicación.

**170 Podar los `requires` que no se usan y quitar a `core` la dependencia del escritorio**

Propuesta: quitar de los `module-info.java` las dependencias sin uso (en `core`, `org.apache.poi.poi` y `java.desktop`; en `inference`, `java.desktop` y `jeval`; en `gui`, `jdk.compiler`), borrar los restos de escritorio de `core`, como el import muerto de `JOptionPane` en `PurposeEdit` y la `CannotUndoException` de `MonteCarloOptionsEdit.undo()`, y retirar entonces `requires java.desktop` de `core`. Es limpieza sin consecuencia visible; el único resto de escritorio que sí se nota, el diálogo de `TreeWithExcludedEventsPotential`, es error 105.

**171 Una sola política para instanciar complementos y descubrirlos en todo el camino de clases**

Propuesta: un ayudante común con la política de `InferenceManager.build` (aislar, registrar y seguir), avisos ante nombres repetidos y clases descartadas, descubrimiento por anotación en todo el camino de clases y no solo bajo `org.openmarkov` (`PluginLoader`), y que el algoritmo preferido lo declare el complemento en vez de nombrarlo `core` por texto. Es una unificación sin consecuencia hoy: por ejemplo, el nombre del algoritmo preferido coincide con la anotación de `VEPropagation`; el gestor que se cae entero por un complemento defectuoso es nota 106.

**172 Reactivar la prueba de requisitos de construcción**

Propuesta: reactivar `ImplementationRequirementsAreMetTest`, hoy con `@Disabled`, congelando la lista actual de incumplimientos para que no crezca; y decidir el destino del `name()` de `@Constraint` (nota 145). La otra parte de la propuesta, borrar `Filter` y `PluginManager`, que nadie importaba, está hecha en `eafc412`. Es red de seguridad de pruebas, sin consecuencia visible; hoy nada comprueba que una subclase nueva de `Potential` tenga el constructor que necesita su copia por reflexión.

**173 Abrir el punto de extensión de los formatos de fichero**

Propuesta: que cada complemento de formato declare su extensión y versión (o sepa decir si puede leer un fichero), mover a `core` el contrato de lectores y escritores, que hoy vive en `io`, y añadir lectura y escritura sobre flujos. Es un rediseño: hoy `FormatManager.getProbNetReader` elige el lector solo por extensión y supone un XML con `formatVersion`, así que un formato nuevo que no sea XML no se podría abrir; el lector de XMLBIF es nota 50.

**174 Que cada excepción construya su mensaje y avisar ante claves de texto repetidas**

Propuesta: que cada excepción construya su mensaje en inglés en vez de buscarlo por el nombre de su clase en los ficheros de textos (hoy renombrarla rompe el mensaje sin aviso del compilador), sacar de `BundleSearch`, en `core`, los nombres de paquetes de `sensitivityAnalysis` y `stochasticPropagationOutput`, usar un registrador por clase y avisar cuando `StringDatabase` funda claves repetidas. Es reorganización del diagnóstico: medido, las 172 subclases concretas de `OpenMarkovException` tienen su texto, y las claves repetidas no tienen hoy consecuencia en inglés (nota 63).

**175 Renombrar el módulo `inference.DES` y separar el motor de simulación de sus ventanas**

Propuesta: renombrar el módulo a la convención `org.openmarkov.*` y dejar el motor `DESInference` abajo y sin escritorio. Desde `eafc412` las ventanas (`CEDESDialog`, `DESResultsWindow` y la exportación a Excel) están en `gui`, paquete `org.openmarkov.gui.desnets`, y ya pueden usar sus componentes. Pero el motor se ha mudado con ellas: sigue sin separarse de las ventanas, y su `ProgressMonitor` obligatorio sigue impidiendo simular sin pantalla. Es una reorganización sin consecuencia visible.

**176 Que la ventana pida los algoritmos al registro de complementos y no los construya a mano**

Propuesta: una fábrica de tareas que resuelva la implementación en el registro, en lugar de los `new VEPropagation`, `new VEEvaluation`, `new VECEAnalysis` y demás que hacen la ventana y los módulos de análisis. No cambia ningún resultado hoy, porque cada tarea de la ventana tiene un solo algoritmo aplicable; lo que sí pasa es que registrar un algoritmo nuevo como complemento no cambia lo que ejecuta la aplicación.

**177 Extraer de la ventana un documento sin Swing con la red, su fichero y su estado**

Propuesta: un `NetworkDocument` sin Swing, que el panel observe y que `NetsIO` reciba, con la red, la ruta del fichero, el lector o escritor con que se abrió, el indicador de modificado y la evidencia, que hoy se reparten `NetworkEditorPanel` y `PNESupport`. Es una reorganización de responsabilidades sin consecuencia visible propia; el defecto que causa que las copias de red hereden ese estado es error 35.

**178 Mover a `core` las ediciones del módulo gráfico que no dependen de Swing**

Propuesta: mover a `core/action` las ediciones de `gui/src/main/java/org/openmarkov/gui/action/` que no tocan clases visuales, como `ChangeNodeTypeEdit`, `ImposePolicyEdit` o `TablePotentialValueEdit`, junto con la redistribución de probabilidades de `PotentialsTablePanelOperations`, y adaptar las que dependen de `VisualNode` o `VisualNetwork`. Es un traslado sin consecuencia visible; hoy quien quiera usarlas sin interfaz depende del módulo Swing, como hace `learning.gui` para `AutoArrangeEdit`.

**179 Desmontar el objeto global `MainGUI.INSTANCE` pasando los colaboradores por constructor**

Propuesta: desmontar `MainGUI.INSTANCE` por capas, pasando los colaboradores por constructor y empezando por los cuatro usos del módulo `learning.gui`. Es una reorganización sin consecuencia visible propia; hoy se llega a ese objeto global desde 59 sitios, a menudo con cadenas largas de llamadas.

**180 Que la red sea la dueña de renombrar variables y cambiar el tipo de los nodos**

Propuesta: que esas modificaciones pasen por `ProbNet` (`renameVariable`, `changeNodeType`) en vez de por `Node.setVariable` y `Node.setNodeType`, que tocan por su cuenta la red, y que `Potential.deepCopy` falle cuando la red destino no tiene una variable. Es una reorganización; los defectos concretos tienen sus fichas: error 166 para el cambio de tipo de un nodo y error 3 para `deepCopy` con nulos.

**181 Decidir qué es el módulo `resttemplate`, hoy un ejemplo que responde «Hello, World!»**

Hay que decidir el destino de `resttemplate`, un envoltorio REST (interfaz de programación por HTTP) con Spring Boot cuyo único controlador ofrece `GET /hello` y dos `POST` de juguete sobre `ExampleData`. No expone ninguna función de OpenMarkov, así que no hay nada que falle; si se decide que vaya en serio, necesita un `core` sin escritorio (nota 170) y leer redes desde un flujo (nota 173).

**182** **Congelar la biblioteca interna `org.openmarkov.java` y revisar qué paquetes exporta `core`**
Propuesta: dejar escrita la regla de mirar la biblioteca estándar de Java y Apache Commons antes de añadir utilidades a `org.openmarkov.java` (18 ficheros, con duplicados como `NullUtils.equals`, que hace lo mismo que `Objects.equals`), y estudiar cuáles de los 48 paquetes que exporta el `module-info.java` de `core` son de verdad interfaz pública. Es una norma de mantenimiento sin consecuencia visible.

**183** **Dejar escrito que el sistema de módulos de Java se usa solo para declarar dependencias**
Propuesta: documentar que los descriptores son `open module` porque el descubrimiento de complementos por reflexión lo exige, y que la aplicación corre en el camino de clases, así que en ejecución no hay encapsulación; si algún día se quiere, el camino sería `java.util.ServiceLoader`. Es una nota de documentación, no trabajo sobre el código.

**195** **Restos sin uso de la rama de la simulación de sucesos**
Propuesta de limpieza: borrar lo que llegó de la rama `DES-development` y no usa nadie: `FunctionPotentialOld` (su `validate` devuelve falso) y `DataFromFileOld`; `EditionToolBar.getEventCreationButton`, `StandardToolBar.monteCarloSimulationButton`, `IconBind.SIMULATION_ENABLED` y `IconFactory.createEventIcon`; las entradas de menú «establecer como imposible o posible», que la propia rama dejó comentadas en `TableWithEventsContextualMenu` (líneas 84-104), con sus constantes en `ActionCommands` y `MenuItemNames` y sus textos; `MCSimulateNetwork` de los ficheros de textos de la barra de herramientas; los ficheros `gui/src/main/resources/cursors/event.gif` y `gui/src/main/resources/icons/simulate.png`; y `evaluation-message.html`, `message-expired.html` y `message-not-expired.html`, de la versión de evaluación con fecha de caducidad. La casilla «Perfom PSA» depende de lo que se decida en la nota 192. No es un error: nada de eso se ejecuta.

# B Procedencia de cada ficha

Cada error y cada nota salen de uno o de varios de estos dos documentos internos: el *plan de cambios* (con los identificadores de sus puntos y de sus hallazgos) y el *registro de fallos* (con el número de su entrada). Cuando dos entradas describían el mismo defecto se han juntado en una sola ficha. Los *hallazgos nuevos* no estaban en ninguno de los dos: se vieron al comprobar otro error de la lista, que se indica, y se comprobaron después con los mismos criterios. Los añadidos el 9 de octubre se vieron al portar OpenMarkov a C++/Qt; los del 15 de octubre, unos al revisar la rama `DES-development` y otros, también, al portar OpenMarkov a C++/Qt. Todos se comprobaron igual.

| Ficha | Origen |
|---|---|
| Error 1 | registro de fallos, nº 49 |
| Error 2 | registro de fallos, nº 54 |
| Error 3 | registro de fallos, nº 180 |
| Error 4 | plan de cambios, RP15; plan de cambios, F2-d; registro de fallos, nº 74 |
| Error 5 | plan de cambios, RP20 |
| Error 6 | plan de cambios, RP29 |
| Error 7 | plan de cambios, RP25; plan de cambios, F8-d |
| Error 8 | registro de fallos, nº 39 |
| Error 9 | hallazgo nuevo, visto junto al error 8 |
| Error 10 | registro de fallos, nº 71 |
| Error 11 | hallazgo nuevo, visto junto al error 12 |
| Error 12 | registro de fallos, nº 69 |
| Error 13 | registro de fallos, nº 70 |

| Ficha | Origen |
|---|---|
| Error 14 | registro de fallos, nº 73 |
| Error 15 | hallazgo nuevo, visto junto al error 10; incluye registro de fallos, nº 32, que se había dado por inalcanzable |
| Error 16 | registro de fallos, nº 75 |
| Error 17 | registro de fallos, nº 150 |
| Error 18 | registro de fallos, nº 152 |
| Error 19 | registro de fallos, nº 96 |
| Error 20 | registro de fallos, nº 95 |
| Error 21 | hallazgo nuevo, visto junto al error 20 |
| Error 22 | registro de fallos, nº 176 |
| Error 23 | registro de fallos, nº 177 |
| Error 24 | registro de fallos, nº 145 |
| Error 25 | hallazgo nuevo, visto junto a la nota 23 |
| Error 26 | registro de fallos, nº 37 |
| Error 27 | hallazgo nuevo, visto junto a la nota 23 |
| Error 28 | plan de cambios, M11; plan de cambios, D4 |
| Error 29 | registro de fallos, nº 29 |
| Error 30 | hallazgo nuevo, visto junto al error 29 |
| Error 31 | plan de cambios, D3 |
| Nota 184 (antes, error 32) | hallazgo nuevo, visto junto a la nota 82 |
| Error 33 | registro de fallos, nº 35 |
| Error 34 | registro de fallos, nº 166 |
| Error 35 | plan de cambios, F5-a |
| Error 36 | registro de fallos, nº 112 |
| Error 37 | plan de cambios, R1; plan de cambios, F3-b |
| Error 38 | registro de fallos, nº 28; plan de cambios, C2; plan de cambios, R2 |
| Error 39 | plan de cambios, R3; plan de cambios, F3-c |
| Error 40 | registro de fallos, nº 40 |
| Error 41 | registro de fallos, nº 120 |
| Error 42 | hallazgo nuevo, visto junto al error 44 |
| Error 43 | registro de fallos, nº 29 |
| Error 44 | registro de fallos, nº 41 |
| Error 45 | hallazgo nuevo, visto junto al error 44 |
| Error 46 | registro de fallos, nº 121 |
| Error 47 | registro de fallos, nº 123 |
| Error 48 | plan de cambios, E1; plan de cambios, F7-g |
| Error 49 | plan de cambios, F5-a |
| Error 50 | registro de fallos, nº 180 |
| Error 51 | registro de fallos, nº 160 |
| Error 52 | registro de fallos, nº 184; registro de fallos, nº 42 |
| Error 53 | registro de fallos, nº 43 |
| Error 54 | hallazgo nuevo, visto junto a la nota 46 |
| Error 55 | registro de fallos, nº 185 |
| Error 56 | registro de fallos, nº 33 |
| Error 57 | registro de fallos, nº 34 |
| Error 58 | registro de fallos, nº 36 |
| Error 59 | plan de cambios, RP24 |
| Error 60 | registro de fallos, nº 167 |
| Error 61 | registro de fallos, nº 169 |
| Error 62 | registro de fallos, nº 168 |
| Error 63 | registro de fallos, nº 48 |
| Error 64 | registro de fallos, nº 47 |
| Error 65 | registro de fallos, nº 147 |
| Error 66 | registro de fallos, nº 182 |
| Error 67 | registro de fallos, nº 46; plan de cambios, E5; plan de cambios, F1-s |

| Ficha | Origen |
|---|---|
| Error 68 | registro de fallos, nº 45; registro de fallos, nº 189 |
| Error 69 | registro de fallos, nº 188 |
| Error 70 | registro de fallos, nº 72 |
| Error 71 | hallazgo nuevo, visto junto al error 10 |
| Error 72 | registro de fallos, nº 38 |
| Error 73 | registro de fallos, nº 134 |
| Error 74 | hallazgo nuevo, visto junto al error 80 |
| Error 75 | registro de fallos, nº 65 |
| Error 76 | registro de fallos, nº 62 |
| Error 77 | registro de fallos, nº 63 |
| Error 78 | registro de fallos, nº 137 |
| Error 79 | registro de fallos, nº 136 |
| Error 80 | registro de fallos, nº 197 |
| Error 81 | registro de fallos, nº 198 |
| Error 82 | registro de fallos, nº 199 |
| Error 83 | hallazgo nuevo, visto junto al error 80 |
| Error 84 | registro de fallos, nº 206 |
| Error 85 | hallazgo nuevo, visto junto a la nota 46 |
| Error 86 | hallazgo nuevo, visto junto a la nota 46; incluye registro de fallos, nº 205, que se había dado por inalcanzable |
| Error 87 | hallazgo nuevo, visto junto a la nota 46 |
| Error 88 | registro de fallos, nº 94 |
| Error 89 | hallazgo nuevo, visto junto al error 88 |
| Error 90 | hallazgo nuevo, visto junto al error 88 |
| Error 91 | registro de fallos, nº 179 |
| Error 92 | registro de fallos, nº 195 |
| Error 93 | registro de fallos, nº 196 |
| Error 94 | registro de fallos, nº 207 |
| Error 95 | registro de fallos, nº 102 |
| Error 96 | registro de fallos, nº 104 |
| Error 97 | registro de fallos, nº 105 |
| Error 98 | registro de fallos, nº 106 |
| Error 99 | hallazgo nuevo, visto junto al error 98 |
| Error 100 | registro de fallos, nº 174 |
| Error 101 | registro de fallos, nº 173 |
| Error 102 | registro de fallos, nº 210 |
| Error 103 | registro de fallos, nº 213 |
| Error 104 | registro de fallos, nº 107 |
| Error 105 | registro de fallos, nº 175; plan de cambios, F6-b |
| Error 106 | hallazgo nuevo, visto junto a la nota 55 |
| Error 107 | registro de fallos, nº 193 |
| Error 108 | hallazgo nuevo, visto junto a la nota 168 |
| Error 109 | registro de fallos, nº 192 |
| Error 110 | registro de fallos, nº 240 |
| Error 111 | plan de cambios, F6-e |
| Error 112 | hallazgo nuevo, visto junto al error 8 |
| Error 113 | hallazgo nuevo, visto junto al error 8 |
| Error 114 | hallazgo nuevo, visto junto al error 8 |
| Error 115 | registro de fallos, nº 100 |
| Error 116 | hallazgo nuevo, visto junto al error 22 |
| Error 117 | registro de fallos, nº 214 |
| Error 118 | registro de fallos, nº 224 |
| Error 119 | registro de fallos, nº 225 |
| Error 120 | registro de fallos, nº 226 |
| Error 121 | registro de fallos, nº 227 |
| Error 122 | registro de fallos, nº 228 |
| Error 123 | registro de fallos, nº 228 |

| Ficha | Origen |
|---|---|
| Error 124 | registro de fallos, nº 68 |
| Error 125 | registro de fallos, nº 85 |
| Error 126 | hallazgo nuevo, visto junto al error 125 |
| Error 127 | registro de fallos, nº 86 |
| Error 128 | hallazgo nuevo, visto junto al error 127 |
| Error 129 | hallazgo nuevo, visto junto al error 127 |
| Error 130 | hallazgo nuevo, visto junto al error 127 |
| Error 131 | registro de fallos, nº 84 |
| Error 132 | registro de fallos, nº 88 |
| Error 133 | hallazgo nuevo, visto junto al error 127 |
| Error 134 | hallazgo nuevo, visto junto al error 127 |
| Error 135 | registro de fallos, nº 87 |
| Error 136 | registro de fallos, nº 76 |
| Error 137 | registro de fallos, nº 83 |
| Error 138 | hallazgo nuevo, visto junto al error 139 |
| Error 139 | registro de fallos, nº 81 |
| Error 140 | hallazgo nuevo, visto junto al error 139 |
| Error 141 | hallazgo nuevo, visto junto al error 139 |
| Error 142 | hallazgo nuevo, visto junto al error 139 |
| Error 143 | registro de fallos, nº 89 |
| Error 144 | registro de fallos, nº 90 |
| Error 145 | registro de fallos, nº 93; registro de fallos, nº 239 |
| Error 146 | hallazgo nuevo, visto junto a la nota 20 |
| Error 147 | plan de cambios, E8 |
| Error 148 | hallazgo nuevo, visto junto al error 147 |
| Error 149 | hallazgo nuevo, visto junto al error 147 |
| Error 150 | registro de fallos, nº 204 |
| Error 151 | registro de fallos, nº 201 |
| Error 152 | registro de fallos, nº 174 |
| Error 153 | registro de fallos, nº 126 |
| Error 154 | registro de fallos, nº 119 |
| Error 155 | hallazgo nuevo, visto junto a la nota 23 |
| Error 156 | registro de fallos, nº 118 |
| Error 157 | registro de fallos, nº 202 |
| Error 158 | registro de fallos, nº 200 |
| Error 159 | registro de fallos, nº 151 |
| Error 160 | registro de fallos, nº 153 |
| Error 161 | hallazgo nuevo, visto junto a la nota 17 |
| Error 162 | registro de fallos, nº 162 |
| Error 163 | registro de fallos, nº 138 |
| Error 164 | registro de fallos, nº 172 |
| Error 165 | registro de fallos, nº 203 |
| Error 166 | plan de cambios, F7-f |
| Error 167 | registro de fallos, nº 115 |
| Error 168 | registro de fallos, nº 122 |
| Error 169 | registro de fallos, nº 131 |
| Error 170 | registro de fallos, nº 128 |
| Error 171 | registro de fallos, nº 232 |
| Error 172 | registro de fallos, nº 241 |
| Error 173 | registro de fallos, nº 142 |
| Error 174 | hallazgo nuevo, visto junto al error 173 |
| Error 175 | registro de fallos, nº 171 |
| Error 176 | hallazgo nuevo, visto junto al error 12 |
| Error 177 | registro de fallos, nº 178 |
| Error 178 | plan de cambios, M1 |
| Error 179 | registro de fallos, nº 125 |
| Error 180 | registro de fallos, nº 144 |

| Ficha | Origen |
|---|---|
| Error 181 | registro de fallos, nº 155 |
| Error 182 | hallazgo nuevo, visto junto a la nota 17 |
| Error 183 | registro de fallos, nº 139 |
| Error 184 | registro de fallos, nº 172 |
| Error 185 | hallazgo nuevo, visto junto a la nota 120 |
| Error 186 | hallazgo nuevo, visto junto al error 48 |
| Error 187 | hallazgo nuevo, visto junto al error 48 |
| Error 188 | hallazgo nuevo, visto junto al error 127 |
| Error 189 | plan de cambios, F7-b |
| Error 190 | plan de cambios, F2-b |
| Error 191 | plan de cambios, F2-a |
| Error 192 | plan de cambios, RP31 |
| Error 193 | hallazgo nuevo, visto junto al error 1 |
| Error 194 | hallazgo nuevo, al corregir el error 41 |
| Error 195 | hallazgo nuevo, al revisar el exportador a Amua tras enlazarlo al menú (nota 51) |
| Error 196 | hallazgo nuevo, al revisar el exportador a Amua tras enlazarlo al menú (nota 51) |
| Error 197 | hallazgo nuevo, al revisar el exportador a Amua tras enlazarlo al menú (nota 51) |
| Error 198 | hallazgo nuevo, al revisar el exportador a Amua tras enlazarlo al menú (nota 51) |
| Error 199 | hallazgo nuevo, al revisar el exportador a Amua tras enlazarlo al menú (nota 51) |
| Error 200 | hallazgo nuevo, al corregir el error 63 |
| Error 201 | hallazgo nuevo, visto en la ventana al comprobar el error 66 |
| Error 202 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 30 de septiembre al 8 de octubre |
| Error 203 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 204 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 205 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 206 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 207 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 208 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 209 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 210 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 211 | hallazgo nuevo, al portar OpenMarkov a C++/Qt |
| Error 212 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 213 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 214 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 215 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 216 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 217 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 218 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 219 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Error 220 | hallazgo nuevo, al portar OpenMarkov a C++/Qt |
| Error 221 | hallazgo nuevo, al portar OpenMarkov a C++/Qt |
| Error 222 | hallazgo nuevo, al portar OpenMarkov a C++/Qt |
| Error 223 | hallazgo nuevo, al portar OpenMarkov a C++/Qt |
| Error 224 | hallazgo nuevo, al portar OpenMarkov a C++/Qt |
| Error 225 | hallazgo nuevo, visto en la ventana al comprobar el arreglo del error 153 |
| Error 226 | hallazgo nuevo, al medir las reglas de familia para el error 76 |
| Error 227 | hallazgo nuevo, al medir la propagación de cada tipo de red para el error 201 |
| Error 228 | hallazgo nuevo, al medir la propagación de cada tipo de red para el error 201 |
| Error 229 | hallazgo nuevo, al medir la propagación de cada tipo de red para el error 201 |
| Error 230 | hallazgo nuevo, al medir la propagación de cada tipo de red para el error 201 |
| Error 231 | hallazgo nuevo, al medir la propagación de cada tipo de red para el error 201 |
| Error 232 | hallazgo nuevo, visto en la aplicación al probar el modo inferencia con los MID para el error 201 |
| Error 233 | hallazgo nuevo, al probar las tres opciones de transición para el error 201 |
| Error 234 | hallazgo nuevo, al corregir el error 21; ampliado al corregirlo |
| Error 235 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |

| Ficha | Origen |
|---|---|
| Error 236 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Error 237 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Error 238 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Error 239 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Error 240 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 9 de octubre |
| Error 241 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 9 de octubre |
| Error 242 | hallazgo nuevo, al llevar a la versión en C++/Qt el modo inferencia de las DAN y los MID, del 9 y 10 de octubre |
| Error 243 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 12 de octubre |
| Error 244 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 245 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 246 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 247 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 248 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 249 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 250 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 251 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 252 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 253 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 254 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 255 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 256 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 257 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 258 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 259 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 260 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 261 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Error 262 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 263 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 264 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 265 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 266 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 267 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 268 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 269 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 270 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 15 de octubre |
| Error 271 | hallazgo nuevo, visto al comprobar el error 257 |
| Error 272 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos de la tarde del 15 de octubre |
| Error 273 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos de la tarde del 15 de octubre |
| Error 274 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos de la tarde del 15 de octubre |
| Error 275 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos de la tarde del 15 de octubre |
| Error 276 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos de la tarde del 15 de octubre |
| Error 277 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos de la tarde del 15 de octubre |
| Error 278 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos de la tarde del 15 de octubre |
| Error 279 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 280 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 281 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 282 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 283 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 284 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 285 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 286 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 287 | hallazgo nuevo, al revisar el código de la edición de redes el 15 de octubre |
| Error 288 | hallazgo nuevo, visto al corregir el error 280 |
| Error 289 | hallazgo nuevo, visto al corregir el error 285 |
| Error 290 | hallazgo nuevo, visto al comprobar el arreglo del error 224 |
| Nota 1 | registro de fallos, nº 31; plan de cambios, RP11 |

| Ficha | Origen |
|---|---|
| Nota 2 | registro de fallos, nº 127 |
| Nota 3 | registro de fallos, nº 24 |
| Nota 4 | registro de fallos, nº 25 |
| Nota 5 | registro de fallos, nº 27 |
| Nota 6 | registro de fallos, nº 51 |
| Nota 7 | registro de fallos, nº 61 |
| Nota 8 | registro de fallos, nº 78 |
| Nota 9 | registro de fallos, nº 82 |
| Nota 10 | registro de fallos, nº 92 |
| Nota 11 | registro de fallos, nº 113 |
| Nota 12 | registro de fallos, nº 116 |
| Nota 13 | registro de fallos, nº 117 |
| Nota 14 | registro de fallos, nº 141 |
| Nota 15 | registro de fallos, nº 143 |
| Nota 16 | registro de fallos, nº 148 |
| Nota 17 | registro de fallos, nº 156 |
| Nota 18 | registro de fallos, nº 164 |
| Nota 19 | registro de fallos, nº 165 |
| Nota 20 | registro de fallos, nº 238 |
| Nota 21 | registro de fallos, nº 242 |
| Nota 22 | plan de cambios, M2 |
| Nota 23 | plan de cambios, M3 |
| Nota 24 | plan de cambios, M12 |
| Nota 25 | plan de cambios, M14 |
| Nota 26 | plan de cambios, M15 |
| Nota 27 | plan de cambios, pot§6.3 |
| Nota 28 | hallazgo nuevo, visto junto al error 10 |
| Nota 29 | registro de fallos, nº 79 |
| Nota 30 | registro de fallos, nº 149 |
| Nota 31 | registro de fallos, nº 159 |
| Nota 32 | registro de fallos, nº 186 |
| Nota 33 | registro de fallos, nº 236 |
| Nota 34 | registro de fallos, nº 26 |
| Nota 35 | registro de fallos, nº 30 |
| Nota 36 | registro de fallos, nº 36; plan de cambios, M5 |
| Nota 37 | registro de fallos, nº 44 |
| Nota 38 | registro de fallos, nº 50 |
| Nota 39 | registro de fallos, nº 52 |
| Nota 40 | registro de fallos, nº 53 |
| Nota 41 | registro de fallos, nº 55 |
| Nota 42 | registro de fallos, nº 56 |
| Nota 43 | registro de fallos, nº 57 |
| Nota 44 | registro de fallos, nº 58 |
| Nota 45 | registro de fallos, nº 59 |
| Nota 46 | registro de fallos, nº 60 |
| Nota 47 | registro de fallos, nº 64 |
| Nota 48 | registro de fallos, nº 66 |
| Nota 49 | registro de fallos, nº 67 |
| Nota 50 | registro de fallos, nº 214 |
| Nota 51 | registro de fallos, nº 222 |
| Nota 52 | registro de fallos, nº 223 |
| Nota 53 | registro de fallos, nº 80 |
| Nota 54 | registro de fallos, nº 91 |
| Nota 55 | registro de fallos, nº 97 |
| Nota 56 | registro de fallos, nº 98 |
| Nota 57 | registro de fallos, nº 99 |
| Nota 58 | registro de fallos, nº 101 |

| Ficha | Origen |
|---|---|
| Nota 59 | registro de fallos, nº 103 |
| Nota 60 | registro de fallos, nº 108 |
| Nota 61 | registro de fallos, nº 109 |
| Nota 62 | registro de fallos, nº 110 |
| Nota 63 | registro de fallos, nº 111 |
| Nota 64 | registro de fallos, nº 129 |
| Nota 65 | registro de fallos, nº 130 |
| Nota 66 | registro de fallos, nº 132 |
| Nota 67 | registro de fallos, nº 140 |
| Nota 68 | registro de fallos, nº 158 |
| Nota 69 | registro de fallos, nº 160 |
| Nota 70 | registro de fallos, nº 161 |
| Nota 71 | registro de fallos, nº 163 |
| Nota 72 | registro de fallos, nº 169 |
| Nota 73 | registro de fallos, nº 170 |
| Nota 74 | registro de fallos, nº 176 |
| Nota 75 | registro de fallos, nº 190 |
| Nota 76 | registro de fallos, nº 194 |
| Nota 77 | registro de fallos, nº 212 |
| Nota 78 | registro de fallos, nº 243 |
| Nota 79 | plan de cambios, G8; plan de cambios, F1-n |
| Nota 80 | plan de cambios, M4 |
| Nota 81 | plan de cambios, M5; registro de fallos, nº 36 |
| Nota 82 | plan de cambios, M6 |
| Nota 83 | plan de cambios, M7 |
| Nota 84 | plan de cambios, M8 |
| Nota 85 | plan de cambios, M9 |
| Nota 86 | plan de cambios, M10 |
| Nota 87 | plan de cambios, M13 |
| Nota 88 | plan de cambios, R6 |
| Nota 89 | plan de cambios, R7; plan de cambios, F3-f |
| Nota 90 | plan de cambios, F8-c |
| Nota 91 | plan de cambios, E10 |
| Nota 92 | plan de cambios, RP22 |
| Nota 93 | plan de cambios, RP23 |
| Nota 94 | plan de cambios, RP26 |
| Nota 95 | plan de cambios, RP27 |
| Nota 96 | plan de cambios, RP28 |
| Nota 97 | plan de cambios, RP3 |
| Nota 98 | plan de cambios, F8-c; plan de cambios, RP-nota1 |
| Nota 99 | plan de cambios, RP-nota2 |
| Nota 100 | plan de cambios, RP-nota3 |
| Nota 101 | plan de cambios, RP-nota4 |
| Nota 102 | plan de cambios, pot§6.4 |
| Nota 103 | plan de cambios, F5-b |
| Nota 104 | plan de cambios, F5-b |
| Nota 105 | plan de cambios, F5-c |
| Nota 106 | plan de cambios, F6-c |
| Nota 107 | registro de fallos, nº 234 |
| Nota 108 | hallazgo nuevo, visto junto a la nota 168 |
| Nota 109 | hallazgo nuevo, visto junto a la nota 168 |
| Nota 110 | hallazgo nuevo, visto junto al error 10 |
| Nota 111 | hallazgo nuevo, visto junto a la nota 58 |
| Nota 112 | registro de fallos, nº 215 |
| Nota 113 | registro de fallos, nº 216 |
| Nota 114 | registro de fallos, nº 217 |
| Nota 115 | registro de fallos, nº 218 |

| Ficha | Origen |
|---|---|
| Nota 116 | registro de fallos, nº 219 |
| Nota 117 | registro de fallos, nº 220 |
| Nota 118 | registro de fallos, nº 221 |
| Nota 119 | registro de fallos, nº 114 |
| Nota 120 | registro de fallos, nº 124 |
| Nota 121 | registro de fallos, nº 133 |
| Nota 122 | registro de fallos, nº 135 |
| Nota 123 | registro de fallos, nº 139 |
| Nota 124 | registro de fallos, nº 146 |
| Nota 125 | registro de fallos, nº 154 |
| Nota 126 | registro de fallos, nº 157 |
| Nota 127 | registro de fallos, nº 164 |
| Nota 128 | registro de fallos, nº 181 |
| Nota 129 | registro de fallos, nº 183 |
| Nota 130 | registro de fallos, nº 187 |
| Nota 131 | registro de fallos, nº 191 |
| Nota 132 | registro de fallos, nº 208 |
| Nota 133 | registro de fallos, nº 209 |
| Nota 134 | registro de fallos, nº 211 |
| Nota 135 | registro de fallos, nº 229 |
| Nota 136 | registro de fallos, nº 230 |
| Nota 137 | registro de fallos, nº 231 |
| Nota 138 | registro de fallos, nº 233 |
| Nota 139 | registro de fallos, nº 235 |
| Nota 140 | registro de fallos, nº 237 |
| Nota 141 | plan de cambios, RP21 |
| Nota 142 | plan de cambios, D4 |
| Nota 143 | plan de cambios, R2 |
| Nota 144 | plan de cambios, R4 |
| Nota 145 | plan de cambios, R5 |
| Nota 146 | plan de cambios, C1 |
| Nota 147 | plan de cambios, C3 |
| Nota 148 | plan de cambios, C4 |
| Nota 149 | plan de cambios, D1 |
| Nota 150 | plan de cambios, D5 |
| Nota 151 | plan de cambios, D6 |
| Nota 152 | plan de cambios, D7 |
| Nota 153 | plan de cambios, F3-a |
| Nota 154 | plan de cambios, F4 |
| Nota 155 | plan de cambios, P1 |
| Nota 156 | plan de cambios, P2 |
| Nota 157 | plan de cambios, P3 |
| Nota 158 | plan de cambios, P4 |
| Nota 159 | plan de cambios, P5 |
| Nota 160 | plan de cambios, F2-c |
| Nota 161 | plan de cambios, RP30 |
| Nota 162 | plan de cambios, F8-c |
| Nota 163 | plan de cambios, F8-b |
| Nota 164 | plan de cambios, F8-f |
| Nota 165 | plan de cambios, F8-a |
| Nota 166 | plan de cambios, F8-h |
| Nota 167 | plan de cambios, F5-c |
| Nota 168 | plan de cambios, F5-d |
| Nota 169 | plan de cambios, F6-a |
| Nota 170 | plan de cambios, F6-b |
| Nota 171 | plan de cambios, F6-c |
| Nota 172 | plan de cambios, F6-d |

| Ficha | Origen |
|---|---|
| Nota 173 | plan de cambios, F6-e |
| Nota 174 | plan de cambios, F6-f |
| Nota 175 | plan de cambios, F6-g |
| Nota 176 | plan de cambios, F7-a |
| Nota 177 | plan de cambios, F7-c |
| Nota 178 | plan de cambios, F7-d |
| Nota 179 | plan de cambios, F7-e |
| Nota 180 | plan de cambios, F7-f |
| Nota 181 | plan de cambios, F8-e |
| Nota 182 | plan de cambios, F8-f |
| Nota 183 | plan de cambios, F8-g |
| Nota 185 | hallazgo nuevo, visto junto al error 1 |
| Nota 186 | hallazgo nuevo, visto junto al error 1 |
| Nota 187 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 30 de septiembre al 8 de octubre |
| Nota 188 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 30 de septiembre al 8 de octubre |
| Nota 189 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 30 de septiembre al 8 de octubre |
| Nota 190 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Nota 191 | hallazgo nuevo, al comparar las ventanas con las de la versión en C++/Qt |
| Nota 192 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Nota 193 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Nota 194 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Nota 195 | hallazgo nuevo, al revisar la rama `DES-development` de Carmen Yago, que nunca se fusionó |
| Nota 196 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 12 de octubre |
| Nota 197 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Nota 198 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |
| Nota 199 | hallazgo nuevo, al llevar a la versión en C++/Qt los arreglos del 13 y 14 de octubre |